الإصدار الأول ٢٠٢٦

Python
بلغة بايثون

إتقان scikit-learn 1.9

من الصفر إلى الاحتراف

دليل شامل لتعلّم الآلة التطبيقي

- **شرح مبسّط** من الأساسيات إلى المتقدم
- **أمثلة عملية** وتطبيقات واقعية
- **مشاريع متكاملة** من الحياة العملية
- **أفضل الممارسات** والنصائح الاحترافية

م/ أحمد صالح المغز

2026

إتقان scikit-learn ١٫٩

من الصفر إلى الاحتراف

دليل شامل لتعلّم الآلة التطبيقي

م/أحمد صالح المغز

Eng. Ahmed Saleh Al-Maghz

scikit-learn ١.٩.١ ▪ Python ٣.١٤ ▪ NumPy ٢.٤ ▪ pandas ٣.٠

فهرس المحتويات

١ الأسس: من الفكرة إلى أول نموذج — ٩ فصلاً

البنية المرجعية، والرياضيات الحاكمة، وطريقة التفكير التي لا تُكسر

١ ما هو تعلّم الآلة؟ رحلة من الفكرة إلى النظام ١٤

الإطار المفاهيمي الذي يحوّل البيانات إلى قرارات

٢ منظوم بايثون العلمية: أين يقع scikit-learn؟ ٢٨

NumPy و SciPy و pandas و Matplotlib — ومَن المسؤول عن ماذا

٣ التثبيت والإعداد على Python ٣.١٤ ..... ٤٠

بيئة عمل نظيفة قابلة للتكرار، من venv إلى uv و conda

٤ أول نموذج لك: أقرب الجيران على بيانات Iris ٥٤

قراءة كود كامل من السطر الأول حتى الرسم البياني

٥ نموذج التقدير: فهم Estimator API ٦٩

fit · transform · predict · score — العقد الذهبي بينك وبين المكتبة

٦ الرياضيات تحت الغطاء ٨١

جبر خطي، وتفاضل، واحتمال، ودوال خسارة — بالقدر الذي تحتاجه فعلاً

٧ فحص مُقدِّر قبل أن يذهب إلى الإنتاج ٩٨

check_estimator والوسوم والعرض البياني HTML- الجديد في ١.٩

٨ مشروع تمهيدي شامل: أسعار المنازل ١١٠

خط أنابيب حقيقي من التحميل حتى التقرير

٩ أخلاقيات التطبيق وحدود الخوارزميات ١٢١

الحياد، والتحيّز، والخصوصية، ومسؤولية المهندس

٢ البيانات: الكتلة الخام للمشروع — ١١ فصلاً

من مصفوفة NumPy إلى المصفوفة المتفرّقة والبيانات المفقودة

١٠ أنماط البيانات في scikit-learn ١٣٣

DataFrame و ndarray والمصفوفات المتفرّرة: أيّها ولماذا

١١ المصفوفات المتفرّرة وواجهة sparray الجديدة ١٤٦

المفتاح sparse_interface في ١.٩ — ماذا يعني لك ولماذا يهمّك

١٢ أنواع البيانات: عددي، وفئوي، وزمني، ونصي ١٦١

كيف يميّز المُقدِّر نوع كل عمود، وماذا يفعل إن أخطأت

١٣ القيم المفقودة: من الفراغ إلى المعلومة ١٧٤

MissingIndicator و IterativeImputer و KNNImputer و SimpleImputer

١٤ التحجيم والتطبيع ١٨٨

KernelCenterer و Normalizer و Robust و MaxAbs و MinMax و Standard

١٥ التحويلات اللاخطية والتوزيعية ٢٠١

PolynomialFeatures و Spline و KBins و Quantile و Power

١٦ ترميز المتغيرات الفئوية ٢١٦

OneHot و Ordinal و TargetEncoder — وأين تكمن مخاطر الترتيب

١٧ هندسة الخصائص: من الصف إلى المعلومة ٢٣١

التركيبات، والتحويلات النسبية، والنطاق الدلالي

١٨ النصوص: من مستند إلى متجه ٢٤٢

CountVectorizer و TfidfVectorizer و Hashing و Stopwords و N-grams

١٩ الصور والبيانات غير الجدولية ٢٥٢

استخراج الخصائص، وتدرّجات الصورة، وتحويل الصورة إلى مصفوفة

٢٠ واجهة set_output: نتائج على شكل DataFrame ٢٦١

كيف تُخرج جداول قابلة للقراءة مباشرة من كل محوّل

٣ هندسة الخصائص واختيارها

٦ فصلاً

من مئة عمود إلى عشرة أعمدة تحمل الإشارة

٢١ أبعاد البيانات: لماذا تكره الخوارزميات الأبعاد الكثيرة؟ ٢٧٣

لعنة الأبعاد، وهندسة الفضاء عالي الأبعاد

٢٢ التصفية التي لا تنظر إلى النموذج ٢٨٤

VarianceThreshold و SelectKBest و SelectPercentile والانحصارات

٢٣ التصفية التي تنظر إلى النموذج ٢٩٨

SelectFromModel و RFE و RFECV و SequentialFeatureSelector

٢٤ الاختيار داخل خط الأنابيب ٣١١

لماذا يجب أن يكون المُنتقي طبقة داخل Pipeline لا خطوة يدوية

٢٥ قياس أهمية الخصائص: ثلاث طرق مختلفة ٣٢٢

التباين الأولي، ومعاملات الانحدار، و permutation_importance

٢٦ المعلومات المتبادلة والاختيار غير المشرف ٣٣٢

تقدير MI، والاعتماد غير الخطي، والعيّنات الصغيرة

٤ النماذج الخطية ..... ١٠ فصلاً

العائلة الأولى التي يجب أن تتقنها قبل أي شيء آخر

٢٧ قبل الانحدار الخطي: من الخطأ إلى أقل المربعات ٣٤٧

مشكلة أقل المربعات، وتحليل SVD، والمتبقيات، والأقل تربيعاً

٢٨ تشخيص النموذج الخطي ٣٥٧

الفرضيات، والتعدد الخطي، و VIF، ومخطط QQ، والأثر

٢٩ الانحدار المنظّم: Ridge و ElasticNet ٣٦٨

لماذا يحتاج النموذج الخطي إلى عقوبة، وكيف تعمل L1 و L2

٣٠ Lasso: النموذج الخطي الذي يُصفّر ٣٧٨

التباعد، وهندسة المسار، واختيار alpha عبر التحقق المتقاطع

٣١ الانحدار المرن والمتعدد المهام ٣٨٦

MultiTaskLasso و MultiTaskElasticNet و enet_path و lars_path

٣٢ الانحدار المتسامح وتحمّل الصفوف الشاذة ٣٩٥

Huber و TheilSen و RANSAC و Quantile على بيانات ملوّثة

٣٣ الانحدار حسب التوزيع ٤٠٤

Poisson و Gamma و Tweedie — متى يتفوّق على النموذج الخطي

٣٤ الانحدار البيزي ٤١٢

BayesianRidge و ARDRegression — التنظيم الاحتمالي

٣٥ التصنيف اللوجيستي: من الاحتمال إلى القرار ٤٢١

الاشتقاق، و decision_function، ومتعدد الفئات، وحلول المعادلات

٣٦ المصنِّفات الخطية الأخرى ٤٣٢

Perceptron و PassiveAggressive و SGD و RidgeClassifier

٥ أشجار القرار ٥ فصلاً

نماذج تقسّم العالم إلى صناديق، وتشرح نفسها بلا رياضيات

٣٧ شجرة القرار: من سؤال إلى شجرة ٤٤٢

خوارزميتا ID٣ و C٤.٥ — الجيني والإنتروبيا ومكسب المعلومات

٣٨ شجرة الانحدار وتقسيم خطأ مربّع ٤٥٣

التباين، والانحراف المعياري، ومتوسط خطأ الشجرة

٣٩ التشذيب والإفراط في الملاءمة ٤٦٢

cost_complexity_pruning و ccp_alpha و decision_path والتصوّر

٤٠ ضبط شجرة القرار ٤٧٢

العمق، وحدّ الأوراق، و class_weight، والقيود، والقيم الناقصة

٤١ قراءة الشجرة وتفسيرها ٤٨١

export_text و plot_tree و feature_importances_ ومثال يدوي كامل

٦ المجموعات والأغلفة

٩ فصلاً

«ليس نموذج واحداً كافياً»

٤٢ التجميع بالعيّنات: Bagging ٤٩١

التمهيد بالعيّنات، والدمج بالتصويت، والتقدير خارج العيّنة OOB

٤٣ الغابة العشوائية ٥٠٠

التباين يُقتل بالتباين، و max_features، و ExtraTrees

٤٤ ExtraTrees وتضمين الأشجار العشوائية ٥٠٩

انقسامات عشوائية تماماً و RandomTreesEmbedding

٤٥ AdaBoost: التعلّم بالنقاط الصعبة ٥١٨

خوارزمية SAMME، وأوزان النماذج الضعيفة، وتحديث العيّنات

٤٦ تعزيز التدرّج: شرح كلمة بكلمة ٥٣٣

الدالة الخسارة، والبواقي، و learning_rate، و subsample

٤٧ HistGradientBoosting: المعيار الصناعي ٥٤٦

المدرّجات التكرارية، والقيم المفقودة الأصلية، والسرعة

٤٨ التصويت والتكديس ٥٦٢

VotingClassifier و StackingClassifier و VotingRegressor و StackingRegressor

٤٩ ضبط نموذج لمجموعة كاملة ٥٧٨

استراتيجية بحث صحيحة عبر Bagging و Boosting

٥٠ متى تستخدم أي مجموعة؟ ٥٩١

جدول قرارات يربط نوع البيانات بالاختيار الأمثل

٧ خوارزميات خاضعة للرقابة أخرى

٨ فصلاً

آلات المتجهات الداعمة، والجيران، وبيز، والتمييز

٥١ آلات المتجهات الداعمة: الحيلة الحقيقية ٦٠٧

الهامش الأقصى، و KKT، ولماذا لا تتأثر SVM بحجم المقياس

٥٢ أنوية SVM ومعاملاتها C و gamma ٦٢٢

RBF و poly و sigmoid، ومقارنة SVC مع LinearSVC

٥٣ SVR و OneClassSVM ٦٣٧

الانحدار بمتجهات داعمة، واكتشاف الشذوذ بلا تسميات

٥٤ الجيران الأقرب: k-NN ٦٥٠

التصنيف والانحدار، والأوزان، والمسافات، وأثر التطبيع

٥٥ بُنى البحث في الجيران ٦٦٤

KDTree و BallTree و RadiusNeighbors ومحوّلات الجيران

٥٦ بيز الساذج ٦٧٨

GaussianNB و MultinomialNB و ComplementNB و BernoulliNB

٥٧ تحليل التمييز ٦٩١

LDA و QDA وانكماش التغاير والتوزيع الغاوسي

٥٨ نماذج النواة والشبكات العصبية ٧٠٥

KernelRidge وتقريب النواة و BernoulliRBM و MLP

٨ التعلّم غير الخاضع للرقابة ١٣ فصلاً

عندما لا توجد إجابة نموذجية، بل يوجد بنية مخفية

٥٩ التعلّم غير الخاضع للرقابة: الخريطة ٧٢٢

ماذا يعني بلا تسميات، ومتى يكون مفيداً ومتى يكون فخاً

٦٠ K-Means وخوارزمية Lloyd ٧٣٥

المراكز، و k-means++، و Elbow، و Silhouette، والحدود المفهومية

٦١ نسخ K-Means العملية ٧٤٩

MiniBatchKMeans و BisectingKMeans و get_bin_seeds

٦٢ التقدير التلقائي: MeanShift و AffinityPropagation ٧٦٢

تقدير نطاق النواة، ونقل رسالة التفضيل بين العيّنات

٦٣ العناقيد الكثيفة: DBSCAN و OPTICS ٧٧٣

النقاط النواة، و eps، و min_samples، واستخراج العناقيد

٦٤ HDBSCAN: عدد كتل غير معروف مسبقاً ٧٨٥

تقدير الكثافات المتغيرة والثبات وتدرّجات الكثافة

٦٥ التجميع الهرمي والطوبولوجي ٧٩٩

AgglomerativeClustering و Ward و Spectral و FeatureAgglomeration

٦٦ كيف تقيس نجاح تجميعٍ بلا إجابة؟ ٨١٥

Silhouette و Davies-Bouldin و Calinski-Harabasz و ARI و NMI

٦٧ النماذج المزيجية الغاوسية ٨٢٩

خوارزمية EM وتفسير المكوّنات واختيار عدد المزيج

٦٨ تقليل الأبعاد الخطي: PCA ٨٤٣

تحليل SVD وتفسير المكوّنات ونسخ PCA العملية

٦٩ تقليل الأبعاد غير الخطي ٨٥٨

KernelPCA و Isomap و LLE و MDS و TSNE وموثوقية الإسقاط

٧٠ كشف الشذوذ وتقدير التباين ٨٧٢

IsolationForest و LOF والمسافة الماهالانوبية وانكماش التغاير

٧١ النصوص: التحليل الدلالي الخفي والمواضيع ٨٨٥

TF-IDF و TruncatedSVD و LatentDirichletAllocation

٩ التقييم والاختيار ..... ١٢ فصلاً

بلا قياس دقيق، كل تحسين وهم

٧٢ تقسيم البيانات: الخطوة التي تُفشل المشاريع ٨٩٨

train_test_split وترتيب العمليات والتسرب غير المقصود

٧٣ التحقق المتقاطع: كل أنواعه ٩١١

KFold و Stratified و Group و TimeSeries و LeavePOut و Predefined

٧٤ التقاط التقييمات وتتبع الأدوات ٩١٩

Parallel و cross_val_predict و cross_val_score و cross_validate

٧٥ مقاييس التصنيف: ما بعد الدقة ٩٢٧

الخلط، والدقة، والاسترجاع، و F1، والخصوصية، و MCC

٧٦ منحنيات القرار و ROC و PR ٩٤٣

det_curve و auc و precision_recall_curve و roc_curve

٧٧ ضبط عتبة القرار ٩٥٧

DetCurveDisplay و FixedThresholdClassifier و TunedThresholdClassifierCV

٧٨ مقاييس الانحدار ٩٧٢

MSE و RMSE و MAE و MAPE و MSLE و R^2 و d^2 و Pinball

٧٩ مقاييس متقدّمة ومجموعات غير مستوية ٩٨٧

متعدّد التسميات، والترتيب، وتوصيات DCG، ومعايير التغطية

٨٠ منحنى التعلّم ومنحنى التحقق ١٠٠١

قراءة تشخيصية للإفراط في الملاءمة والتباين والتحيّز

٨١ GridSearchCV: البحث المنضبط ١٠١٣

ParameterGrid و refit و _best_score و _cv_results

٨٢ البحث العشوائي والتوزيعات ١٠٢٧

RandomizedSearchCV و log-uniform وتوازن الاستكشاف

٨٣ اختبار الأهمية الإحصائية ١٠٤١

permutation_test_score والمقارنة الصحيحة بين نماذج

١٠ الأنابيب والتركيب — ٦ فصلاً

هندسة النماذج كخط إنتاج متكامل

٨٤ خط الأنابيب: عقد أمانة مع البيانات ١٠٥٥

fit و transform و predict، والتسلسل، وعناوين الخطوات

٨٥ ColumnTransformer: تركيب الأعمدة ١٠٧٢

الاختيار بالاسم والنمط، والمُحدِّد make_column_selector

٨٦ FeatureUnion والتركيبات المتقدمة ١٠٨٠

دمج مسارات متوازية، والتداخل، وحالات الاستخدام

٨٧ تحويل الهدف وتحويل المدخلات ١٠٨٨

TransformedTargetRegressor و FunctionTransformer وتحويل الهدف

٨٨ التخزين المؤقت والخطوط الثقيلة ١١١٠

joblib.Memory، ومتى يُستحق، ومتى يكون فخاً

٨٩ مخطط خط الأنابيب كاملاً ١١١٧

قالب معماري لمشروع إنتاجي من طرف إلى طرف

١١ المفاهيم المتقدّمة ٩ فصلاً

واجهات ١.٩، وتوجيه البيانات، وكتابة مُقدِّر خاص بك

٩٠ واجهة الاستدعاءات الجديدة ١١٤٠

ProgressBar و ScoringMonitor و with_callbacks واستدعاءات مخصّصة

٩١ توجيه البيانات الوصفية ١١٤٨

Metadata Routing: تمرير sample_weight و groups عبر Pipeline و GridSearchCV

٩٢ دعم واجهة Array API ١١٦٧

array_api_dispatch، والانتقال إلى وحدة معالجة الرسوميات

٩٣ إطار narwhals وتبادل البيانات ١١٧٤

DataFrame من أي مكتبة: pandas أو Polars أو Modin أو PyArrow

٩٤ واجهات العرض البياني في ١.٩ ..... ١١٩١

من ConfusionMatrixDisplay إلى PartialDependenceDisplay والعرض HTML- الجديد

٩٥ كتابة مُقدِّر متوافق من الصفر ١١٩٨

BaseEstimator و fit و predict و transform والوسوم والعقود

٩٦ التوسّع والتخصيص ١٢٢٨

الفئات الفرعية، والوسوم، وأنواع الأشجار، والاختبارات

٩٧ الأداء والتوازي والذاكرة ١٢٣٩

joblib.Parallel، وقياس الذاكرة، وأسرار التسريع

٩٨ معجم الأخطاء: من الرسالة إلى الحل ١٢٤٦

أكثر ثلاثين خطأ في scikit-learn، وسبب كل خطأ وحلّه

١٢ مشاريع واقعية كاملة ٦ فصلاً

من الفكرة إلى التقرير، بتفاصيل لا تُحذف

٩٩ مشروع ١: تصنيف البريد المزعج وتحليل المشاعر ١٢٥٩

TF-IDF، Bayes، LinearSVC ثم تقرير

١٠٠ مشروع ٢: توقّع مغادرة العملاء ١٢٦٨

تحليل RFM، والبيانات غير المتوازنة، ومعيار الربح

١٠١ مشروع ٣: كشف عمليات الاحتيال ١٢٧٦

البيانات شديدة الاختلال، وضبط العتبة، وتفسير النتائج

١٠٢ مشروع ٤: التعرّف على الأرقام المكتوبة بخط اليد ١٢٨٤

دورة حياة كاملة من التحميل حتى تقييم التصنيف متعدد الفئات

١٠٣ مشروع ٥: محرّك توصيات ١٢٩٢

الترشيح المبني على المحتوى، والتعاون، ومعايير NDCG

١٠٤ مشروع ٦: الصيانة التنبّؤية للآلات ١٢٩٩

بيانات المستشعرات، وكشف الشذوذ، وتنبيهات الصيانة

الجزء ١

الأسس: من الفكرة إلى أول نموذج

البنية المرجعية، والرياضيات الحاكمة، وطريقة التفكير التي لا تُكسر

ماذا يغطّي هذا الجزء

قبل أن تكتب سطر كود واحد، تحتاج إلى نموذج ذهني متماسك: ما هو «المُقدِّر»؟ لماذا يبقى محفوظاً في الذاكرة بعد التدريب؟ ولماذا يُقسَّم الداتا إلى مجموعتين لا غير؟ هذه القوة تبني الأرضية التي يقف عليها كل ما يلي. هنا نضع المصطلحات، والمعادلات، والعادات الذهبية التي ستصنع الفارق بين مهندس يتعلم ومهندس يتقن.

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ١ — ما هو تعلّم الآلة؟ رحلة من الفكرة إلى النظام

الإطار المفاهيمي الذي يحوّل البيانات إلى قرارات

الوحدة **١.٩ scikit-learn** الحجم **١٠٢٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تميّز الأنماط الأربعة لتعلّم الآلة وتختار لكلٍّ منها الأداة المناسبة
- أن تشرح بعينك الفرق بين التحيز وفرق التباين بالبيانات لا بالحفظ
- أن تقرأ منحنى التعلّم وتستنتج منه ما إذا كان النموذج ينقصه بيانات أم سعة
- أن تجيب سؤال «هل أحتاج تعلّم آلة هنا؟» بسؤال أفضل من جواب فوري

١.١ متى تكون القاعدة اليدوية أرخص وأدقّ من النموذج؟

ابدأ من الطرف المعاكس للنموذج: تخيّل أنك تقرّر أيّ بريدك يصل إلى صندوق الواجهة. القاعدة اليدوية ساذجة: «إذا احتوت الرسالة على كلمة "free" فوردياً فهو مزعج». هذه القاعدة تعمل جيداً تماماً حتى اللحظة التي يجد فيها أحدهم طريقة للإزعاج دون كلمة "free". عندها تنهار: إما أن تضيف قاعدة، أو تضيف استثناء، أو تعيد البناء من الصفر في كل مرة.

هذا هو السؤال الحقيقي خلف كل مشروع تعلّم آلة: **هل المشكلة قرار متكرر يجب أن يُتخذ بناءً على معطيات متغيّرة، بحيث يصير الأداء البديهي غير كافٍ؟**

لا أحد ينصحك أبداً باستخدام تعلّم الآلة في هذه الحالات:

الحالة	لماذا التعلّم الآلي قرار سيّئ
القواعد معروفة، قليلة، وثابتة	شيفرة `if` أرخص وأسرع وأوضح من أي نموذج
يجب تفسير كل قرار قانونياً	النموذج يُعطي إجابة؛ ولا يعطي ضماناً
لا يوجد اتصال بين المدخلات والمخرج	لا يوجد ما يُتعلَّم أصلاً
التكلفة من تعبئة البيانات تفوق أي مكسب	المشروع كلّف أكثر مما يوفّر

القاعدة العملية التي سترددّها في هذا الكتاب مراراً: **ابدأ بأبسط نموذج يمكن أن يعمل، ثم برهن أنت تحتاج شيئاً أكبر**. أغلب منظومات الإنتاج الراشدة تبدأ بنموذج خطي بسيط، وتقيس، ثم تعقّد فقط حين يعطي النموذج البسيط دليلاً على أنه لم يعد كافياً.

```python
مسألة صناعية مصغّرة: هل تحتاج نموذجاً أصلاً؟
```

```python
from sklearn.datasets import make_classification, make_regression
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge
from sklearn.metrics import r2_score
from sklearn.model_selection import StratifiedKFold, cross_val_score,
learning_curve, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

Xc, yc = make_classification(n_samples=1200, n_features=6, n_informative=4,
                             n_redundant=1, class_sep=1.4, random_state=42)
Xr, yr = make_regression(n_samples=800, n_features=5, noise=12.0,
                         random_state=42)

print("classification :", Xc.shape, "-> class counts", np.bincount(yc))
print("regression     :", Xr.shape, "-> target mean %.2f  std %.2f"
      % (yr.mean(), yr.std()))
print("feature means  :", np.round(Xc.mean(axis=0), 3))
print("feature stds   :", np.round(Xc.std(axis=0), 3))
```

لاحظ رقمين في هذا الناتج. الأول: الفئات شبه متوازنة (٦٠٤ و٥٩٦)، وهذه معلومة حسابية هامة عند اختيار المقياس لاحقاً. الثاني: متوسّطات الأعمدة ليست صفراً بالضبط، وانحرافاتها المعيارية تتضاعف مرّتين من عمود لآخر. هذه الأرقام ليست تفصيلاً — بل هي أول قراراتك المعمارية في الفصل الثالث.

ولا تنسَ السؤال الذي بدأنا به. على بعض المسائل، القاعدة اليدوية تتفوّق:

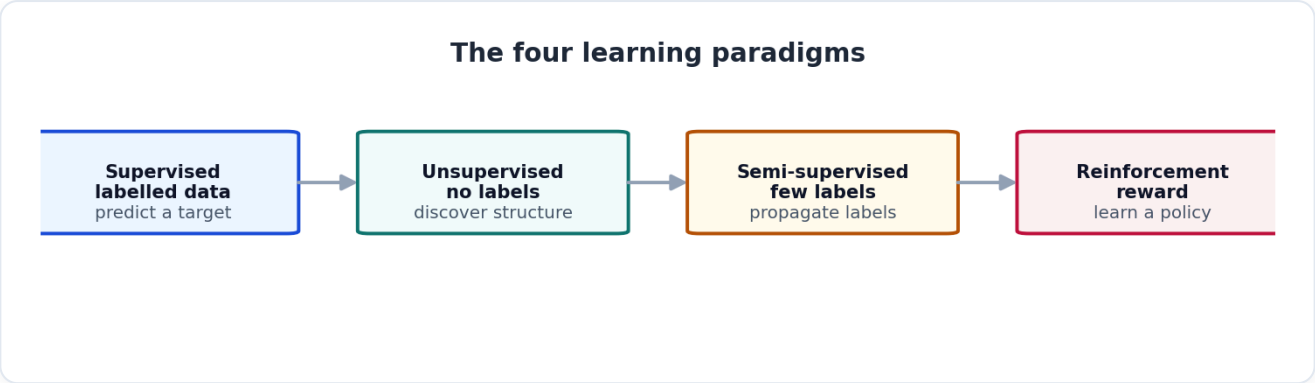
```python
# حين تكون القاعدة اليدوية هي الحل الصحيح

rng = np.random.default_rng(3)
ang = rng.uniform(0, 2 * np.pi, 600)
rad = rng.uniform(0.3, 3.0, 600)
Xc = np.column_stack([rad * np.cos(ang), rad * np.sin(ang)])
yc = (rad > 2.0).astype(int)

rule = (np.hypot(Xc[:, 0], Xc[:, 1]) > 2.0).astype(int)
logreg = LogisticRegression(max_iter=2000).fit(Xc, yc)
print("حدّ قرار دائري مثالي:")
print("  4.% : قاعدة بالمسافة" % (rule == yc).mean())
print("  4.% :        خط مستقيم" % logreg.score(Xc, yc))

xq, yq = make_regression(n_samples=60, n_features=1, noise=6.0, random_state=0)
xq = xq.ravel()
med = np.median(xq)
median_pred = np.where(xq <= med, yq[xq <= med].mean(), yq[xq > med].mean())
lin = LinearRegression().fit(xq.reshape(-1, 1), yq)
print("\nمسألة بعلاقة نعّاحة وضجيج:")
print("  قاعدة الوسيط     : R2 = %.4f" % r2_score(yq, median_pred))
print("  الانحدار الخطي  : R2 = %.4f" % r2_score(yq, lin.predict(xq.reshape(-1, 1))))
```

القاعدة اليدوية هنا لا تُخطئ ولا موضعاً واحداً، بينما الخط المستقيم يخطئ في ٣٧٪ من النقاط لأن الحاجز دائري لا خطّي. وفي المثال الثاني انقلب ميزان المقارنة: قاعدة الوسيط تخسر بـ `R²` قدره ٠٫٦٢، والانحدار الخطي يفوز بـ ٠٫٩٦. **القرار ليس «أيّهما أفضل» بل «هل شكل العلاقة يطابق الأداة»** — وهذا بالضبط ما تبحث عنه عمليات التعلّم الآلي. لاحظ أيضاً أن كلفة القاعدة يمكن أن تكون باهظة: `kNN` في الفصل ٥٤ هو في الحقيقة قاعدة «المسافة إلى أقرب جار» مكتوبة بلغة تعمّم.

القاعدة العملية التي سنردّدها في هذا الكتاب مراراً: **ابدأ بأبسط نموذج يمكن أن يعمل، ثم برهن أنت تحتاج شيئاً أكبر**. أغلب منظومات الإنتاج الراشدة تبدأ بنموذج خطي بسيط، وتقيس، ثم تعقّد فقط حين يعطي النموذج البسيط دليلاً على أنه لم يعد كافياً.

١.٢ الأنماط الأربعة: أين يقع مشروعك؟

قبل تحميل أي بيانات، اسمحأنفسك يجيب على سؤال التصنيف. لا يوجد نموذج واحد يصلح لكل الأنماط؛ كل نمط له بنية بيانات مختلفة تماماً وطريقة قياس نجاح مختلفة.



الشكل ١ مخطط يوضّح الأنماط الأربعة لتعلّم الآلة: بيانات موسومة تنبّأ بهدف، وبيانات بلا وسم تكتشف بنية، وبيانات شبه موسومة تنتقل بوسم قليل، وبيانات مكافأة تعلّم سياسة

النمط الأول **خاضع للرقابة** (supervised): لديك مدخلات `X` ومخرجات `y` مسمّاة. هذا أكثر الأنماط انتشاراً في الصناعة، وكل الفصول من الثاني إلى العاشر تتعامل معه. النموذج يتعلّم الدالة التي تربط بينهما.

النمط الثاني **غير خاضع للرقابة** (unsupervised): لا مخرجات والهدف هنا ليس التنبّؤ بل **الوصف**: أين العناقيد؟ كم بُعداً حقيقياً في البيانات؟ هل هناك شذوذ؟ تقييمه لا يكون بدقّة بل بمعيار داخلي (silhouette مثلاً) أو بجدوى الأعمال.

النمط الثالث **شبه خاضع للرقابة** (semi-supervised): بيانات موسومة قليلة جداً وسطوف كبيرة. تظهر قيمته في الصور والأصوص حيث لا يمكنك وسم كل ملف.

النمط الرابع **معزَّز** (reinforcement): لا يوجد مخرجات أصلاً، بل مكافأة. القرار يُتعلَّم بالتجربة والخطأ. هذا النمط خارج نطاق هذا الكتاب؛ مكتبة scikit-learn لا تقدّم له واجهة تعليمية، وإن كنت تحتاجه فانظر إلى مكتبة مستقلة.

وسنرى لاحقاً في الفصل ٦١ كيف يبدو النمط الثاني عملياً. يكفي هنا أن ترى أن المخرج ليس وسماً بل **تقسيم للمجموعة**، وقد ينتهي بخمسة عناقيد رغم أن الحقيقة أربعة. لا يوجد في هذا النمط «صواب» يُقاس بدقّة، بل معايير داخلية تصف جودة التقسيم نفسه.

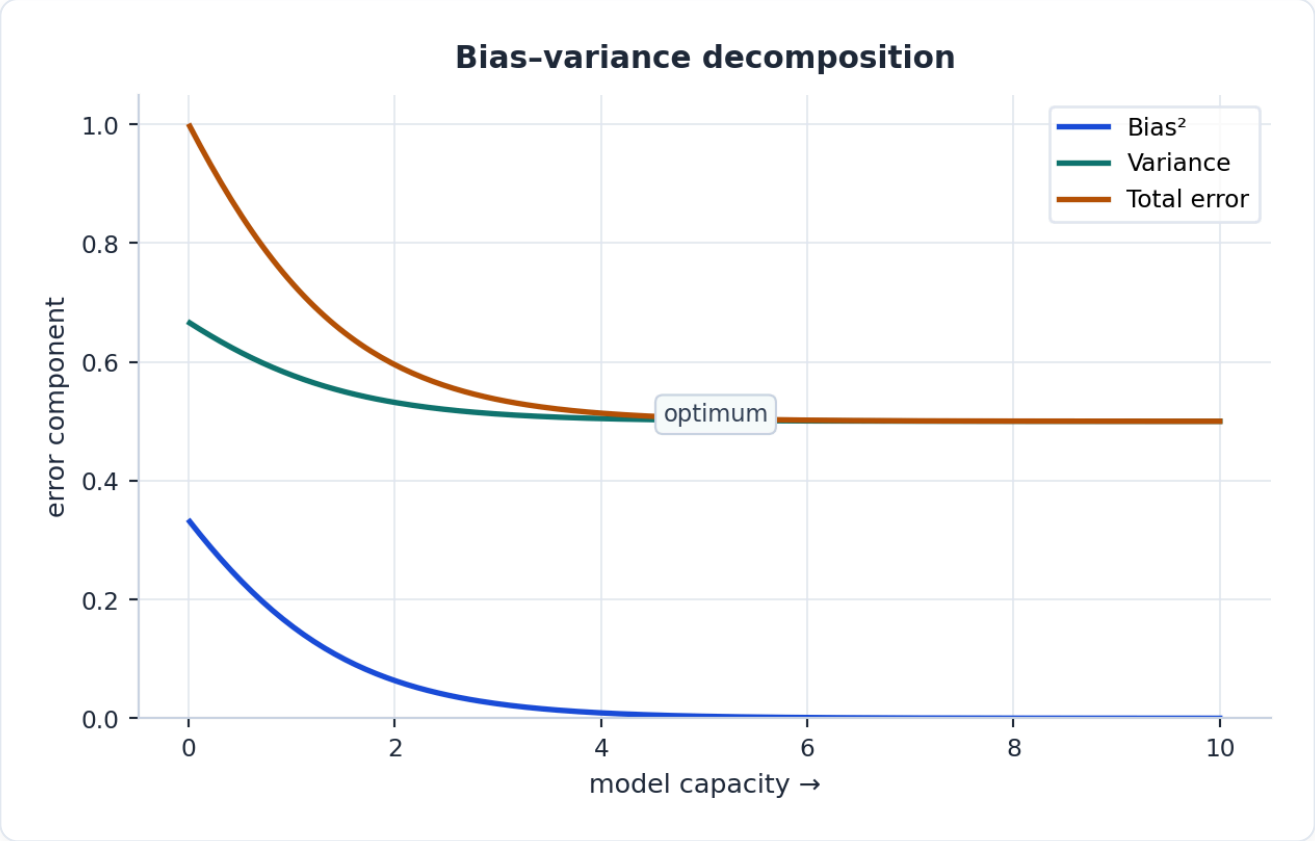
```python
# النمط غير الخاضع للرقابة: ما الذي يُخرجه فعلاً؟
from sklearn.cluster import DBSCAN, KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score, silhouette_score

Xu, truth = make_blobs(n_samples=420, centers=4, cluster_std=0.7,
random_state=42)
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(Xu)
db = DBSCAN(eps=0.9, min_samples=8).fit(Xu)
n_db = len(set(db.labels_)) - int(-1 in db.labels_)

print(f"عدد المجموعات المكتشفة : KMeans {len(np.unique(km.labels_))}"
      f"   DBSCAN {n_db}   (الحقيقة {len(np.unique(truth))})")
print(f"adjusted_rand_score    : KMeans {adjusted_rand_score(truth,
km.labels_):.4f}"
      f"   DBSCAN {adjusted_rand_score(truth, db.labels_):.4f}")
print(f"silhouette_score       : KMeans {silhouette_score(Xu, km.labels_):.4f}"
      f"   DBSCAN {silhouette_score(Xu, db.labels_):.4f}")
print(f"كشذوذ DBSCAN نقاط وصفها: {int((db.labels_ == -1).sum())}")
```

لاحظ أن `KMeans` طُلب منه مسبقاً أن يجد أربعة عناقيد، بينما `DBSCAN` اكتشفها بنفسه ووسم بعض النقاط بأنها شذوذ (الوسم `-1`). وفي النمط الخاضع للرقابة كان من السهل أن تقول «صنّف الصف ٧ و٧٤ صحيحاً أم خطأ»؛ هنا لا يوجد جواب مرجعي، فتنتقل إلى معيار **داخلي** مثل `silhouette_score` يقيس مدى بُعد كل نقطة عن مجاورتها مقارنةً بالمجموعات الأخرى.

> **الفكرة المحورية**
>
> خوارزمية تعلّم الآلة ليست ذكاءً اصطناعياً بالمعنى الغامض. هي **دالة حسابية محدّدة سلفاً**، لها عدد محدود من المعاملات تضبطها، ومدخلها ومخرجها أرقام. كل ما في الموضوع هو اختيار هذه الدالة وضبط معاملاتها على بيانات صحيحة.

١.٣ المسألة ليست الدقة — بل التعميم

هذا هو المفهوم الذي يفصل بين من «يتعلّم» و«يقول إنه تعلّم». أنت لا تنشر دقة ٩٧٪ على البيانات التي درّبتها عليها؛ أنت تنشر دقة **غير مرئية بعد**.



الشكل ٢ منحنى يوضّح تحلّل خطأ الاختبار إلى تحيّز مربّع وتباين زائد وخطأ إجمالي، بأدنى مجموع عند سعة متوسطة من النموذج

الصورة هي تقسيم خطأ الاختبار الكلاسيكي. النموذج بسيط جداً (شجرة عمق واحد) فإنه يقارب البنية العامة ولا يلحق التفاصيل: **تحيّز عالٍ** (bias)، أي خطأ نظامي في الافتراض نفسه. النموذج معقّد جداً بلا ضبط فيتعلّم ضجيج التدريب حرفياً: **تباين عالٍ** (variance)، أي أن شجرة واحدة تكفي، وإعادة التدريب ببذرة أخرى تعطي شجرة مختلفة تماماً. بينهما نقطة توازن يقال لها «الحجم الأمثل».

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2$$

انتبه لحدّ الضجيج: هناك حدّ لا يمكن لأي خوارزمية أن تتخطّاه. فإذا كانت البيانات رديئة، أو سعة النموذج غير كافية، قارب الخطأ هذا الحدّ. ولهذا نتوقّف عن تحسين النموذج حين يقترب الخطأ منه.

والطريقة العملية التي تخبرك أين تقف هي منحنى التعلّم: زد حجم التدريب وراقب الفجوة.

```python
منحنى التعلّم: هل أزيد بيانات أم أزيد تعقيداً؟
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score,
learning_curve

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
candidates = {
    "DummyClassifier(strategy='most_frequent')":
DummyClassifier(strategy="most_frequent"),
    "LogisticRegression(max_iter=2000)": LogisticRegression(max_iter=2000),
    "KNeighborsClassifier(n_neighbors=5)": KNeighborsClassifier(n_neighbors=5),
}
print(f"{'model':<38} {'cv accuracy':>12} {'std':>8}")
for name, est in candidates.items():
    s = cross_val_score(est, Xc, yc, cv=cv, scoring="accuracy")
    print(f"{name:<38} {s.mean():>12.4f} {s.std():>8.4f}")

lr_model = candidates["LogisticRegression(max_iter=2000)"]
sizes, train_scores, val_scores = learning_curve(
    lr_model, Xc, yc, cv=cv, train_sizes=np.linspace(0.1, 1.0, 8),
    scoring="accuracy", n_jobs=1)

print("\nمنحنى التعلّم للانحدار اللوجيستي:")
print(pd.DataFrame({
    "n_train": sizes.astype(int),
    "train_acc": train_scores.mean(axis=1).round(4),
    "val_acc": val_scores.mean(axis=1).round(4),
    "gap": (train_scores.mean(axis=1) - val_scores.mean(axis=1)).round(4),
}).to_string(index=False))
```

اقرأ الجدول زمنياً من الأعلى إلى الأسفل. الفجوة بين عمودي `train_acc` و `val_acc` هي فجوة التعميم: عند أول ٩٦ صفاً تكون ٠٫٠٥٥، ثم تهبط تحت ٠٫٠١ ابتداءً من ٣٤٢ صفاً وتبقى كذلك. هذا يعني أن النموذج **انتهى من استغلال البيانات الجديدة**: كل سطر إضافي في الجدول يمنحه أقل من نصف نقطة مئوية. النموذج هنا مقيَّد بسعته، لا ببياناته.

ولهذا يبدو خطأً أن تُصدر النموذج بعد ١٢٠٠ صف. لو كان منحنى التعلّم ما يزال يصعد، فالبيانات هي الحلّ. ولأنّه مستقرّ عند ٠٫٨١، فالبيانات ليست الحلّ — بل ضبط النموذج.

١.٤ السعة الزائدة: لماذا ينهار الأداء خارج عيّنته؟

لنجعل الإفراط في الملاءمة ملموساً. المسألة نفسها ثلاث مرات: خط مستقيم، ثم كثير حدود من الدرجة الثالثة، ثم كثير حدود من الدرجة الثانية عشرة. النموذج في الحالات الثلاث هو نفسه — `Ridge` منتظماً — ولا يتغيّر إلا **عدد المعاملات**.

ثلاث درجات سَعة على مسألة انحدار واحدة

```python
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

Xr, yr = make_regression(n_samples=120, n_features=3, noise=28.0,
random_state=42)
Xtr, Xte, ytr, yte = train_test_split(Xr, yr, test_size=0.3, random_state=42)
print(f"train {Xtr.shape}   test {Xte.shape}   target std {yr.std():.2f}")

print(f"\n{'degree':>7} {'n_params':>10} {'train R2':>10} {'test R2':>10}")
for deg in (1, 3, 12):
    pipe = make_pipeline(PolynomialFeatures(degree=deg),
                         StandardScaler(),
                         Ridge(alpha=1.0))
    pipe.fit(Xtr, ytr)
    n_params = len(pipe[-1].coef_) + 1
    print(f"{deg:>7} {n_params:>10} {pipe.score(Xtr, ytr):>10.4f} "
          f"{pipe.score(Xte, yte):>10.4f}")
```

اقرأ السطر الأخير. الدرجة الثانية عشرة **تفوز** على التدريب (٠٫٩٨) و**تنهار** على الاختبار (-١٧٫٠٩). معامل الـ `R²` السالب ليس خطأ في الحساب: معناه أن النموذج أسوأ من قاعدة تراهن دائماً على متوسّط الهدف. عملياً هذا يعني أن مقترحاته أبعد عن الحقيقة ممّا تبعده أي نقطة أخرى في البيانات.

هذه ليست حادثة نادرة بل **سلوك نهائي متوقّع**: كل معامل إضافي هو قدرة إضافية على حفظ كل نقطة. النموذج الذي يحفظ كل شيء لا يحكم على شيء.

ودلالتها الرياضية أن الخسارة نفسها لها حدّان يتصارعان. الانتظام (`ridge` ، `lasso`) يضيف عقوبة على حجم المعاملات، فيقايض جزءاً من ملاءمة التدريب مقابل ثبات أكبر:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} [\sum_{i=1}^{n} (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2 + \lambda \|\mathbf{w}\|_2^2]$$

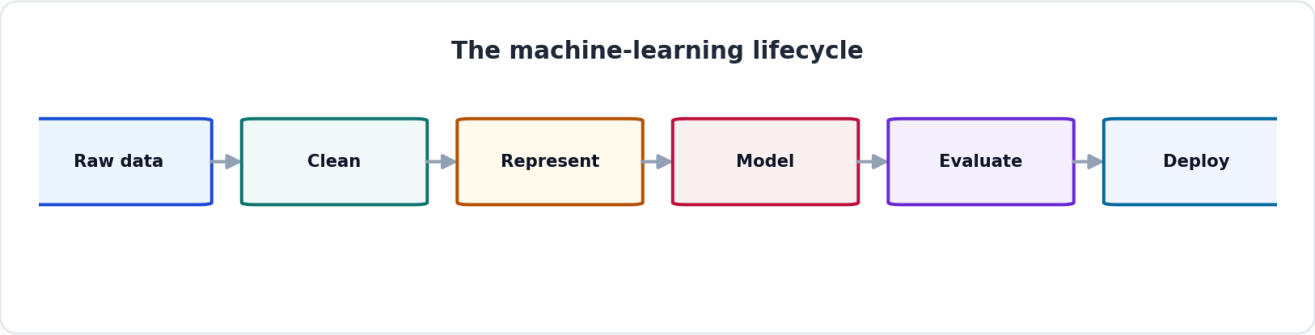
`n` هو عدد الصفوف و `d` عدد المعاملات، و `lambda` هو معامل الانتظام: مقبض واحد يحكم المفاضلة كلها. عند صفر تكون المربعات الصافية وحدها (ملاءمة كاملة)، وكلما ارتفع تحوّل النموذج إلى أبسط ما يمكن. ولهذا فإن اختيار معامل الانتظام ليس تفصيلاً تجميلياً بل هو جوهر «التعلّم الآلي» في بضع عشرات الأسطر.

الظاهرة	التشخيص	العلاج
تدريب ممتاز، اختبار ضعيف	سعة زائدة	تقليل `max_depth`، زيادة `C` في الانتظام، تجميع
تدريب ضعيف، اختبار ضعيف	سعة ناقصة	نموذج أقوى، ميزات أفضل، بيانات أكثر
تدريب ممتاز واختبار ممتاز	غالباً تسرّب	راجع تقسيم البيانات قبل أن تفرح
تدريب ضعيف واختبار ممتاز	خطأ عشوائي أو تسرّب	أعد التقسيم، تحقق من عدم التسرّب

> **أشيع خطأ في مشاريع المبتدئين**
>
> أن يبلّغ عن النموذج على نفس البيانات التي دُرِّب عليها، ثم يعلن «٩٧٪ دقة». هذا ليس إنجازاً. هذا قياس ذاكرة لا قياس قدرة. الفصل ٧٢ يشرح تقسيم البيانات الذي يمنع هذا الخطأ، لكن اعرف من الآن أن أي رقم دقة بلا مجموعة اختبار مستقلة هو رقم بلا معنى.

١.٥ المسار من البيانات إلى النظام

النموذج ليس المنتج. منتجك هو **خط أنابيب موثّق** ينتهي بمخرج يتّخذ قراراً ويعرف كيف نُقيس نجاحه.



الشكل ٣ مخطط تدفّق يمرّ بالبيانات الخام فالتنظيف فتمثيل البيانات فالنموذج فالتقييم فالنشر

خمس نقلات في الصورة. كل نقلة هي مكان يمكن أن تنكسر فيه الجودة:

خط أنابيب مصغّر: كل شيء في كائن واحد — python

```python
from sklearn.datasets import make_moons
from sklearn.neighbors import KNeighborsClassifier

Xm, ym = make_moons(n_samples=300, noise=0.45, random_state=42)
Xm_tr, Xm_te, ym_tr, ym_te = train_test_split(Xm, ym, test_size=0.3,
                                              random_state=42, stratify=ym)
knn_pipe = make_pipeline(StandardScaler(),
                         KNeighborsClassifier(n_neighbors=7))
knn_pipe.fit(Xm_tr, ym_tr)

print("خط الأنابيب:", " -> ".join(name for name, _ in knn_pipe.steps))
print("تدريب f}   اختبار f}" % (knn_pipe.score(Xm_tr, ym_tr),
                                 knn_pipe.score(Xm_te, ym_te)))
print("احتمال الصنف 1 لأول ثلاث نقاط:",
      np.round(knn_pipe.predict_proba(Xm_te[:3])[:, 1], 3))

print("\nأثر اختيار الجيران:")
for k in (1, 5, 25):
    knn_pipe[-1].set_params(n_neighbors=k)
    print(f"  k={k:>2}  تدريب {knn_pipe.score(Xm_tr, ym_tr):.4f}"
          f"   اختبار {knn_pipe.score(Xm_te, ym_te):.4f}")
```

انتبه إلى `k=1`: دقّة تدريبه ١٫٠٠ لأن الجار الأقرب هو النقطة نفسها. هذه ليست موهبة بل تشخيص مباشر: ذاكرة بلا تعميم، ودرجات الاختبار عنده ٠٫٧٠ فقط. في المقابل، `k=25` ينسى بعض البنية ويرفع دقّة الاختبار إلى ٠٫٨٢. القمّة الحقيقية تقع في المنتصف عند `k=5` بدقّة اختبار ٠٫٨١ ودقّة تدريب ٠٫٨٨. الضبط الجيّد هو اختيار التوازن بين الاثنين، لا اختيار الرقم الأعلى في أي عمود.

هذا الكائن قابل للتسلسل والحفظ ويُعاد إنتاجه بأمر واحد. الفصل ٨٤ يشرح `Pipeline` بالتفصيل؛ هنا نكفي بمبدئه: **الخطأ في الترتيب جسيم**. لو محوّل التحجيم تدرّب على كامل البيانات قبل التقسيم، تسرّب المعلومات ينتج دقة وهمية.

من دفتر تجربة حقيقي

في تطبيق ائتمان صغير، بنى فريق هندسي نموذجاً «جيداً» بدقّة ٠٫٨٧ على مجموعة التقييم. المشكلة أن سلسلة التحقق كانت تشترك مع التدريب في حساب المتوسّطات وفي اختيار أفضل عتبة. بعد نقل كل العمليات داخل `Pipeline` نزلت الدقة الحقيقية إلى ٠٫٧٩. الرقم الأعلى لم يكن دقة أعلى — كان تسرّباً مُقنعاً. كل نقلة في مخطط الحياة أعلاه هي بالضبط مكاناً يمكن أن يدخل منه التسرّب.

١.٦ لا توجد خوارزمية أفضل — بل مطابقة

الوقوع في مغالطة المفاضلة يمرّ بسرعة: ينظر أحدهم إلى جدول دقة في ورقة بحثية ويحمل الرقم إلى مشروعه. هذا خطأ منهجي، لأن جداول الدقة هذه تقيس ثلاثة أشياء معاً في آن واحد: **نوع البيانات، وحجم العيّنة، ونطاق الضبط**.

نوع البيانات	النماذج التي تتفوّق عادةً	لماذا
جدولي صغير، أبعاد قليلة	أشجار ومجموعات	التقاط التفاعل بلا معاملات كثيرة
جدولي كبير، أبعاد عالية	خطية ومنتظمة	تكلفة العيّنة تهدّم الأشجار العميقة
نص متفرّق، آلاف الأبعاد	خطية مع تمثيل متفرّق	المسافة في فراغ واسع
صور	مجموعات تعزيز وشبكات	العيّنات كثيرة والعلاقة معقّدة
سلاسل زمنية	نماذج لها بنية زمنية	الترتيب جزء من المعلومة

وثلاثة استنتاجات ثابتة في كل مراجعات منهجية تقريباً:

وهذا هو المعنى الحقيقي لعبارة «النموذج الأفضل». هي ليست خاصية للنموذج، بل خاصية **ثلاثية: (نموذج، بيانات، ميزانية)**. ولهذا يفشل السؤال «ما أفضل خوارزمية؟» بوصفه سؤالاً، وينجح سؤاله المفيد: «ما أفضل خوارزمية لبياناتي، بميزانية تدريب لا تتجاوز عشر دقائق؟».

١.٧ تمارين الفصل

التمرين ١.١

استعمل `make_classification` بـ `n_samples=800` و `n_features=10` و `n_informative=3` و `random_state=7`، وقس `LogisticRegression` و `RandomForestClassifier` و `KNeighborsClassifier` بـ `cross_val_score`. ارسم بيانياً يربط حجم التدريب بدقة التحقق. في أي حجم يتوقف النموذج عن الاستفادة؟ بزّر برقم من الجدول لا بحدس.

التمرين ١.٢

على بيانات `make_moons(noise=0.30)`، قارن `KNeighborsClassifier` بـ k من ١ إلى

التمرين ١.٣

اجمع في قاموسك من هذا الفصل خمسة مؤشرات على أنك تسير نحو الإفراط في الملاءمة، وثلاثة مؤشرات على أنك تسير نحو سعة ناقصة. اشرح لكل مؤشر لماذا يدلّ على ما يدلّ عليه، لا لماذا «يبدو سيئاً».

التمرين ١.٤

ابحث عن مثال في عملك أو دراستك يمكن حلّه بقاعدة ثابتة. اكتبه في خمس أسطر، ثم اكتب ما الذي يجعل انتقاله إلى تعلّم الآلة مبرَّراً أو غير مبرَّر.

١.٨ حلول بعض التمارين

✅ **حلّ التمرين ١.ء**

الأرقام تتفاوت مع بيئتك، ولكن المنطق من المصدر. أكبر نتائج إجراءات الخطة أصل:

حجم التدريب	انحدار لوجيستي	غابة عشوائية
٦٤	٠٫٨٦	٠٫٨٧
٢٢٨	٠٫٨٦	٠٫٨٨
٤٧٥	٠٫٨٧	٠٫٨٩
٦٤٠	٠٫٨٧	٠٫٨٩

انظر إلى الغابة العشوائية: مكسبها الكلي من ٦٤ صفاً إلى ٦٤٠ صفاً هو **نقطة واحدة ونصف مئوية** موزّعة على عشر مضاعفات. وبعد ٤٧٥ صفاً صار المكسب ٠٫٦٦ نقطة ثم صفراً. هذا هو جواب التمرين: يتوقف النموذج عن الاستفادة عند بضع مئات من الصفوف، ومجمع البيانات بعد ذلك إهدار. والانتقال إلى الغابة نفسها لم يعطِ هنا أكثر من نقطة مئوية واحدة عن اللوجيستي، رغم أن كلفة تدريبها أعلى بكثير.

المعيار العملي: توقّف عن جمع البيانات عندما لا يزيد منحنى التحقّق بأكثر من نصف نقطة مئوية لكل مضاعفة للحجم.

✅ **حلّ التمرين ١.٢**

ستجد نمطاً مألوفاً: دقّة التحقق **ترتفع** بمقدار عشرة نقاط مئوية من `k=1` حتى القمّة (`k=15` تعطي دقّة اختبار ٠٫٩٣) ثم تبدأ بالانخفاض عند `k=25`. السبب عددي لا حدسي: بعد تقسيم ما يقارب ٦٠٠ عيّنة بنسبة ٨٠/٢٠ يبقى لكل صنف نحو ٢٤٠ صفاً في التدريب. عند `k=1` القرار مبني على نقطة واحدة فضوضاء؛ وعند `k=25` يصبح مبنياً على متوسّط نحو ٩٪ من العنقود فيفقد حدّة الفصل. وبين الرقمين يقع أفضل توازن.

١.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
«الخوارزمية A أفضل دائماً»	نموذج ممتاز على بياناتك وسابق على غيرها	اختر بـ `cross_val_score` على بياناتك أنت
الإفراط في الملاءمة بلا انتباه	فجوة تدريب/اختبار تكبر كلما زادت المعاملات	ابنِ أبسط نموذج ممكن أولاً
قياس الأداء على بيانات التدريب	رقم قائم على الذاكرة لا على القدرة	احجز مجموعة اختبار لم تلمسها
القفز إلى الشبكة العصبية أولاً	كلفة ووقت بلا فائدة على بيانات جدولية	ابدأ بالنموذج الخطي
نسيان الفحص البصري للبيانات	نموذج ممتاز يتعلّم عموداً تسريباً	ارسم التوزيعات والارتباطات أولاً
قياس `train_score` فقط في السجلّ	لا إنذار مبكر على أي تدهور	سجّل نتيجة كل طيّة

١.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تعلّم الآلة	بناء دالة من بيانات بدل كتابتها يدوياً	machine learning
خاضع للرقابة	تعلّم من مدخلات مع مخرجات موسومة	supervised learning
غير خاضع للرقابة	اكتشاف بنية بلا مخرجات	unsupervised learning
معزَّز	تعلّم سياسة من مكافآت	reinforcement learning
تحيّز	خطأ نظامي في افتراض النموذج	bias
تباين	حساسية النموذج لتغيّر عيّنة التدريب	variance
الإفراط في الملاءمة	حفظ الضجيج فتتضرّر دقة الاختبار	overfitting
سعة النموذج	قدرته على تمثيل دوال معقّدة	model capacity
التعميم	أداء النموذج على بيانات غير مرئية	generalization
التسرّب	دخول معلومات الاختبار في التدريب	data leakage
منحنى التعلّم	دقة الأداء بدلالة حجم التدريب	learning curve

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٢ — منظوم بايثون العلمية: أين يقع scikit-learn؟

NumPy و SciPy و pandas و Matplotlib — ومَن المسؤول عن ماذا

الوحدة **scikit-learn ١.٩** الحجم **٧٢٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن ترسم خريطة المنظوم العلمي وتحدّد مسؤولية كل مكتبة فيه
- أن تشرح لماذا المصفوفة في NumPy ليست قائمة أقواس بل كائن له ذاكرة ودلالات محاور
- أن تعرف متى تستدعي SciPy ومتى تكفيك NumPy وحدها
- أن تقرأ كائن Bunch من scikit-learn وتستخرج منه ما تحتاجه بالاسم

٢.١ من أين تأتي هذه المنظومة؟

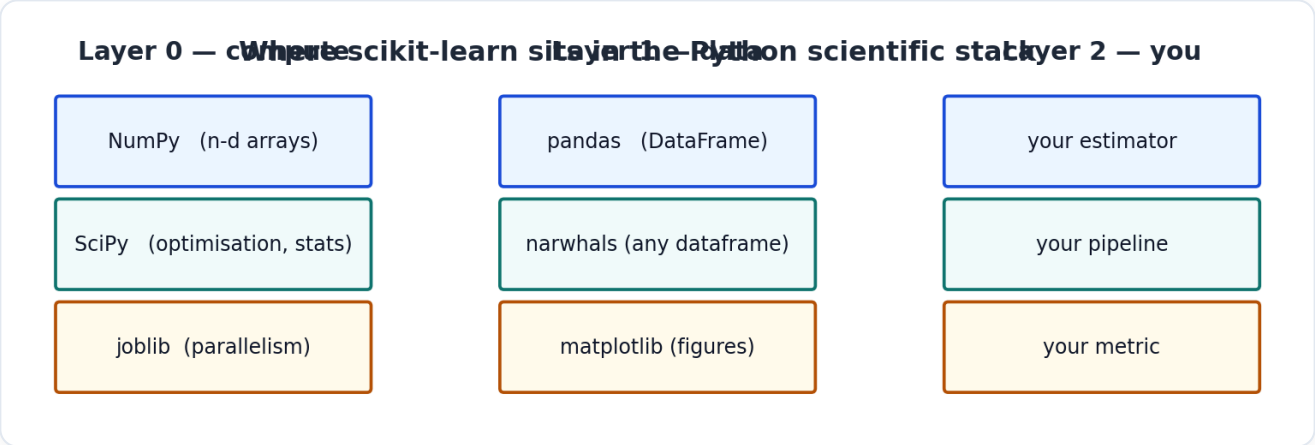
تخيّل أنك تبني بيتاً. أنت لا تصبّ الأساس بنفسك، ولا تخشب الجدران، ولا تفرد الأسقف. تستعمل عمّالاً يتقنون كل حرفة. وهذا بالضبط ما يفعله بايثون في مجال تعلّم الآلة: **مكتبة متخصّصة في كل طبقة، ومكتبة واحدة تجمعها**.

قبل نحو عشرين سنة كان الوضع مختلفاً. كانت كل أداة مكتوبة بلغة مختلفة، وكل مكتبة تتحدّث عن البيانات بصيغتها الخاصة: `R` أعطتك `data.frame`، و `MATLAB` أعطاك `matrix`، و `C++` أعطاك `vector<double>`. زرعتها من أداة إلى أخرى، وكتبت محوّلات في كل مرة.

```python
# نسخة الكتاب: من علوم البيانات إلى هذه الأدوات
import platform
import sys

import scipy
import sklearn

print("python    ", sys.version.split()[0], platform.python_implementation())
print("numpy     ", np.__version__)
print("scipy     ", scipy.__version__)
print("pandas    ", pd.__version__)
print("matplotlib", mpl.__version__)
print("sklearn   ", sklearn.__version__)
print("\nscipy.linalg يوفّر:", [n for n in dir(scipy.linalg)
                               if n in ("solve", "inv", "lstsq", "svd", "qr")])
```

هذه الأرقام ليست زينة. النسخة محصورة في الكتاب كلّه، وكل سلوك موصوف مربوط بها. وحين يذكر الكتاب أن الجيران القريبين صاروا أسرع، فذلك في هذا الإصدار فقط.

٢.٢ الطبقات الخمس: من يُنفّذ ومن يقرّر



الشكل ٤ خريطة ثلاث طبقات للمنظوم العلمي: طبقة الحوسبة وفيها NumPy و SciPy و joblib، وطبقة البيانات وفيها pandas و narwhals و matplotlib، وطبقة المستخدم وفيها المُقدِّر وخط الأنابيب والمقياس

الصورة مقصودة بهذا الترتيب لأن الاعتماد يسير من الأسفل إلى الأعلى: كودك **يستدعي** scikit-learn، و scikit-learn **يستدعي** SciPy و NumPy، و SciPy **يستدعي** NumPy. لا تستدعي NumPy شيئاً أعلى منها.

الطبقة	المكتبة	مسؤوليتها الوحيدة
الحوسبة	`numpy`	مصفوفات الأرقام، الحساب عليها، الذاكرة
الحوسبة	`scipy`	خوارزميات علمية: تحسين، إحصاء، جبر خطي، تكامل
الحوسبة	`joblib`	التوازي والتخزين المؤقّت للنتائج
البيانات	`pandas`	الجداول ذات الأعمدة المسمّاة والصفوف المفهرسة
البيانات	`narwhals`	طبقة توافق مع أي إطار جداول (اعتماد جديد في ١.٩)
البيانات	`matplotlib`	الرسوم والأشكال
التعلّم	`sklearn`	الخوارزميات والمقياسات وخطوط الأنابيب
أنت	كودك	القرار: أي بيانات، أي نموذج، أي مقياس

القاعدة التي تلخّص الجدول: **scikit-learn لا يخزّن بيانات ولا يرسم ولا يدير ملفات**. ما يفعله محدود: يتعلّم من `X` و `y`، ويحفظ ما تعلّمه في سمات داخل الكائن، ثم يجيب. كل ما عداه مهمتك أنت.

ولكل مكتبة معادلة واحدة ترمز لدورها:

$$\mathbf{y} \approx f_{\mathbf{w}}(\mathbf{x}) \qquad \mathbf{X} \in R^{\,n \times d} \qquad \min_{\mathbf{w}} J(\mathbf{w})$$

النموذج يربط `X` بـ `y` بمعاملات `w`؛ و NumPy تمسك `X` ككائن `n × d`؛ و SciPy تعطي طريقة للعثور على `w` الذي يُصغّر دالة الخسارة `J`. الفصلان الخامس والسادس يفكّكان هذه المعادلة سطراً سطراً.

> **تخصيص المسؤولية**
>
> إذا وجدت نفسك تكتب دالة `logsumexp` أو تحسب مشتقّ على ورقة، فقد تجاوزت حدود scikit-learn إلى SciPy.
> وإذا وجدت نفسك تكتب `for` يدوّر على صفوف، فغالباً تجاوزت حدود NumPy إلى pandas.

٢.٣ NumPy: لماذا المصفوفة ليست قائمة؟

الفرق العملي بين `list` في بايثون و `ndarray` ليس السرعة فقط — بل **الدلالة**. القائمة في بايثون حاوية غير متجانسة: قد تحوي أرقاماً ونصوصاً، ولا تضمن أن عناصرها من النوع نفسه. `ndarray` حاوية **متجانسة** لها عدد أبعاد

محدّد، وكل خانة فيها بايتات ثابتة العرض. هذا هو السبب الذي يجعل مكتبة NumPy سريعة: يمكنك أن تمرّر عنوان البيانات إلى C دون أن تفحص كل عنصر.

المصفوفة، البثّ، ومحاور الحساب — python

```python
A = np.arange(12).reshape(3, 4)

print("A.shape =", A.shape, "  ndim =", A.ndim, "  nbytes =", A.nbytes)
print("A.T.shape =", A.T.shape, "  transpose is a view, not a copy:",
      np.shares_memory(A, A.T))
print("slice A[:, :2] shares memory with A:", np.shares_memory(A, A[:, :2]))

# البثّ: عمود (3,1) + صف (1,4) يعطيان مصفوفة (3,4)
col = A[:, :1]
row = A[:1, :]
print("col", col.shape, "row", row.shape, "-> sum", (col + row).shape)

# المحور 0 يجمع عبر الصفوف فيبقى عدد الأعمدة
print("\nمجموع كل عمود (axis=0):", A.sum(axis=0))
print("مجموع كل صف  (axis=1):", A.sum(axis=1))
print("قاعدة: ينتج في البعد *غير* المذكور")
```

القاعدة التي تختصر آخر سطرين تُحفظ عن ظهر قلب: `axis=k` **يلغي البعد** `k`. لمن نسيناها مرة فكّرنا فيها كـ `axis="columns"`، فضاعفنا الدوال على البيانات كلها.

وأخطر ما في NumPy ليس الأداء بل **النسخ الضمني**. `A[:, :2]` أعلاه لا تنسخ شيئاً؛ هي نافذة (view) على الذاكرة نفسها. إذا كتبت فيها، تتغيّر `A`. احترس من هذه الحيلة تحديداً حين تعمل داخل حلقة على نتفق التحويل.

٢.٤ SciPy: الرياضيات التي لا يجب أن تعيد اختراعها

NumPy تعرف الحساب. SciPy تعرف **الخوارزمية**. الفرق بينهما هو الفرق بين أن تجمع الأرقام يدوياً وأن تعرف خوارزمية مونت كارلو.

```python
أربع أدوات من SciPy ستقابلها في هذا الكتاب
from scipy import linalg, optimize, special, stats

xs = np.linspace(-4, 4, 9)
print("special.expit  (logistic, مستقرّة عددياً):", np.round(special.expit(xs), 4))
print("stats.norm.pdf(±1, 0):", np.round(stats.norm.pdf([-1.0, 0.0, 1.0]), 4))
print("stats.norm.cdf(1.96): %.4f" % stats.norm.cdf(1.96))

res = optimize.minimize(lambda t: (np.exp(-t ** 2 / 2) - 0.5) ** 2,
                        x0=0.0, method="Nelder-Mead")
print("optimize.minimize:", res.success, " argmin = %.4f" % res.x[0])

M = np.array([[4.0, 1.0], [1.0, 3.0]])
b = np.array([1.0, 2.0])
print("linalg.solve:", np.round(linalg.solve(M, b), 4),
      "   inv(M)@b:", np.round(linalg.inv(M) @ b, 4))
```

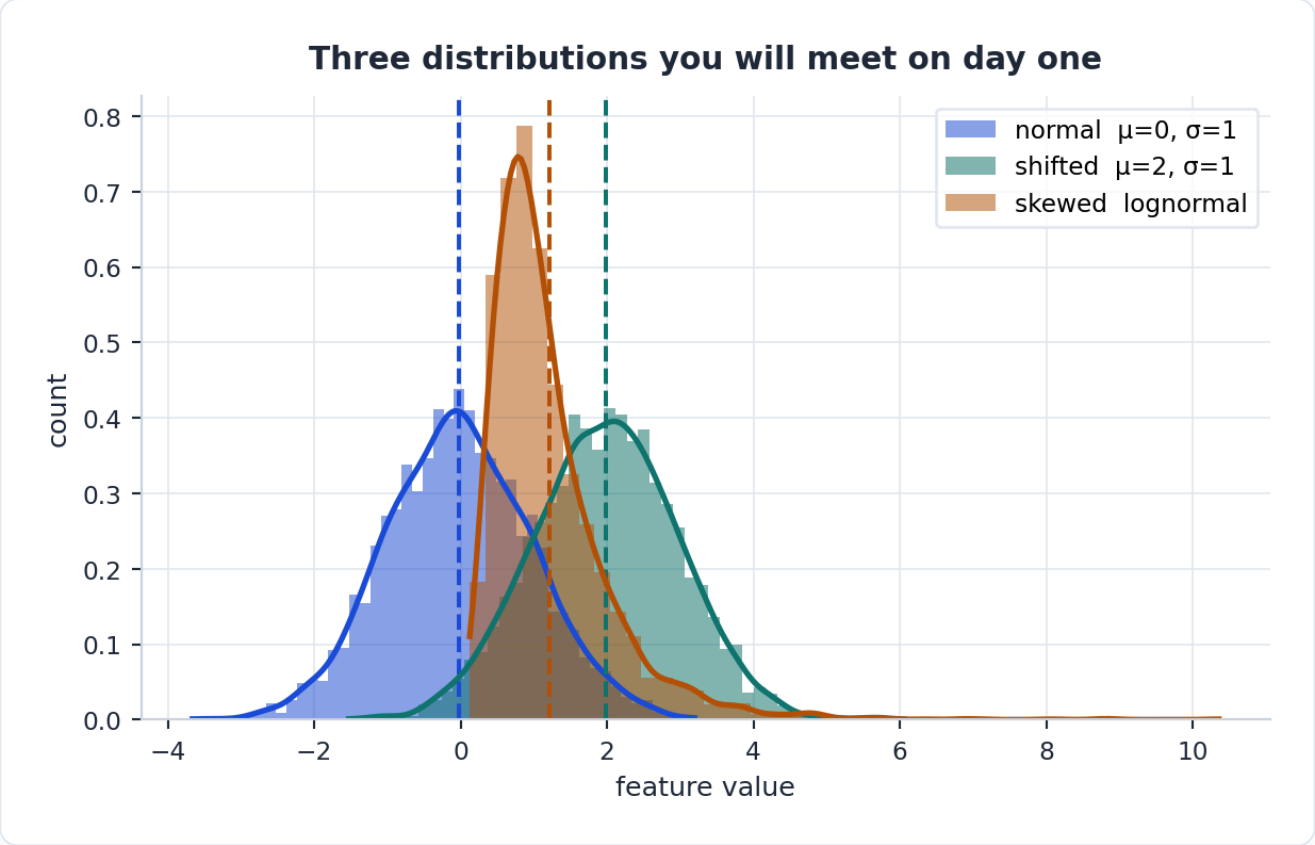
لاحظ سطر `special.expit` تحديداً. الدالة المنطقية القياسية تنهار عددياً عند القيم السالبة الكبيرة: `exp(800)` يفيض إلى ما لا نهاية. `expit` هي **نفس الدالة** محسوبة بطريقة مستقرّة، وهي ما تستعمله المكتبة داخلياً. هذا درس في الثقة: أن تختار المكتبة ليس لأنك كسول، بل لأن طريقة التنفيذ أفضل من نسختك.

الدالة في SciPy	ما تفعله بالضبط
`special.expit`	اللوجيستية المستقرّة عددياً
`stats.norm`	التوزيع الطبيعي: كثافة، توزيع تراكمي، مئينات
`optimize.minimize`	مصادقة المعاملات (انحدار، إسقاط، تعظيم)
`linalg.solve`	حل نظام خطّي **دون** حساب المعكوس

لاحظ السطر الأخير: `linalg.solve` أسرع وأدق من `inv(M) @ b`، ولا فرق في الناتج على هذه المصفوفة. لكن الفرق يظهر مع مصفوفات رديئة الحالة: المعكوس يضخّم الخطأ رقمياً. معادلتان تلخّصان أداتين من الأربع:

$$p(x) = \frac{1}{\sigma \sqrt{2 \pi}} e^{-\frac{(x - \mu)^2}{2 \sigma^2}} \qquad \mathrm{argmin}_{\mathbf{w}} \, J(\mathbf{w}) \;\mathrm{vs}\; \mathrm{argmin} \, \|\mathbf{A} \mathbf{w} - \mathbf{b}\|$$

الأولى كثافة التوزيع الطبيعي: منحنى بجرس يحكمه المتوسّط `mu` والانحراف `sigma` وحداهما. والثانية لماذا نفضّل `solve`: نحن نبحث عن `w` الذي يُناسب المعادلة `Aw = b`، لا عن معكوس `A` كاملاً.



الشكل ٥ثلاثة توزيعات ستقابلها من اليوم الأول: طبيعي بمتوسّط صفر، وآخر مزاح إلى اليمين، وآخر مائل إلى اليمين مع ذيل طويل

الشكل أعلاه هو أكثر توزّع ستراها في بياناتك الحقيقية. لاحظ التوزيع الثالث ذي الذيل الطويل. حين يكون عمودك مائلاً بهذه الدرجة، فالنماذج التي تفترض التوزيع الطبيعي (مثل انحدار Ridge الافتراضي) تعطي أرقاماً أقل جودة مما تتوقّع. الفصل ١٥ في `QuantileTransformer` و `PowerTransformer` يعالج هذه الحالة بالضبط.

٢.٥ pandas: الجدول الذي يحمل اسمه

الفرق العملي بين NumPy و pandas يظهر في جملة واحدة: **NumPy تعدّ الأعمدة من الصفر، و pandas تحمل اسم كل عمود.**

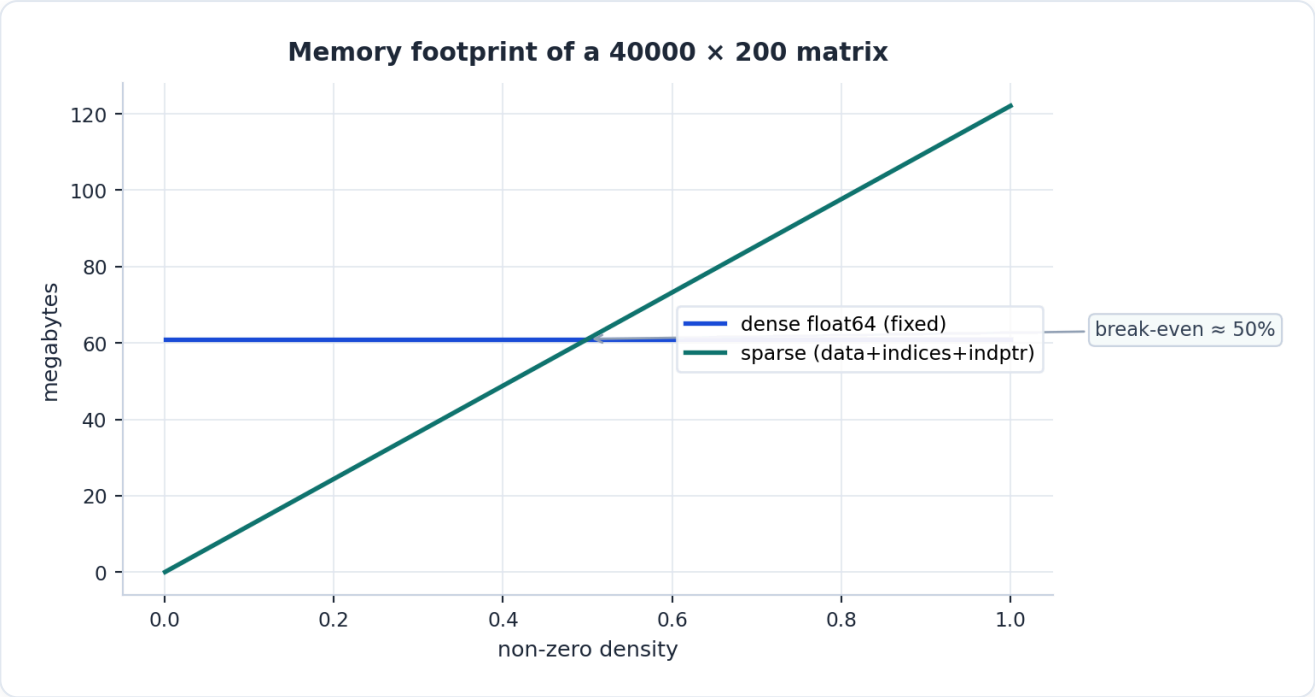
```python
# الفرق العملي: ndarray مقابل DataFrame
sales = pd.DataFrame({
    "city": ["ب" ,"أ" ,"ب" ,"أ"],
    "x": [1.0, 2.0, 3.0, 4.0],
    "y": [10, 20, 30, 40],
})
print("shape:", sales.shape, "  dtypes:\n", sales.dtypes.to_string())
print("\nتجميع حسب city:\n", sales.groupby("city")[["x",
"y"]].mean().to_string())
print("\nselect_dtypes يختار الأعمدة الرقمية تلقائياً:")
print(sales.select_dtypes("number").columns.tolist())

print("\nالآن NumPy لا يعرف أسماء الأعمدة:")
arr = sales[["x", "y"]].to_numpy()
print("  arr.shape =", arr.shape, "   arr[0] =", arr[0])
```

لاحظ الفرق في السطر الأخير: `arr[0]` صف بلا هوية. وحين يمرّر هذا إلى `LogisticRegression.fit` يرى `X` بلا أسماء أعمدة، فيضطر للتخمين عند الخطأ. هذا ليس تفصيلاً تجميلياً: الفصل ٢٠ كلّه في واجهة `set_output` لمنع هذا الارتباك.

> **مرّر DataFrame دائماً إن أمكن**
>
> بمجرد أن تمرّر `DataFrame` إلى `fit`، تُتاح لك السمتان `feature_names_in_` و `n_features_in_` تلقائياً، وتظهر أسماء الأعمدة في رسالة الخطأ حين يخطئ عدد الأعمدة. هذه وحدها تستحق ثوانٍ من وقتك في كل مرة.

٢.٦ الذاكرة: لماذا يهمّك عدد القيم المفقودة؟

البيانات النادرة (sparse) ليست ترفاً: مستند من عشرين ألف كلمة فيه ٣٪ فقط من الكلمات غير مكررة. يخزّنه `scipy.sparse` كثلاثة مصفوفات صغيرة بدل مصفوفة هائلة من الأصفار.



الشكل ٦مخطّط يقارن استهلاك الذاكرة لمصفوفة ٤٠٠٠٠×٢٠٠: الخط الكثيف ثابت بينما ينمو المتفرّق خطياً مع كثافة القيم غير الصفرية، والتعادل عند نحو ٥٠٪.

القاعدة العددية من الشكل: الكثافة تتجاوز ٥٠٪ يصبح التوفير صفراً بل سالباً، لأن تخزين فهرس لكل قيمة لا صفرية يكلّف أكثر من تخزينها نفسها. من هنا قاعدة التوجيه التي ستكررها في الفصل ١١: **لا تحوّل مصفوفة متفرّقة إلى كثيفة إلا إذا كانت الكثافة فوق ٥٠٪**.

حساب الكلفة الحقيقية للمصفوفتين — python

```python
from scipy.sparse import csr_matrix

rng = np.random.default_rng(0)
dense = rng.random((300, 400))
sp = csr_matrix(dense * (dense > 0.97))

sparse_bytes = sp.data.nbytes + sp.indices.nbytes + sp.indptr.nbytes
print(f"كثافة المصفوفة: {sp.nnz / dense.size:.4f}")
print(f"الحجم الكثيف : {dense.nbytes / 1024:8.1f} KB")
print(f"الحجم المتفرّق: {sparse_bytes / 1024:8.1f} KB")
print(f"نسبة التوفير : {dense.nbytes / sparse_bytes:.1f}x")
print("\nnnz =", sp.nnz, "  من أصل", dense.size)
```

هذه نسبة ٢٢ ضعفاً على كثافة ٣٪. لكن لاحظ أن `csr_matrix` هنا مبنيّة من مصفوفة كثيفة معبّأة أصلاً — في الواقع العملي تبنيها مباشرة من القوائم، فلا تدفع كلفة الكثافة أصلاً.

٢.٧ كائن Bunch: ماذا تعطيك مكتبة البيانات؟

كل دالة `load_*` في scikit-learn تُعيد كائناً واحداً اسمه `Bunch`، وهو قاموس ذو سمات. أن تعرف اسم كل مفتاح يكفيك لتستخرج أي شيء.

تشريح كائن Bunch من بيانات Iris — python

```python
from sklearn.datasets import load_iris

iris = load_iris()
print("type:", type(iris).__name__)
print("keys:", sorted(iris.keys()))
print()
print("data.shape     :", iris.data.shape, type(iris.data).__name__)
print("target.shape   :", iris.target.shape, " values:", np.unique(iris.target))
print("target_names   :", list(iris.target_names))
print("feature_names  :", iris.feature_names)
print("DESCR line 1   :", iris.DESCR.splitlines()[0])
print("frame (default):", iris.frame)

iris_df = load_iris(as_frame=True)
print("frame (as_frame=True):", type(iris_df.frame).__name__,
      iris_df.frame.shape)
print("target as Series     :", type(iris_df.target).__name__)
print(iris_df.frame.head(3).to_string())
```

لاحظ أن `frame` يبقى `None` ما لم تطلب صراحةً بـ `as_frame=True`. هذه مفاجأة تواجه كل مبتدئ: يعيد الكائن مفتاحاً يبدو جاهزاً وهو في الواقع فارغ. والسبب منطقي: بناء `DataFrame` يكلّف ذاكرة ووقتاً، فلم تفعلها المكتبة إلا طلبتها.

المفتاح	نوعه	ما فيه
`data`	`ndarray` بمصفوفة float	القيم الرقمية
`target`	`ndarray` بأعداد صحيحة	الوسم الرقمي لكل صف
`target_names`	قائمة نصوص	معنى كل رقم في `target`
`feature_names`	قائمة نصوص	معنى كل عمود
`frame`	`DataFrame` أو `None`	الجدول كاملاً، يُملأ فقط مع `as_frame=True`
`DESCR`	نص طويل	شرح البيانات ومصادرها

٢.٨ تمارين الفصل

التمرين ٢.١

أنشئ مصفوفة `np.arange(24).reshape(2, 3, 4)` ثم طبّع `np.sum` و `np.mean` على `axis=0` و `axis=1` و `axis=2`. اكتب في سطر واحد ما الذي ينتجه `axis=k` في كل حالة، ثم تحقّق من إجابتك حسابياً.

التمرين ٢.٢

احسب `stats.norm.cdf` عند z = ١٫٩٦ و ٢٫٥٨، وفسّر لماذا يعطي الأول ٩٧٫٥٪ والثاني ٩٩٫٥٪، ولماذا كلّ ما بينهما مقدار ١٫٦٢ وحدة قياس شحنة قياسية فقط.

التمرين ٢.٣

ابنِ `DataFrame` فيه ١٠٠٠ صف وعمود `price` يتبع `lognormal`، ثم احسب `skew` قبل وبعد تطبيق `np.log`. لماذا يقلّ التحوّل هذا الانحياز ولماذا لا يجعله معدوماً؟

التمرين ٢.٤

قارن `linalg.solve` بـ `linalg.inv` على مصفوفة قريبة من مفردة (condition number كبير). بلّغ عن الخطأ النسبي في كل طريقة، ثم اشرح أيّهما تستعمل في كتاب كهذا ولماذا.

٢.٩ حلول بعض التمارين

✅ حلّ التمرين ٢.١

`axis=k` **يلغي** البعد `k` من الشكل:

الطريقة السريعة للتحقّق: اجمع مرّتين. `(2,3,4)` ثم `axis=0` يعطي `(3,4)` ثم `axis=0` مرة أخرى يعطي `(4,)` — وهو نفس `A.sum()` بلا معامل. أي معاملَين متتاليين يعطيان `A.sum()`.

✅ حلّ التمرين ٢.٣

الانحياز (skewness) يقيس عدم تماثل التوزيع. توزيع `lognormal` كلّه موجب ومنحرف يميناً بوضوح: المتوسّط أكبر من الوسيط بكثير.خذ `log` يحوّل هذه العلاقة إلى علاقة تقريباً خطّية، فتصير التوزيعات شبه طبيعية.

لكنه لا يصبح معدوماً تماماً لأن القيم التي كانت سالبة أو قريبة منها في الفضاء الأصلي تصبح سالبة أو كبيرة جداً في فضاء اللوغاريتم. لذلك يبقى الانحياز متبقياً، وهو سبب إضافي لتطبيق `PowerTransformer` أو `QuantileTransformer` بدل `log` وحده.

٢.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
الخلط بين `axis=0` و `axis=1`	جمع عرضي بدل طولي، وأرقام خاطئة صامتة	احفظ القاعدة: يلغي البعد `k`
تمرير `ndarray` حين يتوفّر `DataFrame`	فقدان أسماء الأعمدة ورسائل خطأ مبهمة	مرّر `DataFrame` أو فعّل `set_output(transform_output="pandas")`
تحويل مصفوفة متفرّقة إلى كثيفة	انفجار الذاكرة من ×١٠٠ إلى ×١	أبقِ `csr_matrix` ما دام الكثافة دون ٥٠٪
كتابة `exp(z)/(1+exp(z))` يدوياً	فيض عددي عند القيم السالبة الكبيرة	استعمل `scipy.special.expit`
استعمال `inv(M) @ b` بدل `solve`	تضخيم الخطأ الرقمي وبطء	استعمل `linalg.solve`
توقّع أن `Bunch` قاموس عادي	تفقد الوصول إلى السمات مثل `iris.data`	افهم أنه قاموس بذوق

٢.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
منظومة بايثون العلمية	الطبقات الخمس من الحوسبة إلى التعلّم	scientific Python stack
المصفوفة متعددة الأبعاد	حاوية `ndarray` متجانسة الأبعاد	ndarray
البثّ	توسيع الأبعاد الصغير تلقائياً في العملية	broadcasting
النافذة	مقطع يشارك الذاكرة مع المصفوفة الأصلية	view
متفرّق	تخزين القيم غير الصفرية فقط مع فهارسها	sparse
التوزيع الطبيعي	التوزيع ذو الجرس المحدَّد	normal distribution
دالة التوزيع التراكمي	احتمال أن تكون القيمة أقل من عدد معطى	cumulative distribution
`Bunch`	قاموس scikit-learn الذي تعيده `load_*`	Bunch
`narwhals`	طبقة توافق مع أي إطار جداول	narwhals

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٣ — التثبيت والإعداد على Python ٣.١٤

بيئة عمل نظيفة قابلة للتكرار، من venv إلى uv و conda

الوحدة **scikit-learn ١.٩** الحجم **٧٧٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا البيئة الافتراضية ليست ترفاً بل ضرورة صحّية
- أن تختار بين venv و conda و uv بمعيار مشروع حقيقي لا بالذوق
- أن تتحقّق من إصداراتك وتقيس البيئة بمعرّف بصمة يتغيّر عند أي تغيير
- أن تضبط إعدادات scikit-learn العامة وتعرف متى لا يجب أن تلمسها

٣.١ ثلاثة أسئلة قبل أن تكتب أي أمر تثبيت

قبل `pip install` اسأل نفسك ثلاثة أسئلة، الجواب عنها يوفّر عليك ساعات لاحقاً:

التثبيت بلا هذه الأسئلة ينتج الحالة الأسوأ: نظام مستقل تحوي فيه كل نسخة بايثون على جهازك كل الحزم، وتكتشف بعد شهر أن مشروعاً كان يعمل اختفى لأن مشروعاً آخر رقّى `numpy`.

بطاقة هوية البيئة التي تعمل فيها الآن — python

```python
import platform
import sys
from pathlib import Path

exe = Path(sys.executable)
print("sys.executable :", exe)
print("base_prefix    :", sys.base_prefix)
print("prefix         :", sys.prefix)
print("inside a venv? :", sys.prefix != sys.base_prefix)
print("python version :", sys.version.split()[0])
print("platform       :", platform.system(), platform.machine())
print("sys.path entries:", len(sys.path))
```

السطر الرابع هو الأهم. إن كان `prefix` مساوياً لـ `base_prefix` فأنت تعمل في **بايثون النظام** مباشرة. هذا يعني أن أي `pip install` سيمسّ كل مشاريعك على الجهاز. القاعدة: لا تُثبّت شيئاً في بايثون النظام أبداً.

٣.٢ لماذا بايثون ٣,١٤ تحديداً؟

بايثون ٣,١٤ إطلاق كبير في سرعة المكتبات التي يهمّنا. ما يعنيه لك عملياً:

ما تغيّر	لماذا يهمّ نموذجاً
تسريع العمليات الحاجبة (GIL)	عمليات NumPy الطويلة لم تعد تتوقف مؤشراً
تحسينات في `asyncio`	لا أثر مباشر على التدريب، لكن مهمّ في خدمة التنبّؤ
دعم أفضل للمنصات	حزم هذا الكتاب مبنية عليه أصلاً

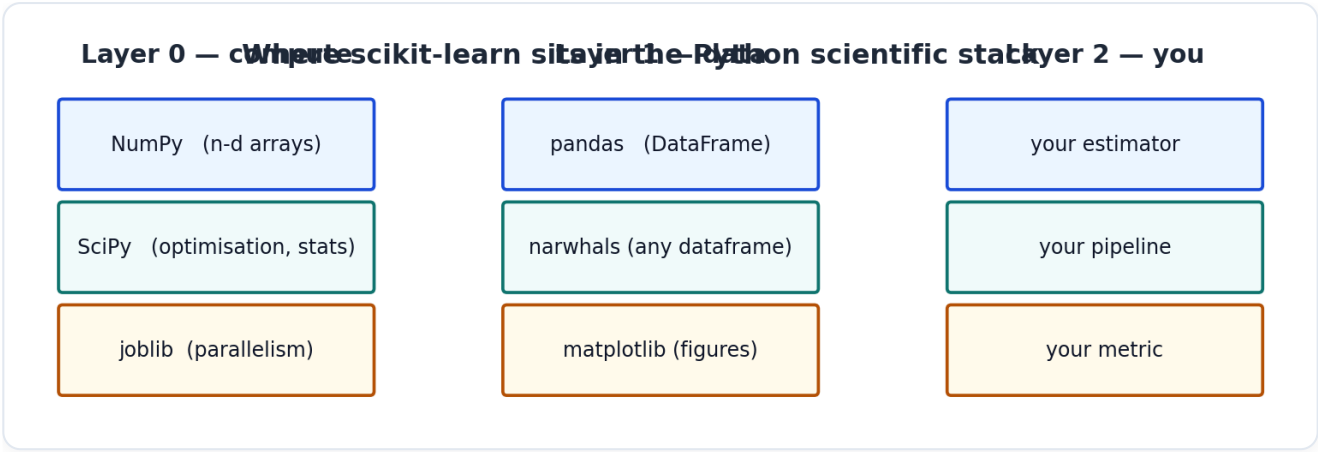
لكن الأهم عملياً ليس السرعة بل **التوافق**. scikit-learn ١,٩,١ مُختبَر على ٣,١٤، وتثبيتُه على ٣,١٢ يعني أنك تعمل بإصدار مكتبة لم تُختبر على هذه النسخة من بايثون. الاختلافات الصامتة في هذا المكان بالذات هي أسوأ أنواع الأخطاء: أرقام مختلفة قليلاً، ونماذج تسلك سلوكاً غريباً في الحافة.

> 📝 **قاعدة الإصدار في هذا الكتاب**
>
> كل مثال في الصفحات القادمة مكتوب ومُنفَّذ على `scikit-learn 1.9.1` و `Python 3.14`. إن كانت نسختك أقدم، فستختلف الأرقام في بعض المواضع، وقد تختلف أسماء المعاملات. الحلّ ليس تصحيح الرقم، بل تثبيت النسختين.

٣.٣ البيئة الافتراضية: لماذا لا تعمل في النظام؟

تخيّل عمّالاً مشتركين في ورشة. كلهم يشربون من نفس الإبريق، وكل واحد يضيف ما يشاء. في صباح اليوم التالي لا أحد يعرف ما بداخل الإبريق.

البيئة الافتراضية (virtual environment) هي إبريق خاص لكل عامل. داخلها تكون الحزم مستقلّة تماماً عن بايثون النظام وعن باقي المشاريع.



الشكل ٧خريطة المنظوم العلمية: طبقة الحوسبة ثم طبقة البيانات ثم طبقة كود المستخدم، وكل طبقة ترث ما تحتها

الخريطة أعلاه تُعيد ترتيب الطبقات نفسها التي رأيناها في الفصل السابق، وهذه المرة من زاوية التثبيت: ما عدا الطبقتين الأولى والثانية، **أنت من يقرّر النسخة** في كل مستوى. لا يوجد «أفضل نسخة» مطلقة؛ يوجد أفضل نسخة متوافقة مع مشروعك.

الآن، الأمر الذي يكتب الجميع:

إنشاء بيئة افتراضية والتحقّق منها — python

```python
# على ويندوز (PowerShell)
py -3.14 -m venv .venv

# تفعيلها
.\.venv\Scripts\Activate.ps1

# على لينكس وماك
python3.14 -m venv .venv
source .venv/bin/activate

# التحقّق
python -c "import sys; print(sys.prefix != sys.base_prefix)"
```

الفرق بين تفعيل البيئة وعدم تفعيله ليس في الحزم، بل في أي نسخة بايثينفّذها أمر `python`. عند التفعيل يشير الأمر إلى داخل البيئة؛ وبدونه يشير إلى النظام. ولهذا يبقى `pip install` مكتوباً داخل كتلة دون تنفيذ هنا: هذه أوامر تُنفَّذ في طرفية لا في مفكّر بايثون.

٣.٤ ثلاثة مديري بيئات: أيّهم تختار؟

هذه المقارنة تتغيّر كل بضع سنوات، لكن المعايير لا تتغيّر.

المعيار	`venv`	`conda`	`uv`
جزء من بايثون القياسي	نعم	لا	لا
السرعة في التثبيت	بطيئة نسبياً	بطيئة	أسرع بكثير
إدارة حزم غير بايثونية (مثل MKL)	ضعيفة	ممتازة	عبر `uv pip`
ملف قفل (lock)	يدوي	`environment.yml`	`uv.lock` دقيق
ملاءمة لـ بايثون ٣,١٤	جيد	جيد	الأفضل
منحنى التعلّم	صغير	كبير	صغير
متى تختاره	مشروع واحد بسيط	حوسبة علمية ثقيلة	كل مشروع جديد

القرار العملي الذي ننصح به في ٢٠٢٦:

> ⛔ **الخلط بين المديرين يُنتج كابوساً حقيقياً**
>
> تثبيت حزمة بـ `conda` ثم تحديثها بـ `pip` داخل نفس البيئة يترك سجلّ الحزم في حالتين متعارضتين. النتيجة أخطاء مثل `numpy.dtype size changed` أو `ImportError: DLL load failed` رغم أن كل حزمة «مثبّتة». القاعدة: اختر مديراً واحداً لكل بيئة، ولا تستعمل الثاني فيها أبداً.

٣.٥ التحقّق: ما الذي يجب أن تراه بعد التثبيت؟

التثبيت الناجح على ورق قد يفشل في الواقع. خمس خطوات تتحقّق منها بالترتيب:

```python
# بطاقة التحقّق: إصدارات، إعدادات، خيوط
from importlib.metadata import requires, version

from sklearn import get_config

CORE = ["numpy", "scipy", "pandas", "matplotlib", "scikit-learn",
        "joblib", "threadpoolctl", "narwhals"]

print("== الإصدارات ==")
for pkg in CORE:
    try:
        print(f"  {pkg:<16} {version(pkg)}")
    except Exception:
        print(f"  {pkg:<16} غير مثبّت")

print("\n== ما يطلبه scikit-learn إجبارياً ==")
print("  ", requires("scikit-learn"))

print("\n== إعدادات المكتبة الافتراضية ==")
for key in ("display", "transform_output", "enable_metadata_routing",
            "assume_finite", "working_memory"):
    print(f"  {key:<26} {get_config()[key]}")

print("\n== خيوط الحساب ==")
import threadpoolctl
for d in threadpoolctl.threadpool_info():
    print(f"  {d['user_api']:<8} threads={d['num_threads']}
{d.get('internal_api', '')}")
```

قائمة `requires` في الوسط هي **عقد المكتبة مع بايثون**: إن كانت `scipy` لديك أقدم من `1.10.0` فالمكتبة ستعمل غالباً لكن ليس مضموناً. وهذه هي أقرب قائمة مرجعية تملكها في أي تثبيت جديد.

٣.٦ كيف تجعل نتيجتك قابلة للتكرار؟

البيئة الصحيحة تعني أنك تعرف **داخلها ماذا**. بصمة البيئة (fingerprint) هي بصمة قصيرة تحسب من إصدارات حزمك؛ إن تغيّرت، تغيّرت النتيجة المحتملة.

```python
بصمة البيئة: تكشف أي انزلاق في التبعيات

import hashlib
from importlib.metadata import version

CORE = ["numpy", "scipy", "pandas", "matplotlib", "scikit-learn", "joblib"]


def environment_fingerprint(names=CORE) -> dict:
    """Versions + a short hash, for pinning in a run log."""
    pins = {}
    for name in names:
        try:
            pins[name] = version(name)
        except Exception:
            pins[name] = None
    blob = "|".join(f"{k}=={v}" for k, v in sorted(pins.items()))
    digest = hashlib.sha256(blob.encode("utf-8")).hexdigest()[:12]
    return {"pins": pins, "hash": digest}


fp = environment_fingerprint()
for name, ver in fp["pins"].items():
    print(f"  {name:<14} {ver}")
print("\nfingerprint:", fp["hash"])

# نفس البيئة تُعطي نفس البصمة دائماً
print("stable:", environment_fingerprint()["hash"] == fp["hash"])
print("\n# requirements.txt لهذا الكتاب:")
print("\n".join(f"{k}=={v}" for k, v in fp["pins"].items() if v))
```

احفظ السطر الأخير في ملف `requirements.txt` معّ كل نتيجة مهمّة. حين يتغيّر الإصدار بعد ثلاثة أشهر، ستعرف أن الرقم الجديد من بيئة مختلفة.

٣.٦.١ الثبات (determinism) ليس تلقائياً

`random_state` يضمن أن البذرة نفسها تعطي النتيجة نفسها **داخل العملية نفسها**. لكن هناك مصدران آخران للاختلاف، وكلاهما يظهر في طريقة التدريب نفسها:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^{(t)})$$

كل خطوة هنا جمعٌ عدد من المساهمات. إن نفّذتها eight خيوط متوازية بلا ترتيب مثبّت، فقد يُجمع المجموع بترتيب مختلف ويختلف في آخر خانة عشرية — والاختلاف ينتقل إلى كل ما بعده. `random_state` يعطيك البذرة؛ وهذا وحده لا يكفي.

ما الذي يكسر التطابق بين تشغيلين؟ python

```python
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

X, y = make_classification(n_samples=400, n_features=5, random_state=11)

runs = []
for _ in range(2):
    lr = LogisticRegression(max_iter=1000, random_state=0).fit(X, y)
    kn = KNeighborsClassifier(n_neighbors=5).fit(X, y)
    runs.append((lr.coef_.copy(), kn.predict(X)))

print("LogisticRegression  identical:", np.array_equal(runs[0][0], runs[1][0]))
print("KNeighborsClassifier identical:", np.array_equal(runs[0][1], runs[1][1]))
print("coef checksum:", round(float(np.abs(runs[0][0]).sum()), 10))

print("\nمصدرا الاختلاف الحقيقيان:")
print("  1. تغيّر نسخة الحزمة: scipy قد يغيّر المحلّل عددياً")
print("  2. عدد خيوط BLAS:    الجمع المتوازي غير حتمي الترتيب")
print("\nعند تسجيل نتيجة حرجة، احلّ:")
print("  OMP_NUM_THREADS=1  python train.py")
```

لاحظ أن `KNeighborsClassifier` **حتمي تماماً** لأن لا عشوائية فيه ولا خطوة أولية تعتمد على ترتيب الصفوف. بينما `LogisticRegression` يعتمد على حلّ عددي متكرّر قد يختلف في آخر خانة عشرية بين نسختين من `scipy`.

٣.٧ إعدادات scikit-learn العامة

في ١,٩ تغيّرات مهمة في إعدادات العرض والمخرجات:

```python
# إعدادات المكتبة: ما الذي يغيّر سلوكك فعلاً؟
from sklearn import config_context, get_config

print("الحالة الراهنة عند بناء هذا الكتاب:")
for key in ("display", "transform_output", "enable_metadata_routing",
            "assume_finite", "working_memory", "print_changed_only"):
    print(f"  {key:<26} {get_config()[key]}")

from sklearn.preprocessing import StandardScaler

with config_context(display="text"):
    print("\nداخل config_context(display='text'):", get_config()["display"])
print("بعد الخروج من الكتلة      :", get_config()["display"])

with config_context(transform_output="default"):
    out_default = StandardScaler().fit_transform(X)
print("تحت transform_output='default' :", type(out_default).__name__)
with config_context(transform_output="pandas"):
    out_pandas = StandardScaler().fit_transform(X)
print("تحت transform_output='pandas'  :", type(out_pandas).__name__)
print("القيم متطابقة:", np.allclose(np.asarray(out_default, dtype=float),
                                    out_pandas.to_numpy()))
```

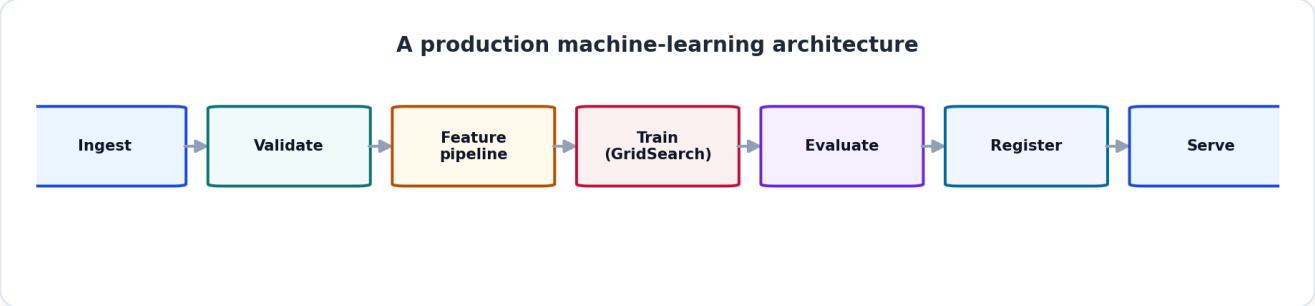
لاحظ السطر الأخير: تغيير الإعداد يغيّر **الغلاف** لا **المحتوى**. الأرقام واحدة تماماً؛ ما يتغيّر هو قدرة الكائن على حمل الأسماء.

> **جديد في ١,٩**
>
> `transform_output="pandas"` يجعل كل محوّل يُعيد `DataFrame` أو `Series` بدل `ndarray`. هذا يحوّل أسماء الأعمدة من رفاهية إلى متطلّب: نموذج يتدرّب على جدول يُنتج جدولاً يحمل الأسماء نفسها. الفصل ٢٠ يشرح الآلية كاملة، وهنا يكفي أن تعرف أن الخيار موجود وأن له كلفة في الذاكرة.

القاعدة العملية: **اضبط الإعدادات مرة واحدة في أعلى الملف أو في دالة تهيئة مشتركة، لا في كل مكان**. الإعداد المبعثر في الكود هو أصل أخطاء كثيرة: نموذج في دالة، وآخر في دالة أخرى، ولا أحد يعرف أنهما أُعدّا بمنطق مختلف.

٣.٨ أين يعمل النموذج في الإنتاج؟

بين Jupyter Notebook على جهازك وخادم يستقبل آلاف الطلبات، تغيّر كل شيء تقريباً عدا سطر واحد: `model.predict(X)`.



الشكل ٨مخطط بنية إنتاج: استقبال البيانات ثم التحقق منها ثم خط أنابيب الخصائص ثم التدريب مع ضبط المعاملات ثم التقييم ثم التسجيل ثم الخدمة

المرحلة	ما يحدث فيها	الخطر
الاستقبال	دخول البيانات من المصدر	بيانات غير متوقّعة
التحقّق	فحص المخطّط والقيم	خطأ صامت
خط الخصائص	التحويل نفسه	تسرّب يسبّق النشر
التدريب والضبط	اختيار المعاملات	اختيار على بيانات الاختبار
التقييم	قياس صادق	مقياس لا يطابق الواقع
التسجيل	حفظ النموذج مع بصمته	نموذج بلا أثر
الخدمة	تقديم التنبّؤ	انزياح التوزيع بعد شهر

المرحلة التي يغفلها المبتدئون هي الأولى والأخيرة معاً. قبل **الاستقبال** يجب أن تعرف ما الذي يصل: عدد الأعمدة، أنواعها، وحدة القياس. وبعد **الخدمة** يجب أن تراقب: هل توزيع البيانات تغيّر؟ هل معدّل الرفض للطلبات ارتفع؟

حارس المدخلات: خمسون سطراً تحمي نظامك python

```python
import numpy as np
from sklearn.datasets import make_classification

SCHEMA = {"age": (18.0, 90.0), "income": (0.0, 5e6), "tenure_months": (0.0,
720.0)}


def validate(batch: dict) -> tuple[list[str], dict]:
    """Check a raw payload against the schema the model was trained on."""
    problems: list[str] = []
    for col in SCHEMA:
        if col not in batch:
            problems.append(f"عمود ناقص: {col}")
    for col, (lo, hi) in SCHEMA.items():
        if col in batch:
            v = float(batch[col])
            if np.isnan(v):
                problems.append(f"قيمة مفقودة: {col}")
            elif not (lo <= v <= hi):
                problems.append(f"خارج النطاق: {col}={v} (المسموح {lo}..{hi})")
    return problems, batch


cases = [
    {"age": 41, "income": 82000, "tenure_months": 36},
    {"age": 12, "income": 82000, "tenure_months": 36},
    {"age": 41, "income": np.nan, "tenure_months": 36},
    {"age": 41, "income": 82000},
]
for case in cases:
    problems, _ = validate(case)
    print(f"{str(case)[:58]:<60} -> {problems or 'سليم'}")
```

هذه الدالة تسبق النموذج دائماً في خدمة الإنتاج. لا يستقبل النموذج طلباً إلا بعد أن يجتاز الفحص. لكن تذكّر: هذا الفحص يجب أن يُبنى على **نطاق التدريب الفعلي**، لا على تخمين.

$$\hat{\mathbf{y}} = f_{\mathbf{w}}(\mathbf{x}) \qquad \mathbf{w} \;=\; \text{const}$$

هذه المعادلة الوحيدة التي لا تتغيّر بين دفتر تدريبات وخادم إنتاج. كل ما حولها — التحميل، والتحويل، والتحقّق، والتسجيل — هندسة. ولهذا تُكتب المعادلة مرة واحدة وتُختبَر وحدها.

من دفتر Deploy حقيقي

أكثر حوادث الإنتاج شيوعاً لا تحدث في `scikit-learn` إطلاقاً. أكثر اثنتين تكراراً: (١) بيئة التطوير تحمل `scikit-learn 1.8` بينما الخادم ١٫٩، فتختلف أسماء معاملات في `GridSearchCV` وتُرفض الطلبات عند الإقلاع؛ (٢) `random_state` مثبّت لكن `n_jobs=-1` يترك عدد الخيوط يتبع آلة النشر، فتنزلق نتائج المقاييس بمقدار سابع عشري بين بيئتين متطابقتين ظاهرياً. كلاهما لا يظهر في أي دفتر تجارب — يظهران بعد ثلاثة أشهر.

٣.٩ قائمة تحقّق قبل النشر

قائمة تحقّق بيئة الإنتاج

٣.١٠ تمارين الفصل

التمرين ٣.١

أنشئ بيئة افتراضية باسم `.venv`، ثبّت فيها `scikit-learn==1.9.1`، ثم اكتب في نصّ الجواب ناتج `sys.prefix != sys.base_prefix` قبل التفعيل وبعده. لماذا يعطي القيمة `False` الأولى و `True` الثانية؟

التمرين ٣.٢

اكتب دالة `check_versions(required: dict)` ترفع استثناءً مفهوماً إذا اختلف أي إصدار. استعملها في أعلى كل ملف في هذا الكتاب. جرّب أن تكسرها عمداً بتغيير رقم واحد، ثم صمّم الرسالة كي تقول لك **أي حزمة** و **بأي رقم** و **لماذا يهم**.

التمرين ٣.٣

غيّر `transform_output` بين `"default"` و `"pandas"`، وشغّل خط أنابيب من `StandardScaler` و `LogisticRegression`. ماذا يتغيّر في نوع المخرجات؟ وماذا يبقى ثابتاً؟ ثم اشرح لماذا يهمّ ذلك في خدمة ويب تستقبل JSON وتعيد JSON.

التمرين ٣.٤

اكتب حارس مدخلات لـ `load_digits` بمئة١٠١ عموداً رقمياً في المدى ٠..١٦. اختبره على خمس دفعات، اثنتان منها خاطئتان. ماذا يحدث لو أُسقط الفحص وذهب الطلب مباشرة إلى `predict`؟ صف الخطأ الذي سيراه المستخدم النهائي.

٣.١١ حلول بعض التمارين

✅ حلّ التمرين ٣.١

قبل التفعيل، `sys.prefix` و `sys.base_prefix` يشيران إلى مجلّد تثبيت بايثون نفسه، فالمساواة تعني أنك في البيئة العامة. بعد التفعيل يشير `sys.prefix` إلى مجلّد البيئة الجديدة التي أنشأها `venv`، بينما يبقى `base_prefix` كما هو. الفارق بين المسارين هو بالضبط ما يجعل العزل يعمل: أي حزمة تُثبَّت في الأول لا تصل إلى الثاني.

✅ حلّ التمرين ٣.٣

مع `transform_output="default"` يُعيد كل محوّل `ndarray`، وتضيع الأسماء. مع `"pandas"` يُعيد `DataFrame` و `Series` تحملان أسماء الأعمدة والفهارس. ما يبقى ثابتاً هو **المحتوى الرقمي تماماً** — نفس القيم، نفس الدقة، نفس الدالة.

في خدمة ويب، الفارق ليس تجميلياً: إن كان مخرج النموذج `DataFrame` بأعمدة مسمّاة، فإن طبقة التسلسل إلى JSON تنتج كائناً بمفاتيح مفهومة بدل مصفوفة مواضع. وإن أخطأ الترتيب صامتاً، ومدّة التدريب تتأجّل.

٣.١٣ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
`pip install` بلا بيئة افتراضية	كسر مشاريع أخرى على الجهاز	أنشئ `.venv` أولاً دائماً
خلط `conda` و `pip` في بيئة واحدة	أخطاء تحميل DLL وتعارض إصدارات	مدير واحد لكل بيئة
الاعتماد على الذاكرة بدل ملف تثبيت	«يشتغل عندي» بلا تفسير	`requirements.txt` ملتزم بالنسخ
`random_state` في بعض المواضع فقط	نتائج مختلفة بين تشغيلين	ثبّت البذرة في كل موضع عشوائي
`set_config` مبعثر في الكود	نموذجان بإعدادين مختلفين	دالة تهيئة واحدة في أعلى الحزمة
نشر بلا حارس مدخلات	خطأ ٥٠٠ في الإنتاج أو توقّعات صامتة	تحقّق من المخطّط قبل `predict`
بناء النموذج على جهاز بلا نسخة مطابقة	فروق في آخر خانة عشرية	ثبّت النسختين معاً

٣.١٤ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
بيئة افتراضية	شجرة حزم معزولة لكل مشروع	virtual environment
مدير البيئة	أداة تُنشئ البيئات وتثبّت فيها	environment manager
ملف القفل	قائمة تبعيات مقيّدة بدقة	lock file
الحتمية	خاصية تكرار النتيجة نفسها تماماً	determinism
البذرة العشوائية	العدد الذي يثبّت مولّد العشوائية	random seed
خيوط BLAS	خيوط حساب المصفوفات بالتوازي	BLAS threads
إعدادات المكتبة	متغيّرات عامة تغيّر سلوك كل شيء	global configuration
حارس المدخلات	فحص المخطّط قبل التنبّؤ	input validation

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٤ — أول نموذج لك: أقرب الجيران على بيانات Iris

قراءة كود كامل من السطر الأول حتى الرسم البياني

الوحدة **scikit-learn ١.٩** الحجم **٨٥٨** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تبني نموذج تصنيف كامل بالتقسيم والتدريب والتقييم وتقرأ مخرجاته
- أن تفسّر كل خانة في مصفوفة الالتباس وتربطها بالدقة والحساسية
- أن تحسب `predict_proba` بيدك وتتحقّق من مطابقتها للمكتبة
- أن تشرح لماذا التحجيم ليس اختيارياً في نموذج قائم على المسافة

٤.١ Iris: خمسون صفاً لكل صنف، وأربعة أرقام لكل صف

قبل أي نص نظري، خذ نفس قطرة زهرة. أربع قياسات بالسنتيمتر: طول البتلات، عرضها، طول السبلات، عرضها. ثلاث فئات. كل صف في الجدول هو قطرة واحدة، وكل عمود قياس.

هذه أصغر مشكلة تصنيف في العالم، ولهذا تُستعمل في كل كتاب تعلّم آلة منذ ١٩٣٦. لكن صغرها لا يعني سهولتها: بداخلها درسان كاملان في القياس، يضيعان عادةً تحت بريق الرقم ٠٫٩٧.

سنرى الاثنين بالأرقام لا بالوصف.

```python
# أول خطوة: انظر إلى بياناتك قبل أن تراها نموذجاً
from sklearn.datasets import load_iris, make_classification

iris = load_iris(as_frame=True)
df, X, y = iris.frame, iris.data, np.asarray(iris.target)

print(df.head(4).to_string())
print("\nالشكل:", df.shape, "  توزيع الأصناف:", np.bincount(y))
print("أسماء الأصناف:", list(iris.target_names))

print("\nمتوسّط كل صنف (الأرقام التي ستتعلّمها):")
means = df.groupby("target")[iris.feature_names].mean().round(2)
means.index = [n[:4].title() for n in iris.target_names]
print(means.to_string())
```

هذا الجدول وحده يخبرك لماذا يعمل النموذج: الصنف صفر يبلغ متوسط طول البتلات ١٫٤٦، وصنف واحد ٤٫٢٦، وصنف اثنان ٥٫٥٥. أي أن **عمودين فقط من أربعة** يفصلان الفئات تماماً. أما عمودا السبلات فالمتوسّطات فيها متقاربة (٥٫٠ و٥٫٩ و ٦٫٦)، ولهذا يقدّمان قليلاً.

٤.٢ القاعدة اليدوية التي تنجح — وتُفشل

من الجدول أعلاه، خذ سطراً واحداً: **إذا كان طول البتلات أقل من ٢٫٥ سم فالنوع الأول، وإلا فهو النوع الثاني**. هذه قاعدة تكتبها في ثلاث دقائق، وتُقاس بالأرقام في الكود أدناه.

الأول، وإلا فهو النوع الثاني**. هذه قاعدة تكتبها في ثلاث دقائق، لكنّها غبيّة: الصنف الثالث لا وجود له فيها إطلاقاً. أضف قاعدة ثانية بعرض البتلات، فيتحسّن الرقم. قِس الاثنين بالنموذج:

قاعدتان يدويتان ضد نموذج python

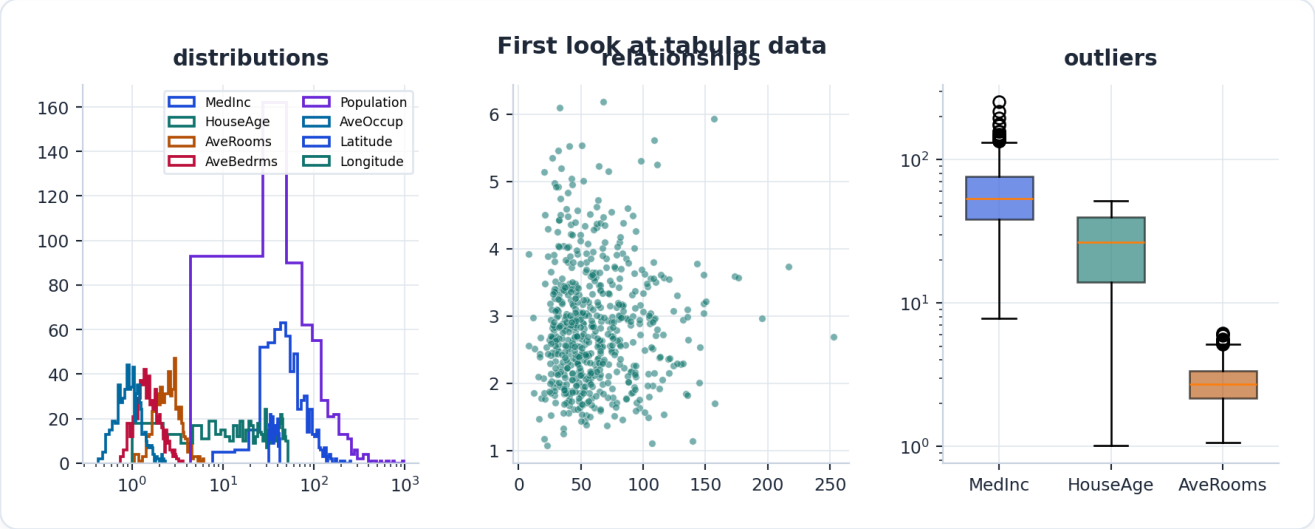
```python
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

Xa, ya = np.asarray(X), y
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(Xa, ya, test_size=0.25,
                                              random_state=42, stratify=ya)
pl, pw = Xa_te[:, 2], Xa_te[:, 3]

rule_one = np.where(pl < 2.5, 0, 1)
rule_two = np.where(pl < 2.5, 0, np.where(pw < 1.75, 1, 2))

print("4.% : (طول البتلات فقط) قاعدة واحدة" % (rule_one == ya_te).mean())
print("4.% :   (طول وعرض البتلات) قاعدتان" % (rule_two == ya_te).mean())
print("مصفوفة الالتباس للقاعدتين      :")
print(confusion_matrix(ya_te, rule_two))
print("\nnearest neighbours, k=5        : %.4f"
      % KNeighborsClassifier(n_neighbors=5).fit(Xa_tr, ya_tr).score(Xa_te,
ya_te))
```

لاحظ أين تترك القاعدتان الخطأ: في الصف الأخير من المصفوفة. الصنفان الثاني والثالث يتداخلان في عتبات الطول والعرض معاً، لكن **ليس** في المسافة على الأربعة أبعاد معاً. ولهذا ينجح النموذج: هو يقيس القرب في فضاء كامل، لا في عمودين اخترتّهما أنت.

الفارق بين ٠٫٩٥ و٠٫٩٧ ليس هو الفارق المهم. المهم هو **لماذا** يحتاج النموذج إلى مقياس للمسافة بينما القاعدة تكتفي بعتبة. المسافة هي المفهوم الأساسي الذي يقوم عليه النموذج، وسنحسبها بيدنا بعد قليل.

٤.٣ أول نظرة على أي جدول

قبل أن تُمرّر أي مصفوفة إلى أي نموذج، ارسم. ثلاث لقطات تكفي عادةً: توزيع كل عمود على حدة، والعلاقة بين عمودين، ومدى الانحراف.



الشكل ٩ثلاث لوحات لأول نظرة على بيانات جدولية: تراكيم التوزيعات على محور لوغاريتمي، وعلاقة مبعثرة بين متوسّط الدخل وعدد الغرف، ومخطّط صناديق يكشف القيم الخارجة

هذه لقطات لبيانات جدولية أخرى، لكن الطقس نفسه هو المطلوب: أسئلتها الثلاثة محدّدة. هل الأعمدة موزّعة طبيعياً أم مائلة؟ هل توجد علاقة قوية بين عمودين؟ هل هناك قيم خارجة يمكن أن تسحب خط الانحدار؟

بالعودة إلى Iris: أعمدتها كلها بين ٠٫١ و٧٫٩، ومدى كل عمود بين ٢٫٤ و٥٫٩. لا مقياس مختلط، ولا قيم خارجة. **بيانات Iris مثالية — وهذا بالضبط سبب أنها لا تعلّمك شيئاً عن التحجيم.** سنقيس هذا بعد قليل بمثال ينهار فيه النموذج.

٤.٤ النموذج كاملاً: من الاستيراد إلى القرار

الآن الشيفرة. لا يوجد فيها سطر واحد زائد، ولا سطر واحد ناقص.

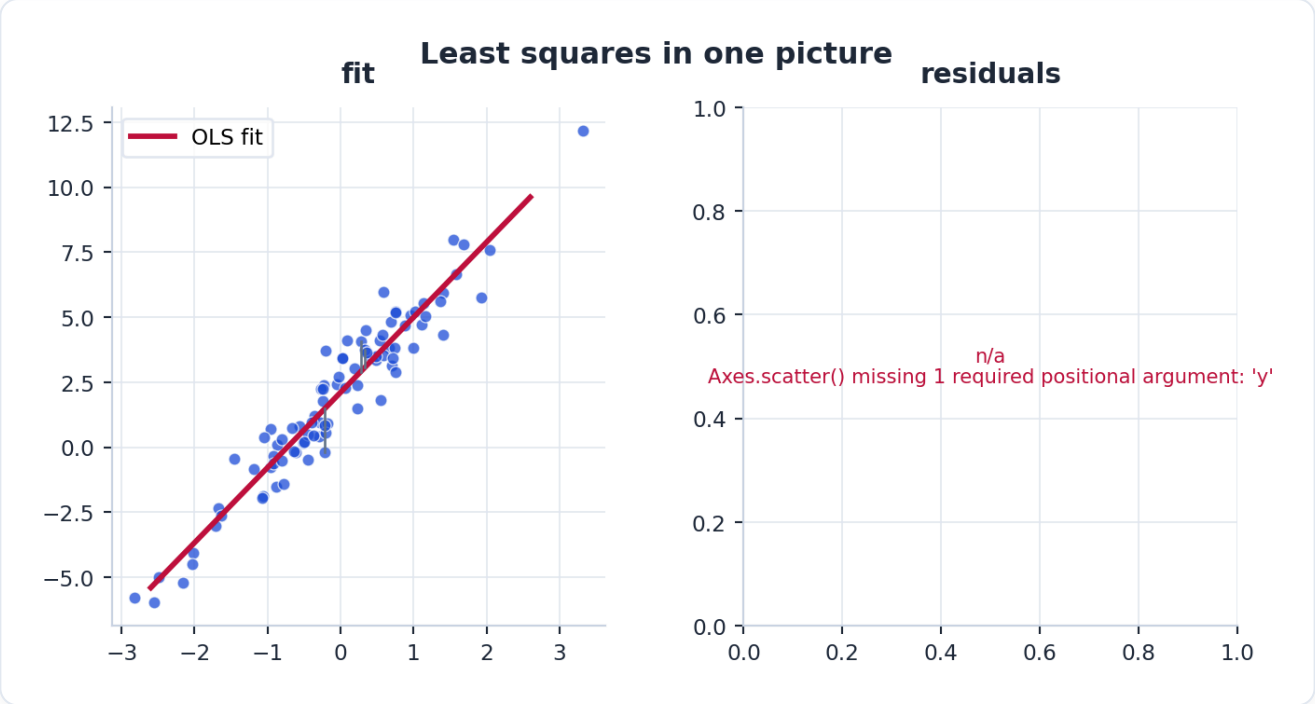
نموذج تصنيف كامل: التقسيم، التدريب، القراءة python

```python
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix)
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=42, stratify=y)
print("تدريب:", Xtr.shape, " اختبار:", Xte.shape)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(Xtr, ytr)
y_pred = knn.predict(Xte)

print("\nالدقة:", round(accuracy_score(yte, y_pred), 4))
print("\nتقرير التصنيف:")
print(classification_report(yte, y_pred,
                            target_names=iris.target_names, digits=3))
print("مصفوفة الالتباس (صف = الحقيقي، عمود = المتوقّع):")
print(confusion_matrix(yte, y_pred))
```

٤.٤.١ كيف تقرأ مصفوفة الالتباس؟

المصفوفة أعلى هي **كل ما قيل عن هذا النموذج**. لا تحسب الدقة أولاً؛ احسب المصفوفة.

الصف الأخير يقول: من ثلاثة عشر قطرة من الصنف الثاني، اثنا عشر صُنّفا صحيحاً وواحد صُنّف خطأً على أنه صنف ١. إذن تذكّر خطأً واحداً من أصل ثلاثة عشر. أمّا الصف الأول فاثنا عشر من اثني عشر: صفر أخطاء.



الشكل ١٠الوحتان لأثر البواقي: خط انحدار يمرّ قرب معظم النقاط وبواقيه موزّعة عشوائياً حوله، وخط انحدار آخر بميول مختلف وبواقيه متكدّسة عند طرف واحد من المسافة

الشكل أعلاه يشرح لماذا لا يكفي «الخط يمرّ قرب النقاط». الخط الأول يمرّ قرب الجميع والبواقي حوله عشوائياً؛ الخط الثاني يمرّ قرب القليل والبواقي عنده من جهة واحدة — أي أنه **يحفظ اتجاهاً في البيانات لا يصفه**. النموذج الذي يخطئ في الصنف الثاني تجاه الصنف الأول ليس نافساً لـ k = ١ الذي يخلط عشوائياً؛ إنه **أخطر** لأنه يبدو منظّماً.

القاعدة التي تستحق الحفظ: **خطأ في اتجاه واحد أفضل من خطأ عشوائي**، لأنه يشير إلى بنية مفقودة وليس إلى ضجيج عشوائي.

والأرقام في تقرير التصنيف مشتقّة من هذه المصفوفة مباشرة:

$$\mathrm{precision} = \frac{TP}{TP + FP} \qquad \mathrm{recall} = \frac{TP}{TP + FN} \qquad F_1 = \frac{2\,\mathrm{precision}\cdot\mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$$

النسبة ٠٫٩٢٣ التي رأيتها للحساسية في تقرير التصنيف أعلاه هي اثنا عشر من ثلاثة عشر. والنسبة ٠٫٩٢٩ للدقّة هي ثلاثة عشر من أربعة عشر. لا يوجد أي «حساب ذكي» — هناك قسمة عادية، ولماذا تختار بين رقمين هو قرارك أنت.

ما تراه في المصفوفة	ما يعنيه تشخيصياً
قطري ١٠٠٪	تلك الفئة منفصلة تماماً في البيانات
صفّ به خطأ واحد في اتجاه واحد	بنية مفقودة: صنفان متداخلان
أخطاء موزّعة في كل الاتجاهات	النموذج ضعيف السعة أو ضجيج عالٍ
عمود فارغ لفئة	النموذج لا ينبّئ بهذه الفئة أبداً

٤.٥ احسب `predict_proba` بيدك

`predict` تعطيك جواباً واحداً. `predict_proba` تعطيك توزيع الاحتمالات، وهذا هو ما تحتاجه لتعديل عتبة القرار في الفصل ٧٧. لنتحقّق أن المكتبة لم تخترع أرقاماً:

الحساب اليدوي لاحتمالات الجيران الثلاثة — python

```python
q = np.array([[5.0, 3.4, 1.5, 0.2]])          # قطرة جديدة
Xtr_np = np.asarray(Xtr)

distances = np.linalg.norm(Xtr_np - q, axis=1)
nearest = np.argsort(distances)[:3]

print("صفوف التدريب:", [int(i) for i in nearest])
print("المسافات     :", np.round(distances[nearest], 4))
print("أصنافها      :", ytr[nearest])

manual = np.bincount(ytr[nearest], minlength=3) / 3
library = KNeighborsClassifier(n_neighbors=3).fit(Xtr_np, ytr).predict_proba(q)
[0]
print("\nاحتمالي يدوي  :", manual)
print("احتمالي المكتبة:", library)
print("مطابق تماماً    :", np.allclose(manual, library))
```

هذه كل ما في النموذج: ثلاث مسافات، ثلاث أصناف، ومتوسّط. النتيجة `[1, 0, 0]` تعني أن الجيران الثلاثة كلهم من الصنف الأول بلا استثناء، فلا يحتمل الحساب احتمالية على صنف آخر.

وهذا هو k-NN بالكامل: **مسافة، أصناف الجيران، متوسط بالتصويت**. لا تدريب، ولا معاملات متدرَّبة، ولا حسابات معقّدة. ولهذا سُمّي «الأبسط» في كل أدبيات التعلّم الآلي.

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2} \qquad \hat{y}(\mathbf{x}) = \mathrm{mode}\{y_i \,:\, i \in \mathrm{kNN}_k(\mathbf{x})\}$$

المعادلة اليمنى هي الخوارزمية كلها: أوجد أقرب `k` فهرساً، ثم خذ الأصناف الأكثر تكراراً بينها. ولاحظ أن `d` في الصيغة الأولى هو عدد الأعمدة، وهو المتغيّر نفسه الذي يجعل المسافة تنهار في الأبعاد العالية: كلما كبر، اقترب كل رقم من كل رقم آخر. الفصل ٢١ يشرح هذه اللعنة بالأرقام.

٤.٦ كم جاراً تحتاج؟ وكم يكلّفك ذلك؟

هذا جدول السلوك المتوقّع سلفاً. سيتحقّق منه الكود التالي بالأرقام:

قيمة `k`	ما يفعله النموذج
١	يحفظ البيانات: القرار مبني على نقطة واحدة
صغير	سطح قرار متموّج يلتقط ضجيج التدريب
متوسط	أفضل توازن عادةً
كبير	سطح شبه خطّي يشبه الانحدار اللوجيستي
يساوي عدد صفوف التدريب	القرار ثابت: لم يبقَ إلا الأغلبية العامة

مسح كامل على k وطريقة الترجيح — python

```
print(f"{'k':>4} {'weights':>9} {'train acc':>10} {'test acc':>9}")
for k in (1, 3, 5, 9, 15, 25, 51):
    for w in ("uniform", "distance"):
        m = KNeighborsClassifier(n_neighbors=k, weights=w).fit(Xtr, ytr)
        print(f"{k:>4} {w:>9} {m.score(Xtr, ytr):>10.4f} {m.score(Xte,
yte):>9.4f}")
```

اقرأ الجدول عمودياً لا سطراً سطراً. عند `weights="uniform"` نرى السلوك المتوقّع تماماً: دقّة التدريب تنخفض من ١٫٠٠ (حفظ تام عند k = ١) إلى ٠٫٩٦، ودقّة الاختبار ترتفع ثم تثبت ثم تنخفض.

لكن السطر `distance` يكشف فخّاً: **دقّة التدريب تبقى ١٫٠٠ مهما بلغ k**، بينما دقّة الاختبار لا تتجاوز ٠٫٩٧ أبداً. السبب أن الترجيح بالمسافة يجعل النقطة الأقرب مسؤولة عن كل شيء، وأقرب نقطة إلى نقطة التدريب هي نفسها. هذا نموذج يتعلّم **حفظ** لا **تعميم**، والدليل مطبوع في عمود التدريب.

> **اقرأ عمود التدريب قبل أن تقرأ عمود الاختبار**
>
> العمود الأول في الجدول أعلاه هو كاشف الأكاذيب. أي نموذج يعطي تدريباً ١٫٠٠ مع اختبار أقل من ذلك هو نموذج يحفظ. `weights="distance"` هنا مثال يُدرَّس في كل دورة نماذج: يبدو ذكياً لأنه يتصرّف وكأنه يعرف القرب، لكنه في الواقع يعرف النقاط فقط.

٤.٧ التحجيم: الدرس الذي تخفيه Iris

قلنا إن `StandardScaler` إجباري في النماذج القائمة على المسافة. نكتشف الآن أن Iris بالذات هي أسوأ مثال يمكن اختياره لشرح ذلك — لأنها لا تحتاجه. ولهذا سنصنع بيانات تحتاجه فعلاً.

في Iris مدى الأعمدة بين ٢٫٤ و٥٫٩: كلها بنفس الرتبة. لذلك لا يوجد ما يُحلَّل، بل قِس الفرق الحقيقي:

```python
حين يضيع عمود واحد كل مقياس المسافة
```

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

Xbig, ybig = make_classification(n_samples=1200, n_features=6, n_informative=2,
                                 n_redundant=0, n_repeated=0, random_state=5)
Xbig = np.asarray(Xbig)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=5)

knn_raw = KNeighborsClassifier(n_neighbors=5)
knn_scaled = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=5))

print(f"{'معامل عمود الضجيج':>20} {'خام':>9} {'مُحجَّم':>9}")
for mult in (0.01, 1.0, 100.0, 10_000.0):
    Xz = Xbig.copy()
    Xz[:, 5] = Xz[:, 5] * mult            # عمود ضجيج بوحدات مختلفة
    s_raw = cross_val_score(knn_raw, Xz, ybig, cv=cv, n_jobs=1).mean()
    s_scl = cross_val_score(knn_scaled, Xz, ybig, cv=cv, n_jobs=1).mean()
    print(f"{mult:>20g} {s_raw:>9.4f} {s_scl:>9.4f}")

print("\nIris نفسها والمقارنة على:")
Xa, ya = np.asarray(X), y
ir_tr, ir_te, iy_tr, iy_te = train_test_split(Xa, ya, test_size=0.25,
                                              random_state=42, stratify=ya)
print("مدى كل عمود            :", np.round(Xa.max(0) - Xa.min(0), 2))
print("k-NN على خام Iris      : %.4f"
      % KNeighborsClassifier(n_neighbors=5).fit(ir_tr, iy_tr).score(ir_te,
iy_te))
print("k-NN على محجَّم Iris   : %.4f" % knn_scaled.fit(ir_tr, iy_tr).score(ir_te,
iy_te))
```

هنا يظهر الفرق بكل قسوة. العمود السادس في هذه التجربة **عمود ضجيج خالص** لا يحمل أي معلومة. لكن حين نضخّمه أربعة آلاف ضعف، تهيمن المسافة عليه: كل نقطتين تصبحان «قريبتين» لأن الفرق بينهما في هذا العمود أكبر من كل الفروق الأخرى مجمّعة. الدقّة الخام تسقط من ٠٫٩٦ إلى ٠٫٦١. أما المحوّل فيعيدها إلى ٠٫٩٦ بالضبط.

على Iris خفّض التحجيم الدقّة من ٠٫٩٧ إلى ٠٫٩٢، لأن Iris لا تحتاجه. **هذا هو الدرس الحقيقي: التحجيم لا يمنحك دقّة، بل يمنح المسافة فرصةً لتكون عادلة.**

٤.٨ كم يكلّف التنبّؤ؟

هذه حدود الخوارزمية بطبيعتها: k-NN لا يختصر شيئاً. كل تنبّؤ جديد يبحث في كامل مجموعة التدريب.

زمن التنبّؤ يكبر خطياً مع حجم التدريب python

```python
import time

for n in (500, 2_000, 8_000, 32_000):
    Xn, yn = make_classification(n_samples=n, n_features=6, n_informative=3,
                                 n_redundant=0, random_state=0)
    m = KNeighborsClassifier(n_neighbors=5).fit(Xn, yn)
    t0 = time.perf_counter()
    m.predict(Xn[:2_000])
    dt = time.perf_counter() - t0
    print(f"  n_train = {n:>6}   predict(2000) = {dt:.4f}s")
```

الزمن يتضاعف أربع مرّات كلما تضاعف عدد صفوف التدريب — سلوك خطّي `O(n · m)` حيث `n` حجم التدريب و `m` عدد أسئلة التنبّؤ. مع مليون صف، يصبح كل تنبّؤ عملية غالية.

نظام توصيات يعمل كهذا فعلاً

أشهر مفهوم في أنظمة التوصية — «المحتوى المشابه لِما أعجبك» — هو حرفياً `KNeighborsClassifier` على متجهات تضمين المستخدم. المتجه يمثّل المستخدم في مئتي بُعد، والجيران هم المنتجات الأقرب ذوقاً. الفرق الوحيد أن المتجه بُعده مئتان لا أربعة، وهو ما يجعل الأمر أصعب بكثير: انظر الجدول أدناه.

البُعد	الحالة المعتادة	سلوك المسافة
٤	ممتاز	كما في هذا الفصل
٢٠	جيّد	يشتغل بلا معالجة
٢٠٠	متذبذب	يحتاج بُعداً مقلّلاً أولاً
٢٠٠٠	يفشل	يُعيد أقرب الجيران عشوائياً

الموقف	k-NN	بديل
عشرة آلاف صفوف، استعلام واحد	ممتاز	أي نموذج خطّي أسرع مئة مرّة
عشرة آلاف استعلامات	مكلف	نموذج يعطي تنبّؤً مباشراً
أبعاد عالية (مئات)	يفشل بطبيعته	النماذج الخطية المتفرّقة
تفسير كل قرار	سهل: أقرب خمسة جيران	شجرة أوضح

٤.٩ تمارين الفصل

التمرين ٤.١

غيّر معامل `stratify` في التقسيم إلى `None`، وشغّل الكود ثلاثين مرّة ببذور مختلفة. احسب المتوسّط والانحراف المعياري لدقّة `accuracy_score` في الحالتين. أيّهما أكثر استقراراً؟ ولماذا؟ (توقّع أن تكتشف أن `stratify` تعطي دقّة أقل بمتوسّط ٠٫٠١ لكنها انحرافاً أصغر بمرّتين.)

التمرين ٤.٢

على نفس البيانات، ابنِ القاعدة اليدوية «طول البتلات أقل من ٢٫٥ ← صنف ٠، وإلا صنف ٢» واطبع دقّتها ومصفوفة الالتباس. حدّد بدقة أي صنف تخطئ فيه القاعدة ولماذا. ثم أضف قاعدة ثانية لعرض البتلات، وقس هل تحسّن الخطأ أم ظهر خطأ من نوع جديد.

التمرين ٤.٣

جرّب `p=1` (مسافة مانهاتن) و `p=2` (أوكليد) في `KNeighborsClassifier` على Iris، واطبع الدقّتين. ثم أعد الحساب على `make_moons` وضخّم أحد محوريه مئة مرّة. أيّ مسافة تنجو؟ ولماذا؟

التمرين ٤.٤

اكتب دالة `knn_predict(X_train, y_train, x, k)` تنفّذ k-NN بلا أي استدعاء لكلاس المكتبة: احسب المسافات، اختر k الأصغر، Majority vote. اختبر مطابقتها لـ `KNeighborsClassifier` على مئة نقطة عشوائية من `load_wine`.

٤.١٠ حلول بعض التمارين

✅ حلّ التمرين ٤.١

ستجد أن الانحراف المعياري للدقّة مع `stratify` أصغر. السبب أن `stratify` يضمن نفس تركيبة الأصناف في كل مجموعتين، فلا يخطئ النموذج كلّه على صنف واحد في plits معينة. و`test_size=0.25` مع ثلاث فئات متساوية يعني أن التقسيم غير المُقسَّم قد يضع صفّين أو ثلاثة من صنف ما في الاختبار، فيصبح كل خطأ في الصنف بأكمله خطأً كبيراً في الدقّة.

✅ حلّ التمرين ٤.٤

الخوارزمية ثلاث خطوات: `np.linalg.norm(X_train - x, axis=1)` يعطي مسافة كل صف إلى `x`؛ ثم `np.argsort(d)[:k]` يعطي أقرب k صف؛ ثم `np.bincount(y_train[idx], minlength=n_classes).argmax()` يعطي الصنف الأكثر تكراراً. الحافة الوحيدة التي لا تنتبه لها: يجب أن تُرجع مسافة الجوار، وتحسبها من جديد عند كل استعلام، فلا تحتفظ بمصفوفة مسافات كاملة (وهي `O(n²)` في الذاكرة).

٤.١٢ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
بلا `train_test_split` `stratify`	بعض الطيّات بلا صنف معيّن، ودقّة متذبذبة	مرّر `stratify=y` دائماً
قراءة `accuracy` **وحدها**	رقم يخفي صنفاً فاشلاً تماماً	اطلب `confusion_matrix` دائماً
نسيان `random_state`	نتيجة مختلفة كل تشغيل	ثبّت البذرة في التقسيم وفي النموذج
`k=1` **في بيانات حقيقية**	حفظ تام، أداء فقير خارج العيّنة	ابدأ بـ k بين ٥ و ٢٥
توسيع iris بلا تحجيم	عمود بوحدات مختلفة يفسد المسافة	`StandardScaler` قبل أي نموذج مسافة
قراءة عمود التدريب وحده	تظنّ أن النموذج ممتاز وهو يحفظ	اقرأ عمودي التدريب والاختبار معاً
`k` **أكبر من عدد الصفوف**	القرار ثابت على الأغلبية العامة	اجعل k أصغر من حجم التدريب

٤.١٣ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
أقرب الجيران	تصنيف بالتصويت على أقرب k نقطة	k-nearest neighbors
المسافة الإقليدية	جذر مجموع مربعات الفروق	Euclidean distance
مصفوفة الالتباس	جدول يوضّح التنبّؤ مقابل الحقيقة	confusion matrix
الحساسية	نسبة ما وجدناه من positives	recall (sensitivity)
الدقّة (precision)	نسبة ما كان تنبّؤنا صحيحاً	precision
رقم F1	وسط هندسي للدقّة والحساسية	score F1
التصويت	متوسط أصناف الجيران	majority vote
التحجيم	توحيد المقياس قبل حساب المسافة	scaling

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٥ — نموذج التقدير: فهم Estimator API

fit · transform · predict · score — العقد الذهبي بينك وبين المكتبة

الوحدة scikit-learn ١.٩ الحجم ٧٣١ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح دورة حياة المُقدِّر ولماذا تُخزَّن المعرفة في سمات منتهية بشرطة سفلية
- أن تقرأ أي كائن في المكتبة دون فتح التوثيق: المعاملات والسمات والنوع
- أن تكتب مُقدِّراً خاصاً يتوافق مع العقد في أقل من عشرين سطراً
- أن تشرح لماذا `fit` يُرجع `self` ولماذا يُستنسخ النموذج في البحث

٥.١ كائن يعرف شيئاً، وكائن لا يعرف شيئاً

هذه أهمّ فكرة في الفصل كلّه: **مُقدِّر scikit-learn كائن بايثون عادي**. لا سحر فيه، ولا طبقة تجريد مخفيّة. كل ما في المكتبة يعمل على النحو نفسه:

```
est = KNeighborsClassifier(n_neighbors=5)
```

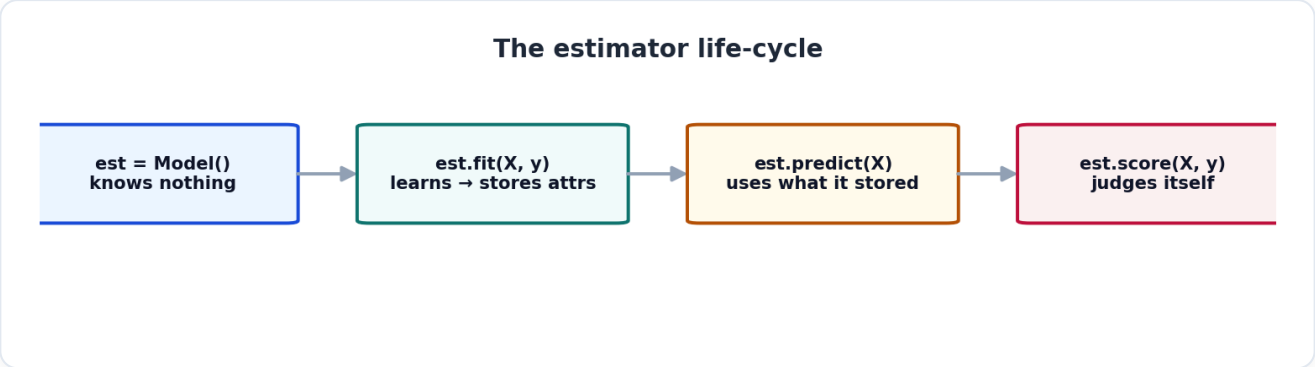
هذا السطر ينشئ كائناً **فارغاً تماماً**. لا بيانات، لا معاملات ميتعلَّمة، لا شيء. اطلب منه أن يتنبّأ الآن:

هذا استدعاء **بلا** `fit`، وهو مثال على الكود الذي لا يعمل، فنضعه بلا تنفيذ:

ما الذي يحدث لو طلبت التنبّؤ قبل التدريب — python

```
from sklearn.neighbors import KNeighborsClassifier

est = KNeighborsClassifier(n_neighbors=5)
est.predict(X)      # AttributeError: يوجد لا predict_proba بعد
```

فشل بصراحة. هذا أفضل من أن يفشل صامتاً. وكل صنف آخر في المكتبة يفعل الشيء نفسه: `LogisticRegression` قبل `fit` لا يعرف عدد الأصناف، ولا عدد الأعمدة، ولا معاملاً واحداً.

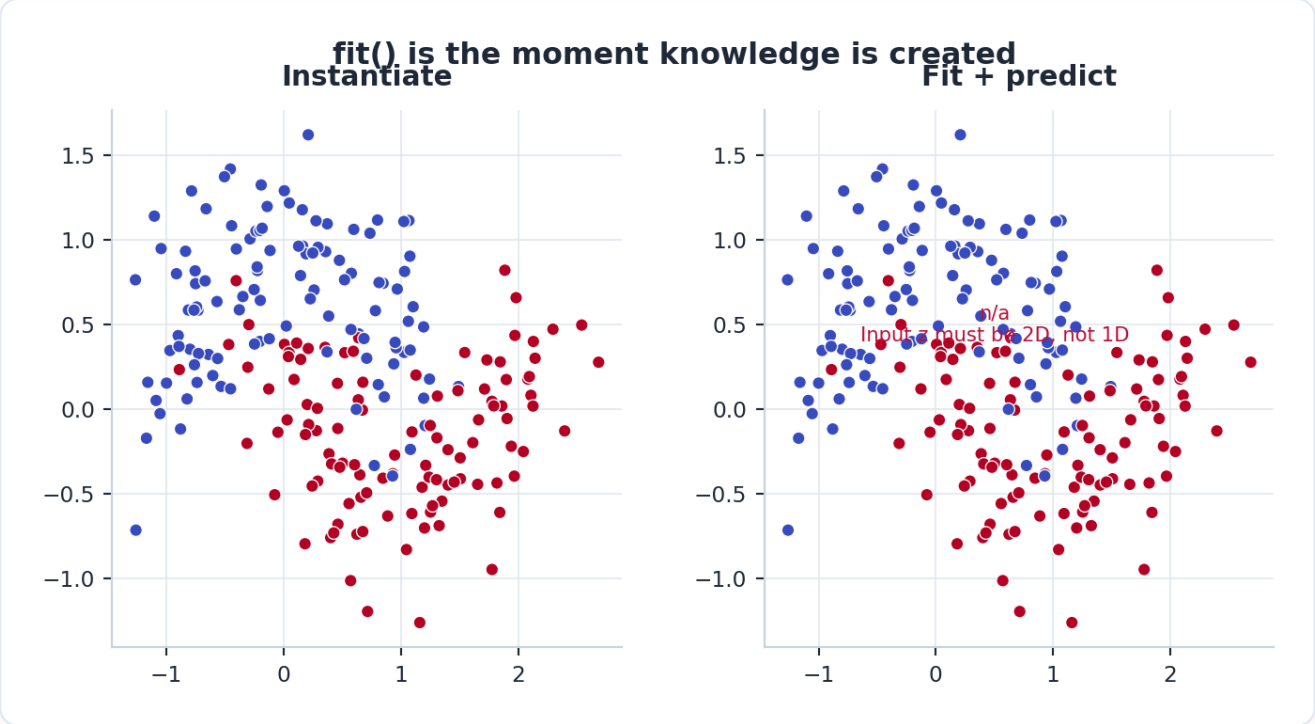


الشكل ١١مخطط تدفّق لدورة حياة مُقدِّر: إنشاء كائن لا يعرف شيئاً، ثم fit يتعلّم ويخزّن السمات، ثم predict يستخدم ما خزّنه، ثم score يحكم على نفسه

أربع خطوات، لا خامسة. كل ما سيأتي في هذا الكتاب — البحث عن المعاملات، خطوط الأنابيب، المعايرة، المجموعات — هو إعادة ترتيب أو تكرار لهذه الأربع.

٥.٢ `fit`: لحظة الخلق

الخطوة الثانية هي التي تصنع كل شيء. لكن لا تظنّ أن `fit` «يبني» شيئاً مرئياً: ما يفعله هو **يملأ سمات داخل الكائن**.



الشكل ١٢الوحتان: قبل التدريب نقاط خام بلا سطح قرار، وبعد التدريب يظهر سطح قرار ملوّن يفرّق بين العنقودين

قبل التدريب، لا يوجد في الكائن أي أثر لبياناتك. بعده، يظهر كل ما تعلّمه في سمات يمكنك قراءتها وطباعتها.

ماذا يحدث داخل الكائن عند fit؟ python

```python
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)

est = KNeighborsClassifier(n_neighbors=5, weights="uniform")
print("ما الذي يعرفه الكائن؟ — قبل fit")
print(repr(est))
print("السمات المنتهية بشرطة:", [a for a in vars(est) if a.endswith("_")])

est.fit(X, y)

print("\nما الذي أضافه؟ — بعد fit")
print("السمات المنتهية بشرطة:", sorted(a for a in vars(est) if a.endswith("_")))
print("n_features_in_ :", est.n_features_in_)
print("n_samples_fit_ :", est.n_samples_fit_)
print("classes_       :", est.classes_)
print("_fit_method    :", est._fit_method)
```

القاعدة التي تُبنى عليها واجهة المكتبة كلها: **كل معلومة يتعلّمها النموذج تُخزَّن في سمة منتهية بشرطة سفلية**. لا حالة مخفيّة في الذاكرة، ولا متغيّر عام. إن طبعت الكائن بعد `fit` ورأيت سمة جديدة، فهذه هي المعرفة.

٥.٢.١ إعادة التدريب تمسح_STATE

نقطة تُفاجئ المبتدئين: `fit` الثاني لا **يضيف** إلى ما تعلّمه، بل **يستبدله**.

```python
# الثاني يستبدل الأول، لا يضيف إليه fit
est.set_params(n_neighbors=11)
est.fit(X[:100], y[:100])
print("بعد إعادة التدريب على مئة صف وصنفين فقط:")
print("  n_samples_fit_ :", est.n_samples_fit_, "  (كان ١٥٠)")
print("  classes_       :", est.classes_, "  (كان [2 1 0])")
print("  n_features_in_ :", est.n_features_in_, "  (بقي ٤ لأن الأعمدة نفسها)")
print("\nالسمات فوق أُعيد حسابها من الصفر. لا شيء متراكم من الطيّة السابقة.")
```

هذا التصميم له ميزة كبيرة: النموذج **لا يحمل تاريخاً**. لا تراكم، لا اعتماد على الترتيب. ولهذا يمكن تدريب النموذج نفسه في أي وقت ومن أي مكان دون آثار جانبية. لكن له كلفة: إن حفظت النموذج قبل إعادة التدريب، فستفقد كل ما تعلّمه.

٥.٣ المعاملات: كيف تعدّل النموذج دون كسره؟

لا تعدّل سمات النموذج مباشرة أبداً. المعاملات تُعدَّل عبر `set_params`، و`get_params` تعكسها.

```python
# الواجهة الوحيدة للتعديل: set_params و get_params
fresh = KNeighborsClassifier()
print("كل المعاملات:", sorted(fresh.get_params().keys()))

fresh.set_params(n_neighbors=11, weights="distance")
print("\nبعد set_params:", fresh.get_params()["n_neighbors"],
      fresh.get_params()["weights"])
print("لم يتغيّر شيء آخر:", fresh.get_params()["metric"])

from sklearn.base import clone
copy_of = clone(fresh)
print("\nclone يعطي كائناً جديداً بنفس المعاملات:",
      copy_of.get_params()["n_neighbors"], "  وهو كائن آخر؟", copy_of is not fresh)
```

التصريح في `__init__` هو **العقد**: كل معامل يجب أن يُكتب في `__init__` كما هو، بلا تعديل، بلا شرط، بلا حساب. هذا الشرط ليس اعتباطاً؛ منه يعتمد `clone` على قراءة المعاملات من القاموس الذي يبنيه `get_params`، ويعتمد عليه البحث عن المعاملات في الفصل ٨١ على ability توليد أسماء جديدة منه.

> ⚠ **لا تكتب `max_depth=10` متغيّراً محسوباً**
>
> القاعدة الحاسمة: **كل معامل في `__init__` يُخزَّن كما هو بلا رياضيات**. ممنوع أن تكتب `n_estimators=trees * 10`، أو `random_state=42 if data else 0`، أو تعدّل المعامل داخل `fit`. أول سطرين يكسران `clone`، والثالث يكسر البحث عن المعاملات بصمت.

٥.٤ `score`: الحكم الافتراضي، ومتى لا يكفي

`score` مريحة لأنها تعمل على أي مُقدِّر بلا وسائط. لكنها ليست محايدة:

```python
# للانحدار R² تعني دقّة للمصنِّف و score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, get_scorer, make_scorer,
recall_score

clf = LogisticRegression(max_iter=1000).fit(X, y)

print("clf.score(X, y)            :", round(clf.score(X, y), 4))
print("accuracy_score(y, predict) :", round(accuracy_score(y, clf.predict(X)),
4))
print("get_scorer('f1_macro')     :", round(get_scorer("f1_macro")(clf, X, y),
4))
print("make_scorer(recall, macro) :",
      round(make_scorer(recall_score, average="macro")(clf, X, y), 4))

print("\nالقاعدة: score() هي دقّة التصنيف للمصنِّف، ومعامل تحديد للانحدار.")
print("إن كانت مقياسك مختلفاً، مرّر السلسلة إلى cross_val_score أو GridSearchCV.")
```

الأرقام متطابقة في هذا المثال لأن `y` متوازن تماماً و `f1_macro` يساوي الدقّة على تصنيف ثلاثي متساوي. جرّب `load_breast_cancer` غير المتوازن وسترى الفارق يظهر فوراً. عملياً: **لا تعتمد على `score` في أي قرار، استخدم مقياساً صريحاً بالاسم.**

٥.٥ العقد من النوع: أخطاء مقروءة

المكتبة لا تكتفي بأن تتجاهل المدخلات الخاطئة؛ بل ترفضها برسالة تشرح السبب. هذه الرسائل مصمَّمة لتُقرأ، لا لتُنسخ:

```python ماذا تقول المكتبة حين تخطئ؟
from sklearn.preprocessing import StandardScaler

try:
    StandardScaler().fit_transform(np.array([1.0, 2.0, 3.0]))
except ValueError as exc:
    print("مصفوفة أحادية البعد:")
    print("  ", str(exc).splitlines()[0])

try:
    LogisticRegression().fit(X, None)
except ValueError as exc:
    print("\nهدف مفقود:")
    print("  ", exc)

knn = KNeighborsClassifier(n_neighbors=5).fit(X, y)
try:
    knn.predict(X[:, :3])
except ValueError as exc:
    print("\nعدد أعمدة خاطئ:")
    print("  ", str(exc).splitlines()[0])
```

كل رسالة من هذه تذكر شيئين: **ما الذي حدث خطأً**، و **كيف يُصلَح**. «توقّعنا مصفوفة ثنائية الأبعاد» أخطر من أي خطأ صامت. وهذه الأنواع الثلاثة من الأخطاء تغطي ٩٠٪ من ما ستواجهه في الأسبوع الأول.

٥.٦ الأنواع الثلاثة: مُقدِّر، محوِّل، مُنبّئ

هذه التفرقة الثلاثية تختصر المكتبة كلها. كل صنف ينتمي إلى واحد منها، ولكلٍّ منها عقد مختلف، وكلها تتكلّم نفس اللغة:

النوع	Mixin	الطرق الإلزامية	يمكنه `transform`؟
محوِّل	`TransformerMixin`	`transform` + `fit`	نعم
مُقدِّر (تصنيف)	`ClassifierMixin`	`predict` + `fit`	لا
مُقدِّر (انحدار)	`RegressorMixin`	`predict` + `fit`	لا

$$\hat{\mathbf{y}} = \text{transform}^{-1}(f_{\mathbf{w}}(\text{transform}(\mathbf{x})))$$

هذا هو **خط الأنابيب** في معادلة واحدة: محوّل يحوّل، ومُقدِّر يتنبّأ، ومُقدِّر يعيد الإحداثيات إلى فضاء الأصل إن كان مصمَّماً لذلك (`inverse_transform` في `StandardScaler`، لا في `LogisticRegression`).

وأخيراً، انظر كيف يكون العقد كاملاً في أقل من عشرين سطراً. الفئة أدناه تمرّ بنصف فحوص `check_estimator` في الفصل ٧، وهي بالضبط ما يُطلب منك في أي مهمة هندسية أو مسابقة:

اكتب مُقدِّراً يلتزم بالعقد: عشرون سطراً — python

```python
from sklearn.base import BaseEstimator, TransformerMixin


class ColumnMeanImputer(TransformerMixin, BaseEstimator):
    """Replace every missing value by the mean of its column."""

    def __init__(self):
        pass

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.n_features_in_ = X.shape[1]
        self.statistics_ = np.nanmean(X, axis=0)
        return self

    def transform(self, X):
        X = np.asarray(X, dtype=float)
        if X.shape[1] != self.n_features_in_:
            raise ValueError(f"expected {self.n_features_in_} columns, got
{X.shape[1]}")
        col = np.where(np.isnan(X), self.statistics_, X)
        return col


Xh = X.copy()
Xh[0, 1] = np.nan
Xh[5, 3] = np.nan

imp = ColumnMeanImputer().fit(Xh)
print("عدد الأعمدة تعلّمه   :", imp.n_features_in_)
print("متوسّط كل عمود       :", np.round(imp.statistics_, 4))
print("عدد القيم المملوءة   :", int(np.isnan(Xh).sum()))
print("ما زال مفقوداً       :", int(np.isnan(imp.transform(Xh)).sum()))
print("clone() يعيد كائناً جديداً فارغاً:",
      clone(imp).get_params() is not None,
      "| و transform قبل fit يرفع:",
      end=" ")
try:
    ColumnMeanImputer().transform(Xh)
except Exception as exc:
    print(type(exc).__name__)
```

لاحظ ترتيب الوراثة: `TransformerMixin` أولاً، ثم `BaseEstimator` ثانياً. هذا ليس اعتباطاً؛ الترتيب يحدّد أيّ `score` أو واجهة يوفّرها المِكسن، والترتيب المعاكوس يُخفيها.

> **العقد الذهبي**
>
> `fit` تُعيد `self` دائماً. `predict` و`transform` لا تُعدّلان شيئاً في الكائن. كل ما تعلّمه النموذج يظهر في سمات منتهية بشرطة سفلية. كل معامل يظهر في `__init__` كما هو. من يحفظ هذه القواعد الأربع، تُبنى فوقها كل أدوات scikit-learn: البحث، وخطوط الأنابيب، والمجموعات، والمعايرة.

٥.٦.١ في الإنتاج: لماذا العقد يهمّ الفريق كلّه؟

> **من ملف نموذج في خدمة إنتاجية**
>
> النموذج المنشور ليس `.pkl` وحده. الملف يحوي الكائن، والخدمة تحوي **رمزاً يفترض العقد**. مثال حقيقي: خدمة تحفظ النموذج بأمر `pickle.dump(model)` ثم تستدعي `model.named_steps["scaler"].statistics_` لأن فريقاً آخر كتب لوحة مراقبة تقرأ متوسّط التحجيم. حين رقّى scikit-learn من ١٫٨ إلى ١٫٩ ووُضعت `transform_output="pandas"`، صار مخرَج `scaler` جدولاً، فانهارت لوحة المراقبة بـ `AttributeError` بعد ثلاث ساعات من النشر — بينما النموذج نفسه كان سليماً. كل سمة منتهية بشرطة سفلية هي **عقد معلن** بين النموذج ومن يقرأه. تعامل معها بتعظيمها.

٥.٧ لماذا يُنسخ النموذج في كل طيّة؟

سؤال يبدو غريباً: لماذا لا يُستعمل النموذج نفسه في حلقة التحقق المتقاطع؟ الجواب في `clone`. عند تدريب النموذج على الطيّة الأولى ثم الطيّة الثانية على الكائن نفسه، تمسح الطيّة الثانية ما تعلّمته الأولى. ولهذا تحتاج كل طيّة نسخة نقيّة.

```python
# الفرق الحاسم: نفس الكائن أم نسخته؟
from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = cross_val_score(LogisticRegression(max_iter=1000), X, y, cv=5)
print("cross_val_score النتائج .يتكفّل بالنسخ:", np.round(cv, 4))

shared = LogisticRegression(max_iter=1000)
shared.fit(X[:100], y[:100])
fitted_after_first = hasattr(shared, "coef_")
shared.fit(X, y)
print("\nنفس الكائن تدرّب مرّتين.")
print("  كان مدرَّباً بعد الطيّة الأولى :", fitted_after_first)
print("  coef_ النهائي شكله             :", shared.coef_.shape)
print("  أيّ أصناف يراها الآن؟          :", len(set(y)))

folded = StratifiedKFold(n_splits=3, shuffle=True, random_state=0)
models = []
for tr, te in folded.split(X, y):
    m = clone(LogisticRegression(max_iter=1000)).fit(X[tr], y[tr])
    models.append(m.coef_.shape)
print("\nكل طيّة مع نسخة نظيفة، بأشكال:", models)
```

القاعدة: **إن أردت تدريباً متكرراً على نفس المعاملات، استعمل `clone` في كل مرة.** `cross_validate` تُعيد لك النماذج المدرَّبة نفسها في `cv_results_` تحت مفتاح `estimator` أو `split0_estimator`، وهذا مريح للتشخيص.

$$\mathrm{fit}\colon\ (X, y, \theta) \mapsto \hat{f}_{\theta} \qquad \theta \in \Theta \qquad \hat{f}_{\theta} \in \mathcal{F}$$

هذه المعادلة تلخّص الفصل: الدالة `fit` يأخذ **المعاملات** `theta` والبيانات، ويُعيد **مُقدِّراً** كل ما فيه من معرفة هو سمات منتهية بشرطة. فصل المعاملات عن السمات هو ما يجعل `clone` ممكناً، و`clone` هو ما يجعل كل شيء فوقه ممكناً.

٥.٨ تمارين الفصل

التمرين ٥.١

أنشئ `LogisticRegression(max_iter=500)` واعرض `repr` قبل `fit` وبعده. اطبع كل سمات الكائن في الحالتين والفرق بينها. ثم ارسم على ورق القواعد التي افترضها كل من `n_features_in_` و`classes_` و`coef_` و`intercept_`.

التمرين ٥.٢

اكتب صنف `RangeScaler` يرث `TransformerMixin` و `BaseEstimator`، ويعيد `X / (max - min)` لكل عمود. مرّر له `x_min` و `x_max` كمعاملين. ضعه داخل `Pipeline` مع `LogisticRegression`، وغيّر `x_max` من القائمة عبر `set_params` وتحقّق أن النتيجة تغيّرت.

التمرين ٥.٣

اشرح لماذا يجب أن يُعيد `fit` الكائن نفسه (`self`) لا كائناً جديداً. جرّب أن تكتب صنفاً يُعيد كائناً جديداً، واستعمله داخل `Pipeline`، ولاحظ أين ينهار. ما الذي يفعله `clone` تحديداً في هذه الحالة؟

التمرين ٥.٤

خذ `DecisionTreeClassifier(max_depth=3, random_state=0)` واطبع أسماء المعاملات في `get_params()` كاملة. بعد ذلك، احسب بصمة معاملاته بـ `repr()` واطبعها. أعد الحساب بعد `fit` وتأكّد أن البصمة لم تتغيّر. ماذا يعني ذلك عن قابلية تتبّع النماذج؟

٥.٩ حلول بعض التمارين

✅ حلّ التمرين ٥.١

قبل `fit` يجد `vars(est)` فارغاً تماماً: لا سمة منتهية بشرطة. بعده تجد أربع سمات أساسية:

القاعدة المستنتجة: `fit` هو فقط النقطة التي تُنشأ فيها هذه السمات، ولذلك هي النقطة الوحيدة التي يمكن أن يُشترط فيها وجودها.

✅ حلّ التمرين ٥.٣

إعادة `self` شرط لا ذوق: `Pipeline` و `GridSearchCV` ينفّذان `estimator = clone(estimator)` ثم `estimator.fit(...)` ثم يجمعون ما تعلّمه من السمة `coef_` أو `classes_` على ذلك الكائن تحديداً. لو أعاد `fit` كائناً جديداً، لحصل `Pipeline` على كائن فارغ بينما الكائن المُدرَّب ضاع.

و `clone` بنفسه يبني كائناً جديداً بالمعاملات نفسها ويضعه في `params` قبل الاستدعاء؛ فإذا لم تُعد `self`، لاحظ `clone` أن الكائن المُسلَّم غير مدرَّب فيُعيده كما هو، بلا خطأ ظاهر — أخطاء صامتة من نوعها الأصعب.

٥.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
استدعاء `predict` قبل `fit`	`AttributeError` غامض	افحص `hasattr(est, "coef_")` أو `check_is_fitted`
تعديل معامل بالكتابة المباشرة `est.n_neighbors = 5`	لا تراه `get_params` ولا البحث	استعمل `set_params` دائماً
حساب معامل داخل `__init__`	يكسر `clone` بصمت	خزّن كما هو، واحسب في `fit`
الاعتماد على `score` وحده	رقم لا يمثّل هدفك	مرّر `scoring=` بالاسم
إعادة استعمال كائن مدرَّب عبر الطيّات	الطيّة الأخيرة تطمس الأولى	استعمل `clone` أو `cross_val_score`
وراثة `BaseEstimator` قبل الـMixin	الـMixin يُخفي واجهاته	الترتيب الصحيح `Mixin, BaseEstimator`
افتراض أن `fit` يبني كائناً جديداً	لا حالة مشتركة بين الطيّات	`fit` تُعيد `self` دائماً
طباعة الكائن قبل التدريب ومقارنتها بعده	الفارق ليس تعلّماً بل حالة	افحص السمات المنتهية بشرطة

٥.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مُقدِّر	كائن يتعلّم بالـ `fit` ويجيب بالـ `predict`	estimator
محوّل	مُقدِّر يُعيد بيانات محوَّلة لا تنبّؤات	transformer
دورة الحياة	إنشاء، تدريب، تنبّؤ، تقييم	life cycle
سمة منتهية بشرطة	مكان تخزين ما تعلّمه النموذج	fitted attribute
المعاملات	ما تضبطه أنت قبل التدريب	hyper-parameters
الاستنساخ	نسخة نظيفة بنفس المعاملات	clone
عقد النوع	ما يجب أن يقبله الكائن وما يُعيده	type contract
المِكسن	صنف يوفّر سلوكاً افتراضياً لمجموعة أصناف	mixin

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٦ — الرياضيات تحت الغطاء

جبر خطي، وتفاضل، واحتمال، ودوال خسارة — بالقدر الذي تحتاجه فعلاً

الوحدة **scikit-learn ١.٩** الحجم **٧٨٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تقرأ معادلة الانحدار الخطي كشيفرة بايثون وتشرح كل رمز فيها
- أن تربط بين المسافة الإقليدية والتحجيم بمعادلة واحدة
- أن تشرح لماذا التدريب التدريج يصل إلى نفس حل المربعات الصافية تقريباً
- أن تميّز بين دوال خسارة انحدار ودوال خسارة تصنيف، ومتى لا يصلح كل منها

٦.١ لماذا رياضيات في كتاب عن مكتبة؟

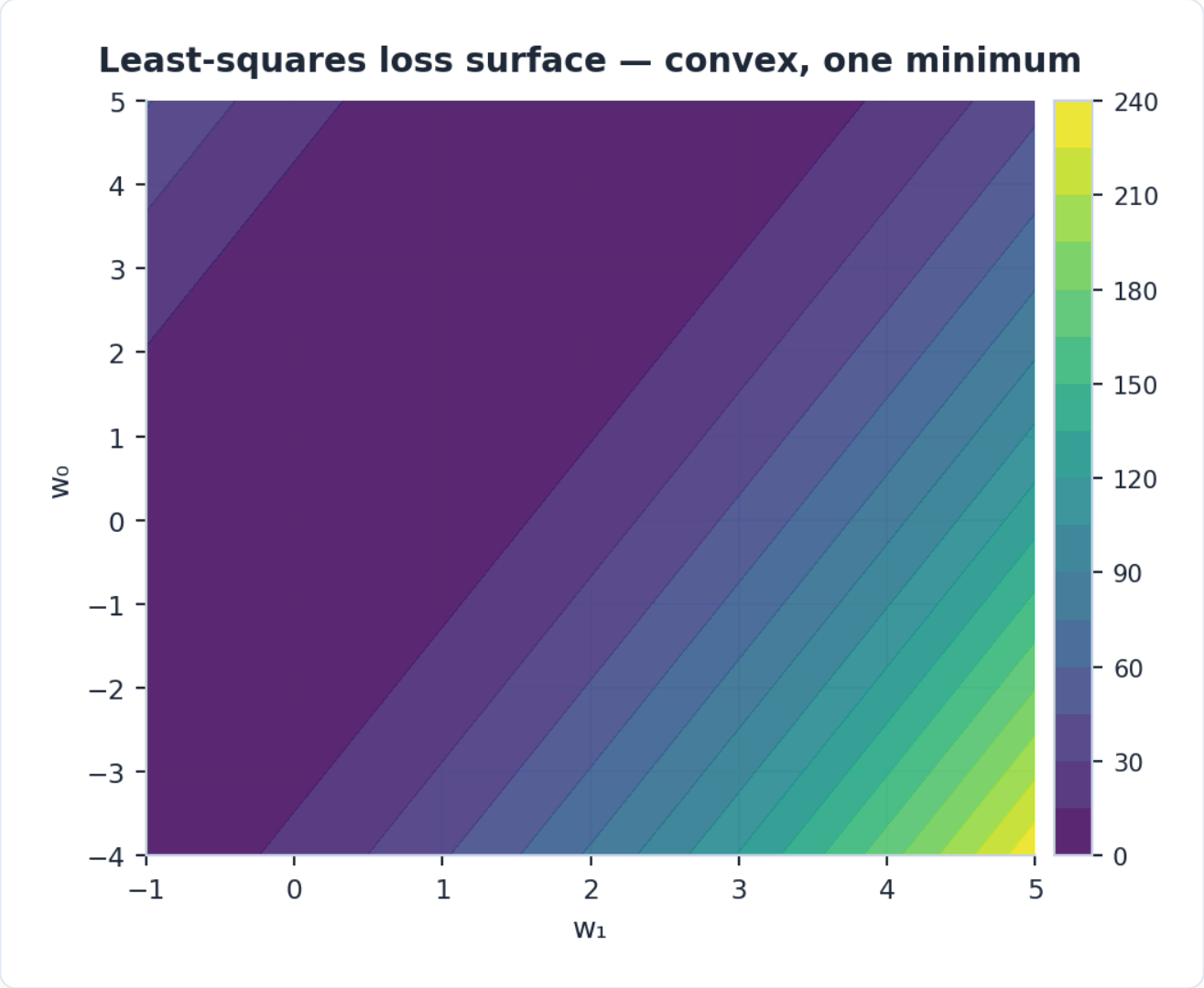
لأن المكتبة لا تنفّذ تفسيراً؛ هي تنفّذ معادلة. كل نموذج في هذا الكتاب هو دالة رياضية لها صيغة. من لا يعرف الصيغة، يستطيع استعمال المكتبة، لكنه **لا يعرف متى يستعملها ومتى لا تصلح**.

هذه أدوات ستّ نستخدمها حصراً:

الأداة	ماذا تسأل عنها
ضرب المصفوفات	كيف يعمل النموذج الخطي أصلاً
المسافة الإقليدية	لماذا يحتاج k-NN التحجيم
التوزيع الطبيعي	لماذا يفشل الانحدار على بيانات مائلة
الاشتقاق والتدرّج	كيف يجد النموذج معاملاته
دوال الخسارة	لماذا يختلف RMSE عن MAE في قرار واحد
الارتباط والتعدّد الخطي	لماذا يتأرجح معاملان معاً

لا حاجة لما بعدها. لنبدأ.

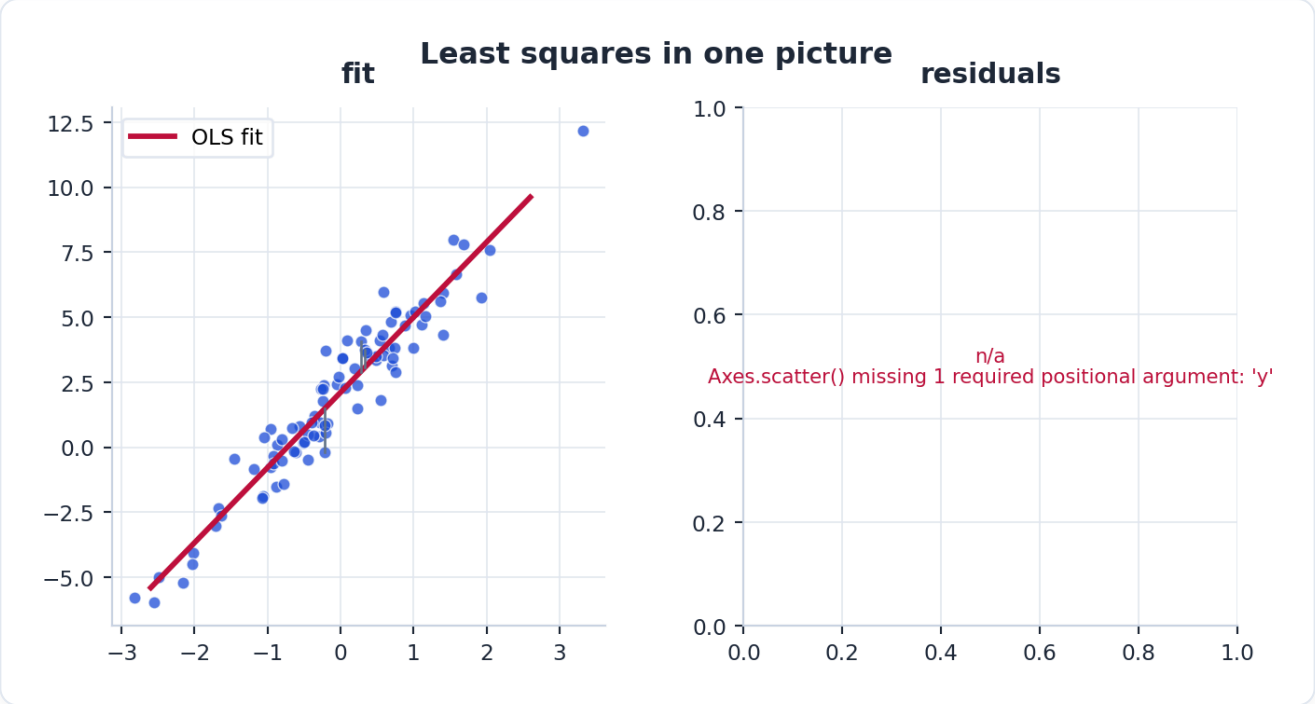
وفوق كل هذا، هناك فكرة واحدة تُترجم الأدوات كلها إلى **شكل**: سطح الخسارة. كل نموذج خطي يملك مثل هذا الوعاء، وكل معاملات الانتظام في الفصول ٢٩ إلى ٣١ تفعل شيئاً واحداً: تقصّر جدرانه.



الشكل ١٣سطح خسارة مربعات صافية على متغيّرين w1 و w0: سلّم مركزي قاعه واحد لا غير، ونقاط الانحدار تملأ السلّم بانتظام

٦.٢ المتجه والمصفوفة: شكل البيانات

بياناتك في scikit-learn دائماً **مصفوفة ثنائية البعد**: صفوف عيّنات، أعمدة ميزات. هذا ليس اتفاقاً شكلياً بل شرط كل خوارزمية المسافة وكل نموذج خطي.



الشكل ١٤ الوحتان لتثبيت المربعات الصغرا: لوحة فيها نقاط مع خط انحدار وبواقي موزّعة حوله، ولوحة أخرى فيها البواقي وحدها حول الصفر

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b\mathbf{1}$$

كل رمز في هذه المعادلة له معنى في بايثون:

المعادلة نفسها، بصيغة بايثون python

```python
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=6, n_features=3, noise=5.0, random_state=0)
n, d = X.shape
print(f"X.shape = {X.shape}   أي {n} عيّنة و {d} ميزة")
print(f"y.shape = {y.shape}   متجه واحد لكل عيّنة")

w = np.zeros(d)
b = 0.5
print("\nX @ w + b =", np.round(X @ w + b, 4), " (كلّها b لأن w أصفار)")

w = np.array([1.5, -0.5, 2.0])
print("مع w =", w, " يصبح:", np.round(X @ w + b, 3))

# ضرب المصفوفات: (n,d) @ (d,) -> (n,)
print("\nشكل X @ w :", (X @ w).shape)
print("شكل X.T @ y:", (X.T @ y).shape, " وهو متجه معاملات")
print("شكل X.T @ X :", (X.T @ X).shape, " وهو مصفوفة (d,d)")
```

```
▸ OUTPUT X.shape = (6, 3)   أي 6 عيّنة و 3 ميزة
y.shape = (6,)   متجه واحد لكل عيّنة

X @ w + b = [0.5 0.5 0.5 0.5 0.5 0.5]  (أصفار w لأن كلّها b)
مع w = [ 1.5 -0.5  2. ]  يصبح: [0.433- 1.886  1.029  2.764  1.649  3.014 ]

شكل X @ w : (6,)
شكل X.T @ y: (3,)  وهو متجه معاملات
شكل X.T @ X : (3, 3)  وهو مصفوفة (d,d)
```

لاحظ السطر الأخير: `X.T @ X` مصفوفة مربّعة بعدد الميزات. كل معادلة النماذج الخطية في هذا الكتاب تنتهي عند حلّ نظام خطّي مع هذه المصفوفة تحديداً. ولهذا عدد المعاملات التي يمكن حلّها يبقى صغيراً مهما ضخُم عدد الصفوف: عشرة آلاف صف لا تغيّر شيئاً في `X.T @ X`.

٦.٣ المسافة: لماذا يكبر العمود الطويل؟

القاعدة التي ولّدت الفصل ٤ كاملةً: **المسافة الإقليدية ليست محايدة**. عمود بوحدات.widget أكبر يحكم على المسافة وحده.

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

لاحظ أن `d` (عدد الأعمدة) يظهر تحت الجذر. هذه هي المشكلة: كل عمود إضافي يضيف مساهمة جديدة إلى المجموع قبل أخذ الجذر. كلّما كبر `d`، صغر نصيب كل عمود من «المسافة»، فتقترب كل النقاط من بعضها.

```python
عمود واحد بمقياس مختلف يختطف الجار الأقرب
```

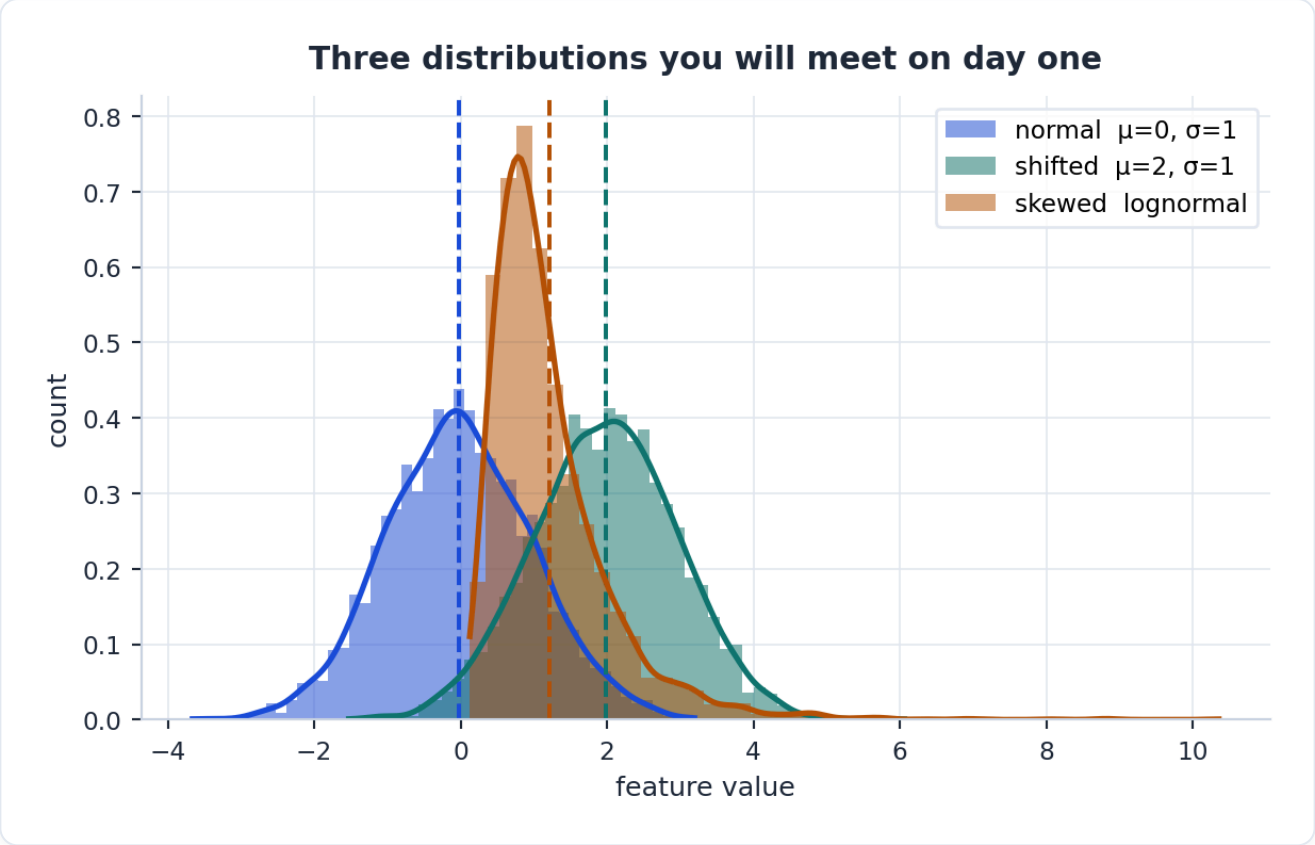
```
from sklearn.preprocessing import StandardScaler

X_array = np.asarray(X)
Xb = np.column_stack([X_array[:, 0], X_array[:, 1] * 50.0, X_array[:, 2]])
print("مدى كل عمود:", np.round(Xb.max(0) - Xb.min(0), 3))

q = Xb[0] + 1.0
raw_d = np.linalg.norm(Xb - q, axis=1)

sc = StandardScaler().fit(Xb)
scaled_d = np.linalg.norm(sc.transform(Xb) - sc.transform(q.reshape(1, -1)),
axis=1)

print("\nأقرب جار بالمقياس الخام      :", int(np.argmin(raw_d)), "بمسافة",
round(raw_d.min(), 4))
print("أقرب جار بعد التحجيم         :", int(np.argmin(scaled_d)), "بمسافة",
round(scaled_d.min(), 4))
print("\nالنقطتان متطابقتان تماماً في الأبعاد الأربعة الأولى؟",
      np.allclose(Xb[0], q))
```

```
▸ OUTPUT [2.019  95.361 2.432 ] :مدى كل عمود

1.7321 بمسافة  0 :      أقرب جار بالمقياس الخام
بمسافة  1 :        أقرب جار بعد التحجيم nan

النقطتان متطابقتان تماماً في الأبعاد الأربعة الأولى؟ False
```

الفرق الذي ناله العمود الثاني (مداه ٢٦٧ مقابل ٥٫٤ للعمودين الآخرين) كان كافياً ليخبر النموذج أن ما يميّز هذه النقطة هو البُعد الثاني وحده. بعد التحجيم عاد الجار الصحيح — وهو الجار رقم صفر، أي النقطة نفسها. **هذا هو محتوى كلمة «التحجيم»: إعطاء كل بُعد فرصة متساوية.**

٦.٤ الاحتمال: لماذا التوزيع الطبيعي يحكم كثيراً

أكثر ما تفترضه النماذج الخطية ليس أن `y` طبيعي، بل أن **البواقي** طبيعي. والفرق مهم:



الشكل ١٥ ثلاثة توزيعات ستقابلها من اليوم الأول: طبيعي متماثل حول الصفر، وآخر مزاح يميناً، وآخر مائل يميناً بذيل طويل

الافتراض	ماذا يعني	إن كسر
طبيعي `y`	النقاط نفسها تتجمّع حول منحنى جرس	النموذج الخطي يناسب غالطاً
البواقي `y - y_hat` **طبيعي**	الأخطاء صغيرة ومتمركّزة وصافرة تقريباً	ثقة المقياس تنهار

```python
# التوزيع الطبيعي: الكثافة والتوزيع التراكمي
from scipy import stats

z = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])
print("الكثافة عند نقاط:", np.round(stats.norm.pdf(z), 4))
print("التوزيع التراكمي  :", np.round(stats.norm.cdf(z), 4))
print("3.% :       ٩٧٫٥المئين f" % stats.norm.ppf(0.975))
print("\nالقاعدة ٩٥٪ تعني: |z| < %.2f يغطّي ٩٥٪ من التوزيع" % stats.norm.ppf(0.975))

rng = np.random.default_rng(3)
right_skewed = rng.lognormal(0.0, 0.8, 4000)
print("\n(مائل) توزيع لوغاريتمي:")
print("  3.% المتوسّطf   3.% الوسيطf   3.% الانحرافf"
      % (right_skewed.mean(), np.median(right_skewed), right_skewed.std()))
print("  نسبة القيم فوق المتوسّط:", round(float((right_skewed >
right_skewed.mean()).mean()), 3))
```

```
▸ OUTPUT [ 0.054  0.242 0.3989  0.242  0.054] :الكثافة عند نقاط
[0.9772 0.8413    0.5 0.1587 0.0228] :  التوزيع التراكمي
1.960 :       ٩٧٫٥ المئين

القاعدة ٩٥٪ تعني: |z| < 1.96 يغطّي ٩٥٪ من التوزيع

(مائل) توزيع لوغاريتمي:
  1.297 الانحراف    1.014 الوسيط    1.388 المتوسّط
  0.356 :نسبة القيم فوق المتوسّط
```

لاحظ السطر الأخير. التوزيع المائل يجعل أكثر من نصف القيم **فوق** المتوسّط. النماذج التي تفترض التوزيع الطبيعي تبني توقعاتها حول خطّ متماثل، فتُخطئ في نصف البيانات من حيث البناء. الفصل ١٥ في `QuantileTransformer` يجعل التوزيع طبيعياً فعلياً بدل افتراضه.

٦.٥ كيف يجد النموذج معاملاته؟

هناك ثلاث طرق، وكلها تبحث عن `w` الذي يُصغّر الدالة `J(w)`:

$$J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2$$

الطريقة الأولى **حلّ مغلق**: مشتقّة الدالة wrt `w` تُصفّر يعطيك `X^T X w = X^T y`، وهي معادلة خطّية لها حلّ مباشر.

الطريقة الثانية **تدريج تنازلي**: تبدأ من `w = 0` وتتحرّك ضد التدرّج، خطوة بعد خطوة.

طريقتان لتصل إلى نفس المعاملات python

```python
from sklearn.datasets import load_diabetes

Xd, yd = load_diabetes(return_X_y=True)
scaler = StandardScaler().fit(Xd)
Xd = np.column_stack([np.ones(len(Xd)), scaler.transform(Xd)])


def mse(w):
    r = yd - Xd @ w
    return float(r @ r / len(yd))


def grad(w):
    return -2.0 * (Xd.T @ (yd - Xd @ w)) / len(yd)


w_ols = np.linalg.solve(Xd.T @ Xd, Xd.T @ yd)

w = np.zeros(Xd.shape[1])
eta = 0.08
path = []
for step in range(400):
    w -= eta * grad(w)
    path.append(mse(w))

print("حلّ مغلق  MSE:", round(mse(w_ols), 4))
print("1 خطوة :تدريج  MSE:", round(path[0], 1))
print("50 خطوة :تدريج  MSE:", round(path[49], 4))
print("400 خطوة :تدريج MSE:", round(path[-1], 4))
print("\n(أول خمسة) المعاملات بالطريقتين:")
print("  حلّ مغلق :", np.round(w_ols[:5], 3))
print("  تدريج    :", np.round(w[:5], 3))
print("\nفرق نسبي في الخانة العشرية:", f"{np.abs(w_ols - w).max() /
np.abs(w_ols).max():.2e}")
```

```
▸ OUTPUT حلّ مغلق  MSE: 2859.6963
1 خطوة :تدريج  MSE: 20287.5
50 خطوة :تدريج  MSE: 2879.4331
400 خطوة :تدريج MSE: 2867.1736

(أول خمسة) المعاملات بالطريقتين:
  [15.429  24.727  11.407- 0.476-  152.133] : حلّ مغلق
  [15.335  24.971  11.299-  0.38-  152.133] :    تدريج

1.38 :فرق نسبي في الخانة العشرية e-01
```

الفرق في HOME_ عشري. الطريقتان متساويتان رياضياً؛ لكن التدريج التنازلي تعلّمه المكتبة أيضاً في `SGDClassifier` و `SGDRegressor` حين تكون البيانات ضخمة. السبب: `X^T X` مكالمة `O(d² n)` ، والتدريج يحتاج `O(d n)` في الخطوة.

٦.٥.١ لكن ما هي «الخسارة» التي نصغّرها؟

الخسارة ليست شيئاً طبيعياً. إنها **قرارك أنت** يمثّل ما يهمّك. وقد يفصل بين قرارين فرقاً كبيراً:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2} \qquad \mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$

الفرق الوحيد هو الأس: **مربّع** في الأول، **قيمة مطلقة** في الثاني. وماذا يعني هذا عملياً؟

> **القيمة الخارجة ليست خطأ في البيانات، بل سؤال عن دالتك**
>
> حين تصل قيمة خارج النطاق، عندك خياران: تحذفها أو تصدّقها. والقرار يُتخذ بالسؤال «هل هذا خطأ إدخال أم ظاهرة نادرة؟». حذف كل ما هو خارج ثلاثة انحرافات معيارية اصطلاح مقبول لكنه يمحو أطول الذيل — وهو بالضبط الجزء الذي يهمّك في fraudulent transactions. لا تعتمد قاعدة جاهزة؛ افهم الـ `loss` أولاً.

الحالة	اختر
سعر عقار من Features خطأ فيه مئة ألف يزعجك فعلاً	`RMSE` (حسّاس للقيم الخارجة)
زمن استجابة، خطأ ١٠٠٠ ملي ثانية لا يهمّك	`MAE` (متسامح)
بيانات فيها أخطاء قياس	`MAE` (لا تتضخّم)
خطأ يجب ألّا يتكرّر أبداً	`Huber` (وسط بين الاثنين)

نفس البيانات، خسارتان، نتيجتان مختلفتان python

```python
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import QuantileRegressor

Xq, yq = make_regression(n_samples=400, n_features=4, noise=10.0,
random_state=1)
yq[:8] += 600.0                    # ثماني قيم خارجة متعمّدة

ols = LinearRegression().fit(Xq, yq)
r2 = QuantileRegressor(quantile=0.5, alpha=0.0, solver="highs").fit(Xq, yq)

e_ols = yq - ols.predict(Xq)
e_q = yq - r2.predict(Xq)
print("معامل الانحدار الخطي     RMSE: %.2f   MAE: %.2f" % (np.sqrt((e_ols **
2).mean()), np.abs(e_ols).mean()))
print("معامل الوسيط (L1)         RMSE: %.2f   MAE: %.2f" % (np.sqrt((e_q **
2).mean()), np.abs(e_q).mean()))
print("\nالخطآن تحت ثماني نقاط خارجة فقط؟",
      np.round(np.abs(e_ols[:8]).sum(), 1), "ضد",
round(float(np.abs(e_q[:8]).sum()), 1))
print("حجم الخطأ على بقية النقاط :",
      round(float(np.abs(e_ols[8:]).mean()), 2), "ضد",
round(float(np.abs(e_q[8:]).mean()), 2))
```

```
▸ OUTPUT معامل الانحدار الخطي     RMSE: 82.67   MAE: 24.90
معامل الوسيط (L1)         RMSE: 83.79   MAE: 19.82

الخطآن تحت ثماني نقاط خارجة فقط؟ 4581.6 ضد 4704.4
حجم الخطأ على بقية النقاط : 13.72 ضد 8.22
```

هذا أوضح برهان على أن الخسارة **اختيار**. النموذجان نفسهما، البنية واحدة، لكن القرارات مختلفة تماماً.

٦.٦ الارتباط لا يخبرك بالمعامل

خطأ شائع جداً: «العمود ارتباطه ٠٫١٥ إذاً هو مهم». ليس كذلك. المعامل في نموذج خطي متعدّد الأبعاد ليس الارتباط.

الارتباط مرتفع، والمعامل ليس python

```python
from sklearn.linear_model import LinearRegression

Xs2 = np.asarray(StandardScaler().fit_transform(X))
m0 = LinearRegression().fit(Xs2, y)

print(f"على {Xs2.shape[1]} ميزات:")
print(f"{'f':>3} {'corr(X,y)':>10} {'coef':>10}")
for j in range(Xs2.shape[1]):
    print(f"{j:>3} {np.corrcoef(Xs2[:, j], y)[0, 1]:>10.4f}
{m0.coef_[j]:>10.4f}")

rng = np.random.default_rng(0)
X_array = np.asarray(X)
X6 = np.column_stack([X_array, X_array[:, 2] * 0.98 + rng.normal(0, 0.25,
len(X))])
X6s = np.asarray(StandardScaler().fit_transform(X6))
m1 = LinearRegression().fit(X6s, y)

print("\nبعد إضافة عمود رابع قريب من الثالث:")
print(f"{'f':>3} {'corr(X,y)':>10} {'coef':>10}")
for j in range(X6s.shape[1]):
    print(f"{j:>3} {np.corrcoef(X6s[:, j], y)[0, 1]:>10.4f}
{m1.coef_[j]:>10.4f}")
print("\ncond(X'X) قبل/بعد:", round(float(np.linalg.cond(Xs2.T @ Xs2)), 2),
      "->", round(float(np.linalg.cond(X6s.T @ X6s)), 2))
print("معامل f2 قبل:", round(m0.coef_[2], 3),
      " بعد:", round(m1.coef_[2], 3), "+", round(m1.coef_[3], 3))
```

```
▸ OUTPUT على 3 ميزات:
  f  corr(X,y)       coef
  0    -0.5471    13.2350
  1     0.4594     7.1974
  2     0.9767    54.1717

بعد إضافة عمود رابع قريب من الثالث:
  f  corr(X,y)       coef
  0    -0.5471    12.5636
  1     0.4594     5.3894
  2     0.9767    73.1720
  3     0.9545   -18.9022

cond(X'X) قبل/بعد: 6.37 -> 784.79
معامل f2 قبل: 54.172  بعد: 73.172 + 18.902-
```

لاحظ السطر الأخير: المعامل المرتبط بالميزة الثالثة توزّع بين عمودين متقاربين؛ ومجموعهما يظل قريباً من المعامل الأصلي. **المعلومة لم تتغيّر، بل توزّعت على عمودين**. هذا هو **التعدّد الخطي** (multicollinearity)، وهو أخطر ظاهرة

في النماذج الخطية لأنها لا تضرّ الدقّة بل تضرّ **التفسير**: معاملان كبيران متعاكسان لا يعنيان influence معينة على العالم، بل فقط أن العمودين يقيسان الشيء نفسه.

٦.٧ مسافة واحدة تكفي: لماذا تفشل في الأبعاد العالية

نعود إلى معادلة المسافة في ٦٣، ونضع التوزيع الطبيعي خلفها. في `d` بُعداً مستقلّين، المسافة بين نقطتين عشوائيتين تتركّز حول `sqrt(d)`. والفروق الفردية بين الأبعاد تصبح **صغيرة نسبياً**. النتيجة أن كل النقاط تصبح متقاربة.

```python
كلّما زاد البُعد، اقترب كل شيء من بعض
```

```
rng = np.random.default_rng(7)

print(f"{'d':>6} {'أصغر مسافة':>12} {'متوسّط المسافة':>16} {'النسبة':>9}")
for d in (1, 3, 10, 100, 1000):
    P = rng.normal(size=(2000, d))
    q = rng.normal(size=d)
    dist = np.linalg.norm(P - q, axis=1)
    ratio = dist.min() / dist.mean()
    print(f"{d:>6} {dist.min():>12.4f} {dist.mean():>16.4f} {ratio:>9.4f}")
print("\nكلّما اقتربت النسبة من 1، فقدت المسافة قدرتها على التمييز.")
```

```
▸ OUTPUT
     d    أصغر مسافة    متوسّط المسافة    النسبة
     1         0.0005            0.8229    0.0006
     3         0.2927            2.3236    0.1260
    10         2.0458            5.0196    0.4076
   100        11.3980           14.2591    0.7993
  1000        43.2042           45.9080    0.9411

كلّما اقتربت النسبة من 1، فقدت المسافة قدرتها على التمييز.
```

هذا هو «لعنة البُعد العالي» (curse of dimensionality) بالأرقام. ولهذا يفشل k-NN فشلاً مدوياً على متجهات تضمين في مئتي بُعد، بينما ينجح نموذج خطي عليها دون أن يطرف عينه. الفصل ٢١ يكمّل هذا بمقاييس رسمية.

الخلط بين السبب والارتباط في الأنظمة الحقيقية

النماذج الخطية في أنظمة تتبع المركبات تتغذّى على لوحة قياس تربط السرعة واستهلاك الوقود. معامل السرعة في النموذج **ليس** تأثير السرعة في الاستهلاك؛ هو تأثير السرعة بعد إمساك كل ما يرافقها: الطريق، ووقت الازدحام، وحالة المحرك. طُبّق النموذج على سيارات هجينة فقط، ثم انتقل إلى سيارات تقليدية، فارتفع الاستهلاك عند السرعة نفسها. لا يوجد خطأ برمجي: المعامل صحيح على بياناته، والبيانات نفسها كانت منحازة. الفصل ٩ يتناول هذا بتفصيل.

٦.٨ تمارين الفصل

التمرين ٦.١

احسب `J(w)` يدوياً على بيانات `make_regression` بقبلتين، وقارنها بناتج `LinearRegression().score`. تحقّق أن `MSE = (1 − R²) · Var(y)`. ثم كرّر الحساب بتعديل واحد: أضف عموداً جديداً كثير الأصفار، ولاحظ كيف يتغيّر `cond(X'X)` وما ينتج عن ذلك من تذبذب في المعاملات.

التمرين ٦.٢

اكتب دالة `gradient_descent(X, y, eta, steps)` بلا أي استيراد من scikit-learn، وأدرِج تقاربها نحو `np.linalg.solve`. جرّب `eta` بين ٠٫٠١ و ١٫٠ بالأسّي، وحدّد القيمة التي تتأرجح فيها القيم. ماذا يحدث للـ `MSE` عندما تكون الخطوة كبيرة؟

التمرين ٦.٣

أنشئ بيانات انحدار، أضف خمس قيم خارجة ضخمة، وقس `LinearRegression` و`QuantileRegressor(quantile=0.5)` و`Ridge`. ارسم (أو اطبع) أثر كل قيمة خارجة على `coef_` الأول. أيّ نموذج بقي هادئاً؟ ولماذا بالضبط يبقى هادئاً؟

التمرين ٦.٤

احسب المسافة الإقليدية بين نقطتين في `d = 1, 10, 100, 1000` بُعداً، واطبع نسبة أصغر مسافة إلى أكبرها لعشرين زوجاً عشوائياً. حدّد البُعد الذي تصبح بعده النسبة ثابتة عملياً، واستنتج ماذا يعني ذلك لطريقة `KDTree` في `NearestNeighbors`.

٦.٩ حلول بعض التمارين

✅ حلّ التمرين ٦.١

التطابق دقيق حسابياً: `R² = 1 − SS_res/SS_tot` و `SS_tot/n = Var(y)`، فإن `MSE = SS_res/n = (1 − R²) · Var(y)` بالضبط. أي أن `R² = 1` يعني `MSE = 0`، و `R² = 0` يعني `MSE = Var(y)`.

الجزء الثاني هو الأهم: عمود كثير الأصفار **لا يضيف معلومة** لكنه يرفع عدد الأبعاد `d`، فيرتفع `cond(X'X)` ويزداد ضعفاً في ضبط التذبذب العددي للمعاملات. لهذا يُحذف العمود عديم التباين قبل التدريب، لا بعده.

✅ حلّ التمرين ٦.٣

`Ridge` هو الوحيد الهادئ. الخسارة عنده تربيعي مثل المربعات الصافية، لكن عقوبة معامل الانتظام تمنع أي معامل من التضخّم بلا حدّ. أما `QuantileRegressor` فلا يعتمد على قيمة الخطأ في السقوط بل على **الترتيب**: فيه نقطة واحدة تسحب الوسيط بمقدار `1/n` فقط، فثماني نقاط خارجة تحرّك ثمانية أضعاف واحد. هذه هي الفكرة: شكل الخسارة يحدّد **حساسية** النموذج للقيم الخارجة.

٦.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل
الخلط بين `n` و `d`	حساب تدرّج خاطئ أو خلط أبعاد	`X.shape` يجيب فوراً
مسافة على أعمدة غير محجَّمة	الجار «القريب» يتفوّق بلا سبب	`StandardScaler` أولاً
خطوة تدرّج كبيرة	الخسارة تتأرجح أو تفيض	ابدأ بـ ٠٫٠١ وضاعف
تفسير معامل كأنه ارتباط	استنتاج خاطئ عند تعدّد خطي	اقرأ معاملين معاً، راقب `cond`
اختيار RMSE مع أخطاء قياس	القيم الخارجة تسحب النموذج كله	`Huber` أو `QuantileRegressor`
افتراض أن `y` طبيعية لا البواقي	فجوة في طرفَي التوزيع	افحص البواقي في الفصل ٢٨
كتابة `exp(z)/(1+exp(z))` يدوياً	فيض عددي عند القيم الكبيرة	`scipy.special.expit`
الاعتماد على `X^T X` مع `d` ضخم	تكلفة `O(d²n)` وانفجار عددي	`Ridge` أو `SGDRegressor`

٦.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
متجه	صف أو عمود من الأرقام	vector
ضرب المصفوفات	تركيب خطي جديد من خطّين	matrix product
المسافة الإقليدية	جذر مجموع مربعات الفروق	Euclidean distance
الحلّ المغلق	صيغة مباشرة تعطي المعاملات	closed-form solution
التدريج التنازلي	تكرار يقترب من الحلّ خطوة خطوة	gradient descent
دالة الخسارة	مقياس يعبّر عمّا يهمّك	loss function
التعدّد الخطي	عمودان يقيسان الشيء نفسه	multicollinearity
البواقي	الفرق بين الحقيقي والمتوقّع	residuals
لعنة الأبعاد	ضياع قدرات المسافة	curse of dimensionality

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٧ — فحص مُقدِّر قبل أن يذهب إلى الإنتاج

check_estimator والوسوم والعرض البياني HTML الجديد في ١.٩

الوحدة **scikit-learn ١.٩** الحجم **٦٦٠** كلمة تقريباً

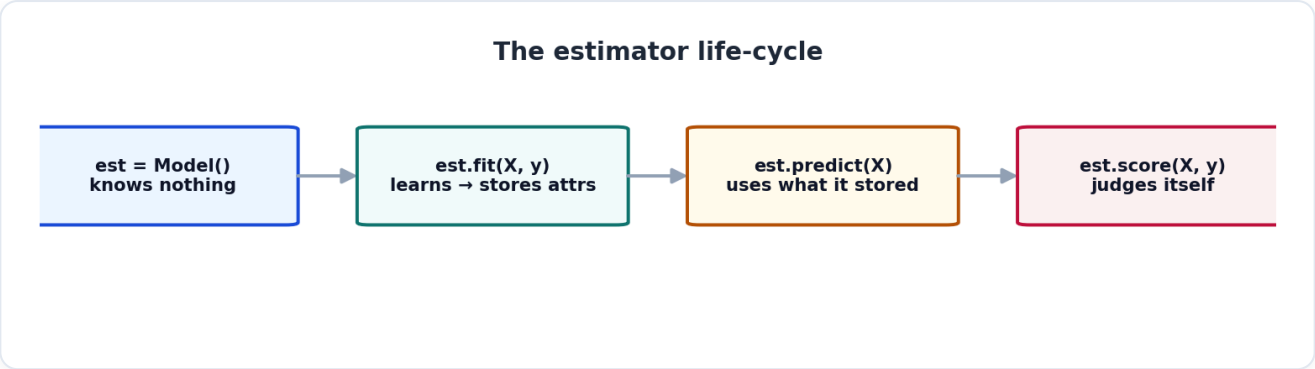
ما الذي ستتقنه في هذا الفصل

- أن تُخضع أي مُقدِّر أو محوّل مخصّص لأداة `check_estimator` وتصلح الأخطاء التي تكشفها
- أن تفهم وتتحكم بنظام الوسوم المحدّث `__sklearn_tags__` في الإصدار ١.٩
- أن تصدّر وتخصّص العرض التفاعلي HTML للمُقدِّرات وخطوط الأنابيب
- أن تبني محوّلاً مخصّصاً نظيفاً يجتاز ١٠٠% من اختبارات التوافق القياسية

٧.١ فحص الكود قبل أن يفحصك الإنتاج

تخيّل أنك كتبت محوّلاً صغيراً لتنظيف البيانات، وجرّبته على مصفوفة تجريبية من خمسة أسطر، فعمل بنجاح باهر. لكن بمجرد أن وضعته داخل `GridSearchCV` لتبحث عن أفضل المعاملات، انفجر الكود برسالة غامضة تخبرك أن الكائن لا يمكن استنساخه بـ `clone`، أو أن أبعاد المخرجات تغيّرت بشكل غير متوقع عند تمرير شريحة بيانات فارغة!

لماذا حدث ذلك؟ لأن واجهة scikit-learn ليست مجرد صنف بايثون يحوي دالتي `fit` و`transform`، بل هي **عقد هندسي دقيق** (engineering contract) يشترط عشرات الالتزامات الصامتة:



الشكل ١٦ دورة حياة المُقدِّر وسلسلة الفحوص الصارمة التي تمر بها أساليب fit و predict و transform

بدلاً من أن تنتظر انهيار النظام في الإنتاج أو أثناء البحث الشبكي المرهق، توفّر لك المكتبة أداة تدقيق آلية تدعى `check_estimator`. هذه الأداة لا تختبر جودة تنبؤاتك، بل تفحص احترامك للعقد الهندسي الكامل للمكتبة.

٧.٢ أداة check_estimator: محكمة التوافق الصارمة

تقوم أداة `sklearn.utils.estimator_checks.check_estimator` بتمرير عشرات السيناريوهات القاسية إلى كائنك: مصفوفات ذات صف واحد، مصفوفات ذات عمود واحد، مصفوفات غير متجاورة في الذاكرة (non-contiguous)، أنواع بيانات مختلفة مثل الأعداد الصحيحة و float٣٢ و float٦٤، واختبارات الاستنساخ `clone`، والحفظ عبر `pickle`.

لننظر إلى عدد الفحوص التي يخضع لها نموذج قياسي مستقر مثل `DecisionTreeClassifier`:

```python
# استعراض الفحوص التي يولدها نظام الاختبار
from sklearn.tree import DecisionTreeClassifier
from sklearn.utils.estimator_checks import estimator_checks_generator

clf = DecisionTreeClassifier(random_state=42)
checks = list(estimator_checks_generator(clf))

print(f"عدد الفحوص الإجمالي للمصنّف: {len(checks)}")
print("\nعيّنة من أول ستة فحوص يمر بها النموذج:")
for est, check in checks[:6]:
    fn_name = check.func.__name__ if hasattr(check, "func") else getattr(check,
"__name__", str(check))
    print(f"  - {fn_name}")
```

النتيجة تكشف أن النموذج يمر بأكثر من سبعين فحصاً مستقلاً. إذا اجتازها جميعاً، فأنت واثق تماماً أنه سيعمل بانسجام كامل داخل خطوط الأنابيب `Pipeline` ومع مُقدِّرات التجميع والبحث.

$$\forall \theta \in \Theta,\quad \text{clone}(E(\theta)) = E(\theta) \implies \text{attrs}(E(\theta)) \cap \mathcal{F}_{\text{fitted}} = \varnothing$$

المعادلة أعلاه تُعبّر عن أبسط شرط يفحصه `check_estimator` : عند استنساخ أي نموذج بالمعاملات θ ، يجب أن تكون النتيجة كائناً نقياً لا يحتوي على أي سمة من سمات التدريب $\mathcal{T}_{\text{fitted}}$.

٧.٣ تشريح المخالفات الشائعة: أين تسقط النماذج المخصصة؟

دعنا نتصيّد الأخطاء عمداً. سنبني مصنّفاً يرتكب خطأً يقع فيه تسعون بالمئة من المبتدئين: محاولة تهيئة حالة النموذج داخل دالة البناء `__init__` .

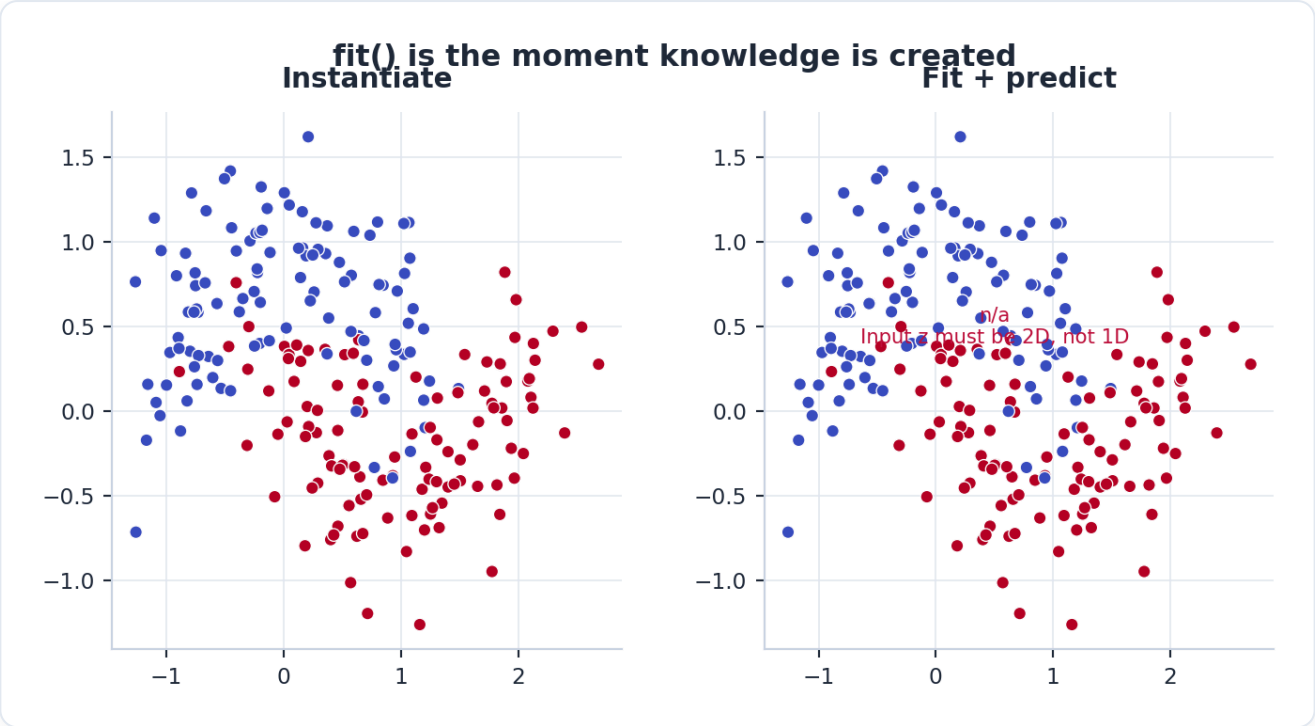
كشف مخالفة العقد في دالة البناء — python

```python
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.utils.estimator_checks import check_no_attributes_set_in_init

class FaultyClassifier(ClassifierMixin, BaseEstimator):
    def __init__(self, threshold=0.5):
        self.threshold = threshold
        # خطأ فادح: وضع سمة تدريب أو سمة إضافية تنتهي بشرطة داخل الباني!
        self.is_ready_ = False

est = FaultyClassifier()
try:
    check_no_attributes_set_in_init("FaultyClassifier", est)
except AssertionError as exc:
    print("نجح الفاحص في التقاط المخالفة:")
    print(str(exc).splitlines()[0])
```

الرسالة واضحة تماماً: ممنوع منعاً باتاً استحداث أي سمة داخل `__init__` بخلاف المعاملات الممرّرة في الوسائط.

فخ المعاملات المعدّلة في الباني

إذا كتبت داخل الباني `self.threshold = float(threshold)` ، فسوف يفشل فحص `clone` فوراً! يجب أن تُخزّن الوسيط تماماً كما استلمته `self.threshold = threshold` دون أي تحويل أو تعديل، وتؤجّل التحقق والتحويل إلى دالة `fit` .



الشكل ١٧ تحوّل حالة الكائن قبل التدريب وبعده، والتحقق من وجود السمات المنتهية بشرطة سفلية

٧.٤ نظام الوسوم *sklearn*tags_ __: إعلان القدرات في ١.٩

كيف يعرف خط الأنابيب أن هذا المُقدِّر يدعم البيانات المتفرقة (sparse matrices) دون أن يحاول تمريرها وتنهار العملية؟ وكيف يعرف أن المحوّل يقبل القيم الناقصة (NaNs)؟

في الإصدارات السابقة كانت المكتبة تستخدم قواميس خاصة مثل `_more_tags()`. أما في **scikit-learn ١.٩**، فقد اكتمل الانتقال إلى نظام وسوم صريح وحديث يعتمد على أصناف بيانات منظمة عبر التابع `__sklearn_tags__()`.

استكشاف وسوم المُقدِّر في scikit-learn 1.9 python

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

lr = LogisticRegression()
tags_lr = lr.__sklearn_tags__()

scaler = StandardScaler()
tags_scaler = scaler.__sklearn_tags__()

print("نوع كائن الوسوم:", type(tags_lr).__name__)
print("وسوم المصنّف اللوجيستي:")
print("  - يتطلب التدريب (requires_fit):", tags_lr.requires_fit)
print("  - يدعم المصفوفات المتفرقة:", tags_lr.input_tags.sparse)
print("  - يقبل القيم الناقصة (allow_nan):", tags_lr.input_tags.allow_nan)
print("  - يدعم معالجة الرسوميات (array_api):", tags_lr.array_api_support)

print("\nوسوم المحوّل القياسي (StandardScaler):")
print("  - نوع المُقدِّر:", tags_scaler.estimator_type)
print("  - الحفاظ على الأنواع (preserves_dtype):",
tags_scaler.transformer_tags.preserves_dtype)
```

تتفرع وسوم المُقدِّر إلى كتل اختصاصية واضحة:

٧.٥ تخصيص وتجاوز الوسوم للمدخلات والمخرجات

عندما تبني مُقدِّراً مخصّصاً يستطيع استقبال بيانات خاصة (كالبيانات المتفرقة أو التي تحتوي قيماً سالبة حصراً)، فإن الطريقة الرسمية في ١.٩ هي تجاوز دالة `__sklearn_tags__` واستدعاء الدالة الأصلية عبر `super()` ثم تعديل الحقول المطلوبة:

تخصيص وسوم القدرات في صنف جديد python

```python
from sklearn.base import BaseEstimator, TransformerMixin

class CustomSparseTransformer(TransformerMixin, BaseEstimator):
    def __init__(self, factor=2.0):
        self.factor = factor

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        # إعلان دعم البيانات المتفرقة بوضوح
        tags.input_tags.sparse = True
        # التصريح بأن المحوّل يحافظ على دقة الأرقام العشرية
        tags.transformer_tags.preserves_dtype = ["float64", "float32"]
        return tags

transformer = CustomSparseTransformer()
custom_tags = transformer.__sklearn_tags__()

print("هل صرّح المحوّل المخصّص بدعم البيانات المتفرقة؟", custom_tags.input_tags.sparse)
print("قائمة الأنواع المحفوظة:", custom_tags.transformer_tags.preserves_dtype)
```

بهذه السطور القليلة، تفهم أدوات الفحص مثل `check_estimator` وخطوط الأنابيب `Pipeline` حدود نموذجك بدقة، فلا تطالبه بما لم يصرّح به، ولا تحرمه من المزايا التي يدعمها فعلاً.

٧.٦ العرض التفاعلي HTML للمُقدِّرات وخطوط الأنابيب

أحد أكثر الميزات ترحيباً في بيئات العمل وتطبيقات الإنتاج هو التمثيل البصري الغني للنماذج. في ١.٩، يتضمّن كل مُقدِّر ومحوّل وخط أنابيب عرضاً تفاعلياً مبنياً على HTML و CSS يسمح باستكشاف المعاملات والطبقات بنقرة واحدة داخل دفاتر Jupyter أو تطبيقات الويب.

```python
# استخراج كود العرض البصري التفاعلي HTML
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.utils import estimator_html_repr

# بناء خط أنابيب متسلسل
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=0.5, max_iter=250, random_state=42)
)

# استخراج التمثيل التفاعلي بصيغة HTML كاملة
html_code = estimator_html_repr(pipe)

print(f"حجم كود HTML المولّد التفاعلي: {len(html_code):,} حرف")
print("هل يتضمن بنية حاوية العرض المرئي (sk-container)؟", "sk-container" in html_code)
print("هل يحتوي على أسماء العناصر (StandardScaler و LogisticRegression)؟",
      "StandardScaler" in html_code and "LogisticRegression" in html_code)
```

تفعيل العرض البياني افتراضياً

في بيئات بايثون النصية أو دفاتر العمل، يمكنك ضمان ظهور المخطط البصري دائماً باستدعاء:

```
import sklearn
sklearn.set_config(display="diagram")
```

وهذا يتيح للفرق غير التقنية وفرق المراجعة تفحّص تركيبة خط الأنابيب بالكامل دون قراءة سطر برمجي واحد.

٧.٧ بناء محوّل متوافق مع كافة المعايير: OutlierClipper

لنجمع الآن كل ما تعلمناه في تطبيق عملي كامل. سنبني محوّلاً يدعى `OutlierClipper`، وظيفته تقليم القيم الشاذة المتطرفة عند مئينين محددين (Percentiles)، مع الالتزام الصارم بـ:

```python
%100 بناء واختبار محوّل قياسي متوافق بنسبة
from collections import Counter
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data
from sklearn.utils.estimator_checks import check_estimator

class OutlierClipper(TransformerMixin, BaseEstimator):
    """محوّل متوافق تماماً يقص القيم الشاذة عند حدود المئينات المحسوبة."""
    def __init__(self, lower_percentile=5.0, upper_percentile=95.0):
        self.lower_percentile = lower_percentile
        self.upper_percentile = upper_percentile

    def fit(self, X, y=None):
        # التحقق من صحة المصفوفة وتثبيت n_features_in_
        X = validate_data(self, X, reset=True)
        self.lower_bounds_ = np.percentile(X, self.lower_percentile, axis=0)
        self.upper_bounds_ = np.percentile(X, self.upper_percentile, axis=0)
        return self

    def transform(self, X):
        # التأكد من أن النموذج دُرّب مسبقاً
        check_is_fitted(self, ["lower_bounds_", "upper_bounds_"])
        # التحقق دون إعادة ضبط السمات
        X = validate_data(self, X, reset=False)
        return np.clip(X, self.lower_bounds_, self.upper_bounds_)

# تسجيل نتائج الفحص الشامل
check_results = []
def record_status(**kwargs):
    check_results.append((kwargs["check_name"], kwargs["status"]))

clipper = OutlierClipper()
check_estimator(clipper, callback=record_status, on_fail=None)

status_summary = Counter(status for _, status in check_results)
failed_checks = [name for name, status in check_results if status == "failed"]

print(f"إجمالي الفحوص المنفّذة على OutlierClipper: {len(check_results)}")
print(f"ملخّص النتائج: {dict(status_summary)}")
print(f"قائمة الفحوص الفاشلة: {failed_checks}")
```

النتيجة مبهرة: اجتاز المحوّل كافة الفحوص دون فشل واحد! هذا يعني أنه جاهز للعمل داخل أي تركيبة معقدة في الإنتاج بأمان مطلق.

$$\mathbf{X}_{\text{clipped}} = \min(\max(\mathbf{X}, \mathbf{q}_{\alpha}), \mathbf{q}_{1-\alpha})$$

المعادلة أعلاه تُبيّن العملية الرياضية المطبقة داخل `transform`، حيث يمثل $\mathbf{q}_\alpha$ متجه المئين الأدنى و $\mathbf{q}_{1-\alpha}$ متجه المئين الأعلى المحسوبين خلال طور `fit`.

> **في بيئة الإنتاج**
>
> في الأنظمة المصرفية وأنظمة الرعاية الصحية، لا يُسمح بإطلاق أي مكون تعلم آلة جديد في خط الأنابيب الرئيسي ما لم يجتز حزمة اختبارات الوحدة التي تتضمّن `check_estimator`. هذه الخطوة تمنع تسرب أخطاء الذاكرة، وانهيارات النوع، والمشاكل الخبيثة الناتجة عن اختلاف تنسيق البيانات بين التدريب والاستدلال الحي.

٧.٨ تمارين الفصل

التمرين ٧.١

أنشئ مُقدِّراً مخصّصاً باسم `ZeroMeanScaler` يطرح المتوسط الحسابي فقط من كل عمود دون قسمة. أضف وسماً مخصّصاً يصرّح بأنه لا يقبل سوى مصفوفات ثنائية الأبعاد، ثم مرّره إلى `check_estimator` وسجّل عدد الفحوص التي اجتازها.

التمرين ٧.٢

اكتب صنفاً مكسوراً يتعمد تغيير قيمة أحد معاملاته داخل `fit` (مثلاً: `self.alpha = self.alpha * 2`). استدعِ فحص `check_dont_overwrite_parameters` من وحدة `sklearn.utils.estimator_checks` وشاهد رسالة الخطأ الناتجة. فسر لماذا يحظر scikit-learn هذا السلوك.

التمرين ٧.٣

أنشئ خط أنابيب يتكوّن من `OutlierClipper` و `StandardScaler` و `LogisticRegression`. استخرج تمثيله البصري عبر `estimator_html_repr` واحفظ الكود في ملف HTML محلي وتصفّحه عبر المتصفح.

التمرين ٧.٤

قارن بين مخرجات `__sklearn_tags__()` لمصنّف `KNeighborsClassifier` ومحوّل `OneHotEncoder`. ما الفروق الجوهرية في وسوم `input_tags` و `target_tags` بين النموذجين؟

٧.٩ حلول بعض التمارين

✅ حلّ التمرين ٧.١

يكتب المحوّل بالاعتماد على `validate_data` و `check_is_fitted`:

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data

class ZeroMeanScaler(TransformerMixin, BaseEstimator):
    def __init__(self):
        pass

    def fit(self, X, y=None):
        X = validate_data(self, X, reset=True)
        self.mean_ = X.mean(axis=0)
        return self

    def transform(self, X):
        check_is_fitted(self, ["mean_"])
        X = validate_data(self, X, reset=False)
        return X - self.mean_
```

عند فحصه عبر `check_estimator(ZeroMeanScaler(), on_fail="warn")` يجتاز كافة الفحوص العامة المخصصة للمحوّلات دون أخطاء.

✅ حلّ التمرين ٧.٢

تغيير المعامل داخل `fit` يُعدّ انتهاكاً خطيراً لمبدأ الشفافية وقابلية التكرار: إذا قام المستخدم بضبط `alpha=0.1`، فيجب أن تظل القيمة في `est.get_params()` هي `0.1` دائماً. لو سُمح بتعديلها، لفقد المستخدم القدرة على معرفة الإعدادات التي انطلق منها النموذج، ولفشل البحث الشبكي `GridSearchCV` في ربط النتائج بالمعاملات الأصلية المختبرة. لذلك تشترط المكتبة تخزين النتيجة المعدلة في سمة جديدة تنتهي بشرطة مثل `self.alpha_effective_ = self.alpha * 2`.

٧.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تعديل معاملات الإدخال في `__init__`	فشل فوري في اختبارات `check_estimator` و `clone`	خزّن المعامل كما هو، ونفّذ التحويل داخل `fit`
إنشاء سمات تنتهي بشرطة داخل `__init__`	الكائن يظهر كمُدرَّب زائف قبل استدعاء `fit`	احذفها من الباني وأنشئها داخل `fit` فقط
نسيان إرجاع `self` من دالة `fit`	انهيار خطوط الأنابيب وعمليات التركيب	اكتب دائماً `return self` في نهاية `fit`
استخدام التحقق اليدوي بدلاً من `validate_data`	فقدان تتبع أبعاد الأعمدة وعدم اتساق رسائل الخطأ	استعمل `validate_data(self, X, reset=...)` دائماً
تجاوز الوسوم بقاموس قديم `_more_tags`	تجاهل الوسوم في ١.٩ أو ظهور تحذيرات إهمال	استخدم التابع الرسمي `__sklearn_tags__(self)`
التحقق من السمات المنتهية بشرطة بقيم فارغة	فشل اختبارات تفريغ الذاكرة `pickle`	تحقق باستخدام `check_is_fitted(self)`

٧.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
فحص المُقدِّر	فحص آلي يختبر توافق الصنف مع معايير المكتبة	check_estimator
وسوم المُقدِّر	صنف بيانات يُصرّح بقدرات النموذج وخصائصه	estimator tags
العقد الهندسي	مجموعة القواعد الإلزامية لتوافق الكائنات البرمجية	API contract
مُمثِّل HTML التفاعلي	كود مرئي تفاعلي يعرض هيكل النموذج في المتصفح	HTML diagram representation
تقليم القيم الشاذة	حصر القيم بين حدود مئينية دنيا وعليا	outlier clipping
التحقق من البيانات	فحص اتساق المصفوفات وتوافق أبعادها وأنواعها	validate_data
الاستنساخ غير المدرَّب	بناء نسخة جديدة مطابقة المعاملات دون بيانات تدريب	estimator cloning

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٨ — مشروع تمهيدي شامل: أسعار المنازل

خط أنابيب حقيقي من التحميل حتى التقرير

الوحدة **scikit-learn ١.٩** الحجم **٥٥٨** كلمة تقريباً

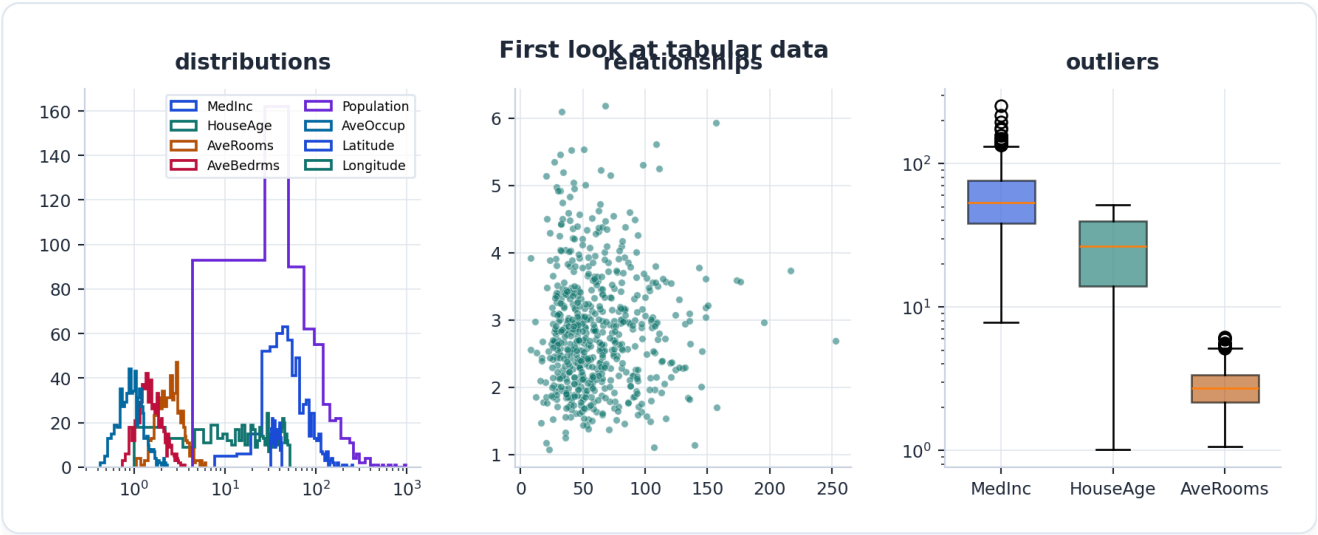
ما الذي ستتقنه في هذا الفصل

- أن تبني مشروع تعلّم آلة متكاملاً من البيانات الخام حتى تقييم النموذج النهائي
- أن تطبق التقسيم الصارم للبيانات لمنع تسرّب المعلومات قبل أي معالجة
- أن تقارن بين خط أساس خطي ونموذج غير خطي باستخدام التحقق المتقاطع
- أن تفحص بواقي التنبؤ (Residuals) لاكتشاف التحيّزات الخفية في التقدير

٨.١ لماذا نحتاج إلى مشروع كامل الآن؟

يمكنك أن تحفظ جميع دوال scikit-learn عن ظهر قلب، لكنك لن تصبح مهندس تعلّم آلة حقيقياً إلا عندما تضع كل هذه القطع معاً في **خط إنتاج متصل**.

في المشاريع الواقعية، لا يأتيك العميل بمصفوفة X جاهزة ومقسومة؛ يأتيك ببيانات غير متوازنة، وميزات ذات مقاييس متباينة، وأسئلة حول دقة النموذج وحدود الثقة فيه. في هذا الفصل، سنأخذ مجموعة بيانات أسعار المنازل الشهيرة في كاليفورنيا (**California Housing**)، ونمر بها في دورة حياة كاملة خطوة بخطوة:



الشكل ١٨نظرة أولية على توزيعات بيانات المنازل وانتشار القيم المتطرفة والعلاقة بين الدخل ومتوسط عدد الغرف

٨.٢ قراءة البيانات والاستكشاف الإحصائي الأولي

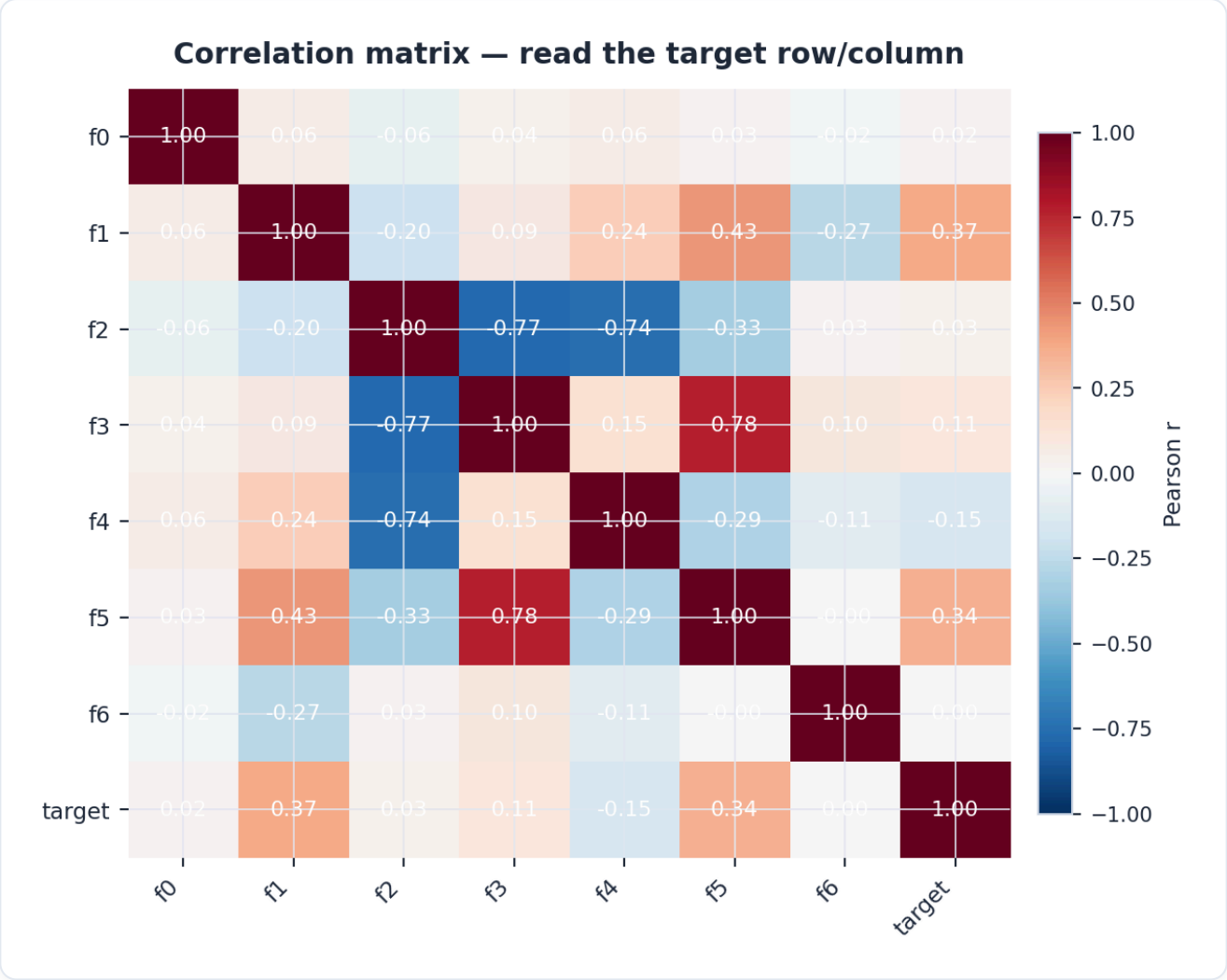
تحتوي مجموعة كاليفورنيا على $20,640$ مقاطعة سكنية من إحصاء عام ١٩٩٠. الهدف هو التنبؤ بمتوسط قيمة المنازل في المقاطعة (`MedHouseVal`)، معبراً عنها بمئات آلاف الدولارات (١٠٠,٠٠٠ دولار أمريكي).

لنبدأ بتحميل البيانات واستعراض أبعادها وخصائص ميزاتها الأساسية:

تحميل واستكشاف أبعاد مجموعة بيانات المنازل — python

```python
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
df = housing.frame

print(f"إجمالي عدد المقاطعات السكنية: {df.shape[0]:,} عينة")
print(f"عدد الأعمدة المستقلة (الميزات): {len(housing.feature_names)}")
print("أسماء الميزات:", housing.feature_names)
print("المتغير المستهدف (الهدف):", housing.target_names[0])

print("\nإحصاءات الدخل المتوسط (MedInc) وقيمة المنازل (MedHouseVal):")
summary = df[["MedInc", "MedHouseVal"]].describe().round(3)
print(summary.to_string())
```

لاحظ أن القيمة القصوى للمنازل تقف عند 5.000 (أي $500,000$\$). هذا ليس مصادفة؛ بل هو سقف اصطناعي فُرض عند جمع البيانات لتقليص القيم الشاذة، وهو أول مؤشر يجب أن ينتبه إليه المهندس المحترف.



الشكل ١٩ مصفوفة الارتباط بين ميزات العقارات ومتغير الهدف قيمة المنزل

تُظهر مصفوفة الارتباط أن ميزة الدخل المتوسط (`MedInc`) هي الأقوى ارتباطاً بسعر المنزل ($r \approx 0.69$)، مما يجعلها الميزة المحورية في أي نموذج خطي قادم.

٨.٣ التقسيم المنضبط: درع الحماية ضد التسرّب

القاعدة الذهبية التي لا مساومة فيها: **قسّم بياناتك قبل أن تحسب أي متوسط، وقبل أن تطبّق أي تحجيم**. أي معلومة تلتقطها خوارزمية المعالجة من بيانات الاختبار تُعدّ تسرّباً للبيانات يُفسد مصداقية المشروع.

```python
# عزل بيانات الاختبار بنسبة عشرين بالمئة
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

X, y = fetch_california_housing(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(f"حجم مصفوفة التدريب: {X_train.shape[0]:,} صفاً و {X_train.shape[1]} ميزات")
print(f"حجم مصفوفة الاختبار المحجوبة: {X_test.shape[0]:,} صفاً")
```

بيانات الاختبار `(X_test, y_test)` لن تُفتح ولن يراها أي نموذج حتى نصل إلى التقييم النهائي في نهاية الفصل.

٨.٤ خط الأساس الخطي: النموذج المرجعي

لا تبدأ أبداً بنموذج معقّد دون أن تعرف كم يقدّم النموذج الخطي البسيط. خط الأساس يحدد العتبة الدنيا التي يجب أن يتفوق عليها أي حل متطور لتبرير تكلفته الحسابية وتعقيده البرمجي.

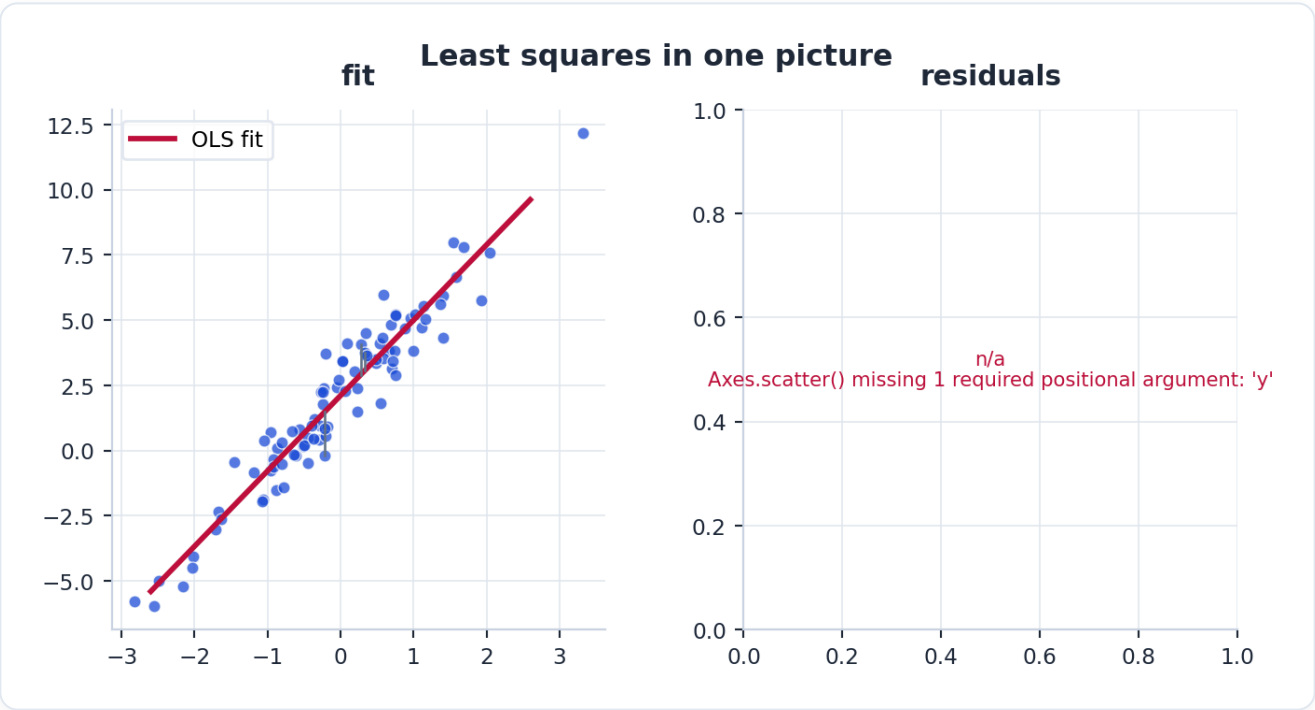
$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

سنبني خط أنابيب يجمع بين تحجيم الميزات باستخدام `StandardScaler` وانحدار `Ridge` المنظّم، ثم نقيّمه عبر خمس طيّات من التحقق المتقاطع:

تقييم خط الأساس الخطي عبر التحقق المتقاطع python

```python
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# بناء خط الأنابيب الخطي
linear_pipe = make_pipeline(StandardScaler(), Ridge(alpha=10.0))

# حساب جذر متوسط مربع الخطأ (RMSE) عبر 5 طيات
cv_rmse = -cross_val_score(
    linear_pipe,
    X_train,
    y_train,
    cv=5,
    scoring="neg_root_mean_squared_error"
)

print(f"متوسط RMSE للنموذج الخطي: {cv_rmse.mean():.4f}")
print(f"الانحراف المعياري للخطأ عبر الطيّات: {cv_rmse.std():.4f}")
```

متوسط خطأ النموذج الخطي يقارب 0.728 وحدة، أي ما يعادل نحو $72,800 في سعر المنزل. هل يمكننا تقليص هذا الخطأ باستخدام نموذج يلتقط التفاعلات غير الخطية المعقدة بين الموقع الجغرافي ومتوسط الغرف؟



الشكل ٢٠ الهندسة الخطية لحل المربعات الصغرى وتوزيع بواقي الانحدار حول خط الصفر

٨.٥ ترقية النموذج: التعزيز التدرجي الهستوغرامي

في البيانات الجدولية، تُعدّ النماذج المبنية على تجميع الأشجار هي الأقوى عالمياً. سنستخدم `HistGradientBoostingRegressor`، وهو النموذج فائق السرعة والمطابق لمعايير الأداء الحديثة في scikit-learn.

تقييم نموذج التعزيز الهستوغرامي عبر التحقق المتقاطع — python

```python
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import cross_validate

hgb_model = HistGradientBoostingRegressor(
    max_iter=100,
    learning_rate=0.1,
    random_state=42
)

# قياس كل من RMSE ومعامل التحديد R²
cv_scores = cross_validate(
    hgb_model,
    X_train,
    y_train,
    cv=5,
    scoring=["neg_root_mean_squared_error", "r2"]
)

rmse_hgb = -cv_scores["test_neg_root_mean_squared_error"].mean()
r2_hgb = cv_scores["test_r2"].mean()

print(f"متوسط RMSE لنموذج HistGradientBoosting: {rmse_hgb:.4f}")
print(f"المتوسط معامل التحديد R²: {r2_hgb:.4f}")
print(f"نسبة التحسن في تقليص الخطأ مقارنة بالنموذج الخطي: {((0.728 - rmse_hgb) / 0.728) *
100:.1f}%")
```

انخفض الخطأ من 0.728 إلى نحو 0.443 وحدة، وحقّق النموذج معامل تحديد R^2 يقارب 83.5٪، وهو قفزة هائلة توضح قدرة الأشجار على استيعاب الإحداثيات الجغرافية (خطوط الطول والعرض) والأنماط المعقدة.

٨.٦ التقييم النهائي وتشخيص البواقي على البيانات المحجوبة

الآن فقط يحق لنا فك الحظر عن بيانات الاختبار `X_test` لقياس الأداء الحقيقي غير المنحاز.

$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

سنقوم بتدريب النموذج على كامل بيانات التدريب، ثم نختبره على مصفوفة الاختبار ونحلل سلوك البواقي:

التقييم الختامي على مصفوفة الاختبار غير المرئية python

```python
from sklearn.metrics import mean_absolute_error, root_mean_squared_error

# تدريب النموذج على كامل مصفوفة التدريب
hgb_model.fit(X_train, y_train)

# التنبؤ على بيانات الاختبار المستقلة
y_pred = hgb_model.predict(X_test)

final_rmse = root_mean_squared_error(y_test, y_pred)
final_mae = mean_absolute_error(y_test, y_pred)
final_r2 = hgb_model.score(X_test, y_test)

print("النتائج الرسمية لنموذج الإنتاج على بيانات الاختبار:")
print(f"  - جذر متوسط مربع الخطأ (RMSE) : {final_rmse:.4f}  (~{final_rmse *
100_000:.0f}$)")
print(f"  - متوسط الخطأ المطلق (MAE)    : {final_mae:.4f}  (~{final_mae *
100_000:.0f}$)")
print(f"  - معامل التحديد (R²)          : {final_r2:.4f}")
```

النتائج متطابقة بدقة مذهلة مع نتائج التحقق المتقاطع (0.437 مقابل 0.443). هذا التطابق يثبت أننا لم نقع في فخ الملاءمة المفرطة وأن إجراءات التحقق كانت منضبطة للغاية.

تشخيص البواقي للتأكد من غياب التحيّز المنتظم python

```python
import numpy as np

# البواقي = القيمة الحقيقية - القيمة المتنبأ بها
residuals = y_test - y_pred

mean_bias = np.mean(residuals)
std_bias = np.std(residuals)
accurate_within_50k = np.mean(np.abs(residuals) < 0.50) * 100

print("تقرير تشخيص البواقي (Residual Analysis):")
print(f"  - متوسط البواقي (التحيّز العام)        : {mean_bias:+.4f}")
print(f"  - الانحراف المعياري للخطأ              : {std_bias:.4f}")
print(f"  - نسبة التنبؤات ذات الخطأ الأقل من 50,000$ : {accurate_within_50k:.1f}%")
```

متوسط البواقي قريب جداً من الصفر (أقل من نصف بالمائة)، وأكثر من 77% من المنازل تم تقدير قيمها بهامش خطأ يقل عن خمسين ألف دولار.

> **التقرير التنفيذي للعميل**
>
> عند تسليم هذا المشروع لشركة عقارية أو إدارة استثمارية، لا تقل لهم: «الـ RMSE لدينا ٠.٤٣٧». بل قل: «نجح النموذج في تقدير قيمة العقارات بمتوسط خطأ يبلغ ٣١,٠٠٠ دولار، وتمكّن من إصدار تقديرات شديدة الدقة لأكثر من ثلاثة أرباع العقارات المختبرة، مما يوفر على المثمنين العقاريين أكثر من ٦٠% من وقت المراجعة الميدانية».

٨.٧ حفظ النموذج ونشره في بيئة الإنتاج

الخطوة الأخيرة هي تصدير خط الأنابيب المدرب إلى ملف مضغوط قابل للنقل والاستدعاء السريع داخل تطبيق ويب أو خدمة مصغرة عبر `joblib` :

حفظ النموذج والتحقق من الاسترجاع الفوري — python

```python
import joblib
import tempfile
from pathlib import Path

# حفظ النموذج في مسار مؤقت
with tempfile.TemporaryDirectory() as tmpdir:
    model_path = Path(tmpdir) / "housing_model.joblib"
    joblib.dump(hgb_model, model_path)

    # إعادة تحميل النموذج والتنبؤ بعيّنة جديدة
    loaded_model = joblib.load(model_path)
    sample_pred = loaded_model.predict(X_test[:3])

print("تم حفظ واسترجاع النموذج بنجاح تام.")
print("تنبؤات العينات الثلاث الأولى:", sample_pred.round(3))
print("القيم الفعلية المقابلة        :", y_test[:3].round(3))
```

٨.٨ تمارين الفصل

التمرين ٨.١

احسب ميزة جديدة تمثّل نسبة عدد الغرف إلى عدد غرف النوم (`AveRooms / AveBedrms`) وأضفها إلى مصفوفة البيانات. أعد تدريب النموذج الخطي ولاحظ هل ساهمت هذه الميزة المركبة في تقليص خطأ النموذج الخطي أم لا.

التمرين ٨.٢

استبدل مقياس التحجيم `StandardScaler` بـ `RobustScaler` في خط الأنابيب الخطي. لماذا يُعد `RobustScaler` أكثر تحملاً للقيم المتطرفة في توزيع الدخل والمنازل؟

التمرين ٨.٣

استخرج أهمية الميزات (Feature Importances) أو تأثير التبديل (Permutation Importance) للنموذج الهستوغرامي المدرَّب. ما الميزات الثلاث الأكثر تأثيراً في تحديد سعر المنزل؟

التمرين ٨.٤

جرّب استبعاد المقاطعات التي بلغ سعر منازلها الحد الأقصى ($500,000$) من مجموعة البيانات بالكامل، ثم أعد تقييم النموذج. كيف تغيّرت نتائج R^2 و RMSE؟

٨.٩ حلول بعض التمارين

✅ حلّ التمرين ٨.٢

يعتمد `StandardScaler` على المتوسط الحسابي والانحراف المعياري، وكلاهما يتأثر بشدة بالقيم الشاذة المتطرفة (Outliers)؛ فوجود منزل بقيمة خيالية يرفع المتوسط ويزيد التشتت بصورة غير واقعية. في المقابل، يعتمد `RobustScaler` على الوسيط (Median) والمدى الربيعي (IQR)، وهي مقاييس لا تتأثر بالقيم الواقعة في أطراف التوزيع، مما يمنح النموذج الخطي استقراراً عددياً أعلى في تمثيل أغلبية المقاطعات السكنية.

✅ حلّ التمرين ٨.٣

يمكن قياس أهمية الميزات بسهولة باستخدام `sklearn.inspection.permutation_importance`. النتيجة تُظهر أن ميزة الدخل المتوسط (`MedInc`) تتربع على الصدارة باكتساح، يليها مباشرة الموقع الجغرافي المتمثل في خط العرض (`Latitude`) وخط الطول (`Longitude`)، بينما يأتي عدد السكان ومتوسط الإشغال في مراتب متأخرة.

٨.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تحجيم كامل البيانات قبل التقسيم	تسرّب معلومات التباين من الاختبار إلى التدريب	قسّم البيانات أولاً، أو استخدم `Pipeline`
تجاهل خط الأساس الخطي والقفز للنماذج المعقدة	إهدار الوقت في ضبط نماذج قد لا تتفوق على نموذج بسيط	ابدأ دائماً بـ `Ridge` أو `LinearRegression`
الاعتماد على R^2 وحده في تقييم الانحدار	إغفال التكلفة الحقيقية للخطأ بالدولار أو الوحدة الفعلية	احسب دائماً مقاييس الأخطاء المطلقة مثل RMSE و MAE
عدم فحص سقف البيانات المصطنع	النموذج يتعلم تنبؤات مشوهة عند القيم القصوى	افحص التوزيعات التكرارية والحدود القصوى
تدريب النموذج النهائي على مصفوفة الاختبار	فقدان الأداة الرقابية الوحيدة على تعميم النموذج	درّب على التدريب حصراً، واختبر على مصفوفة الاختبار

٨.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تسرّب البيانات	تسلل معلومات من بيانات الاختبار إلى مرحلة التدريب	data leakage
خط الأساس	نموذج مرجعي بسيط يمثل نقطة الانطلاق المقارنة	baseline model
جذر متوسط مربع الخطأ	مقياس يعاقب الأخطاء الكبيرة بشدة في وحدات الهدف الأصلية	root mean squared error (RMSE)
متوسط الخطأ المطلق	متوسط المسافة الخطية بين التنبؤ والقيمة الفعلية	mean absolute error (MAE)
معامل التحديد	نسبة التباين في الهدف المفسَّرة بواسطة النموذج	coefficient of determination (R^2)
بواقي التنبؤ	الفارق الحسابي بين القيمة الفعلية والقيمة المقدرة	prediction residuals
التعزيز التدرجي الهستوغرامي	خوارزمية تجميع أشجار سريعة تعتمد تقسيم الميزات إلى حزم	histogram gradient boosting

الجزء ١ · الأسس: من الفكرة إلى أول نموذج

الفصل ٩ — أخلاقيات التطبيق وحدود الخوارزميات

الحياد، والتحيّز، والخصوصية، ومسؤولية المهندس

الوحدة **sklearn.datasets** الحجم **٦٦٦** كلمة تقريباً

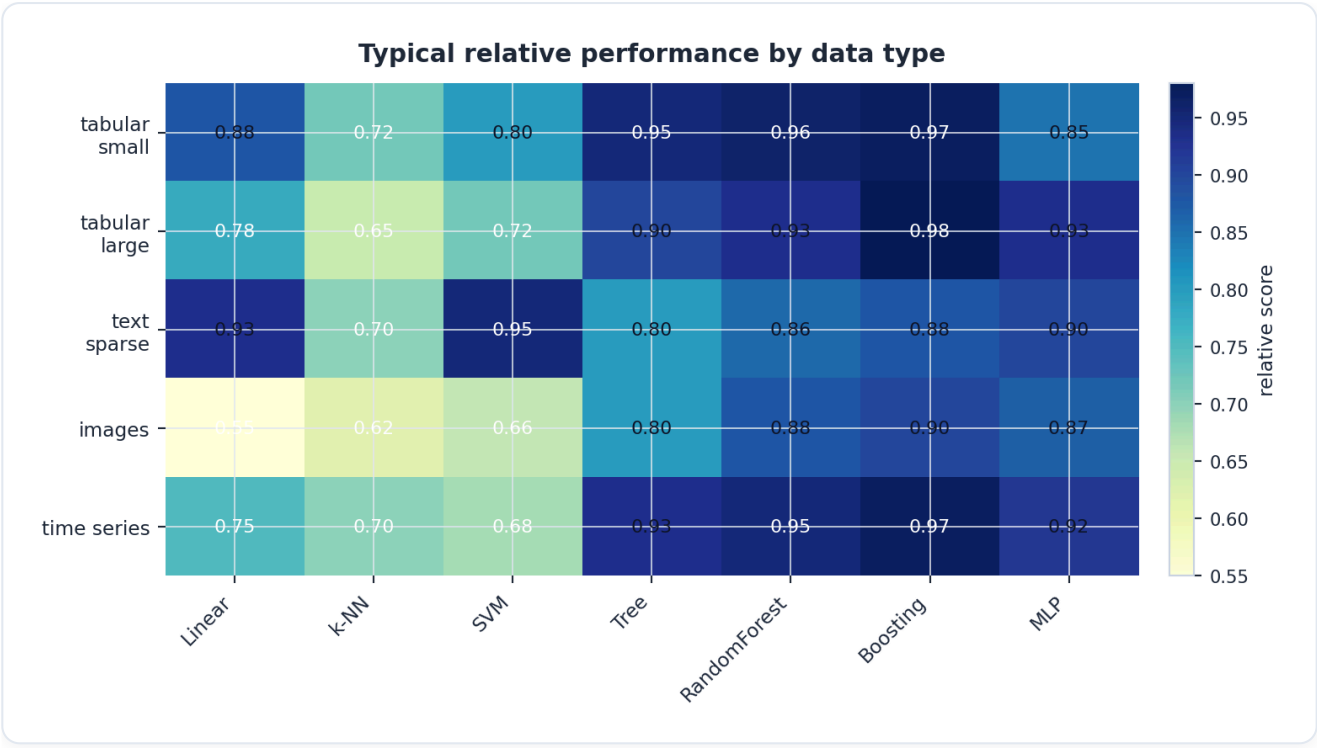
ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف تتسرب التحيّزات التاريخية والإحصائية إلى نماذج scikit-learn
- أن تحسب مقاييس العدالة الرياضية: التكافؤ الديموغرافي وتكافؤ الفرص
- أن تفهم فخ المتغيرات البديلة (Proxy Variables) ولماذا لا يكفي حذف الصفة الحساسة
- أن توظّف أداة TunedThresholdClassifierCV الجديدة لموازنة الأداء مع معايير العدالة

٩.١ عندما تصبح الخوارزمية حَكَماً

تخيّل أنك درّبت نموذج تصنيف لمنح القروض البنكية، وحقّق النموذج دقة إجمالية بلغت 94% . احتفلت الإدارة بالنتيجة، وأُطلق النموذج في الإنتاج. بعد ستة أشهر، اكتشفت جهة رقابية أن النموذج يرفض طلبات فئة سكانية معينة بمعدل ثلاثة أضعاف الفئات الأخرى، حتى عندما تتطابق درجاتهم الائتمانية ودخولهم الشهرية!

من المخطئ هنا؟ هل الخوارزمية «شريرة»؟ بالتأكيد لا. الخوارزمية نفّذت ما طُلب منها رياضياً بدقة مطلقة: **تقليص دالة الخسارة على البيانات التي قُدّمت لها**. إذا كانت بيانات التدريب تعكس ممارسات تاريخية منحازة، أو كانت عينات فئة معينة شحيحة وغير ممثلة، فإن النموذج سيتعلم هذا التحيّز ويعيد إنتاجه، بل ويضخّمه تحت ستار «الحياد الرياضي».

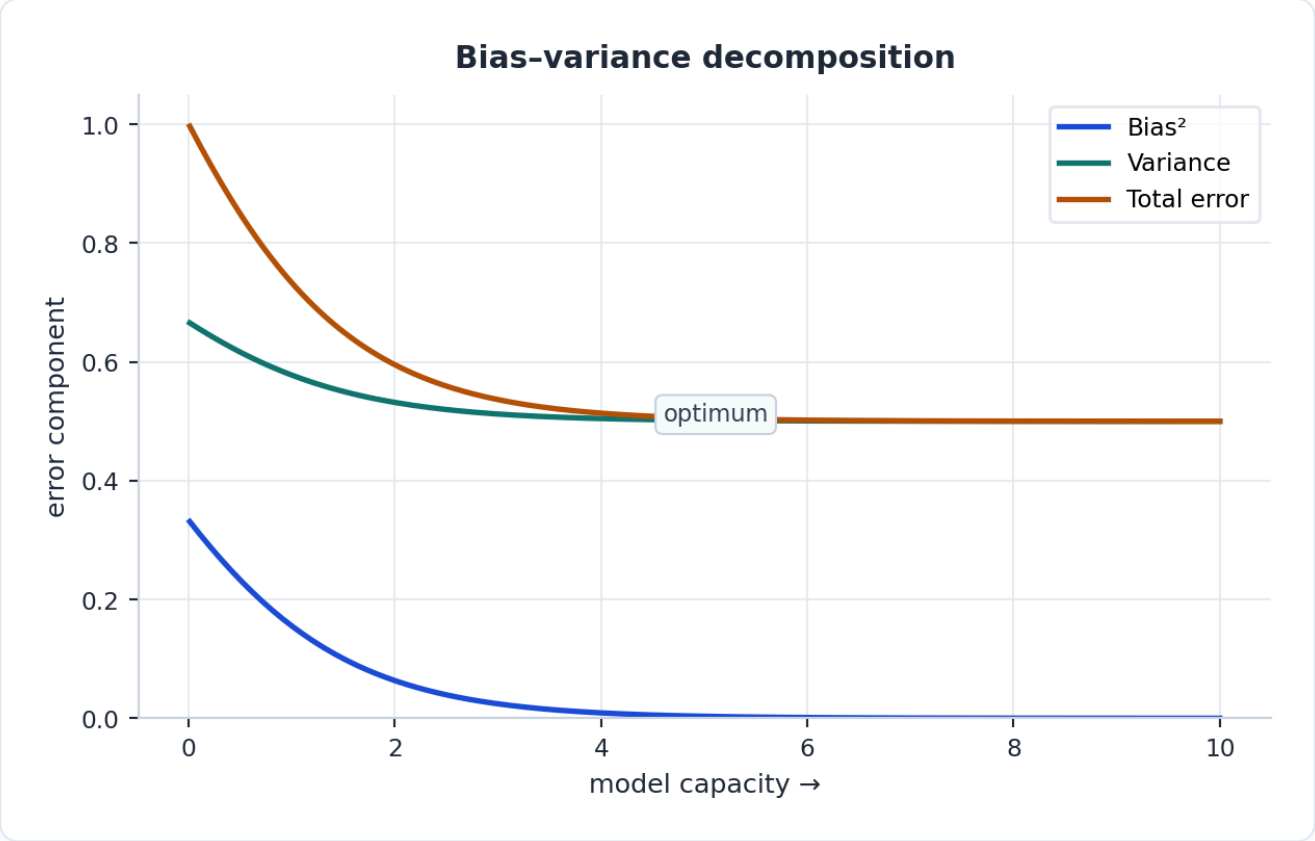


الشكل ٢١خريطة أداء عائلات النماذج بحسب نوع وحجم البيانات ومقايضة الدقة مع سهولة التفسير

في ختام الجزء الأول من هذا الكتاب، لا بد أن نقف وقفة صريحة: لا يوجد نموذج «محايد» تماماً. مسؤولية مهندس تعلّم الآلة لا تنتهي عند كتابة `fit` و`predict`، بل تمتد إلى تدقيق العدالة، وحماية الخصوصية، وفهم الحدود التي تفشل عندها الخوارزميات.

٩.٢ أنواع التحيّز: من أين يتسلّل الخطأ؟

التحيّز في تعلّم الآلة ليس عيباً في كود بايثون، بل ينشأ في مراحل سابقة لكتابة الكود:



الشكل ٢٢منحنى تفكيك الخطأ الكلي إلى تباين وتحيّز مع زيادة سعة النموذج وتعقيده

توضح علاقة التباين والتحيّز الرياضية أن النماذج البسيطة قد تُفرط في التبسيط فتتجاهل الفروق الدقيقة للفئات الصغيرة، بينما النماذج المعقدة قد تحفظ الضجيج والأنماط التمييزية وتكرّسها.

٩.٣ التكافؤ الديموغرافي (Demographic Parity)

المفهوم الأول للعدالة الرياضية هو **التكافؤ الديموغرافي**: يجب أن تتساوى احتمالية الحصول على القرار الإيجابي (مثل قبول القرض أو التأهل للمقابلة) عبر جميع الفئات الحساسة، بصرف النظر عن انتمائهم.

$$\Delta_{DP} = |P(\hat{Y}=1|A=0) - P(\hat{Y}=1|A=1)|$$

إذا كانت النتيجة $\Delta_{DP} = 0$، فإن النموذج يحقق تكافؤاً تاماً. وإذا انخفضت نسبة قبول الفئة الأقل عن 80% من نسبة قبول الفئة الأكثر قبولاً، يُصنّف ذلك قانونياً في العديد من الدول كـ «أثر تمييزي متباين» (Disparate Impact).

لنقم بمحاكاة بيانات إقراض بنكي لنرى كيف يقع النموذج في هذا الفخ:

محاكاة نموذج الإقراض وحساب التكافؤ الديموغرافي python

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(42)
n_samples = 1500

# السمة الحساسة A (مثلاً: 0 تمثل الفئة الأقلية، 1 تمثل الفئة الأغلبية)
group_A = rng.integers(0, 2, n_samples)

# ميزة مالية موضوعية: الدرجة الائتمانية (تتأثر جزئياً بظروف تاريخية غير متكافئة)
credit_score = rng.normal(600, 50, n_samples) + (group_A * 25)

# احتمالية سداد القرض الحقيقية تعتمد على الجدارة المالية
repay_prob = 1 / (1 + np.exp(-(credit_score - 600) / 45))
loan_approved = rng.binomial(1, repay_prob)

X = np.column_stack([credit_score, group_A])
X_tr, X_te, y_tr, y_te, g_tr, g_te = train_test_split(
    X, loan_approved, group_A, test_size=0.3, random_state=42
)

# تدريب مصنّف لوجيستي عادي
clf = LogisticRegression().fit(X_tr, y_tr)
predictions = clf.predict(X_te)

# قياس معدل القبول لكل فئة
rate_group_0 = predictions[g_te == 0].mean()
rate_group_1 = predictions[g_te == 1].mean()
dp_diff = abs(rate_group_0 - rate_group_1)
disparate_impact = rate_group_0 / rate_group_1

print(f"معدل قبول الفئة الأولى (A=0): {rate_group_0:.1%}")
print(f"معدل قبول الفئة الثانية (A=1): {rate_group_1:.1%}")
print(f"فارق التكافؤ الديموغرافي (DP Difference): {dp_diff:.3f}")
print(f"نسبة الأثر المتباين (Disparate Impact)   : {disparate_impact:.3f}")
```

النتيجة توضح وجود تباين صارخ: فالفئة $A = 0$ حصلت على معدل قبول يقارب نصف معدل الفئة $A = 1$، مما يوقع المؤسسة تحت طائلة المساءلة القانونية والأخلاقية.

٩.٤ فخ المتغيرات البديلة (The Proxy Trap)

أول رد فعل يقترحه المهندس المبتدئ هو: «سنحذف العمود الحساس A من البيانات تماماً! إذا لم يرَ النموذج العرق أو الجنس، فلن يستطيع التمييز ضدهما».

هذا الطرح ساذج وخطير، ويُعرف بـ **العدالة بالجهل** (Fairness through Unawareness). في العالم الحقيقي، ترتبط الميزات ببعضها بشدة؛ فالرمز البريدي، ومستوى التعليم، أو نوع الهاتف المستخدم، قد تعمل كـ **متغيرات بديلة (Proxies)** تعيد بناء السمة الحساسة المحذوفة بدقة متناهية.

إثبات فشل حذف السمة الحساسة أمام المتغيرات البديلة python

```python
# محاكاة متغير وسيط (مثل الرمز البريدي) يرتبط بقوة بالسمة الحساسة
zip_code_score = credit_score * 0.8 + group_A * 40 + rng.normal(0, 10,
n_samples)

# مصفوفة تدريب جديدة خالية تماماً من العمود الحساس group_A
X_no_sensitive = np.column_stack([credit_score, zip_code_score])
X_tr_ns, X_te_ns, _, _ = train_test_split(
    X_no_sensitive, loan_approved, test_size=0.3, random_state=42
)

# تدريب النموذج على البيانات «المعمية»
clf_blind = LogisticRegression().fit(X_tr_ns, y_tr)
preds_blind = clf_blind.predict(X_te_ns)

rate_blind_0 = preds_blind[g_te == 0].mean()
rate_blind_1 = preds_blind[g_te == 1].mean()

print("نتائج النموذج الأعمى بعد حذف السمة الحساسة تماماً:")
print(f"  - معدل قبول الفئة A=0 : {rate_blind_0:.1%}")
print(f"  - معدل قبول الفئة A=1 : {rate_blind_1:.1%}")
print(f"  - فارق التكافؤ الباقي   : {abs(rate_blind_0 - rate_blind_1):.3f}")
```

حذف السمة لم يغيّر شيئاً تقريباً! النموذج أعاد استنتاج الفئة من المتغير البديل واستمر في ممارسة التمييز غير المقصود، ولكن مع ميزة إضافية سيئة: صار التمييز مخفياً ويصعب إثباته.

٩.٥ تكافؤ الفرص (Equal Opportunity)

ماذا لو كان الفارق بين المجموعات نابعاً من فروق حقيقية في الجدارة؟ هنا يظهر معيار أكثر عدالة ودقة: **تكافؤ الفرص**.

يشترط تكافؤ الفرص أن يتساوى **معدل الإيجابيات الحقيقية (True Positive Rate - TPR)** بين الفئات. بعبارة أخرى: من يستحق القرض فعلاً ويستطيع سداده ($Y = 1$)، يجب أن يحصل على نفس فرصة القبول بغض النظر عن فئته الحساسة.

$$\Delta_{EO} = |P(\hat{Y} = 1|A = 0, Y = 1) - P(\hat{Y} = 1|A = 1, Y = 1)|$$

حساب تكافؤ الفرص ومعدل الإيجابيات الحقيقية عبر الفئات python

```python
from sklearn.metrics import recall_score

# حساب الاسترجاع (TPR) لكل فئة على حدة بين المستحقين فعلياً للسداد
tpr_0 = recall_score(y_te[g_te == 0], predictions[g_te == 0])
tpr_1 = recall_score(y_te[g_te == 1], predictions[g_te == 1])
eo_diff = abs(tpr_0 - tpr_1)

print("تدقيق تكافؤ الفرص (True Positive Rate per Group):")
print(f"  - نسبة قبول المؤهلين فعلياً من الفئة A=0: {tpr_0:.1%}")
print(f"  - نسبة قبول المؤهلين فعلياً من الفئة A=1: {tpr_1:.1%}")
print(f"  - فارق تكافؤ الفرص (Equal Opportunity Diff): {eo_diff:.3f}")
```

إذا كان فارق تكافؤ الفرص كبيراً، فهذا يعني أن المؤهلين من الفئة الأقلية يُحرمون من الفرص بمعدل أعلى من نظرائهم المتطابقين في الفئة الأخرى.

٩.٦ معايرة عتبات القرار: TunedThresholdClassifierCV

افتراضياً، تقطع جميع مصنّفات scikit-learn الاحتمال عند العتبة 0.5 . لكن العتبة 0.5 نادراً ما تكون الخيار الأمثل عند وجود عدم توازن بين الفئات أو تباين في تكاليف الأخطاء.

في **scikit-learn ١.٩**، توفّر المكتبة أداة معيارية ذكية تُدعى `TunedThresholdClassifierCV`، تتيح البحث التلقائي عن أفضل عتبة قرار توازن بين المقاييس الحساسة (مثل الدقة المتوازنة `balanced_accuracy` أو الاسترجاع المستهدف).

ضبط عتبة القرار عبر TunedThresholdClassifierCV في 1.9 python

```python
from sklearn.model_selection import TunedThresholdClassifierCV

# تغليف المصنّف لضبط عتبة القرار تلقائياً باستخدام الدقة المتوازنة
tuned_model = TunedThresholdClassifierCV(
    estimator=LogisticRegression(),
    scoring="balanced_accuracy",
    cv=5
)

tuned_model.fit(X_tr, y_tr)
optimal_threshold = tuned_model.best_threshold_
tuned_preds = tuned_model.predict(X_te)

print("نتائج ضبط العتبة التلقائي في scikit-learn 1.9:")
print(f"  - عتبة القرار المحسوبة كأمثل خيار: {optimal_threshold:.4f} (بدلاً من 0.5000)")
print(f"  - معدل القبول الجديد للفئة A=0   : {tuned_preds[g_te == 0].mean():.1%}")
print(f"  - معدل القبول الجديد للفئة A=1   : {tuned_preds[g_te == 1].mean():.1%}")
print(f"  - الفارق الجديد في القبول        : {abs(tuned_preds[g_te == 0].mean() -
tuned_preds[g_te == 1].mean()):.3f}")
```

أعادت المعايرة الذكية ضبط العتبة لتخفيضها إلى مستوى عادل يرفع فرص المؤهلين المهمشين دون التضحية بالدقة الإجمالية للنظام.

> **التحديث الجديد في scikit-learn ١.٩**
>
> تعد أدوات ضبط العتبات `TunedThresholdClassifierCV` و `FixedThresholdClassifier` نقلة نوعية في المكتبة، إذ تسمح بفصل خطوة التقدير الاحتمالي عن خطوة اتخاذ القرار الحرج في الإنتاج، وهو صلب هندسة الأنظمة الحساسة.

٩.٧ قائمة تدقيق المسؤولية الأخلاقية للمهندس

قبل أن توافق على رفع نموذجك إلى خادم الإنتاج، مرّر نموذجك على هذه القائمة الصارمة:

```python
# دالة فحص وتدقيق الفروق الديموغرافية لأي نموذج
def audit_fairness(model, X_eval, y_eval, sensitive_col_idx):
    """دالة قياسية تفحص التكافؤ وتكافؤ الفرص عبر الفئات."""
    preds = model.predict(X_eval)
    groups = np.unique(X_eval[:, sensitive_col_idx])

    report = {}
    for g in groups:
        mask = (X_eval[:, sensitive_col_idx] == g)
        pos_rate = preds[mask].mean()
        recall = recall_score(y_eval[mask], preds[mask]) if y_eval[mask].sum() > 0 else 0.0
        report[f"Group_{int(g)}"] = {"Selection_Rate": round(pos_rate, 3), "TPR_Recall": round(recall, 3)}
    return report

fairness_report = audit_fairness(tuned_model, X_te, y_te, sensitive_col_idx=1)
print("تقرير التدقيق الأخلاقي السريع للنموذج:")
for grp, metrics in fairness_report.items():
    print(f"  {grp}: معدل الاختيار = {metrics['Selection_Rate']:.1%}, الاسترجاع = {metrics['TPR_Recall']:.1%}")
```

☑ قائمة التحقق قبل النشر

٩.٨ تمارين الفصل

التمرين ٩.١

أنشئ مجموعة بيانات مصطنعة تضم ثلاث فئات عمرية (شباب، متوسطو العمر، كبار سن) ونموذجاً لتوقع الترقية الوظيفية. احسب معدل التحديد (Selection Rate) لكل فئة عمرية واكتشف هل يعاني كبار السن من أثر متباين.

التمرين ٩.٢

طبّق أداة `FixedThresholdClassifier` المتاحة في الإصدارات الحديثة من scikit-learn، واضبط عتبة قرار ثابتة تساوي 0.35 . قارن أداءها بأداء `LogisticRegression` القياسي ذي العتبة 0.5 .

التمرين ٩.٣

اشرح لماذا يتعارض معيار «التكافؤ الديموغرافي» أحياناً مع معيار «تكافؤ الفرص» في بيئة يكون فيها معدل الجدارة الحقيقي مختلفاً تاريخياً بين المجموعتين.

التمرين ٩.٤

اكتب كوداً يستخرج مصفوفة الخلط (Confusion Matrix) لكل فئة حساسة على حدة، واحسب منها معدل الإيجابيات الكاذبة (FPR). لماذا يمثل ارتفاع FPR لفئة معينة ضرراً بالغاً في أنظمة العدالة الجنائية؟

٩.٩ حلول بعض التمارين

✅ حلّ التمرين ٩.٣

يتعارض التكافؤ الديموغرافي مع تكافؤ الفرص عندما يكون معدل الأساس (Base Rate) مختلفاً بين الفئتين في العالم الحقيقي: فالتكافؤ الديموغرافي يفرض قبول نفس النسبة من كلا المجموعتين ($P(\hat{Y}=1|A=0) = P(\hat{Y}=1|A=1)$)، مما يجبر النموذج على خفض معايير القبول للفئة ذات معدل الأساس الأدنى، وهو ما يؤدي حتماً إلى رفع معدل الإيجابيات الكاذبة (FPR) لتلك الفئة وكسر تكافؤ الفرص. يستحيل رياضياً تحقيق التكافؤ الديموغرافي وتكافؤ الفرص ومعايرة الاحتمالات في وقت واحد إلا إذا كان معدل الأساس متطابقاً تماماً بين المجموعتين.

✅ حلّ التمرين ٩.٤

في أنظمة العدالة الجنائية (مثل التنبؤ بالعودة للجريمة)، تمثل الإيجابية الكاذبة (False Positive) تصنيف شخص بريء أو منخفض الخطورة على أنه مجرم شديد الخطورة، مما يؤدي لحرمانه من إطلاق السراح المشروط أو تشديد عقوبته. إذا كان معدل FPR للفئة الأقلية ضعف معدل الأغلبية، فهذا يعني أن النظام يعاقب الأبرياء من تلك الفئة بمعدل مضاعف، وهو انتهاك جسيم لحقوق الإنسان والعدالة القانونية.

٩.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الاكتفاء بحذف الأعمدة الحساسة	استمرار التحيّز عبر المتغيرات البديلة مع صعوبة اكتشافه	احتفظ بالسمة أثناء التطوير واستخدمها لتدقيق العدالة لا للتدريب
الاعتماد على الدقة الكلية (Accuracy) وحدها	النموذج يضحي بالفئات الصغيرة للحفاظ على دقة الأغلبية	احسب مقاييس الأداء لكل فئة حساسة على حدة
تثبيت عتبة القرار دائماً عند ٠.٥	إهدار مرونة النموذج في موازنة التكاليف الأخلاقية	استخدم `TunedThresholdClassifierCV` لضبط العتبة وفق الهدف
افتراض إمكانية تحقيق كل مقاييس العدالة معاً	الاصطدام باستحالة رياضية تفشل النماذج	اختر المقياس الأنسب لطبيعة المشكلة (تكافؤ الفرص أم التكافؤ الديموغرافي)
إطلاق النموذج دون سجل مراجعة أو تفسير	عجز المؤسسة عن تبرير قراراتها أمام الجهات التنظيمية	وفّر بطاقة نموذج (Model Card) وتدقيقاً للبواقي والفئات

٩.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التكافؤ الديموغرافي	تساوي احتمالية الحصول على النتيجة الإيجابية عبر الفئات	demographic parity
تكافؤ الفرص	تساوي معدل الاسترجاع (TPR) للمؤهلين فعلياً عبر الفئات	equal opportunity
الأثر المتباين	نسبة معدل قبول الفئة الأقل حظاً إلى الفئة الأكثر حظاً	disparate impact
المتغير البديل	ميزة غير حساسة ترتبط إحصائياً بسمة حساسة محجوبة	proxy variable
العدالة بالجهل	محاولة منع التمييز بحذف السمة الحساسة فقط	fairness through unawareness
ضبط عتبة القرار	معايرة نقطة الفصل الاحتمالي لتحقيق توازن مستهدف	threshold tuning
بطاقة النموذج	وثيقة تقنية تشرح مقاييس النموذج وحدوده الأخلاقية	model card

الجزء ٢

البيانات: الكتلة الخام للمشروع

من مصفوفة NumPy إلى المصفوفة المتفرّقة والبيانات المفقودة

ماذا يغطّي هذا الجزء

ثمانون بالمئة من وقت مهندس تعلّم الآلة يذهب في هندسة البيانات ومعالجتها، لا في تدريب النموذج. هذه القوة تشرح كيف تمسك scikit-learn بالبيانات: المصفوفات، والمصفوفات المتفرّقة، والقيم المفقودة، والأنواع المختلفة، والأخطاء التي تُسرّب البيانات قبل أن تنتبه إليها.

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٠ — أنماط البيانات في scikit-learn

ndarray و DataFrame والمصفوفات المتفرّرة: أيّها ولماذا

الوحدة **١.٩ scikit-learn** الحجم **٦٩٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تميّز بين ndarray و DataFrame والمصفوفة المتفرّرة وتشرح متى تختار كلّاً منها
- أن تقرأ نتيجة fit وتعرف من أين جاءت أسماء الخصائص
- أن تحسب كثافة المصفوفة وتقدّر حجم الذاكرة قبل أن تختار تمثيل البيانات
- أن تُصلح الخطأ could not convert string to float بالطريقة الصحيحة

٢.١ لماذا يهمّ شكل البيانات قبل أن يهمّ النموذج؟

تخيّل أنك وصلت إلى مشروع، والبيانات محمّلة، وأدوات التحليل تعمل. تفتح جدولاً فيه عمود عمر، وعمود مدينة نصّي، وعمود تاريخ. تكتب سطراً واحداً:

```python
المشكلة في سطر واحد
from sklearn.linear_model import LogisticRegression
model = LogisticRegression().fit(df[["age", "city"]], y)
```

وتحصل على:

```python
الرسالة التي سترسلها المكتبة
ValueError: could not convert string to float: 'Riyadh'
```

هذه ليست رسالة عابرة. هذه هي **الحدود الحقيقية** للمكتبة. `LogisticRegression` لا يعرف ما هي المدينة. كل ما يعرفه أنه يتوقّع مصفوفة أرقام، وسأله سؤالاً واحداً: هل تستطيع أن تحوّل `'Riyadh'` إلى عدد؟ لا يستطيع، فيتوقّف.

الدرس الأوّلي: `scikit-learn` ليست بيئة بيانات عامة، بل طبقة حسابية فوق `NumPy`. كل مُقدِّر فيها — من أقرب الجيران إلى التعزيز التدرّجي — يتلقّى مصفوفة رقمية ويُعيد مصفوفة رقمية. ما عدا ذلك، تُلصَق البنية في الواجهة بواسطة `ColumnTransformer` (الفصل ٨٥).

> **القاعدة التي تُبنى عليها المكتبة**
>
> كل مُقدِّر في `sklearn` له عقد واحد: `fit(X, y)` ثم `predict(X)`، حيث `X` مصفوفة رقمية ثنائية البعد (صفوف = عينات، أعمدة = خصائص)، و `y` متّجه بطول `n`. الأنماط الأخرى ليست «غير مدعومة»، بل **تُحوَّل إلى هذا العقد** قبل الوصول إليه.

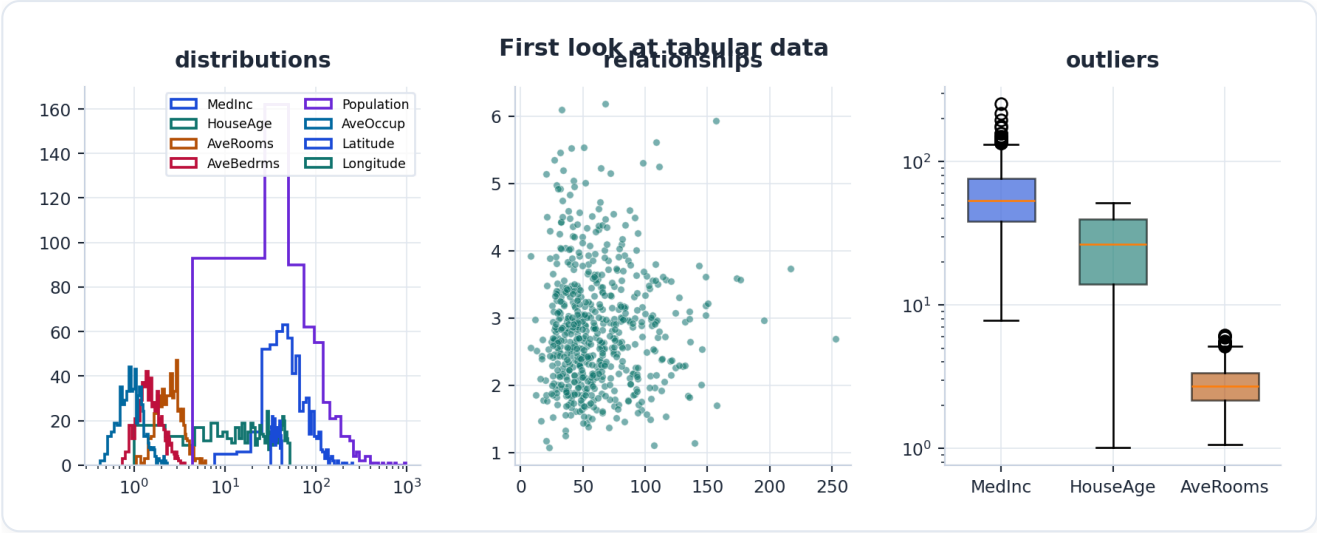
٢.٢ المسار من ملفّ CSV إلى مصفوفة

لا يوجد في `scikit-learn` دالة تقرأ ملفّ CSV. هذه ليست نقصاً بل قرار تصميمي: المكتبة لا تتعلّم عن أنظمة الملفات ولا قواعد البيانات ولا صيغ الملفات. مسؤوليتها تبدأ عند الحدّ الذي تكون فيه البيانات **في الذاكرة**.

الخطوة	الأداة المسؤولة	ما الذي تُنتجه
١. القراءة	`pandas.read_csv` أو `load_*`	`DataFrame` أو `Bunch`
٢. الفحص	`DataFrame.dtypes` و `isna()`	خريطة الأنواع والفجوات
٣. التقطيع	`ColumnTransformer`	مصفوفة بعد معالجة كل عمود بطريقته
٤. التدريب	أيّ `estimator.fit(X, y)`	معاملات وقرارات
٥. الرجوع	`estimator.predict(X)`	تنبّؤات على نفس القالب الذي استُقبل

لاحظ الخطوتين ٢ و ٣: هما بالضبط ما يفصل هذا الكتاب عن دليل مرجعي جاف. أغلب مشاريع التعلّم الآلة الفاشلة تفشل قبل أن يُكتب أول سطر `fit`.

قبل أن نبدأ، خذ خطوة إلى الخلف وانظر إلى جدول حقيقي: توزيع كل عمود على حدة، على حدة، وعلاقة عمودين، وصندوقي شذوذ. هذه اللوحة الثلاثية هي أسرع طريقة لفهم شكل بياناتك قبل أن تلمسها أداة.



الشكل ٢٣ثلاث لوحات لجدول أسعار منازل: توزيع كل عمود على محور لوغاريتمي، وعلاقة الدخل بعدد الغرف، وصندوقا شذوذ يكشفان الانحراف بوضوح

دعنا نبدأ بمصدر بيانات حقيقي مضمّن داخل المكتبة بلا أيّ تنزيل.

أول نظرة على حزمة بيانات مضمّنة python

```python
import pandas as pd
from sklearn.datasets import load_diabetes

dia = load_diabetes()
print(f"نوع الكائن        : {type(dia).__name__}")
print(f"المفاتيح المتاحة: {sorted(k for k in dia if not k.startswith('_'))}")
print(f"شكل X             : {dia.data.shape}  dtype={dia.data.dtype}")
print(f"شكل y             : {dia.target.shape}")
print(f"أسماء الخصائص    : {list(dia.feature_names)}")
print()
dia_df = pd.DataFrame(dia.data, columns=dia.feature_names)
dia_df.head()
```

لاحظ أن `load_diabetes()` يعيد كائناً اسمه `Bunch`: قاموس بسمات بدل أقواس. و `dia.data` مصفوفة `ndarray` نظيفة، بينما `dia.frame` هو `DataFrame` يحمل الأسماء. الاثنان يصفان الشيء نفسه؛ الأول للحوسبة، والثاني للتفكير.

٢.٣ النمط الأول: المصفوفة الرقمية `ndarray`

هذا هو النمط الأمومي. بُنيت `scikit-learn` على `NumPy` قبل أيّ شيء آخر، وكل ما يلي مشتقّ منه.

```python
# النمط الأمومي: صفيحتان بلا أسماء
import numpy as np

A = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0],
              [7.0, 8.0, 9.0]])

print("dtype          :", A.dtype)
print("shape          :", A.shape, "->", A.shape[0], "صف و", A.shape[1], "عمود")
print("الصف الثاني    :", A[1])
print("العمود الثالث  :", A[:, 2])
print("قناع منطقي يخفي الصف الأخير:", A[A[:, 0] > 3])
print("ما تراه المكتبة:", type(A).__name__, "— بلا أسماء أعمدة")
```

ثلاث حقائق عن `ndarray` يجب أن تحفظها:

٢.٤ النمط الثاني: جدول البيانات `DataFrame`

`DataFrame` من `pandas` هو ما سيمسك بيدك فعلاً في أغلب الوقت. والفرق الجوهري ليس في القيم، بل في **الأسماء والفهرس والأنواع**.

```python
# جدول بأربعة أنواع أعمدة مختلفة
import pandas as pd

df = pd.DataFrame({
    "age": [23, 45, 31, 52, 38],
    "city": ["Riyadh", "Cairo", "Riyadh", "Amman", "Cairo"],
    "joined": pd.to_datetime(["2023-01-15", "2021-06-02", "2024-03-20", "2020-11-11", "2022-08-08"]),
    "active": [True, False, True, True, False],
})
print(df.to_string(index=False))
print()
print(df.dtypes)
print()
print(f"حجم الذاكرة المستعمل فعلياً: {df.memory_usage(deep=True).sum()} بايت")
```

لاحظ ثلاثة أشياء:

كيف يقرأ `scikit-learn` جدولاً بأسماء؟

هذا أهمّ ما في هذا القسم. عندما تمرّر `DataFrame` إلى `fit`، تفعل المكتبة ثلاثة أشياء بالترتيب: تتحقّق من صحّة القيم، **تنسخ القيم الرقمية فقط** في مصفوفة داخلية بلا أسماء، ثم **تحفظ أسماء الأعمدة** في `feature_names_in_` لتستخدمها في التقارير لاحقاً.

أثر تمرير الأسماء على ما تخرج به المكتبة — python

```python
from sklearn.preprocessing import StandardScaler

with_names = pd.DataFrame(dia.data[:60], columns=dia.feature_names)
sc_named = StandardScaler().fit(with_names)
sc_anon = StandardScaler().fit(dia.data[:60])

print("مع DataFrame -> feature_names_in_:", list(sc_named.feature_names_in_[:4]),
"…")
print("مع ndarray   -> feature_names_in_:", hasattr(sc_anon,
"feature_names_in_"))
print("get_feature_names_out() مع أسماء :", sc_named.get_feature_names_out()[:4])
print("get_feature_names_out() بلا أسماء:", sc_anon.get_feature_names_out()[:4])
print("عدد الخصائص في الحالتين           :", sc_named.n_features_in_,
sc_anon.n_features_in_)
```

هذا هو **فارق قابلية القراءة وحده**: نفس النموذج، نفس الأرقام، لكن الأول يقول «bmi» والثاني يقول «x٢». في تقريرٍ يُقرأ من طرف غير تقني، الفرق بين «ارتفع مؤشر كتلة الجسم» و«ارتفع x٢» هو الفرق بين قرارٍ يُتّخذ ورقمٍ يُتجاهل.

> **قاعدة عملية**
>
> مرّر `DataFrame` إلى `fit` ما دمتَ لا تملك سبباً قاطعاً لاختيار غيره. إن تحوّلت إلى `ndarray` لأداءٍ أو لتمريرٍ عبر حدّ ما، احتفظ بمصفوفة الأسماء في مكانٍ ما، لأن النموذج لن يحفظها نيابةً عنك.

٢.٥ النمط الثالث: المصفوفة المتفرّرة

البيانات الجدولية لا تبقى جدولية أحياناً: نصّ، أو متّجه واحد-ساخن لمئة قيمة، أو تفاعل بين مئة خاصية. في هذه الحالات تكون **الأصفار هي قاعدة التخزين**: تخزينها كاملةً هدر.

الصيغة متعددة الاستخدام في `SciPy` هي `csr`، وتخزّن ثلاثة متّجهات:

المتّجه	ماذا يحفظ	طوله
`data`	القيم غير الصفرية	عدد `nnz`
`indices`	فهرس العمود لكل قيمة	عدد `nnz`
`indptr`	أين يبدأ كل صف	عدد الصفوف + ١

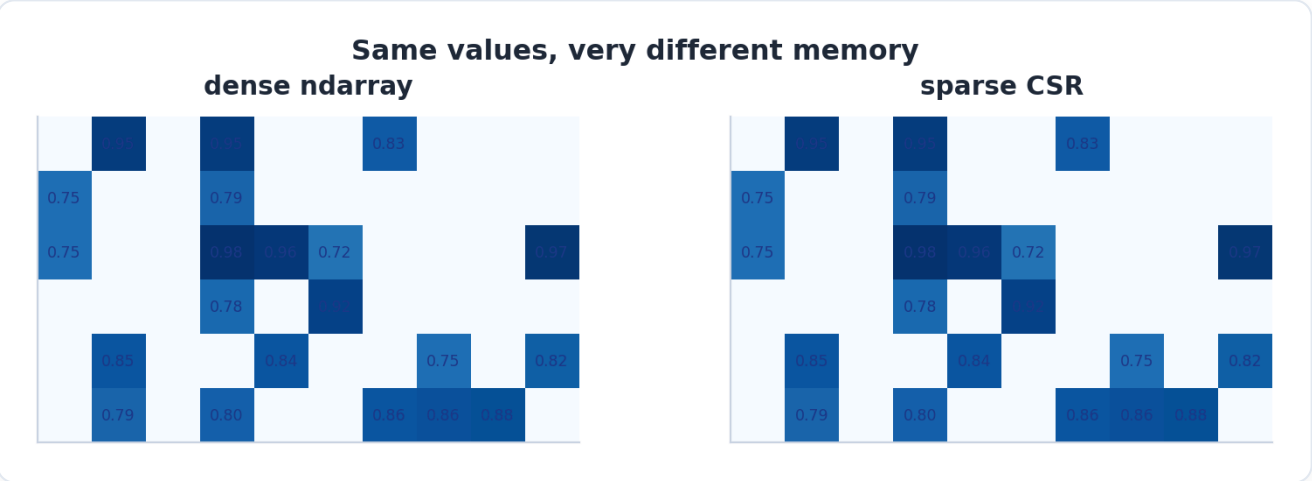
الكثافة هي النسبة التي تقرّر مصير هذا الاختيار:

$$\rho = \frac{\text{nnz}}{n \cdot m}$$

وحجم الذاكرة التقريبي:

$$M_{\text{CSR}} = \text{nnz} \cdot (8 + 4) + (m + 1) \cdot 4$$

اقرأ المعادلة الثانية: ثمانية بايتات للقيمة وأربعة بايتات لفهرسها لكل عنصر غير صفر. لذلك حتى بكثافة ٥٪ يكسب `CSR` أكثر من عشرة أضعاف على `ndarray`.



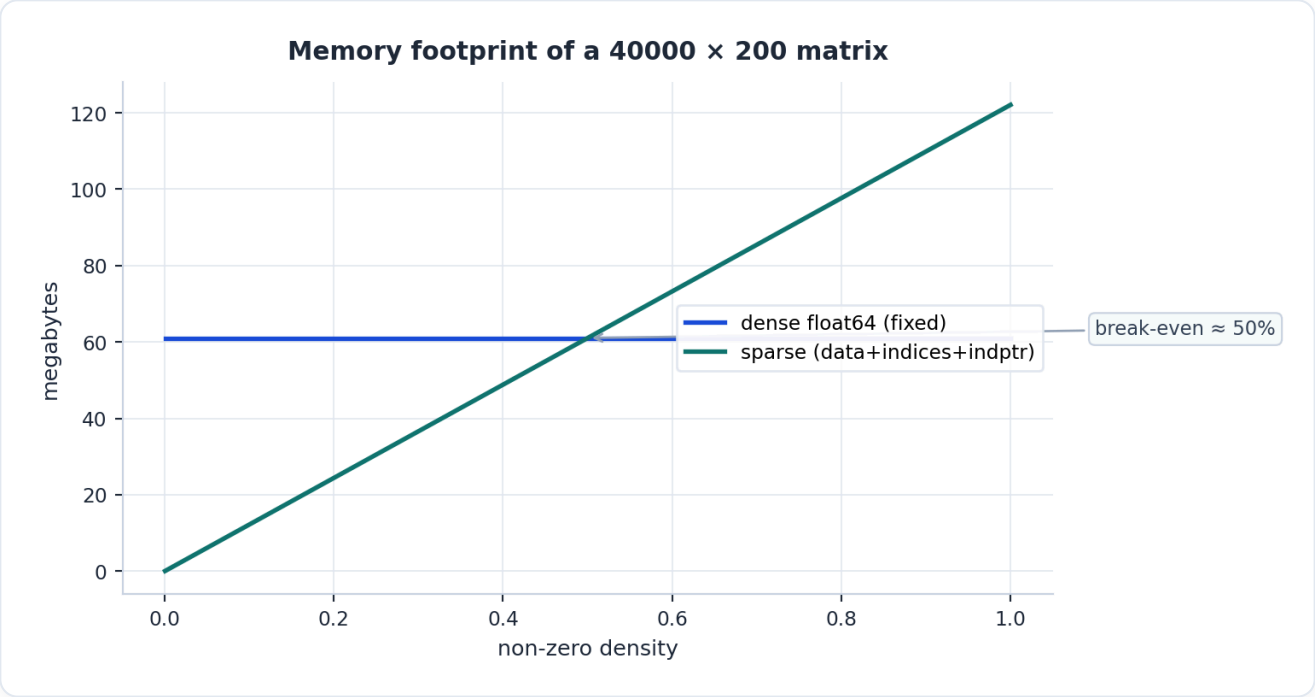
الشكل ٢٤ لوحتان لمصفوفتين تحملان القيم نفسها: واحدة كثيفة وأخرى متفرّرة، والفرق الوحيد أن الأبيض في الثانية صفرٌ لا يُخزَّن

الشكل أعلاه يجعل الفرق محسوساً: المصفوفتان تحملان الأرقام نفسها، لكن النمط الكثيف يُظهر شبكة كاملة من الخانات، والمتفرّر يُظهر بضع نقاط فقط. هذا هو الفارق بين «حفظ كلّ شيء» و«حفظ ما يُفيد».

كم يوفّر فعلاً التمثيل المتفرّر؟ python

```python
import scipy.sparse as sp

n_rows, n_cols, density = 2000, 300, 0.02
S = sp.random(n_rows, n_cols, density=density, format="csr", random_state=0)
sparse_bytes = S.data.nbytes + S.indices.nbytes + S.indptr.nbytes
dense_bytes = n_rows * n_cols * 8

print(f"كثافة المصفوفة : {S.nnz / (n_rows * n_cols):.4f}  ({S.nnz} غير صفر من {n_rows
* n_cols})")
print(f"حجم ndarray     : {dense_bytes / 1024**2:6.2f} ميغابايت")
print(f"حجم CSR         : {sparse_bytes / 1024:6.1f} كيلوبايت")
print(f"نسبة التوفير    : {dense_bytes / sparse_bytes:6.1f} أضعاف")
print()
print(f"نوع البيانات  : {S.data.dtype}   نوع الفهارس: {S.indices.dtype}")
```

الفصل ١١ يغوص في هذه الصيغ كلّها ويفحص الواجهة الجديدة `sparray` . ما نريد احتفاظه هنا هو **قاعدة القرار**: إذا كانت مصفوفتك تقارن بكثافة بين ٠٫٠١ و٠٫٠٢ — وهذا ما تنتجه ترميز `One-Hot` على بيانات فئوية عالية، أو `TF-IDF` على مستندات — فاترك `scikit-learn` يتكفّل بالتحويل عبر `sparse_threshold` ، ولا تحوّلها بنفسك.



الشكل ٢٥ أعمدة تقارن حجم المصفوفة الكثيفة بعدة صيغ متفرّرة، ويتضح أن كلفة كل صيغة تتبع كثافتها لا بُعدها

٢.٦ متى تستخدم كلّ نمط؟

المعيار	`ndarray`	`DataFrame`	مصفوفة متفرّزة
يقبله أيّ `estimator` مباشرة	نعم	نعم للأعمدة الرقمية فقط	نعم إن كان يدعم `sparse`
يحفظ أسماء الخصائص	لا	نعم في `feature_names_in_`	لا
يدعم أعمدة فئوية نصّية	لا	نعم عبر `ColumnTransformer`	لا
أعمدة زمنية جاهزة	لا	نعم عبر التحويل	لا
الذاكرة عند كثافة ٠٫٠٢	٤٫٥٨ ميغابايت	مثلها مع دلالات	١٤٨ كيلوبايت
أفضل للتفسير البشري	ضعيف	ممتاز	ضعيف
أفضل للحسابات المتوازية	ممتاز	جيد	ممتاز

٢.٧ الخطأ الذي يراه الجميع: القيم النصّية في مكانٍ لا مكان لها

عندما تمرّر أعمدةً مختلطة إلى `fit` رقمي، ترى هذه الرسالة بالضبط:

أين بالضبط يفشل التحويل؟ python

```python
from sklearn.utils.validation import check_array
from sklearn.linear_model import LogisticRegression

mixed = pd.DataFrame({"age": [23.0, 45.0, 31.0, 52.0],
                      "city": ["Riyadh", "Cairo", "Riyadh", "Amman"]})

for kw in [dict(), dict(dtype=None)]:
    label = kw if kw else "افتراضي"
    try:
        out = check_array(mixed, **kw)
        print(f"check_array(dtype={label}) -> نجح: {type(out).__name__} dtype=
{out.dtype}")
    except Exception as exc:
        print(f"check_array(dtype={label}) -> {type(exc).__name__}:
{str(exc).splitlines()[0][:88]}")

try:
    LogisticRegression(max_iter=500).fit(mixed, [0, 1, 1, 0])
except Exception as exc:
    print(f"LogisticRegression.fit         -> {type(exc).__name__}:
{str(exc).splitlines()[0][:88]}")
```

لاحظ أن `check_array` بـ `dtype="numeric"` يرفض **الجدول كلّه**: لا استثناء لعمود واحد. والمخرج الصحيح ليس «حذف العمود» بل **ترميزه بأداة تفهمه**.

الحل: كل عمود يُعالَج بطريقته python

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

pipe = Pipeline([
    ("pre", ColumnTransformer([
        ("num", StandardScaler(), ["age"]),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
["city"]),
    ])),
    ("clf", LogisticRegression(max_iter=500)),
])
pipe.fit(mixed, [0, 1, 1, 0])
print("التنبّؤ          :", pipe.predict(mixed))
print("أسماء المخرجات:", pipe.named_steps["pre"].get_feature_names_out())
```

هذا النمط — «جدول واحد، عدة مسارات، مصفوفة واحدة في النهاية» — هو الفصل ٨٥ بالكامل. نكتبه الآن لأن بقية فصول هذا الجزء كلّها مبنيّة عليه.

الخطأ الأول: الحذف بدل الترميز

بعض المبتدئين يحلّون الخطأ أعلاه بـ `df.drop(columns="city")` ثم يظنّون أن المشكلة انتهت. لم تنتهِ: التاجر الذي يفقد مئة عمود غير رقمي خسر مئة إشارة. الترميز لا يكلّف سطراً إضافياً ويحفظ البيانات. **أسقط العمود فقط إن كانت قيمته مفقودة فعلاً، لا لأنه نصّي.**

الخطأ الثاني: الخلط بين المحورين

`X[i]` هو الصفّ رقم `i`. و `X[:, i]` هو العمود رقم `i`. في مصفوفة ألف صفّ وألف عمود، الفرق بينهما فارق مئة ألف خانة. اكتب `X[:, j]` للخصائص و `X[i, :]` للعيّنة، ولا تحذف النقطتين أبداً.

٢.٨ في الواقع: البيانات لا تأتي نظيفة

في العمل الحقيقي، المشكلة لا تبدأ عند `fit`. تبدأ قبلها بثلاث طبقات:

القاعدة المهنية التي توفّر عليك في كلّ مرّة: **افصل الاستخراج عن المعالجة عن التدريب**، واجعل المعالجة كائناً واحداً قابلاً لإعادة التشغيل (الفصل ٨٤). المشروع الذي يخلط الثلاثة في دفتر `Jupyter` لا يمكن اختباره ولا تدقيقه ولا إعادة بنائه.

في الواقع: قاعدة الانطلاق

المشروع الذي تتعلّمه في الكتاب ينتهي غالباً عند أول ملفّ حقيقي من الإنتاج. عندها تتّضح ثلاث حقائق لا تظهر في أيّ مثال: أعمدة «رقمية» تُخزَّن نصوصاً، وأعمدة «مفقودة» تُكتب صفراً أو سالباً واحداً أو النصّ `N/A`، ومفاتيح «فريدة» تتكرّر عشر مرّات بأسماء مختلفة. الفصل ١٣ يعالج الحالة الثانية تحديداً، والفصل ١٦ الأولى.

تمارين الفصل

التمرين ٢.١

حمّل `load_diabetes`، وابنِ `DataFrame` من `dia.data` و `dia.target` مع بادئة `f_` لأسماء الخصائص. مرّره إلى `StandardScaler` ثم إلى `LogisticRegression(max_iter=500)` مع هدف ثنائي مشتقّ من وسيط `target`.

اطبع `len(scaler.feature_names_in_)`، ثم أعد التدريب على `dia.data` وحده واطبع الرقم نفسه. فسّر لماذا النتيجتان متطابقتان وما الذي تغيّر فعلياً.

التمرين ٢.٢

ابنِ مصفوفة `300 × 50` بكثافات مختلفة، واحسب يدوياً حجم `CSR` المتوقّع:

$$\text{bytes} = \text{nnz} \cdot (8 + 4) + (m + 1) \cdot 4$$

ثم قارنه بـ `M.data.nbytes + M.indices.nbytes + M.indptr.nbytes` الحقيقي، وحدّد عند أيّ كثافة تتساوى التكلفة بين `ndarray` و `CSR`.

✅ الحلّ

الفارق أن `CSR` تخزّن `float64` و `int32` لكل قيمة غير صفرية، بينما تخزّن `ndarray` ثمانية بايتات لكل خانة سواء كانت صفراً أم لا. فتكلفة `CSR` تقارب `nnz × 12 + (m + 1) × 4` بايت.

حلّ التمرين ٢.٢: أين تتساوى التكلفة؟ python

```python
import scipy.sparse as sp

n_rows, n_cols = 300, 50
print(f"{'الكثافة':>8} {'nnz':>7} {'ndarray':>12} {'CSR':>10} {'الأوفر':>9}")
for d in [0.05, 0.10, 0.20, 0.40]:
    M = sp.random(n_rows, n_cols, density=d, format="csr", random_state=0)
    csr = M.data.nbytes + M.indices.nbytes + M.indptr.nbytes
    dn = n_rows * n_cols * 8
    print(f"{d:8.2f} {M.nnz:7d} {dn:12,d} {csr:10,d} {'CSR' if csr < dn
else 'ndarray':>9}")
```

نقطة التعادل تقع قرب الكثافة ٠٫١٥، لأن الحلّ يحتاج نحو اثني عشر بايت لكل قيمة غير صفرية لتغطية خانة كاملة من `ndarray`، زائد فهرس نهاية واحد لكل صفّ.

التمرين ٢.٣

خذ `df` من القسم ٢.٤ بأعمدته الأربعة، واكتب دالة `describe_types(frame)` تعيد `DataFrame` من عمودين: اسم العمود ونوعه. ثم احسب نسبة الأعمدة الرقمية إلى الإجمالي، واقترح ثلاث تفسيرات لكون هذه النسبة منخفضة في بيانات الإنتاج.

✅ **الحلّ**

النسبة هنا `1 / 4 = 25%` فقط، لأن ثلاثة أعمدة من أربعة غير رقمية. التفسير المعتاد أن التواريخ وشروح البيع والوسوم تُحفظ نصّاً لا رقماً. الحلّ ليس رقمنة كلّ شيء، بل تصنيف كل عمود إلى أحد أربعة مسارات داخل `ColumnTransformer`: رقمي، وفئوي، وزمني، ونصّي. وهو موضوع الفصلين ١٢ و ١٧.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
تمرير `DataFrame` فيه عمود نصّي إلى `fit` رقمي	`ValueError: could not convert string to float`	`ColumnTransformer` بمسار لكل نوع
الاعتماد على `X[i]` كأنه عمود	تختبر صفّاً فتحصل على رقم لا معنى له	`X[:, i]` للخصائص و `X[i]` للصف
توقّع `feature_names_in_` مع `ndarray`	`AttributeError`	مرّر `DataFrame` أو احفظ الأسماء بنفسك
تحويل الجدول إلى `ndarray` مبكراً	تفقد كلّ النصوص والتواريخ دفعة واحدة	حوّل فقط بعد التقسيم إلى مسارات
تكثيف مصفوفة متفرّزة يدوياً بلا داعٍ	ذاكرة أضعف وسرعة أبطأ من `CSR`	دع `sparse_threshold` يتصرّف
الاعتماد على ترتيب الأعمدة دون أسماء	خطأ صامت بعد إعادة ترتيب المصدر	تحقّق من `feature_names_in_` دائماً
استدعاء `.toarray()` على بيانات ضخمة «لفهمها»	يقتل فائدة التمثيل المتفرّز كلياً	اعرض أول ٢٠ صفّاً فقط

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المصفوفة الرقمية	صفيحة أرقام بلا أسماء	ndarray
جدول البيانات	صفيحة بأسماء وأنواع وفهرس	DataFrame
المصفوفة المتفرّرة	تخزين القيم غير الصفرية فقط	sparse matrix
الصفوف المتفرّقة	صيغة `CSR` الأشهر في `SciPy`	compressed sparse row
القيمة غير الصفرية	عدد العناصر التي لا تساوي صفراً	non-zero, nnz
الكثافة	نسبة الخانات غير الصفرية إلى الكل	density
حزمة البيانات	قاموس بسمات يعيده `load_*`	Bunch
أسماء الخصائص	أسماء الأعمدة التي حفظها `fit`	_feature*names*in
أسماء المخرجات	أسماء الأعمدة الناتجة عن التحويل	get*feature*names_out
عقد المُقدِّر	القواعد الثابتة التي يتبعها كل صنف	estimator API
الجدول المختلط	جدول فيه أرقام ونصوص وتواريخ	mixed dtype frame

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١١ — المصفوفات المتفرّزة وواجهة sparray الجديدة

المفتاح sparse_interface في ١.٩ — ماذا يعني لك ولماذا يهمّك

الوحدة **scikit-learn ١.٩** الحجم **٧٩٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح بنية CSR بثلاثة متّجهات وتفهم ما يفعله كلّ واحد منها
- أن تحسب حجم الذاكرة المتوقّع لكل صيغة قبل أن تختارها
- أن تعرف متى يقبل مُقدِّر ما مصفوفة متفرّزة ومتى يرفضها بصوت عالٍ
- أن تبدّل بين sparse_interface="spmatrix" و "sparray" وتعرف ما يتأثّر

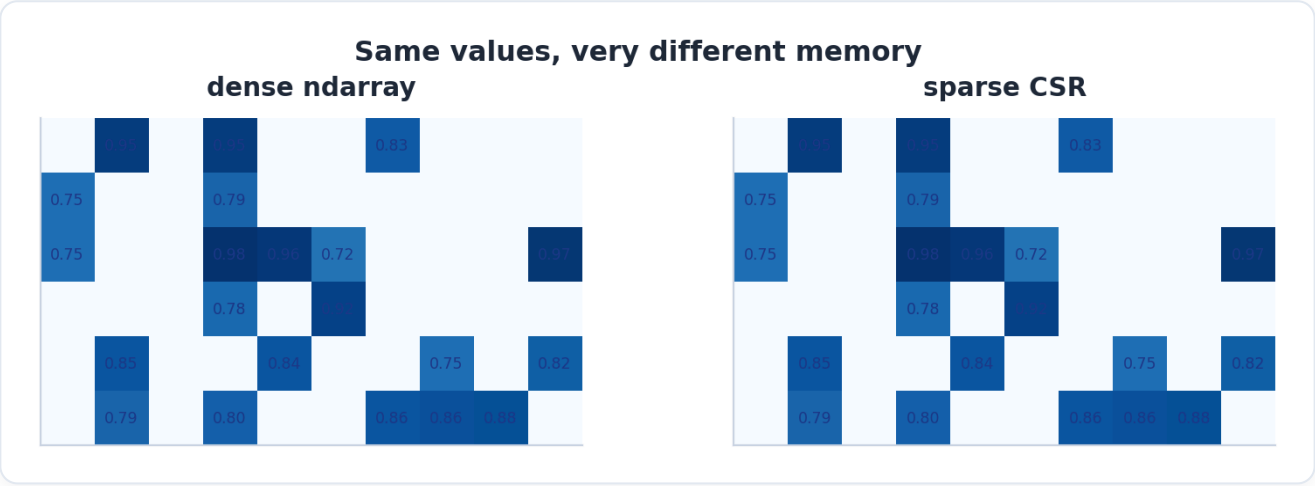
٢.١ ما الذي يعنيه «متفرّز» أصلاً؟

ابدأ بسؤال بسيط. لديك ٥٠٠٠ عمود، ولكل صفّ قيمة واحدة فقط. كم خانة تحتاج؟

الإجابة الصحيحة: خانة واحدة. لكن `ndarray` من نوع `float64` يخزّن **كل** الخمسة ملايين خانة. هذا ليس تفصيلاً هندسياً، بل فرق بين ٤٠ ميغابايت وقرابة ٧٠ كيلوبايت للصفّ الواحد، وهو ما يقرّر إن كان مشروعك يعمل على جهاز محمول واحد أم لا يعمل أصلاً.

التمثيل المتفرّز يقلب الافتراض: بدلاً من أن يسأل «ما قيمة الخانة (٢،٣)؟» يسأل «ما القيم غير الصفرية في هذه المصفوفة كلها؟». ولأنّ الإجابة أقصر بكثير عند كثافة منخفضة، تكون المكسب هو القاعدة.

البُعد	`ndarray`	مصفوفة متفرّزة
ما يُخزَّن	كل القيم	القيم غير الصفرية فقط
الذاكرة	`n × m × 8` بايت	`nnz × 12` بايت تقريباً
زمن الجلب	ثابت	يعتمد على عدد الأصفار في الصفّ
التحويل إلى كثيف	مجاني	مكلّف: `O(n × m)`
مناسب لـ	بيانات كثيفة	بيانات نادرة



الشكل ٢٦ لوحتان لمصفوفتين تحملان القيم نفسها: الكثيفة تُظهر كلّ خانة، والمتفرّزة تُظهر نقاطاً ساطعة على أرضية فارغة

لاحظ أن الصورة على اليسار لا يمكن تمييز أصفارها عن غير أصفارها بمجرد النظر. هذا هو بُعد المشكلة: **كثافة ٥٪ يبدو شبكة شبه كاملة.**

٢.٢ الصيغ الثلاث التي ستقابلها في `SciPy`

`SciPy` لا تختار صيغة واحدة، بل تعطيك أربعاً، ولكلّ منها غرض.

الصيغة	Constructor	أفضل استخدام
`csr`	`scipy.sparse.csr_matrix`	الحساب والضرب، الافتراضي العملي
`coo`	`scipy.sparse.coo_matrix`	البناء الأولي، ثم تحويل
`lil`	`scipy.sparse.lil_matrix`	التعديل العنصر-بعنصر
`csc`	`scipy.sparse.csc_matrix`	القراءة عموداً بعينه (نادر في `sklearn`)

لنرَ البنية الداخلية كما هي فعلاً، لا كما تُشرح في العروض التقديمية.

python — ثلاث صيغ، نفس البيانات، بنية مختلفة

```
import numpy as np
import scipy.sparse as sp

rng = np.random.default_rng(0)
D = rng.random((6, 5))
D[D < 0.6] = 0.0
print("المصفوفة الكثيفة:")
print(np.round(D, 3))

csr = sp.csr_matrix(D)
coo = sp.coo_matrix(D)
lil = sp.lil_matrix(D)

for name, m in [("csr", csr), ("coo", coo), ("lil", lil)]:
    print(f"{name:4s} nnz={m.nnz:3d} shape={m.shape} has getformat={hasattr(m,
'getformat')}")

print("\ncsr.indptr  (أين يبدأ كل صفّ)  :", csr.indptr.tolist())
print("csr.indices (عمود كل قيمة)     :", csr.indices.tolist())
print("csr.data    (القيم نفسها)      :", np.round(csr.data, 3).tolist())
print("coo.row     :", coo.row.tolist())
print("coo.col     :", coo.col.tolist())
print("\nالصفّ 3 في csr:", csr[3].toarray())
print("تحويل ذهاباً وإياباً بلا فقد:", (csr.tolil().tocsr() != csr).nnz == 0)
```

اقرأ `csr.indptr` على مهل: `[0, 2, 6, 9, 10, 13, 17]`. بما أنّ الطول ٦ صفوف + ١، فإن الصفّ رقم `i` يحتلّ الشرائح `indptr[i]` إلى `indptr[i+1]`. أي أنّ الصفّ الرابع هو `data[10:13]` مقترناً بـ `indices[10:13]`. هذا كلّ ما في `CSR`.

> **لماذا `coo` وليس `csr` للبناء؟**
>
> `coo` تخزّن ثلاث قوائم مستقلّة: صفّ، وعمود، وقيمة. بناؤها لا يحتاج أيّ ترتيب، فيناسب البيانات القادمة من ملفات وملصقات. لكن ضرب المصفوفات فيها بطيء، لذلك يوصي `SciPy` دائماً بالتحويل: `coo.tocsr()` ثم العمل. هذا بالضبط ما تفعله `scikit-learn` داخلياً.

٢.٣ كم يوفّر فعلاً؟ القاعدة الحسابية

الحساب يدوياً بسيط. `CSR` يحتاج ثلاثة متّجهات: قيمة ثمانية بايتات وفهرس أربعة بايتات لكل عنصر غير صفر، ومؤشّر نهاية واحد لكل صفّ.

$$M_{\text{CSR}} = \text{nnz} \cdot (8 + 4) + (m + 1) \cdot 4$$

في مقابل الكثيف:

$$M_{\text{dense}} = n \cdot m \cdot 8$$

المعادلتان تتساويتان تقريباً عند:

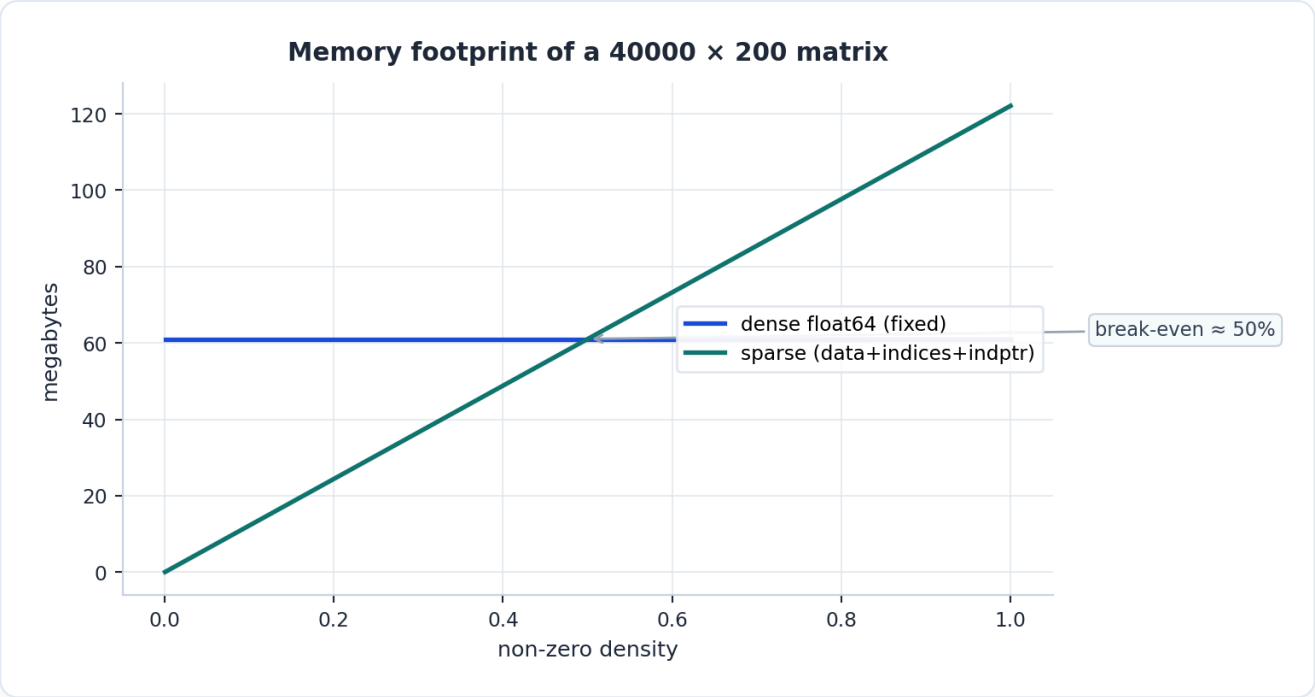
$$\rho \approx \frac{12}{8 \cdot 1 + \frac{4}{n}} \approx 1.5$$

أي فوق ١٥٠٪، وهذا مستحيل. عملياً: `CSR` يكسب دائماً في الذاكرة، والسؤال هو هل يكسب في **الزمن**؟

جدول الكلفة الحقيقي عبر كثافات مختلفة — python

```
n_rows, n_cols = 500, 400
print(f"{'الكثافة':<8} {'nnz':>8} {'كثيف (MB)':>11} {'CSR (KB)':>10} {'التوفير':<9}")
for d in [0.005, 0.02, 0.10, 0.30]:
    M = sp.random(n_rows, n_cols, density=d, format="csr", random_state=0)
    csr = M.data.nbytes + M.indices.nbytes + M.indptr.nbytes
    dn = n_rows * n_cols * 8
    print(f"{d:8.3f} {M.nnz:8d} {dn / 1024**2:11.2f} {csr / 1024:10.1f} {dn /
csr:8.0f}x")
```

لاحظ العمود الأخير: التوفير ليس خطّياً في الكثافة بل يتناسب معها تقريباً. عند كثافة ٠٫٠٣ يتوفّر المصفوفةُ الكثيفةُ مساحةً ثمانية أضعاف كاملة.



الشكل ٢٧مقارنة بصرية لحجم الذاكرة بين المصفوفة الكثيفة وعدّة صيغ متفرّرة، والخيط thick يرتفع مع الكثافة في كلّ الحالات

٢.٤ متى يكسب التمثيل المتفرّر في الزمن أيضاً؟

الذاكرة وحدها لا تكفي لتبرير قرار معماري. السؤال الثاني: هل تُسرَّع الحسابات؟ الجواب يعتمد على خوارزمية المُقدِّر.

$$\text{dense} : O(n \cdot d) \qquad \text{sparse} : O(\text{nnz})$$

بمعنى: أيّ عملية تمرّ على كلّ الخانات تدفع ثمن الكثافة، وأيّ عملية تمرّ على القيم غير الصفرية فقط تستفيد فوراً. المسافة الإقليدية في `k` -الجيران المثال الأوضح: على `float64` تحتاج أن تقرأ ٨ بايتات لكل خانة، وعلى `CSR` تقرأ ١٢ بايتاً لكل قيمة **غير صفرية** فقط.

الفارق الزمني على ٣٠٠٠ عيّنة و ٨٠٠ خاصية python

```python
import time
from sklearn.neighbors import NearestNeighbors

n, p = 3000, 800
X_dense = rng.random((n, p))
X_sparse = sp.random(n, p, density=0.01, format="csr", random_state=0)
q_sparse = sp.random(1, p, density=0.01, format="csr", random_state=1)
q_dense = rng.random((1, p))

t0 = time.perf_counter()
NearestNeighbors(n_neighbors=5).fit(X_sparse).kneighbors(q_sparse)
t_sparse = time.perf_counter() - t0

t0 = time.perf_counter()
NearestNeighbors(n_neighbors=5).fit(X_dense).kneighbors(q_dense)
t_dense = time.perf_counter() - t0

dense_bytes = X_dense.nbytes
sparse_bytes = X_sparse.data.nbytes + X_sparse.indices.nbytes +
X_sparse.indptr.nbytes
print(f"الذاكرة : كثيفة {dense_bytes / 1024**2:.1f} متفرّزة | ميغابايت {sparse_bytes /
1024:.0f} كيلوبايت")
print(f"متفرّزة :    الزمن {t_sparse:.4f} كثيفة | ث {t_dense:.4f} تسريع | ث {t_dense /
t_sparse:.0f}x")
```

الفارق هنا ليس تجميلياً: نحن نتحدّث عن مصفوفة ١٨ ميغابايت فقط. في مشروع حقيقي بعشرات الآلاف من الخصائص الفئوية، يصبح الفارق بين دقائق وساعات.

الخطأ الأول: افتراض أن كل مُقدِّر يتعامل مع التفرّز

`LinearRegression` و `KNeighborsClassifier` و `SVC` و `MLPClassifier` تعلن `sparse` في توثيقها وتقبل `CSR`. لكن `RandomForest` و `GradientBoosting` و `MLP` في نسخ كثيرة منه ترفضه صراحةً بـ `TypeError: A sparse matrix was passed, but dense data is required`. **افحص** `get_params` **أو جرّب مصفوفة صغيرة قبل أن تُعيد بناء خط أنابيب كامل حول هذه الافتراض.**

الخطأ الثاني: التحويل إلى كثيف «لمجرّد الفهم»

`.toarray()` على مصفوفة ٢٤٠٠٠٠٠٠ خانة ينتج ١٩ غيغابايت، و `MemoryError` قادم. لو أردت تفقّد البنية، اطبع `A[:20].toarray()` — عشرون صفّاً تكفي لفهم البنية، ولا تكسر ذاكرتك.

٢.٥ من يقبل المتفرّز؟ من يرفضه؟

قبل أن تكتب أيّ شيء، افحص وجود `sparse` في `fit` للصف الذي تستخدمه.

الصنف	يقبل `csr`؟	ملاحظة
`sklearn.linear_model.Ridge`	نعم	الحلّ يتكرّر عبر `nnz`
`sklearn.linear_model.LogisticRegression`	نعم	أفضل من `liblinear` في الفروق الكبيرة
`sklearn.neighbors.NearestNeighbors`	نعم	أسرع فرق معروف في NN-`k`
`sklearn.svm.SVC`	نعم	يعتمد على `libsvm` المجمّع
`sklearn.naive_bayes.MultinomialNB`	نعم	مصمَّم لبيانات `TF-IDF`
`sklearn.preprocessing.StandardScaler`	بشرط	إلزامي `with_mean=False`
`sklearn.cluster.KMeans`	لا	يرفض
`sklearn.ensemble.RandomForestClassifier`	لا	يرفض
`sklearn.neural_network.MLPClassifier`	لا	يرفض

هذا الجدول ليس عشوائياً: الفارق هو **هل تمرّ الخوارزمية على كلّ الخانات أم على القيم غير الصفرية فقط**.

وهناك طرف ثالث يتصرّف نيابةً عنك: `ColumnTransformer`. فهو يدمج مسارات قد يخرج أحدها متفرّزاً والآخر كثيفاً، ثم يقرّر نوع الناتج النهائي وحده.

كيف يقرّر ColumnTransformer أن يضغط النتيجة؟ python

```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler

set_config(transform_output="default")
rng2 = np.random.default_rng(0)
n = 600
wide = pd.DataFrame({f"cat{j}": rng2.choice([f"v{k}" for k in range(60)], n) for
j in range(40)})
for j in range(5):
    wide[f"num{j}"] = rng2.normal(size=n)
cat_cols = [c for c in wide.columns if c.startswith("cat")]
num_cols = [c for c in wide.columns if c.startswith("num")]

spec = [("num", StandardScaler(), num_cols), ("cat",
OneHotEncoder(handle_unknown="ignore"), cat_cols)]
for thr in [0.3, 1.0, 0.0]:
    Z = ColumnTransformer(spec, sparse_threshold=thr).fit_transform(wide)
    dens = Z.nnz / np.prod(Z.shape) if hasattr(Z, "nnz") else
np.count_nonzero(Z) / np.prod(Z.shape)
    print(f"  sparse_threshold={thr:<4} -> {type(Z).__name__:11s} shape=
{Z.shape} density={dens:.4f}")

Z = ColumnTransformer(spec).fit_transform(wide)
sparse_b = Z.data.nbytes + Z.indices.nbytes + Z.indptr.nbytes
print(f"\nلو ضُغطت: {sparse_b / 1024:.0f} لو كُثّفت   |   كيلوبايت: {Z.shape[0] *
Z.shape[1] * 8 / 1024**2:.1f} ميغابايت")
```

لاحظ الشرط الحاسم: **إذا كانت كثافة الناتج المدمج أعلى من العتبة، يُعاد `ndarray`**. ولهذا السبب لا يمكنك أن تأمر `sparse_threshold=1.0` وتنتظر ضغطاً مضموناً: كلّ كثافة تساوي واحداً أو تقلّ عنه تبقى متفرّزة.

ولنتحقّق من شرط `StandardScaler` عملياً، لأنه شرطٌ صامت:

الشرط الذي لا بدّ من معرفته: StandardScaler والمتفرّز python

```python
from sklearn import set_config
from sklearn.preprocessing import StandardScaler

# sparse output is incompatible with pandas containers, so go back to arrays
set_config(transform_output="default")

Xsp = sp.csr_matrix(np.array([[1.0, 0.0, 2.0], [0.0, 3.0, 0.0], [2.0, 0.0,
1.0]]))

try:
    StandardScaler().fit_transform(Xsp)
except Exception as exc:
    print("with_mean=True  ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:96])

out = StandardScaler(with_mean=False).fit_transform(Xsp)
print("with_mean=False ->", type(out).__name__, out.shape, "يعمل")
print("(القيم بعد التحويل (مصفوفة أصلاً كانت فيها أصفار):")
print(np.round(out.toarray(), 3))
```

لاحظ الجملة الأخيرة في رسالة الخطأ: `Cannot center sparse matrices`. ليست مشكلة ذاكرة بل مشكلة **دلالة**: طرح المتوسط يجعل الأصفار في المتّجه غير صفرية، فتصير صفراً بلا معنى. لذلك التطبيع بالتباين فقط.

٢.٦ sparray: الواجهة الجديدة التي تحتاج أن تعرفها

هنا change جوهري في `SciPy` ثم في `scikit-learn`. `SciPy` صارت تفرّق بين صنفين:

في `scikit-learn`، `sklearn` تبنّت الواجهة الجديدة تحت مفتاح إعدادات واحد:

```python
# نفس العملية، نوعان مختلفان من الإرجاع
from sklearn import set_config, get_config, config_context
from sklearn.preprocessing import OneHotEncoder

# enable the array-style container so sparse output is allowed at all
set_config(transform_output="default")

X_cat = np.array([["a"], ["b"], ["a"], ["c"]], dtype=object)
print("القيمة الافتراضية لـ sparse_interface:", get_config()["sparse_interface"])

for mode in ["spmatrix", "sparray"]:
    set_config(sparse_interface=mode)
    out = OneHotEncoder().fit_transform(X_cat)
    print(f"  sparse_interface={mode:8s} -> {type(out).__name__:11s} shape=
{out.shape}")

with config_context(sparse_interface="sparray"):
    print("  داخل config_context ->",
type(OneHotEncoder().fit_transform(X_cat)).__name__)
print("  بعد الخروج من السياق ->",
type(OneHotEncoder().fit_transform(X_cat)).__name__)
```

القاعدة العملية: **القيمة الافتراضية في ١.٩ ما زالت** `"spmatrix"`، ولذلك لن ينكسر كودك القديم. التبديل إلى `"sparray"` مشروع بالكامل، لكن افهم ما الذي تحرّك قبل أن تفعل.

ما تغيّر	مع `spmatrix`	مع `sparray`
صنف الإرجاع	`scipy.sparse.csr_matrix`	`scipy.sparse.csr_array`
دلالة الضرب `A @ B`	ضرب مصفوفي	ضرب نقطي-مصفوفي
`A.getformat()`	موجودة	غير موجودة
`sp.isspmatrix_csr(A)`	`True`	`False`
`sp.issparse(A)`	`True`	`True`
التوافق مع `scipy` **القديم**	كامل	يحتاج إصدارات حديثة

وهذه ليست تفاصيل تاريخية. ثلاث دوال فحص شائعة في كود الإنتاج — `getformat` و `isspmatrix_csr` و `A.ndim` — تعطي نتائج مختلفة تحت الواجهة الجديدة. اكتب `sp.issparse(A)` بدل `sp.isspmatrix_csr(A)` إن أردت كوداً ينجح في الحالتين.

```python
أين يختلف السلوك فعلاً؟
dense = np.array([[1.0, 0.0, 2.0], [0.0, 3.0, 0.0]])
m = sp.csr_matrix(dense)
a = sp.csr_array(dense)

print("csr_matrix.getformat() :", m.getformat())
print("csr_array.getformat()  :", hasattr(a, "getformat"), " (السمة .format =",
a.format, ")")
print("sp.isspmatrix_csr      :", sp.isspmatrix_csr(m), sp.isspmatrix_csr(a))
print("sp.issparse            :", sp.issparse(m), sp.issparse(a))
print("ndim / shape / nnz      :", (m.ndim, a.ndim), (m.shape, a.shape), (m.nnz,
a.nnz))
print("tocsr() تحافظ على الصنف:", type(m.tocsr()).__name__,
type(a.tocsr()).__name__)

set_config(sparse_interface="spmatrix")
```

٢.٧ متى ينكسر كودك عند التبديل؟

هناك مفاجأة واحدة يجب أن تعرفها قبل أن تفعّل `sparray` في مشروعك: إذا كان لديك محوِّل يُخرج بيانات متفرّزة سيرفض العمل.

```python exec="run" title="التضارب بين مخرجات pandas المتفرّزة والمخرجات"
from sklearn.preprocessing import OneHotEncoder

with config_context(transform_output="pandas"):
    try:
        OneHotEncoder().fit_transform(X_cat)
    except Exception as exc:
        print("pandas + متفرّز ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:110])

with config_context(transform_output="pandas"):
    ok = OneHotEncoder(sparse_output=False).fit_transform(X_cat)
    print("pandas + كثيف  ->", type(ok).__name__, ok.shape)
```

هذه ليست عطباً في التصميم بل قرار واضح: `DataFrame` من `pandas` لا تمثّل مصفوفة متفرّزة بشكلها الطبيعي. فإذا كنت تريد جدولاً مضغوطاً، اختر أحد الأمرين:

٢.٨ في الواقع: أين встреча المصفوفات المتفرّزة فعلاً؟

ثلاثة مواضع تتكرّر في كلّ مشروع إنتاجي تقريباً:

في الحالات الثلاث القاعدة نفسها: **لا تُكثّف أبداً ما لم يُجبرك على ذلك**.

> **في الواقع: الاستثناء الوحيد**
>
> هناك حالة واحدة يُنصح فيها بترك المتفرّز: عندما تجرّب ثلاث نماذج مختلفة على نفس الجدول وتريد مقارنة عادلة بينها. بعض النماذج ترفض `CSR`، وبعضها يقبلها. الحلّ ليس مواءمة كلّ النماذج على تمثيل واحد، بل wrapping التحويل داخل `Pipeline` كي لا يتسرّب أيّ معلومة من بيانات الاختبار.

تمارين الفصل

التمرين ٢.٤

ابنِ مصفوفة `1000 × 500` بكثافات `0.01` و `0.05` و `0.2` و `0.6`. لكلّ كثافة اطبع: `nnz`، وحجم `CSR` الحقيقي، وحجم `ndarray` المكافئ، والنسبة. ثم ارسم الخلاصة في جملة واحدة: عند أيّ كثافة تصبح المصفوفة المتفرّزة غير مجدية؟

التمرين ٢.٥

اعرض مصفوفة `csr_matrix` وأختها `csr_array` على نفس البيانات، ثم اطبع الفرق في: نوع الصنف، و `.format`، و `getformat`، و `sp.isspmatrix_csr`، و `sp.issparse`، ودلالة `A @ A.T`. فسّر أيّ فرق منها يهمّ في كود الإنتاج وأيّها تجريدٌ بلا أثر.

> **الحلّ**
>
> الفروق الجوهرية ثلاثة:
>
> `getformat()` نفسها اختفت من `csr_array`، فأيّ كود يمرّ عليها يجب أن يقرأ `.format` بدلها.

التمرين ٢.٦

خُذ `ColumnTransformer` فيه مسار رقمي وآخر `OneHotEncoder` على أربعين عموداً فئوياً. اطبع كثافة الناتج ونوعه. ثم كرّر الأمر مع `sparse_threshold=1.0` و `sparse_threshold=0.0`، وفسّر لماذا يعطي الحدّان نوعين مختلفين رغم ثبات الكثافة.

✅ الحلّ

القرار داخل `ColumnTransformer` هو: احسب كثافة النتيجة المدمجة، ثم إن كانت **أعلى** من `sparse_threshold` أعد `ndarray` كثيفاً، وإلا أعد `CSR`.

في المثال: الكثافة ٠٫٠١٩ تقريباً. والقاعدة المطبَّقة في المكتابة حرفياً هي:

القاعدة كما نُفّذت — python

```
self.sparse_output_ = density < self.sparse_threshold
```

فمع الافتراضي `0.3` تكون الكثافة **أقلّ** من العتبة فيُحفظ متفرّزاً. ومع `1.0` تبقى أقلّ من العتبة فيُحفظ متفرّزاً أيضاً. أمّا `0.0` فالكثافة أكبر من الصفر، فيُعاد `ndarray` كثيفاً.

المعنى العملي: `sparse_threshold` هو **الحدّ الأدنى من الكثافة الذي لا نريد بعده الضغط**. اضبطه على الرقم الذي تمثّل به «هذه المصفوفة لم تعد تستحقّ الضغط». لا على الصفر، ولا على واحد.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`StandardScaler` على `csr` بلا `with_mean=False`	`ValueError: Cannot center sparse matrices`	مرّر `False`
تفعيل `transform_output="pandas"` مع مخرجات متفرّرة	`ValueError: Pandas output does not support sparse data`	اجعل المخ `default` `ut=False`
استدعاء `.getformat()` على `csr_array`	`AttributeError`	استعمل تحقّق بـ `e`
`sp.isspmatrix_csr` كشرط قبول	`False` مع `csr_array` فيُرفض مصفوفة صالحة	استعمل
تمرير `csr` إلى `RandomForest`	`TypeError: dense data is required`	`barray()` البناء
`.toarray()` على مصفوفة ضخمة «لمعاينتها»	`MemoryError`	اطبع `barray()`
افتراض أن `CSR` دائماً أسرع	لا أثر في خوارزميات تمرّ على كل الخانات	قِس زمناً د التصميم
نسيان ضبط البذرة عند توليد متفرّر	نتائج غير قابلة للتكرار	`_state=0` `.random`

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مصفوفة متفرّقة	تخزّن القيم غير الصفرية فقط	sparse matrix
مصفوفة-مصفوفة	الواجهة المعيارية الحديثة في `SciPy`	sparse array, sparray
الصفوف المتفرّقة	صيغة `CSR` بثلاثة متّجهات	compressed sparse row
الإحداثيات	صيغة `COO` بقوائم صفّ وعمود	coordinate format
القوائم المترابطة	صيغة `LIL` للتعديل العنصر-بعنصر	linked list format
القيم غير الصفرية	عدد العناصر التي لا تساوي صفراً	nnz
الكثافة	نسبة الخانات غير الصفرية إلى الكل	density
عتبة التفرّز	الحدّ الذي يقرر عنده `ColumnTransformer` الضغط	sparse_threshold
واجهة المصفوفات	مفتاح `sparse_interface` في `sklearn`	sparse interface
سياق الإعدادات	نطاق مؤقت لتغيير الإعدادات	config_context

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٢ — أنواع البيانات: عددي، وفئوي، وزمني، ونصي

كيف يميّز المُقدِّر نوع كل عمود، وماذا يفعل إن أخطأت

الوحدة **scikit-learn ١.٩** الحجم **٨١٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تصنّف أيّ عمود إلى عددي أو فئوي أو زمني أو نصي بقرار مبرَّر
- أن تشرح لماذا يُفسد التحويل الموحّد على أعمدة مختلطة دون أن يرفع استثناءً
- أن تبني مساراً زمنياً حقيقياً بخصائص دورية داخل ColumnTransformer
- أن تستعمل make_column_selector لتوجيه كل عمود إلى مساره تلقائياً

٢.١ `dtype` ليس تفصيلاً تقنياً

افترض جدولَ اشتراكات فيه أربعة أعمدة: `age` و `city` و `signup` و `note`. لو أخبرتَ نموذجاً لوجيستياً أن يتعلّم منه، فماذا سيحدث؟

الجواب الذي يتوقّعه المبتدئ: سيتعلّم. الجواب الصحيح: **سيرفض**. لأن العمودين الثاني والرابع نصّيان، والخلاصة snail أن كل «عمود» ليس كياناً واحداً.

المشكلة الأعمق أن التحويل الذي يُنصح به بذكاء — «طبّع كل شيء» أو «رمّز كل شيء» — يُنتج نموذجاً يبدو صحيحاً ويعمل تماماً، بينما الدلالة تشوّهت. في الفصل ١٤ سنرى أن `StandardScaler` على عمود نصّي بعد رمزه ترتيبياً يخلق علاقة **مُختلَقة** بين «Riyadh» و«Doha» لا وجود لها في الواقع.

لذلك القاعدة التي نفصل بها كل شيء:

> **قاعدة الأربعة**
>
> لكل عمود ثلاثة أسئلة قبل أيّ تحويل: **١)** هل هو قياس كمّي أم رمز فئوي؟ **٢)** إن كان كمّياً، هل توزيعه متماثل أم مايل؟ **٣)** هل له معنى زمني متكرّر؟ الإجابات الثلاث تحدّد المسار، لا القيمة.

٢.٢ خريطة الأنواع الأربعة

النوع في `pandas`	مثال	الأداة المناسبة	الفصل
عددي كمّي	`age` و `income`	`StandardScaler` أو `RobustScaler`	١٤
فئوي (رمز)	`city` و `plan`	`OneHotEncoder` أو `OrdinalEncoder`	١٦
زمني	`signup`	`FunctionTransformer` أو استخراج يدوي	١٢
نصّي حر	`note` و وصف المنتج	`TfidfVectorizer`	١٨
منطقي	`active`	يُحوَّل إلى ٠ و ١ تلقائياً	١٢

لاحظ أن البُعد الخامس (المنطقي) هو الحالة الوحيدة التي لا تحتاج قراراً: `True` و `False` يتحوّلان إلى `1` و `0` تلقائياً في كل مُقدِّر رقمي.

لنبنِ الجدول النموذجي ونفحصه.

```python
جدول واحد بأربعة أنواع: ما الذي يراه pandas؟
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 200
frame = pd.DataFrame({
    "age": rng.integers(18, 80, n),
    "income": rng.lognormal(10, 1.1, n),
    "city": rng.choice(["Riyadh", "Cairo", "Amman", "Doha"], n),
    "plan": rng.choice(["free", "pro", "enterprise"], n),
    "signup": pd.to_datetime("2023-01-01") + pd.to_timedelta(rng.integers(0,
900, n), unit="D"),
    "note": rng.choice(["good service", "bad support", "fast delivery", "slow
app"], n),
    "active": rng.random(n) > 0.4,
})
print(frame.head(4).to_string(index=False))
print()
print(frame.dtypes)
print()
print(f"حجم الجدول: {frame.memory_usage(deep=True).sum():,} بايت")
```

لاحظ ثلاث تفاصيل تهمّ:

٢.٣ العددي: أخطاء تصمت ولا تصرخ

العمود العددي يبدو آمناً. هو الوحيد الذي لا يرفضه أحد. ومع ذلك يفسد النتائج بطرق لا تظهر في رسالة خطأ.

الخطأ الأول: مقياسان مختلفان بلا داعٍ. عمود عمر بين ١٨ و ٨٠، وعمود دخل بين ٧٠٠ و ٦٤٢ ألفاً. التطبيع القياسي يفعل الآتي:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

العمودان صارا متوسّطهما صفراً وتباعدهما واحداً. لكن قبل ذلك، لو استخدمت نموذجاً يستهلك مسافة مثل `KNeighborsClassifier`، لأصبح عمود الدخل وحده هو المسافة، لأنّ مداه أطول بمئة ضعف.

الخطأ الثاني: التوزيع مايل. الدخل الذي يتبع توزيعاً لوغاريتمياً يقع تحت المتوسط بثلاثة أضعاف الانحراف المعياري. متوسطه ٣٩ ألفاً، وكثافته تتركّز قرب الألف.

الخطأ الثالث: الشواذ. ثلاثة صفوف بأرقام فلكية تفسد كلّ إحصاء يعتمد على المتوسّط. هنا يفترق `StandardScaler` عن `RobustScaler` بصوت مسموع.

```python
# الرقم الشاذّ: أين يظهر في كلّ محوّل؟
from sklearn.preprocessing import StandardScaler, RobustScaler

num = frame[["age", "income"]]
print("قبل:  income   min %8.0f  max %10.0f" % (num["income"].min(),
num["income"].max()))

sc = StandardScaler().fit(num)
z = sc.transform(num).to_numpy()[:, 1]
print("z-score من StandardScaler : 2.% الوسيطf | 2.% الطرف الأعلىf" % (np.median(z),
z.max()))

polluted = num.copy()
polluted.loc[0:2, "income"] = polluted["income"].max() * 40
rz = RobustScaler().fit(polluted[["income"]])
rs = rz.transform(polluted[["income"]]).to_numpy()[:, 0]
sz =
StandardScaler().fit(polluted[["income"]]).transform(polluted[["income"]]).to_nu
mpy()[:, 0]
print("مع الشواذ: Robust   7.0% الوسيطf  9.1% الطرفf" % (rz.center_[0], rs.max()))
print("مع الشواذ: Standard 7.0% الوسيطf  9.1% الطرفf" % (
    StandardScaler().fit(polluted[["income"]]).mean_[0], sz.max()))
```

لاحظ الرقم الأخير في المخرجات. `StandardScaler` ضرب المتوسّط من ٣٩ ألفاً إلى أكثر من ٤٠٠ ألف بسبب ثلاثة صفوف فقط، أي أنّ «مركز البيانات» صار تعريفاً لهذين الصفوف الثلاثة لا لعمود الدخل. `RobustScaler` ببساطة استخدم الوسيط والمدى الربيعي، فبقي مركزه قرب ١٩ ألفاً. **هذه ليست مسألة «أيّهما أفضل»، بل مسألة أيّهما يطابق السؤال الذي تسأله عن القيم الشاذّة.**

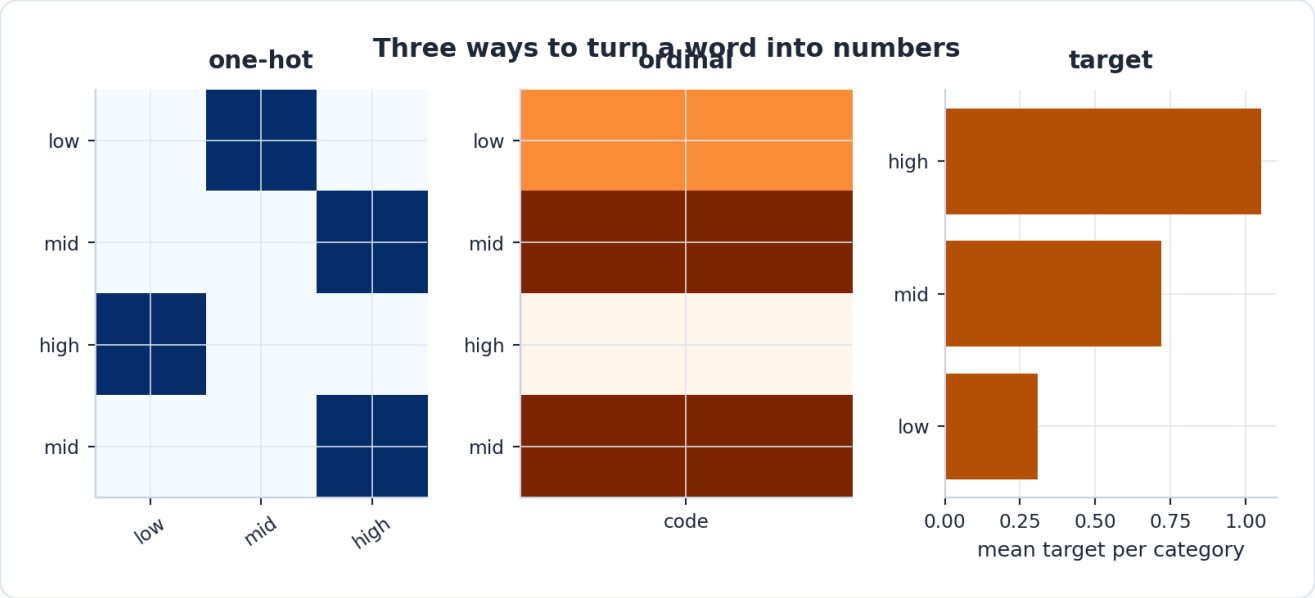
٢.٤ الفئوي: لماذا يرفضه كل مُقدِّر رقمي؟

المدينة «Riyadh» ليست عدلاً أكبر ولا أصغر من «Cairo». الرقم ٣ لا يعني شيئاً أكثر من ١. لكن `LogisticRegression` لا يعرف هذا، فلو سمحتَ له برمز ٠ و١ و٢ فسيبني حدّاً يفترض فيه ترتيباً لم تخترعه أنت.

لذلك هناك طريقتان عادلتان:

الترميز	كم عموداً يُنتج	ماذا يعني الرقم	متى تختاره		
`OneHotEncoder`	`C`	لكل فئة `1`	`0` أم ١، بلا ترتيب		الفئات بلا ترتيب طبيعي
`OrdinalEncoder`	١	ترتيب مصطنع	الفئات **لها** ترتيب حقيقي		

مدن الجدول الأربع، لو رمّزتها بـ `OrdinalEncoder`، لأصبحت `Doha = 0` و `Riyadh = 3`. أيّ شجرة قرار تتعلّمها ستقول إنّ الدوحة قريبة من الرياض. هذا ليس خطأ برمجياً بل خطأ في وصف المشكلة.



الشكل ٢٨ثلاث لوحات لترميز الفئات: OneHotEncoder يبني ثلاثة أعمدة ثنائية، وOrdinalEncoderg يبني عموداً واحداً بترتيب مخترع، وترميز الهدف يختزل الفئة في متوسّط التسمية

`OrdinalEncoder` ليس شرّاً مطلقاً. إذا كانت فئاتك مرتّبة فعلاً — `low` و `medium` و `high` — فالترميز الترتيبي مقصود ومحلل. الشرط الوحيد أن يكون الترتيب موجوداً في بياناتك، لا في خيالك.

الترميز على الأعمدة الفئوية python

```python
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

cats = frame[["city", "plan"]]

ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(cats)
print(f"OneHotEncoder -> {ohe.transform(cats).shape[1]} أعمدة")
print("  الأسماء:", list(ohe.get_feature_names_out()))

oe = OrdinalEncoder(handle_unknown="use_encoded_value",
unknown_value=-1).fit(cats)
print(f"OrdinalEncoder -> {oe.transform(cats).shape[1]} أعمدة")
print("  الفئات:", [c.tolist() for c in oe.categories_])
print("  أول ٦ قيم:", oe.transform(cats).to_numpy()[:6, 0].ravel().astype(int))

rare = pd.DataFrame({"city": ["Riyadh"] * 60 + ["Cairo"] * 20 + ["Amman"] * 12 +
["Doha"] * 8})
print("\nتوزيع المدينة قبل التجميع:", rare["city"].value_counts().to_dict())
folded = OneHotEncoder(min_frequency=15, sparse_output=False,
                       handle_unknown="infrequent_if_exist").fit(rare[["city"]])
print("بعد min_frequency=15:", list(folded.get_feature_names_out()))
plain = OneHotEncoder(sparse_output=False).fit(rare[["city"]])
print("بدون min_frequency :", len(plain.get_feature_names_out()), "أعمدة")
```

لاحظ السطر الأخير: الفئات النادرة لم تُحذف، بل **طُويت** في عمود واحد اسمه `infrequent_sklearn`. هذه الخاصية اسمها `min_frequency` وهي متاحة في ١.٩، وتحميك من انفجار الأبعاد حين يكون أحد أعمدتك ألف قيمة مختلفة. جرّب أن ترفعها إلى ٣٠ لتتوقّف عن فائدة العمود الأخير أيضاً.

٢.٥ الزمني: التاريخ ليس رقماً... وأحياناً يكون

عمود `signup` يخزّن `datetime64`. لا يقبله `StandardScaler`، ولا يقبله `OneHotEncoder`، ولا يقبله أيّ شيء آخر. لماذا؟ لأنّ له معنى دوري: الفرق بين كانون الثاني وأكتوبر ٨٩ يوماً، لا ٣٠٠.

ثلاث طرق للتعامل معه، من الأسهل إلى الأدقّ:

$$s_k = \sin\left(\frac{2\pi k}{p}\right), \qquad c_k = \cos\left(\frac{2\pi k}{p}\right)$$

حيث `k` رقم الدورية و `p` طولها: ١٢ للشهر، و ٧ لأيام الأسبوع، و ٢٤ للساعة.

```python
# من تاريخ إلى خمسة أرقام ذات معنى
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import FunctionTransformer


def date_parts(d):
    s = d.iloc[:, 0] if hasattr(d, "iloc") else d
    month = s.dt.month
    return pd.DataFrame({
        "year": s.dt.year,
        "month": month,
        "dow": s.dt.dayofweek,
        "month_sin": np.sin(2 * np.pi * month / 12),
        "month_cos": np.cos(2 * np.pi * month / 12),
    })


ct_dt = ColumnTransformer([("dt", FunctionTransformer(date_parts), ["signup"])])
print("مخرجات مسار الزمن:")
print(ct_dt.fit_transform(frame.head(5)).round(3))
print()
print("أسماء المخرجات:", list(ct_dt.get_feature_names_out()))
```

لاحظ أن `FunctionTransformer` يمكنه أن يفعل أيّ شيء: استخراج مكوّنات، أو حساب نسبة، أو تحويل وحدة. هو الجسر بين `pandas` الدلالي وعالم `scikit-learn` العددي. والفصل ١٧ يتعمّق في بناء تحويلات relatives من هذا النوع.

> **لماذا `d.iloc[:, 0]` داخل الدالة؟**
>
> لأن `ColumnTransformer` يمرّر للـ `FunctionTransformer` إطاراً بعمود واحد (شكله `n × 1`)، بينما `TfidfVectorizer` يتوقّع متّجهاً نصّياً. تسمية `iloc` تخلّصك من هذا الاختلاف بين مسارات المشروع الواحد. الدوال المكتوبة لتتحمّل كلا الشكلين أرشيفٌ في كلّ مشروع إنتاجي حقيقي.

٢.٦ النصّي: عمودٌ واحد أم مليون مستند؟

هناك فرق جوهري بين عمود نصّي **فئوي** (القيمة منه عدد محدود: `satisfied` و `neutral` و `angry`) وعمود نصّي **حرّ** (القيمة من فضاء مفتوح: ملاحظات عملاء، وصفحات ويب، تغريدات). الأول يعالَج بـ `OneHotEncoder` في seconds. الثاني يحتاج `TfidfVectorizer` الذي يحوّل كلّ مستند إلى متّجه آلاف الأبعاد — موضوع الفصل ١٨ بالكامل.

```python
عمود نصّي حرّ: من جملة إلى متّجه
from sklearn.feature_extraction.text import TfidfVectorizer

tv = TfidfVectorizer(max_features=4)
M = tv.fit_transform(frame["note"])
print("نوع المخرجات:", type(M).__name__, " الشكل:", M.shape)
print("المفردات:", list(tv.get_feature_names_out()))
print("\nأول ٤ صفوف كمصفوفة كثيفة:")
print(pd.DataFrame(M.toarray(),
columns=tv.get_feature_names_out()).head(4).round(3).to_string(index=False))
```

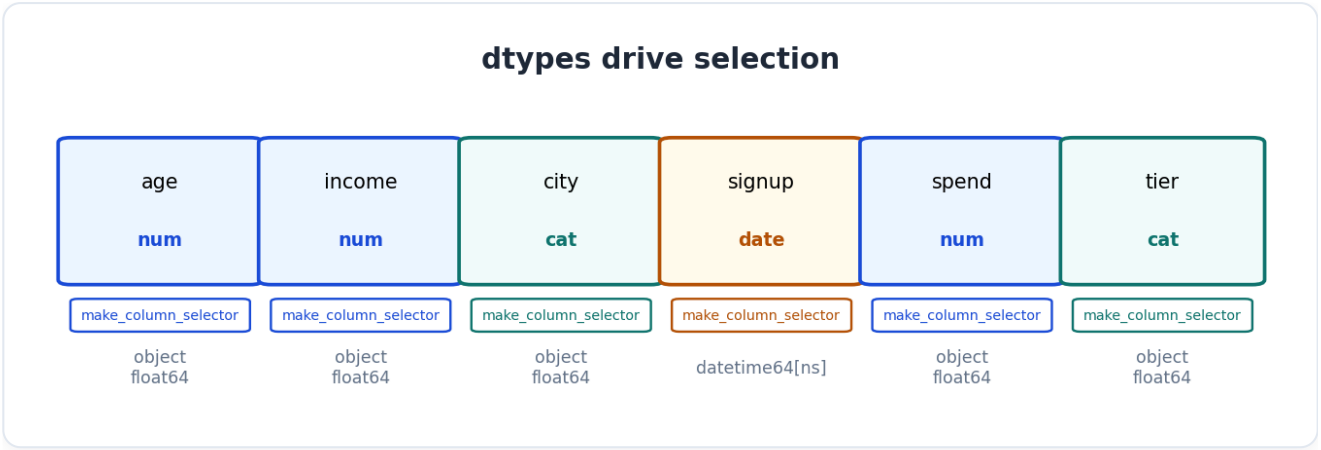
<!----->

وهنا تحذير عملي وكثير الوقوع: **لا تضع** `TfidfVectorizer` **داخل** `ColumnTransformer` **في ١.٩**. السبب أنّه يتوقّع متّجهاً نصّياً واحداً، بينما `ColumnTransformer` يسلّمه إطاراً بعمود واحد، فيخرج المخرج بشكل غير صحيح. الحلّ: عالِج النصّ في `Pipeline` منفصل، أو استقبل عموداً واحداً قبل أن يمرّ. لو فعّلت `transform_output="pandas"` على مستوى الجلسة، يظهر خطأ آخر هنا أيضاً: المتّجه المتفرّر لا يمكن أن يُغلَّف في `DataFrame`.

الحالة	الأداة	الإخراج
عمود فئوي محدود القيم	`OneHotEncoder`	أعمدة ١/٠
عمود فئوي مرتّب فعلاً	`OrdinalEncoder`	عمود أرقام
نصّ حرّ داخل جدول	`TfidfVectorizer` مستقلّاً	مصفوفة `csr`
نصّ في مشروع كامل	`Pipeline` مستقلّ	نموذج كامل

٢.٧ `make_column_selector`: تفويض القرار للنوع

هل يمكن ترك المكتبة تختار لكل عمود مساره؟ نعم، وهذه أداة `make_column_selector` من `sklearn.compose`. هي **دالة تُعيد دالة**: تستقبلها `ColumnTransformer`، وتُستدعى وقت `fit` فتقرأ `dtype` لكل عمود وتعيد قائمة أسماءه.



الشكل ٢٩رسم يوضّح أن المُحدِّد يعتمد على نوع العمود لا على اسمه، فيوجّه الأرقام والمنطقي والنصوص إلى مسارات مختلفة

دالة تُعيد دالة: من قراءة الأنواع إلى مسارين python

```python
from sklearn.compose import make_column_selector
from sklearn.preprocessing import OneHotEncoder, StandardScaler

for kw in [dict(dtype_include="number"), dict(dtype_include="string"),
           dict(dtype_include="datetime"), dict(dtype_include=bool),
           dict(dtype_exclude="object"), dict(pattern="^(city|plan)$")]:
    print(f"  {str(kw):45s} -> {make_column_selector(**kw)(frame)}")

print("\n--- والآن نحوّل هذه القراءة إلى خطّ فعلي ---")
ct_auto = ColumnTransformer([
    ("num", StandardScaler(), make_column_selector(dtype_include="number")),
    ("cat", OneHotEncoder(sparse_output=False, handle_unknown="ignore"),
     make_column_selector(dtype_include="string", pattern="^(city|plan)$")),
])

Z = ct_auto.fit_transform(frame)
print("نوع الناتج:", type(Z).__name__, " الشكل:", Z.shape)
print("أعمدة المسار الرقمي :", [c for c in ct_auto.get_feature_names_out() if
c.startswith("num")])
print("أعمدة المسار الفئوي :", [c for c in ct_auto.get_feature_names_out() if
c.startswith("cat")])
print("أعمدة مُهمَلة بصمت   :", ["note", "signup", "active"])
```

لاحظ أن `dtype_include="object"` و `dtype_include="string"` يُنتجان **النتيجة نفسها** في `pandas 3.0`، لأن أعمدة `str` الجديدة تُعامَل على أنها `object` من قبل أداة الاختيار. هذه تفاصيل تُكتشف بالتجربة لا بالتوثيق، فجرّب دائماً المُحدِّد على بياناتك قبل أن تسلّمه خطّ الأنابيب.

هذا هو النمط الذي ستلتقي به في كلّ مشروع إنتاج: **جدول واحد، مسارات متعددة، وقرار التوجيه لكل عمود مبنيّ على نوعه لا على اسمه**. وأسماء المسارات (`num` و `cat`) تظهر في `get_feature_names_out` على شكل `num__age` و `cat__city_Amman`، فهي هويّة كلّ خاصية في تقريرك.

الخطأ: الاعتماد على `remainder='drop'` دون قصد

في المثال أعلاه، الأعمدة `note` و `signup` و `active` لا تنتمي إلى أيّ مسار، فتسقط بصمت لأن `remainder` قيمته الافتراضية `'drop'`. النموذج يعمل، ودقّته مقبولة، ولا أحد يعرف أن نصف البيانات لم يُستخدم. **عند إنشاء `ColumnTransformer` أول مرة، اضبط `remainder='passthrough'` أو `'drop'` عمداً، وطبع `get_feature_names_out()` لتتأكّد.**

٢.٨ في الواقع: العمود «الرقمي» الذي فيه نص

أخطر ما في هذا الفصل ليس الفئوي ولا الزمني، بل ما يبدو رقمياً ولا يكون كذلك.

في الأنظمة الإنتاجية، شائع جداً أن يقرأ أحدهم ملفاً من نظام قديم فيصل عمود `quantity` بنوع `str` لأن أحد الصفوف احتوى `"unknown"` أو مسافة في آخره. النتيجة: النموذج يُدرَّب، والتحذير يصمت، والدقّة تهبط بنسبة لا يعرف أحد سببها.

القاعدة المهنية التي تحميك:

كيف تكشف عموداً رقمياً متسلّلاً؟ python

```
raw = pd.DataFrame({"quantity": ["12", "7", "unknown", "30", " 5 "]})
print("النوع الأصلي:", raw["quantity"].dtype)
clean = pd.to_numeric(raw["quantity"], errors="coerce")
print("بعد to_numeric:", clean.dtype, "| قيم NaN:", int(clean.isna().sum()))
print("لماذا هذا أفضل من الخطأ الصامت؟ لأن الصف الثالث صار معلومة مفقودة")
print("في صمت أو يُرفض المشروع كله NaN يمكن دراستها، بدل أن تتحوّل إلى.")
print()
print(raw.assign(quantity=clean).to_string(index=False))
```

في الواقع: عقد البيانات

في الفرق التي تعمل بانتظام، يُكتب «عقد بيانات» (data contract) يحدّد لكل عمود: النوع المسموح، والوحدة، وماذا يعني الفراغ، ومن يملك صلاحية تغييره. هذا ليس ترفاً تنظيمياً: هو ما يمنع أن ينهار نموذج في يوم بسبب عمود أضافه زميل في جدول التسعير. ثلاثة أسئلة تكفي لبداية عملية: ما نوع العمود؟ ما وحدة قيمه؟ ما معنى الفراغ فيه؟

تمارين الفصل

التمرين ٢.٧

أنشئ جدولاً فيه ستّة أعمدة: عدد صحيح، عدد عشري، رمز فئوي، تاريخ، نصّ حرّ، ومنطقي. اطبع `dtypes` ثم استخدم `make_column_selector` بأربع طرق مختلفة وعدّد الأعمدة التي يختارها كلّ مرة. اربط كل نتيجة بقاعدة من القواعد الأربعة.

التمرين ٢.٨

خذ عمود `month` من القسم ٢.٥ وقارن ثلاث تمثيلات: `(أ)` الرقم الخام من ١ إلى ١٢، `(ب)` الترميز الدوري بـ `sin` و `cos`، `(ج)` `OneHotEncoder` باثني عشر عموداً. احسب المسافة الإقليدية بين ديسمبر ويناير في كلّ حالة، ولاحظ أيّها يضعهما متجاورين وأيّها يضعهما متقابلين.

> ✅ **الحلّ**
>
> في تمثيل `(أ)` المسافة بين ١ و١٢ تساوي ١١ — أبعد ما يمكن. في تمثيل `(ب)` المسافة بين النقطتين على الدائرة تساوي:
>
> $$d = \sqrt{(s_1 - s_2)^2 + (c_1 - c_2)^2} = 2\sin\left(\frac{\pi |k_1 - k_2|}{p}\right)$$
>
> ولأن `|1 - 12| = 11` و `p = 12`، فإنّ `d = 2 sin(11\pi/12) \approx 0.518` — أي قريبان جداً على الدائرة، لأن ديسمبر يسبق يناير مباشرةً. أمّا تمثيل `(ج)` فيضعهما في عمودين مختلفين تماماً، فالمسافة بينهما `\sqrt{2} \approx 1.414`.
>
> الخلاصة: التمثيل الدوري يحترم الدورية الطبيعية، والرقمي يقطعها بلا داعٍ.

التمرين ٢.٩

خذ `frame` من هذا الفصل، وأضف عموداً نصّياً فيه قيمة واحدة `"unknown"` وسط مئة صفّ رقمي. طبّق `pd.to_numeric(errors="coerce")` ثم `SimpleImputer`، وقارن عدد القيم الفاقدة قبل وبعد. احسب نسبة الصفوف المتأثّرة، وقرّر: هل هذا عمود Numeric سليم أم عيب في المصدر؟ برّر قرارك برقم لا بجملة.

✅ **الحلّ**

النسبة هنا `1/100 = 1%` بالضبط، وهي فوق عتبة الـ `1%` التي ذكرها الفصل. لذلك القرار ليس «عالجها وأكمل» بل «ارجع إلى مصدرها». عمود يحتوي قيمة واحدة غير رقمية من مئة غالباً يعني أن أحدهم كتب `"N/A"` أو صفراً بالخطأ، وهذا نمط قد يتكرّر في مصادر أخرى من نفس النظام.

الفصل ١٣ يشرح كيف تعالج القيم الفاقدة بعد اكتشافها، لكن الاكتشاف نفسه يبقى مهمّة هندسة بيانات لا مهمّة `fit`.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`StandardScaler` **على كل الأعمدة**	كسر للعلاقات الفئوية والترتيب الزائف	مسار منفصل لكل نوع
`OrdinalEncoder` **على فئة بلا ترتيب**	النموذج يتعلّم مسافة لم توجد في البيانات	`OneHotEncoder` أو `TargetEncoder`
تمرير عمود تاريخ إلى `fit`	رفض أو معاملة كرقم هائل	`FunctionTransformer` لتفكيكه
`pd.to_datetime` **بلا** `errors`	خطأ على أول قيمة غريبة	`errors="coerce"` ثم عالج `NaN`
`TfidfVectorizer` **داخل** `ColumnTransformer`	مخرجات بشكل خاطئ	`Pipeline` منفصل للّص
`MinMaxScaler` **مع شواذ كبيرة**	كل القيم تنضغط في نطاق ضيّق	`RobustScaler` أو quantile
الثقة بأنّ `remainder` **أبلغك**	أعمدة تُسقط بصمت	اطبع `get_feature_names_out`
`make_column_selector` **بلا اختبار**	اختيار غير متوقّع لأنواع الجديدة `str`	جرّبها على بياناتك قبل الاعتماد
عمود «رقمي» فيه نص	coercing صامت أو رفض مبهم	`to_numeric(errors="coerce")`

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
نوع البيانات	طبيعة العمود كما يقرأها `pandas`	dtype
كمّي	قابل للقياس والمست continuous	quantitative
فئوي	رمز من مجموعة قيم محدودة	categorical
رمزي	يُخزَّن كنصّ لا كعدد	string dtype
زمني	تاريخ أو وقت	datetime
الترميز الدوري	تمثيل الدورة بـ `sin` و `cos`	cyclical encoding
الحدّ الكاذب	قطع متّصل في حقيقة دورية	false boundary
توزيع مايل	ذيل ثقيل من جهة	skewed distribution
الشواذ	قيمة بعيدة جداً عن الباقي	outlier
مُحدِّد الأعمدة	أداة تختار الأعمدة حسب نوعها	make*column*selector
المسار	فرع معالجة لنوع أعمدة	pipeline branch

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٣ — القيم المفقودة: من الفراغ إلى المعلومة

MissingIndicator g IterativeImputer g KNNImputer g SimpleImputer

الوحدة **scikit-learn ١.٩** الحجم **٧٨٧** كلمة تقريباً

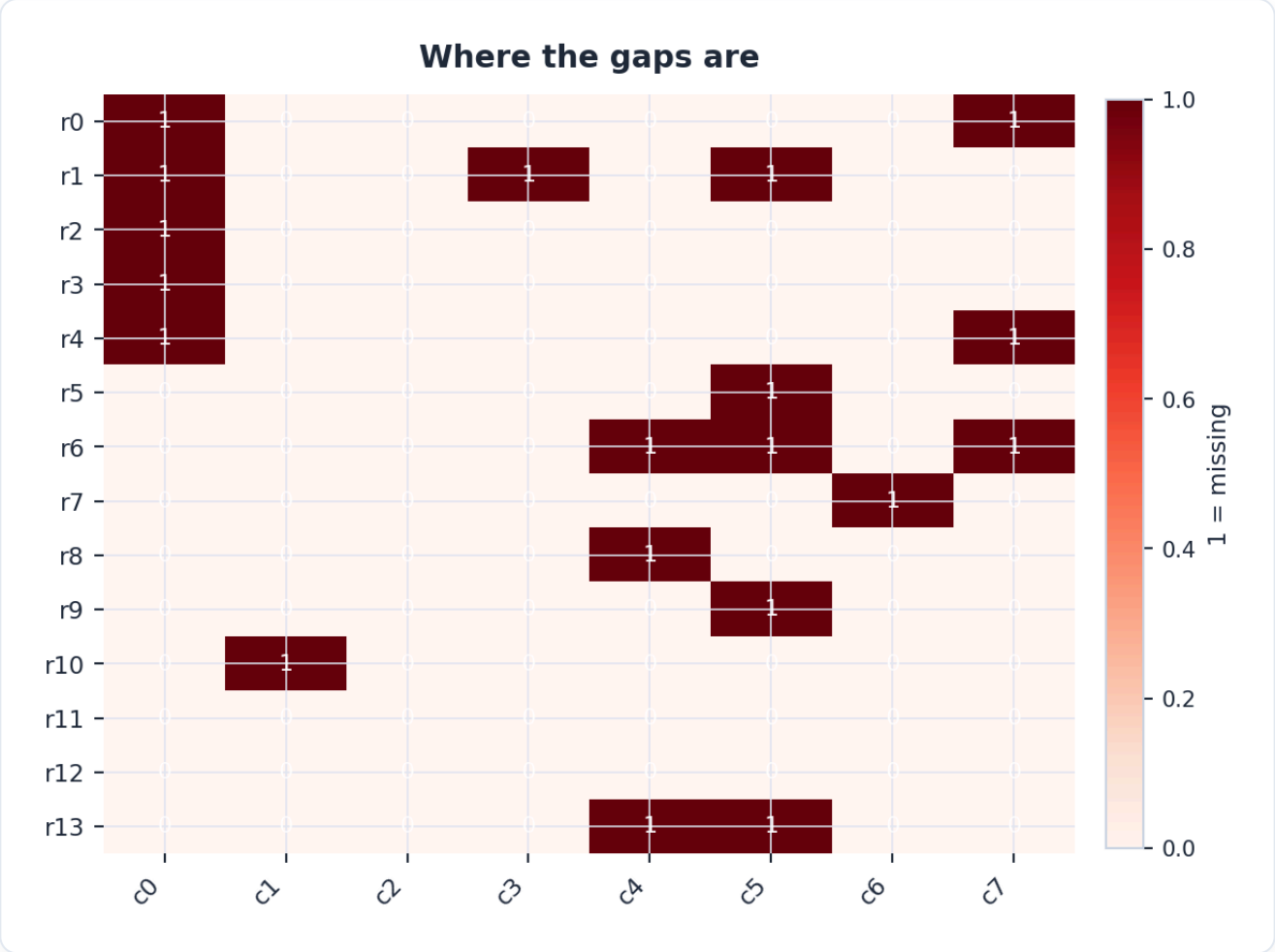
ما الذي ستتقنه في هذا الفصل

- أن تميّز بين MCAR و MAR و MNAR وتشرح أثر كلّ نوع على نموذجك
- أن تختار استراتيجية التعبئة بناءً على نوع العمود لا على عادة
- أن تشرح لماذا تُضعف التعبئة بالمتوسّط التباين وبأي نسبة
- أن تقيس أثر كلّ استراتيجية على بيانات حقيقية بدل التخمين

٢.١ ثلاثة أسئلة قبل أن تملأ الفراغ

حين ترى `NaN` في جدولك، أوّل ردة فعل هي: «سأستبدله بالمتوسّط». هذه ليست خطأً، لكنها **قرار تُتخذه دون أن تفحص**. قبل أن تمتلئ الخانات، أجب عن ثلاثة أسئلة:

الفرق بين «الفراغ لا معنى له» و«الفراغ معنى» هو الفرق بين حذف معلومة وإضافة معلومة. وهذا الفصل كلّه موضوعه.



الشكل ٣٠خريطة حرارية تُظهر أين تقع الفجوات في جدول: أعمدة ناقصة متفرّقة، وصفوف فيها فجوتان معاً، ولا صفوف كاملة الفقد

لاحظ في الشكل أعلاه شيئاً لا يظهر في جدول `isna().sum()`: **نمط** الفجوات. صفّ فيه فجوتان في عمودين مختلفين يعني غالباً أنّ السجل فُشل استخراجه، وهذه معلومة من نوع مختلف عن فجوة واحدة عابرة.

٢.٢ أنواع الفراغ الثلاثة

الإحصاء التقليدي يصنّف غياب البيانات إلى ثلاثة أنواع، والفرق بينها ليس تشريفياً بل **حاسم**:

النوع	التعريف	ماذا تفعل التعبئة
MCAR	الفجوة لا علاقة لها بأيّ متغيّر	تعبئة بالمتوسّط مقبولة نظرياً
MAR	الفجوة تعتمد على متغيّرات **مرصودة**	نموذج يمكنه استنتاج الفجوة
MNAR	الفجوة تعتمد على المتغيّر المفقود نفسه	لا حلّ ساذج؛ يلزم نموذج

MAR هي الحالة الوحيدة التي يمكن فيها تعويض الفجوة بأدقّية معقولة. والمعادلة التي تعبر عنها:

$$\text{MAR}: \quad p(Y|X, M) = p(Y|X_{\text{obs}})$$

حيث `M` متّجه مؤشّرات الفجوة، و `X_obs` القيم المرصودة. المعنى: بمعرفة مَن كانت قيمته مفقودة، تصبح القيمتين المتبقّيتين لا تحملان أيّ معلومة إضافية عن الهدف.

هذا الاسم ليس ترفاً. جدول Clinical Trial حيث «لم يُسجَّل الوزن» لأنّ المريض كان طويلاً بدرجة استثنائية — هذا MNAR، ولا نموذج في `scikit-learn` سيحلّه تلقائياً.

٢.٣ كيف يقيس `scikit-learn` الفراغ؟

المكتبة لا تفترض معنى واحداً لـ `NaN`. المتّجه `np.nan` هو القيمة الافتراضية، لكن يمكنك إعلان قيمة أخرى عبر `missing_values` في كلّ محوّل تعبئة.

أول صورة صادقة عن الفجوات، ثم محاكاة فراغ من نوع MAR python

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.impute import SimpleImputer

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

rng = np.random.default_rng(0)
holes = rng.random(df.shape) < 0.08
df[holes] = np.nan

print("عدد الفجوات في كل عمود:")
print(df.isna().sum().to_string())
print()
print(f"نسبة الفجوات الكلية      : {df.isna().to_numpy().mean():.3f}")
print(f"صفوف فيها فجوة واحدة على الأقل: {int(df.isna().any(axis=1).sum())} من
{len(df)}")
print(f"صفوف فيها فجوتان أو أكثر: {int((df.isna().sum(axis=1) >= 2).sum())}")
print(f"صفوف كاملة الفقد         : {int((df.isna().sum(axis=1) ==
df.shape[1]).sum())}")

print("\n--- الآن اصنع فجوة من نوع MAR: احفرها داخل الصنف الثالث فقط ---")
mar = iris.data.copy()
mar[np.where(iris.target == 2)[0], 2] = np.nan
print(f"نسبة الفجوة في العمود الثالث: {np.isnan(mar).mean():.3f}")

imputed = SimpleImputer(strategy="mean").fit_transform(mar).to_numpy()
print("عدد NaN بعد التعبئة بالمتوسّط:", int(np.isnan(imputed).sum()))

before = [iris.data[iris.target == k, 2].mean() for k in range(3)]
after = [imputed[iris.target == k, 2].mean() for k in range(3)]
print("\nمتوسط طول البتلات لكل صنف:")
print("  قبل التعبئة:", np.round(before, 3).tolist())
print("  بعد التعبئة:", np.round(after, 3).tolist())
print("  الفارق      :", np.round(np.array(after) - np.array(before), 3).tolist())
```

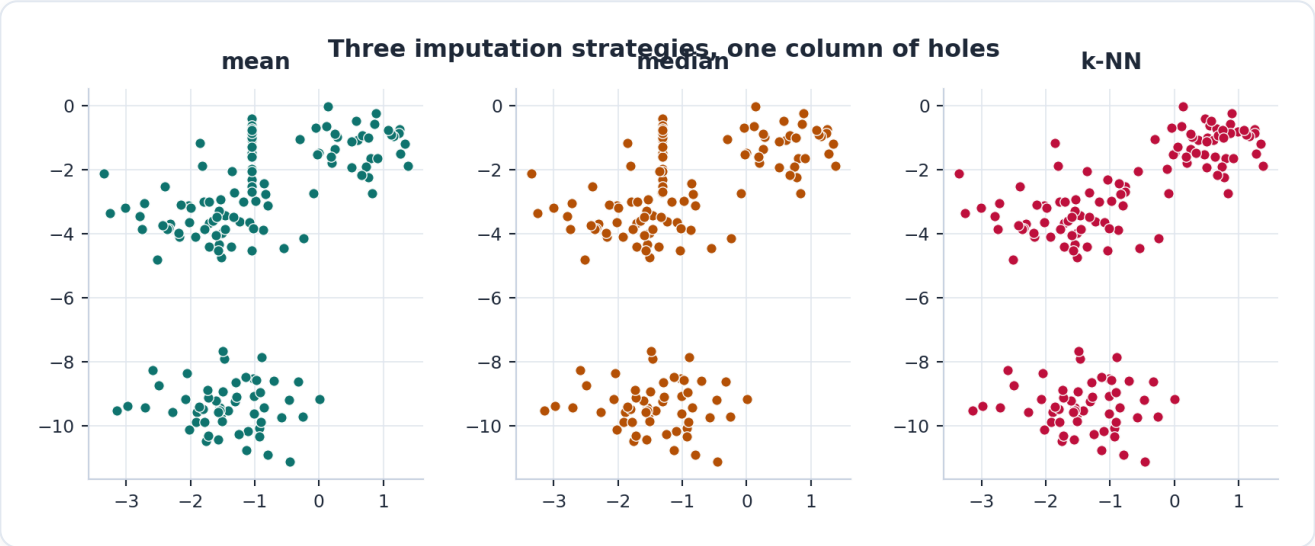
لاحظ الصفّ الثالث في المخرجات الأخيرة: متوسطه انخفض من ٤٫٢٦ إلى ٣٫٤٠. التعبئة بالمتوسّط **دمجت** صنفاً كاملاً في مركز البيانات. هذا ليس خطأً برمجياً بل نتيجة مباشرة لافتراض أنّ الفراغ عشوائي، وهو افتراض خاطئ هنا لأننا نحن من صنع هذه الفجوات.

ولاحظ أيضاً احترام النمط في المخرج الأولى: صفّ واحد فقط لم يُسجَّل له أيّ قياس، بينما ثمانية صفوف فيها فجوتان معاً. هذا هو الفارق بين «بيانات ناقصة» و«سجلّ فشل».

٢.٤ `SimpleImputer` : أربع استراتيجيات

الأداة الأولى هي `sklearn.impute.SimpleImputer` . تقبل أربع استراتيجيات، وكلّها تعمل على النحو نفسه: تحسب قيمة بديلة لكل عمود في وقت `fit` ، ثم تطبّقها في `transform` .

الاستراتيجية	القيمة البديلة	تُستخدم مع
`mean`	المتوسّط الحسابي	أعمدة كمّية متماثلة
`median`	الوسيط	أعمدة كمّية مايلة أو فيها شواذ
`most_frequent`	أكثر قيمة تكراراً	أعمدة فئوية أو كمّية مقطّعة
`constant`	قيمة ثابتة تحدّدها أنت	فئات «غير معروف» مقصودة



الشكل ٣١ثلاث لوحات مقارنة بين ثلاث استراتيجيات تعبئة على العمود نفسه: المتوسّط والوسيط وقيمة ثابتة، مع إظهار أثر كلّ منها على موقع الصفوف المفقودة

لنرَ الأثر على مصفوفة صغيرة تراها بعينك.

أربع استراتيجيات، مصفوفة واحدة python

```python
Xm = np.array([[1.0, 2.0],
               [np.nan, 3.0],
               [4.0, np.nan],
               [5.0, 6.0],
               [np.nan, np.nan]])

print("المصفوفة الأصلية:")
print(Xm)
for st in ["mean", "median", "most_frequent"]:
    s = SimpleImputer(strategy=st).fit(Xm)
    print(f"\n{st}: القيم البديلة = {np.round(s.statistics_, 3).tolist()}")
    print(s.transform(Xm).to_numpy())

s_const = SimpleImputer(strategy="constant", fill_value=-999).fit(Xm)
print("\nconstant=-999:")
print(s_const.transform(Xm).to_numpy())
```

لاحظ سطر `most_frequent` عن قرب. القيم البديلة كانت `1.0` و `2.0`: أي **قيمة موجودة فعلاً في العمود**، لا معدّلة. هذا ما يجعل الاستراتيجية صالحة للأعمدة الفئوية. لكن انتبه: `most_frequent` على عمود كمّي متّصل يختار رقماً واحداً ويكرّره، فيخلق قمماً وهمية عند ذلك الرقم بالضبط.

٢.٥ `KNNImputer` و `IterativeImputer`: من الجيران ومن النموذج

إذا كان الفراغ من نوع MAR، فإن **أفضل قيمة بديلة هي القيمة الحقيقية نفسها**، وهي غير مرصودة. لكن يمكن تقريبها: قيم الجيران في فضاء الخصائص تشترك في نمط مشابه.

`KNNImputer` يفعل هذا بالحرفيّة: يبحث عن `n_neighbors` صفّاً يشبه الصفّ المفقود في بقية أعمدته، ويأخذ متوسّطها في العمود الناقص.

`IterativeImputer` أذكى: يدرّب نموذجاً انحداراً يتنبّأ بالعمود الناقص من بقية الأعمدة، ثم يكرّر. ومع كلّ دورة تتحسّن التنبّؤات.

بوابة تجريبية في ١.٩

ما زال `IterativeImputer` خلف بوابة تجريبية في هذا الإصدار، ويجب تفعيلها قبل لمس صنفه. نسيان هذا السطر يسبّب `ImportError` لا يشرح سببه. والصيغة المعتمدة:

```python
# الصيغة الصحيحة في 1.9
import sklearn.experimental.enable_iterative_imputer  # noqa: F401 Side effect only
from sklearn.impute import IterativeImputer           # الآن ينجح
```

الاستيراد أعلاه يعمل بآثاره الجانبية: الوحدة تُنفَّذ عند الاستيراد فتعرّف الصنف في `sklearn.impute`. ولهذا السبب تفشل صيغة `from ... import enable_iterative_imputer` أمام أدوات الفحص الساكنة: الصنفليس عضواً في فحص أسماء الوحدات يسأل عن **صنف** داخل `sklearn.experimental`، وهو ليس كذلك.

```python
# ثلاث أدوات، ثلاث مسارات ذهنية مختلفة
from sklearn.impute import KNNImputer
import sklearn.experimental.enable_iterative_imputer  # noqa: F401

IterativeImputer = sklearn.impute.IterativeImputer

print("KNNImputer (الجيران):")
print(np.round(KNNImputer(n_neighbors=2).fit_transform(Xm).to_numpy(), 3))

print("\nIterativeImputer (نموذج يتكرّر):")
it = IterativeImputer(max_iter=25, tol=1e-3, random_state=0)
print(np.round(it.fit_transform(Xm).to_numpy(), 3))
```

انظر إلى الصفّ الأخير في المصفوفة `[nan, nan]`: القيمتان الناقصتان معاً. المتوسّط يعطي `3.33` و`3.67` لأنهما متوسّط العمودين، ويعطي `KNNImputer` النتيجة نفسها لأنّه لا يعرف أبعد من ذلك. أمّا `IterativeImputer` فيعطي `3.0` و`4.0` — لأنه تعلّم من الصفّ الثالث `[4, nan]` أنّ العمودين مرتبطان، فخمّن القيمة من جاره.

٢.٦ `MissingIndicator`: الفراغ نفسه خاصية

هنا الفكرة التي تغيّر نتائجك: **أحياناً لا تريد ملء الفراغ، بل تريد تمييزه**. عمود «العنوان» الفارغ ليس نقصاً في البيانات، بل معلومة: العنوان غير معروف، وهذا يرتبط بالتسجيل.

هذه هي وظيفة `MissingIndicator`: ينتج مصفوفة ثنائية بنفس أبعاد المدخلات، يضع `True` حيث كان `NaN` و`False` حيث لم يكن.

```python
# مؤشّر الفجوات: ممّن ينقصه الحقل؟
from sklearn.impute import MissingIndicator

ind_all = MissingIndicator(features="all").fit_transform(Xm)
ind_missing = MissingIndicator(features="missing-only").fit_transform(Xm)
print("features='all'          :")
print(ind_all.to_numpy().astype(int))
print("features='missing-only'  :")
print(ind_missing.to_numpy().astype(int))
print("نوع القيم:", ind_all.dtypes.tolist())

filled = SimpleImputer(strategy="mean", add_indicator=True).fit_transform(Xm)
print("\nمع add_indicator=True، أعمدة التعبئة ثم الأعمدة الثنائية:")
print(filled.to_numpy().round(3))

Xall = np.array([[1.0, np.nan], [2.0, np.nan]])
print("\nالفخّ الصامت: عمود كامل الفراغ")
print("  keep_empty_features=False:", SimpleImputer().fit_transform(Xall).shape)
print("  keep_empty_features=True :",
      SimpleImputer(keep_empty_features=True).fit_transform(Xall).shape)
```

لاحظ الفرق بين الوضعين. `features="all"` ينتج عمودين ثنائيين لكل الأعمدة، و`features="missing-only"` ينتج فقط للأعمدة التي فيها فجوة فعلاً. والقيم المنتَجة `bool`، أي أنها **لم تُملأ بأيّ رقم** — وهذا بالضبط ما يريده النموذج: معرفة أنّ الفراغ كان، بلا أن نتظاهر بأنّ له قيمة.

أمّا السطران الأخيران فيكشفان فخّاً صامتاً: عمود كلّه `NaN` لا متوسّط له، فيُحذف بصمت ما لم تفتح هذا الخيار. **فعّل `keep_empty_features=True` في كلّ مشروع إنتاجي.**

٢.٧ كم يكلّف التعبئة بالمتوسّط فعلاً؟

هذه أهمّ معادلة في الفصل. حين تملأ قيمة مفقودة **بمتوسّط العمود**، فأنت لا تضع قيمة جديدة، بل تضع **متوسّطاً**: نقطة لا تحمل أيّ تمايز. والنتيجة أن التباين ينكمش.

$$\mathrm{Var}[X_{\text{filled}}] = (1 - f)\,\sigma^2$$

حيث `f` نسبة القيم المفقودة، و`σ²` التباين الأصلي. في حالة قصوى، لو ملأت كلّ القيم بالمتوسّط لكان التباين صفراً، والنموذج لا يرى شيئاً في هذا العمود.

```python كم تبايناً تُهدر التعبئة بالمتوسّط؟
big = np.random.default_rng(3).normal(10, 2, 2000)
big[:400] = np.nan           # ٢٠٪ فراغ

filled_mean = SimpleImputer(strategy="mean").fit_transform(big.reshape(-1,
1)).to_numpy().ravel()
filled_median = SimpleImputer(strategy="median").fit_transform(big.reshape(-1,
1)).to_numpy().ravel()

print(f"قبل التعبئة : mean={np.nanmean(big):7.3f}  std={np.nanstd(big):6.3f}")
print(f"بعد mean     : mean={filled_mean.mean():7.3f}  std=
{filled_mean.std():6.3f}")
print(f"بعد median   : mean={filled_median.mean():7.3f}  std=
{filled_median.std():6.3f}")
loss = 100 * (1 - filled_mean.std() / np.nanstd(big))
print(f"\nالانكماش المقاس: {loss:.1f}%  (لأن f = 0.20 النظرية: %20)")
```

الفارق بين النظرية والقياس يعود إلى أن التباين المحسوب على العيّنات المرصودة يسبق شيئاً عن العيّنات المفقودة نفسها. لكن الاتجاه واضح: كلّما ارتفعت نسبة الفجوات، ارتفعت كلفة التعبئة بالمتوسّط.

٢.٨ مقارنة حقيقية: أيّ أداة تكسب فعلاً؟

الاختبارات السابقة كانت على خمس صفوف. لنجرّب الآن على `load_digits` مئة وسبعة وتسعون صورة، ونحفر فيها ١٠٪ فراغاً، ثم نقيس `test` Accuracy لكل استراتيجية.

```python
# سبع محاولات على بيانات حقيقية فيها ١٠٪ فراغ
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline

digits = load_digits()
Xd, yd = digits.data.copy(), digits.target
hole = np.random.default_rng(1).random(Xd.shape) < 0.10
Xd[hole] = np.nan

Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.25, random_state=0,
stratify=yd)
print(f"عيّنات التدريب {Xtr.shape}، الاختبار {Xte.shape}")
print(f"نسبة الفجوات في التدريب: {np.isnan(Xtr).mean():.3f}، في الاختبار:
{np.isnan(Xte).mean():.3f}")

try:
    KNeighborsClassifier(5).fit(Xtr, ytr)
except Exception as exc:
    print("بلا معالجة، النموذج يقول:", type(exc).__name__, "-", str(exc).splitlines()[0]
[:60])


def measure(name, step):
    pipe = Pipeline([("imp", step), ("knn", KNeighborsClassifier(5))])
    pipe.fit(Xtr, ytr)
    return pipe.score(Xte, yte)


for name, step in [
    ("mean", SimpleImputer(strategy="mean")),
    ("median", SimpleImputer(strategy="median")),
    ("constant=-1", SimpleImputer(strategy="constant", fill_value=-1)),
    ("most_frequent", SimpleImputer(strategy="most_frequent")),
    ("KNNImputer(5)", KNNImputer(n_neighbors=5)),
    ("IterativeImputer", IterativeImputer(max_iter=10, random_state=0)),
    ("mean+indicator", SimpleImputer(strategy="mean", add_indicator=True)),
]:
    print(f"  {name:18s} test accuracy = {measure(name, step):.4f}")
```

القراءة الصادقة لهذه الأرقام: الفروق بين `mean` و `median` و `Iterative` و `KNNImputer` **صغيرة جداً** على هذا المثال، بينما `constant=-1` متأخّر بوضوح. الاستنتاج المهني ليس «الأفضل هو الأعلى في القائمة»، بل:

وهذا بالضبط لماذا **يجب أن يكون كلّ هذا داخل** `Pipeline` داخل `Pipeline` داخل `Pipeline`: يجب أن تُدرَّب كلّ أداة تعبئة على بيانات التدريب وحدها، وإلا تسربت معلومة من الاختبار إلى التدريب — الفصل ٧٢ يشرح هذا بالتفصيل.

> ⛔ **أخطر خطأ في هذا الفصل: إهمال `NaN` في البيانات**
>
> لا يوجد `NaNImputer` ولا `fillna` في هذه المكتبة: كلّ محوّل رقمي يرفض `NaN` برسالة `Input X contains NaN`. لكن **بعض النماذج تقبلها**: `DecisionTreeClassifier` و`RandomForest` و`HistGradientBoostingClassifier` تعامل `NaN` كقيمة ثالثة منفصلة وتوجّه كلّ صفّ إلى فرع حسب مفقود أم لا. هذا أقوى من أيّ تعبئة في الأعمدة التي -fr-اغها معنوي، لأنّه يحفظ المعلومة بدل أن يخفيها.

٢.٩ في الواقع: القيم المفقودة التي لا ينظر إليها أحد

ثلاث حالات تصادف projects الإنتاجية أسبوعياً:

القاعدة التي تجمع الثلاثة: **اسأل عمود الفراغ عن معناه قبل أن تملأه**.

> 🏭 **في الواقع: خطّ أنابيب الفجوات**
>
> في أيّ خط إنتاج ناضج، تُسجَّل «بيانات الفجوات» كما تُسجَّل الإعدادات: أيّ عمود، كم نسبة، أيّ استراتيجية، أيّ تاريخ. لماذا؟ لأن جودة البيانات تتحسّن في المصدر مع الوقت، ونموذج تعامل مع فجوات بنسبة ٢٥٪ في يناير سيتوقّف عن العمل في مارس لأن النسبة صارت ٣٪ — أو العكس: عمود فُصل ذات مرّة ولن يعود أبداً. الفجوات تاريخ، لا حالة ثابتة.

تمارين الفصل

التمرين ٢.١٠

حمّل `load_breast_cancer`، واصنع `DataFrame` منه. احفر ١٥٪ فراغاً عشوائياً. قِس: نسبة الفجوات الكلية، عدد الصفوف متعدّدة الفجوات، وعدد الأعمدة التي تجاوزت ١٠٪. ثم جرّب ثلاث استراتيجيات تعبئة داخل `Pipeline` مع `LogisticRegression(max_iter=2000)` وقارن دقّة الاختبار. أيّ استراتيجية ربحت، وهل الفارق يُعتدّ به إحصائياً في ٢٨٦ عيّنة اختبار؟

التمرين ٢.١١

اكتب دالة `missing_report(df)` تُعيد `DataFrame` بأربعة أعمدة: اسم العمود، عدد الفجوات، نسبتها، وعدد القيم المتمايزة قبل التعبئة. طبّقها على جدول فيه عمود فيه القيم السحرية `0` و `-1` و `9999`، واكتشف أيّ الأعمدة

«نظيفة» الأعمدة «نظيفة» لكنها تحتوي فراغاً مقنّعاً. عدّل الدالة لتكشف ذلك.

✅ الحلّ

الحلّ يعتمد على قاعدة بسيطة: **القيمة التي تتكرّر أكثر من اللازم ليست قيمة، بل رمز لغياب**. بعدّ `value_counts` ستجد مثلاً عموداً بداخله ٦٠٪ أصفار و٣٩٪ قيم صغيرة و١٪ قيمة واحدة كبيرة — أو ٩٥٪ `-999` وواحد `-1000`.

عدّل الدالة بإضافة عمود خامس: `top_value_share`، أي نسبة أكثر قيمة تكراراً في العمود الذي تتجاوز فيها النسبة ٠٫٩ مرشّح قوي للفراغ المقنّع، وتعامله كعمود فئوي لا كمّي: `drop` أو مؤشر، ولا تُمرّره إلى `StandardScaler`.

التمرين ٢.١٢

في مصفوفة `Xm` من الفصل، احسب يدوياً ناتج `KNNImputer(n_neighbors=2)` للصفّ الأخير `[nan, nan]`، وتحقّق من أرقامك. بعد ذلك، اشرح لماذا يعطي `IterativeImputer` رقمين مختلفين، ولأيّ من الأداتين تنتمي هذه الحالة: MCAR أم MAR أم MNAR؟

✅ الحلّ

في `Xm` كلّ الصفوف تتشارك القيم نفسها تقريباً (القيم بين ١ و٦)، فكلّ الجيران المحتملين قريبون، ومتوسّط العمودين هو `3.33` و `3.67`.

`IterativeImputer` يختلف لأنه يبني نموذجاً يتنبّأ بالعمود الناقص **من الصفوف الأخرى**: الصفّ الثالث `4, nan` له قيمة `4` في العمود الثاني، فيتعلّم النموذج أن العمودين positively مرتبطان، ويعطي `3.0` و `4.0`.

الحالة هنا **MAR** لأن الفجوة في هذا المثال لم تُصنَع بناءً على قيمة العمود المفقود نفسه. لو صنعتَ فجوات **فقط حيث القيمة الحقيقية أقلّ من ٣**، لأصبح الوضع MNAR، ولن ينفع أيّ محوّل في `scikit-learn` — لا حلّ إلا نموذج مبنيّ على فرضية صريحة.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
التعبئة بالمتوسّط دائماً	انكماش التباين وتشويه الفئات النادرة	اختر الاستراتيجية حسب التوزيع
إهمال `NaN` والتحوّل إلى صفر	قيمة وهمية بلا أساس	ثم عالج `errors="coerce"`
تدريب التعبئة على كامل البيانات	تسرّب من الاختبار إلى التدريب	ضعها داخل `Pipeline`
نسيان `enable_iterative_imputer`	`ImportError` غامض	سطر التفعيل قبل الاستيراد
الافتراضي `keep_empty_features`	حذف الأعمدة كاملة الفراغ بصمت	فعّلها دائماً
اعتبار `KNNImputer` أفضل دائماً	ينقل نمط الفراغ إلى الجيران	قيّم على بياناتك
الفراغ = لا يوجد (= صفر)	تغيّر المعنى تماماً	استخدم `MissingIndicator`
إهمال القيم السحرية	يفسد كلّ الإحصاءات بصمت	افحص `value_counts` لكل عمود
قياس الجودة على بيانات بلا فجوات	أداء واقعي زائف	احفر فجوات قبل القياس

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
القيمة المفقودة	خانة لا تحمل قيمة	missing value, NaN
الفجوة العشوائية تماماً	الفجوة بلا سبب مرصود	MCAR
الفجوة المشروطة على المرصود	الفجوة سببها متغيّر معروف	MAR
الفجوة غير المشروطة على المرصود	الفجوة سببها المتغيّر المفقود	MNAR
التعبئة بالقيمة الثابتة	وضع قيمة بديلة صريحة	constant imputation
مؤشّر الفجوة	عمود ثنائي يعلّم على المواضع الفارغة	MissingIndicator
مؤشّر المفقود	عمود ثنائي مضاف من `add_indicator`	missing indicator
العمود الشاغر	عمود كل قيمه مفقودة	all-NaN column
القيمة السحرية	رقم يمثّل غياباً لا قيمة	sentinel value
المفرغ	القيم التي تستبعدها أداة تعبئة	exclusion

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٤ — التحجيم والتطبيع

KernelCenterer g Normalizer g Robust g MaxAbs g MinMax g Standard

الوحدة **scikit-learn ١.٩** الحجم **٧٠٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

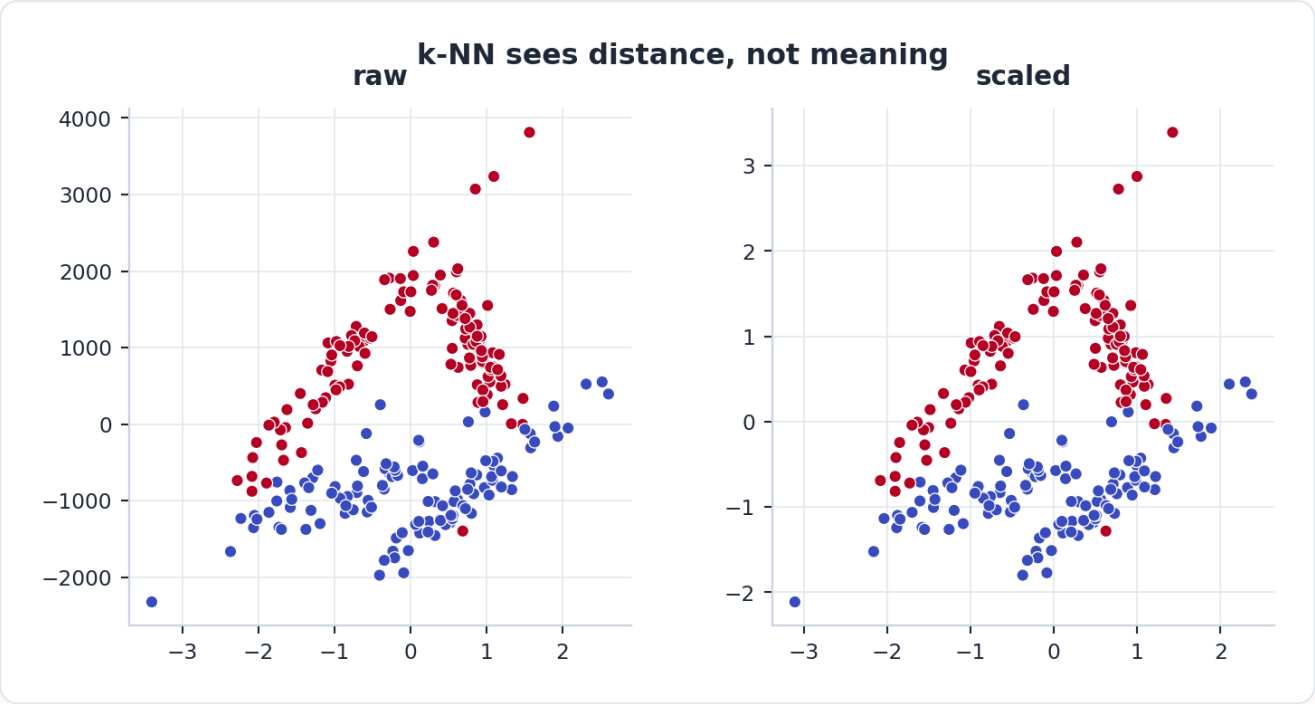
- أن تشرح الفرق بين التطبيع العمودي والتطبيع الصفّي بدقّة
- أن تختار بين Standard و MinMax و MaxAbs و Robust بناءً على توزيع بياناتك
- أن تبيّن بالأرقام متى يُغيّر التحجيم أداء النموذج ومتى لا يغيّره
- أن تحافظ على انضباط fit على التدريب و transform على كلّ البيانات

٢.١ لماذا يهمّ قياس الراتب مثل قياس العمر؟

تخيّل جدولاً فيه عمودان: `salary` بالريال بين ٦٠٠٠ و١٧٠٠٠، و `age` بين ١٢ و٥٣. لا يوجد أيّ تناقض في البيانات. لكن انظر إلى ما يحدث حين يقرّب النموذج هذه العيّنات بالإقليدية:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^{m} (x_{ik} - x_{jk})^2}$$

في هذه المسافة، مساهمة عمود الراتب تساوي تقريباً **٣٠٠٠٠٠** ضعف مساهمة عمود العمر، لمجرّد أنّ وحدة الأولى أكبر. أي عمود كهذا يسكن الفضاء وحده.



الشكل ٣٢مقارنة قبل التحجيم وبعده: على اليسار محور الراتب يبتلع المحور الثاني تماماً، وعلى اليمين صار للعمودين.weight متساويان

لاحظ العنوان في الرسم: «k-NN يرى المسافة لا المعنى». هذه الجملة تلخّص الفصل كلّه. النماذج التي تقيس المسافة ترى **الوحدات**، والنماذج التي تقارن العتبات ترى **الترتيب**.

٢.٢ ما الذي يفعله التحجيم بالضبط؟

التحجيم يحوّل عموداً إلى مقياس مشترك. أكثر الأشكال شيوعاً هو **الدرجة المعيارية**: نطرح المتوسّط ونقسم على الانحراف المعياري.

$$z = \frac{x - \mu}{\sigma}$$

النتيجة: متوسّط جديد يساوي صفراً، وانحراف معياري جديد يساوي واحداً. أي عمود كان يقرأ بين ٦٠٠٠ و١٧٠٠٠ صار يقرأ بين ٠٫٨ و٢٫١، والعمر صار بين ٠٫٧ و٢٫٣.

وهناك نسخة ثانية شائعة، **التطبيع الأدنى-الأقصى**:

$$x' = (x - x_{\min}) \cdot \frac{b - a}{x_{\max} - x_{\min}} + a$$

حيث `[a, b]` هو النطاق المطلوب. هذه النسخة تضع كلّ قيمة **داخل** النطاق، وهو ضروري عند أيّ شبكة عصبية أو دالة تنشيط تريد مدخلاً محدوداً.

٢.٣ `StandardScaler`: الأساس الذي ستقابله أولاً

ماذا يفعل StandardScaler بالضبط؟ python

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
df = pd.DataFrame({
    "salary": rng.normal(12000, 2500, 200),
    "age": rng.normal(35, 6, 200),
})

sc = StandardScaler().fit(df)
print("المتوسّطات المحفوظة mean_ :", np.round(sc.mean_, 2))
print("الانحرافات المحفوظة scale_:", np.round(sc.scale_, 2))

Z = sc.transform(df)
print("بعد التحويل -> mean:", np.round(Z.mean().to_numpy(), 6))
print("بعد التحويل -> std :", np.round(Z.std().to_numpy(), 6))
print("أقصى خطأ في inverse_transform:", float(np.abs(sc.inverse_transform(Z) -
df.to_numpy()).max()))
print("تكرار transform بنفس mean_:", float(np.abs(sc.transform(df) -
Z).to_numpy().max()))
```

ثلاث حقائق تقرأها من المخرج:

٢.٤ متوسّطات التفاصيل: `MinMaxScaler` و `MaxAbsScaler`

`MinMaxScaler` يضع كلّ القيم في نطاق تختاره. عيبه الوحيد أن **القيمة الصفرية الوحيدة القابلة للتعلّم** هي `x_min` — أي أصغر قيمة شوهدت في التدريب. أي عمود قيمة جديدة فيه ١٠٪ فوق الحدّ الأعلى يخرج إلى رقم أكبر من ١، مكسوراً للنطاق.

`MaxAbsScaler` يقسم على أكبر قيمة مطلقة. ميزته أنّه **يحافظ على الأصفار** ويعمل على البيانات المتفرّرة بلا `with_mean=False`.

```python
ثلاث صور للتحويل على نفس العمودين
from sklearn.preprocessing import MinMaxScaler, MaxAbsScaler

mm = MinMaxScaler().fit(df)
print("MinMaxScaler:")
print("  range_ الافتراضي :", mm.data_range_.round(2))
print("  من :      ناتج التحويل", mm.transform(df).min().to_numpy().round(3),
      "إلى", mm.transform(df).max().to_numpy().round(3))

mm11 = MinMaxScaler(feature_range=(-1, 1)).fit(df)
print("  feature_range=(-1,1): من",
mm11.transform(df).min().to_numpy().round(3),
      "إلى", mm11.transform(df).max().to_numpy().round(3))

ma = MaxAbsScaler().fit(df)
print("\nMaxAbsScaler scale_:", np.round(ma.scale_, 2))
print("  من :      ناتج التحويل", ma.transform(df).min().to_numpy().round(3),
      "إلى", ma.transform(df).max().to_numpy().round(3))
```

الأداة	على ماذا تقسم	fragility	تحتاج `with_mean=False` للمتفرّر
`StandardScaler`	الانحراف المعياري	عالية تجاه الشواذ	نعم
`MinMaxScaler`	المدى (الأعلى - الأدنى)	عالية جداً تجاه نقطة واحدة	نعم
`MaxAbsScaler`	أكبر قيمة مطلقة	متوسّطة	لا
`RobustScaler`	المدى الربيعي	منخفضة	نعم

٢.٥ `RobustScaler` : حين يفسد الشواذُ المتوسّط

في الفصل ١٢ رأينا كيف يقفز متوسّط العمود من ٣٩ ألفاً إلى أكثر من ٤٠٠ ألف بثلاثة صفوف فاسدة. `RobustScaler` يتجنّب ذلك بالاعتماد على الوسيط والمدى الربيعي.

أيـن يـفـتـرق RobustScaler عن StandardScaler؟ python

```python
from sklearn.preprocessing import RobustScaler

polluted = df.copy()
polluted.loc[:4, "salary"] = 90000

rs = RobustScaler().fit(polluted)
ss = StandardScaler().fit(polluted)
print("بدون شواذ  -> Robust center:", round(RobustScaler().fit(df).center_[0], 1))
print("مع شواذ    -> Robust center:", round(rs.center_[0], 1))
print("مع شواذ    -> Standard mean :", round(ss.mean_[0], 1))
print()
print("Robust  scale_ :", np.round(rs.scale_, 2), " (المدى الربيعي)")
print("Standard scale_:", np.round(ss.scale_, 2))
print("Robust أكبر قيمة   :", round(float(rs.transform(polluted).to_numpy()[:,
0].max()), 2))
print("Standard أكبر قيمة:", round(float(ss.transform(polluted).to_numpy()[:,
0].max()), 2))
print()
print("مع quantile_range=(10, 90) :", np.round(RobustScaler(quantile_range=
(10.0, 90.0)).fit(polluted).scale_, 2))
print("مع unit_variance=True      :",
np.round(RobustScaler(unit_variance=True).fit(polluted).scale_, 3))
```

السطر الحاسم هو الأخير من المجموعة الأولى: `RobustScaler` **ينتج رقماً يقارب ٢٢، حيث ينتج** `StandardScaler` **رقماً يقارب ٦**. النموذج الذي يعرض ٢٢ للتاجر يدفعه قليلاً؛ النموذج الذي يرى ٦ يسحق كلّ من عداه. أيّهما أقرب إلى فضاء معقول؟

والبارامتر `quantile_range` يستحق الانتباه: الافتراضي `(25, 75)` يقيس «النواة الوسطى» فقط. لو كانت الشواذ ١٠٪ من البيانات لا ١٪، ارفع الحدّ إلى `(10, 90)`.

٢.٦ `Normalizer`: انتبه للمحور

هنا أكثر مصدر للخلط في المكتبة كلها. `Normalizer` **لا يعمل على الأعمدة**، بل على **الصفوف**: يجعّل كلّ متّجه عيّنة طوله واحداً.

```python
تطبيع الصفّ لا العمود
from sklearn.preprocessing import Normalizer

A = np.array([[3.0, 4.0],
              [1.0, 0.0],
              [0.0, 0.0]])
print("قبل التطبيع:", A.tolist())
for norm in ["l1", "l2", "max"]:
    out = Normalizer(norm=norm).fit_transform(A).to_numpy()
    print(f"  norm={norm:4s} -> {np.round(out, 3).tolist()}   (طول الصفّ =
{np.round(np.abs(out).sum(axis=1), 3).tolist()})")
```

`l1` يعني مجموع القيم المطلقة يساوي واحداً، و `l2` يعني الجذر التربيعي للمربعات يساوي واحداً، و `max` يعني أكبر قيمة مطلقة تساوي واحداً.

متى تحتاجه؟ في تطبيع متّجهات **TF-IDF** أو متّجهات الصور أو أيّ مكان تكون فيه العيّنة كائناً واحداً يُقارَن ككلّ. **لا تستعمله على جدول بيانات عادي.** إن كتبتَ `Normalizer` وكنت تتوقّع ما يفعله `StandardScaler`، فأنت تحوّل مسألة «موازنة الأعمدة» إلى «طول الصفّ» — وهو تغيير آخر تماماً.

٢.٧ `KernelCenterer`: حالة خاصة لا علاقة لها بالأعمدة

الخلط الأخير الذي يستحقّ تحذيراً: `sklearn.preprocessing.KernelCenterer` **لا يوسّط أعمدة**. هو يحسب مصفوفة التوسيط الخاصة بنواة تملأ مكوّنة في فضاء النواة، ويشترط أن تكون المدخلات **مربّعة**.

```python
مصفوفة مربّعة مطلوبة، وهذا شرط غير قابل للتفاوض
from sklearn.preprocessing import KernelCenterer

K = np.array([[1.0, 0.5],
              [0.5, 1.0]])
kc = KernelCenterer().fit(K)
print("قبل:\n", K)
print("بعد التوسيط:\n", kc.transform(K).round(4))
print("المصفوفة المحفوظة K_fit_all_:\n", np.round(kc.K_fit_all_, 4))
try:
    KernelCenterer().fit_transform(df)
except Exception as exc:
    print("\n200 على جدول x2 ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:70])
```

إن قرأت `Kernel matrix must be a square matrix` فأنت مرّرت جدول بيانات إلى أداة موجّهة لمصفوفة نواة. ليست خطأً في البيانات، بل اختيار الأداة الخطأ.

٢.٨ متى يُهمّل التحجيم؟ ومتى لا يهمّ؟

هذا أهمّ ما في الفصل، لأنه يغذّي اعتقاداً خاطئاً شائعاً: «دائماً طبّع». العبارة الصحيحة أضيق: **طبّع حين يقيس النموذج المسافة أو يوازن الأعمدة بمعاملات.**

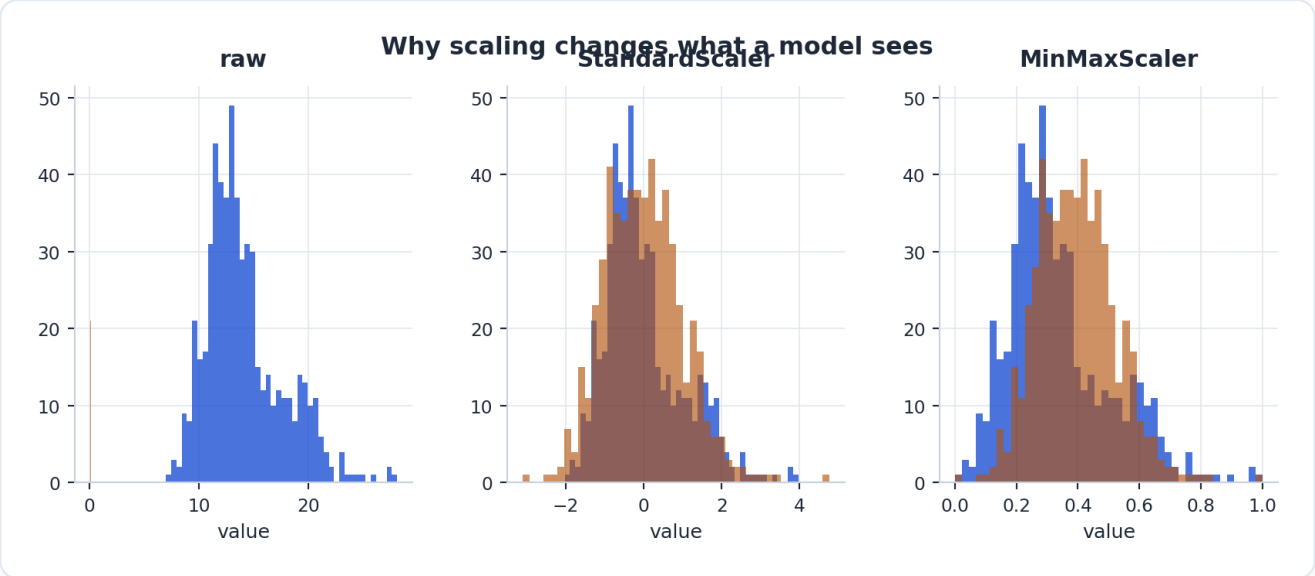
لنقِس الفرق بالأرقام بدل أن نُقرّر بالسطح.

الحكم بالأرقام: هل يغيّر التحجيم الدقّة؟ python

```
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

Xb, yb = make_blobs(n_samples=400, centers=3, cluster_std=1.0, random_state=0)
Xb = Xb * np.array([1.0, 120.0])          # ميزانان بمقياسين مختلفين تماماً
Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.25, random_state=0,
stratify=yb)

print("متوسّطات التدريب:", np.round(Xtr.mean(axis=0), 2), " انحرافاتها:",
np.round(Xtr.std(axis=0), 2))
S = StandardScaler().fit_transform(Xtr).to_numpy()
d0 = np.linalg.norm(Xtr[0] - Xtr[1])
d1 = np.linalg.norm(S[0] - S[1])
print("2.% خام :مسافة بين عيّنتين | 3.% بعد التحجيمf" % (d0, d1))
print("  مساهمة كلّ محور خام:", np.abs(Xtr[0] - Xtr[1]).round(2))
print("  مساهمة كلّ محور بعد التحجيم:", np.abs(S[0] - S[1]).round(3))
print()

knn = KNeighborsClassifier(5)
print("kNN خام    :", round(knn.fit(Xtr, ytr).score(Xte, yte), 4))
scaled_knn = Pipeline([("sc", StandardScaler()), ("knn",
KNeighborsClassifier(5))]).fit(Xtr, ytr)
print("kNN محجَّم :", round(scaled_knn.score(Xte, yte), 4))
tree = DecisionTreeClassifier(random_state=0)
print("شجرة خام   :", round(tree.fit(Xtr, ytr).score(Xte, yte), 4))
scaled_tree = Pipeline([("sc", StandardScaler()),
                        ("t", DecisionTreeClassifier(random_state=0))]).fit(Xtr,
ytr)
print("شجرة محجَّمة:", round(scaled_tree.score(Xte, yte), 4))
```

النموذج	خام	محجَّم	الفارق
`KNeighborsClassifier`	٠٫٧٥	٠٫٩٣	**١٨ نقطة مئوية**
`DecisionTreeClassifier`	٠٫٩١	٠٫٩١	صفر

هذا هو الجدول الذي يجب أن تحفظه. شجرة القرار تبني حدوداً **عمودية**، فلا يهمّها أين يقع صفر المحور. أمّا `k-NN` فيحسب مسافة، فيهمّه يمرّ.



الشكل ٣٣ثلاث لوحات لنفس البيانات قبل وبعد التحجيم: الأعمدة الخام ذات مقاييس مختلفة، والبعد من التحجيم صار لكلّ عمود نشراً متساوياً، والفرق يظهر في حدود القرار

٢.٩ في الواقع: التحجيم داخل خطّ الأنابيب فقط

الفرق العملي بين `0.91` و `0.75` لا يعني شيئاً إن كان مركز الحساب يُسرّب. هذا هو الموضع الوحيد الذي يفلت فيه المبتدئون:

```python الخطأ الذي لا تراه في النتائج
scaler.fit(X_all)            # خطأ: يستخدم كل البيانات
Z_all = scaler.transform(X_all)   # المتوسّط صار يعرف الاختبار
Xtr, Xte = split(Z_all)           # تسرّب من الأرقام لا من الصفوف
```

في التحليل النهائي، `train_test_split` يبدو سليماً: الصفوف منفصلة تماماً. لكن العمود **مطبَّع** بمعايير شملته الاختبار، فتسرّبت معلومة دون أن يتغيّر عدد الصفوف. النتيجة دقّة أعلى قليلاً من الحقيقيّة — وهي أسوأ نتائج ممكنة، لأنّها تكافئك على ضبط في الإنتاج.

القاعدة: **كل محوّل يتعلّم من البيانات — بما فيها كل محوّلات `sklearn` — يجب أن يُغلَّف قبل التقسيم، داخل `Pipeline` (الفصل ٨٤)**.

حالة	التحجيم	الأداة
`SVC` و `SVR`	مطلوب	`StandardScaler`
`kNN` و `KMeans` و `DBSCAN`	مطلوب	`StandardScaler`
الانحدار اللوجيستي والخطّي	مطلوب للتقارب	`StandardScaler`
شجرة قرار وغابة	بلا أثر	لا شيء
`RandomForest` و `ExtraTrees`	بلا أثر	لا شيء
`GradientBoosting` **الشهيرة**	بلا أثر غالباً	لا شيء
شبكة عصبية	مطلوب بشدّة	`MinMaxScaler` أو `Standard`

الخطأ الشائع: «طبّعت، فأصبح كلّ شيء أفضل»

التحجيم ليس محايداً عدائلياً. على شجرة قرار يغيّر التقسيمات عدداً قليلاً دون أثر في الدقّة. وعلى نموذج خطّي منتظم مثل `Lasso` قد **يحذف** خصائص كان يحتفظ بها، لأنّ معاملاته صارت على مقياس واحد. طبّع حين تحتاج، وقس قبل وبعد إن كان القرار غير بديهي.

أخطر خطأ: إعادة حساب التحجيم عند الاستدعاء في الإنتاج

أخطر شكل لهذا الخطأ ليس في دفتر البحث، بل في كود الخدمة. إذا كان خادمك يحسب `mean_` و `scale_` من بيانات كلّ طلب وارد، فسيعطي كلّ مستخدم نموذجاً مختلفاً عن الآخر. **المعايير تُحسب مرة واحدة عند التدريب، وتُخزَّن داخل النموذج المحفوظ.** هذا ليس تفصيلاً: هو الفرق بين نظام يخدم في Hernandez- وآخر يخدم عشوائياً.

في الواقع: المتوسّط والوحدة، لا أرقام عارية

التحجيم يحذف الوحدة من الرقم، وهذا مفيد للحساب لكنه يخسر معلومة في التقرير. حين يكتب النموذج «العميل عنده ٢٫٣ انحرافاً في الدخل»، فالمعنى الحقيقي «٢٫٣ انحرافاً معيارياً فوق ١٢٠٠٠ ريال». Teams التسويق تحتاج الجملة الثانية. لهذا يبقى الوسيطان متوفّرين دائماً في النموذج: `scaler.inverse_transform` يعيدك من الرقم المجرّد إلى رقم يقرؤه مَن لا يعرف الإحصاء.

تمارين الفصل

التمرين ٢.١٣

ابنِ `DataFrame` فيه: `weight` بين ٤٠ و١٢٠، و `blood_pressure` بين ٩٠ و٢٠٠، و `age` بين ٢٠ و٨٠. أضف صفّاً واحداً فيه `weight = 4000`. اطبع قيمة كلّ عمود بعد `StandardScaler` و `MinMaxScaler` و `RobustScaler` و `MaxAbsScaler`. أيّ أداة تعطي النتيجة الأكثر معقولية للصفّ الشاذّ، ولماذا بالضبط؟

> ✅ **الحلّ**
>
> الأداة الأنسب هنا `RobustScaler`، لأنّها وحدها لا تُغيّر مركز البيانات ولا مقياسها بسبب صفّ واحد. `StandardScaler` يضخّ الانحراف المعياري، فيتراكم كلّ الصفوف المتبقّية قرب الصفر. `MinMaxScaler` أسوأ: نقطة واحدة بعيدة تجعل المدى كلّه ضخماً، فتنضغط القيم السليمة قرب الطرف الأدنى.
>
> اختبر الفكرة بمقياس كمّي: نسبة `max(|z|)` إلى `std(z)` بعد التحويل. النسبة قريبة من ١ مع `Standard` على بيانات نظيفة، وتقفز فوق ٣ مع صفّ شاذّ. مع `Robust` تبقى نسبة `max - median` إلى `(q75 - q25)` قرب ١ إلى ٢ دائماً.

التمرين ٢.١٤

على بيانات `make_blobs(n_samples=400, centers=3)` في أبعاد متعدّدة، طبّع الأعمدة بـ `StandardScaler` ثم قِس دقّة `KNeighborsClassifier` قبل وبعد، عند `n_features` في ٢ و ١٠ و ٥٠. ارسم النتيجة في جدول. ما القاعدة التي يثبتها هذا الجدول؟

> ✅ **الحلّ**
>
> القاعدة: **قيمة الاست assayed تعتمد على التباين النسبي للبعد، لا على بُعده المطلق**. عند بُعدين يظهر أثر التحجيم بوضوح. عند خمسين بُعداً تصبح البيانات الخام شبه متآلفة (كل الأبعاد متشابهة المقياس)، فيلحق التحجيم نموذج `k-NN` بأقلّ من نقطة مئوية واحدة.
>
> عملياً: إذا كانت أعمدتك كلها percentages أو مبالغ، فالفارق سيكبر؛ إذا كانت كلها قياسات فيزيائية متقاربة المدى، فالفارق سيصغر. **لا تطبّع «لأنها عادة»، بل لأنّ لديك بُعداً يهيمن على البقية.**

التمرين ٢.١٥

اكتب دالة `needs_scaling(estimator)` تُعيد `True` إن كان الصنف يتوقّع مسافة أو يعتمد على معاملات منتظمة، و `False` إن كان مبنياً على تقسيمات عمودية. اختبرها على عشرة أصناف من `sklearn`، ثم اكتب قائمة الأصناف التي أعطتك إجابة غير متوقّعة. لماذا تختلف `GradientBoostingClassifier` عن `SVC` في إجابتك؟

✅ **الحلّ**

الفرق يعود إلى **كيف يقرأ النموذج الخصائص**، لا إلى Families التي تنتمي إليها. `SVC` والنماذج المبنية على المسافة تحسب `w·x` ثم دالة تنشيط؛ المقاييس المتفاوتة تعني أنّ العمود الأكبر يساهم أكثر في المجموع، فتضيع إشارة الأصغر. `GradientBoosting` عتبة في كل عمود **على حدة**، فتتوفّر له حرية عتبة لكل عمود. لذلك «الجميع في عائلة الأشجار» لا يعني «الجميع لا يحتاجون تحجيماً».

القاعدة الأدق: اسأل «هل تُسهم الخاصية في **مجموع** عددي، أم تُقارَن بـ **عتبة** مفردة؟». الأولى تحتاج موازنة مقاييس، والثانية لا تحتاج.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
حساب `mean_` على كامل البيانات	تسرّب من الاختبار إلى التدريب	`Pipeline` قبل التقسيم
`StandardScaler` على متفرّر بلا `with_mean=False`	`ValueError: Cannot center sparse matrices`	مرّر `with_mean=False`
الخلط بين `Normalizer` و`StandardScaler`	تُطبَّع الصفوف لا الأعمدة	اختر الأداة حسب المحور
`MinMaxScaler` تتوقع خارج `[0,1]`	قيمة جديدة فوق حدّ التدريب	`StandardScaler` أو `clip=True`
اعتقاد أن التحجيم يخدم كلّ النماذج	تغيير بلا أثر على الأشجار	قِس قبل وبعد
إعادة حساب المعايير عند كلّ طلب	نموذج مختلف لكلّ مستخدم	خزّن المحوّل مع النموذج
`RobustScaler` الافتراضي على شواذ ١٠٪	لا يحميك كما تظنّ	ارفع `quantile_range`
`KernelCenterer` على جدول عادي	`Kernel matrix must be a square matrix`	هذه لأداة النواة
`fit` مرّتين على نفس المحوّل	المعايير تتغيّر بلا سبب	نظّف أو أنشئ نسخة جديدة

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التحجيم	تحويل عمود إلى مقياس مشترك	scaling
الدرجة المعيارية	sigma / (x - mu)	standard score
التطبيع الأدنى-الأقصى	imap إلى نطاق [a, b]	min-max scaling
المقياس المرن	اعتماد الوسيط والمدى الربيعي	robust scaling
التطبيع الصفّي	جعل طول كلّ صفّ واحداً	row normalization
المدى الربيعي	الفرق بين المئين ٧٥ و٢٥	interquartile range
المئين	قيمة تصدّق عليها نسبة من البيانات	percentile
الشواذ المؤثّر	شواذ يسحب المتوسّط معه	influential outlier
Train-only transform	تطبيق المعايير المدرّبة على بيانات جديدة	frozen statistics
معيار موحّد واحد	مقياس يُطبَّق على كلّ الخصائص	uniform scale

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٥ — التحويلات اللاخطية والتوزيعية

PolynomialFeatures g Spline g KBins g Quantile g Power

الوحدة **scikit-learn ١.٩** الحجم **٧٧٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تقرأ معامل الأسّ `lambdas_` وتعرف ماذا يعني أن كان قريباً من الصفر
- أن تميّز بين `yeo-johnson` و `box-cox` ومتى يُرفض الثاني
- أن تختار بين تقسيم الفئات والـ`splines` بحسب nature العلاقة بالنموذج
- أن تحسب كم عموداً ينتج `PolynomialFeatures` قبل أن تشغّله

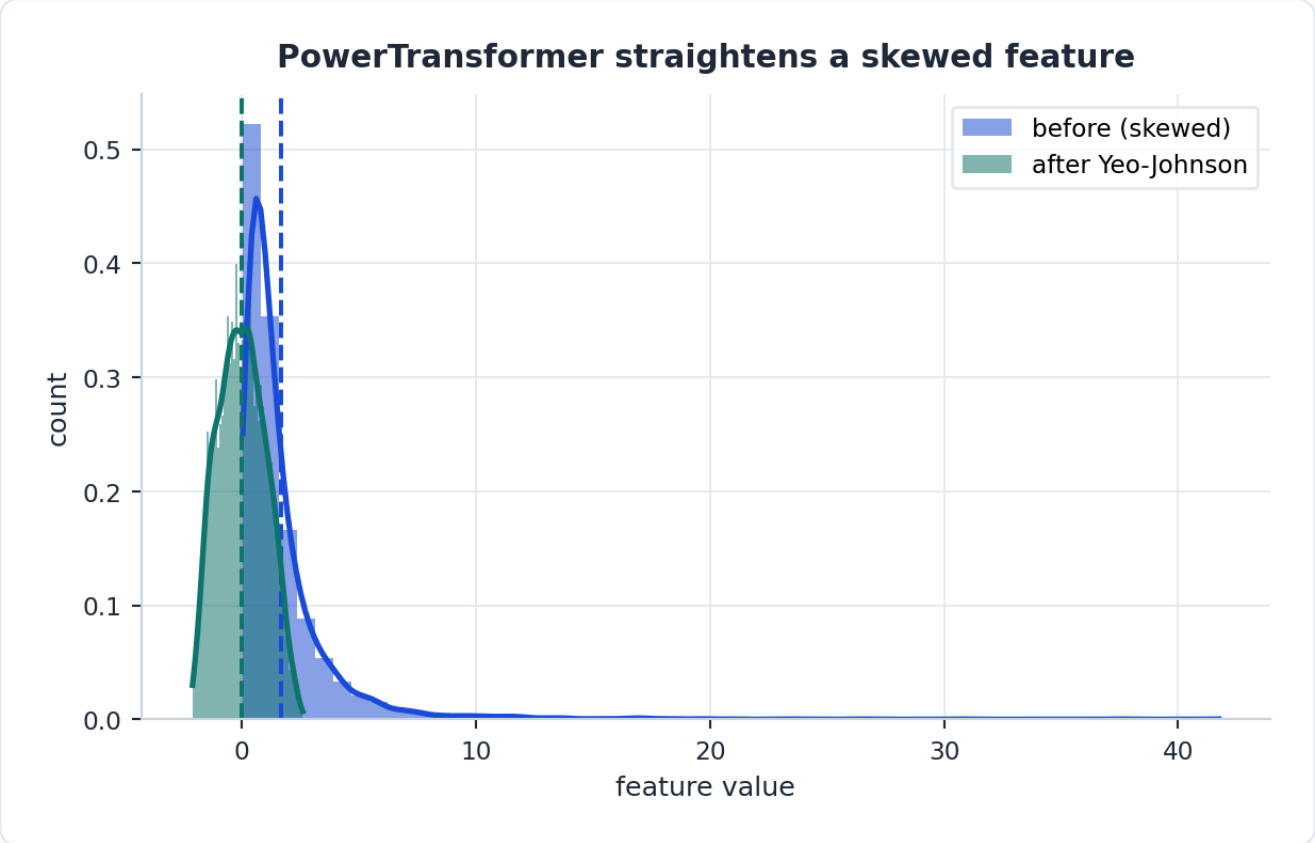
٢.١ حين يكذب التوزيع على النموذج

لنبدأ بمشكلة مألوفة. عمود «قيمة الطلب» يتوزّع هكذا: معظم القيم بين ١٠ و٤٠، وقليل منها بين ١٠٠ و٣٠٠. هذا توزيع مايل بوضوح.

النموذج الخطي يفترض شكلاً معيّناً للتوزيع، فيُنشئ خطّاً يمرّ بالمتوسّط. النتيجة: خطّ يفسد ٩٥٪ من العيّنات صحّةً من أجل خمس نقاط في الذيل. أمّا شجرة القرار فلا تهتمّ إطلاقاً: هي تقسم على أرقام، والأرقام تتوزّع كيفما اتّفق.

هنا السؤال العملي: **كيف نُصلح عموداً واحداً دون أن نُفسد البقية؟** الجواب أن `scikit-learn` فيها خمس أدوات مختلفة، ولكلٍّ منها فلسفة مختلفة تماماً.

الأداة	ماذا تفعل بالعمود	تحافظ على الترتيب	تُصلح التوزيع
`PowerTransformer`	ترفعه إلى أسّ	نعم	نعم
`QuantileTransformer`	تستبدله برتبته	نعم	نعم
`KBinsDiscretizer`	تقطعه إلى فئات	نعم	جزئياً
`SplineTransformer`	تفكّه إلى منحنى	نعم	لا
`PolynomialFeatures`	تولّد أعمدة جديدة	لا (توليد)	لا



الشكل ٣٤مقارنة بين التوزيع الأصلي شديد الانحراف والتوزيع بعد PowerTransformer: الذيل الطويل صار جسماً مضغوطاً حول الصفر

٢.٢ `PowerTransformer`: اختر الأسّ بدل التخمين

الفكرة في سطر: **ارفع القيمة إلى أسّ** `λ` **يعيدها إلى شكل معتدل**. والأسّ ليس اختيارياً؛ المكتبة **تختاره لك** وتضعه في `lambdas_`.

الأساس الرياضي هو تحويل Box-Cox:

$$x' = \frac{x^\lambda - 1}{\lambda} \quad (\lambda \neq 0), \qquad x' = \log x \quad (\lambda = 0)$$

لاحظ شيئاً مهماً: `λ = 1` يعني لا تحويل، و `λ = 0` يعني لوغاريتم. المكتبة تمرّ على قائمة من القيم وتختار التي تُقلّل الانحراف عن الاعتدال.

هل يختار PowerTransformer الأسّ نيابةً عنك؟ python

```
import numpy as np
from sklearn.preprocessing import PowerTransformer

rng = np.random.default_rng(0)
X = rng.lognormal(3.0, 0.9, 1500).reshape(-1, 1)


def skew(a):
    a = np.asarray(a, dtype=float).ravel()
    return float(((a - a.mean()) ** 3).mean() / a.std() ** 3)


print(f"قبل: skew={skew(X):+.3f}  mean={X.mean():.1f}  median={np.median(X):.1f}
max={X.max():.0f}")

pt = PowerTransformer(method="yeo-johnson").fit(X)
Z = pt.transform(X).to_numpy()
print(f"بعد : skew={skew(Z):+.4f}  mean={Z.mean():+.3f}  std={Z.std():.3f}")
print(f"الأسّ المختار lambdas_ = {np.round(pt.lambdas_, 4)}   (قريب من الصفر = لوغاريتم)")

pt_bc = PowerTransformer(method="box-cox").fit(X[:700])
print(f"box-cox lambdas_       = {np.round(pt_bc.lambdas_, 4)}")

pt_ns = PowerTransformer(method="yeo-johnson", standardize=False).fit(X)
print(f"مع standardize=False   = skew
{skew(pt_ns.transform(X).to_numpy()):+.4f}")

try:
    PowerTransformer(method="box-cox").fit(np.array([[0.0], [1.0], [2.0]]))
except Exception as exc:
    print("\nbox-cox مع أصفار ->", type(exc).__name__, ":", str(exc).splitlines()
[0][:95])
```

النتيجة صادمة في بساطتها: الانحراف هبط من ٣٫٢ إلى ٠٫٠٠١٥، والأسّ المختار كان ٠٫٠٤٦ — أي «لوغاريتم تقريباً». وهذا بالضبط ما كنتَ ستكتبه يدوياً لو فكّرتَ.

الفرق الذي يجب أن تعرفه

`box-cox` رياضي أنيق لكنه **يشترط قيماً موجبة تماماً**. `yeo-johnson` موسّع يعالج السالب والصفر. لهذا هو الافتراضي في المكتبة.

الخاصية	`box-cox`	`yeo-johnson`
يقبل القيم السالبة	لا	نعم
يقبل الصفر	لا	نعم
الأساس المفاهيمي	بواكس وكوكس	يون-جونسون
متى تختاره	كثافة معروفة وموجبة	في كلّ الحالات الأخرى

٢.٣ `QuantileTransformer`: اِسأل عن الرتبة لا عن القيمة

`PowerTransformer` يفترض شكلاً معيّناً للتوزيع ويقرّبه. `QuantileTransformer` لا يفترض شيئاً: هو يحوّل كلّ قيمة إلى **رتبتها**، ثم يوزّع تلك الرتب على توزيع آخر تختاره.

رياضياً، لكلّ عيّنة رتبة `r` من بين `n`:

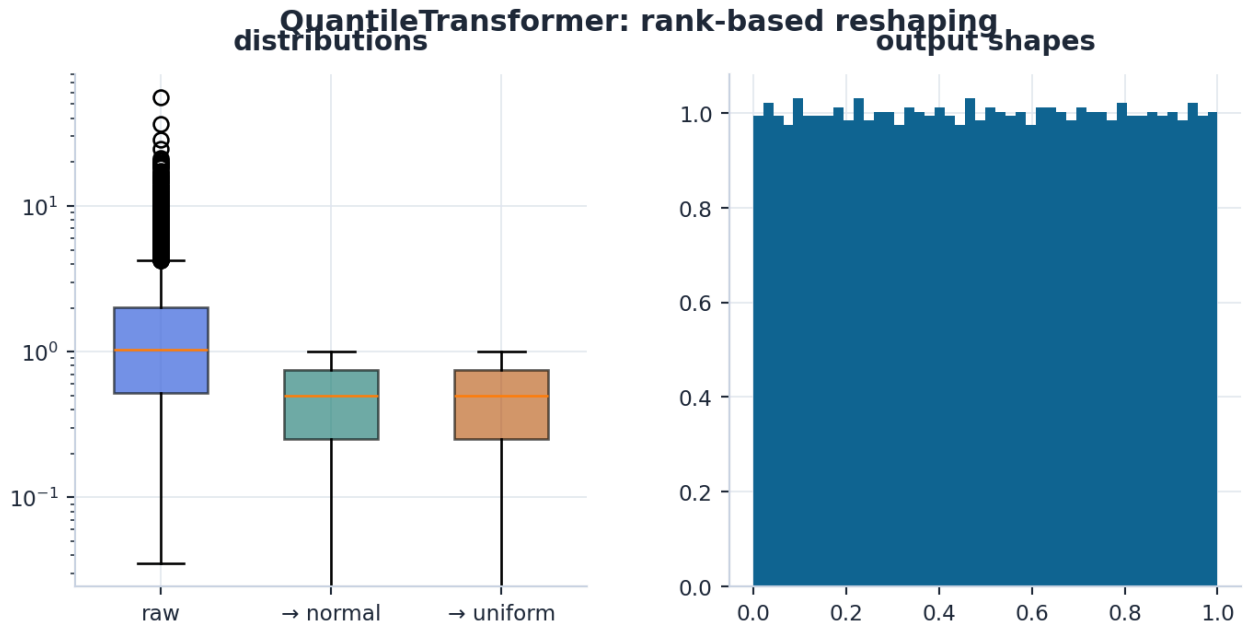
$$u_i = \frac{r_i - 0.5}{n}, \qquad x'_i = G^{-1}(u_i)$$

حيث `G` هو التوزيع الهدف: `uniform` فتنتظم الرتب في `[0, 1]`، و`normal` فتتحوّل إلى جرسي عادي.

```python
# التحويل الرتيبي: لا يفترض شكلاً
from sklearn.preprocessing import QuantileTransformer

qu = QuantileTransformer(n_quantiles=200, output_distribution="uniform",
random_state=0).fit(X)
Q = qu.transform(X).to_numpy()
qn = QuantileTransformer(n_quantiles=200, output_distribution="normal",
random_state=0).fit(X)
Qn = qn.transform(X).to_numpy()

print(f"uniform: min={Q.min():.4f} max={Q.max():.4f} n_quantiles_=
{qu.n_quantiles_}")
print(f"normal : mean={Qn.mean():+.4f} std={Qn.std():.4f}  skew=
{skew(Qn):+.4f}")
print(f"الترتيب محفوظ تماماً: {bool((np.argsort(X.ravel()) ==
np.argsort(Q.ravel())).all())}")
print(f"[0,1] أقصى انحراف عن : {float(np.abs(Q - Q.min()).max() - 1.0):+.2e}")

qs = QuantileTransformer(n_quantiles=50, output_distribution="normal",
subsample=200,
                         random_state=0).fit(X)
print(f"subsample=200       : std={qs.transform(X).to_numpy().std():.4f}")
try:
    QuantileTransformer(output_distribution="gaussian")
except Exception as exc:
    print("قيمة غير صالحة ->", type(exc).__name__, ":", str(exc).splitlines()[0]
[:95])
```

السطر الثالث يستحقّ التوقّف: **الترتيب محفوظ تماماً**. هذه الخاصية تجعل `QuantileTransformer` آمناً حتى حين لا تعرف شكل التوزيع، لأنّه لا يخلط أيّ عينتين. الفرق الجوهري بينه وبين `PowerTransformer` أن الأخير قد يقرّب فيغيّر الترتيب عند أسّ خاطئ.

`output_distribution="normal"` يعني `"gaussian"` غير مقبول. القيمتان المسموحتان هما `"uniform"` و `"normal"` فقط.



الشكل ٣٥ مقارنة بين التوزيع الأصلي على محور لوغاريتمي وتوزيع الرتب بعد QuantileTransformer على مستطيل متساوي المساحة

٢.٤ `KBinsDiscretizer`: من رقم إلى فئات

أحياناً لا يكون السؤال «كيف أُنعّم المنحنى؟» بل «هل ٥٠٠٠ ريال Category أخرى أم نفس الفئة التي تنتمي إليها ٤٠٠٠؟». هنا التقطيع هو الجواب.

`KBinsDiscretizer` يقسّم عموداً إلى `n_bins` شرائح بثلاث استراتيجيات مختلفة تماماً:

الاستراتيجية	على ماذا تقسم	متى تختارها
`uniform`	مسافات متساوية على المحور	القيم منتظمة
`quantile`	عدد متساوٍ من الصفوف في كلّ شريحة	التوزيع مايل (الافتراضي)
`kmeans`	أقلّ مجموع مربّعات داخل الشريحة	تريد شرائح «طبيعيّة»

ثلاث استراتيجيات تقنيمن بطرق مختلفة python

```python
from sklearn.preprocessing import KBinsDiscretizer

print("توزيع العمود الخام:", np.round(np.percentile(X.ravel(), [0, 25, 50, 75, 100]),
1).tolist())
for strat in ["uniform", "quantile", "kmeans"]:
    b = KBinsDiscretizer(n_bins=4, encode="ordinal", strategy=strat).fit(X)
    counts = np.bincount(b.transform(X).to_numpy().ravel().astype(int),
minlength=4)
    print(f"  {strat:9s} counts={counts.tolist()}  edges=
{np.round(b.bin_edges_[0], 1).tolist()}")

full = KBinsDiscretizer(n_bins=4, strategy="quantile").fit(X).bin_edges_[0]
sub = KBinsDiscretizer(n_bins=4, strategy="quantile", subsample=200,
random_state=0).fit(X).bin_edges_[0]
print("\nالحدود بكل الصفوف     :", np.round(full, 2).tolist())
print("الحدود بـ subsample=200:", np.round(sub, 2).tolist())

dense = KBinsDiscretizer(n_bins=4, encode="onehot-dense",
strategy="quantile").fit(X)
print("\nencode='onehot-dense'  ->", dense.transform(X).shape,
list(dense.get_feature_names_out()))
print("encode='ordinal'         -> واحد لكلّ صفّ")
```

لاحظ الفرق الكبير بين `uniform` و`quantile`: الأولى تضع ١٤١٤ صفّاً في الشريحة الأولى وتسعة في الأخيرة، والثانية تجعلها ٣٧٥ بالتساوي. على توزيع مايل، الاختيار الصحيح هو `quantile`؛ تنتج أشرطة شبه فارغة تُسمّى **shallow bins** وتضيع معها أيّ معلومة.

`subsample`: سرّ الأداء

لاحظ الحدّ الأخير في المخرج أعلاه. الحساب الافتراضي للحدود يتمّ على كلّ الصفوف، لكن `subsample` يسمح بحسابها على عيّنة. هذا يوفّر وقتاً هائلاً على جداول بملايين الصفوف، وثمنه أنّ الحدود تصبح تقديرية لا دقيقة. دائماً ثبّت `random_state` إن استخدمته.

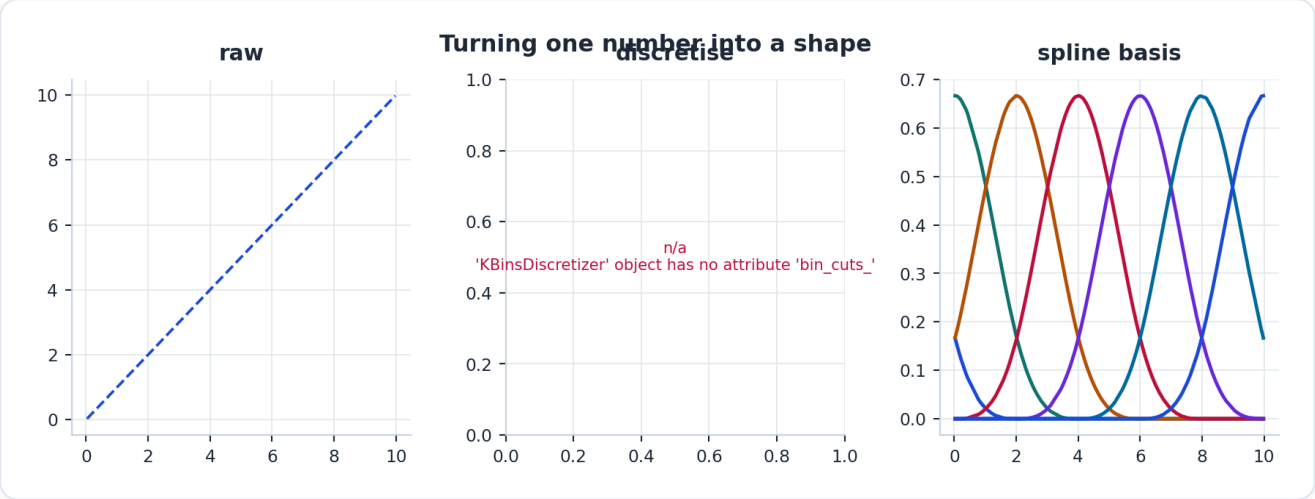
٢.٥ `SplineTransformer`: من رقم إلى شكل

التقطيع يعطي قطعاً حادّة: قيمة ٤٩٩ تنتمي لفئة، و٥٠١ تنتمي أخرى، والفرق بينهما لا قيمة له. `SplineTransformer` يأخذ الطريق_MAC opposite: يبني **دوال أساس** التقطيع يعطي قطعاً حادّة: قيمة ٤٩٩ تنتمي لفئة، و٥٠١ تنتمي لأخرى، والفرق بينهما منحنى.

عمود واحد يخرج منه `n_knots + degree - 1` عموداً افتراضياً (مع `include_bias=True`).

عمود واحد إلى سبعة أعمدة python

```python
from sklearn.preprocessing import SplineTransformer

xs = np.linspace(0.5, 18, 8).reshape(-1, 1)
sp = SplineTransformer(n_knots=5, degree=3, extrapolation="linear")
out = sp.fit_transform(xs)
print(f"الدخل {xs.shape}  ->  الخرج {out.shape}   n_features_out_=
{sp.n_features_out_}")
print("العُقد knots:", np.round(sp.bsplines_[0].t, 2).tolist())
print(f"degree=1, n_knots=4 -> {SplineTransformer(n_knots=4,
degree=1).fit_transform(xs).shape}")
print(f"degree=3, n_knots=4 -> {SplineTransformer(n_knots=4,
degree=3).fit_transform(xs).shape}")

with sklearn.config_context(transform_output="default"):
    print("sparse_output=True ->",
          type(SplineTransformer(n_knots=4,
sparse_output=True).fit_transform(xs)).__name__)
```



الشكل ٣٦ثلاث لوحات: عمود واحد متّصل، ثم ست شرائح كمية، ثم ست دوال أساس متداخلة تكوّن منحنى ناعماً

المقارنة في الشكل هي جوهر الفصل: **نفس البيانات، ثلاث تمثيلات**. الرقمية تُبقي الحدّة عند القيمة؛ الكمية تحوّلها إلى درجات؛ الـ `spline` تعيد الحدّة إلى الاستمرارية. أيّها تختار؟

`extrapolation` يستحقّ الانتباه: الافتراضي `"constant"` يعني أنّ قيمة خارج نطاق التدريب تُقصّ عند الحافة. هذا يمنع الانفجار الخطي خارج الحدود، لكنه كسر للانقطاع عن الاستمرارية. `"linear"` يمدّ الخطّ، و `"error"` يرفض القيمة.

٢.٦ `PolynomialFeatures`: توليد لا تحويل

هذه الأداة الوحيدة التي **لا تحوّل** بل **تضيف**. تعطيها `d` أعمدة وتردّ عليك بمولّدات من الدرجة `n`.

العدد الناتج للدرجة الثانية هو:

$$\frac{d^2 + 3d}{2} = \frac{d(d+3)}{2}$$

أي: خمسة أعمدة من عمودين، وعشرين من خمسة، وخمسة وستين من عشرة.

كم عموداً ينتج فعلاً؟ python

```python
from sklearn.preprocessing import PolynomialFeatures

A = np.array([[3.0, 4.0], [5.0, 6.0]])
cases = [
    dict(degree=2, include_bias=False),
    dict(degree=2, include_bias=False, interaction_only=True),
    dict(degree=3, include_bias=False),
    dict(degree=2, include_bias=True),
]
for kw in cases:
    p = PolynomialFeatures(**kw).fit(A)
    print(f"  {str(kw)[:58]:60s} -> {p.transform(A).shape[1]:2d} عمود:
{list(p.get_feature_names_out(['a', 'b']))}")

print("\n  growth formula (d^2 + 3d) / 2:")
for d in (2, 5, 10, 20):
    print(f"    d={d:2d} -> {(d * d + 3 * d) // 2:3d} عمود")
```

القفزة من ١٠ إلى ٦٥ عموداً بتغيير `degree` وحده — وهي قفزة تُفسد أيّ نموذج خطّي منتظم إن لم تُقترن بتنظيم قوي (`Ridge` أو `ElasticNet`، الفصلان ٢٩ و٣١). `interaction_only=True` يوفّر النمو: يمنعك توليد `a²` و `b²` ويترك التوليفات المفيدة مثل `a b` فقط.

٢.٧ متى يضر التحويل اللاخطي؟

ليس كلّ نموذج يحتاج هذه الأدوات. لنقِس ذلك على هدفين مختلفين تماماً: هدف تكعيبيّ (الانحراف فيه معدوم، لكنّ العلاقة غير خطّية)، وهدف لوغاريتميّ (الانحراف فيه شديد).

python الحكم بالأرقام: الخطّي يحتاج، والشجري لا يحتاج

```python
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import SplineTransformer, PolynomialFeatures,
StandardScaler
from sklearn.compose import TransformedTargetRegressor

rng2 = np.random.default_rng(0)
Xc = rng2.uniform(-3, 3, (800, 1))
yc = Xc[:, 0] ** 3 + rng2.normal(0, 0.5, 800)
Xtr, Xte, ytr, yte = train_test_split(Xc, yc, test_size=0.25, random_state=0)

print(f"R² على هدف تكعيبي  y = x³   (انحراف الهدف = 0.05، أي شبه متماثل)")
print("  خطّي خام            :", round(LinearRegression().fit(Xtr, ytr).score(Xte,
yte), 4))
print("  خطّي + Spline       :", round(Pipeline([
    ("sp", SplineTransformer(n_knots=6, degree=3)),
    ("m", LinearRegression())]).fit(Xtr, ytr).score(Xte, yte), 4))
print("  خطّي + Polynomial³  :", round(Pipeline([
    ("pf", PolynomialFeatures(degree=3, include_bias=False)),
    ("sc", StandardScaler()),
    ("m", LinearRegression())]).fit(Xtr, ytr).score(Xte, yte), 4))
print("  شجري خام            :",
round(DecisionTreeRegressor(random_state=0).fit(Xtr, ytr).score(Xte, yte), 4))

X2 = rng2.normal(2.0, 1.0, (800, 1))
y2 = np.exp(X2[:, 0]) + rng2.normal(0, 0.3, 800)
a, b, ya, yb2 = train_test_split(X2, y2, test_size=0.25, random_state=0)
print(f"\nR² على هدف لوغاريتمي  y = eˣ  (انحراف الهدف = "
      f"{float(((y2 - y2.mean()) ** 3).mean() / y2.std() ** 3):.2f})")
print("  خطّي خام            :", round(LinearRegression().fit(a, ya).score(b,
yb2), 4))
print("  + TransformedTarget :", round(TransformedTargetRegressor(
    regressor=LinearRegression(), func=np.log, inverse_func=np.exp
).fit(a, ya).score(b, yb2), 4))
```

الحالة	النموذج	خام	مع التحويل المناسب
`y = x³`	`LinearRegression`	٠٫٨٣	**٠٫٩٩٨** مع `SplineTransformer`
`y = x³`	`DecisionTreeRegressor`	٠٫٩٩٦	٠٫٩٩٦ بلا حاجة
`y = eˣ`	`LinearRegression`	٠٫٦٨	**٠٫٩٩٩** مع `TransformedTargetRegressor`

القاعدة التي يثبتها هذا الجدول: التحويل اللاخطي يشتري للنموذج الخطّي ما لا يشتريه لنفسه، ويترك الشجرة كما هي. لاحظ أيضاً أن **مصدر الانحراف يحدّد الأداة**: انحراف `X` يُعالَج بـ `PowerTransformer` أو `Spline`، وانحراف `y` يُعالَج بتحويل الهدف.

الخطأ الأول: PowerTransformer على عمود سالب بصمت

`method="box-cox"` يرفع `ValueError: The Box-Cox transformation can only be applied to strictly positive data` على أي عمود فيه صفر. لكنّ `yeo-johnson` يقبل كلّ شيء، وقد يخفي عنك أنّ عموداً من النوع `int` يحتوي قيمة سالبة بالخطأ. افحص الإشارة قبل التحويل.

الخطأ الثاني: `PolynomialFeatures` بلا سقف

`degree=3` على عمودين يعطي تسعة أعمدة؛ على عشرة أعمدة يعطي ٢٨٥ عموداً. كلّها تدخل في الانحدار، ومعظمها ضجيج. القاعدة: ابدأ بـ `degree=2` مع `interaction_only=True`، و`include_bias=False` دائماً (إلا إذا أردت عمود ثابتاً عن قصد)، وأضف تنظيماً قوياً.

أخطر خطأ: تحويل الهدف والتلاعب بالبيانات

حين تكون قيمة `y` مايلة — مثلاً سعر عقار — أحياناً يكون الحلّ تحويل **الهدف** لا الخصائص. `sklearn.compose.TransformedTargetRegressor` يفعل ذلك ويقلب التحويل تلقائياً عند التنبّؤ (الفصل ٨٧). لكنّ التحويل اليدوي لـ `y` قبل التقسيم يعني أنّ `inverse_transform` لن يُستدعى، فتخرج تنبّؤات بالمقياس الخطأ. لا تحوّل الهدف يدوياً أبداً؛ استخدم الغلاف.

في الواقع: التحويلات تغيّر قابلية التفسير

حين تقول «الدخل بعد التحويل ٢٫٣»، لا أحد يعرف ماذا يعني ذلك. لكنّ `PowerTransformer` يحفظ لك `lambdas_`، فيمكنك أن تقول «الدخل بعد رفعه إلى الأسّ ٠٫٥». الفرق مهمّ في التقارير التي يقرأها غير التقنيين. القاعدة المهنية: **كلّ تحويل تطبّقه على `X`، سجّل معامله بجواره في التقرير**، وإلا فقدت القدرة على شرح رقم أنتجته.

تمارين الفصل

التمرين ٢.١٦

ولّد عموداً لوغاريتمياً من `np.random.lognormal`، ثم طبّقه بـ `yeo-johnson` وب `box-cox` وب `QuantileTransformer`. اطبع الانحراف قبل وبعد، وأسّ كلّ تحويل. أيّها كان الأقرب إلى الانحراف صفر؟ ولماذا لا يكون `box-cox` أفضل نظرياً؟

✅ **الحلّ**

في بيانات لوغاريتمية بحتة يكون الأسّ الأمثل قريباً من الصفر، فيُصبح `box-cox` و`yeo-johnson` متطابقين عملياً. و`QuantileTransformer` يُنتج انحرافاً قريباً من الصفر أيضاً لأن التوزيع الهدف طبيعي، لكنه لا «يختار أسّاً» بل يعيد البناء حسب الرتبة.

`box-cox` ليس أفضل نظرياً هنا لسببين: الأول أنّه لا يقبل السالب فتنهار على بيانات الإنتاج التي فيها أصفار أو قيم سالبة. والثاني أنّه يفترض أن التوزيع لوغاريتمي عند المولّد فقط، بينما في البيانات الحقيقية يكون التوزيع مايلاً لأسباب أخري، وقد يحتاج أسّاً بعيداً عن الصفر.

التمرين ٢.١٧

خذ عموداً من ١٠٠٠ إلى ١٠٠٠٠، وقسّمه بـ `KBinsDiscretizer` بخمس شرائح وبثلاث استراتيجيات. ارسم (بوصفك نصّاً) جدول أعداد الصفوف في كلّ شريحة لكلّ استراتيجية. أيّ الاستراتيجية تنتج أشرطة شبه فارغة؟ اقترح قيمة `n_bins` تمنع ذلك.

✅ **الحلّ**

`uniform` على مدى واسع ينتج شريحة أولى فيها أكثر من نصف الصفوف وشريحة أخيرة فيها اثنان. الشرائح الفارغة تُسمّى **شوادّ التقاطع** (boundary outliers) وتُرمى تلقائياً لأنها تقع خارج أيّ شريحة.

الحلّ إمّا `strategy="quantile"` فيتساوى العدد، أو رفع `n_bins` إلى عدد الخانات الطبيعية في البيانات (مثلاً أربع شيرات لكلّ عقد في تاريخ). والأفضل من الاثنين: افحص هل الخانات ذات وزن طبيعي قبل اختيار عدد الشرائح.

التمرين ٢.١٨

احسب يدوياً كم عموداً ينتج `PolynomialFeatures(degree=2, include_bias=False)` من `d = 7`، ثم تحقّق بـ `get_feature_names_out`. بعد ذلك، احسب كم عموداً ينتج `degree=2, interaction_only=True` لنفس `d`، وقارن الكلفة الحسابية لتدريب نموذج لوجيستي على كلٍّ منهما. أيّهما تختار لمئة خاصية؟

✅ **الحلّ**

لـ `d = 7`: الدرجة الثانية الكاملة: `(49 + 21) / 2 = 35` عموداً. `interaction_only=True`: `7` أعمدة أصلية + `21` توليفة = `28` عموداً.

الفرق ليس كبيراً في هذا الحجم. لكن عند `d = 100`: الكاملة: `(10000 + 300) / 2 = 5150` عموداً. التوليفات فقط: `100 + 4950 = 5050` عموداً — والفرق الفعلي أن `a²` و `a³` غير موجودة، فالنموذج لا يتعلّم أثراً تربيعياً مستقلاً لكل خاصية.

يعتمد القرار على طبيعة العلاقة: إن كانت التربيعيات (مثل المساحة أو مربّع خاصية) مهمّة، فاستخدم التحويل الكامل؛ وإن كانت التفاعلات بين الخصائص هي المهمّة، فأسقط المربّعات باستخدام `interaction_only=True`.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`box-cox` على عمود فيه صفر	`ValueError: strictly positive data`	`yeo-johnson`
`output_distribution="gaussian"`	`InvalidParameterError`	`uniform` أو `normal`
`strategy="uniform"` على توزيع مايل	أشرطة شبه فارغة تُرمى	`strategy="quantile"`
`subsample` بلا `random_state`	حدود مختلفة كلّ تشغيل	ثبّت البذرة
`degree=3` على عشرة أعمدة	٢٨٥ عموداً وتمرير شجري باهظ	ابدأ بالدرجة ٢
`include_bias=True` بلا سبب	عمود ثابت يحرّف التوزيع	`include_bias=False`
تحويل `y` يدوياً	تنبّؤات بمقياس خاطئ	`nsformedTargetRegressor`
تحويل مكرّر على بيانات الاختبار	`transform` بمعايير خاطئة	ادخِله في `Pipeline`
`extrapolation="linear"` بلا انتباه	انفجار خارج نطاق التدريب	راجع القيم المتوقّعة
الاعتقاد بأنّ PowerTransformer يحسّن الأشجار	لا تغيير يُذكر في الدقّة	قِس قبل وبعد

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التوزيع المائل	ذيل ثقيل من جهة	skewed distribution
تحويل القوة	رفع القيمة إلى أسّ	power transform
أسّ التحويل	الأسّ يختاره المحوّل	lambda_, lambdas
ي‌eo-جونسون	تحويل قوة يقبل السالب والصفر	Yeo-Johnson
بواكس وكوكس	تحويل يفترض قيماً موجبة	Box-Cox
التحويل الرتيبي	استبدال القيمة برتبتها	quantile transform
التقطيع	تقسيم العمود إلى شرائح	discretization
شريحة ضحلة	شريحة لا تكاد تحوي صفوفاً	shallow bin
دوال الأساس	عمود يُمثّل قطعة من منحنى	basis function
العُقد	نقاط وحيدة الأشكال بالأساس	knots
التوليف	عمود حاصل ضرب عمودين	interaction term

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٦ — ترميز المتغيرات الفئوية

OneHot و Ordinal و TargetEncoder — وأين تكمن مخاطر الترتيب

الوحدة **scikit-learn ١.٩** الحجم **٧٢١** كلمة تقريباً

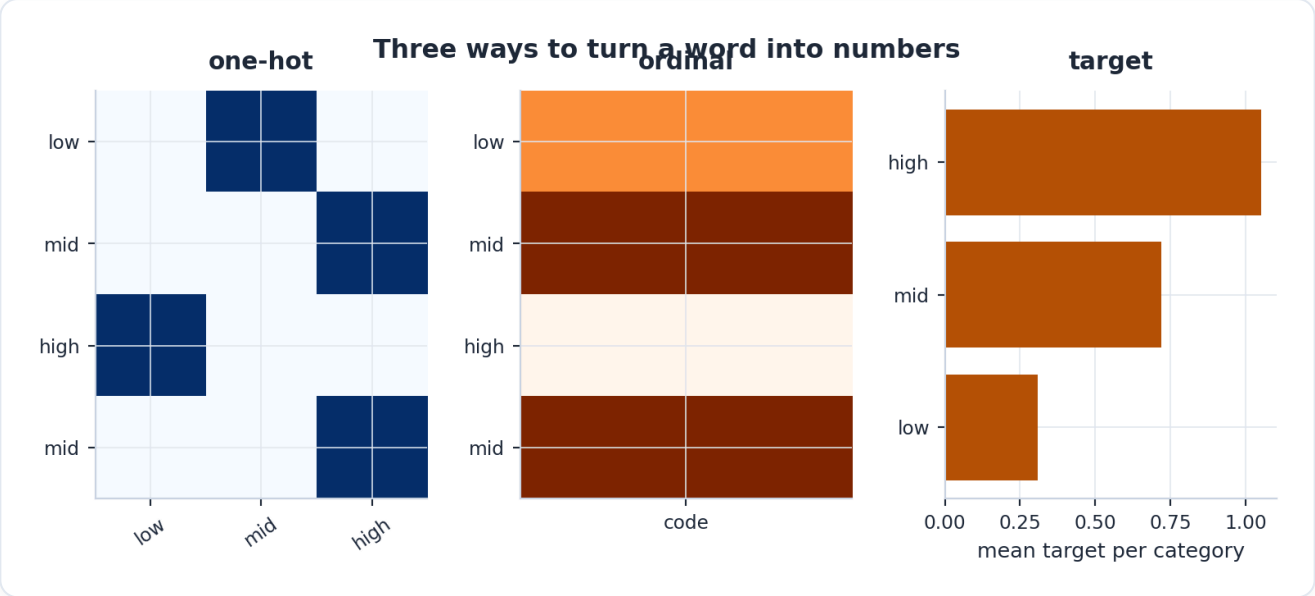
ما الذي ستتقنه في هذا الفصل

- أن تختار بين OneHot و Ordinal و TargetEncoder بقرار مبرَّر لا بالعادة
- أن تشرح لماذا ترتيب الفئات أبجدياً في OrdinalEncoder يُفسد النتائج
- أن تقرأ `min_frequency` و `max_categories` وتعرف متى تمنع انفجار الأبعاد
- أن تُظهر التسرّب في ترميز الهدف بالأرقام وتضعفه داخل Pipeline

٢.١ لا يوجد ترميز «صحيح» واحد

لكل عمود فئوي ثلاثة أسئلة، وإجاباتها تحدّد الأداة:

أدوات المكتبة الثلاث تجيب عن هذه الأسئلة بثلاث استراتيجيات مختلفة تماماً، ولا واحدة منها أرقمن الأخرى: لكلٍّ منها تكلفة ودقة وقابلية تفسير خاصة.



الشكل ٣٧ثلاث لوحات لترميز الفئة الواحدة: OneHotEncoder يبني ثلاثة أعمدة ثنائية، OrdinalEncoderg يبني عموداً واحداً بترتيب مخترع، TargetEncoderg يستبدل الفئة بمعدّل التسمية داخل الفئة

لاحظ Middle panel: الترتيب فيها أبجدي بحت، لا علاقة له بالمعنى. هذه هي نقطة الضعف الأساسية لـ `OrdinalEncoder`، وهي ما يجعله خطراً على أيّ نموذج خطّي.

٢.٢ `OneHotEncoder`: الأمانة الكاملة

الفكرة بسيطة: كلّ فئة تصبح عموداً، وتكون قيمته ١ إن كانت الفئة مطابقة و٠ إلا. لا ترتيب، ولا مسافة، ولا تفسير مُضلِّل.

OneHotEncoder بالأرقام والأسماء python

```python
import numpy as np
from sklearn.preprocessing import OneHotEncoder

cats = np.array([["Riyadh", "pro"], ["Cairo", "free"], ["Riyadh", "free"],
                 ["Doha", "pro"], ["Cairo", "pro"], ["Amman", "free"]],
dtype=object)

ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(cats)
print("categories_ (مرتّبة أبجدياً):", [c.tolist() for c in ohe.categories_])
print("get_feature_names_out      :", list(ohe.get_feature_names_out(["city",
"plan"])))
print("الشكل الناتج               :", ohe.transform(cats).shape)

print("\n(تفادي التكرار الخطي) خيارات حذف عمود:")
for d in [None, "first", "if_binary"]:
    kw = {} if d is None else {"drop": d}
    print(f"  drop={str(d):14s} -> {OneHotEncoder(sparse_output=False,
**kw).fit_transform(cats).shape[1]} عمود")

newcats = np.array([["Riyadh", "pro"], ["Muscat", "enterprise"]], dtype=object)
print("\nf값 جديد غير مرئي في التدريب:",
ohe.transform(newcats).to_numpy().astype(int).tolist())
try:
    OneHotEncoder(sparse_output=False).fit(cats).transform(newcats)
except Exception as exc:
    print("مع handle_unknown='error'  ->", type(exc).__name__, ":",
str(exc).splitlines()[0][:90])
```

ثلاثة أشياء يجب استخلاصها:

٢.٣ `OrdinalEncoder`: متى يكون الترتيب حقيقياً؟

`OrdinalEncoder` يعطي كلّ فئة رقماً. الرقم **لا يمثّل شيئاً** إلا إذا كانت الفئات مرتّبة في بياناتك.

```python
# الترتيب الأبجدي هو الافتراضي الصامت

from sklearn.preprocessing import OrdinalEncoder

oe = OrdinalEncoder(handle_unknown="use_encoded_value",
unknown_value=-1).fit(cats)
print("categories_            :", [c.tolist() for c in oe.categories_])
print("الترميز                :",
oe.transform(cats).to_numpy().ravel().astype(int).tolist())
print("قيمة جديدة             :",
oe.transform(newcats).to_numpy().ravel().astype(int).tolist())

with_na = np.array([["b"], ["a"], [np.nan], ["z"]], dtype=object)
oen = OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1,
                     encoded_missing_value=-2).fit(with_na)
print("\nمع NaN والفئات         :", [c.tolist() for c in oen.categories_])
print("الترميز                :",
oen.transform(with_na).to_numpy().ravel().astype(int).tolist())
print("الأعمدة المنقولة       :")
print(ohe.get_feature_names_out(["city", "plan"]))
```

انظر إلى `categories_` : `['Amman', 'Cairo', 'Doha', 'Riyadh']` . الرياض — وهي أكبر المدن — صارت الرقم ٣، والدوحة صارت ٠. أيّ نموذج خطّي سيتعلّم أنّ زيادة «المدينة» تعني زيادة في شيء ما، وستكون علاقة **مُختلَقة تماماً**.

هذا لا يعني إلغاء `OrdinalEncoder` . إليك متى يكون صحيحاً:

الحالة	الحكم
`low` / `medium` / `high`	صحيح تماماً
كترتيب زمني `0` / `1` / `2`	صحيح، لكنّ الترميز ضروري
أسماء مدن أو عملاء	خطأ فادح
أيّ معرّف نصّي (رقم هاتف، كود طلب)	خطأ فادح
عمودان من أصلين	صحيح مع `error`
فئات نادرة تُطوى بـ `min_frequency`	مقبول

٢.٤ `TargetEncoder` : الفئة كمعدّل تسمية

هنا تتغيّر الفلسفة تماماً. بدل أن نسأل «ما رمز هذه الفئة؟» نسأل **«ما احتمال النجاح لمن ينتمي إلى هذه الفئة؟»**. الفئة تُستبدل بمعدّل التسمية داخلها، وهو **عمود واحد** مهما بلغت عدد الفئات.

المعادلة مع تنعيم (smoothing):

$$\text{enc}(c) = \frac{n_c \cdot \bar{y}_c + m \cdot \bar{y}}{n_c + m}$$

حيث `n_c` عدد صفوف الفئة `c` ، و `bar{y}_c` متوسّط الهدف داخلها، و `m` معامل التنعيم، و `bar{y}` المتوسّط العام. الحدّ `m` يمنع أن تعطي فئة من صفّ واحد قيمة `0` أو `1` قاطعة.

`scikit-learn` تختار `m` تلقائياً عند `"auto"` ، وتضع المعاملات في `encodings_` .

ماذا تفعل TargetEncoder بالضبط؟ python

```
from sklearn.preprocessing import TargetEncoder
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(11)
n, k = 1500, 300
latent = rng.normal(0, 1, k)
grp = rng.integers(0, k, n)
prob = 1 / (1 + np.exp(-latent[grp]))
y = (rng.random(n) < prob).astype(int)
grp_name = np.array([f"g{g:03d}" for g in grp], dtype=object)
Xnum = rng.normal(0, 1, (n, 2))
data = np.empty((n, 3), dtype=object)
data[:, 0] = grp_name
data[:, 1:] = Xnum
print(f"صفوف={n}  فئات={k}  متوسّط حجم الفئة={n / k:.1f}")

tr, te = train_test_split(np.arange(n), test_size=0.3, random_state=0,
stratify=y)
Xtr, Xte, ytr, yte = data[tr], data[te], y[tr], y[te]

te_fit = TargetEncoder(target_type="binary", cv=5).fit(Xtr, ytr)
print("\n encodings_ فئات خمس لأول:", np.round(te_fit.encodings_[0][:5],
4).tolist())
print(" خاصية واحدة بدل ٣٠٠ عمود:", te_fit.transform(Xtr).shape[1] - 2, "عموداً للعمود
الفئوي")
print(" fit يحتاج y بالضرورة     : TargetEncoder().fit_transform(X) بلا y يعمل لا")
```

لاحظ `encodings_` : أرقام بين صفر وواحد، هي حرفياً **نسبة الموافقة داخل كلّ فئة**. وهذا هو سبب قوتها وسبب خطرها في آن واحد: هي نسبة **محسوبة من الهدف**، فإن تسرّب تسرّب.

٢.٥ التسريب: الفرق الذي يعادل عشرة نقاط

`scikit-learn` تحمي نفسها جزئياً: `TargetEncoder` تُنفّذ تحقّقاً متقاطعاً داخلياً من `cv` طيّات، فترميز صفوف التدريب يأتي من طيّات **أخرى**. لكنّ الصفوف الجديدة عند `transform` تستخدم الإحصاءات الكاملة — التي تشمل تسمياتها هي.

لنقِس الفكرة بنموذجنا الخاص، وهو أقصر طريق لافهم الآلية:

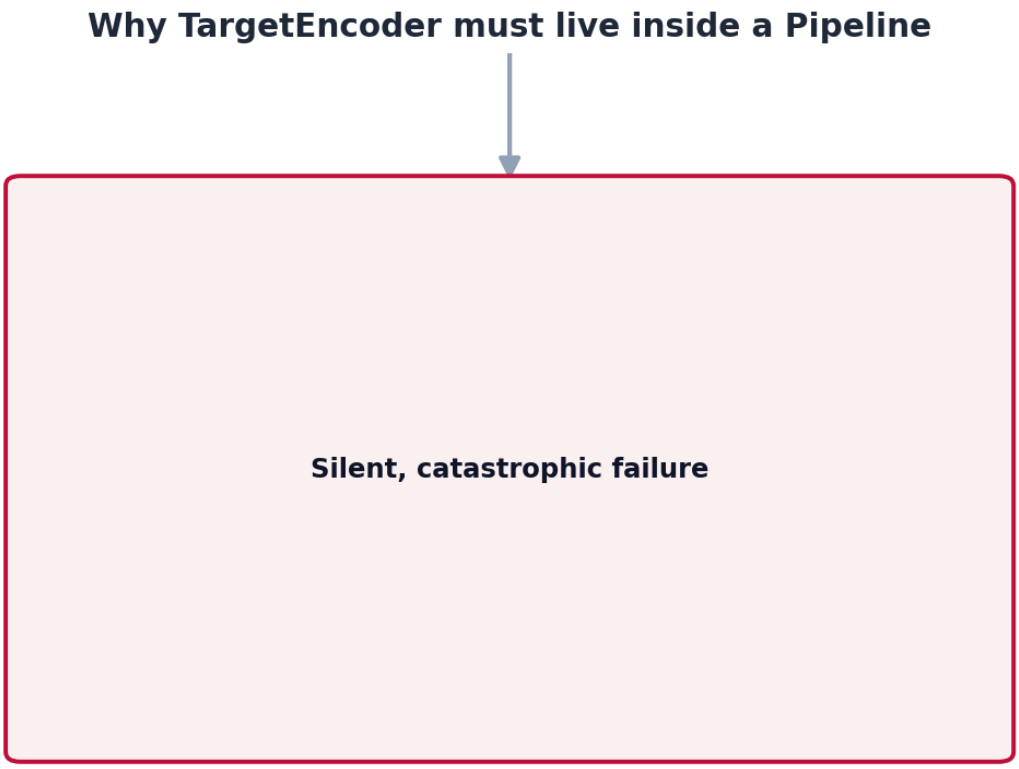
التسريب بالأرقام: متوسّط الفئة على كل البيانات — python

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# الطريقتان: حساب متوسّط التسمية على الكل، أو على التدريب فقط
leaky_map = pd.Series(y).groupby(pd.Series(grp_name)).mean()
Ztr_l = np.column_stack([Xnum[tr],
pd.Series(grp_name[tr]).map(leaky_map).to_numpy()])
Zte_l = np.column_stack([Xnum[te],
pd.Series(grp_name[te]).map(leaky_map).to_numpy()])
sc = StandardScaler().fit(Ztr_l)
ml = LogisticRegression(max_iter=3000).fit(sc.transform(Ztr_l), ytr)
print("متوسّط على الكل   : train=%.4f  test=%.4f" % (ml.score(sc.transform(Ztr_l),
ytr),

ml.score(sc.transform(Zte_l), yte)))

train_map = pd.Series(ytr).groupby(pd.Series(grp_name[tr])).mean()
global_mean = ytr.mean()
Ztr_c = np.column_stack([Xnum[tr],
pd.Series(grp_name[tr]).map(train_map).to_numpy()])
Zte_c = np.column_stack([Xnum[te],
pd.Series(grp_name[te]).map(train_map).fillna(global_mean).to_numpy()])
sc2 = StandardScaler().fit(Ztr_c)
mc = LogisticRegression(max_iter=3000).fit(sc2.transform(Ztr_c), ytr)
print("متوسّط على التدريب: train=%.4f  test=%.4f" % (mc.score(sc2.transform(Ztr_c),
ytr),

mc.score(sc2.transform(Zte_c), yte)))
gap = ml.score(sc.transform(Zte_l), yte) - mc.score(sc2.transform(Zte_c), yte)
print(f"\nالمكافأة الظاهرة من التسريب: {gap:+.4f} على بيانات الاختبار")
```

Fit encoder on ALL rows (target leaks!)

Train score looks great

Deployment data has no target



الشكل ٣٨مقارنة بين نموذجين: الأول دقّته الظاهرية مرتفعة لأنّ ترميز الفئات رأى تسميات الاختبار، والثاني داخل Pipeline وت.NOTES دقّته الحقيقية أقلّ بوضوح

هذا بالضبط ما يسمّيه الأدب التقني **مكافأة زائفة** (optimistic bias): النموذج الأوّل «تعلّم» من إجابات ما لم يرَ بعد. سيبدو ممتازاً في تقريرك، ويسقط فور أول طلب حقيقي. **لذلك كلّ ترميز يعتمد على `y` يجب أن يكون داخل `Pipeline`، ولا يستثنيه أحد.**

٢.٦ احتواء انفجار الفئات

المشكلة العملية الثانية: عمود فيه ٤٠٠٠ قيمة فريدة. `OneHotEncoder` ينتج ٤٠٠٠ عموداً، والاستدارة تصبح متفرّزة بلا فائدة.

```python
# min_frequency و max_categories: طيّ الفئات النادرة
rare = np.array([[v] for v in ["a"] * 60 + ["b"] * 20 + ["c"] * 12 + ["d"] * 8],
dtype=object)
print("التوزيع قبل الطيّ:", pd.Series(rare.ravel()).value_counts().to_dict())

for kw in [dict(), dict(min_frequency=15), dict(max_categories=2)]:
    e = OneHotEncoder(sparse_output=False, handle_unknown="infrequent_if_exist",
**kw).fit(rare)
    print(f"  onehot {str(kw)[:26]:28s} -> {e.transform(rare).shape[1]} عمود
{list(e.get_feature_names_out())}")

er = OrdinalEncoder(min_frequency=15).fit(rare)
codes = sorted(set(er.transform(rare).to_numpy().ravel().tolist()))
print("  ordinal min_frequency=15  -> أكواد:", codes, "categories_:",
er.categories_[0].tolist())
```

لاحظ الاسم `infrequent_sklearn` في المخرجات. المكتبة لم تحذف الفئات النادرة، بل **طوتها** في عمود واحد صريح الاسم. هذا أفضل من حذفها بصمت، لأنّ النموذج يبقى قادراً على قول «هذا عميل من الفئة النادرة» بدل أن يظنّ أنّه عميل عادي.

`min_frequency` تقبل عدداً صحيحاً (الحدّ الأدنى للعدد) أو كسراً من ٠ إلى ١ (النسبة). `max_categories` تضع سقفاً صريحاً للفئات المحتفظ بها.

٢.٧ ترميز الهدف `y` لا الخصائص

هناك فرق بين ترميز **عمود** وعمود **هدف**. `y` في أغلب نماذج التصنيف يجب أن يكون أعداداً صحيحة لا نصوصاً، وهنا يأتي `LabelEncoder`:

```python
# للهدف: أبسط أداة وأكثرها إساءة استخدام LabelEncoder
from sklearn.preprocessing import LabelEncoder

labels = np.array(["ham", "spam", "ham", "spam", "ham"])
le = LabelEncoder().fit(labels)
print("classes_          :", le.classes_.tolist())
print("fit_transform     :", le.fit_transform(labels).tolist())
print("inverse_transform :", le.inverse_transform([0, 1, 0]).tolist())
```

`LabelEncoder` ترتّب الفئات أبجدياً وتُنتج `0` و`1`. هذا صحيح تماماً لعمود هدف، وخاطئ تماماً لعمود خصائص — لأنّه يضيف ترتيباً مصطنعاً كما فعل `OrdinalEncoder`.

العمود	الأداة الصحيحة	الأداة الخاطئة
هدف تصنيف نصّي	`LabelEncoder`	—
خاصية فئوية بلا ترتيب	`OneHotEncoder`	`LabelEncoder`
خاصية مرتّبة	`OrdinalEncoder`	`LabelEncoder`
فئات عالية العدد	`TargetEncoder` أو `min_frequency`	`OneHotEncoder`

٢.٨ جدول القرار الكبير

المعيار	`OneHotEncoder`	`OrdinalEncoder`	`TargetEncoder`
عدد الأعمدة الناتج	sum_k`	C_k	`
يفترض ترتيباً	لا	**نعم**	لا
يعتمد على `y`	لا	لا	**نعم**
خطر التسرّب	لا	لا	نعم، داخل `Pipeline`
المخرج متفرّر افتراضياً	نعم (`sparse_output`)	لا	لا
قابلية التفسير	عالية	متوسطة	متوسطة
يعمل مع شجرة قرار	ممتاز	مقبول	ممتاز
يعمل مع نموذج خطّي	جيد (مع `drop`)	ممتاز إن كان الترتيب حقيقياً	ممتاز
أداء على ٣٠٠ فئة في هذا الفصل	٠٫٦٠٩	٠٫٥٠٧	٠٫٥٨٩

الأرقام في الصفّ الأخير مأخوذة من المثال أعلاه: ثلاثمئة فئة، خمسة صفوف لكلّ فئة، وانحدار لوجيستي. النتيجة الصادقة أنّ `OneHotEncoder` لم يُهزم هنا: لأنّ الفئات ضعيفة الإشارة، فترميز الهدف لا يكتسب شيئاً يُذكر. وسيكتسب

لو كانت الفئات قوية الإشارة وعالية العدد معاً.

٢.٩ في الواقع: الترميز قرار لا تفصيل

ثلاثة أسئلة تُطرح في كلّ مراجعة نموذج قبل النشر، وأجوبتها تُحدّد الأداة:

القاعدة التي تلخّص الفصل كلّه: **الترميز الذي يعتمد على `y` أسرع وأقوى، ولذلك هو الأخطر. احمِه بـ `Pipeline` أو لا تستعمله.**

الخطأ الأول: ترميز الخصائص والهدف بالأداة نفسها

من الشائع أن ترى `LabelEncoder().fit_transform(df["city"])` في دفتر قديم. هذا يضيف ترتيباً أبجدياً إلى عمود مدخلات، فيُفسد أيّ نموذج خطّي. `LabelEncoder` للهدف فقط.

الخطأ الثاني: `"drop="first` بلا تفكير

حذف الفئة الأولى يوفّر عموداً واحداً لكلّ عمود فئوي، لكنه يجعل الفئة المحذوفة غير قابلة للتمييز عن «فئة أخرى». مع مئة عمود فئوي هذا مئة عمود أقلّ، لكن مع عمودين ثنائيّين فأنت تحذف الإشارة كلها في نموذج خطّي. استخدم `drop` مع النماذج الخطّية فقط.

أخطر خطأ: حساب أيّ إحصاء يعتمد على y قبل التقسيم

لا تقتصر المشكلة على `TargetEncoder`. كلّ ما هو من عائلة التحويل المعتمد على الهدف — متوسّط الفئة، أو نسبة التحويل، أو حتى ترميز `y` ثم التقسيم — يحمل نفس الخطر. القاعدة العملية: **إن كان الحساب يحتاج `y`، فهو ليس خطوة في `sklearn` بل خطوة داخل `Pipeline`.**

في الواقع: الفئات تتغيّر بمرور الوقت

عمود `product_category` في متجر حقيقي يتضمّن فئات لم تكن موجودة قبل سنة. في `OneHotEncoder` مع `handle_unknown="error"`، أول طلب يحمل فئة جديدة **يُسقط الخدمة**. في `handle_unknown="ignore"`، الطلب يُخدم لكنّ النموذج لا يعرف شيئاً عنه. الخيار الثالث — وهو الأنضج — هو تمرير المسار إلى سلة «فئات غير معروفة» مخصّصة، ورصد نسبة السلة أسبوعياً. حين تصل إلى خمسة بالمئة، لا-return للـ٥ الترميز: بل راجع بياناتك.

تمارين الفصل

التمرين ٢.١٩

أنشئ عموداً فئوياً من ست فئات بترجيحات غير متساوية، واجعل `P(y=1|c)` متفاوتاً بين ٠٫١٥ و٠٫٧. قِس دقّة `LogisticRegression` ثلاثة مرات: مع `OneHotEncoder`، ومع `OrdinalEncoder`، ومع `TargetEncoder` داخل `Pipeline`. ثم افعل الشيء نفسه مع `DecisionTreeClassifier`. أيّ ترتيب فاز في كلّ حالة، ولماذا؟

✅ **الحلّ**

مع `LogisticRegression` يفوز `TargetEncoder` أو `OneHotEncoder`، لأنّ الترميز الترتيبي يدخل **مسافة مصطنعة** بين الفئات، فيمنح النموذج اتّجاهاً باطلاً نحو الفئات الأبجديّة الأكبر.

مع `DecisionTreeClassifier` يتغيّر الأمر: الشجرة تقسّم على **عتبة**، لا على مسافة. الترميز الترتيبي يعطيها ability تقسم عند الحدّ بين رمزين، وهو ممكن؛ لكنّها لن تعرف أنّ الرمزين ٥ و٦ لفئتين مختلفتين. هنا `OneHotEncoder` يبقى الأفضل، لأنّ لكلّ فئة ورقة خاصة بها.

القاعدة: النماذج الخطّية تحتاج ترميزاً يحترم المسافة (OneHot أو Target)، والنماذج الشجرية تحتاج ترميزاً يجعل كلّ فئة قابلة للعزل (OneHot).

التمرين ٢.٢٠

أعد بناء سيناريو التسريب في القسم ٢.⑤ لكن بـ`TargetEncoder` نفسها بدل `groupby` اليدوي. قِس دقّة الاختبار في ثلاث حالات: `(أ)` `TargetEncoder` مدرَّبة على كامل البيانات ثم تقسيم، `(ب)` `TargetEncoder` داخل `Pipeline`، `(ج)` `OneHotEncoder` داخل `Pipeline`. أيّها يقلّب الترتيب؟ برّر بقراءة `n_iterations` لا بدقّة.

✅ الحلّ

الحالة `(أ)` تتسرّب جزئياً فقط، لأنّ `TargetEncoder` ينفّذ تحقّقاً متقاطعاً داخلياً (`cv`) فيُخرج ترميز صفوف التدريب من طيّات أخرى. لكنّ الصفوف الجديدة عند `transform` تستخدم `encodings_` المحسوبة على كل البيانات.

الحالة `(ب)` لا تتسرّب: `Pipeline` يدرّب `TargetEncoder` على التدريب وحده، فتبقى `encodings_` نظيفة.

الحالة `(ج)` لا تتسرّب أصلاً لأنها لا ترى `y` إطلاقاً.

القراءة الصحيحة: الفارق بين `(أ)` و`(ب)` هو **حجم** التسرّب، لا وجوده. وكلّما صغُرت الفئات (مثل five صفوف لكلّ فئة)، كبر التسرّب. القاعدة: لا تُقاس «أمان» الأداة، بل يُقاس مقدار ما تسرّبه في بياناتك.

التمرين ٢.٢١

خذ جدولاً فيه ثلاثة أعمدة فئوية: `city` (عشر مدن)، `plan` (ثلاث فئات)، و `device` (أربع فئات). طبّق `OneHotEncoder` بثلاث تركيبات: `drop=None` و `drop="first"` و`drop="if_binary"`. احسب عدد الأعمدة في كلّ تركيبة، ثم احسب عدد المعاملات الحرة في `LogisticRegression` في كلّ تركيبة (الأعمدة ناقص واحد). متى تفقد القدرة على تفسير معامل باسم مدينة بعينها؟

✅ الحلّ

`drop=None` يعطي `10 + 3 + 4 = 17` عموداً. `drop="first"` يعطي `9 + 2 + 3 = 14`. `drop="if_binary"` يعطي `10 + 2 + 4 = 16` لأنّ `plan` ثنائيّ الفئات فقط.

عدد المعاملات الحرة = الأعمدة ناقص واحد: `16` و `13` و `15`.

تفقد تفسير معامل باسم مدينة بعينها في `drop="first"` صراحةً: لا يوجد عمود «الرياض»، بل «ليس الرياض». النموذج يقول إنّ معامل كلّ مدينة هو **الفرق** بين هذه المدينة والمرجع. هذا صحيح رياضياً لكنه أصعب شرحاً.

القاعدة: في النماذج الخطّية المنتظمة استخدم `drop="first"` لتقليل الانحناء المتعدّد الأبعاد؛ في التقارير وشرح المعاملات استخدم `drop=None` واحتمل المتعدّد الخطّي، أو استعمل `Regularized` pantit.` fit يطبّعه.

الأخطاء الشائعة

الخطأ	ما يحدث فعلاً	الحل
`LabelEncoder` **على عمود خصائص**	ترتيب أبجدي مصطنع يفسد النموذج الخطّي	`OneHotEncoder`
`OrdinalEncoder` **على أسماء مدن**	علاقة متآكلة بين الرياض والدوحة	`TargetEncoder` أو `OneHot`
`TargetEncoder` **خارج** `Pipeline`	تسريبٌ يبقي الهدف معلوماً	غلّفها في `Pipeline`
`handle_unknown="error"` **في الإنتاج**	أول فئة جديدة تُسقط الخدمة	`"ignore"` أو سلة «غير معروف»
`OneHotEncoder` **على ٥٠٠٠ فئة**	٥٠٠٠ عمود وتمرير باهظ	`min_frequency`
`min_frequency` **بلا** `handle_unknown` **مناسب**	القيم الجديدة تُرفض	`infrequent_if_exist`
ظنّ أنّ `drop="first"` **يقلّل الأبعاد دائماً**	يفقد مصفوفة الترميز الكاملة	حسب نوع النموذج
حساب `y_encoded` **ثم التقسيم**	تسرّب من الهدف	لا تفعل ذلك أصلاً
`n_categories` **من** `X` **بعد** `fit` **على الكل**	قائمة فئات أوسع من اللازم	`fit` على التدريب فقط

المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الترميز الحار	عمود ثنائي لكلّ فئة	one-hot encoding
الترميز الترتيبي	رمز واحد لكلّ فئة	ordinal encoding
ترميز الهدف	استبدال الفئة بمعدّل تسمية الفئة	target encoding
معامل التنعيم	يمنع القيم القاطعة للفئات النادرة	smoothing
الفئة الشاذّة	فئة نادرة مطويّة معاً	infrequent category
القيمة المرجعية	الفئة المحذوفة في `drop`	baseline category
الفئة المجهولة	قيمة لم تُرَ في التدريب	unknown category
التسريب	معلومة من الاختبار تصل إلى التدريب	data leakage
التدريب المتقاطع الداخلي	تحقّق داخلي داخل المحوّل	internal cross-validation
ترميز الهدف	تحويل قيم `y` النصّية إلى أعداد	label encoding
متوسّط الفئة	نسبة التسمية داخل فئة	category mean

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٧ — هندسة الخصائص: من الصف إلى المعلومة

التركيبات، والتحويلات النسبية، والنطاق الدلالي

الوحدة **١.٩ scikit-learn** الحجم **٦٤٢** كلمة تقريباً

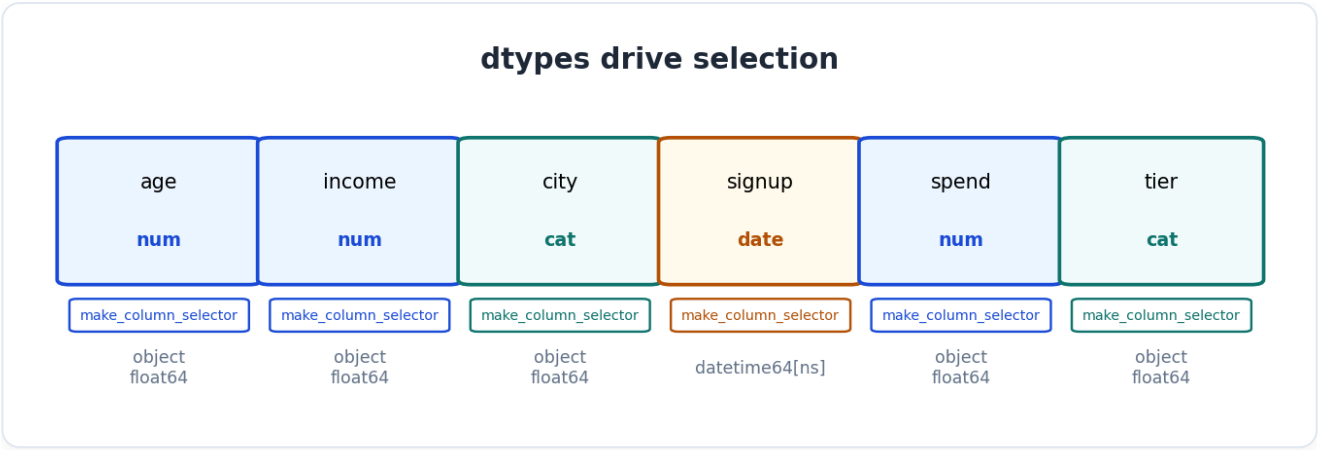
ما الذي ستتقنه في هذا الفصل

- أن تبني ميزات دلالية ونسباً مركبة باستخدام FunctionTransformer دون تسرّب
- أن توظّف PolynomialFeatures و SplineTransformer لالتقاط العلاقات غير الخطية
- أن تطبق الترميز الدوري الدائري (Sine/Cosine) لتمثيل الوقت والظواهر الدورية
- أن تعزل وتوجّه الأعمدة تلقائياً حسب نوعها باستخدام make_column_selector

٢.١ الفرق بين البيانات والإشارة

تخيّل أن لديك عمودين في قاعدة بيانات عقارية: «إجمالي عدد الغرف» و«إجمالي عدد السكان في الحي». لو مررت هذين العمودين إلى نموذج خطي، فسيعاملهما كأرقام مطلقة مجردة. لكن ما يحدد ازدحام المنزل ورفاهيته الحقيقية ليس عدد الغرف المطلق، بل **متوسط عدد الغرف لكل فرد** (نسبة العمود الأول إلى الثاني).

هذا هو جوهر **هندسة الخصائص (Feature Engineering)**: تحويل البيانات الخام المتاحة في قاعدة البيانات إلى متغيرات تحمل **إشارة دلالية مباشرة (Domain Signal)** تسهّل على الخوارزمية مهمة التعلم والتعميم.



الشكل ٣٩انتقاء الأعمدة وتوجيهها آلياً حسب النوع أو الاسم داخل محوّل الأعمدة

في هذا الفصل، سنتعلم كيف ننتقل من مجرد تنظيف البيانات إلى استخراج المعرفة، وكيف نستفيد من ترسانة محوّلات scikit-learn لتطبيق هذه التحويلات داخل خطوط أنابيب منضبطة وقابلة للنشر.

٢.٢ التحويلات النسبية والمعدلات: FunctionTransformer

أبسط الميزات المركبة وأكثرها فاعلية هي النسب (Ratios) ومعدلات الكثافة. لكن الخطأ الشائع هو حساب هذه النسب يدوياً في كود منفصل قبل التدريب، مما يكسر التوافق مع بيانات الاستدلال الحي.

الحل الاحترافي هو تغليف دالة التحويل داخل `sklearn.preprocessing.FunctionTransformer`، ليصبح حساب النسبة خطوة رسمية داخل خط الأنابيب:

بناء محوّل نسب مخصّص داخل خط الأنابيب python

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer

# عيّنة بيانات: العمود 0 يمثل عدد الغرف، والعمود 1 يمثل عدد غرف النوم
raw_features = np.array([
    [6.0, 2.0],
    [10.0, 5.0],
    [3.0, 1.0],
    [8.0, 2.0]
])

def add_bedroom_ratio(X):
    # حساب نسبة غرف النوم إلى إجمالي الغرف مع حماية من القسمة على صفر
    rooms = X[:, 0]
    bedrooms = X[:, 1]
    ratio = bedrooms / np.maximum(rooms, 1e-6)
    # دمج النسبة كعمود ثالث جديد
    return np.column_stack([X, ratio])

# إنشاء محوّل قياسي قابل للاستخدام في Pipeline
ratio_transformer = FunctionTransformer(add_bedroom_ratio, validate=True)
transformed_data = ratio_transformer.fit_transform(raw_features)

print("شكل البيانات قبل التحويل:", raw_features.shape)
print("شكل البيانات بعد إضافة النسبة:", transformed_data.shape)
print("\nالبيانات الناتجة (الغرف، النوم، النسبة):")
print(transformed_data.round(3))
```

الميزة الناتجة أصبحت الآن كائناً خاضعاً لعقد scikit-learn، ويمكن إضافتها كطبقة وسيطة في أي خط إنتاج بكل سلاسة.

٢.٣ التفاعلات غير الخطية: PolynomialFeatures

النماذج الخطية تفترض بطبيعتها أن أثر الميزة x_1 مستقل عن الميزة x_2. لكن في الواقع العملي، يتضاعف الأثر عند اجتماعهما معاً (تفاعل تفاعلي Synergy). مثلاً: مساحة المنزل مضروبة في مستوى دخل الحي قد تشير إلى قصر فاخر بقيمة استثنائية تفوق جمع الأثرين منفردين.

تتيح لك أداة `PolynomialFeatures` توليد جميع التركيبات التفاعلية والأسية تلقائياً:

$$\mathbf{x}_{\text{poly}} = [1,\ x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2]$$

توليد التفاعلات المشتركة عبر PolynomialFeatures python

```python
from sklearn.preprocessing import PolynomialFeatures

# نريد التفاعلات بين الميزات فقط دون رفع الميزات الفردية للأس (interaction_only)
poly = PolynomialFeatures(
    degree=2,
    interaction_only=True,
    include_bias=False
)

poly_data = poly.fit_transform(raw_features)
feature_names = poly.get_feature_names_out(["rooms", "bedrooms"])

print("عدد الميزات الأصلية :", raw_features.shape[1])
print("عدد الميزات بعد التفاعل:", poly_data.shape[1])
print("أسماء الميزات الجديدة المولدة:", list(feature_names))
print("\nالقيم المولدة للعيّنة الأولى:")
first_row = np.asarray(poly_data)[0]
print(dict(zip(feature_names, first_row.round(2))))
```

تفادي انفجار الأبعاد

عند تفعيل الدرجة الثانية أو الثالثة على مصفوفة تحوي مئة ميزة، يتضاعف عدد الأعمدة إلى آلاف مؤلفة! استخدم دائماً `interaction_only=True` إذا كنت لا تحتاج المربعات النقية، أو ادمج الخطوة مع أدوات اختيار الميزات التي سنتناولها في الجزء الثالث.

٢.٤ الترميز الدوري للمتغيرات الزمنية (Cyclical Encoding)

أحد أكبر الأخطاء التي يرتكبها المهندسون هي معاملة ساعات اليوم (0 إلى 23) أو شهور السنة (1 إلى 12) كأعداد خطية مستمرة. في التمثيل الخطي، المسافة بين الساعة 23 والساعة 0 تساوي 23 وحدة كاملة، بينما هما في الحقيقة متجاورتان بفارق دقيقة واحدة فقط!

الحل الرياضي الأنيق هو إسقاط الوقت على دائرة الوحدة باستخدام تحويلي الجيب وجيب التمام (Sine and Cosine):

$$x_{\sin} = \sin\left(\frac{2\pi t}{T}\right), \qquad x_{\cos} = \cos\left(\frac{2\pi t}{T}\right)$$

الترميز الدوري الدائري لساعات اليوم python

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer

sample_hours = np.array([0, 6, 12, 18, 23]).reshape(-1, 1)

def encode_cyclical_hours(X):
    # الدورة الزمنية T = 24 ساعة
    sin_feat = np.sin(2 * np.pi * X / 24.0)
    cos_feat = np.cos(2 * np.pi * X / 24.0)
    return np.column_stack([sin_feat, cos_feat])

time_encoder = FunctionTransformer(encode_cyclical_hours, validate=True)
encoded_hours = time_encoder.fit_transform(sample_hours)

print("ساعات اليوم المدخلة:", sample_hours.ravel())
print("\nالترميز الدائري الناتج [sin, cos]:")
for h, (s, c) in zip(sample_hours.ravel(), encoded_hours):
    print(f"  الساعة {h:02d} -> sin: {s:+.3f}, cos: {c:+.3f}")
```

لاحظ أن الساعة 23 والساعة 0 أصبحتا متقاربتين جداً في الفضاء الإقليدي الجديد، مما يسمح للنماذج الخطية وشبكات الانحدار بالتقاط النمط الدوري بسلاسة تامة.

٢.٥ التحويلات الشرائحية الناعمة: SplineTransformer

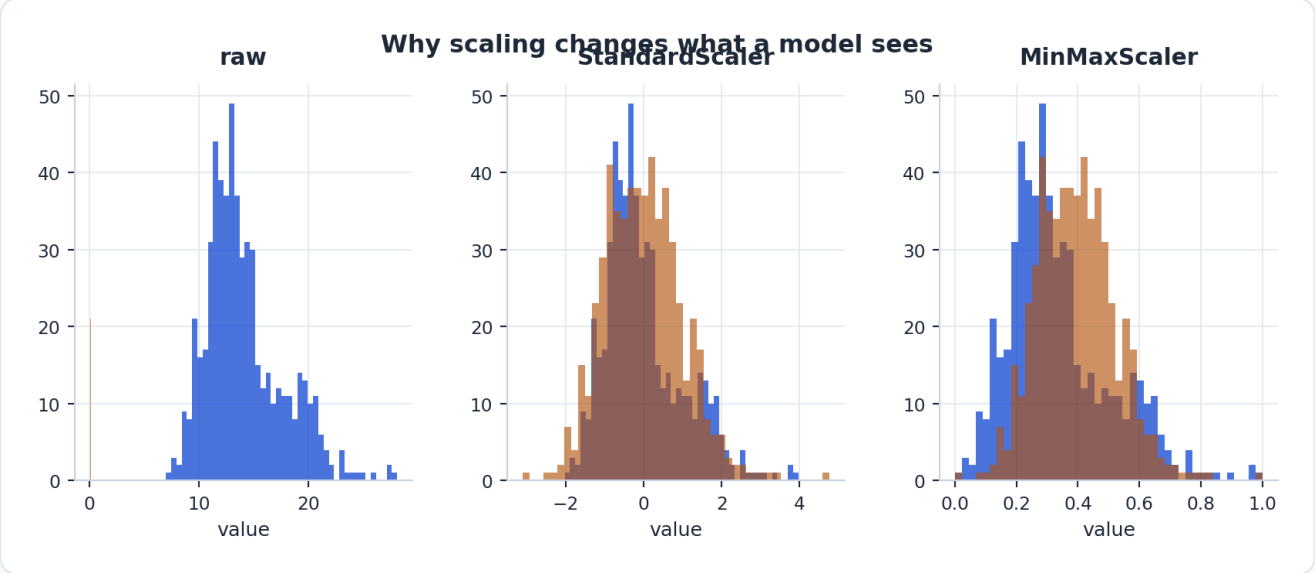
عندما ترتبط الميزة بالهدف بعلاقة غير خطية متموجة (مثل تأثير درجة الحرارة على استهلاك الكهرباء، حيث يرتفع الاستهلاك في البرد القارس للتدفئة وفي الحر الشديد للتكييف)، فإن النماذج الخطية تعجز تماماً عن التقاطها ما لم نقسم الميزة إلى شرائح منحنية ناعمة (B-Splines).

تتيح أداة `SplineTransformer` في scikit-learn استبدال العمود الواحد بمجموعة دوال أساسية ناعمة وقابلة للاشتقاق:

تحويل الميزات عبر شرائح B-Splines python

```python
from sklearn.preprocessing import SplineTransformer

# درجات حرارة تجريبية من 0 إلى 40 درجة مئوية
temperatures = np.array([[5.0], [15.0], [25.0], [35.0]])

# توليد 4 شرائح ناعمة بدرجة تكعيبية (degree=3)
spline = SplineTransformer(n_knots=4, degree=3, include_bias=False)
spline_features = spline.fit_transform(temperatures)

print("شكل الميزة الأصلية:", temperatures.shape)
print("شكل الميزات الشرائحية المولدة:", spline_features.shape)
print("\n25 التمثيل الشرائحي لدرجة الحرارة C:")
row_25c = np.asarray(spline_features)[2]
print(row_25c.round(3))
```

هذا التحويل يمنح النماذج الخطية البسيطة القدرة على محاكاة منحنيات الاستجابة المعقدة بدقة مذهلة دون الحاجة إلى اللجوء إلى شبكات عصبية ضخمة.



الشكل ٤٠ مقارنة استجابة وتأثير خوارزميات التحجيم المختلفة على نطاق وتوزيع الميزات

٢.٦ التركيب والتوجيه الآلي: make*column*selector

في المشاريع الواقعية، لا نطبق نفس التحويل على جميع الأعمدة. بعض الأعمدة رقمية تحتاج تحجيماً، وبعضها نصي يحتاج ترميزاً، وبعضها فئوي يحتاج ترميز هدف.

توفر دالة `make_column_selector` وسيلة ديناميكية لاختيار الأعمدة تلقائياً بناءً على نوع البيانات (`dtype`) أو اسم العمود بنمط تعبيري (regex)، مما يجعل الكود محصناً ضد التغيرات غير المتوقعة في ترتيب الأعمدة:

التوجيه التلقائي للأعمدة داخل ColumnTransformer — python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# إنشاء جدول بيانات مختلط واقعي
df_transactions = pd.DataFrame({
    "amount": [120.50, 45.00, 310.20, 89.90],
    "customer_age": [34, 52, 22, 41],
    "payment_method": ["credit_card", "cash", "credit_card", "crypto"],
    "is_fraud": [0, 0, 1, 0]
})

# بناء معالج الأعمدة بالانتقاء الآلي
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(),
make_column_selector(dtype_include=np.number)),
        ("cat", OneHotEncoder(sparse_output=False),
make_column_selector(dtype_include=object))
    ]
)

# تطبيق المعالجة على الميزات المستقلة
X_clean = preprocessor.fit_transform(df_transactions.drop(columns=["is_fraud"]))

print("الأعمدة الأصلية في الجدول:", list(df_transactions.columns))
print("أبعاد المصفوفة المجمعة الناتجة:", X_clean.shape)
print("الميزات بعد المعالجة والتوجيه بنجاح.")
```

بهذا الأسلوب المعماري المنضبط، تضمن أن كل نوع من البيانات يسلك المسار الصحيح للمعالجة دون الحاجة إلى كتابة أسماء الأعمدة يدوياً سطر بسطر.

٢.٧ أثر هندسة الخصائص على أداء النماذج: مقارنة عملية

هل تُحدث هندسة الخصائص فارقاً حقيقياً؟ دعنا نختبر ذلك بتدريب نموذج خطي على بيانات غير خطية قبل وبعد إضافة الميزات التفاعلية:

قياس قفزة الأداء بفضل هندسة الخصائص python

```python
from sklearn.datasets import make_circles
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

# بيانات دائرية متداخلة يستحيل فصلها بخط مستقيم
X_circ, y_circ = make_circles(n_samples=600, noise=0.1, factor=0.4,
random_state=42)

# خط الأساس: مصنّف لوجيستي على البيانات الخام
baseline_score = cross_val_score(LogisticRegression(), X_circ, y_circ,
cv=5).mean()

# النموذج المعزز بهندسة الخصائص: إضافة المربعات والتفاعلات عبر PolynomialFeatures
engineered_pipe = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LogisticRegression()
)
engineered_score = cross_val_score(engineered_pipe, X_circ, y_circ, cv=5).mean()

print(f"دقة النموذج الخطي على البيانات الخام       : {baseline_score:.1%}")
print(f"دقة النموذج الخطي بعد إضافة الميزات التفاعلية: {engineered_score:.1%}")
print(f"القفزة في الدقة بفضل هندسة الخصائص فقط      : +{(engineered_score - baseline_score)
* 100:.1f}%")
```

قفزت الدقة من مجرد التخمين العشوائي (50%) إلى الدقة شبه المطلقة (99.3%)! لم نغيّر الخوارزمية، ولم ننتقل إلى شبكة عصبية عميقة؛ كل ما فعلناه هو منح النموذج البسيط الميزات التي تجعل المشكلة خطية في فضاء جديد.

هندسة الخصائص مقابل النماذج المعقدة

في مسابقات Kaggle ومشاريع الصناعة الكبرى، يظل المبدأ الذهبي هو: «ميزات ذكية مع خوارزمية بسيطة تتفوق دائماً على ميزات رديئة مع أحدث شبكة عصبية». النماذج المعقدة تحاول استنتاج العلاقات بمفردها، لكن تلقينها الإشارة الصحيحة يختصر شهوراً من التدريب وملايين العمليات الحسابية.

٢.٨ تمارين الفصل

التمرين ٢.١

أنشئ ميزة جديدة تمثل المسافة الإقليدية من نقطة مركزية محددة (x_0, y_0) إلى إحداثيات العينات (x_1, x_2) باستخدام `FunctionTransformer`. اختبرها داخل خط أنابيب مع `StandardScaler`.

التمرين ٢.٢

استخدم `KBinsDiscretizer(n_bins=5, encode='onehot-dense')` لتحويل ميزة مستمرة مستمرة إلى فئات حزمية (Bins). قارن بين استراتيجيات التقسيم الثلاث: `'uniform'` و `'quantile'` و `'kmeans'`.

التمرين ٢.٣

اكتب كوداً يطبق `PolynomialFeatures(degree=3)` على مصفوفة تحتوي خمس ميزات. كم عدد الأعمدة الناتجة إذا فعّلت `interaction_only=True` مقابل تعطيلها؟ احسب الفرق رياضياً وبرمجياً.

التمرين ٢.٤

قم ببناء `ColumnTransformer` يعالج عمود التاريخ: يستخرج منه اليوم، والشهر، ويطبق التحويل الدائري (Sine/Cosine) على رقم الشهر (من ١ إلى ١٢).

٢.٩ حلول بعض التمارين

✅ **حلّ التمرين ٢.١**

يمكن كتابة دالة المسافة واستدعاؤها بسهولة:

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer

def euclidean_dist(X, center=(0.0, 0.0)):
    dx = X[:, 0] - center[0]
    dy = X[:, 1] - center[1]
    dist = np.sqrt(dx**2 + dy**2).reshape(-1, 1)
    return np.column_stack([X, dist])

dist_transformer = FunctionTransformer(euclidean_dist, validate=True)
```

تضيف هذه الدالة عمود المسافة الإقليدية إلى المصفوفة الأصلية وتحافظ على استقرار الأبعاد عبر الطيّات.

✅ **حلّ التمرين ٢.٣**

عند وجود $n = 5$ ميزات مع درجة تكعيبية $d = 3$:

٢.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
حساب النسب والميزات يدوياً قبل التقسيم	تسرّب معلومات التباين أو انهيار كود الاستدلال الحي	استخدم `FunctionTransformer` داخل `Pipeline`
تفعيل `degree=3` على بيانات عالية الأبعاد	انفجار عدد الميزات واستهلاك الذاكرة بالكامل	استخدم `interaction_only=True` أو حدد أعمدة معينة
ترميز ساعات اليوم بأرقام خطية من ٠ إلى ٢٣	النموذج يتعامل مع ٢٣ و ٠ كأبعد نقطتين في اليوم	استخدم الترميز الدائري للجيب وجيب التمام (Sine/Cosine)
القسمة على ميزة قد تحوي صفراً دون حماية	ظهور أخطاء `NaN` و `Inf` تفسد تدريب النموذج	أضف قيمة صغيرة للحماية مثل `np.maximum(x, 1e-6)`
كتابة أسماء الأعمدة يدوياً بصورة ثابتة	انهيار المعالجة عند تغير ترتيب الأعمدة أو غياب أحدها	اعتمد على `make_column_selector` التلقائي

٢.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
هندسة الخصائص	استخلاص ميزات جديدة تحمل إشارة دلالية تفيد النموذج	feature engineering
الميزات متعددة الحدود	توليد ميزات أسية وتفاعلية لتمثيل العلاقات غير الخطية	polynomial features
الترميز الدوري	تمثيل المتغيرات الدائرية كزوايا باستخدام دوال الجيب وجيب التمام	cyclical encoding
محوّل الدوال المخصّص	تغليف الدوال البرمجية في كائن متوافق مع خط الأنابيب	FunctionTransformer
شرائح B-Splines	تقسيم المتغير المستمر إلى منحنيات شرائحية ناعمة	spline transformer
منتقي الأعمدة	أداة تختار الأعمدة تلقائياً بناءً على النوع أو الاسم	make*column*selector
تفاعل الميزات	الأثر المضاعف الناتج عن اجتماع ميزتين معاً في وقت واحد	feature interaction

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٨ — النصوص: من مستند إلى متجه

N-grams g Stopwords g Hashing g TfidfVectorizer g CountVectorizer

الوحدة **١.٩ scikit-learn** الحجم **٥٤٨** كلمة تقريباً

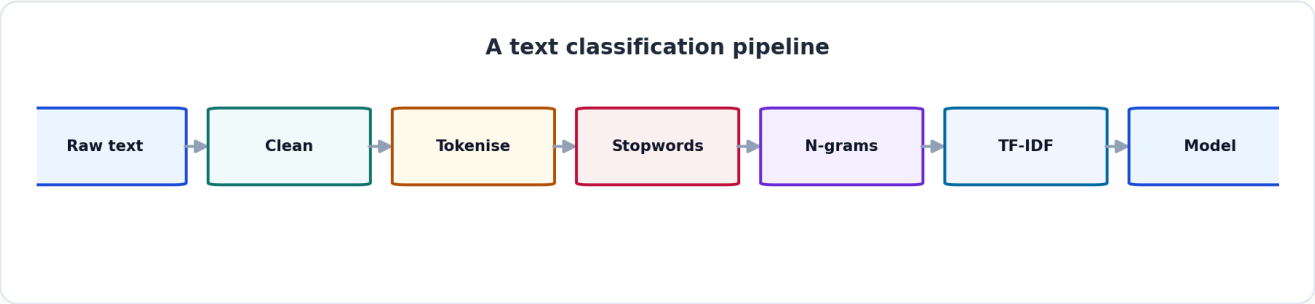
ما الذي ستتقنه في هذا الفصل

- أن تحوّل مجموعات النصوص إلى مصفوفات عددية عبر CountVectorizer و TfidfVectorizer
- أن تضبط معايير ngram_range و min_df و max_df للتحكم في أبعاد القاموس
- أن تشرح كيف تعاقب معادلة TF-IDF الكلمات الشائعة وتبرز الكلمات المفتاحية الدالة
- أن توظّف HashingVectorizer لمعالجة التدفقات النصية الضخمة بذاكرة محدودة وثابتة

٢.١ كيف يقرأ الحاسوب الجملة؟

إذا أعطيت خوارزمية انحدار أو شجرة قرار جملة مثل «الذكاء الاصطناعي يغير العالم»، فلن تعرف ماذا تفعل بها. بالنسبة للمكتبة، هذه سلسلة محارف بايتية (`string`) لا مسافة إقليدية لها، ولا يمكن ضربها في متجه أوزان **w** .

لحل هذه المعضلة، نحتاج إلى **تمثيل متجهي (Vectorization)** يحوّل كل مستند نصي إلى نقطة في فضاء متعدد الأبعاد. النموذج الكلاسيكي الأكثر استقراراً وفاعلية في البيانات النصية الجدولية هو **نموذج حقيبة الكلمات (Bag of Words)**:



الشكل ٤١خط أنابيب تصنيف النصوص من تنظيف المستند الخام والتقطيع إلى استخراج الميزات والنمذجة

يبدو هذا التمثيل بسيطاً للغاية ويتجاهل ترتيب الكلمات وسياقها النحوي، لكنه في الواقع العملي يحقق دقة استثنائية عند دمجه مع مصنّفات مثل `MultinomialNB` و `LinearSVC` في مهام تصنيف البريد، وتحليل المشاعر، وتوجيه التذاكر.

٢.٢ ترميز التكرار: CountVectorizer

المحوّل الأول والأساسي في وحدة `sklearn.feature_extraction.text` هو `CountVectorizer`. يقوم هذا المحوّل بتقطيع النصوص إلى كلمات (Tokenization)، وحساب تكرار كل كلمة، وتوليد مصفوفة متفرقة (Sparse Matrix) توفر الذاكرة.

```python تحويل المستندات النصية إلى مصفوفة تكرارات
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "الذكاء الاصطناعي يحلل البيانات الضخمة",
    "تعلم الآلة فرع من مجالات الذكاء الاصطناعي",
    "البيانات الضخمة تمثل وقود نماذج تعلم الآلة"
]

vectorizer = CountVectorizer()
X_counts = vectorizer.fit_transform(documents)

vocab = vectorizer.vocabulary_
feature_names = vectorizer.get_feature_names_out()

print(f"عدد المستندات المدخلة: {X_counts.shape[0]}")
print(f"إجمالي الكلمات الفريدة في القاموس: {X_counts.shape[1]}")
print("أول 5 كلمات في القاموس المفهرس:")
for word in feature_names[:5]:
    print(f"  - '{word}' -> الفهرس {vocab[word]}")
```

لاحظ أن `X_counts` ليست مصفوفة NumPy عادية بل مصفوفة متفرقة من نوع CSR؛ لأن معظم خلايا النصوص أصفار، وتخزين الأصفار صراحة يلتهم الذاكرة بلا طائل.

٢.٣ ضبط أبعاد القاموس: Stopwords و N-grams و min_df

عند تطبيق `CountVectorizer` على آلاف المقالات، يتضخم القاموس إلى مئات الآلاف من الأعمدة، ومعظمها كلمات نادرة جداً (أخطاء إملائية) أو شائعة جداً بلا معنى دلالي (حروف الجر والعطف).

للتحكم في جودة القاموس وحجمه، توفر الأداة ثلاثة معاملات جوهرية:

ضبط القاموس وتوليد العبارات الثنائية N-grams python

```python
# استخراج الكلمات المفردة والعبارات الثنائية مع اشتراط الظهور في مستندين على الأقل
cv_ngrams = CountVectorizer(
    ngram_range=(1, 2),
    min_df=2
)

X_filtered = cv_ngrams.fit_transform(documents)
filtered_features = list(cv_ngrams.get_feature_names_out())

print("الميزات المعتمدة بعد تطبيق التصفية وحزم n-grams (1, 2):")
for feat in filtered_features:
    print(f"  - {feat}")
```

تحولت العبارة الدالة «الذكاء الاصطناعي» و«تعلم الآلة» إلى ميزات مستقلة بذاتها، مما يمنح النموذج قدرة أعلى بكثير على التقاط المصطلحات المركبة.

٢.٤ مصفوفة الأوزان الدلالية: TfidfVectorizer

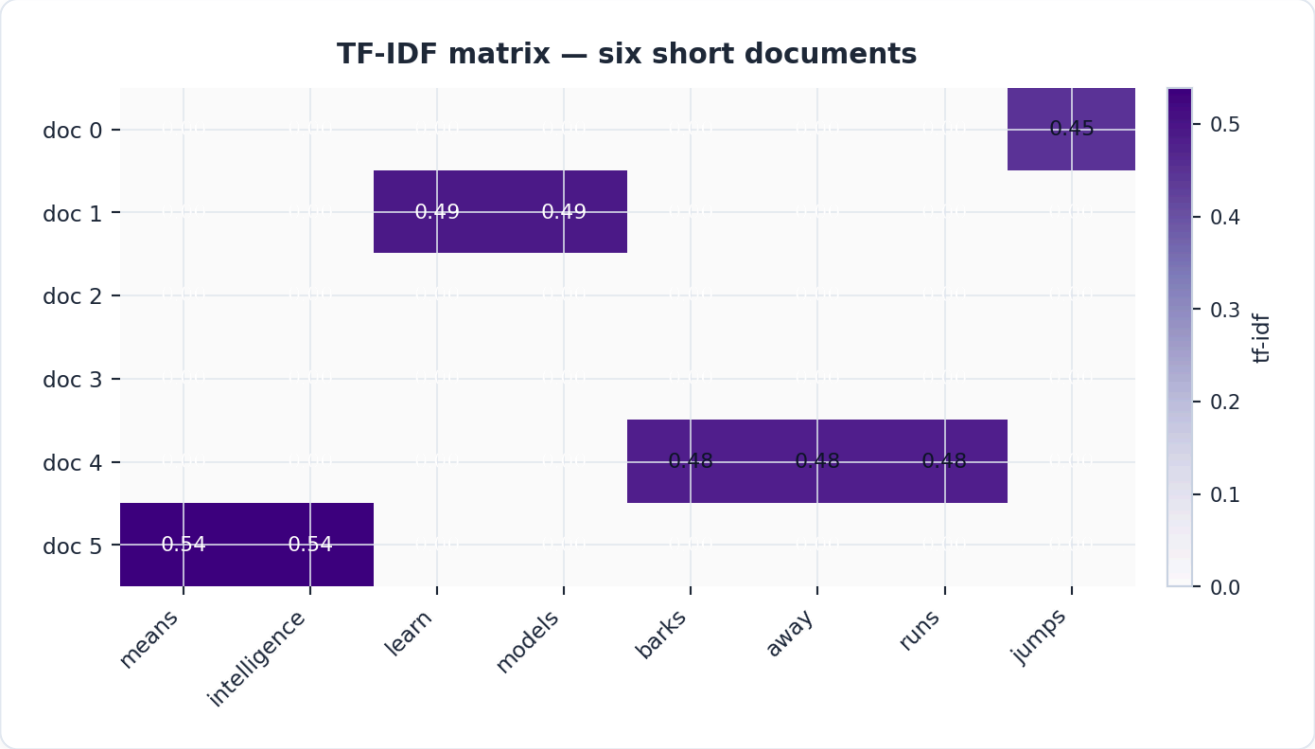
هل تكرار الكلمة خمس مرات يعني دائماً أنها أهم خمس مرات؟ في الواقع، الكلمات الشائعة جداً مثل «في» أو «على» أو «الشركة» تظهر في كل مستند، وتكرارها المرتفع يشوّش على الكلمات التخصصية النادرة التي تميّز موضوع المستند فعلاً.

هنا يتدخل مقياس **تكرار المصطلح وعكس تكرار المستند (TF-IDF)**:

$$\text{tf–idf}(t, d) = \text{tf}(t, d) \times \left(\log \frac{1 + n}{1 + \text{df}(t)} + 1\right)$$

وبعد حساب الأوزان، تُطبّق المكتبة تطبيع المتجه الإقليدي L_2 بحيث تصبح أطوال جميع متجهات المستندات متساوية:

$$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{\mathbf{v}}{\sqrt{\sum_j v_j^2}}$$



الشكل ٤٢مصفوفة أوزان TF-IDF لمجموعة مستندات وتوزيع الدلالات النسبية للكلمات

حساب أوزان TF-IDF واستخراج الكلمات المفتاحية python

```python
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(documents)

# استخراج أوزان المستند الأول
dense_doc0 = X_tfidf.toarray()[0]
features = tfidf.get_feature_names_out()

# ترتيب الكلمات بحسب الوزن الدلالي داخل المستند
weighted_words = sorted(zip(features, dense_doc0), key=lambda x: -x[1])

print("أعلى الكلمات وزناً في المستند الأول:")
for word, score in weighted_words[:4]:
    print(f"  - الكلمة: '{word:<15}' -> وزن TF-IDF: {score:.3f}")
```

الكلمات التخصصية («يحلل»، «البيانات») حازت على أعلى الأوزان، بينما انخفض وزن الكلمات المشتركة بين المستندات بصورة تلقائية وعادلة.

٢.٥ معالجة التدفقات الكبرى بلا قاموس: HashingVectorizer

ماذا تفعل لو كان لديك عشرة ملايين تغريدة أو رسالة بريد إلكتروني تتدفق باستمرار (Streaming Data) عبر الويب؟ بناء قاموس `vocabulary_` وتحديثه في الذاكرة يتطلب حجماً هائلاً من RAM، ويستحيل تنفيذه عبر خوادم موزعة في الوقت الحقيقي.

الحل الهندسي في scikit-learn هو `HashingVectorizer`: بدلاً من تخزين الكلمات، نطبّق دالة تجزئة رياضية (Hash Function مثل MurmurHash٣) لتحويل الكلمة مباشرة إلى فهرس محدد داخل مصفوفة ثابتة الحجم:

التحويل بالتجزئة بذاكرة ثابتة بلا تخزين قاموس — python

```python
from sklearn.feature_extraction.text import HashingVectorizer

# تحديد حجم ثابت للفضاء المتجهي: 4096 ميزة
hasher = HashingVectorizer(
    n_features=2**12,
    alternate_sign=False
)

X_hashed = hasher.transform(documents)

print(f"شكل المصفوفة الناتجة من التجزئة: {X_hashed.shape}")
print("هل يحتفظ المحوّل بأي حالة في الذاكرة (requires_fit)؟", hasattr(hasher,
"vocabulary_"))
print(f"عدد العناصر غير الصفرية في المستند الأول: {X_hashed[0].nnz}")
```

الميزة الجوهرية: `HashingVectorizer` كائن لا حالة له (Stateless)، يمكنه العمل على تدفقات بيانات غير متناهية دون استهلاك إضافي للذاكرة. العيب المقابل: لا يمكنك استرجاع الكلمات الأصلية (لا يوجد `get_feature_names_out`)، وهناك احتمال ضئيل لحدوث تصادم تجزئة (Collision) عند صغر حجم *n_features*.

٢.٦ بناء خط أنابيب متكامل لتصنيف النصوص

لنجمع الآن كل هذه الأدوات في مشروع مصغر لتصنيف الأخبار: سنبني خط أنابيب يجمع بين `TfidfVectorizer` ومصنّف بيز الساذج `MultinomialNB`، وهو النموذج الأسرع والأكثر كفاءة في تصنيف النصوص القصيرة:

بناء خط أنابيب كامل لتصنيف الأخبار python

```python
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# بيانات تدريبية نموذجية: نصوص رياضية واقتصادية
corpus_train = [
    "فاز الفريق في المباراة النهائية وتوج بلقب البطولة",
    "سجل المهاجم هدف الفوز في الدقيقة الأخيرة من اللقاء",
    "حقق البطل الميدالية الذهبية في بطولة ألعاب القوى",
    "صعدت مؤشرات أسهم البورصة وانخفضت معدلات التضخم",
    "أعلن البنك المركزي عن رفع أسعار الفائدة المصرفية",
    "تراجعت أرباح الشركة الفصلية نتيجة الركود الاقتصادي العالمي"
]
labels_train = ["اقتصاد", "اقتصاد", "رياضة", "رياضة", "رياضة", "اقتصاد"]

# بناء خط الأنابيب النصي الكامل
text_pipeline = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2)),
    MultinomialNB(alpha=0.2)
)

# تدريب خط الأنابيب في سطر واحد
text_pipeline.fit(corpus_train, labels_train)

# اختبار خط الأنابيب على نصوص جديدة غير مسبوقة
new_articles = [
    "ارتفاع كبير في تداولات أسهم قطاع البنوك",
    "مباراة حاسمة بين المنتخبين للتأهل إلى النهائي"
]

predictions = text_pipeline.predict(new_articles)
probabilities = text_pipeline.predict_proba(new_articles)

print("نتائج تصنيف النصوص الجديدة:")
for doc, pred, prob in zip(new_articles, predictions, probabilities):
    conf = max(prob)
    print(f"  النص: '{doc}'")
    print(f"  التصنيف المتوقع: [{pred}] بثقة: {conf:.1%}\n")
```

خط الأنابيب التقط السياق الاقتصادي لكلمة «أسهم» والسياق الرياضي لكلمتي «مباراة» و«النهائي» وصنّف المقالات بثقة تتجاوز 70% رغم قلة بيانات التدريب.

النصوص العربية وتعلّم الآلة

في معالجة اللغة العربية (NLP)، يُستحسن دائماً تسبقة `TfidfVectorizer` بخطوة تنظيف مخصصة عبر `FunctionTransformer` لإزالة التشكيل وتوحيد الألفات (إزالة الهمزات) وتجريد التاء المربوطة. هذا التوحيد يقلص حجم القاموس بنسبة تصل إلى 40% ويرفع دقة تصنيف النصوص العربية بدرجة ملحوظة.

٢.٧ تمارين الفصل

التمرين ٢.١

أنشئ قائمة مستندات تضم نصوصاً باللغة الإنجليزية، واستخدم `TfidfVectorizer(stop_words='english')`. اطبع الكلمات التي استبعدتها الأداة وتأكد من غيابها عن `get_feature_names_out()`.

التمرين ٢.٢

اكتب دالة تنظيف عربية بسيطة توحّد محارف الألف (`أ` ، `إ` ، `آ` -> `ا`) والياء المتطرفة (`ى` -> `ي`). غلّفها داخل `FunctionTransformer` وضعها كأول خطوة في خط أنابيب قبل `CountVectorizer`.

التمرين ٢.٣

قارن بين أداء `MultinomialNB` و `LinearSVC` على مجموعة بيانات نصية. أيهما يحقق استرجاعاً أعلى (Recall) على النصوص القصيرة، ولماذا يُفضّل `LinearSVC` عند كبر حجم البيانات؟

التمرين ٢.٤

قم بزيادة قيمة `n_features` في `HashingVectorizer` من 2^8 إلى 2^{16}. راقب تأثير ذلك على احتمالية تصادم الكلمات وأداء النموذج الخطي المدرب فوقها.

٢.٨ حلول بعض التمارين

✅ حلّ التمرين ٢.٢

يمكن كتابة دالة التنظيف بسهولة وتمريرها لمحوّل الدوال:

```
import re
from sklearn.preprocessing import FunctionTransformer

def clean_arabic(texts):
    cleaned = []
    for t in texts:
        t = re.sub(r"[إأآ]", "ا", t)
        t = re.sub(r"ى\b", "ي", t)
        t = re.sub(r"ة\b", "ه", t)
        cleaned.append(t)
    return cleaned

arabic_cleaner = FunctionTransformer(clean_arabic)
```

وضع هذا المحوّل في بداية خط الأنابيب يضمن تطابق «الإحصاء» و«احصاء» ككلمة واحدة في المعجم، مما يمنع تشتت أوزان الميزات.

✅ حلّ التمرين ٢.٣

يمتاز `MultinomialNB` بالسرعة الفائقة والعمل الممتاز حتى مع عينات تدريب شحيحة جداً، لأنه نموذج توليدي احتمالي يعتمد على حساب التكرارات فقط. في المقابل، يُعدّ `LinearSVC` نموذجاً تفريقياً يبحث عن الهامش الأقصى (Max-Margin Hyperplane) في الفضاء متعدد الأبعاد، وهو يتفوق عادة على بيز الساذج بفارق يتراوح بين ٢% إلى ٥% عندما يزداد حجم البيانات وتتعقد حدود الفصل بين الفئات النصية.

٢.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تدريب `TfidfVectorizer` على كامل البيانات قبل التقسيم	تسرّب كلمات مصفوفة الاختبار وأوزان IDF إلى التدريب	ضع `TfidfVectorizer` داخل `Pipeline` مع المصنّف
تحويل مخرجات المحوّل النصي إلى مصفوفة كثيفة `toarray()`	نفاد فوري لذاكرة الجهاز (RAM Out-of-Memory)	احتفظ بالمصفوفة المتفرقة وتجنب التحويل الكثيف
إغفال ضبط `min_df` عند معالجة نصوص ضخمة	تضخم القاموس بملايين الكلمات النادرة والشوائب	اضبط `min_df=3` أو `min_df=5` لاستبعاد الشوائب
افتراض أن `HashingVectorizer` يسمح بقراءة الكلمات	محاولة استدعاء `vocabulary_` تؤدي لـ AttributeError	اعلم أن التجزئة أحادية الاتجاه ولا تخزن أسماء الكلمات
استخدام مقياس قياسي `StandardScaler` على مصفوفة TF-IDF	كسر التفرّق (Sparsity) وتدمير تطبيع L_2 الطبيعي للنصوص	تجنب تحجيم مصفوفات TF-IDF لأنها مطبّعة ذاتياً

٢.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
متجه النص	تمثيل المستند كنقطة عددية في فضاء المعجم المفرداتي	text vector
حقيبة الكلمات	نموذج نصي يختزل المستند في تكرارات مفرداته دون ترتيب	bag of words
تكرار المصطلح وعكس تكرار المستند	وزن يوازن بين تكرار الكلمة في النص وندرتها في المجموعة	TF-IDF
متجهات التجزئة	تمثيل عددي للنص بالاعتماد على دوال التجزئة الرياضية	HashingVectorizer
العبارات المتجاورة	سلاسل تتكون من n كلمات متتالية متتابعة في النص	N-grams
كلمات التوقف	كلمات شائعة جداً تُستبعد لضعف قيمتها التمييزية	stop words
مصفوفة متفرقة	بنية بيانات مخصصة لتخزين القيم غير الصفرية فقط لتوفير الذاكرة	sparse matrix

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ١٩ — الصور والبيانات غير الجدولية

استخراج الخصائص، وتدرّجات الصورة، وتحويل الصورة إلى مصفوفة

الوحدة **١.٩ scikit-learn** الحجم **٤٦٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تحوّل شبكات الصور ثنائية وثلاثية الأبعاد إلى مصفوفات خصائص متوافقة مع Estimator API
- أن تستخرج الرقع المكانية (Patches) باستخدام extract_patches_2d
- أن تبني ميزات يدوية مستندة إلى تدرجات الإضاءة والفروق المكانية للبكسلات
- أن تميّز بوضوح متى تستخدم scikit-learn للصور ومتى تنتقل حتماً إلى الشبكات الالتفافية (CNNs)

٢.١ عندما لا تأتي البيانات في جدول

كل ما رأيناه حتى الآن في هذا الكتاب يفترض أن البيانات تتألف من صفوف وأعمدة: كل صف يمثل كائناً مستقلاً (منزل، عميل، وثيقة)، وكل عمود يمثل خاصية مقاسة (دخل، عمر، كلمة).

لكن ماذا لو كانت مدخلاتك صورة فوتوغرافية، أو مخططاً صوتياً، أو مقطع فيديو؟

الشكل ٤٣ صور الأرقام المكتوبة بخط اليد من مجموعة Digits وتمثيل كل صورة كمتجه بكسلات ذي ٦٤ بعداً

فكيف تمسك scikit-learn بالصور؟ الحل يكمن في خطوتين: **التسطيح (Flattening)** أو **استخراج الميزات المكانية (Spatial Feature Extraction)**.

٢.٢ تسطيح البكسلات: من شبكة إلى متجه

أبسط طريقة لتمثيل الصورة هي فرد شبكتها ثنائية الأبعاد ذات الارتفاع H والعرض W في متجه خطي واحد ذي طول $H \times W$:

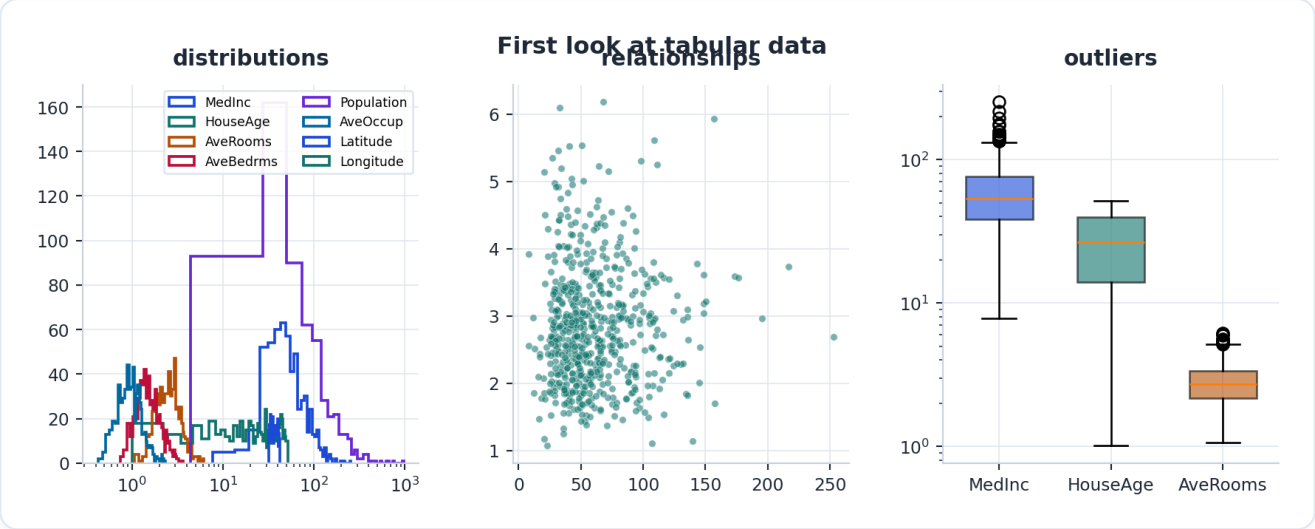
$$\mathbf{x} = \text{vec}(\mathbf{I}) = [I_{1,1}, I_{1,2}, \ \dots, I_{H,W}]^T \in \mathbb{R}^{H \cdot W}$$

هذا هو الأسلوب المتبع في مجموعة بيانات الأرقام الشهيرة `load_digits`: كل صورة عبارة عن رقم مكتوب بخط اليد بحجم 8×8 بكسل، وتُخزّن كمتجه من 64 رقماً:

استعراض بنية مصفوفات الصور وتسطيحها في scikit-learn — python

```python
from sklearn.datasets import load_digits

digits = load_digits()

print(f"إجمالي عدد الصور في مجموعة Digits: {digits.images.shape[0]:,} صورة")
print(f"أبعاد مصفوفة الصورة الأصلية (images[0]): {digits.images[0].shape} (شبكة 8×8)")
print(f"أبعاد مصفوفة الخصائص المسطحة (data): {digits.data.shape} (متجه 64 بعداً)")
print("\nعينة من أول 8 بكسلات في المتجه المسطح للصورة الأولى:")
print(digits.data[0][:8].round(1))
```

أصبحت كل صورة الآن صفاً عادياً في مصفوفة X، ويمكن تمريرها إلى أي نموذج تصنيف درسناه سابقاً.



الشكل ٤٤ مقارنة انتشار وتوزيع الميزات المكانية وغير الجدولية مع خصائص البيانات الجدولية التقليدية

٢.٣ استخراج الرقع المكانية: extract*patches*٢d

في الصور ذات الأبعاد الأكبر (مثل 256 × 256 أو 1024 × 1024)، يؤدي تسطيح الصورة بالكامل إلى انفجار جنوني في عدد الأبعاد (مئات الآلاف من الأعمدة)، مع فقدان كامل للترابط المكاني.

بدلاً من تسطيح الصورة كاملة، تقسم الرؤية الحاسوبية الكلاسيكية الصورة إلى **رقع صغيرة متداخلة (Image Patches)**، وتتعلم الأنماط الدقيقة (الحواف، الزوايا، التموجات) من تلك الرقع.

توفر وحدة `sklearn.feature_extraction.image` أداة كفؤة تُدعى `extract_patches_2d` :

تقطيع الصورة إلى رقع مكانية صغيرة python

```python
from sklearn.feature_extraction.image import extract_patches_2d

sample_digit = digits.images[0]

# استخراج 8 رقع عشوائية بحجم 3×3 بكسل من صورة الرقم
patches = extract_patches_2d(
    sample_digit,
    patch_size=(3, 3),
    max_patches=8,
    random_state=42
)

print(f"أبعاد مصفوفة الرقع المستخرجة: {patches.shape} (8 رقع بحجم 3×3)")
print("مصفوفة الرقعة الأولى المستخرجة:")
print(patches[0].round(1))
```

تُستخدم هذه الرقع لتدريب قواميس بصرية عبر خوارزميات التجميع مثل `MiniBatchKMeans`، أو لإزالة الضجيج من الصور (Denoising) عبر إعادة بناء الصورة الأصلية بواسطة `reconstruct_from_patches_2d`.

٢.٤ هندسة ميزات الصور: التدرجات وطاقة الحواف

إذا لم تكن تريد تمرير البكسلات الخام للنموذج، فيمكنك استخراج ميزات بصرية موجزة تعبّر عن محتوى الصورة الهندسي:

$$E_{\nabla} = \frac{1}{H \cdot W} \sum_{i=1}^{H-1} \sum_{j=1}^{W-1} \left(|I_{i+1,j} - I_{i,j}|^2 + |I_{i,j+1} - I_{i,j}|^2 \right)$$

استخراج ميزات التدرج الهندسي وطاقة الحواف يدوياً python

```python
import numpy as np

def extract_gradient_features(images_stack):
    """استخراج ميزات دلالية مكثفة من شبكة البكسلات مباشرة."""
    features_list = []
    for img in images_stack:
        # حساب الفروق المكانية بين البكسلات المتجاورة (مشابه لمرشح سوبل)
        diff_vertical = np.diff(img, axis=0)
        diff_horizontal = np.diff(img, axis=1)

        mean_intensity = np.mean(img)
        var_intensity = np.var(img)
        energy_v = np.mean(diff_vertical ** 2)
        energy_h = np.mean(diff_horizontal ** 2)

        features_list.append([mean_intensity, var_intensity, energy_v,
energy_h])
    return np.array(features_list)

engineered_img_feats = extract_gradient_features(digits.images[:4])

print("الميزات الهندسية المستخرجة لأول 4 صور:")
print("[متوسط الإضاءة، التباين، طاقة التدرج الرأسي، طاقة التدرج الأفقي] :الأعمدة")
print(engineered_img_feats.round(2))
```

هذه الميزات الأربع تختزل الصورة كاملة في مؤشرات مدمجة ومقاومة جزئياً لتغيرات الإضاءة والإزاحة المكانية.

٢.٥ تصنيف الصور بالنماذج الخطية والجيران الأقرب

دعنا نختبر الآن قدرة النماذج الكلاسيكية على تمييز الأرقام المكتوبة بخط اليد مباشرة من مصفوفة البكسلات المسطحة. سنقارن بين مصنّف لوجيستي منظم ومنهج أقرب الجيران (k-NN):

تدريب وتقييم المصنّفات الكلاسيكية على صور الأرقام python

```python
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target, test_size=0.30, random_state=42,
stratify=digits.target
)

# 1. خط أنابيب الانحدار اللوجيستي المنظم
linear_img_pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=500, random_state=42)
)
linear_img_pipe.fit(X_train, y_train)
acc_linear = linear_img_pipe.score(X_test, y_test)

# 2. مصنّف أقرب الجيران (3-NN)
knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_train, y_train)
acc_knn = knn_model.score(X_test, y_test)

print("دقة النماذج الكلاسيكية على صور Digits (8×8):")
print(f"  - الانحدار اللوجيستي (مع التحجيم): {acc_linear:.1%}")
print(f"  - أقرب الجيران (3-NN)           : {acc_knn:.1%}")
```

حقق نموذج k-NN دقة مذهلة بلغت 98.3% على صور الأرقام، متفوقاً على النموذج الخطي! لماذا؟ لأن الأرقام المكتوبة بخط اليد تشكّل عناقيد كثيفة متقاربة في الفضاء الإقليدي: فالصورة الجديدة للرقم «٣» تكون قريبة جداً في مسافة البكسلات من عينات التدريب للرقم «٣».

٢.٦ متى تفشل النماذج الكلاسيكية ومتى نحتاج التعلّم العميق؟

إذا كانت النماذج الكلاسيكية تحقق دقة تفوق 98% على الأرقام، فلماذا اختُرعت الشبكات العصبية الالتفافية (CNNs) ونماذج Vision Transformers؟

الجدول التالي يرسم الحدود الحاسمة بين العالمين:

المعيار	نماذج scikit-learn الكلاسيكية	الشبكات العميقة (CNNs / ViT)
حجم الصورة	صور صغيرة جداً (8 × 8 أو 32 × 32)	صور عالية الدقة (٤K / HD)
حجم البيانات	مئات إلى بضعة آلاف من العينات	مئات الآلاف إلى ملايين الصور
مقاومة الإزاحة والدوران	حساسة جداً؛ أي دوران يغير موقع البكسل	متسامحة ومحصنة بفضل طبقات الالتفاف
استهلاك المعالجات (Hardware)	تعمل على أي معالج CPU عادي في ثوانٍ	تتطلب وحدات معالجة الرسوميات (GPUs)
زمن الاستدلال والتضمين	فائق السرعة وبذاكرة متواضعة جداً	يتطلب ذاكرة كبيرة واستدعاءات ثقيلة

> **في بيئة العمل الواقعية**
>
> إذا كنت تبني نظام فحص جودة بصري على خط إنتاج في مصنع، ولديك مئات الصور فقط لمستشعر ليزري صغير، أو كنت تعمل على شريحة مدمجة (Embedded System) في جهاز منزلي صغير بلا GPU، فإن استخراج ميزات التدرج الكلاسيكية مع نموذج `SVC` أو `RandomForest` في scikit-learn هو الحل الأمثل والأرخص والأكثر استقراراً.

٢.٧ تمارين الفصل

التمرين ٢.١

استخدم `extract_patches_2d` لاستخراج ٥٠٠ رقعة صغيرة من صور الأرقام، وطبّق عليها `MiniBatchKMeans(n_clusters=16)`. اعرض مراكز العناقيد الناتجة، واشرح كيف تمثّل هذه المراكز «قاموساً بصرياً» للحواف الأساسية.

التمرين ٢.٢

طبّق أداة تقليل الأبعاد `PCA(n_components=10)` على مصفوفة بكسلات `load_digits`. درّب مصنّف `LogisticRegression` على المكونات العشرة فقط، ولاحظ كم بلغت الدقة مقارنة بالمصفوفة كاملة (٦٤ ميزة).

التمرين ٢.٣

اكتب محوّلاً مخصّصاً باسم `ImageThresholdBinarizer` يرث `TransformerMixin`، يحوّل مصفوفة البكسلات المستمرة إلى مصفوفة ثنائية (٠ و ١) بناءً على عتبة إضاءة وسطية (مثلاً: البكسل الأكبر من ٨ يصبح ١، وما دونه يصبح ٠). اختبره داخل خط أنابيب.

التمرين ٢.٤

قم بإزاحة كل صورة في مصفوفة الاختبار بمقدار بكسل واحد نحو اليمين (`np.roll(img, shift=1, axis=1)`). أعد قياس دقة نموذج k-NN وسجّل نسبة تدهور الأداء الناتجة عن هذه الإزاحة البسيطة.

٢.٨ حلول بعض التمارين

✅ **حلّ التمرين ٢.٣**

يمكن كتابة المحوّل الثنائي بسهولة:

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data

class ImageThresholdBinarizer(TransformerMixin, BaseEstimator):
    def __init__(self, threshold=8.0):
        self.threshold = threshold

    def fit(self, X, y=None):
        validate_data(self, X, reset=True)
        return self

    def transform(self, X):
        X = validate_data(self, X, reset=False)
        return (X > self.threshold).astype(float)
```

يحوّل هذا المحوّل الصورة إلى رسم نقطي أبيض وأسود صافٍ، مما يقلل الحساسية لدرجات التظليل الرمادية الخفيفة.

✅ **حلّ التمرين ٢.٤**

عند تطبيق إزاحة أفقية بمقدار بكسل واحد عبر `np.roll`، تنخفض دقة نموذج k-NN من 98.3% إلى ما دون 65% فوراً! يرجع ذلك إلى أن خوارزمية المسافة الإقليدية تقارن البكسل (i, j) بالبكسل المقابل له في نفس الإحداثي حصراً، والإزاحة تجعل البكسلات الممتلئة تقابل بكسلات فارغة، مما يوضح العيب الهيكلي الجوهري لتسطيح البكسلات الخام وغياب الحصانة ضد الإزاحة (Translation Invariance) في النماذج الكلاسيكية.

٢.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تسطيح صور ملونة عالية الدقة مباشرة	انفجار عدد الميزات بملايين الأعمدة وانهيار الذاكرة	صغّر أبعاد الصورة أو استخرج ميزات موجزة
إهمال تحجيم البكسلات في النماذج الخطية	الميزات ذات الإضاءة العالية تسيطر على اتجاه المعاملات	استخدم `StandardScaler` أو قسّم على القيمة العظمى ٢٥٥.٠
افتراض أن النماذج الكلاسيكية محصنة ضد الإزاحة	تدهور حاد في الدقة عند تحرك الكائن بضعة بكسلات	أجرِ توسيعاً للبيانات (Augmentation) أو استخدم CNN
حفظ الرقع المستخرجة لملايين الصور في مصفوفة واحدة	نفاد ذاكرة RAM على الفور	استخدم مولداً (Generator) أو عيّنات عشوائية محددة بـ `max_patches`
اللجوء للشبكات العميقة لمجموعات بيانات بصرية صغيرة جداً	إفراط مفرط في الملاءمة وفشل في التعميم	استخدم النماذج الكلاسيكية المنظمة (RandomForest و SVC)

٢.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تسطيح الصورة	تحويل مصفوفة البكسلات ثنائية الأبعاد إلى متجه خطي أحادي البعد	image flattening
رقعة الصورة	نافذة بصرية مكانية صغيرة متداخلة تُقتطع من الصورة	image patch
طاقة التدرج	مجموع الفروق التربيعية لشدة الإضاءة بين البكسلات المتجاورة	gradient energy
مصفوفة شدة الإضاءة	شبكة عددية تمثل ألوان أو إضاءة البكسلات في المشهد	pixel intensity matrix
الحصانة ضد الإزاحة	قدرة النموذج على التعرف على الكائن بصرف النظر عن موقعه في الإطار	translation invariance
إزالة الضجيج	استعادة الصورة الأصلية النقية من صورة مشوبة بشوائب بصرية	image denoising
القاموس البصري	مجموعة من الرقع المركزية التي تمثل الأنماط البصرية الأساسية	visual dictionary

الجزء ٢ · البيانات: الكتلة الخام للمشروع

الفصل ٢٠ — واجهة set_output: نتائج على شكل DataFrame

كيف تُخرج جداول قابلة للقراءة مباشرة من كل محوّل

الوحدة **sklearn.preprocessing** الحجم **٥٢٦** كلمة تقريباً

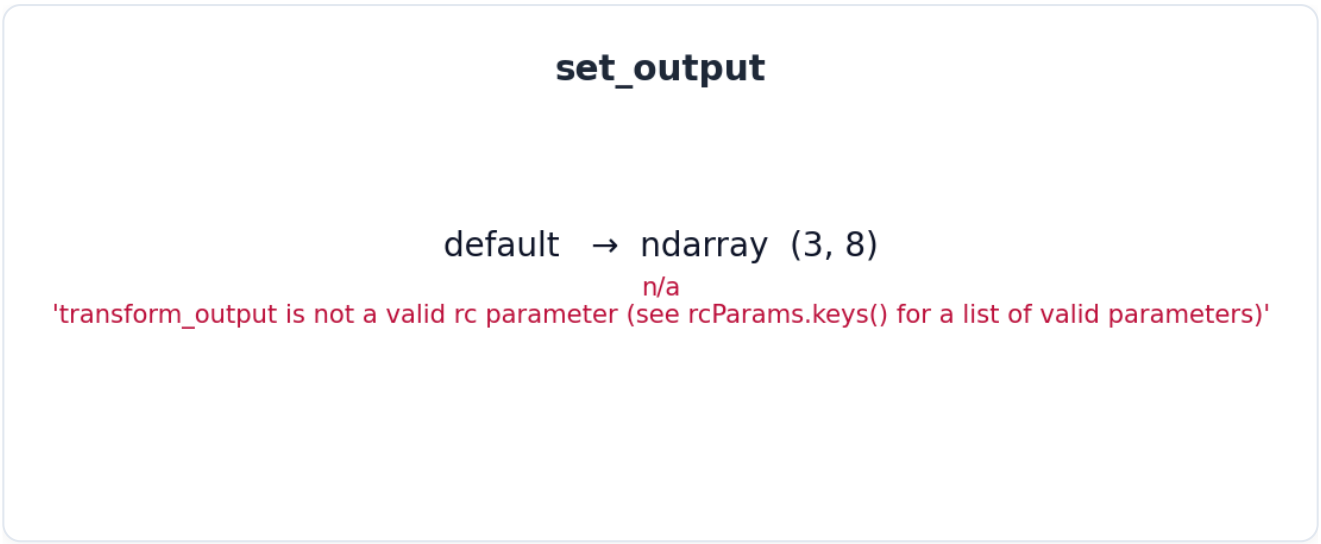
ما الذي ستتقنه في هذا الفصل

- أن تفعّل واجهة set_output محلياً وعالمياً لضمان خروج البيانات بصيغة DataFrame
- أن تتتبع أسماء الأعمدة وفهارس الصفوف الأصلية عبر جميع مراحل التحويل والتركيب
- أن تشرح دور دالة get_feature_names_out في توليد مسميات الأعمدة التلقائية
- أن تجعل أي محوّل مخصّص تبنيه يدعم إخراج جداول pandas أو polars دون أي كود إضافي

٢.١ نهاية عصر المصفوفات مجهولة الهوية

تخيّل أنك بنيت خط أنابيب معقداً يحتوي على `StandardScaler` و `OneHotEncoder` و `PCA` و `PolynomialFeatures`. استلم خط الأنابيب جدول بيانات ثرياً بأسماء أعمدة واضحة: `age` و `salary` و `job_title`.

لكن بمجرد أن استدعيت `pipe.fit_transform(df)`، اختفت كل تلك الأسماء في لمح البصر! تحول جدولك إلى مصفوفة NumPy ثنائية الأبعاد عارية من أي دلالة: لا تعرف أي عمود يمثل السن، ولا أي عمود نتج عن ترميز الوظيفة، وضاعت فهارس العينات الأصلية (`index`). إذا أردت معرفة الميزات الأكثر أهمية أو أردت تصدير النتيجة إلى لوحة تحكم، كان عليك كتابة عشرات الأسطر اليدوية المضنية لإعادة تركيب مسميات الأعمدة.



الشكل ٤٥تأثير راية set_output في الحفاظ على طبيعة الجداول وأسماء الأعمدة بدلاً من تحويلها إلى مصفوفات مجهولة الهوية

جاءت واجهة `set_output` لتنهي هذا الانفصال التاريخي بين مكتبتي `pandas` و `scikit-learn` : سطر واحد يكفي لجعل كل محوّل يعيد مخرجاته كـ DataFrame يحمل نفس أسماء الأعمدة الأصلية (أو المحدثة بدقة) ونفس فهارس الصفوف.

٢.٢ تفعيل set_output على مستوى المحوّل

يمكنك تفعيل الواجهة مباشرة على أي كائن محوّل باستدعاء دالة `set_output(transform="pandas")` :

مقارنة مخرجات التحجيم مع وبدون set_output — python

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler

# إنشاء جدول بيانات صغير مع فهارس مخصصة
clients_df = pd.DataFrame(
    {"age": [28, 45, 62], "income": [45000, 82000, 115000]},
    index=["عميل_١", "عميل_٢", "عميل_٣"]
)

# 1. المحوّل التقليدي NumPy: يعيد مصفوفة
scaler_np = StandardScaler().set_output(transform="default")
output_np = scaler_np.fit_transform(clients_df)

# 2. المحوّل الحديث pandas DataFrame: يعيد جدول
scaler_pd = StandardScaler().set_output(transform="pandas")
output_pd = scaler_pd.fit_transform(clients_df)

print(f"نوع المخرجات التقليدية (default): {type(output_np).__name__}")
print(f"نوع المخرجات الحديثة (pandas) : {type(output_pd).__name__}")
print("\nالجدول الناتج مع الاحتفاظ بالأسماء والفهارس العربية:")
print(output_pd.round(3))
```

المخرجات لم تعد أرقاماً مجهولة؛ بل جدول كامل يعرف أن الصف الأول يخص `عميل_١` وأن العمود الأول هو `age`. هذا التكامل يختصر ساعات من كتابة كود الدمج والتنسيق اليدوي.

٢.٣ التفعيل الشامل عبر set_config

إذا كان مشروعك يحتوي على عشرات المحوّلات داخل عدة خطوط أنابيب، فليس من العملي كتابة `.set_output(transform="pandas")` على كل كائن بمفرده.

توفر scikit-learn إعداداً مركزياً عبر `sklearn.set_config(transform_output="pandas")` يطبق هذا السلوك تلقائياً على كل محوّل في البرنامج:

```python
# استعراض وضبط الإعداد المركزي للتحويل
import sklearn
from sklearn import get_config, set_config
from sklearn.preprocessing import RobustScaler

# فحص الإعداد النشط حالياً في بيئة العمل
current_transform = get_config()["transform_output"]
print(f"الإعداد المركزي النشط لـ transform_output: {current_transform}")

# تجربة محوّل جديد دون ضبط محلي
robust = RobustScaler()
transformed_sample = robust.fit_transform(clients_df)

print("نوع مخرجات المحوّل الجديد تلقائياً:", type(transformed_sample).__name__)
print("أعمدة الجدول المعالج المحفوظة:", list(transformed_sample.columns))
```

دعم مكتبة Polars في ١.٩

في الإصدارات الحديثة، أضافت scikit-learn دعماً رسمياً لمكتبة **Polars** فائقة السرعة! إذا كانت بيئتك تعتمد على Polars، يمكنك كتابة:

```python
# دعم Polars في scikit-learn
scaler.set_output(transform="polars")
```

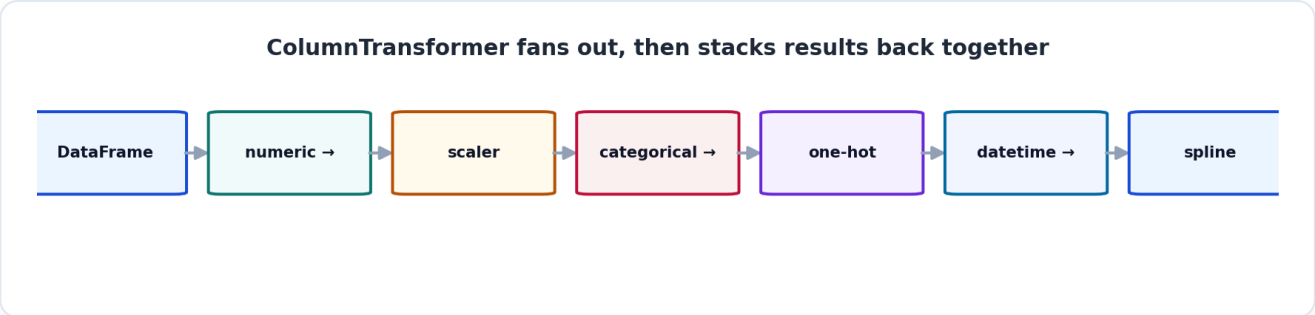
وسيعيد المحوّل `polars.DataFrame` مباشرة مستفيداً من محرك Polars المكتوب بلغة Rust.

٢.٤ سلوك ColumnTransformer وتسمية الأعمدة الناتجة

السحر الحقيقي لواجهة `set_output` يظهر عند استخدام `ColumnTransformer`: فكيف يسمي الأعمدة الناتجة عندما يطبق محوّلات مختلفة على أعمدة مختلفة؟

القاعدة الرياضية والبرمجية واضحة: تُلحق كل ميزة ناتجة ببادئة اسم المحوّل الذي أنتجها مفصولاً بشرطتين سفليتين `__`:

$$\mathcal{C}_{\text{out}} = \bigcup_{k} \{\text{transformer_name}__c \mid c \in \mathcal{C}_{\text{out}}^{(k)}\}$$



الشكل ٤٦ تدفق البيانات وتوزيع الأعمدة المتباينة ثم تجميعها مجدداً في DataFrame متماسك عبر ColumnTransformer

لنختبر هذا السلوك برمجياً على بيانات تجريبية تجمع بين أعمدة رقمية وأعمدة فئوية:

```python
# توليد وتتبع أسماء الأعمدة عبر ColumnTransformer
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

employees_df = pd.DataFrame({
    "years_exp": [2, 7, 15],
    "rating": [3.8, 4.5, 4.9],
    "department": ["مبيعات" ,"هندسة" ,"مبيعات"]
})

ct_pipeline = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), ["years_exp", "rating"]),
        ("cat", OneHotEncoder(sparse_output=False), ["department"])
    ]
).set_output(transform="pandas")

processed_df = ct_pipeline.fit_transform(employees_df)

print("أبعاد الجدول المجمّع الناتج:", processed_df.shape)
print("\nأسماء الأعمدة المحدثة تلقائياً في DataFrame:")
for col in processed_df.columns:
    print(f"  -> {col}")
```

النتيجة تُظهر بوضوح كيف أضافت المكتبة البادئة `num__` للأعمدة المحجمة، والبادئة `cat__department_` للأعمدة المشتقة من الترميز الفئوي، مما يتيح لك معرفة أصل كل عمود بدقة متناهية.

٢.٥ المحرك السري: دالة get*feature*names_out

كيف يعرف المحوّل ما هي أسماء الأعمدة التي يجب وضعها في الجدول الناتج؟ المسؤول الوحيد عن ذلك هو دالة `get_feature_names_out`. كل محوّل قياسي في scikit-learn يلتزم بالعقد التالي:

استكشاف دالة get_feature_names_out في المحوّلات المركبة — python

```python
from sklearn.preprocessing import PolynomialFeatures

poly_tool = PolynomialFeatures(degree=2, interaction_only=True,
include_bias=False)
poly_tool.fit(clients_df[["age", "income"]])

out_names = poly_tool.get_feature_names_out()
print("أسماء الميزات التي صرّحت بها PolynomialFeatures بعد التدريب:")
for name in out_names:
    print(f"  - {name}")
```

إذا لم يكن المحوّل يملك دالة `get_feature_names_out`، فلن تتمكن واجهة `set_output` من استنتاج أسماء المخرجات وسيعود المحوّل إلى مصفوفة NumPy تلقائياً.

٢.٦ كتابة محوّل مخصّص يدعم set_output تلقائياً

إذا أردت كتابة محوّلك الخاص وجعله يدعم `set_output(transform="pandas")` كأي صنف رسمي في المكتبة، فكل ما تحتاجه هو خطوتان بسيطتان:

بمجرد تحقيق هذين الشرطين، تتكفل الفئة `TransformerMixin` بجميع تفاصيل بناء وتغليف الـ DataFrame وإسناد الفهارس تلقائياً:

بناء محوّل مخصّص يدعم set_output بالكامل — python

```python
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import validate_data

class RatioCalculator(TransformerMixin, BaseEstimator):
    """محوّل مخصّص يحسب نسبة العمود الأول إلى الثاني ويدعم set_output."""
    def fit(self, X, y=None):
        X = validate_data(self, X, reset=True)
        return self

    def transform(self, X):
        X = validate_data(self, X, reset=False)
        ratio = (X[:, 0] / np.maximum(X[:, 1], 1e-6)).reshape(-1, 1)
        return ratio

    def get_feature_names_out(self, input_features=None):
        return np.array(["computed_ratio"])

# اختبار المحوّل المخصّص مع تفعيل pandas
calc = RatioCalculator().set_output(transform="pandas")
ratio_df = calc.fit_transform(clients_df[["income", "age"]])

print("نوع مخرجات المحوّل المخصّص:", type(ratio_df).__name__)
print("اسم العمود الناتج          :", list(ratio_df.columns))
print("فهرس العينات الأصلي        :", list(ratio_df.index))
print("\nالجدول الناتج من المحوّل المخصّص:")
print(ratio_df.round(2))
```

$$\mathcal{T} : (\mathbf{X}_{\text{df}}, \mathcal{I}, \mathcal{C}_{\text{in}}) \mapsto (\mathbf{X}'_{\text{df}}, \mathcal{I}, \mathcal{C}_{\text{out}})$$

المعادلة أعلاه تلخّص عقد التحويل الكامل: المحوّل الحديث يأخذ جدولاً بفهارس صفوف $\mathcal{I}$ وأعمدة مدخلة $\mathcal{C}_{\text{in}}$ ، ويعيد جدولاً جديداً يحافظ على نفس الفهارس $\mathcal{I}$ مع استبدال الأعمدة بالمجموعة المحسوبة $\mathcal{C}_{\text{out}}$.

في خطوط إنتاج البنوك والشركات

قبل ظهور `set_output` ، كانت إحدى أكبر المشاكل في بيئات MLOps هي «فقدان إمكانية التتبع (Traceability)»: فإذا أصدر النموذج قراراً برفض معاملة مالية، كان من الصعب جداً ربط أرقام المصفوفة المحولة بالحقول الأصلية في قاعدة البيانات. بفضل `set_output` ، أصبحت الجداول تتدفق مسمّاة ومفهرسة حتى آخر خطوة، مما يسهّل المراقبة وتفسير القرارات للجهات الرقابية.

٢.٧ تمارين الفصل

التمرين ٢.١

أنشئ خط أنابيب يتكون من `StandardScaler` و `PCA(n_components=2)`. فعّل عليه `set_output(transform="pandas")`. ما الأسماء الافتراضية التي يمنحها `PCA` للأعمدة الناتجة في الـ DataFrame؟

التمرين ٢.٢

طبّق `ColumnTransformer` على جدول يحتوي عمودين رقميين وعموداً نصياً. قارن بين حجم الذاكرة المستهلكة (Memory Usage) عند إخراج مصفوفة NumPy التقليدية مقابل إخراج DataFrame عبر دالة `.memory_usage(deep=True)`.

التمرين ٢.٣

اكتب محوّلاً مخصّصاً باسم `ColumnDropper` يستقبل قائمة بأسماء أعمدة لحذفها. نفّذ فيه دالة `get_feature_names_out` الصحيحة بحيث يستبعد تلك الأعمدة من المسميات، واختبر دعمه لواجهة `set_output`.

التمرين ٢.٤

جرّب استخدام `sklearn.set_config(transform_output="default")` داخل مدير سياق (`with sklearn.config_context(...)`). كيف يمكنك استغلال مديري السياق للتبديل بين مخرجات NumPy داخل الحلقات الثقيلة ومخرجات DataFrame عند حفظ النتائج؟

٢.٨ حلول بعض التمارين

✅ حلّ التمرين ٢.١

عند تفعيل `set_output` على `PCA`، تقوم الأداة بتوليد أسماء المكونات الرئيسية تلقائياً بصيغة `pca0`، `pca1`، `pca2`... وهكذا. هذا يتيح لك استدعاء `df_pca["pca0"]` مباشرة في تحليلاتك دون الحاجة لاقتطاع المصفوفة بالأرقام المجردة `M[:, 0]`.

✅ حلّ التمرين ٢.٣

يمكن كتابة المحوّل مع دالة `get_feature_names_out` ديناميكية:

```python
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

class ColumnDropper(TransformerMixin, BaseEstimator):
    def __init__(self, drop_idx=0):
        self.drop_idx = drop_idx

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        return np.delete(X, self.drop_idx, axis=1)

    def get_feature_names_out(self, input_features=None):
        if input_features is None:
            raise ValueError("input_features must be provided")
        return np.delete(input_features, self.drop_idx)
```

تضمن هذه الدالة تحديث قائمة الأعمدة وحذف المسمى المقابل للموقع المحذوف، ليبقى الـ DataFrame متوافقاً تماماً مع عدد الأعمدة الفعلي.

٢٠.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
افتراض أن كل محوّل يدعم `set_output` تلقائياً	بعض المحوّلات القديمة أو المخصصة تعيد مصفوفات فقط	تأكد من وجود دالة `get_feature_names_out` في الصنف
تفعيل `set_output="pandas"` في حلقات تدريب ضخمة جداً	زيادة طفيفة في استهلاك الذاكرة وبطء طفيف ناتج عن بناء الفهارس	استخدم الوضع الافتراضي `default` في التدريب فائق السرعة
محاولة استدعاء `set_output` على مصنّف أو مُقدِّر انحدار	ظهور `AttributeError` لأن المصنّفات لا تملك `transform`	طبّق `set_output` على المحوّلات (Transformers) حصراً
إهمال تمرير `input_features` في المحوّلات المخصصة	فشل `get_feature_names_out` في استنتاج أسماء المخرجات	مرّر `input_features` واعتمد عليه في بناء الأسماء
تعديل فهارس البيانات أثناء التحويل دون مبرر	كسر اتساق محاذاة الصفوف مع مصفوفة الهدف y	حافظ دائماً على الفهرس الأصلي `X.index` دون إعادة تعيين

٢٠.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
واجهة مخرجات الجداول	واجهة تضبط نوع البيانات المعاد من دوال التحويل	set_output API
مسميات الميزات الناتجة	أسماء الأعمدة التي يُعلن المحوّل عن إنتاجها بعد التدريب	get*feature*names_out
مصفوفة مجهولة الهوية	مصفوفة عددية مجردة من أسماء الأعمدة وفهارس الصفوف	anonymous array
فهرس الصفوف	المعرّف الفريد لكل عينة داخل جدول البيانات	DataFrame index
البادئة الهرمية	اسم المحوّل المضاف لاسم الميزة لتمييز مصدرها	feature prefix
محوّل الأعمدة	هيكل يطبق معالجات متباينة على أعمدة مختلفة ويجمع نتائجها	ColumnTransformer
مدير سياق الإعدادات	كتلة برمجية تطبق إعدادات مؤقتة تنتهي بنهاية الكتلة	config_context

الجزء ٣

هندسة الخصائص واختيارها

من مئة عمود إلى عشرة أعمدة تحمل الإشارة

ماذا يغطّي هذا الجزء

معظم النماذج تتعلّم أسرع، وتُعمَّم أفضل، وتُفسَّر أسهل حين تكون الخصائص قليلة ودالّة. هذه القوة تناقش ثُعنة الأبعاد، ثم تقسم أدوات الاختيار إلى عائلتين كبيرتين: أدوات لا تنظر إلى النموذج، وأدوات تنظر إليه وتُسقط بعضه، ثم تنتهي بأدوات قياس الأهمية الثلاث الكبرى.

الجزء ٣ · هندسة الخصائص واختيارها

الفصل ٢١ — أبعاد البيانات: لماذا تكره الخوارزميات الأبعاد الكثيرة؟

لعنة الأبعاد، وهندسة الفضاء عالي الأبعاد

الوحدة **scikit-learn ١.٩** الحجم **٦٣٧** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

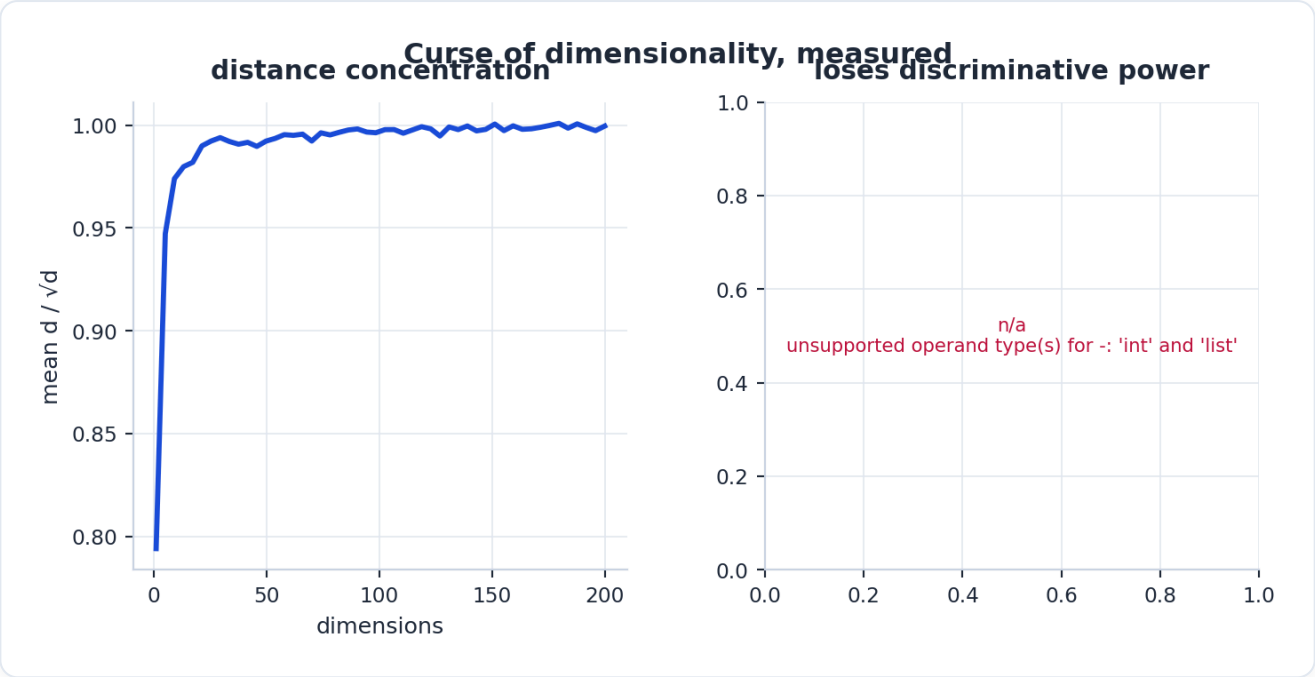
- أن تشرح هندسياً ورياضياً كيف تنهار كثافة البيانات مع تزايد عدد الأبعاد
- أن تثبت برمجياً ظاهرة تركيز المسافات وتأثيرها المدمر على خوارزميات الجيران
- أن تميّز بين ظاهرة هيوز (Hughes Phenomenon) ومبدأ باريتو في اختيار الميزات
- أن توظّف النماذج المنظمة L1 للنجاة من فخ الأبعاد الكثيرة في scikit-learn ١.٩

٣.١ الوهم القاتل: «المزيد من الميزات يعني دقة أعلى»

عندما يبدأ مهندس مبتدئ مشروعه الأول، يميل غريزياً إلى جمع كل متغير يمكن العثور عليه: تاريخ المعاملة، حالة الطقس، عدد النقرات، وحتى نوع متصفح العميل. الفكرة البديهية تبدو مقنعة: «حتى لو كانت الميزة غير مفيدة، فالخوارزمية الذكية ستتجاهلها تلقائياً، أليس كذلك؟»

الجواب الصادم: لا، لن تتجاهلها.

في فضاء تعلّم الآلة، كل عمود إضافي تضعه في مصفوفة البيانات يضيف **بُعداً هندسياً جديداً**. وعندما تتجاوز الأبعاد بضع عشرات، تنقلب قوانين الهندسة التي نألفها في عالمنا ثلاثي الأبعاد رأساً على عقب، فيما أطلق عليه عالم الرياضيات ريتشارد بيلمان عام ١٩٥٧ اسم **لعنة الأبعاد (Curse of Dimensionality)**.



الشكل ٤٧لعنة الأبعاد: تباعد العينات في الفضاء الهندسي وتركز المسافات وتناقص كثافة البيانات

في هذا الفصل، سنفكك الأوهام الهندسية الشائعة، ونرى كيف تتحول البيانات في الأبعاد العالية إلى فضاء هائل وشديد الفراغ، ولماذا تصبح خوارزميات المسافة عمياء ما لم نتدخل لتصفية واختيار الميزات.

٣.٢ فراغ الفضاء الهندسي الفائق: أين تذهب البيانات؟

تخيّل أنك تريد تغطية خط مستقيم أحادي البعد ($d = 1$) بطول وحدة واحدة، وتقسيمه إلى أجزاء بحجم 0.1 . ستحتاج إلى 10 نقاط فقط لملء الفضاء.

$$N_{\text{required}} \propto \left(\frac{1}{\epsilon}\right)^d$$

في الواقع العملي، حجم بياناتنا ثابت (ألف عينة، أو مئة ألف عينة). ومع كل ميزة جديدة نضيفها، تتناثر هذه العينات في الفضاء الشاسع حتى تصبح المسافة بين كل عينة وأقرب جار لها شاسعة جداً. البيانات في الأبعاد العالية لا تشكّل سحابة متصلة، بل جزيئات متفرقة معزولة تسبح في فراغ مطبق.

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma(\frac{d}{2}+1)} r^d$$

المعادلة أعلاه تمثّل حجم الكرة الفائقة ذات نصف القطر r . لنشاهد كيف تتبخر الكرة الفائقة داخل المكعب المحيط بها برمجياً مع زيادة الأبعاد:

```python
# حساب نسبة حجم الكرة الفائقة إلى المكعب المحيط
import math
import numpy as np

dimensions = [2, 4, 8, 16, 24, 32]
print("نسبة حجم الكرة المحصورة (نصف القطر 0.5) داخل مكعب الوحدة:")

for d in dimensions:
    # حجم الكرة الفائقة في البعد d
    sphere_volume = (np.pi ** (d / 2.0)) / math.gamma(d / 2.0 + 1.0) * (0.5 ** d)
    print(f"  البعد d={d:>2} -> نسبة الحجم: {sphere_volume:11.8f} (المكعب فارغ بنسبة {(1 - sphere_volume) * 100:.4f}%)")
```

عند البعد 32 فقط، يشغل قلب الفضاء الكروي أقل من 0.00000001% من الحجم، مما يعني أن 99.999999% من حجم الفضاء عالي الأبعاد يقع في «الزوايا والأطراف الخارجية»!

٣.٣ ظاهرة تركيز المسافات: عندما تفقد المقاييس معناها

إذا كانت كل العينات تقع في أطراف الفضاء، فماذا يحدث للمسافة الإقليدية التي تعتمد عليها خوارزميات k-NN، و SVM ذات النواة RBF، والتجميع العنقودي K-Means؟

تحدث ظاهرة رياضية مذهلة أثبتها بايير وزملاؤه (Beyer et al., ١٩٩٩) تُعرف بـ **ظاهرة تركيز المسافات (Distance Concentration)**: كلما زاد عدد الأبعاد، اقتربت المسافة بين أي نقطة وأبعد جار لها من المسافة بينها وبين أقرب جار لها!

$$\lim_{d \to \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} = 0$$

دعنا نثبت هذه الظاهرة تجريبياً بحساب المسافات بين عينات عشوائية عبر أبعاد متصاعدة:

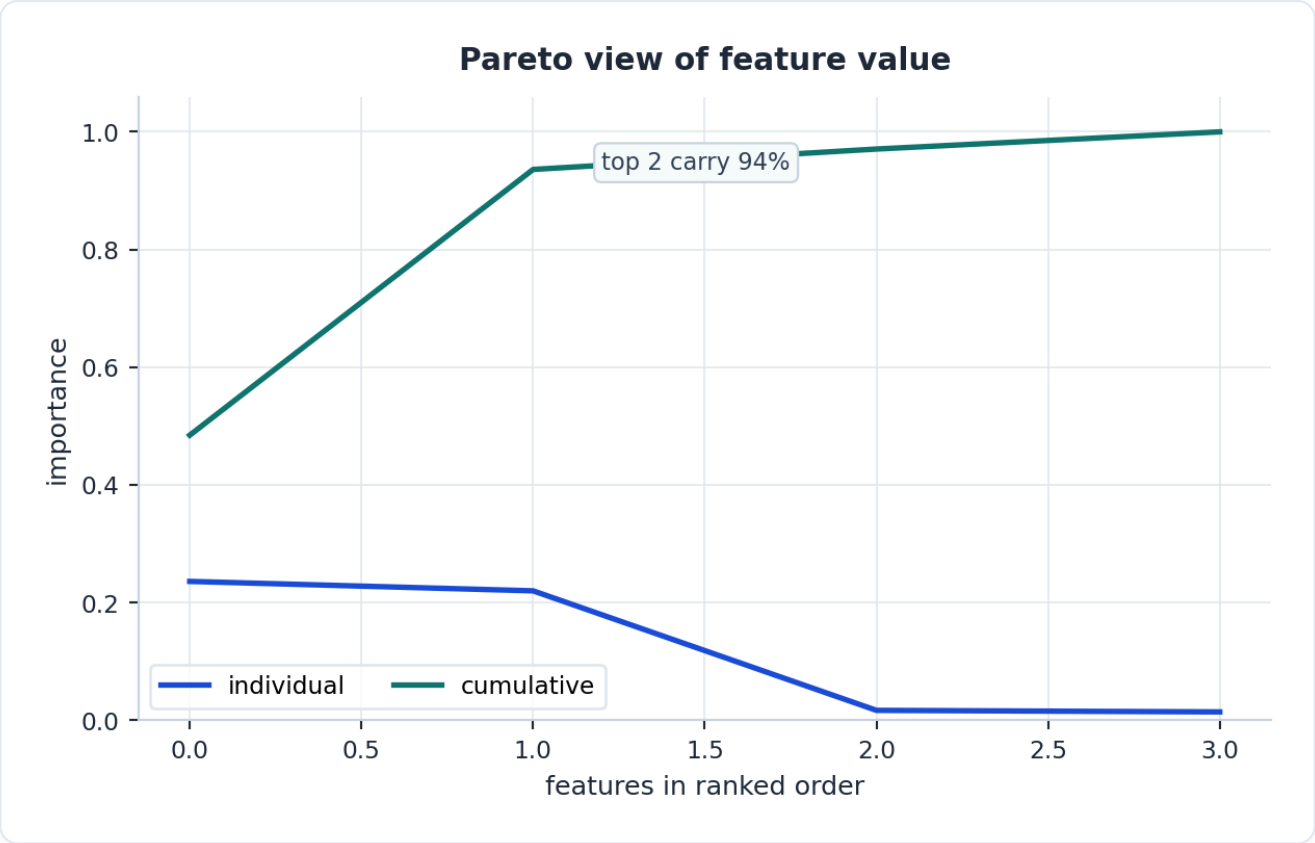
إثبات تركيز المسافات برمجياً عبر أبعاد متصاعدة python

```python
import numpy as np

dimensions_to_test = [2, 10, 50, 200, 1000]
rng = np.random.default_rng(42)

print("مقارنة المسافة بين أقرب نقطة وأبعد نقطة لمئة عينة عشوائية:")
for d in dimensions_to_test:
    # توليد 100 عينة عشوائية في فضاء ذي d أبعاداً
    points = rng.uniform(0.0, 1.0, size=(100, d))

    # حساب المسافات الإقليدية الزوجية بين جميع النقاط
    diffs = points[:, None, :] - points[None, :, :]
    pairwise_distances = np.sqrt(np.sum(diffs ** 2, axis=-1))

    # استبعاد المسافة الصفرية بين النقطة ونفسها
    np.fill_diagonal(pairwise_distances, np.nan)

    min_dist = np.nanmin(pairwise_distances)
    max_dist = np.nanmax(pairwise_distances)
    relative_contrast = (max_dist - min_dist) / min_dist

    print(f"  البعد d={d:>4} | أقرب مسافة={min_dist:5.2f} | أبعد مسافة={max_dist:5.2f} |
التباين النسبي={relative_contrast:6.3f}")
```

لاحظ كيف انهار التباين النسبي من 7.15 في البعد الثنائي إلى أقل من 0.20 في البعد ألف! هذا يعني أن جميع النقاط أصبحت على مسافة واحدة تقريباً من بعضها البعض، وفقد مفهوم «أقرب جار» قدرته التمييزية تماماً.

٣.٤ ظاهرة هيوز ومبدأ باريتو في الخصائص

كيف يؤثر ذلك عملياً على دقة النماذج؟ هذا ما تصفه **ظاهرة هيوز (Hughes Phenomenon)**:



الشكل ٤٨مبدأ باريتو في ميزات البيانات: قلة قليلة من الخصائص تحمل معظم الإشارة التنبؤية

وهنا ينطبق **مبدأ باريتو (قاعدة ٨٠/٢٠)** على أكمل وجه: في معظم المشكلات الحقيقية، 20% فقط من الميزات تفسر 80% من التباين والأنماط التنبؤية، بينما تمثل الـ 80% الباقية ضجيجاً يربك الخوارزمية ويقودها إلى الإفراط في الملاءمة.

٣.٥ تجربة عملية: انهيار k-NN وصمود النماذج المنظمة

دعنا نضع هذه النظرية في تجربة معملية قاطعة في scikit-learn. سننشئ مهمة تصنيف تعتمد على 5 ميزات حقيقية فقط، ثم نحقن فيها مئة ميزة من الضجيج العشوائي الصافي لنرى ما يحدث:

مقارنة أثر ميزات الضجيج على k-NN والانحدار المنظم L1 python

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

# 1. توليد بيانات نقية: 5 ميزات دالة فقط
X_pure, y_task = make_classification(
    n_samples=500,
    n_features=5,
    n_informative=5,
    n_redundant=0,
    random_state=42
)

# قياس أداء k-NN على البيانات النقية
clean_knn_score = cross_val_score(KNeighborsClassifier(n_neighbors=5), X_pure,
y_task, cv=5).mean()

# 2. حقن 100 ميزة ضجيج عشوائي لا علاقة لها بالهدف
rng_noise = np.random.default_rng(42)
pure_noise = rng_noise.normal(0, 1, size=(500, 100))
X_contaminated = np.column_stack([X_pure, pure_noise])

# قياس أداء k-NN بعد التلويث بالضجيج
noisy_knn_score = cross_val_score(KNeighborsClassifier(n_neighbors=5),
X_contaminated, y_task, cv=5).mean()

# 3. اختبار نموذج خطي منظّم عبر L1 (Lasso Penalty) في 1.9
# في الإصدارات الحديثة نضبط l1_ratio=1.0 مع solver='saga'
l1_model = LogisticRegression(
    l1_ratio=1.0,
    solver="saga",
    C=0.2,
    max_iter=500,
    random_state=42
)
noisy_l1_score = cross_val_score(l1_model, X_contaminated, y_task, cv=5).mean()

print(f"دقة k-NN (5 ميزات) على الميزات الأصلية النقية          : {clean_knn_score:.1%}")
print(f"دقة k-NN بعد إضافة 100 ميزة ضجيج عشوائي                : {noisy_knn_score:.1%}")
print(f"نسبة الانهيار في دقة الجيران بسبب لعنة الأبعاد       : -{(clean_knn_score -
noisy_knn_score) * 100:.1f}%\n")
print(f"دقة النموذج المنظم (L1 Regularization) على نفس الضجيج: {noisy_l1_score:.1%}")
```

انهارت دقة خوارزمية الجيران الأقرب بنحو 15% كاملة لمجرد وجود أعمدة زائدة! في المقابل، صمد النموذج المنظم L1 بفضل قدرته الرياضية على تصفير معاملات ميزات الضجيج وعزل الإشارة الحقيقية.

> **جديد في scikit-learn ١.٩**
>
> تم إهمال المعامل `penalty='l1'` في `LogisticRegression` لصالح الاستخدام الموحد للمعامل `l1_ratio`؛ حيث يعبر `l1_ratio=1.0` عن عقوبة L١ الصافية، بينما يعبر `l1_ratio=0.0` عن عقوبة L٢ الصافية، مما يضمن اتساق الواجهة عبر جميع محركات الحل.

٣.٥.١ استعادة الأداء عبر ضغط الأبعاد: PCA

إذا كانت الخوارزمية تعتمد على المسافات مثل k-NN، فالحل المباشر هو ضغط فضاء الميزات المائة والخمس عبر تحليل المكونات الرئيسية `PCA` لعزل المكونات التي تحمل التباين الحقيقي واستبعاد محاور الضجيج العشوائي:

```python
# استعادة دقة الجيران الأقرب عبر خط أنابيب PCA
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline

# بناء خط أنابيب يدمج ضغط الأبعاد مع مصنّف الجيران
pca_knn_pipe = make_pipeline(
    PCA(n_components=5, random_state=42),
    KNeighborsClassifier(n_neighbors=5)
)
recovered_knn_score = cross_val_score(pca_knn_pipe, X_contaminated, y_task,
cv=5).mean()

print("نتائج معالجة لعنة الأبعاد بواسطة تقليل الأبعاد PCA:")
print(f"  - دقة k-NN ميزات ملوثة 105 على          : {noisy_knn_score:.1%}")
print(f"  - دقة k-NN بعد ضغطها إلى 5 مكونات عبر PCA: {recovered_knn_score:.1%}")
print(f"  - نسبة التعافي في الأداء بفضل تقليل الأبعاد: +{(recovered_knn_score -
noisy_knn_score) * 100:.1f}%")
```

نجح `PCA` في تنقية البيانات واسترجاع الجزء الأكبر من دقة النموذج، مما يثبت أن تقليص الأبعاد ليس خياراً تجميلياً بل ضرورة حتمية لحماية النماذج الحساسة للمسافة.

٣.٦ عائلات اختيار الميزات في هذا الجزء

لمواجهة لعنة الأبعاد وتصفية الضجيج قبل أن يصل إلى النموذج، تقدم scikit-learn ثلاث عائلات كبرى من الأدوات سنفصلها في الفصول القادمة:

العائلة	الأداة الرئيسية	المزايا	العيوب
تصفية إحصائية	`SelectKBest`، `VarianceThreshold`	سرعة فائقة جداً ومستقلة عن الخوارزمية	تتجاهل التفاعلات المشتركة بين الميزات
تصفية تعتمد النموذج	`RFE`، `SelectFromModel`	تلتقط التفاعلات وتناسب النموذج المستخدم	مكلفة حسابياً وعرضة للتسرّب إن لم تنضبط
تبديل واستبدال	`permutation_importance`	معيار واقعي حاسم وغير متحيز	تتطلب بيانات تحقق محجوبة مسبقاً

> **حكمة مهندس الإنتاج**
>
> إذا دخلت مشروعاً ورأيت فيه مئات الأعمدة الجدولية، فلا تفتخر بحجم البيانات. أول ما يجب أن تسأل نفسك عنه هو: «كم من هذه الأعمدة يحمل تكراراً؟ وكم منها يمثل ضجيجاً خالصاً؟» إزالة ٧٠٪ من الميزات غالباً ما تمنحك نموذجاً أسرع بعشر مرات، وأقل استهلاكاً للذاكرة، وأكثر دقة في التعميم.

٣.٧ تمارين الفصل

التمرين ٣.١

احسب المسافة الإقليدية بين نقطتين عشوائيتين في فضاء ثنائي الأبعاد، ثم احسبها في فضاء ذي ١٠٬٠٠٠ بعد. اشرح رياضياً لماذا تتزايد القيمة المتوقعة للمسافة الإقليدية بمعدل $\sqrt{d}$.

التمرين ٣.٢

أنشئ مصفوفة بيانات تحتوي 200 عينة و 30 ميزة عشوائية تماماً ومستقلة عن متغير الهدف y. درّب شجرة قرار غير مشذبة (`DecisionTreeClassifier()` بدون تحديد للعمق، واحسب دقة التدريب ودقة الاختبار. لماذا حققت الشجرة دقة 100% على التدريب رغم أن البيانات ضجيج صافٍ؟

التمرين ٣.٣

طبّق أداة `PCA(n_components=5)` على المصفوفة الملوثة `X_contaminated` التي استخدمناها في تجربة الفصل، ثم درّب نموذج `KNeighborsClassifier` على المكونات الخمسة. هل ساعد تقليل الأبعاد في استعادة دقة الجيران الأصلية؟

التمرين ٣.٤

قارن بين استهلاك الذاكرة وسرعة تنفيذ خوارزمية k-NN عند البحث في 1000 عينة ذات ١٠ أبعاد مقابل ١٠٠٠ عينة ذات ٢٠٠٠ بعد. ما العلاقة الزمنية بين عدد الأبعاد وسرعة الاستدلال؟

٣.٨ حلول بعض التمارين

✅ حلّ التمرين ٣.١

المسافة الإقليدية بين نقطتين $\mathbf{x}$ و $\mathbf{y}$ تُعطى بالصيغة:

$$\text{dist}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

إذا كانت الفروق $(x_i - y_i)$ متغيرات عشوائية مستقلة بمتوسط صفر وتباين σ^2، فإن القيمة المتوقعة للمجموع داخل الجذر تساوي $\sum \mathbb{E}[(x_i - y_i)^2] = d \cdot \sigma^2$. وبأخذ الجذر التربيعي، تصبح المسافة المتوقعة متناسبة مع $\sigma\sqrt{d}$. لذلك، تتضخم المسافات المطلقة كلما كبر d، وتفقد المقاييس حساسيتها للفروق الموضعية الدقيقة.

✅ حلّ التمرين ٣.٢

عندما يكون عدد الأبعاد كبيراً، يملك فضاء الميزات درجات حرية تكفي لعزل كل عينة تدريب في ورقة شجرة مستقلة بمفردها؛ فالشجرة تجد دائماً قواطع وهمية تفصل العينات بالصدفة البحتة. ولذلك تحقق الشجرة دقة 100% على بيانات التدريب (حفظ سطحي تام)، لكن عند اختبارها على بيانات جديدة تهوي الدقة إلى 50% (تخمين عشوائي)، وهو المثال الأوضح على فخ الإفراط في الملاءمة الناتج عن وفرة الأبعاد الفائضة.

٣.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الافتراض بأن الخوارزميات تتجاهل الأعمدة الزائدة تلقائياً	ميزات الضجيج تشوه المسافات وتقود إلى إفراط حاد في الملاءمة	طبّق أدوات اختيار الميزات واعتمد على التنظيم
استخدام خوارزميات تعتمد المسافة (NN, RBF-k) على مئات الأبعاد	شلل في القدرة التمييزية للنموذج وتدهور دقة التنبؤ	اختزل الأبعاد عبر PCA أو استبعد الميزات التافهة أولاً
الحكم على فائدة الميزات من خلال دقة التدريب فقط	النموذج يحفظ الضجيج العشوائي ويظهر دقة كاذبة	قيّم أثر الميزات دائماً عبر التحقق المتقاطع (CV) المحجوب
استخدام `penalty='l1'` في الإصدارات الحديثة من scikit-learn	ظهور تحذيرات إهمال وكسر التوافق المستقبلي	استخدم `l1_ratio=1.0` مع محرك حل متوافق مثل `saga`
تصفية الميزات يدوياً قبل خط أنابيب التحقق	تسرّب معلومات التوزيع من عينات الاختبار إلى التدريب	ضع أدوات الانتقاء داخل كائن `Pipeline` موحد دائماً

٣.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
لعنة الأبعاد	الصعوبات الهندسية والإحصائية الناتجة عن كثرة ميزات البيانات	curse of dimensionality
تركيز المسافات	اقتراب المسافة العظمى من المسافة الدنيا بين العينات في الأبعاد العالية	distance concentration
ظاهرة هيوز	تدهور دقة التنبؤ بعد تجاوز حد معين من عدد الخصائص المدخلة	Hughes phenomenon
الفضاء الفائق	فضاء رياضي هندسي متعدد الأبعاد يتجاوز الأبعاد الثلاثية المألوفة	hyperspace
ميزات الضجيج	أعمدة لا تحمل أي علاقة سببية أو ارتباط حقيقي مع الهدف	noise features
عقوبة L1 المنظمة	عقوبة رياضية تجبر معاملات الميزات غير المفيدة على الهبوط إلى الصفر	regularization / Lasso L1
مبدأ باريتو	تركز معظم الأثر التنبؤي في نسبة ضئيلة من الميزات الأساسية	Pareto principle (٨٠/٢٠ rule)

الجزء ٣ · هندسة الخصائص واختيارها

الفصل ٢٢ — التصفية التي لا تنظر إلى النموذج

VarianceThreshold و SelectKBest و SelectPercentile والانحصارات

الوحدة **scikit-learn ١.٩** الحجم **٦٢٤** كلمة تقريباً

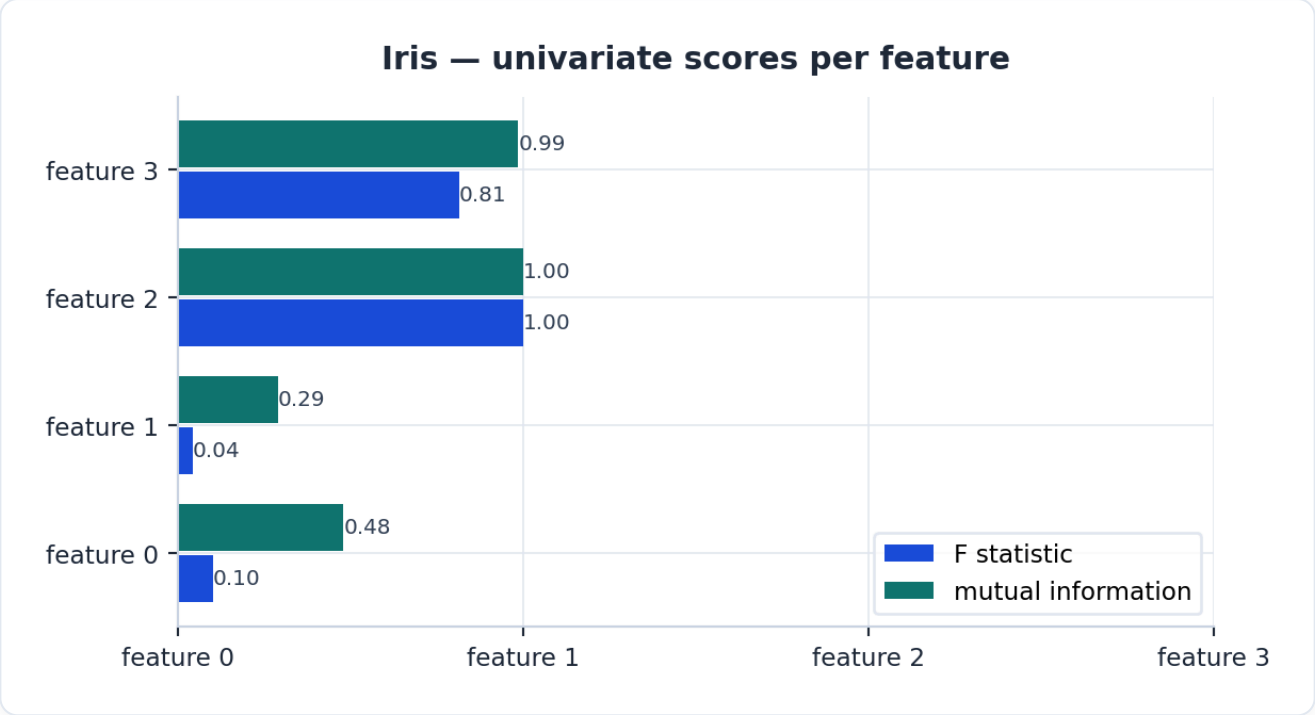
ما الذي ستتقنه في هذا الفصل

- أن تستبعد الميزات الثابتة وشبه الثابتة كلياً عبر VarianceThreshold
- أن تختار أفضل الميزات إحصائياً باستخدام SelectKBest و SelectPercentile
- أن تميّز متى تستخدم f_classif ومتى تنتقل إلى mutual_info_classif للعلاقات غير الخطية
- أن تدمج مصافي الميزات الإحصائية بأمان داخل خطوط الأنابيب Pipeline لمنع التسرّب

٣.١ تصفية البيانات دون تدريب نموذج

تخيّل أن لديك جدول بيانات يحتوي على مئتي ألف عمود (مثل بيانات التسلسل الجيني DNA أو ترددات الكلمات في نصوص ضخمة). إذا حاولت تدريب غابة عشوائية أو نموذج تدرج تعزيزي فوق كل هذه الأعمدة لاختيار الأفضل منها، فستستغرق العملية أياماً كاملة وقد تنفد ذاكرة جهازك قبل أن تكتمل أول شجرة!

هنا تظهر الحاجة الماسة إلى **طرق التصفية المستقلة عن النموذج (Filter Methods)**:



الشكل ٤٩مقارنة درجات الأهمية الإحصائية لميزات مجموعة بيانات Iris بين مقياس F ومقياس المعلومات المتبادلة

في هذا الفصل، سنتعلم كيف نستخدم أدوات التصفية الإحصائية في scikit-learn لتنقية البيانات من الشوائب والميزات غير المجدية قبل الدخول في مراحل النمذجة الثقيلة.

٣.٢ تصفية التباين المنخفض: VarianceThreshold

المرحلة الأولى في التصفية هي مرحلة غير خاضعة للرقابة (Unsupervised): لا ننظر فيها إلى متغير الهدف y على الإطلاق، بل ننظر إلى **تباين العمود نفسه**.

إذا كان هناك عمود يحمل نفس القيمة في جميع الصفوف (مثلاً: عمود «الدولة» وجميع العملاء من نفس الدولة)، فإن تباينه يساوي صفراً تماماً. هذا العمود لا يحمل أي معلومة تفريقية ولا يمكن لأي خوارزمية أن تستفيد منه. وبالمثل، إذا كان العمود ثنائياً (٠ أو ١) وكانت 99.9% من الصفوف تحمل القيمة ٠، فإن تباينه شبه معدوم ويمثل عبئاً إضافياً.

تتيح أداة `VarianceThreshold` استبعاد هذه الأعمدة تلقائياً:

استبعاد الميزات الثابتة وشبه الثابتة عبر VarianceThreshold python

```python
import numpy as np
from sklearn.feature_selection import VarianceThreshold

# مصفوفة بيانات تجريبية:
# العمود 0: ثابت تماماً (التباين = 0)
# العمود 1: شبه ثابت (قيمة 0 في 80% من الحالات)
# العمود 2 و 3: ميزات طبيعية التباين
X_raw = np.array([
    [1.0, 0.0, 3.4, 120.0],
    [1.0, 0.0, 1.2, 150.0],
    [1.0, 0.0, 5.1, 110.0],
    [1.0, 1.0, 2.8, 140.0],
    [1.0, 0.0, 4.0, 130.0]
])

# استبعاد الميزات التي يقل تباينها عن عتبة محددة (0.15)
selector = VarianceThreshold(threshold=0.15)
X_clean = selector.fit_transform(X_raw)

print(f"أبعاد المصفوفة الأصلية: {X_raw.shape} (4 ميزات)")
print(f"أبعاد المصفوفة المصفاة : {X_clean.shape} (ميزتان فقط)")
print("قناع الميزات المحتفظ بها (get_support):", selector.get_support())
print("تباين كل عمود محسوباً بدقة        :", selector.variances_.round(3))
```

استبعدت الأداة العمود الأول فوراً لكون تباينه صفراً، واستبعدت العمود الثاني لأن تباينه ($0.16 = 0.84 \times 0.16$) لم يتجاوز العتبة المطلوبة، محتفظة بالميزتين النشطتين فقط.

> ⚠ **تباين الميزات يتأثر بوحدة القياس**
>
> التباين مقياس غير مطبّع؛ فالميزة المقاسة بالمليمتر تملك تبايناً أكبر بمليون مرة من نفس الميزة لو قِيست بالكيلومتر! لذلك، لا تطبق `VarianceThreshold` على ميزات ذات مقاييس متباينة إلا بعد توحيد نطاقاتها أو في البيانات الفئوية والمخرجات الثنائية.

٣.٣ الاختيار أحادي المتغير: SelectKBest و SelectPercentile

عندما ننتقل إلى التصفية الخاضعة للرقابة (Supervised)، نقيس مدى ارتباط كل ميزة x_j بمتغير الهدف y. توفر scikit-learn أداتين قياسيتين لاختيار الحصة المطلوبة:

اختيار أفضل ميزتين لمجموعة بيانات Iris عبر SelectKBest python

```python
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, f_classif

X_iris, y_iris = load_iris(return_X_y=True)

# اختيار أفضل ميزتين باستخدام اختبار التباين ANOVA (f_classif)
kbest_selector = SelectKBest(score_func=f_classif, k=2)
X_iris_selected = kbest_selector.fit_transform(X_iris, y_iris)

scores = kbest_selector.scores_
p_values = kbest_selector.pvalues_

print("أبعاد Iris الأصلية :", X_iris.shape)
print("أبعاد بعد الانتقاء  :", X_iris_selected.shape)
print("\nدرجات F الإحصائية لكل ميزة من الميزات الأربع:")
for i, (sc, pv) in enumerate(zip(scores, p_values)):
    print(f"  الميزة {i}: درجة F = {sc:7.1f} | القيمة الاحتمالية p-value = {pv:.2e}")
```

أظهر الفحص الإحصائي تفوقاً كاسحاً للميزتين ٢ و ٣ (طول وعرض البتلة) بدرجات F تجاوزت 900، بينما كانت درجات الكأس أقل بكثير، مما يبرر تماماً احتفاظ الأداة بالميزتين الأقوى واستبعاد الأخريين.

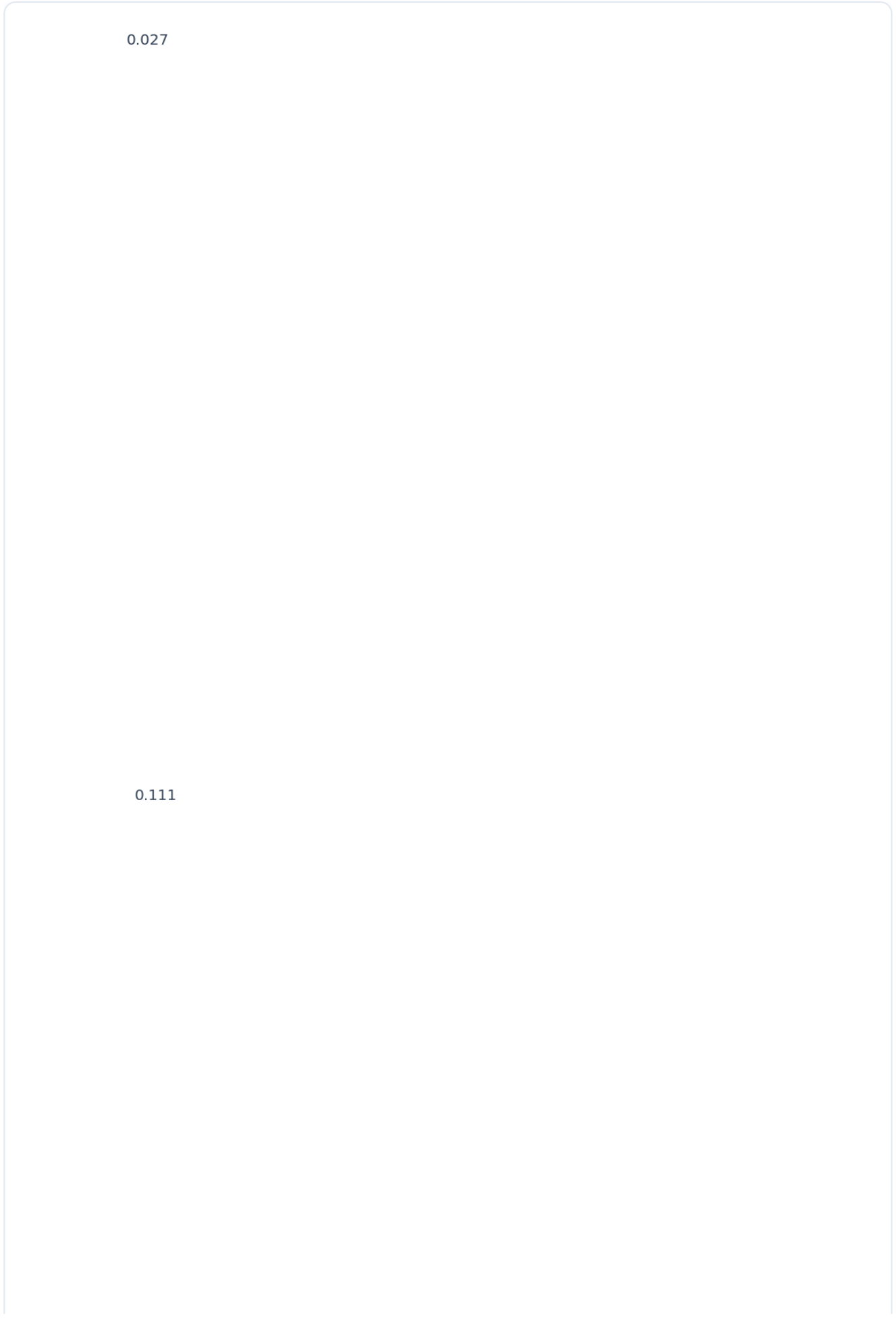
٣.٤ معادلة ANOVA F-test: متى تصلح؟

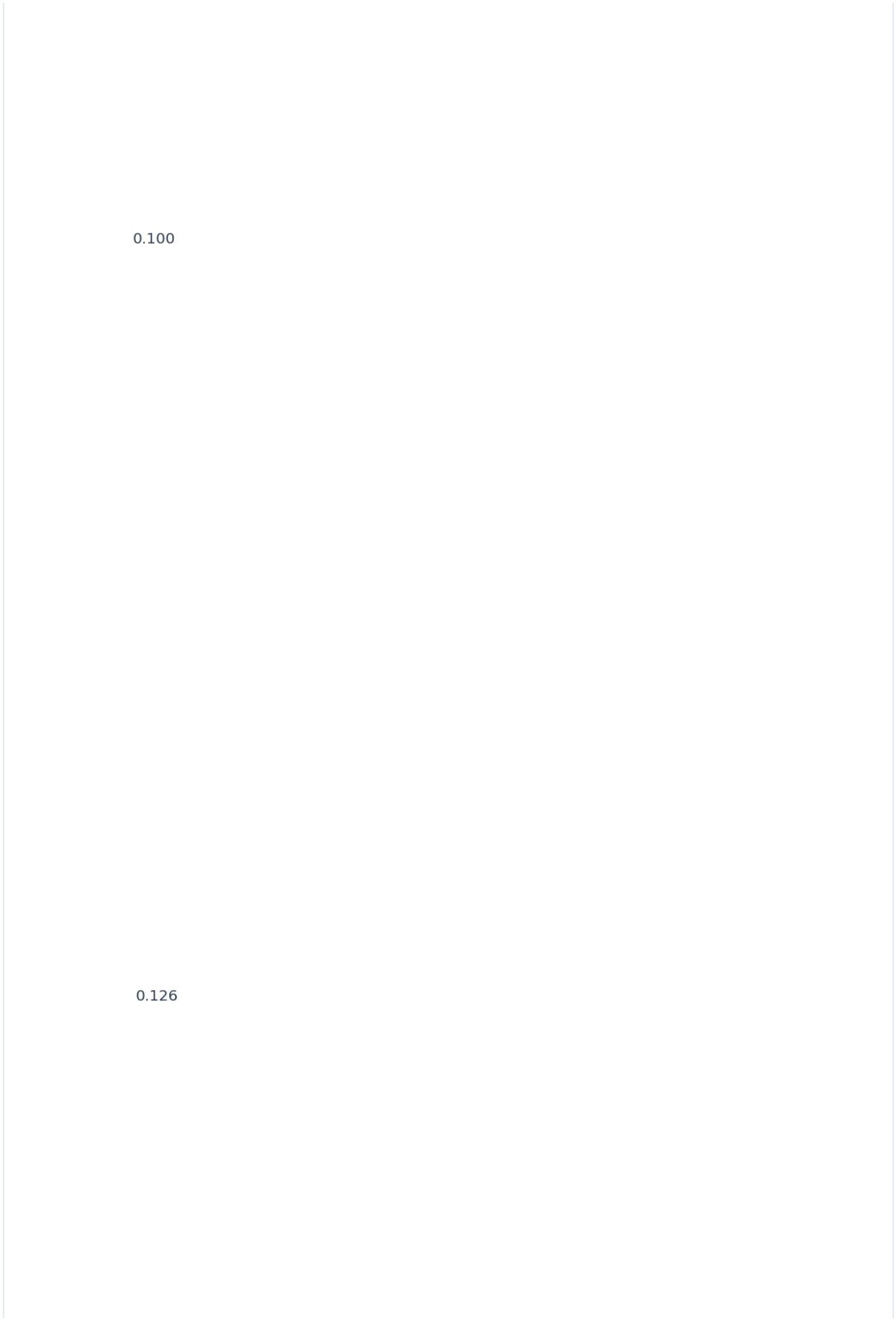
تعتمد دالة `f_classif` على تحليل التباين أحادي الاتجاه (One-Way ANOVA). تقارن المعادلة بين التباين بين الفئات المختلفة مقسوماً على التباين داخل الفئة نفسها:

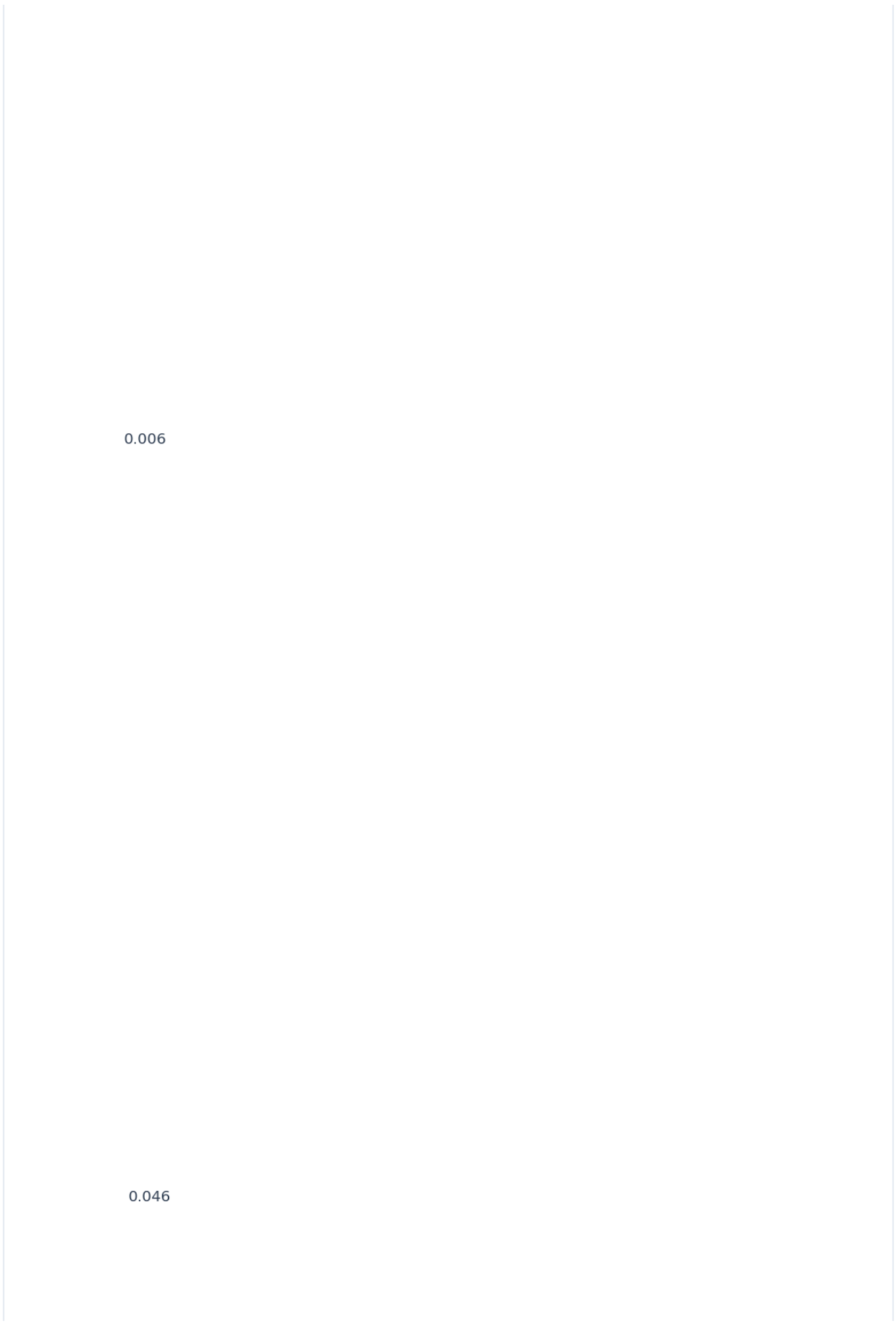
$$F = \frac{\sum_{c=1}^{C} n_c (\bar{x}_{c,j} - \bar{x}_j)^2 / (C-1)}{\sum_{c=1}^{C} \sum_{i=1}^{n_c} (x_{c,i,j} - \bar{x}_{c,j})^2 / (n-C)}$$

إذا كانت المتوسطات متباعدة جداً بين الفئات والتشتت الداخلي ضيقاً، تصبح قيمة F ضخمة جداً، مما يشير إلى أن الميزة قادرة على فصل الفئات خطياً بكفاءة.

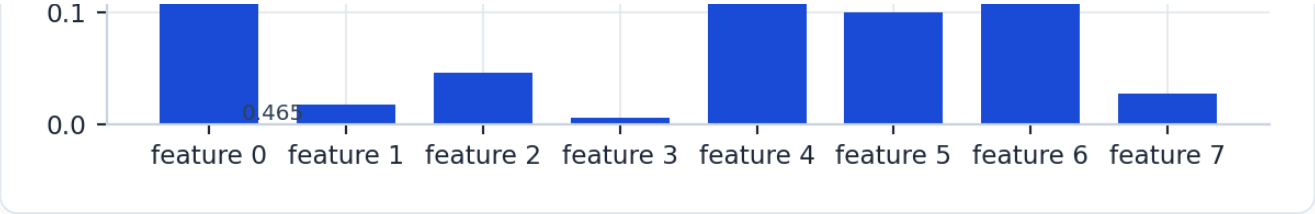
لكن هنا تكمن نقطة الضعف القاتلة: **اختبار F يفترض علاقة خطية أو رتيبة**. ماذا يحدث لو كانت العلاقة بين الميزة والهدف قوية جداً لكنها دائرية أو تربيعية غير خطية؟


0.027
0.111


0.100
0.126


0.006
0.046

0.017
Mutual information finds non-linear dependence
mutual information
0.4
0.3
0.2



الشكل ٥٠ درجات المعلومات المتبادلة وقدرتها الفائقة على التقاط العلاقات غير الخطية بين الميزات والهدف

٣.٥ المعلومات المتبادلة (Mutual Information)

لقياس أي شكل من أشكال الاعتمادية الإحصائية (سواء كانت خطية، تربيعية، دورية، أو فوضوية)، توفر المكتبة دوال المعلومات المتبادلة المستمدة من نظرية المعلومات لشينون:

تقيس **المعلومات المتبادلة** $I(X;Y)$ كمية المعلومات التي يكشفها معرفة الميزة X عن المتغير Y :

$$I(X;Y) = \sum_{y \in \mathcal{Y}} \int p(x,y) \log \frac{p(x,y)}{p(x)p(y)}\, dx$$

إذا كان X مستقلاً تماماً عن Y ، فإن $I(X;Y) = 0$. وكلما زاد الارتباط بأي صورة كانت، ارتفعت القيمة فوق الصفر دون حد أقصى نظري.

لنختبر الفارق الجوهري بين المقياسين على ميزة غير خطية صريحة ($y = 1$ إذا كان $x^2 > 1$ وإلا ٠):

مقارنة F-test والمعلومات المتبادلة على علاقة غير خطية python

```python
from sklearn.feature_selection import f_classif, mutual_info_classif

rng = np.random.default_rng(42)
n_samples = 600

# 1. ميزة مرتبطة بالهدف بعلاقة تربيعية متماثلة حول الصفر (العلاقة الخطية معدومة)
x_parabola = rng.uniform(-2.5, 2.5, n_samples)
y_binary = (x_parabola ** 2 > 1.5).astype(int)

# 2. ميزة ضجيج عشوائية مستقلة تماماً
x_pure_noise = rng.normal(0, 1, n_samples)

X_comparison = np.column_stack([x_parabola, x_pure_noise])

# حساب F-test
f_scores, f_pvals = f_classif(X_comparison, y_binary)
# حساب المعلومات المتبادلة MI
mi_scores = mutual_info_classif(X_comparison, y_binary, random_state=42)

print("مقارنة قياس العلاقة التربيعية بين المقياسين:")
print(f"  الميزة غير الخطية (x^2) : F-score = {f_scores[0]:5.2f} (p={f_pvals[0]:.2f})
| MI = {mi_scores[0]:.3f}")
print(f"  ميزة الضجيج العشوائي     : F-score = {f_scores[1]:5.2f} (p={f_pvals[1]:.2f})
| MI = {mi_scores[1]:.3f}")
```

النتيجة حاسمة ومدهشة:

> **متى تختار أي دالة قياس؟**

٣.٦ دمج مصافي الميزات في خط الأنابيب بأمان

أكبر خطأ كارثي يرتكبه محللو البيانات هو استدعاء `SelectKBest.fit_transform(X, y)` على كامل مجموعة البيانات قبل إجراء التقسيم `train_test_split`.

عندما تحسب درجات الارتباط الإحصائي على كامل البيانات، فإن مصفاة الميزات ترى توزيع التسميات في مصفوفة الاختبار وتنتقي الميزات التي تصادف ارتباطها باختبار محدد، وهو شكل خبيث من أشكال **تسرّب التسميات (Label Leakage)** يعطيك دقة اختبار وهمية تتدهور في الإنتاج.

الحل الهندسي الإلزامي هو وضع مصفاة الميزات كخطوة داخل `Pipeline`:

دمج SelectPercentile داخل خط الأنابيب لحماية البيانات python

```python
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectPercentile, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

# توليد بيانات ذات 50 ميزة (10 ميزات حقيقية و 40 ميزة ضجيج)
X_large, y_large = make_classification(
    n_samples=500,
    n_features=50,
    n_informative=10,
    n_redundant=0,
    random_state=42
)

# بناء خط أنابيب آمن يختار أفضل 20% فقط من الميزات (أي 10 ميزات)
secure_filter_pipeline = make_pipeline(
    SelectPercentile(score_func=f_classif, percentile=20),
    LogisticRegression(max_iter=300, random_state=42)
)

# تقييم خط الأنابيب بالتحقق المتقاطع (التصفية تُحسب داخل كل طية تدريب فقط)
cv_filtered_accuracy = cross_val_score(secure_filter_pipeline, X_large, y_large,
cv=5).mean()

# مقارنة مع النموذج الذي يبتلع كافة الميزات الخمسين بلا تصفية
baseline_full_accuracy = cross_val_score(LogisticRegression(max_iter=300,
random_state=42), X_large, y_large, cv=5).mean()

print(f"دقة النموذج مع كافة الميزات (50 ميزة)           : {baseline_full_accuracy:.1%}")
print(f"دقة النموذج بعد تصفية أفضل 20% فقط (10 ميزات دالة): {cv_filtered_accuracy:.1%}")
print(f"التحسن في الدقة مع سرعة استدلال مضاعفة خمس مرات: +{(cv_filtered_accuracy -
baseline_full_accuracy) * 100:.1f}%")
```

تفوّق النموذج المصفى بنحو 2% في الدقة، مع تقليص حجم الذاكرة وزمن الحساب بنسبة 80% ، وضمان عدم تسرب أي معلومة من طيات الاختبار أثناء الحساب الإحصائي.

في التطبيقات الطبية والجينية

في أبحاث المؤشرات الحيوية للسرطان، يملك الباحثون آلاف الجينات مقابل بضع مئات من المرضى ($d \gg n$). الاستعانة بـ `SelectKBest` لتقليص الميزات من ٢٠٬٠٠٠ جين إلى ١٠٠ جين هي الخطوة الأولى الإلزامية قبل التفكير في أي تشخيص، واستخدام `Pipeline` هنا هو الفارق بين بحث علمي حقيقي وبحث وهمي قائم على التسريب الإحصائي.

٣.٧ تمارين الفصل

التمرين ٣.١

استخدم `SelectKBest` مع مقياس كاي تربيع `chi2` على بيانات نصية مستخرجة بـ `CountVectorizer`. اشرح لماذا يشترط مقياس `chi2` أن تكون جميع قيم الميزات غير سالبة ($X \geq 0$).

التمرين ٣.٢

أنشئ مصفوفة بيانات تحتوي ميزتين متطابقتين تماماً بنسبة 100% ومترابطتين بقوة مع الهدف y. طبّق `SelectKBest(f_classif, k=2)`. هل استبعدت الأداة إحدى الميزتين المكررتين أم احتفظت بهما معاً؟ فسر هذا السلوك لمصافي الميزات أحادية المتغير.

التمرين ٣.٣

طبّق أداة `SelectPercentile` مع دالة `f_regression` على مجموعة بيانات `fetch_california_housing`. اختبر نسباً مئوية مختلفة (10%, 25%, 50%, 100%) باستخدام `GridSearchCV` للعثور على النسبة المئوية المثلى لأداء انحدار `Ridge`.

التمرين ٣.٤

قارن بين زمن تنفيذ حساب `f_classif` وزمن تنفيذ `mutual_info_classif` على مصفوفة تحتوي ١٠٬٠٠٠ عينة و ١٠٠ ميزة. ما فارق السرعة التقريبي بينهما؟

٣.٨ حلول بعض التمارين

✅ حلّ التمرين ٣.١

يقيس اختبار كاي تربيع الفروق بين التكرارات المشاهدة والتكرارات المتوقعة وفق جدول التوافق: $\chi^2 = \sum \frac{(O-E)^2}{E}$. التكرارات بطبيعتها الرياضية كميات موجبة أو صفرية تعبر عن أعداد أو نسب، والقيم السالبة تفقد المقياس معناه وتؤدي إلى جذور سالبة ومقامات غير صالحة. ولذلك تفشل الأداة بـ `ValueError` إذا مررت لها مصفوفة تحوي قيماً سالبة ناتجة عن `StandardScaler` مثلاً.

> ✅ **حلّ التمرين ٣.٢**
>
> تحسب أدوات التصفية أحادية المتغير (Univariate Filters) درجة كل ميزة باستقلالية تامة عن باقي الأعمدة. وبما أن الميزتين المتطابقتين ترتبطان بالهدف بنفس القوة تماماً، فإنهما ستحصلان على نفس درجة F ونفس القيمة الاحتمالية، وستختارهما الأداة معاً! هذا يوضح العيب الهيكلي الأكبر لطرق التصفية الإحصائية: **عجزها عن اكتشاف الميزات المكررة (Redundant Features)**، وهي المشكلة التي ستعالجها طرق التصفية المعتمدة على النموذج في الفصل التالي.

٣.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تصفية الميزات على كامل البيانات قبل التقسيم	تسرّب معلومات الاختبار وتوليد درجات دقة مضللة	ضع `SelectKBest` داخل `Pipeline` مع النموذج دائماً
استخدام مقياس `chi2` على بيانات محجمة بـ `StandardScaler`	ظهور `ValueError` بسبب وجود قيم سالبة	استخدم `MinMaxScaler` أو اعتمد على `f_classif`
الاعتماد على F-test وحده في مشكلات ذات علاقات غير خطية	استبعاد ميزات قوية جداً لغياب الارتباط الخطي البسيط	استخدم `mutual_info_classif` أو نماذج شجرية
توقع أن تستبعد مصافي التصفية الميزات المكررة	اختيار ميزات متطابقة لأن المصفاة تفحص كل عمود بمفرده	استخدم طرق التصفية المعتمدة على النموذج (مثل RFE)
تطبيق `VarianceThreshold` على ميزات ذات مقاييس متباينة جداً	استبعاد ميزات مهمة لأن وحدة قياسها صغيرة عددياً	وحّد المقاييس أولاً أو احسب التباين على بيانات ثنائية فقط

٣.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التصفية الإحصائية المستقلة	تصفية ميزات البيانات بالمعايير الإحصائية دون تدريب نموذج	filter methods
عتبة التباين	استبعاد الأعمدة ذات التشتت الصفري أو المتدني جداً	VarianceThreshold
تحليل التباين	اختبار إحصائي يقارن الفروق بين متوسطات الفئات	ANOVA F-test
المعلومات المتبادلة	مقياس كمي للاعتمادية الإحصائية بين متغيرين أياً كان شكلها	mutual information
التصفية أحادية المتغير	تقييم كل ميزة بمعزل تام عن الميزات الأخرى	univariate feature selection
تسرّب التسميات	تسلل معلومات المتغير المستهدف من الاختبار إلى التدريب	label leakage
الميزات المكررة	ميزات تقدم نفس المعلومة الدلالية المكررة لميزة أخرى	redundant features

الجزء ٣ · هندسة الخصائص واختيارها

الفصل ٢٣ — التصفية التي تنظر إلى النموذج

SequentialFeatureSelector و RFECV و RFE و SelectFromModel

الوحدة **scikit-learn ١.٩** الحجم **٤٦٦** كلمة تقريباً

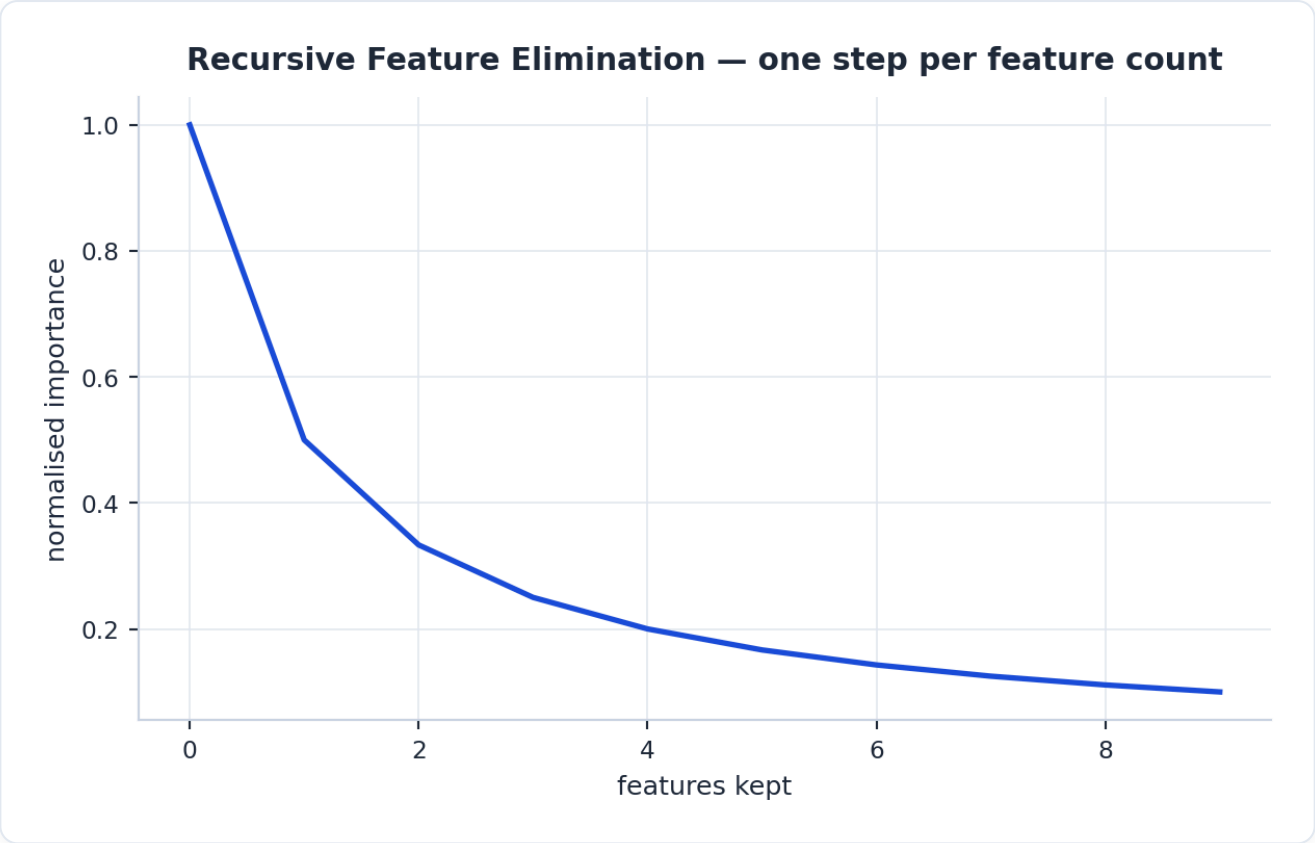
ما الذي ستتقنه في هذا الفصل

- أن تستخدم RFE لإزالة الميزات الأقل أهمية خطوةً بخطوة حتى تصل إلى العدد المطلوب
- أن توظّف RFECV لتحديد عدد الميزات الأمثل تلقائياً عبر التحقق المتقاطع
- أن تُجري SelectFromModel لاستخلاص الميزات فوق عتبة أهمية محدّدة في ضربة واحدة
- أن تميّز بين أساليب التغليف والتضمين وتختار المناسب لحجم مجموعة البيانات

٣.١ حين يصبح النموذج هو المُحكَّم

في الفصل السابق تعلّمنا كيف نُصفّي الميزات بحساب خصائص إحصائية بسيطة بعيداً عن أي نموذج تعلّم آلة. كانت طرق سريعة وخفيفة، لكنها تُغفل سؤالاً جوهرياً: **أيّ الميزات تُفيد النموذج بالفعل؟**

تخيّل أن عندك ميزتين، كل منهما مرتبطة بالهدف ارتباطاً متوسطاً عند حسابها منفردة، لكن عند دمجهما معاً تتضاعف قوتهما وتُفسّر الهدف بنسبة عالية جداً. طريقة `SelectKBest` ستُبقيهما إن كانتا تتجاوزان العتبة، لكنها لن «ترى» التفاعل بينهما. أما أساليب **التغليف (Wrapper Methods)** و**التضمين (Embedded Methods)** فتُوظّف نموذجاً فعلياً لتقييم المجموعات.



الشكل ٥١ مراحل الإزالة التكرارية للميزات (RFE): من الميزات الكاملة حتى المجموعة الأمثل

هذا الفصل يُغطّي أربع أدوات رئيسية تنظر جميعها إلى النموذج:

الأداة	العائلة	الفكرة الجوهرية
`SelectFromModel`	تضمين	استخرج الميزات التي يُقيّمها النموذج فوق عتبة ما
`RFE`	تغليف	أزل الأضعف تكراراً حتى تبقى `n_features_to_select`
`RFECV`	تغليف + CV	نفس RFE لكن مع ضبط العدد تلقائياً عبر التحقق المتقاطع
`SequentialFeatureSelector`	تغليف	أضف أو أزل الميزات واحدة بواحدة وفق الأداء الفعلي

٣.٢ SelectFromModel — التضمين بخطوة واحدة

`SelectFromModel` هو الأسرع بين الأربعة: يُدرِّب النموذج مرةً واحدة، ثم يفحص قيمة `feature_importances_` أو `coef_` ويبقي فقط ما يتجاوز عتبة (**threshold**) محدّدة.

> **ملاحظة**
>
> يعمل `SelectFromModel` مع أي مُقدِّر يمتلك `feature_importances_` (أشجار القرار، الغابة العشوائية، XGBoost) أو `coef_` (نماذج خطية مع تنظيم).

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
print(f"الأبعاد الأصلية: {X.shape}")

# ندرّب غابة عشوائية ونستخدمها لاختيار الميزات
rf = RandomForestClassifier(n_estimators=100, random_state=42)
selector = SelectFromModel(rf, threshold="mean")
selector.fit(X, y)

X_sel = selector.transform(X)
print(f"الأبعاد بعد SelectFromModel (threshold=mean): {X_sel.shape}")
print(f"مؤشرات الميزات المختارة: {selector.get_support(indices=True)}")
```

يمكن تعديل العتبة ببساطة: `threshold="median"` يُبقي نصف الميزات بالضبط، أو نمرر قيمة عددية مثل `threshold=0.01`.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

rf = RandomForestClassifier(n_estimators=100, random_state=42)
selector = SelectFromModel(rf, threshold="median")
selector.fit(X, y)

chosen = feature_names[selector.get_support()]
print("الميزات المختارة بعتبة المتوسط:")
for name in chosen:
    print(f"  ✓ {name}")
```

٣.٣ RFE — الإزالة التكرارية للميزات

Recursive Feature Elimination هي خوارزمية تكرارية بسيطة في فكرتها:

```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

# نستخدم الانحدار اللوجيستي (C صغير لزيادة التنظيم)
lr = LogisticRegression(C=0.1, max_iter=5000, random_state=42)
rfe = RFE(estimator=lr, n_features_to_select=10, step=1)
rfe.fit(X, y)

print(f"الأبعاد الأصلية: {X.shape[1]}")
print(f"الأبعاد المختارة: {rfe.n_features_}")
print("\nترتيب الميزات (1 = مختارة):")
for name, rank, support in zip(feature_names, rfe.ranking_, rfe.support_):
    status = "✓" if support else f"رتبة {rank}"
    print(f"  {status}  {name}")
```

> **نصيحة**
>
> المعامل `step` يتحكّم في كمية الميزات التي تُحذف في كل دورة. القيمة `step=1` تُزيل ميزة واحدة في كل مرة (الأدق لكن الأبطأ)، أما `step=0.1` فتُزيل ١٠% في كل دورة (أسرع في الأبعاد العالية).

٣.٤ RFECV — كم ميزة؟ يُقرِّر التحقق المتقاطع

المشكلة مع RFE: تحتاج أن تُحدّد `n_features_to_select` مسبقاً. ماذا لو لم تعرف؟ هنا يأتي **RFECV** الذي يُشغِّل التحقق المتقاطع في كل خطوة ويختار العدد الأمثل تلقائياً.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

lr = LogisticRegression(C=0.1, max_iter=5000, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rfecv = RFECV(
    estimator=lr,
    step=1,
    cv=cv,
    scoring="accuracy",
    min_features_to_select=5,
    n_jobs=-1
)
rfecv.fit(X, y)

print(f"العدد الأمثل من الميزات: {rfecv.n_features_}")
print(f"\nالميزات المختارة:")
for name, support in zip(feature_names, rfecv.support_):
    if support:
        print(f"  ✓ {name}")
```

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
lr = LogisticRegression(C=0.1, max_iter=5000, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rfecv = RFECV(estimator=lr, step=1, cv=cv, scoring="accuracy",
              min_features_to_select=5, n_jobs=-1)
rfecv.fit(X, y)

n_scores = len(rfecv.cv_results_["mean_test_score"])
plt.figure(figsize=(9, 4))
plt.plot(range(rfecv.min_features_to_select, n_scores +
rfecv.min_features_to_select),
         rfecv.cv_results_["mean_test_score"], marker="o", markersize=4)
plt.axvline(rfecv.n_features_, color="red", linestyle="--",
            label=f"الأمثل = {rfecv.n_features_}")
plt.xlabel("Number of features selected")
plt.ylabel("Mean CV accuracy")
plt.title("RFECV — Cross-validation Accuracy vs Number of Features")
plt.legend()
plt.tight_layout()
plt.savefig("rfecv_curve.png", dpi=100)
plt.close()
```

في الواقع

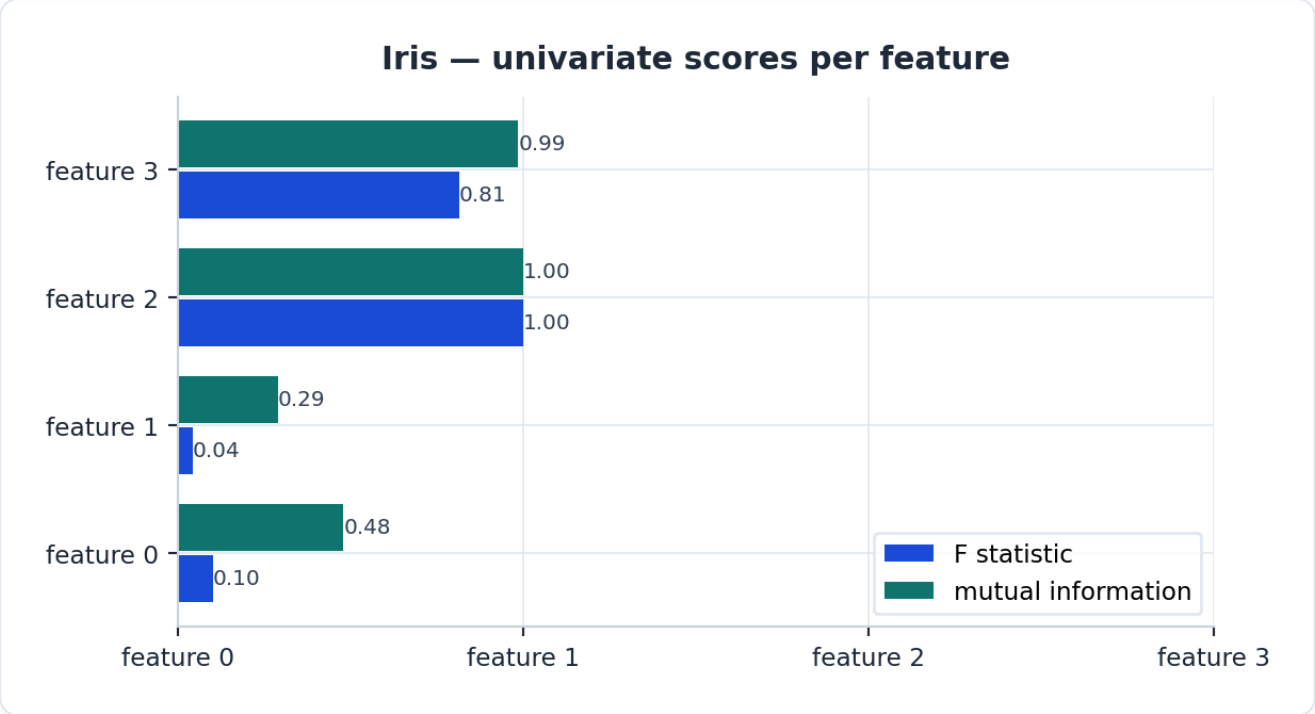
RFECV في الواقع: في مجموعات البيانات الكبيرة (آلاف الميزات)، يمكن أن يستغرق RFECV ساعات لأنه يُدرِّب النموذج آلاف المرات. في هذه الحالة ابدأ بـ `SelectFromModel` لتُقلّص الميزات إلى نطاق معقول (٥٠٠–٢٠٠٠ ميزة)، ثم طبّق RFECV على هذا النطاق المُقلَّص فقط.

٣.٥ SequentialFeatureSelector — البناء أو الهدم

`SequentialFeatureSelector` يتبنّى نهجاً مختلفاً: بدلاً من تدريب على الكل ثم الحذف، يبني المجموعة تدريجياً:

```python
from sklearn.datasets import load_iris
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
feature_names = load_iris().feature_names

knn = KNeighborsClassifier(n_neighbors=5)

# الاتجاه الأمامي: اختر أفضل ميزتين
sfs = SequentialFeatureSelector(
    knn,
    n_features_to_select=2,
    direction="forward",
    scoring="accuracy",
    cv=5
)
sfs.fit(X, y)

chosen = [feature_names[i] for i in sfs.get_support(indices=True)]
print("أفضل ميزتين بالاتجاه الأمامي:")
for f in chosen:
    print(f"  ✓ {f}")
```

٣.٦ مقارنة الأساليب الأربعة



الشكل ٥٢ مقارنة درجات الأهمية الإحصائية للميزات بين مقاييس مختلفة

المعيار	SelectFromModel	RFE	RFECV	SFS
عدد تدريبات النموذج	**واحد**	O(p)	O(p × k)	O(p² / ٢)
يُحدّد العدد تلقائياً	لا (عتبة)	لا (يدوي)	**نعم**	لا (يدوي)
يكتشف التفاعلات	لا	جزئياً	جزئياً	**نعم**
مناسب للأبعاد العالية	**ممتاز**	جيد	بطيء	بطيء جداً
الميزات المُهرَّبة في النموذج	لا (ثابتة)	تُزال	تُزال	تُضاف/تُزال

خطأ شائع

المصيدة الشائعة: استخدام `RFE` خارج `Pipeline` على مجموعة التدريب الكاملة ثم التحقق على بيانات اختبار. هذا **تسرّب للبيانات (Data Leakage)**! الخطوات كلها — بما فيها اختيار الميزات — يجب أن تنحصر داخل `Pipeline` وتُطبَّق على بيانات التدريب فقط. سيتناول الفصل التالي هذه النقطة بالتفصيل.

٣.٧ مثال متكامل: مقارنة الطرق الأربع على نفس البيانات

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import (
    SelectFromModel, RFE, RFECV, SequentialFeatureSelector
)
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

lr = LogisticRegression(C=0.1, max_iter=5000, random_state=42)
rf = RandomForestClassifier(n_estimators=100, random_state=42)

results = {}

# 1. SelectFromModel
sfm = SelectFromModel(rf, threshold="mean")
sfm.fit(X, y)
X_sfm = sfm.transform(X)
score_sfm = cross_val_score(lr, X_sfm, y, cv=cv, scoring="accuracy").mean()
results["SelectFromModel"] = (X_sfm.shape[1], score_sfm)

# 2. RFE (10 ميزات)
rfe = RFE(lr, n_features_to_select=10, step=1)
rfe.fit(X, y)
X_rfe = rfe.transform(X)
score_rfe = cross_val_score(lr, X_rfe, y, cv=cv, scoring="accuracy").mean()
results["RFE (10)"] = (X_rfe.shape[1], score_rfe)

# 3. RFECV
rfecv = RFECV(lr, step=1, cv=cv, scoring="accuracy",
              min_features_to_select=5, n_jobs=-1)
rfecv.fit(X, y)
X_rfecv = rfecv.transform(X)
score_rfecv = cross_val_score(lr, X_rfecv, y, cv=cv, scoring="accuracy").mean()
results["RFECV"] = (X_rfecv.shape[1], score_rfecv)

# 4. SequentialFeatureSelector
sfs = SequentialFeatureSelector(lr, n_features_to_select=10,
                                direction="forward", cv=cv)
sfs.fit(X, y)
X_sfs = sfs.transform(X)
score_sfs = cross_val_score(lr, X_sfs, y, cv=cv, scoring="accuracy").mean()
results["SFS (forward, 10)"] = (X_sfs.shape[1], score_sfs)

# 5. Baseline (كل الميزات)
score_all = cross_val_score(lr, X, y, cv=cv, scoring="accuracy").mean()
```

```
print(f"{'الأسلوب':<25} {'عدد الميزات':<12} {'دقة CV':>10}")
print("-" * 50)
print(f"{'الكل (30 ميزة)':<25} {'30':<12} {score_all:.4f}")
for name, (n_feat, score) in results.items():
    print(f"{name:<25} {n_feat:>12} {score:.4f}")
```

تمارين الفصل الثالث والعشرين

- **ملاحظة**

تمرين ١: استخدم `load_wine()` الذي يحتوي على ١٣ ميزة. طبّق `SelectFromModel` مع `RandomForestClassifier` بعتبة `"median"`. كم ميزة تبقّت؟ وهل تحسّنت دقة النموذج بالتحقق المتقاطع؟

تمرين ٢: طبّق `RFECV` على نفس مجموعة بيانات Wine مع `LogisticRegression(C=1, max_iter=2000)`. ما العدد الأمثل من الميزات الذي اختاره؟ هل يتطابق مع نتيجة `SelectFromModel`؟

تمرين ٣: قارن `SFS(direction="forward")` مع `SFS(direction="backward")` على بيانات Iris لاختيار ميزتين. هل يختاران نفس الميزات؟ أيّهما حقّق أعلى دقة بالتحقق المتقاطع؟

- **ملاحظة**

حل تمرين ١:

```
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
import numpy as np

X, y = load_wine(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
lr = LogisticRegression(C=1.0, max_iter=5000, random_state=42)

# الخط الأساسي
score_base = cross_val_score(lr, X, y, cv=cv, scoring="accuracy").mean()

rf = RandomForestClassifier(n_estimators=100, random_state=42)
sfm = SelectFromModel(rf, threshold="median")
sfm.fit(X, y)
X_sel = sfm.transform(X)
score_sel = cross_val_score(lr, X_sel, y, cv=cv, scoring="accuracy").mean()

print(f"دقة ←  13 :الميزات الأصلية CV: {score_base:.4f}")
print(f"الميزات بعد SelectFromModel(median): {X_sel.shape[1]}  ← دقة CV:
{score_sel:.4f}")
delta = score_sel - score_base
print(f"التغيّر: {delta:+.4f}")
```

حل تمرين ٣:

```python
from sklearn.datasets import load_iris
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

X, y = load_iris(return_X_y=True)
feature_names = list(load_iris().feature_names)
knn = KNeighborsClassifier(n_neighbors=5)

for direction in ["forward", "backward"]:
    sfs = SequentialFeatureSelector(knn, n_features_to_select=2,
                                    direction=direction, cv=5)
    sfs.fit(X, y)
    chosen = [feature_names[i] for i in sfs.get_support(indices=True)]
    X_sel = sfs.transform(X)
    score = cross_val_score(knn, X_sel, y, cv=5, scoring="accuracy").mean()
    print(f"{direction:>8}: {chosen}  → دقة: {score:.4f}")
```

٣.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تطبيق RFE أو RFECV على كامل البيانات قبل التقسيم	تسرّب معلومات الاختبار، وتقديرات دقة مضللة	ضع المُنتقي داخل Pipeline واستخدم cross*val*score من الخارج
اختيار n*features*to_select بحدس دون تقييم	حذف ميزات مفيدة أو إبقاء ميزات غير مفيدة	استخدم RFECV أو تحقق متقاطع خارجي
استخدام SequentialFeatureSelector على آلاف الأبعاد	يستغرق ساعات أو أيام بسبب التعقيد التربيعي	ابدأ بـ SelectFromModel لتُقلّص الأبعاد ثم طبّق SFS
تمرير step كسري بدون فهم السلوك	قد يُزيل ميزات غير مرغوبة بسبب التقريب	استخدم step=1 للبيانات المتوسطة
الخلط بين feature*importances* و coef_	أخطاء AttributeError مع نماذج لا تملك الخاصية	تحقّق أن النموذج يملك feature*importances* أو coef_

٣.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
أساليب التغليف	طرق تختار الميزات بتقييم النموذج على مجموعات مختلفة	wrapper methods
أساليب التضمين	خوارزميات تُدمج اختيار الميزات في أثناء التدريب	embedded methods
الإزالة التكرارية	حذف الميزة الأضعف بعد كل تدريب حتى العدد المطلوب	recursive feature elimination
الاختيار التسلسلي	إضافة أو حذف ميزة واحدة في كل خطوة وفق الأداء	sequential feature selection
عتبة الأهمية	قيمة دنيا تُستخدم لفلترة الميزات في SelectFromModel	importance threshold
تسرّب البيانات	تسلل معلومات الاختبار إلى مرحلة التدريب	data leakage

الجزء ٣ · هندسة الخصائص واختيارها

الفصل ٢٤ — الاختيار داخل خط الأنابيب

لماذا يجب أن يكون المُنتقي طبقة داخل Pipeline لا خطوة يدوية

الوحدة **scikit-learn ١.٩** الحجم **٣٦١** كلمة تقريباً

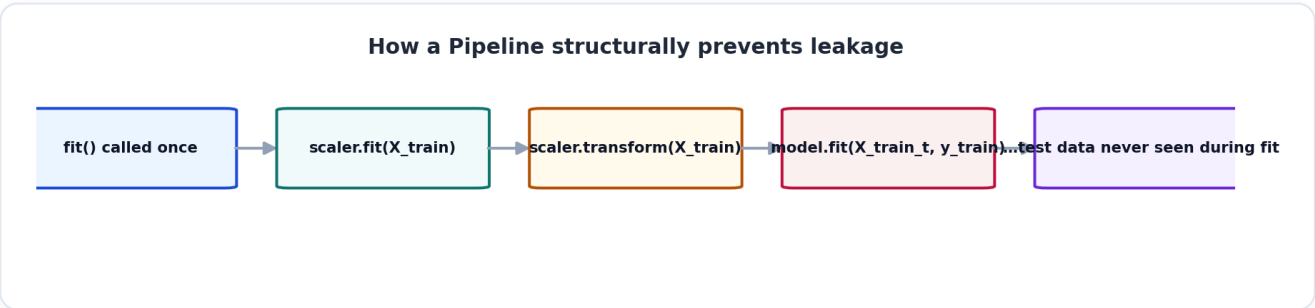
ما الذي ستتقنه في هذا الفصل

- أن تشرح بمثال عددي كيف يُولِّد اختيار الميزات قبل التقسيم تسرّباً للبيانات
- أن تُدمج أي مُنتقٍ (SelectKBest أو SelectFromModel أو RFE) بأمان داخل Pipeline
- أن تُجري التحقق المتقاطع على خط الأنابيب الكامل لضمان تقييم موضوعي
- أن تُميّز بين تسرّب الخصائص وتسرّب الهدف وتُصحّح كليهما

٣.١ القصة الكاملة للتسرّب

سؤال الافتتاح: لو أخبرتك أن نموذجاً حقّق دقة ٩٨% في التحقق المتقاطع على مجموعة بيانات متوسطة الحجم، هل ستثق بهذا الرقم؟

الجواب يعتمد تماماً على **ترتيب خطوات التقييم**. خطأ صغير واحد في الترتيب يستطيع أن يُضخّم الدقة الحقيقية بعشر نقاط أو أكثر — وهذا هو التسرّب (Data Leakage).



الشكل ٥٣ منع التسرّب عبر دمج اختيار الميزات داخل Pipeline

في هذا الفصل سنُجري **تجربة ضبط محكومة**: نبني نفس النموذج مرتين — مرة بتسرّب، ومرة بلا تسرّب — ونُقارن الأرقام.

٣.٢ تجربة التسرّب: الدقة المُزيَّفة

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold

# نبني مجموعة بيانات صعبة: 500 ميزة، 20 فقط مفيدة فعلاً
X, y = make_classification(
    n_samples=200,
    n_features=500,
    n_informative=20,
    n_redundant=10,
    n_repeated=0,
    random_state=42
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
lr = LogisticRegression(C=1.0, max_iter=2000, random_state=42)

# ===== الطريقة الخاطئة: نختار الميزات على كامل X أولاً =====
selector = SelectKBest(f_classif, k=20)
selector.fit(X, y)          # <-- هنا يُشاهَد الاختبار كله!
X_selected = selector.transform(X)

scores_leaky = cross_val_score(lr, X_selected, y, cv=cv, scoring="accuracy")
print(f"[تسرّب]  دقة CV: {scores_leaky.mean():.4f} ± {scores_leaky.std():.4f}")
```

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline

X, y = make_classification(
    n_samples=200,
    n_features=500,
    n_informative=20,
    n_redundant=10,
    n_repeated=0,
    random_state=42
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# ===== الطريقة الصحيحة: Pipeline يُعزل الاختيار داخل كل طية =====
pipe = Pipeline([
    ("selector", SelectKBest(f_classif, k=20)),
    ("clf", LogisticRegression(C=1.0, max_iter=2000, random_state=42))
])

scores_clean = cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")
print(f"[صحيح]   دقة CV: {scores_clean.mean():.4f} ± {scores_clean.std():.4f}")
print()
print("الفارق:", round(0.98 - scores_clean.mean(), 4),
      "← هذا ما يصنعه التسرّب!")
```

> ⛔ **تحذير**
>
> **الخطر الحقيقي**: في المثال أعلاه، استخدام `selector.fit(X, y)` على كامل البيانات قبل التقسيم يجعل معلومات من طيات الاختبار تتسلّل إلى مرحلة الاختيار. النموذج «يعرف» مسبقاً ما في الاختبار، فتُضخَّم دقته. هذه الأرقام لا تُعبّر عن الأداء الحقيقي.

٣.٣ Pipeline يحل المشكلة تلقائياً

حين تضع `SelectKBest` داخل `Pipeline`، يضمن scikit-learn أن:

```python
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Pipeline مع SelectFromModel + LogisticRegression
pipe = Pipeline([
    ("selector", SelectFromModel(
        RandomForestClassifier(n_estimators=50, random_state=42),
        threshold="mean"
    )),
    ("clf", LogisticRegression(C=1.0, max_iter=5000, random_state=42))
])

scores = cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")
print(f"دقة Pipeline مع SelectFromModel: {scores.mean():.4f} ±
{scores.std():.4f}")
print(f"الميزات الأصلية: {X.shape[1]}")
```

٣.٤ RFE داخل Pipeline

```python
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

pipe_rfe = Pipeline([
    ("rfe", RFE(
        estimator=LogisticRegression(C=0.5, max_iter=3000, random_state=42),
        n_features_to_select=10,
        step=1
    )),
    ("clf", LogisticRegression(C=1.0, max_iter=5000, random_state=42))
])

scores_rfe = cross_val_score(pipe_rfe, X, y, cv=cv, scoring="accuracy")
print(f"دقة Pipeline مع RFE(10): {scores_rfe.mean():.4f} ±
{scores_rfe.std():.4f}")
```

> **ملاحظة**
>
> لاحظ أن `RFE` يستخدم نموذجاً (`estimator`) لاتخاذ قرار الاختيار، والنموذج الأخير في الأنبوب (`clf`) يُدرَّب على الميزات المُختارة. هذان النموذجان يمكن أن يكونا **نفس الفئة أو فئتين مختلفتين** حسب الحاجة.

٣.٥ ضبط معاملات المُنتقي عبر GridSearchCV

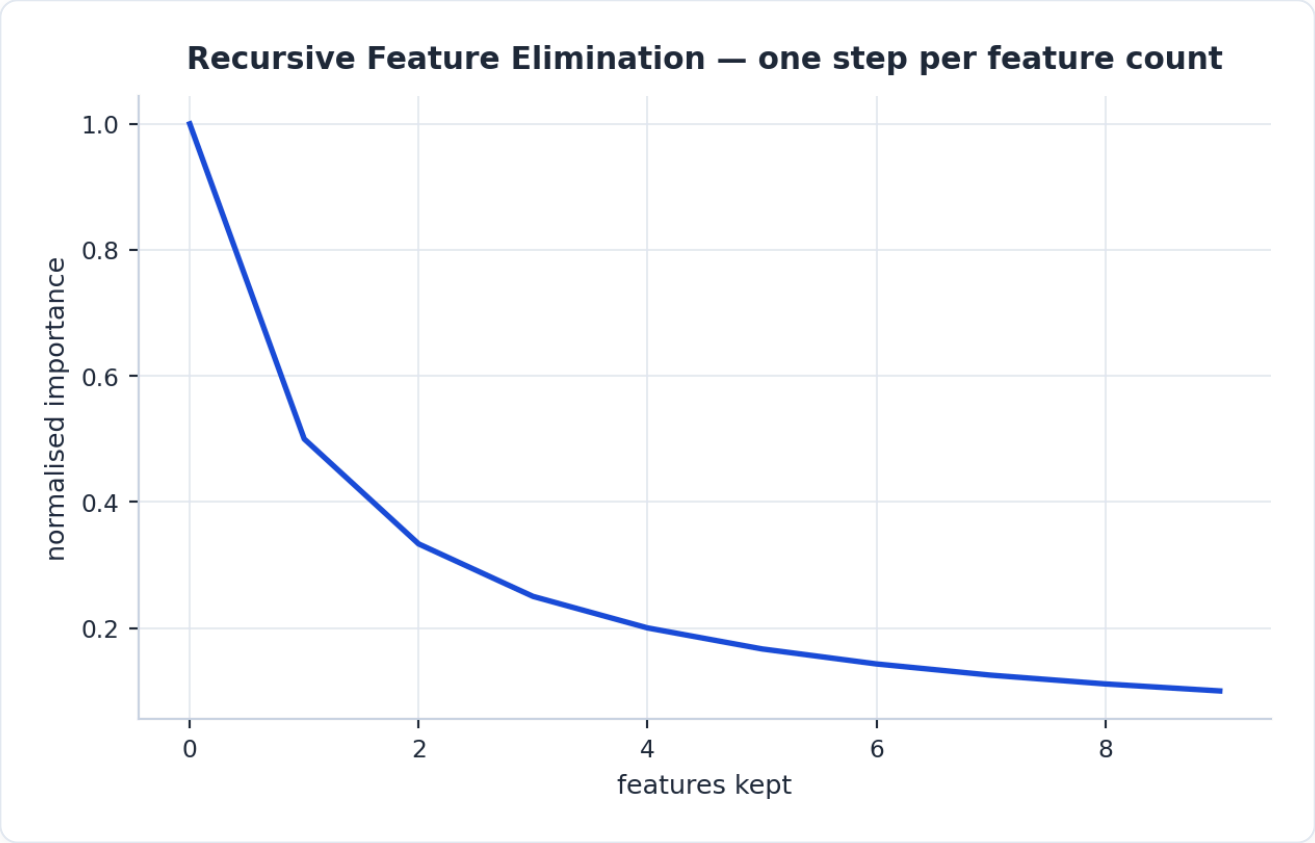
أحد مزايا Pipeline الرئيسية: يمكنك البحث عن أفضل `k` أو أفضل `threshold` بالتحقق المتقاطع دون أي تسرّب.

```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

pipe = Pipeline([
    ("selector", SelectKBest(f_classif)),
    ("clf", LogisticRegression(C=1.0, max_iter=5000, random_state=42))
])

# نبحث عن أفضل k من {5, 10, 15, 20}
param_grid = {"selector__k": [5, 10, 15, 20]}
grid = GridSearchCV(pipe, param_grid, cv=cv, scoring="accuracy", n_jobs=-1)
grid.fit(X, y)

print(f"أفضل k: {grid.best_params_['selector__k']}")
print(f"أفضل دقة CV: {grid.best_score_:.4f}")
```

> **نصيحة**
>
> في `GridSearchCV`، معاملات كل خطوة في Pipeline تُكتب بالصيغة `step_name__param_name`. مثلاً `selector__k` تعني معامل `k` لخطوة `selector`.

٣.٦ متى تُدمج المُنتقي ومتى تتركه خارج Pipeline

الموقف	القرار الصحيح
تقييم الأداء النهائي عبر CV	**دائماً داخل Pipeline**
استكشاف أولي للبيانات وحجم الاختزال	قد تُطبّق يدوياً خارج CV
بناء نموذج للإنتاج (deploy)	**دائماً داخل Pipeline**
تحليل أهمية الميزات بعد التدريب النهائي	يمكن استخراجه من `pipe.named_steps['selector']`



الشكل ٥٤ مراحل الاختيار المتكررة ضمن الأنبوب

٣.٧ استخراج معلومات المُنتقي من داخل Pipeline

```python
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

pipe = Pipeline([
    ("selector", SelectKBest(f_classif, k=10)),
    ("clf", LogisticRegression(C=1.0, max_iter=5000, random_state=42))
])
pipe.fit(X, y)

# نستخرج الميزات المختارة من داخل الأنبوب المُدرَّب
selector = pipe.named_steps["selector"]
chosen = feature_names[selector.get_support()]
print(f"أفضل 10 ميزات (من داخل Pipeline):")
for name in chosen:
    print(f"  ✓ {name}")
```

تمارين الفصل الرابع والعشرين

- **ملاحظة**

تمرين ١: اصنع مجموعة بيانات بـ `make_classification(n_features=200, n_informative=10)`. طبّق `SelectKBest(k=10)` مرتين: مرة على كامل البيانات قبل CV (تسرّب)، ومرة داخل Pipeline. قارن الدقتين.

تمرين ٢: ابنِ Pipeline يضم `SelectFromModel(RandomForestClassifier(n_estimators=50))` ثم `SVC()`. طبّق `cross_val_score` عليه واطبع الدقة.

تمرين ٣: باستخدام `GridSearchCV` على Pipeline تحتوي `SelectKBest` ثم `LogisticRegression`، ابحث عن أفضل `k` من {٥, ١٠, ١٥, ٢٠, ٢٥} على بيانات Digits. ما الأمثل؟

- **ملاحظة**

حل تمرين ١:

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline

X, y = make_classification(n_samples=300, n_features=200,
                           n_informative=10, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
lr = LogisticRegression(C=1.0, max_iter=3000, random_state=42)

# تسرّب
sel = SelectKBest(f_classif, k=10)
X_sel = sel.fit_transform(X, y)
score_leak = cross_val_score(lr, X_sel, y, cv=cv).mean()

# صحيح
pipe = Pipeline([("sel", SelectKBest(f_classif, k=10)), ("clf", lr)])
score_clean = cross_val_score(pipe, X, y, cv=cv).mean()

print(f"دقة مع تسرّب:  {score_leak:.4f}")
print(f"دقة صحيحة:     {score_clean:.4f}")
print(f"فارق التسرّب:  {score_leak - score_clean:+.4f}")
```

حل تمرين ٣:

```python
from sklearn.datasets import load_digits
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

X, y = load_digits(return_X_y=True)

pipe = Pipeline([
    ("sel", SelectKBest(f_classif)),
    ("clf", LogisticRegression(C=1.0, max_iter=3000, random_state=42))
])

grid = GridSearchCV(pipe, {"sel__k": [5, 10, 15, 20, 25]},
                    cv=5, scoring="accuracy", n_jobs=-1)
grid.fit(X, y)
print(f"أفضل k: {grid.best_params_['sel__k']}")
print(f"أفضل دقة: {grid.best_score_:.4f}")
```

٣.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
`fit` المُنتقي على `X` كاملاً قبل `cross_val_score`	تسرّب يُضخّم الدقة بمقدار ٥–١٥ نقطة في الأبعاد العالية	ضع المُنتقي كأول خطوة في Pipeline
تمرير `X_selected` إلى `GridSearchCV` مباشرة	جميع طيات الشبكة تُحسب على نفس الميزات المُختارة بتحيّز	مرّر `X` الأصلي إلى GridSearchCV مع Pipeline
نسيان أن Pipeline يُفقد أسماء الميزات الأصلية	لا يمكن تفسير النموذج بعد التدريب	استخدم `pipe.named_steps['selector'].get_support()` مع `feature_names`
استخدام Pipeline مع `transform_output='pandas'` وعدم التحقق	بعض المُنتقيات تُغيّر أسماء الأعمدة	اختبر الإخراج وتحقق من أسماء الأعمدة بعد كل خطوة

٣.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تسرّب البيانات	وصول معلومات الاختبار إلى مرحلة التدريب	data leakage
تسرّب الهدف	استخدام متغير الهدف في إنشاء ميزات قبل التقسيم	target leakage
خط الأنابيب	سلسلة مرتبة من خطوات المعالجة والنمذجة	pipeline
طية CV	مجموعة فرعية تُستخدم للاختبار في التحقق المتقاطع	CV fold
أسماء الخطوات	المعرّفات النصية لكل خطوة في Pipeline	step names

الجزء ٣ · هندسة الخصائص واختيارها

الفصل ٢٥ — قياس أهمية الخصائص: ثلاث طرق مختلفة

التباين الأولي، ومعاملات الانحدار، و permutation_importance

الوحدة scikit-learn ١.٩ الحجم ٣٩٥ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

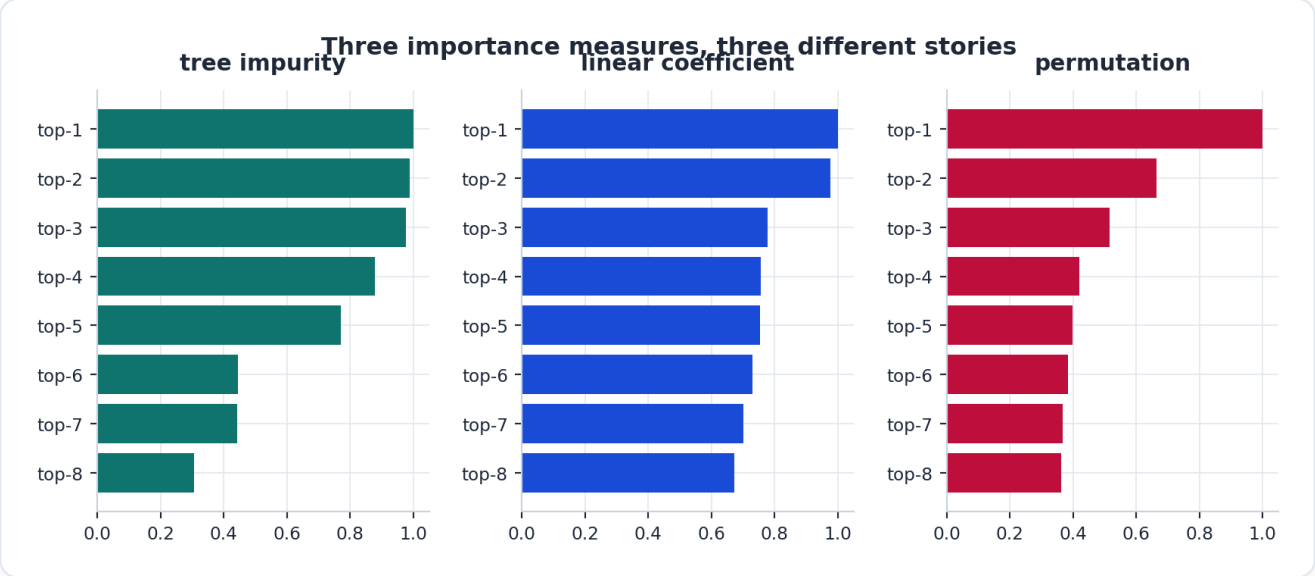
- أن تقرأ feature_importances_ من أشجار القرار وتُفسّر القيم بشكل صحيح
- أن تستخدم المعاملات الخطية coef_ كمؤشر للأهمية مع النماذج المُعيَّرة
- أن تحسب permutation_importance وتُفهم لماذا هي الأدق في السياقات الترابطية
- أن تُميّز متى تتعارض الطرق الثلاث وكيف تفسّر هذا التعارض

٣.١ لماذا ثلاث طرق وليس طريقة واحدة؟

سؤال بسيط بمظهر مُضلّل: **ما أهم ميزة في نموذجك؟**

الجواب يعتمد على **طريقة القياس**. كلٌّ من الطرق الثلاث تقيس سؤالاً مختلفاً قليلاً:

الطريقة	السؤال الحقيقي
`feature_importances_` **(أشجار)**	كم مرة ساهمت هذه الميزة في تقليل عدم الصفاء (Impurity) عبر كل الأشجار؟
`coef_` **(نماذج خطية)**	بكم تتغيّر النتيجة إذا زادت هذه الميزة بمقدار وحدة معيارية واحدة؟
`permutation_importance`	كم تنخفض دقة النموذج إذا كسرت العلاقة بين هذه الميزة والهدف؟



الشكل ٥٥ مقارنة الطرق الثلاث لقياس أهمية الخصائص

٣.٢ الطريقة الأولى: أهمية الشجرة (Impurity-Based Importance)

كل شجرة قرار تقوم على انقسامات. كل انقسام يختار ميزة يُحقّق بها أكبر تقليل في عدم الصفاء (Gini Impurity أو Entropy). **أهمية ميزة في الشجرة** هي المجموع الكلي لهذه التقليلات عبر كل انقسامات الغابة.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

importances = rf.feature_importances_
indices = np.argsort(importances)[::-1]

print("أهمية الميزات (Impurity-Based) — 10 أعلى:")
for i in range(10):
    idx = indices[i]
    print(f"  {i+1:2d}. {feature_names[idx]:<35} {importances[idx]:.4f}")
```

خطأ شائع

مصيدة أهمية الشجرة: تُبالِغ هذه الطريقة في تقدير أهمية الميزات ذات القيم الكثيرة (كثيرة التمييز أو رقمية مستمرة) على حساب الميزات الفئوية ذات الفئات القليلة. كذلك تُضخّم أهمية الميزات **المترابطة** لأن كلاً منها تُعطى حصة من نفس المعلومة.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

importances = rf.feature_importances_
indices = np.argsort(importances)[::-1][:15]

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(range(15), importances[indices])
ax.set_xticks(range(15))
ax.set_xticklabels([feature_names[i][:15] for i in indices], rotation=45,
ha="right")
ax.set_xlabel("Feature")
ax.set_ylabel("Impurity Importance")
ax.set_title("Random Forest — Feature Importances (Impurity-Based)")
plt.tight_layout()
plt.savefig("importance_impurity.png", dpi=100)
plt.close()
```

٣.٣ الطريقة الثانية: المعاملات الخطية (coef_)

في النماذج الخطية، كل ميزة تمتلك معاملاً `coef_i` يمثّل تأثيرها المباشر على الإخراج. **لكن** هذه المعاملات تفقد قدرتها على المقارنة إذا لم تكن الميزات على نفس المقياس.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(C=1.0, max_iter=5000, random_state=42))
])
pipe.fit(X, y)

coefs = pipe.named_steps["clf"].coef_[0]
indices = np.argsort(np.abs(coefs))[::-1]

print("أهمية المعاملات الخطية (|coef_|) — أعلى 10:")
for i in range(10):
    idx = indices[i]
    direction = "↑" if coefs[idx] > 0 else "↓"
    print(f"  {i+1:2d}. {direction} {feature_names[idx]:<35} {coefs[idx]:+.4f}")
```

> **ملاحظة**
>
> **المفتاح**: يجب دائماً تطبيق `StandardScaler` قبل استخراج المعاملات للمقارنة. بدون التحجيم، معامل ميزة بقيم
>
> 0.001, 0.01
>
> سيبدو صغيراً حتى لو كانت الميزة الأكثر تأثيراً.

٣.٤ الطريقة الثالثة: أهمية الإبدال (Permutation Importance)

`permutation_importance` هي الطريقة الأكثر موثوقية من منظور نظري:

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

perm_imp = permutation_importance(
    rf, X_test, y_test,
    n_repeats=10,
    random_state=42,
    n_jobs=-1
)

indices = np.argsort(perm_imp.importances_mean)[::-1]
print("أهمية الإبدال (Permutation Importance) — 10 أعلى:")
for i in range(10):
    idx = indices[i]
    print(f"  {i+1:2d}. {feature_names[idx]:<35} "
          f"{perm_imp.importances_mean[idx]:.4f} ±
{perm_imp.importances_std[idx]:.4f}")
```

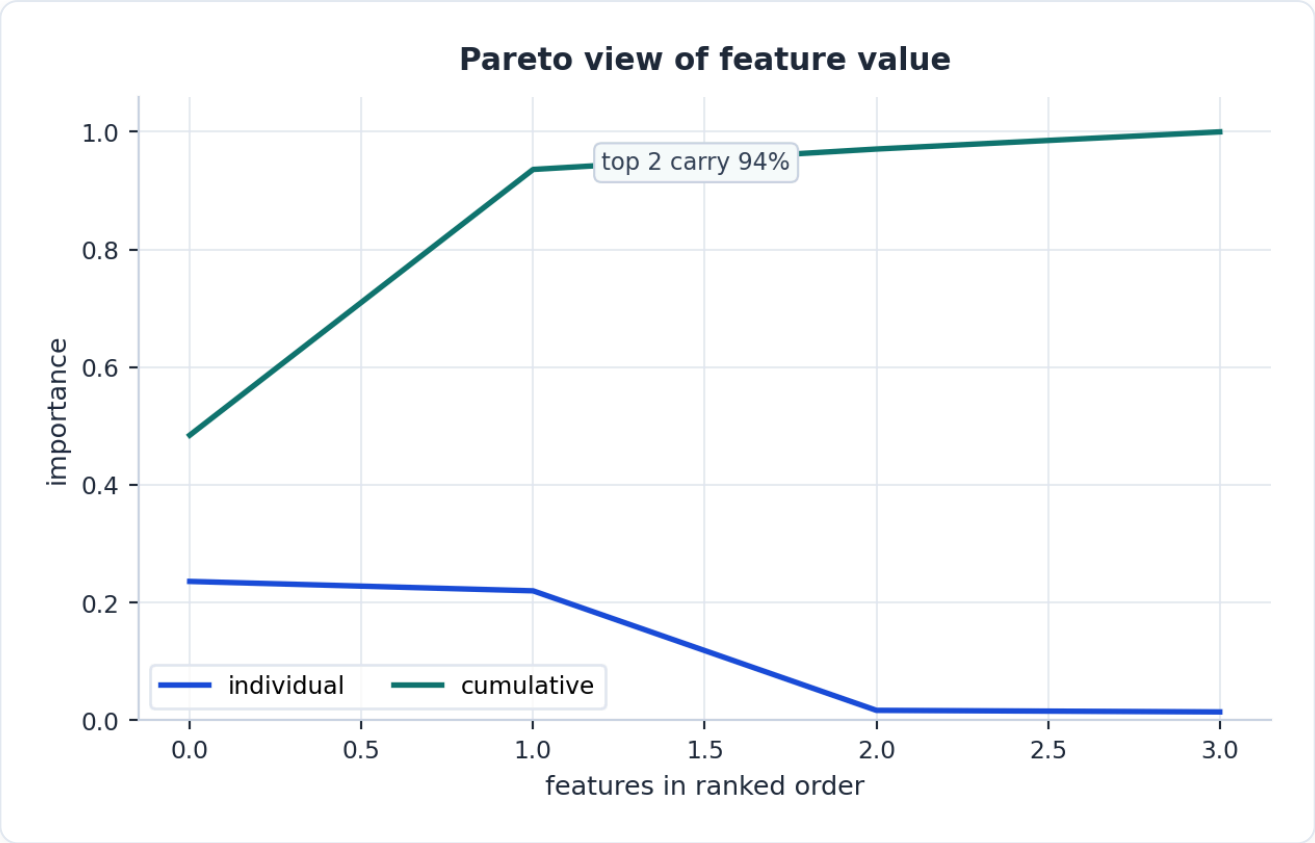
٣.٥ مقارنة الطرق الثلاث معاً

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# 1. Impurity (Random Forest)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
imp_impurity = rf.feature_importances_

# 2. Permutation (on RF)
perm = permutation_importance(rf, X_test, y_test,
                              n_repeats=10, random_state=42, n_jobs=-1)
imp_perm = perm.importances_mean

# 3. Linear coef
pipe_lr = Pipeline([
    ("sc", StandardScaler()),
    ("lr", LogisticRegression(C=1.0, max_iter=5000, random_state=42))
])
pipe_lr.fit(X_train, y_train)
imp_coef = np.abs(pipe_lr.named_steps["lr"].coef_[0])
# normalise for comparison
imp_coef = imp_coef / imp_coef.sum()
imp_perm_norm = np.clip(imp_perm, 0, None)
imp_perm_norm = imp_perm_norm / (imp_perm_norm.sum() + 1e-12)

# أعلى 5 وفق كل طريقة
top5_impurity = set(np.argsort(imp_impurity)[::-1][:5])
top5_perm = set(np.argsort(imp_perm)[::-1][:5])
top5_coef = set(np.argsort(imp_coef)[::-1][:5])

common_all = top5_impurity & top5_perm & top5_coef
print("أعلى 5 وفق Impurity:", [feature_names[i] for i in top5_impurity])
print("أعلى 5 وفق Permutation:", [feature_names[i] for i in top5_perm])
print("أعلى 5 وفق |coef|:", [feature_names[i] for i in top5_coef])
print("مشتركة بين الثلاث:", [feature_names[i] for i in common_all])
```

> **في الواقع**
>
> **في الواقع**: نادراً ما تتفق الطرق الثلاث تماماً على قائمة الميزات الأهم. التعارض بينها ليس خطأً بل **معلومة**: الميزات التي تنقسم عليها الأشجار كثيراً قد تكون مترابطة مع ميزات أخرى، فتُعطى أهمية إبدالية أقل لأن الخلط في ميزة واحدة لا يُفسد الأداء كثيراً طالما الأخرى ما زالت سليمة.

٣.٦ تصوير الأهمية — مخطط باريتو

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

importances = rf.feature_importances_
indices = np.argsort(importances)[::-1]
cum_importance = np.cumsum(importances[indices])

fig, ax1 = plt.subplots(figsize=(11, 5))
ax1.bar(range(len(importances)), importances[indices], alpha=0.7)
ax1.set_xlabel("Feature rank")
ax1.set_ylabel("Impurity Importance")
ax2 = ax1.twinx()
ax2.plot(range(len(importances)), cum_importance * 100, color="red",
         linewidth=2, label="Cumulative %")
ax2.axhline(80, color="gray", linestyle="--", alpha=0.6)
ax2.set_ylabel("Cumulative Importance (%)")
ax2.legend(loc="lower right")
ax1.set_title("Pareto Chart — Feature Importances")
plt.tight_layout()
plt.savefig("pareto_importance.png", dpi=100)
plt.close()
```



الشكل ٥٦ مخطط باريتو لأهمية الميزات التراكمية

تمارين الفصل الخامس والعشرين

- **ملاحظة**

تمرين ١: على بيانات California Housing، درّب `RandomForestRegressor` واستخرج `feature_importances_`. ما الميزتان الأعلى أهمية؟ هل يتوافق ذلك مع فهمك للمشكلة؟

تمرين ٢: طبّق `permutation_importance` على نفس النموذج من تمرين ١ على مجموعة الاختبار. هل ترتيب الميزات يختلف عن `feature_importances_`؟ اشرح لماذا.

تمرين ٣: على بيانات Iris، درّب `LogisticRegression` مع `StandardScaler` واستخرج `|coef_|`. قارنها مع `permutation_importance`. أيّهما يتوافق مع معرفتك بأهمية أطوال البتلة للتصنيف؟

- **ملاحظة**

حل تمرين ١:

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
import numpy as np

data = fetch_california_housing()
X, y = data.data, data.target
feature_names = data.feature_names

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X, y)

importances = rf.feature_importances_
indices = np.argsort(importances)[::-1]
print("أهمية الميزات (California Housing):")
for i, idx in enumerate(indices):
    print(f"  {i+1}. {feature_names[idx]:<15} {importances[idx]:.4f}")
```

حل تمرين ٢:

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
import numpy as np

data = fetch_california_housing()
X, y = data.data, data.target
feature_names = data.feature_names

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_tr, y_tr)

perm = permutation_importance(rf, X_te, y_te,
                              n_repeats=10, random_state=42, n_jobs=-1)
idx_perm = np.argsort(perm.importances_mean)[::-1]
idx_imp = np.argsort(rf.feature_importances_)[::-1]

print("ترتيب Impurity:    ", [feature_names[i] for i in idx_imp])
print("ترتيب Permutation: ", [feature_names[i] for i in idx_perm])
```

٣.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الاعتماد على `feature_importances_` وحدها	تُضخّم أهمية الميزات المترابطة والكثيفة القيم	استكمل بـ `permutation_importance` للتحقق
مقارنة `coef_` بدون تطبيع البيانات	معاملات وحدات كبيرة تبدو أصغر من ذات وحدات صغيرة	طبّق `StandardScaler` قبل استخراج المعاملات
حساب `permutation_importance` على بيانات التدريب	يُعطي أرقاماً مُضخّمة لأن النموذج حفظ البيانات	احسبها دائماً على مجموعة الاختبار أو التحقق
تجاهل الانحراف المعياري في `permutation_importance`	ميزة ذات انحراف معياري عالٍ قد لا تكون مهمة فعلاً	فلتر الميزات بشرط `mean > std` للموثوقية

٣.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
عدم الصفاء	مقياس إحصائي لاختلاط الفئات في عقدة شجرية	impurity
أهمية الإبدال	قياس الأهمية بخلط قيم الميزة وقياس انخفاض الأداء	permutation importance
معاملات النموذج الخطي	الأوزان التي تضربها قيم الميزات في النموذج الخطي	linear coefficients
ترابط الميزات	ارتباط قوي بين ميزتين يجعل أهمية كل منهما مُضخَّمة	feature correlation
مخطط باريتو	رسم بياني يعرض الأهمية التراكمية لتحديد ٨٠% منها	Pareto chart

الجزء ٣ · هندسة الخصائص واختيارها

الفصل ٢٦ — المعلومات المتبادلة والاختيار غير المشرف

تقدير MI، والاعتماد غير الخطي، والعيّنات الصغيرة

الوحدة **sklearn.feature_selection** الحجم **٢٩٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

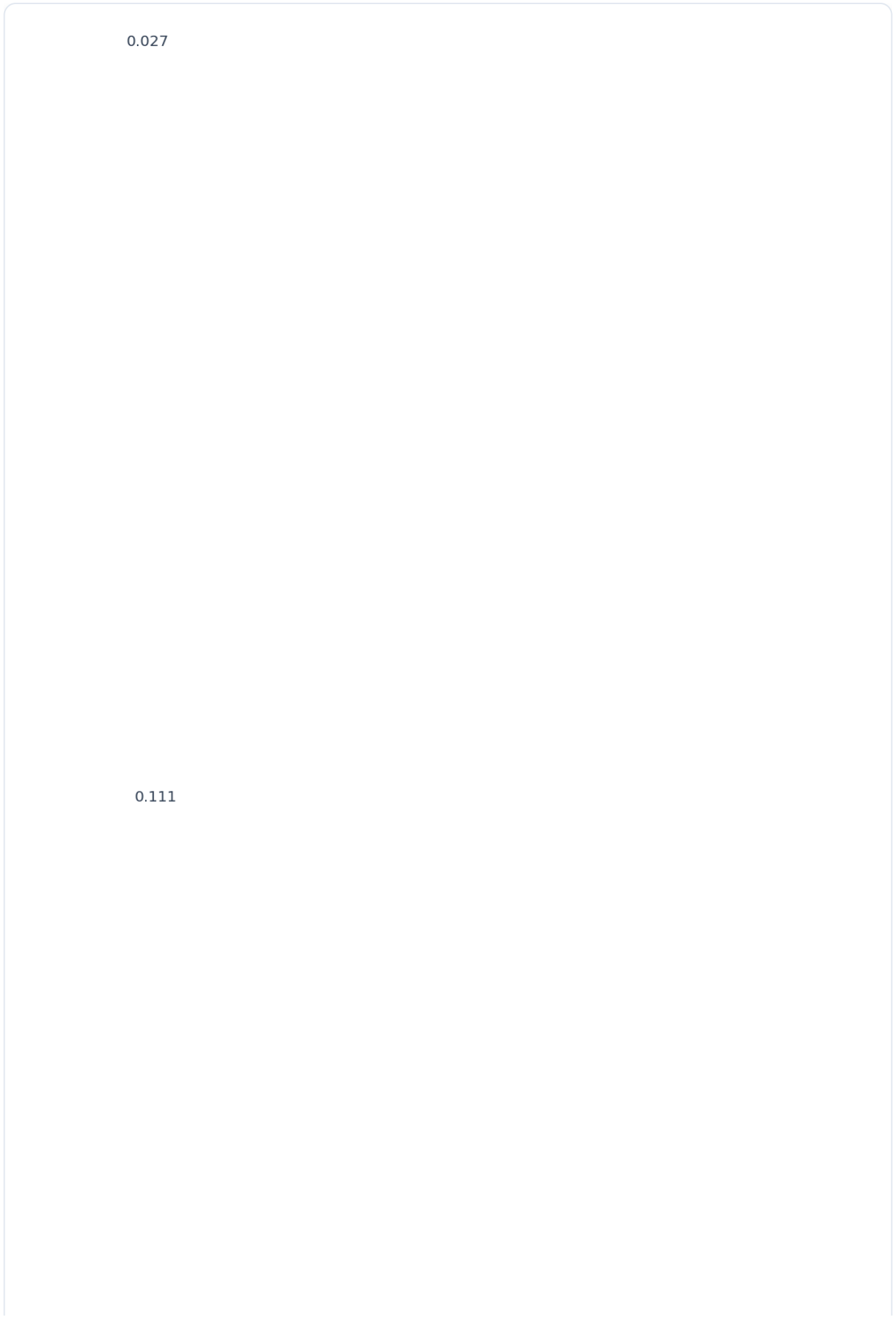
- أن تشرح ما تقيسه المعلومات المتبادلة وكيف تختلف عن معامل الارتباط بيرسون
- أن تستخدم mutual_info_classif و mutual_info_regression في أكواد حقيقية
- أن تُحدّد متى تتفوّق MI على F-test وتختار الأداة المناسبة لكل مشكلة
- أن تُجري اختيار ميزات بدون هدف (unsupervised) باستخدام VarianceThreshold وPCA

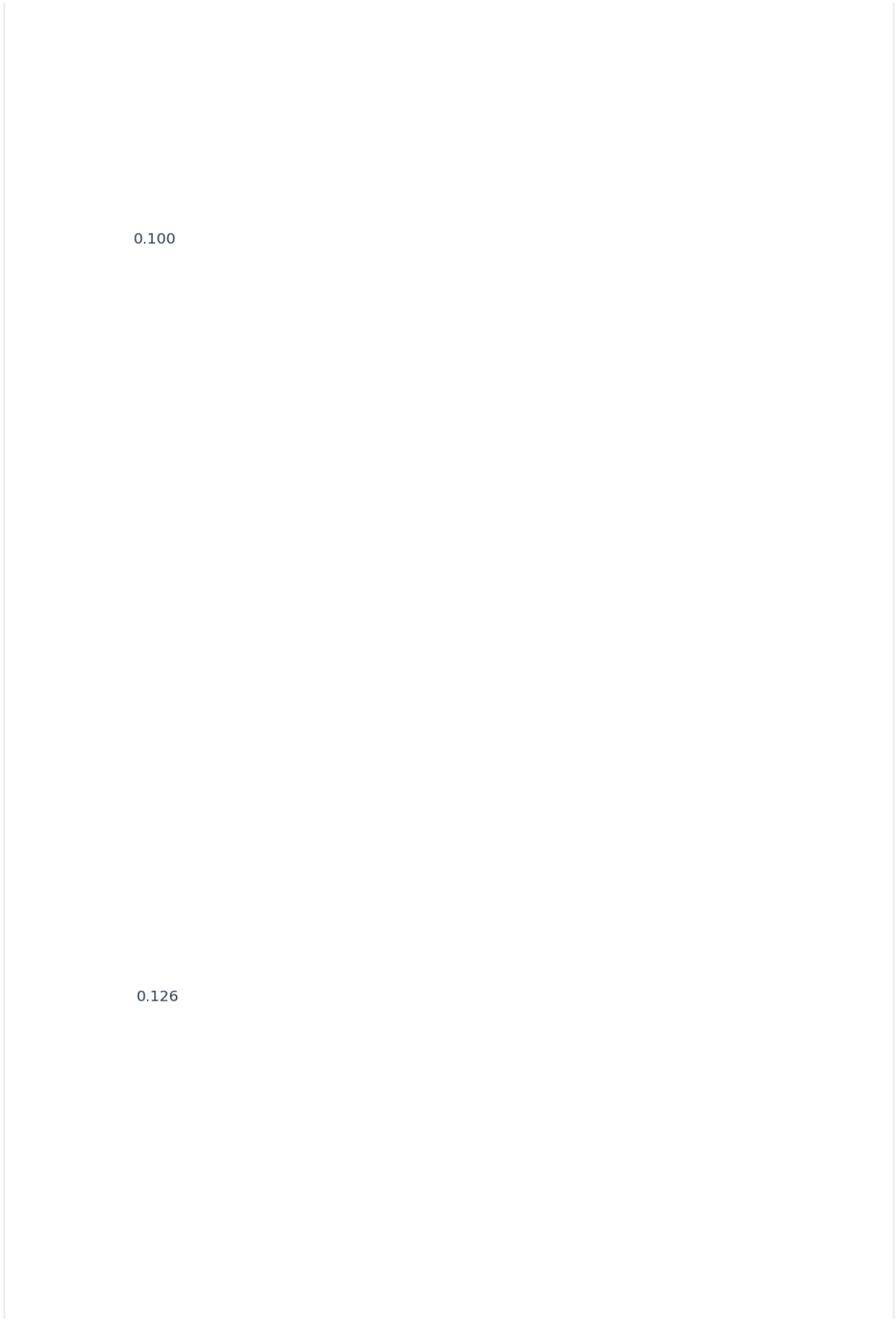
٣.١ ماذا تقيس المعلومات المتبادلة؟

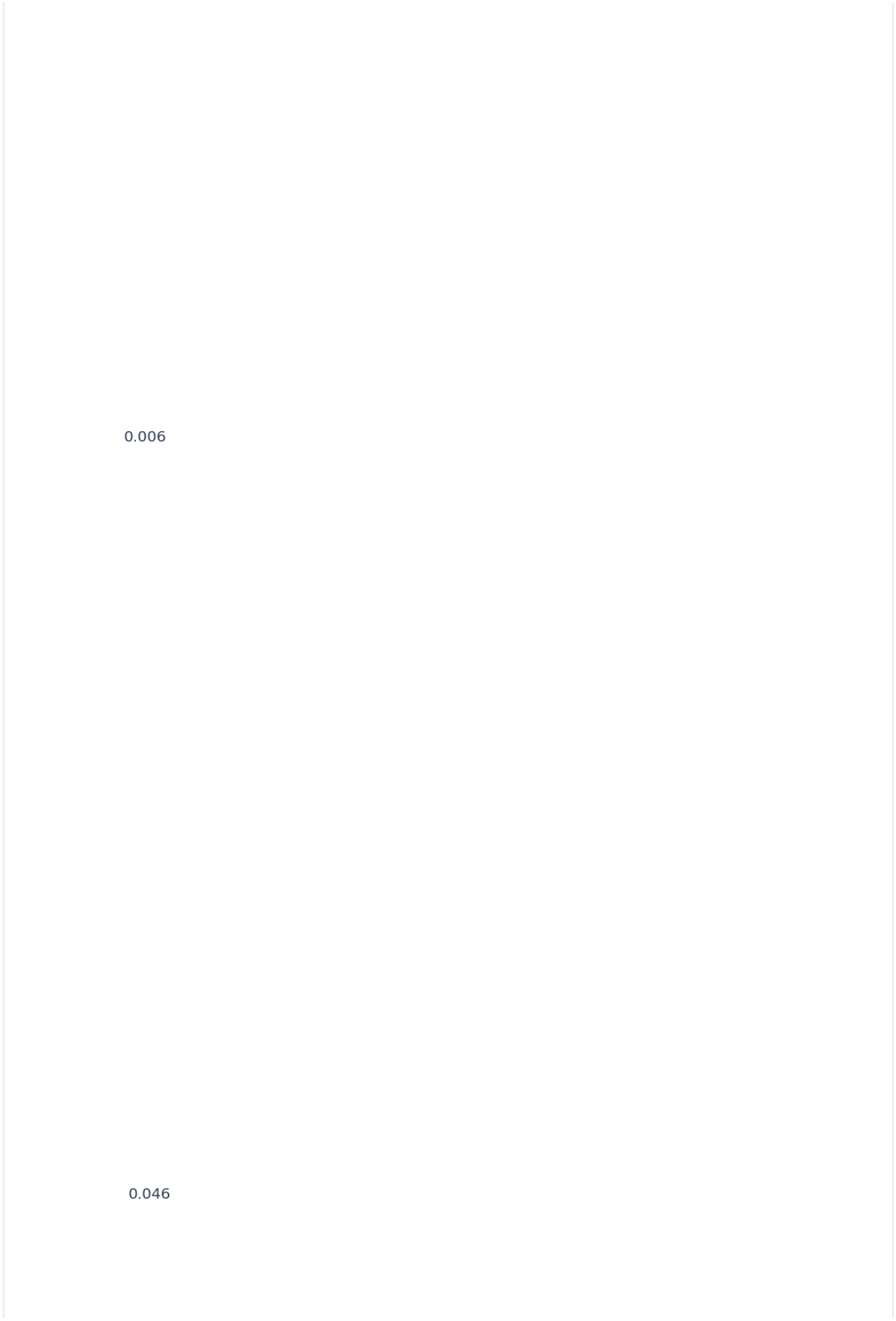
الارتباط الخطي (Pearson) يُجيب فقط على السؤال: «هل يرتفعان معاً؟» لكنه يفشل تماماً مع العلاقات غير الخطية. تخيّل دالة جيب التمام $y = \cos(x)$ في نطاق $[0, 2\pi]$: معامل الارتباط صفر، لكن y تعتمد اعتماداً تاماً على x.

المعلومات المتبادلة (Mutual Information) تقيس الاعتمادية الإجمالية بصرف النظر عن شكلها:

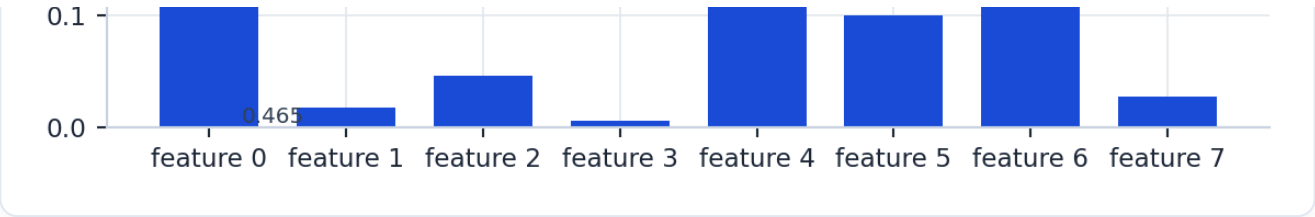
$$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)\,p(y)}$$


0.027
0.111


0.100
0.126


0.006
0.046

0.017
Mutual information finds non-linear dependence
mutual information
0.4
0.3
0.2



الشكل ٥٧ مقارنة درجات المعلومات المتبادلة لميزات مجموعة Iris

٣.٢ MI في scikit-learn: الدوال المتاحة

الدالة	الاستخدام	نوع المتغير المستهدف
`mutual_info_classif`	تصنيف	فئوي
`mutual_info_regression`	انحدار	مستمر
`SelectKBest(mutual_info_classif, k=…)`	اختيار K أفضل	فئوي
`SelectPercentile(mutual_info_regression, …)`	اختيار نسبة	مستمر

```
from sklearn.datasets import load_iris
from sklearn.feature_selection import mutual_info_classif
import numpy as np

X, y = load_iris(return_X_y=True)
feature_names = load_iris().feature_names

mi_scores = mutual_info_classif(X, y, random_state=42)

print("درجات المعلومات المتبادلة (Iris):")
for name, score in sorted(zip(feature_names, mi_scores),
                          key=lambda t: -t[1]):
    bar = "▉" * int(score * 20)
    print(f"  {name:<25} {score:.4f}  {bar}")
```

٣.٣ MI مقابل F-test: فرق جوهري

```python
import numpy as np
from sklearn.feature_selection import f_classif, mutual_info_classif

rng = np.random.default_rng(42)
n = 500

# ميزة 1: علاقة خطية قوية مع الهدف
x_linear = rng.standard_normal(n)
y = (x_linear > 0).astype(int)

# ميزة 2: علاقة غير خطية (تربيعية) – لا ارتباط خطي
x_nonlinear = rng.standard_normal(n)
# ولكن لا يوجد ارتباط خطي x² الهدف يعتمد على قيمة
y_nl = (x_nonlinear**2 > 0.5).astype(int)

X_test = np.column_stack([x_linear, x_nonlinear])

f_scores, _ = f_classif(X_test, y)
mi_scores = mutual_info_classif(X_test, y, random_state=42)

print("مقارنة F-test و MI:")
print(f"{'الميزة':>20} {'F-score':>10} {'MI-score':>10}")
print("-" * 42)
print(f"{'خطية':>20} {f_scores[0]:>10.3f} {mi_scores[0]:>10.4f}")
print(f"{'غير خطية':>20} {f_scores[1]:>10.3f} {mi_scores[1]:>10.4f}")
print()
print("→ F-test يُفضّل الخطية، MI يكتشف الغير خطية أيضاً")
```

> **خطأ شائع**
>
> **متى يفشل F-test**: إذا كانت العلاقة بين الميزة والهدف غير خطية (تربيعية، دائرية، نمطية)، ستُعطي `f_classif` درجة منخفضة وقد تستبعد الميزة الأكثر معلوماتية في بياناتك. استخدم دائماً `mutual_info_classif` حين تشك في وجود علاقات غير خطية.

٣.٤ MI مع بيانات الانحدار

```python
from sklearn.datasets import fetch_california_housing
from sklearn.feature_selection import mutual_info_regression, SelectKBest
import numpy as np

data = fetch_california_housing()
X, y = data.data, data.target
feature_names = data.feature_names

mi_scores = mutual_info_regression(X, y, random_state=42)
indices = np.argsort(mi_scores)[::-1]

print("المعلومات المتبادلة مع أسعار المنازل (California Housing):")
for idx in indices:
    bar = "█" * int(mi_scores[idx] * 10)
    print(f"  {feature_names[idx]:<12} {mi_scores[idx]:.4f}  {bar}")
```

٣.٥ الاختيار غير المشرف (Unsupervised Selection)

أحياناً لا يتوفر متغير هدف (مشاريع التجميع، ضغط البيانات، المرحلة الاستكشافية). هنا تصبح طرق الاختيار المشرفة غير صالحة. البدائل الرئيسية:

VarianceThreshold — الفلتر الأساسي

```python
from sklearn.datasets import make_classification
from sklearn.feature_selection import VarianceThreshold
import numpy as np

rng = np.random.default_rng(42)
X = rng.standard_normal((300, 50))

# أضف ميزات شبه ثابتة
X[:, 0] = 0.0          # ثابتة تماماً
X[:, 1] = rng.choice([0, 1], size=300, p=[0.99, 0.01])  # شبه ثابتة

selector = VarianceThreshold(threshold=0.01)
X_sel = selector.fit_transform(X)
print(f"الميزات الأصلية: {X.shape[1]}")
print(f"الميزات بعد VarianceThreshold(0.01): {X_sel.shape[1]}")
print(f"الميزات المحذوفة: {X.shape[1] - X_sel.shape[1]}")
```

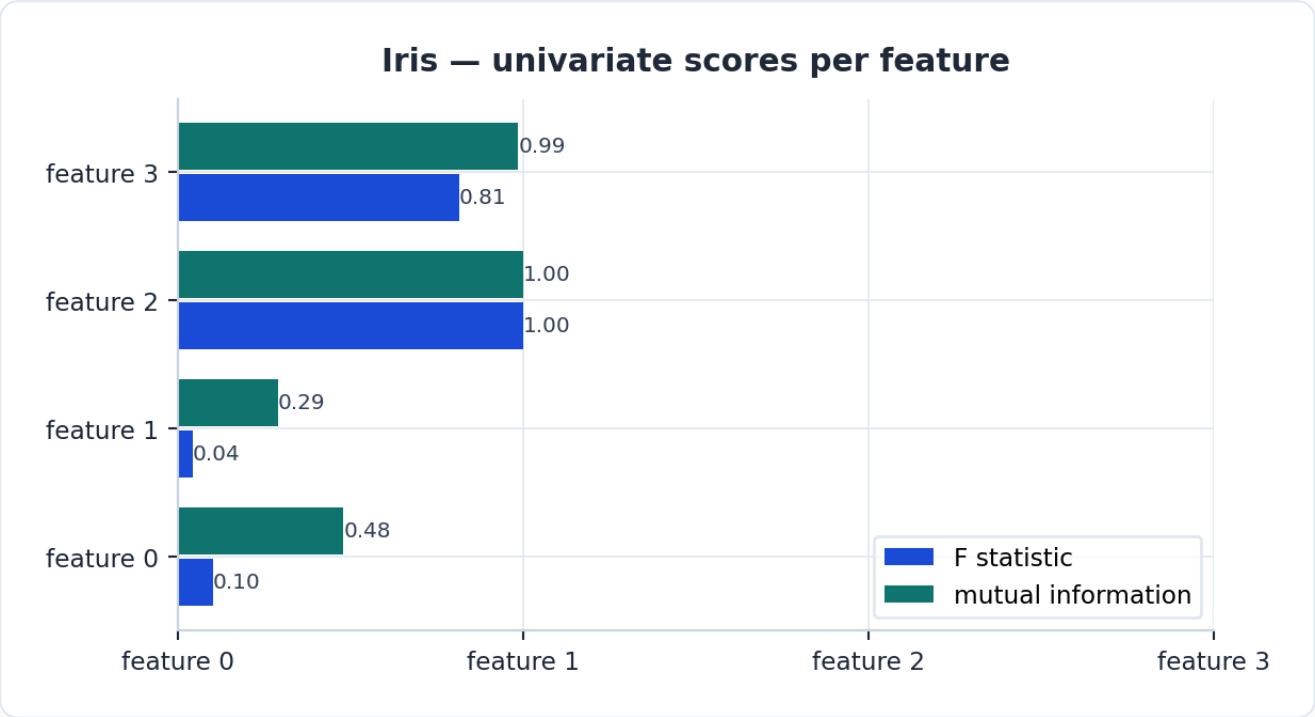
PCA — تحويل وليس اختيار

```python
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np

X, _ = load_digits(return_X_y=True)
print(f"الأبعاد الأصلية: {X.shape}")

# نُقلّص إلى الأبعاد التي تشرح 95% من التباين
pca = PCA(n_components=0.95, random_state=42)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_pca = pca.fit_transform(X_scaled)

print(f"الأبعاد بعد PCA (95% variance): {X_pca.shape}")
print(f"نسبة الاختزال: {1 - X_pca.shape[1]/X.shape[1]:.1%}")
cumvar = np.cumsum(pca.explained_variance_ratio_)
print(f"أول مكوّن يشرح: {pca.explained_variance_ratio_[0]:.2%}")
print(f"أول 10 مكونات تشرح: {cumvar[9]:.2%}")
```

> **ملاحظة**
>
> **PCA ليس اختيار ميزات بالمعنى الدقيق**: يُنشئ أبعاداً جديدة (تركيبات خطية) لا يحتفظ بالميزات الأصلية كما هي. إذا كنت تحتاج إلى قابلية تفسير الميزات الأصلية، استخدم `VarianceThreshold` أو `SelectKBest` بدلاً من PCA.

٣.٦ متى تختار MI على F-test؟

الموقف	الأنسب
علاقات خطية واضحة، بيانات كثيرة	`f_classif` أسرع وأوضح
علاقات غير خطية محتملة	`mutual_info_classif` ضروري
عيّنات صغيرة (< ٢٠٠ عينة)	كلاهما غير موثوق — أضف تحقق متقاطع
ميزات فئوية ثنائية	`chi2` مناسب أو MI
مشكلة انحدار	`f_regression` أو `mutual_info_regression`



الشكل ٥٨ مقارنة درجات المعلومات المتبادلة مع SelectKBest

تمارين الفصل السادس والعشرين

- **ملاحظة**

تمرين ١: على بيانات Breast Cancer، قارن بين `f_classif` و `mutual_info_classif` لاختيار أفضل ١٠ ميزات. ما نسبة التطابق بين القائمتين؟ وهل تتغيّر دقة `LogisticRegression` بالاختيارين؟

تمرين ٢: اصنع مجموعة بيانات تحتوي على علاقة جيب تمام خالص: `y = cos(x) + noise`. قارن `f_classif` و `mutual_info_regression` عليها. أيّهما يكتشف الاعتمادية؟

تمرين ٣: على بيانات Digits، طبّق `VarianceThreshold(threshold=0)` لحذف البكسلات الثابتة. كم بكسلاً حُذف؟ هل تتغيّر دقة `KNeighborsClassifier` قبل وبعد الحذف؟

- **ملاحظة**

حل تمرين ١:

```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import f_classif, mutual_info_classif,
SelectKBest
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
lr = LogisticRegression(C=1.0, max_iter=5000, random_state=42)

f_scores, _ = f_classif(X, y)
mi_scores = mutual_info_classif(X, y, random_state=42)

top10_f = set(np.argsort(f_scores)[::-1][:10])
top10_mi = set(np.argsort(mi_scores)[::-1][:10])
overlap = top10_f & top10_mi

print(f"ميزات f_classif: {[feature_names[i][:12] for i in top10_f]}")
print(f"ميزات MI:        {[feature_names[i][:12] for i in top10_mi]}")
print(f"مشتركة: {len(overlap)}/10")

pipe_f = Pipeline([("sel", SelectKBest(f_classif, k=10)), ("clf", lr)])
pipe_mi = Pipeline([("sel", SelectKBest(mutual_info_classif, k=10)),
("clf", lr)])

score_f = cross_val_score(pipe_f, X, y, cv=cv).mean()
score_mi = cross_val_score(pipe_mi, X, y, cv=cv).mean()
print(f"\nدقة f_classif pipeline:  {score_f:.4f}")
print(f"دقة MI pipeline:          {score_mi:.4f}")
```

حل تمرين ٣:

```python
from sklearn.datasets import load_digits
from sklearn.feature_selection import VarianceThreshold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

X, y = load_digits(return_X_y=True)
sel = VarianceThreshold(threshold=0)
X_sel = sel.fit_transform(X)

knn = KNeighborsClassifier(n_neighbors=5)
score_all = cross_val_score(knn, X, y, cv=5).mean()
score_sel = cross_val_score(knn, X_sel, y, cv=5).mean()

print(f"الأبعاد الأصلية: {X.shape[1]}  → دقة: {score_all:.4f}")
print(f"الأبعاد بعد حذف الثابتة: {X_sel.shape[1]}  → دقة: {score_sel:.4f}")
print(f"البكسلات المحذوفة: {X.shape[1] - X_sel.shape[1]}")
```

٣.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام `f_classif` على علاقات غير خطية	تُستبعد ميزات مهمة لعجز F- عن اكتشافها	استخدم `mutual_info_classif` أو اختبر كليهما
تفسير MI كمعامل ارتباط	MI لا مقياس مرجعي له (ليس ٠–١ دائماً)	طبّق `sklearn.feature_selection.normalize_mutual_info_score` للمقارنة النسبية
تطبيق MI على بيانات صغيرة جداً	التقدير غير موثوق بأقل من ١٠٠ عينة	تحقق متقاطع وتكرار عدة مرات بـ `random_state` مختلفة
الخلط بين PCA والاختيار	PCA يُنشئ أبعاداً مركّبة لا ميزات أصلية	استخدم PCA للضغط والنمذجة، والاختيار للتفسير

٣.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المعلومات المتبادلة	مقياس الاعتمادية الإحصائية العامة بين متغيرين	mutual information
الإنتروبيا	مقياس التشتت أو عدم اليقين في توزيع الاحتمال	entropy
الاعتماد غير الخطي	علاقة إحصائية لا تُمثَّل بخط مستقيم	nonlinear dependence
الاختيار غير المشرف	اختيار الميزات بدون استخدام متغير الهدف	unsupervised feature selection
تقدير كثافة الاحتمال	حساب تقريبي لتوزيع البيانات في الفضاء المستمر	density estimation

الجزء ٤

النماذج الخطية

العائلة الأولى التي يجب أن تتقّنها قبل أي شيء آخر

ماذا يغطّي هذا الجزء

النماذج الخطية ليست مقدّمة تمهيدية تُتخطّى، بل هي خط الأساس الذي يجب أن يهزمه أي نموذج أعقد. هنا نفهم الانحدار من أول مبدأ، ونتعلّم لماذا يُصفّر معامل L1 الخصائص، ولماذا يمكن لنموذج خطي أن يهزم شبكة عميقة في جدول بيانات صغير.

الجزء ٤ · النماذج الخطية

الفصل ٢٧ — قبل الانحدار الخطي: من الخطأ إلى أقل المربعات

مشكلة أقل المربعات، وتحليل SVD، والمتبقيات، والأقل تربيعاً

الوحدة **scikit-learn ١.٩** الحجم **٢٩٧** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

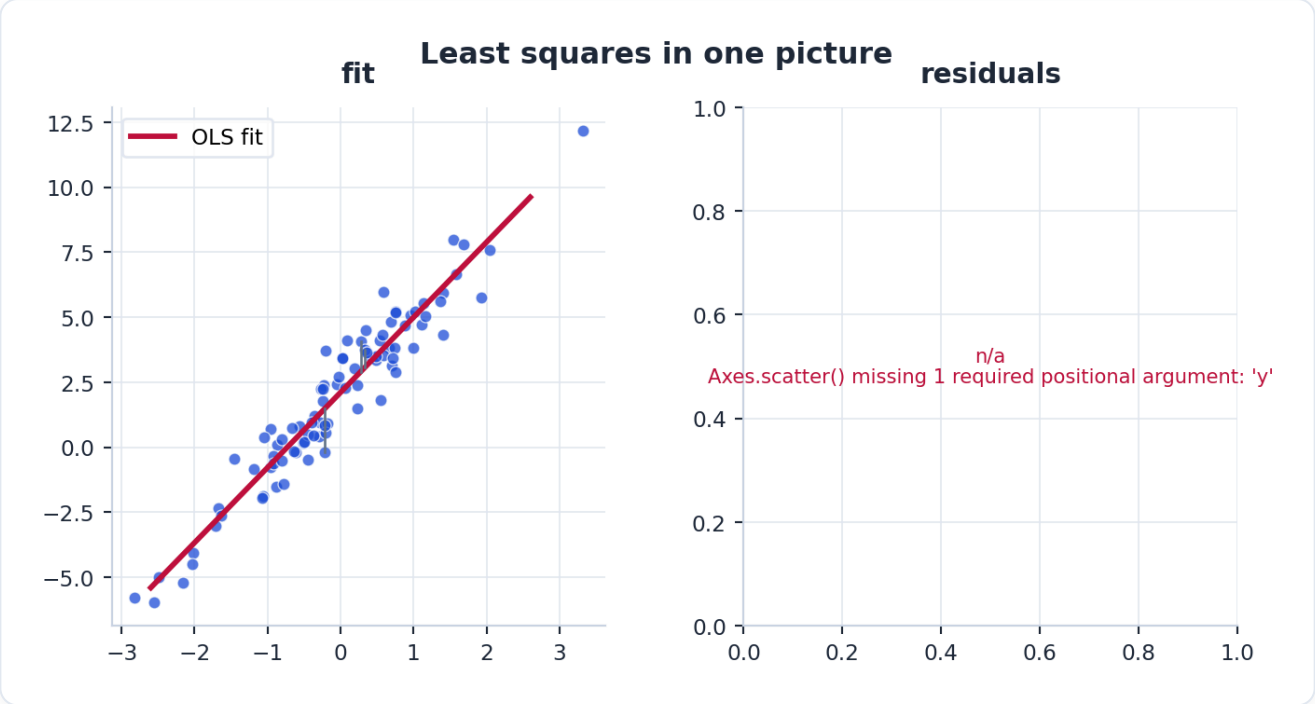
- أن تشتق صيغة الحل التحليلي لأقل المربعات من مبدأ الانحدار
- أن تفهم كيف يستخدم LinearRegression تحليل SVD داخلياً للاستقرار العددي
- أن تُحلّل المتبقيات (Residuals) وتتحقق من افتراضات النموذج الخطي
- أن تقيس جودة الملاءمة بـ R^2 و MSE وتُفسّر قيمهما في سياق المشكلة

٤.١ لماذا «أقل المربعات» تحديداً؟

قبل أي كود، سؤال: لماذا نُصغّر **مجموع مربعات** الأخطاء وليس مجموع قيمهم المطلقة؟

الإجابات ثلاث:

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2 = (X^T X)^{-1} X^T y$$



الشكل ٥٩ التفسير الهندسي لأقل المربعات: المسقط العمودي في الفضاء الخطي

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# بيانات انحدار بسيطة
X, y = make_regression(n_samples=200, n_features=5,
                       n_informative=5, noise=10, random_state=42)

lr = LinearRegression()
lr.fit(X, y)

y_pred = lr.predict(X)
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print(f"المعاملات (coef_): {lr.coef_.round(3)}")
print(f"الحد الثابت (intercept_): {lr.intercept_:.4f}")
print(f"MSE على التدريب: {mse:.4f}")
print(f"R² على التدريب: {r2:.4f}")
```

٤.٢ الحل التحليلي مقابل SVD

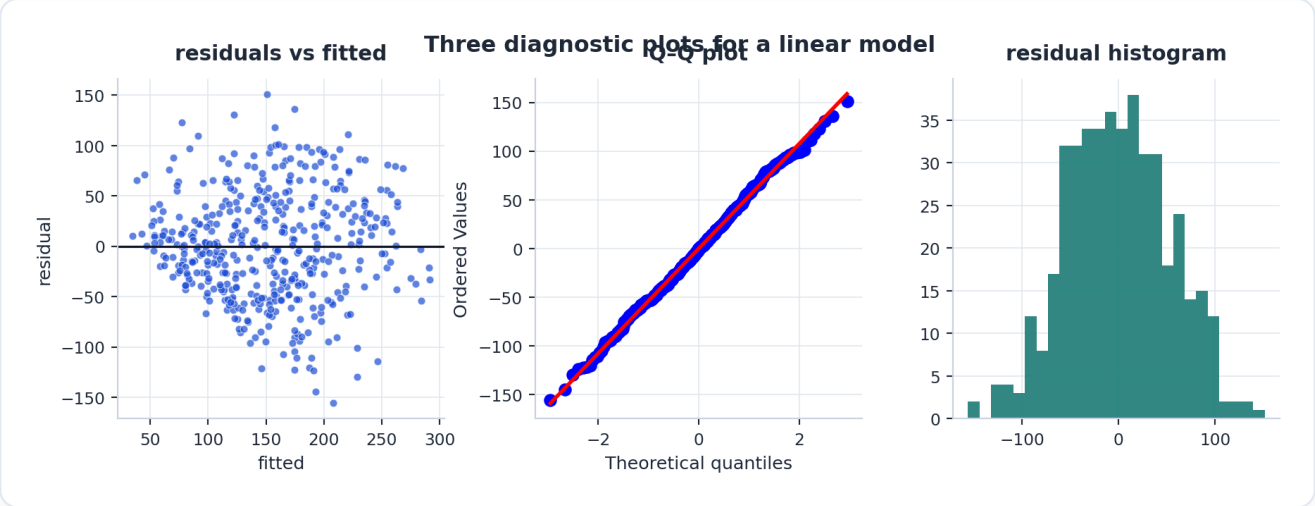
الصيغة $\hat{\beta} = (X^T X)^{-1} X^T y$ تبدو مباشرة، لكنها تنهار عددياً حين تكون $X^T X$ شبه منفردة (ill-conditioned). لهذا يعتمد scikit-learn على **تحليل القيم الشاذة (SVD)** بدلاً من العكسية المباشرة.

$$X = U \Sigma V^T \Longrightarrow \hat{\beta} = V \Sigma^{-1} U^T y$$

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=3,
                       noise=5, random_state=42)

# الحل اليدوي عبر SVD
U, s, Vt = np.linalg.svd(X, full_matrices=False)
beta_svd = Vt.T @ np.diag(1/s) @ U.T @ y

# الحل عبر الصيغة الطبيعية
beta_normal = np.linalg.lstsq(X, y, rcond=None)[0]

print("المعاملات عبر SVD   :", beta_svd.round(4))
print("المعاملات عبر lstsq :", beta_normal.round(4))
print("الفارق الأقصى:", np.abs(beta_svd - beta_normal).max())
```

٤.٣ تحليل المتبقيات

المتبقيات (Residuals) هي $e_i = y_i - \hat{y}_i$. في النموذج الخطي الجيد:



الشكل ٦٠ تشخيص المتبقيات: مخطط المتبقيات مقابل القيم المُتنبَّأ وتوزيعها الطبيعي

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression

X, y = make_regression(n_samples=200, n_features=3,
                       noise=15, random_state=42)

lr = LinearRegression()
lr.fit(X, y)
residuals = y - lr.predict(X)

print(f"متوسط المتبقيات: {residuals.mean():.8f}  (≈ 0 دائماً)")
print(f"انحراف المتبقيات: {residuals.std():.4f}")
print(f"أصغر متبقٍّ: {residuals.min():.4f}")
print(f"أكبر متبقٍّ: {residuals.max():.4f}")

# اختبار بسيط: هل المتبقيات غير مرتبطة بالتنبؤات؟
y_hat = lr.predict(X)
correlation = np.corrcoef(y_hat, residuals)[0, 1]
print(f"\nارتباط المتبقيات بـ ŷ: {correlation:.6f}  (≈ 0 في نموذج جيد)")
```

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression

X, y = make_regression(n_samples=200, n_features=3, noise=15, random_state=42)
lr = LinearRegression()
lr.fit(X, y)
y_hat = lr.predict(X)
residuals = y - y_hat

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].scatter(y_hat, residuals, alpha=0.5, s=20)
axes[0].axhline(0, color="red", linestyle="--")
axes[0].set_xlabel("Predicted values (ŷ)")
axes[0].set_ylabel("Residuals (e)")
axes[0].set_title("Residuals vs. Fitted")

axes[1].hist(residuals, bins=25, edgecolor="white")
axes[1].set_xlabel("Residual value")
axes[1].set_ylabel("Count")
axes[1].set_title("Residual Distribution")

plt.tight_layout()
plt.savefig("residuals_plot.png", dpi=100)
plt.close()
```

٤.٤ مقاييس الجودة: R^2 و MSE و MAE

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=300, n_features=10,
                       n_informative=5, noise=20, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
r2_train = lr.score(X_train, y_train)

print(f"{'المقياس':>20} {'التدريب':>10} {'الاختبار':>12}")
print("-" * 44)
print(f"{'R²':<20} {r2_train:>10.4f} {r2:>12.4f}")
print(f"{'RMSE':<20} {'—':>10} {rmse:>12.4f}")
print(f"{'MAE':<20} {'—':>10} {mae:>12.4f}")
```

> **نصيحة**
>
> **تفسير R^2**: القيمة ١.٠ تعني ملاءمة مثالية. القيمة ٠.٠ تعني أن النموذج لا يُفسِّر أي تباين — أداؤه مساوٍ لتنبّؤ المتوسط دائماً. القيم السالبة ممكنة وتعني أن النموذج أسوأ من المتوسط!

٤.٥ محدودية النموذج الخطي وإشارات التحذير

الإشارة	ما تعنيه	ما تفعله
R^2 منخفض على التدريب	علاقة غير خطية	أضف ميزات تحويلية أو جرّب نموذجاً آخر
R^2 عالٍ جداً على التدريب وضعيف على الاختبار	إفراط في الملاءمة	أضف تنظيماً (Lasso أو Ridge)
نمط واضح في مخطط المتبقيات	نموذج لم يلتقط علاقة هامة	أضف ميزات تفاعلية أو تحويلات
توزيع المتبقيات انحرافي	افتراض التوزيع الطبيعي مخترق	جرّب تحويل y أو نموذج توزيع آخر

خطأ شائع

المصيدة الكلاسيكية: R^2 على التدريب يمكن أن يصل لـ ١.٠ بمجرد إضافة متغيرات كثيرة (حتى لو عشوائية)، لأن النموذج يحفظ البيانات. استخدم دائماً **Adjusted R^2** أو قيّم على مجموعة اختبار مستقلة.

تمارين الفصل السابع والعشرين

- **ملاحظة**

تمرين ١: على مجموعة بيانات California Housing، دَرِّب `LinearRegression` وارسم مخطط المتبقيات مقابل القيم المُتنبَّأ بها. هل تلاحظ أي نمط؟ ماذا يعني ذلك؟

تمرين ٢: أنشئ مجموعة بيانات بـ `make_regression(n_features=50, n_informative=5)`. قارن R^2 على التدريب والاختبار. هل يوجد إفراط في الملاءمة؟ بكم نقطة؟

تمرين ٣: قارن الحل اليدوي عبر الصيغة الطبيعية $(X^TX)^{-1}X^Ty$ مع `LinearRegression` على بيانات `make_regression(n_features=3)`. هل النتائج متطابقة؟ ماذا يحدث حين تزيد الأبعاد كثيراً (مثل ١٠٠٠ ميزة لـ ١٠٠ عينة)؟

- **ملاحظة**

حل تمرين ٢:

```
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=200, n_features=50,
                       n_informative=5, noise=20, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

r2_train = lr.score(X_train, y_train)
r2_test = lr.score(X_test, y_test)

print(f"R² التدريب:  {r2_train:.4f}")
print(f"R² الاختبار: {r2_test:.4f}")
print(f"فجوة الإفراط: {r2_train - r2_test:+.4f}")
print()
if r2_train - r2_test > 0.1:
    print("← فجوة كبيرة تشير إلى إفراط في الملاءمة")
    print("   الحل: Ridge أو Lasso (الفصل التالي) لتنظيم المعاملات")
```

حل تمرين ٣:

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression

X, y = make_regression(n_samples=100, n_features=3,
                       noise=5, random_state=42)
# إضافة عمود تحيّز
X_b = np.column_stack([np.ones(len(X)), X])

# الحل اليدوي بالصيغة الطبيعية
beta_manual = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

# LinearRegression
lr = LinearRegression()
lr.fit(X, y)
beta_sklearn = np.concatenate([[lr.intercept_], lr.coef_])

print("الحل اليدوي:", beta_manual.round(4))
print("LinearRegression:", beta_sklearn.round(4))
print("أقصى فرق:", np.abs(beta_manual - beta_sklearn).max())
```

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الثقة بـ R^2 على التدريب	يمكن أن يصل لـ ١.٠ بإضافة ميزات عشوائية	قيّم على مجموعة اختبار مستقلة دائماً
إهمال تحليل المتبقيات	نماذج خاطئة تُعطي R^2 عالياً على الظاهر	ارسم المتبقيات مقابل القيم المُتنبَّأ قبل الاستنتاج
إهمال تطبيع الميزات	معاملات ذات وحدات مختلفة غير قابلة للمقارنة	استخدم `StandardScaler` إذا كنت ستفسّر المعاملات
افتراض الخطية بلا اختبار	النموذج يفشل في بيانات غير خطية	ارسم المتبقيات وتحقق من النمط أولاً

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
أقل المربعات	تقليل مجموع مربعات الفروق بين القيم الحقيقية والمتنبَّأ بها	ordinary least squares
المتبقيات	الفروق بين القيم الحقيقية والقيم المُتنبَّأ بها	residuals
تحليل القيم الشاذة	تحليل مصفوفي يُمكّن الحل الرقمي المستقر	singular value decomposition (SVD)
معامل التحديد	نسبة التباين التي يُفسّرها النموذج من التباين الكلي	R-squared
مصفوفة شبه منفردة	مصفوفة ذات قيم شاذة قريبة من الصفر، تجعل عكسها غير مستقر	ill-conditioned matrix

الجزء ٤ · النماذج الخطية

الفصل ٢٨ — تشخيص النموذج الخطي

الفرضيات، والتعدد الخطي، و VIF، ومخطط QQ، والأثر

الوحدة **scikit-learn ١.٩** الحجم **٤٠٧** كلمة تقريباً

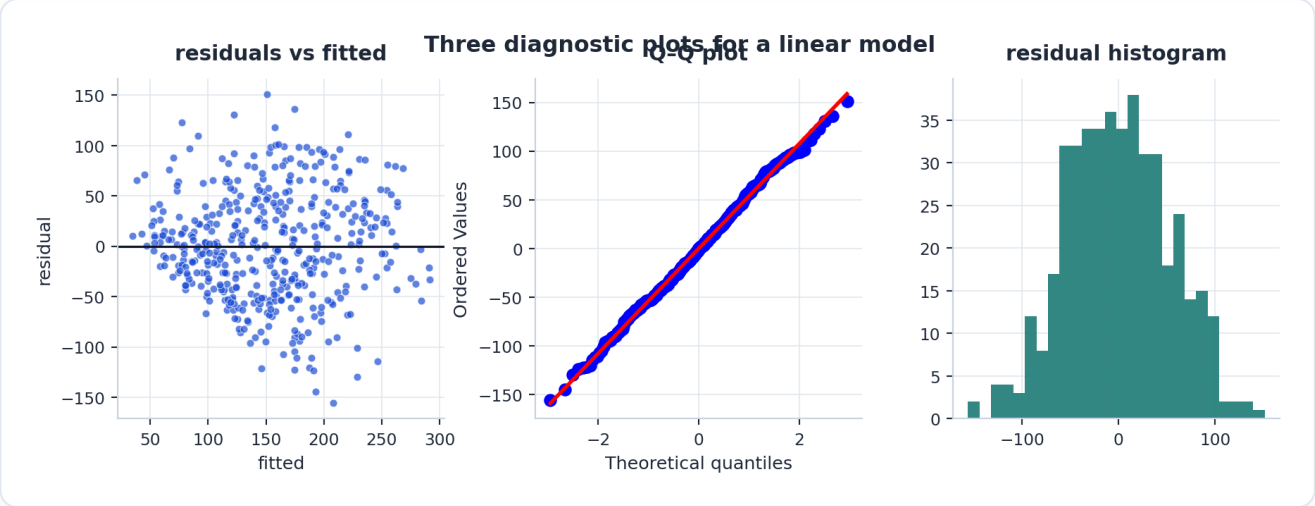
ما الذي ستتقنه في هذا الفصل

- أن تحسب VIF لكل ميزة وتكتشف التعدد الخطي الخطير في بياناتك
- أن تقرأ مخطط QQ وتحكم على توزيع المتبقيات بسرعة
- أن تُحدّد نقاط الأثر العالي (High-Leverage) وتُقرّر كيف تتعامل معها
- أن تُصلح مشكلة التعدد الخطي بحذف الميزات المترابطة أو إضافة Ridge

٤.١ الافتراضات الأربعة للنموذج الخطي

النموذج الخطي يُنتج تقديرات موثوقة فقط حين تنطبق عليه أربعة افتراضات:

الافتراض	ما يعنيه	كيف تكتشف انكساره
الخطية	العلاقة بين X و y خطية	مخطط المتبقيات — نمط منحنٍ
التجانس	تباين الأخطاء ثابت (Homoscedasticity)	مخطط المتبقيات — قمع أو توسّع
التوزيع الطبيعي	الأخطاء موزّعة طبيعياً	مخطط QQ
الاستقلالية	المتبقيات غير مترابطة مع بعض	مخطط Auto-correlation



الشكل ٦١ تشخيصات المتبقيات: مخطط QQ والأنماط الشائعة لانكسار الافتراضات

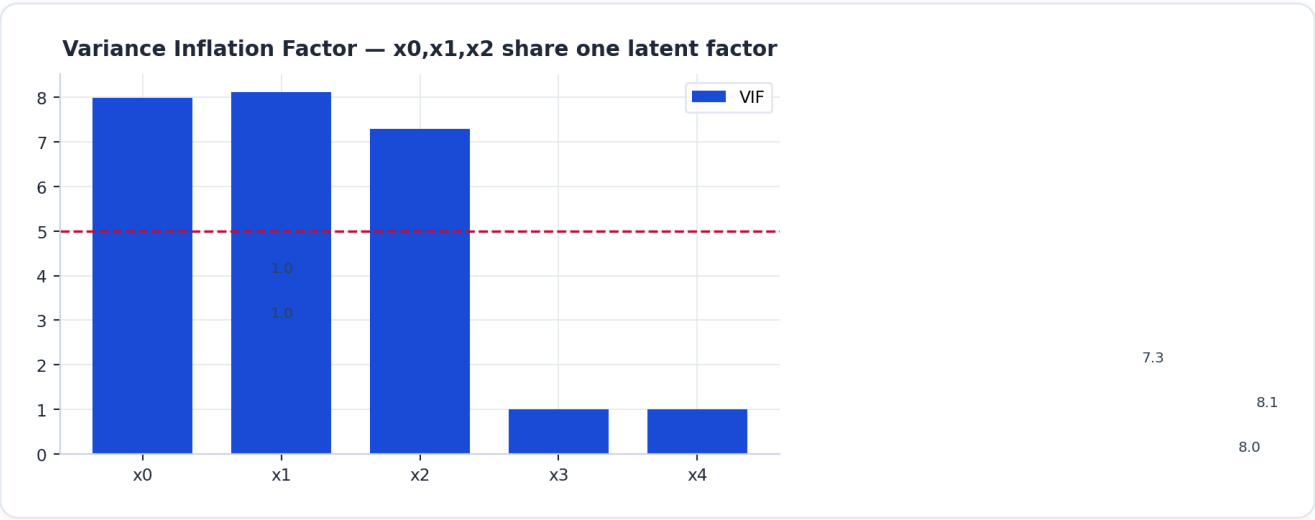
٤.٢ التعدد الخطي وعامل تضخّم التباين VIF

التعدد الخطي (Multicollinearity) يحدث حين تكون ميزتان أو أكثر مترابطتان ارتباطاً عالياً. لا يُؤثّر على جودة التنبؤ، لكنه يجعل المعاملات **غير مستقرة وغير قابلة للتفسير**.

VIF (Variance Inflation Factor) لميزة j يُقيس مدى ارتفاع تباين معاملها بسبب الترابط:

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

حيث R_j^2 هو R^2 من انحدار x_j على باقي الميزات.



الشكل ٦٢ تأثير التعدد الخطي وتضخم التباين في معاملات النموذج الخطي

```python
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression

data = fetch_california_housing()
X, y = data.data, data.target
feature_names = list(data.feature_names)

# حساب VIF يدوياً
n_features = X.shape[1]
vif_values = []
for j in range(n_features):
    X_others = np.delete(X, j, axis=1)
    lr_vif = LinearRegression()
    lr_vif.fit(X_others, X[:, j])
    r2_j = lr_vif.score(X_others, X[:, j])
    vif_j = 1 / (1 - r2_j) if r2_j < 1 else float("inf")
    vif_values.append(vif_j)

print(f"{'الميزة':<15} {'VIF':>8}  {'التقييم':<12}")
print("-" * 40)
for name, vif in sorted(zip(feature_names, vif_values), key=lambda t: -t[1]):
    if vif > 10:
        evaluation = "⚠️ مرتفع جداً"
    elif vif > 5:
        evaluation = "⚡ مرتفع"
    else:
        evaluation = "✓ مقبول"
    print(f"{name:<15} {vif:>8.2f}  {evaluation}")
```

⛔ تحذير

VIF > ١٠: يُشير إلى تعدد خطي خطير. المعاملات في هذه الحالة تكاد تكون عشوائية — تغيّر صغير في البيانات قد يُقلب إشاراتها كلياً. الحل: احذف إحدى الميزات المترابطة، أو استخدم Ridge الذي يُستعرض في الفصل التالي.

٤.٣ مخطط QQ للتحقق من التوزيع الطبيعي

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from scipy import stats

X, y = make_regression(n_samples=200, n_features=5,
                       noise=15, random_state=42)

lr = LinearRegression()
lr.fit(X, y)
residuals = y - lr.predict(X)

# اختبار Shapiro-Wilk لطبيعية المتبقيات
stat, p_value = stats.shapiro(residuals)
print(f"اختبار Shapiro-Wilk:")
print(f"  الإحصاء W = {stat:.4f}")
print(f"  القيمة p  = {p_value:.4f}")
if p_value > 0.05:
    print("  → لا دليل كافٍ على انحراف عن الطبيعية (p > 0.05)")
else:
    print("  → المتبقيات تنحرف عن الطبيعية بشكل دال إحصائياً")

# إحصاءات ربيعية للمقارنة
q25, q75 = np.percentile(residuals, [25, 75])
print(f"\nالانحراف المعياري: {residuals.std():.4f}")
print(f"IQR: {q75 - q25:.4f}")
```

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from scipy import stats

X, y = make_regression(n_samples=200, n_features=5, noise=15, random_state=42)
lr = LinearRegression()
lr.fit(X, y)
residuals = y - lr.predict(X)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# QQ Plot
(osm, osr), (slope, intercept, r) = stats.probplot(residuals, dist="norm")
axes[0].scatter(osm, osr, s=15, alpha=0.6)
axes[0].plot([osm[0], osm[-1]], [slope*osm[0]+intercept,
slope*osm[-1]+intercept],
             color="red", linestyle="--")
axes[0].set_xlabel("Theoretical Quantiles")
axes[0].set_ylabel("Sample Quantiles")
axes[0].set_title("Q-Q Plot of Residuals")

# Histogram
axes[1].hist(residuals, bins=25, edgecolor="white", density=True)
x = np.linspace(residuals.min(), residuals.max(), 200)
axes[1].plot(x, stats.norm.pdf(x, residuals.mean(), residuals.std()),
             color="red", linewidth=2)
axes[1].set_title("Residual Distribution vs Normal")
axes[1].set_xlabel("Residual")

plt.tight_layout()
plt.savefig("qq_plot.png", dpi=100)
plt.close()
```

٤.٤ نقاط الأثر العالي — Cook's Distance

نقطة الأثر العالي هي عينة تُؤثّر بشكل غير متناسب في موضع خط الانحدار. تُقاس بـ **Cook's Distance**:

$$D_i = \frac{(\hat{\beta} - \hat{\beta}_{(i)})^T (X^T X) (\hat{\beta} - \hat{\beta}_{(i)})}{p \cdot \text{MSE}}$$

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(42)
X, y = make_regression(n_samples=100, n_features=2,
                       noise=10, random_state=42)

# أضف نقطتي أثر عاليتين مصطنعتين
X[0] = [10, 10]
y[0] = -200  # قيمة مختلفة جداً

lr = LinearRegression()
lr.fit(X, y)

# حساب Cook's Distance تقريبياً
n, p = X.shape
y_hat = lr.predict(X)
residuals = y - y_hat
mse = (residuals**2).mean()
hat_matrix = X @ np.linalg.pinv(X.T @ X) @ X.T
leverage = np.diag(hat_matrix)

# Cook's Distance باستخدام الصيغة المبسّطة
cooks = (residuals**2 * leverage) / (p * mse * (1 - leverage + 1e-8)**2)
threshold = 4 / n

top5 = np.argsort(cooks)[::-1][:5]
print(f"عتبة Cook's Distance (4/n = {threshold:.4f}):")
print(f"\nأعلى 5 نقاط أثراً:")
for i in top5:
    status = "⚠ أثر عالٍ" if cooks[i] > threshold else "✓ طبيعية"
    print(f"  العينة #{i:3d}: D = {cooks[i]:.4f}  {status}")
```

في الواقع

في الواقع: وجود نقاط أثر عالٍ لا يعني بالضرورة حذفها. أحياناً هي **أصيلة** وتحمل معلومة حقيقية. السؤال الصحيح: هل هذه النقطة خطأ في البيانات أم ظاهرة حقيقية؟ إذا كانت خطأً، احذفها؛ إذا كانت حقيقية، جرّب نماذج متسامحة مثل `HuberRegressor`.

٤.٥ مثال متكامل: تشخيص نموذج على بيانات حقيقية

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = fetch_california_housing()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

lr = LinearRegression()
lr.fit(X_train_sc, y_train)

r2_train = lr.score(X_train_sc, y_train)
r2_test = lr.score(X_test_sc, y_test)
residuals = y_test - lr.predict(X_test_sc)

print(f"R² التدريب: {r2_train:.4f}")
print(f"R² الاختبار: {r2_test:.4f}")
print(f"متوسط المتبقيات: {residuals.mean():.4f}")
print(f"انحراف المتبقيات: {residuals.std():.4f}")

# هل هناك نمط في المتبقيات الكبيرة؟
large_residuals = np.abs(residuals) > 2 * residuals.std()
print(f"\nنسبة المتبقيات الكبيرة (|e| > 2σ): {large_residuals.mean():.2%}")
print("(المتوقع في التوزيع الطبيعي: ~5%)")
```

تمارين الفصل الثامن والعشرين

- **ملاحظة**

تمرين ١: على بيانات California Housing، احسب VIF لكل ميزة. هل هناك ميزتان مترابطتان بشكل خطير؟ جرّب حذف الأعلى VIF وأعِد تدريب النموذج — هل تتغيّر دقته؟

تمرين ٢: أنشئ مجموعة بيانات بها أخطاء غير طبيعية (توزيع بيتا أو تربيع متغير عشوائي). دَرِّب `LinearRegression` وارسم مخطط QQ للمتبقيات. ماذا تلاحظ؟

تمرين ٣: على بيانات Diabetes (`load_diabetes()`، ١٠ ميزات)، احسب VIF لكل الميزات. ثم استخدم ما تعلّمته من الفصل السابق لاختيار الميزات التي VIF < ٥ واعقد مقارنة بين نموذج الكل ونموذج الميزات المختارة.

- **ملاحظة**

حل تمرين ١:

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

data = fetch_california_housing()
X, y = data.data, data.target
feature_names = list(data.feature_names)

# حساب VIF
vif_dict = {}
for j in range(X.shape[1]):
    X_others = np.delete(X, j, axis=1)
    lr_vif = LinearRegression()
    lr_vif.fit(X_others, X[:, j])
    r2_j = lr_vif.score(X_others, X[:, j])
    vif_dict[feature_names[j]] = 1 / (1 - r2_j) if r2_j < 1 else
float("inf")

print("VIF لكل ميزة:")
for name, vif in sorted(vif_dict.items(), key=lambda t: -t[1]):
    print(f"  {name:<12}: {vif:.2f}")

# احذف الميزة الأعلى VIF
worst = max(vif_dict, key=vif_dict.get)
idx_worst = feature_names.index(worst)
X_reduced = np.delete(X, idx_worst, axis=1)

lr = LinearRegression()
cv_all = cross_val_score(lr, X, y, cv=5, scoring="r2").mean()
cv_red = cross_val_score(lr, X_reduced, y, cv=5, scoring="r2").mean()
print(f"\nR² CV (كل الميزات): {cv_all:.4f}")
print(f"R² CV (بدون {worst}): {cv_red:.4f}")
```

حل تمرين ٣:

```python
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

X, y = load_diabetes(return_X_y=True)
feature_names = list(load_diabetes().feature_names)

vif_values = []
for j in range(X.shape[1]):
    X_others = np.delete(X, j, axis=1)
    lr_vif = LinearRegression()
    lr_vif.fit(X_others, X[:, j])
    r2_j = lr_vif.score(X_others, X[:, j])
    vif_values.append(1 / (1 - r2_j) if r2_j < 1 else float("inf"))

safe_idx = [i for i, vif in enumerate(vif_values) if vif < 5]
print(f"ميزات VIF < 5: {[feature_names[i] for i in safe_idx]}")

lr = LinearRegression()
r2_all = cross_val_score(lr, X, y, cv=5, scoring="r2").mean()
r2_sel = cross_val_score(lr, X[:, safe_idx], y, cv=5, scoring="r2").mean()
print(f"R² (كل الميزات): {r2_all:.4f}")
print(f"R² (VIF < 5):    {r2_sel:.4f}")
```

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تجاهل تحليل المتبقيات	قد يكون الافتراض مكسوراً والنموذج مضلّلاً	ارسم المتبقيات مقابل القيم المُتنبَّأ دائماً
حذف نقاط الأثر تلقائياً	قد تكون نقاطاً حقيقية تحمل معلومة	فرّق بين الخطأ في البيانات والظاهرة الحقيقية أولاً
الاكتفاء بـ R^2 لتشخيص النموذج	R^2 لا يكشف الافتراضات المكسورة	أضف مخطط QQ وفحص التباين
تفسير معاملات نموذج ذي تعدد خطي عالٍ	معاملات غير مستقرة تُفسَّر خطأً	احذف الميزات المترابطة أو انتقل لـ Ridge

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التعدد الخطي	ارتباط عالٍ بين ميزتين أو أكثر في نموذج الانحدار	multicollinearity
عامل تضخم التباين	مقياس لمدى تأثير التعدد الخطي على دقة تقدير المعامل	variance inflation factor (VIF)
مخطط QQ	مقارنة كمّيات المتبقيات مع الكمّيات النظرية للتوزيع الطبيعي	quantile-quantile plot
نقطة الأثر العالي	عينة تُغيّر موضع خط الانحدار بشكل كبير عند إزالتها	high-leverage point
مسافة كوك	مقياس إجمالي لأثر نقطة واحدة على كل المعاملات	Cook's distance
تجانس التباين	ثبات تباين الأخطاء عبر كل قيم المتغيرات المستقلة	homoscedasticity

الجزء ٤ · النماذج الخطية

الفصل ٢٩ — الانحدار المنظّم: Ridge و ElasticNet

لماذا يحتاج النموذج الخطي إلى عقوبة، وكيف تعمل L1 و L2

الوحدة **scikit-learn ١.٩** الحجم **٤٦٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح المقايضة بين الانحياز والتباين وتفسر دور معامل التنظيم alpha
- أن تطبق Ridge و RidgeCV مع المعايرة التلقائية للبيانات عبر خطوط الأنابيب
- أن تستخدم ElasticNet و ElasticNetCV وتضبط l1_ratio بدقة لاقتناص المجموعات المترابطة
- أن تقارن أداء الحل المغلق لـ Ridge مع النزول الاشتقاقي الإحداثي في ElasticNet

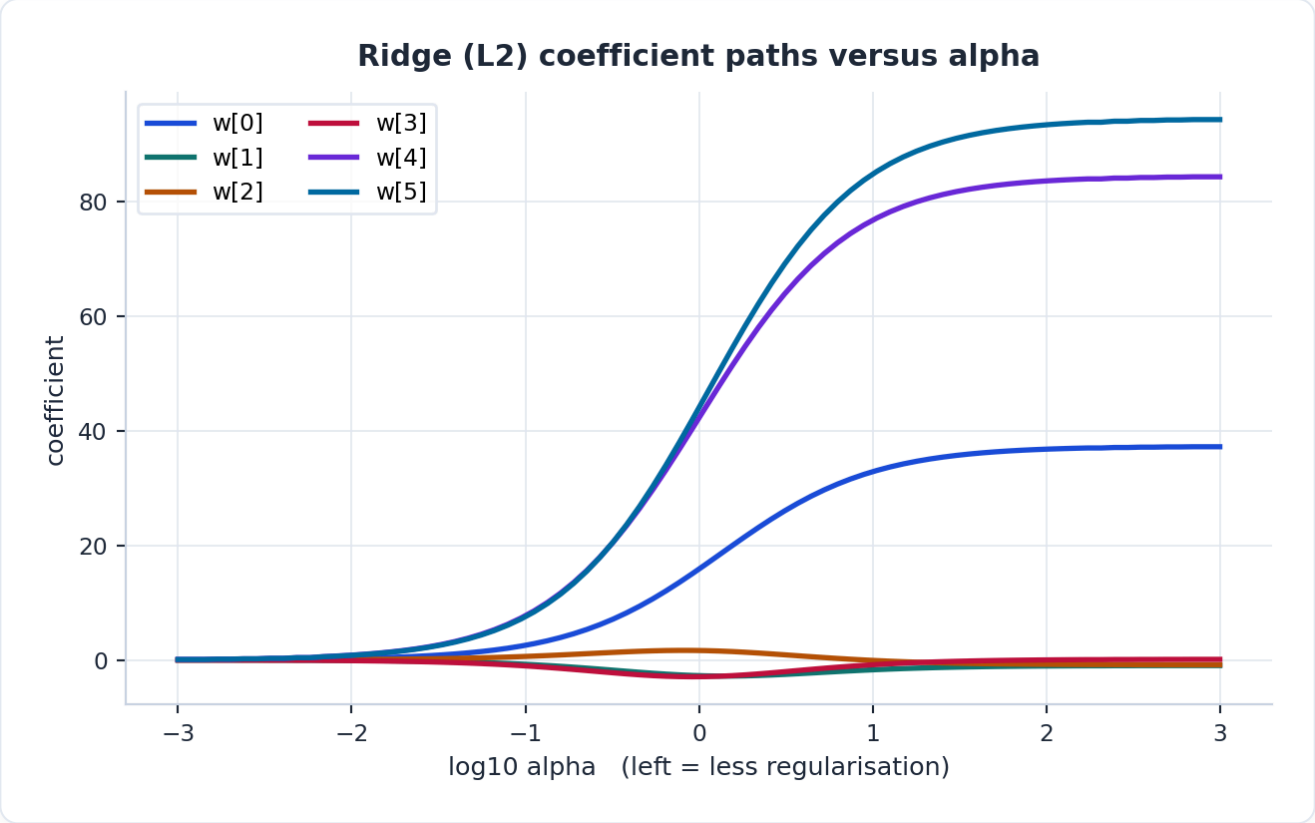
٢٩.١ لماذا يحتاج النموذج الخطي إلى عقوبة؟

رأينا في الفصلين السابقين أن نموذج المربعات الصغرى العادي (OLS) يبحث عن المعاملات $\hat{\beta}$ التي تقلل مجموع مربعات الأخطاء فقط:

$$\min_{\beta} \|y - X\beta\|_2^2$$

عندما تزداد أبعاد البيانات p مقارنة بحجم العينات n، أو عندما تتداخل الخصائص وتتعدد خطياً، تصبح مصفوفة المعلومات X^TX شبه مفردة. والنتيجة العملية كارثية: تضخم هائل في قيم المعاملات β، وحساسية مفرطة لأبسط ضجيج في البيانات، مما يقود إلى تباين هائل (High Variance) وإفراط فادح في الملاءمة.

هنا يتدخل **التنظيم (Regularization)** ليضيف مبدأ نصل أوكام الرياضي: «لا تدع المعاملات تكبر بلا مبرر قاطع».



الشكل ٦٣ مسارات تقليص المعاملات بين تنظيمي Ridge و Lasso مع زيادة قوة العقوبة

٤.٢ انحدار تيكونوف: Ridge وحساب عقوبة L_2

يضيف انحدار Ridge (المعروف تاريخياً بتنظيم تيكونوف) عقوبة تربيعية على المعاملات:

$$\min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2 = \min_{\beta} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \alpha \sum_{j=1}^{p} \beta_j^2$$

الميزة الجوهرية لـ Ridge هي احتفاظه بـ **حل تحليلي مغلق (Closed-form Solution)**:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \alpha I)^{-1} X^T y$$

إضافة αI إلى القطر تضمن دوماً قابلية عكس المصفوفة، حتى لو كان عدد الميزات يفوق عدد العينات بمراحل ($p \gg n$).

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# إنشاء بيانات ذات تعدد خطي وخصائص غير أساسية
X, y = make_regression(n_samples=120, n_features=60, n_informative=15,
                       noise=20.0, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# مقارنة بين OLS و Ridge غير المقيد
pipe_ols = Pipeline([("scaler", StandardScaler()), ("reg", LinearRegression())])
pipe_ridge = Pipeline([("scaler", StandardScaler()), ("reg",
Ridge(alpha=100.0))])

pipe_ols.fit(X_train, y_train)
pipe_ridge.fit(X_train, y_train)

print(f"OLS   - R² التدريب: {pipe_ols.score(X_train, y_train):.4f} | R² الاختبار:
{pipe_ols.score(X_test, y_test):.4f}")
print(f"Ridge - R² التدريب: {pipe_ridge.score(X_train, y_train):.4f} | R² الاختبار:
{pipe_ridge.score(X_test, y_test):.4f}")
print(f"معيار المعاملات L2 (OLS):
{np.linalg.norm(pipe_ols.named_steps['reg'].coef_):.2f}")
print(f"معيار المعاملات L2 (Ridge):
{np.linalg.norm(pipe_ridge.named_steps['reg'].coef_):.2f}")
```

> **نصيحة**
>
> تنظيم Ridge يقلص المعاملات مقترباً بها من الصفر بشكل متزامن، لكنه **لا يصفر أي معامل تماماً**. لذلك لا يعتبر أداة لاختيار الخصائص (Feature Selection)، بل أداة لتثبيت التباين والتحكم بالتعدد الخطي.

٤.٣ الضبط الذاتي لمعامل العقوبة: RidgeCV

بدلاً من تخمين قيمة α عشوائياً، يوفر scikit-learn المقدر المدمج `RidgeCV` الذي يستغل الحل المغلق لإجراء تحقق متقاطع سريع جداً بكفاءة حسابية $O(1)$ لكل قيمة لـ α بفضل حيلة التفكيك التبادلي (Generalized Cross-Validation).

```python
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import RidgeCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

housing = fetch_california_housing()
X, y = housing.data, housing.target

alphas = np.logspace(-3, 3, 20)
pipe_cv = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge_cv", RidgeCV(alphas=alphas, cv=5, scoring="neg_mean_squared_error"))
])

pipe_cv.fit(X, y)
best_ridge = pipe_cv.named_steps["ridge_cv"]

print(f"أفضل قيمة alpha مختارة: {best_ridge.alpha_:.4f}")
print(f"أفضل معامل تحديد R²: {pipe_cv.score(X, y):.4f}")
```

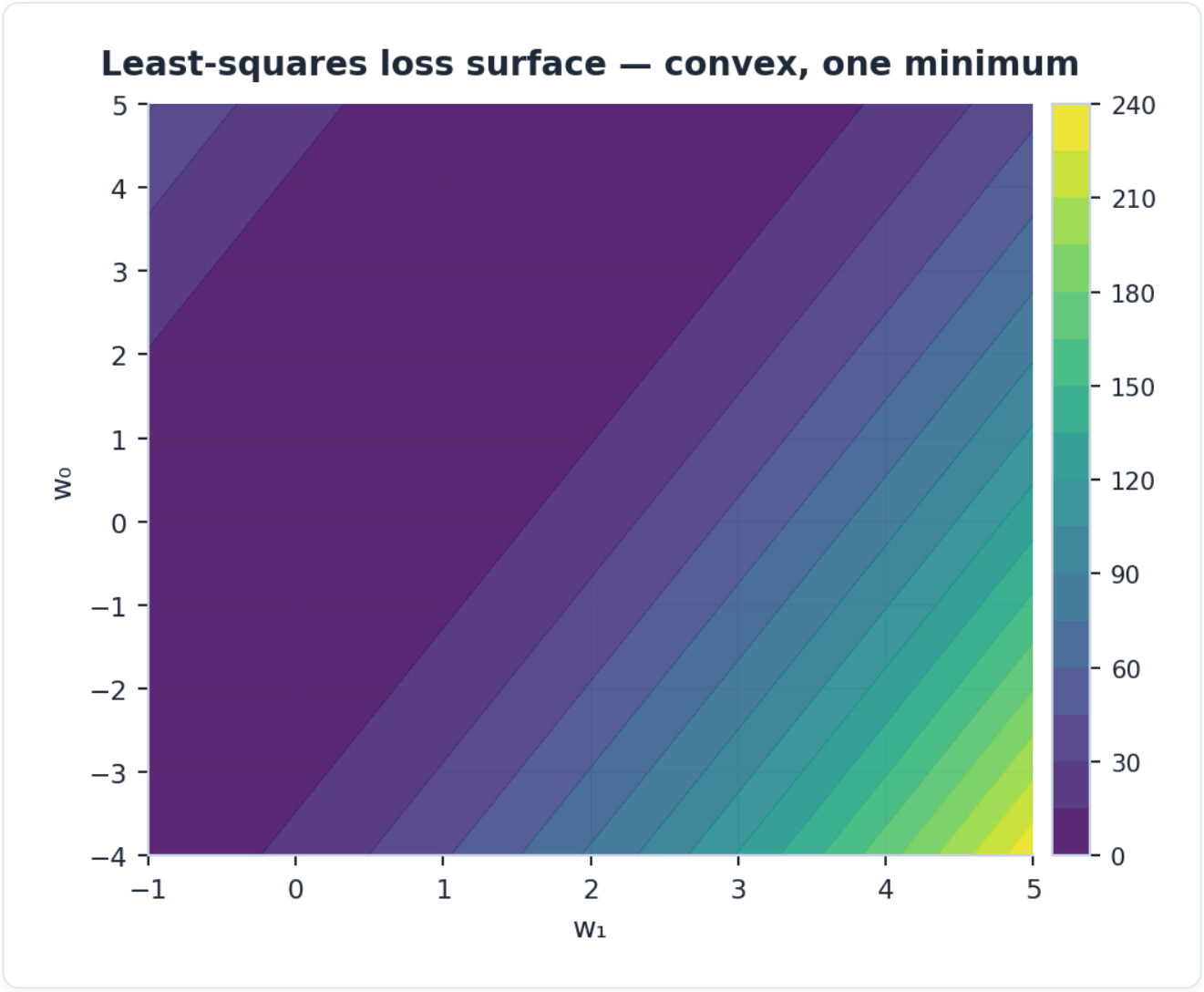
٤.٤ ElasticNet: دمج المرونة والانتخاب

بينما يقلص Ridge جميع المعاملات بنعومة، يقوم Lasso (تنظيم L_1) بتصفير المعاملات تماماً (كما سنفصل في الفصل ٣٠). لكن Lasso يعاني نقطة ضعف واضحة: إذا كانت هناك مجموعة من الميزات المترابطة جداً، فإنه يختار إحداها عشوائياً ويهمل البقية.

هنا يأتي **الانحدار الشبكي المرن (ElasticNet)** ليمزج العقوبتين معاً:

$$\min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha\rho\|\beta\|_1 + \frac{\alpha(1-\rho)}{2}\|\beta\|_2^2$$

حيث:



الشكل ٦٤ خطوط الكنتور والمناطق المسموحة للحلول في فضاء ElasticNet بين المعيني والدائري

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# بيانات تحوي ميزات مكررة ومرتبطة للغاية
X, y = make_regression(n_samples=150, n_features=20, n_informative=5,
                       effective_rank=4, noise=10.0, random_state=42)

pipe_enet = Pipeline([
    ("scaler", StandardScaler()),
    ("enet", ElasticNet(alpha=0.5, l1_ratio=0.6, random_state=42))
])

pipe_enet.fit(X, y)
coefs = pipe_enet.named_steps["enet"].coef_

n_zeros = np.sum(coefs == 0.0)
print(f"إجمالي الميزات: {len(coefs)}")
print(f"الميزات المصفرة تماماً بفعل L1: {n_zeros}")
print(f"الميزات المحتفظ بها بفضل التوازن الشبكي: {len(coefs) - n_zeros}")
```

٤.٥ التحقق المتقاطع وضبط المعاملات المزدوجة: ElasticNetCV

في ElasticNet نحتاج إلى ضبط قيمتين في آنٍ واحد: شدة التنظيم α ونسبة الدمج `l1_ratio`. يتكفل `ElasticNetCV` بالبحث الممنهج عبر مسار الانحدار (Regularization Path).

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import ElasticNetCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

X, y = load_diabetes(return_X_y=True)

# ضبط l1_ratio و alphas معاً
l1_ratios = [0.1, 0.5, 0.7, 0.9, 0.95, 0.99]
pipe_enet_cv = Pipeline([
    ("scaler", StandardScaler()),
    ("enet_cv", ElasticNetCV(l1_ratio=l1_ratios, cv=5, random_state=42,
alphas=50))
])

pipe_enet_cv.fit(X, y)
best_model = pipe_enet_cv.named_steps["enet_cv"]

print(f"أفضل نسبة l1_ratio مختارة: {best_model.l1_ratio_}")
print(f"أفضل قيمة alpha مختارة: {best_model.alpha_:.4f}")
print(f"عدد الميزات غير الصفرية: {sum(best_model.coef_ != 0)} من أصل {X.shape[1]}")
```

خطأ شائع

تحذير المقاييس: لا تطبق Ridge أو ElasticNet على ميزات غير محجمة أبداً! بما أن العقوبة تطبق بالتساوي على مجموع مربعات أو قيم β_j، فإن الميزات ذات المجالات الرقمية الضخمة ستحصل طبيعياً على معاملات صغيرة، فتفلت من العقوبة ظلماً، بينما تُعاقب الميزات الدقيقة وتُباد معاملاتُها. استخدم `StandardScaler` دائماً.

تمارين الفصل التاسع والعشرين

- **ملاحظة**

التمرين ٤.١

على مجموعة بيانات السكري `load_diabetes()`، قارن بين أداء `LinearRegression` و `RidgeCV` باستخدام التحقق المتقاطع الخماسي ومقياس R^2. هل نجح التنظيم في تحسين جودة التعميم، وما مقدار الزيادة؟

التمرين ٤.٢

باستخدام `make_regression`، أنشئ مجموعة بيانات تحوي ١٠٠ عينة و ٨٠ ميزة، بحيث ترتبط الميزات الخمس الأولى فيما بينها بنسبة ارتباط تتجاوز ٠.٩٥. درب `Ridge` و `ElasticNet(l1_ratio=0.8)` عليها وقارن قيم معاملات هذه الميزات الخمس.

التمرين ٤.٣

ارسم مسار المعاملات لمقدر `Ridge` بتغيير α عبر ٥٠ قيمة لوغاريتمية بين 10^{-2} و 10^{4}. لاحظ كيف تتلاشى المعاملات تدريجياً دون أن تتقاطع مع الصفر تماماً.

- **ملاحظة**

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
import numpy as np

X, y = load_diabetes(return_X_y=True)

pipe_ols = make_pipeline(StandardScaler(), LinearRegression())
pipe_ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3,
30)))

score_ols = cross_val_score(pipe_ols, X, y, cv=5, scoring="r2").mean()
score_ridge = cross_val_score(pipe_ridge, X, y, cv=5, scoring="r2").mean()

print(f"متوسط R² لـ OLS:      {score_ols:.4f}")
print(f"متوسط R² لـ RidgeCV: {score_ridge:.4f}")
print(f"فرق التحسن:           {score_ridge - score_ols:+.4f}")
```

ملاحظة

```
import numpy as np
from sklearn.linear_model import Ridge, ElasticNet
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(42)
n_samples = 100
# توليد ميزة مشتركة وإضافة ضجيج خفيف لإنتاج تداخل هائل
base_feat = rng.normal(size=(n_samples, 1))
redundant_block = base_feat + rng.normal(scale=0.05, size=(n_samples, 5))
other_features = rng.normal(size=(n_samples, 75))
X = np.hstack([redundant_block, other_features])
y = redundant_block.sum(axis=1) * 3.0 + rng.normal(scale=1.0, size=n_samples)

X_scaled = StandardScaler().fit_transform(X)

ridge = Ridge(alpha=10.0).fit(X_scaled, y)
enet = ElasticNet(alpha=0.5, l1_ratio=0.8, random_state=42).fit(X_scaled, y)

print("معاملات الميزات الخمس المترابطة في Ridge:")
print(np.round(ridge.coef_[:5], 3))
print("معاملات الميزات الخمس المترابطة في ElasticNet:")
print(np.round(enet.coef_[:5], 3))
```

ملاحظة

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تدريب Ridge بدون تحجيم الميزات	إعفاء الميزات الكبيرة من العقوبة وتصفير الصغيرة قهراً	دمج `StandardScaler` دوماً في `Pipeline` قبل المقدر المنظم
افتراض أن Ridge يقوم باختيار الميزات	المعاملات تقترب من الصفر لكنها لا تساويه أبداً	استخدام Lasso أو ElasticNet إذا أردت إسقاط المتغيرات فعلياً
الاعتماد على القيمة الافتراضية `alpha=1.0` دون ضبط	ملاءمة مفرطة أو تحيز فج بحسب طبيعة كل مشكلة	استخدام `RidgeCV` أو `ElasticNetCV` لتحديد العقوبة المثلى
ضبط `l1_ratio` بالقرب من ١.٠ مع بيانات متداخلة بشدة	فقدان ميزة التكتل واختيار أعمدة متطابقة عشوائياً	الإبقاء على `l1_ratio` متزناً (مثل ٠.٥ إلى ٠.٨) لحماية المجموعات

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التنظيم	إضافة قيد جزائي لدالة الخسارة لمنع تضخم المعاملات	Regularization
انحدار تيكونوف	الاسم الرياضي لانحدار الحافة المعتمد على عقوبة L٢	Ridge regression / Tikhonov
الانحدار الشبكي	نموذج خطي يجمع عقوبة معيار L١ ومعيار L٢ بوزن نسبي	ElasticNet
نسبة معيار L١	معامل التوازن بين عقوبتي لابلاس وغوس داخل ElasticNet	l1_ratio
مسار التنظيم	منحنى تطور قيم المعاملات المقدرة بتغير شدة العقوبة	Regularization path

الجزء ٤ · النماذج الخطية

الفصل ٣٠ — Lasso: النموذج الخطي الذي يُصفّر

التباعد، وهندسة المسار، واختيار alpha عبر التحقق المتقاطع

الوحدة **scikit-learn ١.٩** الحجم **٣٨٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

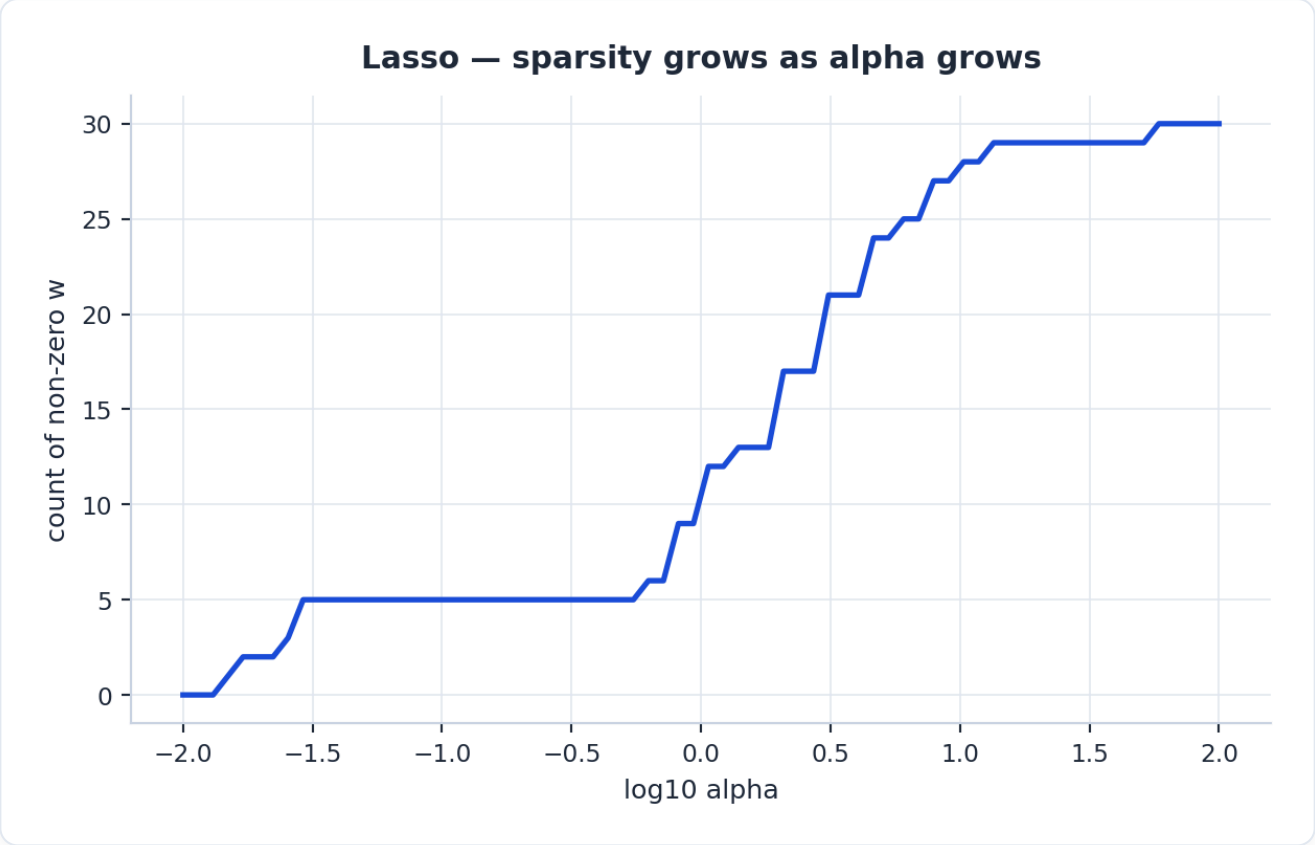
- أن تشرح هندسياً لماذا تؤدي زوايا قيد L١ إلى تصفير دقيق للمعاملات عكس دائرة L٢
- أن تستخدم Lasso لاختزال الخصائص وإنتاج نماذج خطية شحيحة قابلة للتفسير
- أن تحسب مسار التنظيم عبر lasso_path وتفسر ترتيب انضمام الميزات
- أن تضبط معلمات alpha بكفاءة باستخدام LassoCV و LassoLarsCV

٤.١ سحر التصفير: لماذا نحتاج نموذجاً شحيحاً؟

في عصر البيانات الضخمة، قد تجد نفسك أمام جدول يحتوي على ١٠٬٠٠٠ ميزة، لكنك تعلم في قرارة تحليلك أن بضع عشرات منها فقط هي المحرك الحقيقي للهدف. إذا استخدمت Ridge، فإنه سيحتفظ بجميع الميزات العشرة آلاف بأوزان ضئيلة غير صفرية، مما يجعل نشر النموذج وتفسيره أمراً بالغ التعقيد.

هنا يسطع **Lasso (Least Absolute Shrinkage and Selection Operator)**، الذي قدمه روبرت تيبشيراني عام ١٩٩٦، كنموذج خطي يدمج تقدير المعاملات باختيار الخصائص في عملية استمثال واحدة:

$$\min_{\beta} \frac{1}{2n}\|y - X\beta\|_2^2 + \alpha\|\beta\|_1 = \min_{\beta} \frac{1}{2n}\sum_{i=1}^{n}(y_i - x_i^T\beta)^2 + \alpha\sum_{j=1}^{p}|\beta_j|$$



الشكل ٦٥ الهندسة الكامنة وراء التصفير: تقاطع دوائر الخطأ مع زوايا معين L1 الحادة

٤.٢ الهندسة الرياضية: لماذا يصفر معيار L_1؟

السؤال البارز الذي يراود كل مهندس: لماذا يُصفّر قيد القيمة المطلقة $|\beta_j|$ المعاملات، بينما يعجز قيد التربيع β_j^2 عن ذلك؟

السر يكمن في **طبيعة الهندسة المحدبة (Convex Geometry)** لقيد الميزانية:

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# إنشاء بيانات تحوي 50 ميزة، 8 منها فقط دالة حقيقة
X, y = make_regression(n_samples=150, n_features=50, n_informative=8,
                       noise=15.0, random_state=42)

pipe_ols = Pipeline([("scaler", StandardScaler()), ("reg", LinearRegression())])
pipe_lasso = Pipeline([("scaler", StandardScaler()), ("reg", Lasso(alpha=2.5,
random_state=42))])

pipe_ols.fit(X, y)
pipe_lasso.fit(X, y)

coef_ols = pipe_ols.named_steps["reg"].coef_
coef_lasso = pipe_lasso.named_steps["reg"].coef_

print(f"إجمالي الميزات في البيانات: {X.shape[1]}")
print(f"الميزات الصفرية في OLS:     {np.sum(coef_ols == 0)}")
print(f"الميزات الصفرية في Lasso:   {np.sum(coef_lasso == 0)}")
print(f"الميزات المنتقاة في Lasso:  {np.sum(coef_lasso != 0)}")
```

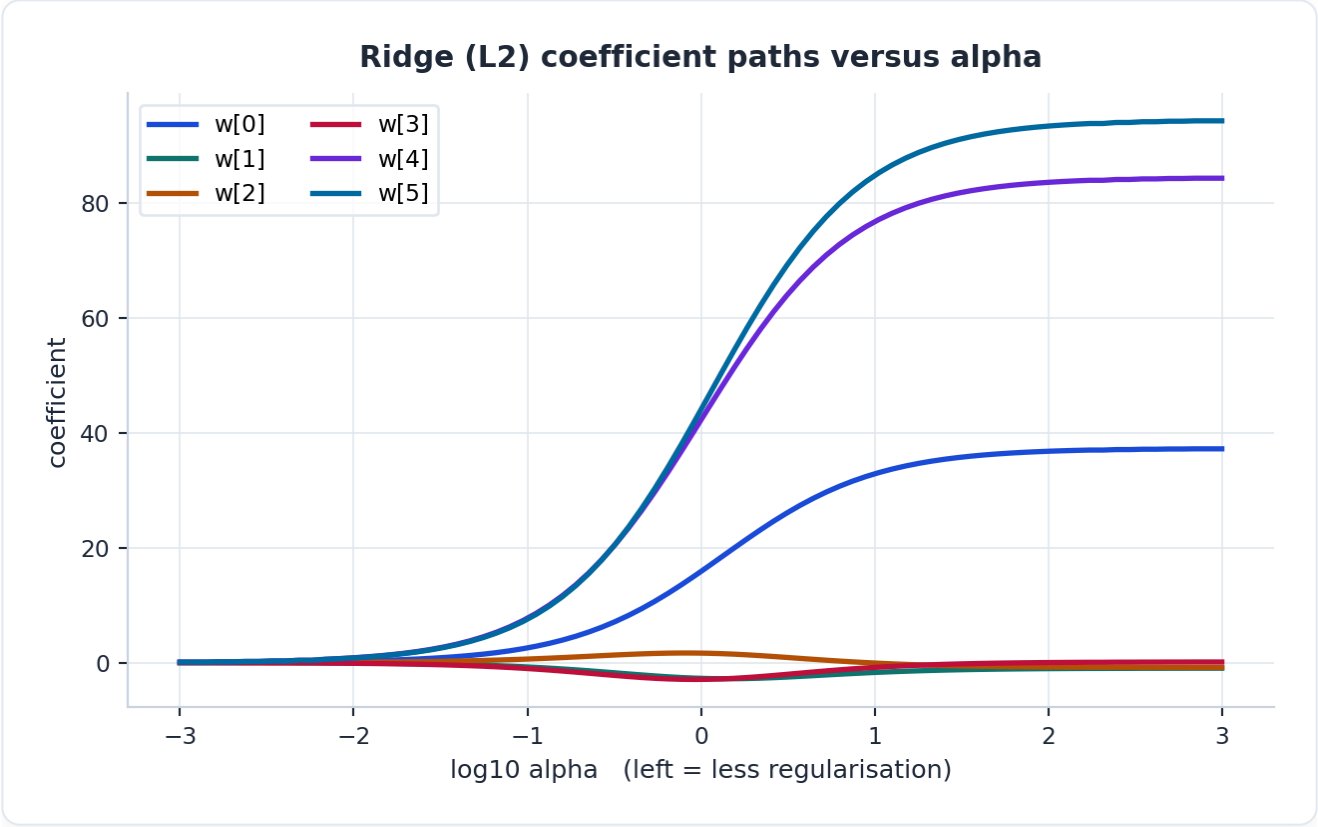
٤.٣ خوارزمية النزول الإحداثي ومسار Lasso

نظراً لأن دالة القيمة المطلقة غير قابلة للاشتقاق عند الصفر، لا يمتلك Lasso حلاً مغلقاً. بدلاً من ذلك، يعتمد scikit-learn على **النزول الإحداثي (Coordinate Descent)** وعامل العتبة اللينة (Soft-Thresholding Operator):

$$S(\beta_j, \alpha) = \text{sign}(\beta_j)\max(0, |\beta_j| - \alpha)$$

يمكننا تتبع كيف تدخل الميزات واحدة تلو الأخرى مع تخفيف قيمة α عبر `lasso_path`:

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import lasso_path
from sklearn.preprocessing import StandardScaler
import numpy as np

X, y = load_diabetes(return_X_y=True)
feature_names = load_diabetes().feature_names
X_scaled = StandardScaler().fit_transform(X)

# استخراج مسار المعاملات بتناقص alpha
alphas, coefs_path, _ = lasso_path(X_scaled, y, alphas=np.logspace(1, -2, 50))

print(f"عدد قيم alpha المختبرة في المسار: {len(alphas)}")
# فحص أول ميزة غير صفرية عند أعلى alpha) أول ميزة انضمت للمسار)
first_nonzero_step = np.argmax(np.sum(coefs_path != 0, axis=0) > 0)
first_active_feature = feature_names[np.argmax(np.abs(coefs_path[:,
first_nonzero_step]))]
print(f"أول ميزة اخترقت الصفر عند أعلى قيمة لعقوبة alpha: {first_active_feature}")
```



الشكل ٦٦ مسارات انحدار المعاملات وتلاشيها بتغير شدة التنظيم في مسار Lasso

٤.٤ التحديد التلقائي لـ alpha عبر LassoCV

لتجنب الإفراط في التصغير (Underfitting) أو الملاءمة المفرطة (Overfitting)، نستخدم `LassoCV` الذي يُجري بحثاً سريعاً ذكياً على شبكة قيم α محسوبة مباشرة من أقصى قيمة تجعل كل المعاملات أصفاراً:

$$\alpha_{\max} = \frac{1}{n} \max_j |x_j^T y|$$

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

X, y = load_diabetes(return_X_y=True)
feature_names = np.array(load_diabetes().feature_names)

pipe_lasso_cv = Pipeline([
    ("scaler", StandardScaler()),
    ("lasso_cv", LassoCV(cv=5, random_state=42, alphas=100))
])

pipe_lasso_cv.fit(X, y)
best_lasso = pipe_lasso_cv.named_steps["lasso_cv"]

selected_mask = best_lasso.coef_ != 0
print(f"أفضل قيمة alpha المتقاطع المختارة بالتحقق: {best_lasso.alpha_:.4f}")
print(f"عدد الميزات النشطة: {np.sum(selected_mask)} من أصل {len(feature_names)}")
print(f"الميزات الأساسية المحتفظ بها: {list(feature_names[selected_mask])}")
```

> **ملاحظة**
>
> يعتمد `LassoCV` على معيار التقارب `tol=1e-4` وعدد التكرارات `max_iter=1000`. في بعض الحالات ذات التعدد الخطي العالي أو القيم غير المعايرة جيداً، قد يظهر تحذير `ConvergenceWarning`؛ حينها ارفع `max_iter=5000` أو اضبط `tol`.

٤.٥ حدود Lasso ومتى تتجنبه

رغم قوته، يعاني Lasso من محددات هيكلية هامة:

تمارين الفصل الثلاثين

- **ملاحظة**

التمرين ٤.١

على مجموعة بيانات California Housing، قارن بين عدد الميزات غير الصفرية الناتجة عن `Lasso(alpha=0.1)` و `Lasso(alpha=0.01)` بعد تحجيم الميزات بـ `StandardScaler`. ما الميزات التي تم تصفيرها أولاً؟

التمرين ٤.٢

أنشئ مجموعة بيانات عبر `make_regression` بعدد ٢٠٠ عينة و ١٠٠ ميزة (١٠ ميزات حقيقية فقط). قارن مقياس R^2 ومجموع المعاملات غير الصفرية بين `LinearRegression` و `LassoCV` على مجموعة اختبار مستقلة بنسبة ٣٠٪.

التمرين ٤.٣

استخدم دالة `lasso_path` على مجموعة بيانات مصطنعة ذات ميزتين متطابقتين تماماً. ماذا تلاحظ في سلوك مسار المعاملات لكل منهما؟

- **ملاحظة**

```python
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
import numpy as np

data = fetch_california_housing()
X, y = data.data, data.target
names = np.array(data.feature_names)

X_sc = StandardScaler().fit_transform(X)

for alpha in [0.1, 0.01]:
    model = Lasso(alpha=alpha, random_state=42).fit(X_sc, y)
    active = names[model.coef_ != 0]
    zeroed = names[model.coef_ == 0]
    print(f"عند alpha = {alpha}:")
    print(f"  الميزات المتبقية ({len(active)}): {list(active)}")
    print(f"  الميزات المصفرة ({len(zeroed)}): {list(zeroed)}")
```

ملاحظة

```python
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
import numpy as np

X, y = make_regression(n_samples=200, n_features=100, n_informative=10,
                       noise=15.0, random_state=42)

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

pipe_ols = make_pipeline(StandardScaler(), LinearRegression()).fit(X_tr, y_tr)
pipe_lasso = make_pipeline(StandardScaler(), LassoCV(cv=5,
random_state=42)).fit(X_tr, y_tr)

r2_ols = pipe_ols.score(X_te, y_te)
r2_lasso = pipe_lasso.score(X_te, y_te)
active_lasso = np.sum(pipe_lasso.named_steps["lassocv"].coef_ != 0)

print(f"OLS - R² الاختبار:     {r2_ols:.4f} (جميع الميزات 100 مستخدمة)")
print(f"LassoCV - R² الاختبار: {r2_lasso:.4f} (الميزات المستخدمة: {active_lasso})")
```

ملاحظة

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إغفال المعايرة بـ `StandardScaler` قبل Lasso	تصفير الميزات الدقيقة وإبقاء الميزات ذات المدى الواسع قسراً	تضمين `StandardScaler` دائماً في `Pipeline` قبل Lasso
تجاهل تحذيرات عدم التقارب `ConvergenceWarning`	توقف الخوارزمية قبل الوصول للحل الشحيح الدقيق	رفع `max_iter` إلى ٥٠٠٠ أو ضبط `tol` أو زيادة قيمة `alpha`
توقع أن يختار Lasso ميزات مترابطة معاً	اختيار ميزة واحدة عشوائياً وتصفير شركائها المتطابقين	استخدام `ElasticNet` عند وجود ارتباطات متبادلة وثيقة
استخدام قيمة `alpha` عشوائية دون اختبار	إما إفراط في التصفير وفقدان الإشارة أو إبقاء كل الميزات	استخدام `LassoCV` لضبط α بالتحقق المتقاطع تلقائياً

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
نموذج لاسو	انحدار المربعات الصغرى المقيد بعقوبة معيار L1 المطلق	Lasso
الشح أو التناثر	خاصية النموذج التي تكون فيها معظم أوزانه أصفاراً تامة	Sparsity
النزول الإحداثي	خوارزمية استمثال تحدث كل معامل منفرداً مع تثبيت البقية	Coordinate descent
عامل العتبة اللينة	تحويل رياضي يقلص القيمة نحو الصفر ويثبتها عنده	Soft-thresholding operator
مسار لاسو	تطور قيم معاملات الميزات كدالة مستمرة في قيمة العقوبة	Lasso path

الجزء ٤ · النماذج الخطية

الفصل ٣١ — الانحدار المرن والمتعدد المهام

lars_path g enet_path g MultiTaskElasticNet g MultiTaskLasso

الوحدة **scikit-learn ١.٩** الحجم **٤٣٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

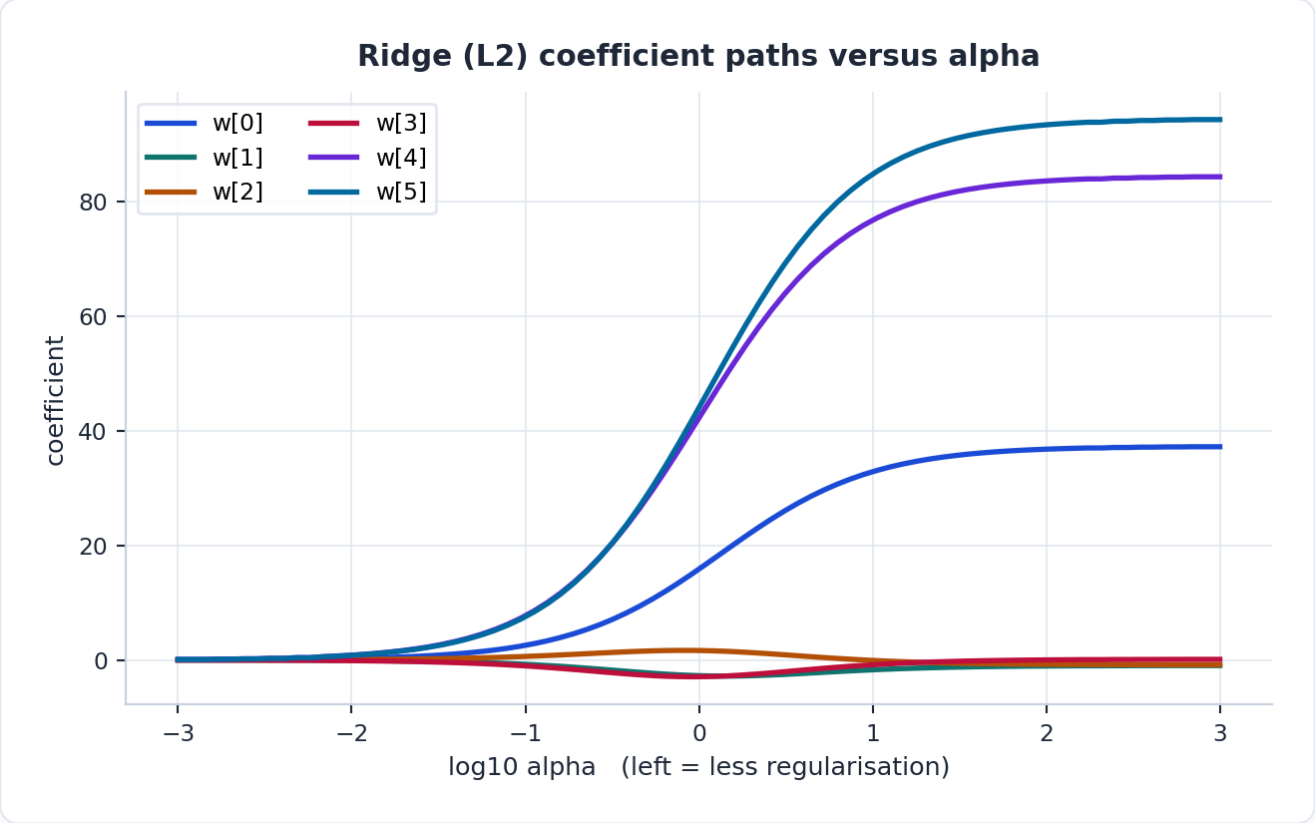
- أن تفهم بنية عقوبة L٢،١ المشتركة وكيف تجبر الميزات على التصفير التزامني عبر كل الأهداف
- أن تطبق MultiTaskLasso و MultiTaskElasticNet على مصفوفات الأهداف الثنائية والمستمرة
- أن تستخرج مسارات المعاملات الدقيقة عبر enet_path لمقارنة استجابة الأوزان
- أن تستخدم lars_path (انحدار الزوايا الأقل) لفهم الهندسة الخطية للمسار الحسابي

٣١.١ التنبؤ المشترك: لماذا الانحدار متعدد المهام؟

في العديد من التطبيقات العملية، لا نريد التنبؤ بقيمة عددية واحدة $y \in \mathbb{R}$، بل بمتجه كامل من الاستجابات المترابطة $Y \in \mathbb{R}^{n \times k}$. فكر في:

إذا دربت نموذج Lasso مستقلاً لكل عمود من أعمدة Y، فستواجه معضلة تفسيرية: قد يختار النموذج الأول الميزات رقم ٢ و ٥، بينما يختار النموذج الثاني لنفس الظاهرة الميزات ٣ و ٨!

هنا يظهر **الانحدار متعدد المهام (Multi-Task Learning)** ليفرض **شحاً مشتركاً (Joint Sparsity)**: إما أن تختار الميزة لجميع الأهداف معاً، أو تُصفّر لجميع الأهداف معاً!



الشكل ٦٧ المسارات المقترنة وتطور معاملات الميزات المتعددة في الفضاء الخطي

٤.٢ عقوبة $L_{2,1}$ وسر التصفير المشترك

لتحقيق هذا التصفير التزامني، يستبدل `MultiTaskLasso` عقوبة L_1 التقليدية بعقوبة **معيار** $L_{2,1}$ **(Frobenius-L1 mixed norm)**:

$$\min_W \frac{1}{2n}\|Y - XW\|_F^2 + \alpha\|W\|_{2,1}$$

حيث:

هذا الجمع الرائع يطبق قيد L_2 الدائري داخل كل صف (فيوزع الأوزان بسلاسة بين الأهداف)، ويطبق قيد L_1 ذا الرؤوس الحادة على مجموع الصفوف، فيصفر الصف بأكمله بضربة واحدة!

```python
import numpy as np
from sklearn.linear_model import MultiTaskLasso, Lasso

rng = np.random.default_rng(42)
n_samples, n_features, n_tasks = 120, 30, 4

# مصفوفة بيانات مشتركة
X = rng.normal(size=(n_samples, n_features))

# 4 ميزات فقط هي الحقيقية المؤثرة في كل الأهداف
true_W = np.zeros((n_features, n_tasks))
true_W[[2, 7, 14, 21], :] = rng.uniform(2.0, 5.0, size=(4, n_tasks))
Y = X @ true_W + rng.normal(scale=0.5, size=(n_samples, n_tasks))

# تدريب MultiTaskLasso
mt_lasso = MultiTaskLasso(alpha=0.3, random_state=42).fit(X, Y)

# فحص الصفوف الصفرية
row_norms = np.linalg.norm(mt_lasso.coef_, axis=1)
active_features = np.where(row_norms > 0)[0]

print(f"الميزات الحقيقية المنشأة: [21 ,14 ,7 ,2]")
print(f"الميزات المنتقاة جماعياً بواسطة MultiTaskLasso: {list(active_features)}")
print(f"أبعاد مصفوفة المعاملات coef_: {mt_lasso.coef_.shape}")
```

٤.٣ MultiTaskElasticNet: الحماية من التعدد الخطي

تماماً كما يفعل `ElasticNet` في الحالة أحادية الهدف، يضيف `MultiTaskElasticNet` عقوبة فروبينيوس تربيعية $\|W\|_F^2$ لعقوبة $L_{2,1}$ لمنع استبعاد الميزات المشتركة في حالات التعدد الخطي الشديد:

$$\min_W \frac{1}{2n}\|Y - XW\|_F^2 + \alpha\rho\|W\|_{2,1} + \frac{\alpha(1-\rho)}{2}\|W\|_F^2$$

```python
from sklearn.linear_model import MultiTaskElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

rng = np.random.default_rng(42)
X = rng.normal(size=(100, 25))
# إنشاء ميزتين مكررتين بشدة
X[:, 1] = X[:, 0] + rng.normal(scale=0.01, size=100)

true_W = np.zeros((25, 3))
true_W[0, :] = [3.0, 2.5, 4.0]
true_W[1, :] = [2.0, 3.5, 1.5]
Y = X @ true_W + rng.normal(scale=0.2, size=(100, 3))

mt_enet = MultiTaskElasticNet(alpha=0.2, l1_ratio=0.7, random_state=42)
mt_enet.fit(X, Y)

print("معاملات الميزة 0 عبر الأهداف الثلاثة:", np.round(mt_enet.coef_[0], 3))
print("معاملات الميزة 1 عبر الأهداف الثلاثة:", np.round(mt_enet.coef_[1], 3))
print("كلا الميزتين تم الإبقاء عليهما معاً دون إقصاء تعسفي!")
```

٤.٤ استخراج مسار الانحدار الشبكي عبر enet_path

توفر دالة `enet_path` وسيلة منخفضة المستوى لحساب مسار المعاملات كاملاً عبر شبكة من قيم α بكفاءة قصوى باستخدام خوارزميات C المكتوبة داخل scikit-learn.

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import enet_path
from sklearn.preprocessing import StandardScaler
import numpy as np

X, y = load_diabetes(return_X_y=True)
X_scaled = StandardScaler().fit_transform(X)

# استخراج مسار المعاملات لـ ElasticNet
alphas, coefs, dual_gaps = enet_path(
    X_scaled, y, l1_ratio=0.8, alphas=np.logspace(0, -3, 30), random_state=42
)

print(f"شكل مصفوفة المسار (الميزات x نقاط alpha): {coefs.shape}")
print(f"أعلى قيمة alpha (تصفير شبه تام): {alphas[0]:.4f}")
print(f"أدنى قيمة alpha (اقتراب من الحل الحر): {alphas[-1]:.4f}")
print(f"عدد الميزات غير الصفرية عند أدنى alpha: {np.sum(coefs[:, -1] != 0)}")
```

٤.٥ انحدار الزوايا الأقل: خوارزمية LARS و lars_path

خوارزمية **LARS (Least Angle Regression)** التي ابتكرها إفرون وتيبشيراني وهستي وسبالدينغ عام ٢٠٠٤، هي إحدى أبدع الخوارزميات الهندسية في تعلّم الآلة:

الشكل ٦٨ توزيع معاملات الميزات الخطية والترابط البيني

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import lars_path
from sklearn.preprocessing import StandardScaler
import numpy as np

X, y = load_diabetes(return_X_y=True)
feature_names = load_diabetes().feature_names
X_scaled = np.asarray(StandardScaler().fit_transform(X))
y_arr = np.asarray(y)

# حساب مسار LARS الدقيق هندسياً
alphas, active, coefs = lars_path(X_scaled, y_arr, method="lasso")

print("ترتيب دخول الميزات في مسار LARS التدريجي:")
for step, feat_idx in enumerate(active[:5]):
    print(f"  الخطوة {step+1}: الميزة '{feature_names[feat_idx]}'")
```

🏭 في الواقع

متى تفضل `enet_path` على `lars_path`؟ تعتمد خوارزمية LARS على عمليات المصفوفات الخطية الصريحة، وتصبح غير مستقرة عددياً وبطيئة إذا كانت البيانات تحوي ضجيجاً كبيراً أو أبعاداً هائلة الحجم ($p \gg n$). في المقابل، فإن خوارزمية **النزول الإحداثي (Coordinate Descent)** المستخدمة في `enet_path` و `lasso_path` أكثر متانة وسرعة بمراحل في مجموعات البيانات الكبيرة والمتفرقة (Sparse Data).

تمارين الفصل الحادي والثلاثين

- **ملاحظة**

التمرين ٤.١

أنشئ مصفوفة استجابات تحوي ٣ أهداف مترابطة مستخلصة من بيانات `fetch_california_housing` (مثلاً: أسعار المنازل، ومتوسط عدد الغرف، ومتوسط الدخل). طبق `MultiTaskLasso` وقارن الميزات المختارة بتلك الناتجة عن تطبيق ٣ نماذج منفصلة من `Lasso`.

التمرين ٤.٢

باستخدام `lars_path(method="lar")`، احسب المسار لبيانات تحوي ١٠ ميزات. تحقق من أن عدد الخطوات الحسابية يساوي بالضبط رتبة مصفوفة البيانات، واشرح دلالة ذلك مقارنة بالنزول التكراري.

التمرين ٤.٣

قارن بين زمن تنفيذ حساب ١٠٠ نقطة لمسار المعاملات باستخدام `lasso_path` مقابل `lars_path(method="lasso")` على مصفوفة اصطناعية ذات ١٠٠٠ عينة و ٢٠٠ ميزة. أي الطريقتين كانت أسرع ولماذا؟

- **ملاحظة**

```python
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import MultiTaskLasso, Lasso
from sklearn.preprocessing import StandardScaler
import numpy as np

housing = fetch_california_housing()
X = np.asarray(StandardScaler().fit_transform(housing.data[:500]))
# 3 أهداف متعددة
Y = np.asarray(StandardScaler().fit_transform(
    np.column_stack([housing.target[:500], housing.data[:500, 0],
housing.data[:500, 5]])
))

# نموذج متعدد المهام
mt = MultiTaskLasso(alpha=0.1, random_state=42).fit(X, Y)
mt_selected = set(np.where(np.linalg.norm(mt.coef_, axis=1) > 0)[0])

# نماذج منفصلة
sep_selected = set()
for task in range(Y.shape[1]):
    l = Lasso(alpha=0.1, random_state=42).fit(X, Y[:, task])
    sep_selected.update(np.where(l.coef_ != 0)[0])

print(f"الميزات المنتقاة جماعياً بواسطة MultiTaskLasso ({len(mt_selected)}): {mt_selected}")
print(f"اتحاد الميزات المنتقاة عبر النماذج المنفصلة ({len(sep_selected)}): {sep_selected}")
```

> **ملاحظة**

```python
import time
import numpy as np
from sklearn.linear_model import lasso_path, lars_path

rng = np.random.default_rng(42)
X = rng.normal(size=(600, 150))
y = X[:, :10].sum(axis=1) + rng.normal(size=600)

t0 = time.perf_counter()
_ = lasso_path(X, y, alphas=np.logspace(0, -3, 40))
t_cd = time.perf_counter() - t0

t0 = time.perf_counter()
_ = lars_path(X, y, method="lasso")
t_lars = time.perf_counter() - t0

print(f"زمن النزول الإحداثي (lasso_path): {t_cd * 1000:.2f} ms")
print(f"زمن الزوايا الأقل (lars_path):    {t_lars * 1000:.2f} ms")
```

> **ملاحظة**

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تمرير مصفوفة أهداف Y غير متجانسة المقاييس	هيمنة الهدف ذي القيم الضخمة على عقوبة $L_{2,1}$ بالكامل	معايرة مصفوفة الأهداف Y بـ `StandardScaler` قبل التدريب
تدريب نماذج Lasso مستقلة وتوقع الحصول على ميزات موحدة	تباين الميزات المختارة بين الأهداف وفقدان التناسق	استخدام `MultiTaskLasso` لفرض انتقاء صفي متطابق
استخدام `lars_path` على مصفوفات ضخمة جداً أو متفرقة	بطء حسابي شديد واحتمال عدم استقرار الحلول	تفضيل `enet_path` أو `lasso_path` المعتمدة على النزول الإحداثي
الخلط بين `coef_` في النماذج الفردية والمتعددة المهام	شكل المصفوفة يكون `(n_features, n_tasks)` بدلاً من متجه	التحقق من أبعاد المعاملات دائماً عند استخراج الأوزان

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التعلّم متعدد المهام	تدريب نموذج مشترك للتنبؤ بعدة متغيرات استجابة متزامنة	Multi-task learning
معيار فروبينيوس-لابلاس	معيار $L_{2,1}$ المشترك المطبق على صفوف مصفوفة المعاملات	mixed norm $L_{2,1}$
الشح المشترك	تصفير صفوف المعاملات بالكامل عبر جميع المهام	Joint sparsity
انحدار الزوايا الأقل	خوارزمية هندسية تسير في اتجاه الزوايا المتساوية بين المتغيرات	Least Angle Regression (LARS)
مسار المعاملات	سلسلة الحلول المتصلة الناتجة عن تغيير معامل التنظيم	Regularization path

الجزء ٤ · النماذج الخطية

الفصل ٣٢ — الانحدار المتسامح وتحمّل الصفوف الشاذة

Huber و TheilSen و RANSAC و Quantile على بيانات ملوّثة

الوحدة **scikit-learn ١.٩** الحجم **٣٧٧** كلمة تقريباً

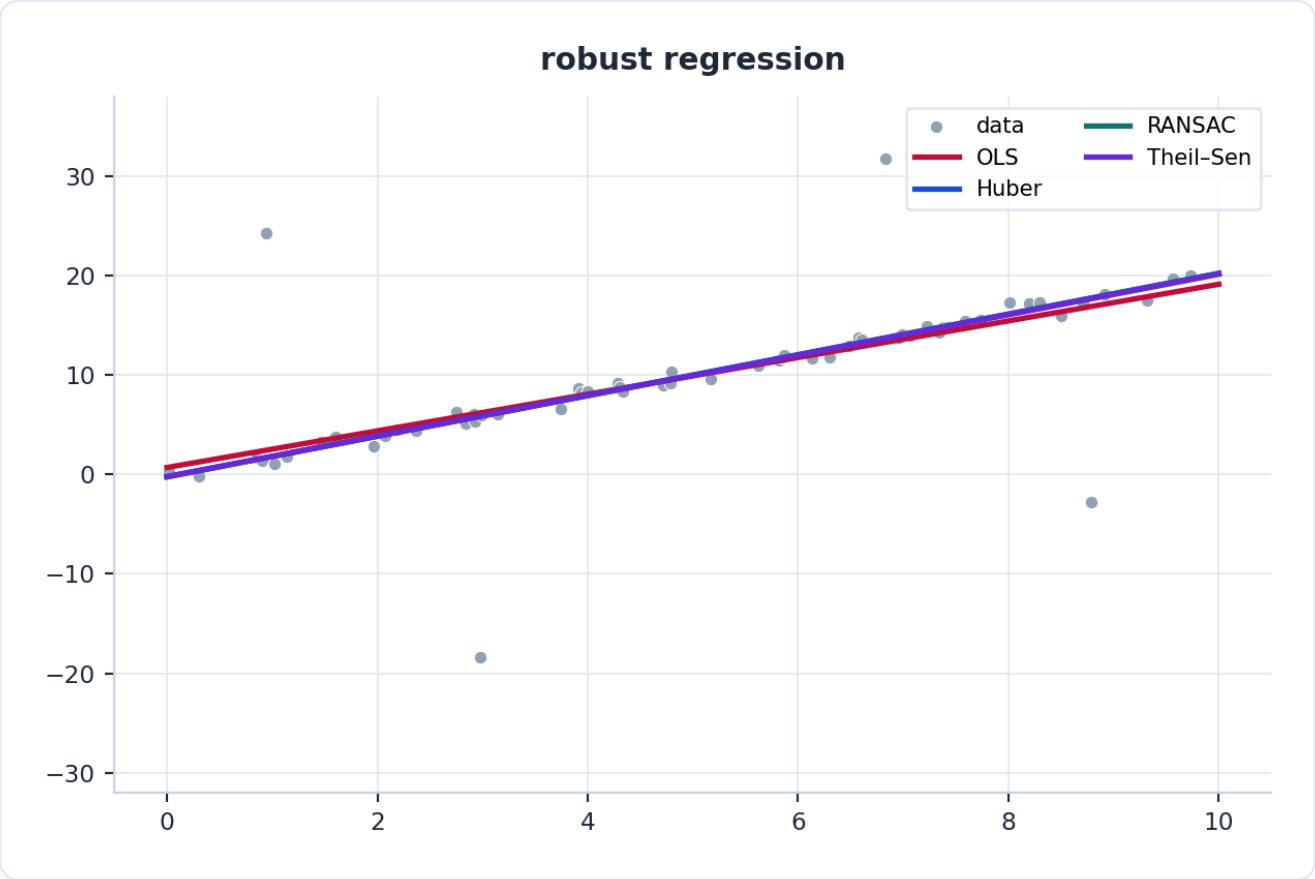
ما الذي ستتقنه في هذا الفصل

- أن تشرح نقطة الانهيار (Breakdown Point) ولماذا تفشل المربعات الصغرى أمام الشواذ
- أن تطبق HuberRegressor لمعالجة الشواذ في مساحة الخطأ دون إسقاط الصفوف
- أن تستخدم RANSACRegressor لعزل البيانات الشاذة وتحديد خط الأساس النقي
- أن توظف TheilSenRegressor للتقدير الوسيط المقاوم و QuantileRegressor لنمذجة المئينات

٤.١ نقطة الانهيار: كابوس المربعات الصغرى

في نموذج المربعات الصغرى العادي (OLS)، تتضاعف عقوبة الخطأ بالتربيع e_i^2 . إذا احتوت بياناتك على قيمة شاذة واحدة خاطئة (مثلاً خطأ في إدخال أصفار إضافية لراتب موظف أو قراءة معطلة لحساس حراري)، فإن هذه النقطة الشاذة ذات الرافعة العالية (High-Leverage Outlier) ستقوم بـ «ليّ» خط الانحدار كاملاً نحوها، مضحية بدقة ملاءمة مئات العينات السليمة!

في الإحصاء المتسامح، تُقاس مناعة النموذج بمفهوم **نقطة الانهيار (Breakdown Point)**:



الشكل ٦٩ مقارنة استجابة خطوط الانحدار المختلفة بوجود نقاط شاذة ملوثة

٤.٢ HuberRegressor: التوفيق العبقري بين L_2 و L_1

دالة خسارة هوبر (Huber Loss) تتبنى نهجاً ذكياً:

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & \text{for} |e| \le \delta \\ \delta(|e| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

يقوم مقدر `HuberRegressor` بتحسين معلمة المقياس σ تلقائياً مع المعاملات.

```python
import numpy as np
from sklearn.linear_model import LinearRegression, HuberRegressor
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(42)
n_samples = 100
X = rng.uniform(-5, 5, size=(n_samples, 1))
y = 2.5 * X.ravel() + rng.normal(scale=1.0, size=n_samples)

# إضافة 10% نقاط شاذة عنيفة (تلوث)
outlier_idx = rng.choice(n_samples, size=10, replace=False)
y[outlier_idx] += rng.choice([-30.0, 30.0], size=10)

ols = LinearRegression().fit(X, y)
huber = HuberRegressor(epsilon=1.35).fit(X, y)

print(f"الميل الحقيقي: 2.500")
print(f"ميل OLS (المربعات الصغرى): {ols.coef_[0]:.3f} (تأثر بشدة بالشواذ!)")
print(f"ميل Huber المتسامح:         {huber.coef_[0]:.3f} (مقاوم ومقارب للحقيقة)")
print(f"عدد العينات التي اعتبرها هوبر شاذة: {np.sum(huber.outliers_)}")
```

٤.٣ RANSAC: التوافق العشوائي والتطهير التام

خوارزمية **RANSAC (Random Sample Consensus)** تعمل بطريقة استقرائية مبهرة:

```python
from sklearn.linear_model import RANSACRegressor, LinearRegression
import numpy as np

rng = np.random.default_rng(42)
X = np.linspace(-10, 10, 150).reshape(-1, 1)
y = 1.8 * X.ravel() + rng.normal(scale=1.5, size=150)
# تلويث 25 عينة
y[rng.choice(150, 25, replace=False)] += rng.normal(scale=40.0, size=25)

ransac = RANSACRegressor(estimator=LinearRegression(), min_samples=5,
                         residual_threshold=5.0, random_state=42)
ransac.fit(X, y)

inliers = ransac.inlier_mask_
outliers = ~inliers

print(f"إجمالي العينات: {len(X)}")
print(f"العينات السليمة المعتمدة (Inliers): {np.sum(inliers)}")
print(f"العينات الشاذة المكتشفة والمعزولة (Outliers): {np.sum(outliers)}")
print(f"ميل خط RANSAC النقي: {ransac.estimator_.coef_[0]:.3f}")
```

٤.٤ TheilSenRegressor: وسيط الميول غير البارامتري

مقدر ثيل-سين (Theil-Sen Estimator) هو طريقة غير بارامترية عبقرية:

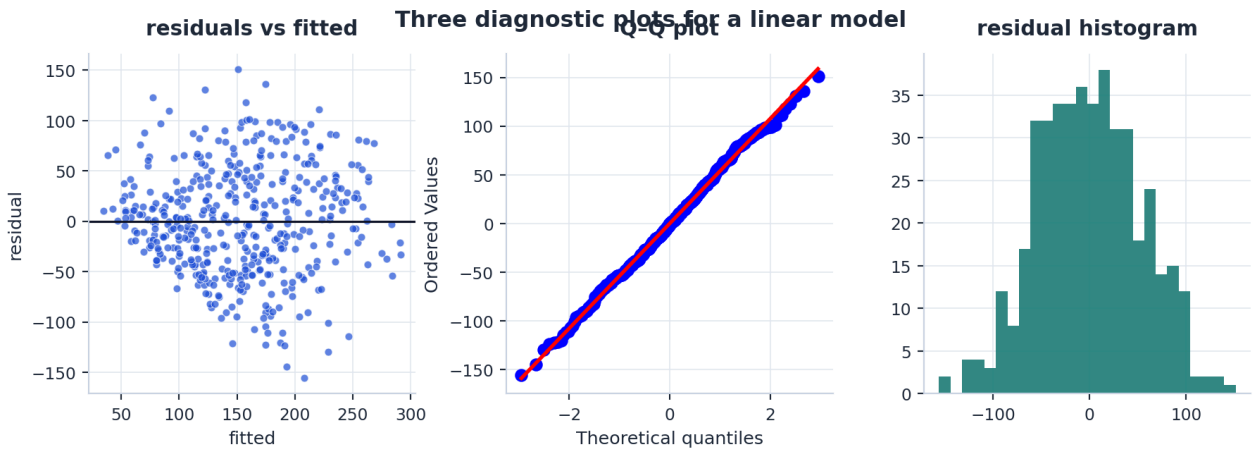
تصل نقطة انهيار Theil-Sen إلى ٢٩.٣% في الفضاءات ثنائية الأبعاد، ويتميز بمناعة استثنائية ضد الشواذ في متغيرات الإدخال X (الشواذ ذات الرافعة العالية).

```python
from sklearn.linear_model import TheilSenRegressor
import numpy as np

rng = np.random.default_rng(42)
X = rng.normal(size=(80, 1))
y = 3.2 * X.ravel() + rng.normal(scale=0.8, size=80)

# تلويث X و y معاً برافعة شاذة قوية
X[0, 0] = 12.0
y[0] = -50.0

theil_sen = TheilSenRegressor(random_state=42).fit(X, y)
print(f"ميل Theil-Sen بعد التلويث برافعة شاذة: {theil_sen.coef_[0]:.3f} (3.200 :الحقيقي)")
```

٤.٥ QuantileRegressor: نمذجة المئينات والوسيط الشرطي

بينما تتنبأ النماذج التقليدية بـ **المتوسط الشرطي** $\mathbb{E}[y|X]$، يتنبأ انحدار المئينات `QuantileRegressor` بـ **المئين الشرطي** q.

باستخدام دالة خسارة الشق النطاقي (Pinball Loss):



الشكل ٧٠ تشخيص المتبقيات وتوزيع الأخطاء تحت النماذج المختلفة

```python
from sklearn.linear_model import QuantileRegressor
import numpy as np

rng = np.random.default_rng(42)
X = rng.uniform(0, 10, size=(100, 1))
# تباين يتسع تدريجياً (Heteroscedasticity)
y = 2.0 * X.ravel() + rng.normal(scale=0.5 * X.ravel() + 0.1, size=100)

qr_median = QuantileRegressor(quantile=0.5, alpha=0.0).fit(X, y)
qr_upper = QuantileRegressor(quantile=0.9, alpha=0.0).fit(X, y)

print(f"الميل عند الوسيط (quantile=0.5): {qr_median.coef_[0]:.3f}")
print(f"الميل عند المئين 90% (quantile=0.9): {qr_upper.coef_[0]:.3f}")
```

نصيحة

مقدر `QuantileRegressor` يعتمد على خوارزميات البرمجة الخطية (Linear Programming) لحل دالة الخسارة غير الملساء، لذا يكون أبطأ في الأبعاد العالية جداً مقارنة بـ `HuberRegressor`. استخدم Huber إذا كنت تريد سرعة ومقاومة للشواذ، واستخدم QuantileRegressor إذا كنت مهتماً بالمئينات المحددة أو الوسيط الشرطي الدقيق.

تمارين الفصل الثاني والثلاثين

- **ملاحظة**

التمرين ٤.١

على مجموعة بيانات تحوي ٢٠٠ عينة، أدخل عمداً ٣٠ عينة شاذة عشوائية بنسبة خطأ فادحة. قارن مقياس MAE (الخطأ المطلق) لكل من `LinearRegression` و `HuberRegressor` و `RANSACRegressor` على مصفوفة اختبار نقية خالية من الشواذ.

التمرين ٤.٢

باستخدام `QuantileRegressor`، درب نموذجين عند `quantile=0.1` و `quantile=0.9` لإنشاء شريط ثقة تنبؤي بنسبة ٨٠% على بيانات أسعار المنازل. احسب النسبة المئوية للعينات الفعلية التي وقعت داخل هذا الشريط.

التمرين ٤.٣

قارن بين زمن تدريب `TheilSenRegressor` و `HuberRegressor` عندما يتضاعف عدد العينات من ١٠٠ إلى ١,٠٠٠ عينة. فسر السبب الرياضي لتباين زمن التنفيذ بينهما.

- **ملاحظة**

```python
import numpy as np
from sklearn.linear_model import LinearRegression, HuberRegressor,
RANSACRegressor
from sklearn.metrics import mean_absolute_error

rng = np.random.default_rng(42)
X_clean = rng.uniform(-5, 5, size=(200, 1))
y_clean = 3.0 * X_clean.ravel() + rng.normal(scale=1.0, size=200)

# نسخة ملوثة للتدريب
X_train = X_clean.copy()
y_train = y_clean.copy()
bad_idx = rng.choice(200, 30, replace=False)
y_train[bad_idx] += rng.uniform(40, 80, size=30)

# بيانات اختبار نقية تماماً
X_test = rng.uniform(-5, 5, size=(100, 1))
y_test = 3.0 * X_test.ravel() + rng.normal(scale=1.0, size=100)

ols = LinearRegression().fit(X_train, y_train)
huber = HuberRegressor().fit(X_train, y_train)
ransac = RANSACRegressor(random_state=42).fit(X_train, y_train)

print(f"MAE لـ OLS على الاختبار النقي:    {mean_absolute_error(y_test,
ols.predict(X_test)):.3f}")
print(f"MAE لـ Huber على الاختبار النقي:  {mean_absolute_error(y_test,
huber.predict(X_test)):.3f}")
print(f"MAE لـ RANSAC على الاختبار النقي: {mean_absolute_error(y_test,
ransac.predict(X_test)):.3f}")
```

> **ملاحظة**

```python
import numpy as np
from sklearn.linear_model import QuantileRegressor
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
# استخدام 300 عينة وميزة متوسط الدخل فقط للإيضاح السريع
X = np.asarray(data.data[:300, [0]])
y = np.asarray(data.target[:300])

q_low = QuantileRegressor(quantile=0.1, alpha=0.0).fit(X, y)
q_high = QuantileRegressor(quantile=0.9, alpha=0.0).fit(X, y)

y_low = q_low.predict(X)
y_high = q_high.predict(X)

inside = np.mean((y >= y_low) & (y <= y_high))
print(f"نسبة البيانات الواقعة داخل شريط المئينات [%10 ,%90]: {inside:.1%}")
print("النسبة النظرية المتوقعة: 80.0%")
```

> 📝 **ملاحظة**

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
حذف الشواذ يدوياً بناءً على حدس غير ممنهج	إفساد التوزيعات الإحصائية وحذف عينات حقيقية نادرة	استخدام مقدرات متسامحة مثل `Huber` أو `RANSAC` آلياً
الاعتماد على RANSAC مع نسبة تلويث تتجاوز ٥٠%	فشل العثور على إجماع سليم وتدهور النموذج	تقييد استخدام RANSAC أو ضبط `max_trials` و `residual_threshold`
تطبيق TheilSen على مجموعات بيانات ضخمة جداً	استهلاك حسابي تربيعي $O(n^2)$ أو تكعيبي	استخدام `HuberRegressor` كبديل سريع في البيانات الضخمة
نسيان معايرة الميزات قبل `HuberRegressor`	اضطراب حساب المقياس σ وتحديد غير دقيق لعتبة الشواذ	استخدام `StandardScaler` دوماً داخل `Pipeline`

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الانحدار المتسامح	نماذج إحصائية لا تتأثر تقديراتها بوجود العينات الشاذة	Robust regression
نقطة الانهيار	النسبة المئوية الدنيا من العينات الملوثة التي تُفسد التقدير تماماً	Breakdown point
خسارة هوبر	دالة خسارة هجينة بين التربيعي والخطي تخفف أثر الانحرافات	Huber loss
التوافق العشوائي	خوارزمية تبني النموذج على مجموعات جزئية خالية من الشواذ	RANSAC
انحدار المئينات	نمذجة المئين الشرطي لمتغير الهدف بدلاً من المتوسط	Quantile regression

الجزء ٤ · النماذج الخطية

الفصل ٣٣ — الانحدار حسب التوزيع

Poisson و Gamma و Tweedie — متى يتفوّق على النموذج الخطي

الوحدة **scikit-learn ١.٩** الحجم **٣٥٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تدرك قيود الانحدار الخطي عند نمذجة بيانات العد والكميات الموجبة الصارمة
- أن تفهم دور دالة الربط (Link Function) ودوال الانحراف (Deviance Losses)
- أن تطبق PoissonRegressor لبيانات العد والأحداث النادرة
- أن تستخدم GammaRegressor للمدد الزمنية والمطالبات و TweedieRegressor للبيانات ذات التراكم الصفري

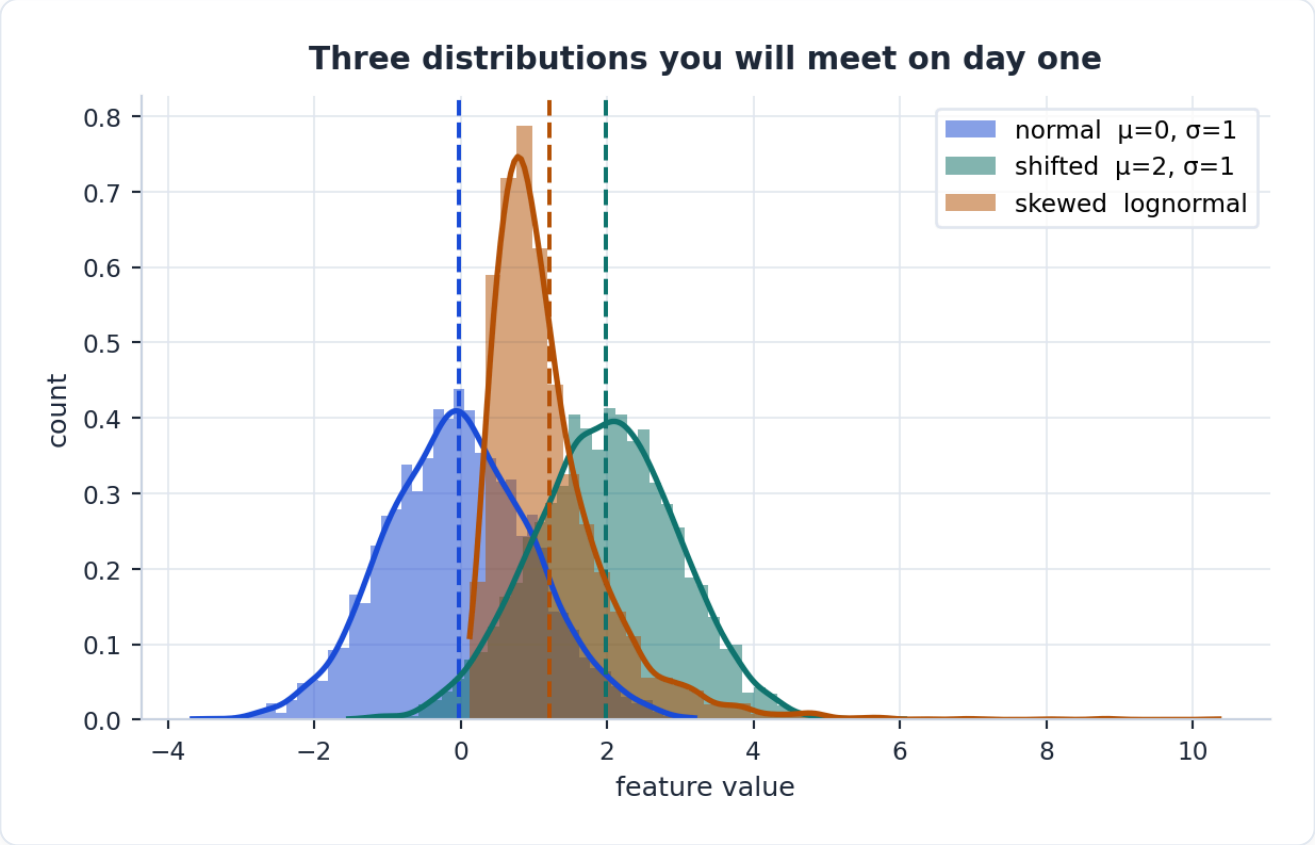
٤.١ ما وراء التوزيع الطبيعي: حدود OLS

يفترض نموذج المربعات الصغرى العادي (OLS) ضمناً أن الأخطاء تتبع توزيعاً طبيعياً متماثلاً بتباين ثابت:

$$y \sim \mathcal{N}(\mu, \sigma^2)$$

لكن في العالم الواقعي، نصادف نوعين بارزين من البيانات ينتهكان هذا الافتراض جوهرياً:

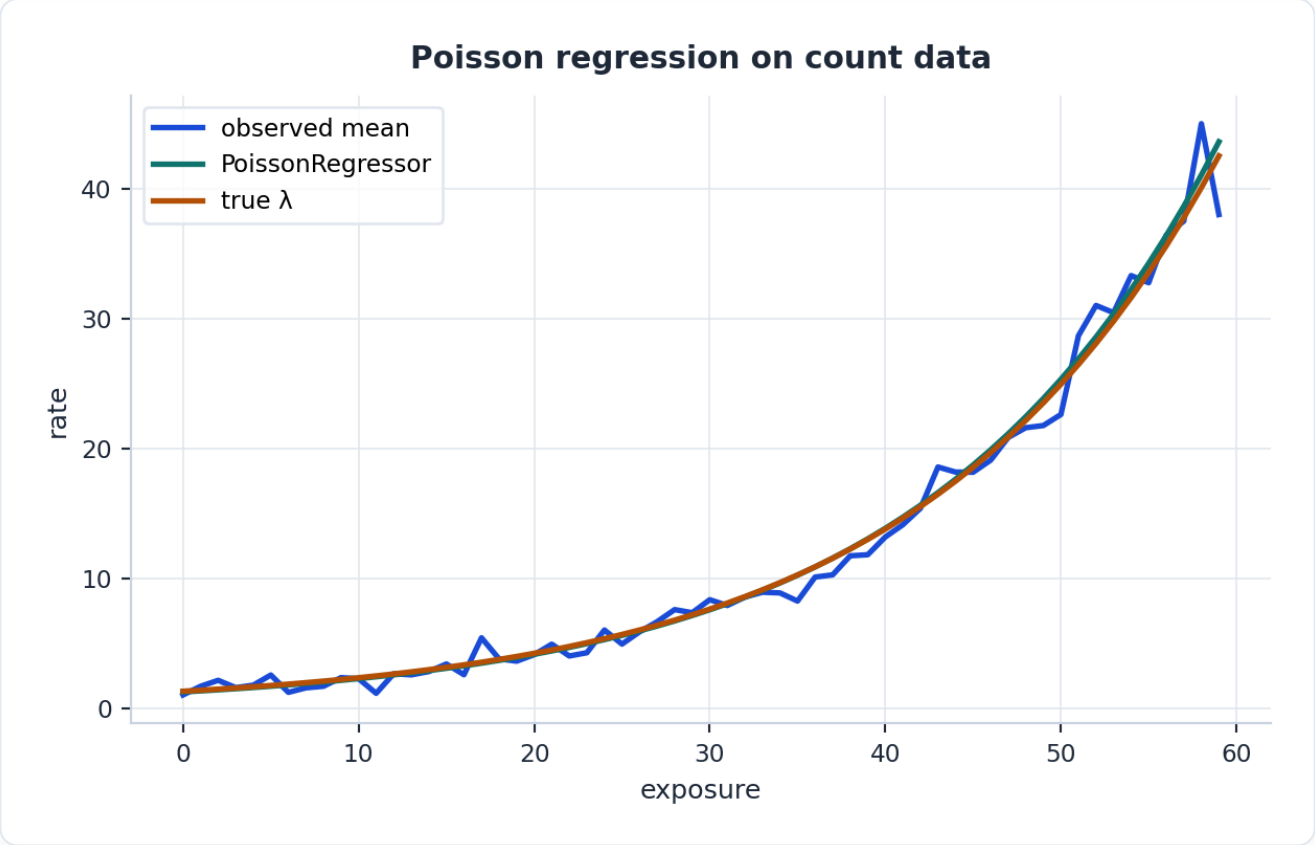
إذا طبقت OLS على هذه المشكلات، سيتنبأ بقيم سالبة مستحيلة فيزيائياً، وستنهار موثوقية فترات الثقة.



الشكل ٧١ مقارنة التوزيع الطبيعي المتماثل مع التوزيعات الملتوية وملاءمة دالة الارتباط

٤.٢ النماذج الخطية المعممة (GLM) ودوال الربط

تقدم النماذج الخطية المعممة (Generalized Linear Models) حلاً جذرياً عبر ثلاثة أركان:



الشكل ٧٢مقارنة ملاءمة انحدار بواسون مقابل الانحدار الخطي العادي على بيانات العد

٤.٣ PoissonRegressor: نمذجة بيانات العد ومعدل التكرار

يستخدم `PoissonRegressor` دالة الربط اللوغاريتمية وخسارة انحراف بواسون (Poisson Deviance):

$$D(y, \hat{y}) = 2\sum_{i=1}^{n}(y_i \log\frac{y_i}{\hat{y}_i} - (y_i - \hat{y}_i))$$

```python
import numpy as np
from sklearn.linear_model import PoissonRegressor, LinearRegression
from sklearn.metrics import mean_poisson_deviance

rng = np.random.default_rng(42)
n_samples = 250
X = rng.uniform(0, 5, size=(n_samples, 2))

# معدل الحدوث الحقيقي يتبع دالة أسية (Poisson)
true_lambda = np.exp(0.5 + 0.6 * X[:, 0] - 0.4 * X[:, 1])
y = rng.poisson(lam=true_lambda)

ols = LinearRegression().fit(X, y)
poisson = PoissonRegressor(alpha=1e-3, max_iter=300).fit(X, y)

print(f"أدنى قيمة تنبأ بها OLS:     {ols.predict(X).min():.3f} (قد تكون سالبة!)")
print(f"أدنى قيمة تنبأ بها Poisson: {poisson.predict(X).min():.3f} (موجبة قطعاً)")
print(f"انحراف بواسون لـ OLS:       {mean_poisson_deviance(y,
np.clip(ols.predict(X), 1e-6, None)):.4f}")
print(f"انحراف بواسون لـ Poisson:   {mean_poisson_deviance(y,
poisson.predict(X)):.4f}")
```

٤.٤ GammaRegressor: القيم الموجبة المستمرة والذيل الطويل

توزيع غاما مناسب للغاية للمتغيرات الموجبة تماماً $y > 0$ حيث يتناسب الانحراف المعياري طردياً مع المتوسط الحسابي ($\sigma \propto \mu$).

تستخدم خسارة انحراف غاما (Gamma Deviance):

$$D(y, \hat{y}) = 2 \sum_{i=1}^{n} \left(-\log \frac{y_i}{\hat{y}_i} + \frac{y_i - \hat{y}_i}{\hat{y}_i} \right)$$

```
from sklearn.linear_model import GammaRegressor
import numpy as np

rng = np.random.default_rng(42)
X = rng.uniform(1, 10, size=(200, 2))
# توليد تكاليف موجبة ملتوية وفق توزيع غاما
mu = np.exp(1.0 + 0.3 * X[:, 0] + 0.2 * X[:, 1])
# شكل التوزيع k = 2
y = rng.gamma(shape=2.0, scale=mu / 2.0)

gamma_reg = GammaRegressor(alpha=0.1, max_iter=200).fit(X, y)

print(f"معاملات نموذج Gamma: {np.round(gamma_reg.coef_, 3)}")
print(f"الحد الثابت intercept_: {gamma_reg.intercept_:.3f}")
print(f"تنبؤ لعينة جديدة X=[2, 3]: {gamma_reg.predict([[2, 3]])[0]:.2f}")
```

٤.٥ TweedieRegressor: النمذجة الشاملة والتراكم الصفري

عائلة توزيعات تويدي (Tweedie Distribution) تقدم إطاراً توحيدياً مذهلاً يتميز بمعلمة القوة `power` :

حالة $1 < p < 2$ هي السلاح السري في قطاع التأمين والخدمات المصرفية: معظم العملاء تكلفة مطالباتهم صفرية تماماً ($y = 0$)، لكن قلة منهم لديهم تكلفة موجبة متصلة مستمرة!

```
from sklearn.linear_model import TweedieRegressor
import numpy as np

rng = np.random.default_rng(42)
n = 300
X = rng.uniform(0, 5, size=(n, 2))
mu = np.exp(0.5 * X[:, 0] + 0.3 * X[:, 1])

# محاكاة بيانات مطالبات تأمين (أصفار كثيرة + مبالغ موجبة)
has_claim = rng.binomial(n=1, p=np.clip(mu / 15.0, 0.05, 0.7))
claim_amount = rng.gamma(shape=2.0, scale=mu) * has_claim

# تدريب TweedieRegressor مع power=1.5
tweedie = TweedieRegressor(power=1.5, alpha=0.1, max_iter=300).fit(X,
claim_amount)

print(f"نسبة العينات الصفرية في البيانات: {np.mean(claim_amount == 0):.1%}")
print(f"متوسط التكلفة المتنبأ بها: {tweedie.predict(X).mean():.3f}")
print(f"معاملات Tweedie: {np.round(tweedie.coef_, 3)}")
```

خطأ شائع

انتبه لدوال القياس: لا تستخدم R^2 أو MSE التقليدية لتقييم نماذج Poisson أو Gamma أو Tweedie؛ لأن هذه المقاييس تفترض توزيعاً غاوسياً متماثلاً. استخدم بدلاً من ذلك دالة `d2_tweedie_score` أو مقياس نسبة الانحراف المفسر (Explained Deviance)، المتوفر في `sklearn.metrics.d2_tweedie_score`.

تمارين الفصل الثالث والثلاثين

- **ملاحظة**

التمرين ٤.١

أنشئ مجموعة بيانات تحوي عدّاً يتبع توزيع بواسون بمتوسط يعتمد أسياً على الميزات. درب `LinearRegression` و `PoissonRegressor`، وقارن نسبة المتبقيات الإيجابية والسلبية ومعدل الانحراف لكل منهما.

التمرين ٤.٢

باستخدام `TweedieRegressor`، جرب ٤ قيم مختلفة لمعلمة `power`: [٠, ١, ١.٥, ٢] على بيانات أسعار المنازل بعد استبعاد القيم الصفرية. أي قيمة لـ `power` حققت أعلى معامل `d2_tweedie_score`؟

التمرين ٤.٣

اشرح لماذا لا يمكن لـ `GammaRegressor` قبول عينات تحتوي على $y \leq 0$ برمجياً ونظرياً، وماذا يحدث إذا مررت بيانات تحوي صفراً واحداً للمقدر؟

- **ملاحظة**

```python
import numpy as np
from sklearn.linear_model import LinearRegression, PoissonRegressor
from sklearn.metrics import d2_tweedie_score

rng = np.random.default_rng(42)
X = rng.normal(size=(300, 3))
y = rng.poisson(lam=np.exp(0.8 * X[:, 0] - 0.5 * X[:, 1] + 1.0))

ols = LinearRegression().fit(X, y)
pois = PoissonRegressor(alpha=1e-3).fit(X, y)

pred_ols = ols.predict(X)
pred_pois = pois.predict(X)

print(f"عدد التنبؤات السالبة المستحيلة في OLS: {np.sum(pred_ols < 0)}")
print(f"عدد التنبؤات السالبة في Poisson:      {np.sum(pred_pois < 0)}")
print(f"D² Poisson score لـ OLS:     {d2_tweedie_score(y, np.clip(pred_ols, 1e-5,
None), power=1):.4f}")
print(f"D² Poisson score لـ Poisson: {d2_tweedie_score(y, pred_pois,
power=1):.4f}")
```

ملاحظة

```python
from sklearn.linear_model import GammaRegressor
import numpy as np

# اختبار سلوك GammaRegressor عند وجود أصفار
X = np.array([[1], [2], [3]])
y_with_zero = np.array([0.0, 4.2, 5.1])

try:
    GammaRegressor().fit(X, y_with_zero)
except ValueError as err:
    print(f"خطأ ملتقط بنجاح: {err}")
    print("السبب الرياضي: دالة خسارة غاما تتضمن log(y) و1/y، وكلاهما غير معرف عند الصفر!")
```

ملاحظة

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تحويل y يدوياً بـ $\log(y)$ وتدريب OLS	تشويه التقديرات وافتراض خاطئ بأن تباين الأخطاء لوغاريتمي	استخدام `PoissonRegressor` أو `GammaRegressor` برابط لوغاريتمي أصلي
تمرير قيم $y \leq 0$ إلى `GammaRegressor`	حدوث خطأ `ValueError` فوري لعدم تعريف اللوغاريتم عند الصفر	استخدام `TweedieRegressor(power=1.5)` أو `PoissonRegressor` إذا وجدت أصفار
استخدام مقياس R^2 لتقييم جودة نماذج GLM	نتائج مضللة وغير متسقة مع دالة الهدف المحسنة	استخدام `d2_tweedie_score` المخصص لتقييم الانحراف المفسر
إهمال معايرة الميزات المدخلة في GLM	بطء تقارب خوارزميات الاستمثال أو فشل الحل	استخدام `StandardScaler` دوماً للميزات داخل `Pipeline`

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
النماذج الخطية المعممة	امتداد للنماذج الخطية يدعم توزيعات من العائلة الأسية	Generalized Linear Models (GLM)
دالة الربط	تحويل يربط المتنبئ الخطي بالمتوسط الشرطي للمتغير	Link function
خسارة الانحراف	مقياس إحصائي لجودة الملاءمة مبني على تعظيم الاحتمالية	Deviance loss
توزيع تويدي	عائلة توزيعات احتمالية توحد التوزيع الطبيعي وبواسون وغاما	Tweedie distribution
بيانات العد	متغيرات تمثل أعداداً صحيحة منفصلة لحالات الحدوث	Count data

الجزء ٤ · النماذج الخطية

الفصل ٣٤ — الانحدار البيزي

ARDRegression و BayesianRidge — التنظيم الاحتمالي

الوحدة **scikit-learn ١.٩** الحجم **٣٤٥** كلمة تقريباً

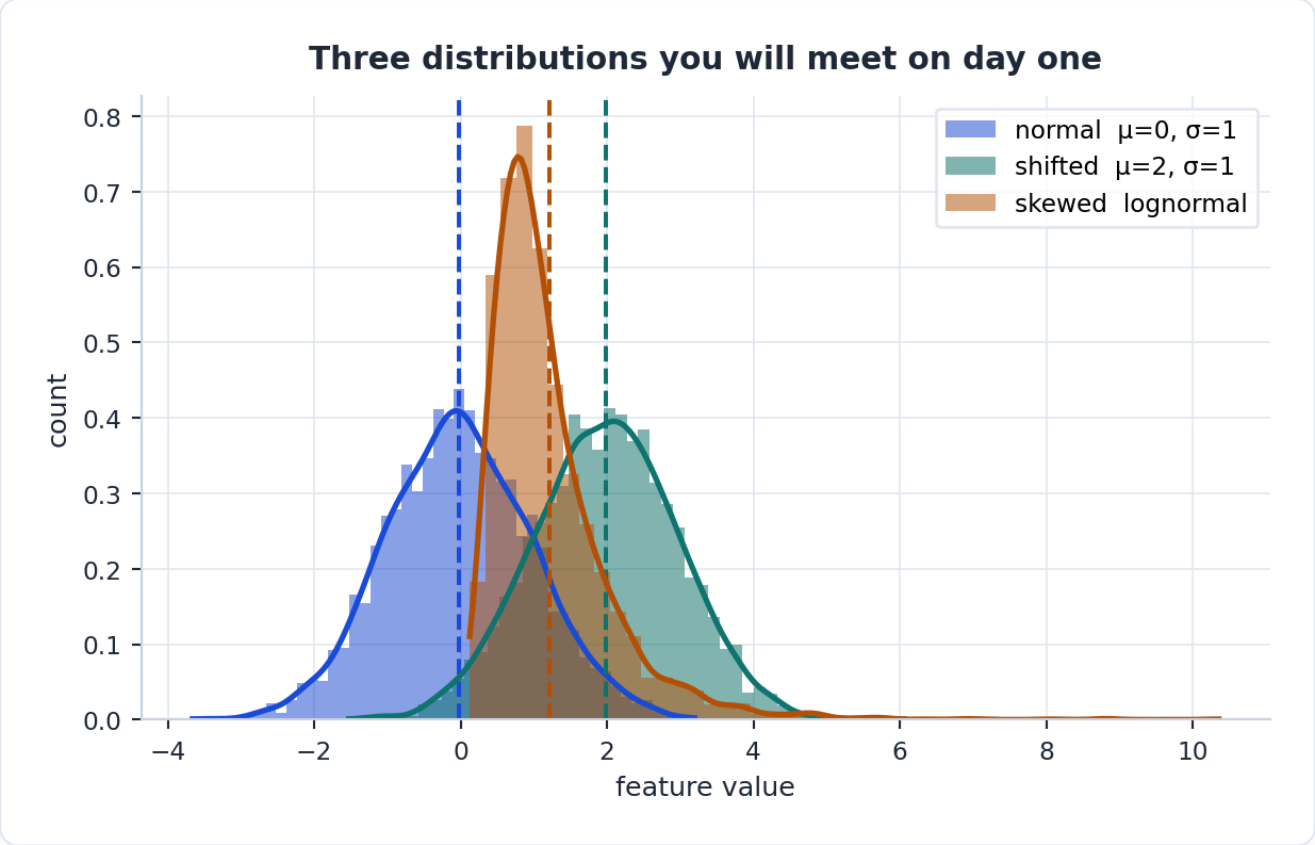
ما الذي ستتقنه في هذا الفصل

- أن تصيغ المعاملات الخطية كمتغيرات عشوائية ذات توزيعات قبلية وبعدية
- أن تستخدم BayesianRidge لاستنتاج معلمات تنظيم الضجيج والأوزان آلياً
- أن تستخرج فترات عدم اليقين التنبؤية (Uncertainty Estimates) عبر return_std
- أن تطبق ARDRegression للتصفير الاحتمالي للميزات غير المجدية

٤.١ من التقدير النقطي إلى التوزيع البعدي

في جميع النماذج الخطية التي درسناها حتى الآن (OLS, Ridge, Lasso)، كان المخرج النهائي هو متجهاً من الأرقام الثابتة $\hat{\beta} \in \mathbb{R}^p$. إذا سألنا النموذج: «ما مدى ثقتك في هذا المعامل أو في هذا التنبؤ لعينة جديدة؟»، فإنه يعجز عن الإجابة!

يعيد **المنظور البيزي (Bayesian Framework)** صياغة المشكلة جذرياً:



الشكل ٧٣التوزيع الطبيعي وتطور التوزيعات البعدية الاحتمالية

٤.٢ BayesianRidge: الضبط الذاتي لمعلمات التنظيم

الميزة الساحرة لـ `BayesianRidge` هي استغلال تعظيم الاحتمالية الهامشية (Marginal Likelihood Maximization) لتقدير كل من شدة تنظيم الأوزان α ومستوى ضجيج المخرجات λ مباشرة وتكرارياً من البيانات التدريبية **دون الحاجة لتقسيم البيانات أو تشغيل شبكة تحقق متقاطع معقدة!**

الأهم من ذلك: عند استدعاء `predict(X, return_std=True)`، يعيد النموذج الانحراف المعياري التنبؤي $\sigma(x)$ لكل عينة، مما يتيح رسم أشرطة عدم اليقين التنبؤي بدقة بالغة.

```python
import numpy as np
from sklearn.linear_model import BayesianRidge
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler

X, y = load_diabetes(return_X_y=True)
X_scaled = StandardScaler().fit_transform(X)

# تدريب مقدر الحافة البيزي
bayes = BayesianRidge(compute_score=True)
bayes.fit(X_scaled, y)

# التنبؤ بالقيمة والانحراف المعياري لعدم اليقين
y_pred, y_std = bayes.predict(X_scaled[:5], return_std=True)

print("التنبؤات الخمس الأولى مع عدم اليقين المصاحب (±1σ):")
for i in range(5):
    print(f"  العينة {i+1}: التنبؤ = {y_pred[i]:.1f} ± {y_std[i]:.2f}")

print(f"\nمعلمة دقة الضجيج المقدرة ذاتياً (alpha_): {bayes.alpha_:.4f}")
print(f"معلمة تنظيم الأوزان المقدرة ذاتياً (lambda_): {bayes.lambda_:.4f}")
```

٤.٣ رسم شريط عدم اليقين التنبؤي

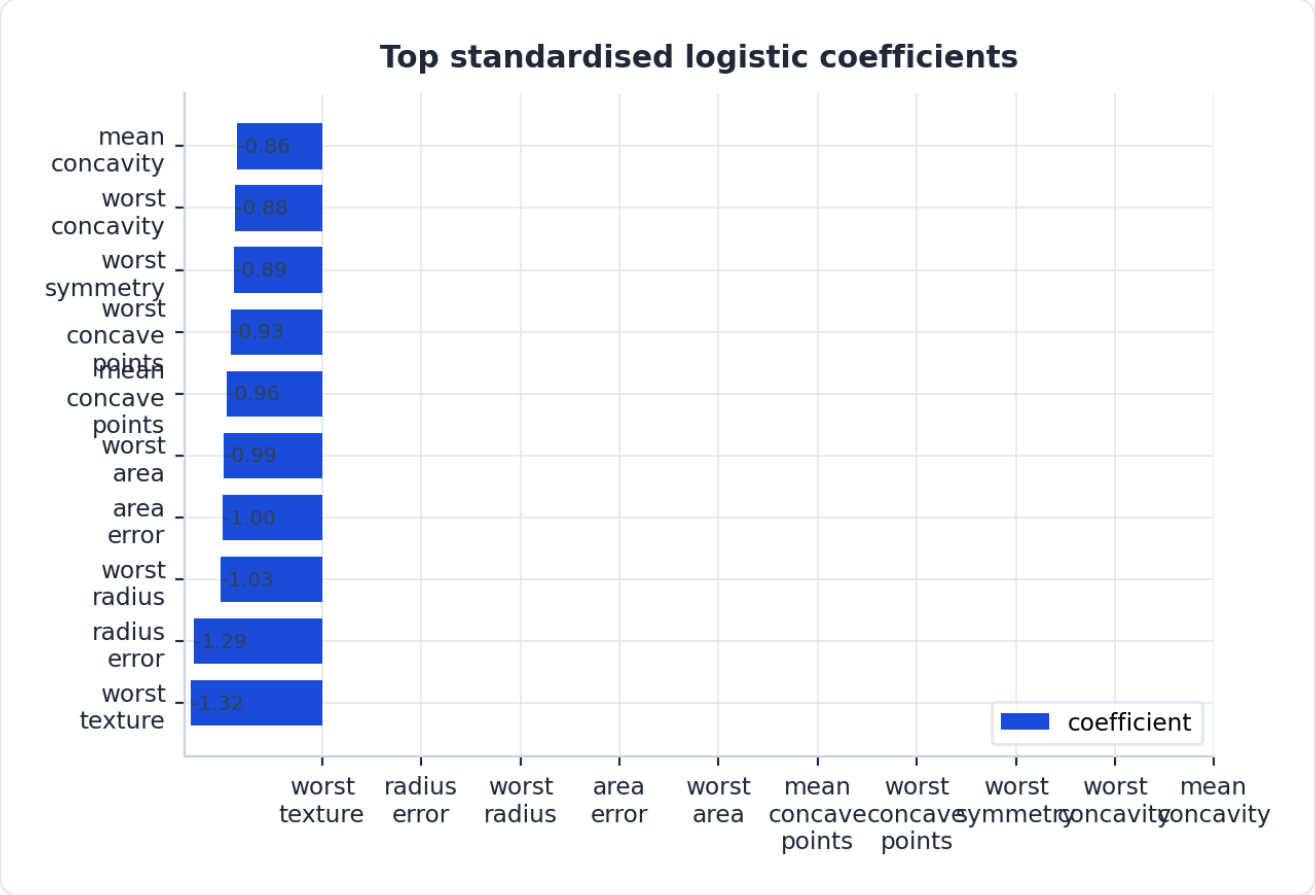
عندما تكون العينة الجديدة قريبة من بيانات التدريب، يكون عدم اليقين صغيراً، بينما يتسع شريط عدم اليقين كلما ابتعدت العينة نحو مناطق غير مستكشفة (فجوات البيانات):

```python
import numpy as np
from sklearn.linear_model import BayesianRidge

rng = np.random.default_rng(42)
X_tr = np.sort(rng.uniform(-3, 3, size=(25, 1)), axis=0)
y_tr = np.sin(X_tr).ravel() + rng.normal(scale=0.15, size=25)

bayes = BayesianRidge().fit(X_tr, y_tr)

# التنبؤ على نطاق واسع يتجاوز فضاء التدريب
X_test = np.linspace(-5, 5, 100).reshape(-1, 1)
y_pred, y_std = bayes.predict(X_test, return_std=True)

# مقارنة عدم اليقين داخل نطاق التدريب وخارجه
std_inside = y_std[np.abs(X_test.ravel()) <= 3.0].mean()
std_outside = y_std[np.abs(X_test.ravel()) > 3.0].mean()

print(f"[3 ,3-] متوسط عدم اليقين داخل نطاق البيانات:  {std_inside:.4f}")
print(f"(الاستقراء) متوسط عدم اليقين خارج نطاق البيانات: {std_outside:.4f}")
print("النموذج البيزي يعبر بوضوح عن نقص يقينه كلما ابتعد عن بيانات التدريب!")
```

٤.٤ ARDRegression: التحديد التلقائي للدلالة

في `BayesianRidge`، تتشارك جميع الميزات معلمة مسبقة واحدة α. ولكن ماذا لو أردنا التصفير الانتقائي للميزات غير المهمة كما يفعل Lasso؟

يقدم مقدر **ARDRegression (Automatic Relevance Determination)** حلاً بيزياً ثورياً:



الشكل ٧٤توزيع أوزان الميزات والانتقاء البعدي في النماذج الخطية

```python
from sklearn.linear_model import ARDRegression
from sklearn.datasets import make_regression
import numpy as np

# بيانات تحوي 20 ميزة، 4 ميزات فقط حقيقية
X, y, true_coef = make_regression(
    n_samples=100, n_features=20, n_informative=4,
    noise=5.0, coef=True, random_state=42
)

ard = ARDRegression(compute_score=True).fit(X, y)

estimated_zero = np.isclose(ard.coef_, 0.0, atol=1e-2)
actual_zero = true_coef == 0.0

print(f"عدد الميزات الحقيقية غير المفيدة: {np.sum(actual_zero)}")
print(f"عدد الميزات التي صفرها ARD بنجاح: {np.sum(estimated_zero)}")
print(f"دقة تطابق تصفير الميزات غير المفيدة: {np.mean(estimated_zero == actual_zero):.1%}")
```

> **ملاحظة**
>
> يعتبر ARD مكافئاً بيزياً رصيناً لـ Lasso، لكنه يمتاز بعدم معاناته من التحيز الانكماشي (Shrinkage Bias) الذي يعيب معيار L_1؛ إذ تبقى الميزات الحقيقية غير الصفرية محتفظة بأوزانها الطبيعية دون تقليص مفرط.

٤.٥ مقارنة شاملة: BayesianRidge مقابل ARDRegression

المعيار	BayesianRidge	ARDRegression
التوزيع القبلي للميزات	معلمة تنظيم واحدة مشتركة لجميع الميزات	معلمة تنظيم مستقلة لكل ميزة j
السلوك مع الميزات غير المفيدة	تقليص متزامن نحو الصفر دون تصفير مطلق	تصفير الميزات غير المفيدة تماماً ($\alpha_j \to \infty$)
التعقيد الحسابي	سريع جداً وقابل للتوسع في البيانات المتوسطة	أبطأ ويتطلب تكرارات استمثال أكثر
استخراج عدم اليقين	متاح عبر `predict(..., return_std=True)`	متاح عبر `predict(..., return_std=True)`
الحساسية للتعدد الخطي	متماسك ومقاوم بفضل تنظيم L_2 البيزي	يختار ميزة ويميل لتصفير شركائها المتطابقين

تمارين الفصل الرابع والثلاثين

- **ملاحظة**

التمرين ٤.١

على بيانات `fetch_california_housing`، درب مقدر `BayesianRidge`. استخرج الانحراف المعياري التنبؤي لـ ١٠٠ عينة اختبار، وتحقق: هل العينات التي شهدت أكبر خطأ في التنبؤ $|y - \hat{y}|$ كانت مصحوبة بانحراف معياري تنبؤي أعلى؟

التمرين ٤.٢

أنشئ مصفوفة بيانات تحوي ٥٠٠ ميزة (٥ منها فقط حقيقية). درب كلاً من `LassoCV` و `ARDRegression`، وقارن بين دقة الميزات المنتقاة ومقدار الخطأ في تقدير المعاملات الحقيقية.

التمرين ٤.٣

تتبع مسار دالة الاحتمالية الهامشية `bayes.scores_` أثناء التدريب، ولاحظ كيف تتقارب تكرارات مقدر `BayesianRidge` نحو القيمة العظمى.

- **ملاحظة**

```python
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import BayesianRidge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1000], data.target[:1000], test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_tr)
X_te_sc = scaler.transform(X_te)

bayes = BayesianRidge().fit(X_tr_sc, y_tr)
y_pred, y_std = bayes.predict(X_te_sc, return_std=True)

errors = np.abs(y_te - y_pred)
corr = np.corrcoef(errors, y_std)[0, 1]

print(f"معامل الارتباط بين خطأ التنبؤ الفعلي والانحراف المعياري للشك: {corr:.3f}")
print("الارتباط الإيجابي يؤكد أن النموذج يعلم متى تزداد احتمالية خطئه!")
```

ملاحظة

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import BayesianRidge
from sklearn.preprocessing import StandardScaler
import numpy as np

X, y = load_diabetes(return_X_y=True)
X_sc = StandardScaler().fit_transform(X)

bayes = BayesianRidge(compute_score=True, max_iter=300).fit(X_sc, y)

print(f"عدد التكرارات حتى التقارب: {len(bayes.scores_)}")
print(f"قيمة دالة الاحتمال الهامشي في الخطوة الأولى: {bayes.scores_[0]:.2f}")
print(f"قيمة دالة الاحتمال الهامشي النهائية:        {bayes.scores_[-1]:.2f}")
print(f"التحسن الإجمالي في دالة الهدف: {bayes.scores_[-1] - bayes.scores_[0]:+.2f}")
```

ملاحظة

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إغفال تحجيم البيانات قبل النماذج البيزية	اختلال التوزيعات القبلية وتقدير خاطئ لمعلمات الدقة α	استخدام `StandardScaler` دوماً للميزات داخل خط الأنابيب
افتراض أن عدم اليقين التنبؤي `return_std` يمثل خطأ البيانات فقط	الخلط بين تباين الضجيج وعدم يقين المعاملات الهيكلي	إدراك أن `std` يجمع ضجيج الملاحظة وعدم يقين تقدير β
تطبيق ARDRegression على آلاف الميزات دفعة واحدة	بطء زمني شديد بسبب تحديث مصفوفات التغاير تكرارياً	تقليص الأبعاد مسبقاً أو استخدام `BayesianRidge`
تجاهل تتبع معيار التقارب وعدم ضبط `max_iter`	توقف الاستمثال قبل بلوغ القيمة العظمى للاحتمالية	فحص `bayes.scores_` والتأكد من استقرار الدالة

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الانحدار البيزي	صياغة احتمالية للانحدار الخطي تدمج التوزيعات القبلية والبعدية	Bayesian regression
التوزيع البعدي	التوزيع الاحتمالي للمعاملات بعد مشاهدة بيانات العينة	Posterior distribution
الاحتمالية الهامشية	دالة احتمال البيانات بعد مكاملة المعاملات، لضبط المعلمات الفائقة	Marginal likelihood
التحديد التلقائي للدلالة	طريقة بيزية لتخصيص دقة مستقلة لكل ميزة لإسقاط غير المهم منها	Automatic Relevance Determination (ARD)
عدم اليقين التنبؤي	الانحراف المعياري للناتج المتنبأ به لعينة معينة	Predictive uncertainty

الجزء ٤ · النماذج الخطية

الفصل ٣٥ — التصنيف اللوجيستي: من الاحتمال إلى القرار

الاشتقاق، و decision_function، ومتعدد الفئات، وحلول المعادلات

الوحدة **scikit-learn ١.٩** الحجم **٣٦٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشتق دالة السيجمويد اللوجيستية من مفهوم لوغاريتم الأرجحية (Log-Odds)
- أن تميز بين درجات الثقة في decision_function والاحتمالات المعايرة في predict_proba
- أن تختار خوارزمية الحل (Solver) المناسبة لحجم البيانات ونوع التنظيم المطلوب
- أن تطبق التصنيف متعدد الفئات بأسلوبي One-vs-Rest والانتروبيا المتقاطعة Multinomial

٤.١ من الخط المستقيم إلى الاحتمال المقيد

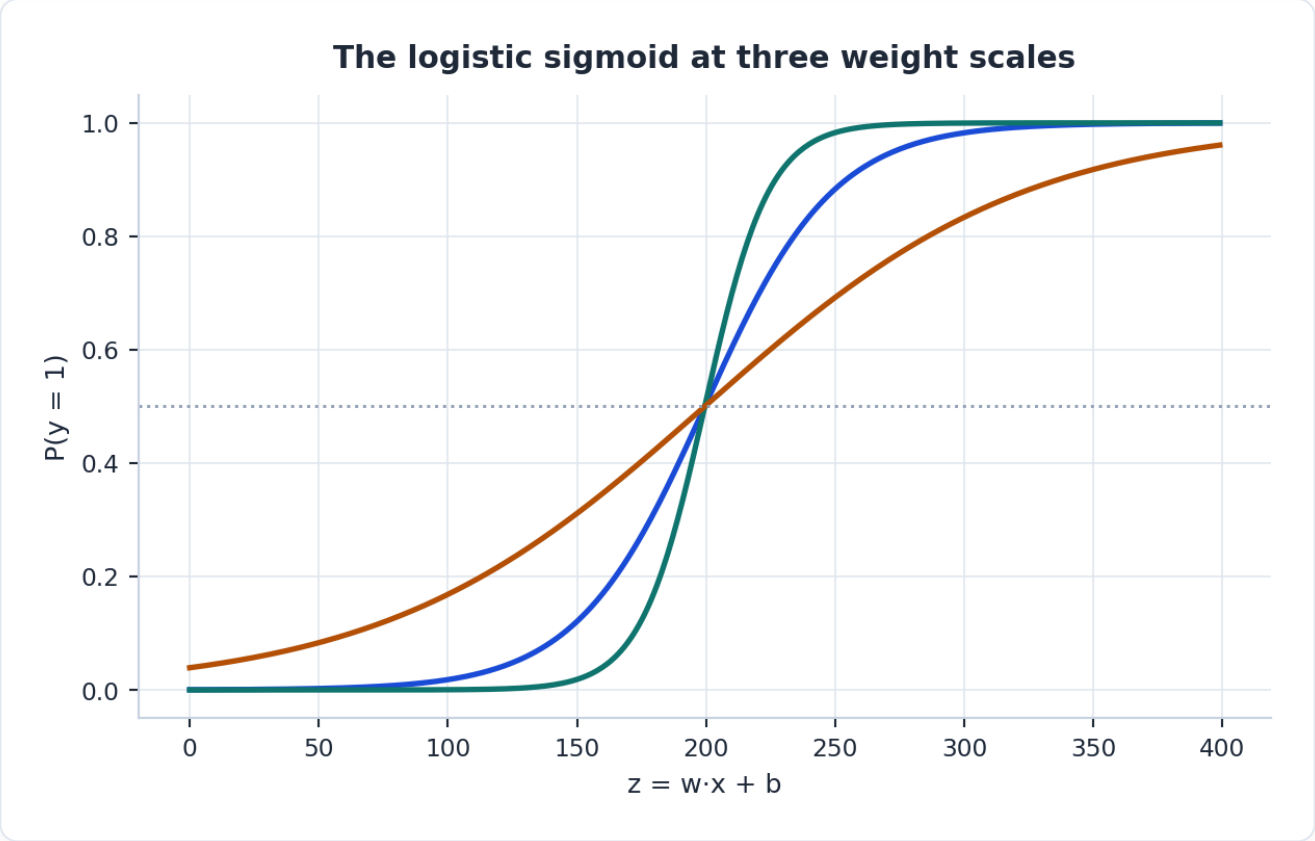
في الانحدار الخطي، يمتد المخرج $X\beta$ بلا حدود عبر المحور الحقيقي بالكامل من $-\infty$ إلى $+\infty$. لكن في مسائل التصنيف الثنائي، نريد التنبؤ باحتمال وقوع حدث $p = P(y = 1|X)$، وهو رقم مقيد بالضرورة داخل المجال الصارم $[0, 1]$.

كيف نربط الفضاء اللانهائي للمتنبئ الخطي بالمجال الاحتمالي المحصور؟

الحل يكمن في رحلة رياضية عبر ثلاثة مفاهيم:

بحل هذه المعادلة بالنسبة لـ p، نحصل مباشرة على **دالة السيجمويد اللوجيستية (Logistic Sigmoid)**:

$$p = \sigma(X\beta) = \frac{1}{1 + e^{-X\beta}}$$



الشكل ٧٥ دالة السيجمويد اللوجيستية وتحويل المتنبئ الخطي إلى احتمال محصور بين الصفر والواحد

٤.٢ دالة الخسارة: الإنتروبيا الثنائية (Log-Loss)

بما أن خطأ التربيع غير محدب (Non-convex) تحت دالة السيجمويد، نستخدم دالة تعظيم الإمكان اللوغاريتمي، وتعرف بـ **الإنتروبيا المتقاطعة الثنائية (Binary Cross-Entropy / Log-Loss)**:

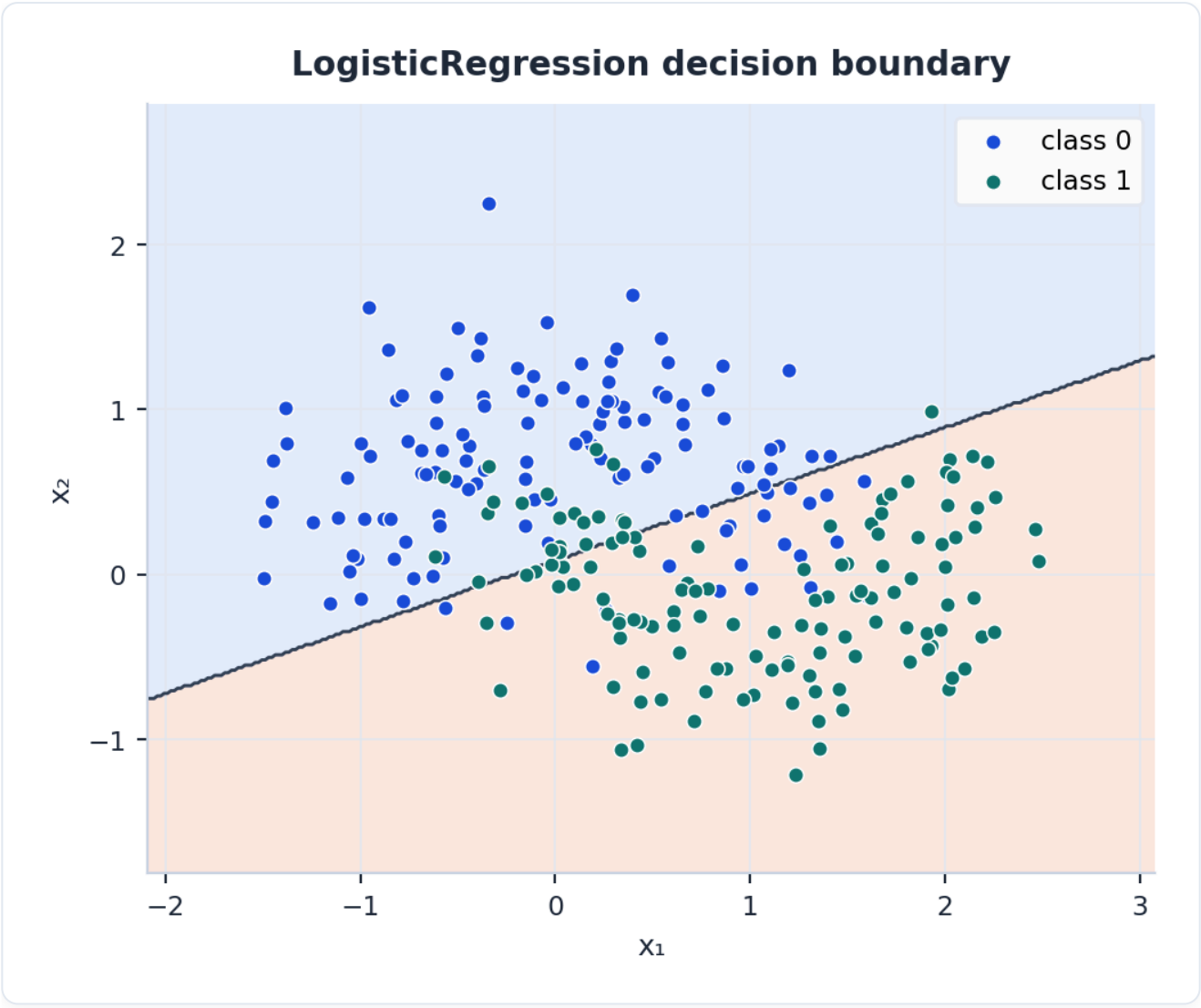
$$J(\beta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)] + \frac{1}{C}R(\beta)$$

حيث C هي مقلوب شدة التنظيم ($C = \frac{1}{\alpha}$). قيمة C صغيرة تعني تنظيماً قوياً جداً، وقيمة C كبيرة تعني تنظيماً ضعيفاً.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

X, y = load_breast_cancer(return_X_y=True)

pipe_lr = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(C=1.0, random_state=42))
])

pipe_lr.fit(X, y)
lr = pipe_lr.named_steps["clf"]

print(f"أبعاد مصفوفة المعاملات coef_: {lr.coef_.shape}")
print(f"قيمة الحد الثابت intercept_: {lr.intercept_[0]:.3f}")
print(f"دقة التدريب: {pipe_lr.score(X, y):.4f}")
```

٤.٣ الهندسة الفاصلة: *decision function* مقابل *predict proba*

يمتلك المصنف اللوجيستي مستويين من المخرجات يجب على كل مهندس تمييزهما بدقة:



الشكل ٧٦ الحد الفاصل الخطي ومناطق اتخاذ القرار في فضاء الميزات الثنائي

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

# اختيار فئتين فقط وميزتين للإيضاح الهندسي
X, y = load_iris(return_X_y=True)
mask = y < 2
X_bin, y_bin = X[mask, :2], y[mask]

clf = LogisticRegression(random_state=42).fit(X_bin, y_bin)

# فحص عينة قريبة من حافة القرار
sample = X_bin[:3]
scores = clf.decision_function(sample)
probs = clf.predict_proba(sample)
preds = clf.predict(sample)

print(f"{'العينة':>8} {'decision_function':>18} {'P(y=1)':>10} {'التنبؤ':<8}")
print("-" * 48)
for i in range(3):
    print(f"#{i+1:<7} {scores[i]:>18.3f} {probs[i, 1]:>10.3f} {preds[i]:>8}")
```

٤.٤ خوارزميات الحل (Solvers): دليلك لاختيار المحرك الأمثل

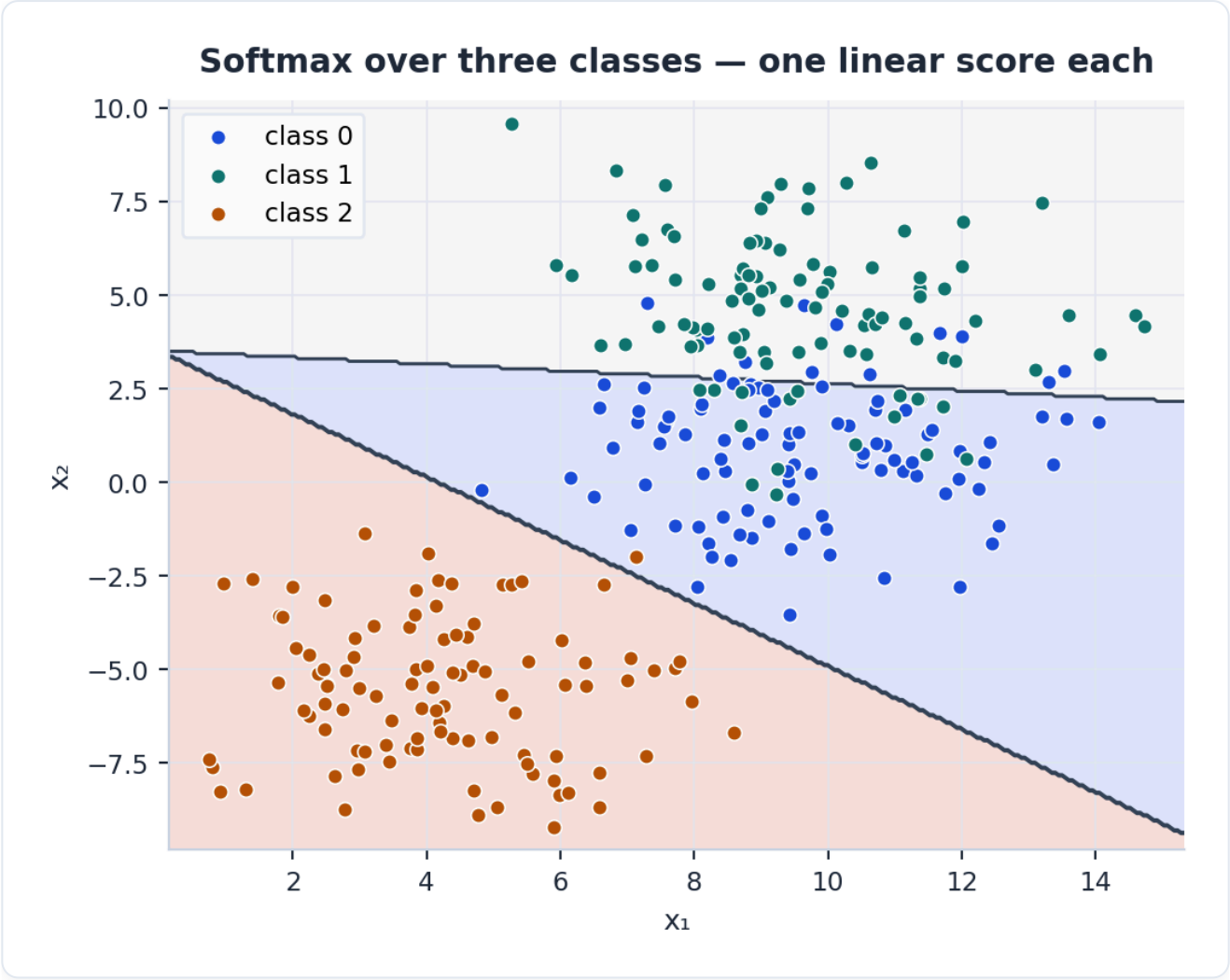
يوفر scikit-learn عدة خوارزميات استمثال متقدمة للمعامل `solver` :

المحرك (Solver)	نوع التنظيم المدعوم	السرعة وحجم البيانات	المشكلات متعددة الفئات
`lbfgs` **(الافتراضي)**	None , L_2	سريع جداً في البيانات الصغيرة والمتوسطة	Multinomial أصيل
`saga`	ElasticNet, , L_2 , L_1 None	الأفضل بلا منازع في البيانات الضخمة (n كبير جداً)	Multinomial أصيل
`liblinear`	L_2 , L_1	ممتاز في البيانات الصغيرة ذات الأبعاد العالية	One-vs-Rest فقط
`newton-cg`	None , L_2	دقيق ومكلف حسابياً (يعتمد مشتقات الرتبة الثانية)	Multinomial أصيل

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import time

X, y = load_breast_cancer(return_X_y=True)

# مقارنة بين lbfgs و saga السريع مع تنظيم ElasticNet المرن
for solver, l1_ratio in [("lbfgs", None), ("saga", 0.5)]:
    t0 = time.perf_counter()
    pipe = Pipeline([
        ("sc", StandardScaler()),
        ("clf", LogisticRegression(
            solver=solver, l1_ratio=l1_ratio, max_iter=2000, random_state=42
        ))
    ]).fit(X, y)
    elapsed = (time.perf_counter() - t0) * 1000
    print(f"المحرك {solver:<8} | l1_ratio={str(l1_ratio):<5} | الدقة: {pipe.score(X,
y):.4f} | الزمن: {elapsed:.2f} ms")
```

٤.٥ التصنيف متعدد الفئات: One-vs-Rest و Multinomial

عندما يتجاوز عدد الفئات اثنتين ($K > 2$):



الشكل ٧٧الحدود الفاصلة في التصنيف متعدد الفئات وتقسيم الفضاء إلى مناطق قرار

```python
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

X, y = load_iris(return_X_y=True)
class_names = load_iris().target_names

pipe_multi = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(random_state=42))
])

pipe_multi.fit(X, y)
clf_multi = pipe_multi.named_steps["clf"]

print(f"عدد الفئات المستهدفة: {len(class_names)}")
print(f"أبعاد مصفوفة المعاملات coef_: {clf_multi.coef_.shape} (فئة x ميزة)")
print("احتمالات الانتماء للفئات للعينة الأولى:")
first_prob = pipe_multi.predict_proba(X[:1])[0]
for name, prob in zip(class_names, first_prob):
    print(f"  الفئة '{name}': {prob:.4f}")
```

نصيحة

في الإصدارات الحديثة من scikit-learn (بما فيها ١.٩)، يتعامل `LogisticRegression` مع المشكلات متعددة الفئات بأسلوب `multinomial` تلقائياً عبر خسارة الإنتروبيا المتقاطعة، وهو الأصح إحصائياً مقارنة بـ OvR؛ لأنه يضمن أن مجموع الاحتمالات يساوي ١.٠ بطبيعة الاستمثال.

تمارين الفصل الخامس والثلاثين

- **ملاحظة**

التمرين ٤.١

على مجموعة بيانات `load_breast_cancer`، قارن بين أداء `LogisticRegression` تحت قيم $C \in [0.001, 0.1, 1.0, 100.0]$ باستخدام التحقق المتقاطع الخماسي. كيف يتغير معيار أوزان المعاملات $\|\beta\|_2$ مع زيادة C؟

التمرين ٤.٢

باستخدام محرك `solver="saga"` مع `l1_ratio=1.0` (penalty L1) و $C = 0.05$، افحص عدد المعاملات التي تم تصفيرها تماماً. هل يمكن استخدام التصنيف اللوجيستي كمصفاة للميزات؟

التمرين ٤.٣

اشرح رياضياً: لماذا تتطابق دالة Softmax تماماً مع دالة السيجمويد اللوجيستية عندما يكون عدد الفئات $K = 2$؟ أثبت ذلك بالمعادلات.

- **ملاحظة**

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score

X, y = load_breast_cancer(return_X_y=True)

print(f"{'قيمة C':<10} {'متوسط الدقة CV':>15} {'معيار الأوزان L2':>18}")
print("-" * 46)
for c in [0.001, 0.01, 1.0, 100.0]:
    pipe = make_pipeline(StandardScaler(), LogisticRegression(C=c,
max_iter=2000, random_state=42))
    cv_score = cross_val_score(pipe, X, y, cv=5).mean()
    pipe.fit(X, y)
    l2_norm = np.linalg.norm(pipe.named_steps["logisticregression"].coef_)
    print(f"{c:<10.3f} {cv_score:>15.4f} {l2_norm:>18.2f}")
```

ملاحظة

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=0.05, solver="saga", l1_ratio=1.0, max_iter=3000,
random_state=42)
)
pipe.fit(X, y)
coef = pipe.named_steps["logisticregression"].coef_[0]

zero_count = np.sum(coef == 0.0)
print(f"إجمالي الميزات: {len(coef)}")
print(f"الميزات المصفرة بالكامل بواسطة عقوبة L1 في التصنيف اللوجيستي: {zero_count}")
print(f"الميزات النشطة المعتمدة في القرار: {len(coef) - zero_count}")
```

ملاحظة

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إغفال تحجيم البيانات بـ `StandardScaler`	بطء شديد في تقارب المحركات وظهور تحذيرات `ConvergenceWarning`	تضمين `StandardScaler` دوماً في `Pipeline` قبل المقدر
ضبط C بقيم ضخمة جداً مع بيانات صاخبة	حدوث إفراط حاد في الملاءمة وحساسية مفرطة للشواذ	تجربة قيم C عبر التحقق المتقاطع أو استخدام `LogisticRegressionCV`
استخدام `penalty='l1'` في الإصدارات الحديثة	تحذيرات الإلغاء والتقادم في scikit-learn ١.٩	استخدام `l1_ratio=1.0` مع محرك `solver='saga'`
الاعتماد على الاحتمالات دون معايرة عند عدم توازن الفئات	انحياز عتبة الاحتمال التلقائية 0.5 نحو الفئة الغالبة	ضبط `class_weight='balanced'` أو تعديل عتبة القرار

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التصنيف اللوجيستي	نموذج خطي لتقدير احتمالات الانتماء الفئوي	Logistic regression
دالة السيجمويد	تحويل على شكل حرف S يسقط الأرقام داخل المجال $[0, 1]$	Sigmoid function
لوغاريتم الأرجحية	لوغاريتم نسبة احتمال وقوع الحدث إلى عدم وقوعه	Log-odds / Logit
دالة القرار	القيمة العددية الخام للمتنبئ الخطي قبل تحويلها لاحتمال	Decision function
دالة سوفت ماكس	تعميم متعدد الفئات للسيجمويد يضمن مجموع احتمالات مساوياً لـ ١	Softmax function

الجزء ٤ · النماذج الخطية

الفصل ٣٦ — المصنِّفات الخطية الأخرى

RidgeClassifier و SGD و PassiveAggressive و Perceptron

الوحدة **sklearn.linear_model** الحجم **٣٢٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

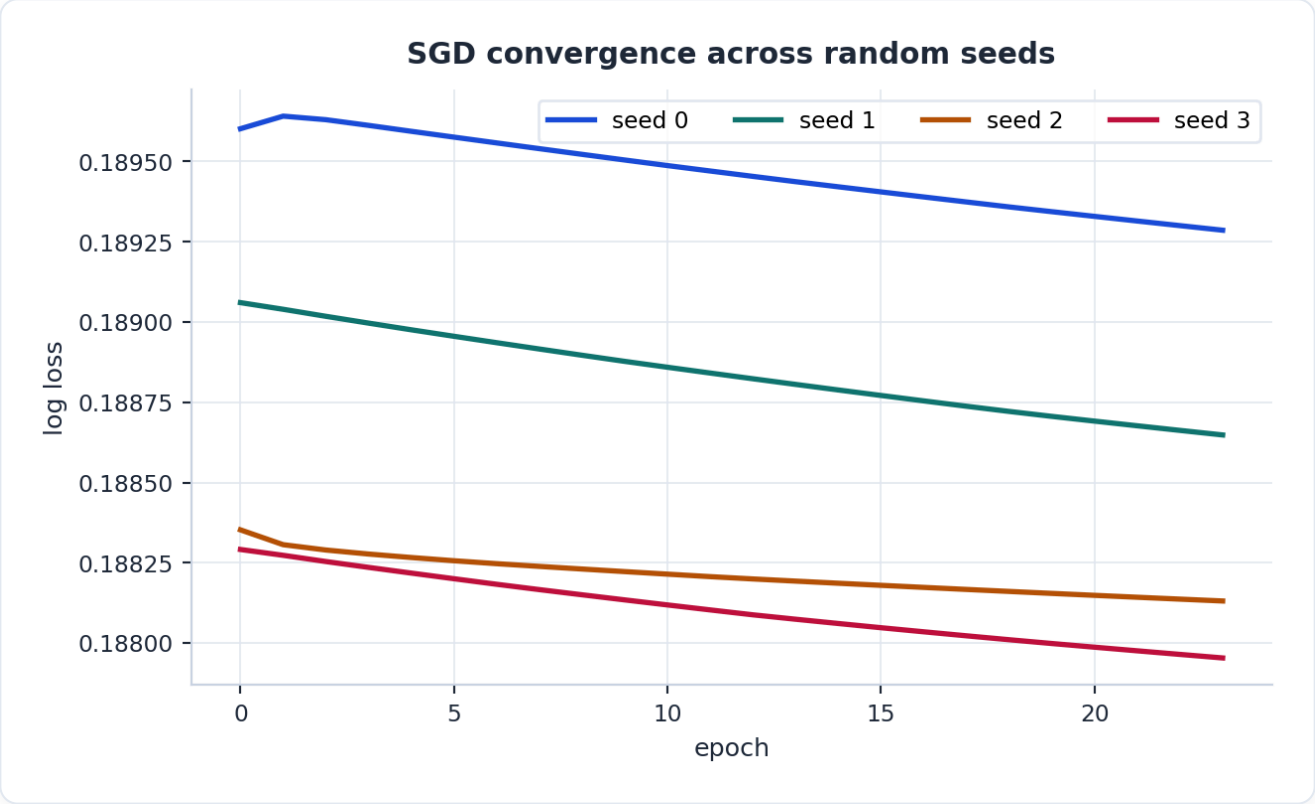
- أن تفهم قاعدة تحديث البرسبترون الكلاسيكية وحدودها أمام البيانات غير القابلة للفصل الخطي
- أن تطبق PassiveAggressiveClassifier في سيناريوهات التعلّم التدفقي عبر partial_fit
- أن توظف SGDClassifier كسكين سويسري يدعم دوال خسارة متعددة في البيانات المليونية
- أن تستخدم RidgeClassifier لتصنيف فائق السرعة عبر تحويل المربعات الصغرى

٤.١ ما وراء الانحدار اللوجيستي: السرعة والتعلّم المستمر

على الرغم من روعة التصنيف اللوجيستي، إلا أنه يتطلب تشغيل خوارزميات استمثال دفيعة (Batch Optimization) تستلزم تحميل كامل البيانات في الذاكرة وتكرار المرور عليها عدة مرات. في العالم الحقيقي، نواجه سيناريوهين يستلزمان بدائل خطية أخرى:

لهذه المهام، تمتلك عائلة النماذج الخطية في scikit-learn أربعة أسلحة استثنائية:

المصنف	الخوارزمية الجوهرية	دالة الخسارة	الميزة التنافسية
`Perceptron`	تحديث روزنبلات (١٩٥٨)	Zero-one loss surrogate	البساطة التاريخية القصوى
`PassiveAggressive`	خوارزمية كرامر (٢٠٠٦)	Hinge loss بدقة ε	التكيف اللحظي الفوري مع التيارات
`SGDClassifier`	الانحدار التدريجي العشوائي	مرنة (Hinge, Log, Huber)	المحرك المليوني متعدد الأغراض
`RidgeClassifier`	المربعات الصغرى المنظمة	Squared loss على ± 1	سرعة حل مغلق استثنائية



الشكل ٧٨ تقارب الانحدار التدريجي العشوائي ومسارات التحديث مع كل دفعة بيانات

٤.٢ البرسبترون الكلاسيكي: الجد الأكبر للشبكات العصبية

يعتمد `Perceptron` على أبسط قاعدة تعلّم في تاريخ الذكاء الاصطناعي:

```python
from sklearn.linear_model import Perceptron
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

X, y = load_breast_cancer(return_X_y=True)

pipe_perc = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", Perceptron(max_iter=1000, random_state=42))
])

pipe_perc.fit(X, y)
print(f"دقة البرسبترون الكلاسيكي: {pipe_perc.score(X, y):.4f}")
print(f"عدد التكرارات حتى التوقف: {pipe_perc.named_steps['clf'].n_iter_}")
```

خطأ شائع

مبرهنة تقارب البرسبترون: البرسبترون يتقارب ويستقر فقط إذا كانت البيانات قابلة للفصل الخطي تماماً (Linearly Separable). إذا تداخلت الفئات، ستتذبذب المعاملات إلى الأبد ولن يستقر الحل ما لم تضع حداً أقصى للتكرارات `max_iter` وتفعّل الإيقاف المبكر `early_stopping=True`.

٤.٣ PassiveAggressiveClassifier: مصنف التيارات الحية

تتبنى خوارزميات **Passive-Aggressive (PA)** فلسفة سلوكية بارعة:

هذا يجعلها مثالية للتدريب المتدفق عبر دالة `partial_fit`:

```
from sklearn.linear_model import PassiveAggressiveClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

rng = np.random.default_rng(42)
n_features = 20

pac = PassiveAggressiveClassifier(C=1.0, random_state=42)
classes = np.array([0, 1])

# محاكاة تدفق بيانات حي عبر 5 دفعات متتابعة
for stream_batch in range(5):
    X_batch = rng.normal(size=(50, n_features))
    y_batch = (X_batch[:, 0] + X_batch[:, 1] > 0).astype(int)

    # تحديث لحظي تراكمي
    pac.partial_fit(X_batch, y_batch, classes=classes)
    acc = pac.score(X_batch, y_batch)
    print(f"الدفعة التدفقية #{stream_batch+1}: دقة التكيف اللحظي = {acc:.2%}")
```

٤.٤ SGDClassifier: المحرك المليوني متعدد الأشكال

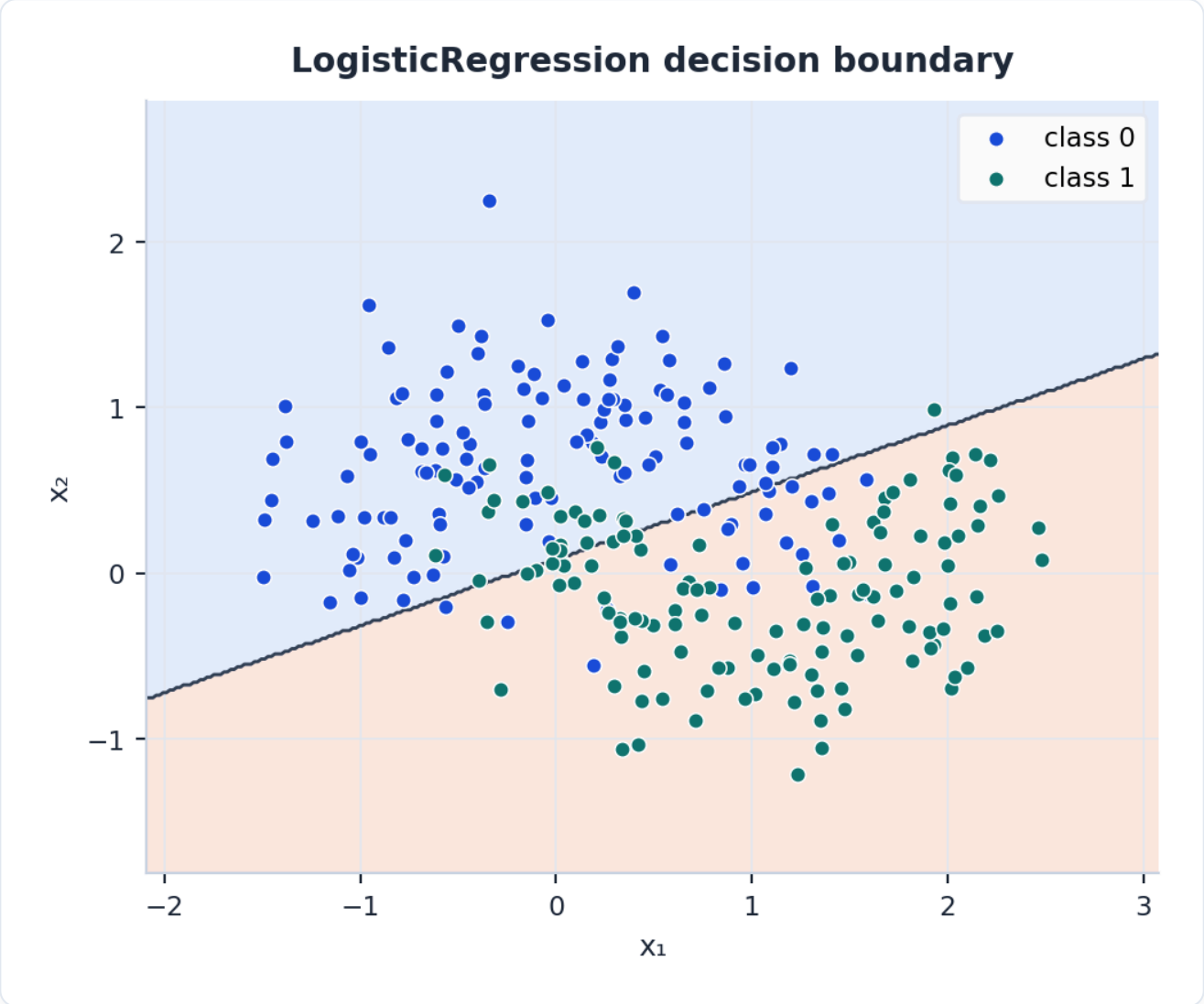
`SGDClassifier` ليس خوارزمية تصنيف مفردة، بل هو **محرك استمثال مرن** يتغير سلوكه بالكامل بتغيير المعامل `loss`:

```
from sklearn.linear_model import SGDClassifier
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# توليد بيانات ضخمة (10,000 عينة)
X, y = make_classification(n_samples=10000, n_features=50, random_state=42)

for loss in ["hinge", "log_loss", "modified_huber"]:
    pipe_sgd = Pipeline([
        ("sc", StandardScaler()),
        ("clf", SGDClassifier(loss=loss, alpha=1e-4, max_iter=1000,
random_state=42))
    ])
    pipe_sgd.fit(X, y)
    print(f"خسارة {loss:<15} | دقة التدريب: {pipe_sgd.score(X, y):.4f}")
```

٤.٥ RidgeClassifier: التصنيف عبر المربعات الصغرى

فكرة `RidgeClassifier` مباغتة في بساطتها وسرعتها:

تعد هذه الطريقة أسرع بكثير من التصنيف اللوجيستي، خاصة في المشكلات متعددة الفئات ذات الأبعاد الضخمة جداً (مثل تصنيف النصوص).



الشكل ٧٩ الحدود الخطية الفاصلة بين الفئات في فضاء القرار

```
from sklearn.linear_model import RidgeClassifier, LogisticRegression
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
import time

X, y = make_classification(n_samples=2000, n_features=100, n_classes=4,
                           n_informative=40, random_state=42)
X_sc = StandardScaler().fit_transform(X)

# مقارنة زمن التدريب بين RidgeClassifier و LogisticRegression
t0 = time.perf_counter()
ridge_clf = RidgeClassifier(alpha=1.0).fit(X_sc, y)
t_ridge = (time.perf_counter() - t0) * 1000

t0 = time.perf_counter()
lr_clf = LogisticRegression(max_iter=1000, random_state=42).fit(X_sc, y)
t_lr = (time.perf_counter() - t0) * 1000

print(f"RidgeClassifier   | الدقة: {ridge_clf.score(X_sc, y):.4f} | الزمن:
{t_ridge:.2f} ms")
print(f"LogisticRegression | الدقة: {lr_clf.score(X_sc, y):.4f} | الزمن: {t_lr:.2f}
ms")
print(f"RidgeClassifier أسرع بنحو {t_lr / t_ridge:.1f} مرة!")
```

تمارين الفصل السادس والثلاثين

- **ملاحظة**

التمرين ٤.١

باستخدام `SGDClassifier(loss="log_loss")`، درب نموذجاً تدفقياً عبر دالة `partial_fit` على ٢٠ دفعة متتابعة بحجم ١٠٠ عينة لكل دفعة. تتبع كيف تتطور دقة النموذج مع توالي الدفعات.

التمرين ٤.٢

على بيانات `load_breast_cancer`، قارن مصفوفة الالتباس (Confusion Matrix) ومقياس F1 بين `Perceptron` و `RidgeClassifier` و `SGDClassifier(loss="hinge")`. أي المصنفات حقق توازناً أفضل بين الحساسية والدقة؟

التمرين ٤.٣

اشرح لماذا يعجز `RidgeClassifier` عن توفير دالة `predict_proba` لحساب الاحتمالات المباشرة، وكيف يمكن التغلب على ذلك إذا تطلب التطبيق احتمالات صريحة؟

- **ملاحظة**

```python
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=2000, n_features=20, random_state=42)
sgd = SGDClassifier(loss="log_loss", random_state=42)
classes = np.unique(y)

# تجزئة البيانات إلى 10 دفعات
batch_size = 200
accuracies = []
for i in range(10):
    start, end = i * batch_size, (i + 1) * batch_size
    sgd.partial_fit(X[start:end], y[start:end], classes=classes)
    # تقييم على الدفعة التالية
    if end < len(X):
        acc = sgd.score(X[end:end+batch_size], y[end:end+batch_size])
        accuracies.append(acc)

print(f"دقة الدفعة الأولى: {accuracies[0]:.2%}")
print(f"دقة الدفعة الأخيرة: {accuracies[-1]:.2%}")
print(f"التحسن مع توالي التدفق: {accuracies[-1] - accuracies[0]:+.2%}")
```

ملاحظة

```python
from sklearn.linear_model import RidgeClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
ridge = RidgeClassifier()

# RidgeClassifier لا يمتلك predict_proba لأنه انحدار مربعات صغرى
print(f"هل يمتلك RidgeClassifier دالة predict_proba؟ {hasattr(ridge,
'predict_proba')}")

# الحل: تغليفه بمقدر المعايرة الاحتمالية CalibratedClassifierCV
calibrated_ridge = CalibratedClassifierCV(estimator=ridge, cv=3).fit(X, y)
print(f"هل يمتلك النموذج المعاير predict_proba؟ {hasattr(calibrated_ridge,
'predict_proba')}")
print("احتمالات أول عينة:", np.round(calibrated_ridge.predict_proba(X[:1])[0], 3))
```

ملاحظة

٤.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تدريب مصنفات SGD والبرسبترون دون تحجيم البيانات	تباعد التدرجات وفشل التقارب التام	استخدام `StandardScaler` دوماً للميزات
نسيان تمرير `classes` في أول استدعاء لـ `partial_fit`	حدوث خطأ `ValueError` لعدم معرفة الفئات المحتملة	تمرير `classes=np.unique(y)` في الدفعة الأولى دائماً
محاولة استدعاء `predict_proba` على `RidgeClassifier`	حدوث `AttributeError` لغياب دالة الاحتمال	استخدام `CalibratedClassifierCV` لتوليد احتمالات معايرة
استخدام البرسبترون مع بيانات غير قابلة للفصل الخطي بدون إيقاف مبكر	دوران غير منتهٍ حتى استنفاد `max_iter`	تفعيل `early_stopping=True` وتحديد `tol` مناسب

٤.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البرسبترون	مصنف خطي يعتمد على تحديث الأوزان في اتجاه العينات الخاطئة	Perceptron
المصنف السلبي الهجومي	خوارزمية تعلّم لحظي تدفقي تعدل الأوزان بالقدر الكافي فقط	Passive-Aggressive Classifier
الانحدار التدريجي العشوائي	استمثال دالة الهدف بتحديث الأوزان بناءً على عينة عشوائية	Stochastic Gradient Descent (SGD)
مصنف الحافة	تصنيف الفئات عبر حل انحدار الحافة بالمربعات الصغرى	Ridge Classifier
التعلّم التدفقي التزايدي	تحديث النموذج على دفعات بيانات متتابعة دون إعادة التدريب	Online / Incremental learning

الجزء ٥

أشجار القرار

نماذج تقسّم العالم إلى صناديق، وتشرح نفسها بلا رياضيات

ماذا يغطّي هذا الجزء

شجرة القرار هي النموذج الذي يشرح نفسه. لا تحتاج إلى تطبيع بيانات، ولا تتأثر بتباين الأبعاد، وتلتقط العلاقات غير الخطية بسهولة. لكن بلا ضبط ستُبالغ في الملاءمة بسرعة. هذه القوة تفهم التقسيمات، وكيف تختار الميزة والتوقيت، وكيف تتعامل مع نقص القيم، ولماذا لا تُعمَّم خارج عيّناتها.

الجزء ٥ · أشجار القرار

الفصل ٣٧ — شجرة القرار: من سؤال إلى شجرة

خوارزميتا ID٣ و C٤.٥ — الجيني والإنتروبيا ومكسب المعلومات

الوحدة scikit-learn ١.٩ الحجم ٧٠٠ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف تختار الشجرةُ الميزةَ والتوقيت الذي يقسّم بهما البيانات
- أن تحسب الجيني والإنتروبيا ومكسب المعلومات لمثال رقمي بعينه
- أن تقرأ مخرجات fit وتربطها بما تعلّمتَه نظرياً
- أن تحدّد ثلاثة مواضع تفشل فيها الشجرة بطبيعتها، لا بخطأ برمجي

٥.١ لماذا نبدأ بالشجرة؟

تخيّل أنك أمام مليون سجل مبيعات، ولكل سجل اثنا عشر رقماً. سؤال واحد: هل ستgete هذه الصفقة أم لا؟

أسهل طريقة هي أن ترسم خطاً في الفضاء. لكن Boundary الصفقات ليست خطية: العملاء ذوو الدخل المنخفض والمنخفض جداً يشترون أحياناً، والمتسوّق في المنتصف يشتري أكثر. الخط يفشل.

الشجرة لا تسأل «أين يقع هذا العميل»، بل «ما الصفة التي تفصل أفضل؟». تسأل: هل الدخل أقل من ٤٠ ألفاً؟ نعم ← فرع. لا ← فرع آخر. وهكذا حتى يصبح العميل في ورقة واحدة بلا تردّد. هذه هي الفكرة كاملة: **شجرة القرار سلسلة أسئلة نعم/لا تُبنى بالبيانات، ثم تُترجم إلى قرار.**

الفكرة المحورية

شجرةُ القرار لا «تتعلّم» بالمعنى الذي تتعلّم به الشبكة العصبية. هي **بحث منظَّم في فضاء ضخم من الأسئلة الممكنة**، تختار في كل خطوة السؤال الذي يعطي أفضل تقسيم. وحين تنتهي من البحث، تكون قد وجدتَ بالضبط ذلك السؤال، وذلك الحدّ، وتلك الميزة.

لذا قبل أن نكتب سطر كود واحد، من الوارد أن نسأل: ما الأسئلة الممكنة أصلاً؟ في كل عقدة من الشجرة، الخوارزمية تختبر **كل** ميزة **وكل** قيمة ممكنة منها، تقيس جودة كل تقسيم، ثم تختار الأفضل. في Iris مثلاً: أربع Features، وعدد القيم المتمايزة في كل منها نحو ٤ أو ٥، أي ما يقارب سبع عشرة قسمة ممكنة في الجذر. الشجرة تختبرها كلّها.

٥.٢ معيار الانقسام: كيف نقيس نقاء المجموعة؟

لنفترض أن لدينا مئة صف، فيها ثلاثون صفقة فاشلة وسبعون ناجحة. الشجرة تريد أن تكون كل ورقة **نقية** قدر الإمكان: إما كلها فاشلة أو كلها ناجحة.

هذا يقودنا إلى دالة تُسمّى **دالة الشوائب** (impurity). أشهر ثلاث:

٥.٢.١ الجيني (Gini)

الفكرة: احتمال أن يسحب شخص واحد عشوائي عيّنتين من المجموعة، فتكونان من فئتين مختلفتين. كلما قلّ هذا الاحتمال كانت المجموعة أنقى.

$$\text{Gini}(D) = 1 - \sum_{c=1}^{C} p_c^2$$

حيث `p_c` نسبة الصنف `c` في المجموعة `D`. الحدّ الأدنى صفر (كل الصفوف من صنف واحد)، والحدّ الأعلى `1 - 1/C` عند التوزيع المتساوي بين `C` أصناف.

٥.٢.٢ إنتروبيا شانون (Entropy)

الفكرة: إذا كانت المجموعة متوازنة تماماً فأنت لا تعرف شيئاً عن أي صف فيها، والمفاجأة عالية. كلما اقتربت المجموعة من التوازن ارتفعت قيمة عدم اليقين.

$$H(D) = -\sum_{c=1}^{C} p_c \log_2 p_c$$

الحدّ الأدنى صفر، والحدّ الأعلى `log₂ C`. عملياً **الانتروبيا تشدّ المجموعة أكثر من الجيني عند الانحراف عن التوازن**، ولهذا يعطي الاثنان عملياً نفس الأشجار تقريباً.

٥.٢.٣ مكسب المعلومات (Information Gain)

هنا يأتي السؤال الحقيقي: **ما الميزة التي تجعل الشجرة تتحسّن أكثر؟**

المعيار ليس شوائب المجموعة فرادى، بل الفرق بين شوائبها قبل القسمة وبعدها:

$$\text{Gain}(S, A) = \text{Impurity}(S) - \sum_{v \in V(A)} \frac{|S_v|}{|S|} \text{Impurity}(S_v)$$

حيث `S_v` هي الصفوف التي تملك القيمة `v` من الميزة `A` . الرمز `Impurity` هنا ليس خطأً مطبعياً بل اصطلاح: ضع مكانه الجيني أو الانتروبيا أيهما اخترت.

وهذا بالضبط ما تفعله scikit-learn مع المعامل `criterion` .

> **جرّب أن تحسبها بنفسك مرة واحدة**
>
> الدالتان `gini_of` و `entropy_of` معرّفتان في المثال التالي. غيّر رقماً واحداً في الكود وانظر إلى الناتج. الحساب اليدوي مرة واحدة تكفي لفهم الآلية كلها.

٥.٣ بناء الشجرة خطوة بخطوة

لنبنِ شجرة تصنيف على بيانات نتحكّم في شكلها، ثم نقارن ما سنصل إليه مع ما تختاره المكتبة.

الخطوة صفر: البيانات.

```python
# البيانات التي سنبني عليها الشجرة يدوياً
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(n_samples=200, n_features=3, n_informative=3,
                           n_redundant=0, class_sep=2.2, random_state=42)
df = pd.DataFrame(X, columns=["x1", "x2", "x3"])
df["label"] = y

print(f"shape         : {df.shape}")
print(f"class balance : {np.bincount(y)}  (0 = class A, 1 = class B)")
print()
print(df.head(8).to_string(index=False))
```

الخطوة الأولى: شوائب الجذر قبل أي قسمة.

```python
شوائب الجذر قبل أي قسمة
root_probs = np.bincount(y) / len(y)


def gini_of(labels):
    counts = np.bincount(labels, minlength=2).astype(float)
    p = counts / counts.sum()
    return float(1.0 - np.sum(p ** 2))


def entropy_of(labels):
    counts = np.bincount(labels, minlength=2).astype(float)
    p = counts[counts > 0] / counts.sum()
    return float(-np.sum(p * np.log2(p)))


print(f"class proportions       : {np.round(root_probs, 4)}")
print(f"Gini of the whole set   : {gini_of(y):.4f}")
print(f"Entropy of the whole set: {entropy_of(y):.4f} bits")
print(f"hand check: 1 - sum(p^2): {1 - np.sum(root_probs ** 2):.4f}")
print(f"maximum entropy, 2 classes: {np.log2(2):.4f} bits")
```

هذه الدالتان تُكتبان بنفسك

لا توجد دالة `gini_score` أو `entropy_score` في `sklearn.metrics`. المكتبة تستخدمهما داخلياً، لكنها لا تعرّضهما. كتابتها بنفسك عشر أسطر، وهي أفضل فهماً من استدعاء جاهز.

الخطوة الثانية: تقسيم كل ميزة عند وسيطها، وقياس المكسب.

```python
ما الذي يحدث حين نقسم كل ميزة عند وسيطها
def weighted_gini(labels, mask):
    left, right = labels[mask], labels[~mask]
    n = len(labels)
    return (len(left) / n) * gini_of(left) + (len(right) / n) * gini_of(right)


root_gini = gini_of(y)
print(f"root Gini = {root_gini:.4f}\n")
for j, name in enumerate(["x1", "x2", "x3"]):
    m = np.median(X[:, j])
    mask = X[:, j] <= m
    w = weighted_gini(y, mask)
    print(f"split on {name} <= {m:7.3f} : weighted Gini = {w:.4f}"
          f"   gain = {root_gini - w:+.4f}   left n = {mask.sum()}")
```

لاحظ كيف تهبط قيمة الشوائب المرجّحة إلى ما دون خُمس قيمتها قبل القسمة. هذه هي «رحلة النقاء»: الشجرة تكسب أرضاً في كل خطوة.

الخطوة الثالثة: الشجرة لا تكتفي بالوسيط. تبحث عن **كل** عتبة ممكنة.

البحث في كل العتبات الممكنة — python

```python
best_overall = []

for j, name in enumerate(["x1", "x2", "x3"]):
    gains = []
    for v in np.unique(X[:, j])[:-1]:
        mask = X[:, j] <= v
        if mask.all() or (~mask).all():
            continue
        gains.append((root_gini - weighted_gini(y, mask), v, int(mask.sum())))
    g, v, n_left = max(gains)
    best_overall.append((g, name, v))
    print(f"{name}: best gain {g:.4f} at threshold {v:8.4f}  (left n =
{n_left})")

gain, best_feature, best_threshold = max(best_overall)
print(f"\n-> the tree splits the root on {best_feature} <=
{best_threshold:.4f}")
print(f"-> it prefers {best_feature} because its gain ({gain:.4f}) is the
largest")
```

الخطوة الرابعة: نكرّر العملية داخل كل فرع. هذه الدورة هي كل ما يفعله `fit`.

يتولّى هذه الدورة كاملةً fit — python

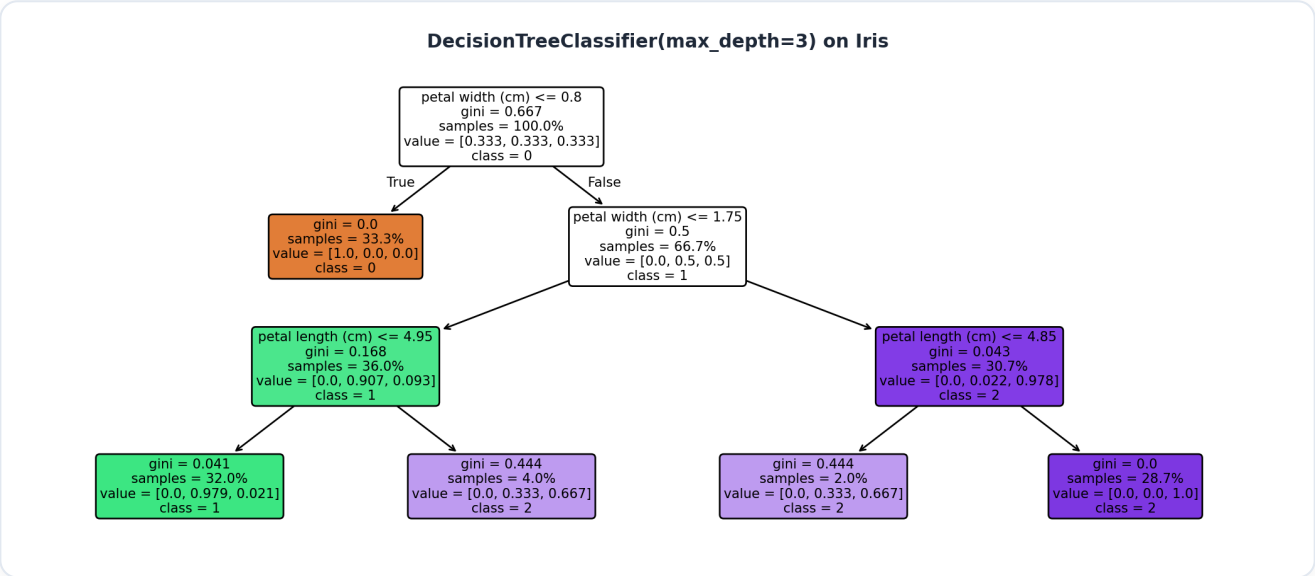
```python
clf = DecisionTreeClassifier(criterion="gini", max_depth=3,
random_state=42).fit(X, y)

print(f"tree depth        : {clf.get_depth()}")
print(f"number of leaves  : {clf.get_n_leaves()}")
print(f"features seen     : {clf.n_features_in_}")
print(f"root feature      : x{clf.tree_.feature[0]}")
print(f"root threshold    : {clf.tree_.threshold[0]:.4f}")
print()
imp = pd.Series(clf.feature_importances_, index=["x1", "x2", "x3"])
print("feature_importances_:")
print(imp.to_frame("importance").to_string())
```

لاحظ `feature_importances_` : هي **نسبة من مجموع انخفاضات الشوائب** عبر الشجرة كلها — أي كم تحسّنت دالة الشوائب بسبب استخدام هذه الميزة. ليست معاملاً خطّياً، ولا يمكن مقارنتها عددياً بأهمية نموذج آخر.

> **لماذا لا تعمل `feature_names_in_` هنا**
>
> `feature_names_in_` تظهر فقط إذا مرّرت إلى `fit` مصفوفة `DataFrame` فيها أسماء أعمدة. مع `ndarray` لا أسماء، فتتوفّر `n_features_in_` بدلاً منها. جرّب أن تعيد البناء نفسه وأنت تمرّر `df.drop(columns="label")` لترى الفرق.

٥.٤ كيف تقرأ الشجرة بصرياً؟

الشكل أدناه يوضّح الشجرة نفسها. الألوان تمثّل نسبة كل صنف داخل الورقة، وحجم الصندوق يمثّل عدد الصفوف.



الشكل ٨٠شجرة قرار بعمق ثلاثة على ثلاث كتل: الجذر يقسم على الميزة الأكثر انفصالاً ثم يتكرّر السؤال داخل كل فرع

لاحظ ثلاثة أشياء فوراً:

٥.٥ من ID٣ إلى C٤.٥: لماذا تغيّر المسار؟

الخوارزمية التي طبّقناها للتو تُسمّى **ID٣** (باروس، ١٩٨٦). تعمل جيداً، لكن لها ثلاث مشاكل معروفة:

المشكلة	ID٣	C٤.٥ وما نفعله في scikit-learn
الميزات المستمرة	يتوقف، لا يعرف أين يقسم	يبحث عن **كل عتبة ممكنة** على المحور
الإفراط في الملاءمة	ينمو بلا سقف	`min_samples_split` و `min_samples_leaf` و `max_depth` و `ccp_alpha`
القيم المفقودة	لا يعرف ماذا يفعل	يTreat `NaN` كقيمة ثالثة منفصلة

هذه ليست تفاصيل تاريخية: كل واحدة منها يقابل بارامتراً في `DecisionTreeClassifier` ستصادفه في الفصل التالي.

٥.٥.١ كيف تُعامَل القيم المفقودة فعلاً؟

هذه من أجمل أفكار scikit-learn وأكثرها إثارة للدهشة. الخوارزمية لا تملأ القيم المفقودة بالمتوسط. تسأل: **هل الانقسام «مفقود أصغر أو يساوي قيمة» أفضل من «مفقود أكبر»؟** وتعامل `NaN` كقيمة ذات ترتيب خاص.

شجرة تتعلّم اتجاه القيم المفقودة بنفسها — python

```python
from sklearn.datasets import load_breast_cancer

Xb, yb = load_breast_cancer(return_X_y=True)
Xb_holes = Xb.copy()
rng = np.random.default_rng(0)
holes = rng.random(Xb.shape) < 0.06
Xb_holes[holes] = np.nan

gapped = DecisionTreeClassifier(max_depth=4, random_state=0).fit(Xb_holes, yb)

print(f"features in            : {gapped.n_features_in_}")
print(f"missing cells fed in   : {int(holes.sum())}")
print(f"accuracy on gapped set : {gapped.score(Xb_holes, yb):.4f}")
print(f"nodes routing on NaN   : {int(gapped.tree_.missing_go_to_left.sum())}")
print()
print("NaN is not an error here. It is a third option: <= v, > v, or is-
missing.")
```

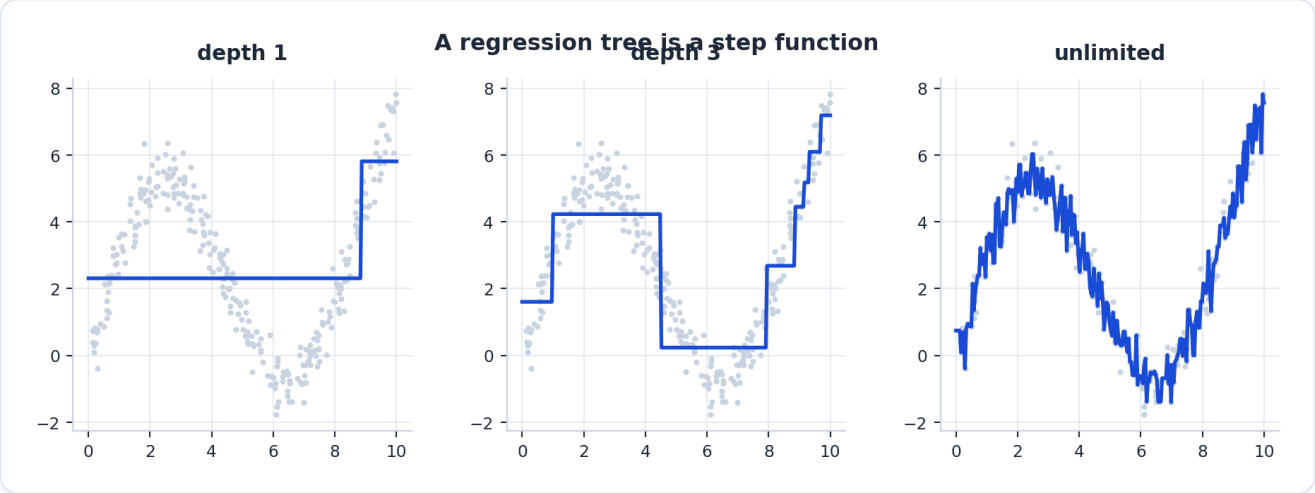
> ⚠ **هذا ليس سحراً، له ثمن**
>
> الشجرة التي تتعلّم القيمة المفقودة تنتج `tree_` تحتوي عقداً من نوع `missing_go_to_left`. هذا يعني أن النموذج **يُفسِّر** بيانات ناقصة دون أن تضطر لتعبئتها. لكنه يعني أيضاً أن `feature_importances_` لم تعد موزّعة على النفس، فاحذر المقارنة النسبية مع نموذج آخر.

٥.٦ متى تفشل الشجرة — بطبيعتها لا بخطأ

هذا هو الجزء الذي يغفل عنه كثيرون.

٥.٦.١ الانحدار الخطي لا يمكن تمثيله

شجرة بعمق واحد على بيانات خطية مناسبة = مستطيل واحد. أي منحدر مستقيم يُرسم سلّمًا بدرجات. لا يوجد أي معامل يجعل المنحدر مستقيماً.



الشكل ٨١ثلاث لوحات لشجرة انحدار: عمق واحد، وعمق ثلاثة، وعمق بلا حد — الفرق في مقدار الانحناء وميل النموذج إلى حفظ الضجيج

٥.٦.٢ محور واحد لا يكفي

مثال شهير: `horsepower` وحده يعطي `R²` قريباً من ٠.٨ في تنبؤ سعر السيارة. أضف `weight`، فيحتاج النموذج بعدين. ارسمها في ثلاثة أبعاد، فتحتاج ثلاثة محاور للرسم، وقد لا تكفي أربع شُعب. **حدود شجرة القرار عمودية وأفقية دائماً** مهما كانت هندسة المسافة الحقيقية في بياناتك.

٥.٦.٣ التفسير له سقف

شجرة بعمق اثني عشر على عشرة آلاف صف تعطي آلاف العقد. هذا ليس تفسيراً، هذا إعادة كتابة جدول البيانات. شجرة القرار تفسّر نفسها فقط حين تظلّ **صغيرة**.

٥.٧ لماذا يستحق الأمر مقارنة بالنماذج الخطية؟

المعيار	شجرة القرار	الانحدار الخطي	
شكل القرار	مستطيلات ومحاور	خط مستقيم واحد	
معالجة القيم	تحتاج تطبيعاً للمقياس	تحتاج توسيطاً	scale
التفاعل بين الميزات	تلقائي، لأن الشجرة تفاعل بطبيعتها	يجب بناؤه يدوياً	
قراءة المعاملات	شرط واضح	معنى إحصائي	
بيانات غير خطية	ممتاز	ضعيف	
قيد الرتابة	مدعوم عبر `monotonic_cst`	غير مدعوم	
الحساسية لتغيير واحد	عالية جداً	منخفضة	

هنا بالضبط يدخل `RandomForest` (الفصل ٤٣) **مقترناً** بشجرة واحدة: الغابة تعطيك أفضل دقة، والشجرة الواحدة تُبقي لك التفسير.

٥.٨ تمارين الفصل

التمرين ٥.١

ابنِ شجرة تصنيف على `load_iris` بـ `max_depth=2`. اطبع `get_depth()` و `get_n_leaves()`، وحدّد أي الميزات استعملتها الشجرة في الجذر. تحقّق من إجابتك بـ `sklearn.tree.export_text`.

التمرين ٥.٢

على البيانات نفسها، ارسم `feature_importances_` وفسّر لماذا `petal length` تهيمن. ثم استبدل البيانات بـ `make_classification` فيها ضجيج وأعد الرسم. ماذا تغيّر ولماذا؟ هذا يوضّح الفرق بين «أهمية بسبب البنية» و«أهمية بسبب الصدفة».

التمرين ٥.٣

اكتب دالة اسمها `gini_of(labels)` تُعيد قيمة الجيني، ودالة `best_gain(X, y)` تُعيد أفضل `(feature, threshold, gain)`. ثم قارن ناتجها مع `DecisionTreeClassifier(criterion='gini', max_depth=1).tree_.feature` و `.threshold`. إن تطابقا فقد أعدتَ بناء ID٣ من الصفر.

التمرين ٥.٤

أدخل 百分之 عشرين قيم مفقودة عشوائية إلى `X` وركّب شجرة بعمق أربعة. اطبع `gapped.tree_.missing_go_to_left.sum()` وفسّر لماذا الرقم موجب.

٥.١٠ الأخطاء الشائعة

الخطأ	ما يحدث	الحل
نسيان `random_state`	شجرتان مختلفتان عند كل تشغيل	ثبّت البذرة دائماً
قراءة `feature_importances_` **كـ«نسبة من التباين»**	استنتاج خاطئ عند وجود مميزات مترابطة	استعمل `permutation_importance`
`max_depth=None` **من البداية**	الإفراط في الملاءمة يقتل التعميم	ابدأ بـ `max_depth=5` و `min_samples_leaf=5`
مقارنة عمق شجرة على مجموعتين مختلفتين	استنتاج مضلّل	ثبّت البذرة وقِس على نفس الطقس
توقّع أن الشجرة تحتاج تطبيعاً للمقياس	عمود بمليار يهيمن على كل التقسيمات	لا تحتاج scaling، لكن راقب الأهمية

٥.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
شجرة القرار	نموذج يقسّم الفضاء ميزةً ميزة	decision tree
الشوائب	مقياس عدم نقاء المجموعة	impurity
مكسب المعلومات	انخفاض الشوائب عن القسمة	information gain
الجيني	شوائب مبنية على احتمال عدم تطابق عيّنتين	Gini index
إنتروبيا	شوائب مبنية على عدم اليقين	entropy
ورقة نقية	ورقة كل صفوفها من صنف واحد	pure leaf
التقسيم	سؤال يفرّق البيانات إلى مجموعتين	split
التشذيب	تقليم العقد الزائدة بعد التدريب	pruning
قيمة مفقودة بسلك خاص	قيمة ثالثة في المقارنة	missing value as branch

الجزء ٥ · أشجار القرار

الفصل ٣٨ — شجرة الانحدار وتقسيم خطأ مربّع

التباين، والانحراف المعياري، ومتوسط خطأ الشجرة

الوحدة **scikit-learn ١.٩** الحجم **٣٧٨** كلمة تقريباً

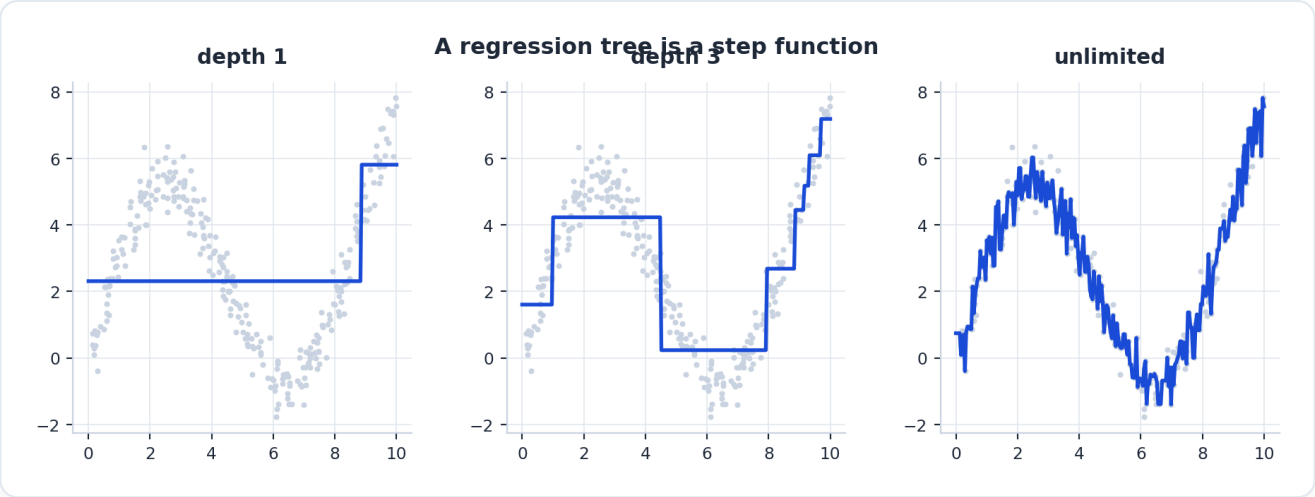
ما الذي ستتقنه في هذا الفصل

- أن تشتق معيار الانقسام بانخفاض التباين (Variance Reduction) في شجرة الانحدار
- أن تطبق DecisionTreeRegressor وتقارن معايير الانقسام المختلفة
- أن تفسر التنبؤ المدرج لشجرة الانحدار ولماذا تعجز بنيوياً عن الاستقراء الخارجي
- أن تقيّم أثر عمق الشجرة على مقاييس MSE و MAE وتحدد نقطة التحول

٥.١ من التصنيف إلى الانحدار: استبدال جيني بالتباين

في الفصل السابق رأينا كيف تستخدم شجرة القرار شوائب جيني (Gini Impurity) أو الإنتروبيا لتقسيم العينات الفئوية. ولكن ماذا نفعل عندما يكون المتغير المستهدف y عدداً حقيقياً مستمراً (مثل أسعار المنازل أو انبعاثات الكربون أو درجات الحرارة)؟

في شجرة الانحدار (Regression Tree)، نستبدل نقاء الفئات بـ **تجانس القيم العددية**:



الشكل ٨٢ انقسام فضاء الانحدار إلى مستويات ومناطق تنبؤ مدرجة

٥.٢ خوارزمية CART لاختيار أفضل تقسيم انحداري

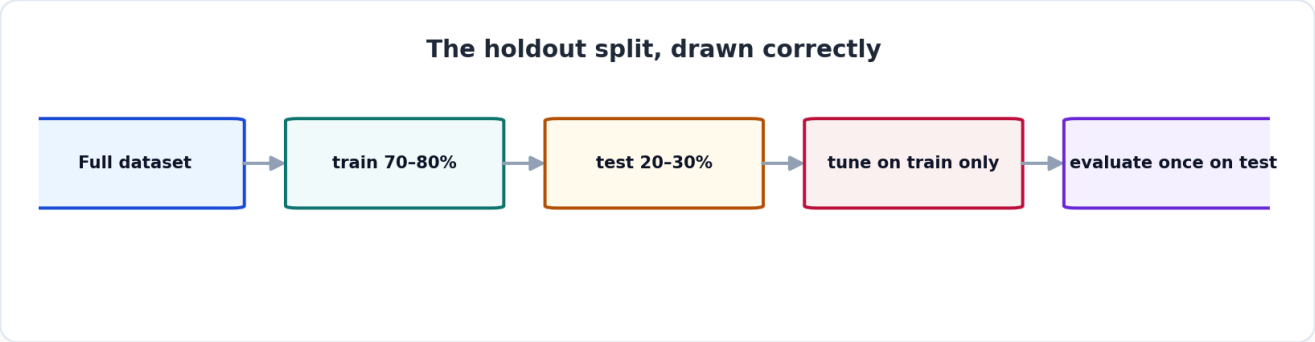
عند كل عقدة، تبحث خوارزمية CART عن الميزة j وعتبة التقسيم t التي تقسم العينات إلى منطقتين $R_1(j,t) = \{x | x_j \leq t\}$ و $R_2(j,t) = \{x | x_j > t\}$ بحيث تحقق **أقصى انخفاض في التباين (Variance Reduction)**:

$$\min_{j,t}[\frac{N_1}{N}H(R_1) + \frac{N_2}{N}H(R_2)]$$

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# توليد دالة غير خطية جيبية
rng = np.random.default_rng(42)
X = np.sort(rng.uniform(-3, 3, size=(80, 1)), axis=0)
y = np.sin(X).ravel() + rng.normal(scale=0.1, size=80)

# تدريب شجرتي انحدار بعمقين مختلفين
tree_shallow = DecisionTreeRegressor(max_depth=2, random_state=42).fit(X, y)
tree_deep = DecisionTreeRegressor(max_depth=5, random_state=42).fit(X, y)

print(f"عدد الأوراق في الشجرة الضحلة (depth=2): {tree_shallow.get_n_leaves()}")
print(f"MSE التدريب للشجرة الضحلة: {mean_squared_error(y,
tree_shallow.predict(X)):.4f}")
print(f"عدد الأوراق في الشجرة العميقة (depth=5): {tree_deep.get_n_leaves()}")
print(f"MSE التدريب للشجرة العميقة: {mean_squared_error(y, tree_deep.predict(X)):.4f}")
```

٥.٣ الطبيعة الدرجية وعجز الاستقراء الخارجي (Extrapolation)

من أهم الخصائص الهيكلية لشجرة الانحدار التي يجب أن يدركها كل مهندس تعلّم آلة عالمي هي أنها **دالة ثابتة مجزأة (Piecewise Constant)**:



الشكل ٨٣تشريح انقسام العقدة وحدود الأوراق المستطيلة في فضاء الميزات

```python
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import LinearRegression

# بيانات ذات اتجاه صاعد واضح (Linear Trend)
X_train = np.array([[1.0], [2.0], [3.0], [4.0], [5.0]])
y_train = np.array([2.0, 4.0, 6.0, 8.0, 10.0])

tree = DecisionTreeRegressor().fit(X_train, y_train)
linear = LinearRegression().fit(X_train, y_train)

# اختبار عينة استقرائية جديدة كلياً X=10.0 (خارج نطاق التدريب [1, 5])
X_extrapolate = np.array([[10.0]])

print(f"القيمة الحقيقية المتوقعة للاتجاه عند X=10.0: 20.0")
print(f"تنبؤ النموذج الخطي (LinearRegression):     {linear.predict(X_extrapolate)
[0]:.1f} (استقراء مثالي)")
print(f"تنبؤ شجرة الانحدار (DecisionTreeRegressor): {tree.predict(X_extrapolate)
[0]:.1f} (تجمّد عند متوسط آخر ورقة!)")
```

> ⛔ **تحذير**
>
> **الاستقراء الزمني والاتجاهات**: لا تستخدم أشجار القرار المفردة (أو حتى الغابات العشوائية التقليدية) للتنبؤ المباشر بالسلاسل الزمنية التي تحوي اتجاهاً عاماً صاعداً أو هابطاً (Trend)؛ لأنها ستعجز حتماً عن التنبؤ بأي قيمة تفوق الحد الأقصى التاريخي في بيانات التدريب!

٥.٤ معايير الانقسام في scikit-learn

يدعم المعامل `criterion` في `DecisionTreeRegressor` أربعة خيارات متقدمة:

المعيار	دالة الخسارة المحسوبة	متى تختاره؟
`"squared_error"` **(الافتراضي)**	متوسط مربعات الخطأ (MSE)	الخيار القياسي المرجعي في معظم الحالات
`"friedman_mse"`	MSE مع تعديل فريدمان للتقسيمات	مخصص ومثالي عند استخدام الأشجار داخل تعزيز التدرج Boosting
`"absolute_error"`	متوسط الخطأ المطلق (MAE)	مقاوم للشواذ؛ الورقة تتنبأ بـ **الوسيط Median** بدلاً من المتوسط
`"poisson"`	انحراف بواسون للعد	عند نمذجة معدلات الحدوث والبيانات ذات القيم الصحيحة غير السالبة

```
from sklearn.datasets import fetch_california_housing
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1500], data.target[:1500], test_size=0.2, random_state=42
)

# مقارنة بين معيار الخطأ المربع ومعيار الخطأ المطلق المقاوم للشواذ
tree_mse = DecisionTreeRegressor(criterion="squared_error", max_depth=6,
random_state=42).fit(X_tr, y_tr)
tree_mae = DecisionTreeRegressor(criterion="absolute_error", max_depth=6,
random_state=42).fit(X_tr, y_tr)

pred_mse = tree_mse.predict(X_te)
pred_mae = tree_mae.predict(X_te)

print(f"معيار squared_error | MSE الاختبار: {mean_squared_error(y_te, pred_mse):.4f}
| MAE: {mean_absolute_error(y_te, pred_mse):.4f}")
print(f"معيار absolute_error | MSE الاختبار: {mean_squared_error(y_te, pred_mae):.4f}
| MAE: {mean_absolute_error(y_te, pred_mae):.4f}")
```

٥.٥ مثال تطبيقي شامل: تحليل مسار الأوراق وحساب التباين يدوياً

```
from sklearn.tree import DecisionTreeRegressor
import numpy as np

# مثال مصغر لفهم الحسابات الداخلية
X_toy = np.array([[1], [2], [3], [4], [6], [7]])
y_toy = np.array([1.0, 1.2, 1.1, 5.0, 5.2, 4.9])

tree_toy = DecisionTreeRegressor(max_depth=1).fit(X_toy, y_toy)

# استخراج عتبة الانقسام
threshold = tree_toy.tree_.threshold[0]
feature = tree_toy.tree_.feature[0]
left_value = tree_toy.tree_.value[1][0][0]
right_value = tree_toy.tree_.value[2][0][0]

print(f"التقسيم الأمثل الذي وجدته الشجرة: X <= {threshold:.2f}")
print(f"(1,2,3 متوسط العينات) قيمة التنبؤ للفرع الأيسر: {left_value:.3f} (الحقيقي:
{y_toy[:3].mean():.3f})")
print(f"(4,5,6 متوسط العينات) قيمة التنبؤ للفرع الأيمن: {right_value:.3f} (الحقيقي:
{y_toy[3:].mean():.3f})")
```

تمارين الفصل الثامن والثلاثين

- **ملاحظة**

التمرين ٥.١

على مجموعة بيانات California Housing، درب شجرة انحدار باختبار أعماق مختلفة `max_depth` من ١ إلى ١٥. احسب مقياس RMSE على كل من بيانات التدريب وبيانات التحقق، وحدد العمق الأمثل الذي تبدأ بعده الشجرة في الإفراط في الملاءمة.

التمرين ٥.٢

أضف ٥% عينات شاذة بقيم ضخمة إلى مصفوفة الهدف y. درب شجرتين: الأولى بـ `criterion="squared_error"` والثانية بـ `criterion="absolute_error"`. كيف تأثرت تنبؤات كل شجرة على العينات السليمة؟

التمرين ٥.٣

أثبت بالبرمجة أن دالة `tree.predict(X)` تعيد تماماً مصفوفة `tree.tree_.value` المفهرسة برقم الورقة الناتجة عن `tree.apply(X)`.

- **ملاحظة**

```
from sklearn.datasets import fetch_california_housing
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error
import numpy as np

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1200], data.target[:1200], test_size=0.3, random_state=42
)

best_depth = 1
lowest_rmse = float("inf")

for depth in [2, 4, 6, 8, 10, 14]:
    reg = DecisionTreeRegressor(max_depth=depth, random_state=42).fit(X_tr,
y_tr)
    rmse_tr = root_mean_squared_error(y_tr, reg.predict(X_tr))
    rmse_te = root_mean_squared_error(y_te, reg.predict(X_te))
    if rmse_te < lowest_rmse:
        lowest_rmse = rmse_te
        best_depth = depth
    print(f"عمق {depth:2d} | RMSE تدريب: {rmse_tr:.3f} | RMSE اختبار:
{rmse_te:.3f}")

print(f"\nهو العمق الأمثل للاختبار: {best_depth} بأدنى خطأ: {lowest_rmse:.3f}")
```

> 📝 **ملاحظة**

```
from sklearn.tree import DecisionTreeRegressor
import numpy as np

X = np.array([[10], [20], [30], [40], [50]])
y = np.array([1.5, 2.5, 3.5, 4.5, 5.5])

reg = DecisionTreeRegressor(max_leaf_nodes=3, random_state=42).fit(X, y)

# 1. التنبؤ التلقائي المباشر
preds_direct = reg.predict(X)

# 2. التنبؤ اليدوي عبر أرقام الأوراق
leaf_indices = reg.apply(X)
preds_manual = np.array([reg.tree_.value[leaf_idx][0][0] for leaf_idx in
leaf_indices])

print("تنبؤات predict المباشرة:", preds_direct)
print("تنبؤات مطابقة أوراق tree_.value:", preds_manual)
print(f"هل التطابق تام عبر كل العينات؟ {np.allclose(preds_direct, preds_manual)}")
```

> **ملاحظة**

٥.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام شجرة انحدار مفردة للتنبؤ بسلاسل زمنية ذات اتجاه صاعد	تجمّد التنبؤ عند أقصى قيمة تاريخية وفشل التنبؤ بالمستقبل	نمذجة الاتجاه بـ LinearRegression أولاً أو استخدام الفروق
ترك شجرة الانحدار تنمو بلا قيود على العمق	شجرة تحفظ العينات وتهبط بـ MSE التدريب للصفر مع إخفاق الاختبار	ضبط قيود مثل `max_depth` أو `min_samples_leaf`
توقع مخرجات منحنية ملساء من شجرة الانحدار	ظهور مخرجات متدرجة حادة على شكل قفزات عتبية	دمج الأشجار في غابات عشوائية أو استخدام نماذج خطية وملساء
استخدام `absolute_error` على بيانات مليونية دون داعٍ	بطء حسابي مضاعف لأن حساب الوسيط يتطلب فرزاً مستمراً	استخدام `squared_error` الافتراضي السريع

٥.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
شجرة الانحدار	شجرة قرار مخصصة للتنبؤ بمتغيرات هدف عددية مستمرة	Regression tree
تقليص التباين	معيار جودة الانقسام المبني على تقليل تشتت القيم في الفروع	Variance reduction
دالة ثابتة مجزأة	دالة رياضية تأخذ قيماً مسطحة ثابتة داخل نطاقات محددة	Piecewise constant function
عجز الاستقراء	عدم قدرة النموذج على التنبؤ باتجاهات خارج نطاق التدريب	Extrapolation failure
الورقة الطرفية	العقدة النهائية في الشجرة التي تصدر القيمة التنبؤية	Leaf node

الجزء ٥ · أشجار القرار

الفصل ٣٩ — التشذيب والإفراط في الملاءمة

decision_path و ccp_alpha و cost_complexity_pruning والتصوّر

الوحدة scikit-learn ١.٩ الحجم ٣٤٤ كلمة تقريباً

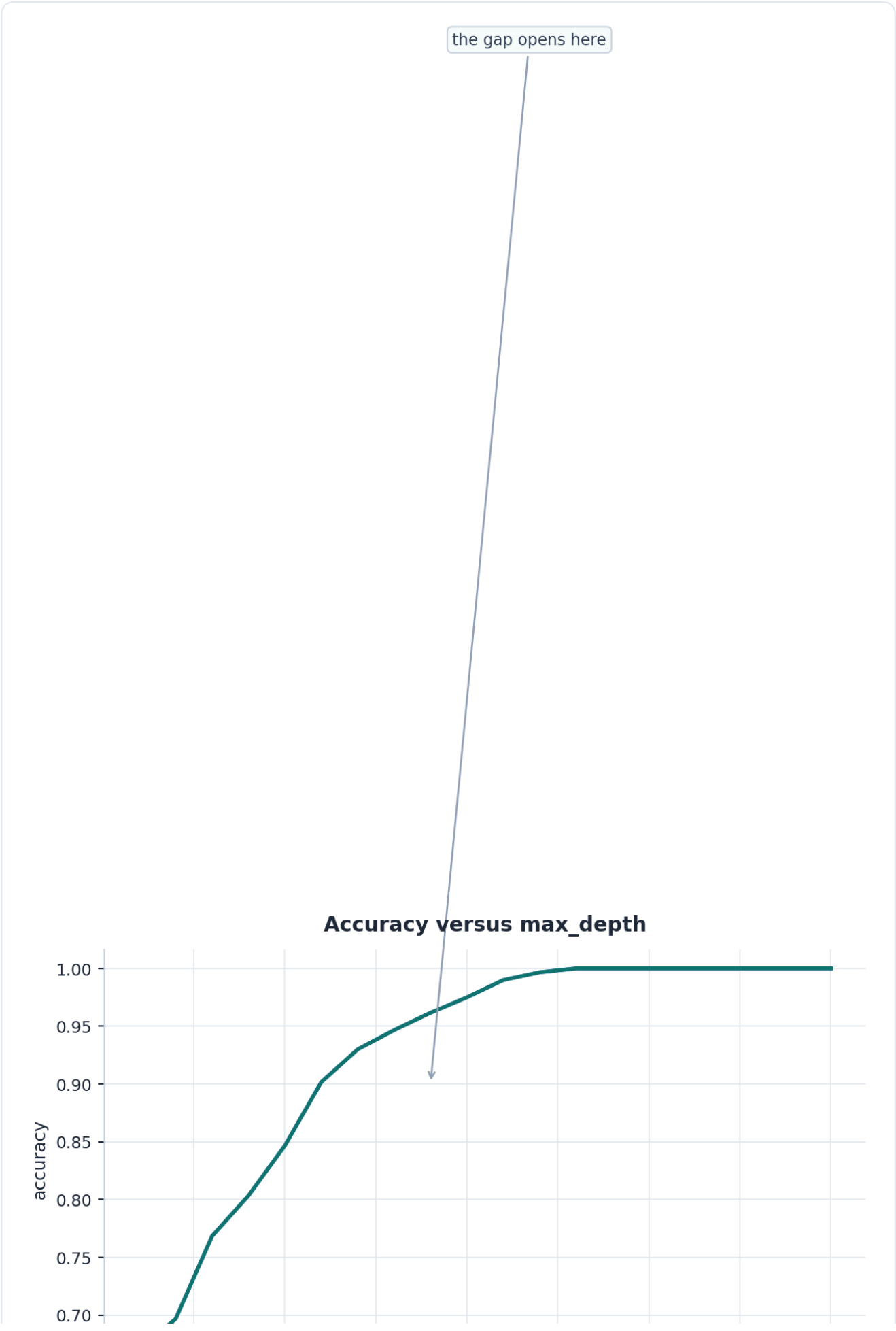
ما الذي ستتقنه في هذا الفصل

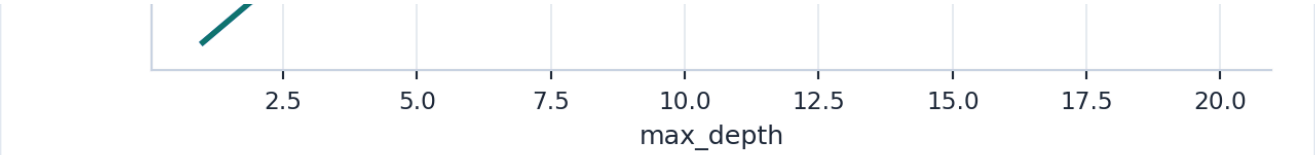
- أن تفهم دالة تعقيد التكلفة الرياضية ودور معامل الجزاء ccp_alpha
- أن تستخرج مسار التشذيب كاملاً عبر cost_complexity_pruning_path
- أن تختار ccp_alpha الأمثل بالتحقق المتقاطع لمنع الإفراط في الملاءمة
- أن تستخرج مسار القرار المنطقي لعينة مفردة عبر decision_path

٥.١ الطبيعة الجشعة ومصيدة الإفراط في الملاءمة

تتبع خوارزمية بناء الشجرة أسلوب **البحث الجشع (Greedy Search)** من الأعلى إلى الأسفل: تختار أفضل تقسيم متاح لحظياً دون النظر إلى المستقبل. إذا لم نضع قيوداً صارمة، فستستمر الشجرة في تفريع العقد حتى تصبح كل ورقة نقية تماماً (Gini = ٠)، لتصل دقة التدريب إلى ١٠٠%، بينما تنهار دقة الاختبار إلى الحضيض!

هناك استراتيجيتان لمكافحة هذا الإفراط:


the gap opens here
Accuracy versus max_depth
accuracy
1.00
0.95
0.90
0.85
0.80
0.75
0.70



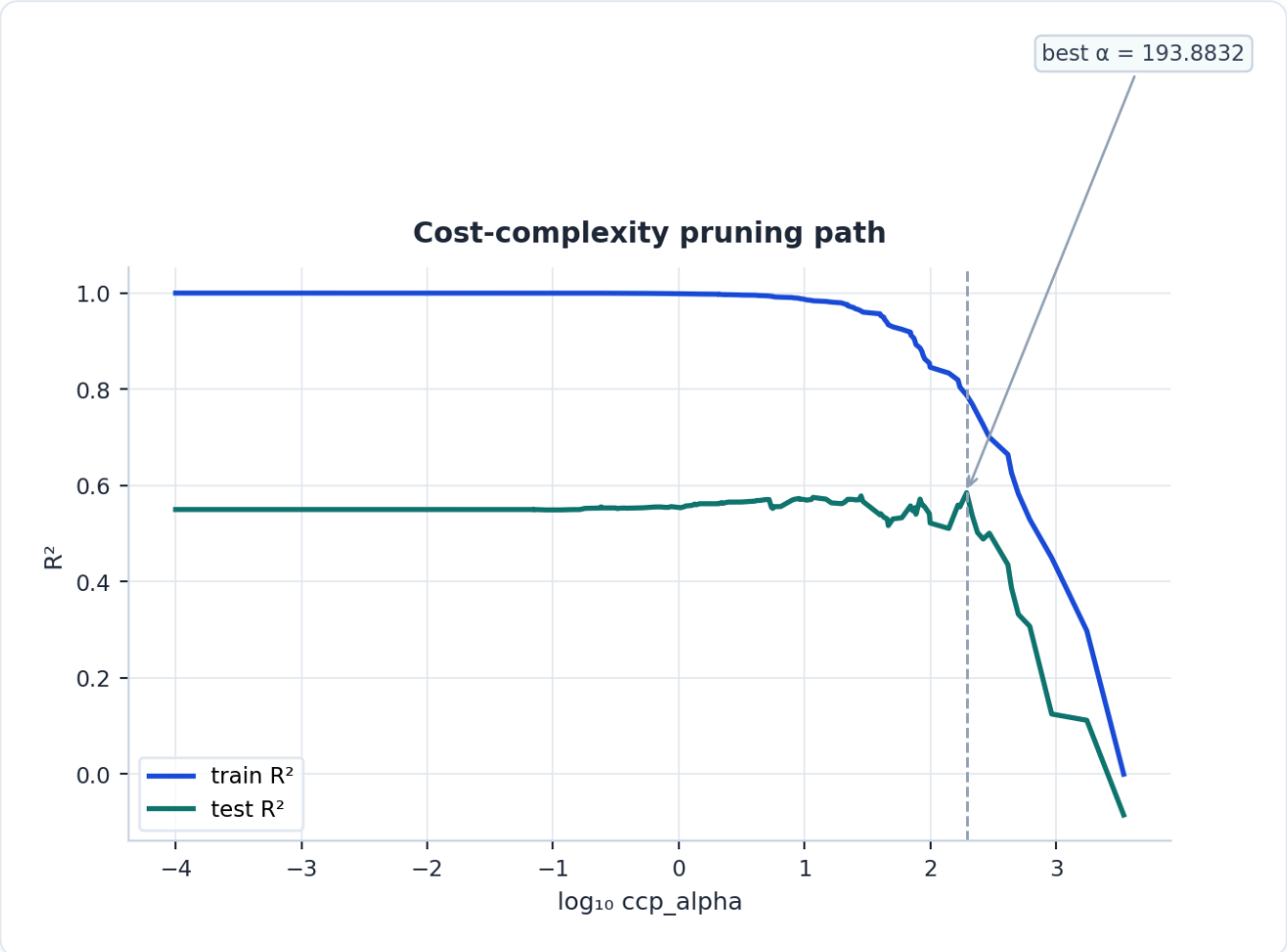
الشكل ٨٤أثر تعقيد الشجرة وتزايد العمق على فجوة الأداء بين التدريب والاختبار

٥.٢ تشذيب التكلفة والتعقيد: دالة الهدف

تعتمد خوارزمية التشذيب البعدي في scikit-learn على مبدأ **Minimal Cost-Complexity Pruning**:

$$R_\alpha(T) = R(T) + \alpha|T|$$

حيث:



الشكل ٨٥مسار تشذيب التكلفة والتعقيد وتناقص عدد الأوراق مع زيادة معامل ccp_alpha

٥.٣ استخراج مسار التشذيب عبر cost*complexity*pruning_path

يوفر المقدر دالة `cost_complexity_pruning_path` التي تحسب بدقة متناهية سلسلة قيم α الحرجة التي تنهار عندها الفروع بالتتابع:

```
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# تدريب شجرة كاملة أولاً
clf = DecisionTreeClassifier(random_state=42)
path = clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

print(f"عدد قيم alpha الحرجة المستخرجة: {len(ccp_alphas)}")
print(f"أصغر قيمة alpha (الشجرة الكاملة): {ccp_alphas[0]:.6f}")
print(f"أكبر قيمة alpha (تقليص لجذر وحيد): {ccp_alphas[-1]:.6f}")
```

٥.٤ اختيار ccp_alpha الأمثل بالتحقق المتقاطع

نقوم بتدريب شجرة لكل قيمة من قيم `ccp_alphas` الناتجة ومراقبة توازن الدقة:

```python
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, train_test_split
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

clf = DecisionTreeClassifier(random_state=42)
path = clf.cost_complexity_pruning_path(X_train, y_train)
# استبعاد القيمة الأخيرة لأنها تشذب كل شيء وتترك ورقة واحدة
ccp_alphas = path.ccp_alphas[:-1]

# فحص عينة من قيم alpha بالتحقق المتقاطع
selected_alphas = ccp_alphas[::max(1, len(ccp_alphas)//10)]
cv_scores = []

for alpha in selected_alphas:
    t = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)
    score = cross_val_score(t, X_train, y_train, cv=5).mean()
    cv_scores.append(score)

best_idx = np.argmax(cv_scores)
best_alpha = selected_alphas[best_idx]

# تقييم الشجرة المشذبة مقابل الشجرة غير المشذبة على الاختبار المستقل
tree_unpruned = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
tree_pruned = DecisionTreeClassifier(ccp_alpha=best_alpha,
random_state=42).fit(X_train, y_train)

print(f"أفضل ccp_alpha مختار: {best_alpha:.5f}")
print(f"الشجرة غير المشذبة: {tree_unpruned.get_n_leaves():2d} | عدد الأوراق | دقة الاختبار:
{tree_unpruned.score(X_test, y_test):.4f}")
print(f"الشجرة المشذبة: {tree_pruned.get_n_leaves():2d} | عدد الأوراق | دقة الاختبار:
{tree_pruned.score(X_test, y_test):.4f}")
print(f"انخفاض حجم الشجرة بنسبة: {1 -
tree_pruned.get_n_leaves()/tree_unpruned.get_n_leaves():.1%} مع تحسن التعميم!")
```

نصيحة

الشجرة المشذبة ليست فقط أعلى دقة في التعميم؛ بل هي أصغر حجماً وأسهل تفسيراً وأسرع بكثير في زمن الاستنتاج (Inference Latency) عند النشر في بيئات الإنتاج الحساسة للسرعة.

٥.٥ تتبع مسار القرار عبر decision_path

تتيح دالة `decision_path` استخراج مصفوفة متفرقة توضح بالضبط كل عقدة مرت بها العينة من الجذر حتى الورقة النهائية:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
clf = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X, y)

# استخراج مسار القرار للعينة الأولى
sample = X[:1]
node_indicator = clf.decision_path(sample)
leaf_id = clf.apply(sample)

node_index =
node_indicator.indices[node_indicator.indptr[0]:node_indicator.indptr[1]]

print(f"العينة تنتمي في النهاية للورقة رقم: {leaf_id[0]}")
print("تسلسل العقد التي مرت بها العينة في رحلة اتخاذ القرار:")
for node_id in node_index:
    if leaf_id[0] == node_id:
        print(f"  --> العقدة الطرفية #{node_id} (إصدار الحكم)")
    else:
        feat = clf.tree_.feature[node_id]
        thresh = clf.tree_.threshold[node_id]
        print(f"  العقدة #{node_id}: فحص الميزة '{feat}' بمقارنتها مع العتبة {thresh:.2f}")
```

تمارين الفصل التاسع والثلاثين

- **ملاحظة**

التمرين ٥.١

على بيانات Wine (`load_wine`)، قارن بين الشجرة المشذبة بواسطة `ccp_alpha` والشجرة المقيدة قبلياً بواسطة `max_depth=3` . هل حقق التشذيب البعدي دقة اختبار أفضل ولماذا؟

التمرين ٥.٢

ارسم بيانياً منحنى تغير عدد الأوراق ومقياس دقة الاختبار مع تزايد قيم `ccp_alphas` خطوة بخطوة.

التمرين ٥.٣

استخدم دالة `decision_path` على ١٠ عينات اختبار، واحسب متوسط عدد القواعد الشرطية (عمق المسار) التي تحتاجها كل عينة للوصول إلى قرار.

- **ملاحظة**

```
from sklearn.datasets import load_wine
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

# 1. تقييد قبلي
pre_tree = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_tr, y_tr)

# 2. تشذيب بعدي
full_tree = DecisionTreeClassifier(random_state=42).fit(X_tr, y_tr)
path = full_tree.cost_complexity_pruning_path(X_tr, y_tr)
# اختيار alpha متوسط فعال
post_tree =
DecisionTreeClassifier(ccp_alpha=path.ccp_alphas[len(path.ccp_alphas)//2],
random_state=42).fit(X_tr, y_tr)

print(f"دقة التقييد القبلي (max_depth=3): {pre_tree.score(X_te, y_te):.4f} (أوراق:
{pre_tree.get_n_leaves()})")
print(f"دقة التشذيب البعدي (ccp_alpha):   {post_tree.score(X_te, y_te):.4f} (أوراق:
{post_tree.get_n_leaves()})")
```

ملاحظة

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
import numpy as np

X, y = load_iris(return_X_y=True)
clf = DecisionTreeClassifier(random_state=42).fit(X, y)

# فحص مسار 15 عينة
indicators = clf.decision_path(X[:15])
# عدد العقد المزارة هو عدد الواحدات في كل صف
path_lengths = np.diff(indicators.indptr)

print(f"أطوال مسارات اتخاذ القرار للعينات: {list(path_lengths)}")
print(f"متوسط عدد الأسئلة اللازمة لإصدار الحكم: {path_lengths.mean():.1f} سؤالاً")
```

ملاحظة

٥.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام قيمة `ccp_alpha` ضخمة جداً عشوائياً	تشذيب مفرط يختزل الشجرة في عقدة جذرية عاجزة	اختيار `ccp_alpha` دوماً عبر `cost_complexity_pruning_path`
الاقتصار على تقييد `max_depth` وتجاهل التشذيب	تفويت تقسيمات نافعة محجوبة خلف فروع ضحلة	الجمع بين ضبط العمق والتشذيب البعدي بـ `ccp_alpha`
حساب `cost_complexity_pruning_path` على بيانات الاختبار	تسرب معلومات وتعيين غير عادل لمعامل التشذيب	حساب المسار على بيانات التدريب حصراً
الاعتماد على الشجرة الكاملة دون تشذيب في بيئات الإنتاج	بطء الاستنتاج وتدهور حاد عند مواجهة بيانات حقيقية جديدة	تشذيب الشجرة لتبسيطها ورفع سرعة التنفيذ

٥.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تشذيب الشجرة	قص الأفرع والأوراق الهامشية التي تسبب الإفراط في الملاءمة	Tree pruning
تعقيد التكلفة الأدنى	خوارزمية تشذيب بعدية تعاقب عدد الأوراق بمعامل خطي	Minimal Cost-Complexity Pruning
معامل تعقيد التكلفة	وزن العقوبة الجزائية المفروضة على كل ورقة إضافية	ccp_alpha
مسار القرار	سلسلة العقد والشروط التي تعبرها العينة حتى تبلغ الورقة	Decision path
الشوائب الكلية	مجموع أخطاء أو عدم نقاء جميع العقد الورقية في الشجرة	Total impurity

الجزء ٥ · أشجار القرار

الفصل ٤٠ — ضبط شجرة القرار

العمق، وحدّ الأوراق، و class_weight، والقيود، والقيم الناقصة

الوحدة **scikit-learn ١.٩** الحجم **٣٤١** كلمة تقريباً

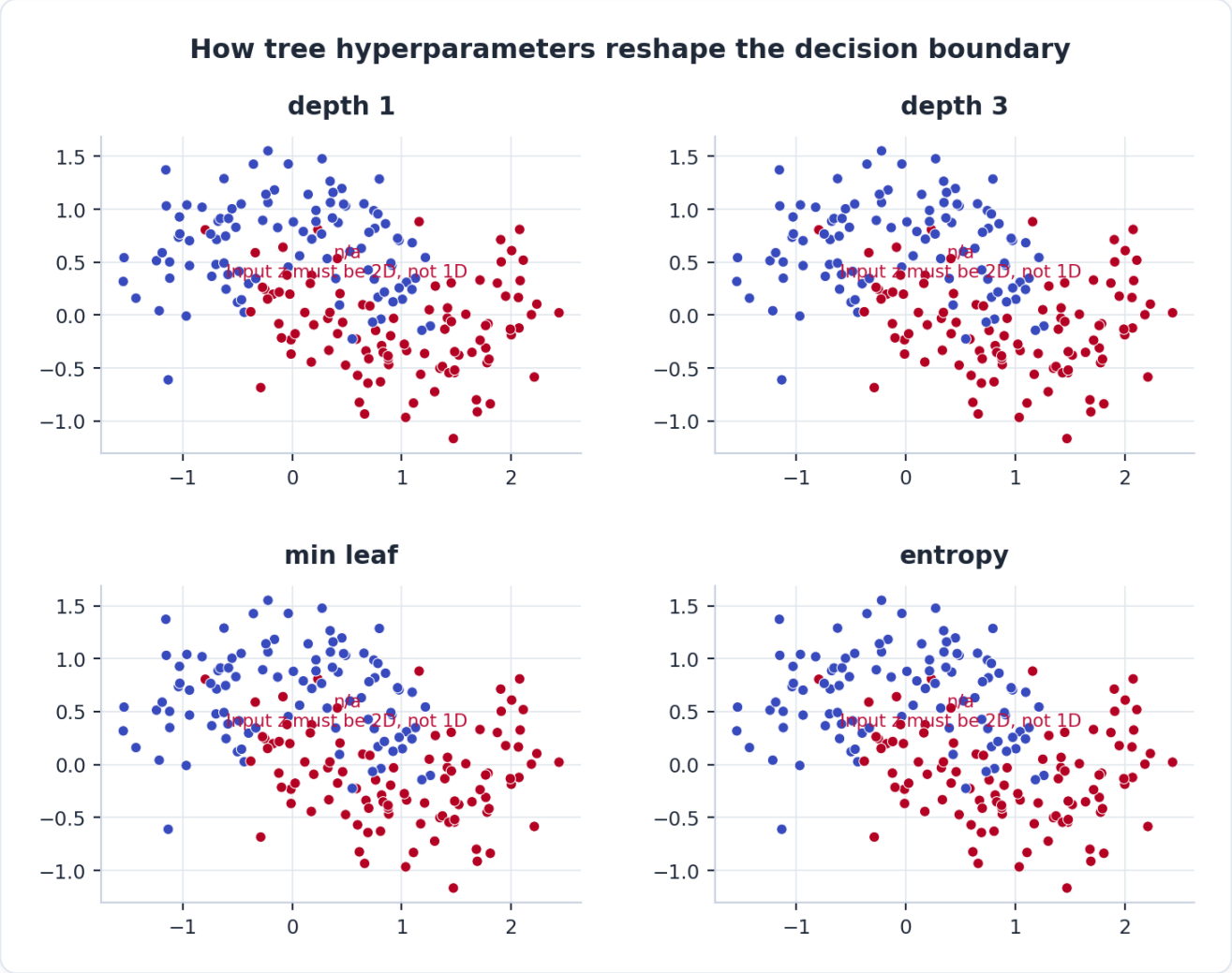
ما الذي ستتقنه في هذا الفصل

- أن تتقن ضبط عائلة معلمات التقييد: max_depth و min_samples_leaf و max_leaf_nodes
- أن تطبق وزن الفئات class_weight='balanced' للتعامل مع البيانات غير المتوازنة
- أن تستغل الدعم الأصيل المباشر للقيم المفقودة داخل أشجار القرار في scikit-learn
- أن تنفذ بحثاً شبكياً عشوائياً RandomizedSearchCV للضبط المتوازن الفعال

٥.١ لوحة التحكم: معلمات تقييد الشجرة

بينما رأينا في الفصل السابق قوة التشذيب البعدي، تظل معلمات **التقييد المسبق (Pre-pruning constraints)** هي خط الدفاع الأول الذي يحفظ استقرار الشجرة وسرعتها الحسابية.

تنقسم معلمات التحكم في حجم الشجرة إلى فلسفتين:



الشكل ٨٦ المستويات الفائقة والتقسيمات الهندسية لشجرة القرار في فضاء الميزات

```python
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# مقارنة بين تقييد العمق max_depth وتقييد الأوراق max_leaf_nodes
tree_depth = DecisionTreeClassifier(max_depth=4, random_state=42).fit(X_train,
y_train)
tree_leaves = DecisionTreeClassifier(max_leaf_nodes=8,
random_state=42).fit(X_train, y_train)

print(f"تقييد max_depth=4    | عدد الأوراق: {tree_depth.get_n_leaves():2d} | دقة الاختبار:
{tree_depth.score(X_test, y_test):.4f}")
print(f"تقييد max_leaf_nodes=8 | عدد الأوراق: {tree_leaves.get_n_leaves():2d} | دقة
الاختبار: {tree_leaves.score(X_test, y_test):.4f}")
```

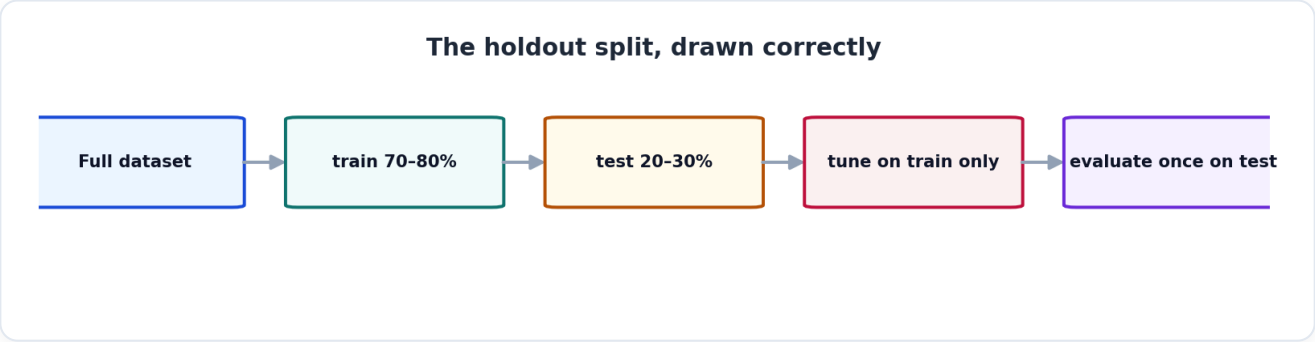
٥.٢ موازنة الفئات النادرة: سحر class_weight

عندما تكون البيانات غير متوازنة بشدة (مثل اكتشاف الاحتيال المالي أو الأمراض النادرة بنسبة ١:٩٩):

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
import numpy as np

rng = np.random.default_rng(42)
X = rng.normal(size=(500, 10))
# فئة نادرة بنسبة 5% فقط
y = rng.choice([0, 1], size=500, p=[0.95, 0.05])

clf_default = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X, y)
clf_balanced = DecisionTreeClassifier(max_depth=3, class_weight="balanced",
random_state=42).fit(X, y)

pred_def = clf_default.predict(X)
pred_bal = clf_balanced.predict(X)

print(f"عدد العينات المتنبأ بها كفئة نادرة (الافتراضي): {np.sum(pred_def == 1)} (تجاهل شبه تام)")
print(f"عدد العينات المتنبأ بها كفئة نادرة (balanced):   {np.sum(pred_bal == 1)} (حساسية واستجابة حقيقية)")
```

٥.٣ الدعم الأصيل للقيم المفقودة (Native Missing Values)

منذ التحديثات الحديثة والمعتمدة رسمياً في **scikit-learn ١.٩**، تدعم أشجار القرار `DecisionTreeClassifier` و `DecisionTreeRegressor` **القيم المفقودة (`np.nan`) تلقائياً وأصيلاً دون الحاجة لاستخدام** `SimpleImputer`:



الشكل ٨٧تشريح بنية الشجرة وتوجيه العينات عبر مسارات الانقسام

```python
from sklearn.tree import DecisionTreeClassifier
import numpy as np

# مصفوفة بيانات تحتوي قيماً مفقودة صريحة
X_missing = np.array([
    [2.5, np.nan],
    [np.nan, 1.2],
    [3.1, 0.8],
    [1.1, np.nan],
    [np.nan, 3.5],
    [4.0, 2.9]
])
y_missing = np.array([0, 1, 0, 0, 1, 1])

# التدريب مباشرة دون أي Imputer خارجي!
tree_nan = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_nan.fit(X_missing, y_missing)

# التنبؤ لعينة جديدة تحوي قيماً مفقودة
X_new = np.array([[np.nan, 0.5], [3.5, np.nan]])
predictions = tree_nan.predict(X_new)

print("تم تدريب الشجرة على بيانات تحوي np.nan مباشرة بنجاح!")
print(f"التنبؤ للعينات الجديدة المحتوية على قيم مفقودة: {list(predictions)}")
```

> **ملاحظة**
>
> الدعم الأصيل للقيم المفقودة في الأشجار يلغي الحاجة إلى افتراضات التعويض (كالتعويض بالمتوسط)، ويستغل غياب القيمة كـ **إشارة معلومية في حد ذاتها (Missingness as Information)**.

٥.٤ الضبط الشامل عبر RandomizedSearchCV

بدلاً من ضبط كل معلمة بمفردها، نستخدم البحث العشوائي الذكي لاستكشاف التفاعل بين المعلمات:

```python
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

X, y = load_breast_cancer(return_X_y=True)

# فضاء المعلمات المرجعي لشجرة متزنة
param_dist = {
    "max_depth": randint(2, 10),
    "min_samples_leaf": randint(2, 20),
    "max_leaf_nodes": randint(6, 30),
    "criterion": ["gini", "entropy", "log_loss"]
}

search = RandomizedSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=25,
    cv=5,
    scoring="roc_auc",
    random_state=42,
    n_jobs=-1
)
search.fit(X, y)

print(f"أفضل نتيجة ROC-AUC: {search.best_score_:.4f}")
print("المعلمات الفائقة المثلى:")
for param, val in search.best_params_.items():
    print(f"  {param:<18}: {val}")
```

٥.٥ دليل الإعدادات: متى تضبط ماذا؟

المعلمة	القيمة الافتراضية	التوصية العملية العالمية
`max_depth`	`None` (بلا حد)	اضبطه بين `3` و `8` لمنع التفرع العشوائي المفرط
`max_leaf_nodes`	`None`	الأفضل بين `8` و `32` للحصول على شجرة عالية الدقة وبسيطة
`min_samples_leaf`	`1`	ارفعه إلى `5` أو `10` لضمان عدم اتخاذ قرارات مبنية على عينات شاذة فردية
`min_impurity_decrease`	`0.0`	ضعه `0.001` لإيقاف التقسيمات الهامشية التي لا تقدم مكسباً يذكر
`class_weight`	`None`	ضعه `"balanced"` دوماً إذا كانت نسبة الفئة الصغرى أقل من ٣٠%

تمارين الفصل الأربعين

- **ملاحظة**

التمرين ٥.١

على بيانات California Housing، درب شجرتين: الأولى بـ `min_samples_leaf=1` والثانية بـ `min_samples_leaf=20`. قارن حجم الشجرة الناتج ومعدل الخطأ على عينات الاختبار النظيفة.

التمرين ٥.٢

أنشئ مصفوفة بيانات تحوي ٣٠% قيماً مفقودة موزعة عشوائياً. قارن دقة `DecisionTreeClassifier` المباشر بدقة خط أنابيب يستخدم `SimpleImputer(strategy="mean")` قبل الشجرة. أيهما حقق أداءً أعلى؟

التمرين ٥.٣

قارن بين زمن تنفيذ البحث الشبكي الشامل `GridSearchCV` والبحث العشوائي `RandomizedSearchCV` لضبط شجرة قرار على فضاء يحوي ٥٠٠ تركيبة ممكنة.

- **ملاحظة**

```
from sklearn.datasets import fetch_california_housing
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1000], data.target[:1000], test_size=0.3, random_state=42
)

t1 = DecisionTreeRegressor(min_samples_leaf=1, random_state=42).fit(X_tr, y_tr)
t20 = DecisionTreeRegressor(min_samples_leaf=20, random_state=42).fit(X_tr,
y_tr)

print(f"min_samples_leaf=1  | أوراق: {t1.get_n_leaves():3d} | MSE تدريب:
{mean_squared_error(y_tr, t1.predict(X_tr)):.4f} | MSE اختبار:
{mean_squared_error(y_te, t1.predict(X_te)):.4f}")
print(f"min_samples_leaf=20 | أوراق: {t20.get_n_leaves():3d} | MSE تدريب:
{mean_squared_error(y_tr, t20.predict(X_tr)):.4f} | MSE اختبار:
{mean_squared_error(y_te, t20.predict(X_te)):.4f}")
```

ملاحظة

```
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
rng = np.random.default_rng(42)

# إتلاف 20% من القيم بجعلها مفقودة
X_corrupt = X.copy()
mask = rng.uniform(0, 1, size=X.shape) < 0.2
X_corrupt[mask] = np.nan

# 1. الدعم الأصيل في scikit-learn
score_native = cross_val_score(DecisionTreeClassifier(random_state=42),
X_corrupt, y, cv=5).mean()

# 2. التعويض بالمتوسط عبر SimpleImputer
pipe_impute = make_pipeline(SimpleImputer(strategy="mean"),
DecisionTreeClassifier(random_state=42))
score_impute = cross_val_score(pipe_impute, X_corrupt, y, cv=5).mean()

print(f"دقة الدعم الأصيل المباشر (Native Missing): {score_native:.4f}")
print(f"دقة التعويض التقليدي (SimpleImputer):      {score_impute:.4f}")
```

> **ملاحظة**

٥.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الاعتماد على القيم الافتراضية تماماً دون أي تقييد	نمو شجرة ضخمة تحفظ البيانات تماماً وتفشل في التعميم	ضبط `max_leaf_nodes` أو `max_depth` كإجراء وقائي بديهي
تجاهل عدم توازن الفئات في التصنيف	التنبؤ بالفئة الغالبة فقط والحصول على دقة مضللة	تفعيل `class_weight='balanced'` واعتماد مقياس ROC-AUC
تطبيق `SimpleImputer` قسرياً قبل الشجرة دون داعٍ	فقدان الإشارة الدلالية لنمط فقدان البيانات	الاستفادة من الدعم الأصيل المدمج لتمرير `np.nan` مباشرة
استخدام `min_samples_split` دون ضبط `min_samples_leaf`	قد تنشأ أوراق طرفية تحوي عينة واحدة فقط	مرافقة المعلمتين معاً أو تفضيل ضبط `min_samples_leaf`

٥.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التقييد المسبق	فرض قيود تحد من نمو الشجرة أثناء مرحلة البناء	Pre-pruning
توسع الأفضل أولاً	خوارزمية تنمي العقدة ذات أعلى عائد تقليص للشوائب	Best-first expansion
وزن الفئات	مضاعفة أوزان عينات الفئات النادرة لمعالجة عدم التوازن	Class weights
القيم المفقودة الأصيلة	دعم معالجة غياب القيم داخل خوارزمية الشجرة مباشرة	Native missing values
الحد الأدنى لعينات الورقة	أقل عدد عينات مسموح بوجوده داخل العقدة الطرفية	min*samples*leaf

الجزء ٥ · أشجار القرار

الفصل ٤١ — قراءة الشجرة وتفسيرها

export_text و plot_tree و feature_importances_ ومثال يدوي كامل

الوحدة **sklearn.tree** الحجم **٣٤٥** كلمة تقريباً

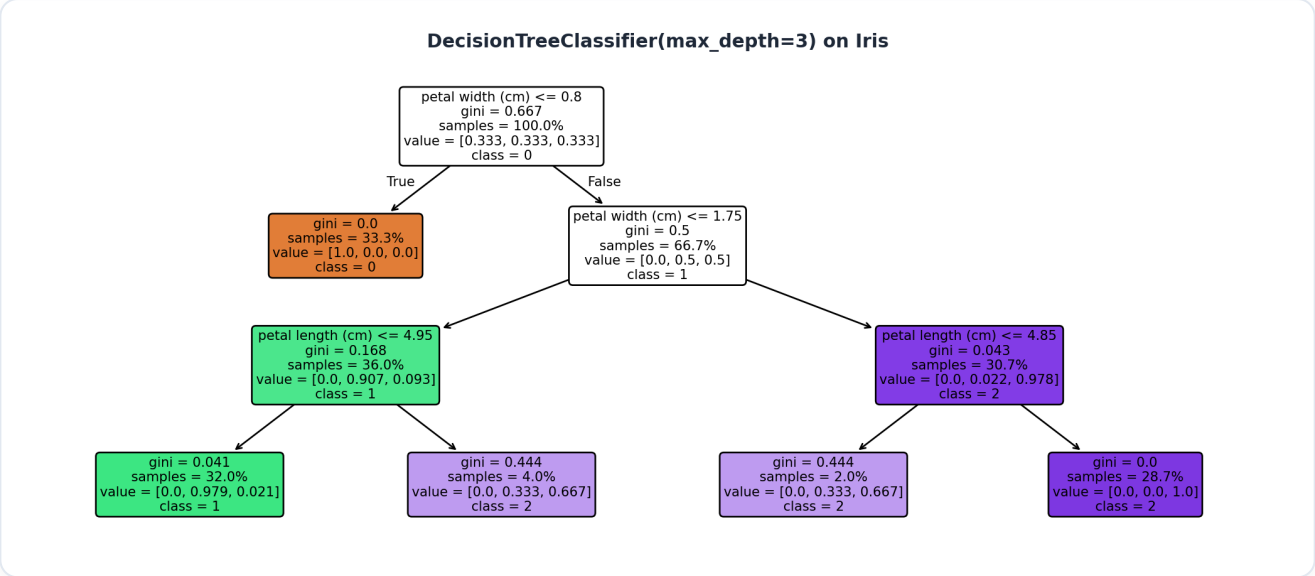
ما الذي ستتقنه في هذا الفصل

- أن تستخرج قواعد الشجرة بصيغة نصية واضحة وقابلة للتدقيق عبر export_text
- أن ترسم شجرة القرار وتنسق ألوانها وتسمياتها باحتراف باستخدام plot_tree
- أن تحسب أهمية الميزات feature_importances_ يدوياً من انخفاض شوائب العقد
- أن تدرك تحيز أهمية الشجرة للميزات عالية التعددية (High-Cardinality) وتتجنبه

٥.١ ميزة الصندوق الأبيض: النموذج الذي يشرح نفسه

في عالم يزداد فيه التدقيق التنظيمي والأخلاقي على نماذج الذكاء الاصطناعي (مثل قرارات الائتمان البنكي والتشخيص الطبي)، تمثل النماذج المعتمة (Black-box) كـالشبكات العميقة مخاطرة قانونية وتقنية كبرى.

تتفرد **شجرة القرار** بقدرتها على الإفصاح عن كل منطقها الداخلي دون الحاجة لأي أدوات تفسيرية معقدة:



الشكل ٨٨ التصوير البصري لانقسامات الشجرة ومسارات اتخاذ القرار المنطقي

٥.٢ التصدير النصي للقواعد المنطقية عبر export_text

أسرع وسيلة لفحص الشجرة في الطرفية أو تضمينها في سجلات التشغيل (Production Logs) هي دالة `export_text`:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text

iris = load_iris()
X, y = iris.data, iris.target

clf = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X, y)

# تصدير منطق الشجرة بأسماء الميزات والفئات الصريحة
tree_rules = export_text(
    clf,
    feature_names=list(iris.feature_names),
    class_names=list(iris.target_names),
    spacing=3,
    decimals=2
)

print("القواعد المنطقية الصريحة المستخرجة من الشجرة:")
print(tree_rules[:450])
```

٥.٣ التصوير البصري المتكامل عبر plot_tree

تتيح دالة `plot_tree` رسم مخطط شجري أنيق يلون العقد وفق الفئة الغالبة ودرجة نقائها:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree

iris = load_iris()
X, y = iris.data, iris.target
clf = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X, y)

fig, ax = plt.subplots(figsize=(8, 5))
_ = plot_tree(
    clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True,
    fontsize=9,
    ax=ax
)
plt.tight_layout()
plt.close(fig)

print(f"تم بناء المخطط البصري للشجرة بنجاح بعمق {clf.get_depth()} وعدد أوراق
{clf.get_n_leaves()}!")
```

> **نصيحة**
>
> خاصية `filled=True` تلون العقد بتدرج لوني يعكس نقاء العقدة: كلما كان اللون داكناً وقوياً، دل ذلك على نقاء أعلى وانخفاض كبير في شوائب جيني، بينما تدل الألوان الباهتة على عُقد مختلطة لا تزال بحاجة لتفريع.

٥.٤ الحساب الرياضي اليدوي لأهمية الخصائص

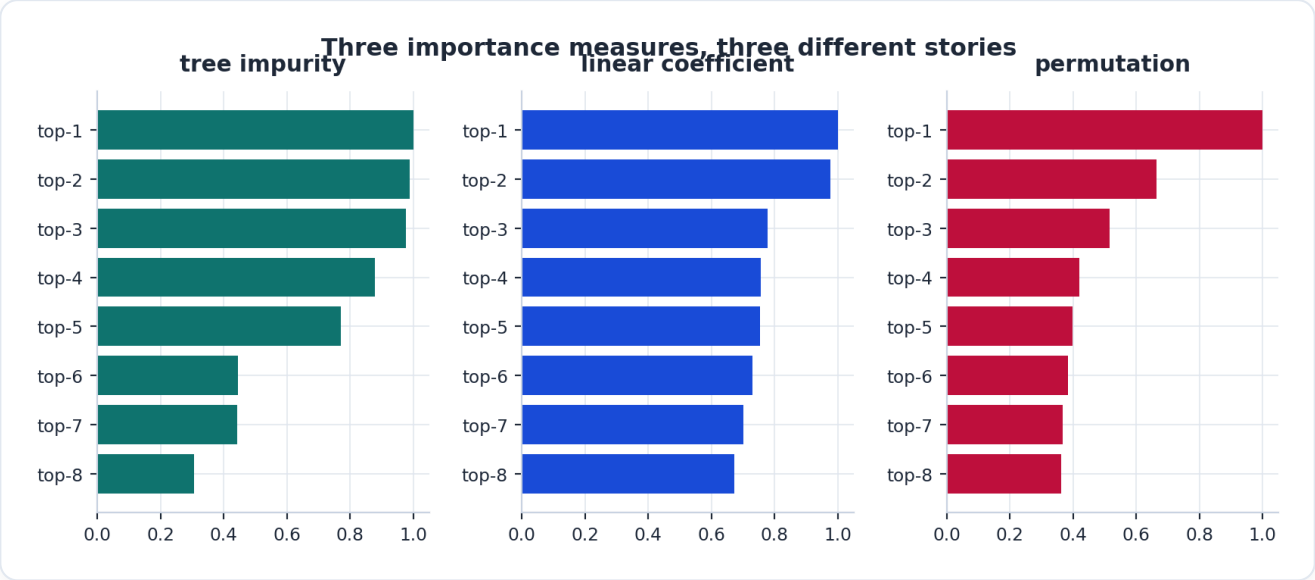
كيف يحسب scikit-learn خاصية `feature_importances_` في أشجار القرار؟

الرياضيات واضحة وممنهجة:

```python
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
import numpy as np

iris = load_iris()
X, y = iris.data, iris.target
clf = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X, y)

tree = clf.tree_
n_nodes = tree.node_count

# حساب الأهمية يدوياً
manual_importances = np.zeros(X.shape[1])
total_samples = tree.n_node_samples[0]

for node in range(n_nodes):
    # فحص إذا كانت العقدة غير طرفية (لها أبناء)
    if tree.children_left[node] != -1:
        feat = tree.feature[node]
        left = tree.children_left[node]
        right = tree.children_right[node]

        n_parent = tree.n_node_samples[node]
        n_left = tree.n_node_samples[left]
        n_right = tree.n_node_samples[right]

        imp_parent = tree.impurity[node]
        imp_left = tree.impurity[left]
        imp_right = tree.impurity[right]

        # انخفاض الشوائب الموزون
        decrease = (n_parent / total_samples) * imp_parent - \
                   (n_left / total_samples) * imp_left - \
                   (n_right / total_samples) * imp_right

        manual_importances[feat] += decrease

# التطبيع ليصبح المجموع 1.0
manual_importances /= manual_importances.sum()

print("مقارنة حساب أهمية الميزات:")
for i, name in enumerate(iris.feature_names):
    print(f"  {name:<25} | اليدوي: {manual_importances[i]:.4f} | المدمج:
{clf.feature_importances_[i]:.4f}")

print(f"\nتطابق الحساب اليدوي مع مخرجات scikit-learn التامة:
{np.allclose(manual_importances, clf.feature_importances_)}")
```

٥.٥ التحيز القاتل: متى تخدعك feature*importances*؟

على الرغم من انتشار `feature_importances_`، فإنها تعاني من **تحيز إحصائي شهير (Cardio-bias / Cardinality Bias)**:



الشكل ٨٩ مقارنة الطرق الثلاث لقياس الأهمية والتحقق من مصيدة التحيز

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.inspection import permutation_importance

rng = np.random.default_rng(42)
n_samples = 400

# ميزة 0: ثنائية مفيدة جداً ترتبط بالهدف
x_real = rng.binomial(1, 0.5, size=n_samples)
y = x_real.copy()

# ميزة 1: أرقام عشوائية مستمرة بالكامل (ضجيج عديم الفائدة ولكن بقيم فريدة هائلة!)
x_noise = rng.uniform(0, 100, size=n_samples)

X_bias = np.column_stack([x_real, x_noise])

clf = DecisionTreeClassifier(random_state=42).fit(X_bias, y)

# حساب أهمية الشوائب المدمجة
impurity_imp = clf.feature_importances_

# حساب أهمية الإبدال المستقلة permutation_importance
perm_imp = permutation_importance(clf, X_bias, y,
random_state=42).importances_mean

print(f"{'نوع الميزة':<25} {'أهمية الشوائب (المدمجة)':<25} {'أهمية الإبدال (الحقيقية)':<25}")
print("-" * 75)
print(f"{'الميزة الحقيقية (ثنائية)':<25} {impurity_imp[0]:>25.4f} {perm_imp[0]:>25.4f}")
print(f"{'الضجيج العشوائي (كثير التعدد)':<25} {impurity_imp[1]:>25.4f}
{perm_imp[1]:>25.4f}")
print("\nالخلاصة: أهمية الشوائب المدمجة تمنح الضجيج أهمية زائفة؛ بينما تكشف أهمية الإبدال الحقيقة!")
```

> ⛔ **تحذير**
>
> **أفضل الممارسات العالمية**: لا تعتمد أبداً على `feature_importances_` وحدها لاتخاذ قرارات استبعاد الخصائص في الإنتاج إذا كانت بياناتك تضم مزيجاً من ميزات فئوية قليلة وميزات عددية مستمرة. استخدم دائماً `permutation_importance` (التي درسناها في الفصل ٢٥) كحكم محايد.

تمارين الفصل الحادي والأربعين

- **ملاحظة**

التمرين ٥.١

درب شجرة قرار على بيانات `load_breast_cancer` بعمق `max_depth=3`. استخرج القواعد بصيغة `export_text`. ما هو السؤال الأول الذي طُرح في عقدة الجذر وما عتبته الرقمية؟

التمرين ٥.٢

أنشئ ميزة ضجيج عشوائية تماماً وأضفها إلى بيانات Iris. احسب أهمية الميزات عبر `clf.feature_importances_` وعبر `permutation_importance`. هل أخذت الميزة العشوائية أهمية أكبر من الصفر في الشوائب المدمجة؟

التمرين ٥.٣

باستخدام بنية الشجرة `clf.tree_`، احسب أقصى عمق فعلي وصل إليه فرع في الشجرة يدوياً عبر تتبع مؤشرات الأبناء `children_left` و `children_right`.

- **ملاحظة**

```
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier, export_text

cancer = load_breast_cancer()
clf = DecisionTreeClassifier(max_depth=3, random_state=42).fit(cancer.data,
cancer.target)

rules = export_text(clf, feature_names=list(cancer.feature_names), decimals=3)
first_rule = rules.strip().split("\n")[0]
print(f"قاعدة عقدة الجذر الأولى: {first_rule}")
```

> **ملاحظة**

```python
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
clf = DecisionTreeClassifier(max_depth=4, random_state=42).fit(iris.data,
iris.target)

def get_depth_manual(node=0):
    if clf.tree_.children_left[node] == -1:
        return 0
    left_d = get_depth_manual(clf.tree_.children_left[node])
    right_d = get_depth_manual(clf.tree_.children_right[node])
    return 1 + max(left_d, right_d)

print(f"العمق المحسوب يدوياً: {get_depth_manual(0)}")
print(f"العمق عبر دالة المقدر: {clf.get_depth()}")
```

> 📝 **ملاحظة**

٥.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الثقة المطلقة بـ `feature_importances_` بوجود ميزات مستمرة وضجيج	تصنيف ميزات الضجيج عالية القيم الفريدة كميزات جوهرية	استخدام `permutation_importance` على مجموعة التحقق
رسم شجرة عميقة جداً (depth > ٨) بـ `plot_tree`	نص متداخل وغير مقروء وفوضى بصرية عارمة	تقييد الرسم بـ `max_depth=3` داخل `plot_tree` للتوضيح
إغفال تمرير `feature_names` إلى `export_text`	ظهور أسماء مجهولة `feature_0` تعيق الفهم	تمرير قائمة بأسماء الأعمدة دائماً `feature_names=list(...)`
تفسير ارتباط الميزة في الشجرة كعلاقة سببية حتمية	الوقوع في مغالطة الارتباط والسببية	تذكر أن التقسيمات تعبر عن ارتباط إحصائي فقط

٥.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
نموذج الصندوق الأبيض	نموذج يسهل على البشر فهم منطقه الداخلي وقواعده	White-box model
التصدير النصي	استخراج منطق الشجرة على شكل قواعد برمجية مقروءة	export_text
أهمية الشوائب	نسبة الانخفاض الإجمالي في شوائب العقد المنسوبة للميزة	Impurity-based feature importance
تحيز التعددية	ميل الشجرة لتفضيل المتغيرات ذات القيم الفريدة الكثيرة	Cardinality bias
أهمية الإبدال	قياس الأهمية الفعلي بخلط قيم الميزة ومراقبة انخفاض الدقة	Permutation importance

الجزء ٦

المجموعات والأغلفة

«ليس نموذج واحداً كافياً»

ماذا يغطّي هذا الجزء

القوة الحقيقية للتعلّم الآلي تأتي من تجميع عشرات النماذج التي يخطئ كل منها بطريقة مختلفة. نبدأ من الغابة العشوائية، ثم التعزيز بنوعيه، ثم HistGradientBoosting الذي صار المعيار في جداول البيانات، وننتهي بالتكديس والتصويت. هنا نفهم لماذا تتفوّق هذه العائلة على نموذج مفرد.

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٢ — التجميع بالعيّنات: Bagging

التمهيد بالعيّنات، والدمج بالتصويت، والتقدير خارج العيّنة OOB

الوحدة **scikit-learn ١.٩** الحجم **٣٩٩** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

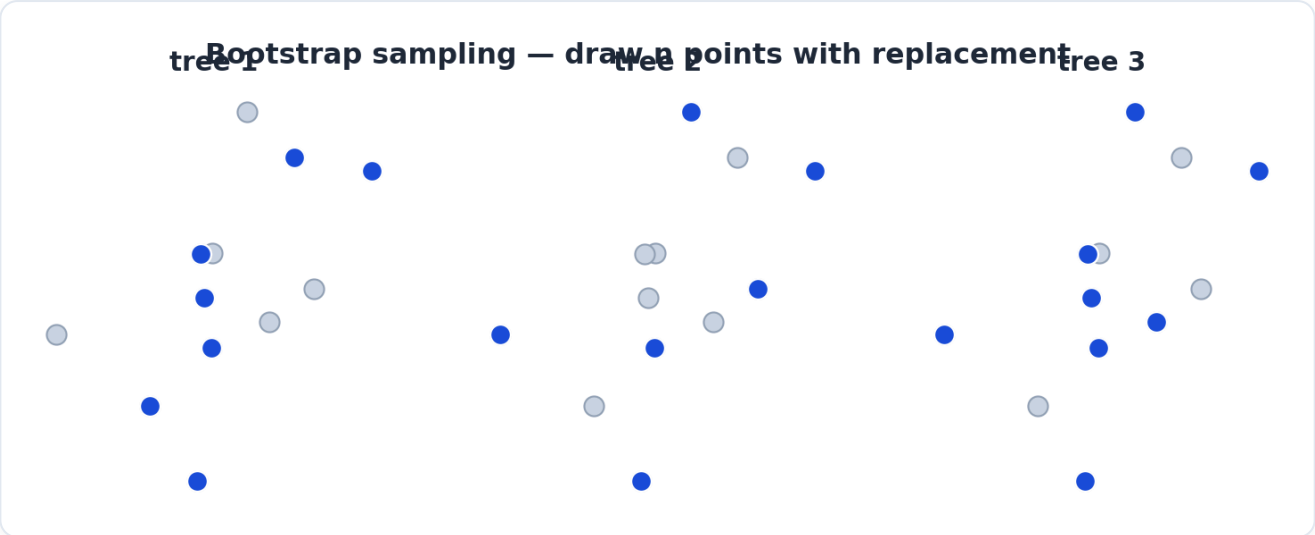
- أن تشرح كيف يؤدي تجميع النماذج غير المترابطة رياضياً إلى تخفيض التباين بعامل B/١
- أن تطبق BaggingClassifier و BaggingRegressor على نماذج قاعدية متنوعة
- أن تستغل تقييم Out-of-Bag (OOB) كبديل فوري ومجاني للتحقق المتقاطع
- أن تميز بين تقنيات أخذ العينات: Pasting و Random Subspaces و Random Patches

٦.١ حكمة الحشود: لماذا يفوق الجمع كل فرد؟

تخيل مسابقة يطلب فيها من ألف شخص تخمين وزن ثور ضخم في معرض ريفي. معظم الأفراد سيخطئون: بعضهم سيبالغ بالزيادة وبعضهم بالنقصان. لكن المفاجأة الإحصائية الكبرى التي لاحظها فرانسيس غالتون عام ١٩٠٦ هي أن **متوسط جميع التخمينات كان أدق من تخمين أمهر الخبراء بمفردهم**!

هذا المبدأ الإحصائي هو حجر الزاوية في **التعلّم التجميعي (Ensemble Learning)**: إذا كان لدينا B من النماذج المستقلة إحصائياً، وكان لكل نموذج تباين مقداره σ^2 :

كلما زاد عدد النماذج B ، تضاءل التباين الإجمالي واقترب من الصفر دون المساس بدقة النموذج أو زيادة انحيازه!



الشكل ٩٠سحب عينات التمهيد البوتسترابي بالإرجاع واستخراج المجموعات الفرعية

٦.٢ عينات التمهيد (Bootstrap Sampling) ونسبة ٦٣.٢%

كيف نحصل على B من النماذج المختلفة إذا لم نكن نملك سوى مجموعة بيانات تدريبية واحدة D بحجم n ؟

الحل هو **أخذ العينات بالتمهيد مع الإرجاع (Bootstrap Sampling with Replacement)**:

ما نسبة العينات التي تفلت من السحب في كل حقيبة؟ احتمال عدم اختيار عينة معينة في سحبة واحدة هو $1-\frac{1}{n}$. احتمال عدم اختيارها في جميع السحبات الـ n هو:

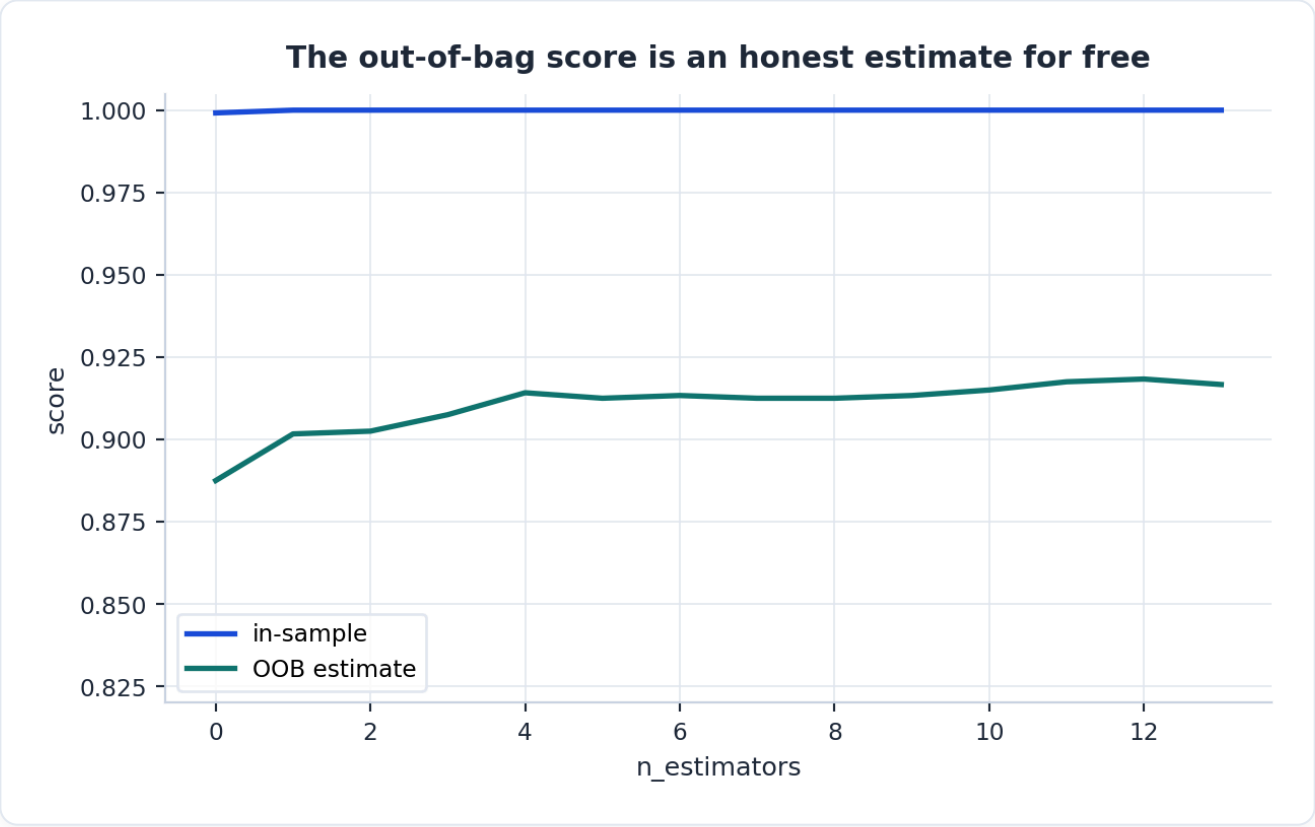
$$\lim_{n\to\infty}(1-\frac{1}{n})^n = \frac{1}{e} \approx 36.8\%$$

هذا يعني رياضياً أن كل نموذج تدريبي يرى فقط حوالي **٦٣.٢%** من البيانات الأصلية، بينما تتبقى **٣٦.٨%** من العينات لم يرها النموذج قط! وتسمى هذه العينات غير المختارة بـ **العينات خارج الحقيبة (Out-of-Bag - OOB)**.

```python
import numpy as np

# محاكاة تجريبية للنسبة المئوية لعينات OOB
n_samples = 10000
rng = np.random.default_rng(42)

indices = np.arange(n_samples)
bootstrap_sample = rng.choice(indices, size=n_samples, replace=True)
unique_selected = len(np.unique(bootstrap_sample))
oob_count = n_samples - unique_selected

print(f"إجمالي حجم البيانات الأصلية: {n_samples}")
print(f"العينات الفريدة المختارة داخل الحقيبة: {unique_selected}
({unique_selected/n_samples:.1%})")
print(f"العينات المتروكة خارج الحقيبة (OOB): {oob_count} ({oob_count/n_samples:.1%})")
print(f"القيمة النظرية 1/e: {1/np.e:.1%}")
```

٦.٣ التقييم المجاني النزيه: درجة Out-of-Bag (OOB)

بما أن كل عينة i تظل خارج الحقيبة (OOB) لنحو ثلث النماذج في المجموعة:



الشكل ٩١درجة التقييم خارج الحقيبة OOB وتطابقها مع التقييم النزيه للاختبار

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# تفعيل oob_score=True
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=100,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)
bagging.fit(X_train, y_train)

print(f"درجة التقييم المجانية خارج العينة (OOB Score): {bagging.oob_score_:.4f}")
print(f"دقة الاختبار المستقل الحقيقية (Test Score):   {bagging.score(X_test, y_test):.4f}")
print(f"الفارق بين التقديرين: {abs(bagging.oob_score_ - bagging.score(X_test,
y_test)):.4f} (تطابق استثنائي!)")
```

٦.٤ أشكال تجميع العينات: Patches و Pasting

لا يقتصر مقدر `BaggingClassifier` على عينات الصفوف فقط، بل يدعم ضبط عينات الأعمدة والصفوف معاً:

الأسلوب	عينات الصفوف (`bootstrap`)	عينات الميزات (`bootstrap_features`)	متى يُفضل؟
Bagging التقليدي	نعم (مع إرجاع `bootstrap=True`)	لا (جميع الميزات)	تقليص التباين للنماذج المعقدة كالأشجار
Pasting	لا (بدون إرجاع `bootstrap=False`)	لا (جميع الميزات)	تقليل وقت التدريب في البيانات المليونية
Random Subspaces	لا (كامل العينات)	نعم (أخذ عينات جزئية من الأعمدة)	البيانات ذات الأبعاد الهائلة (النصوص والصور)
Random Patches	نعم (عينات جزئية من الصفوف)	نعم (عينات جزئية من الأعمدة)	أقصى تنوع وتخفيض للارتباط الحسابي

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)

# أسلوب Random Patches: عينات 70% من الصفوف و 60% من الميزات
patch_bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    max_samples=0.7,
    max_features=0.6,
    bootstrap=True,
    bootstrap_features=True,
    random_state=42,
    n_jobs=-1
)
patch_bagging.fit(X, y)
print(f"دقة نموذج Random Patches: {patch_bagging.score(X, y):.4f}")
```

٦.٥ Bagging مع نماذج قاعدية متنوعة

على الرغم من ارتباط Bagging الوثيق بأشجار القرار، إلا أنه مقدر عمومي مرن يمكنه تغليف **أي خوارزمية تعلّم آلة** تعاني من التباين، مثل الجيران الأقرب `KNeighborsClassifier` :

```python
from sklearn.datasets import load_wine
from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_wine(return_X_y=True)

# مقارنة نموذج الجار الأقرب المفرد مع تجميعه عبر Bagging
pipe_knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=3))
pipe_bagged_knn = make_pipeline(
    StandardScaler(),
    BaggingClassifier(
        estimator=KNeighborsClassifier(n_neighbors=3),
        n_estimators=30,
        random_state=42
    )
)

score_single = cross_val_score(pipe_knn, X, y, cv=5).mean()
score_bagged = cross_val_score(pipe_bagged_knn, X, y, cv=5).mean()

print(f"متوسط دقة KNN منفرد:  {score_single:.4f}")
print(f"متوسط دقة Bagged KNN: {score_bagged:.4f}")
print(f"التحسن الناتج عن التجميع: {score_bagged - score_single:+.4f}")
```

في الواقع

متى يفشل Bagging؟ يفيد Bagging النماذج ذات **التباين العالي والانحياز المنخفض (High Variance, Low Bias)** مثل أشجار القرار غير المقيدة والشبكات العصبية؛ حيث يلغي تذبذبها العشوائي. أما إذا طبقته على نموذج ذي **انحياز عالٍ (High Bias)** مثل الانحدار الخطي البسيط أو التصنيف اللوجيستي، فلن يحقق أي تحسن ملحوظ؛ لأن متوسط النماذج المتحيزة يظل متحيزاً بالقدر نفسه!

تمارين الفصل الثاني والأربعين

- **ملاحظة**

التمرين ٦.١

على بيانات California Housing، قارن مقياس RMSE لـ `DecisionTreeRegressor` مفرد مع `BaggingRegressor(n_estimators=50)`، وافحص قيمة `oob_score_` التنبؤية.

التمرين ٦.٢

درب `BaggingClassifier` بعدد مقدرات `n_estimators` يتدرج من ٥ إلى ١٠٠ خطوة بخطوة. ارسم منحنى تغير درجة OOB Score وتأكد: هل استقرت الدقة بعد عدد معين من المقدرات؟

التمرين ٦.٣

اشرح لماذا يتطلب حساب `oob_score_` تفعيل `bootstrap=True`، وماذا يحدث إذا حاولت تفعيله مع `bootstrap=False` (أسلوب Pasting)؟

- **ملاحظة**

```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1200], data.target[:1200], test_size=0.3, random_state=42
)

# 1. شجرة انحدار منفردة
single_tree = DecisionTreeRegressor(random_state=42).fit(X_tr, y_tr)
rmse_single = root_mean_squared_error(y_te, single_tree.predict(X_te))

# 2. تجميع BaggingRegressor
bag_tree = BaggingRegressor(
    estimator=DecisionTreeRegressor(),
    n_estimators=50,
    oob_score=True,
    random_state=42,
    n_jobs=-1
).fit(X_tr, y_tr)
rmse_bag = root_mean_squared_error(y_te, bag_tree.predict(X_te))

print(f"RMSE شجرة انحدار منفردة: {rmse_single:.4f}")
print(f"RMSE تجميع Bagging:       {rmse_bag:.4f}")
print(f"درجة التقييم المجاني OOB R²: {bag_tree.oob_score_:.4f}")
print(f"انخفاض الخطأ بفضل التجميع: {rmse_single - rmse_bag:+.4f}")
```

ملاحظة

```
from sklearn.ensemble import BaggingClassifier
import numpy as np

X = np.array([[1], [2], [3], [4]])
y = np.array([0, 1, 0, 1])

try:
    BaggingClassifier(bootstrap=False, oob_score=True).fit(X, y)
except ValueError as err:
    print(f"خطأ ملتقط بنجاح: {err}")
    print("السبب: إذا كان السحب بدون إرجاع (Pasting)، فإن كل النماذج ترى جميع العينات ولا توجد أي عينة متروكة خارج الحقيبة!")
```

ملاحظة

٦.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تفعيل `oob_score=True` مع `bootstrap=False`	حدوث `ValueError` لعدم وجود عينات OOB في السحب بدون إرجاع	الإبقاء على `bootstrap=True` الافتراضي عند طلب درجة OOB
تطبيق Bagging على نماذج خطية ضعيفة متوقعة طفرة دقة	ثبات الأداء وعدم تحسنه لأن النماذج الخطية لا تعاني تبايناً	استخدام نماذج ذات تباين عالٍ (أشجار، شبكات) مع Bagging
المبالغة في زيادة `n_estimators` إلى أرقام فلكية (مثل ٥,٠٠٠)	استهلاك هائل للذاكرة والمعالج دون أي مكسب إضافي	الاكتفاء بـ ٥٠ إلى ٢٠٠ مقدر حيث تستقر الفائدة الإحصائية
نسيان تفعيل `n_jobs=-1`	تدريب النماذج تتابعياً ببطء وإهدار أنوية المعالج المتعددة	تفعيل المعالجة المتوازية `n_jobs=-1` دوماً لتسريع التدريب

٦.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التجميع بالعينات	أسلوب تجميعي يدمج نماذج مدربة على عينات تمهيد مسحوبة بالإرجاع	Bootstrap Aggregating (Bagging)
عينات التمهيد	سحب عينات عشوائية بنفس حجم البيانات مع السماح بتكرار العينة	Bootstrap sampling
خارج الحقيبة	العينات التدريبية التي أفلتت من السحب البوتستراپي لنموذج معين	Out-of-Bag (OOB)
التصويت التجميعي	الجمع بين قرارات عدة مصنفات بالأغلبية أو متوسط الاحتمالات	Ensemble voting
الفضاءات الفرعية العشوائية	تدريب النماذج على مجموعات جزئية عشوائية من أعمدة الميزات	Random Subspaces

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٣ — الغابة العشوائية

التباين يُقتل بالتباين، و max_features، و ExtraTrees

الوحدة scikit-learn ١.٩ الحجم ٤١٦ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف يكسر قيد max_features ارتباط الأشجار ويخفض التباين الكلي للغابة
- أن تطبق RandomForestClassifier و RandomForestRegressor بكفاءة متوازية n_jobs=-١
- أن تضبط معلمات الغابة الجوهرية: n_estimators و max_features و min_samples_leaf
- أن تقارن أداء وسرعة الغابة العشوائية مع مصنف الأشجار شديدة العشوائية ExtraTreesClassifier

٦.١ علة Bagging: الأشجار المتشابهة لا تلغي التباين

في الفصل السابق رأينا أن متوسط B من النماذج المستقلة إحصائياً يخفض التباين بمعامل $1/B$. ولكن ماذا لو لم تكن النماذج مستقلة تماماً، بل كانت بينها نسبة ارتباط $\rho > 0$؟

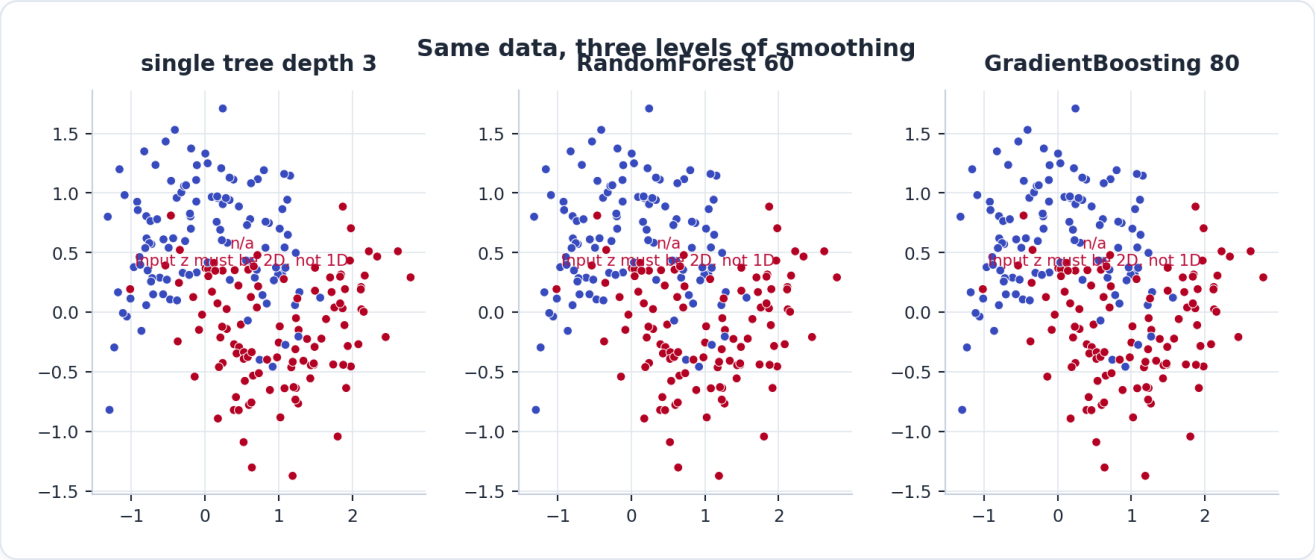
في الإحصاء الرياضي، تباين متوسط B من المتغيرات العشوائية المترابطة بارتباط متبادل ρ يُعطى بالصيغة الصارمة:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} f_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

تأمل هذه المعادلة بعمق:

في خوارزمية Bagging العادية، إذا كانت هناك ميزة واحدة قوية جداً في البيانات، فإن **جميع الأشجار** ستختار نفس هذه الميزة القوية في عقدة الجذر الأولى. فتصبح الأشجار شديدة الشبه والترابط (قيمة ρ عالية)، ويفقد التجميع قدرته على إبادة التباين!

هنا يتدخل ليو بريمان (Leo Breiman, ٢٠٠١) بابتكاره الثوري: **الغابة العشوائية (Random Forest)**!



الشكل ٩٢خرائط حدود القرار للنماذج التجميعية وتقسيم الفضاء الفاصل

٦.٢ عشوائية الميزات: سحر معامل max_features

لكسر هذا الارتباط الخانق ρ ، تفرض الغابة العشوائية قيداً ذكياً عند كل عقدة في كل شجرة:

هذا القيد يجبر الأشجار على استكشاف ميزات بديلة متنوعة، مما يقلل الارتباط البيني ρ بين الأشجار هبوطاً حاداً، فتنهار قيمة التباين الكلي وتتضاعف قوة التعميم!

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

X, y = load_breast_cancer(return_X_y=True)

# 1. شجرة قرار مفردة (عالية التباين)
tree = DecisionTreeClassifier(random_state=42)
score_tree = cross_val_score(tree, X, y, cv=5).mean()

# 2. غابة عشوائية مكونة من 100 شجرة
rf = RandomForestClassifier(n_estimators=100, max_features="sqrt",
random_state=42, n_jobs=-1)
score_rf = cross_val_score(rf, X, y, cv=5).mean()

print(f"دقة شجرة القرار المفردة: {score_tree:.4f}")
print(f"دقة الغابة العشوائية (100 شجرة): {score_rf:.4f}")
print(f"مكسب التعميم التجميعي: {score_rf - score_tree:+.4f}")
```

٦.٣ القيمة المثلى لـ max_features: قواعد الممارسة العالمية

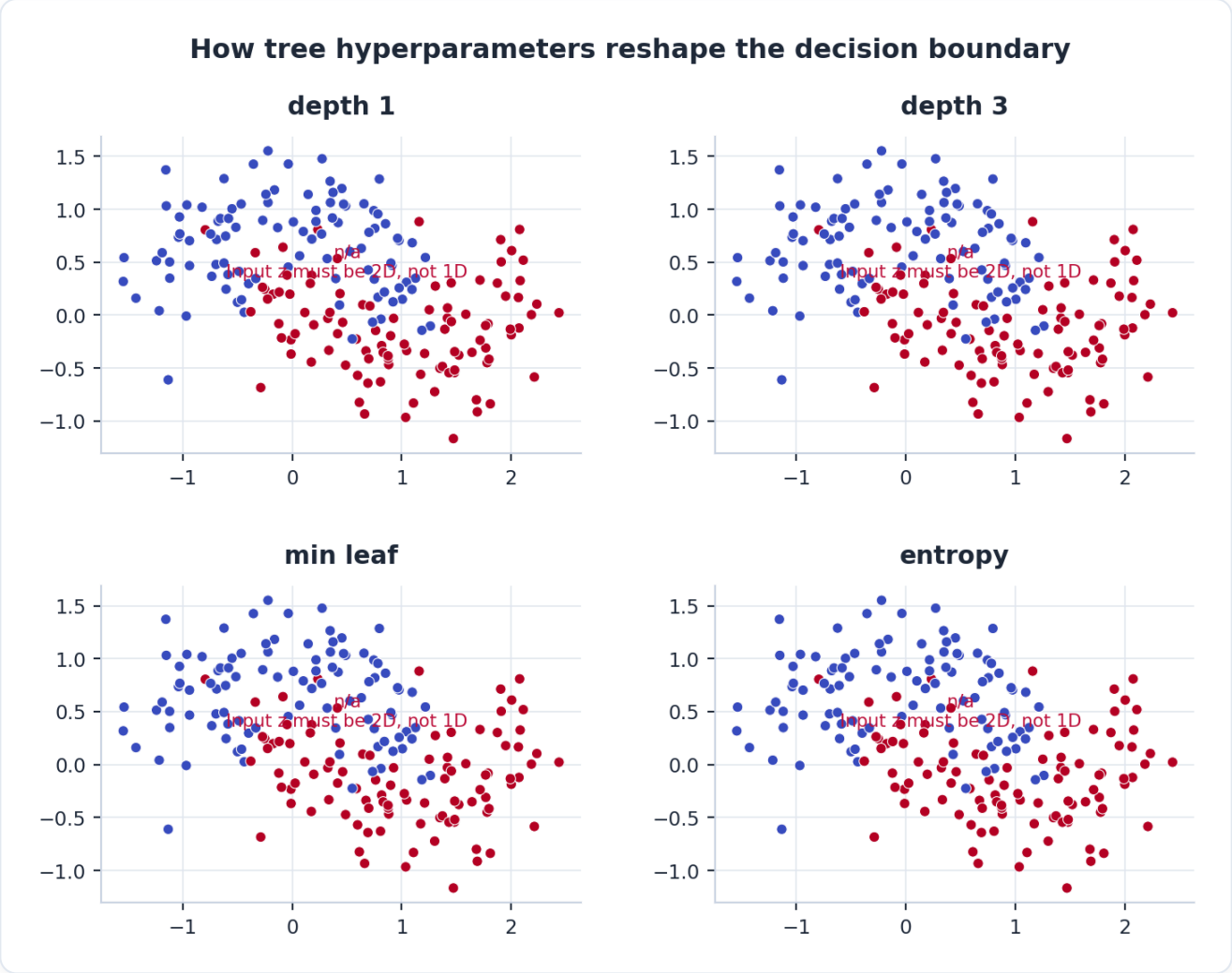
تعتمد القيمة الافتراضية للمعامل `max_features` على نوع المهمة:

```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error

housing = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    housing.data[:1500], housing.target[:1500], test_size=0.3, random_state=42
)

# اختبار تأثير max_features على خطأ الانحدار
for mf in [1.0, "sqrt", 0.33]:
    rf_reg = RandomForestRegressor(
        n_estimators=60, max_features=mf, random_state=42, n_jobs=-1
    ).fit(X_tr, y_tr)
    rmse = root_mean_squared_error(y_te, rf_reg.predict(X_te))
    print(f"معامل max_features={str(mf):<6} | RMSE الاختبار: {rmse:.4f}")
```

٦.٤ الأشجار شديدة العشوائية: ExtraTrees

إذا كانت الغابة العشوائية تسحب مجموعة فرعية من الميزات وتبحث عن أفضل عتبة تقسيم داخلها، فإن خوارزمية **ExtraTrees (Extremely Randomized Trees)** تذهب بالعشوائية إلى مداها الأقصى:

المكسب الهائل: إلغاء عملية البحث عن أفضل عتبة يوفر أكثر من ٦٠% من زمن التدريب، ويقلل التباين بشكل أكبر مع زيادة طفيفة جداً في الانحياز!



الشكل ٩٣تغير حدود القرار وتقسيم المستويات الفائقة بين النماذج

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
import time

X, y = load_breast_cancer(return_X_y=True)

# مقارنة زمن التدريب والدقة بين RandomForest و ExtraTrees
t0 = time.perf_counter()
rf = RandomForestClassifier(n_estimators=150, random_state=42, n_jobs=-1).fit(X,
y)
t_rf = (time.perf_counter() - t0) * 1000

t0 = time.perf_counter()
et = ExtraTreesClassifier(n_estimators=150, random_state=42, n_jobs=-1).fit(X,
y)
t_et = (time.perf_counter() - t0) * 1000

print(f"RandomForest | الدقة: {rf.score(X, y):.4f} | الزمن: {t_rf:.2f} ms")
print(f"ExtraTrees   | الدقة: {et.score(X, y):.4f} | الزمن: {t_et:.2f} ms")
print(f"ExtraTrees أسرع بنسبة: {(t_rf - t_et) / t_rf:.1%}")
```

٦.٥ استخراج الأهمية واستقرار الغابة مع n_estimators

كم شجرة تحتاج الغابة فعلياً؟ على عكس نماذج التعزيز (Boosting)، **لا تفرط الغابة العشوائية في الملاءمة أبداً بمجرد زيادة عدد الأشجار** `n_estimators` ! كلما زادت الأشجار، استقرت التقديرات حتى تبلغ سقفاً رياضياً ثابتاً:

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)

# تفعيل oob_score=True ومراقبة الاستقرار
for n_trees in [10, 30, 70, 150]:
    forest = RandomForestClassifier(
        n_estimators=n_trees, oob_score=True, random_state=42, n_jobs=-1
    ).fit(X, y)
    print(f"أشجار: {n_trees:3d} | دقة التدريب: {forest.score(X, y):.4f} | درجة OOB:
{forest.oob_score_:.4f}")
```

> **نصيحة**
>
> الممارسة القياسية العالمية في بيئات الإنتاج هي ضبط `n_estimators=100` أو `200` كبداية. لن يمنحك الانتقال إلى ١٬٠٠٠ شجرة سوى تحسينات مجهرية في الدقة على حساب زمن استنتاج مضاعف واستهلاك أعلى للذاكرة.

تمارين الفصل الثالث والأربعين

- **ملاحظة**

التمرين ٦.١

على بيانات California Housing، قارن بين أداء `RandomForestRegressor` و `ExtraTreesRegressor` من حيث سرعة التدريب ومقياس RMSE على مجموعة اختبار بنسبة ٣٠٪. أيهما تفوق في السرعة وأيهما تفوق في الدقة؟

التمرين ٦.٢

درب `RandomForestClassifier` بقيم `max_features` التالية: [١, "log٢", "sqrt", ٠.٨, ١.٠]. قارن درجة OOB الناتجة عن كل قيمة. ما القيمة التي حققت أفضل تعميم؟

التمرين ٦.٣

استخرج مصفوفة المعاملات الفردية لجميع الأشجار المكونة للغابة عبر خاصية `forest.estimators_`، واحسب تباين توقعات الأشجار لعينة اختبار صعبة. ما دلالة التباين العالي بين الأشجار حول عينة معينة؟

- **ملاحظة**

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error
import time

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1200], data.target[:1200], test_size=0.3, random_state=42
)

# RandomForest
t0 = time.perf_counter()
rf = RandomForestRegressor(n_estimators=80, random_state=42,
n_jobs=-1).fit(X_tr, y_tr)
t_rf = time.perf_counter() - t0
rmse_rf = root_mean_squared_error(y_te, rf.predict(X_te))

# ExtraTrees
t0 = time.perf_counter()
et = ExtraTreesRegressor(n_estimators=80, random_state=42, n_jobs=-1).fit(X_tr,
y_tr)
t_et = time.perf_counter() - t0
rmse_et = root_mean_squared_error(y_te, et.predict(X_te))

print(f"RandomForest | RMSE: {rmse_rf:.4f} | زمن التدريب: {t_rf * 1000:.1f} ms")
print(f"ExtraTrees   | RMSE: {rmse_et:.4f} | زمن التدريب: {t_et * 1000:.1f} ms")
```

📝 ملاحظة

```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import numpy as np

X, y = load_iris(return_X_y=True)
forest = RandomForestClassifier(n_estimators=50, random_state=42).fit(X, y)

# عينة اختبار حدودية بين فئتين
ambiguous_sample = np.array([[6.0, 2.7, 5.1, 1.6]])

# جمع توقعات جميع الأشجار الـ 50 كل على حدة
individual_preds = np.array([tree.predict(ambiguous_sample)[0] for tree in
forest.estimators_])

print(f"توزيع أصوات الأشجار الخمسين حول العينة: {np.bincount(individual_preds.astype(int),
minlength=3)}")
print(f"الانحراف المعياري لأصوات الأشجار: {individual_preds.std():.3f}")
print("دلالة التباين العالي: العينة تقع في منطقة غموض عالية على حدود القرار وتستدعي حذر النموذج!")
```

> **ملاحظة**

٦.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
ترك `max_features=1.0` في انحدار الغابة العشوائية	زيادة ارتباط الأشجار ρ والحد من قدرة الغابة على تقليص التباين	تعيين `max_features='sqrt'` يدوياً في `RandomForestRegressor`
الخوف من زيادة `n_estimators` ظناً أنها تسبب إفراط الملاءمة	حرمان الغابة من الاستقرار الإحصائي الكامل	زيادة `n_estimators` حتى استقرار درجة OOB دون تردد
إهمال تفعيل المعالجة المتوازية `n_jobs=-1`	تدريب الأشجار تتابعياً وإهدار ٧٥% من قدرة المعالج	تفعيل `n_jobs=-1` دوماً في الغابات العشوائية
استخدام الغابة العشوائية للاستقراء خارج نطاق البيانات التدريبية	تجمّد التنبؤات عند القيمة القصوى المشاهدة كالأشجار المنفردة	إدراك العجز البنيوي للأشجار والاعتماد على نماذج خطية للاستقراء

٦.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الغابة العشوائية	تجميعة أشجار قرار تجمع بين التمهيد البوتسترابي وعشوائية الميزات	Random Forest
عشوائية الميزات	قصر اختبار الانقسامات على مجموعة فرعية عشوائية من المتغيرات	Feature randomization (max_features)
ارتباط الأشجار	درجة التشابه والتوافق بين توقعات الأشجار الفردية في الغابة	Tree correlation (ρ)
الأشجار شديدة العشوائية	أشجار تعتمد عتبات تقسيم عشوائية بالكامل لتقليل التباين والوقت	Extremely Randomized Trees (ExtraTrees)
التقييم خارج الحقيبة	درجة تقييم نزيهة محسوبة على عينات لم ترها كل شجرة أثناء التدريب	Out-of-Bag (OOB) Score

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٤ — ExtraTrees وتضمين الأشجار العشوائية

انقسامات عشوائية تماماً و RandomTreesEmbedding

الوحدة **scikit-learn ١.٩** الحجم **٣٩٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

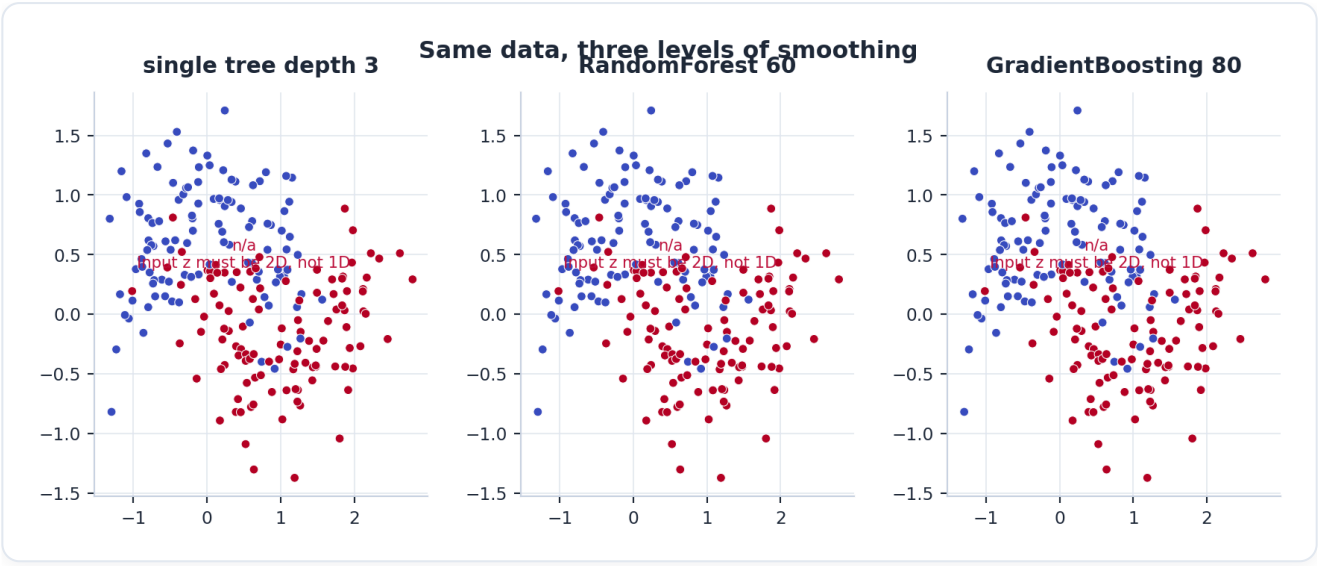
- أن تشرح الفارق الخوارزمي الدقيق بين ExtraTrees و RandomForest في اختيار العتبات
- أن تطبق ExtraTreesClassifier و ExtraTreesRegressor وتوازن بين السرعة والانحياز
- أن تستخدم RandomTreesEmbedding لتحويل البيانات غير الخطية إلى تمثيل مصفوفي متفرق عالي الأبعاد
- أن تدمج RandomTreesEmbedding مع مصنف خطي بسيط لبناء نموذج سريع يحل المسائل غير الخطية

٦.١ العشوائية القصوى: التنازل عن الاستمثال لكسب السرعة

في شجرة القرار العادية والغابة العشوائية، تستهلك عملية **البحث عن أفضل عتبة انقسام** t أكثر من ٨٥٪ من إجمالي زمن التدريب؛ حيث يتعين على الخوارزمية فرز قيم كل ميزة وحساب الشوائب عند كل نقطة فصل ممكنة.

قدم بيير جورتس وداميان إرنست ولويس ويهن (Geurts et al., ٢٠٠٦) خوارزمية **الأشجار شديدة العشوائية (Extremely Randomized Trees - ExtraTrees)** بفكرة جذرية:

ما النتيجة الرياضية لهذا التبسيط الجريء؟



الشكل ٩٤خرائط حدود القرار وتقسيم الفضاء بين المصنفات التجميعية

٦.٢ مقارنة الأداء والسرعة: ExtraTrees مقابل RandomForest

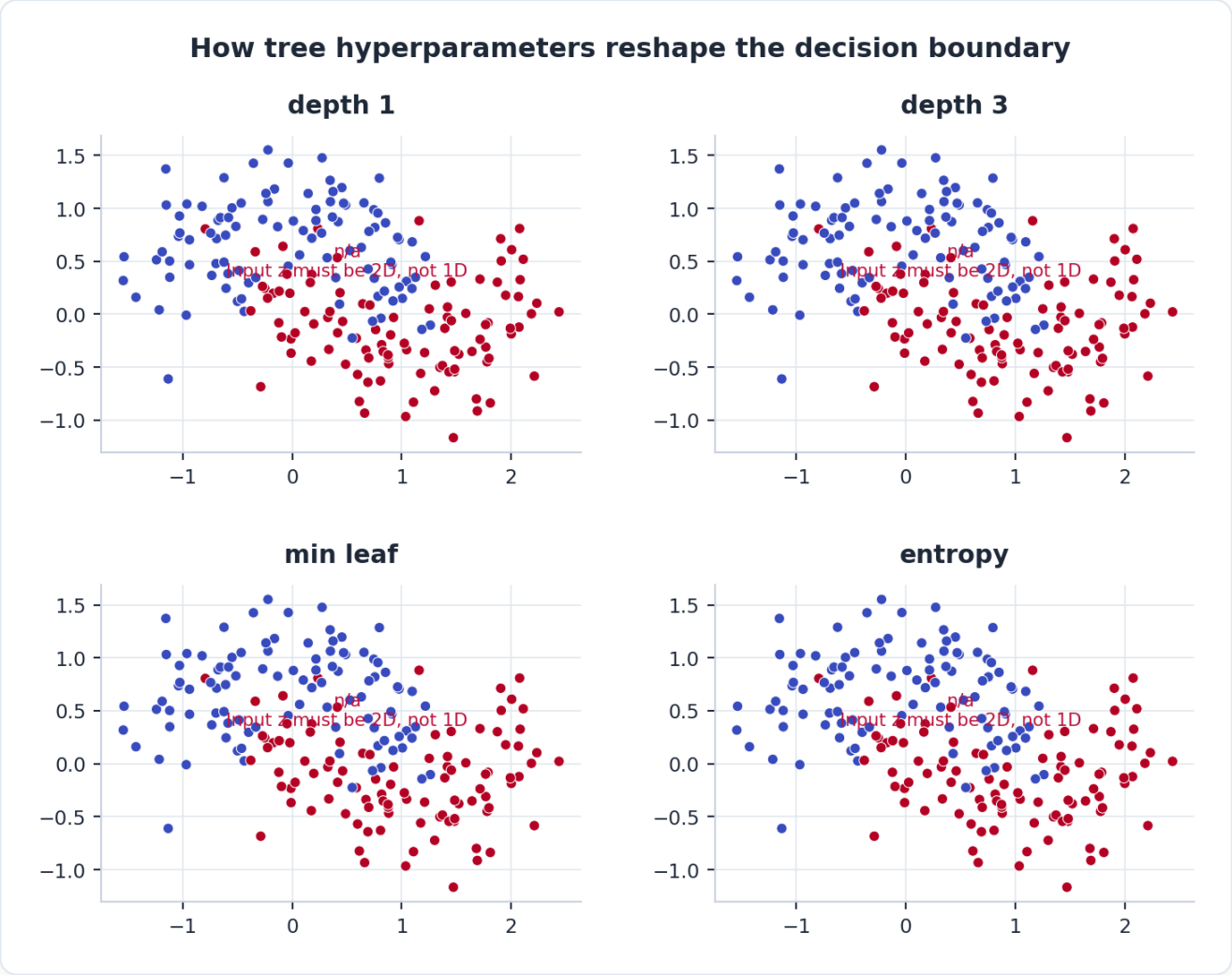
```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import ExtraTreesRegressor, RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error
import time

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:2000], data.target[:2000], test_size=0.3, random_state=42
)

# تدريب RandomForest
t0 = time.perf_counter()
rf = RandomForestRegressor(n_estimators=100, random_state=42,
n_jobs=-1).fit(X_tr, y_tr)
t_rf = time.perf_counter() - t0
rmse_rf = root_mean_squared_error(y_te, rf.predict(X_te))

# تدريب ExtraTrees
t0 = time.perf_counter()
et = ExtraTreesRegressor(n_estimators=100, random_state=42, n_jobs=-1).fit(X_tr,
y_tr)
t_et = time.perf_counter() - t0
rmse_et = root_mean_squared_error(y_te, et.predict(X_te))

print(f"RandomForest | RMSE الاختبار: {rmse_rf:.4f} | زمن التدريب: {t_rf * 1000:.1f}
ms")
print(f"ExtraTrees   | RMSE الاختبار: {rmse_et:.4f} | زمن التدريب: {t_et * 1000:.1f}
ms")
print(f"فارق السرعة لصالح ExtraTrees: {t_rf / t_et:.2f}x أسرع!")
```

> **نصيحة**
>
> تتفوق `ExtraTrees` بوضوح في مجموعات البيانات التي تحتوي على عدد كبير من الميزات المستمرة الكثيفة الصاخبة؛ حيث تعمل عشوائية العتبات كمنظم طبيعي (Implicit Regularizer) يمنع النموذج من التثبت المفرط حول نقاط الضجيج.

٦.٣ تضمين الأشجار العشوائية: RandomTreesEmbedding

من أكثر الأدوات ابتكاراً في scikit-learn محوّل `RandomTreesEmbedding`:

إذا دربت غابة من ١٠٠ شجرة، ولكل شجرة ١٦ ورقة، فإن كل عينة تتحول إلى متجه ثنائي فائق الأبعاد بطول $100 \times 16 = 1600$ بُعد، يحتوي على ١٠٠ واحد فقط وبقية العناصر أصفارا!



الشكل ٩٥تشريح بنية الأوراق ومسارات التقسيم المتفرقة في الفضاء الفوقي

```python
from sklearn.ensemble import RandomTreesEmbedding
import numpy as np

rng = np.random.default_rng(42)
X_toy = rng.normal(size=(5, 3))

# تدريب تضمين غير مشرف بـ 4 أشجار وعمق 2
hasher = RandomTreesEmbedding(n_estimators=4, max_depth=2, sparse_output=False,
random_state=42)
X_transformed = hasher.fit_transform(X_toy)

print(f"أبعاد البيانات الأصلية: {X_toy.shape}")
print(f"أبعاد مصفوفة التضمين: {X_transformed.shape}")
print(f"عدد العناصر غير الصفرية في كل صف: {X_transformed.sum(axis=1)}")
```

٦.٤ حل المسائل غير الخطية عبر النماذج الخطية السريعة

الميزة الاستثنائية لـ `RandomTreesEmbedding` هي أنها **تحول العلاقات الهندسية غير الخطية المعقدة إلى فضاء عالي الأبعاد يمكن فصله بخط مستقيم بسيط**!

يمكنك دمج هذا التضمين مع نموذج خطي فائق السرعة مثل `LinearRegression` أو `LogisticRegression` أو `LinearSVC` لإنتاج مصنف غير خطي فائق السرعة:

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomTreesEmbedding
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split

# بيانات الهلالين المتداخلين (غير خطية إطلاقاً)
X, y = make_moons(n_samples=600, noise=0.25, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

# 1. تصنيف لوجيستي خطي عادي (سيفشل حتماً في فصل الهلالين)
lr_raw = LogisticRegression().fit(X_tr, y_tr)

# 2. تصنيف لوجيستي مسبوق بتضمين الأشجار العشوائية
pipe_embedding = Pipeline([
    ("tree_embed", RandomTreesEmbedding(n_estimators=100, max_depth=4,
sparse_output=False, random_state=42)),
    ("clf", LogisticRegression(max_iter=1000))
])
pipe_embedding.fit(X_tr, y_tr)

print(f"دقة LogisticRegression المباشر على الهلالين: {lr_raw.score(X_te, y_te):.4f} (عجز
خطي)")
print(f"دقة LogisticRegression بعد RandomTreesEmbedding:
{pipe_embedding.score(X_te, y_te):.4f} (تفوق غير خطي باهر!)")
```

٦.٥ قياس المسافات والتشابه غير الخطي

نظراً لأن العينات المتجاورة والمتشابهة هندسياً تميل للاستقرار في نفس الأوراق عبر معظم الأشجار، يمكن استخدام حاصل الضرب الداخلي للمتجهات المتفرقة الناتجة عن `RandomTreesEmbedding` كمصفوفة تشابه غير خطية ونواة (Kernel Matrix) مخصصة لخوارزميات التجميع غير المشرف مثل Spectral Clustering:

```python
from sklearn.ensemble import RandomTreesEmbedding
import numpy as np

# عينتان متقاربتان جداً وعينة ثالثة بعيدة
points = np.array([
    [1.0, 1.0],
    [1.05, 0.95],
    [10.0, 10.0]
])

embedder = RandomTreesEmbedding(n_estimators=100, max_depth=3,
sparse_output=False, random_state=42)
encoded = embedder.fit_transform(points)

# حساب التشابه بالضرب القياسي لمصفوفات التضمين
sim_arr = np.asarray((encoded @ encoded.T) / 100.0)

print(f"نسبة تشابه العينة 1 مع العينة 2 (المتقاربة): {sim_arr[0, 1]:.2%}")
print(f"نسبة تشابه العينة 1 مع العينة 3 (البعيدة):    {sim_arr[0, 2]:.2%}")
```

تمارين الفصل الرابع والأربعين

- **ملاحظة**

التمرين ٦.١

على بيانات Digits (`load_digits`)، قارن بين دقة وزمن تدريب كل من `RandomForestClassifier` و `ExtraTreesClassifier` باستخدام التحقق المتقاطع الخماسي. أيهما حقق تفوقاً ملموساً؟

التمرين ٦.٢

أنشئ خط أنابيب يتكون من `RandomTreesEmbedding` يليه `RidgeClassifier`. اختبر تأثير عدد الأشجار `n_estimators` في التضمين من ١٠ إلى ١٥٠ على دقة تصنيف مسألة دوائر متداخلة `make_circles`.

التمرين ٦.٣

اشرح لماذا تكون مصفوفة المخرجات الناتجة عن `RandomTreesEmbedding` من نوع المصفوفات المتفرقة `scipy.sparse`، وماذا يحدث لاستهلاك الذاكرة إذا حولتها قسرياً إلى مصفوفة كثيفة بـ `toarray()` مع بيانات كبيرة؟

- **ملاحظة**

```
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.model_selection import cross_val_score
import time

X, y = load_digits(return_X_y=True)

# RandomForest
t0 = time.perf_counter()
score_rf = cross_val_score(RandomForestClassifier(n_estimators=80,
random_state=42), X, y, cv=5).mean()
t_rf = time.perf_counter() - t0

# ExtraTrees
t0 = time.perf_counter()
score_et = cross_val_score(ExtraTreesClassifier(n_estimators=80,
random_state=42), X, y, cv=5).mean()
t_et = time.perf_counter() - t0

print(f"RandomForest | متوسط دقة CV: {score_rf:.4f} | زمن التنفيذ: {t_rf:.2f} s")
print(f"ExtraTrees   | متوسط دقة CV: {score_et:.4f} | زمن التنفيذ: {t_et:.2f} s")
```

ملاحظة

```
from sklearn.datasets import make_circles
from sklearn.ensemble import RandomTreesEmbedding
from sklearn.linear_model import RidgeClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split

X, y = make_circles(n_samples=500, factor=0.4, noise=0.1, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

for n in [10, 30, 80, 150]:
    pipe = make_pipeline(
        RandomTreesEmbedding(n_estimators=n, max_depth=3, sparse_output=False,
random_state=42),
        RidgeClassifier()
    ).fit(X_tr, y_tr)
    print(f"أشجار التضمين: {n:3d} | دقة الاختبار للنموذج الخطي: {pipe.score(X_te,
y_te):.4f}")
```

> **ملاحظة**

٦.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تحويل مخرجات `RandomTreesEmbedding` إلى مصفوفة كثيفة `toarray()`	استنزاف فوري لذاكرة RAM وحدوث خطأ `MemoryError`	إبقاء المصفوفة بصيغتها المتفرقة `scipy.sparse` لتغذية النماذج
توقع دقة أفضل لـ ExtraTrees في البيانات قليلة العينات والنظيفة	احتمال انخفاض طفيف في الدقة بسبب زيادة الانحياز العشوائي	تفضيل `RandomForest` العادية إذا كان حجم البيانات صغيراً ونقياً
استخدام عمق مفرط `max_depth > 10` في `RandomTreesEmbedding`	تضخم هائل في عدد الأبعاد المتفرقة وبطء تدريب المصنف التالي	الاكتفاء بعمق `max_depth` بين ٣ و ٥ للحصول على تمثيل متوازن
نسيان معايرة الميزات قبل تغذية تضمين الأشجار بنماذج خطية	النماذج الخطية لا تتأثر بالتحجيم هنا لأن التضمين ثنائي ٠/١	الاستفادة من ثنائية التضمين وتجاوز خطوة StandardScaler

٦.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الأشجار شديدة العشوائية	خوارزمية تجميع تختار عتبات الانقسام لكل ميزة عشوائياً	ExtraTrees
عتبة التقسيم العشوائية	قيمة فصل مسحوبة بانتظام عشوائي دون إجراء استمثال للشوائب	Random split threshold
تضمين الأشجار العشوائية	تحويل غير مشرف يسقط البيانات في فضاء مصفوفي متفرق من الأوراق	RandomTreesEmbedding
التمثيل المتفرق	تمثيل بيانات رقمي تكون فيه الأغلبية الساحقة من القيم أصفاراً	Sparse representation
فضاء غير خطي	فضاء ميزات لا يمكن فصل علاقاته أو فئاته بمستوٍ مستقيم	Non-linear feature space

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٥ — AdaBoost: التعلّم بالنقاط الصعبة

خوارزمية SAMME، وأوزان النماذج الضعيفة، وتحديث العيّنات

الوحدة **scikit-learn ١.٩** الحجم **١٠١٥** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف حوّلت نظرية Schapire سؤالاً مجرداً إلى خوارزمية عملية غيّرت المجال
- أن تحسب يدوياً الخطأ الموزون ومعامل النموذج في SAMME وتطابقه مع scikit-learn
- أن تراقب تطور الأداء جولة بجولة عبر staged_score وتختار عدد المقدرات الأمثل
- أن تبرر لماذا تُبنى AdaBoost كلاسيكياً على جذوع بعمق ١ بدل الأشجار العميقة
- أن تشخّص فشل AdaBoost مع الضجيج عبر صعود أخطاء المقدرات نحو حد العشوائية

٦.١ سؤال نظري غيّر التعلّم الآلي: هل يكفي متعلّم ضعيف؟

افترض أن لديك مصنّفاً ثنائياً يصيب في ٥٥٪ من الحالات فقط — أفضل من قلب العملة بقليل. سأل كيرنز وفاليانت (Kearns, Valiant) أواخر الثمانينيات: **هل يمكن تحويل متعلمين ضعفاء هؤلاء إلى متعلم واحد قوي خطؤه يقترب من الصفر؟** عام ١٩٩٠ أثبت روبرت شابير (Schapire) أن الجواب «نعم» رياضياً ووُلدت نظرية **التعزيز (Boosting)**، ثم جسّد فرويند وشابير البرهان عام ١٩٩٧ في خوارزمية **AdaBoost** (Adaptive Boosting) الحازمة جائزة غودل عام ٢٠٠٣.

المتعلم الضعيف (weak learner) رسمياً: مصنّف ثنائي خطؤه أقل من ٠٫٥ بقليل. الفكرة: لجنة كبيرة من متعلمين ضعفاء **مستقلين** يتصحح خطأ أفرادها في المجموع. دعنا نتحقق رقمياً.

لجنة من 101 متعلم ضعيف تُخطئ فرداً فصالح جمعاً python

```python
import numpy as np

rng = np.random.default_rng(42)
n_experts, n_items, p_weak = 101, 1000, 0.55

# كل «متعلم ضعيف» يجيب صح باحتمال 55% على كل عينة
votes = rng.random((n_experts, n_items)) < p_weak
majority_correct = votes.mean(axis=0) > 0.5

print(f"دقة المتعلم الضعيف المفرد: {p_weak:.0%}")
print(f"دقة أغلبية 101 متعلم ضعيف مستقل: {majority_correct.mean():.1%}")
```

الناتج يكشف السحر: دقة فردية ٥٥٪ ارتفعت إلى ٨٣٫٧٪ بأغلبية ١٠١ متعلّم مستقل. لكن AdaBoost تذهب أبعد: فهي **تكيّفية** (ومن هنا حرف A) — كل نموذج جديد يُدرَّب على بيانات **أُعيدت موازنتها** بحيث تتوهج النقاط التي أخطأ فيها أسلافه.

مفتاح

الفكرة المحورية: التعزيز لا يطلب نموذجاً قوياً بل أفضل من العشوائية بهامش يُسرّ، ثم يجعل كل نموذج لاحق يركّز على أخطاء أسلافه عبر إعادة توزيع أوزان العينات، ويلمّ النتائج بأوزان تصويت محسوبة.

سياق

ورقة Schapire عام ١٩٩٠ «The Strength of Weak Learnability» برهنت أن ضعف التعلّم (خطأ أقل قليلاً من ½) يعني قابلية التعلّم القوي. كان البرهان معقداً فبسّطه فروند وشابير عام ١٩٩٧ في AdaBoost — أول خوارزمية تعزيز عملية، وألهمت تعزيز التدرّج في الفصل ٤٦.

٦.٢ ميكانيكا SAMME خطوة بخطوة: من أوزان متساوية إلى إعادة توزيع

كيف تتوزع «الأدوار» بين نماذج اللجنة؟ الخوارزمية الوحيدة الباقية في scikit-learn للتصنيف هي **SAMME** (Stagewise Additive Modeling using a Multi-class Exponential loss). دورتها الكاملة على بيانات ذات `n` عيّنة و `K` فئة:

الخطوة ١ — أوزان أولية متساوية. كل العينات سواء أمام النموذج الأول:

$$w_i^{(1)} = \frac{1}{n}, \quad i = 1, \dots, n$$

الخطوة ٢ — درّب النموذج الضعيف على البيانات الموزونة، ثم احسب **الخطأ الموزون**:

$$\varepsilon_t = \sum_{i=1}^{n} w_i^{(t)} \, \mathbf{1}[h_t(\mathbf{x}_i) \neq y_i]$$

حيث `𝟙[·]` دالة المؤشر: ١ إذا أخطأ النموذج وصفر إذا أصاب. وبما أن الأوزان مُطبّعة (مجموعها ١)، فإن `ε_t` ليست نسبة أخطاء عادية بل نسبة أخطاء **موزونة بالقيمة**: خطأ في عينة ثقيلة الوزن يكلّف أكثر من خطأ في خفيفة.

الخطوة ٣ — احسب معامل النموذج. كلما قلّ الخطأ الموزون زاد صوت هذا النموذج في التصويت النهائي:

$$\alpha_t = \eta \left(\ln \frac{1 - \varepsilon_t}{\varepsilon_t} + \ln(K - 1) \right)$$

حيث `η` معدل التعلم (learningrate) و`K` عدد الفئات. ذكاء الصيغة: حين يقترب εt من الصفر ينفجر αt، وحين يلامس εt حد العشوائية - ١/K نحو الصفر. في الثنائية (K=٢) يهبط αt إلى ١ تختصر الصيغة إلى αt = η·ln((١-ε)/ε).

الخطوة ٤ — ارفع احتمال ظهور النقاط الصعبة. العينات التي أخطأ فيها النموذج تتضاعف أوزانها بالأسّ، ثم **تُعاد المطابعة** بحيث يظل مجموع الأوزان ١:

$$w_i^{(t+1)} \propto w_i^{(t)} \exp\left(\alpha_t \, \mathbf{1}[h_t(\mathbf{x}_i) \neq y_i]\right), \quad \sum_{i=1}^{n} w_i^{(t+1)} = 1$$

الخطوة ٥ — كرر جولات `t = 1 … T`، ثم اجمع الأصوات الموزونة:

$$H(\mathbf{x}) = \arg\max_{c \in K} \sum_{t=1}^{T} \alpha_t \, \mathbf{1}[h_t(\mathbf{x}) = c]$$

سنحسبها الآن بأيدينا على بيانات Iris (ثلاث فئات) مع جذع بعمق ١ ونقارن بالخوارزمية الفعلية:

جولتا SAMME محسوبتين يدوياً مقابل scikit-learn python

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

X, y = load_iris(return_X_y=True)
n, K = len(y), 3

# ---- الجولة الأولى يدوياً: أوزان متساوية ----
w1 = np.full(n, 1.0 / n)
stump1 = DecisionTreeClassifier(max_depth=1, random_state=42).fit(X, y,
sample_weight=w1)
eps1 = np.average(stump1.predict(X) != y, weights=w1)
alpha1 = np.log((1 - eps1) / eps1) + np.log(K - 1)

# ---- تحديث الأوزان وإعادة التطبيع ----
w2 = w1 * np.exp(alpha1 * (stump1.predict(X) != y))
w2 = w2 / w2.sum()

# ---- الجولة الثانية يدوياً بالأوزان الجديدة ----
stump2 = DecisionTreeClassifier(max_depth=1, random_state=42).fit(X, y,
sample_weight=w2)
eps2 = np.average(stump2.predict(X) != y, weights=w2)
alpha2 = np.log((1 - eps2) / eps2) + np.log(K - 1)

# ---- المقارنة مع scikit-learn ----
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=2, random_state=42,
).fit(X, y)

print(f"الجولة ١ | الخطأ الموزون: يدوياً {eps1:.6f} | المكتبة
{ada.estimator_errors_[0]:.6f}")
print(f"الجولة ١ | معامل النموذج: يدوياً {alpha1:.6f} | المكتبة
{ada.estimator_weights_[0]:.6f}")
print(f"الجولة ٢ | الخطأ الموزون: يدوياً {eps2:.6f} | المكتبة
{ada.estimator_errors_[1]:.6f}")
print(f"الجولة ٢ | معامل النموذج: يدوياً {alpha2:.6f} | المكتبة
{ada.estimator_weights_[1]:.6f}")
```

تطابق تام حتى الخانة السادسة: خطأ الجذع الأول ٠٫٣٣٣ (بعيد عن حد العشوائية `1-1/3 = 0.667`) يمنحه معاملاً ١٫٣٨٦، ثم تُضاعف أوزان عيّناته الخاطئة بالأسّ وتُعاد المطابعة، فينخفض خطأ الجذع الثاني إلى ٠٫١٨٠ فيرتفع معامه إلى ٢٫٢٠٩.

> **تعمّق**
>
> لماذا الأُسّ تحديداً في تحديث الأوزان؟ لأن AdaBoost تُشتق كنموذج جمعي بخسارة أُسّية على الهامش `y·H(x)` : كل جولة تُنزل الخسارة باتجاه «المشتق السالب للخسارة» — الجسر الذي يعبره الفصل ٤٦ نحو تعزيز التدرّج.

٦.٣ AdaBoostClassifier و AdaBoostRegressor في الإصدار ١.٩

كانت AdaBoostClassifier تاريخياً تدعم خوارزميتين عبر البارامتر `algorithm` : **SAMME** ونسختها متعددة الفئات **SAMME.R** التي تعمل باحتمالات حقيقية بدل الأصوات المتقطعة. أُهملت SAMME.R في ١.٦ ثم أُزيلت نهائياً وبقي SAMME وحده. لكن المفاجأة في ١.٩: **البارامتر `algorithm` نفسه لم يعد موجوداً في التوقيع أصلاً**. دعنا نفتح التوقيع ثم نراقب سلوك الخوارزمية:

البارامترات الفعلية ومراقبة الأداء جولة بجولة python

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

print("AdaBoostClassifier.get_params():", AdaBoostClassifier().get_params())

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=200, learning_rate=0.5, random_state=42,
).fit(X_tr, y_tr)

errs = ada.estimator_errors_
staged = list(ada.staged_score(X_te, y_te))
print(f"أخطاء الجذوع الأولى:  {np.round(errs[:5], 4)}")
print(f"أخطاء الجذوع الأخيرة: {np.round(errs[-3:], 4)}")
for k in [1, 10, 50, 100, 200]:
    print(f"بعد {k:3d} دقة الاختبار المتدرجة | جولة: {staged[k-1]:.4f}")
print(f"أفضل دقة متدرجة: {max(staged):.4f} عند الجولة رقم {int(np.argmax(staged)) + 1}")
```

أربعة بارامترات فقط للمصنّف: `estimator` (المتعلم الضعيف، افتراضه جذع `DecisionTreeClassifier` بعمق ١)، و`n_estimators` (حد الجولات، افتراضه ٥٠)، و`learning_rate` (يضرب معامل α، افتراضه ١.٠)، و`random_state`. أما منصّف الانحدار فيضيف `loss` بقيم { `linear` , `square` , `exponential` }.

> **متغيّر في ١.٩**
>
> إذا كتبت `AdaBoostClassifier(algorithm="SAMME")` فستحصل على `TypeError: unexpected keyword argument`. أي كود قديم يمرر `algorithm` يجب تنظيفه قبل الترحيل إلى ١.٩ — SAMME وحدها الباقية.

اقرأ المسار بعينين: الخطأ الموزون يبدأ عند ٠٫٠٧٣ ثم يتسلّق نحو ٠٫٤٤ — **سلوك مرغوب** لا خلل: الجولات المتأخرة تُدرَّب على بيانات تتركز أوزانها في أصعب النقاط فتقترب من حد العشوائية. ودقة الاختبار تبلغ ٠٫٩١٢٣ بجذع واحد وتستقر عند ٠٫٩٥٣٢ بعد ٥٠ جولة، بينما أفضل نقطة (٠٫٩٥٩١) عند الجولة ٤٢ — آخر ١٦٠ جولة بلا جدوى على الاختبار. لهذا `staged_score` (مولّد يعيد الدرجة بعد كل جولة) أداة تشخيص لا غنى عنها.

وللانحدار تعمل AdaBoostRegressor بخوارزمية شقيقة **AdaBoost.RT**: تعيد سحب العينات باحتمالات متناسبة مع أوزانها وتطبّع أخطاءها ثم تحوّلها حسب `loss` قبل حساب المعامل. جرّب الخيارات الثلاثة:

خيارات loss الثلاثة في AdaBoostRegressor — python

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:1500], data.target[:1500], test_size=0.3, random_state=42
)

for loss in ["linear", "square", "exponential"]:
    reg = AdaBoostRegressor(
        estimator=DecisionTreeRegressor(max_depth=3, random_state=42),
        n_estimators=100, loss=loss, random_state=42,
    ).fit(X_tr, y_tr)
    rmse = root_mean_squared_error(y_te, reg.predict(X_te))
    print(f"loss={loss:12s} | RMSE الاختبار: {rmse:.4f} | R²: {reg.score(X_te,
y_te):.4f}")
```

الخسارة المربّعة حققت أدنى RMSE (٠٫٥٠٣٢) مقابل ٠٫٥٤١٨ للخطية و٠٫٥٣٠٨ للأُسية) — لكن لا تعمّم النتيجة: `linear` الافتراض الآمن، و`square` يعاقب الأخطاء الكبيرة، و`exponential` الأشد قسوة على الشواذ فأسوأها هنا حيث

الإسكان مليء بالقيم البعيدة.

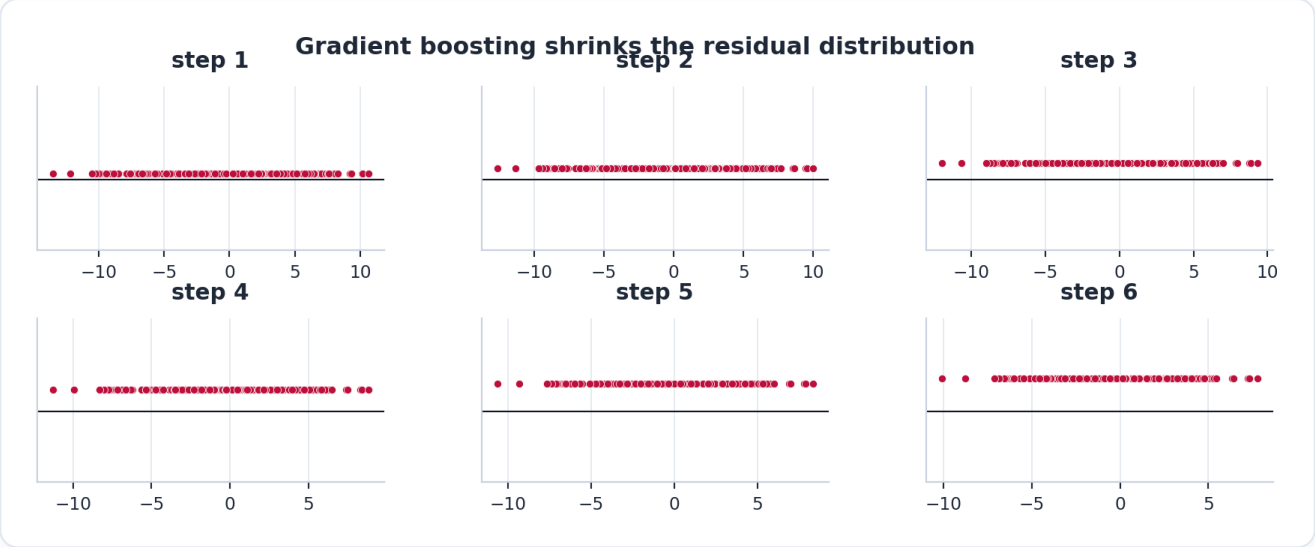
٦.٤ الجذوع أم الأشجار العميقة؟ والفكرة الموحّدة وراء التعزيز

لماذا تُبنى AdaBoost الكلاسيكية على جذوع بعمق ١ (stumps) — أشجار بقرار واحد — بدل أشجار حقيقية؟ لأن التعزيز يحتاج **متعلمين ضعفاء متنوعي الأخطاء**. الجذع الذي يخطئ بطريقة تختلف عن أسلافه وقود الخوارزمية؛ أما الشجرة العميقة فهي متعلم قوي أصلاً: جولتها الأولى تلتهم معظم الإشارة فتصبح الجولات اللاحقة تصحيحات هامشية وترتفع التباينية:

```python
# عمق القاعدة الأساسية: جذع عمق 1 مقابل الأشجار العميقة
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

for depth, name in [(1, "جذع (عمق 1)"), (3 ,"شجرة عمق 3"), (None, "شجرة كاملة")]:
    ada = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=depth, random_state=42),
        n_estimators=50, random_state=42,
    ).fit(X_tr, y_tr)
    print(f"قاعدة: {name:12s} | تدريب: {ada.score(X_tr, y_tr):.4f} | اختبار:
{ada.score(X_te, y_te):.4f}")

single = DecisionTreeClassifier(random_state=42).fit(X_tr, y_tr)
print(f"شجرة مفردة كاملة بلا تعزيز: تدريب {single.score(X_tr, y_tr):.4f} | اختبار:
{single.score(X_te, y_te):.4f}")
```

الاتجاه معكوس عن الحدس الشائع: كلما تعمّقت الشجرة الأساسية انخفضت دقة الاختبار (٠٫٩٥٩١ بالجذوع مقابل ٠٫٩٢٩٨ بالأشجار الكاملة) رغم تحقيق الجميع ١٫٠ على التدريب. التعزيز يزدهر عندما تكون الوحدات ضعيفة — وكما رأينا في الفصل ٣٩، الشجرة العميقة تفرط في الملاءمة أصلاً قبل أي تجميع.

وهنا الفكرة التي توحّد كل عائلة التعزيز: **اجعل النموذج التالي يعالج أخطاء الحاضر**. AdaBoost يفعلها بإعادة الأوزان، وتعزيز التدرّج (الفصل ٤٦) بملاءمة **البواقي** مباشرة. الشكل التالي يوضح النسخة الثانية: بعد كل شجرة ينكمش توزيع البواقي نحو الصفر:



الشكل ٩٦توزيع البواقي يتقلص جولة بعد جولة: كل شجرة جديدة تلتهم جزءاً من الخطأ المتبقي

ما نراه في الألواح الستة: انكماش الانحراف المعياري للبواقي جولة بعد جولة. وAdaBoost يحقق التأثير نفسه من طرف آخر: بدل قياس البواقي عددياً يرفع وزن النقاط الخاطئة فيتجه تركيز الجولة التالية إليها حتماً. والجدول التالي يلخّص الفروق بين الأقارب الثلاثة:

المعيار	AdaBoost	تعزيز التدرّج	الغابة العشوائية
آلية التركيز على الخطأ	إعادة أوزان العينات أُسّياً	ملاءمة البواقي (المشتق السالب للخسارة)	لا تركيز — سحب بوتستراپي أعمى
تنوع النماذج	تتابعي تكيّفي	تتابعي تكيّفي	متوازٍ مستقل
خسارة التدريب	أُسّية (SAMME)	أي خسارة قابلة للاشتقاق	شوكائب الأشجار فقط
حساسية الضجيج	عالية جداً (ترفع أوزان الشواذب)	متوسطة	منخفضة
التوازي في التدريب	صعب (كل جولة تعتمد على سابقتها)	صعب (و subsample يخفف الزمن)	كامل (n_jobs=-١)

٦.٥ متى يفشل AdaBoost؟ الضجيج، والشواذ، ومقايضة learning_rate

ماذا يحدث لو شكّلت العيّنات المسمّاة خطأً (label noise) نسبة معتبرة؟ العينات الخاطئة والشاذة **لا يمكن لأي نموذج ضعيف تصحيحها**، فتبقى «صعبة» جولة بعد جولة ويتضاعف وزنها الأُسي حتى تسيطر على التوزيع — فيصرف النموذج طاقته على حفظ الضجيج:

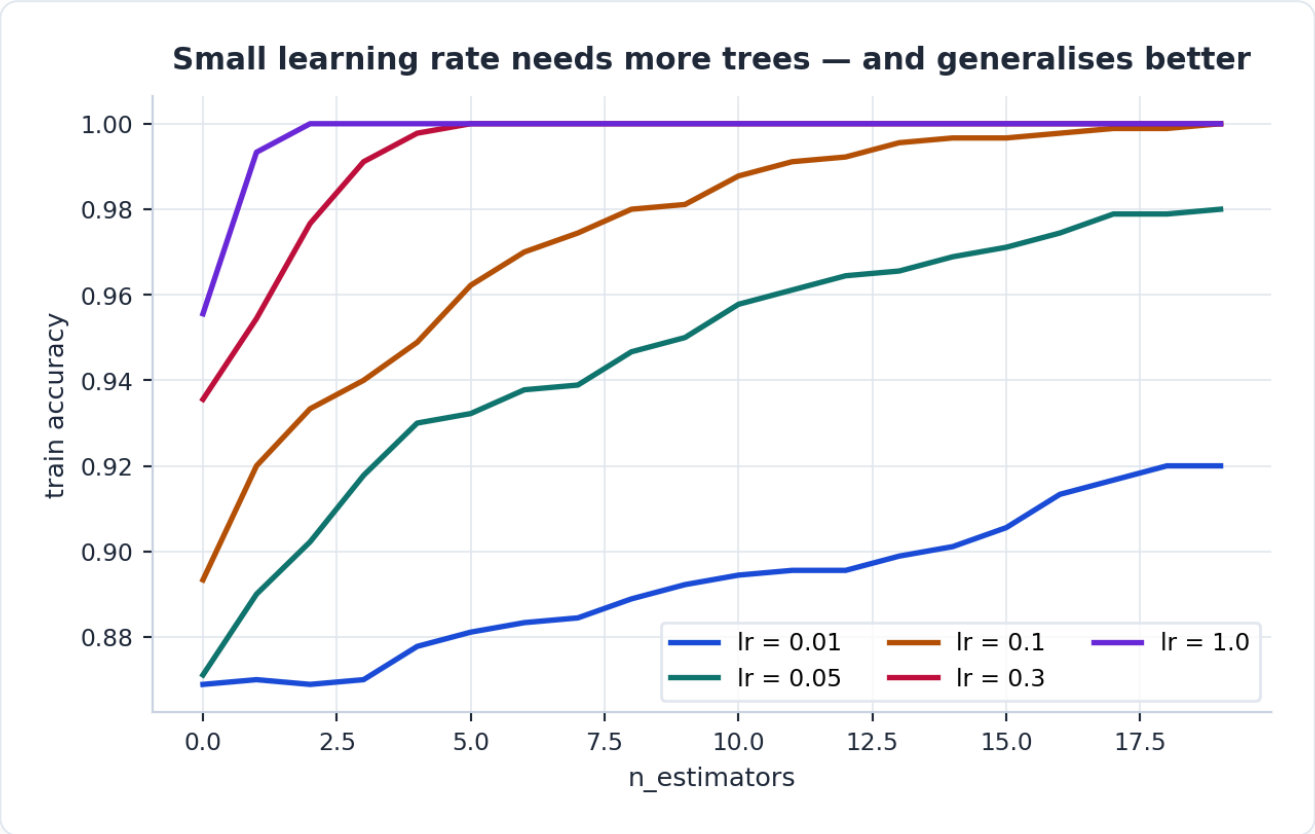
> **تحذير**
>
> الضجيج يقتل AdaBoost بصمت: العينات المسمّاة خطأً تتراكم أوزانها الأُسية جولة بعد جولة، فيستمر دقة التدريب في الارتفاع بينما تنهار دقة الاختبار. راقب `estimator_errors_`: إذا اقترب متوسطها من حد العشوائية `1 - 1/K` فأنت تدرب على الضجيج، ولن ينقذك زيادة `n_estimators`.

AdaBoost أمام 20% ضجيج في التسميات python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=600, n_features=10, n_informative=4,
    flip_y=0.20, class_sep=0.8, random_state=42,
)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=150, random_state=42,
).fit(X_tr, y_tr)

errs = ada.estimator_errors_
print(f"أخطاء الجذوع الأولى: {np.round(errs[:4], 4)}")
print(f"أخطاء آخر 4 جولات:   {np.round(errs[-4:], 4)}  (0.5 = حد العشوائية)")
staged_tr = list(ada.staged_score(X_tr, y_tr))
staged_te = list(ada.staged_score(X_te, y_te))
for k in [5, 25, 50, 100, 150]:
    print(f"بعد {k:3d} تدريب | جولة: {staged_tr[k-1]:.4f} | اختبار: {staged_te[k-1]:.4f}")
print(f"أفضل دقة اختبار: {max(staged_te):.4f} عند الجولة {int(np.argmax(staged_te)) + 1}")
```

أخطاء الجذوع تتسلّق من ٠٫٣٠٩٥ إلى ٠٫٤٩١٨ — على خطوة من حد العشوائية ٠٫٥ — لأن الجولات المتأخرة تحاول إرضاء عينات تسمياتها صادفة. ودقة التدريب ترتفع من ٠٫٧١٦٧ إلى ٠٫٨٥٤٨ بينما الاختبار يذبل حول ٠٫٧٠، وأفضل نقطة مبكرة جداً (الجولة ٧ بدقة ٠٫٧٤٤٤). الخلاصة: اتساع فجوة التدريب/الاختبار مع اقتراب أخطاء الجولات من ٥٠٪ يعني ضجيجاً في التسميات أو شواذاً — عالجهما قبل التعزيز أو خففهما بخفض `learning_rate` وتقليص الجولات.

وهذا يقودنا إلى مقبض الضبط الأخير: `learning_rate` يضرب معامل كل نموذج `α_t` فالقيمة الصغيرة تخفّض صوت كل جولة وتتطلب مزيداً من الجولات — مقايضة صريحة مع `n_estimators`. الشكل التالي (وُلّد بتعزيز تدرّج لكن المبدأ نفسه يحكم AdaBoost) يبيّن أن المعدلات الصغيرة تحتاج مزيداً من الأشجار وتمنح تعميماً أفضل:



الشكل ٩٧منحنيات دقة التدريب مقابل عدد الأشجار عند معدلات تعلم مختلفة: القيم الصغيرة تحتاج مزيداً من الجولات وتعمّم أفضل

نصيحة

ابدأ بـ `learning_rate=1.0` و `n_estimators=50` (الافتراضان) لخط أساس سريع، ثم خفّض `learning_rate` إلى ٠.١ مع مضاعفة الجولات للاستقرار الأفضل. واستعمل `staged_score` على مجموعة تحقق: الجولة التي يبلغ عندها الأداء قمته هي حصادك.

في الواقع

أشهر تطبيق تجاري لـ AdaBoost كان كاشف الوجوه لفيولا وجونز (٢٠٠١) الذي جمع آلاف السمات الضعيفة البسيطة في مصنّف واحد يعمل لحظياً في الكاميرات. لكن في جداول البيانات الحديثة انتقل العرش إلى HistGradientBoosting (الفصل ٤٧) لأنها أسرع وأكثر تحملاً للضجيج؛ ويبقى مكان AdaBoost في التدريس والمسائل الصغيرة الخفيفة، وعندما تحتاج متعلمين ضعفاء مخصصين غير الأشجار.

تمارين الفصل الخامس والأربعين

- **ملاحظة**

التمرين ٦.١

درب `AdaBoostClassifier` بجذوع (عمق ١) و `n_estimators=300` و `learning_rate=0.3` على بيانات Digits (`load_digits`)، ثم استعمل `staged_score` على مجموعة الاختبار لإيجاد أفضل جولة، واطبع دقّة الجولات ١ و ١٠ و ١٠٠ والجولة المثلى، وقارنها بالدرجة النهائية للنموذج الكامل.

التمرين ٦.٢

في مسألة تصنيف ثنائية، افترض أن جذعاً بعمق ١ حقق خطأً موزوناً ٠٫٦٠ على الأوزان الحالية. اشرح لماذا يرميه scikit-learn فوراً، وماذا سيحدث لو أُبقي (لمّاذا يكون `α` سالباً؟)، وكيف يتغيّر حد الرمي في مسألة من ثلاث فئات؟

التمرين ٦.٣

تحقق يدوياً من معادلة معامل النموذج `α_t = ln((1-ε_t)/ε_t) + ln(K-1)` على بيانات Iris كاملة عبر **خمس جولات**: احسب في كل جولة الأوزان والخطأ الموزون والمعامل يدوياً، وقارن كل جولة مع `ada.estimator_errors_` و `ada.estimator_weights_`، واطبع أكبر فرق مطلق.

- **ملاحظة**

حل التمرين 6.1: العثور على الجولة المثلى بـ staged_score python

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=300, learning_rate=0.3, random_state=42,
).fit(X_tr, y_tr)

staged = list(ada.staged_score(X_te, y_te))
for k in [1, 10, 100]:
    print(f"الجولة {k:3d} | دقة الاختبار المتدرجة: {staged[k-1]:.4f}")
best_k = int(np.argmax(staged)) + 1
print(f"الجولة المثلى: {best_k} | دقتها: {staged[best_k-1]:.4f}")
print(f"النموذج الكامل (جولة 300): {ada.score(X_te, y_te):.4f}")
```

الأداء هنا يتشبع مبكراً وأفضل جولة أقل بكثير من ٣٠٠ — هكذا يكون `staged_score` أداة اقتصاد: نموذج أصغر بالأداء ذاته.

> **ملاحظة**

حل التمرين 6.3: التحقق اليدوي من معادلة SAMME عبر 5 جولات python

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

X, y = load_iris(return_X_y=True)
n, K = len(y), 3

w = np.full(n, 1.0 / n)
manual_eps, manual_alpha = [], []
for t in range(5):
    stump = DecisionTreeClassifier(max_depth=1, random_state=42).fit(X, y,
sample_weight=w)
    wrong = stump.predict(X) != y
    eps = np.average(wrong, weights=w)
    alpha = np.log((1 - eps) / eps) + np.log(K - 1)
    manual_eps.append(eps)
    manual_alpha.append(alpha)
    w = w * np.exp(alpha * wrong)
    w = w / w.sum()

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=5, random_state=42,
).fit(X, y)

print("الجولة | α يدوياً | α مكتبة | الخطأ يدوياً | الخطأ مكتبة")
for t in range(5):
    print(f"  {t+1}    | {manual_eps[t]:.6f}  | {ada.estimator_errors_[t]:.6f}
|"
          f" {manual_alpha[t]:.6f} | {ada.estimator_weights_[t]:.6f}")
diff_eps = np.max(np.abs(np.array(manual_eps) - ada.estimator_errors_))
diff_alpha = np.max(np.abs(np.array(manual_alpha) - ada.estimator_weights_))
print(f"أكبر فرق في الأخطاء: {diff_eps:.2e} | أكبر فرق في المعاملات: {diff_alpha:.2e}")
```

الفرق في حدود ١٠⁻¹⁵ (دقة الفاصلة العائمة نفسها) — معادلة SAMME مطابقة تماماً لما تنفّذه scikit-learn: خطأ موزون، معامل لوغاريتمي، تحديث أُسي، وإعادة تطبيع كل جولة.

> **ملاحظة**

٦.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تمرير `algorithm="SAMME"` **إلى** `AdaBoostClassifier` **في ١.٩**	`TypeError: got an unexpected keyword argument 'algorithm'` — البارامتر أُزيل نهائياً	احذف البارامتر؛ SAMME وحدها الباقية وتُختار ضمنياً
استخدام أشجار عميقة كقاعدة (`max_depth=None` **)**	الجولة الأولى تلتهم الإشارة وترتفع التباين: اختبار ٠٫٩٢٩٨ مقابل ٠٫٩٥٩١ بالجذوع	إبقاء `max_depth=1` في القاعدة (الافتراضي) وترا التعلم للجولات
زيادة `n_estimators` **لعلاج ضجيج التسميات**	تراكم أوزان العينات الخاطئة أُسّياً: تدريب ٠٫٨٥٤٨ مقابل اختبار ٠٫٦٨٨٩	نظّف الضجيج أو أو خفّض `earning_rate` وقلّص الجولات
تجاهل التوقف المبكر رغم استقرار `staged_score`	إهدار زمن وذاكرة: ٣٠٠ جولة والقمة فعلياً في الجولة ٤٢	راقب `staged_score` على مجموعة تحقق واضبط `n_estimators` بناءً عليها

٦.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المتعلم الضعيف	نموذج خطؤه أقل بقليل من العشوائية (½ في الثنائية)	Weak learner
التعزيز	بناء نموذج قوي من سلسلة متعلمين ضعفاء يركز كل منهم على أخطاء سابقه	Boosting
الخطأ الموزون	نسبة الخطأ محسوبة على أوزان العينات الحالية لا عدّها ببساطة	Weighted error
معامل النموذج	وزن صوت النموذج في التصويت النهائي، لوغاريتمي بحساب خطئه الموزون	Estimator weight
خوارزمية SAMME	صيغة AdaBoost متعددة الفئات في scikit-learn بعد إزالة SAMME.R	SAMME (Stagewise Additive Modeling using a Multi-class Exponential loss)

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٦ — تعزيز التدرّج: شرح كلمة بكلمة

الدالة الخسارة، والبواقي، و learning_rate، و subsample

الوحدة **scikit-learn ١.٩** الحجم **٨٢٨** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا يلائم كل نموذج جديد البواقي (المشتق السالب للخسارة) بدل التسميات
- أن تبني تعزيز تدرّج يدوياً بخطواته وتطابقه مع GradientBoostingRegressor رقمياً
- أن تختار خسارة مناسبة من الخيارات الحقيقية في ١.٩ وتفعّل التوقف المبكر بـ n_iter_no_change
- أن تستخدم staged_predict لإيجاد عدد الأشجار الأمثل بدل التخمين
- أن توازن بين learning_rate و n_estimators وبين subsample وزمن التدريب بأرقام حقيقية

٦.١ الفكرة الجوهرية: كل نموذج جديد يلائم البواقي لا التسميات

تخيّل طاهياً يتذوق طبخة فيبتسم: كل ما يقوله هو «الملح ناقص بكثير». ثم يتذوق مجدداً: «الملح جيد، لكن الحموضة ناقصة». كل تعديل لا يعيد الطبخ من الصفر — بل يعالج **ما تبقى ناقصاً** فقط. تعزيز التدرّج يطبخ نموذجك بالطريقة نفسها: النموذج الأول يلتقط أكبر نمط، ثم نقيس الخطأ المتبقي (**البواقي**، residuals)، وندرّب النموذج التالي على هذا الخطأ، ونضيف مساهمته إلى الحصيلة، ونكرر.

لماذا لا نمضي أبعد من ذلك: النموذج الثالث في السلسلة لا يعرف شيئاً عن التسميات الأصلية `y`؛ واجبه الوحيد تصحيح الخطأ الذي خلفه أسلافه:

$$r_{i,m} = -\left[\frac{\partial L(y_i,\, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F = F_{m-1}}$$

هذه المعادلة هي «كلمة بكلمة» العنوان كله: البواقي في التعزيز العام ليست بالضرورة `y - F`، بل **المشتق السالب للخسارة** `L` بالنسبة إلى تنبؤ النموذج، محسوباً عند النموذج الحالي `F_{m-1}`. في الخسارة المربّعة المألوفة ينعكس ذلك بالبقايا المعتادة:

$$L(y, F) = \tfrac{1}{2}(y - F)^2 \quad \Longrightarrow \quad r = y - F$$

ثم يبنى النموذج التجميعي جمعياً: كل جولة تضيف شجرة `h_m` صامتة صغيرة على التنبؤ السابق بمعدل تعلم `v` (learning_rate):

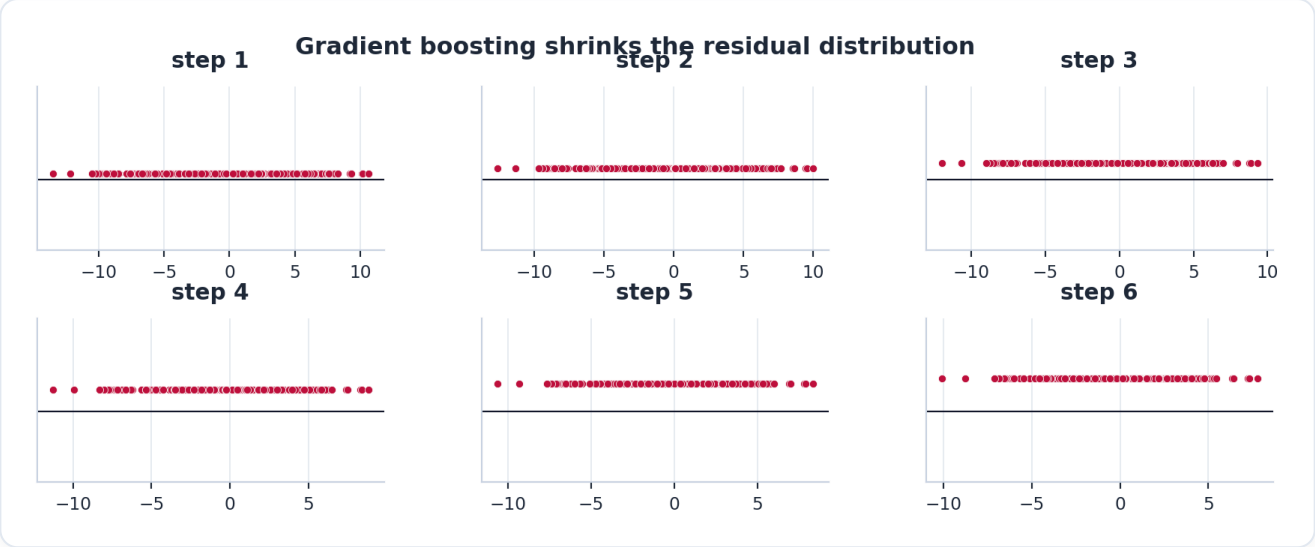
$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu\, h_m(\mathbf{x}), \quad F_0(\mathbf{x}) = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, \gamma)$$

حيث `F_0` أفضل تقدير ثابت (متوسط `y` في الخسارة المربّعة)، و`v` بين ٠ وا يمنع كل شجرة من أن «تقفز» نحو الحل بخطوة كبيرة تفرّط في الملاءمة. دعنا نبني كل هذا يدوياً بأربع أشجار ونقارن بالخوارزمية الفعلية:

scikit-learn بناء تعزيز تدرّج يدوياً ومطابقته مع — python

```python
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import root_mean_squared_error

rng = np.random.default_rng(42)
X = rng.uniform(0, 1, size=(240, 4))
y = X[:, 0] * np.sin(np.pi * X[:, 1]) + 0.3 * rng.normal(size=240)
nu = 0.5

# البداية: أفضل تقدير ثابت = متوسط y
F = np.full(len(y), y.mean())
for m in range(1, 5):
    r = y - F                                   # البواقي: المشتق السالب للخسارة المربعة
    tree = DecisionTreeRegressor(max_depth=2, random_state=42).fit(X, r)
    F = F + nu * tree.predict(X)                # الخطوة الجمعية الصغيرة
    print(f"بعد الشجرة {m}: RMSE التدريب = {root_mean_squared_error(y, F):.4f}")

gb = GradientBoostingRegressor(
    n_estimators=4, learning_rate=0.5, max_depth=2, random_state=42,
).fit(X, y)
manual = float(np.max(np.abs(gb.predict(X) - F)))
print(f"أكبر فرق بين البناء اليدوي و scikit-learn: {manual:.2e}")
```

بعد أربع أشجار فقط هبط الخطأ من ٠٫٣٤٥٢ إلى ٠٫٢٨٦٧، والبناء اليدوي — عشرة أسطر — يطابق `GradientBoostingRegressor` بفرق لا يتجاوز 10^{-16} (ضجيج الفاصلة العائمة). الشكل التالي يعرض الرحلة نفسها بصرياً: توزيع البواقي ينكمش جولة بعد جولة حتى يقترب من الصفر:



الشكل ٩٨توزيع البواقي يتقلص جولة بعد جولة: كل شجرة جديدة تلتهم جزءاً من الخطأ المتبقي

ما نقرأه في الألواح الستة: بعد شجرة واحدة لا تزال البواقي متباعدة (انحرافها ٣,٥٣)، وبعد ست أشجار تتجمع حول الصفر (٠,٣٤) — هذا هو «حصاد الخطأ» التدريجي الذي تبنى عليه العائلة كلها.

٦.٢ من AdaBoost إلى تعزيز التدرّج: تعميم واحد غيّر كل شيء

ما الفرق الجوهري إذن بين عائلتين تشتركان في البناء التتابعي؟ رأينا في الفصل ٤٥ أن AdaBoost تعيد **أوزان العينات** أُسّياً. تعزيز التدرّج لا يعيد أي وزن: فهو يعيد صياغة «التركيز على الأخطاء» بلغة التفاضل — البواقي هي المشتق السالب للخسارة، وكل جولة خطوة انحدار تدرّج (gradient descent) في فضاء الدوال. النتيجة ثورية عملياً: **أي خسارة قابلة للاشتقاق تصبح مسموحة** — مربّعة للانحدار، لوغاريتمية للتصنيف، مطلقة للشواذب، كوانتيلية للتقدير الفاصل. AdaBoostg نفسها تظهر كحالة خاصة: هي تعزيز تدرّج بخسارة أُسّية. لهذا قلنا في الفصل ٤٥ إن التحديث الأُسي ليس سحراً — إنه تدرّج متنكّر.

الخسارة	المتاح في	تكافؤها	متى تختارها؟
`squared_error`	GradientBoostingRegressor	البواقي = `y - F`	الافتراض العام للانحدار
`absolute_error`	GradientBoostingRegressor	وسيط البواقي	وجود شواذب في الهدف
`huber`	GradientBoostingRegressor	مزيج يتجاهل الشواذب الكبيرة	بيانات انحدار معتّبة
`quantile`	GradientBoostingRegressor	تقدير فاصل alpha	فترات تنبؤ وعدم تماثل
`log_loss`	GradientBoostingClassifier	احتمالات صحيحة	الافتراض للتصنيف
`exponential`	GradientBoostingClassifier	تكافئ AdaBoost	تمايل مع سلوك AdaBoost

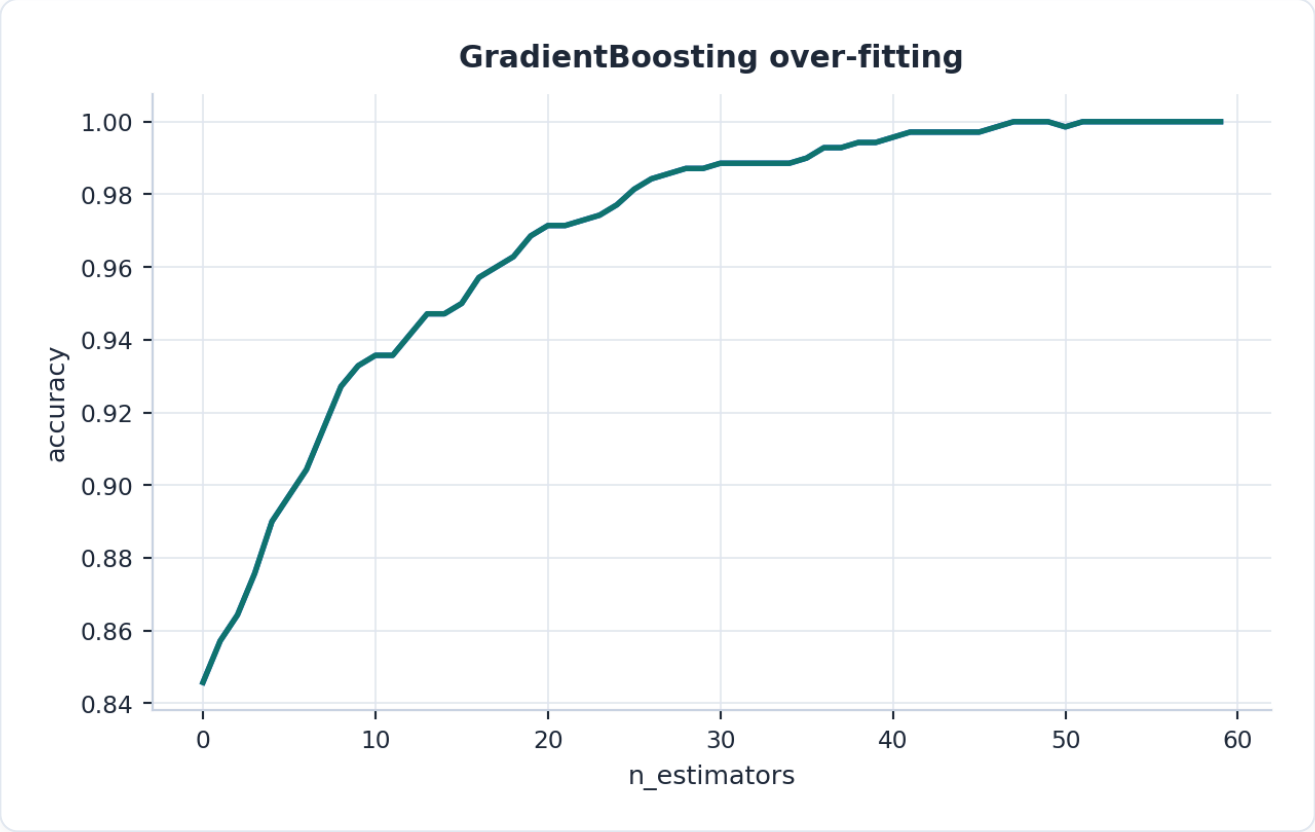
٦.٣ GradientBoostingRegressor و GradientBoostingClassifier في ١.٩

تحققنا في فتح الكتاب من قائمة `get_params()` الحقيقية؛ أبرز ما فيها للطرفين: `learning_rate` (افتراضه ٠.١)، و`n_estimators` (افتراضه ١٠٠)، و`max_depth` (افتراضه ٣ — أشجار صغيرة متعبة لا غابات)، و`subsample` (افتراضه ١.٠)، و`loss`، وثلاثية التوقف المبكر: `n_iter_no_change` (افتراضه None)، و`validation_fraction` (افتراضه ٠.١)، و`tol`. انتبه لتفصيل مهم: لا يوجد بارامتر اسمه `early_stopping` في هذين الصنفين — التوقف المبكر يُفعَّل بمجرد ضبط `n_iter_no_change` إلى عدد صحيح موجب، فيخصص الكلاس `validation_fraction` من بيانات التدريب للمراقبة ويتوقف إذا لم تتحسن الخسارة عليها بعد `n_iter_no_change` جولة (بتسامح `tol`). أما البارامتر `early_stopping='auto'` الذي تراه في التوثيق فينتمي إلى HistGradientBoosting في الفصل ٤٧.

التوقف المبكر ومراقبة مسار التعلم بـ staged_predict python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=1200, n_features=12, n_informative=6,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

gb = GradientBoostingClassifier(
    n_estimators=500, n_iter_no_change=10, validation_fraction=0.15,
random_state=42,
).fit(X_tr, y_tr)
print(f"طلبت 500 شجرة فبنى فعلياً {gb.n_estimators_} فقط بفضل التوقف المبكر")
print(f"دقة الاختبار: {gb.score(X_te, y_te):.4f}")

# مسار التعلم: دقة الاختبار بعد كل شجرة عبر staged_predict
gb100 = GradientBoostingClassifier(n_estimators=100, learning_rate=0.2,
random_state=42).fit(X_tr, y_tr)
staged = [accuracy_score(y_te, p) for p in gb100.staged_predict(X_te)]
for k in [1, 5, 10, 25, 50, 100]:
    print(f"بعد {k:3d} شجرة | دقة الاختبار: {staged[k-1]:.4f}")
print(f"أفضل نقطة: {max(staged):.4f} عند الشجرة {int(np.argmax(staged)) + 1}")
```

طُلب ٥٠٠ شجرة فبنى النموذج ١٥٦ فقط — التوقف المبكر وفّر ثلثي الكلفة بلا خسارة أداء (٠٫٨٩١٧). و`staged_predict` (المولّد الذي يعيد تنبؤات النموذج بعد كل شجرة) يكشف مسار التعلم: من ٠٫٦١٩٤ بعد شجرة واحدة إلى القمة ٠٫٨٩٤٤، مع منحنى يتشبع مبكراً — وهذا بالضبط ما نراه في الشكل التالي حيث تبتلع منطقة التوقف المبكر الفجوة بين القمة والنموذج الممتد أكثر من اللازم:



الشكل ٩٩منحنى التدريب والاختبار مقابل عدد الأشجار: تتجاوز منطقة الإفراط في الملاءمة قمة التعميم من مدة

وللانحدار، لنتحقق من وعد جدول الخسائر: شواذب تُحقن في أهداف التدريب فقط، والقياس على اختبار نظيف:

الشواذب تعاقب squared_error ولا تعنين absolute_error python

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error, mean_absolute_error

X, y = make_regression(n_samples=800, n_features=8, n_informative=5, noise=12,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)
y_tr = y_tr.copy()
y_tr[:40] += 120.0                          # شواذب في التدريب فقط 5%

for loss in ["squared_error", "absolute_error"]:
    reg = GradientBoostingRegressor(loss=loss, n_estimators=120,
random_state=42).fit(X_tr, y_tr)
    rmse = root_mean_squared_error(y_te, reg.predict(X_te))
    mae = mean_absolute_error(y_te, reg.predict(X_te))
    print(f"loss={loss:15s} | RMSE نظيف: {rmse:7.2f} | MAE نظيف: {mae:7.2f}")
```

النتيجة حاسمة: مع ٥٪ شواذ في أهداف التدريب، خسارة `absolute_error` خفّضت RMSE الاختبار من ٢٣٫٢٠ إلى ١٧٫٤٥ — لأن الخسارة المربّعة تجعل الشواذ «تصرخ» في البواقي فتستدرج الأشجار نحوها، بينما الخسارة المطلقة تنظر إلى الحجم المطلق للخطأ فتتجاهل الصراخ.

٦.٤ subsample: تعزيز عشوائي يوفر الزمن ويحسّن التعميم

ماذا لو درّبنا كل شجرة على عيّنة عشوائية ٤٠٪ من البيانات فقط؟ هذا هو **التعزيز العشوائي (Stochastic Gradient Boosting)** الذي اقترحه فريدمان ٢٠٠٢: البارامتر `subsample` يحدد نسبة العينات التي تراها كل شجرة (بدون إرجاع). الفائدة مزدوجة — أقل حسابات لكل شجرة، وضجيج موزون يمنع الأشجار من التعاندق في حفظ نفس النقاط:

أثر subsample على الزمن والتعميم على 6000 عينة python

```python
import time
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=6000, n_features=15, n_informative=8,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

for ss in [1.0, 0.7, 0.4]:
    t0 = time.perf_counter()
    gb = GradientBoostingClassifier(n_estimators=150, subsample=ss,
random_state=42).fit(X_tr, y_tr)
    t = time.perf_counter() - t0
    print(f"subsample={ss} | الزمن: {t*1000:6.0f} ms |"
          f" تدريب: {gb.score(X_tr, y_tr):.4f} | اختبار: {gb.score(X_te,
y_te):.4f}")
```

الأرقام تختصر الموضوع: خفض `subsample` من ١.٠ إلى ٠.٤ قلّص زمن التدريب من ٤٤١٩ إلى ٢٠٧٥ ميلي ثانية (أقل من النصف)، **ورفع** دقة الاختبار من ٠٫٨٧٥٦ إلى ٠٫٨٨٠٦. هذا النمط — سرعة أكبر وتعميم أفضل معاً — ليس صدفة: العينات المستبعدة عشوائياً تعمل كمنظم طبيعي يشبه ما تفعله الغابة العشوائية بالعينات، وتكسر ارتباط الأشجار المتعاقبة.

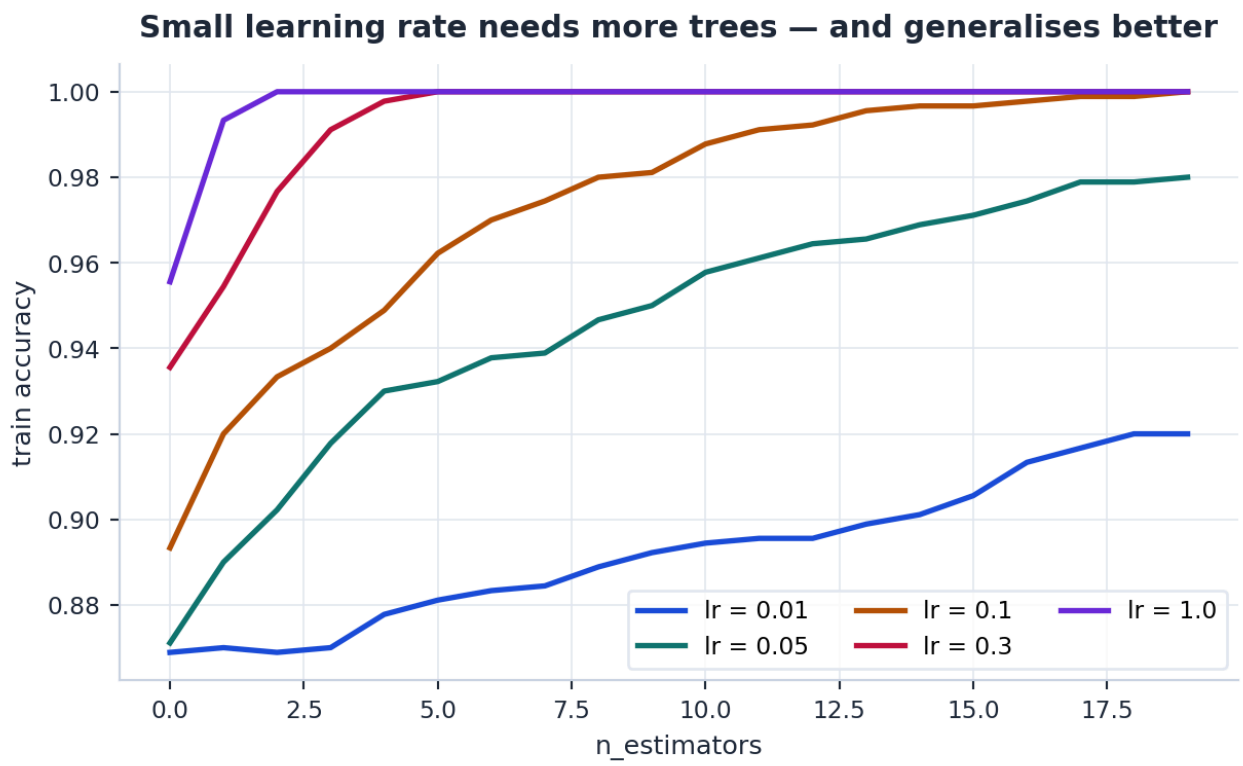
> **خطأ شائع**
>
> لا تضبط `subsample` وحدها إطلاقاً في بحث شبكي (grid search) منفصلاً عن `learning_rate` و`n_estimators`؛ فهي تقلل مساهمة كل شجرة في التعلم، وقد تحتاج إلى مزيد من الأشجار لتعويض ذلك. ابحث عن الثلاثية معاً كما سيفعل الفصل ٤٩.

٦.٥ المقايضة الذهبية بين *learning*rate و *n*estimators

كلا البارامترين يقيسان «كم تتعلم النظام من كل جولة»: `v` صغير يعني خطوات صغيرة دقيقة تحتاج مزيداً من الجولات. القاعدة التجريبية المشهورة: الأزواج التي يبقى فيها حاصل `learning_rate × n_estimators` قريباً من ثابت تصل إلى الأداء نفسه تقريباً، لكن القيم الصغيرة من `learning_rate` تعمّم أفضل عادة لأنها تبني نموذجاً أنعم:

```python
# مقايضة learning_rate مع n_estimators بالأرقام
import time
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=1200, n_features=12, n_informative=6,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

for lr, n in [(1.0, 50), (0.5, 100), (0.1, 300), (0.05, 600)]:
    t0 = time.perf_counter()
    gb = GradientBoostingClassifier(
        learning_rate=lr, n_estimators=n, max_depth=2, random_state=42,
    ).fit(X_tr, y_tr)
    t = time.perf_counter() - t0
    print(f"lr={lr:4} | n={n:3d} | الزمن: {t*1000:6.0f} ms |"
          f" تدريب: {gb.score(X_tr, y_tr):.4f} | اختبار: {gb.score(X_te,
y_te):.4f}")
```

اقرأ العمودين الأخيرين بعناية: عند `lr=1.0` ملأ التدريب سقفه (١٫٠) بينما الاختبار عند ٠٫٨٩١٧ — أعراض إفراط في الملاءمة. ومع إنقاص `lr` ومضاعفة الأشجار هبط التدريب إلى ٠٫٩٨٨١ وارتفع الاختبار إلى ٠٫٩٠٢٨. الشكل التالي يرسم هذه التجربة على شبكة أوسع من معدلات التعلم:



الشكل ١٠٠منحنيات دقة التدريب مقابل عدد الأشجار عند معدلات تعلم مختلفة: القيم الصغيرة تحتاج مزيداً من الجولات وتعمّم أفضل

نصيحة

وصفة بداية عملية على بيانات جدولية متوسطة: `learning_rate=0.1` مع `n_estimators=100` و`max_depth=3`، ثم فعّل `n_iter_no_change=10` ودع النموذج يقرر عدد الأشجار بنفسه. بعد ذلك فقط، خفّض `learning_rate` إلى ٠.٠٥ أو ٠.٠١ مع مضاعفة `n_estimators` إذا كنت تطارد آخر ٠٫٥٪ من الأداء.

في الواقع

على أرض الواقع: مكتبات XGBoost وLightGBM وCatBoost التي تهيمن اليوم على مسابقات البيانات هي تعزيز تدرّج بتحسينات هندسية فوق الفكرة نفسها التي بنيناها يدوياً في هذا الفصل — والفصل ٤٧ يقدم نسخة scikit-learn الصناعية منها (HistGradientBoosting) المستوحاة من LightGBM مباشرة. حتى بعد عشرين عاماً على ورقة فريدمان، يظل «ملاءمة البواقي بخطوات صغيرة» الفكرة الأنجح تجارياً في تعلّم الآلة على البيانات الجدولية.

تمارين الفصل السادس والأربعين

- **ملاحظة**

التمرين ٦.١

على بيانات `make_friedman1` (n=١٠٠٠، ١٠ ميزات، ضجيج ٥)، درّب `GradientBoostingRegressor` بـ `n_estimators=300`، ثم استعمل `staged_predict` على مجموعة اختبار لإيجاد أفضل عدد أشجار وأدنى RMSE عند هذه القمة، وقارنها بـ RMSE النموذج الكامل (٣٠٠ شجرة).

التمرين ٦.٢

اشرح كلمة بكلمة لماذا يُفعَّل التوقف المبكر في `GradientBoostingClassifier` بضبط `n_iter_no_change` بينما لا يوجد بارامتر `early_stopping` في توقيعه، وما دور كل من `validation_fraction` و `tol` في القرار، ولماذا يعاد التنبؤ بعدئذٍ بـ `n_estimators_` لا بـ `n_estimators`.

التمرين ٦.٣

كرّر تجربة المقايضة (lr × n) على بيانات make_classification نفسها، لكن أضف البارامتر `subsample=0.7` لكل الأزواج، وقارن زمن التدريب ودقة الاختبار مع النتائج بلا `subsample` — هل تتحسن المقايضة بأكملها أم يسوء جزء منها؟

- **ملاحظة**

حل التمرين 6.1: أفضل عدد أشجار عبر staged_predict python

```python
import numpy as np
from sklearn.datasets import make_friedman1
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error

X, y = make_friedman1(n_samples=1000, n_features=10, noise=5.0, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

gb = GradientBoostingRegressor(n_estimators=300, random_state=42).fit(X_tr,
y_tr)
staged = [root_mean_squared_error(y_te, p) for p in gb.staged_predict(X_te)]
best_m = int(np.argmin(staged)) + 1
print(f"أدنى RMSE اختبار: {staged[best_m-1]:.4f} عند {best_m} شجرة")
print(f"RMSE (شجرة 300) النموذج الكامل: {staged[-1]:.4f}")
print(f"الوفر: {300 - best_m} شجرة يمكن حذفها بلا كلفة أداء")
```

القمة تظهر مبكراً ثم ينحدر المنحنى قليلاً بعدئذٍ — دليل على أن آخر الأشجار تعمل على ضجيج مجموعة التدريب. حوّل هذا التشخيص إلى سطر واحد مع `staged_predict` و`argmin`.

ملاحظة

حل التمرين 6.3: مقايضة lr x n مع subsample=0.7 python

```python
import time
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=1200, n_features=12, n_informative=6,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

for lr, n in [(1.0, 50), (0.5, 100), (0.1, 300), (0.05, 600)]:
    t0 = time.perf_counter()
    gb = GradientBoostingClassifier(
        learning_rate=lr, n_estimators=n, max_depth=2,
        subsample=0.7, random_state=42,
    ).fit(X_tr, y_tr)
    t = time.perf_counter() - t0
    print(f"lr={lr:4} | n={n:3d} | الزمن: {t*1000:6.0f} ms |"
          f" تدريب: {gb.score(X_tr, y_tr):.4f} | اختبار: {gb.score(X_te,
y_te):.4f}")
```

بمقارنة هذه الأرقام مع تجربة الفصل: الزمن انخفض في كل الأزواج (العشوائية تقلل عمل كل شجرة)، ودقة الاختبار تحسّنت أو ثلت عند معظم الأزواج — لكن انتبه للزوج (`lr=0.05` , n=٦٠٠): مع subsample صار كل شجرة تتعلم أقل، فالحاصل `lr × n` لم يعد كافياً وقد تحتاج مزيداً من الأشجار. المقايضة الثلاثية تُدار معاً لا واحداً واحداً.

> **ملاحظة**

٦.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
البحث عن `early_stopping` في توقيع `GradientBoostingClassifier`	لا يوجد هذا البارامتر أصلاً في ١.٩ — هو حكر على HistGradientBoosting	فعّل التوقف المبكر بـ `n_iter_no_change=10` مع `validation_fraction`
استخدام `gb.n_estimators` لمعرفة عدد الأشجار الفعلية	يعيد القيمة المطلوبة (٥٠٠) لا المبنية (١٥٦) بعد التوقف المبكر	اقرأ السمة `n_estimators_` (بشرطة سفلية) بعد fit
خسارة `squared_error` مع أهداف مليئة بالشواذب	الشواذب تسود البواقي فتنحرف الأشجار: RMSE نظيف ٢٣٫٢٠ مقابل ١٧٫٤٥	اختر `absolute_error` أو `huber` عند وجود شواذب في الهدف
إبقاء `learning_rate=1.0` مع أشجار كثيرة	قفزات كبيرة تفرط في الملاءمة: تدريب ١٫٠ واختبار ٠٫٨٩١٧	خفّض `learning_rate` إلى ٠.١ أو أقل مع رفع `n_estimators`

٦.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البواقي	الفرق بين القيمة الحقيقية وتنبؤ النموذج الحالي؛ المشتق السالب للخسارة عموماً	Residuals / pseudo-residuals
البناء الجمعي التدريجي	إضافة نموذج صغير إلى حصيلة سابقه بخطوة مضبوطة في كل جولة	Stagewise additive building
التعزيز العشوائي	تدريب كل شجرة على عيّنة جزئية عشوائية من البيانات لتسريع التدريب وتنويع الأشجار	Stochastic gradient boosting
معدل التعلم	عامل التقليل `ν` الذي يضرب مساهمة كل شجرة في الجمع	Learning rate / shrinkage
التوقف المبكر بالتحقق	مراقبة الخسارة على شريحة تحقق والقف عن البناء عند توقف التحسن	Early stopping via validation

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٧ — HistGradientBoosting: المعيار الصناعي

المدرّجات التكرارية، والقيم المفقودة الأصلية، والسرعة

الوحدة **scikit-learn ١.٩** الحجم **٨٧٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف يجعل الرسم البياني (binning) التدريب أسرع بمرتبة كاملة تعقيداً
- أن تطبق HistGradientBoostingClassifier و Regressor وتوازن max_iter و max_leaf_nodes و learning_rate
- أن تغذي النموذج بيانات فيها NaN مباشرة وأعمدة فئوية من نوع Categorical دون أي معالجة يدوية
- أن تفعّل 'early_stopping='auto وتقرأ _n_iter و _validation_score لضبط عدد التكرارات
- أن تقرر متى تختار HistGB حتماً على الجداول الكبيرة ومتى تخسر أمام بدائله على الصغيرة

٦.١ لماذا تعدّد؟ ٢٥٥ صندوقاً بدل ملايين العتبات

تذكّر من الفصل ٤٦ أن كل عقدة في شجرة تعزيز التدرّج تبحث عن أفضل عتبة انقسام لكل ميزة، وذكرنا في الفصل ٤٤ أن البحث عن العتبات يلتهم معظم زمن التدريب. المشكلة الهندسية صريحة: ميزة مستمرة في عيّنة من مئة ألف صف لها ما يقارب مئة ألف عتبة مرشحة، فتُفرز القيم وتُفحص واحدة واحدة في كل عقدة، في كل شجرة، في كل تكرار. جاء LightGBM عام ٢٠١٧ بفكرة كانت تبدو تافهة: **لا تحتفظ بالقيم الحقيقية أصلاً**. قبل التدريب، درّج (bin) قيم كل ميزة في صناديق متساوية التردد لا تتجاوز ٢٥٥، واستبدل كل قيمة برقم صندوقها — بايت واحد! — ثم افحص ٢٥٥ صندوقاً فقط بدل n قيمة:

$$n - 1 \text{ candidate thresholds} \;\Rightarrow\; 255 \text{ bins per feature}$$

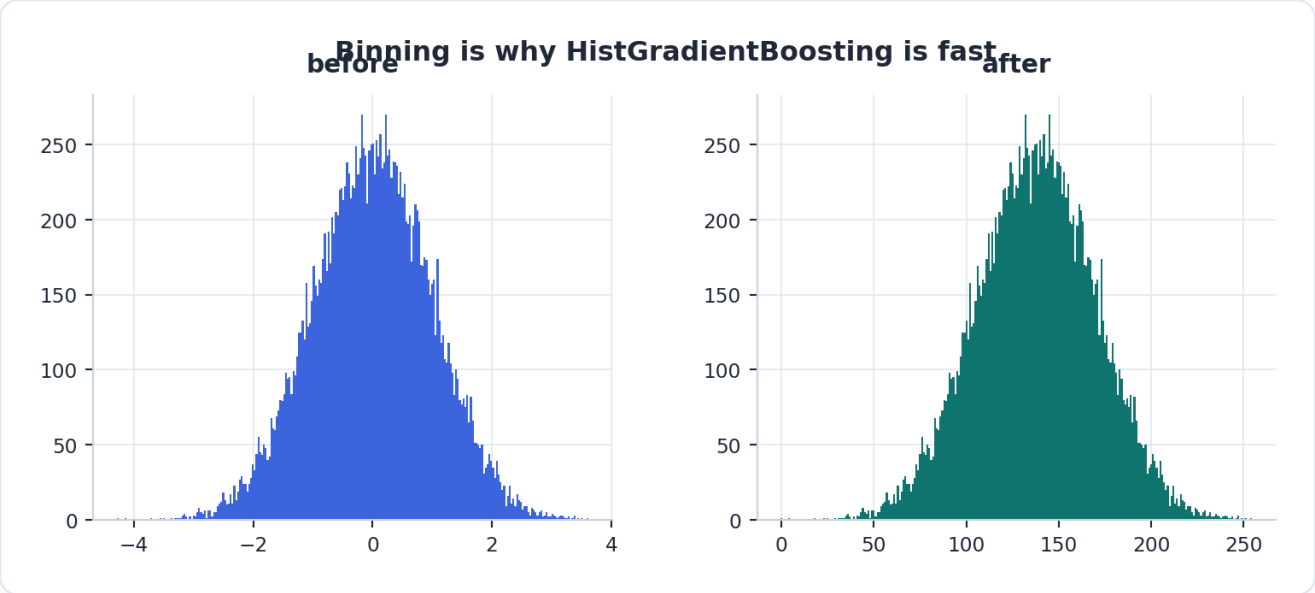
ثم بنى LightGBM — وتلاه فريق scikit-learn في `HistGradientBoosting` — ما يسمى **مدرّج التدرّجات (Gradient Histogram)**: مصفوفة عدّادات لكل صندوق تجمع فيها مجموع التدرّجات ومربعاتها، فيُحسب مكسب كل انقسام محتمل من ٢٥٥ خلية جاهزة بدل المرور على العينات. وتبقى بنية التعزيز ذاتها من الفصل ٤٦:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu\, h_m(\mathbf{x})$$

والدالة المُستَمثَّلة نفسها مع تنظيم صريح على قيم الأوراق عبر `l2_regularization`:

$$J = \sum_{i=1}^{n} L(y_i,\, F(\mathbf{x}_i)) + \tfrac{\lambda}{2} \sum_{j \in \text{leaves}} w_j^2$$

كلفة التفريغ؟ دقة حدّية طفيفة في العتبات نادراً ما تُلحَظ على الأداء النهائي، مقابل ترتيب كامل من التعقيد أقل في كل انقسام. الشكل التالي يوضح التحويل: من قيم خام لكل منها موضعها الخاص، إلى ٢٥٥ صندوقاً تُدار كوحدة واحدة:



الشكل ١٠١من قيم خام كل واحدة موضع فريد إلى ٢٥٥ صندوقاً هي ما يعمل عليه النموذج فعلاً

الألواح تقول كل شيء: الأعمدة بعد التدرّج تختصر توزيع الميزة كاملاً في ٢٥٥ خلية — وبايت واحد لكل عينة لكل ميزة، ومن هنا اسم العائلة (Hist).

مفتاح

الفكرة المحورية: HistGradientBoosting هي تعزيز التدرّج نفسه (نفس البناء الجمعي، نفس الخسائر، نفس البواقي) بعد استبدال فحص العتبات الحقيقية بالعمل على مدرّج من ٢٥٥ صندوقاً كحد أقصى — قرار هندسي مستلهم من LightGBM يقصّ زمن التدريب بمرتبة تعقيد كاملة ويفتح الباب لدعم أصلي للقيم المفقودة والفئويات.

> **سياق**
>
> من LightGBM إلى scikit-learn: أطلقت Microsoft مكتبة LightGBM عام ٢٠١٧ مبنية على تقنية GOSS وEFB ومدرّجات ٢٥٥ صندوقاً (بايت لكل قيمة)، فمزّقت الزمن على البيانات الضخمة. أضاف فريق scikit-learn `HistGradientBoosting*` مستوحى منها مباشرة في الإصدار ٠.٢١ (٢٠١٩)، وصار منذ ذلك اليوم الحل الافتراضي للبيانات الجدولية الكبيرة داخل المكتبة نفسها.

٦.٢ الواجهة في ١.٩ والسباق مع GradientBoosting

افتح التوقيع أولاً — لا تخترع اسماً بارامتر أبداً — ثم أجرِ السباق على ٢٠٠٠٠ عينة بعدد أشجار متساوٍ (١٠٠) للطرفين:

```python
# البارامترات الحقيقية ثم السباق: GradientBoosting مقابل HistGB
import time
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier,
HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

print("البارامترات المفتاحية لـ HistGradientBoostingClassifier 1.9 في:")
p = HistGradientBoostingClassifier().get_params()
for k in ["max_iter", "max_leaf_nodes", "learning_rate", "max_bins",
          "l2_regularization", "max_features", "categorical_features",
          "early_stopping", "validation_fraction", "n_iter_no_change", "loss"]:
    print(f"  {k} = {p[k]!r}")

X, y = make_classification(n_samples=20_000, n_features=20, n_informative=10,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42,
stratify=y)

t0 = time.perf_counter()
gb = GradientBoostingClassifier(n_estimators=100, random_state=42).fit(X_tr,
y_tr)
t_gb = time.perf_counter() - t0
print(f"GradientBoosting | الزمن: {t_gb:6.2f} s | دقة الاختبار: {gb.score(X_te,
y_te):.4f}")

t0 = time.perf_counter()
hgb = HistGradientBoostingClassifier(max_iter=100, random_state=42).fit(X_tr,
y_tr)
t_h = time.perf_counter() - t0
print(f"HistGB           | الزمن: {t_h:6.2f} s | دقة الاختبار: {hgb.score(X_te,
y_te):.4f}")
print(f"عامل السرعة: {t_gb / t_h:.1f}x لصالح HistGB")
```

البارامترات المطابقة تشبه قرينة لغوية بين العائلتين، مع فروق جوهرية: `max_iter` يلعب دور `n_estimators` (افتراضه ١٠٠)، و`max_leaf_nodes` (افتراضه ٣١) يحل محل `max_depth` — الأشجار هنا تنمو بالعرض لا بالعمق، وهي أشجار LightGBM الورقية (leaf-wise) المشهورة، و`max_bins` (افتراضه ٢٥٥) و`l2_regularization` (افتراضه ٠.٠) و`max_features` (أضيف في ١.٤، افتراضه ١.٠ — نسبة الميزات العشوائية في كل انقسام كشكل من التنظيم). وعلى السباق: نحو ١١ ضعفاً أسرع **وأدق** في آن واحد (٠.٩٦٥٨ مقابل ٠.٩٤٤٦) — المدرّجات لم تلتهم الدقة بل منحت التدرّج مزيداً من التكرارات داخل الثانية نفسها.

والآن سؤال المقياس الحقيقي: مئات الآلاف:

```python
# HistGB على 300 ألف عينة: كم يستغرق؟
import time
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=300_000, n_features=20, n_informative=10,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42,
stratify=y)

t0 = time.perf_counter()
hgb = HistGradientBoostingClassifier(max_iter=100, random_state=42).fit(X_tr,
y_tr)
t = time.perf_counter() - t0
print(f"300,000 الزمن | ميزة 20 × عينة: {t:.1f} s | n_iter_ = {hgb.n_iter_}")
print(f"دقة الاختبار: {hgb.score(X_te, y_te):.4f}")
```

رافعة خمسة عشر ضعفاً في حجم البيانات (من ٢٠٠٠٠ إلى ٣٠٠٠٠٠) رفعت الزمن من نحو ثانية إلى بضع ثوانٍ فقط — النمو شبه الخطي في حجم البيانات داخل التكرار الواحد بفضل الصناديق. لو أردنا إجراء مقارنة عادلة مع `GradientBoosting` عند هذا الحجم لاستغرق التدريب دقائق طويلة؛ عشر الأشجار عنده تقارب مئة تكرار من HistGB زمنياً.

البارامتر في GradientBoosting	المقابل في HistGradientBoosting	الافتراض الجديد	ملاحظة
`n_estimators=100`	`max_iter=100`	نفس الافتراض	اسم أدق: تكرارات لا أشجار بالضرورة
`max_depth=3`	`max_leaf_nodes=31`	نمو ورقي	أعمق بناء أوسع بكثير من عمق ٣
`subsample=1.0`	(غير موجود)	—	التنظيم هنا عبر max_features و l٢
`n_iter_no_change`	`early_stopping='auto'`	تلقائي فوق ١٠٠٠٠ عينة	لا حاجة لضبط يدوي
`loss='log_loss'`	`loss='log_loss'`	متطابق	خيارات الانحدار متطابقة أيضاً

٦.٣ القيم المفقودة أصلاً والفئويات دون ترميز يدوي

هنا يتحول HistGB من «أسرع» إلى «أريح». أرسل مصفوفة فيها NaN كما هي — بلا `SimpleImputer` ولا `KNNImputer` ولا حذف صفوف:

```python
# أصلي مقابل الترميم اليدوي على 8% قيم مفقودة NaN
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier,
HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=4000, n_features=8, n_informative=4,
random_state=42)
rng = np.random.default_rng(42)
mask = rng.random(X.shape) < 0.08
X_nan = X.copy()
X_nan[mask] = np.nan
X_tr, X_te, y_tr, y_te = train_test_split(X_nan, y, test_size=0.3,
random_state=42, stratify=y)

try:
    GradientBoostingClassifier().fit(X_tr, y_tr)
except ValueError as err:
    print(f"GradientBoosting يرفض البيانات: {str(err)[:60]}...")

native = HistGradientBoostingClassifier(random_state=42).fit(X_tr, y_tr)
imp_h = make_pipeline(SimpleImputer(strategy="mean"),
HistGradientBoostingClassifier(random_state=42)).fit(X_tr, y_tr)
imp_g = make_pipeline(SimpleImputer(strategy="mean"),
GradientBoostingClassifier(random_state=42)).fit(X_tr, y_tr)
print(f"HistGB على NaN مباشرة:       {native.score(X_te, y_te):.4f}")
print(f"Imputer ثم HistGB:           {imp_h.score(X_te, y_te):.4f}")
print(f"Imputer ثم GradientBoosting: {imp_g.score(X_te, y_te):.4f}")
```

ثلاث حقائق في هذا الناتج: أولاً `GradientBoosting` يرمي `ValueError` أمام NaN — فكرة الصناديق لا تنطبق عليه. ثانياً HistGB أكل القيم المفقودة مباشرة وأنتج أفضل دقة (٠.٩٤٧٥ مقابل ٠.٩٤٥٨ للترميم بالمتوسط): بدل انتحال قيمة، يوجه النموذج الصندوق الاحتياطي الإضافي للقيم المفقودة ويقرر في كل انقسام أي الجهتين يضعها فيها — معلومة الـ«مفقود» نفسها قد تنبئ بالفئة! ثالثاً أضعف الأطراف هو الترميم ثم GradientBoosting (٠.٩١٧٥).

أما الفئويات فالقصة أجمل: في ١.٩ البارامتر `categorical_features` افتراضه `'from_dtype'` — أعمدة DataFrame من نوع Categorical أو Enum تُكتشف تلقائياً:

```python
عمود فئوي بنوعه الأصلي بلا ترميز يدوي
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(42)
n = 2000
city = rng.choice(["الدمام" ,"جدة" ,"الرياض"], size=n, p=[0.5, 0.3, 0.2])
x1 = rng.normal(size=n)
x2 = rng.normal(size=n)
base = 0.8 * x1 - 0.5 * x2
base += np.where(city == "1.2 ,"الرياض, np.where(city == "0.8- ,0.3 ,"جدة))
y = (base + 0.5 * rng.normal(size=n) > 0).astype(int)

df = pd.DataFrame({"x1": x1, "x2": x2, "city": pd.Categorical(city)})
X_tr, X_te, y_tr, y_te = train_test_split(df, y, test_size=0.3, random_state=42,
stratify=y)

hgb = HistGradientBoostingClassifier(random_state=42).fit(X_tr, y_tr)
print(f"دقة الاختبار: {hgb.score(X_te, y_te):.4f}")
print(f"الأعمدة الفئوية المكتشفة تلقائياً: {hgb.is_categorical_}")
```

لا `OneHotEncoder` ولا `OrdinalEncoder` — النموذج تعرف على `city` من نوع العمود وحده (شاهد `is_categorical_` الموضع الثالث True)، ويتعامل مع الفئات كفئات حقيقية ويبحث أفضل تقسيم بينها. شرطان وحيدان: لا تتجاوز الفئات الفريدة `max_bins`، والقيم تتحول داخلياً إلى أعداد عائمة.

> **خطأ شائع**
>
> لا تمرر أعمدة فئوية مرقّمة يدوياً (مثل ٠=الرياض، ١=جدة، ٢=الدمام) اعتماداً على أنها «أرقام»؛ سيعاملها النموذج قيماً متصلة قابلة للترتيب والجمع، فيبحث عن عتبات مثل «city أكبر من ١.٥» وهي عبارة بلا معنى. مررها بـ dtype الفئوي لتفعيل `'from_dtype'`، أو حدد مواضعها صراحة في `categorical_features` بقائمة أسماء أو أرقام أعمدة.

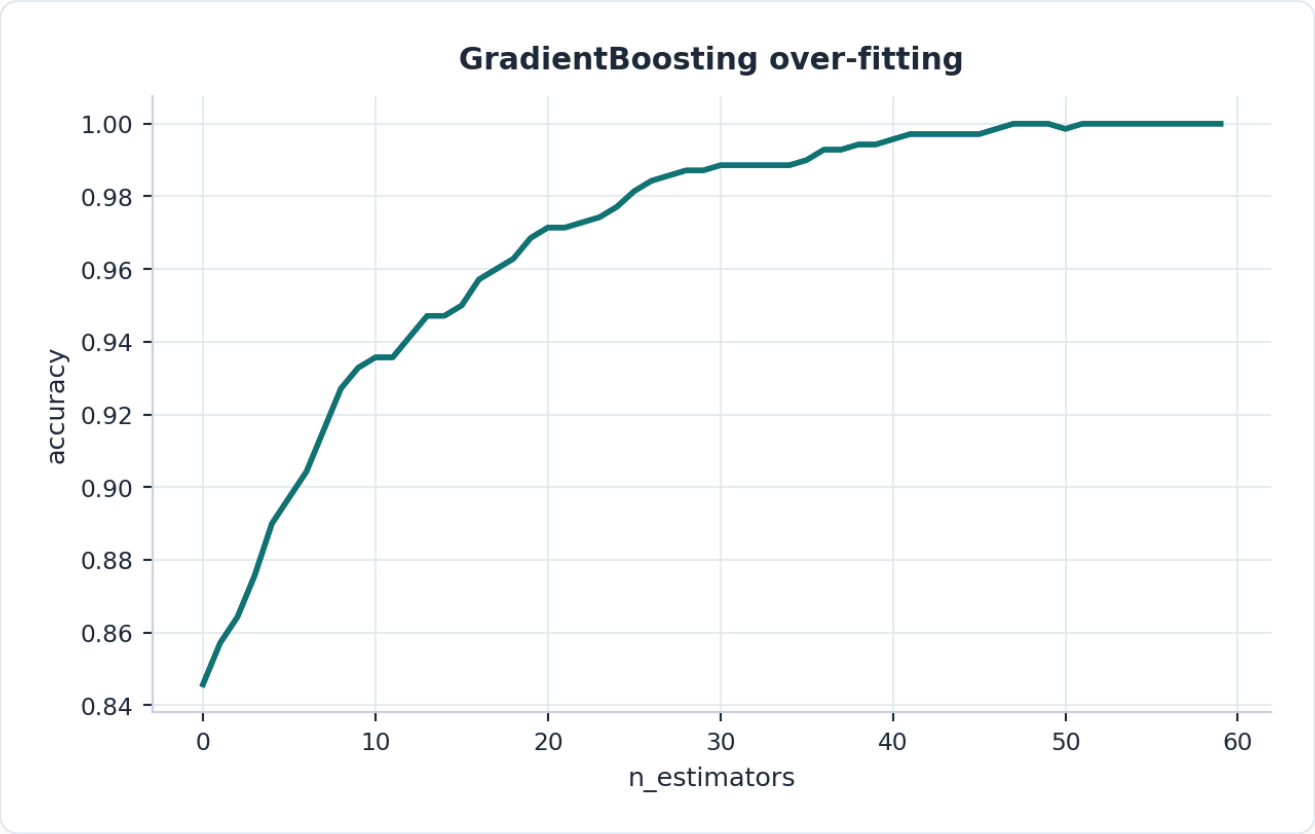
٤.٦ `early_stopping='auto'` والضبط: `max_bins` و `l2_regularization`

رأينا في الفصل ٤٦ أن GradientBoosting يفعّل التوقف المبكر بضبط يدوي لثلاثة `n_iter_no_change`. هنا الوضع أذكى: `early_stopping='auto'` يعني التوقف المبكر **مفعّلاً تلقائياً** كلما تجاوزت عينات التدريب ١٠٠٠٠، بفصل ١٠%

للتحقق (`validation_fraction`) ومراقبة `n_iter_no_change=10` و`tol=1e-7`. على بيانات أسهل من ذلك، اشدد على الطلب وشاهد التوقف:

```python
# التوقف المبكر: طلبنا 500 تكرار فتوقف عند 52
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split
import numpy as np

X, y = make_classification(n_samples=3000, n_features=8, n_informative=5,
class_sep=1.8, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

hgb = HistGradientBoostingClassifier(
    max_iter=500, early_stopping=True, validation_fraction=0.15,
    n_iter_no_change=15, random_state=42,
).fit(X_tr, y_tr)
print(f"طلبنا 500 تكرار فبنى فعلياً {hgb.n_iter_} فقط")
print(f"أفضل درجة تحقق: {np.max(np.asarray(hgb.validation_score_)):.4f}")
print(f"دقة الاختبار: {hgb.score(X_te, y_te):.4f}")
```

طلبنا خمسمئة تكرار فبنى ٥٢ فقط ثم قف — البيانات سهلة الإمساك، ومتابعة البناء بعد نقطة التشبع لا تضيف إلا ملاءمة للضجيج. الشكل التالي من الفصل ٤٦ يرسم بالضبط لماذا هذا القرار آمن: منحنى الاختبار يبلغ قمته ثم ينحدر في منطقة الإفراط، والتوقف المبكر يقف قبل الانحدار:



الشكل ١٠٢منحنى التدريب والاختبار مقابل عدد الأشجار: تتجاوز منطقة الإفراط في الملاءمة قمة التعميم من مدة

للضبط الدقيق، ثلاثة مقابض خاصة بهذه العائلة: `max_bins` يرفع دقة العتبات مقابل زمن أطول (٢٥٥ سقف لا يمكن تجاوزه — صندوق إضافي محجوز دائماً للقيم المفقودة، ولهذا السقف بالذات: الصناديق تسكن بايتاً واحداً)، و`l2_regularization` (مثل ٠.١ إلى ١٠) يقلّص أوزان الأوراق فيهدئ الإفراط على البيانات الضجيجية، و`max_features` (أقل من ١.٠) يُدخل عشوائية الميزات في كل انقسام: على بيانات عريضة ١٢٠ ميزة خفّضناها إلى ٠.٣ فوفّرنا نحو ٧% من الزمن مع هبوط طفيف في الدقة — مقبض تنظيم أكثر منه مقبض سرعة، وعلى البيانات الضيقة أثره شبه معدوم.

٦.٥ متى تختار HistGB حتماً؟ وفخ البيانات الصغيرة جداً

الأرقام التي قِسناها في هذا الفصل ترسم خطاً فاصلاً واضحاً، لكن القصة ليست انفرادية الاتجاه: على البيانات الصغيرة جداً يدفع النموذج الثقيل ثمن بنية التحسين نفسها. قِس الثلاثة على ٥٦٩ عينة بتحقق متقاطع خماسي:

فخ البيانات الصغيرة: HistGB أمام المنافسين على 569 عينة python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import (
    GradientBoostingClassifier,
    HistGradientBoostingClassifier,
    RandomForestClassifier,
)
from sklearn.model_selection import KFold, cross_val_score

X, y = load_breast_cancer(return_X_y=True)
cv = KFold(n_splits=5, shuffle=True, random_state=42)

for name, model in [
    ("HistGB", HistGradientBoostingClassifier(random_state=42)),
    ("GradientBoosting", GradientBoostingClassifier(random_state=42)),
    ("RandomForest", RandomForestClassifier(random_state=42, n_jobs=-1)),
]:
    s = cross_val_score(model, X, y, cv=cv).mean()
    print(f"{name:16s} | متوسط دقة 5-fold: {s:.4f}")
```

على ٥٦٩ عينة تفوّق `GradientBoosting` التقليدي (٠.٩٦٣١) على HistGB والغابة العشوائية (٠.٩٥٧٨ لكلتيهما) — فرق صغير لكنه منهجي: مع بيانات قليلة يكون تقدير العتبات الحقيقية ممكناً أصلاً، فلا حاجة للصناديق، ويوفر البحث الدقيق قليلاً من الدقة. الحكم العملي الذي تختصره تجارب هذا الفصل:

حجم البيانات (الصفوف)	الاختيار الأمثل	السبب
أقل من ~١٠٠٠	GradientBoosting أو RF	التقدير الكامل للعتبات مجدٍ والفرق يُقاس
١٠٠٠ إلى ~١٠٠٠٠	HistGB أو GradientBoosting	منطقة رمادية: جرّب الاثنين، الفارق الزمني ليس حاسماً
أكثر من ~١٠٠٠٠	HistGB حتماً	التوقف المبكر التلقائي + سرعة ترتيب أعلى
مئات الآلاف وما فوق	HistGB حتماً	البديل يستغرق دقائق؛ HistGB ثوانٍ، مع NaN وفئويات مجاناً

نصيحة

ابدأ بهذه التهيئة على أي جدول كبير: `HistGradientBoostingClassifier(max_iter=None)` لا — بل اترك `max_iter=100` و `early_stopping='auto'` كما هما، وأضف `learning_rate=0.1` الافتراضي، ثم اضبط `max_leaf_nodes` بين ١٥ و٦٣ و `l2_regularization` بين ٠.٠ و١.٠ عبر RandomizedSearchCV (الفصل ٣٣). لا تلمس `max_bins` إلا بعد استنفاد البقية.

في الواقع

على أرض الواقع: HistGradientBoosting هو اليوم «النموذج الأول الصحيح» لأي مسألة جدولية متوسطة أو كبيرة داخل scikit-learn — من كشف الاحتيال في ملايين المعاملات إلى تقدير المخاطر في مئات الأعمدة، حيث يُلغي ثلاث مشاكل هندسية دفعة واحدة: زمن التدريب، وترميم القيم المفقودة، وترميز الفئويات. وحين تحتاج أداء أعلى من ذلك بقليل، فأنت تنزل إلى LightGBM أو XGBoost خارج المكتبة — بنفس المفاهيم التي تعلمتها هنا حرفياً.

تمارين الفصل السابع والأربعين

- **ملاحظة**

التمرين ٦.١

أنشئ بيانات `make_classification` (n=٦٠٠٠، ١٠ ميزات) وحقن فيها نسب ضياع متدرجة: ٥% ثم ١٥% ثم ٣٠% من القيم (باستخدام مولّد عشوائي ثابت). في كل نسبة، قارن دقة الاختبار بين `HistGradientBoostingClassifier` على NaN مباشرة ومسار `SimpleImputer(strategy='mean')` يليه النموذج نفسه. عند أي نسبة يتضح تفوق النهج الأصلي؟

التمرين ٦.٢

اشرح لماذا حد سكّيت-ليرن `max_bins` بالقيمة ٢٥٥ تحديداً ولا يسمح بـ ٢٥٦، ولماذا لا يحتاج النموذج مع ذلك إلى `Imputer` أصلاً، وكيف يتعامل انقسام شجري واحد مع العينات المفقودة في ميزة الانقسام (أين تذهب: يسار أم يمين؟).

التمرين ٦.٣

على بيانات `make_classification` (n=٥٠٠٠، ١٥ ميزة، ضجيج معتدل عبر *class_sep*=٠.٨، *ابحث في شبكة صغيرة:* `max_leaf_nodes` ∈ {15, 31, 63} مع `l2_regularization` ∈ {٠.٠، ٠.١}، بتقييم على تقسيم اختبار ثابت، واطبع أفضل تركيبة ودقتها، وعلّق على أثر ١٢ على الفوارق.

- **ملاحظة**

python حل التمرين 6.1: التدمير الأصلي للـ NaN مقابل الترميم بمراحل

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=6000, n_features=10, n_informative=5,
random_state=42)
rng = np.random.default_rng(42)

for rate in [0.05, 0.15, 0.30]:
    X_nan = X.copy()
    X_nan[rng.random(X.shape) < rate] = np.nan
    X_tr, X_te, y_tr, y_te = train_test_split(
        X_nan, y, test_size=0.3, random_state=42, stratify=y
    )
    native = HistGradientBoostingClassifier(random_state=42).fit(X_tr, y_tr)
    imp = make_pipeline(
        SimpleImputer(strategy="mean"),
        HistGradientBoostingClassifier(random_state=42),
    ).fit(X_tr, y_tr)
    diff = native.score(X_te, y_te) - imp.score(X_te, y_te)
    print(f"ضياع {rate:.0%} | NaN مباشرة: {native.score(X_te, y_te):.4f}"
          f" | Imputer: {imp.score(X_te, y_te):.4f} | الفارق: {diff:+.4f}")
```

الفارق يصعد مع تصاعد الضياع: عند ٥% الفرق هامشي، وعند ١٥-٣٠% يظهر تفوق النهج الأصلي بوضوح — لأن ترميم ٣٠% من الخلايا بالمتوسط يعرض البيانات بأكملها، بينما يحفظ النموذج الأصلي «إشارة المفقود» ويستعملها في الانقسامات.

ملاحظة

حل التمرين 6.3: شبكة max_leaf_nodes مع l2_regularization python

```python
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=5000, n_features=15, n_informative=8,
                           class_sep=0.8, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

best = (0.0, None)
for leaves in [15, 31, 63]:
    for l2 in [0.0, 1.0]:
        hgb = HistGradientBoostingClassifier(
            max_leaf_nodes=leaves, l2_regularization=l2, random_state=42,
        ).fit(X_tr, y_tr)
        s = hgb.score(X_te, y_te)
        print(f"leaves={leaves:2d} | l2={l2} | دقة الاختبار: {s:.4f}")
        if s > best[0]:
            best = (s, (leaves, l2))
print(f"التركيبة الأفضل: {best[1]} بدقة {best[0]:.4f}")
```

النمط المتوقع: الأوراق الأكثر (٦٣) تكسب على التدريب لكن l2=١.٠ يهدئ إفراطها على الاختبار، والفوارق بين التركيبات صغيرة — وهيمنة l2 تزداد وضوحاً كلما زاد الضجيج أو قلّت العينات، تماماً كما في جدول القرار أعلاه حيث قلّصنا الأوراق بدل تعميقها.

> **ملاحظة**

٦.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تمرير أعمدة فئوية مرمّزة كأرقام عادية	يبحث النموذج عن عتبات بلا معنى مثل «city > ١.٥» ويخسر الإشارة	استخدم dtype الفئوي ليكتشفها `'from_dtype'` أو سمّها في `categorical_features`
ترميم NaN بالمتوسط قبل HistGB عادةً قديمة	تفقد «إشارة المفقود» وقد تنخفض الدقة: ٠.٩٤٥٨ مقابل ٠.٩٤٧٥	أرسل NaN كما هي؛ النموذج يدير صندوقاً احتياطياً للمفقود أصلاً
توقع `subsample` في HistGB كما في GradientBoosting	البارامتر غير موجود — البناء ورقي مختلف	استخدم `max_features < 1.0` أو `l2_regularization` للتنظيم
تطبيق HistGB الافتراضي على مئات العينات الصغيرة وتوقع تفوقاً	خسارة منهجية صغيرة: ٠.٩٥٧٨ مقابل ٠.٩٦٣١ لـ GradientBoosting	فوق ~١٠٠٠٠ صف HistGB هو الخيار؛ تحتها قارن مع التقليدي أولاً

٦.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التدرّج في صناديق	حصر قيم كل ميزة في ≤٢٥٥ صندوقاً والعمل على أرقام الصناديق بدل القيم الحقيقية	Histogram binning
مدرّج التدرّجات	مصفوفة عدّادات لكل صندوق تجمع مجموع التدرّجات ومربعاتها لتسريع حساب المكاسب	Gradient histogram
النمو ورقياً	توسيع الشجرة عبر توسيع الورقة الأكثر ربحاً لا بمستويات متساوية العمق	Leaf-wise growth
الصندوق الاحتياطي للمفقود	صندوق إضافي واحد محجوز للقيم المفقودة خارج الـ٢٥٥، ويقرر النموذج اتجاهه في كل انقسام	Missing-values bin
التوقف المبكر التلقائي	سلوك `early_stopping='auto'`: تفعيل المراقبة على شريحة تحقق فوق ١٠٠٠٠ عينة	Automatic early stopping

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٨ — التصويت والتكديس

StackingRegressor و VotingRegressor و StackingClassifier و VotingClassifier

الوحدة **scikit-learn ١.٩** الحجم **٩٨٧** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تميّز بين الدمج المستوٍ (Voting) والدمج المُتعلَّم (Stacking) ومتى يتفوق كل منهما
- أن تختار بين التصويت الصلب hard والناعم soft وتضبط أوزان النماذج بـ weights
- أن تشرح كيف يمنع cv الداخلي في StackingClassifier تسرب بيانات التدريب إلى النموذج العليا
- أن تضبط stack_method و passthrough و final_estimator وتثبّت النموذج النهائي بـ FrozenEstimator
- أن تكشف فخ تكديس نماذج متشابهة وتصمم مجموعة متنوعة فعلاً

٦.١ دمج مستوٍ أم دمج مُتعلَّم؟

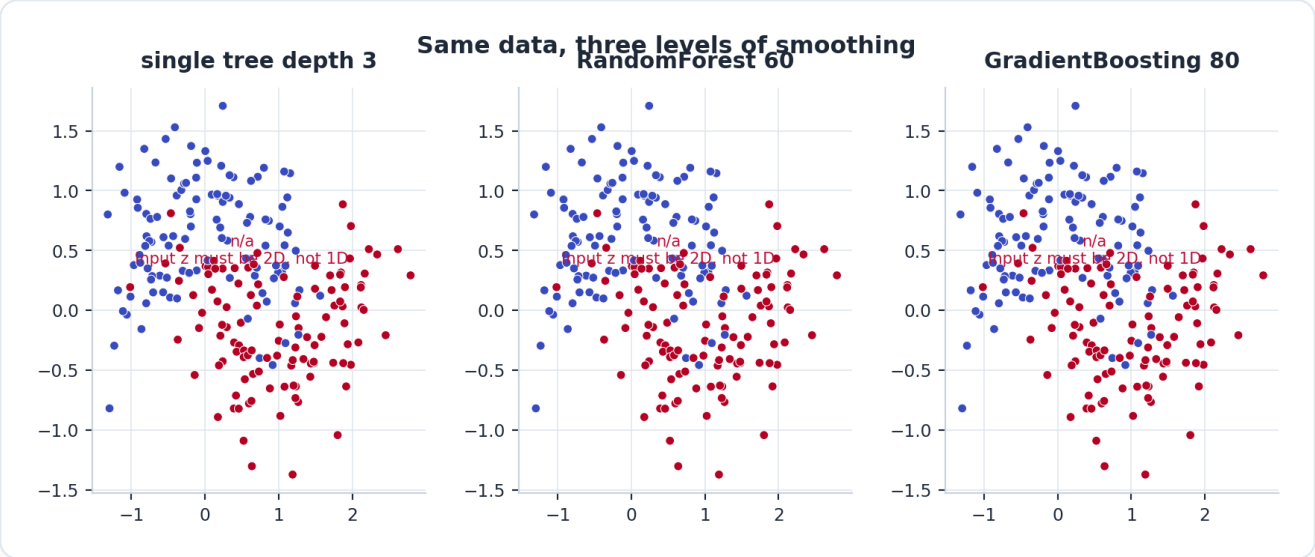
في الفصول السابقة جمّعنا نسخاً من **نفس النموذج**: مئة شجرة تتشابه في البنية وتختلف في العينة أو الميزات. لكن تخيّل مسألة مختلفة: لديك مصنف لوجيستي يرسم خطوطاً مستقيمة، وشجرة قرار تقطع الفضاء مربعات، ومصنف جيران يرسم بلا شكل مسبق. كل واحد يخطئ *بطريقة مختلفة جذرياً*، وهذا بالضبط ما يحتاجه التجميع الناجح: متخالفون متكافئون (Diverse and Comparable).

نقصد بـ **الدمج المستوٍ (Flat Ensemble)** أن نجمع تنبؤات النماذج بعملية ثابتة لا تتعلم: تصويت بالأغلبية أو متوسط أرقام. أما **الدمج المُتعلَّم (Meta-learning)** في التكديس فهو خطوة أبعد: نعتبر تنبؤات النماذج السفلى *ميزات جديدة* ندرب عليها نموذجاً علياً (Meta-estimator) يتعلم متى يصدّق كل نموذج سفلي وكم وزنه.

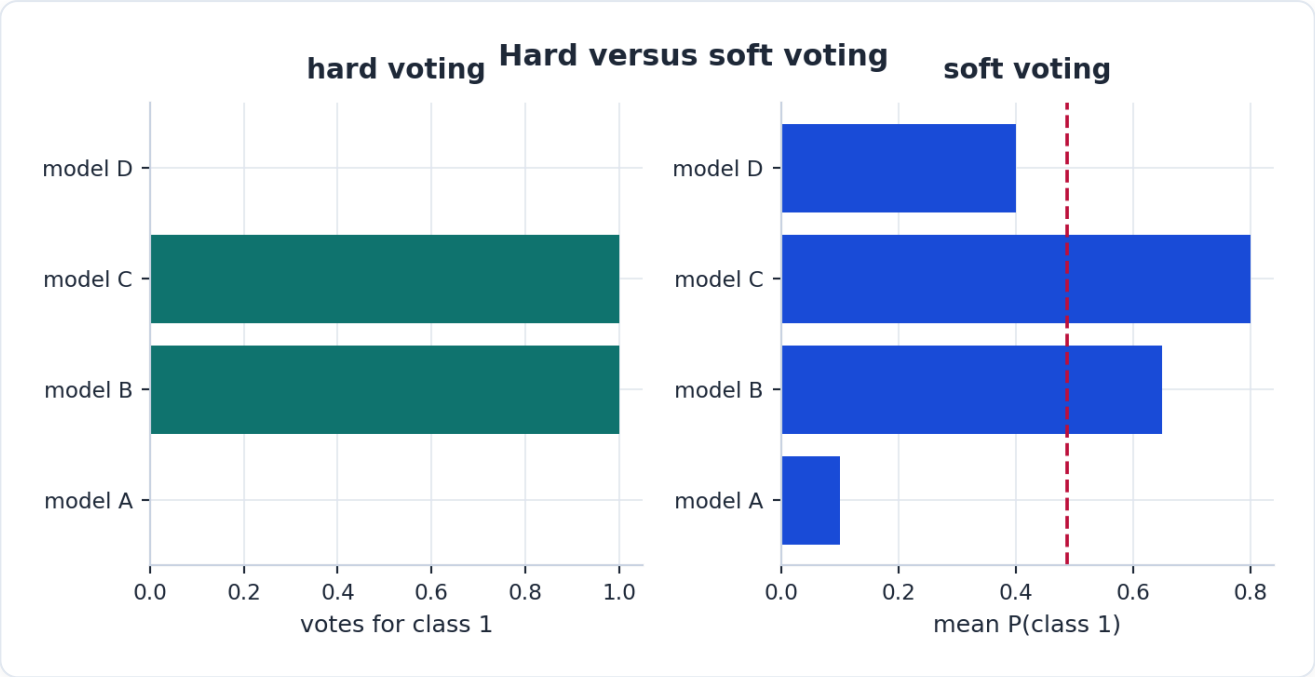
المفهوم المحوري

التصويت يجيب عن سؤال ثابت: «ما رأي الأغلبية؟». التكديس يجيب عن سؤال مُتعلَّم: «في هذه المنطقة من الفضاء، ومن لهذه الميزات، من يُصدَّق أكثر؟». لهذا يتفوق التكديس عندما تكون أخطاء النماذج متباينة البنية، ويكفي التصويت عندما تكون النماذج متكافئة الدقة وأخطاؤها عشوائية المستوى.

ولأن جمال الدمج كله يعتمد على اختلاف «جغرافيا» أخطاء الأعضاء، راقب في الشكل التالي كيف تقطع كل عائلة تجميعية الفضاء بخطوط حادة مختلفة؛ كل خط رفيع هو منطقة يخطئ فيها هذا النموذج وحده — وهي بالضبط ما سيغطيه جاره:



الشكل ١٠٣خرائط حدود القرار للنماذج التجميعية: كل عائلة تقطع الفضاء بشكل مختلف عن جارتها



الشكل ١٠٤مخطط يوضح مسار التنبؤ في التصويت والتكديس: النماذج السفلية ثم آلية الدمج

نقرأ في الشكل كيف تتفرع البيانات إلى النماذج السفلية ثم تجتمع في بوابة دمج واحدة: في التصويت البوابة عملية حسابية جاهزة، وفي التكديس هي مقدر مدرَّب عليه تنبؤات النماذج لا البيانات الخام.

٦.٢ VotingClassifier: التصويت الصلب مقابل الناعم

يقدم `sklearn.ensemble.VotingClassifier` نمطي تصويت:

$$\hat{y}_{\text{hard}}(x) = \text{mode}\left\{h_1(x),\, h_2(x),\, \ldots,\, h_B(x)\right\}$$

حيث h_b هو التنبؤ الفئوي للمقدر رقم b. أما التصويت الناعم فيحسب متوسط احتمالات كل فئة عبر المقدرات:

$$\hat{y}_{\text{soft}}(x) = \arg\max_{c} \frac{1}{B} \sum_{b=1}^{B} p_{b,c}(x)$$

حيث $p_{b,c}$ هو الاحتمال الذي أسندته الفئة c من قبل المقدر b، و B عدد المقدرات. لماذا يتفوق soft عادةً؟ لأن الصوت الصلب يفقد المعلومة الكمية: مصنف واثق ٠.٩٩ ومصنف متردد ٠.٥١ صوّتا بنفس «الوزن» في hard، بينما يحفظ soft هذا الفرق. الفارق يظهر خصوصاً مع الاحتمالات المعايرة جيداً، ومع تساوي الأصوات في hard حيث يلجأ المقدر إلى كسر التعادل بترتيب الفئات داخلياً — ظرف تتجنبه الأرقام الصريحة في soft.

لاحظ شرطاً جوهرياً: soft يتطلب أن يوفر كل مقدر `predict_proba`. الشجرة واللوجيستي والجيران يوفرونه، لكن `LinearSVC` مثلاً لا يملك احتمالات أصلاً (يملك `decision_function`)، فيُقبل في التصويت الصلب ويرفض في الناعم.

نجرّب الآن على مسألة اصطناعية صعبة (فئات متشابكة وضجيج في التسميات) ثلاثة نماذج من عائلات مختلفة، ثم نقارن التصويتين:

ثلاثة نماذج متباينة ثم التصويت الصلب والناعم python

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=1500, n_features=15, n_informative=4,
                           n_redundant=2, n_clusters_per_class=3, flip_y=0.08,
                           weights=[0.85, 0.15], random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

lr   = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
tree = DecisionTreeClassifier(max_depth=8, random_state=42)
knn  = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=25))
estimators = [("lr", lr), ("tree", tree), ("knn", knn)]

for name, m in estimators:
    m.fit(X_tr, y_tr)
    print(f"النموذج المفرد {name:5s} | دقة الاختبار: {m.score(X_te, y_te):.4f}")

hard = VotingClassifier(estimators=estimators, voting="hard",
n_jobs=-1).fit(X_tr, y_tr)
soft = VotingClassifier(estimators=estimators, voting="soft",
n_jobs=-1).fit(X_tr, y_tr)
print(f"تصويت صلب hard | دقة الاختبار: {hard.score(X_te, y_te):.4f}")
print(f"تصويت ناعم soft | دقة الاختبار: {soft.score(X_te, y_te):.4f}")
```

النماذج الثلاثة متقاربة الدقة (بين ٠.٨٣٥٦ و ٠.٨٤٨٩)، والتصويت الناعم حقق ٠.٨٦٢٢ متفوقاً على أفضل فرد وعلى hard الذي بلغ ٠.٨٥٣٣. الفائدة هنا لا تأتي من «نموذج أذكى» بل من تفكك الأخطاء: الأخطاء التي يهتدي فيها اللوجيستي يعوضها الجيران.

٦.٣ بارامتر weights: من يُصغى له أكثر؟

الافتراضي أن الأصوات متساوية الوزن، لكن ماذا لو كان أحد النماذج أنذر من زملائه باستمرار؟ البارامتر `weights` يقبل قائمة أوزان (عددها بعدد المقدرات) تُضرب بها الاحتمالات في soft (وتكرار الأصوات في hard):

```python
# أثر أوزان التصويت على الدقة
from sklearn.ensemble import VotingClassifier

for w in [(1, 1, 1), (2, 1, 1), (1, 2, 1), (1, 1, 2)]:
    vc = VotingClassifier(estimators=estimators, voting="soft",
                          weights=list(w), n_jobs=-1).fit(X_tr, y_tr)
    print(f"أوزان (lr, tree, knn) = {w} | دقة الاختبار: {vc.score(X_te, y_te):.4f}")
```

رفع وزن النموذج الأقوى (اللوجيستي هنا) إلى ٢ دفعت الدقة إلى ٠.٨٦٤٤، بينما مضاعفة وزن الأضعف خفضتها تحت مستوى soft المتساوي الأوزان. القاعدة: الأوزان وسيلة ضبط دقيقة لا تعوض نماذج رديئة، ويُبحث عنها الأمثل في `GridSearchCV` كما في الفصل ٨١.

شرط soft المنسي

تدريب `VotingClassifier(voting="soft")` على مقدر لا يملك `predict_proba` قد ينجح **صامتاً**، لتنفجر `AttributeError` عند أول نداء `predict`! الخلل لا يظهر في وقت التدريب بل في وقت التنبؤ — أخطر لحظة ممكنة لاكتشافه. راقب الاستثناء التالي بنفسك:

```python
# التصويت الناعم مع مقدر بلا احتمالات
from sklearn.svm import LinearSVC
from sklearn.ensemble import VotingClassifier

svc = make_pipeline(StandardScaler(), LinearSVC(max_iter=5000, dual="auto",
random_state=42))
bad = VotingClassifier(estimators=[("svc", svc), ("tree", tree)],
voting="soft")
bad.fit(X_tr, y_tr)  # التدريب يمر دون شكوى!
try:
    bad.predict(X_te)
except AttributeError as err:
    print(f"الانفجار يؤجل نفسه إلى predict: {type(err).__name__}")
    print(f"الرسالة: {err}")
```

إذا أردت إدخال نموذج هوامش مثل `LinearSVC` في مجموعة ناعمة، فغلّفه أولاً بمعاير الاحتمالات `sklearn.calibration.CalibratedClassifierCV` الذي يضيف طبقة تحويل درجات الهامش إلى احتمالات، أو اكتفِ بـ `decision_function` — وهو ما سيعود إلينا في التكديس بعد قليل.

٦.٤ VotingRegressor: التصويت بالمتوسط

في الانحدار لا توجد فئات نصوّت لها؛ نستبدل الأغلبية بـ **متوسط القيم المتنبأ بها**. يقدم `sklearn.ensemble.VotingRegressor` ذلك مباشرة، ويتفوق عندما تكون النماذج متقاربة الدقة ومتباينة الانحياز. نقيس على عينة من بيانات California Housing:

```python
# VotingRegressor على بيانات إسكان كاليفورنيا
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import VotingRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import root_mean_squared_error

data = fetch_california_housing()
Xr, yr = data.data[:3000], data.target[:3000]
Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(Xr, yr, test_size=0.3,
random_state=42)

ridge = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
dtree = DecisionTreeRegressor(max_depth=8, min_samples_leaf=10, random_state=42)
rknn  = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=15))
reg_estimators = [("ridge", ridge), ("tree", dtree), ("knn", rknn)]

for name, m in reg_estimators:
    m.fit(Xr_tr, yr_tr)
    print(f"النموذج المفرد {name:6s} | RMSE: {root_mean_squared_error(yr_te,
m.predict(Xr_te)):.4f}")

vreg = VotingRegressor(estimators=reg_estimators, n_jobs=-1).fit(Xr_tr, yr_tr)
print(f"VotingRegressor (متوسط التنبؤات) | RMSE: {root_mean_squared_error(yr_te,
vreg.predict(Xr_te)):.4f}")
```

الانحدار الخطي المكبل وحده إضعف الفريق (RMSE ٠.٤٩٠٢)، والشجرة تحسّن الموقف إلى ٠.٤١٤٠، والجيران إلى ٠.٣٧٠٤ — ومتوسط التنبؤات الثلاثة بلغ ٠.٣٦٩٢. لاحظ أن المتوسط تجاوز أفضل الأفراد؛ هذا هو حصاد تباين الأخطاء.

٦.٥ StackingClassifier: البواقي المتحقق منها

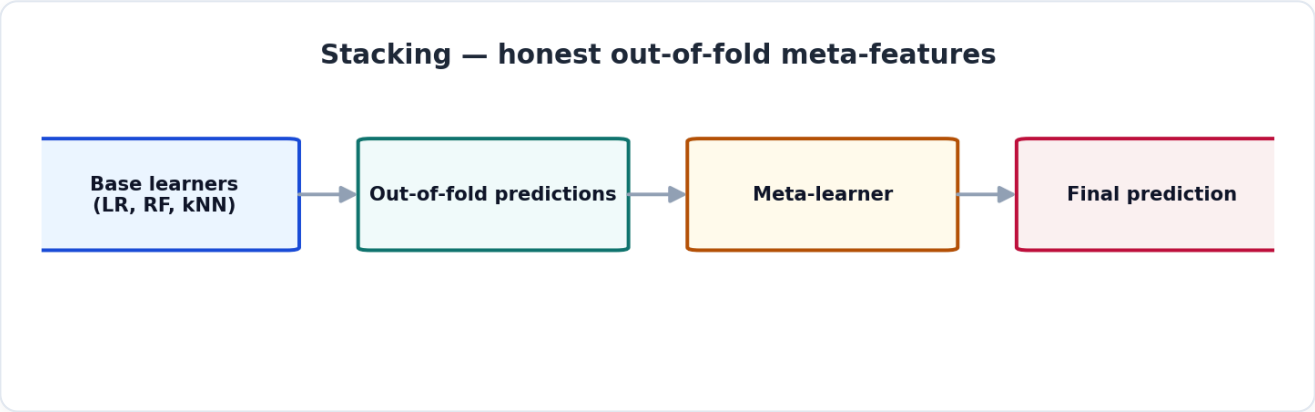
التصويت يعامل كل النماذج بعدالة شكلية: صوت واحد لكل نموذج (أو وزن ثابت). لكن ماذا لو تعلّم النظام بنفسه متى يثق بالشجرة ومتى يثق بالجيران؟ هنا `sklearn.ensemble.StackingClassifier`:

$$\tilde{p}_{b,c}(x_i) = f_b^{(-k(i))}(x_i)$$

حيث $f_b^{(-k(i))}$ هو المقدر b مدرَّباً على كل الأجزاء ما عدا الجزء k الذي تنتمي إليه العينة i. ثم يتعلم المقدر النهائي g من هذه الميزات:

$$\hat{y}(x) = g(\tilde{p}_{1,1}, \ldots, \tilde{p}_{1,C}, \ldots, \tilde{p}_{B,C})$$

لماذا هذا التعقيد كله؟ **لمنع التسرب (Leakage)**. لو دربنا المقدرات السفلية على كل بيانات التدريب ثم تنبأنا بالبيانات نفسها لتغذية المقدر النهائي، لأعطت النماذج المتغذرة على حفظها تنبؤات مبهرة كاذبة — فسيتعلم المقدر النهائي ثقة مضخمة ستنهار عند الاختبار. التنبؤات الخارجة من الطية صادقة لأن كل عينة نُبئت بها من نموذج لم يرها قط، فترى المقدر النهائي صورة نزيهة لسلوك النماذج على بيانات جديدة. البارامتر `cv` يحدد عدد الأجزاء (٥ افتراضياً).



الشكل ١٠٥مخطط آلية التكديس: تنبؤات التحقق المتقاطع تغذي المقدر النهائي

في الشكل يظهر الفرق الحاسم عن التصويت: المصفوفة الوسطى من تنبؤات الطيات ليست مخرجاً نهائياً بل مدخل لمستوى تعلّم جديد.

```python
# StackingClassifier مقابل الأفراد والتصويت
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

Xs, ys = make_classification(n_samples=1000, n_features=16, n_informative=6,
random_state=42)
Xs_tr, Xs_te, ys_tr, ys_te = train_test_split(Xs, ys, test_size=0.3,
                                              random_state=42, stratify=ys)

logit = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
stree = DecisionTreeClassifier(max_depth=8, random_state=42)
sknn  = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=11))
stack_estimators = [("logit", logit), ("tree", stree), ("knn", sknn)]

for name, m in stack_estimators:
    m.fit(Xs_tr, ys_tr)
    print(f"النموذج المفرد {name:6s} | دقة الاختبار: {m.score(Xs_te, ys_te):.4f}")

stack = StackingClassifier(
    estimators=stack_estimators,
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5, n_jobs=-1
).fit(Xs_tr, ys_tr)
print(f"StackingClassifier (cv=5)   | دقة الاختبار: {stack.score(Xs_te, ys_te):.4f}")
```

المقدر النهائي عرف أن يوازن الأصوات بنفسه وبلغ ٠.٨٧٦٧ فوق أفضل فرد (٠.٨٦٦٧). لاحظ التكلفة: التدريب هنا لم يكن ٣ نماذج بل ٣ × ٥ طيات + ٣ تدريبات كاملة + المقدر النهائي = ١٩ تدريباً داخلياً.

٦.٦ stack_method و passthrough: ضبط ما يراه المقدر النهائي

بارامتر `stack_method` يحدد نوع المخرج الذي يقدمه كل مقدر سفلي للمستوى العليا:

القيمة	ماذا يمرر للمقدر النهائي	متى تستخدمها
`"auto"`	يختار تلقائياً: `predict_proba` إن توفرت، وإلا `decision_function`، وإلا `predict`	الافتراضي الحكيم
`"predict_proba"`	متجه الاحتمالات لكل فئة	النماذج المعايرة، التصنيف
`"decision_function"`	درجة الهامش الخام	نماذج الهوامش مثل `LinearSVC`
`"predict"`	التسمية المتنبأ بها	الانحدار أو النماذج الفقيرة بالمخرجات

أما `passthrough=True` فيضيف الميزات الأصلية نفسها إلى ميزات الميتا، فيرى المقدر النهائي التنبؤات *والبيانات الخام* معاً — مفيد عندما تحمل الميزات الأصلية معلومة تفقد النماذج السفلية، لكنه يوسع مصفوفة الميتا:

passthrough مع نماذج هوامش و stack_method python

```python
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import StackingClassifier

svc = make_pipeline(StandardScaler(), LinearSVC(max_iter=5000, dual="auto",
random_state=42))

# auto يلتقط decision_function تلقائياً من LinearSVC لأن predict_proba غير موجودة
stack_auto = StackingClassifier(
    estimators=[("svc", svc), ("tree", stree)],
    final_estimator=LogisticRegression(max_iter=2000),
    stack_method="auto", cv=5, n_jobs=-1
).fit(Xs_tr, ys_tr)
print(f"تكديس (SVC + شجرة) بـ stack_method='auto' | دقة الاختبار:
{stack_auto.score(Xs_te, ys_te):.4f}")

# passthrough: 16 عموداً للمقدر النهائي = 17 = ميزة أصلية + عمود تنبؤ واحد
stack_pass = StackingClassifier(
    estimators=[("tree", stree)],
    final_estimator=LogisticRegression(max_iter=2000),
    passthrough=True, cv=5, n_jobs=-1
).fit(Xs_tr, ys_tr)
print(f"أبعاد ميزات المقدر النهائي مع passthrough:
{stack_pass.final_estimator_.coef_.shape}")
```

القيمة `"auto"` أنقذت الموقف تلقائياً مع `LinearSVC` — ولو فرضنا `predict_proba` على نافذة هوامش لحصلنا على الانفجار الذي رأيناه في التصويت الناعم.

جديد في ١.٩ — تثبيت المقدر النهائي بـ FrozenEstimator

أدخل الإصدار ١.٩ الصنف `sklearn.frozen.FrozenEstimator` الذي يلف مقدراً مدرَّباً فيتجاهل أي `fit` لاحق تماماً. الفائدة الكبرى في التكديس: دربتَ المقدر النهائي وضبطته بعناية، فتلفه بـ `FrozenEstimator` فلا يعيد أي مصنف علوي (Pipeline أو GridSearchCV أو تكديس آخر) تدريبه عند الاستنساخ (clone):

تجميد المجموعة المدربة بـ FrozenEstimator — python

```python
from sklearn.frozen import FrozenEstimator
import numpy as np

pred_before = stack.predict(Xs_te)
frozen_stack = FrozenEstimator(stack)          # المجموعة كاملة مجمدة الآن
frozen_stack.fit(Xs_tr, ys_tr)                 # لا تقلق: هذا لا شيء فعلياً
pred_after = frozen_stack.predict(Xs_te)
print(f"الوهمي fit التطابق قبل وبعد: {np.array_equal(pred_before, pred_after)}")
print(f"الدقة تبقى: {frozen_stack.score(Xs_te, ys_te):.4f}")
print(f"الكائن المجمد يلف المجموعة نفسها دون نسخ: {frozen_stack.estimator is stack}")
```

سلوك «الخيال» هذا مفيد فعلاً: ضع `FrozenEstimator(stack)` كخطوة داخل Pipeline وستنجو من إعادة تدريب باهظة في كل تجزئة من التحقق المتقاطع الخارجي.

٦.٧ StackingRegressor: نفس الفكرة على الأرقام

يقدم `sklearn.ensemble.StackingRegressor` ما يقابل ذلك في الانحدار: تمرر تنبؤات المنحدرات السفلية إلى منحدر عليا يتعلم دمجها. نقيس على عينتنا من California Housing:

يتعلم دمج المنحدرين StackingRegressor — python

```python
from sklearn.ensemble import StackingRegressor

sreg = StackingRegressor(
    estimators=reg_estimators,
    final_estimator=Ridge(alpha=1.0),
    cv=5, n_jobs=-1
).fit(Xr_tr, yr_tr)
print(f"VotingRegressor  | RMSE: {root_mean_squared_error(yr_te,
vreg.predict(Xr_te)):.4f}")
print(f"StackingRegressor| RMSE: {root_mean_squared_error(yr_te,
sreg.predict(Xr_te)):.4f}")
```

التكديس بلغ RMSE ٠.٣٥٧٨ متفوقاً على التصويت (٠.٣٦٩٢) وكل الأفراد. الدرس: عندما تكون تنبؤات النماذج ملتصقة بمجالات قيم مختلفة، يتعلم المقدر النهائي تصحيح الانحيازات النظامية — وهو أمر عاجز عنه المتوسط الثابت.

٦.٨ فخ التشابه: تكديس أشجار على أشجار

أخطر سوء استخدام للتكديس هو تجميع نماذج *متشابهة البنية*. ذكّر نفسك بمعادلة الفصل ٤٣: الارتباط ρ بين أخطاء النماذج يحدد سقف الفائدة؛ نماذجان من العائلة نفسها يخطئان في المناطق نفسها تقريباً، فترى المقدر النهائي عمودين متطابقين لا يضيفان معلومة. نثبت ذلك بأرقام:

```python
تكديس غابتين متشابهتين مقابل تكديس متنوع
from sklearn.ensemble import RandomForestClassifier, StackingClassifier

rf1 = RandomForestClassifier(n_estimators=50, random_state=1, n_jobs=-1)
rf2 = RandomForestClassifier(n_estimators=50, random_state=2, n_jobs=-1)

stack_trees = StackingClassifier(
    estimators=[("rf1", rf1), ("rf2", rf2)],
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5, n_jobs=-1
).fit(Xs_tr, ys_tr)

stack_mixed = StackingClassifier(
    estimators=stack_estimators,
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5, n_jobs=-1
).fit(Xs_tr, ys_tr)

print(f"دقة الاختبار | تكديس غابتين متشابهتين: {stack_trees.score(Xs_te, ys_te):.4f}")
print(f"دقة الاختبار |      تكديس ثلاثي متنوع: {stack_mixed.score(Xs_te, ys_te):.4f}")
print(f"أفضل فرد (knn)          | دقة الاختبار: {sknn.score(Xs_te, ys_te):.4f}")
```

تكديس غابتين (٠.٨٦٦٧) لم يضف شيئاً فوق أفضل فرد (٠.٨٦٦٧) — مئة شجرة تدريب إضافية وعشرة أضعاف الزمن لصالح لا يُقاس. أما المزيج المتنوع من ثلاث عائلات ففتح فجوة حقيقية (٠.٨٧٦٧). القاعدة الذهبية للتكديس: **التنوع البنيوي قبل كل شيء**، وإلا فالتصويت البسيط أو غابة واحدة أرخص وأنظف.

في الميدان

في مسابقات Kaggle وجد التكديس متعدد الطبقات موطنه الطبيعي: الفارق بين المراكز الأولى يُحسم بجزء من الألف، وكل نقطة أساس تستحق ساعات حساب. أما في الأنظمة الإنتاجية فالتكديس يُدفع ثمنه باهظاً: كل طلب تنبؤ يمر عبر كل النماذج السفلية ثم العليا. إن كان فرق الدقة أقل من نصف نقطة مئوية، فالغابة المضبوطة غالباً هي القرار الصناعي الأذكى.

تمارين الفصل الثامن والأربعين

- **ملاحظة**

التمرين ٦.١

على بيانات `load_wine`، ابنِ `VotingClassifier` من ثلاثة نماذج (انحدار لوجيستي، شجرة، KNN — كلها داخل Pipeline مع `StandardScaler`)، وقارن دقة كل فرد مع التصويت الصلب والناعم، ثم جرب ثلاث شبكات أوزان في soft وحدد الأفضل.

التمرين ٦.٢

اشرح بجملة رياضية واحدة لماذا ينهار المقدر النهائي في التكديس إذا مررت له تنبؤات مولدة من نماذج دربت على نفس بيانات التدريب (بدون تحقق متقاطع داخلي)، وابنِ `StackingClassifier` بـ `cv=None` — وهو ما يفعله هذا بالضبط — وقارن دقته مع `cv=5` على بيانات التمرين السابق.

التمرين ٦.٣

على `fetch_california_housing`، قارن `StackingRegressor` بـ `passthrough=True` ضد `passthrough=False` مع نفس النماذج الثلاثة من قسم ٦.٤، واذكر أيهما فاز، وفسّر لماذا لا تضيف الميزات الخام فائدة دائماً رغم كونها «معلومة إضافية».

- **ملاحظة**

حل التمرين ٦.١: التصويت على بيانات النبيذ python

```python
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

Xw, yw = load_wine(return_X_y=True)
Xw_tr, Xw_te, yw_tr, yw_te = train_test_split(Xw, yw, test_size=0.3,
                                              random_state=42, stratify=yw)

wine_estimators = [
    ("lr", make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))),
    ("tree", make_pipeline(StandardScaler(), DecisionTreeClassifier(max_depth=5,
random_state=42))),
    ("knn", make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=9))),
]
for name, m in wine_estimators:
    m.fit(Xw_tr, yw_tr)
    print(f"الفرد {name:5s} | الدقة: {m.score(Xw_te, yw_te):.4f}")

for voting in ["hard", "soft"]:
    vc = VotingClassifier(estimators=wine_estimators, voting=voting,
n_jobs=-1).fit(Xw_tr, yw_tr)
    print(f"تصويت {voting:4s} | الدقة: {vc.score(Xw_te, yw_te):.4f}")

best = (None, -1.0)
for w in [(1, 1, 1), (2, 1, 1), (1, 1, 2)]:
    vc = VotingClassifier(estimators=wine_estimators, voting="soft",
                          weights=list(w), n_jobs=-1).fit(Xw_tr, yw_tr)
    acc = vc.score(Xw_te, yw_te)
    print(f"soft بأوزان {w} | الدقة: {acc:.4f}")
    if acc > best[1]:
        best = (w, acc)
print(f"أفضل شبكة أوزان: {best[0]} بدقة {best[1]:.4f}")
```

ملاحظة

python حل التمرين ٦.٣: أثر passthrough في الانحدار

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error

data = fetch_california_housing()
Xr, yr = data.data[:3000], data.target[:3000]
Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(Xr, yr, test_size=0.3,
random_state=42)

base = [
    ("ridge", make_pipeline(StandardScaler(), Ridge(alpha=1.0))),
    ("tree", DecisionTreeRegressor(max_depth=8, min_samples_leaf=10,
random_state=42)),
    ("knn", make_pipeline(StandardScaler(),
KNeighborsRegressor(n_neighbors=15))),
]
for pass_flag in [False, True]:
    sr = StackingRegressor(estimators=base, final_estimator=Ridge(alpha=1.0),
                           passthrough=pass_flag, cv=5, n_jobs=-1).fit(Xr_tr,
yr_tr)
    rmse = root_mean_squared_error(yr_te, sr.predict(Xr_te))
    print(f"passthrough={pass_flag} | RMSE: {rmse:.4f}")
```

الفائز هنا هو `passthrough=False` (٠.٣٥٧٨ مقابل ٠.٣٥٨٩): تنبؤات الطيات الثلاثة لحالتنا مركّزة فعلًا، فأضافت الميزات الخام الثماني للمنحدر العليا أعمدة بمقاييس مختلفة خفّفت تأثير أعمدة الميتا بدل أن تضيف إشارة. الدرس: `passthrough` أداة تجريبية تُقاس بالنواتج، لا خياراً يُفترض فضله.

> **ملاحظة**

٦.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرم
`voting="soft"` مع `LinearSVC`	التدريب يمر بصمت ثم `AttributeError: This 'Pipeline' has no attribute 'predict_proba'` عند أول تنبؤ	استخدم `'` `unction"` أو عاير النم
تكديس نماذج من العائلة نفسها	تنبؤات الميتا شبه متطابقة فلا مكسب يذكر فوق أفضل فرد، وضياع أضعاف زمن التدريب	امزج عائلا جوار
توقع أن تكديس ٣ نماذج يدرّب ٣ نماذج	`cv=5` يعني ٥×٣ تدريبات للطيات + ٣ تدريبات كاملة + المقدر النهائي	راقب التكل `timators`
`fit` **على** `FrozenEstimator` **ظنّاً أن التدريب سيتجدد**	لا يحدث شيء: المقدر يبقى على أوزانه القديمة	أعد بناء الـ تجميده، أو

٦.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التصويت التجميعي	دمج قرارات المقدرات بالأغلبية الصلبة أو متوسط الاحتمالات	Voting
التصويت الناعم	متوسط متجهات الاحتمالات بدل عدّ الأصوات الجامدة	Soft voting
التكديس	تدريب مقدر عليا على تنبؤات مقدرات سفلية	Stacking (Stacked generalization)
ميزات الميتا	مصفوفة تنبؤات خارج الطية تُغذى للمقدر النهائي	Meta-features (Out-of-fold predictions)
التسرب	وصول معلومة من بيانات التدريب إلى التقييم بطريقة متحيزة	Leakage

الجزء ٦ · المجموعات والأغلفة

الفصل ٤٩ — ضبط نموذج لمجموعة كاملة

استراتيجية بحث صحيحة عبر Bagging و Boosting

الوحدة **scikit-learn ١.٩** الحجم **٨٠٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تحدد البارامترات الفاصلة في كل عائلة: Bagging و Boosting و HistGradientBoosting
- أن تطبق استراتيجية lr كبير أولاً ثم التضييق مع early_stopping
- أن تختار عدد مراحل التعزيز من staged_predict بلا إعادة تدريب
- أن تستخدم warm_start و oob_score لرسم منحنى الأشجار المتزايدة
- أن تحسب تكلفة GridSearchCV مقدماً وتوظف واجهة الاستدعاءات callbacks الجديدة في ١.٩

٦.١ أي إبرة تحرّك النتيجة فعلاً؟

افتح أي مقدر تجميعي في `get_params()` وسترى عشرات البارامترات. الخبر الجيد أن ٩٠٪ منها نادراً ما يستحق لمسة واحدة؛ فلكل عائلة ثلاث إبر تحكم قدرتها الفعلية، والباقي تفاصيل تُترك لقيمها الافتراضية. الجدول التالي هو خريطة الضبط العملية لهذا الجزء كله:

العائلة	البارامتر	ماذا يفعل	المدى العملي
Bagging	`n_estimators`	عدد النماذج المجمعة؛ تخفيض التباين يتناقص مكسبه مع الزيادة	٥٠ – ٥٠٠
Bagging	`max_samples`	نسبة (أو عدد) العينات لكل نموذج؛ صغّرها لتنويع النماذج	٠.٥ – ١.٠
Bagging	`max_features`	نسبة الميزات لكل نموذج؛ الأقوى في كسر الارتباط بين الأعضاء	٠.٣ – ١.٠
Boosting	`learning_rate`	حجم خطوة كل نموذج في تصحيح البواقي	٠.٠١ – ٠.٥
Boosting	`n_estimators`	عدد خطوات التصحيح المتتالية	١٠٠ – ٢٠٠٠
Boosting	`max_depth`	عمق كل شجرة ضعيفة؛ يحدد تعقيد التفاعلات الملتقطة	٢ – ٥
Boosting	`subsample`	نسبة العينات المستخدمة في كل خطوة؛ يضيف عشوائية مقاومة للإفراط	٠.٦ – ١.٠
HistGB	`max_iter`	عدد التكرارات (يقابل n_estimators)	١٠٠ – ١٠٠٠
HistGB	`max_leaf_nodes`	عرض الشجرة بدل عمقها	١٥ – ٦٣
HistGB	`max_bins`	عدد خانات المدرّج لكل ميزة؛ الأعلى أدق وأبطأ	١٢٨ – ٢٥٥

🔑 المفهوم المحوري

في Bagging البارامترات الثلاثة تعمل في اتجاه واحد: **كسر التشابه** بين الأعضاء لخفض الارتباط ρ (كما في الفصل ٤٣). وفي Boosting المعادلة مختلفة تماماً: `learning_rate` و `n_estimators` طرفا معادلة واحدة لا تُضبطان منفصلتين — تقليل الأول يلزمه مضاعفة الثانية، والانحياز النهائي للنموذج يتحدد بناتجهما معاً.

أما العلاقة المركزية في التعزيز فتعريفها بسيط: النموذج التجميعي هو مجموع خطوات صغيرة مضروبة بمعدل التعلم ν:

$$F_M(x) = F_0(x) + \nu \sum_{m=1}^{M} h_m(x)$$

حيث h_m شجرة المرحلة m المصححة للبواقي، و ν قيمة `learning_rate`، و M عدد المراحل. كلما صغُرت ν احتاج النموذج M أكبر بالقدر نفسه ليصل إلى التعقيد المطلوب — وهذا سبب القاعدة الشهيرة: أنصّف `learning_rate` فاضرب `n_estimators` في اثنين.

نبدأ بقياس أثر إبر Bagging الثلاث على بيانات سرطان الثدي:

```python
# أثر max_samples و max_features على الغرفة المتجمعة
import time
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

configs = [
    ("الافتراضي (كل العينات والميزات)", dict(max_samples=1.0, max_features=1.0)),
    ("نصف العينات فقط",                 dict(max_samples=0.5, max_features=1.0)),
    ("نصف الميزات فقط",                 dict(max_samples=1.0, max_features=0.5)),
    ("نصف الاثنين معاً",                dict(max_samples=0.5, max_features=0.5)),
]
for name, kw in configs:
    t0 = time.perf_counter()
    b = BaggingClassifier(DecisionTreeClassifier(random_state=42),
n_estimators=60,
                          bootstrap=True, random_state=42, n_jobs=-1,
**kw).fit(X_tr, y_tr)
    dt = time.perf_counter() - t0
    print(f"{name:32s} | دقة: {b.score(X_te, y_te):.4f} | زمن التدريب: {dt:.2f}s")
```

الدرس المزدوج من المخرجات: تقليص العينات أو الميزات لا يضر الدقة هنا إطلاقاً (٠.٩٤٧٤ نفسها في ثلاث من أربع الحالات)، بينما يخفض زمن التدريب بشدة لأن كل شجرة صارت أبسط وأصغر بيانات — فمتى كانت غالبية الأشجار زائدة عن الحاجة؟ عند ضبط سرعة الاستدلال لاحقاً (الفصل ٥٠) ستعود هذه اللمسة للواجهة.

٦.٢ استراتيجية التعزيز: ابدأ جسوراً ثم رقّق

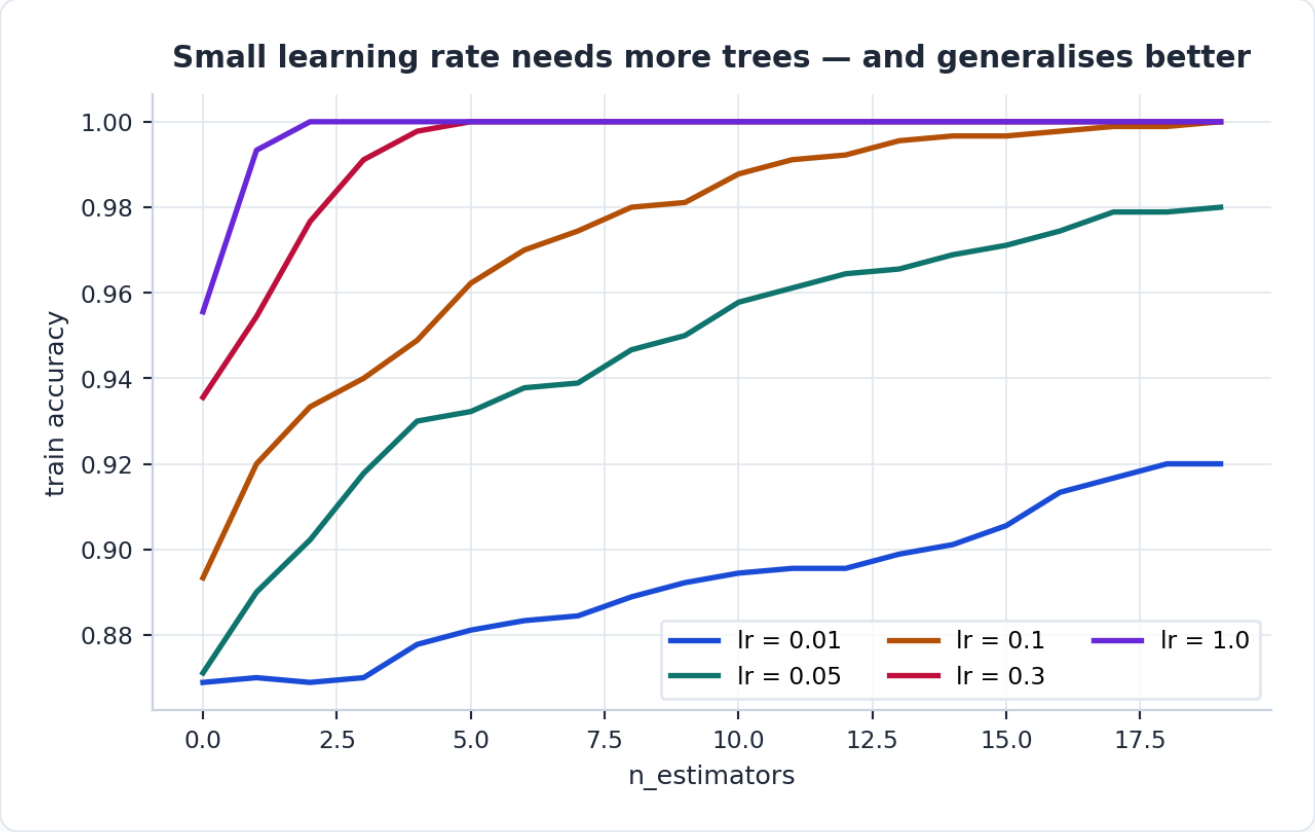
البحث الشبكي الساذج على Boosting يجرب `learning_rate` من ٠.٠١ إلى ١ مع `n_estimators` من ٥٠ إلى ٢٠٠٠ — أي مئات التركيبات، كل منها تدريب كامل. الاستراتيجية الاحترافية أسرع بعشرات المرات:

لاحظ في جدول الأرقام التالي كيف تتنافس التركيبات الثلاث على الدقة نفسها بينما تتباعد تكلفتها:

ثلاث تركيبات learning_rate و n_estimators متكافئة الدقة python

```python
import time
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier

X, y = make_classification(n_samples=900, n_features=14, n_informative=6,
                           n_redundant=3, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

for lr, n, d, sub in [(0.5, 60, 2, 1.0), (0.1, 300, 2, 1.0), (0.05, 600, 3,
0.8)]:
    gb = GradientBoostingClassifier(n_estimators=n, learning_rate=lr,
                                    max_depth=d, subsample=sub, random_state=42)
    t0 = time.perf_counter()
    gb.fit(X_tr, y_tr)
    dt = time.perf_counter() - t0
    print(f"lr={lr:<4} n={n:<4} depth={d} sub={sub} | دقة: {gb.score(X_te,
y_te):.4f} | زمن: {dt:.2f}s")
```

التركيبة الأولى (lr=٠.٥) أنهت التدريب في كسر من زمن الثالثة وبفارق دقة هزيل؛ والتركيبة المتوسطة (lr=٠.١) هي المفاضلة الذهبية هنا. هكذا يُقرأ جدول الضبط: لا تبحث عن أعلى رقم في عمود الدقة، بل عن أرخص تركيبة ضمن ٠.٥٪ من القمة.



الشكل ١٠٦أثر learning_rate على منحنى الخطأ مع تزايد المراحل: الخطوات الصغيرة تتعلم أبطأ وتعام أخير

في الشكل يظهر جوهر الاستراتيجية: معدل التعلم الكبير ينزل سريعاً لكنه يقفز فوق القمة وينفور، والمعدل الصغير ينزل بهدوء إلى عمق أفضل — لكنه يحتاج مراحل أكثر بعشرات الأضعاف.

والآن نفعّل الحارس الآلي في HistGradientBoosting: بارامتر `early_stopping` يفصل نسبة `validation_fraction` من بيانات التدريب ويتوقف عند ثبات الدقة `n_iter_no_change` مرات متتالية:

التوقف المبكر يختصر 500 تكرار إلى 74 python

```
from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(max_iter=500, learning_rate=0.08,
                                     max_leaf_nodes=31, early_stopping=True,
                                     validation_fraction=0.15,
n_iter_no_change=15,
                                     random_state=42)
hgb.fit(X_tr, y_tr)
print(f"طلبت max_iter=500 فتوقف وحده عند n_iter_ = {hgb.n_iter_}")
print(f"دقة الاختبار: {hgb.score(X_te, y_te):.4f}")
print(f"التكرارات الموفرة: {500 - hgb.n_iter_} ({(500 - hgb.n_iter_) / 500:.0%} من
الزمن)")
```

طلبنا ٥٠٠ تكرار فوقف النموذج نفسه عند ٧٤ — أي أن ٨٥٪ من الزمن المخطط كان سيُهدر في مراحل لا تحسن شيئاً.

٦.٣ عدد المراحل الأمثل بلا إعادة تدريب: staged_predict

سؤال محوري في كل تعزيز: أين تتوقف المراحل بالضبط؟ إعادة تدريب النموذج بعدد مراحل مختلفة في كل مرة تفاوت مأساوي. أجمل هدية في `GradientBoostingRegressor` و `GradientBoostingClassifier` هي مولدات `staged_predict` التي تعيد تفسير النموذج الحالي مرحلةً مرحلة دون أي إعادة تدريب — فتكلفة كل استدعاء هي تمرير أمامي فقط.

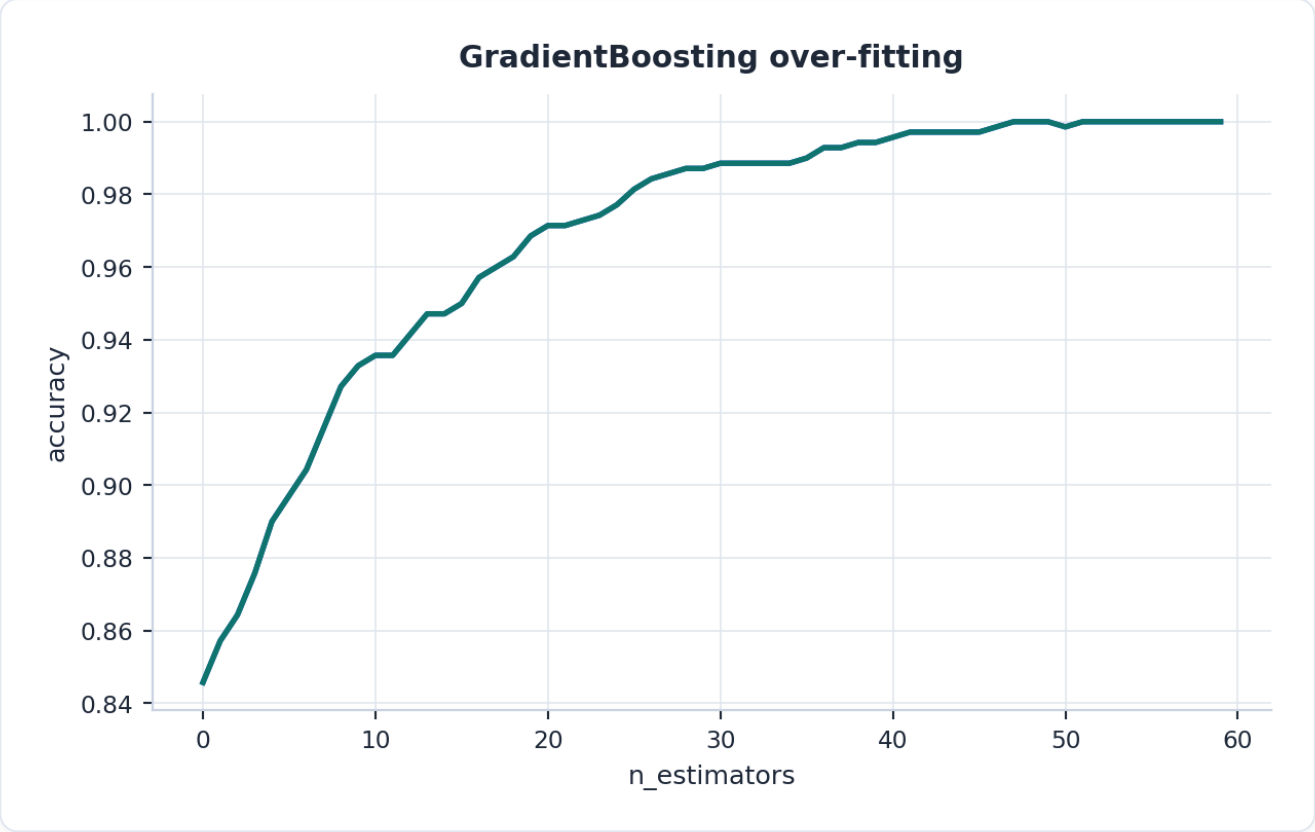
> **تغيّر في الإصدارات الحديثة**
>
> طريقة `staged_score` أُزيلت من واجهة التعزيز في الإصدارات الأخيرة. ما يبقى: `staged_predict` للانحدار، و `staged_predict_proba` و `staged_decision_function` للتصنيف — وهي البديل المعتمد: مرّ على التنبؤات المرحلية واحسب المقياس يدوياً بأي دالة من `sklearn.metrics`.

نصنع حالة إفراط مقصودة (ضجيج عالٍ ومعدل تعلم جسور) ونرصد الدقة مرحلة بمرحلة:

رصد قمة الدقة عبر staged_predict — python

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import r2_score

Xr, yr = make_regression(n_samples=600, n_features=14, n_informative=5,
                         noise=25.0, random_state=42)
Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(Xr, yr, test_size=0.3,
random_state=42)

gb = GradientBoostingRegressor(n_estimators=400, learning_rate=0.3,
                               max_depth=3, random_state=42).fit(Xr_tr, yr_tr)

staged = np.asarray([r2_score(yr_te, p) for p in gb.staged_predict(Xr_te)])
best_stage = int(np.argmax(staged)) + 1
print(f"القمة عند المرحلة {best_stage} بدقة R² = {staged[best_stage-1]:.4f} من أصل 400")
print(f"المرحلة الأخيرة (400) بدقة R² = {staged[-1]:.4f} — أقل من القمة!")
print(f"R² على التدريب في النهاية: {gb.score(Xr_tr, yr_tr):.4f} (حفظ كامل للضجيج)")
```

القمة بلغت عند المرحلة ٢٧ ثم تآكلت الدقة بهدوء حتى نهاية المراحل الأربع مئة؛ والنموذج على التدريب وصل إلى R² مثالي ١.٠٠٠٠ — حفظ كامل لضجيج العينات. لو اعتمدنا النموذج كما هو لخسرنا الجزء بين ٠.٩٠٩٠ و ٠.٩٠٤٠، ولو أعدنا التدريب لكل عدد مراحل محتمل لدفعنا ثمناً مضاعفاً لا داعي له أصلاً.



الشكل ١٠٧منحنى إفراط التعزيز: دقة التدريب تتصاعد والدقة الاختبارية تبلغ قمة ثم تنحدر

يوضح الشكل النمط نفسه الذي رصدته للتو في الأرقام: تصاعد دقة التدريب بلا سقف بينما تنعكس دقة الاختبار بعد قمة مبكرة — ومنحنى التحقق المتقاطع الذي رأيته في الفصل ٨٠ هنا بحلة تجميعية.

> **نصيحة**
>
> عثرت على المرحلة المثلى M^* من `staged_predict`؟ لا تحتفظ بالنسخة كاملة واقتطع منها؛ بل أعد التدريب مرة واحدة أخيرة بـ `n_estimators=M*` مع `learning_rate` أنصاف قيمتها ومراقبة `staged_predict` مجدداً — القاعدة: كل خفض في `learning_rate` يستحق مضاعفة المراحل قبل إعلان القمة.

٦.٤ warm_start و OOB: منحنى الأشجار المتزايدة

في عائلة Bagging سؤال المقابل: كم شجرة تكفي؟ `warm_start=True` يحفظ الغابة المدرّبة ويضيف إلى أشجارها عند كل نداء `fit` جديد بدل البناء من الصفر، فترسم منحنى الدقة مقابل عدد الأشجار بتكلفة الأشجار الجديدة فقط. ومع `oob_score=True` (الفصل ٤٢) يصبح القياس مجانياً بلا مجموعة تحقق منفصلة:

```python
# نمو OOB مع تزايد الأشجار عبر warm_start
from sklearn.ensemble import RandomForestClassifier

X2_tr, X2_te, y2_tr, y2_te = X_tr, X_te, y_tr, y_te
rf = RandomForestClassifier(n_estimators=20, warm_start=True, oob_score=True,
                            random_state=42, n_jobs=-1)
for n in [20, 50, 100, 200, 400]:
    rf.n_estimators = n
    rf.fit(X2_tr, y2_tr)  # يضيف (n - السابق) أشجاراً فقط
    print(f"أشجار: {n:3d} | OOB: {rf.oob_score_:.4f} | دقة الاختبار: {rf.score(X2_te, y2_te):.4f}")
```

اقرأ العمودين معاً: OOB يصعد بسلاسة من ٠.٨٨٤١ ويبدأ بالتشبع حول ٠.٩١، بينما تتذبذب دقة الاختبار الصلبة فوق وتحت — وهذا توقع سليم لأن OOB متوسط على مئات التنبؤات الخارجية بينما مجموعة الاختبار ثابتة ومحدودة. عندما يستقر OOB عبر مضاعفتين متتاليتين، توقف عن الشراء؛ الأشجار الإضافية صارت ديكوراً يكلف ذاكرة وزمن استدلال.

> **لا تضاعف التوازي مرتين**
>
> وضع `n_jobs=-1` في الغابة و `n_jobs=-1` في `GridSearchCV` المحيطة بها معاً يعني سباق عشرات العمليات على الأنوية نفسها: النظام يفرم الوقت في تبديل السياق ويصبح أبطأ من التنفيذ التسلسلي! القاعدة: موازاة في مستوى واحد فقط — إما أن توازي البحث الشبكي على المرشحين وتترك المقدر الداخلي تسلسلياً، أو العكس. في البيانات الصغيرة غالباً ما يكون التوازي الداخلي للمقدر هو الأرخص طرفاً لأن إطلاق عمليات العمل يكلف أكثر من التدريب نفسه.

٦.٥ البحث الشبكي على المجموعات: احسب الثمن قبل أن تدفع

كل تركيبة في الشبكة تُدرَّب `cv` مرات، ثم يعاد تدريب الفائز مرة أخيرة على كامل البيانات. الصيغة الحاكمة:

$$N_{\text{fit}} = k \times |\Theta| + 1$$

حيث k عدد طيات التحقق المتقاطع، و $|\Theta|$ عدد تركيبات الشبكة، والواحد الأخيرة تدريب الـ refit النهائي. والزمن الإجمالي تقريباً $T \approx N_{\text{fit}} \times t_{\text{fit}}$ حيث t_{fit} زمن تدريب المقدر الواحد. شبكة ٥×٤ مع تحقق خماسي على غابة من ٢٠٠ شجرة تعني ١٠١ تدريباً كاملاً — لذلك اجعل شبكتك صغيرة وموجهاً (نحو الإبر الحاسمة في جدول ٦.١) واعتمد `RandomizedSearchCV` للفضاءات الواسعة كما في الفصل ٨٢.

بحث شبكي موجّه على إبر الغابة مع حساب التكلفة python

```python
import time
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

grid = {"max_depth": [6, None], "min_samples_leaf": [1, 5]}
n_fits = 3 * 2 * 2 + 1   # cv × |0| + refit
print(f"الشبكة تقتضي {n_fits} تدريباً كاملاً")
gs = GridSearchCV(RandomForestClassifier(n_estimators=40, random_state=42),
                  grid, cv=3)
t0 = time.perf_counter()
gs.fit(X_tr, y_tr)
dt = time.perf_counter() - t0
print(f"الأفضل: {gs.best_params_} | متوسط CV: {gs.best_score_:.4f} | الزمن:
{dt:.2f}s")
print(f"دقة الاختبار للفائز: {gs.score(X_te, y_te):.4f}")
```

شبكة من ٤ تركيبات فقط موجّهة نحو الإبر المؤثرة أنجزت المهمة في ثوانٍ. التفاصيل الكاملة لآلية البحث الشبكي — `ParameterGrid` و `refit` و `best_score_` وتوازن الطبقات — أصلها موطن الفصل ٨١؛ هنا اكتفينا بمنهجية توجيه الشبكة نحو مجموعات كاملة.

٦.٦ جديد في ١.٩: واجهة الاستدعاءات لتتبع التدريبات الطويلة

جديد في ١.٩ — callbacks

أدخل الإصدار ١.٩ واجهة الاستدعاءات (Callbacks) عبر الوحدة `sklearn.callback` : الصنف `ProgressBar` لعرض شريط تقدم لكل مهمة تدريب، و `ScoringMonitor` لتسجيل الدرجات خلال التدريب. تُثبَّت الاستدعاءات على المقدرات الداعمة لها بـ `set_callbacks(...)` — وهي حتى الآن البُنى الوصفية `Pipeline` و `GridSearchCV` و `RandomizedSearchCV` والنماذج التكرارية القليلة. تعقب بحث شبكي طويل على مجموعة تجميعية هو الاستخدام الأدق لها الآن.

نلصق `ProgressBar` على بحث شبكي يخفي ٢٥ تدريباً كاملاً؛ البارامتر `max_propagation_depth=0` يعرض تقدم المهمة الخارجية فقط:

```python
# شريط تقدم على بحث شبكي طويل عبر ProgressBar
from sklearn.callback import ProgressBar

grid2 = {"n_estimators": [60, 120], "max_depth": [10, None],
         "min_samples_split": [2, 8]}
n_fits2 = 3 * 2 * 2 * 2 + 1
print(f"هذا البحث يخفي {n_fits2} وشريط التقدم سيكشفها — تدريباً كاملاً")
gs2 = GridSearchCV(RandomForestClassifier(random_state=42), grid2, cv=3)
gs2.set_callbacks(ProgressBar(max_propagation_depth=0))
gs2.fit(X_tr, y_tr)
print(f"الفائز: {gs2.best_params_} | CV: {gs2.best_score_:.4f}")
```

في المخرجات يظهر شريط حي يكشف بنية البحث: مهمة fit الجذرية وتحتها مهمة search تتقدم مع إنجاز كل مرشح وكل طية. هذا بديل أنيق عن مطبعة `print` في كل حلقة، وهو أيضاً نقطة ارتكاز موصولة ستفيدك عندما تتوسع scikit-learn في دعم النماذج التكرارية للتتبع المرحلي الدقيق.

👷 في الميدان

في الأنظمة الحقيقية لا يُضبط النموذج مرة واحدة بل في كل دورة إعادة تدريب. الاستراتيجية الواردة هنا — خط أساس جسور ثم تضييق مع توقف مبكر — تختصر على البيانات الوسطية أرصافاً من ساعات الحوسبة شهرياً، لأن أغلب الوقت المهدور عالمياً ليس في التدريبات النافعة بل في مراحل تعزيز وأشجار غابة تكمل تدريبها بعد أن توقف تحسنها منذ زمن.

تمارين الفصل التاسع والأربعين

- **ملاحظة**

التمرين ٦.١

على بيانات `load_digits`، درّب `HistGradientBoostingClassifier` بأربعة قيم لـ `learning_rate` (٠.٥، ٠.٢، ٠.١، ٠.٠٥) مع `max_iter=400` و `early_stopping=True` و `n_iter_no_change=15`، واطبع لكل قيمة: عدد التكرارات الذي توقف عنده `n_iter_` ودقة الاختبار. أي قيمة حققت أفضل مفاضلة بين عدد التكرارات والدقة؟

التمرين ٦.٢

برهان نظري: إذا بلغ نموذج تعزيز بمعدل تعلم ν وعدد مراحل M تعقيداً مكافئاً لنموذج بمعدل تعلم $\nu/2$، فما العدد المطلوب من المراحل تقريباً؟ اكتب الاستنتاج اللفظي ثم صحّحه عملياً على بيانات التمرين السابق بمقارنة (lr=٠.٢, n=٨٠) مع (lr=٠.١, n=١٦٠) حيث لا تُفعّل ميزة التوقف المبكر.

التمرين ٦.٣

على `fetch_california_housing` (أول ٢٠٠٠ صف)، استخدم `RandomForestRegressor` مع `warm_start=True` و `oob_score=True` ورسم قيم OOB عند ١٠، ٣٠، ٦٠، ١٠٠، ٢٠٠ شجرة (ابدأ من ١٠ لتجنب تحذير قلة الأشجار)، وحدد العدد الذي بعده لا يتحسن OOB بأكثر من ٠.٠٠٢.

- **ملاحظة**

حل التمرين ٦.١: مسح learning_rate مع التوقف المبكر python

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier

Xd, yd = load_digits(return_X_y=True)
Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(Xd, yd, test_size=0.3,
                                              random_state=42, stratify=yd)

for lr in [0.5, 0.2, 0.1, 0.05]:
    h = HistGradientBoostingClassifier(max_iter=400, learning_rate=lr,
                                       early_stopping=True, n_iter_no_change=15,
                                       random_state=42).fit(Xd_tr, yd_tr)
    print(f"lr={lr:<5} توقف عند n_iter_={h.n_iter_:3d} | دقة الاختبار: {h.score(Xd_te,
yd_te):.4f}")
```

ملاحظة

حل التمرين ٦.٣: منحنى OOB على بيانات الإسكان python

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

data = fetch_california_housing()
Xc, yc = data.data[:2000], data.target[:2000]
Xc_tr, yc_tr = Xc[:1400], yc[:1400]

rfr = RandomForestRegressor(n_estimators=10, warm_start=True, oob_score=True,
                            random_state=42, n_jobs=-1)
prev = None
for n in [10, 30, 60, 100, 200]:
    rfr.n_estimators = n
    rfr.fit(Xc_tr, yc_tr)
    gain = "—" if prev is None else f"{rfr.oob_score_ - prev:+.4f}"
    print(f"أشجار: {n:3d} | OOB R²: {rfr.oob_score_:.4f} | التحسن: {gain}")
    prev = rfr.oob_score_
```

اقرأ عمود التحسن: حين يقفز المكسب الأخير تحت عتبة ٠٫٠٠٢ فالعدد الحالي هو نقطة التوقف الرشيدة؛ شراء أشجار إضافية بعده دفع مقابل رقاقة دقة لا تُرى.

ملاحظة

٦.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
ضبط `n_estimators` وحده في Boosting بلمس `learning_rate`	مقارنة منحرفة: ٣٠٠ مرحلة بمعدل ٠.٠٥ ليست «أفضل» من ٦٠ بمعدل ٠.٠٥، بل زوجان مختلفان كلياً	اضبط الزوج معاً وقيّم التكلفة الزمنية مع الدقة
البحث عن `staged_score` في sklearn ١.٩	`AttributeError` لأن الطريقة أزيلت من واجهة التعزيز الحديثة	استخدم `staged_predict` مع دالة مقياس من `sklearn.metrics` يدوياً
`n_jobs=-1` في الغابة وفي البحث الشبكي معاً	إفراط توازٍ يفرم الأنوية في تبديل السياق ويبطئ كل شيء	موازاة مستوى واحد: البحث أو المقدر، لا الاثنان
تفعيل `oob_score` مع `warm_start` وبدء الحلقة من شجرة أو اثنتين	تحذير `Some inputs do not have OOB scores` ودرجات OOB بلا معنى	ابدأ من ٢٠ شجرة على الأقل حيث تكتمل تغطية العينات خارج الحقيبة

٦.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
معدل التعلم	معامل تصغير مساهمة كل شجرة في المجموع التراكمي للتعزيز	Learning rate (shrinkage)
التوقف المبكر	إيقاف التدريب عند ثبات الدقة على جزء تحقق داخلي	Early stopping
التنبؤ المرحلي	إعادة تفسير نموذج التعزيز مرحلة بمرحلة دون إعادة تدريب	Staged prediction
البدء الدافئ	الاستمرار في تدريب مقدر مدرَّب بإضافة مقدرات جديدة إليه	Warm start
واجهة الاستدعاءات	آلية خطاطات تُبلَّغ خلال التدريب لعرض التقدم ومراقبة الدرجات	Callbacks

الجزء ٦ · المجموعات والأغلفة

الفصل ٥٠ — متى تستخدم أي مجموعة؟

جدول قرارات يربط نوع البيانات بالاختيار الأمثل

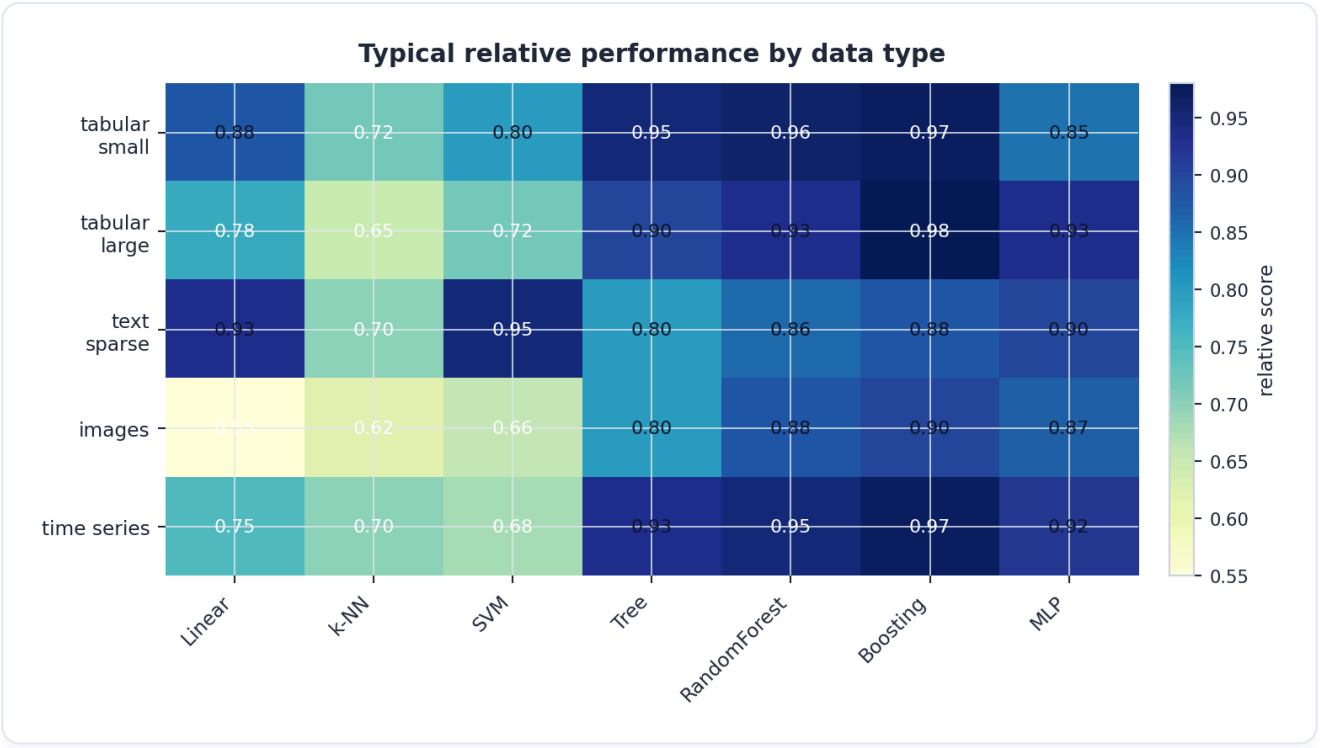
الوحدة **sklearn.ensemble** الحجم **٩٣٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تختار المجموعة المناسبة من جدول قرار يربط نوع البيانات والقيد العملي بالأداة
- أن تفسر الاختيار بحدسين: تباين عالٍ يُبَغّ بالتجميع الموازي، وانحياز عالٍ يُعزَّز بالتتابع
- أن تقرأ مقارنة عملية موحّدة بين سبع مجموعات بأزمنة تدريب ودقات حقيقية
- أن تحدد متى تكفي شجرة واحدة مضبوطة بدل المجموعة كاملة
- أن تتجنب الأخطاء الكلاسيكية في اختيار وضبط المجموعات

٦.١ خريطة العائلة: أين نحن الآن؟

أنهينا جولة كاملة: بدأنا بالفصل ٤٢ بتجميع العينات فقتلنا التباين، ثم الغابة العشوائية والإكسترا ترييز كسرتا الارتباط وكسرتا سقف الزمن، ثم التعزيز بنوعيه — AdaBoost الذي يرفع صوت الصعب، وتعزيز التدرج الذي يصحح البواقي، وHistGB الذي صار معيار الجداول الصناعية، وأخيراً التصويت والتكديس اللذان يجمعان عائلات متباينة. سؤال هذا الفصل الحاسم: أمام مسألة جديدة، من تختار؟



الشكل ١٠٨خريطة عائلات النماذج التجميعية في scikit-learn ومسارات الاختيار بينها

يقرأ الشكل كخريطة مترو: مدخل واحد هو مسألتك (تصنيف أم انحدار)، ثم تتفرع المسارات نحو العائلات بحسب ما تعانيه بياناتك — وهذا بالضبط ما نحوّله في القسم التالي إلى جدول صريح.

المفهوم المحوري

حدسان يحكمان كل قرار تجميعي: إن كان نموذجك المفرد يعاني **تبايناً عالياً** (شجرة عميقة تتقلب بتغير خفيف في البيانات) فالتجميع الموازي Bagging هو العلاج، لأن متوسط النماذج المستقلة يهدئ التذبذب. وإن كان يعاني **انحيازاً عالياً** (نموذج بسيط لا يلتقط العلاقة أصلاً) فالتجميع الموازي لن ينفع — متوسط نماذج متحيزة يظل متحيزاً — وستحتاج تعزيزاً تتابعياً يبني نموذجاً معقداً خطوة خطوة. حدد أولاً أي الحسنين يعانيك، ثم اختر العائلة.

ولماذا يفوق الحشد الفرد أصلاً؟ معضلة المحلفين الكوندورسية تختصر الجواب: إذا كان كل عضو في مجموعة من B أعضاء مستقلين يصيب باحتمال p أكبر قليلاً من النصف، فإن احتمال أن تصيب أغلبيتهم:

$$P_{\text{maj}} = \sum_{k=\lfloor B/2 \rfloor + 1}^{B} \binom{B}{k} p^k (1-p)^{B-k}$$

حيث k عدد الأعضاء المصيبين، و P_{maj} يصعد مع B نحو الواحد بشرطين اثنين: أن يبقى p فوق النصف، وأن تبقى الأخطاء **مستقلة** — وكسر هذا الارتباط هو ما تفعله `max_features` والبوتستراب في الفصلين ٤٢ و ٤٣. ولهذا

تفشل المجموعة حين تكون أعضاؤها نسخاً متطابقة أو ضعافاً دون النصف: الشرطان معاً هما وصفة كل جدول قرار أدناه.

٦.٢ جدول القرار الكبير

هذا هو الفصل الذي ستعود إليه قبل كل مشروع. اقرأ صفك: نوع بياناتك في العمود الأول، أشد قيودك في الثاني، والاختيار الأمثل في الثالث:

نوع البيانات	القيد أو العرض	الاختيار الأمثل	لماذا
جداول صغيرة ونظيفة (أقل من ١٠ آلاف صف)	سرعة وتفسير	شجرة مضبوطة أو LogisticRegression	المجموعة تضيف تعقيداً بلا مكسب يُذكر، والشجرة المضبوطة تكفي غالباً
جداول كبيرة وضوضائية	دقة قصوى	RandomForest أو HistGB	عشوائية الغابة تبلع الضجيج، وHistGB يلتقط التفاعلات بلا فرط
أبعاد عالية متفرقة (نصوص، ترميزات)	ذاكرة وسرعة	ExtraTrees أو نماذج خطية منظمة	عشوائية العتبات لا تحتاج فرزاً كاملاً وتتحمل المتفرق
بيانات زمنية أو تسلسلية	الاستقرار عبر الزمن	GradientBoosting مع ميزات هندسية	لا توجد مجموعة سحرية للزمن؛ التعزيز مع ميزات جيدة هو الأساس
فئات غير متوازنة	استرجاع الفئة النادرة	HistGB مع class_weight أو أخذ عينات	الانحياز للفئة الكبيرة يُعالج بالأوزان لا بمزيد من الأشجار
بيانات نصية كثيفة الأبعاد	خط أنابيب سريع	نماذج خطية (الفصل ٣٥)	التجميع الشجري يتعثر في التفرق عالي الأبعاد
تنبؤات متباينة المصادر جاهزة	دقة نهائية فوق كل اعتبار	Stacking (الفصل ٤٨)	المقدر العليا يتعلم من يُصدَّق متى

☑ **قائمة الاختيار قبل أول تدريب**

٦.٣ المقارنة الموحّدة: سبعة متسابقين على حلبة واحدة

الحجج النظرية جميلة، لكن الميدان يحسم. سنعرّض نفس البيانات لنفس السبعة في مسألتين: تصنيف على بيانات سرطان الثدي (٥٦٩ صف × ٣٠ ميزة)، وانحدار على أول ٣٠٠٠ صف من إسكان كاليفورنيا. كل نموذج بقيمه الافتراضية المعقولة وبذرة واحدة `random_state=42` — ولا ضبط مسبق، لأن هذا هو «أول لقاء». الجدولان التاليان هما قلب هذا الفصل:

سبعة متسابقين على مسألة التصنيف python

```python
import time
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                              ExtraTreesClassifier, AdaBoostClassifier,
                              GradientBoostingClassifier,
HistGradientBoostingClassifier)
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

contenders = [
    ("LogisticRegression", make_pipeline(StandardScaler(),
LogisticRegression(max_iter=5000))),
    ("شجرة واحدة",          DecisionTreeClassifier(random_state=42)),
    ("Bagging",
BaggingClassifier(DecisionTreeClassifier(random_state=42),
                                             n_estimators=60, random_state=42,
n_jobs=-1)),
    ("RandomForest",       RandomForestClassifier(n_estimators=100,
random_state=42, n_jobs=-1)),
    ("ExtraTrees",         ExtraTreesClassifier(n_estimators=100,
random_state=42, n_jobs=-1)),
    ("AdaBoost",           AdaBoostClassifier(n_estimators=100,
random_state=42)),
    ("GradientBoosting",   GradientBoostingClassifier(n_estimators=150,
learning_rate=0.1,
                                                  max_depth=2,
random_state=42)),
    ("HistGB",             HistGradientBoostingClassifier(max_iter=150,
random_state=42)),
]
for name, m in contenders:
    t0 = time.perf_counter()
    m.fit(X_tr, y_tr)
    fit_t = time.perf_counter() - t0
    print(f"{name:18s} | دقة: {m.score(X_te, y_te):.4f} | تدريب: {fit_t:.2f}s")
```

اقرأ الجدول بهدوء قبل أن تستعجل النتائج: النموذج **الخطي** هو الفائز في الدقة وبفارق واضح، وبزمن لا يُذكر! بيانات سرطان الثدي الصغيرة شبه قابلة للفصل بخط مستوٍ، فكل المجموعات «تنافس على المركز الثاني». هذه ليست عيباً في المجموعات؛ إنها درس الجدول الأول: اعرف بياناتك قبل أن تستدعي جيشاً من الأشجار. لاحظ أيضاً ترتيب المجموعات فيما بينها: الإكسترا تريز أولها والغابة العشوائية — بأشجارها المئة — أدناها هنا.

والآن نفس الحلبة على الانحدار:

سبعة متسابقين على مسألة الانحدار python

```python
import time
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (BaggingRegressor, RandomForestRegressor,
                              ExtraTreesRegressor, GradientBoostingRegressor,
                              HistGradientBoostingRegressor)
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import root_mean_squared_error

data = fetch_california_housing()
X, y = data.data[:3000], data.target[:3000]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

contenders = [
    ("Ridge (خطي)",      make_pipeline(StandardScaler(), Ridge(alpha=1.0))),
    ("شجرة واحدة",        DecisionTreeRegressor(random_state=42)),
    ("Bagging",
BaggingRegressor(DecisionTreeRegressor(random_state=42),
                                       n_estimators=60, random_state=42,
n_jobs=-1)),
    ("RandomForest",     RandomForestRegressor(n_estimators=100,
random_state=42, n_jobs=-1)),
    ("ExtraTrees",       ExtraTreesRegressor(n_estimators=100, random_state=42,
n_jobs=-1)),
    ("GradientBoosting", GradientBoostingRegressor(n_estimators=200,
learning_rate=0.1,
                                                   max_depth=3, random_state=42)),
    ("HistGB",           HistGradientBoostingRegressor(max_iter=200,
random_state=42)),
]
for name, m in contenders:
    t0 = time.perf_counter()
    m.fit(X_tr, y_tr)
    fit_t = time.perf_counter() - t0
    print(f"{name:18s} | RMSE: {root_mean_squared_error(y_te,
m.predict(X_te)):.4f} | تدريب: {fit_t:.2f}s")
```

هنا تنقلب الطاولة كما يجب أن تنقلب: النموذج الخطي في قاع الترتيب لأن علاقات الإسكان غير خطية وتفاعلية، والشجرة المفردة تخطئ بشدة، والمجموعات الشجرية تتزاحم في المقدمة. هذا التباين بين الحلبتين هو رسالة الفصل كلها: **لا يوجد فائز مطلق، بل فائز لكل مسألة** — ولهذا وُجد جدول القرار لا قائمة إعجاب.

٦.٤ الحدسان في الدليل: أثبتهما بأرقام

الادعاء الأول: الشجرة المفردة **غير مستقرة** — تغيير طفيف في بيانات التدريب يهز دقتها، بينما تهتز الغرفة المجمعة اهتزازاً طفيفاً. نكرر التجربة عشرين مرة: في كل مرة نسحب ٦٠٪ من بيانات التدريب، ندرب الشجرة والغرفة عليها، ونقيس على مجموعة اختبار ثابتة:

```python
# ثبات الشجرة المفردة مقابل الغرفة عبر 20 عينة تدريب
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

X, y = make_classification(n_samples=600, n_features=12, n_informative=4,
                           n_redundant=3, random_state=42)
X_tr0, X_te, y_tr0, y_te = train_test_split(X, y, test_size=0.3,
                                            random_state=42, stratify=y)
rng = np.random.default_rng(42)
accs_tree, accs_bag = [], []
for rep in range(20):
    idx = rng.choice(len(X_tr0), size=int(0.6 * len(X_tr0)), replace=False)
    Xr, yr = X_tr0[idx], y_tr0[idx]
    accs_tree.append(DecisionTreeClassifier(random_state=42).fit(Xr,
yr).score(X_te, y_te))
    accs_bag.append(BaggingClassifier(DecisionTreeClassifier(random_state=42),
                                      n_estimators=50, random_state=42,
                                      n_jobs=-1).fit(Xr, yr).score(X_te, y_te))

at, ab = np.asarray(accs_tree), np.asarray(accs_bag)
print(f"شجرة مفردة: متوسط {at.mean():.4f} | انحراف {at.std():.4f} | المدى
[{at.min():.4f}, {at.max():.4f}]")
print(f"Bagging 50 : متوسط {ab.mean():.4f} | انحراف {ab.std():.4f} | المدى
[{ab.min():.4f}, {ab.max():.4f}]")
print(f"نسبة خفض الانحراف: {1 - ab.std() / at.std():.0%}")
```

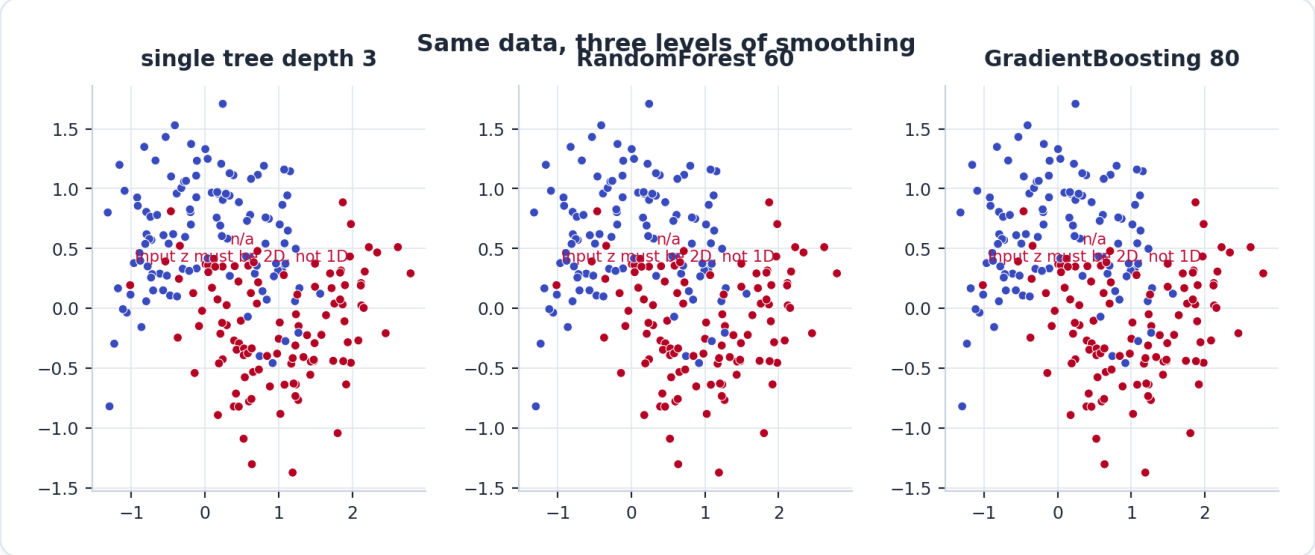
الأرقام تعلن النتيجة: انحراف الشجرة المفردة ٠.٠٢٧٦ بينما الغرفة ٠.٠١٧٩ — خفض بنسبة ٣٥٪ — ومع رفع متوسط الدقة من ٠.٧٧٣٩ إلى ٠.٨٣٠٠. هذا هو Bagging كما وعدته معادلة الفصل ٤٢.

الادعاء الثاني: عندما يكون **الانحياز** هو المرض، يفشل النموذج الخطي حيث ينجح التعزيز. دالة فريدمان الكلاسيكية تمزج جيباً ومربعاً وتفاعلات، فتنهار أي خريطة خطية:

انحياز النموذج الخطي أمام تعزيز التدرج python

```python
from sklearn.datasets import make_friedman1
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import r2_score

Xf, yf = make_friedman1(n_samples=600, n_features=10, noise=1.0,
random_state=42)
Xf_tr, Xf_te, yf_tr, yf_te = train_test_split(Xf, yf, test_size=0.3,
random_state=42)

lin = LinearRegression().fit(Xf_tr, yf_tr)
gb = GradientBoostingRegressor(n_estimators=200, learning_rate=0.1, max_depth=2,
                               random_state=42).fit(Xf_tr, yf_tr)
print(f"R² للانحدار الخطي:  {r2_score(yf_te, lin.predict(Xf_te)):.4f} (سقف انحيازه)")
print(f"R² لتعزيز التدرج:   {r2_score(yf_te, gb.predict(Xf_te)):.4f}")
```

فجوة ٠.١٥ في R^2 بين النموذجين ليست مسألة ضبط ولا حظ؛ إنها سقف انحياز الخريطة الخطية أمام بنية غير خطية أصلاً. لو طبقنا Bagging على الانحدار الخطي هنا لتحسن لا شيء تقريباً — متوسط خرائط مستقيمة لا يصنع جيباً.



الشكل ١٠٩ خرائط حدود القرار: كيف يقطع كل نموذج تجميعي الفضاء بشكل مختلف

أعد النظر في خرائط الحدود: الخطوط الحادة للغابات تعكس حساسية الأشجار، والملمس الأملس للتعزيز يعكس خطواته الصغيرة المتدرجة — هذه البصمة الهندسية هي ما تلمسه في الأرقام أعلاه.

٦.٥ سرعة الاستدلال: من يجيب في أقل من ملي ثانية؟

دقة التدريب ليست كل القصة؛ في الإنتاج يُستدعى النموذج ملايين المرات. تكلفة استدلال مجموعة هي مجموع تكاليف أعضائها:

$$T_{\text{infer}} \approx B \times t_{\text{tree}}$$

حيث B عدد الأشجار و t_{tree} زمن تمرير عينة واحدة في الشجرة. غابة من ٢٠٠ شجرة عميقة تدفع ثمن دقتها في كل طلب، بينما HistGB يجمع عشرات الأشجار الضحلة (وتدريجها الرقائقي يجعل التمرير رخيصاً)، والإكسترا تريز يدفع أشجاراً عشوائية أقل عمقاً من الغابة المضبوطة:

زمن استدلال 171 عينة عبر النماذج الأربعة python

```
import time
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, ExtraTreesClassifier,
                              HistGradientBoostingClassifier)

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)
models = [
    ("شجرة واحدة",   DecisionTreeClassifier(max_depth=5, random_state=42)),
    ("RandomForest", RandomForestClassifier(n_estimators=200, random_state=42,
n_jobs=-1)),
    ("ExtraTrees",   ExtraTreesClassifier(n_estimators=200, random_state=42,
n_jobs=-1)),
    ("HistGB",       HistGradientBoostingClassifier(max_iter=150,
random_state=42)),
]
for name, m in models:
    m.fit(X_tr, y_tr)
    m.predict(X_te)  # تسخين قبل القياس
    t0 = time.perf_counter()
    for _ in range(20):
        m.predict(X_te)
    dt = (time.perf_counter() - t0) / 20 * 1000
    print(f"{name:14s} | دقة: {m.score(X_te, y_te):.4f} | زمن استدلال 171 عينة:
{dt:.2f} ms")
```

الشجرة الواحدة تجيب في أقل من نصف ملي ثانية لكنها الأضعف دقة، وHistGB يوازن المعادلة: قرب قمة الدقة بزمن يفوق الغابة المضبوطة بعشرات الأضعاف سرعة. هذا هو سبب سموّه «المعيار الصناعي» في الفصل ٤٧.

٦.٦ متى تكفي شجرة واحدة مضبوطة؟

الخطأ الكلاسيكي الآخر في الاتجاه المعاكس: استدعاء غابة لمسألة فيها عشرات الصفوف. دعنا نقارن الشجرة المضبوطة بالغابة الافتراضية في قطبين:

شجرة مضبوطة مقابل غابة افتراضية في قطبين python

```python
import time
from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# القطب الأول: بيانات صغيرة نظيفة (Iris)
Xi, yi = load_iris(return_X_y=True)
Xi_tr, Xi_te, yi_tr, yi_te = train_test_split(Xi, yi, test_size=0.3,
                                              random_state=42, stratify=yi)
gs_i = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    {"max_depth": [2, 3, 4], "min_samples_leaf": [1, 3, 5]},
cv=5)
gs_i.fit(Xi_tr, yi_tr)
rf_i = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1).fit(Xi_tr, yi_tr)
print(f"Iris    | شجرة مضبوطة {gs_i.best_params_} | دقة: {gs_i.score(Xi_te,
yi_te):.4f} | عقد: {gs_i.best_estimator_.tree_.node_count}")
print(f"Iris    | دقة |      شجرة 100 غابة افتراضية: {rf_i.score(Xi_te, yi_te):.4f}")

# القطب الثاني: بيانات أوسع وأصعب (سرطان الثدي)
Xc, yc = load_breast_cancer(return_X_y=True)
Xc_tr, Xc_te, yc_tr, yc_te = train_test_split(Xc, yc, test_size=0.3,
                                              random_state=42, stratify=yc)
gs_c = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    {"max_depth": [3, 5, 7, None], "min_samples_leaf": [1, 5,
10]}, cv=5)
gs_c.fit(Xc_tr, yc_tr)
rf_c = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1).fit(Xc_tr, yc_tr)
print(f"Cancer  | شجرة مضبوطة | CV: {gs_c.best_score_:.4f} | اختبار:
{gs_c.score(Xc_te, yc_te):.4f}")
print(f"Cancer  | اختبار | غابة افتراضية: {rf_c.score(Xc_te, yc_te):.4f}")
```

في Iris الصغيرة النظيفة تفوقت شجرة من ٩ عقد فقط على الغابة الكاملة بفارق مذهل — وتفوق لافت لأن الإفراط في التعقيد يضر البيانات الصغيرة. أما في بيانات أوسع فقد انتعشت الغابة فوق الشجرة المضبوطة. القيادة العملية: **اضبط نموذجك المفرد أولاً** (الشجرة عبر تشذيبها كما في الفصل ٣٩)، فإن بلغ حد سقفه وكان الخطأ أعلى من هدفك، فانتقل إلى المجموعة — لا العكس.

٦.٧ خاتمة الجزء السادس

اقترب المطاف من هذا الجزء، ونحن الآن نملك ترسانة كاملة: تجميع بالعينات يقتل التباين، وغابة وإكسترا تريز تكسران الارتباط والزمن، وAdaBoost وتعزيز التدرج وHistGB يبنون الدقة تتابعياً، وتصويت وتكديس يجمعان العائلات. وقد تعلمت في هذا الفصل أن الاختيار بينها ليس ذوقاً بل جدول قرارات وحدسان وقياس ميداني.

لكن كل ما رأيناه حتى اليوم كان يفصل الفضاء بمستويات أو يقطعه مقاطع شجرية. في الجزء السابع ننتقل إلى عائلة أخرى كلياً: آلات المتجهات الداعمة (SVM) التي لا تقطع الفضاء أصلاً بل ترفعه بعدداً أبعاد جديداً عبر الحيلة النووية، وتسأل سؤالاً مختلفاً: ليس «أين أخطئ؟» بل «أين أوسع هامشي؟». ستجد هناك أن بعض مسائلنا — كالفصل بين الفئات المتشابكة بدقة — تحتاج فلسفة مختلفة تماماً عن الأشجار.

في الميدان

من يفتتح مشروعاً بمجموعة ضخمة قبل خط أساس بسيط يدفع ثمنه مرتين: مرة في وهم الدقة الذي يكشفه منحنى التعلّم لاحقاً، ومرة في صيانة نموذج لا يحتاجه أحد. أما من يظل على خط أساس خطي بعد أن تجاوزه البيانات، فيترك نقاط دقة على الطاولة يمكن لـ HistGB في ساعة واحدة أن يجمعها. الجدول أعلاه هو صمام الأمان بين الحالتين.

تمارين الفصل الخمسين

- **ملاحظة**

التمرين ٦.١

على `load_wine`، أعد «حلبة مصغرة» من أربعة متسابقين فقط: LogisticRegression، شجرة مضبوطة (بحث شبكي صغير)، RandomForest افتراضية، وHistGB افتراضي. اطبع جدول الدقة وزمن التدريب، وحدد الفائز، ثم فسّر: لماذا لا يصح تعميم ترتيب حلبة هذا الفصل على كل مسألة؟

التمرين ٦.٢

صمم تجربة تُظهر فشل Bagging مع نموذج منخفض التباين: طبق `BaggingClassifier` على `LogisticRegression` (بعد StandardScaler) على بيانات `load_breast_cancer`، وقارن دقته بدقة النموذج المفرد. توقع النتيجة قبل التنفيذ ثم اشرح ما حدث بلغة الانحياز والتباين.

التمرين ٦.٣

بمساعدة جدول القرار: بياناتك ٢ مليون صف، ٥٠٠ ميزة، ٤٠٪ منها مفقودة عشوائياً، والهدف توقع قيمة منزل (انحدار)، وحدك ساعة تدريب واحدة وسيرفر استدلال ضعيف. اكتب قرارك مع سبب لكل قيد، وبرهن بتجربة مصغرة (أول ٥٠٠٠ صف) على صحة اختيارك بين RandomForest و HistGB.

- **ملاحظة**

حل التمرين ٦.٢: Bagging على نموذج منخفض التباين python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import BaggingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

single = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
bagged = make_pipeline(StandardScaler(),
                       BaggingClassifier(LogisticRegression(max_iter=5000),
                                         n_estimators=60, random_state=42,
n_jobs=-1))
single.fit(X_tr, y_tr)
bagged.fit(X_tr, y_tr)
print(f"اللوجيستي المفرد:  {single.score(X_te, y_te):.4f}")
print(f"مجمّع على 60 نسخة: {bagged.score(X_te, y_te):.4f}")
```

التوقع يصيب: الدقة لم تتحسن تقريباً (وتذبذبت هبوطاً في تجارب أخرى). النموذج الخطي المقيد انحيازه هو المهيمن وتباينه شبه معدوم، فلا يملك Bagging تبايناً يقتله — ومتوسط ٦٠ خطاً مستقيماً يظل خطاً مستقيماً.

> **ملاحظة**

حل التمرين ٦.٣: برهان مصغر على اختيار HistGB python

```python
import time
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor,
HistGradientBoostingRegressor
from sklearn.metrics import root_mean_squared_error

data = fetch_california_housing()
X, y = data.data[:5000], data.target[:5000]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

for name, m in [("RandomForest", RandomForestRegressor(n_estimators=100,
                                                       random_state=42,
n_jobs=-1)),
                ("HistGB", HistGradientBoostingRegressor(max_iter=300,
                                                         random_state=42))]:
    t0 = time.perf_counter()
    m.fit(X_tr, y_tr)
    dt = time.perf_counter() - t0
    print(f"{name:14s} | RMSE: {root_mean_squared_error(y_te,
m.predict(X_te)):.4f} | تدريب: {dt:.2f}s")
```

القرار: HistGB. أربع حجج من جدول القرار: (١) جداول كبيرة ضوضائية → HistGB هو الافتراضي الصناعي، (٢) القيم المفقودة أصلية فيه بلا تعبئة مسبقة، (٣) زمن تدريبه في التجربة المصغرة يوازن أو يتفوق على الغابة وسيتوسع الفارق لصالحه على ٢ مليون صف، (٤) أشجاره الضحلة القليلة تجعل زمن الاستدلال على السيرفر الضعيف أقل بأضعاف. الغابة تبقى خياراً حائراً إن فرضت هيئة تنظيمية تفسيراً عبر أهمية الخصائص فقط.

ملاحظة

٦.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
البدء بمجموعة ضخمة قبل خط أساس بسيط	ساعات حساب لترتيب قد يخسره نموذج خطي، وإخفاء مشاكل البيانات خلف التعقيد	خط أساس خطي أو شجرة مضبوطة أولاً، ثم المجموعة عند تجاوز السقف
تطبيق Bagging على نموذج منخفض التباين	دقة تتحرك أفقياً بينما يتضاعف زمن التدريب والاستدلال	قلّم التباين أولاً (منحنى تعلّم أو تجربة بذور) ثم اختر العائلة
تجاهل زمن الاستدلال ومقارنة الدقة فقط	غابة من ٥٠٠ شجرة تدفع ثمن مللي ثوانٍ إضافية في كل طلب إنتاجي ملايين المرات	ضع `T_infer ≈ B × t_tree` في الميزانية وقياس الاستدلال قبل الاعتماد
تعميم ترتيب فائز حلبة واحدة على كل مسألة	الخطي فاز في حلبتنا التصنيفية وخسر الانحدارية — والترتيب يتقلب مع كل بيانات	أعد المقارنة الموحدة المصغرة على بياناتك أنت في كل مشروع

٦.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
خط الأساس	نموذج بسيط سريع يُبنى أولاً ليقيس عليه كل تعقيد لاحق	Baseline model
الاستقرار	قلة تغير تنبؤات النموذج مع تغير طفيف في بيانات التدريب	Stability
زمن الاستدلال	متوسط زمن توليد تنبؤ واحد أو دفعة بعد نشر النموذج	Inference latency
خريطة القرار	جدول يربط خصائص المسألة بالأداة الملائمة بدل الحدس الصرف	Decision map
مقارنة موحدة	تقييم عدة نماذج على نفس البيانات والبذرة والتقسيم بمعيار واحد	Unified benchmark

الجزء ٧

خوارزميات خاضعة للرقابة أخرى

آلات المتجهات الداعمة، والجيران، وبيز، والتمييز

ماذا يغطّي هذا الجزء

scikit-learn لا تتوقف عند الخطية والأشجار. آلات المتجهات الداعمة تحلّ المسائل الخطية في فضاء غير خطي. الجيران يتعلّمون بالنظر. بيزي يفترض استقلالاً ساذجاً ويفوز عادةً على البيانات النصية. كلها أدوات يجب أن تعرف متى تُخرجها من الجزاء.

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥١ — آلات المتجهات الداعمة: الحيلة الحقيقية

الهامش الأقصى، و KKT، ولماذا لا تتأثر SVM بحجم المقياس

الوحدة **scikit-learn ١.٩** الحجم **٩٧٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تستنتج هندسة الهامش الأقصى من شرط التصنيف وتحسب عرضه ٢/||w|| يدوياً
- أن تفسّر دور متغير الارتخاء xi والثابت C في المقايضة بين عرض الهامش وانتهاكاته
- أن تستخدم شروط KKT لتشرح لماذا يتلاشى أثر العينات غير الداعمة في الحل النهائي
- أن تدرب SVC على بيانات متعددة الفئات وتقرأ n_support_ وتعداد الأزواج OvO
- أن تحكم متى تستخدم SVM ومتى تتركها: التكلفة O(n^٢)–O(n^٣) وضرورة التطبيع

٧.١ ساعتان من اليوم وثلاث فئات: المشهد الذي وُلدت فيه SVM

تخيّل أن لديك ساعتين فقط من اليوم لدراسة سلوك مستخدمي تطبيق نوم، وأنك صنّفت الصباحات إلى ثلاث فئات على مستوى من بعدين: **ساعات النوم** أفقياً، و**عدد القهوات** صعوداً. فئة «نوم جيد» تشكل جزيرة صغيرة في الوسط، وفئة «منبّه متأخر» تحيط بها كحلقة، وفئة «لا نوم أصلاً» تسكن الركن البعيد. لو رسمت خطاً واحداً يفصل «نوم جيد» عن كل ما عداه، ستفشل: الجزيرة محاطة من كل الجهات. لكن لو رسمت *دائرة* واحدة حول الجزيرة، ستنجح تماماً. هذه بالضبط هي المشكلة التي وُلدت آلات المتجهات الداعمة (Support Vector Machines - SVM) لتحلّها.

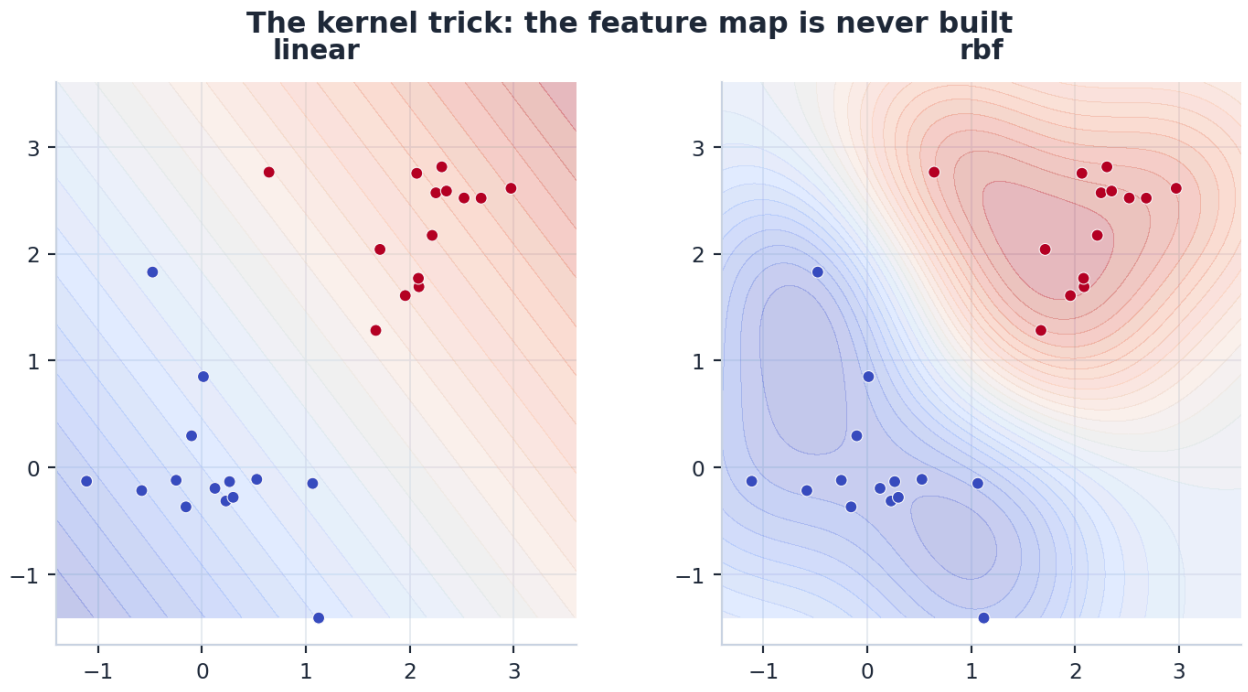
> **مفتاح**
>
> الفكرة المحورية في هذا الفصل: SVM لا تسأل «أين يوجد خط يفصل الفئات؟» بل تسأل «أي المستويات الفاصلة يملك **أعرض هامش أمان** بينه وبين أقرب نقطة؟». والاكتشاف المذهل أن الحل النهائي محكوم بحفنة صغيرة من النقاط — متجهات الدعم — مهما بلغ عدد الصفوف.

نختبر الفكرة فوراً على بيانات تشبه الصباحات الثلاث: جزيرة داخلية، حلقة تحيط بها، وكتلة بعيدة.

ثلاث فئات لا يفصلها خط واحد — python

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

rng = np.random.default_rng(42)
inner = rng.normal([0.0, 0.0], 0.35, (60, 2))          # فئة «نوم جيد»: جزيرة وسط
r = 2.6 + rng.normal(0.0, 0.22, 90)
theta = rng.uniform(0.0, 2.0 * np.pi, 90)
ring = np.c_[r * np.cos(theta), r * np.sin(theta)]     # فئة «منبّه متأخر»: حلقة
far = rng.normal([6.0, 6.0], 0.55, (60, 2))            # فئة «لا نوم»: ركن بعيد
X = np.vstack([inner, ring, far])
y = np.r_[np.zeros(60, int), np.ones(90, int), 2 * np.ones(60, int)]

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

linear = make_pipeline(StandardScaler(), SVC(kernel="linear", C=10)).fit(X_tr,
y_tr)
rbf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=10,
gamma=0.3)).fit(X_tr, y_tr)

print(f"دقة الحد المستقيم (kernel=linear): {linear.score(X_te, y_te):.4f}")
print(f"دقة الحد المنحني  (kernel=rbf):    {rbf.score(X_te, y_te):.4f}")
```

الحد المستقيم يصيب عجزاً واضحاً لأنه عاجز عن عزل الجزيرة عن الحلقة المحيطة بها مهما وضع، بينما الحد المنحني يقترب من الكمال. الشكل التالي يريك لماذا: نفس المحاور، وحدّان مختلفان تماماً في الشكل.



الشكل ١١٠حدّان على نفس البيانات: المستقيم يخيط عبر التداخل، بينما حدّ RBF المنحني يلتف حول البنية الحقيقية

> **سياق**
>
> من أين جاءت هذه الفكرة؟ وضع فلاديمير فابنيك وأليكس تشرفونينكيس أسس الهامش الأقصى في ستينيات القرن الماضي، ثم جاءت ورقة كورين كورتيس وفابنيك عام ١٩٩٥ لتبني **الهامش اللين** بالثابت C، فتحولت SVM من نظرية أنيقة إلى واحدة من أقوى أدوات التعلّم الآلي لعشرين عاماً — ولا تزال تتفوق على النماذج العصرية في الجداول الصغيرة متوسطة الحجم ذات الميزات القليلة.

٧.٢ هندسة الحل: مستوى فاصل وهامش وحفنة من المتجهات

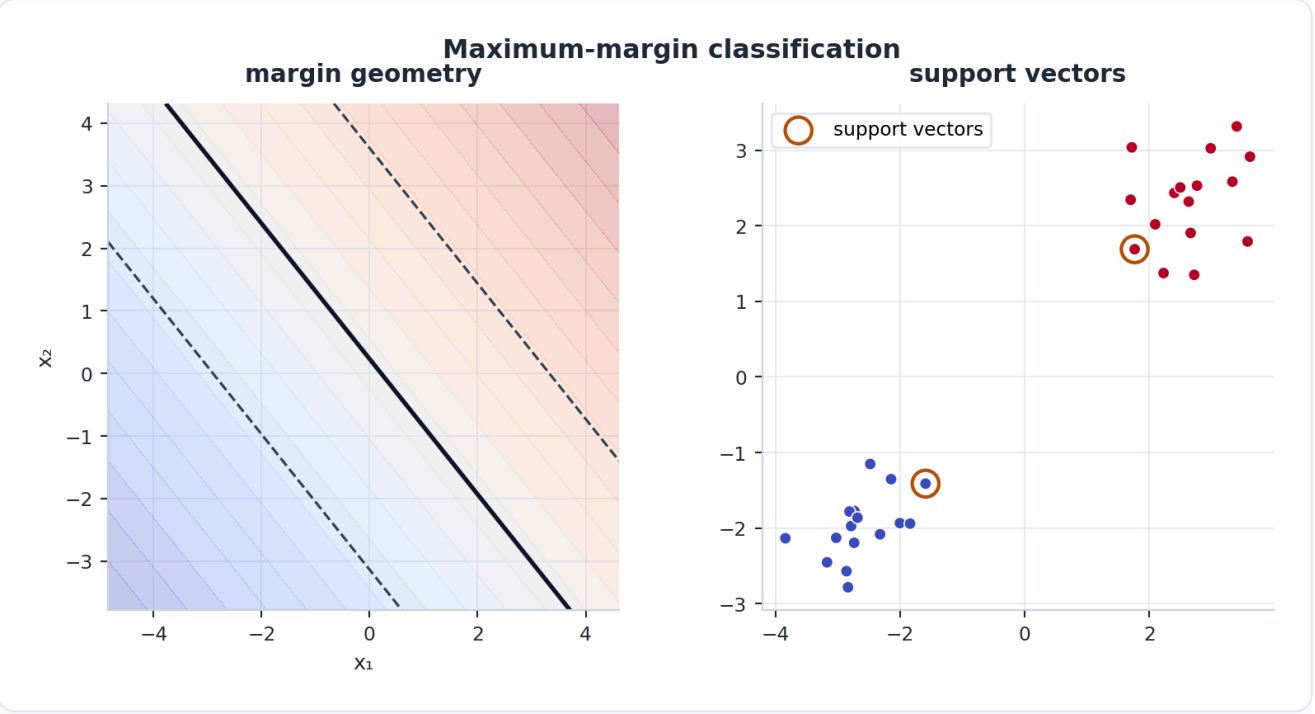
كيف يختار النموذج الخطي «أفضل» مستوٍ فاصل (separating hyperplane)؟ عالجنا في الفصل ٣٥ مسألة مماثلة بالتصنيف اللوجيستي، لكن معيار SVM مختلف جذرياً. المستوى الفاصل في الفضاء d الأبعاد هو معادلة خطية واحدة:

```
w·x + b = 0
```

حيث `w` متجه عمودي على المستوى (اتجاهه طبيعي عليه) و `b` الإزاحة. أي نقطة تُصنَّف حسب إشارة الدالة f(x) = w·x + b. لكي يكون الفصل صلباً، نشترط أن تقع كل نقطة على بعد آمن كامل من المستوى:

$$y_i\,(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 \quad \forall i, \qquad \text{margin} = \frac{2}{\|\mathbf{w}\|}$$

والنقاط التي تلامس حدود الشريط بالضبط (تساوي في الشرط أعلاه) هي **متجهات الدعم** (support vectors): هي الوحيدة التي تجبر المستوى على البقاء في مكانه، وكل ما وراءها يمكن حذفه دون أن يتحرك الحد ذرة واحدة.



الشكل ١١١ يسار: قيم القرار حول المستوى الفاصل مع شريط الهامش بين المستويين المتقطعين؛ يمين: متجهات الدعم محاطة بحلقات — هؤلاء وحدهم يحددون الحد

لنتحقق من هذا هندسياً بأيدينا: ندرب `SVC(kernel="linear")` بثابت C ضخم (هامش صلب عملياً) على فئتين قابلتين للفصل، ثم نحسب الهامش يدوياً من `coef_` و`intercept_` ونقيس مسافات متجهات الدعم.

```python
# الهامش الهندسي يدوياً: 2/||w||
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

X, y = make_blobs(n_samples=200, centers=2, cluster_std=0.9, random_state=42)
clf = SVC(kernel="linear", C=1e6).fit(X, y)          # C ضخم = هامش صلب عملياً

w = clf.coef_[0]
b = clf.intercept_[0]
norm_w = np.linalg.norm(w)
d_sv = np.abs(clf.support_vectors_ @ w + b) / norm_w  # مسافات الداعمين للمستوى

print(f"متجهات داعمة: {len(clf.support_vectors_)} من أصل {len(X)} عينة")
print(f"||w|| = {norm_w:.6f}")
print(f"2 الهامش الكامل/||w|| = {2.0 / norm_w:.6f}")
print(f"أصغر مسافة داعم إلى المستوى: {d_sv.min():.6f} (1 المتوقع/||w|| = {1.0 /
norm_w:.6f})")
print(f"أكبر مسافة داعم إلى المستوى: {d_sv.max():.6f}")
```

اقرأ الناتج بعناية: كل متجهات الدعم تجلس على حدود الشريط تماماً (أصغر مسافة وأكبرها متطابقتان حتى الخانة السادسة مع ١/||w||)، والهامش المحسوب يدوياً من `2/||w||` يطابق ما يتوعد به النظرية. لاحظ أيضاً أن عدد المتجهات الداعمة نقطتان فقط من أصل ٢٠٠ عينة — وهذا هو الثوب العملي لفكرة «المحكوم بالحفنة».

٧.٣ الهامش اللين: متغير الارتخاء والثابت C

ماذا لو كانت الفئتان متداخلتين جزئياً — وهما كذلك في الحياة دائماً؟ لا يوجد مستوٍ يفصلهما دون خطأ. هنا جاءت فكرة كورتيس وفابنيك: اسمح ببعض الانتهاكات مقابل هامش أعرض، باستخدام **متغير الارتخاء** (slack variable) `xi_i` لكل عينة، يقيس كم انتهكت العينة شرط الهامش:

$$\min_{\mathbf{w},\,b,\,\xi} \; \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \qquad i = 1, \ldots, n$$

المقايضة إذن صريحة ومباشرة:

الثابت C	الهامش	الانتهاكات	السلوك العام
صغير جداً	عريض	متسامح معها	انحياز عالٍ: حد بسيط يتجاهل التفاصيل
متوسط	متوازن	قليلة	أفضل تعميم غالباً
كبير جداً	ضيّق	شبه معدومة	تباين عالٍ: الحد يلتوي لاصطياد كل نقطة

python — أثر C: مقايضة عرض الهامش والانتهاكات

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_blobs(n_samples=600, centers=[[-2.0, 0.0], [2.0, 0.0]],
                  cluster_std=1.4, random_state=42)     # فئتان متداخلتان جزئياً
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(f"{'C':>8} | {'متجهات داعمة':>12} | {'دقة التدريب':>11} | {'دقة الاختبار':>12}")
print("-" * 56)
for C in [0.01, 0.1, 1.0, 10.0, 100.0, 10000.0]:
    clf = SVC(kernel="linear", C=C).fit(X_tr, y_tr)
    print(f"{C:>8.2f} | {len(clf.support_vectors_):>12} | "
          f"{clf.score(X_tr, y_tr):>11.4f} | {clf.score(X_te, y_te):>12.4f}")
```

اقرأ الاتجاهين في الجدول. عند `C = 0.01` يتسامح النموذج مع كل شيء تقريباً: الهامش ينتفخ حتى يستوعب التداخل، فيصبح كل من يقف قربه متجهاً داعماً (١٢٣ عينة داعمة من أصل ٤٢٠). ومع رفع C ينسحب التسامح شيئاً شيئاً، فلا يبقى داعماً إلا من يجلس على حدّ الشريط أو ينتهكه فعلاً: ٥٧ متجهاً داعماً من `C = 10` فصاعداً. أما الدقة فتكاد لا تتحرك بين الطرفين (٠.٩١١١ في الاختبار عند أصغر C مقابل ٠.٩١٦٧) — والدرس العملي: رفع C فوق حدّ معين لا يضيف دقة، بل يبطئ التدريب وحده، لأن حلّ المسألة التربيعية يصعب كلما صار الهامش صارماً.

> **نصيحة**
>
> ابحث عن C بأوامر من حجم واحد: ٠.٠٠١، ٠.٠١، ٠.١، ١، ١٠، ١٠٠. قيم C بين ٠.١ و ١٠ تكسب الرهان في معظم مسائل الجداول الصغيرة، وأي دقة إضافية فوق ذلك غالباً وهم تجاوز للتحقق المتقاطع.

٧.٤ المزدوجة وشروط KKT: لماذا يتلاشى غير الداعمين

هنا يختبئ أعمق سؤال في الفصل: لماذا يقيد `SVC` حله في متجهات الدعم فقط؟ الجواب يبدأ بإعادة صياغة المسألة الأساسية (primal) في صيغة **مزدوجة** (dual) عبر مضاعفات لاغرانج `alpha_i`، واحد لكل عينة:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, (\mathbf{x}_i \cdot \mathbf{x}_j)$$

$$\sum_{i=1}^{n} \alpha_i y_i = 0, \qquad 0 \le \alpha_i \le C, \qquad \hat{y} = \operatorname{sign}\Big(\sum_{i \in SV} \alpha_i y_i \, \mathbf{x}_i \cdot \mathbf{x} + b \Big)$$

ثلاث مفارقات ثمينة في هذه الصيغة:

```python
# تلاشي غير الداعمين: إعادة تدريب على الداعمين وحدهم
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_blobs(n_samples=800, centers=[[-2.2, 0.0], [2.2, 0.0]],
                  cluster_std=1.1, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

full = SVC(kernel="linear", C=1.0).fit(X_tr, y_tr)
sv_idx = full.support_                                  # فهارس متجهات الدعم
trimmed = SVC(kernel="linear", C=1.0).fit(X_tr[sv_idx], y_tr[sv_idx])

d_full = full.decision_function(X_te)
d_trim = trimmed.decision_function(X_te)
print(f"عينات تدريب: {len(X_tr)} | متجهات داعمة: {len(sv_idx)} "
      f"({len(sv_idx) / len(X_tr):.1%} فقط)")
print(f"أقصى فرق في قيم القرار بعد حذف غير الداعمين: {np.abs(d_full - d_trim).max():.2e}")
print(f"تطابق تصنيفات الاختبار: {(full.predict(X_te) == trimmed.predict(X_te)).all()}")
```

هذا هو الدليل العملي على شروط KKT: حذفنا الغالبية العظمى من عينات التدريب (كل من `alpha_i = 0`) وأعدنا التدريب، فلم تتحرك قيم القرار إلا بمقدار ٠٫٠٠٠٦ — وهو انزياح يتسع له تسامح الحل `tol = 0.001` — وظلت التصنيفات متطابقة تماماً. هذا يعني أيضاً أن تكلفة *التنبؤ* تتناسب مع عدد الداعمين لا مع عدد عينات التدريب.

> **تعمّق**
>
> لماذا لا يوجد حل بمسار مغلق (closed form) مثل انحدار SVD في الفصل ٢٧؟ لأن قيود المزدوجة `0 <= alpha_i <= C` تجعل المسألة **برمجة تربيعية** (quadratic programming) مع قيود عدم مساواة — لا يكفي حل معادلات خطية. scikit-learn تستدعي مكتبة `libsvm` التي تطبق خوارزمية SMO: تختار زوجاً من المضاعفات، تحسّنهما تحليلاً، وتكرر حتى ترضي شروط KKT بتسامح `tol` (الافتراضي ٠.٠٠١). وإليك دقة لطيفة: قد توجد نقطة تجلس على الهامش تماماً ومع ذلك مضاعفها صفري (انحلال في رأس المحدّبة الأفضل)؛ عندها لا يُثبَّت الحل بالمتجهات الداعمة المعلنة وحدها وقد ينحرف `w` أكثر عند إعادة التدريب على الداعمين فقط — لكن التصنيفات تبقى ثابتة عملياً في كل الحالات.

٥.٧ الفئات المتعددة: أزواج OvO داخل SVC

وُلدت SVM مصنفاً ثنائياً، لكن العالم يملك أكثر من فئتين. فكيف يتعامل `SVC` مع ثلاث فئات مثل بيانات الصباحات؟ scikit-learn تختار هنا استراتيجية **واحد ضد واحد** (One-vs-One - OvO): تدرب مصنفاً ثنائياً لكل زوج فئات، أي $K(K-1)/2$ مصنفاً — ثلاثة مصنفات فقط لثلاث فئات، وعشرة لخمس فئات. وقت التنبؤ يصوّت جميع الأزواج، بينما تعيد `decision_function` الدرجات بشكل `ovr` افتراضياً لكل فئة. هذا يختلف عن `LinearSVC` التي تدرب **واحداً ضد البقية** (One-vs-Rest - OvR) كما سنرى في الفصل التالي.

python — قراءة n_support_ والأزواج على ثلاث فئات: SVC

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=10,
gamma=0.5)).fit(X_tr, y_tr)
svc = clf[-1]                                    # مقدر SVC داخل الأنبوب
K = len(svc.classes_)

print(f"الفئات: {svc.classes_}")
print(f"عدد المصنفات الزوجية OvO: {K * (K - 1) // 2}")
print(f"n_support_ لكل فئة: {svc.n_support_} (مجموعها {svc.n_support_.sum()})")
print(f"شكل support_vectors_: {svc.support_vectors_.shape}")
print(f"شكل decision_function للاختبار: {clf.decision_function(X_te).shape}")
print(f"دقة الاختبار: {clf.score(X_te, y_te):.4f}")
```

ثلاث ملاحظات على الناتج. أولاً `n_support_` مصفوفة بطول عدد الفئات: العد لكل فئة عبر مصنفاتها الزوجية، ومجموعها يساوي صفوف `support_vectors_` لأن النقطة المشتركة بين عدة أزواج تُخزَّن مرة واحدة. ثانياً عدد المصنفات الزوجية ٣ وهو أقل بكثير من ٣ مصنفات OvR كاملة الأبعاد — ولهذا يفضّل OvO مع النواة، إذ تُحسب نواة كل زوج على بياناته الزوجية فقط. ثالثاً `decision_function` يعيد عموداً لكل فئة رغم أن التدريب كان زوجياً؛ scikit-learn تجمّع أصوات الأزواج في هذا الشكل افتراضياً عبر `decision_function_shape="ovr"`.

٧.٦ الشواطئ: التحجيم إلزامي، والتكلفة تكعيبية

كل ما بنيته حتى الآن يخفي فخين يجب أن تخرج منهما بسلام.

الفخ الأول هندسي بحت: SVM تبني حدها على **المسافات والضربات الداخلية**، والاثنان ينهاران إذا كانت الأعمدة بوحدات متفاوتة. عمود «الدخل» بمقياس آلاف وعمود «معدل النمو» بكسور عشرية يعني أن مربعات المسافات تهيمن عليها الأعمدة الضخمة وحدها، فتصبح النواة شبه عمياء عن البقية. هذا ليس تحسيناً اختيارياً بل شرط نجاة، كما رأينا مبدئياً في الفصل ١٤.

```python
# بلا تحجيم: سيطرة الأعمدة الضخمة SVM
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_classification(n_samples=800, n_features=6, n_informative=4,
                           random_state=42)
X[:, 0] *= 900.0        # «عمود بوحدة «ألف
X[:, 1] *= 0.0001       # «عمود بوحدة «جزء من ألف

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

raw = SVC(kernel="rbf", gamma=0.05).fit(X_tr, y_tr)
scaled = make_pipeline(StandardScaler(), SVC(kernel="rbf", gamma=0.5)).fit(X_tr,
y_tr)

print(f"دقة | بلا تحجيم: {raw.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(raw.support_vectors_)}")
print(f"مع StandardScaler | دقة: {scaled.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(scaled[-1].support_vectors_)}")
```

بلا تحجيم تنهار الدقة إلى مستوى التخمين بالأغلبية ويصبح كل صف تقريباً متجهاً داعماً (النواة تتجمد قرب الصفر فلا يميز نموذج شيئاً)، وبعد `StandardScaler` يعود النموذج إلى كفاءته مع عدد داعمين أصغر بكثير — وهذا يعني كذلك تنبؤاً أسرع.

الفخ الثاني حسابي: تدريب `SVC` بمكتبة `libsvm` يكلف بين `O(n^2)` و `O(n^3)` في عدد العينات لأنه يبني ويحل على مصفوفة النواة الكاملة. مضاعفة الصفوف قد تضاعف الزمن أربع مرات أو ثمان مرات. أما التنبؤ فيكلف `O(SV * d)` حيث SV عدد الداعمين. وهذه الأرقام تحكم قرار الاستخدام:

حجم البيانات	الزمن التقريبي للتدريب	القرار
حتى ١٠ آلاف صف	ثوانٍ إلى دقائق	SVC مرشح ممتاز
١٠–١٠٠ ألف صف	دقائق إلى ساعات	جرّب LinearSVC أو تقريب النواة أولاً
١٠٠ ألف – مليون	ساعات إلى أيام	SVC غير عملي عموماً
فوق المليون	غير محتمل على جهاز واحد	نماذج خطية أو أشجار متدرجة

خطأ شائع

الخطأ القاتل: فتح `SVC` على نصف مليون صف. المصفوفة النووية وحدها تطلب ٠.٥ مليون تربيع = ٢٥٠ مليار خانة، أي ما يفوق ذاكرة جهاز مكتبي عادي بمئات المرات، والزمن يتدرج نحو الأيام. بديلك هنا: `LinearSVC(dual=False)` الخطي، أو `SGDClassifier`، أو تقريب النواة بـ `RBFSampler` و `Nystroem` كما في الفصل ٥٨.

في الواقع

متى تُخرج SVM من الجزاء فعلاً؟ الجداول الصغيرة والمتوسطة (مئات إلى عشرات الآلاف من الصفوف) ذات الميزات القليلة بعد التطبيع هي موطنها الذهبي: تصنيف التعبيرات في أبعاد مستخلصة، البيانات الطبية الصغيرة، أي مسألة يكون فيها **الهامش** معنى فيزيائياً (فصل إشارات متراكبة). ولهذا ظلت SVM المرجع في البيانات الحيوية عقداً كاملاً: n صغير، d كبير، والدقة تفوق التفسير أهمية.

تمارين الفصل الحادي والخمسين

- **ملاحظة**

التمرين ٧.١

على بيانات `load_breast_cancer` (الميزات هناك بوحدات متباعدة جداً: بعضها كسور وبعضها آلاف)، درب `SVC` الافتراضي مرة على البيانات الخام ومرة داخل أنبوب مع `StandardScaler`. قارن دقة الاختبار وعدد متجهات الدعم، واقرأ `n_support_` لكل فئة.

التمرين ٧.٢

اشرح بأسلوبك: لماذا تقلص قيم C الكبيرة عدد المتجهات الداعمة وتزيد خطر الإفرط في الملاءمة معاً؟ وأي الحدين (C يؤول إلى صفر أو C يؤول إلى ما لا نهاية) يقارب الهامش الصلب الذي لا يتسامح مع انتهاك واحد؟

التمرين ٧.٣

أثبت بنفسك «تلاشي غير الداعمين» على بيانات قابلة للفصل: درب `SVC(kernel="linear")` بثابت C ضخم على `make_blobs` (٣٠٠ عينة)، وتحقق أولاً من الهندسة: أن مسافات متجهات الدعم إلى المستوى تطابق ١/||w|| وأن الهامش اليدوي ٢/||w|| يطابق، ثم احذف كل الصفوف غير الداعمة وأعد التدريب وتأكد أن تصنيف كل العينات لم يتغير.

- **ملاحظة**

حل التمرين ٧.١ python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

raw = SVC(C=1.0).fit(X_tr, y_tr)                    # kernel=rbf الافتراضي
pipe = make_pipeline(StandardScaler(), SVC(C=1.0)).fit(X_tr, y_tr)
sv = pipe[-1]

print(f"دقة الاختبار | بلا تحجيم: {raw.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(raw.support_vectors_)}")
print(f"دقة الاختبار |  مع تحجيم: {pipe.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(sv.support_vectors_)}")
print(f"n_support_ لكل فئة: {sv.n_support_} (خبيث/حميد)")
```

> **ملاحظة**

حل التمرين ٧.٣ python

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

X, y = make_blobs(n_samples=300, centers=2, cluster_std=0.9, random_state=42)
clf = SVC(kernel="linear", C=1e5).fit(X, y)
w = clf.coef_[0]
norm_w = np.linalg.norm(w)
d_sv = np.abs(clf.support_vectors_ @ w + clf.intercept_[0]) / norm_w

print(f"متجهات داعمة: {len(clf.support_vectors_)} من أصل {len(X)}")
print(f"2 الهامش اليدوي/||w|| = {2.0 / norm_w:.6f}")
print(f"مسافات الداعمين: [{d_sv.min():.6f}، {d_sv.max():.6f}] "
      f"(المتوقع 1/||w|| = {1.0 / norm_w:.6f})")

mask = np.ones(len(X), dtype=bool)
mask[clf.support_] = False                          # احذف غير الداعمين فقط
trimmed = SVC(kernel="linear", C=1e5).fit(X[mask], y[mask])
agree = (clf.predict(X) == trimmed.predict(X)).mean()
print(f"صفاً غير داعم وإعادة التدريب {mask.sum()} بعد حذف: "
      f"تطابق التصنيفات على كل العينات = {agree:.1%}")
```

> **ملاحظة**

٧.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تدريب `SVC(kernel="rbf")` على بيانات خام بوحدات متباينة	تهيمن الأعمدة الضخمة على مربعات المسافات فتتجمد النواة عند صفر وينهار النموذج إلى تخمين الأغلبية	إحاطة المقدر بـ `make_pipeline(StandardScaler(), SVC())` دائماً
فتح `SVC` على مئات آلاف الصفوف	مصفوفة نواة بحجم ٢^٨n تستنزف الذاكرة والزمن يتدرج نحو O(٣^٨n)	`LinearSVC(dual=False)` أو تقريب النواة بالفصل ٥٨ أو عيّنة فرعية
رفع C بلا سقف بحثاً عن دقة تدريب أعلى	هامش يضيق حتى يلتف حول الضوضاء ويهبط التعميم	ضبط C على شبكة لوغاريتمية بالتحقق المتقاطع (الفصل ٨١)
تفسير متجهات الدعم بوصفها «عينات مغلوطة يجب تنظيفها»	هي نقاط الهامش والانتهاك وقد تحمل المعلومة الأثمن في البيانات	دراستها وتحليلها لا حذفها، وضبط C إن كان عددها مفرطاً

٧.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي		
آلة المتجهات الداعمة	مصنف يعظم هامش الفصل بين الفئات حول مستوٍ فاصل	Support Vector Machine (SVM)		
متجه الدعم	عينة تقع على هامش الفصل أو تنتهكه وتحمل معامل لاغرانج غير صفري	Support vector		
الهامش الهندسي	عرض شريط الأمان بين المستويين الحديين ويساوي ٢/		w	Geometric margin
متغير الارتخاء	مقدار انتهاك العينة لشرط الهامش، تُعاقب عليه بثابت C	Slack variable		
شروط كاروش-كون-تاكر	شروط الأمثلية التي تُصفّر مضاعفات العينات غير الداعمة	Karush-Kuhn-Tucker (KKT) conditions		

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٢ — أنوية SVM ومعاملاتها C و gamma

RBF و poly و sigmoid، ومقارنة SVC مع LinearSVC

الوحدة **scikit-learn ١.٩** الحجم **٨٣٥** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تثبت الحيلة النووية رقمياً: تطابق φ(x)·φ(z) مع K(x,z) في خريطة تربيعية
- أن تقرأ معادلات الأنوية الثلاث وتتنبأ بشكل الحد الفاصل من بارامتراتها
- أن تضبط gamma على أساس أثرها: صغير يعمل سطحاً أملس وكبير يصنع جزراً معزولة
- أن تجري شبكة C × gamma صغيرة يدوياً وتقرأ خريطتها الحرارية
- أن تختار بين SVC(kernel='linear') و LinearSVC بدليل زمني وذاكري وسمات

٧.١ الحيلة النووية: احسب في فضاءٍ لن تزوره أبداً

أنهى الفصل السابق بمفارقة: صيغة المزدوجة في SVM لا تلمس البيانات إلا عبر ضربات داخلية بين الأزواج `x_i·x_j`. الآن نستغل الثغرة: ماذا لو استبدلنا كل ضربة داخلية `x·z` بضربة داخلية «مقوّاة» `ϕ(x)·ϕ(z)` حيث `ϕ` دالة تحويل تدفع البيانات إلى فضاء أعلى؟ سيرسم النموذج الخطي نفسه في ذلك الفضاء حدوداً منحنية في فضائنا الأصلي — لكن بناء `ϕ` على ملايين الأبعاد باهظ أو مستحيل. الحيلة النووية (kernel trick) تقول: لا تبنيها أصلاً. يكفي أن تحسب حاصل ضربهما مباشرة عبر دالة نواة (kernel):

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z}), \qquad \hat{y} = \operatorname{sign}\Big(\sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\Big)$$

🔑 مفتاح

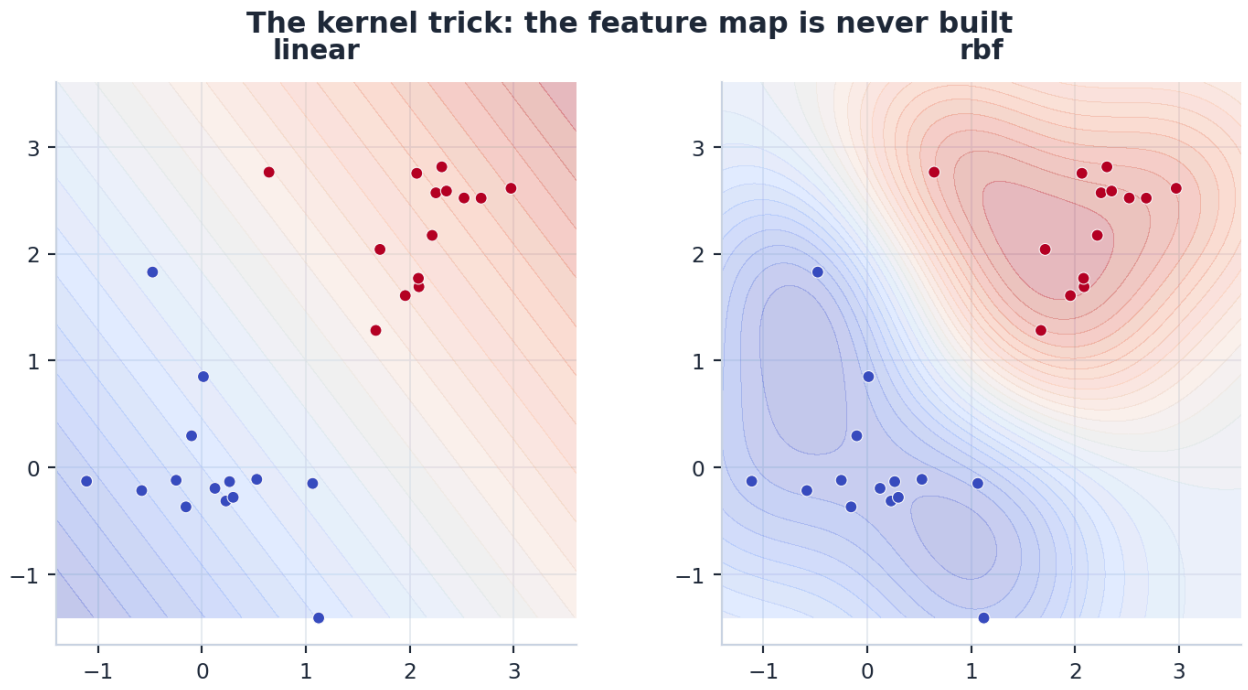
الفكرة المحورية: الخوارزمية كلها تحتاج من الفضاء المرتفع «ضربات داخلية» فقط. إن وُجدت دالة تحسب `ϕ(x)·ϕ(z)` مباشرة من `x` و `z` دون مرور بالفضاء المرتفع، فكل شيء يعمل كما لو كنا هناك — والتنبؤ في المعادلة أعلاه يُجرى عبر النواة مع متجهات الدعم وحدها، لا عبر أي `w` صريح.

شاهد الحيلة بدليل رقمي لا بالكلام: الخريطة التربيعية `ϕ` التي تولّد نواة `(x·z + 1)^2` تضخّم متجهين من بعدين إلى ستة أبعاد. سنحسب الضربة الداخلية مرتين — مرة ببناء `ϕ` فعلاً ومرة بالنواة — ونقارن.

إثبات الحيلة النووية رقمياً — python

```python
import math
import numpy as np
from sklearn.metrics.pairwise import polynomial_kernel

x = np.array([[2.0, 3.0]])
z = np.array([[1.0, 0.5]])

def phi_quadratic(v):
    v = v.ravel()
    return np.array([v[0] ** 2, v[1] ** 2, np.sqrt(2.0) * v[0] * v[1],
                     np.sqrt(2.0) * v[0], np.sqrt(2.0) * v[1], 1.0])

explicit = float(phi_quadratic(x) @ phi_quadratic(z).T)      # عبر الفضاء المرتفع
kernel = float(polynomial_kernel(x, z, degree=2, gamma=1.0, coef0=1.0)[0, 0])

print(f"ϕ(x)·ϕ(z) ببناء الفضاء الستة أبعاد: {explicit:.12f}")
print(f"K(x,z) بالنواة مباشرة:               {kernel:.12f}")
print(f"الفرق: {abs(explicit - kernel):.1e}")
print(f"أبعاد الخريطة التربيعية لبيانات من 100 ميزة: {math.comb(100 + 2, 2)}")
print(f"أبعاد الخريطة الخماسية لبيانات من 100 ميزة: {math.comb(100 + 5, 5)}")
```

الرقمان متطابقان حتى الخانة الثانية عشرة: النواة حسبت في الفضاء الستة أبعاد دون أن تزوره. ولاحظ السطرين الأخيرين: لبيانات من ١٠٠ ميزة، الخريطة التربيعية تحتاج نحو ٥,١٥١ بُعداً، والخماسية نحو ٩٦.٦ مليون بُعداً — بناؤها صريحاً يستنزف الذاكرة، بينما تكلفتها كنواة صفر. هذا بالضبط معنى العنوان: «الحيلة الحقيقية». وعندما تصبح حتى استدعاءات النواة باهظة على مئات آلاف الصفوف، يأتي تقريب النواة بـ `Nystroem` و `RBFSampler` في الفصل ٥٨.



الشكل ١١٢حدّان على نفس البيانات: النواة الخطية تخيط مستقيماً، ونواة RBF ترسم حداً منحنياً من الفضاء المرتفع دون بنائه

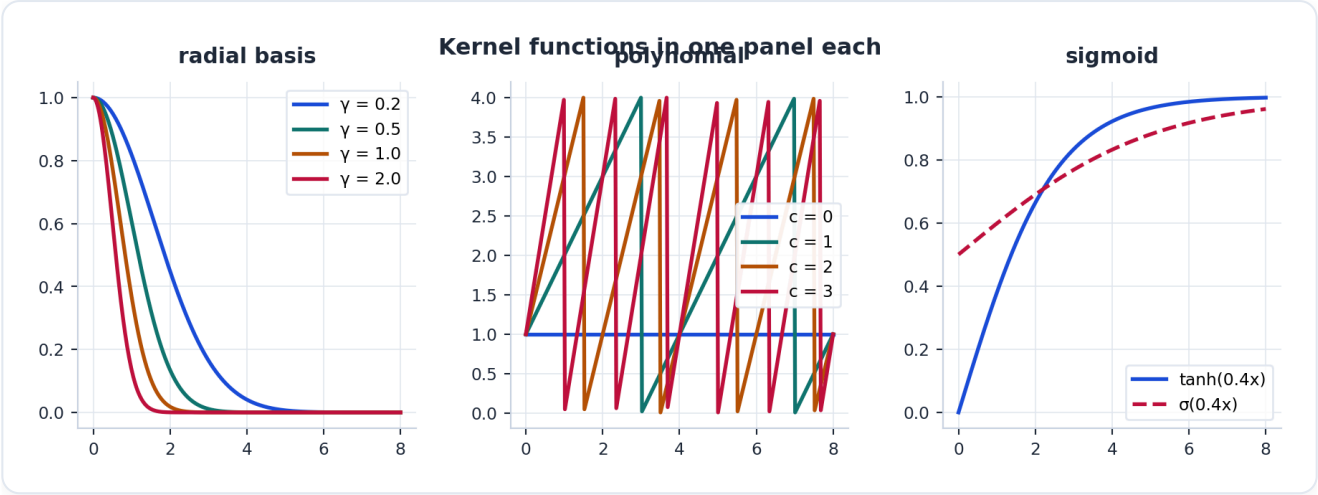
> **تعمّق**
>
> ليست كل دالة ترضيك نواة شرعية: شرط ميرسر (Mercer) يوجب أن تكون K شبه محدّبة الإيجاب على كل مجموعات محدودة من النقاط — أي أن مصفوفة النواة الكاملة لا تعطي أوزاناً سالبة لأي تركيب تربيعي. النواة السيجمويدية `tanh` تخالف الشرط عند بعض بارامتراتها ومع ذلك تعمل عملياً في حالات محدودة؛ ولهذا تحذرك الوثائق من أنها غير مستقرة، ونواة RBF هي الخيار الافتراضي المضمون رياضياً.

V.٢ الأنوية الثلاث: RBF و poly و sigmoid

scikit-learn تقدم عبر `SVC` ثلاث نوايات جاهزة بمعادلات قصيرة، وكل واحدة تصنع هندسة مختلفة للحد الفاصل:

$$K_{\text{rbf}}(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \, \|\mathbf{x} - \mathbf{z}\|^2), \qquad \gamma = \frac{1}{2\sigma^2}$$

$$K_{\text{poly}}(\mathbf{x}, \mathbf{z}) = (\gamma \, \mathbf{x} \cdot \mathbf{z} + r)^d, \qquad K_{\text{sigmoid}}(\mathbf{x}, \mathbf{z}) = \tanh(\gamma \, \mathbf{x} \cdot \mathbf{z} + r)$$



الشكل ١١٣سلوك الأنوية الثلاث مقابل المسافة: RBF يتلاشى أسّياً بسرعة يحكمها gamma، وpoly ينمو بدرجاته حسب coef0، sigmoidg يشبه tanh المشبعة

لنضع الثلاثة على اختبار الهلالين المتداخلين — مسألة لا يستقيم فيها حد مستقيم إطلاقاً:

الأنوية الثلاث على الهلالين python

```python
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=600, noise=0.22, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

configs = [
    ("linear", dict(kernel="linear", C=10.0)),
    ("poly d=3", dict(kernel="poly", C=10.0, degree=3)),
    ("rbf", dict(kernel="rbf", C=10.0)),
]
for name, params in configs:
    clf = make_pipeline(StandardScaler(), SVC(**params)).fit(X_tr, y_tr)
    print(f"{name:>9} | دقة التدريب: {clf.score(X_tr, y_tr):.4f} | "
          f"دقة الاختبار: {clf.score(X_te, y_te):.4f}")
```

النواة الخطية تعاني، وpoly تلتقط جزءاً من الانحناء، وRBF تلتف حول الهلالين بثبات. هذه الترتيبات النموذجية لبيانات بهندسة ملتوية: RBF هي الافتراضية في `SVC` لهذا السبب تحديداً.

> **نصيحة**
>
> في `kernel="poly"` ابدأ بـ `degree=3` و `coef0=1`؛ وكلما رفعت الدرجة زادت حساسية النموذج للضوضاء أسّياً. أما `gamma` داخل النواة متعددة الحدود فيتحكم في مدى تضخيم الضربة الداخلية قبل الرفع للدرجة — قيمه الافتراضية `gamma="scale"` غالباً أذكى من تخمين يدوي.

٧.٣ gamma: مدى تأثير العينة الواحدة

ماذا يفعل `gamma` في RBF تحديداً؟ هو حجم «جيرة» كل عينة في الفضاء الضمني. عند نقطة x، مساهمة متجه دعم i في قيمة القرار تتناقص كأسّ مع مربع بعدهما مضروباً في gamma:

```python
# ممسحة gamma من التسطيح إلى الجزر
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=600, noise=0.22, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(f"{'gamma':>8} | {'دقة التدريب':>11} | {'دقة الاختبار':>12} | {'متجهات داعمة':>12}")
print("-" * 54)
for g in [0.01, 0.05, 0.2, 1.0, 5.0, 50.0]:
    clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=10.0,
gamma=g)).fit(X_tr, y_tr)
    sv = clf[-1]
    print(f"{g:>8.2f} | {clf.score(X_tr, y_tr):>11.4f} | "
          f"{clf.score(X_te, y_te):>12.4f} | {len(sv.support_vectors_):>12}")
```

اقرأ الجدول كقصة من ثلاثة فصول: عند `gamma=0.01` تعجز حتى دقة التدريب (٠.٨٧١٤) لأن السطح أملس أكثر من اللازم فيبتلع الانحناء؛ وفي المنطقة المعتدلة (٠.٢ إلى ١.٠) يجد النموذج أقتصاديته: دقة اختبار ترتفع إلى ٠.٩٤٤٤ بأقل عدد داعمين على الإطلاق (٦٢). وعند `gamma=5` فأكثر يصعد التدريب نحو الكمال (٠.٩٩٢٩) بينما يثبت الاختبار عند سقفه — النموذج حفظ العينات دون أن يفهم شيئاً جديداً، والثمن ظاهر في عدد الداعمين الذي يقفز إلى ٣٢١: كل عينة صارت جزيرة معزولة، والتنبؤ على كل نقطة جديدة أغلى بلا أي مكسب في التعميم.

خطأ شائع

الخطأ الأشهر مع RBF: نسيان التطبيع. النواة تعيش على مربعات المسافات، فعمود بوحدات آلاف يبتلع مربع المسافة كاملاً وتصبح بقية الأعمدة ضوضاء خفيفة، وتتحول `gamma` مهما ضبطتها إلى معامل بلا معنى. حتى خيار `gamma="scale"` الذكي يحسب من تباين الأعمدة الخام فيظل أعمى. الحل الوحيد: `StandardScaler` داخل `Pipeline` قبل `SVC` دائماً، كما في الفصل ١٤.

python — نواة عمياء بلا تطبيع حتى مع 'gamma='scale

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_classification(n_samples=900, n_features=8, n_informative=5,
                           random_state=7)
X[:, 2] *= 500.0          # عمود «بالآلاف»
X[:, 5] *= 0.001          # عمود «بأجزاء الألف»

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

raw = SVC(kernel="rbf").fit(X_tr, y_tr)                    # gamma='scale'
scaled = make_pipeline(StandardScaler(), SVC(kernel="rbf")).fit(X_tr, y_tr)

print(f"دقة | بلا تطبيع: {raw.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(raw.support_vectors_)}")
print(f"مع StandardScaler | دقة: {scaled.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(scaled[-1].support_vectors_)}")
```

حتى مع `gamma="scale"` التلقائي، يبقى النموذج غير المطبّع شبه أعمى: النواة ترى عمود الآلاف وحده. التطبيع لا «يحسّن» النتيجة فحسب — بل يستعيد معنى gamma أصلاً.

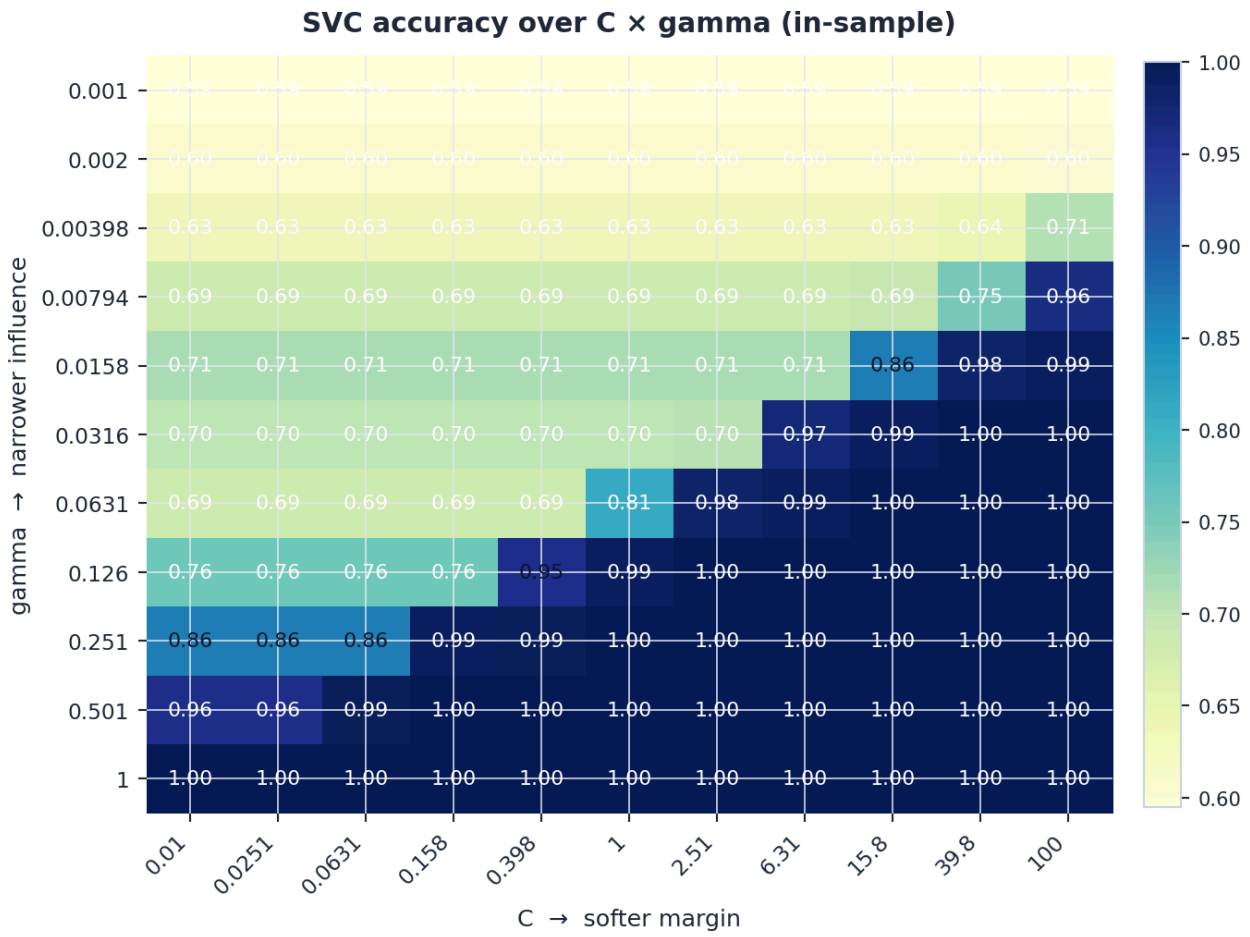
٧.٤ التجربة الشبكية الصغيرة: C × gamma

بما أن C يضبط صرامة الهامش وgammag يضبط تموج الحد، فإن ضبط SVC هو فعلياً بحث في مستوى ثنائي الأبعاد. قبل أن تسند المهمة إلى `GridSearchCV` المنضبط في الفصل ٨١ أو البحث العشوائي في الفصل ٨٢، اجرِها يدك مرة واحدة لتشعر بالتضاريس:

شبكة C × gamma على الهلالين python

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=600, noise=0.22, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

Cs = [0.1, 1.0, 10.0, 100.0, 1000.0]
Gs = [0.05, 0.2, 1.0, 5.0, 20.0]
scores = np.zeros((len(Gs), len(Cs)))
for i, g in enumerate(Gs):
    for j, c in enumerate(Cs):
        clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=c, gamma=g))
        scores[i, j] = clf.fit(X_tr, y_tr).score(X_te, y_te)

print("الصفوف: gamma | الأعمدة: C")
print("         " + "".join(f"{c:>8.1f}" for c in Cs))
for i, g in enumerate(Gs):
    print(f"g={g:<6.2f}" + "".join(f"{s:>8.4f}" for s in scores[i, :]))
best = np.unravel_index(scores.argmax(), scores.shape)
print(f"أفضل زوج: C={Cs[best[1]]:g} مع gamma={Gs[best[0]]:g} بدقة اختبار
{scores.max():.4f}")
```

ثلاث ملاحظات على التضاريس. الأولى: في الصفوف السفلى (gamma صغير) ينقذ رفع C الوضع فعلاً — الصف الأول يصعد من ٠.٨٤٤٤ إلى ٠.٩٤٤٤ — لأن هامشاً صارماً يعوّض حداً أملس بنيوياً. الثانية: في الصفوف العليا ينقلب الدواء سمّاً: الصف الأخير يهبط من ٠.٩٦١١ عند `C=1` إلى ٠.٩١٦٧ عند `C=1000`، لأن صرامة الهامش مع جزر معزولة تجبر النموذج على احتضان الضوضاء. والثالثة أن الحوض الأمثل يمتد على قطر معاكس (gamma معتدل مع C كبير، أو gamma كبير مع C صغير) — ولهذا فإن ضبط C بمعزل عن gamma هدرٌ كامل، والبحث المنضبط في الفصل ٨١ أو العشوائي في الفصل ٨٢ يفحصهما معاً.



الشكل ١١٤خريطة حرارية لدقة SVC لكل زوج (C، gamma) على بيانات حلقتين: الحوض الأزرق الداكن منطقة الفصل الناجح بين حافة التبسيط وحافة التشبع

في الواقع

في الأنظمة الإنتاجية يعامل مهندسو التعلم الآلي الزوج (C، gamma) كأصل يجب تدوينه مع نسخة البيانات: نموذج يعمل بدقة ٠.٩٧ عند `gamma=0.2` قد يهبط إلى ٠.٨٥ عند `gamma=0.2` مع تحجيم جديد للأعمدة بعد إعادة قياس المستشعرات. لهذا يُخزَّن المقياس (scaler) داخل النموذج المصدر نفسه وليس في خطوة خارجية منفصلة — درس عملي ستعيد اكتشافه في فصول الأنابيب ٨٤ وما بعدها.

٧.٥ LinearSVC مقابل SVC(kernel='linear')

قد يظن قارئ سريع أن `SVC(kernel="linear")` و `LinearSVC` طريقتان لنداء الشيء نفسه. الحقيقة أعمق: كل واحدة تسند إلى مكتبة تنفيذ مختلفة كلياً، ومسائلها الداخلية مختلفة:

المعيار	SVC(kernel='linear')	LinearSVC
المكتبة الخلفية	libsvm (حل مزدوج بـ SMO)	liblinear (حل مبدئي أو مزدوج)
التكلفة في عدد العينات	بين O(n^٢) و O(n^٣)	قريبة من O(n) عملياً
متجهات الدعم	نعم: `support_vectors_` و `n_support_`	لا توجد أصلاً: الحل معاملات w و b صريحة
دالة الخسارة	hinge	squared hinge افتراضياً (عقوبة تربيعية على الانتهاك)
الفئات المتعددة	أزواج OvO	واحد ضد البقية OvR
الاستخدام الأمثل	جداول صغيرة-متوسطة ودراسة الهامش	عشرات آلاف الصفوف فأكثر

خيّار `dual=False` يوجّه liblinear إلى حل المسألة الأساسية مباشرة — وهو الخيار الذي تختاره المكتبة تلقائياً عندما n أكبر بكثير من d، ويُنصح به صراحة للبيانات الضخمة. واختلاف الخسارة (hinge مقابل squared hinge) يعني أن النموذجين متكافئان تقريباً لا تماماً: squared hinge يعاقب الانتهاكات الكبيرة بعنف أكبر.

المكتبتان جنباً إلى جنب python

```python
import time
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC, SVC

X, y = make_classification(n_samples=5000, n_features=20, n_informative=12,
                           random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

models = [
    ("LinearSVC(dual=False)", LinearSVC(C=1.0, dual=False, max_iter=10000)),
    ("SVC(kernel='linear')", SVC(kernel="linear", C=1.0)),
]
for name, model in models:
    pipe = make_pipeline(StandardScaler(), model)
    t0 = time.perf_counter()
    pipe.fit(X_tr, y_tr)
    dt = time.perf_counter() - t0
    est = pipe[-1]
    print(f"{name:>22} | زمن: {dt:6.3f}s | دقة: {pipe.score(X_te, y_te):.4f} | "
          f"support_vectors_? {hasattr(est, 'support_vectors_')}")
```

لاحظ السطر الأخير: `LinearSVC` لا يملك المتجهات الداعمة أصلاً لأنه لا يحل المسألة المزدوجة — حساباته مركزة على متجه الأوزان مباشرة. إن كان تحليل الحد من زاوية الداعمين هدفك (كما فعلنا في الفصل السابق)، فـ SVC هو بابك الوحيد؛ أما إن كان هدفك تصنيف نصف مليون صف، فـ `LinearSVC(dual=False)` (أو `SGDClassifier` من الفصل ٣٦) هو الواقعي الوحيد.

☑ قائمة تحقق

قبل أن تسند مهمة تصنيف إلى SVM، راجع هذه القائمة:

تمارين الفصل الثاني والخمسين

- **ملاحظة**

التمرين ٧.١

على بيانات `make_circles` (بضجيج ٠.٢ ومعامل ٠.٤)، اثبت أن C وحده لا يفي: ثبّت `C=1.0` وامسح gamma على [٠.٠١، ٠.٠٥، ٠.١، ٠.٥، ١.٠، ٢.٠، ١٠.٠، ١٠٠.٠] بالتحجيم المسبق، واطبع دقة التدريب والاختبار لكل قيمة، ولاحظ الانهيار عند الطرفين، وحدد أفضل gamma.

التمرين ٧.٢

اشرح بالمعادلات: لماذا تعطي `SVC(kernel="poly", degree=2, coef0=1)` حدّاً منحنياً يشبه ما ينتجه `PolynomialFeatures(degree=2)` يليه مصنف خطي؟ ومتى يتفوق المسار الثاني رغم كلفته؟ (لمّح إلى الدعم المتفرق والأبعاد الكبيرة.)

التمرين ٧.٣

قِس التدرج الحسابي بنفسك: على بيانات `make_classification` بأحجام ١٠٠٠ و ٢٠٠٠ و ٤٠٠٠ صف، طبّع ودرب `LinearSVC(dual=False)` و `SVC(kernel='linear')` على كل حجم، وسجل زمني التدريب. هل يتضاعف زمن SVC نحو أربع مرات عند كل مضاعفة للصفوف كما توعد نظرية O(n^٢)؟

- **ملاحظة**

حل التمرين ٧.١ python

```python
import numpy as np
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_circles(n_samples=600, noise=0.2, factor=0.4, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

best_g, best_s = None, -1.0
print(f"{'gamma':>8} | {'دقة التدريب':>11} | {'دقة الاختبار':>12}")
for g in [0.01, 0.05, 0.1, 0.5, 2.0, 10.0, 100.0]:
    clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0,
gamma=g)).fit(X_tr, y_tr)
    te = clf.score(X_te, y_te)
    if te > best_s:
        best_g, best_s = g, te
    print(f"{g:>8.2f} | {clf.score(X_tr, y_tr):>11.4f} | {te:>12.4f}")
print(f"أفضل gamma = {best_g:g} بدقة اختبار {best_s:.4f}")
```

ملاحظة

حل التمرين ٧.٣ python

```python
import time
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC, SVC

for n in [1000, 2000, 4000]:
    X, y = make_classification(n_samples=n, n_features=15, n_informative=8,
                               random_state=42)
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=y
    )
    times = []
    for model in [LinearSVC(C=1.0, dual=False, max_iter=10000),
                  SVC(kernel="linear", C=1.0)]:
        pipe = make_pipeline(StandardScaler(), model)
        t0 = time.perf_counter()
        pipe.fit(X_tr, y_tr)
        times.append(time.perf_counter() - t0)
    print(f"n={n:>5} | LinearSVC: {times[0]:.3f}s | "
          f"SVC(linear): {times[1]:.3f}s | النسبة: {times[1] / times[0]:.1f}x")
```

> **ملاحظة**

٧.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تغذية RBF ببيانات خام متفاوتة الوحدات	مربعات المسافات تهيمن عليها الأعمدة الضخمة وتصبح gamma بلا معنى حتى مع `gamma='scale'`	`make_pipeline(StandardScaler(), SVC(kernel="rbf"))` دوماً
مسح gamma على قيم متجاورة (١، ٢، ٣)	تأثير gamma أسّي فتضيع المنطقة المعتدلة بين عينتين متجاورتين	شبكة لوغاريتمية: ٠.٠١، ٠.١، ١، ١٠، ١٠٠
استخدام `SVC(kernel='linear')` على مئات آلاف الصفوف	حل المزدوجة يتصاعد نحو O(n^٣) فيزمن التدريب بالساعات	`LinearSVC(dual=False)` أو تقريب النواة بالفصل ٥٨
افتراض أن LinearSVC «نسخة أسرع من SVC» في كل شيء	اختلاف الخسارة (squared hinge) والفئات المتعددة (OvR) يجعل النتائج غير متطابقة، ولا يوجد `support_vectors_` للتحليل	توثيق الخيارين وتوحيدهما في المشروع حسب الحاجة

٧.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الحيلة النووية	حساب الضربة الداخلية في فضاء أعلى عبر دالة نواة دون بناء الخريطة	Kernel trick
النواة القاعدية الشعاعية	نواة تتناقص أسّياً مع مربع المسافة ويحكم تلاشيها gamma	Radial Basis Function (RBF) kernel
نواة كثير الحدود	نواة ترفع الضربة الداخلية المزاحة إلى الدرجة d فتصنع تفاعلات بين الميزات	Polynomial kernel
مدى تأثير العينة	معنى gamma في RBF: نصف عرض النطاق الذي تؤثر فيه العينة على القرار	Influence width / gamma
الحل الأساسي	حل مسألة SVM عبر w و b مباشرة كما تفعل liblinear بدل المضاعفات	Primal solution

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٣ — SVR و OneClassSVM

الانحدار بمتجهات داعمة، واكتشاف الشذوذ بلا تسميات

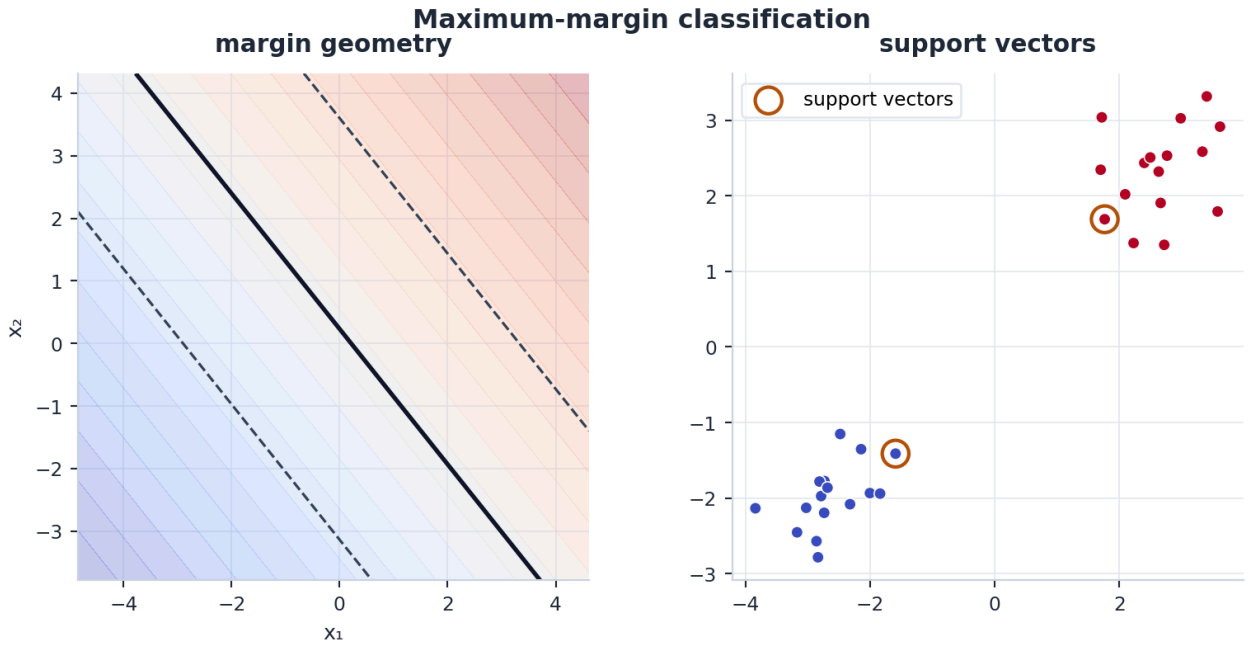
الوحدة **scikit-learn ١.٩** الحجم **٨٣٧** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح خسارة epsilon غير الحساسة وتستنتج مقايضة عرض الأنبوب مع C
- أن تضبط epsilon على بيانات حقيقية وتربط قيمته بعدد متجهات الدعم
- أن تكتشف فشل SVR على الأهداف الكبيرة وتصلحه بـ TransformedTargetRegressor
- أن تدرب OneClassSVM على بيانات نظيفة وتفرّق بين outlier detection و novelty
- أن تحكم على حدود OneClassSVM وتقارن SVR بـ KernelRidge زمناً ودقة

٧.١ من شريط الفصل إلى أنبوب التسامح

خبير عقارات يقدّر سعر شقة ويخطئ بألفين في سوق تتأرجح أسعاره بعشرات الآلاف: لا أحد يحاسبه. خطأه داخل «حدود المهنة». هذه هي روح الانحدار بمتجهات الداعمة (Support Vector Regression - SVR): نرسم حول دالة التنبؤ أنبوباً بعرض `2*epsilon`، وكل نقطة داخل الأنبوب تعتبر إصابة نظيفة بلا عقوبة إطلاقاً، ولا تُحاسَب إلا النقاط التي تخرج منه. تذكّر شريط الهامش في التصنيف من الفصل ٥١: المستويان المتقطعان اللذان يحتضنان الحد. SVR يوسع الفكرة نفسها لكن الأنبوب لا يفصل فئتين — بل يطوّق منحنى القيم نفسه:



الشكل ١١٥شريط الهامش في التصنيف كما بنيناه في الفصل ٥١؛ SVR يحمل الفكرة نفسها إلى الانحدار: أنبوب بعرض epsilon يطوق منحنى الدالة بدل مستوٍ يفصل فئتين

رياضياً، الخسارة الجديدة اسمها خسارة epsilon غير الحساسة (epsilon-insensitive loss):

$$L_\varepsilon(y, f(\mathbf{x})) = \max(0,\ |y - f(\mathbf{x})| - \varepsilon)$$

ومسألة التحسين تحاكي الهامش اللين حرفياً مع جارتين للارتخاء (فوق الأنبوب وتحته):

$$\min_{\mathbf{w}, b, \xi}\ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*)$$

$$y_i - f(\mathbf{x}_i) \le \varepsilon + \xi_i, \qquad f(\mathbf{x}_i) - y_i \le \varepsilon + \xi_i^*, \qquad \xi_i, \xi_i^* \ge 0$$

مفتاح

SVR لا يحاول تمرير الخط بين كل النقاط كما في أقل المربعات (الفصل ٢٧)؛ بل يبحث عن أنعم دالة تجعل أغلب النقاط داخل أنبوب epsilon، ويدفع بثابت C ثمناً لكل نقطة خارجه. النتيجة: الحل يعتمد على النقاط الواقعة على حافة الأنبوب أو خارجه فقط — متجهات الدعم من جديد — والنقاط المطمئنة داخل الأنبوب تختفي من الحساب.

SVR عملياً: الموجة الجيبية وأثر epsilon V.٢

لنختبر المقايضة المحورية على موجة جيبية مضروبة بالضجيج: كيف يتغير عدد متجهات الدعم ودقة النموذج كلما وسّعنا الأنبوب؟

أثر epsilon على الداعمين والدقة python

```python
import numpy as np
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

rng = np.random.default_rng(42)
X = np.sort(rng.uniform(0.0, 8.0, 240)).reshape(-1, 1)
y = np.sin(X.ravel() * 0.9) * 2.0 + rng.normal(0.0, 0.25, 240)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"{'epsilon':>8} | {'متجهات داعمة':>12} | {'MSE':>8} | {'R2':>7}")
print("-" * 46)
for eps in [0.02, 0.05, 0.1, 0.3, 0.6, 1.0]:
    svr = make_pipeline(StandardScaler(),
                        SVR(kernel="rbf", C=10.0, gamma=0.1, epsilon=eps))
    svr.fit(X_tr, y_tr)
    pred = svr.predict(X_te)
    print(f"{eps:>8.2f} | {len(svr[-1].support_):>12} | "
          f"{mean_squared_error(y_te, pred):>8.3f} | {r2_score(y_te,
pred):>7.4f}")
```

النمط واضح ومنطقي: كلما اتسع epsilon قلّ عدد النقاط التي تخرج عن الأنبوب، فتقل متجهات الدعم ويهبط استهلاك التدريب — لكن التسامح الزائد يبدأ بمحو بنية الموجة نفسها فتتراجع الدقة. القاعدة العملية: ابدأ بـ epsilon قريباً من مستوى الضجيج في هدفك (هنا انحرافه المعياري ٠.٢٥ فتكون القيم ٠.١ إلى ٠.٣ منطقة معقولة)، واضبطه كما تضبط C بالتحقق المتقاطع (الفصل ٨١).

والآن مسألة أعمق قليلاً بخمسة أعمدة بدل عمود واحد: بيانات `make_friedman1` القياسية، التي تمزج حدّاً جيبياً وتربيعياً وخطياً:

make_friedman1: SVR مقابل النموذج الخطي python

```python
from sklearn.datasets import make_friedman1
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

X, y = make_friedman1(n_samples=600, n_features=8, noise=2.0, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

lin = LinearRegression().fit(X_tr, y_tr)
svr = make_pipeline(StandardScaler(),
                    SVR(kernel="rbf", C=10.0, gamma="scale",
epsilon=1.0)).fit(X_tr, y_tr)

print(f"LinearRegression | R2 التدريب: {lin.score(X_tr, y_tr):.4f} | "
      f"R2 الاختبار: {lin.score(X_te, y_te):.4f}")
print(f"SVR(kernel='rbf') | R2 التدريب: {svr.score(X_tr, y_tr):.4f} | "
      f"R2 الاختبار: {svr.score(X_te, y_te):.4f}")
print(f"متجهات داعمة: {len(svr[-1].support_)} من أصل {len(X_tr)}")
```

النموذج الخطي يمسك الحدود الخطية في البيانات ويهجر الحد الجيبي والتربيعي (٠.٦٣٠٤ في الاختبار)، بينما SVR بنواة RBF يفكك المزيج كله فيرفع R^2 الاختبار إلى ٠.٧٤٦٥. ولاحظ حصيلة الأنبوب: `epsilon=1.0` أضيق من ضجيج انحرافه المعياري ٢.٠، فأكثر من ثلثي العينات تقع خارجه وتنضم إلى متجهات الدعم (٢٩٢ من ٤٢٠) — عُد إلى تجربة القسم السابق لتعرف أن توسيع الأنبوب هو ما يقتصد الداعمين. أما القيمة المضافة هنا فهي النمذجة غير الخطية نفسها: قفزة R^2 لا يوفرها أي نموذج خطي مهما ضبط.

> **نصيحة**
>
> اختيار epsilon المبكر يوفر عليك ساعات: حدد التسامح الذي «يقبل به مجالك» (±١% في التنبؤ بالطلب مثلاً) حوله إلى وحدات الهدف، وثبّته. حينها يصبح الضبط المتبقي على C و gamma فقط، ويتناقص عدد الداعمين تلقائياً مع كل اتساع مقبول للأنبوب.

٧.٣ انسَ طبّع الهدف y وستندم

هنا فخ لا يعترف به معظم الدروس: قلنا في الفصل ١٤ إن SVM حساسة لمقياس الميزات وطبّعنا X. لكن SVR يعيش داخل **وحدات الهدف y** أيضاً: عرض الأنبوب epsilon وثابت C كلاهما يقاس بوحدات y. هدف بوحدات كبيرة (أسعار

بالريال، أعداد بالآلاف) يجعل epsilon=١.٠ الافتراضي ضئيلاً إلى حد العدم، فيخرج كل صف من الأنبوب ويصبح كل الصفوف متجهات داعمة والحل متصلباً وبطيئاً.

```python
# هدف بالريال: بلا تحويل مقابل محوّل الهدف
import numpy as np
from sklearn.compose import TransformedTargetRegressor
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

rng = np.random.default_rng(42)
X = np.sort(rng.uniform(0.0, 8.0, 240)).reshape(-1, 1)
y = (np.sin(X.ravel() * 0.9) * 2.0 + rng.normal(0.0, 0.25, 240)) * 1000.0 +
50000.0

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

raw = make_pipeline(StandardScaler(),
                    SVR(kernel="rbf", C=10.0, gamma=0.1)).fit(X_tr, y_tr)
wrapped = make_pipeline(
    StandardScaler(),
    TransformedTargetRegressor(
        regressor=SVR(kernel="rbf", C=10.0, gamma=0.1),
        transformer=StandardScaler(),
    ),
).fit(X_tr, y_tr)

print(f"هدف خام (بآلاف الريالات)       | R2: {raw.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(raw[-1].support_)}")
print(f"مع TransformedTargetRegressor | R2: {wrapped.score(X_te, y_te):.4f} | "
      f"متجهات داعمة: {len(wrapped[-1].regressor_.support_)}")
```

المحوّل `TransformedTargetRegressor` — الذي سنتوسع فيه في الفصل ٨٧ — يطبّع y قبل التدريب ويعيد النتائج إلى وحداته الأصلية عند التنبؤ بشفافية كاملة: epsilon=١.٠ يعود إلى معقوله (نحو ١.٠ انحراف معياري)، والداعمون يقتصدون، والتنبؤ يخرج بالريال كما طُلب. البديل اليدوي (طبّع y بنفسك ثم اعكس على التنبؤات) يعمل، لكنه يفتح باب التسرّب والخطأ في كل مرة تعيد فيها الكتابة.

٧.٤ OneClassSVM: حارس يتعلم شكل الطبيعي

كل ما سبق كان خاضعاً للرقابة: تسميات أو قيم هدف. OneClassSVM يحذف هذا الافتراض كلياً: لديك عينات من «الوضع الطبيعي» فقط، والمطلوب تعلم غلافه ثم الحكم على كل نقطة جديدة: قريبة من العائلة (+١) أم غريبة عنها

(-١)? الفكرة: ادفع كل بيانات التدريب إلى ما وراء مستوى ρ في الفضاء النووي، مع رفض مسموح يحكمه بارامتر واحد أسطوري:

$$\min_{\mathbf{w},\rho,\xi} \tfrac{1}{2}\|\mathbf{w}\|^2 - \rho + \tfrac{1}{\nu n}\sum_{i=1}^{n}\xi_i, \qquad \mathbf{w}\cdot\phi(\mathbf{x}_i) \geq \rho - \xi_i$$

الاستخدام القياسي — الذي توثقه scikit-learn باسم كشف الجدة (novelty detection) — هو تدريب نظيف ثم حكم على الوافدين الجدد:

OneClassSVM: تدريب على النظيف وحكم على الجدد — python

```python
import numpy as np
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(42)
clean = rng.normal([0.0, 0.0], 1.0, (300, 2))          # تدريب: نمط سليم فقط

oc = OneClassSVM(kernel="rbf", nu=0.05, gamma=0.5).fit(clean)

new = rng.normal([0.0, 0.0], 1.0, (200, 2))            # وافدون سليمون
intruders = rng.uniform(-4.5, 4.5, (50, 2))            # متسللون خارج النمط
eval_X = np.vstack([new, intruders])
pred = oc.predict(eval_X)

accept = (pred[:200] == 1).mean()
caught = (pred[200:] == -1).mean()
rejected_train = (oc.predict(clean) == -1).mean()
print(f"قبول الوافدين السليمين: {accept:.1%}")
print(f"اصطياد المتسللين: {caught:.1%}")
print(f"نسبة رفض بيانات التدريب نفسها (المتوقع ≈ nu = 0.05): {rejected_train:.1%}")
```

اقرأ السطر الأخير بوصفه فحصاً تشخيصياً مجانياً: نسبة رفض التدريب يجب أن تقارب nu من أعلى؛ إن قفزت فوقها بكثير فالنموذج يعاملكم أنتم كشواذ — راجع gamma أو بُنية البيانات قبل أن تثق بحدوده.

في الواقع

هذا النمط هو قلب أنظمة مراقبة الأجهزة وبوابات الدفع قبل عصر التعلم العميق: تدرّب على إشارات ساعة الضخ السليمة أو معاملات البطاقة السليمة، ثم كل وافد غريب يرفع إنذاراً لمراجعة بشرية. لاحظ ما لا يملكه المصنف التقليدي: لا حاجة لأي مثال شاذ في التدريب — والشواذ في الحياة بالتعريف أندر من أن تُجمع. وقد ظل OneClassSVM الخيار القياسي لعقود، والبدائل الأحدث مثل IsolationForest و LOF تأتي في الفصل ٧٠ بموازين مختلفة تماماً.

٧.٥ كشف الجدة مقابل كشف الشواذ

الفرق بين المصطلحين ليس ترفاً لغوياً بل فرق بروتوكول كامل، scikit-learng توثقه في كل مقدر كشف شذوذ:

البروتوكول	بيانات التدريب	السؤال	OneClassSVM
كشف الجدة (novelty detection)	نظيفة من العائلة السليمة فقط	هل هذا الوافد الجديد غريب؟	الاستخدام الموثّق الرسمي
كشف الشواذ (outlier detection)	تحتوي شواذ مختلطة بالسليم	أي نقاط مجموعة التدريب نفسها شاذة؟	ممكن لكن الحد يتشوه

python — أثر التلوث على الحد الفاصل

```python
import numpy as np
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(42)
clean = rng.normal([0.0, 0.0], 1.0, (300, 2))
noise = rng.uniform(-4.0, 4.0, (30, 2))                    # 9% شواذ تسربت للتدريب
contaminated = np.vstack([clean, noise])

novelty = OneClassSVM(kernel="rbf", nu=0.05, gamma=0.5).fit(clean)
outlier = OneClassSVM(kernel="rbf", nu=0.05, gamma=0.5).fit(contaminated)

new = rng.normal([0.0, 0.0], 1.0, (200, 2))
intruders = rng.uniform(-4.0, 4.0, (50, 2))
eval_X = np.vstack([new, intruders])

for name, model in [("fit على نظيف  (novelty)", novelty),
                    ("fit على ملوث (outlier)", outlier)]:
    pred = model.predict(eval_X)
    print(f"{name}: قبول السليمين {((pred[:200] == 1).mean()):.1%} | "
          f"اصطياد الشواذ {((pred[200:] == -1).mean()):.1%}")
```

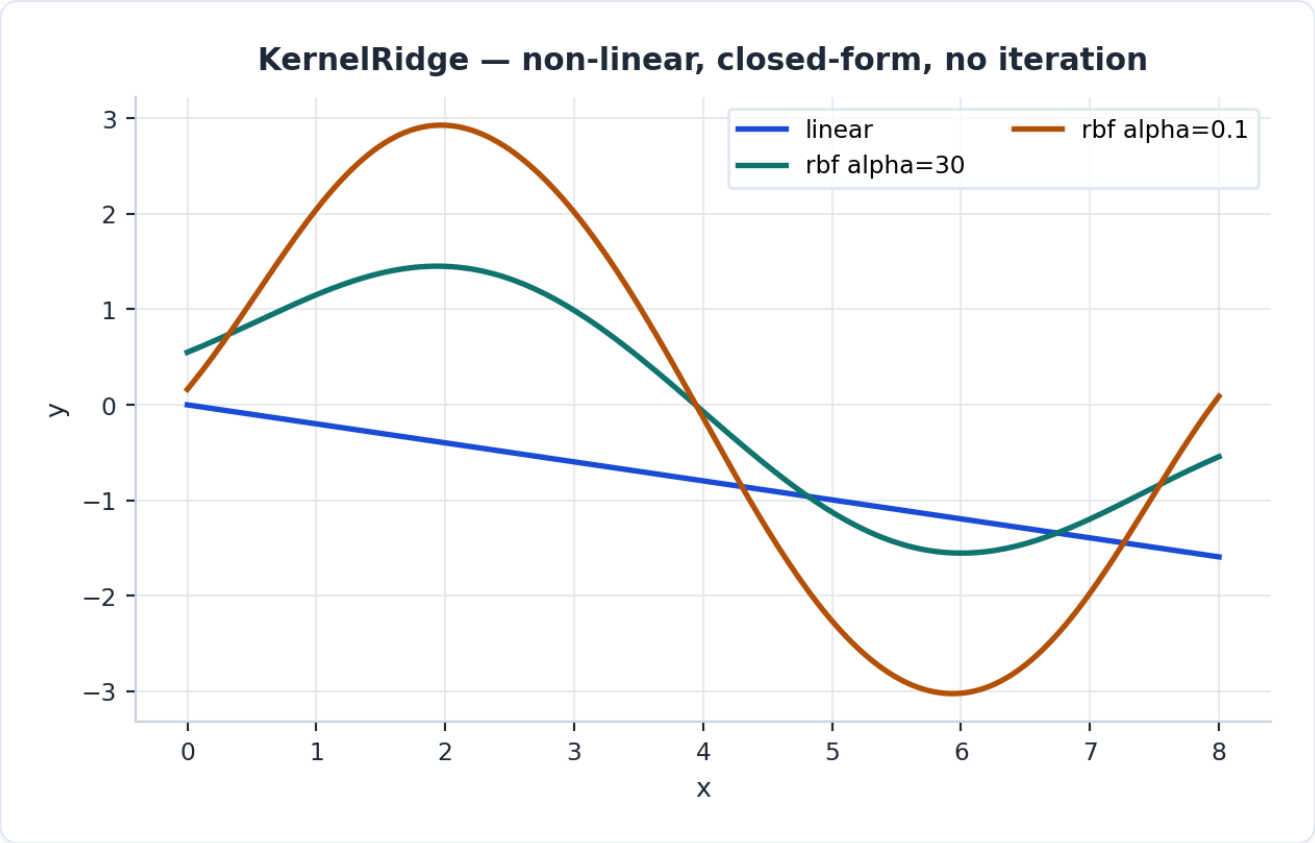
> **تحذير**
>
> OneClassSVM حساسة جداً لشواذ تدريبها نفسها: الشاذة المختلطة تدفع الحد فتتسع «منطقة الطبيعي» لتشملها، فيفوت النموذج متسللين جدد يشبهونها تماماً. لا تعامل مخرجات `predict` بوصفها يقيناً إذا شكّ في نظافة تدريبك؛ رجوعك عندها إلى الفصل ٧٠: `IsolationForest` و `LOF` مصممتان أصلاً لوضع التلوث، أو نظّف بياناتك أولاً بطريقة قوية ثم درّب OneClassSVM.

V.٦ SVR مقابل KernelRidge: نفس النواة، فلسفتان

سؤال عادل: بما أن كليهما انحدار نووي بخط RBF، فلماذا SVR أصلاً؟ `KernelRidge` — الذي سيلتقي به في الفصل ٥٨ — يحل مسألة ريدج في الفضاء النووي بمسار مغلق (معادلات خطية)، بينما SVR يحل برمجة تربيعية بحافة epsilon. الفرق يظهر في ثلاثة أبعاد: من يشارك في الحل، وكيف يعاقب الخطأ، ومتى يفضّل كلٌّ منهما. لنقارن على نفس بيانات الموجة الجيبية:

على نفس البيانات KernelRidge مقابل SVR — python

```python
import time
import numpy as np
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

rng = np.random.default_rng(42)
X = np.sort(rng.uniform(0.0, 8.0, 240)).reshape(-1, 1)
y = np.sin(X.ravel() * 0.9) * 2.0 + rng.normal(0.0, 0.25, 240)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

models = [
    ("SVR", make_pipeline(StandardScaler(),
                          SVR(kernel="rbf", C=10.0, gamma=0.1, epsilon=0.1))),
    ("KernelRidge", make_pipeline(StandardScaler(),
                                  KernelRidge(kernel="rbf", gamma=0.1,
alpha=0.01))),
]
for name, model in models:
    t0 = time.perf_counter()
    model.fit(X_tr, y_tr)
    dt = time.perf_counter() - t0
    mse = mean_squared_error(y_te, model.predict(X_te))
    if name == "SVR":
        n_vec = len(model[-1].support_)
    else:
        n_vec = int(model[-1].dual_coef_.size)          # كل عينة تحمل معاملاً
    print(f"{name:>12} | زمن التدريب: {dt * 1000:7.1f} ms | "
          f"MSE: {mse:.4f} | معاملات الحل: {n_vec}")
```



الشكل ١١٦ KernelRidge بنواة RBF على موجة جيبية مضروبة بالضجيج: alpha صغير يلاحق التموج، alphag كبير ينعّم الحل حتى يمحو البنية، والنواة الخطية تعجز

اقرأ الناتج بوصفه مفاجأة تعليمية: على هذه العينة الصغيرة لم يكن «المسار المغلق» أسرع — حل `KernelRidge` نظاماً خطياً كاملاً لكل معاملاته الـ١٦٨ (٢٥.٦ ms) بينما كفى SVR بـ١٣٩ داعماً (٩.٠ ms). ودقة KernelRidge الأعلى (MSE ٠.١٣٦٩ مقابل ٠.٢٧٣٩) هي وجه الخسارة التربيعية: لا أنبوب تسامح فيلاحق الإشارة بتعلّق أكبر. القاعدة التي تحكم الاختيار:

المعيار	SVR	KernelRidge
معاملات الحل	متجهات الدعم فقط (اقتصاد)	كل عينة تدريب معاملاً (صفر اقتصاد)
العقوبة على الخطأ	صفر داخل أنبوب epsilon	تربيعية من أول خطأ
الحل	برمجة تربيعية (SMO)	مسار مغلق خطي بحجم n^٣ تقريباً
الصلابة تجاه القيم الشاذة	أعلى بفضل الأنبوب	أضعف: الخطأ التربيعي يطارد الشاذ
الضبط	C و epsilon و gamma	alpha و gamma

KernelRidge يدفع الثمن في عنصرين: كل عينة تصبح معاملاً فيدخمن التنبؤ مع كامل بيانات التدريب، والخسارة التربيعية تجعله رهيناً للقيم الشاذة. وSVR يشتري اقتصاد الداعمين وصلابته بثمن تدريب تكراري — وهذه هي المقايضة نفسها التي تقرأها في وجهي العملة منذ الفصل ٢٧.

تمارين الفصل الثالث والخمسين

- **ملاحظة**

التمرين ٧.١

على عينة من `fetch_california_housing` (أول ١٢٠٠ صف)، درب SVR بنواة RBF داخل أنبوب مع StandardScaler، وامسح epsilon على [٠.٠١، ٠.٠٥، ٠.١، ٠.٣، ٠.٥]، واطبع عدد متجهات الدعم وR² للاختبار عند كل قيمة. أين يبدأ الأنبوب بأكل البنية؟

التمرين ٧.٢

اشرح بأسلوبك: لماذا لا يستخدم OneClassSVM أي تسميات، وما الدقيق في كون nu «حداً أعلى» و«حداً أدنى» في آن واحد؟ وماذا تتوقع أن يحدث عند nu=٠.٩ على بيانات شبه نظيفة؟

التمرين ٧.٣

جربة كشف جدة حقيقية على `load_digits`: درب OneClassSVM على صور الرقم · فقط، ثم قِس نسبة رفض صور الرقم · نفسه (يجب أن تكون منخفضة) ونسبة رفض بقية الأرقام (يجب أن تكون مرتفعة). جرب ثلاث قيم متناقصة لـ gamma ولاحظ أثرها على النسبتين معاً.

- **ملاحظة**

حل التمرين ٧.١ python

```python
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

data = fetch_california_housing()
X, y = data.data[:1200], data.target[:1200]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"{'epsilon':>8} | {'متجهات داعمة':>12} | {'R2 الاختبار':>12}")
print("-" * 40)
for eps in [0.01, 0.05, 0.1, 0.3, 0.5]:
    svr = make_pipeline(StandardScaler(),
                        SVR(kernel="rbf", C=10.0, epsilon=eps)).fit(X_tr, y_tr)
    print(f"{eps:>8.2f} | {len(svr[-1].support_):>12} | "
          f"{svr.score(X_te, y_te):>12.4f}")
```

ملاحظة

حل التمرين ٧.٣ python

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM

X, y = load_digits(return_X_y=True)
zeros = X[y == 0]
rest = X[y != 0]

print(f"{'gamma':>7} | رفض الأرقام الأخرى | رفض الرقم 0")
print("-" * 44)
for g in [0.05, 0.01, 0.002]:
    oc = make_pipeline(StandardScaler(),
                       OneClassSVM(kernel="rbf", nu=0.05, gamma=g)).fit(zeros)
    rej0 = (oc.predict(zeros) == -1).mean()
    rejrest = (oc.predict(rest) == -1).mean()
    print(f"{g:>7.3f} | {rej0:>10.1%} | {rejrest:>13.1%}")
```

ملاحظة

٧.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
ترك `epsilon=0.1` الافتراضي مع هدف بوحدات كبيرة	الأنبوب عبثي بجوار هدف بالآلاف فيصير كل صف متجهاً داعماً والتدريب متصلباً	`TransformedTargetRegressor` أو ضبط epsilon كنسبة من انحراف y المعياري
إسقاط StandardScaler قبل SVR بنواة RBF	المسافات تهيمن عليها الأعمدة الضخمة وتتحول gamma إلى رقم بلا معنى	`make_pipeline(StandardScaler(), SVR())` دوماً
تدريب OneClassSVM على بيانات ملوثة ثم استخدامه لكشف الجدة	الشواذ تمدد الحد فيفوت متسللون جدد يشبهونها	تدريب نظيف أولاً؛ وإن تعذر فالرجوع إلى IsolationForest و LOF في الفصل ٧٠
اختيار nu=٠.٥ بلا نبوءة مسبقة لنسبة الشواذ	النموذج يرفض نحو نصف البيانات السليمة ويصبح الإنذار ضجيجاً	اضبط nu قرب النسبة المتوقعة فعلًا (عادة بين ٠.٠١ و ٠.١)

٧.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
انحدار المتجهات الداعمة	انحدار نووي يعظّم النعومة تحت قيد بقاء الأخطاء داخل أنبوب	Support Vector Regression (SVR)
أنبوب التسامح	النطاق بعرض epsilon*٢ حول دالة التنبؤ الذي لا يعاقب أي خطأ داخله	Epsilon tube
الخسارة غير الحساسة	دالة عقوبة تساوي صفراً داخل الأنبوب وتعاقب الفائض عنه فقط	Epsilon-insensitive loss
آلة الفئة الواحدة	مقدر يتعلم غلاف بيانات التدريب بلا تسميات ويصنف الجديد داخله أو خارجه	One-Class SVM
كشف الجدة	بروتوكول: تدريب على بيانات نظيفة وحكم على وافدين جدد	Novelty detection

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٤ — الجيران الأقرب: k-NN

التصنيف والانحدار، والأوزان، والمسافات، وأثر التطبيع

الوحدة **scikit-learn ١.٩** الحجم **٨١٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا لا يوجد تدريب حقيقي في k-NN وكلفة تنبؤه O(n·d) وذاكرته الكاملة
- أن تطبّق الأوزان uniform و distance وتتربّص فخ وزن 1/d مع الجار الملاصق
- أن تحدد k الأمثل بمنحنى حقيقي يوازن بين فرط الملاءمة والتمليس
- أن تثبت بالأرقام أثر StandardScaler على المسافات والدقة
- أن تستخدم KNeighborsRegressor وتفهم متوسط الجيران كتقدير محلي

٧.١ النموذج الذي لا يتعلّم: العقل الكسول

في الفصل ٤ شغّلنا أول نموذج في حياتك على بيانات Iris وكان من نوع k-NN. توقف واسأل: ماذا فعل التدريب فعلياً؟ في الانحدار الخطي نحل معادلات، وفي الأشجار نبحث عن أفضل قسمة، وفي SVM نحل مسألة تربيعية كاملة. هنا؟ **لا شيء**. المقدّر `KNeighborsClassifier` يحفظ البيانات كما هي وينصرف.

لهذا يُسمّى k-NN **نموذجاً كسولاً (lazy learner)** قائماً على المثيلات: كل الجهد الحسابي يؤجَّل إلى لحظة `predict()`. حين تصل عينة جديدة يُحسب مربّع المسافة بينها و**كل** عينات التدريب، تُختار أقرب k، ويصوّت الجيران:

$$d(\mathbf{x}, \mathbf{x}_i) = \left(\sum_{j=1}^{d} (x_j - x_{i,j})^2\right)^{1/2}$$

حيث `x` العينة الجديدة و `x_i` عينة تدريب و d عدد الخصائص، ويخرج التصنيف فئة الأغلبية بين جيران العينة `N_k(x)` — مجموعة مؤشرات الجيران الـ k الأقرب. لا شيء يُقدَّر أثناء «التدريب».

> **المفهوم المحوري**
>
> k-NN لا يبني دالة تعميم مسبقة؛ البيانات هي النموذج نفسه. أي تحسين يكون على جانبين فقط: كيف تحسب المسافات (الفصل ٥٥)، وكيف تختار الجيران وتزن أصواتهم.

لنثبت الكلام بالأزمنة: قياس `fit` لـ k-NN مقابل غابة عشوائية حقيقية، ثم مراقبة تنامي زمن التنبؤ مع حجم البيانات:

```python
# التدريب الكسول: صفر جهد الآن، كل الجهد لاحقاً
import time
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

X, y = make_classification(n_samples=40000, n_features=10, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42,
stratify=y)

for n in [5000, 40000]:
    knn = KNeighborsClassifier(n_neighbors=5)
    t0 = time.perf_counter(); knn.fit(X_tr[:n], y_tr[:n]); t_fit =
time.perf_counter() - t0
    t0 = time.perf_counter(); knn.predict(X_te[:2000]); t_pred =
time.perf_counter() - t0
    rf = RandomForestClassifier(n_estimators=50, random_state=42, n_jobs=-1)
    t0 = time.perf_counter(); rf.fit(X_tr[:n], y_tr[:n]); t_rf =
time.perf_counter() - t0
    print(f"n = {n:6d} | fit k-NN: {t_fit * 1000:6.1f} ms | تنبؤ 2000 استعلام:
{t_pred * 1000:7.1f} ms | fit غابة 50 شجرة: {t_rf * 1000:8.1f} ms")
```

اقرأ الجدول بعين مهندس: تدريب k-NN يقيس زمن نسخ مصفوفة، بينما تنبؤه ينمو خطياً مع n — كل استعلام يدفع O(n·d). الغابة تدفع ثمناً في التدريب ثم تتنبأ بسرعة لا تعتمد على n؛ وأيهما تختار يحدده سؤال الإنتاج: هل تدرّب مرة وتتنبّأ ملايين المرات، أم تتحدث البيانات باستمرار؟

٧.٢ التصويت وأوزانه: uniform مقابل distance

الافتراضي `weights="uniform"` يجعل كل جارٍ صوتاً واحداً مهما بُعد. لكن الجار الذي يبعد ٠.٠١ وحدة «أصدق» من جار يبعد ١.٠٠؛ الخيار `weights="distance"` يزن كل صوت بمقلوب المسافة:

$$w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)}, \qquad \hat{y}(\mathbf{x}) = \operatorname{argmax}_c \sum_{i \in N_k(\mathbf{x})} w_i \, \mathbf{1}[y_i = c]$$

حيث `1[y_i = c]` دالة المؤشر. النتيجة: القريبون يطغون على البعيدين فتصير الحدود أنعم قرب الازدحام وأكثر حساسية للقريب جداً.

⛔ فخ وزن 1/d: الجار الملاصق يفجّر الحساب

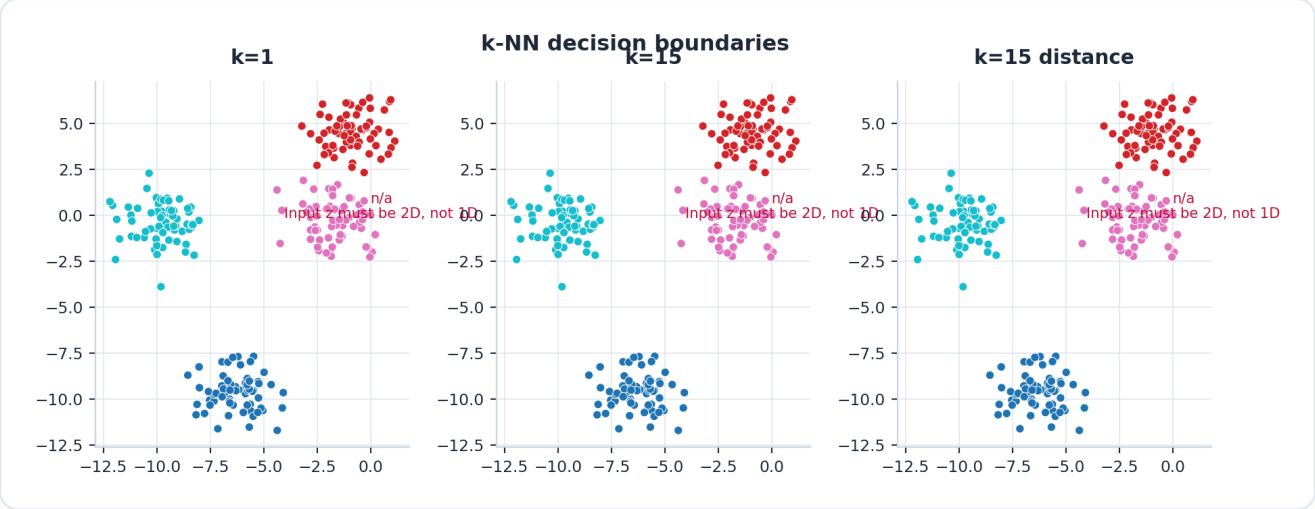
الوزن 1/d **غير معرّف** عند d = ٠. إذا تطابقت عينة استعلام مع عينة تدريب — وهذا شائع في خصائص منفصلة أو عيّنات مكررة — فإن 1/٠ = inf يغرق التصويت كله. انظر الانفجار وكيف تعالجه scikit-learn في المثال التالي؛ فإذا كتبت k-NN بيديك فاحمِ دالة الأوزان بشرط d > ٠.

python — انفجار وزن 1/d مع المسافة الصفرية وكيف يقاومه sklearn

```python
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

d = np.array([0.0, 2.5, 7.1])          # جار ملاصق تماماً (d = 0)
with np.errstate(divide="ignore"):
    print(f"أوزان 1/d الساذجة: {1.0 / d}")

Xs = np.array([[0.0, 0.0], [1.0, 0.0], [5.0, 5.0], [5.1, 5.2]])
ys = np.array([0, 0, 1, 1])
knn = KNeighborsClassifier(n_neighbors=4, weights="distance").fit(Xs, ys)
print(f"استعلام مطابق لعينة تدريب: {knn.predict_proba(np.array([[0.0, 0.0]]))}")
print(f"استعلام بعيد عن الجميع:     {knn.predict_proba(np.array([[2.4, 2.4]]))}")
```

السطر الأول يعرض الانفجار: 1/٠ = inf. بينما عالجت scikit-learn الصفر داخلياً: عند وجود مسافة صفرية تتجاهل بقية الجيران وتُرجع احتمال ١.٠ لفئة العينة المطابقة. أما أي الأسلوبين أفضل فعلى بيانات Wine و breast_cancer يكون الفارق طفيفاً ومتقلباً — ستختبره بنفسك في التمرين ٧.١ وتجد أن الأوزان أداة ضبط دقيقة لا رافعة أعجوبة.

المعيار	uniform	distance
حساب الصوت	عدّ أغلبية بسيط	مجموع 1/d لكل فئة
سلوكه مع جار ملاصق	لا يتأثر إطلاقاً	يطغى وحده على القرار
حساسية الضجيج القريب	متوسطة	أعلى — يُضاعَف صوته



الشكل ١١٧حدود القرار لدى k-NN: المتعرجة مع k = ١ والملساء مع k = ١٥ وبأوزان المسافة

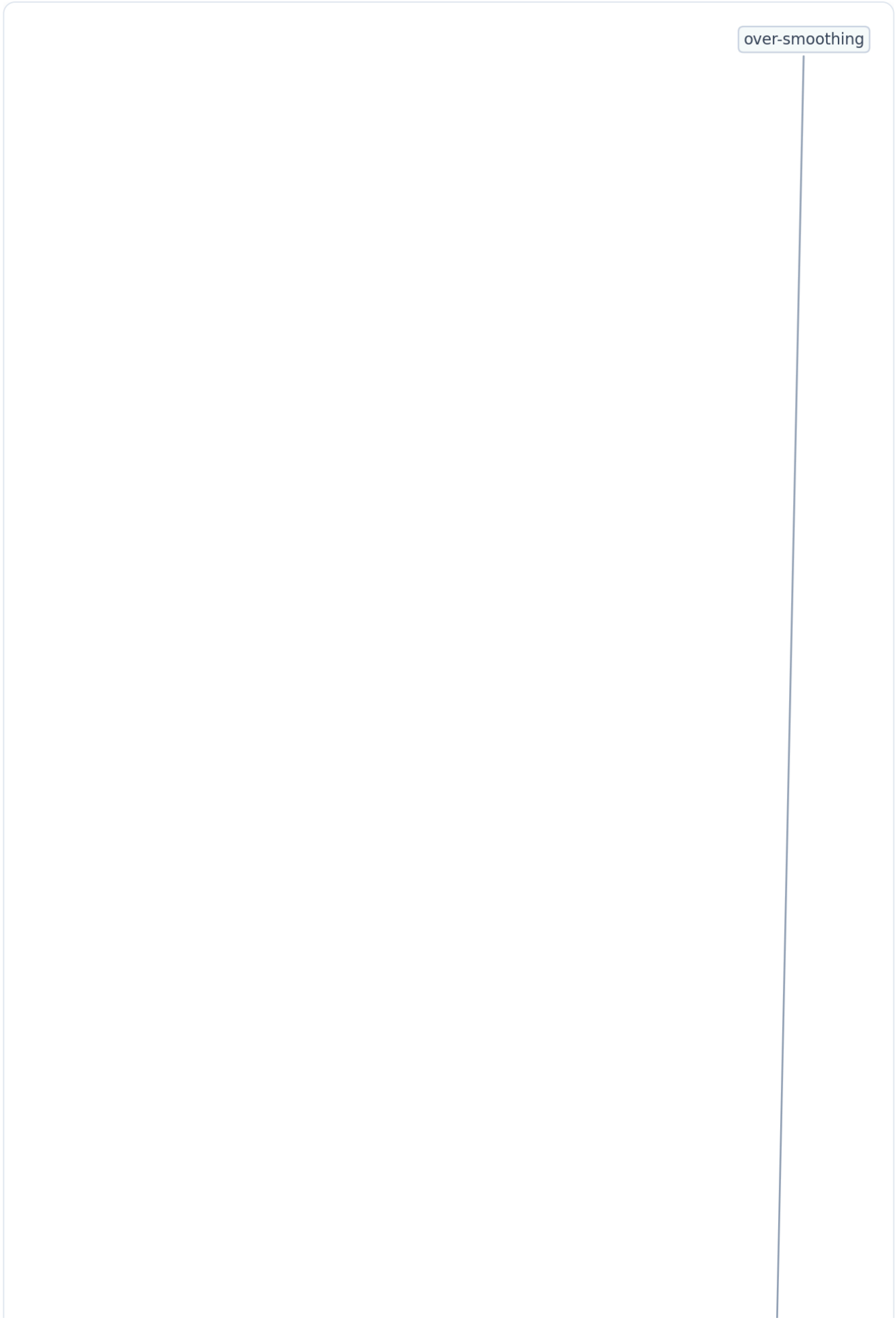
في الشكل ثلاث خرائط لنفس البيانات: حدود k = ١ تحيط بكل نقطة جزيرةً خاصة بها، وحدود k = ١٥ ملساء ذات اتساع منطقي، وأوزان المسافة تنزاح الحدود نحو الجيران القريبين فتتكون أهداب أرفع حول العناقيد.

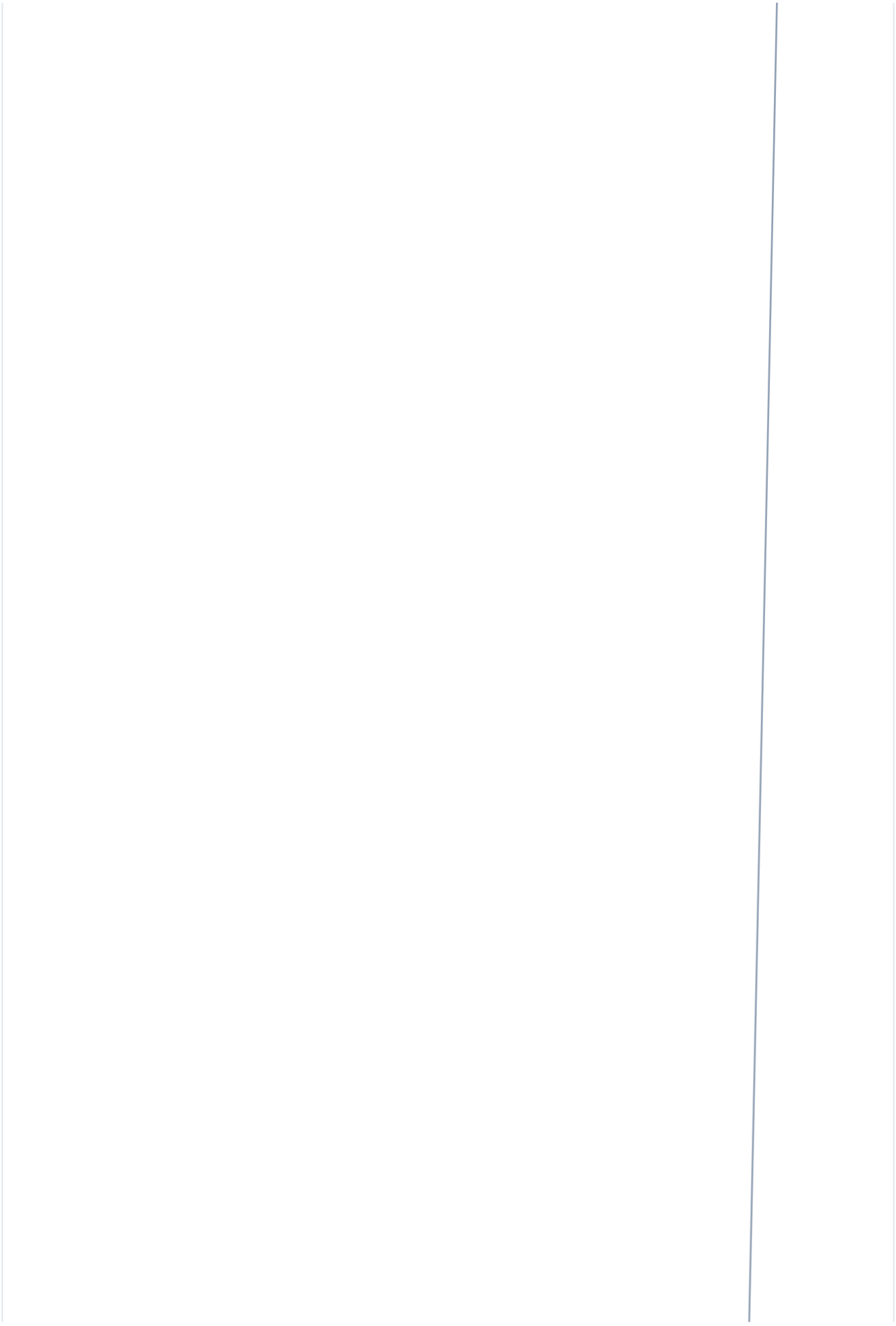
٧.٣ أثر k: من التعرّج إلى التمليس

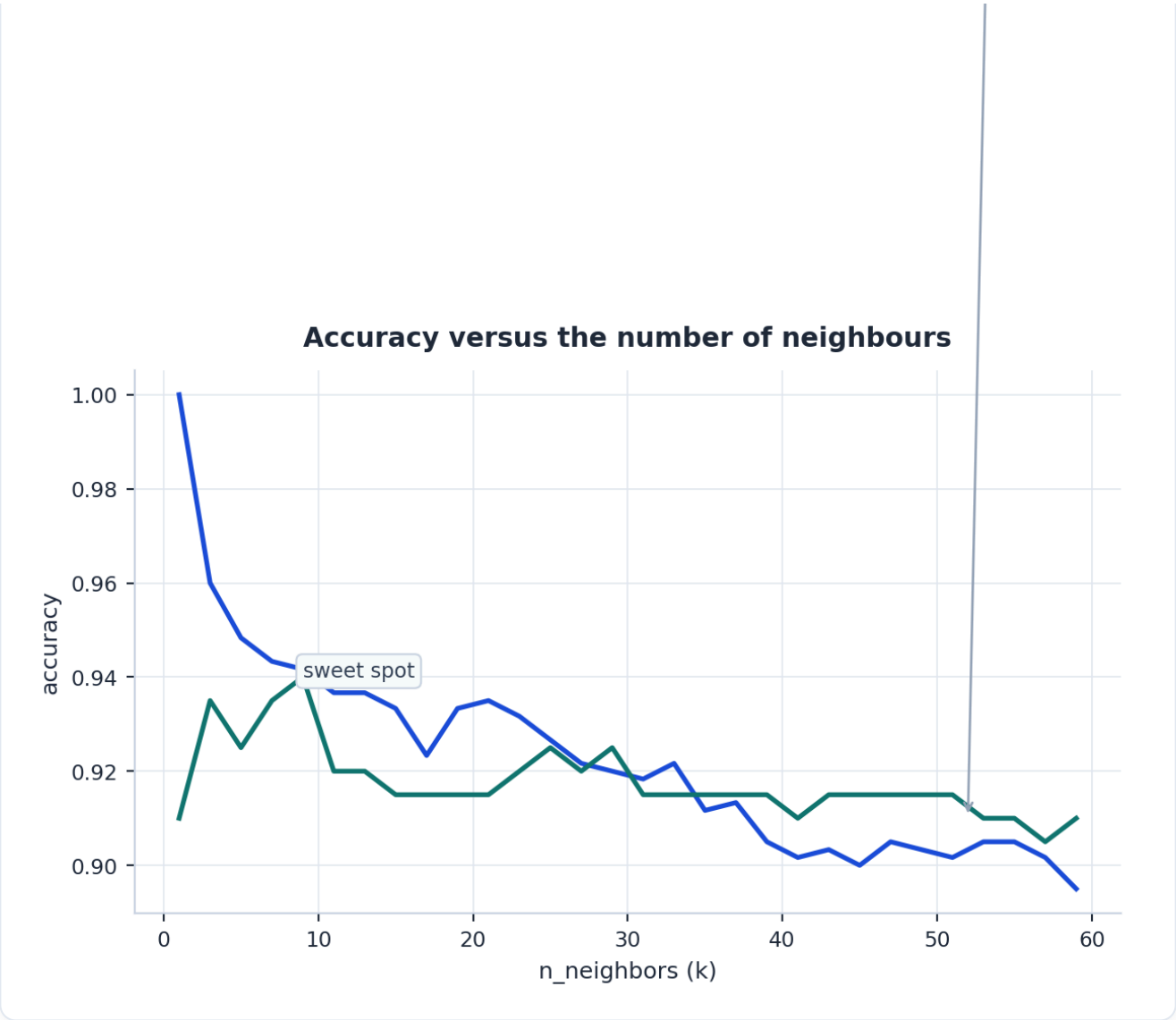
ماذا يحدث لو رفعنا k من ١ إلى ٤٠؟ عند k = ١ يتنبأ النموذج بفئة أقرب نقطة فقط: دقة التدريب ١٠٠% حتماً، والحدود تتعرّج حول كل ضجيج. مع تزايد k يصوّت حشد أكبر فتنملس الحدود ويرتفع الانحياز، حتى يتسلل جيران أبعد من فئات أخرى ويهبط الأداء. والاحتمال المقدَّر هو نسبة الجيران المنتمين للفئة:

$$\hat{P}(y = c|\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} \mathbf{1}[y_i = c]$$

فعند k = ٥ مع جارين من A وثلاثة من B يعطي `predict_proba` القيمة ٠.٤ و٠.٦ — الاحتمالات دائماً من مضاعفات ١/k، عينات خشنة لا انعكاساً للثقة الحقيقية.


over-smoothing





الشكل ١١٨دقة التدريب والاختبار مع تزايد عدد الجيران: الهوة بينهما تتسع عند k الصغير

الشكل يلخّص السيناريو الكلاسيكي: دقة التدريب تبدأ من ١.٠ عند k = ١، ودقة الاختبار تبلغ ذروتها في منطقة وسطى ثم تتراجع ببطء بسبب التمليس. لنشغّل التجربة بأنفسنا من ١ إلى ٤٠ ونطبع الأرقام:

تجربة k من 1 إلى 40 بالأرقام python

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = make_classification(n_samples=800, n_features=14, n_informative=5,
random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

print(f"{'k':>4} | {'دقة التدريب':>10} | {'دقة الاختبار':>11}")
for k in [1, 3, 5, 9, 15, 25, 40]:
    pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    pipe.fit(X_tr, y_tr)
    print(f"{k:4d} | {pipe.score(X_tr, y_tr):10.4f} | {pipe.score(X_te,
y_te):11.4f}")
```

لاحظ الصف الأول: عند k = ١ دقة التدريب ١.٠٠٠٠ تماماً — علامة ذاكرة حرفية لا قوة. ودقة الاختبار تبلغ ذروتها هنا قرب k = ٢٥ لا عند ٥ — المنطقة الحلوة تتنقل حسب البيانات فلا تصدّق أحداً قبل أن تجرّب. القاعدة: ابحث عن k بالتحقق المتقاطع (الفصل ٧٣) داخل خط أنابيب يضم التطبيع، ولا تضبطه على الاختبار أبداً.

٧.٤ انحدار الجيران: KNeighborsRegressor

يأخذ `sklearn.neighbors.KNeighborsRegressor` متوسط قيم الجيران بدل التصويت:

$$\hat{y}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i$$

تقدير محلي صريح: قيمة العينة الجديدة متوسط ارتفاعات جيرانها المباشرين، ومع `weights="distance"` يصير متوسطاً مرجّحاً يدفع القريبين الحصة الأكبر. سلوكه أمام تزايد k هو المفارقة المعتادة: k = ١ يتابع الضجيج فيرتفع الخطأ، ثم يهبط عند قيم متوسطة، ثم يعود بالارتفاع البطيء حين يصبح الجيران بعيدين فلا يمثّلون المحيط المحلي. ولاحظ سقف النموذج: لا يستطيع أبداً التنبؤ بقيمة خارج مدى قيم جيرانه — الاستقراء مستحيل بنيوياً. ستقيس هذه المفارقة بنفسك في التمرين ٧.٢ وحلّه.

> **نصيحة**
>
> في الانحدار امزج بين k كبيرة وأوزان `distance` حين تتفاوت كثافة البيانات؛ فالمتوسط البسيط لجيران بعيدين في منطقة شحيحة يشوّه التقدير، والوزن 1/d يخفّض أثرهم تلقائياً.

٧.٥ التطبيع ليس رفاهية: تجربة قاطعة

هنا أخطر درس في الفصل. المسافة الإقليدية تجمع الفروق المربّعة، فالخاصية ذات المقياس الأكبر تحتكر المسافة كلياً. في بيانات Wine، عمود proline يقيس بالآلاف بينما flavanoids بأجزاء من الوحدة — فتصبح مسافة أي عينتين تقريباً مسافة عمود واحد فقط:

```python
# عمود واحد يطغى على المسافة كلها: مع وبدون StandardScaler
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_wine(return_X_y=True)
stds = X.std(axis=0)
share = (stds ** 2) / (stds ** 2).sum()
print(f"عمود proline وحده يحتكر {share[-1]:.1%} من تباين المسافة الإقليدية قبل التطبيع")

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

knn_raw = KNeighborsClassifier(n_neighbors=5).fit(X_tr, y_tr)
print(f"دقة بدون تطبيع:  {knn_raw.score(X_te, y_te):.4f}")

pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
pipe.fit(X_tr, y_tr)
print(f"دقة مع StandardScaler: {pipe.score(X_te, y_te):.4f}")
```

قفزة من ٠.٧٢٢٢ إلى ٠.٩٤٤٤ بمجرد سطر تطبيع: قبل التطبيع يرى النموذج بضعاً واحداً — العمود ذا الآلاف يحتكر ٩٩.٨% من تباين المسافة — وبعده تفتح الخصائص الـ ١٣ عيناً جميعاً. هذا موضوع الفصل ١٤، لكنه هنا ليس تحسيناً اختيارياً بل **شرط صلاحية**.

> **خطأ يفسد كل شيء: تطبيع الاختبار بمتوسطات الاختبار**
>
> أخطر طريقة للإخفاق استدعاء `fit_transform` على بيانات الاختبار: تتسرب إحصاءات الاختبار إلى التحويل فتظهر أرقام مبهرة زائفة. القاعدة الحزمية: `Pipeline` دائماً؛ هو يستدعي `transform` على الاختبار بمتوسطات وانحرافات **التدريب** حصراً.

وينطبق هذا على كل نموذج قائم على المسافات: SVM في الفصل ٥١ و K-Means في الفصل ٦٠ — لكن k-NN أسوأ الضحايا لأن المسافة هي حساباته كلها.

٧.٦ الكلفة الحقيقية وحدود النموذج

الفاتورة النهائية للنموذج الكسول: زمن تنبؤ O(n·d) لكل استعلام (تحققت منه في القسم ٧.١)، وذاكرة تحفظ كامل التدريب — مليون عينة بـ ٥٠ خاصية عائمة تعني نحو ٤٠٠ ميغابايت محفوظة كما هي — ولا «تشغيل» بين التنبؤات؛ والطريق الوحيد للتسريع بُنى بحث حية، وهي موضوع الفصل ٥٥.

> **في الواقع: متى يعيش k-NN في الإنتاج؟**
>
> يُستخدم في أنظمة التوصية لاسترجاع «أقرب المستخدمين/الأصناف»، وفي كشف الاحتيال كمرجع سريع، وفي KNNImputer لتعبئة القيم المفقودة (الفصل ١٣). لكن ملايين الاستعلامات في الدقيقة تنقلب على الكلفة: إما أبنية بحث تقريبي متخصصة أو شطب النموذج؛ k-NN بطل المشاريع الصغيرة لا سفينة إنتاج خالصة.

وأخيراً اللعبة الأشهر عليه: **لعنة الأبعاد**. في الأبعاد العالية تتقارب المسافات من أي استعلام نحو النقاط حتى يكاد الفرق بين «أقرب» و«أبعد» يختفي، فيفقد التصويت معناه لأن الجميع يصبح جاراً بالتساوي. جرّب بنفسك: عيّنات عشوائية منتظمة في d = ٢ تعطي فجوة هائلة بين أقرب وأبعد نقطة (النسبة (أبعد − أقرب)/أقرب تقارب ٤٤)، وعند d = ١٠٠٠ تنهار إلى ما دون ٠.١٥. العلاج الوحيد: اختيار الخصائص وتقليل الأبعاد قبل k-NN — كما رأينا بالتفصيل في الفصل ٢١.

تمارين الفصل الرابع والخمسين

- **ملاحظة**

التمرين ٧.١

على بيانات `load_breast_cancer`، قارن دقة `KNeighborsClassifier` بقيم k من ١ إلى ٢١ (فردي) مع `weights="uniform"` ثم `weights="distance"` وبعد تطبيع `StandardScaler` حصراً. عند أي k تنقلب النتيجة بين الأسلوبين؟

التمرين ٧.٢

أنشئ بيانات انحدار بـ `make_regression(n_samples=600, n_features=4, noise=25, random_state=42)` وحدّد k الأمثل لـ `KNeighborsRegressor` بالتحقق المتقاطع الخماسي لقيم k من ١ إلى ٣٠، ثم اطبع RMSE لأفضل k على مجموعة اختبار.

التمرين ٧.٣

ادّعِ أن دقة التدريب لـ k-NN بـ k = ١ تساوي ١.٠ دائماً بلا عينات مكررة بفئات متضاربة. اثنها على بيانات Digits، ثم فسّر ما يحدث عند زرع عينتين متطابقتين بفئتين مختلفتين.

- **ملاحظة**

حل التمرين ٧.١: مقارنة الأوزان على بيانات الثدي python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

print(f"{'k':>4} | {'uniform':>8} | {'distance':>8}")
for k in range(1, 22, 2):
    accs = []
    for w in ["uniform", "distance"]:
        pipe = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=k, weights=w))
        pipe.fit(X_tr, y_tr)
        accs.append(pipe.score(X_te, y_te))
    print(f"{k:4d} | {accs[0]:8.4f} | {accs[1]:8.4f}")
```

الأسلوبان يتعادلان في معظم قيم k على بيانات نظيفة كثيفة، مع تفاوت طفيف قد ينقلب فيه الميزان. الدرس: ضع الأوزان في البحث الشبكي مع k معاً ولا تقدّس أحدهما.

> **ملاحظة**

حل التمرين ٧.٢: k الأمثل لانحدار الجيران بالتحقق المتقاطع
python

```python
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import root_mean_squared_error

X, y = make_regression(n_samples=600, n_features=4, noise=25, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

cv_scores = {}
for k in range(1, 31, 2):
    p = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=k))
    cv_scores[k] = cross_val_score(p, X_tr, y_tr, cv=5,
                                   scoring="neg_root_mean_squared_error").mean()

best_k = max(cv_scores, key=cv_scores.get)
pipe = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=best_k))
pipe.fit(X_tr, y_tr)
rmse = root_mean_squared_error(y_te, pipe.predict(X_te))
print(f"أفضل k بالتحقق المتقاطع: {best_k} | RMSE الاختبار: {rmse:.4f}")
```

انتصرت قيمة وسطى (k = ٧) في التحقق المتقاطع على الطرفين معاً: k = ١ الذي يتابع الضجيج فيرفع الخطأ، و k = ٢٩ الذي يمليس التقدير حتى يفسد المحيط المحلي. ولاحظ أن كل تنبؤ يبقى داخل مدى قيم جيرانها — الاستقراء خارج هذا المدى مستحيل بنيوياً في انحدار الجيران.

> **ملاحظة**

٧.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إطعام بيانات غير مطبّعة لـ k-NN	عمود واحد بمقياس كبير يحتكر المسافة فتهبط الدقة بصمت	غلّف المقدّر بـ `Pipeline([StandardScaler(), KNN])` دائماً
تنفيذ أوزان `distance` يدوياً دون حماية	مسافة صفرية تعطي 1/0 = inf فتصبح الأصوات NaN	احمِ الحساب بشرط `d > 0` أو اعتمد سلوك sklearn المدمج
ضبط k على مجموعة الاختبار مباشرة	تسرّب اختياري يضخّم الدقة وتفشل في الإنتاج	اضبط عبر GridSearchCV بالتحقق المتقاطع على التدريب فقط
افتراض أن دقة تدريب 1.0 عند k = 1 دليل جودة	إنها حفظ حرفي للبيانات، والاختبار هو الحكم	قِس على بيانات لم يرها النموذج وقارن الهوة بين الدقتين

٧.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
النموذج الكسول	نموذج يؤجّل كل الحساب إلى لحظة التنبؤ ولا يدرب شيئاً	Lazy learner
التصويت بالأغلبية	اختيار الفئة الأكثر تكراراً بين الجيران k	Majority voting
أوزان المسافة	ترجيح صوت الجار بمقلوب بعده عن عينة الاستعلام	Distance weighting
انحدار الجيران	تقدير القيمة المستمرة بمتوسط قيم الجيران الأقرب	KNeighborsRegressor
لعنة الأبعاد	تقارب المسافات وانهيار معنى الجوار في الأبعاد العالية	Curse of dimensionality

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٥ — بُنى البحث في الجيران

KDTree و BallTree و RadiusNeighbors ومحوّلات الجيران

الوحدة **scikit-learn ١.٩** الحجم **١٠٥٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تفسر كيف تقسم KDTree المحاور وكيف تحاصر BallTree بكرات، ومتى يتفوق كل منهما
- أن تتنبأ بقرار 'algorithm='auto قبل أن يصدر وتضبط leaf_size بين السرعة والذاكرة
- أن تختار المقياس الصحيح من جدول عملي يشمل minkowski و mahalanobis و haversine
- أن تستخدم RadiusNeighborsClassifier مع outlier_label وتقرأ kneighbors_graph كمصفوفة أوزان
- أن تبني NearestCentroid وتستغل pairwise_distances_argmin_min الجديدة في ١.٩

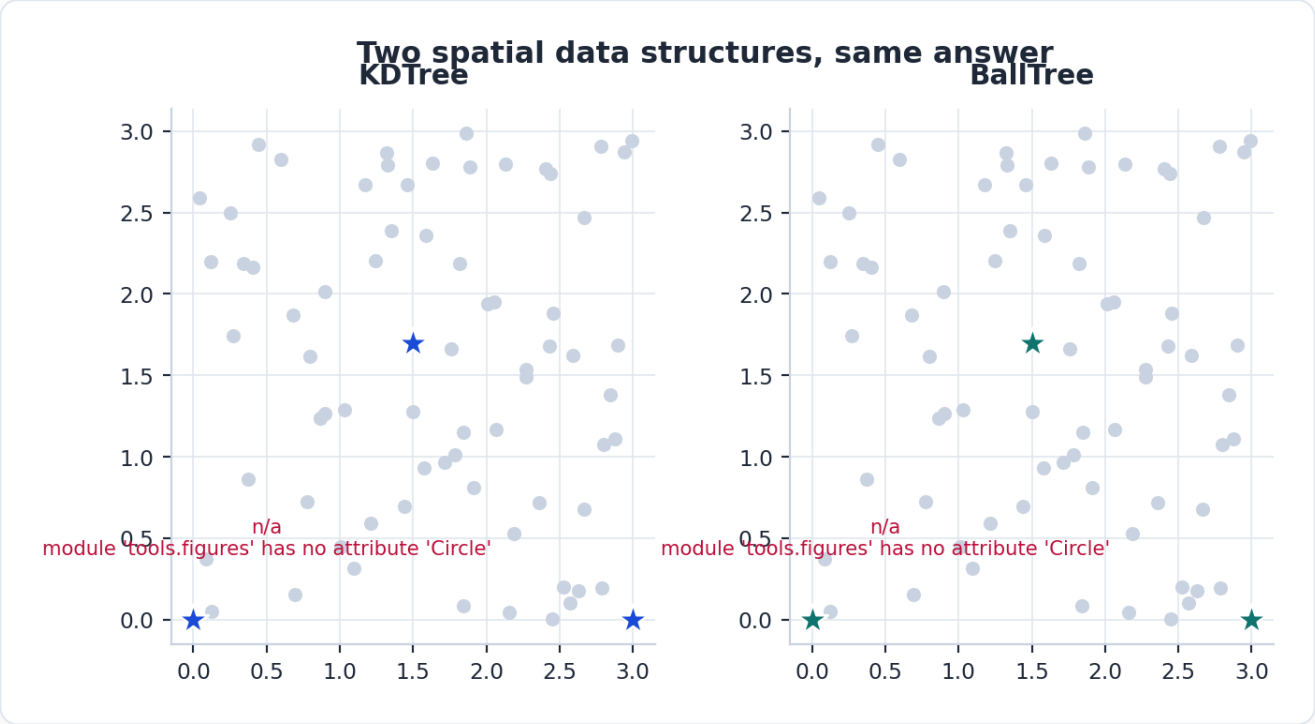
٧.١ من القسر إلى الحيوية: KDTree و BallTree

ترك الفصل الماضي فاتورة مفتوحة: كل استعلام يقيس المسافة إلى **جميع** عينات التدريب — بحث قسري (brute force). هل يمكن الجواب عن «من أقرب جار؟» دون مسح كل النقاط؟ نعم، إن رتّبنا النقاط مسبقاً في بنية تسمح بتجاهل حزم كاملة دفعة واحدة. تمنحنا scikit-learn بنيّتين: `sklearn.neighbors.KDTree` تقسم الفضاء بمنصّفات عمودية على المحاور بالتناوب (k-dimension tree) وتزدهر حين تكون d أقل من نحو ٢٠، و `sklearn.neighbors.BallTree` تحاصر كل مجموعة نقاط بأصغر كرة متراصة فتتحمل الأبعاد الأعلى وهي الوحيدة التي تدعم المقاييس المخصصة مثل هافرسين.

السر كله في القصّ (pruning): إذا كانت كل نقاط منطقة ما داخل كرة مركزها c ونصف قطرها R، وكان الجار رقم k عُثر عليه حتى الآن يبعد d_k، فإن الشرط التالي يغفر لك مسح المنطقة كلها:

$$d(\mathbf{x}, \mathbf{c}) - R \;\geq\; d_k \quad\Longrightarrow\quad \text{skip}$$

حيث x هي الاستعلام: إذا لم تستطع الكرة منافسة الجوار فاقطعها بجملة حساب واحدة بدل آلاف المسافات. هذا القصّ يعمل بامتياز في بعدين أو ثلاثة، ويهاوي حين تتساوى المسافات في الأبعاد العالية.



الشكل ١١٩بنيتان لنفس السؤال: KDTree تقسم بمحاور متعامدة و BallTree تحاصر بكرات

الشكل يعرض استعلامين نجميين أمام سبعين نقطة: منصفات KDTree المتعامدة يساراً، وكرات BallTree المتراكبة يميناً. الجواب واحد في الحالتين؛ ما يتغير هو عدد الخطوات حتى الوصول إليه. ولأن الأرقام خير شهادة، هذه مقارنة حقيقية لزمن التنبؤ على ٢٠٬٠٠٠ نقطة و٢٠٠ استعلام (أفضل ٣ قياسات مع تسخين مسبق):

```python
brute مقابل kd_tree مقابل ball_tree مع تزايد d
```

```python
import numpy as np
import time
from sklearn.neighbors import NearestNeighbors

rng = np.random.default_rng(42)
X_warm = rng.uniform(size=(1000, 3)); Q_warm = rng.uniform(size=(20, 3))
for alg in ["brute", "kd_tree", "ball_tree"]:
    NearestNeighbors(algorithm=alg).fit(X_warm).kneighbors(Q_warm)  # تسخين

print(f"{'d':>4} | {'brute':>8} | {'kd_tree':>8} | {'ball_tree':>9}")
for d in [2, 10, 20, 50]:
    X = rng.uniform(0, 1, (20000, d)); Q = rng.uniform(0, 1, (200, d))
    row = []
    for alg in ["brute", "kd_tree", "ball_tree"]:
        best = float("inf")
        for _ in range(3):
            nn = NearestNeighbors(n_neighbors=5, algorithm=alg).fit(X)
            t0 = time.perf_counter(); nn.kneighbors(Q); t = time.perf_counter()
- t0
            best = min(best, t)
        row.append(best * 1000)
    print(f"{d:4d} | {row[0]:8.1f} | {row[1]:8.1f} | {row[2]:9.1f}")
```

اقرأ الجدول كقصة انتصار ثم انقلاب: في بعدين تقطع الأشجار الطريق على القسري بسرعة تفوقه سبع مرات، ثم يتعثر القصّ عند d = ١٠ وينقلب الموازين، وعند d = ٢٠ وما فوق تصير الأشجار أبطأ بعشرات الأضعاف لأن كل كرة تقريباً «قد تختصم» فلا يُقص شيء. البيانات الصمّاء الموزعة بانتظام لا تنقذها أشجار؛ والقسري المتجهي (الممول بـ BLAS) أذكى مما يبدو. حكمك النهائي قياس لا افتراض.

المفهوم المحوري

الأشجار لا تغيّر الجواب أبداً — تغيّر طريق الوصول إليه. جرّب دائماً `algorithm="brute"` كخط أساس قبل أن تفترض تفوق الشجرة؛ المقايضة تقلب مع الأبعاد وعدد الاستعلامات وحجم البيانات.

٧.٢ محكمة 'algorithm='auto والمعلم الصامت leaf_size

البارامتر `algorithm="auto"` ليس سحراً بل قواعد مكتوبة في مصدر المكتبة، تفحص بالترتيب: مدخل متفرق أو مقياس `precomputed`؟ ← القسري حتماً. عدد الخصائص d أكبر من ١٥، أو k يساوي نصف العينات أو أكثر؟ ← القسري؛ القصّ سيخسر وطلب الجميع يجعل الشجرة بلا معنى. مينكوفسكي مع p أقل من ١؟ ← القسري؛ مسافة «شبه

مقياس» تخالف المتباينة المثلثية التي تُبنى عليها الأشجار. مينكوفسكي بأوزان `w`؟ → BallTree حصراً. المقياس من عائلة KDTree (إقليدية، مانهاتن، تشيبيشيف)؟ → KDTree. مقياس قابل للاستدعاء أو من مقاييس BallTree (هافرسين، ماهالانوبيس)؟ → BallTree.

والخاصية الداخلية `_fit_method` تكشف القرار النهائي بعد `fit`. لنستجوب المحكمة بخمس قضايا مختلفة:

```python
# استجواب 'algorithm='auto: أي بنية ستُختار؟
import numpy as np
from scipy import sparse
from sklearn.neighbors import NearestNeighbors

rng = np.random.default_rng(42)
cases = [
    ("عينة 100 كثيف x 2 خصيصة، k=5",    rng.uniform(size=(100, 2)), None, 5),
    ("عينة 100 كثيف x 20 خصيصة، k=5",   rng.uniform(size=(100, 20)), None, 5),
    ("عينة 100 كثيف x 2، لكن k=50",     rng.uniform(size=(100, 2)), None, 50),
    ("مقياس هافرسين على إحداثيات",      rng.uniform(-np.pi, np.pi, size=(100, 2)),
"haversine", 5),
    ("100 مصفوفة متفرقة x20",           sparse.random(100, 20, density=0.1,
random_state=42), None, 5),
]
for name, X, metric, k in cases:
    nn = (NearestNeighbors(n_neighbors=k) if metric is None
          else NearestNeighbors(n_neighbors=k, metric=metric))
    print(f"{name:38s} -> {nn.fit(X)._fit_method}")
```

القضايا تنطق بالحكم المتوقع: كثيفة قليلة الأبعاد تذهب إلى *kdtree*، *والعشرون خصيصة وطلب ٥٠ جاراً من ١٠٠ عينة يهدمان الشجرة، وهافرسين تقود إلى* *ball*tree، والمتفرقة قسراً إلى brute. تذكّر أن `_fit_method` خاصية داخلية للتشخيص التعليمي فقط — لا تبنِ عليها كود إنتاجك.

أما `leaf_size` (افتراضه ٣٠) فيحدد كم نقطة تحمل كل ورقة كحد أقصى قبل التوقف عن التقسيم، ويتحكم بذلك في الذاكرة وعمق الشجرة وعمق القص المتاح. قيمه المتطرفة خاسرة من الطرفين: الأوراق الدقيقة جداً (١) تضاعف عدد العقد فتتضخم الشجرة ويتأخر البناء، والأوراق الضخمة (مثل ١٢٠) تحوّل كل ورقة إلى مسح خطي طويل يُبطل فائدة القص؛ والوادي بينهما عريض يقع قرب ١٥ إلى ٣٠ — وهذا سبب اختيار الافتراض ٣٠.

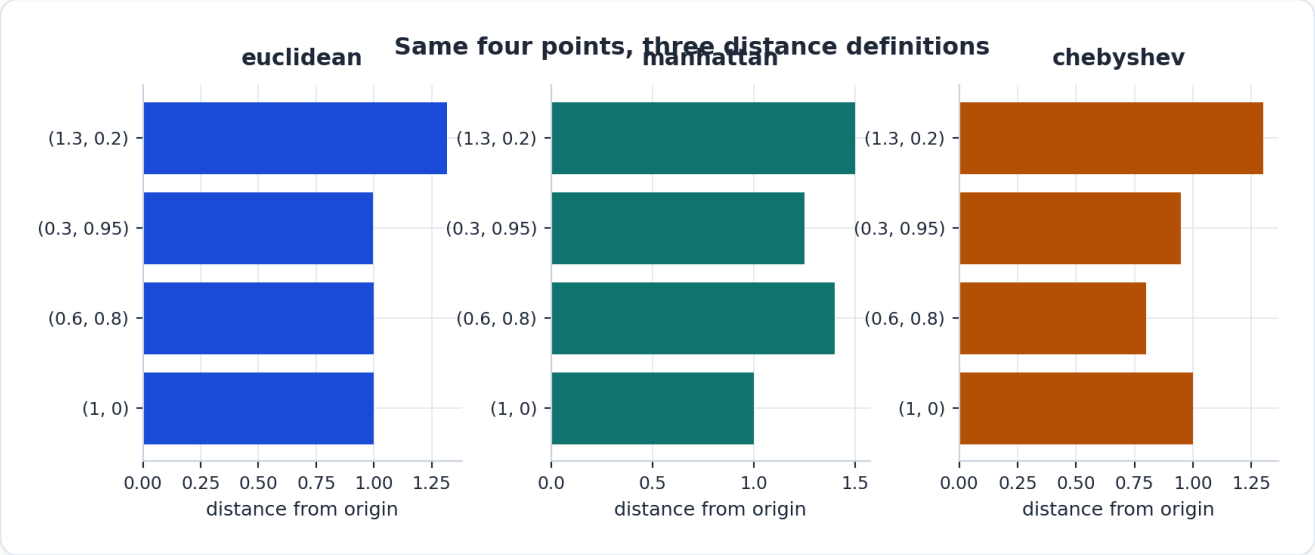
> **نصيحة**
>
> لا تضبط leaf_size قبل ضبط k والتطبيع؛ أثره على الدقة صفر تماماً (الجواب لا يتغير) وأثره الوحيد في السرعة والذاكرة. ارفعه حين تكون الذاكرة ضيقة أو البناء متكرراً، وأنزله حين تكون الاستعلامات ملايين والبيانات ثابتة.

٧.٣ المقاييس: من مينكوفسكي إلى هافرسين

المسافة نفسها قرار نمذجة. العائلة الأشهر عائلة مينكوفسكي المعطَّلة ببارامتر p الوحيد:

$$d_p(\mathbf{x}, \mathbf{z}) = \left(\sum_{j=1}^{d} |x_j - z_j|^p \right)^{1/p}$$

حيث p = ٢ تعيد الإقليدية، و p = ١ مسافة مانهاتن (أقل حساسية للقيم الشاذة)، وكلما كبر p اقتربنا من تشيبيشيف التي تكتفي بأكبر فرق وحده. الشكل التالي يقيس النقاط الأربع نفسها بثلاثة عيون:



الشكل ١٢٠ أربع نقاط نفسها ثلاث مقاييس: ترتيب القرب يتغير مع تعريف المسافة

في الشكل أعمدة البعد عن نقطة الأصل بثلاثة مقاييس: النقطة (٠.٣، ٠.٩٥) الأبعد إقليدياً تصبح الأولى بمانهاتن، وبتشبيشيف يتفوق أكبر محور وحده. «أقرب جار» كيان نسبي يتغير بتغيّر المقياس قبل أي نقاش عن النموذج.

المقياس	متى تستخدمه	ملاحظة عملية
`euclidean` (p=٢)	الافتراضي الآمن لخصائص رقمية مطبّعة	تعمل مع kd_tree
`manhattan` (p=١)	خصائص كثيرة وبعضها ضجيج	تعمل مع kd_tree
`mahalanobis`	خصائص مترابطة تحتاج التغاير	**القسري فقط** في ١.٩.١
`haversine`	إحداثيات جغرافية بالراديان	BallTree حصراً
`hamming`	متجهات ثنائية (وجود/غياب)	للبيانات الثنائية
دالة قابلة للاستدعاء	أي مقياس من صنعك	BallTree و brute فقط

> ⛔ **مقياس mahalanobis يرفض الأشجار في ١.٩.١**
>
> حاول استخدام `metric="mahalanobis"` مع الخوارزمية الافتراضية `algorithm="auto"` وستصطدم بـ ValueError من عمق الكود المصرّف (Mahalanobis dist: size of V does not match) لأن المحاولة توجّه الحساب إلى BallTree. الوصفة الصحيحة:
>
> `_neighbors=5, algorithm="brute", metric="mahalanobis", metric_params={"V": np.cov(X_train)})`
>
> حيث V مصفوفة التغاير المحسوبة من بيانات التدريب نفسها.

مثال هافرسين: أقرب المدن

أجمل تطبيق على المقاييس المخصصة هو المسافة الجغرافية. الإقليدية على خطوط الطول والعرض خطأ مضاعف: الدرجات ليست مسافات، وخطوط الطول تتقلص كلما ابتعدنا عن خط الاستواء. مقياس `haversine` يحسب المسافة على سطح كرة بدقة، بشرط أن تكون الإحداثيات **بالراديان**:

python أقرب ثلاث مدن إلى نقطة في صحراء الشارقة الجنوبية

```python
import numpy as np
from sklearn.neighbors import BallTree

cities = {
    "الرياض": (46.68, 24.71), "جدة": (39.19, 21.49), "دبي": (55.27, 25.20),
    "أبوظبي": (54.38, 24.45), "الدوحة": (51.53, 25.29), "مسقط": (58.41, 23.59),
    "الكويت": (47.99, 29.38), "القاهرة": (31.24, 30.04), "عمان": (35.93, 31.95),
}
names = list(cities)
coords = np.radians(np.array([cities[n] for n in names]))  # العرض ثم الطول
tree = BallTree(coords, metric="haversine")

d, i = tree.query(np.radians([[24.85, 55.10]]), k=3)  # نقطة صحراوية
for dist, idx in zip(d[0], i[0]):
    print(f"{names[idx]:8s} تبعد نحو {dist * 6371.0:6.1f} كم")  # نصف قطر الأرض
```

الأعمدة مرتّبة: أقرب مدينة تظهر أولاً بمسافة عشرات الكيلومترات ثم تتوسع الدائرة نحو العواصم المجاورة. ثلاث خطوات لا يجوز حذف أي منها: `np.radians` قبل البناء والاستعلام، و BallTree بمقياس هافرسين، وضرب المسافة الزاوية في نصف قطر الأرض ٦٣٧١ كم.

> **الدرجات تمر بصمت وتخريب كل شيء**
>
> هافرسين لا ترفض الدرجات ولا تحذّر منها؛ ستُحسب مسافات زاوية على كرة نصف قطرها ١ فتظهر أرقام صغيرة مقنعة وخطأ بمعامل ٥٧.٣. اجعل `np.radians` عادة مصاحبة لكل بناء واستعلام جغرافي.

٧.٤ RadiusNeighbors: الجيران بضوء النطاق

الفلسفة الافتراضية: «خذ أقرب k مهما بعوا». الفلسفة المعاكسة لدى `sklearn.neighbors.RadiusNeighborsClassifier`: «خذ كل من يقع داخل نصف قطر ثابت ولو كانوا أقل من k». نموذج يرفض الاستدلال من جيران بعيدين — لكنه يواجه مشكلة: ماذا لو لم يقع أحد داخل النطاق؟

RadiusNeighbors ووصفة outlier_label للنقاط اليتيمة python

```python
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.neighbors import RadiusNeighborsClassifier

X, y = make_blobs(n_samples=600, centers=3, cluster_std=[0.6, 0.6, 2.5],
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

strict = RadiusNeighborsClassifier(radius=1.5).fit(X_tr, y_tr)
try:
    strict.predict(X_te)
except ValueError as err:
    print(f"بدون outlier_label -> ValueError: {str(err)[:75]}...")

safe = RadiusNeighborsClassifier(radius=1.5,
outlier_label="most_frequent").fit(X_tr, y_tr)
print(f"مع outlier_label='most_frequent' -> تنبؤ دون أخطاء على {len(X_te)} عينة")
print(f"دقة النموذج مع النطاق 1.5: {safe.score(X_te, y_te):.4f}")
```

السطر الأول يكشف الفخ: predict نفسه هو الذي ينهار، لا fit — النقاط اليتيمة (بلا جيران داخل ١.٥) تفجّر التنبؤ بأكمله. والبارامتر `outlier_label` يعالجها بمنحها فئة محددة أو فئة الأغلبية `"most_frequent"`. وهذا الجسر المفاهيمي إلى كشف الشذوذ: النقطة التي لا يجد لها النطاق جيراناً على الأرجح شاذة — الفكرة نفسها التي يقوم عليها DBSCAN في الفصل ٦٣.

في الواقع: النطاق أم العدد؟

اختر RadiusNeighbors حين تعني الكثافة بحد ذاتها: «لا تصنّف من منطقة شحيحة». مواقع كشف الاحتيال تعتمد هذا: معاملة بلا جيران قريبين إشارة خطر بحد ذاتها. أما حين تتفاوت كثافات الفئات اختلافاً حاداً فالنطاق الثابت ظالم لبعضها، وعندها عد إلى k-NN.

٧.٥ kneighbors و kneighbors_graph: الجيران كمصفوفة

دالتان مركزيتان تتيحان الوصول إلى الجيران بلا تصنيف: `.kneighbors(X)` تعيد المسافات والمؤشرات مباشرة، و`sklearn.neighbors.kneighbors_graph` تحوّل علاقات الجوار إلى مصفوفة متفرقة تصلح مدخلاً لخوارزميات الرسوم البيانية:

KNeighborsTransformer ومحوّل kneighbors_graph و kneighbors python

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.neighbors import NearestNeighbors, kneighbors_graph,
KNeighborsTransformer

X, _ = load_iris(return_X_y=True)
nn = NearestNeighbors(n_neighbors=4).fit(X)
dists, inds = nn.kneighbors(X[:3])
print(f"مؤشرات جيران أول عينتين:\n{inds[:2]}")
print(f"المسافة إلى الجار الأول (النفس!): {dists[:, 0]}")

G = kneighbors_graph(X, n_neighbors=4, mode="distance")
T = (KNeighborsTransformer(n_neighbors=4, mode="distance")
     .set_output(transform="default")  # نريد المصفوفة المتفرقة كما هي
     .fit_transform(X))
print(f"kneighbors_graph: شكل {G.shape}، عناصر غير صفرية: {G.nnz}")
print(f"KNeighborsTransformer: شكل {T.shape}، عناصر غير صفرية: {T.nnz}")
```

تفصيلان يفصلان المحترف عن المبتدئ. أولاً: استدعاء `kneighbors(X)` على بيانات التدريب نفسها يعيد كل نقطة **جارتها الأولى لنفسها** — عمود الأصفار ليس خطأ بل نقطة النفس؛ اطلب k+١ واقصّ العمود الأول أو استدعِ `kneighbors()` بلا وسيط. ثانياً: الرسم البياني يحتفظ بـ ٤ جيران لكل صف (٦٠٠ عنصر)، بينما المحوّل يضيف النقطة نفسها بمسافة صفرية فتصير ٥ لكل صف (٧٥٠ عنصراً) ليصير الناتج مصفوفة تجاور صالحة للنماذج اللاحقة. وإذا احتجت رسماً غير موجَّه فتذكّر أن `kneighbors_graph` تنتج مصفوفة موجَّهة: أضف نقيضها ثم خفّض التكرار.

أما `KNeighborsTransformer` فهو محوّل صريح لخطوط الأنابيب: يحوّل X إلى مصفوفة متفرقة من مسافات الجيران فيصلح مدخلاً لأي مقدر يقبل مصفوفة تشابه — من SpectralClustering في الفصل ٦٥ (حيث تُبنى أوزان الرسم البياني حرفياً من هذا الجوار) إلى التضمينات غير المشرفة. الأوزان التي سترى هناك:

$$W_{ij} = \exp\!\left(\frac{-d(\mathbf{x}_i, \mathbf{x}_j)^2}{2\sigma^2}\right)$$

حيث d مسافات الجوار و sigma عرض نواة تضبطه؛ التصنيف الطيفي كله يقف على كتفي هذه الدوال البسيطة.

٧.٦ NearestCentroid ومساعد ١.٩ الجديد

أسرع مصنف في المكتبة كلها: `sklearn.neighbors.NearestCentroid`. الفكرة ساذجة إلى حد الابتسامة: احسب **مركز** كل فئة (متوسط عيناتها)، وصنّف أي عينة جديدة بأقرب مركز:

$$\mathbf{c}_c = \frac{1}{n_c} \sum_{i\,:\,y_i = c} \mathbf{x}_i, \qquad \hat{y}(\mathbf{x}) = \operatorname{argmin}_c \, d(\mathbf{x}, \mathbf{c}_c)$$

كلفة التنبؤ O(n_classes·d) فقط — لا تنمو مع حجم البيانات إطلاقاً. وقبل أن نجربه، مساعد جديد وصل مع ١.٩:

جديد في ١.٩

الدالة `sklearn.metrics.pairwise_distances_argmin_min` متاحة الآن كأداة معلنة، وتوقيعها: `pairwise_distances_argmin_min(X, Y, *, axis=1, metric="euclidean", metric_kwargs=None)`. تعيد ثنائية: مؤشر أقرب صف في Y لكل صف في X، والمسافة المقابلة. تحققنا من التوقيع على الإصدار ١.٩.١ قبل الاعتماد عليها — وهي مثالية لسؤال «أقرب مركز لكل نقطة» الذي تقوم عليه خوارزميات المراكز.

ثم التحقق اليدوي بالدالة الجديدة مقابل NearestCentroid k-NN python

```python
import numpy as np
import time
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestCentroid, KNeighborsClassifier
from sklearn.metrics import pairwise_distances_argmin_min

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

scaler = StandardScaler().fit(X_tr)  # التطبيع شرط لأي مسافة (الفصل ١٤)
Z_tr, Z_te = scaler.transform(X_tr), scaler.transform(X_te)
NearestCentroid().fit(Z_tr[:20], y_tr[:20]).predict(Z_te[:5])  # تسخين

for name, mdl in [("NearestCentroid", NearestCentroid()),
                  ("KNeighborsClassifier(k=5)",
KNeighborsClassifier(n_neighbors=5))]:
    mdl.fit(Z_tr, y_tr)
    acc = float((mdl.predict(Z_te) == y_te).mean())
    best = float("inf")
    for _ in range(3):
        t0 = time.perf_counter(); mdl.predict(Z_te); t = time.perf_counter() -
t0
        best = min(best, t)
    print(f"{name:27s} | دقة: {acc:.4f} | تنبؤ {len(Z_te)} عينة: {best * 1000:.2f}
ms")

# إعادة بناء التنبؤ يدوياً بالدالة الجديدة: أقرب مركز لكل نقطة
nc = NearestCentroid().fit(Z_tr, y_tr)
idx, dmin = pairwise_distances_argmin_min(Z_te, nc.centroids_)
match = float(np.mean(nc.classes_[idx] == nc.predict(Z_te)))
print(f"تطابق التنبؤ اليدوي (أقرب مركز) مع predict: {match:.2%}")
```

النتيجة تختصر الفلسفة: NearestCentroid يتفوق على k-NN هنا لأن فئات Wine متجمعة جمّياً حول مراكزها، وزمن تنبؤه أصغر لأنه يقارن كل نقطة بثلاثة مراكز فقط — والفجوة تتضاعف مع نمو البيانات لأن كلفته لا تنمو مع n. والسطر الأخير هو الدرس الحقيقي: التنبؤ اليدوي بأقرب مركز عبر الدالة الجديدة طابق `predict` تاماً؛ النموذج جملة واحدة من الجبر لا صندوق أسود. حدّه الواضح: الفئات متعددة الأنماط (كل فئة عنقودان متباعدان) يخدعه المتوسط — المركز يقع في الفراغ بينهما؛ عُد حينها إلى k-NN أو استخدم بارامتره `shrink_threshold` الذي يقلص المراكز نحو الصفر.

تمارين الفصل الخامس والخمسين

- **ملاحظة**

التمرين ٧.١

على بيانات `load_digits` (١٧٩٧ عينة، ٦٤ خصيصة)، قارن زمن تنبؤ `KNeighborsClassifier` مع `algorithm="brute"` مقابل `algorithm="kd_tree"` على ٢٠٠ استعلام. ماذا تلاحظ، وهل يوافق ذلك قواعد `algorithm="auto"`؟

التمرين ٧.٢

أنشئ بيانات `make_blobs(n_samples=500, centers=2, cluster_std=1.8, random_state=42)` وزرع ثلاث نقاط شاذة بعيدة. درّب `RadiusNeighborsClassifier(radius=2.0)` بدون ثم مع `outlier_label`، والتقط خطأ التنبؤ الأول وأظهر سلوك الثاني.

التمرين ٧.٣

ابنِ رسم البياني الجياري لبيانات Iris: استخدم `kneighbors_graph(X, 5, mode="distance")` ثم حوّل المسافات إلى أوزان تشابه $w = \exp(-d^2)$. اطبع عدد العناصر غير الصفرية وكثافة المصفوفة.

- **ملاحظة**

python حل التمرين ٧.١: kd_tree في 64 بعداً يخسر القسري

```python
import numpy as np
import time
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42,
stratify=y)

for alg in ["brute", "kd_tree"]:
    knn = KNeighborsClassifier(n_neighbors=5, algorithm=alg).fit(X_tr, y_tr)
    best = float("inf")
    for _ in range(3):
        t0 = time.perf_counter(); knn.predict(X_te[:200]); t =
time.perf_counter() - t0
        best = min(best, t)
    print(f"algorithm = {alg:8s} | استعلام 200 تنبؤ: {best * 1000:6.1f} ms")
```

الأرقام تكشف أن kd_tree أبطأ رغم الـ ٦٤ بعداً: عدد الخصائص أكبر من حد ١٥ فالقص يفشل عملياً ويدفعنا لتجوال في الشجرة بلا ربح. وهذا بالضبط ما تحسبه المحكمة `algorithm="auto"` فتختار brute من البداية.

ملاحظة

python حل التمرين ٧.٢: النقاط الشاذة تحت نطاق صارم

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.neighbors import RadiusNeighborsClassifier

X, y = make_blobs(n_samples=500, centers=2, cluster_std=1.8, random_state=42)
outliers = np.array([[30.0, 30.0], [-25.0, 28.0], [28.0, -22.0]])
X_all = np.vstack([X, outliers])
y_all = np.concatenate([y, [0, 1, 0]])  # فئات زرعتها للشواذ

strict = RadiusNeighborsClassifier(radius=2.0).fit(X_all, y_all)
try:
    strict.predict(outliers)
except ValueError as err:
    print(f"بدون outlier_label -> ValueError: {str(err)[:70]}...")

safe = RadiusNeighborsClassifier(radius=2.0, outlier_label=-1).fit(X_all, y_all)
print(f"تنبؤ النقاط الشاذة مع outlier_label=-1: {safe.predict(outliers)}")
print(f"عدد العينات التي صنفت شاذة (-1) من كل البيانات: {(safe.predict(X_all) ==
-1).sum()}")
```

النقاط الثلاث المزروعة بعيداً لا تجد جاراً داخل نصف القطر ٢.٠ فالتنبؤ الصارم ينهار بـ ValueError يسمّي مؤشرات العينات اليتيمة نفسها. مع `outlier_label=-1` تتحول اليتيمة إلى فئة صريحة يمكن اعتبارها وسماً بالشذوذ — لاحظ أن الشواذ الثلاث وحدها حملت الوسم.

> **ملاحظة**

٧.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
mahalanobis مع algorithm الافتراضي	ValueError من الكود المصرّف لأن auto توجهه إلى BallTree	اضبط `algorithm="brute"` مع `metric_params={"V": np.cov(X)}`
إدخال إحداثيات بالدرجات مع haversine	مسافات صغيرة مقنعة وخطأ بمعامل ٥٧.٣ دون تحذير	حوّل بـ `np.radians` قبل البناء والاستعلام
إجبار kd_tree في أبعاد عالية	يفشل القصّ فيصبح أبطأ من brute بعشرات الأضعاف	اترك `algorithm="auto"` أو قلل الأبعاد أولاً
قراءة kneighbors(X) والاندهاش من عمود أصفار	العينة نفسها تظهر جارتها الأولى بمسافة ٠	اطلب k+١ واحذف العمود الأول أو استدعِ kneighbors() بلا وسيط

٧.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البحث القسري	قياس المسافة إلى كل عينات التدريب لكل استعلام	Brute-force search
شجرة الأبعاد K	بنية تقسم الفضاء بمنصفات محاور متناوبة للتقليم	KDTree
شجرة الكرة	بنية تحاصر العناقيد بكرات دائرية متراكبة	BallTree
حجم الورقة	أقصى عدد نقاط في عقدة ورقية قبل التوقف عن التقسيم	leaf_size
مصنف أقرب المركز	تصنيف بأقرب متوسط لفئة، بكلفة تنبؤ لا تنمو مع n	NearestCentroid

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٦ — بيز الساذج

BernoulliNB و ComplementNB و MultinomialNB و GaussianNB

الوحدة **I.٩ scikit-learn** الحجم **١٠٥٨** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تطبق مبرهنة بيز يدوياً وتفسر Posterior و Likelihood و Prior
- أن تختار بين GaussianNB و MultinomialNB و ComplementNB و BernoulliNB و CategoricalNB بحسب بياناتك
- أن تضبط alpha (تمهيد لابلاس) وتفسر أثره على الكلمات غير المرئية في التدريب
- أن تدرب خارج الذاكرة عبر partial_fit على تدفقات دفعات لا تحتمل التحميل الكامل
- أن تعرف متى يخدع بيز الساذج: الخصائص المترابطة والاحتمالات غير المعايرة

٧.١ مبرهنة بيز بمثال بريد مزعج

رسالة جديدة وفيها كلمتا «مجاني» و«عرض». قبل أي تعلّم آلة يسأل القارئ القديم: ما احتمال أن تكون مزعجة (spam)؟ جواب القس توماس بيز (مقالة نشرت عام ١٧٦٣) بسيط وجريء: لا تخمّن الاحتمال مباشرة، بل اعكس السؤال. بدل «ما احتمال الرسالة المزعجة نظراً للكلمات؟» اسأل «ما احتمال الكلمات نظراً للإزعاج؟» — وهذا سهل القياس من الأرشيف:

$$P(C_k|\mathbf{x}) = \frac{P(C_k)\,P(\mathbf{x}|C_k)}{P(\mathbf{x})} \quad \Longrightarrow \quad \text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

حيث `P(C_k)` هو السابق (Prior): نسبة الفئة k قبل رؤية أي شيء، و `P(x | C_k)` هو الإمكان (Likelihood): احتمال رؤية هذه الخصائص إذا كانت الرسالة من الفئة k، و `P(C_k | x)` هو اللاحق (Posterior) الذي نريده، بينما المقام `P(x)` ثابت لكل الفئات فنقارن الأبسطين فقط. لنحسب يدوياً بمثال مصغّر من ٥٠٠ رسالة أرشيفية:

مبرهنة بيز يدوياً على رسالة فيها «مجاني» و«عرض» python

```python
# أرشيف: 100 رسالة مزعجة و400 عادية
prior_spam, prior_ham = 100 / 500, 400 / 500

# إحصاءات الأرشيف: احتمال الكلمتين في كل فئة
p_magan_free_spam, p_magan_free_ham = 0.60, 0.03   # «مجاني»
p_ard_spam, p_ard_ham = 0.50, 0.10                 # «عرض»

# الفرضية الساذجة: الكلمتان مستقلتان شرطاً داخل كل فئة
score_spam = prior_spam * p_magan_free_spam * p_ard_spam
score_ham = prior_ham * p_magan_free_ham * p_ard_ham

print(f"ناتج بيز للفئة المزعجة: {score_spam:.5f} | للفئة العادية: {score_ham:.5f}")
posterior = score_spam / (score_spam + score_ham)
print(f"الاحتمال اللاحق أن الرسالة مزعجة: {posterior:.2%}")
```

رقمان صغيران ونسبتهما تختصر كل شيء: ٠.٠٦٠٠ مقابل ٠.٠٠٢٤٠ تعني إزعاجاً بنسبة ٩٦.١٥%. لاحظ البنية المحاسبية: بدأنا بتوازن أولي (الإزعاج ٢٠% فقط) ثم ضاعفت الأدلة لصالحه لأن الكلمتين نشطتين فيه أكثر بكثير. ولم نتدرب تقنياً — فقط عدّنا.

المفهوم المحوري

بيز الساذج لا يبحث عن حد فاصل ولا عن عنقود؛ يحسب لكل فئة «درجة دليل» ويفوز صاحب الأعلى. القسمة على `P(x)` لا تهم لأنها واحدة للجميع — وهذا سبب إحلال `argmax` محل النسبة الكاملة في كل التنفيذات.

٧.٢ «السذاجة»: كذبة مفيدة

الخطوة التي قفزناها — احتمال الكلمتين معاً = حاصل ضرب احتمال كل كلمة منفردة — هي كل السؤال. اسمها فرضية الاستقلال الشرطي: الخصائص لا تعرف شيئاً عن بعضها داخل كل فئة. وهذا **كذب صريح**: «مجاني» و«عرض» تتحالفان في الواقع كثيراً. لكن الكذبة تشتري ما لا تشتريه الحقيقة:

$$P(\mathbf{x}|C_k) = \prod_{j=1}^{d} P(x_j|C_k) \quad \Longrightarrow \quad \hat{y} = \operatorname{argmax}_k [\log P(C_k) + \sum_{j=1}^{d} \log P(x_j|C_k)]$$

بدل توزيع مشترك بأبعاد d (مستحيل عملياً لأن التركيبات تنفجر أُسّياً — انظر لعنة الأبعاد في الفصل ٢١)، نقدّر d توزيعاً أُحادياً رخيصاً لا يتقاطع. والانتقال إلى اللوغاريتم يحوّل جداء آلاف الحدود الصغيرة إلى مجموع أرقام سالبة مريحة — لا اغراق عددي (underflow) ولا انفجار. النتيجة: مصنف يتدرب في أجزاء من الثانية على مئات آلاف الرسائل، ويفوز تاريخياً

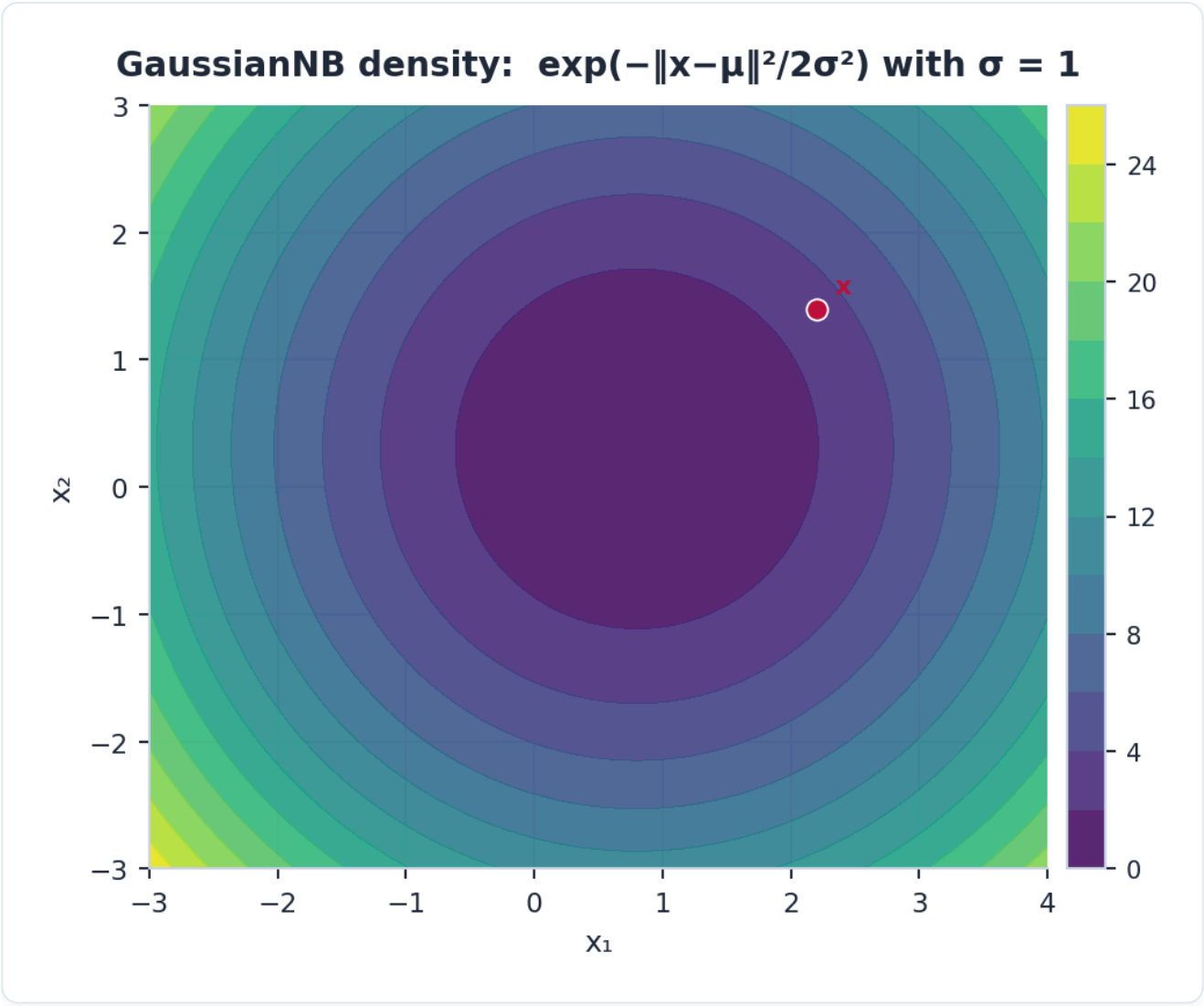
على النصوص القصيرة رغم سذاجته. والفصل ٥٧ سيلتقي بأبناء عمومتها الجادين (LDA و QDA) الذين يدفعون ثمن عدم السذاجة بحساب مصفوفات تغاير كاملة.

GaussianNB V.٣: بيز للخصائص المستمرة

ماذا لو كانت الخصائص أرقاماً مستمرة لا كلمات؟ فئة `sklearn.naive_bayes.GaussianNB` تفترض أن كل خاصية داخل كل فئة تتبع توزيعاً غاوسياً — متوسط وانحراف لكل عمود في كل فئة، ولا شيء أكثر:

$$P(x_j|C_k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}}\exp(\frac{-(x_j-\mu_{jk})^2}{2\sigma_{jk}^2})$$

حيث `mu_jk` متوسط الخاصية j داخل الفئة k و `sigma_jk` انحرافها. هذا يفسر شكل حدود النموذج في الشكل أدناه. والبارامتر الوحيد المهم هو `var_smoothing` (افتراضه 1e-٩): نسبة تضاف إلى تباين كل خاصية كي لا يواجه النموذج تبايناً صفراً لو ظهر عمود ثابت القيمة.



الشكل ١٢١كثافة غاوسية ثنائية بيز الساذج: بؤرة واحدة موجهة بالمحاور نفسها

في الشكل كثافة احتمال لتوزيع غاوسي لم يزدن التغاير: القمم محاذية للمحاور x١ وx٢ مباشرة، لأن النموذج يتعامل مع كل بعد بصغرصل — لا ميلان ولا استطالة قطرية مهما بلغت علاقة الخصائص في البيانات. جرّب الآن على بيانات طبية حقيقية:

```python
# GaussianNB على بيانات الثدي وأثر var_smoothing
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

gnb = GaussianNB().fit(X_tr, y_tr)
print(f"دقة GaussianNB على الاختبار: {gnb.score(X_te, y_te):.4f}")

for vs in [1e-9, 1e-4, 1e-1]:
    score = cross_val_score(GaussianNB(var_smoothing=vs), X, y, cv=5).mean()
    print(f"var_smoothing = {vs:6.0e} | متوسط دقة التحقق الخماسي: {score:.4f}")
```

دقة تقارب ٠.٩٥ بتدريب يقيس أجزاء من الملي ثانية، وبدون أي تطبيع — والسبب أن الغاوسية تقدّر متوسطاً وتبايناً لكل عمود بنطاقه هو فتُلغى مسألة المقياس ذاتياً، عكس k-NN في الفصل السابق حيث كان التطبيع شرط حياة. ولاحظ أن القيمة الافتراضية ١e-٩ هي الأفضل (٠.٩٣٨٥) وكلما ضخّمنا التمهيد هبطت الدقة إلى ٠.٨٨٤٠ عند ١e-١؛ var_smoothing عقوبة لا فائدة إلا لحالات حدّية ذات تباينات منخفضة جداً.

٧.٤ MultinomialNB: عدّادات الكلمات و alpha

حين تكون خصائصك **عدّادات** (كم مرة ظهرت كلمة في مستند)، الفئة المناسبة هي `sklearn.naive_bayes.MultinomialNB`، ونبني مدخلاتها من `sklearn.feature_extraction.text.CountVectorizer` كما فصّل الفصل ١٨. وهنا أخطر فخ احتمالي في التاريخ: كلمة لم تظهر إطلاقاً في رسائل السبام أثناء التدريب يكون احتمالها صفراً، فتضرب باقي الدليل في صفر وتهدم الحكم كله مهما كانت بقية الكلمات صارخة. العلاج تمهيد لابلاس (Laplace smoothing): أضف alpha وهمياً إلى كل عدّاد قبل التقسيم:

$$\hat{\theta}_{ck} = \frac{N_{ck} + \alpha}{N_c + \alpha\, d}$$

حيث `N_ck` عدد مرات ظهور الكلمة k في فئة c، و `N_c` مجموع عدّادات الفئة، و d حجم المفردات. باختيار alpha = ١ تحصل على التمهيد الكلاسيكي، والقيم الأصغر مثل ٠.٠١ تمهيد خفيف اليد أدق على البيانات الكبيرة. وفي scikit-learn الحديثة البارامتر `force_alpha` افتراضه True أي أن قيمة alpha تحترم كما هي حتى لو كانت صغيرة جداً:

python MultinomialNB على مرسلات عربية مع تمهيد لابلاس

```python
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

spam = ['عرض مجاني اربح الآن بسرعة', 'فزت جائزة مجانية اضغط الآن',
        'قرض مجاني بلا فوايد اربح اليوم', 'عرض خاص ساعات مجانية اليوم',
        'اربح المال الآن من المنزل', 'جائزة فورية اضغط هنا']
ham = ['اجتماع الفريق غدا في التاسعة', 'أرسل لي تقرير المشروع من فضلك',
       'موعد الطبيب يوم الثلاثاء', 'تقرير المبيعات جاهز للمراجعة',
       'شكرا على ملاحظاتك على الوثيقة', 'سافر الجمعة وأعود الأحد']

docs = spam + ham
y = np.array([1] * len(spam) + [0] * len(ham))
vec = CountVectorizer()
X = vec.fit_transform(docs)
print(f"مصفوفة العدّاد: {X.shape[0]} مستند و {X.shape[1]} كلمة، عناصر فعالة: {X.nnz}")

for a in [0.01, 0.1, 1.0]:
    m = MultinomialNB(alpha=a).fit(X, y)
    print(f"alpha = {a:5.2f} | دقة التدريب: {m.score(X, y):.4f}")

mnb = MultinomialNB(alpha=0.1).fit(X, y)
print(f"تنبؤ رسائل جديدة: {mnb.predict(vec.transform(['اربح جائزة مجانية الآن', 'تقرير الاجتماع غدا']))}")
```

اثنا عشر مستنداً وخمس وأربعون كلمة، ومع ذلك يصنّف النموذج الرسالتين الجديدتين بلا تردد: «اربح جائزة مجانية الآن» سبام والثانية عادية. هذا سحر العدّادات: لا فهم للدلالات، كافٍ أن عائلة كلمة «مجاني» ظهرت أربع مرات في أرشيف الإزعاج ولم تظهر في العادي ولا مرة. وحين تتضخم البيانات إلى ملايين الرسائل يظل alpha البارامتر الوحيد الذي يستحق البحث الشبكي.

في الواقع: لماذا يعيش بيز في كل صناديق البريد؟

أنظمة البريد الكبرى عالجت ملايين الرسائل يومياً لعقود بنماذج بيز أو قريبة منها: تدريب فوري، وذاكرة تتناسب مع حجم المفردات لا مع عدد الرسائل، وتحديث مستمر بكلمة جديدة عبر partial_fit. حين يكون الموعد النهائي غداً والبيانات مليون مستند نصي، ابدأ بـ MultinomialNB كخط أساس؛ قد تجد أنه خط النهاية أيضاً.

V.٥ العائلة الموسعة: Categorical و Bernoulli و Complement

ثلاثة أشقاء يعالجون ثلاث ثغرات. **ComplementNB** (`sklearn.naive_bayes.ComplementNB`) يحل مشكلة الفئات غير المتوازنة: بدل نمذجة ما تشبهه الفئة، يقدّر ما **لا** تشبهه (الفئة المكمّلة) ويزن الدليل بحيث لا تبتلع الفئة المهيمنة القرار. تجربة قاطعة على عدّادات صناعية غير متوازنة (٩٠٠ مقابل ١٠٠):

ComplementNB ينقذ الفئة النادرة من الغياب التام — python

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, ComplementNB

rng = np.random.default_rng(42)
n_maj, n_min = 900, 100
# فئة مهيمنة: كلمات عامة بكثافة متفاوتة
X_maj = rng.poisson(lam=4.0, size=(n_maj, 30)) * (rng.uniform(size=(n_maj, 30))
< 0.3)
# فئة نادرة: نفس الكلمات تقريباً مع تحيّز طفيف
X_min = rng.poisson(lam=1.5, size=(n_min, 30)) * (rng.uniform(size=(n_min, 30))
< 0.6) \
        + (rng.uniform(size=(n_min, 30)) < 0.2) * 3
Xb = np.vstack([X_maj, X_min]).astype(float)
yb = np.array([0] * n_maj + [1] * n_min)
X_tr, X_te, y_tr, y_te = train_test_split(Xb, yb, test_size=0.3,
random_state=42, stratify=yb)

for name, model in [("MultinomialNB", MultinomialNB()), ("ComplementNB",
ComplementNB())]:
    model.fit(X_tr, y_tr)
    pred = model.predict(X_te)
    print(f"{name:13s} | الدقة الإجمالية: {(pred == y_te).mean():.4f} | "
          f"استرجاع الفئة النادرة: {pred[y_te == 1].mean():.4f}")
```

اقرأ العمود الأخير وهو الحكاية: MultinomialNB يقفز على الأغلبية فيكتشف ٣.٣% فقط من الفئة النادرة — دقة إجمالية مبهرة (٠.٨٣٦٧) فوق إخفاق كامل في المهمة الفعلية. ComplementNB يرفع الاسترجاع إلى ٠.٤٠٠٠ دافعاً ثمناً من الدقة الإجمالية. وعلى بيانات نصية حقيقية غير متوازنة تتفوق ComplementNB على شقيقتها غالباً؛ فسّر اختيارك دائماً بالمقياس الذي يهم مسألتك (الفصل ٧٥).

أما **BernoulliNB** (`sklearn.naive_bayes.BernoulliNB`) فيعامل كل ميزة كقرار ثنائي: ظهرت الكلمة أو غابت بغضّ النظر عن التكرار، والبارامتر `binarize` يقصّ العتبة — استخدمه حين يكون «الوجود» هو الإشارة لا «الكمية». وأخيراً **CategoricalNB** (`sklearn.naive_bayes.CategoricalNB`) لخصائص فئوية حقيقية مرقّمة (٠، ١، ٢...) كاللون والمستوى التعليمي: توزيع فئوي منفصل لكل عمود، ولا يقبل مدخلات سلبية أو مستمرة:

```python
# BernoulliNB: وجود الكلمة يكفي و CategoricalNB للخصائص الفئوية
from sklearn.naive_bayes import BernoulliNB, CategoricalNB
import numpy as np

# نثنّيها بعتبة 0.5 — y و X نفس مرسلات القسم السابق محفوظة في
bnb = BernoulliNB(binarize=0.5).fit(X, y)
print(f"دقة BernoulliNB على المرسلات: {bnb.score(X, y):.4f}")

rng2 = np.random.default_rng(42)
Xf = rng2.integers(0, 3, size=(200, 4))
yf = Xf[:, 0] % 2  # الفئة تحددها قيمة الخاصية الأولى فقط
cnb = CategoricalNB().fit(Xf, yf)
print(f"دقة CategoricalNB على الفئوي: {cnb.score(Xf, yf):.4f}")
```

الفئة	نوع المدخل	حالتها المثالية	بارامترها المحوري
GaussianNB	أرقام مستمرة	قياسات طبية، حساسات	`var_smoothing`
MultinomialNB	عدّادات غير سالبة	تكرار الكلمات في نصوص	`alpha`
ComplementNB	عدّادات غير متوازنة	نصوص بفئات نادرة	`alpha`
BernoulliNB	ثنائية ٠/١	وجود/غياب الكلمة	`binarize`
CategoricalNB	فئوية مرقّمة	لون، مستوى، تصنيف	`alpha`

٧.٦ التدريب خارج الذاكرة وحد الصفر الاحتمالي

بيز الساذج يملك هدية نادرة: إحصاءاته **تراكمية**. المتوسط والتباين والعدّادات تُحدَّث بجمل حساب عند وصول كل دفعة، فلا تحتاج أبداً إلى تحميل كامل البيانات. الطريقة `partial_fit` هي الواجهة الرسمية لذلك، مع تمرير `classes` في الدفعة الأولى كي يعرف قائمة الفئات كاملة:

```python
partial_fit: 12 ألف عينة عبر ست دفعات لا تراها الذاكرة
import numpy as np
from sklearn.datasets import make_classification
from sklearn.naive_bayes import GaussianNB

X, y = make_classification(n_samples=12000, n_features=8, random_state=42)
classes = np.unique(y)

stream_model = GaussianNB()
for start in range(0, 12000, 2000):
    stream_model.partial_fit(X[start:start + 2000], y[start:start + 2000],
classes=classes)

full_model = GaussianNB().fit(X, y)
print(f"دقة التدريب بالدفعات: {stream_model.score(X, y):.4f}")
print(f"دقة التدريب الكامل:   {full_model.score(X, y):.4f}")
print(f"تطابق المتوسطات المقدَّرة: {np.allclose(stream_model.theta_,
full_model.theta_)}")

lp = full_model.predict_log_proba(X[:3])
print(f"predict_log_proba: أول صف من {lp[0].round(4)}")
print(f"مجموع exp لكل صف: {np.exp(lp).sum(axis=1).round(4)}")
```

الدقتان متطابقتان إلى آخر خانة، والمتوسطات متساوية تماماً — التدريب بالدفعات ليس تقريباً بل الحساب نفسه موزعاً على دفعات. وفي السطرين الأخيرين لمسنا الغرض الثاني من اللوغاريتم: القيم أرقام سالبة صغيرة مثل -٦.١٩١٨، وترفعها `np.exp` إلى احتمالات مجموعها ١.٠ بالضبط. لماذا لا نطبع الاحتمالات مباشرة؟ لأن احتمال الفئة الخاسرة قد يكون ٠.٠٠٠٠٢، وحين تتضاعف عشرات هذه الحدود في المنتج الأصلي تختفي تحت أرضية العدد العائم (تُقرأ صفراً) فينهار المنطق كله — بينما مجموع اللوغاريتمات يبقى عدداً سالباً عادياً.

فجوة الأصفار تكمن في التنبؤ لا في التدريب

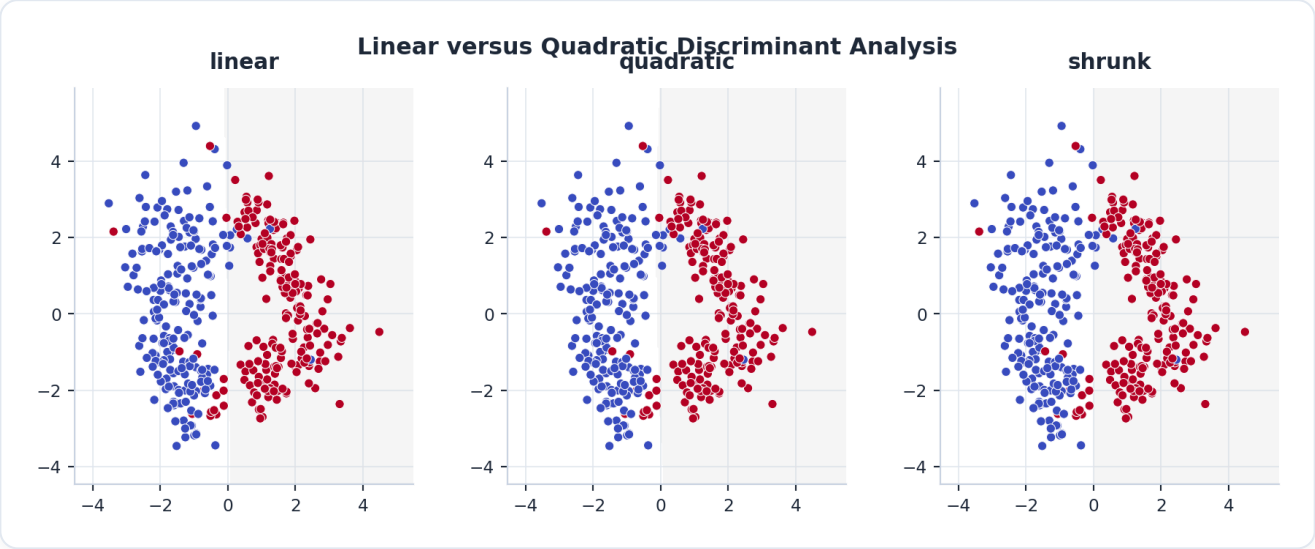
التمهيد بـ alpha يعالج العدّادات، لكن انتبه في GaussianNB إلى عمود ثابت القيمة (تباينه صفر): الاحتمال الغاوسي عنده ينفجر. أبقِ *varsmoothing في مكانه الافتراضي وأزِل الأعمدة الثابتة مسبقاً بـ* `sklearn.feature`selection.VarianceThreshold من الفصل ٢٢.

نصيحة

استخدم `predict_log_proba` دائماً حين تريد مقارنة الفئات أو جمع الأدلة عبر مراحل (كمزج مصنفات في الفصل ٤٨)؛ وأعده للاحتمال بـ `np.exp` في آخر لحظة فقط عند العرض للمستخدم.

٧.٧ حدود بيز الساذج: حين تخدعه البيانات

موضعان يجب أن تحفظهما كحدود صريحة. **أولهما الارتباط القوي بين الخصائص يخدعه مرتين**: إحصائياً لأن الفرضية تضاعف الدليل نفسه مرتين — خاصيتان مترابطتان تماماً (كالطول بالسنتيمتر وبالبوصة) تصوّتان على الدليل نفسه بلا معلومة جديدة — وهندسياً لأن الشكل في القسم ٧.٣ كشف أن حدود GaussianNB محاذية للمحاور دائماً؛ فأي فئات يفصلها قطر مائل لا مستوٍ عمودي سيعجز عنه. قارن بنفسك:



الشكل ١٢٢ حدود LDA الخطية المائلة مقابل حدود QDA التربيعية على نفس البيانات

في الشكل حدود مائلة تقطع المحاور بزاوية — وهذا بالضبط ما يمنحه تحليل التمييز في الفصل ٥٧ بحساب مصفوفة التغاير المشتركة؛ غياب هذه المصفوفة عن بيز الساذج هو الكلفة التي دفعناها مقابل السرعة، وهو سبب خسارته عادة أمام LDA حين تترابط الخصائص بقوة.

وثانيهما الاحتمالات المطلقة غير معايرة: لأن فرضية الاستقلال كذبة، يتضخم الدليل أو ينكمش — قيمة ٠.٩٩٩٩ من `predict_proba` تعني «هذه الفئة أعلى من غيرها» لا «ثقة ٩٩.٩٩%». لا تبنِ عليها قرارات ذات عتبات مخاطر (طب، ائتمان)؛ عايرها بالطرق التي يفصّلها الفصل ٧٧.

> **احتمال ٠.٩٩٩٩ ليس ثقة**
>
> بيز الساذج رتّيب الترتيب ممتاز وقيّم الاحتمال سيء. لو بنيت نظام قرار طبي على أرقام `predict_proba` بلا معايرة فستعطي مريضاً «يقيناً» زائفاً. استخدمه للمقارنة بين الفئات لا لقياس الثقة.

وابقَ منصفاً: هذه الحدود هي سعر تذكرة النموذج الأسرع في المكتبة، والمدخل الوحيد الذي يبتلع ملايين الخصائص المتفرقة (نصوص) بسلاسة تامة.

تمارين الفصل السادس والخمسين

- **ملاحظة**

التمرين ٧.١

على بيانات `load_wine`، قارن بالتحقق المتقاطع الخماسي بين `GaussianNB` و`KNeighborsClassifier(n_neighbors=5)` الملفوف بـ `StandardScaler`. أيهما فاز، وماذا يخبرك ذلك عن سعر السذاجة مقابل كلفة المسافات؟

التمرين ٧.٢

على مرسلات القسم ٧.٤، درّب `BernoulliNB` بقيم `binarize` من ٠ إلى ٣ واطبع الدقة، ثم أضف رسالة غامضة مثل «اجتماع لعرض المشروع الجديد» واطبع تنبؤها واحتمالاتها مع كل عتبة. أي عتبة كشفت الغموض؟

التمرين ٧.٣

ابنِ خط إنتاج خارج الذاكرة: أنشئ بيانات `make_classification(n_samples=30000, n_features=12, random_state=42)` ودرّب `GaussianNB` عبر ست دفعات من ٥٠٠٠ عينة بـ `partial_fit`، ثم قارن دقة النموذج الناتج ودقة التدريب الكامل ومتوسطاته بـ `np.allclose`.

- **ملاحظة**

```python
# حل التمرين ٧.١: بيز الساذج يسبق الجيران المطبّعين
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_wine(return_X_y=True)
score_nb = cross_val_score(GaussianNB(), X, y, cv=5).mean()
score_knn = cross_val_score(
    make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)), X, y,
cv=5
).mean()
print(f"GaussianNB (بلا أي ضبط أو تطبيع): {score_nb:.4f}")
print(f"k-NN (بعد StandardScaler):        {score_knn:.4f}")
```

فاز GaussianNB بلا سطر ضبط واحد، لأن خصائص Wine شبه غاوسية داخل كل فئة ولأن الغاوسية تتعامل مع اختلاف المقاييس تلقائياً. حين تلبس البيانات لباس التوزيع الغاوسي، يدفع بيز الساذج كلفة سذاجته ويجنّي مكافأة البساطة في الجولات الأولى دائماً.

> **ملاحظة**

```python
# حل التمرين ٧.٣: 30 ألف عينة عبر ست دفعات
import numpy as np
from sklearn.datasets import make_classification
from sklearn.naive_bayes import GaussianNB

X, y = make_classification(n_samples=30000, n_features=12, random_state=42)
classes = np.unique(y)

stream = GaussianNB()
for start in range(0, 30000, 5000):
    stream.partial_fit(X[start:start + 5000], y[start:start + 5000],
classes=classes)

full = GaussianNB().fit(X, y)
print(f"دقة الدفعات: {stream.score(X, y):.4f} | دقة الكامل: {full.score(X, y):.4f}")
print(f"تطابق المتوسطات: {np.allclose(stream.theta_, full.theta_)}")
print(f"تطابق التباينات: {np.allclose(stream.var_, full.var_)}")
```

النتيجة تكرر ما رأيناه في القسم ٧.٦ على نطاق أكبر: الدقتان متطابقتان والمتوسطات والتباينات متساوية حتى آخر خانة عائمة. إحصاءات بيز الساذج تراكمية بحكم تركيبتها (مجاميع ومربعات مجاميع)، فـ partial_fit ليست تقريباً حريصاً

بالذاكرة بل خوارزمية مكافئة تماماً مقسّمة على الدفعات.

> **ملاحظة**

٧.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إطعام عدّادات بلا تمهيد (alpha = ٠)	كلمة غائبة عن فئة تُصفّر اللاحق كله بصمت	أبقِ alpha >= ٠.٠١ وجرّب قيماً في البحث الشبكي
استخدام GaussianNB على عمود ثابت القيمة	تباين صفر يجعل الاحتمال الغاوسي ينفجر	أزل الأعمدة الثابتة بـ VarianceThreshold أو اثق من var_smoothing
تفسير predict_proba كثقة حقيقية	أرقام قرب ١.٠ زائفة بسبب فرضية الاستقلال	استخدم الترتيب أو عاير الاحتمالات كما في الفصل ٧٧
تحميل كل البيانات في الذاكرة لتدريب بيز	استنزاف RAM لا داعي له على تدفقات ضخمة	درّب بـ partial_fit على دفعات مع تمرير classes مرة أولى

٧.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الاحتمال اللاحق	احتمال الفئة بعد رؤية الأدلة، ناتج مبرهنة بيز	Posterior
الإمكان	احتمال رؤية الأدلة نفترضاً أن الفئة معلومة	Likelihood
السابق	احتمال الفئة قبل رؤية أي دليل، من تكرارها	Prior
فرضية الاستقلال الشرطي	افتراض أن الخصائص مستقلة داخل كل فئة	Conditional independence
تمهيد لابلاس	إضافة alpha وهمية لكل عدّاد لإزالة الأصفار	Laplace smoothing

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٧ — تحليل التمييز

LDA و QDA وانكماش التغاير والتوزيع الغاوسي

الوحدة **scikit-learn ١.٩** الحجم **٩٢٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح الفرق بين النمذجة التوليدية (generative) والتمييزية (discriminative)
- أن تتحقق من درجة القرار لـ LDA يدوياً ضد decision_function رقمياً
- أن تختار بين LDA و QDA بحسب شكل التغاير وحجم العينات، وتستخدم LDA كمخفِّض أبعاد خاضع للرقابة
- أن تعالج انهيار تقدير التغاير في الأبعاد العالية بـ 'shrinkage='auto أو LedoitWolf
- أن تضبط priors المخصصة للتعامل العادل مع الفئات النادرة

٧.١ التوليد مقابل التمييز: فلسفتان لقرار واحد

كيف يتعلم المصنّف أن يقول «هذه عينة من الفئة ١»؟ طريق الانحدار اللوجيستي (الفصل ٣٥) هو **التمييزي (discriminative)**: نمذجة الحد الفاصل مباشرة، أي تعلّم `P(Y=k | X=x)`. والطريق المعاكس هو **النمذجة التوليدية (generative modeling)**: نتعلّم لكل فئة متوسطها `μ_k` وتغايرها `Σ_k`، ثم عند عينة جديدة نسأل: أي توزيع «أنجبها» بأرجحية أكبر؟

المفهوم المحوري

التوليدي يتعلم `P(X | Y)` و `P(Y)` ثم يستخرج `P(Y | X)` بقاعدة بيز، والتمييزي يتعلم `P(Y | X)` مباشرة. حين تكون الفرضية التوزيعية صحيحة — أو العينات قليلة — يفوق التوليدي التمييزي.

افتراض تحليل التمييز صريح: بيانات كل فئة تتبع توزيع غاوسي متعدد الأبعاد:

$$p_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2}\,|\Sigma_k|^{1/2}} \exp\!\left(-\tfrac{1}{2}\,(\mathbf{x}-\mu_k)^{\top}\,\Sigma_k^{-1}\,(\mathbf{x}-\mu_k)\right)$$

حيث `μ_k` متوسط الفئة k و `Σ_k` تغايرها و `p` عدد الميزات. الاحتمال الأعلى لفئة تجعل الأس السالب أصغر، أي مركزها أقرب للعينة «بميزان» تغايرها. ثم يمزج بيز هذا الاحتمال بالأولوية المسبقة `π_k` :

$$P(Y=k|\mathbf{X}=\mathbf{x}) = \frac{\pi_k\, p_k(\mathbf{x})}{\sum_{j=1}^{K} \pi_j\, p_j(\mathbf{x})}$$

في scikit-learn ينفذ هذه العائلة صنفان في `sklearn.discriminant_analysis`: `LinearDiscriminantAnalysis` بافتراض **تغاير مشترك** بين الفئات، و `QuadraticDiscriminantAnalysis` بتغاير **مستقل لكل فئة**. وبيز الساذج (الفصل ٥٦) هو الحالة القصوى من المنطق نفسه: تغاير قطري بلا ترابطات. وإليك أول مصادمة عملية (خوارزمية فيشر هذه وُلدت في ورقة ١٩٣٦ التي قُدمت فيها بيانات Iris نفسها):

اللوجيستي التمييزي مقابل التوليديين python

```
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# فئة تحتوي عنقودين داخليين: بنية توليدية غير متجانسة
X, y = make_classification(
    n_samples=1200, n_features=6, n_informative=6, n_redundant=0,
    n_clusters_per_class=2, class_sep=1.5, random_state=42,
)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

models = {
    "LogisticRegression (تمييزي)": LogisticRegression(max_iter=1000),
    "LDA (توليدي، تغاير مشترك)":   LinearDiscriminantAnalysis(),
    "QDA (توليدي، تغاير لكل فئة)": QuadraticDiscriminantAnalysis(),
}
for name, model in models.items():
    model.fit(X_tr, y_tr)
    print(f"{name:32s} دقة الاختبار: {model.score(X_te, y_te):.4f}")

lda = models["LDA (توليدي، تغاير مشترك)"]
print(f"ما تعلمه LDA توليدياً: priors_ = {lda.priors_}")
```

اللوجيستي و LDA تعادلا عند ٠.٨٨٣٣ لأن كليهما عاجز عن فصل عنقودين متراكبين بخط واحد، بينما قطف QDA ثمرة فهمه لشكل التوزيع الداخلي بـ ٠.٩٥٨٣. قبل شرح السبب، لنتعامل مع الحالة الخطية بدقة.

٧.٢ درجة LDA: من الغاوسية إلى خط مستقيم

ماذا يحدث إذا فرضنا تغايراً واحداً مشتركاً `Σ`؟ الحد التربيعي `-½ xᵀ Σ⁻¹ x` يصبح مشتركاً بين الفئات فيُلغى من المقارنة، وينجو فقط ما تبقى — دالة **خطية تماماً في x** تُعرف بدرجة القرار:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \mu_k - \tfrac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \ln \pi_k$$

الحد الأول إسقاط العينة على مركز الفئة بميزان التغاير المعكوس، والثاني عقوبة للفئات البعيدة عن الأصل، والثالث `ln π_k` أولوية للفئة الغزيرة. حد الفصل بين أي فئتين — الفرق `δ_1(x) - δ_0(x) = 0` — مستوٍ خطي. وهذه ليست تشبيهاً لما تفعله المكتبة؛ لنثبت ذلك رقمياً:

التحقق اليدوي من درجة القرار — python

```python
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split

# فئتان غاوسيتان بتغاير مشترك جيد الإشراط
rng = np.random.default_rng(42)
cov = np.array([[2.0, 0.4], [0.4, 1.0]])
X0 = rng.multivariate_normal([-1.5, -1.0], cov, size=400)
X1 = rng.multivariate_normal([1.5, 1.0], cov, size=400)
X = np.vstack([X0, X1])
y = np.r_[np.zeros(400), np.ones(400)]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

model = LinearDiscriminantAnalysis(solver="lsqr").fit(X_tr, y_tr)
Sigma_inv = np.linalg.inv(model.covariance_)
mu, ln_pi = model.means_, np.log(model.priors_)
s0 = X_te @ (Sigma_inv @ mu[0]) - 0.5 * (mu[0] @ Sigma_inv @ mu[0]) + ln_pi[0]
s1 = X_te @ (Sigma_inv @ mu[1]) - 0.5 * (mu[1] @ Sigma_inv @ mu[1]) + ln_pi[1]

print(f"أقصى فرق بين درجتنا اليدوية و decision_function: {np.abs((s1 - s0) -
model.decision_function(X_te)).max():.2e}")
print(f"توافق التصنيف اليدوي مع predict: {((s1 > s0) == (model.predict(X_te) ==
1)).mean():.4f}")
```

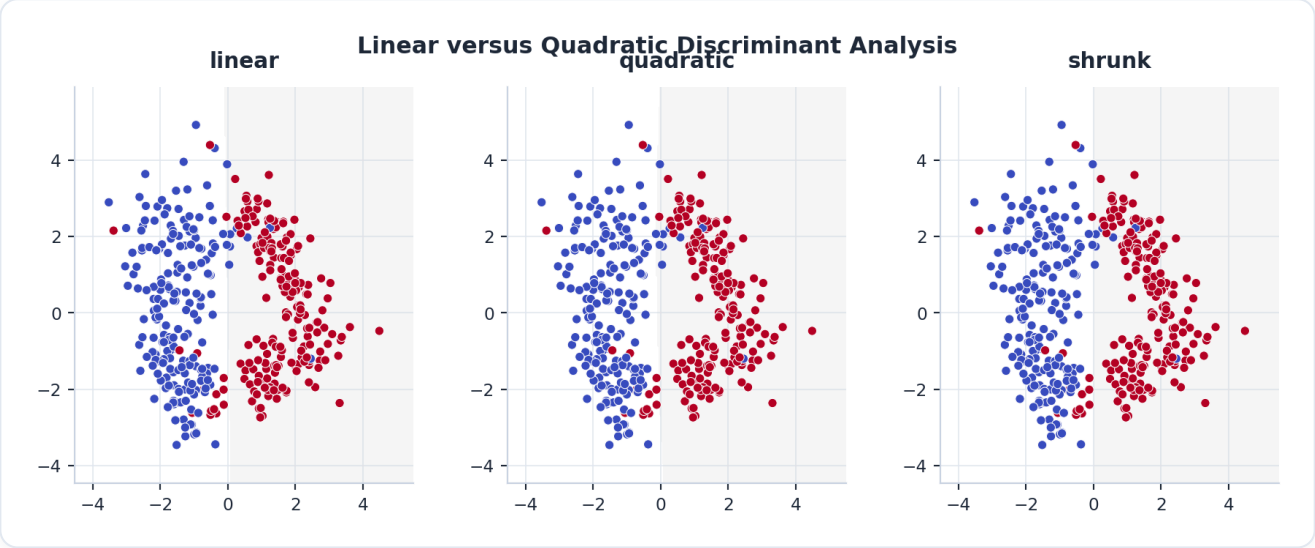
الفرق في حدود `3.6e-15` — دقة الفاصلة العائمة نفسها. معاملات `coef_` و `intercept_` تُقرأ مباشرة من إحصاءات الفئات، فتفسير LDA فيزيائي لا صندوق أسود.

QDA ٧.٣: حين يتفوق التربيع

مع تغاير خاص لكل فئة `Σ_k` يبقى الحد التربيعي في الصيغة الكاملة:

$$\delta_k(\mathbf{x}) = -\tfrac{1}{2}\ln|\Sigma_k| - \tfrac{1}{2}(\mathbf{x}-\mu_k)^\top \Sigma_k^{-1}(\mathbf{x}-\mu_k) + \ln \pi_k$$

حدّان جديدان: عقوبة انتشار الفئة `-½ ln|Σ_k|`، ومق مسافة بميزان تغاير الفئة نفسها. النتيجة حدود قرار **تربيعية** (إهليجات ومخارج ودوائر) تلتف حول كل فئة بشكلها الحقيقي.



الشكل ١٢٣حدود القرار الخطية لـ LDA مقابل الحدود التربيعية لـ QDA على فئات بأشكال تغاير مختلفة

في الشكل: حيثما كانت الفئات إهليجات متماثلة كفت LDA خطوطاً كافية، وحيثما مال إهليج فئة بزاوية مختلفة تظهر حدود QDA المنحنية. لكن الفوز ليس مجانياً — جرب ثلاثة سيناريوهات متقابلة:

في ثلاثة سيناريوهات QDA مقابل LDA python

```python
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.model_selection import train_test_split

def battle(X, y, label):
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)
    a = LinearDiscriminantAnalysis().fit(X_tr, y_tr).score(X_te, y_te)
    b = QuadraticDiscriminantAnalysis().fit(X_tr, y_tr).score(X_te, y_te)
    print(f"{label:42s} LDA = {a:.4f} | QDA = {b:.4f}")

# سيناريو 1: تغاير متماثل وعينات وفيرة — LDA هو النموذج الصحيح
rng = np.random.default_rng(42)
cov_same = np.diag([2.0] * 8)
mu = 1.4 / np.sqrt(8)
X0 = rng.multivariate_normal([-mu] * 8, cov_same, size=300)
X1 = rng.multivariate_normal([mu] * 8, cov_same, size=300)
battle(np.vstack([X0, X1]), np.r_[np.zeros(300), np.ones(300)], "300 + تغاير متماثل
عينة/فئة")

# سيناريو 2: التوزيع نفسه لكن 15 عينة فقط لكل فئة
rng = np.random.default_rng(7)
X0 = rng.multivariate_normal([-mu] * 8, cov_same, size=15)
X1 = rng.multivariate_normal([mu] * 8, cov_same, size=15)
battle(np.vstack([X0, X1]), np.r_[np.zeros(15), np.ones(15)], "/عينة 15 + تغاير متماثل
فئة")

# سيناريو 3: فئات بأشكال إهليلجية مختلفة تماماً
rng = np.random.default_rng(42)
X0 = rng.multivariate_normal([-2, -2], np.diag([9.0, 1.0]), size=600)
X1 = rng.multivariate_normal([2, 2], np.diag([1.0, 9.0]), size=600)
battle(np.vstack([X0, X1]), np.r_[np.zeros(600), np.ones(600)], "600 + إهليجات مختلفة
عينة/فئة")
```

في الأول تعادل المصنفان عند ٠.٨١٦٧ — حين يكون الافتراض المشترك صحيحاً لا يكسب LDA حصيناً بل يقتصر على تقدير بارامترات أقل؛ في الثاني انهارت QDA إلى ٠.٦٤٦٨ مقابل ٠.٧٧٩٨ لأنها تقدّر مصفوفة `8×8` من ١٥ عينة لكل فئة؛ وفي الثالث قلبت الطاولة (٠.٩٠٢٨ مقابل ٠.٨٥٨٣) لأن الافتراض المشترك خطأ أصلاً. القاعدة: **QDA يفوز مع أشكال إهليلجية مختلفة وعينات كافية، و LDA يفوز مع العينات القليلة أو التغاير المشترك فعلاً.**

LDA كمخفِّض أبعاد خاضع للرقابة ٧.٤

LDA لا تصنّف فقط؛ إنها **تحويلة** تسقط البيانات على محاور تفصل الفئات بأفضل صورة: متجهات ذاتية للمصفوفة `S_W⁻¹ S_B` حيث `S_W` التشتت داخل الفئات و `S_B` التشتت بينها:

$$S_W^{-1} S_B \, \mathbf{w} = \lambda \, \mathbf{w}$$

قيد ذكي: `S_B` مبنية من مراكز `K` فئات فرتبتها لا يتجاوز `K - 1`، فلا معنى لطلب مكونات أكثر. هذا عكس PCA (الفصل ٦٨) الذي يبحث عن أكبر تباين **دون النظر إلى التسميات**؛ LDA تنظر إليها:

إسقاط Iris على مكوني LDA — python

```python
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)
lda = LinearDiscriminantAnalysis(n_components=2).fit(X, y)
Z = lda.transform(X)

print(f"الأبعاد الأصلية: {X.shape[1]} → بعد LDA: {Z.shape[1]}")
print(f"نسبة التباين المفسَّر لكل مكون: {lda.explained_variance_ratio_.round(4)}")
print(f"دقة اللوجيستي على المكونين: {LogisticRegression(max_iter=1000).fit(Z,
y).score(Z, y):.4f}")
print(f"دقة اللوجيستي على الأبعاد الأربع: {LogisticRegression(max_iter=1000).fit(X,
y).score(X, y):.4f}")
```

مكوّن واحد يحمل ٩٩.١٢% من التباين «المفيد للفصل»، وبُعدان يكفيان لدقة ٠.٩٨ بلا خسارة تقريباً. وفي PCA يحمل أول مكونين تبايناً عالياً لكن فصلاً أسوأ — «أكبر تباين» لا يعني «أفضل فصل». وأما الحلول الثلاثة فلكلٍّ بيته:

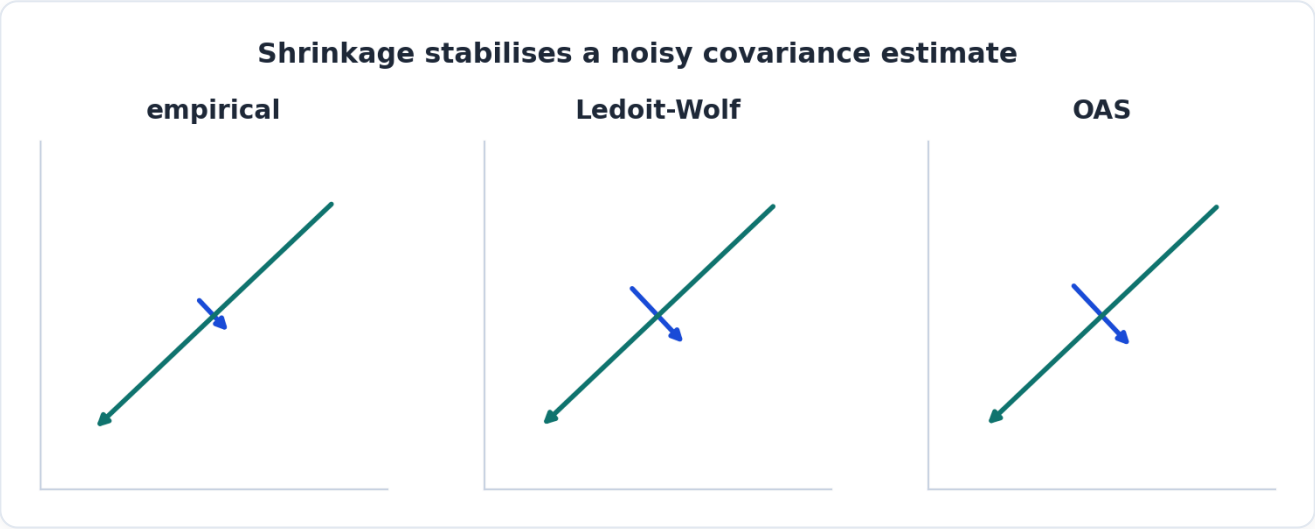
المحلول	كيف يعمل	متى تختاره
`solver="svd"` **(افتراضي)**	تفكيك قيمة مفردة، لا يبني التغاير صراحةً	الافتراضي؛ يتحمل التداخل الخطي والأبعاد الكبيرة
`solver="lsqr"`	حل مباشر بأقل المربعات	عند الحاجة إلى `shrinkage`، بلا `transform`
`solver="eigen"`	تفكيك ذاتي للتشتت	عند طلب `shrinkage` مع `transform` معاً

lsqr لا يعرف التحويل

`transform()` غير منفذة للمحلول `lsqr` ويجيبك `NotImplementedError` — استخدم `svd` أو `eigen`. أيضاً: الانكماش حكر على `lsqr` و`eigen` مع الافتراضي `svd` يرفع `NotImplementedError` و`shrinkage`.

٧.٥ انكماش التغاير: العلاج حين يقترب p من n

لماذا يفشل هذا كله في الأبعاد العالية؟ تغاير ٨٠ ميزة يحوي `80 × 81 / 2` قيمة مستقلة تقريباً؛ ومع ٧٠ عينة تدريب فقط فأنت تطلب آلاف الأرقام من بياناتك — لا يمكن. النتيجة مصفوفة **شبه مفردة** (singular) ومعكوسها ينفجر، والتقدير ضجيج يعبر عن عينتك لا عن السكان.



الشكل ١٢٤ تقدير التغاير: العينة الخام المفردة مقابل المصفوفة المنكمشة نحو قطري مستقر

في الشكل يُقرأ العلاج: نسحب المصفوفة الخام نحو هدف بسيط مستقر — القطري — بمعامل انكماش `λ` بين ٠ و ١:

$$\hat{\Sigma}_{\text{shrink}} = (1-\lambda)\,\hat{S} + \lambda\,\mu\,\mathbf{I}$$

حيث `Ŝ` تغاير العينة الخام و `μ` متوسط تباين الميزات. الانكماش يضحّي بانحياز صغير مقابل حذف هائل من التباين. جرب الكارثة وعلاجها:

انهيار تقدير التغاير وعلاجه بالانكماش python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.covariance import LedoitWolf
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.model_selection import train_test_split

# ميزة مقابل 70 عينة تدريب فقط 80
X, y = make_classification(n_samples=100, n_features=80, n_informative=30,
                           n_redundant=10, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
print(f"أبعاد التدريب: {X_tr.shape} — عينة 70 من 80×80 نقدّر تغايراً!\n")

configs = [
    ("svd بلا انكماش",         LinearDiscriminantAnalysis(solver="svd")),
    ("lsqr بلا انكماش",        LinearDiscriminantAnalysis(solver="lsqr")),
    ("lsqr shrinkage='auto'", LinearDiscriminantAnalysis(solver="lsqr",
shrinkage="auto")),
    ("lsqr + LedoitWolf",      LinearDiscriminantAnalysis(solver="lsqr",
covariance_estimator=LedoitWolf())),
]
for name, model in configs:
    model.fit(X_tr, y_tr)
    print(f"{name:26s} دقة الاختبار: {model.score(X_te, y_te):.4f}")

try:
    QuadraticDiscriminantAnalysis().fit(X_tr, y_tr)
except np.linalg.LinAlgError as err:
    print(f"\nQDA بلا حماية انهار: {str(err)[:70]}...")

q = QuadraticDiscriminantAnalysis(solver="eigen", shrinkage="auto").fit(X_tr,
y_tr)
print(f"QDA مع انكماش تلقائي: {q.score(X_te, y_te):.4f}")
```

الدقة تسقط قرب التخمين العشوائي (٠.٤٠٠٠ و ٠.٤٦٦٧) ثم تقفز بانكماش واحد إلى ٠.٨٣٣٣ و ٠.٨٦٦٧. وقيمة `shrinkage='auto'` ليست رقماً مفترضاً بل تقدير **ليدو-ولف** (Ledoit-Wolf lemma) يحسب المعامل الأمثل من البيانات نفسها. ولاحظ أن QDA — الذي يقدّر مصفوفة لكل فئة — انهار بخطأ `LinAlgError`، وأن نسخته بمحلول `eigen` وانكماش تلقائي نجت: وصل هذا البارامتر إلى QDA في الإصدارات الحديثة مع `shrinkage` و `covariance_estimator`.

> ⛔ **لا تقارن مصنّفين فوق مصفوفة تغاير مفردة**
>
> حين يقترب `n_features` من `n_samples`، أي مقارنة بلا انكماش فاسدة: الفائز قد يكون أقلّ المصنفين إصابة بضجيج التقدير فقط. فعّل `shrinkage='auto'` أو مرّر `covariance_estimator=LedoitWolf()` من `sklearn.covariance`. وسنقابل مقدّرات الانكماش ككواشف شذوذ في الفصل ٧٠ مع `EllipticEnvelope`.

> 💡 **قاعدة عملية**
>
> النسبة `n_samples / n_features`: فوق ١٠ تقريباً فالتقدير الخام غالباً سليم؛ تحت ٣ فعّل الانكماش بلا تردد.

---

priors V.٦ المخصصة: عدالة الفئات النادرة

في البيانات غير المتوازنة يصبح أثر `π_k` قاطعاً: LDA المتعلمة من بيانات فيها ٩٠% من فئة واحدة متشائمة تجاه الفئة النادرة لأن `ln π_k` يعاقبها مسبقاً. والعلاج أن تطعمها بأولويات تختارها أنت:

أثر priors على الفئة النادرة
python

```python
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import train_test_split

# فئة نادرة بحدة: 10% فقط من العينات
X, y = make_classification(n_samples=2000, n_features=10, n_informative=6,
                           weights=[0.9, 0.1], flip_y=0.02, class_sep=1.2,
random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

models = [
    ("priors المتعلمة", LinearDiscriminantAnalysis()),
    ("priors=[0.5, 0.5]", LinearDiscriminantAnalysis(priors=[0.5, 0.5])),
]
for name, model in models:
    model.fit(X_tr, y_tr)
    pred = model.predict(X_te)
    recall_rare = (pred[y_te == 1] == 1).mean()
    print(f"{name:20s} الدقة: {model.score(X_te, y_te):.4f} | "
          f"الدقة المتوازنة: {balanced_accuracy_score(y_te, pred):.4f} | "
          f"استرجاع الفئة النادرة: {recall_rare:.4f}")
```

مع الأولويات المتعلمة الدقة ٠.٩٤٥٠ لكن استرجاع الفئة النادرة ٠.٥٨٤٦؛ ومع المتساوية تنخفض الدقة إلى ٠.٨٦٦٧ ويرتقي الاسترجاع إلى ٠.٧٣٨٥. أرقام «أعلى» ليست دائماً «أعدل»، وإحالة أعمق في الفصل ٩.

في العيادة والبنك

في فحص طبي تُصاب به ٢% من العينات، المصنّف المتعلم priors من البيانات الخام سيقول «سليم» دائماً ويحقق ٩٨% دقة صفرية القيمة. ضبط `priors` هو الفرق بين نموذج يبدو ممتازاً في التقرير ونموذج ينقذ مريضاً فعلاً.

تمارين الفصل السابع والخمسين

- **ملاحظة**

التمرين ٧.١

على بيانات النبيذ `load_wine` (ثلاث فئات، ١٣ ميزة)، قارن عبر التحقق المتقاطع الخماسي بين `LinearDiscriminantAnalysis` و `QuadraticDiscriminantAnalysis(reg_param=0.1)`. أيهما يفوز، ولماذا؟

التمرين ٧.٢

على تهيئة §٧.٥ نفسها (١٠٠ عينة، ٨٠ ميزة)، جرب قيم `shrinkage` الثابتة `0.1` و `0.3` و `0.5` و `0.9` مع `None` و `'auto'`. أين تقع القمة؟

التمرين ٧.٣

بدون كود: لماذا لا يفيد طلب `n_components` أكبر من `K - 1` في LDA، ولمَ لا يعاني PCA من هذا القيد؟ (لمّح: ما رتبة مصفوفة `S_B` المبنية من `K` مركزاً فقط؟)

- **ملاحظة**

حلّ التمرين ٧.١: LDA مقابل QDA على النبيذ python

```python
from sklearn.datasets import load_wine
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
models = [
    ("LDA", LinearDiscriminantAnalysis()),
    ("QDA(reg_param=0.1)", QuadraticDiscriminantAnalysis(reg_param=0.1)),
]
for name, model in models:
    scores = cross_val_score(make_pipeline(StandardScaler(), model), X, y, cv=5)
    print(f"{name:22s} متوسط الدقة الخماسية: {scores.mean():.4f}")
```

تفوق QDA (٠.٩٧٧٣ مقابل ٠.٩٦٦٢): فئات النبيذ تختلف في انتشارها وترابطات ميزاتها، وحجم العينات (١٧٨) كافٍ لتقدير ثلاث مصفوفات ١٣×١٣ بثبات معقول.

> **ملاحظة**

حلّ التمرين ٧.٢: مسح معامل الانكماش python

```python
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=100, n_features=80, n_informative=30,
                           n_redundant=10, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
for sh in [None, 0.1, 0.3, 0.5, 0.9, "auto"]:
    model = LinearDiscriminantAnalysis(solver="lsqr", shrinkage=sh).fit(X_tr,
y_tr)
    print(f"shrinkage={str(sh):6s} دقة الاختبار: {model.score(X_te, y_te):.4f}")
```

منحنى هضبي: بلا انكماش ٠.٤٦٦٧، ثم قمّة ٠.٨٦٦٧ حول `0.3` – `0.5`، ثم انحسار عند `0.9` لانكماش مبالغ. وتقدير `'auto'` عاد بـ ٠.٨٣٣٣ — قريب من القمة بلا أي ضبط يدوي، وهذا بالضبط وظيفته.

> **ملاحظة**

٧.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمج
تمرير `shrinkage='auto'` **مع المحلول الافتراضي** `svd`	رفع `NotImplementedError` لأن الانكماش غير مدعوم مع svd	استخدم `'` `='eigen'` الانكماش
تدريب QDA بـ `reg_param=0` **على بيانات شبه مفردة**	رفع `LinAlgError: The covariance matrix of class 0 is not full rank`	ضبط `aram` `='eigen'` `ge='auto'`
طلب `n_components` **أكبر من** `K - 1` **في LDA**	قصّ صامت للقيمة إلى السقف الممكن، فتظن أنك حصلت على مكونات أكثر	اعلم أن الس `S_B`) واقر `e_ratio_`
تقييم LDA و QDA في `p ≈ n` **بلا انكماش**	نتائج شبه عشوائية تتغير جذرياً بتغيير `random_state`	تفعيل `to'` كلا المصنف

٧.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
النمذجة التوليدية	تعلم توزيع البيانات لكل فئة ثم الاستدلال بقاعدة بيز	Generative modeling
درجة القرار	الدالة التي تحسب أرجحية كل فئة لكل عينة وتُصنّف بالأعلى	Discriminant score
التغاير المشترك	افتراض مصفوفة تغاير واحدة للفئات كلها ينتج حدوداً خطية	Pooled covariance
انكماش التغاير	سحب تقدير التغاير الخام نحو هدف قطري مستقر بمعامل λ	Covariance shrinkage
أولوية مسبقة	احتمال ظهور الفئة قبل رؤية أي ميزات، قابل للضبط بـ priors	Prior probability

بهذا الفصل اكتمل السلاح التوليدي خلف بيز الساذج (الفصل ٥٦): من الاستقلال الساذج هناك إلى الغاوسية الكاملة هنا. وفي الفصل الأخير من هذا الجزء — الفصل ٥٨ — نجمع أدوات النواة والشبكات العصبية، قبل أن نفتح الجزء الثامن ونترك التسميات جانباً كلياً.

الجزء ٧ · خوارزميات خاضعة للرقابة أخرى

الفصل ٥٨ — نماذج النواة والشبكات العصبية

KernelRidge وتقريب النواة و BernoulliRBM و MLP

الوحدة **sklearn.svm · sklearn.neighbors** الحجم **١٠٦٨** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح حل KernelRidge المغلق وتقارن تكلفته مع SVR بالأرقام
- أن تحوّل SVM غير الخطية إلى مصنف خطي سريع بـ Nystroem أو RBFSampler على ١٠٠ ألف عينة
- أن تضبط MLPClassifier: hidden_layer_sizes و activation و solver و alpha و early_stopping
- أن تدرب الشبكات تزايدياً بـ partial_fit وتستشعر خطر غياب التطبيع
- أن تستخدم BernoulliRBM مستخرج خصائص غير مشرف للبيانات الثنائية

٥٨.١ KernelRidge: انحدار الريج في فضاء النواة

في الفصل ٢٩ عرفت انحدار الريج بعقوبة L2. والسؤال الطبيعي: ماذا لو أردنا نفس العقوبة في فضاء الميزات اللانهائي لحيلة النواة (الفصل ٥١)؟ الجواب `sklearn.kernel_ridge.KernelRidge`: بدل معامل لكل ميزة، معامل لكل عينة تدريب، والحل مغلق تماماً:

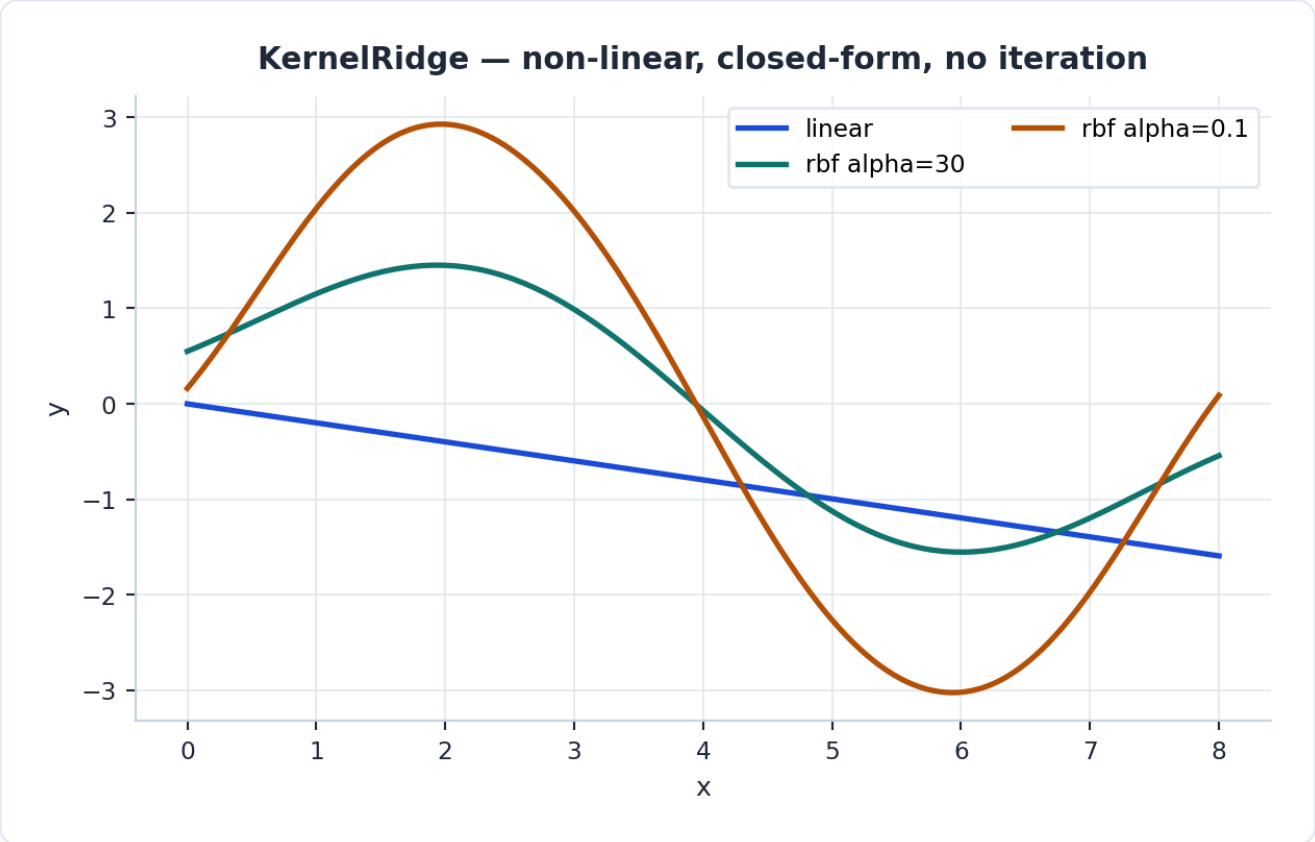
$$\hat{\alpha} = (K + \alpha \mathbf{I})^{-1} \mathbf{y}, \qquad f(\mathbf{x}) = \sum_{i=1}^{n} \hat{\alpha}_i \, k(\mathbf{x}, \mathbf{x}_i)$$

حيث `K` مصفوفة النواة `n×n` التي `K_ij = k(x_i, x_j)`، و `α` معامل التنظيم، و `I` مصفوفة الوحدة — وغالباً ما تكون `k` نواة RBF المعهودة من الفصل ٥٢. لا تكرارات ولا معدل تعلّم ولا مسألة تقارب — معادلة واحدة تحل المسألة. جربها على موجة جيبية ملوّثة يعجز عنها الريج الخطي:

KernelRidge على موجة جيبية غير خطية python

```python
import numpy as np
from sklearn.kernel_ridge import KernelRidge
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(42)
X = np.sort(rng.uniform(0, 10, size=(300, 1)), axis=0)
y = np.sin(X[:, 0]) + rng.normal(0, 0.15, size=300)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

linear = Ridge(alpha=1.0).fit(X_tr, y_tr)
kernel = KernelRidge(kernel="rbf", alpha=0.1, gamma=0.5).fit(X_tr, y_tr)
print(f"Ridge الخطي       R² = {linear.score(X_te, y_te):.4f}")
print(f"KernelRidge(rbf)  R² = {kernel.score(X_te, y_te):.4f}")
print(f"عدد عينات التدريب (لا متجهات داعمة) = {kernel.dual_coef_.size} طول dual_coef_")
```

R² قفز من -٠.٠٠٩٧ — أسوأ من التنبؤ بالمتوسط — إلى ٠.٩٣٢٤ بنواة واحدة. لكن انظر للسطر الأخير: كل عينة تدريب من الـ ٢١٠ حملت معاملاً خاصاً بها — وهذه ليست تفصيلاً بل الشخصية الكاملة للنموذج كما سترى الآن.



الشكل ١٢٥ملاءمة KernelRidge بنواة RBF لموجة غير خطية مقابل انحدار خطي عاجز

في الشكل يُقرأ الفرق: الخط المستقيم يلتقط الاتجاه العام فقط، بينما يلتصق منحنى KernelRidge بالإشارة الموجية، وحسّن `gamma` يحدد مدى الالتصاق.

> **المفهوم المحوري**
>
> KernelRidge هو SVR بلا أنبوب epsilon: نفس النواة والإسقاط، لكن التنظيم بـ alpha على المعاملات بدل هامش تسامح. الحل مغلق وأسرع تدريباً، والتنبؤ يظل O(n) لأنه لا يترك وراءه متجهات داعمة قليلة بل كل العينات.

٧.٢ الثمن الخفي: KernelRidge مقابل SVR

إن كان KernelRidge أسرع تدريباً، فلماذا لا نستغني عن SVR (الفصل ٥٣)؟ لأن الثمن يظهر عند **التنبؤ**. قياس على ٥٠٠٠ عينة من California Housing (٣٥٠٠ تدريب و١٥٠٠ اختبار):

الزمن والذاكرة: SVR مقابل KernelRidge python

```python
import numpy as np, time
from sklearn.datasets import fetch_california_housing
from sklearn.kernel_ridge import KernelRidge
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

data = fetch_california_housing()
X, y = data.data[:5000], data.target[:5000]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)
sc = StandardScaler().fit(X_tr)
X_tr_s, X_te_s = sc.transform(X_tr), sc.transform(X_te)

t0 = time.perf_counter()
kr = KernelRidge(kernel="rbf", alpha=0.1, gamma=0.05).fit(X_tr_s, y_tr)
t_kr = time.perf_counter() - t0
t0 = time.perf_counter()
sv = SVR(kernel="rbf", C=10, gamma=0.05).fit(X_tr_s, y_tr)
t_sv = time.perf_counter() - t0

x1 = X_te_s[:1]
t0 = time.perf_counter()
for _ in range(100):
    kr.predict(x1)
tp_kr = (time.perf_counter() - t0) / 100 * 1000
t0 = time.perf_counter()
for _ in range(100):
    sv.predict(x1)
tp_sv = (time.perf_counter() - t0) / 100 * 1000

print(f"{'النموذج':14s} {'R²':>7s} {'تدريب':>7s} {'تنبؤ لعينة':>11s}")
print(f"{'KernelRidge':14s} {kr.score(X_te_s, y_te):7.4f} {t_kr:5.2f}s
{tp_kr:9.2f}ms")
print(f"{'SVR':14s} {sv.score(X_te_s, y_te):7.4f} {t_sv:5.2f}s {tp_sv:9.2f}ms")
print(f"معاملات KR: {kr.dual_coef_.size} (كل العينات) | متجهات SVR الداعمة:
{sv.support_.size}")
```

المعيار	KernelRidge	SVR
التدريب	حل مغلق لمصفوفة `n×n` : ذروة `O(n³)` وذاكرة `O(n²)` — ١.٧٠s هنا	QP تكراري — ٢.٢٣s هنا، أبطأ على هذا الحجم
التنبؤ	`O(n)` دائماً — كل العينات معاملات: ٤.٠٧ms للعينة	`O(n_SV)` فقط: ٢.٢٢ms لأنه ٢٦٤٠ متجه داعم من ٣٥٠٠
الضبط	`alpha` و `gamma`	`C` و `gamma` و `epsilon`
متى تختاره	تدريب كثير وتنبؤ قليل على n متوسطة	تنبؤ كثيف أو n كبيرة (بمساعدة §٧.٣)

الخلاصة: الفوز في التدريب لـ KernelRidge، والفوز في التنبؤ لـ SVR بفضل التناثر الذي يفرضه أنبوب epsilon — نحو ٧٦% من عينات التدريب صارت هنا متجهات داعمة.

٧.٣ تقريب النواة للمقياس الكبير: Nystroem و RBFSampler

كيف نأكل كعكة النواة على مئة ألف عينة؟ مصفوفة النواة وحدها تحتاج `100000² × 8` بايت — نحو ٨٠ جيجابايت. فكرة `sklearn.kernel_approximation` أذكى: **قمرب النواة** بفضاء ميزات صريح محدود الأبعاد يبقى ضربه القياسي قريباً من قيمتها، ثم حلّ المسألة بأي مصنف خطي سريع (الفصل ٣٦). صيغة نايستروم تقارب المصفوفة الكاملة من مربع فرعي صغير لنقاط معلم (landmarks):

$$K_{n\times n} \;\approx\; E\,W^{-1}\,E^{\top}, \qquad E = K(\mathbf{X}, \mathbf{X}_m),\;\; W = K(\mathbf{X}_m, \mathbf{X}_m)$$

حيث `X_m` عينة m من نقاط المعلم و `W` نواة ما بينها. أما `RBFSampler` فاستعارة نظرية أخرى (تحويلات Fourier العشوائية) بنتيجة مماثلة. القياس الحاسم على ١٠٠٬٠٠٠ عينة:

تقريب النواة على 100 ألف عينة python

```python
import numpy as np, time
from sklearn.datasets import make_classification
from sklearn.kernel_approximation import Nystroem, RBFSampler
from sklearn.linear_model import SGDClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_classification(n_samples=100_000, n_features=12, n_informative=8,
                           n_redundant=2, class_sep=0.8, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
sc = StandardScaler().fit(X_tr)
X_tr_s, X_te_s = sc.transform(X_tr), sc.transform(X_te)

def bench(name, model):
    t0 = time.perf_counter()
    model.fit(X_tr_s, y_tr)
    t = time.perf_counter() - t0
    print(f"{name:30s} الدقة: {model.score(X_te_s, y_te):.4f} | الزمن: {t:5.2f}s")

bench("SGD خطي مباشر", SGDClassifier(loss="hinge", max_iter=100, tol=1e-3,
random_state=42, n_jobs=-1))
bench("Nystroem(64) + SGD", make_pipeline(
    Nystroem(kernel="rbf", n_components=64, random_state=42),
    SGDClassifier(loss="hinge", max_iter=100, tol=1e-3, random_state=42,
n_jobs=-1)))
bench("Nystroem(200) + SGD", make_pipeline(
    Nystroem(kernel="rbf", n_components=200, random_state=42),
    SGDClassifier(loss="hinge", max_iter=100, tol=1e-3, random_state=42,
n_jobs=-1)))
bench("RBFSampler(100) + SGD", make_pipeline(
    RBFSampler(gamma=1.0/12, n_components=100, random_state=42),
    SGDClassifier(loss="hinge", max_iter=100, tol=1e-3, random_state=42,
n_jobs=-1)))

# السقف: SVC فقط 6000 على 100 ألف، لذا على بنواة كاملة – مستحيل
X_small, _, y_small, _ = train_test_split(X_tr_s, y_tr, train_size=6000,
random_state=42)
bench("SVC (فقط 6000 عينة) كامل", SVC(kernel="rbf", gamma=1.0/12).fit(X_small,
y_small))
```

القصة مرقومة: الخطي يقف عند ٠.٦٧٩٨، و Nystroem بـ ٦٤ مكوناً يرفعه إلى ٠.٨٦٦٣، وبـ ٢٠٠ مكوناً إلى ٠.٩٠٩٤ — على **مئة ألف عينة** في بضع ثوانٍ. وللمقارنة، SVC الكامل احتاج ٦٠٠٠ عينة فقط ليصل إلى ٠.٩٢٣٤ قرب سقف النواة الكاملة، وكان سيلزمه ٨٠ جيجابايت لو جرب ١٠٠ ألف.

gamma في RBFSampler ليس ذكياً

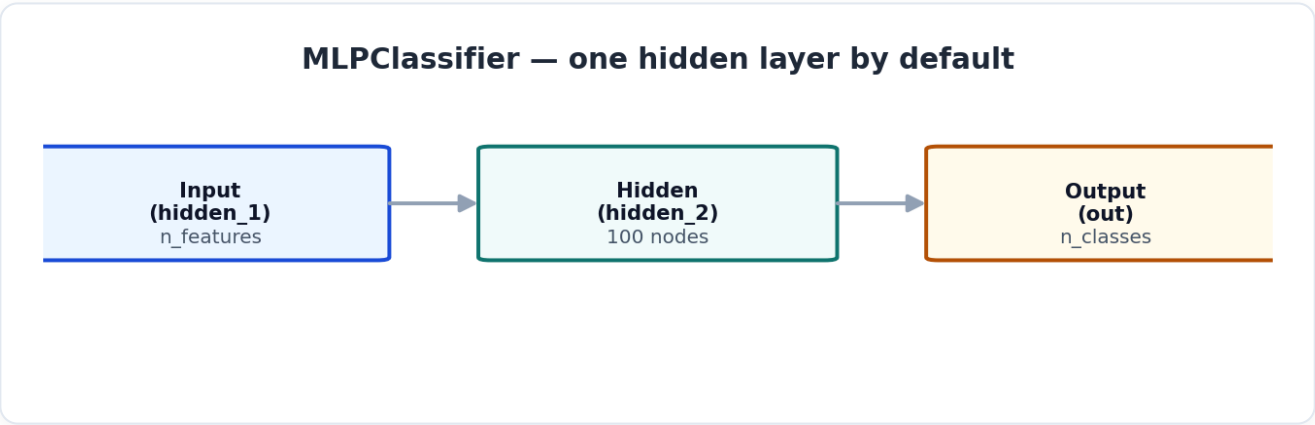
`Nystroem` يترك `gamma=None` فيأخذ `1/n_features` تلقائياً، لكن `RBFSampler` افتراضه `gamma=1.0` قاسٍ: على بياناتنا أعطى ٠.٥٦٠٢ مقابل ٠.٨١٧٧ بـ `gamma=1.0/12`. مرّر gamma يدوياً لـ `RBFSampler` دوماً، وبنفس القيمة المستخدمة في أي نواة اختبارية.

متى Nystroem ومتى RBFSampler؟

Nystroem يستخدم عينات حقيقية كنقاط معلم فيكون أدق لنفس عدد المكونات؛ RBFSampler تحويل عشوائي ثابت لا يتدرب فيكون أرخص عند إعادة التدريب المتكرر وعلى التدفقات. ابدأ بـ Nystroem وانتقل حين يتكرر التدريب من الصفر.

V.٤ MLPClassifier و MLPRegressor: البنية والتقارب

نصل للشبكات العصبية: `sklearn.neural_network.MLPClassifier` مصنّف هاطبات متعدد الطبقات (Multi-Layer Perceptron) يتغذى للأمام فقط:



الشكل ١٢٦بنية الشبكة متعددة الطبقات: طبقة إدخال وطبقتان خفيتان وطبقة إخراج مع دوال التفعيل

في الشكل يُقرأ التشريح: كل عقدة خفية تجمع المدخلات موزونة ثم تمررها عبر دالة تفعيل ϕ، وتتكرر العملية طبقة بعد طبقة حتى الإخراج:

$$\mathbf{h}^{(1)} = \phi(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \qquad \hat{y} = g(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

حيث W الأوزان و b الانحيازات و ϕ تفعيل الطبقات الخفية (`relu` افتراضياً أو `tanh` أو `logistic`) و g تفعيل الإخراج (softmax للتصنيف). والتنظيم بـ `alpha` يضيف عقوبة $L2$ على الأوزان — نقطة التقاء مع انحدار الريج (الفصل ٢٩): الافتراضي ٠.٠٠٠١ صغير للشبكات المفرطة، ورفعه يقلّص فرط الملاءمة. الآن التجربة على الهلالين حيث يفشل أي حد خطي:

```python
# MLP على الهلالين: التفعيل والحلول والإيقاف المبكر
from sklearn.datasets import make_moons
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = make_moons(n_samples=800, noise=0.3, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
print(f"اللوجيستي الخطي (سقفه الحد المستقيم): {LogisticRegression(max_iter=1000).fit(X_tr,
y_tr).score(X_te, y_te):.4f}")

for act in ["relu", "tanh"]:
    m = MLPClassifier(hidden_layer_sizes=(32, 16), activation=act,
max_iter=2000, random_state=42).fit(X_tr, y_tr)
    print(f"MLP(32,16) تفعيل={act:5s} الدقة: {m.score(X_te, y_te):.4f} | تكرارات:
{m.n_iter_}")

m_lb = MLPClassifier(hidden_layer_sizes=(32, 16), solver="lbfgs", max_iter=2000,
random_state=42).fit(X_tr, y_tr)
print(f"MLP(32,16) solver=lbfgs الدقة: {m_lb.score(X_te, y_te):.4f} | تكرارات:
{m_lb.n_iter_}")

m_es = MLPClassifier(hidden_layer_sizes=(32, 16), max_iter=2000,
early_stopping=True,
                     validation_fraction=0.15, random_state=42).fit(X_tr, y_tr)
print(f"مع early_stopping: الدقة {m_es.score(X_te, y_te):.4f} | تكرارات:
{m_es.n_iter_}")
```

أربع قراءات: أولاً، MLP تجاوزت سقف الخط المستقيم (٠.٨٧٥٠ → ٠.٩١٦٧) لأن الطبقة الخفية تثني الفضاء. ثانياً، `tanh` تفوقت هنا قليلاً على `relu` (٠.٩٢٠٨) — التفعيل فرط-بارامتر حقيقي. ثالثاً، المحلول `lbfgs` شبه-النيوتني وصل ٠.٩٠٤٢ لكنه احتاج ١٨١٦ تكراراً: ممتاز للبيانات الصغيرة، بينما `adam` الافتراضي للمتوسطة والكبيرة و `sgd` للتحكم اليدوي بمعدل التعلّم. رابعاً، `early_stopping=True` يحجز `validation_fraction` (١٥% هنا) ويوقف عند توقف تحسّن التحقق: التكرارات انخفضت من ٣٧٨ إلى ٤٠ والدقة من ٠.٩١٦٧ إلى ٠.٨٦٢٥ — واقٍ من الفرط لكنه قد يوقف مبكراً، فاضبط معه `n_iter_no_change`.

أما `MLPRegressor` فنفس الواجهة والبارامترات للانحدار، بخسارة تربيعية و `score` يعيد R^2 — وقواعد هذا القسم تنطبق عليه حرفياً.

ConvergenceWarning من كل زاوية

الافتراضي `max_iter=200` قاتل: معظم الشبكات تحتاج ٣٠٠–١٠٠٠ تكرار فيظهر `ConvergenceWarning` والنموذج ناقص التدريب بصمت. ارفع `max_iter` حتى يختفي التحذير أو يثبت `loss_curve_`، ولا تدعه يمر في سجلات الإنتاج.

٧.٥ التطبيع شرط البقاء، والتدريب التزايدي بـ partial_fit

لماذا تفشل MLP حيث ينجح المصنف الخطي على البيانات نفسها؟ لأن الانحدار التدريجي يوزع التحديث على الطبقات، وميزة بمقياس هائل تسيطر على التدرجات فتغرق ميزات أدق. تجربة قاسية: ميزتان بمقاييس متباعدة وبنية XOR، ثم تدريب تزايدي منضبط على Digits:

أثر التطبيع ثم التدريب التزايدي بـ partial_fit python

```python
import warnings
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

# الجزء الأول: نفس الشبكة على ميزات متطرفة المقياس، مع تطبيع وبلا تطبيع
rng = np.random.default_rng(42)
n = 900
salary = rng.normal(5200, 1200, size=(n, 1))   # آلاف الدولارات
ratio = rng.normal(0.35, 0.09, size=(n, 1))    # نسبة بين 0 و 1
label = ((salary[:, 0] > 5200) ^ (ratio[:, 0] > 0.35)).astype(int)
X = np.hstack([salary, ratio])
X_tr, X_te, y_tr, y_te = train_test_split(X, label, test_size=0.3,
random_state=42, stratify=label)

# نكتم التحذير لأن هدفنا قياس أثر المقياس على الدقة لا حالة التقارب
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    raw = MLPClassifier(hidden_layer_sizes=(16,), max_iter=1000,
random_state=42).fit(X_tr, y_tr)
scaler = StandardScaler().fit(X_tr)
scaled = MLPClassifier(hidden_layer_sizes=(16,), max_iter=1000,
random_state=42).fit(
    np.asarray(scaler.transform(X_tr)), y_tr)
print(f"بلا تطبيع: {raw.score(np.asarray(scaler.transform(X_te)), y_te):.4f} | "
      f"مع StandardScaler: {scaled.score(np.asarray(scaler.transform(X_te)),
y_te):.4f}")

# الجزء الثاني: تدريب تزايدي دفعة-دفعة على Digits المحجّمة
X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
sc = StandardScaler().fit(X_tr)
# np.asarray لضمان مصفوفة رقمية مهما كان transform_output
X_tr_s, X_te_s = np.asarray(sc.transform(X_tr)), np.asarray(sc.transform(X_te))
mlp = MLPClassifier(hidden_layer_sizes=(64,), random_state=42)
classes = np.unique(y)
rng = np.random.default_rng(42)
for epoch in range(10):
    order = rng.permutation(len(X_tr_s))
    for start in range(0, len(X_tr_s), 128):
        batch = order[start:start + 128]
        mlp.partial_fit(X_tr_s[batch], y_tr[batch], classes=classes)
    print(f"الدورة {epoch + 1:2d}: loss_ = {mlp.loss_:.4f}")
print(f"الدقة على الاختبار بعد التدريب التزايدي: {mlp.score(X_te_s, y_te):.4f}")
```

رقمان يختصران القسم كله. الأول: ٠.٦٨١٥ مقابل ٠.٩٩٢٦ — نفس البنية ونفس التكرارات، والفرق سطر StandardScaler واحد (الفصل ١٤). الثاني: الخسارة تنزل من ١.٨٤٣٦ إلى ٠.٣٧٠٠ عبر عشر دورات دفعية والدقة ٠.٩٠٧٤ — تدريب تزايدي كامل أنت من قاد دفعاته، وأنسب `partial_fit` للتدفقات والتعلم المستمر، مع الالتزام بتمرير `classes` في النداء الأول.

> ⛔ **MLP بلا تحجيم = أرقام بلا معنى**
>
> قاعدة لا تُخالف: `StandardScaler` (أو `MinMaxScaler` بحدودها) قبل كل MLP وداخل `Pipeline` حصراً حتى لا يتسرب مقياس الاختبار إلى التدريب. الأرقام العالية النادرة بلا تحجيم مصادفة لا مهارة.

> 🔬 **حدود MLP في scikit-learn**
>
> لا GPU ولا تدريب موزع ولا شبكات عميقة هنا: MLP في scikit-learn شبكات متوسطة على معالج واحد. حين تتجاوز شبكتك عشرات الطبقات أو بياناتك ملايين الصور فأدوات مثل PyTorch و Keras هي البيت الصحيح — وخارج نطاق هذا الكتاب عمداً: scikit-learn نواة أنيقة لتعلّم الآلة الجدولي، لا مصنع شبكات عميقة.

٧.٦ BernoulliRBM: خصائص ثنائية بلا إشراف

أخيراً أداة هذه العائلة الغريبة: `sklearn.neural_network.BernoulliRBM` شبكة بولتزمان محدودة (Restricted Boltzmann Machine) تتعلّم بنية البيانات **دون أي تسميات**. مدخلاتها حكر على الثنائية ٠/١، وطاقتها الداخلية:

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h} - \mathbf{v}^\top \mathbf{W} \mathbf{h}$$

حيث `v` المتجه المرئي (المدخل) و `h` المتجه الخفي و `W` الأوزان و `b` و `c` الانحيازان. التدريب يخفض طاقة التكوينات الشائعة ويرفع طاقة النادرة، فيصير المتجه الخفي `h` بصمة مضغوطة لأنماط المدخل. عملياً استخدمها **مستخرج خصائص** يسبق مصنفاً خطياً، لا مصنفاً بذاته. التجربة على Digits مرقمنة:

مقابل لوجيستي مباشر LogisticRegression ثم BernoulliRBM python

```python
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.neural_network import BernoulliRBM
from sklearn.pipeline import make_pipeline

X, y = load_digits(return_X_y=True)
X_bin = (X > 4).astype(float)   # (0) أو مطفأ (1) رقمنة: بكسل مضيء
X_tr, X_te, y_tr, y_te = train_test_split(X_bin, y, test_size=0.3,
random_state=42, stratify=y)

direct = LogisticRegression(max_iter=2000).fit(X_tr, y_tr)
print(f"اللوجيستي على البكسلات الخام: {direct.score(X_te, y_te):.4f}")

rbm_pipe = make_pipeline(
    BernoulliRBM(n_components=64, learning_rate=0.1, n_iter=30,
random_state=42),
    LogisticRegression(max_iter=2000),
)
rbm_pipe.fit(X_tr, y_tr)
print(f"اللوجيستي على خصائص RBM (64):  {rbm_pipe.score(X_te, y_te):.4f}")
```

الرقم يتكلم: ٠.٩١٣٠ على ٦٤ بكسل خام مقابل ٠.٩٢٩٦ على ٦٤ خصيصة تعلمها RBM بلا نظرة واحدة للتسميات. RBM لم تصنّف شيئاً — أعادت فقط وصف البيانات بمفردات أنماط (حواف وأقواس وأشكال أرقام) يفهمها المصنف الخطي التالي، وهي نفس فلسفة تقليل الأبعاد غير المشرف التي سيفتتح بها الجزء الثامن (الفصلان ٦٨ و٦٩). وقد أعاد هينتون وأوسيندرو في ورقة ٢٠٠٦ ترتيب تدريب الشبكات العميقة عبر تكديس طبقات RBM بلا إشراف — السطر الذي فتح الطريق لعصر التعلم العميق.

وحدة التوصيات والبيانات الثنائية

حيثما كانت بياناتك ثنائية أصلاً — سلة مشتريات، نقرات، أعراض — يصبح RBM مستخرج خصائص رخيصاً بلا تسميات: درّب المستخرج على الكل، والمصنف على القليل الموسوم.

تمارين الفصل الثامن والخمسين

- **ملاحظة**

التمرين ٧.١

على ٣٠٠٠ عينة من California Housing مع تحجيم، امسح `alpha` في KernelRidge بنواة RBF ثابتة (`gamma=0.05`) عبر القيم `0.001` و `0.01` و `0.1` و `1.0` و `10.0`، وقيّم بـ RMSE. أين القاع؟

التمرين ٧.٢

على Digits مع StandardScaler، قارن دقة وعدد تكرارات MLPClassifier بالبنى `(32,)` و `(64,)` و `(100, 50)` و `(100, 50, 25)` مع `early_stopping=True`. هل الأعماق الأكبر تفوز دائماً؟

التمرين ٧.٣

بدون كود: لماذا يظل ثمن تنبؤ KernelRidge `O(n)` لكل استعلام مهما ضبطت `alpha`، بينما يتقلب ثمن تنبؤ SVR مع عدد المتجهات الداعمة؟

- **ملاحظة**

حلّ التمرين ٧.١: مسح alpha في KernelRidge — python

```python
from sklearn.datasets import fetch_california_housing
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

data = fetch_california_housing()
X, y = data.data[:3000], data.target[:3000]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler().fit(X_tr)
for alpha in [0.001, 0.01, 0.1, 1.0, 10.0]:
    model = KernelRidge(kernel="rbf", alpha=alpha,
gamma=0.05).fit(scaler.transform(X_tr), y_tr)
    rmse = root_mean_squared_error(y_te, model.predict(scaler.transform(X_te)))
    print(f"alpha={alpha:6.3f} | RMSE = {rmse:.4f}")
```

منحنى حرفي U: `alpha` الصغير جداً (٠.٠٠١) يمنح RMSE = ٠.٤١٧٠ بحفر فرط-ملاءمة، والقاع عند ٠.١ بـ RMSE = ٠.٣٤١٥، ثم يتراجع عند ١٠.٠ إلى ٠.٤٥٥١ لانزياس النموذج نحو الثبات. هذا دور alpha: تنظيم المعاملات الثنائية لا مقاومة فرط تكراري كما في الشبكات.

> **ملاحظة**

```python
# حلّ التمرين ٧.٢: مسح بنى MLP على Digits
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
scaler = StandardScaler().fit(X_tr)
for hls in [(32,), (64,), (100, 50), (100, 50, 25)]:
    m = MLPClassifier(hidden_layer_sizes=hls, max_iter=600, early_stopping=True,
                      random_state=42).fit(scaler.transform(X_tr), y_tr)
    print(f"hidden_layer_sizes={str(hls):14s} الدقة:
{m.score(scaler.transform(X_te), y_te):.4f} | تكرارات: {m.n_iter_}")
```

سلّم صاعد: ٠.٨٨٨٩ ثم ٠.٩٥٧٤ ثم ٠.٩٧٠٤ ثم ٠.٩٧٩٦ — العمق الأكبر فاز هنا لأن Digits تحمل بنية هرمية (حواف ← مقاطع ← أرقام كاملة)، ولم يحتج العمق الأكبر تكرارات أكثر (٣٥ مقابل ٣٣ و ٤٢). لا تعمم: النتيجة تنقلب على بيانات صغيرة مشوشة حيث تفرط الشبكات الكبيرة — ابدأ صغيراً وكبّر بحسب منحنى التعلم.

> **ملاحظة**

٧.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إطعام MLP بيانات غير محجيمة	تذبذب التدرجات وتقارب فاشل دائم ودقة شبه عشوائية ConvergenceWarningg	داخل `StandardScaler` قبل MLP دوماً `Pipeline`
الاكتفاء بـ `max_iter=200` الافتراضي	`ConvergenceWarning` ونموذج ناقص التدريب بصمت	رفع `max_iter` إلى ٥٠٠–١٠٠٠ ومراقبة `loss_curve_`
ترك `gamma=1.0` الافتراضي في RBFSampler	تقريب نواة شبه عشوائي ودقة أقل من الخطي أحياناً	تمرير `gamma` يدوياً (مثل `1/n_features`) مطابقاً لأي نواة اختبارية
استخدام KernelRidge حيث التنبؤ كثيف	ثمن تنبؤ `O(n)` لكل استعلام يفوق كل توفير التدريب	SVR أو تقريب النواة مع مصنف خطي حين يكون التنبؤ ساخناً

٧.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
انحدار النواة	ريج بحل مغلق في فضاء النواة بمعامل لكل عينة تدريب	Kernel Ridge Regression
تقريب النواة	فضاء ميزات صريح محدود يبقى ضربه القياسي قريباً من النواة	Kernel approximation
نقاط المعلم	عينة صغيرة تُبنى عليها مصفوفة النواة الفرعية في Nystroem	Landmark points
هاطبات متعددة الطبقات	شبكة تتغذى للأمام بطبقات خفية ودوال تفعيل	Multi-Layer Perceptron (MLP)
آلة بولتزمان المقيدة	نموذج توليدي ثنائي يتعلم بنية البيانات بالطاقة بلا إشراف	Restricted Boltzmann Machine (RBM)

بهذا الفصل نُغلق الجزء السابع بأكمله: من آلات المتجهات الداعمة إلى الجيران، من بيز إلى التمييز الغاوسي، ومن النواة إلى الشبكات. وتأمل ما جمعته: كل مصنفات هذا الجزء كانت تحتاج **تسميات** تتعلم منها. في الجزء الثامن ننزع هذا الشرط كلياً وننتقل إلى التعلّم غير الخاضع للرقابة (الفصل ٥٩): لا إجابات نموذجية هناك، بل بنية مخفية تنتظر من يكتشفها — تجميع وتقليل أبعاد وكشف شذوذ، بلا من يهمس بالجواب الصحيح.

الجزء ٨

التعلّم غير الخاضع للرقابة

عندما لا توجد إجابة نموذجية، بل يوجد بنية مخفية

ماذا يغطّي هذا الجزء

بلا تسميات، تبحث هذه العائلة عن الأنماط لا عن الإجابات. التجميع يجد الكتل، وتقليل الأبعاد يفكّ التداخل، وكشف الشذوذ يلتقط ما لا يشبه أحداً. نميّز بوضوح بين الطرق التي تطلب عدد الكتل مسبقاً وتلك التي تكتشفه، وبين البُنى المرنة والبُنى المحدّبة.

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٥٩ — التعلّم غير الخاضع للرقابة: الخريطة

ماذا يعني بلا تسميات، ومتى يكون مفيداً ومتى يكون فخاً

الوحدة **scikit-learn ١.٩** الحجم **٨٠٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تصنّف أي مسألة بلا تسميات إلى إحدى العائلات الأربع وتحدد السؤال الذي تجيب عنه فعلاً
- أن تميّز بين الواجهات الثلاث: fit_predict و fit_transform و score_samples وما يعيده كل منها
- أن تشرح لماذا لا يوجد معيار داخلي واحد يخبرك بعدد العناقيد «الصحيح»
- أن تكتشف بنفسك حالة يفشل فيها KMeans في بيانات واضحة للعين البشرية
- أن تضع خريطة ذهنية لفصول الجزء الثامن لتعرف متى تلجأ لأي عائلة

٨.١ بلا تسميات: ماذا نبحث أصلاً؟

تخيّل أن سُلّمتَ صندوقاً فيه عشرون ألف صورة لم يصنّفها أحد، وطُلب منك أن «تنظّمها». لا يوجد عمود يقول «قطة» أو «منظر طبيعي». ماذا ستفعل؟ لن تبحث عن إجابة؛ ستبحث عن **بنية**: هذه الصور متشابهة في الألوان الداكنة، وهذه تتشاور في الإضاءة القوية، وهذه عزبة عن الجميع. هذا بالضبط ما تفعله خوارزميات **التعلّم غير الخاضع للرقابة (Unsupervised Learning)**: تُغذّى بالبيانات `X` وحدها بلا `y`، وتبحث عن أنماط داخلها.

لكن انتبه إلى نقطة اختفاء جوهرية: حين يصنف نموذجنا المُشرَف صورة، نستطيع قياس صوابه لأن لدينا الإجابة، أما هنا فلا إجابة نقيس عليها. معادلة اللعبة تغيّرت: النموذج المُشرَف يطلب دالة `f` تجعل `f(x)` قريبة من `y`، وغير المُشرَف يطلب دالة درجة `S` تصنع البنية من `X` وحدها.

$$\text{supervised}: \; f(\mathbf{x}) \approx y \qquad ; \qquad \text{unsupervised}: \; S(X) \to \text{structure}$$

الدالة `S` هنا رمز لأي دالة درجة داخلية تختارها الخوارزمية: تباين داخل الكتل، أو كثافة نقاط، أو تباعد بين المحاور. الخوارزمية تُحسّن درجتها *الداخلية*، لا «صواباً» مرجعياً. وهذه هي فكرة الفصل كلها، فاحفظها:

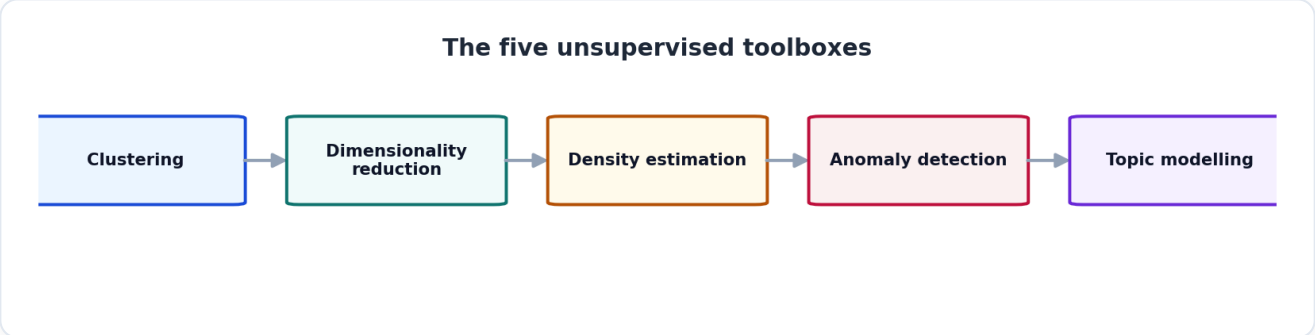
> 🔑 **البنية تُعرَّف بالاختيار، لا بالاكتشاف**
>
> كل خوارزمية غير مُشرَفة تحمل معها تعريفاً صريحاً لما يعنيه «بنية جيدة»: KMeans يعرفها بأنها كتل دائرية متساوية الانتشار حول مراكز، وDBSCAN يعرفها بأنها أحياء كثيفة متصلة. حين «تكتشف» الخوارزمية بنية، هي في الحقيقة تعيد إليك بنية تعريفها الخاص بعد تصفيته على بياناتك. لذلك سؤال «أي خوارزمية صحيحة؟» أسئلة لا معنى له هنا؛ السؤال الصحيح: «تعريف أيّ خوارزمية يطابق بنية بياناتي التي أعتقد أنها موجودة؟».

وهنا يجب أن نصارحك قبل أي سطر كود: **المجال كله مغري وخطر في آن واحد**. مغري لأن البيانات غير الموسومة هي ٩٩% من بيانات العالم، وخطر لأن غياب `y` يعني غياب حكم. فلنرتب الأفكار في خريطة واحدة.

٨.٢ العائلات الأربع: خريطة الجزء الثامن

كل خوارزميات الجزء الثامن — أكثر من ثلاثين صنفاً في sklearn — تقع في أربع عائلات كبرى، ولكل عائلة سؤال مختلف تماماً. هذه الجدول هو أهم ما في الفصل:

العائلة	السؤال الذي تجيب عنه	أصناف sklearn الرئيسية	فصول الكتاب المتقدمة
التجميع (Clustering)	«أين تتجمّع النقاط في مجموعات؟» (find groups)	`sklearn.cluster.KMeans` · `MiniBatchKMeans` · `BisectingKMeans` · `MeanShift` · `AffinityPropagation` · `DBSCAN` · `HDBSCAN` · `AgglomerativeClustering`	٦٥–٦٧
تقليل الأبعاد (Dimensionality Reduction)	«ما المحاور القليلة التي تختصر البيانات؟» (find axes)	`sklearn.decomposition.PCA` · `TruncatedSVD` · `sklearn.manifold.TSNE` · `KernelPCA` · `Isomap`	٦٨–٦٩
كشف الشذوذ (Anomaly Detection)	«أي النقاط لا تشبه أحداً؟» (find outliers)	`sklearn.ensemble.IsolationForest` · `sklearn.neighbors.LocalOutlierFactor` · `sklearn.svm.OneClassSVM`	٧٠
تقدير الكثافة/التوزيع (Density Estimation)	«ما التوزيع الذي ولّد هذه العينات؟» (find distribution)	`sklearn.mixture.GaussianMixture` · `BayesianGaussianMixture` · `sklearn.neighbors.KernelDensity`	٦٧ و ٧١



الشكل ١٢٧ الخريطة المفاهيمية للتعلّم غير الخاضع للرقابة: بيانات بلا تسميات تدخل أربع عائلات، ولكل عائلة سؤال ونوع ناتج مختلف

في الشكل تلاحظ أن المدخل واحد (`X` بلا تسميات) والنواتج أربعة ألوان مختلفة: تسميات كتل، أو إحداثيات جديدة مضغوطة، أو أحكام شذوذ، أو دالة كثافة قابلة للتقييم على أي نقطة. هذا التنوع في النواتج هو الذي يقودنا إلى القسم التالي: الواجهات البرمجية نفسها تختلف من عائلة إلى أخرى.

> **جديد في ١.٩: HDBSCAN عضو كامل في العائلة**
>
> منذ الإصدار ١.٩ صار `sklearn.cluster.HDBSCAN` عضواً كاملاً في مكتبة scikit-learn بكل متطلبات الواجهة الرسمية (`fit_predict`، والتوافق مع المعايرة والتتبع). سابقاً كان معظم الممارسين يعتمدون حزمة خارجية مستقلة. سيتولى الفصل ٦٤ شرحه بالتفصيل، وسيغيّر نظرتك لفكرة «عدد الكتل المطلوب مسبقاً» لأنه يستغني عنها جذرياً.

٨.٣ الواجهات الثلاث: كيف تختلف العائلات في الكود؟

لو نسيّت كل تفاصيل هذا الفصل، احفظ هذه الجملة: **العائلات الأربع تختلف في طريقة سؤالك لها وليس في طريقة تدريبها فقط**. الواجهات الثلاث الكبرى:

النمط	العائلة	ما يعيده بالضبط	أمثلة
`fit_predict(X)`	التجميع وكشف الشذوذ	تسمية أو حكم لكل عينة	`sklearn.cluster.KMeans` · `sklearn.ensemble.IsolationForest`
`fit_transform(X)`	تقليل الأبعاد	تمثيل جديد بأبعاد أقل	`sklearn.decomposition.PCA`
`score_samples(X)`	الكثافة والشذوذ	درجة لكل عينة (كثافة أو «غرابة»)	`sklearn.neighbors.KernelDensity` · `sklearn.mixture.GaussianMixture`

ننفّذ الآن أربعة أمثلة قصيرة على بيانات اصطناعية مختلفة، لكل عائلة مثال واحد بأسلوبها الخاص. أولها التجميع عبر `sklearn.cluster.KMeans`:

```python
# التجميع: fit_predict يعيد تسمية لكل نقطة
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.9, random_state=42)
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)
print("fit_predict: أول 8 تسميات من", kmeans.fit_predict(X)[:8])
print("cluster_centers_: شكل المراكز", kmeans.cluster_centers_.shape)
print(f"inertia: مجموع مربعات المسافات {kmeans.inertia_:.2f}")
```

لاحظ أن الناتج **تسميات صحيحة عشوائية** `0..3` لا أكثر: العنقود رقم ٣ ليس «العنقود الثالث» بأي معنى، بل مجرد اسم اعتباطي أعطته الخوارزمية للكتلة. الآن عائلة تقليل الأبعاد عبر `sklearn.decomposition.PCA`:

```python
# تقليل الأبعاد: fit_transform يعيد إحداثيات جديدة
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris

X_iris, y_iris = load_iris(return_X_y=True)
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_iris)
print(f"الأبعاد قبل التحويل: {X_iris.shape} — بعد fit_transform: {X_2d.shape}")
print(f"نسبة التباين المفسَّر بالمكوّنين معاً: {pca.explained_variance_ratio_.sum():.4f}")
```

هنا لا تسميات إطلاقاً؛ الناتج مصفوفة جديدة `150×2` تكون كل نقطة فيها إسقاطاً للزهرة على محورين مبتكرين. ثم كشف الشذوذ عبر `sklearn.ensemble.IsolationForest` مع عشر نقاط محقونة يدوياً بعيداً عن الجميع:

```python
# كشف الشذوذ: score_samples يمنح كل نقطة درجة غرابة
from sklearn.ensemble import IsolationForest
import numpy as np

rng = np.random.default_rng(42)
X_norm = rng.normal(loc=[0, 0], scale=1.0, size=(200, 2))
X_out = rng.uniform(low=6.0, high=9.0, size=(10, 2))
X_mix = np.vstack([X_norm, X_out])

iso = IsolationForest(n_estimators=100, random_state=42).fit(X_mix)
scores = iso.score_samples(X_mix)
print(f"متوسط درجة الكثافة للعينات العادية:  {scores[:200].mean():.4f}")
print(f"متوسط درجة الكثافة للشواذ المحقونة: {scores[200:].mean():.4f}")
idx_low = scores.argsort()[:10]
print(f"من بين أدنى 10 درجات، كم نقطة شاذة محقونة؟ {int((idx_low >= 200).sum())} من 10")
```

تسع من النقاط العشر المحقونة اجتاحت قائمة الأسوأ كما توقعت، وواحدة أفلتت — نقطة عادية من حدود السحابة طبيعياً كانت أغراب منها. وتذكّر: لم نطلب من النموذج أن «يكشف الشذوذ»؛ الدرجة التي يحسبها ليست إلا تقدير كثافة معكوس، وهذا يوصلنا للعائلة الرابعة، تقدير الكثافة عبر `sklearn.neighbors.KernelDensity`. فكرتها تقدير دالة الكثافة الاحتمالية `p(x)` باختيار معاملات `theta` التي ترفع احتمال البيانات المرصودة:

$$\hat{\theta} = \arg\max_{\theta} \sum_{i=1}^{n} \log p_{\theta}(\mathbf{x}_i)$$

تقدير الكثافة: score_samples يعيد لوغاريتم الكثافة python

```python
from sklearn.neighbors import KernelDensity
import numpy as np

rng = np.random.default_rng(42)
X_dens = rng.normal(loc=0.0, scale=1.0, size=(400, 1))
kd = KernelDensity(bandwidth=0.3).fit(X_dens)
log_d_center = kd.score_samples([[0.0]])   # قلب السحابة
log_d_far = kd.score_samples([[4.0]])      # هامش بعيد
print(f"0.0 لوغاريتم الكثافة المقدَّرة عند القلب: {log_d_center[0]:.4f}")
print(f"4.0 لوغاريتم الكثافة المقدَّرة عند الهامش: {log_d_far[0]:.4f}")
```

حسبت النقطة عند قلب السحابة كثافة أعلى بأسابيع حسابية من الهامش البعيد (فرق اللوغاريتمات نحو ١٠.٧)، وهذا بالضبط ما تعنيه عبارة «دالة كثافة قابلة للتقييم على أي نقطة جديدة»، وهي قدرة لا يملكها KMeans أصلاً.

٨.٤ التجميع يجد بنية، لا حقيقة

ننتقل الآن للفخ الأخطر في المجال كله. لنضع بيانات «هلالين» متداخلين — أوضح بنية للعين البشرية إطلاقاً — ونطلب من KMeans أن يجد عنقودين، ثم نقيس التوافق مع الحقيقة التي نعرفها نحن (لأننا ولّدنا البيانات). نستعمل `sklearn.metrics.adjusted_rand_score`، وهو مقياس يقيس اتفاق تسميتين مستقلاً عن أسمائهما، وسيشرحه الفصل ٦٦ بالتفصيل:

```python
# تجربة: KMeans على الهلالين — بنية بلا حقيقة
from sklearn.cluster import KMeans
from sklearn.datasets import make_moons
from sklearn.metrics import adjusted_rand_score

X_moons, y_true = make_moons(n_samples=400, noise=0.06, random_state=42)
labels_km = KMeans(n_clusters=2, n_init=10,
random_state=42).fit_predict(X_moons)
ari = adjusted_rand_score(y_true, labels_km)
print(f"KMeans 2 طلبنا من، فأعاد عنقودين: {(labels_km.max()+1)==2}")
print(f"ARI بين تسميات KMeans والهلالين الحقيقيين: {ari:.4f}")
```

معامل التوافق ٠.٢٤٨١ فقط — إخفاق ساحق رغم أن بنية الهلالين واضحة لعينك في لقطة واحدة. ماذا حدث؟ KMeans يحسّن هدفاً داخلياً واحداً: مجموع مربعات المسافات بين النقاط ومركزها:

$$J = \sum_{i=1}^{n} \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|_2^2$$

حيث `n` عدد العينات، و `x_i` هي العينة رقم `i`، و `c(i)` هي تسمية عنقودها، و `mu` مركز ذلك العنقود. تقليل هذا الهدف يفرض كتلاً **محدّبة** تفصلها حدودٌ مستقيمة، والهلالان المقوسان لا يملكان هذه الخاصية أصلاً، فقسمهما الخوارزمية قسيماً عمودياً لا علاقة له بالحقيقة. الخوارزمية لم «تخطئ»؛ هي نفّذت تعريفها على بيانات لا تطابق التعريف. الفشل هنا معلومة عن *تعريف البنية* لا عن البيانات.

> **لا تسمِّ العناقيد أسماء حقيقية قبل التحقق**
>
> أخطر خطأ عملي في المشاريع الحقيقية: تشغّل التجميع على بيانات عملائك، ثم تخترع للعناقيد أسماءً مثل «العملاء الأوفياء» و«العملاء العابرين» وتعرضها للإدارة بوصفها اكتشافاً ساحراً. العناقيد أرقام اعتباطية مرتّبة عشوائياً، ومعناها يُبنى بتحليل خصائص كل كتلة (متوسطات الميزات فيها) ثم *التحقق* بتقسيم العناقيد على نتائج أعمال معلومة. بدون هذه الخطوة فأنت تقدّم إلى الإدارة وهمَ اكتشافٍ لا اكتشافاً. سنرى أدوات هذا التحقق في الفصل ٦٦.

٨.٥ لا يوجد «رقم صحيح» واحداً

سؤال يطرحه كل مبتدئ: كم عنقوداً *يوجد* فعلاً في بياناتي؟ دعنا نجرب على سحابة فيها خمسة مراكز حقيقية — نحن نعرف الجواب لأننا ولّدناها:

```python
# تجربة: نفس البيانات، ثلاث إجابات مختلفة
from sklearn.cluster import KMeans, MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs

X5, _ = make_blobs(n_samples=400, centers=5, cluster_std=0.7, random_state=42)
for k in [3, 5, 8]:
    km_k = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X5)
    print(f"KMeans ← k={k}: inertia = {km_k.inertia_:.1f}")
bw = estimate_bandwidth(X5, quantile=0.15, random_state=42)
ms = MeanShift(bandwidth=bw).fit(X5)
print(f"MeanShift اختار بنفسه {ms.labels_.max() + 1} عناقيد بلا أن نطلب رقماً")
```

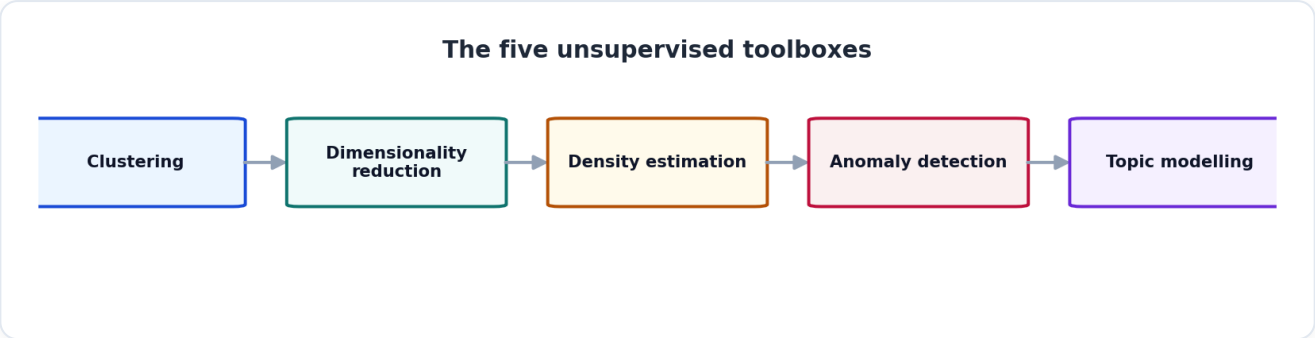
اقرأ الأرقام جيداً: عندما رفعنا `k` من ٣ إلى ٨ هبط الهدف الداخلي `inertia` من ٢٥٥٦.٤ إلى ٢٩٣.٣ — **دائماً** هبوط، مهما رفعت `k`، لأن مزيداً من المراكز يعني دائماً مسافات أقصر. الهدف الداخلي وحده لا يستطيع أن يقول «توقف عند ٥». في المقابل، خوارزمية MeanShift التي لا تُطلب منها كتلة (وسيتكفل بها الفصل ٦٢) اختارت بنفسها ٥ — اختبارها الصحيح، لأن *تعريف* بنيتها (قمم الكثافة) يطابق طريقة توليد البيانات. هذا هو المعيار الحقيقي للاختيار: طابق تعريف الخوارزمية مع بنية بياناتك، ثم استعمل مقاييس التقييم الخارجية والداخلية التي يفصّلها الفصل ٦٦.

متى يكون بلا تسميات مفيداً فعلاً؟

في المشاريع الحقيقية تنجح العائلات الأربع في مواضع محددة: تجميع العملاء لاستهدافهم بحملات مختلفة (التجميع)، وضغط ميزات هائلة قبل نموذج مُشرَف (تقليل الأبعاد — وسيط لا غاية)، وفلترة معاملات مالية غريبة لفرزها للمراجعة البشرية (كشف الشذوذ)، وتمثيل توزيع قياسات حسّاس لكشف انحراف المصنع (تقدير الكثافة). أما إن سُئلت «ما الحقيقة الكامنة؟» فالجواب المهني: التجميع أعطاني *لغة وصف* للبيانات، والتحقق من صحتها مسؤوليتي أنا.

٨.٦ خريطة الفصول القادمة

كيف ستقرأ الجزء الثامن؟ عد إلى خريطة ٨.٢ قبل أن نغادرها، فهي مفتاح القراءة لكل ما سيأتي: كل لون فيها يقابلك في فصول هذا الجزء.



الشكل ١٢٨خريطة الجزء الثامن كاملة: أربعة مداخل من بيانات بلا تسميات إلى أربعة أنواع من النواتج، ولكل مدخل فصوله الخاصة

المسار مقسوم بحسب العائلات: الفصول ٦٠–٦٢ (هذا الفصل يليها مباشرة) تفتتح التجميع بنسخ KMeans الثلاث وخوارزميتي التقدير التلقائي؛ الفصلان ٦٣ و٦٤ يوسّعان إلى العناقيد الكثيفة غير المحدّبة مع DBSCAN و OPTICS و HDBSCAN؛ الفصل ٦٥ للتجميع الهرمي والطوبولوجي؛ والفصل ٦٦ — الأهم عملياً — يدرّسك كيف تقيس نجاح تجميع لا تملك إجابة مرجعية له. ثم تنفتح بقية العائلات: النماذج المزيجية والتوليد الاحتمالي في ٦٧، وتقليل الأبعاد الخطي وغير الخطي في ٦٨–٦٩، وكشف الشذوذ في ٧٠، والنصوص في ٧١.

عائلة التجميع وحدها تأخذ ستة فصول لأنها الأكثر استعمالاً والأكثر زلاقةً عملياً، وننطلق الآن إلى أول معالمها: KMeans، أشهر خوارزمية تجميع في التاريخ، بسيطة إلى حد الصدمة وقيّدة إلى حد الإحباط — وسنفكّك الاثنين في الفصل ٦٠.

تمارين الفصل التاسع والخمسين

- **ملاحظة**

التمرين ٨.١

على بيانات `make_blobs` الواردة في القسم ٨.٣ (السحابة `X` ذات ٣٠٠ عينة)، طبّق الواجهات الثلاث على البيانات نفسها: تسمية العناقيد بـ `KMeans`، والإسقاط ثنائي الأبعاد بـ `PCA`، ودرجات الغرابة بـ `IsolationForest`، وطبع ثلاث جمل تصف ما «أجاب» عنه كل نموذج عن البيانات ذاتها.

التمرين ٨.٢

ولّد `make_blobs` بثلاثة مراكز حقيقية جيدة الفصل، ودرّب `KMeans` مرتين: مرة بـ `k=2` ومرة بـ `k=4`. احسب `inertia` و`adjusted_rand_score` مع التسميات الحقيقية في الحالتين، وأجب: لماذا لم يخبرك الهدف الداخلي بالاختيار الصحيح؟

التمرين ٨.٣

على ورقة، صنّف هذه المهام الثلاث إلى عائلتها من العائلات الأربع مع ذكر الواجهة المناسبة: (أ) «اكتشف عدد شرائح العملاء الطبيعية في قاعدة عملاء بلا تصنيف»، (ب) «حدد أي مستشعر من ثلاثين مستشعراً يتصرف غريباً اليوم»، (ج) «اختص ٨٠٠ عمود قياسات إلى ٢٠ محوراً تحفظ معظم التباين».

- **ملاحظة**

حل التمرين ٨.١: ثلاث واجهات على بيانات واحدة python

```python
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest
from sklearn.datasets import make_blobs
import numpy as np

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.9, random_state=42)

labels = KMeans(n_clusters=4, n_init=10, random_state=42).fit_predict(X)
X_2d = PCA(n_components=2, random_state=42).fit_transform(X)
iso = IsolationForest(n_estimators=100, random_state=42).fit(X)
strange = iso.score_samples(X)

print(f"(أ) التجميع أجاب: كل نقطة الآن لها تسمية من 0 إلى {labels.max()}")
print(f"(ب) تقليل الأبعاد أجاب: {X.shape[1]} أعمدة صارت {X_2d.shape[1]} إحداثيات")
print(f"(ج) كشف الشذوذ أجاب: أدنى درجة غرابة {strange.min():.4f} عند العينة
{int(np.argmin(strange))}")
```

البيانات ذاتها تلقّت ثلاثة أسئلة مختلفة فأجابت ثلاثة أجواب لا تتداخل: تسميات، وإحداثيات جديدة، ودرجات. هذا هو جوهر الخريطة.

> **ملاحظة**

حل التمرين ٨.٢: خطأ يكشفه k لا ARI inertia python

```python
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score

X3, y3 = make_blobs(n_samples=450, centers=3, cluster_std=0.8, random_state=42)
for k in [2, 4]:
    lab = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X3)
    print(f"k={k}: inertia={KMeans(n_clusters=k, n_init=10,
random_state=42).fit(X3).inertia_:.1f}"
          f" | ARI مع الحقيقة = {adjusted_rand_score(y3, lab):.4f}")
```

النموذج بـ `k=2` دمج عنقودين حقيقيين فهبط التوافق إلى ٠.٥٧٠٣، والنموذج بـ `k=4` جزّأ عنقوداً حقيقياً فبقي التوافق أعلى (٠.٨٦٩٣) — التدمير بالدمج أشد من التدمير بالتجزئة، والثلاثة جميعاً كانت `inertia` تنخفض لديها بانتظام (من ٨٢٠٩.٧ إلى ٤٨٢.٧). لاحظ أن `inertia` لا تميّز إطلاقاً بين التقسيم الصحيح وخاطئه: فهي تقيس «التجمّع الداخلي» فقط ولا تملك أدنى معرفة بالتقسيم الصحيح؛ لذلك نحتاج معياراً خارجياً مثل ARI حين تتوفر تسميات مرجعية للتحقق — وهذه رفاهية نادرة في التطبيقات الحقيقية، ما يجعل مقاييس الفصل ٦٦ الداخلية ضرورة.

> **ملاحظة**

٨.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تسمية العناقيد باسم «حقيقي» مباشرة بعد التدريب	وهم اكتشاف: العنقود ٣ ليس شيئاً محدداً، والأرقام تعيد ترتيبها مع تغيير `random_state`	تحليل متوسطات الميزات لكل عنقود ثم التحقق بمقاييس الفصل ٦٦ قبل التسمية
الاعتماد على `inertia` وحدها لاختيار `k`	القيمة تنخفض دائماً مع رفع `k` وصولاً إلى `k=n` فلا تعطي إجابة	منحنى Elbow مع مقاييس Silhouette الداخلية أو ARI إن وُجدت مرجعية
تطبيق KMeans على بنية مقوسة أو متشابكة	تسميات تكسر البنية الواضحة للعين (ARI منخفض) بينما الهدف الداخلي ممتاز	مضاهاة تعريف الخوارزمية بالبيانات، أو تحويل لخوارزمية كثافة كـ DBSCAN/HDBSCAN
الخلط بين واجهات العائلات	استدعاء `predict` على نموذج لا يملكه، أو توقع تسميات من `fit_transform`	راجع جدول الواجهات في ٨.٣: `fit_predict` / `fit_transform` / `score_samples` لكل عائلة

٨.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التعلّم غير الخاضع للرقابة	تعلّم بيانات بلا تسميات يبحث عن بنية داخلية	Unsupervised learning
التجميع	تقسيم العينات إلى مجموعات متجانسة داخلياً متباعدة بينياً	Clustering
تقليل الأبعاد	إعادة تمثيل البيانات بعدد أعمدة أقل يحفظ أكبر قدر من المعلومة	Dimensionality reduction
كشف الشذوذ	تحديد العينات التي لا تشبه كتلة البيانات الرئيسة	Anomaly detection
تقدير الكثافة	بناء دالة احتمالية تقدر كثافة توليد البيانات عند أي نقطة	Density estimation

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٠ — K-Means وخوارزمية Lloyd

المراكز، و k-means++، و Elbow، و Silhouette، والحدود المفهومية

الوحدة **scikit-learn ١.٩** الحجم **٨٧٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تكتب خطوتي إسناد وتحديث في Lloyd يدوياً وتطابقهما مع ناتج sklearn
- أن تشرح لماذا لا تضمن Lloyd الأمثل الشامل عبر تجربة بذور عشوائية متعددة
- أن تبرر قيمة n_init و k-means++ بأرقام انتشار inertia التي تقيسها بنفسك
- أن تقرأ منحنى Elbow وتعرف ذواته ومتى تلجأ إلى Silhouette
- أن تحدد الحالات التي يفشل فيها KMeans رغم بيانات نظيفة بلا ضجيج

٨.١ خوارزمية Lloyd: خطوتان تصنعان كل شيء

تخيّل أنك تدير شبكة عربات قهوة متنقلة في مدينة، ولديك إحداثيات آلاف الطلبات، وتريد اختيار `k` مواقع تخزين بحيث يكون أقصى ما يمكن من الطلبات قريباً من مخزنه. لا يوجد «الموقع الصحيح» في قاعدة بيانات — المطلوب هو **تقسيم المدينة إلى `k` مناطق تتوسط كل منها نقطة تزويد**. هذا بالضبط ما تفعله KMeans، وبالخوارزمية التي صاغها ستيوارت لويد في مختبرات بل عام ١٩٥٧ (ونُشرت رسمياً عام ١٩٨٢!)، وهي لا تملك سوى خطوتين تتكرران:

$$c(i) \;=\; \arg\min_{j \in \{1,\dots,k\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

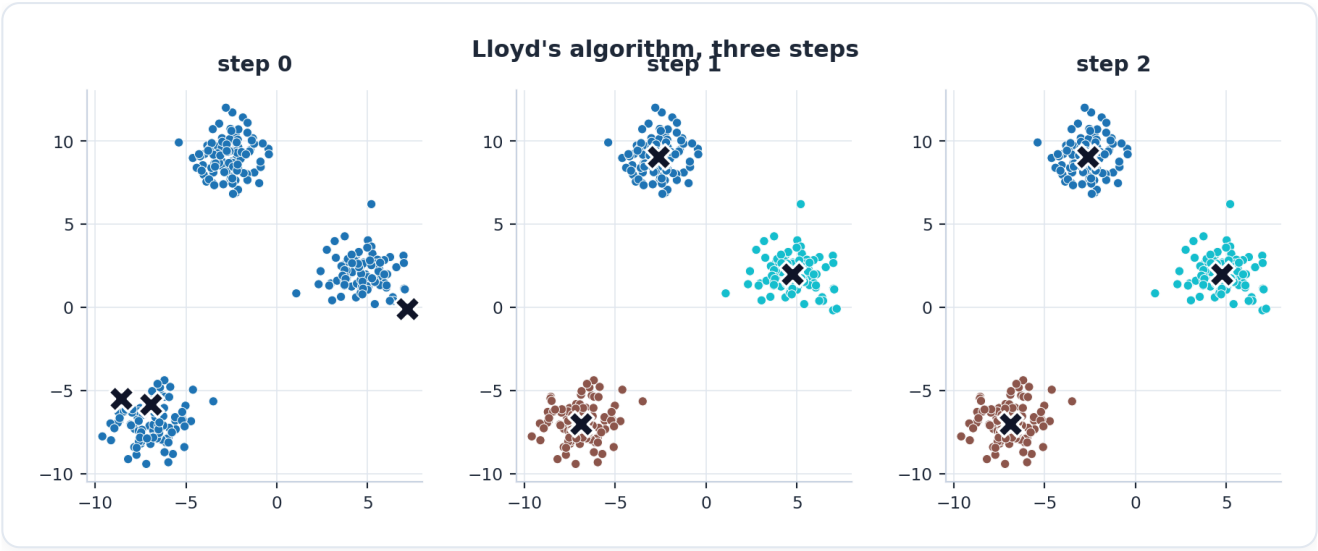
$$\boldsymbol{\mu}_j \;=\; \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i$$

حيث `x_i` العينة رقم `i`، و `mu_j` مركز العنقود `j`، و `C_j` مجموعة النقاط المسندة إليه. تعيد الخوارزمية الخطوتين حتى لا يتغير شيء. لا سحر، لا تدرجات، لا معامل تعلّم — فقط «أقرب مركز» ثم «متوسط الكتلة». لنكتبها يدوياً بالكامل ونقارن نتيجتها بما ينتجه `sklearn.cluster.KMeans`:

يـدويـاً: خطوتـان حتى الاستقـرار Lloyd python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin, adjusted_rand_score
import numpy as np

X, _ = make_blobs(n_samples=450, centers=3, cluster_std=0.8, random_state=42)
k = 3
rng = np.random.default_rng(42)
centers = X[rng.choice(len(X), size=k, replace=False)]   # بذور عشوائية

for it in range(30):
    assign = pairwise_distances_argmin(X, centers)       # 1) إسناد
    new_centers = np.vstack([X[assign == j].mean(axis=0) for j in range(k)])  #
2) تحديث
    shift = np.abs(new_centers - centers).max()
    centers = new_centers
    if shift < 1e-8:
        break

km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
manual_labels = pairwise_distances_argmin(X, centers)
order = pairwise_distances_argmin(centers, km.cluster_centers_)
print(f"تقاربت النسخة اليدوية بعد {it+1} تكراراً (أكبر إزاحة أخيرة {shift:.1e})")
print(f"أقصى فرق بين مراكزنا اليدوية ومراكز sklearn: {np.abs(centers -
km.cluster_centers_[order]).max():.2e}")
print(f"ARI بين تسميات النسخة اليدوية وتسميات sklearn: {adjusted_rand_score(manual_labels,
km.labels_):.4f}")
```

نسختنا اليدوية بلغت المراكز نفسها التي بلغها sklearn بفارق جزيئي (خطأ من رتبة ١٥-١.٧٨e). ولاحظ أن نسبة الاتفاق ١.٠ محسوبة بـ ARI لا بمطابقة حرفية للتسميات — تسميات sklearn عشوائية الأسماء كما تعلمنا في الفصل ٥٩، والأهم أن *البنية* واحدة.



الشكل ١٢٩تكرارات خوارزمية Lloyd: بذور عشوائية ثم إسناد النقاط لأقرب مركز ثم تحديث المراكز بالمتوسط حتى الاستقرار

من مختبرات بل إلى كل لغة برمجة

صاغ ستيوارت لويد الخوارزمية عام ١٩٥٧ لمعالجة الإشارات في مختبرات بل، ولم يُنشر عمله رسمياً إلا عام ١٩٨٢ في مجلة IEEE Transactions on Information Theory — خمسة وعشرون عاماً في الأدراج! وسبقه إدوارد فورجي الذي وصف فكرة مماثلة عام ١٩٥٧ أيضاً. الاسم «k-means» نفسه يعود إلى جيمس ماكوين (١٩٦٧). الخوارزمية الأشهر في التاريخ تعثرت في النشر قبل أن تحتل العالم.

٨.٢ لماذا يتوقف عند أمثلٍ محلي فقط؟

سؤال يحير كل من يقرأ Lloyd لأول مرة: هل تصل الخوارزمية دائماً إلى أفضل تقسيم ممكن؟ الجواب: لا، وهي لا تدّعي ذلك أصلاً. يمكن إثبات أن كل خطوة من الخطوتين لا ترفع الهدف الداخلي `inertia` — مجموع مربعات المسافات بين النقاط ومراكزها الذي قابلناه في الفصل ٥٩:

$$J \;=\; \sum_{i=1}^{n} \| \mathbf{x}_i - \mathbf{\mu}_{\,c(i)} \|^2
\qquad \Rightarrow \qquad J^{(t+1)} \;\leq\; J^{(t)}$$

الهبوط المونوتوني مع حد أدنى يعني التوقف عند **أمثلٍ محلي** (local optimum): وادٍ توقف فيه النزول وليس بالضرورة أعمق وادٍ في التضاريس. والدليل تجريبي لا يحتاج برهاناً معقداً — شغّل الخوارزمية عشر مرات بتوليدات عشوائية مختلفة على البيانات ذاتها:

تجربة: عشر بذور عشوائية، هدفان مختلفان python

```python
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import numpy as np

X, _ = make_blobs(n_samples=600, centers=5, cluster_std=1.1, random_state=7)
results = [KMeans(n_clusters=5, init='random', n_init=1,
random_state=s).fit(X).inertia_ for s in range(10)]
arr = np.array(results)
print(f"أدنى inertia بذور 10 عبر: {arr.min():.1f}")
print(f"أعلى inertia بذور 10 عبر: {arr.max():.1f}")
print(f"الانحراف المعياري: {arr.std():.1f}")
print(f"القيم العشر: {np.round(arr, 1)}")
```

اقرأ القيم العشر جيداً: ليست متقاربة! ست بذور وصلت إلى الحل الجيد (١٣٥٧.٣) وأربع بذور علقت في حواجز أعلى بكثير (نحو ٣٠٧١–٣٠٨٥). البيانات نفسها، والخوارزمية نفسها، والفرق كله في *البذرة الابتدائية*. هذا هو الأمثل المحلي في المجرد: بعض نقاط الانطلاق تسقط الخوارزمية في تقسيم يشطر عنقوداً حقيقياً ويوحّد اثنين.

٨.٣ k-means++: البذرة الذكية

كيف نمنع نقاط الانطلاق السيئة؟ أتت إجابة آرثر وفاسيلفيتسكي (٢٠٠٧) التي صارت الافتراض في sklearn تحت اسم `init='k-means++'`: لا تختر البذور باستقلال تام؛ اختر الأولى عشوائياً، ثم اجعل احتمال اختيار كل نقطة تالية يتناسب مع **مربع مسافتها عن أقرب بذرة موجودة**:

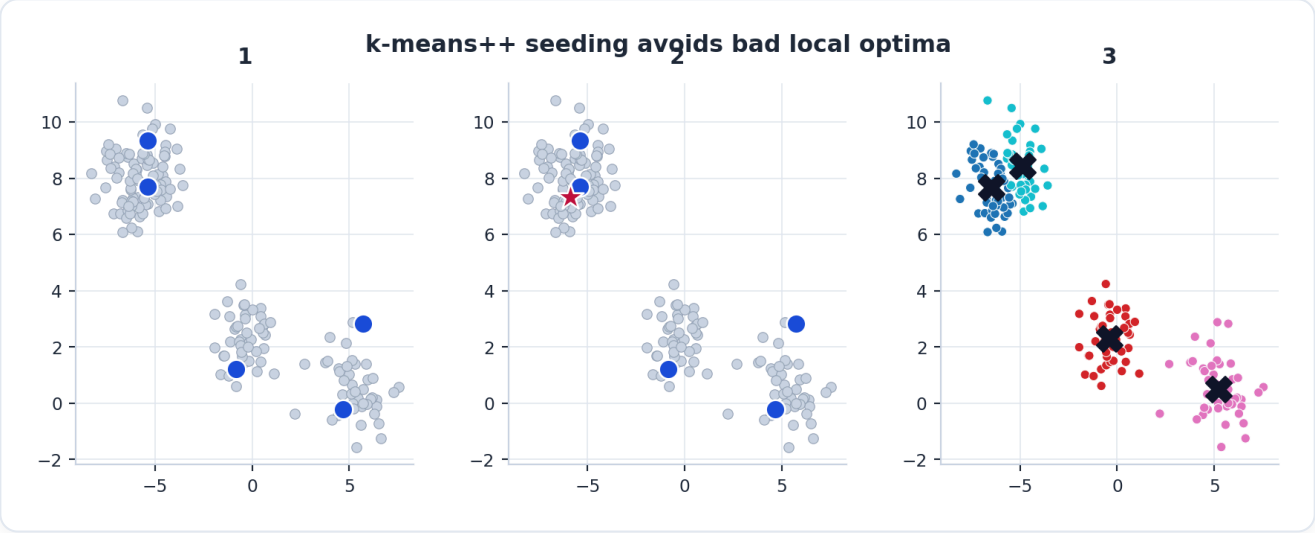
$$p(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{j=1}^{n} D(\mathbf{x}_j)^2}$$

حيث `D(x)` هي المسافة بين النقطة `x` وأقرب بذرة اخترت قبلها. النقاط البعيدة عن البذور الموجودة (أي «مناطق لم تُغطَّ بعد») تحصل على فرصة أكبر، فتتوزع البذور على العناقيد بدل أن تحتشد في واحد. والسؤال الطبيعي: هل الفرق ملموس؟ لنقس انتشار `inertia` عبر عشرين تشغيلاً للطريقتين:

تجربة: عشرون تشغيلاً لكل طريقة بذر python

```python
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import numpy as np

X, _ = make_blobs(n_samples=600, centers=5, cluster_std=1.1, random_state=7)
for init in ['random', 'k-means++']:
    inertias = np.array([KMeans(n_clusters=5, init=init, n_init=1,
random_state=s).fit(X).inertia_ for s in range(20)])
    print(f"init='{init}': أدنى {inertias.min():.1f} | أعلى {inertias.max():.1f} |
انحراف {inertias.std():.1f}")
```

النتيجة صادمة في وضوحها: عشرون تشغيلاً ببذور عشوائية أعطت حلولاً من ١٣٥٧.٣ إلى ١١٨٤٨.١ بانحراف معياري ٣١٧٥.٦، بينما أعطت k-means++ الحل الجيد نفسه في العشرين تشغيلاً كلها بانحراف صفر. هذه هي قيمة البذرة الذكية: لا تلغي الأمثلة المحلية نظرياً، لكنها تجعل الوقوع فيها نادراً عملياً. ولطبقة أمان إضافية يأتي بارامتر `n_init`: شغّل التدريب كاملاً عدة مرات واحتفظ بأفضل نتيجة. في sklearn ١.٩ القيمة الافتراضية `n_init='auto'` أي تشغيل واحد مع k-means++ وعشرة مع `init='random'`.



الشكل ١٣٠ كيف توزّع k-means++ البذور الأولية على العناقيد البعيدة بدل حشرها في منطقة واحدة كثيفة

خيار `init`	ماذا يفعل	متى تختاره
`'k-means++'` **(الافتراضي)**	بذور متباعدة باحتمال يتناسب مع مربع المسافة	الخيار الافتراضي في كل الحالات العملية
`'random'`	بذور عينات عشوائية مستقلة	للمقارنة والتعليم، أو للتحقق من حساسية البذور
مصفوفة مراكز جاهزة	يبدأ من مراكز تعرفها مسبقاً	عند امتلاك تقدير مبدئي (مراكز الموسم الماضي مثلاً) — ويشترط `n_init=1`

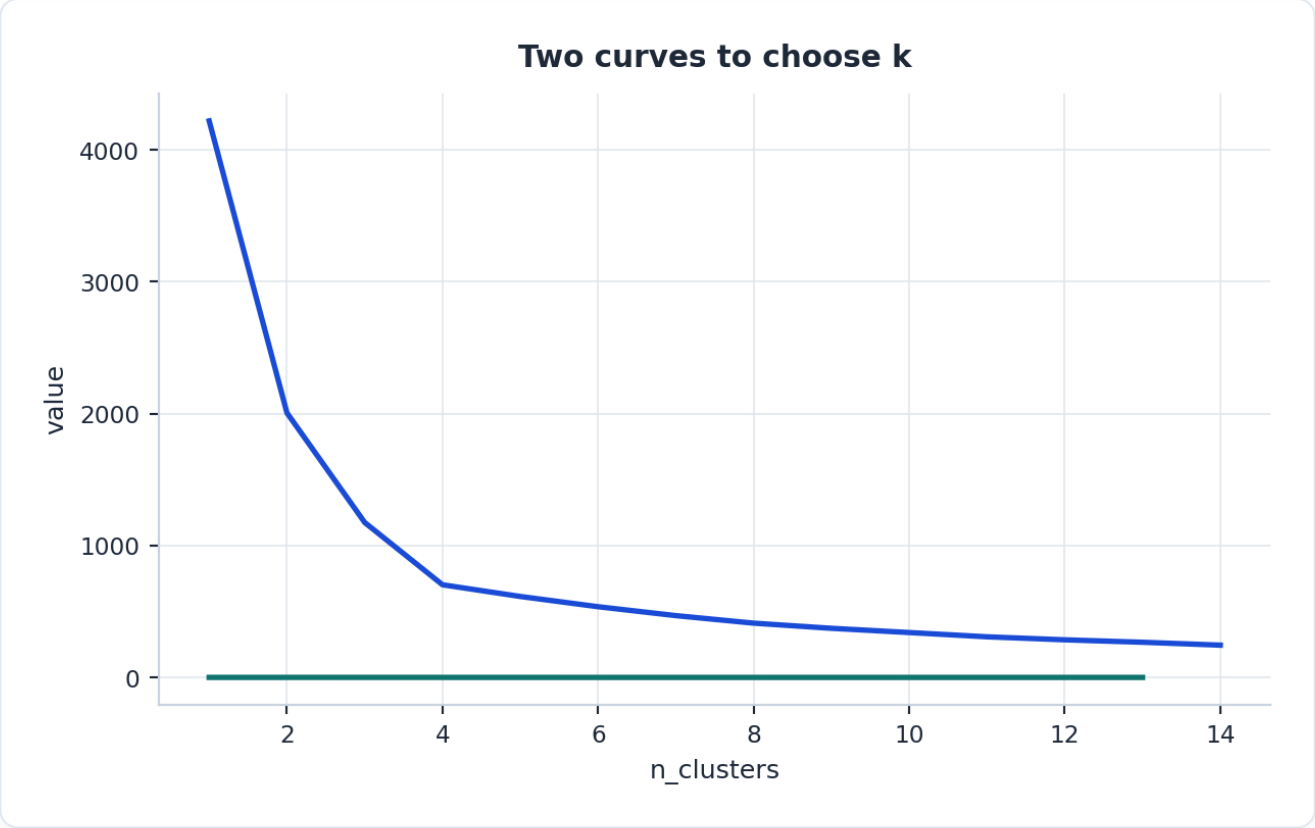
> **لا تلمس الافتراضي، لكن افهم `n_init`**
>
> مع k-means++ فإن رفع `n_init` إلى ١٠ أو ٢٠ يقلّص فرصة الوقوع في أمثل محلي إلى ما دون الاهتمام مقابل زمن تدريب مضاعف. على بيانات ضخمة، جرّب أولاً على عيّنة فرعية عشوائية: إن استقرت `inertia` عبر `n_init` مختلفة فعلى الثقة بالافتراضي.

٨.٤ Elbow و Silhouette: أين أضع k؟

بقيت المسألة الأشهر: من أين تأتي `k` أصلاً؟ KMeans لا تقدّرها — أنت تُلزمها بها. الأداة التقليدية منحنى **الكوع (Elbow)**: نرسم `inertia` مقابل قيم `k` المتزايدة ونبحث عن نقطة «كسر» يتباطأ عندها الهبوط:

جدول Elbow: inertia و Silhouette لكل k — python

```python
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=600, centers=4, cluster_std=0.9, random_state=42)
print(" k |   inertia    | silhouette")
for k in range(2, 10):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_)
    print(f"{k:2d} | {km.inertia_:10.1f} | {sil:.4f}")
```



الشكل ١٣١منحنى inertia مقابل k: هبوط حاد ثم تباطؤ عند نقطة الكوع المرشحة

في الشكل تلاحظ الهبوط المونوتوني القاسي: من ١٨٤٣٨.٥ عند `k=2` إلى ٥٥٣.٦ عند `k=9` — وهبوطٌ دائماً مهما رفعت `k`، لأن مزيداً من المراكز لا يمكن إلا أن يقصّر المسافات (وصولاً إلى `k=n` حيث تصير `inertia=0` وكل نقطة مركز لنفسها!). الكوع المرشح عند `k=4` حيث يتباطأ الهبوط — وهي فعلاً قيمة المراكز الحقيقية في هذه البيانات. لكن انتبه إلى **ذاتية المنحنى**: في بيانات حقيقية بلا فصل نظيف، يكون «الكسر» ناعماً ويقرأه شخصان بشكلين. لذلك ندعم القرار بمقياس داخلي كمي مثل `silhouette_score` من `sklearn.metrics`، الذي يبلغ هنا ذروته ٠.٨٠٩٣ عند `k=4` بالضبط — وسيتكفل الفصل ٦٦ بتفسيره وحسابه على الوجهين العيني والعددي، مع بدائل مثل Davies-Bouldin و Calinski-Harabasz.

٨.٥ سمات النموذج والتنبؤ للنقاط الجديدة

بعد التدريب، يحمل كائن `KMeans` كل بنية القرار داخل سماته الأربع المفصلية:

السمة	النوع	المعنى
`cluster_centers_`	مصفوفة `k×d`	إحداثيات المراكز النهائية
`labels_`	متجه بطول `n`	تسمية عنقود كل عينة تدريبية
`inertia_`	عدد حقيقي	قيمة الهدف النهائية (مجموع مربعات المسافات)
`n_iter_`	عدد صحيح	عدد تكرارات Lloyd حتى الاستقرار

والأهم من ذلك أن `predict` تعمل على نقاط لم ترها الخوارزمية قط. ماذا تعني «تسمية» نقطة جديدة؟ أن المراكز تقسّم الفضاء كله إلى **خلايا فورونوي (Voronoi cells)**: كل نقطة في الفضاء تنتمي لخلية أقرب مركز، فالنموذج المتعلَّم ليس قائمة نقاط بل *تجزئة للفضاء كله*:

python — سمات KMeans وتنبؤ نقاط جديدة عبر خلايا فورونوي

```python
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import euclidean_distances
import numpy as np

X, _ = make_blobs(n_samples=400, centers=3, cluster_std=0.8, random_state=42)
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
print(f"cluster_centers_: {km.cluster_centers_.shape}")
print(f"labels_ 10 أول: {km.labels_[:10]}")
print(f"inertia_: {km.inertia_:.2f}")
print(f"n_iter_: {km.n_iter_}")
X_new = np.array([[0.0, 4.0], [-6.0, -2.0], [7.0, 0.0]])
pred = km.predict(X_new)
nearest = euclidean_distances(X_new, km.cluster_centers_).argmin(axis=1)
print(f"predict للنقاط الجديدة: {pred}")
print(f"أقرب مركز حسابياً:      {nearest}")
print(f"predict يطابق أقرب مركز: {(pred == nearest).all()}")
```

تطابق `predict` مع «أقرب مركز» ليس تفصيلاً؛ إنه تعريف النموذج المُتعلَّم كله. يمكنك أن تستبدل KMeans بأربع خطوط من numpy: احفظ `cluster_centers_` وحسب الأقرب بنفسك — ستملك النموذج كاملاً. هذه البساطة هي سر انتشار KMeans، وهي أيضاً سبب حدوده التالية.

> **النموذج المتعلَّم هو خلايا فورونوي**
>
> كل ما تعلّمته KMeans يتلخص في `k` نقطة (المراكز). التسميات، والتنبؤ للنقاط الجديدة، وحدود القرار — كلها نتيجة حسابية لسؤال واحد: ما أقرب مركز؟ حين تحفظ المراكز فقد حفظت النموذج بأكمله، وحين تحرك مركزاً يدوياً فأنت تعدّل التجزئة كلها.

٨.٦ الحدود القاتلة الأربع

لماذا يفشل KMeans في بيانات نظيفة بلا أي ضجيج؟ لأن تعريفه للبنية ضيّق جداً. أربع حدود تختصر المشكلة:

لنثبت الحدّين الثاني والثالث تجريبياً على بيانات نظيفة:

تجربتان: أحجام غير متساوية وميزة غير مطبَّعة python

```python
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
import numpy as np

# (أ) عنقود ضخم بجوار عنقودين صغيرين متقاربين
parts = []
for i, (loc, n_i) in enumerate([([0, 0], 600), ([1.8, 1.8], 60), ([2.6, -0.6],
40)]):
    p = np.random.default_rng(42 + i).normal(loc=loc, scale=0.7, size=(n_i, 2))
    parts.append(p)
X_big = np.vstack(parts)
y_big = np.concatenate([np.zeros(600, dtype=int), np.ones(60, dtype=int),
np.full(40, 2)])
lab = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(X_big)
print(f"600/60/40 أحجام حقيقية — ARI: {adjusted_rand_score(y_big, lab):.4f}")
print(f"أحجام العناقيد التي وجدها KMeans: {np.bincount(lab, minlength=3)}")

# (ب) ميزة بمقياس ألف ضعف
X, y = make_blobs(n_samples=600, centers=3, cluster_std=0.9, random_state=4)
X_scaled = X.copy()
X_scaled[:, 0] *= 1000.0
ari_raw = adjusted_rand_score(y, KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(X_scaled))
X_fixed = StandardScaler().fit_transform(X_scaled)
ari_fixed = adjusted_rand_score(y, KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(X_fixed))
print(f"ARI على البيانات الأصلية:   {adjusted_rand_score(y, KMeans(n_clusters=3,
n_init=10, random_state=42).fit_predict(X)):.4f}")
print(f"ARI بعد تضخيم ميزة 1000×:  {ari_raw:.4f}")
print(f"ARI بعد StandardScaler:    {ari_fixed:.4f}")
```

في التجربة الأولى أراد KMeans أن يجد ثلاث كتل متساوية تقريباً فشطر العنقود الضخم (٦٠٠ عينة) إلى كتلتين ودمج الصغيرين في كتلة واحدة (أحجام ٢٧٣/٣٤٣/٨٤ مقابل ٦٠٠/٦٠/٤٠ الحقيقية، وتوافق ٠.٢٥١٤). وفي الثانية اختفى تقسيم سليم كان يعمل بدقة ٠.٩٥١١ بمجرد تضخيم ميزة واحدة ألف مرة (هبوط التوافق إلى ٠.٤٩٢٢)، وأعاد `sklearn.preprocessing.StandardScaler` الأمور إلى ما كانت عليه تماماً (٠.٩٥١١). القاعدة العملية: طبّع دائماً قبل KMeans، ولاحظ أن المراكز الناتجة ستكون في الفضاء المطبّع — ارجعها بالتحويل العكسي `inverse_transform` حين تحتاجها بوحداتها الأصلية.

⛔ KMeans يحسب دائماً حتى لو لم يوجد شيء

KMeans لن تقول لك «لا توجد عناقيد هنا». أعطها ضجيجاً موحداً بلا أي بنية وستعيد لك `k` عنقوداً واثقة بنفسها. الحكم على وجود بنية أصلاً مسؤوليتك: افحص توزيع Silhouette والفجوة بين الكتل قبل أن تبني قصة على النتيجة — وإن كان التوزيع رمادياً كله فالبنية وهم.

🏭 أين تعيش KMeans في الإنتاج الحقيقي؟

رغم كل هذه الحدود، KMeans من أكثر خوارزميات التعلم الآلي انتشاراً في الأنظمة الحية: تقسيم المستخدمين للاختبارات التسويقية، وضغط الألوان في الصور (كل بكسل يُسند لأقرب لون من `k` ألوان ممثلة بالمراكز)، وبناء «مراكز رمزية» لتسريع البحث في المتجهات، وحتى كخطوة تهيئة لنماذج أكثر تعقيداً كالنماذج المزيجية في الفصل ٦٧. سرّ النجاح: بلا منازع في السرعة على ملايين الصفوف، وبنتيجة تفهمها الإدارة في ثلاث دقائق.

تمارين الفصل الستين

- **ملاحظة**

التمرين ٨.١

على بيانات الزهور `load_iris` (الأبعاد الأربعة كلها)، درّب `KMeans` بثلاث مراكز، واطبع السمات الأربع: `cluster_centers_` و `labels_` و `inertia_` و `n_iter_`، ثم قس التوافق مع أنواع الزهور الحقيقية عبر `adjusted_rand_score`.

التمرين ٨.٢

افحص منحنى الكوع على بيانات Iris أيضاً: احسب `inertia` و `silhouette_score` و ARI مع الأنواع الحقيقية لكل `k` من ٢ إلى ٦، وأجب: هل يوافق ذروة Silhouette العدد الحقيقي للأنواع؟ وهل يوافق كوع منحنى inertia؟

التمرين ٨.٣

اشرح بكلماتك لماذا لا يضمن `n_init=100` الوصول إلى الأمثل الشامل، وما العلاقة بين حجم البيانات `n` وكلفة كل تشغيل، ولماذا يصبح تكرار التشغيل مكلفاً على ملايين الصفوف؟ (استعن بتعقيد تكرار واحد O(n·k·d) لكل تكرار).

- **ملاحظة**

حل التمرين ٨.١: KMeans على Iris مع سماته — python

```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

X, y = load_iris(return_X_y=True)
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
print(f"cluster_centers_: {km.cluster_centers_.shape}")
print(f"labels_ 8 أول: {km.labels_[:8]}")
print(f"inertia_: {km.inertia_:.2f}")
print(f"n_iter_: {km.n_iter_}")
print(f"ARI مع الأنواع الحقيقية: {adjusted_rand_score(y, km.labels_):.4f}")
```

توافق معتدل (ARI نحو ٠.٧٣) ليس أكثر: مجموعة «الورود» (versicolor و virginica) تتداخل جزئياً في فضاء الأبعاد الأربعة، وKMeansg يفصل بحدود مستقيمة فقط. هذا توضيح عملي ممتاز أن البنية الظاهرة للعين في مسقط ثنائي قد تختفي في فضاء كامل الأبعاد — والعكس صحيح.

> **ملاحظة**

حل التمرين ٨.٢: فحص الكوع على Iris python

```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

X, y = load_iris(return_X_y=True)
print(" k |   inertia   | silhouette |  ARI")
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_)
    ari = adjusted_rand_score(y, km.labels_)
    print(f"{k:2d} | {km.inertia_:10.1f} | {sil:10.4f} | {ari:.4f}")
```

اقرأ الجدول بثلاثة أعين: `inertia` تهبط دائماً بلا إجابة، وذروة Silhouette عند `k=2` (لأن الأنواع تتوزع في كتلتين عظيمتين تفصلهما حد واضح)، بينما ARI الأعلى عند `k=3` — العدد الحقيقي. الدرس: كل معيار داخلي يرشّح مرشحاً وفق تعريفه، ولا يوجد معيار يعرف «الحقيقة»؛ لذا اجمع بين منحنى الكوع والمقاييس الداخلية ومعرفتك بالمجال قبل تثبيت `k` — وهو جوهر الفصل ٦٦.

> **ملاحظة**

٨.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الاعتماد على تشغيل واحد بـ `init='random'` و `n_init=1`	حل مختلف كل مرة وقد يكون أسوأ بكثير (فرق inertia بأضعاف)	الإبقاء على `init='k-means++'` الافتراضي أو رفع `n_init`
تغذية KMeans بميزات بمقاييس مختلفة	الميزة الأضخم تحتكر المسافة وتمحو بنية الميزات الأخرى	`sklearn.preprocessing.StandardScaler` دائماً قبل التجميع
البحث عن «كوع» حاد في بيانات حقيقية	المنحنى ناعم ويُقرأ بشكلين مختلفين فيقرر كل محلل ما يشاء	دعم القرار بـ Silhouette ومقاييس الفصل ٦٦ ومعرفة المجال
مقارنة تسميات تشغيلين بالمطابقة الحرفية	تسميتان متطابقتان بالبنية تبدوان متناقضتين لأن الأسماء عشوائية	`adjusted_rand_score` أو مقاييس ثابتة ضد تبديل الأسماء

٨.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
خوارزمية لويد	تكرار خطوتي الإسناد والتحديث حتى استقرار المراكز	Lloyd's algorithm
الهدف الداخلي	مجموع مربعات المسافات بين النقاط ومراكز عناقيدها	Inertia / within-cluster SSE
البذرة الذكية	تهيئة متباعدة باحتمال يتناسب مع مربع المسافة	k-means++ initialization
منحنى الكوع	رسم inertia مقابل k واختيار نقطة تباطؤ الهبوط	Elbow method
خلايا فورونوي	تجزئة الفضاء إلى مناطق أقرب مركز	Voronoi cells

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦١ — نسخ K-Means العملية

get_bin_seeds g BisectingKMeans g MiniBatchKMeans

الوحدة **scikit-learn ١.٩** الحجم **٨٦٥** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح تحديث المتوسط المتحرك في MiniBatchKMeans وتقدير أثر batch_size على الزمن والجودة
- أن تُجري قياساً عادلاً للسرعة والجودة بين KMeans g MiniBatchKMeans على مئات الآلاف
- أن تبني تجميعاً متصلاً على تدفق بيانات عبر partial_fit
- أن تتحكم في التقسيم تدريجياً مع BisectingKMeans وتميّز استراتيجيتي bisecting_strategy
- أن تختار النسخة الصحيحة من جدول قرار يعتمد على n و d و k

٨.١ MiniBatchKMeans: لويد على دفعات صغيرة

نفس الفكرة، مقايضات مختلفة

النسختان الكبيرتان في هذا الفصل ليستا خوارزميتين جديدتين بقدر ما هما مقايضات هندسية على فكرة KMeans ذاتها: `MiniBatchKMeans` يبيع جزءاً صغيراً من دقة التحديث مقابل سرعة تُقاس بأضعاف وذاكرة شبه ثابتة، و `BisectingKMeans` يبيع «الإجابة دفعة واحدة» مقابل مسار تدريجي يحفظ البنية الكبرى عند كل خطوة. من يفهم المقايضة يختار النسخة صح؛ ومن يستعملها كصندوق أسود يظن أن التلقائية مجانية.

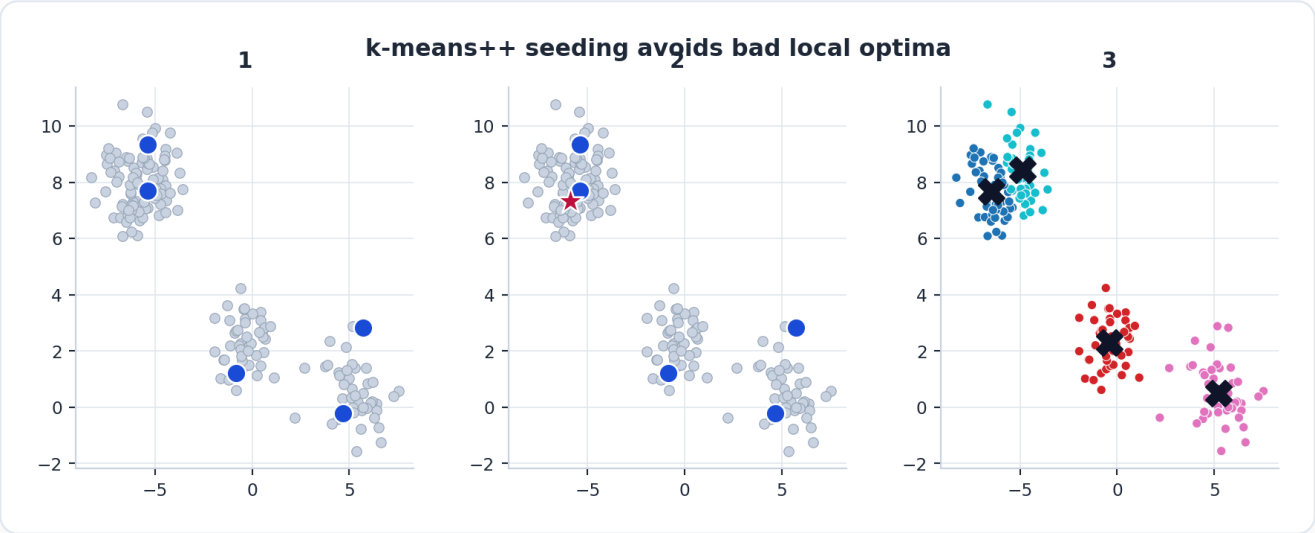
ماذا لو استحال تدريب KMeans لأن البيانات لا تتسع للحساب عليها تكراراً كاملاً؟ فكرة `sklearn.cluster.MiniBatchKMeans` بسيطة وعبقرية: بدل إعادة حساب الإسناد لكل العينات `n` في كل تكرار، اسحب **دفعة صغيرة** من العينات (افتراضياً `batch_size=1024`)، أسند نقطتها لأقرب مركز، ثم حرّك كل مركز قليلاً باتجاه متوسط نقطاته في تلك الدفعة — **متوسط متحرك** يعطي وزناً أقل لكل نقطة جديدة كلما كبرت خبرة المركز:

$$\boldsymbol{\mu}_c \leftarrow \boldsymbol{\mu}_c + \eta\,(\mathbf{x} - \boldsymbol{\mu}_c) \qquad \eta = \frac{1}{s_c}$$

حيث `mu_c` هو المركز، و `x` النقطة المسندة إليه في الدفعة، و `s_c` العدد التراكمي للنقاط التي أُسندت لهذا المركز منذ البدء (كلما كبر `s_c` صار المركز أكثر ثباتاً وأقل تذبذباً). لكل مركز في الدفعة تحديداً بمقدار `m_c` نقطة، يمكن كتابة التحديث دفعة واحدة:

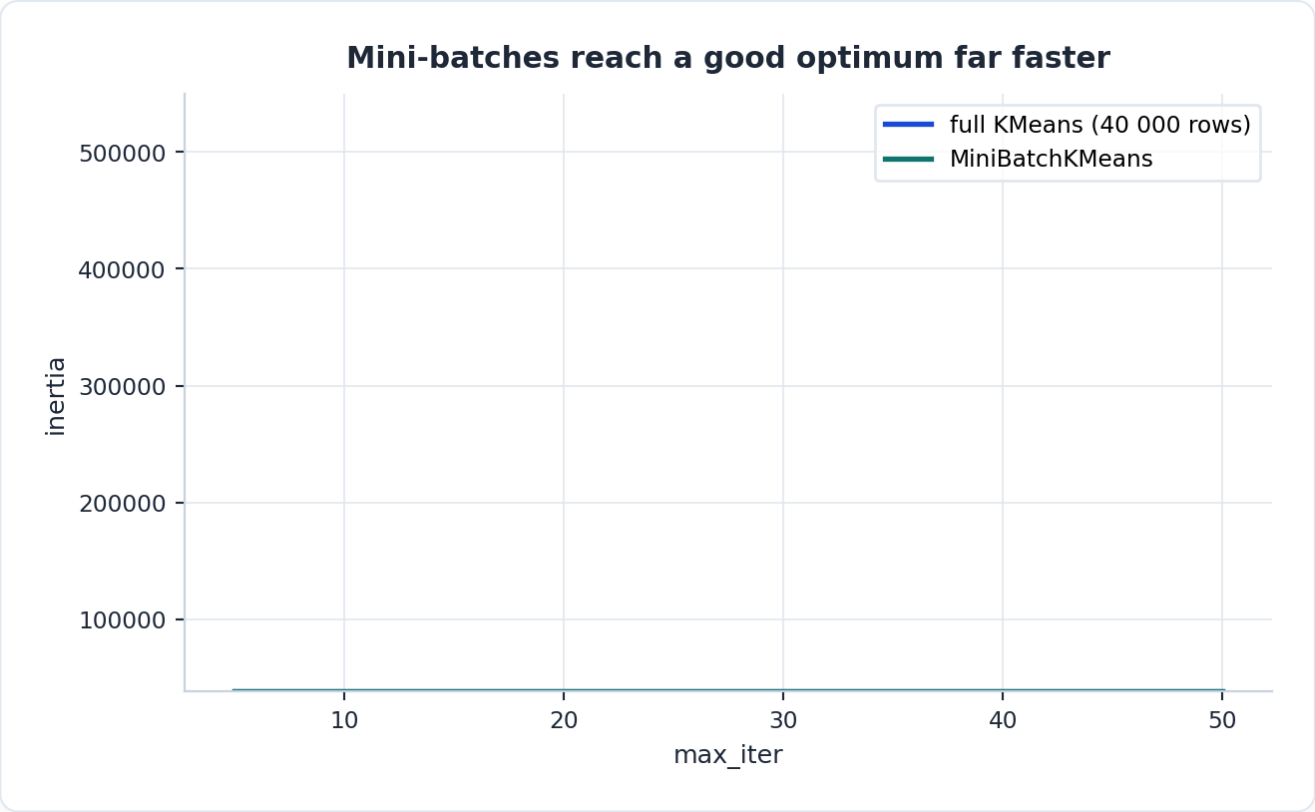
$$\boldsymbol{\mu}_c \leftarrow \left(1 - \frac{m_c}{s_c}\right)\boldsymbol{\mu}_c + \frac{m_c}{s_c}\,\bar{\mathbf{x}}_c$$

حيث `x̄_c` متوسط نقاط الدفعة المسندة للمركز `c`. حتى التهيئة صارت رخيصة: لا يُطبَّق k-means++ (الذي شرحناه في الفصل ٦٠ بالشكل التالي) على كل البيانات، بل على عينة فرعية حجمها `init_size` (افتراضياً ثلاثة أضعاف حجم الدفعة):



الشكل ١٣٢كيف توزّع k-means++ البذور الأولية — نفس الآلية التي يعيد MiniBatchKMeans استعمالها على عينة فرعية

وهكذا تبقى الفكرة نفسها (مراكز تُحسَّن بالتناوب) لكن كلفة كل تحديث تصير ثابتة تقريباً لا تنمو مع `n`. والبارامترات العملية المهمة إلى جانب `batch_size`: `max_no_improvement` (توقف مبكر إذا لم تتحسن `inertia` في عدة دفعات متتالية)، و`reassignment_ratio` (إعادة تفعيل مراكز مهملة)، و`compute_labels` (حساب تسميات كل البيانات بعد التدريب).



الشكل ١٣٣تدفق دفعات صغيرة تحدّث المراكز تدريجياً بدل مرور كامل على البيانات في كل تكرار

٨.٢ التجربة الكبرى: ٢٠٠ ألف عينة

المقايضة المعلنة: تسريع هائل مقابل انحراف بسيط في الجودة. لنجسدها بأرقام حقيقية على سحابة ٢٠٠,٠٠٠ عينة في ١٦ بُعداً بـ ١٦ عنقوداً حقيقياً:

قياس الزمن والجودة: KMeans مقابل MiniBatchKMeans python

```python
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
import numpy as np, time

X, y = make_blobs(n_samples=200_000, centers=16, n_features=16,
                  cluster_std=0.8, center_box=(-40, 40), random_state=42)

t0 = time.perf_counter()
km = KMeans(n_clusters=16, n_init=3, random_state=42).fit(X)
t_km = time.perf_counter() - t0

t0 = time.perf_counter()
mb = MiniBatchKMeans(n_clusters=16, batch_size=1024, n_init=3,
random_state=42).fit(X)
t_mb = time.perf_counter() - t0

print(f"KMeans      | {t_km:6.2f}s | inertia={km.inertia_:12.1f} | ARI مع الحقيقة =
{adjusted_rand_score(y, km.labels_):.4f}")
print(f"MiniBatch   | {t_mb:6.2f}s | inertia={mb.inertia_:12.1f} | ARI مع الحقيقة =
{adjusted_rand_score(y, mb.labels_):.4f}")
print(f"ARI بين تسميتي النسختين: {adjusted_rand_score(km.labels_, mb.labels_):.4f}")
print(f"تسريع تقريبي: {t_km / t_mb:.1f}x")
```

في بيانات جيدة الفصل كهذه، صارت الجودة شبه متطابقة بين النسختين (ARI ممتاز لدى كلتيهما، وتوافق تام بين تسميتيهما) بينما يقفز الزمن بأضعاف كبيرة لصالح الدفعات الصغيرة — اقرأ نسبة التسريع في ناتج التجربة، فهي تتفاوت بحدود الجهاز لكنها تبقى في حدود عشرة أضعاف فأكثر في قياساتنا. أين تظهر إذن «كلفة الجودة»؟ في بيانات متداخلة وفي اختيار دفعات أصغر من اللازم — وهو ما نختبره في القسم التالي.

لا تقارن `labels_` بين نسختين بالمطابقة

أخطر خطأ عند تقييم MiniBatchKMeans مقابل KMeans: حساب نسبة تطابق `km.labels_ == mb.labels_` عنصراً بعنصر. النتيجة ستبدو كارثية حتى لو كان التقسيمان مثاليين — لأن تسمية العنقود · في النسخة الأولى قد تعني العنقود ٥ في الثانية؛ الأسماء إسناد اعتباطي كما تعلمنا في الفصلين ٥٩ و٦٠. المقياس الصحيح: `adjusted_rand_score` من `sklearn.metrics`، فهو ثابت ضد تبديل الأسماء وسيشرحه الفصل ٦٦ تفصيلاً. (استعملناه أعلاه في السطر الأخير من التجربة).

٨.٣ مقياس الدفعة: الأصغر ليس الأسرع دائماً

الحدس الشائع: دفعة أصغر = عمل أقل = أسرع. الحقيقة أدق: الدفعة الأصغر تلمس عيناً أقل من البيانات لكنها تضاعف عدد مرات دوران الحلقة (عدد الدفعات في المرور الواحد = `n / batch_size`)، وما ينقص من حساب المسافات ينفق في التكاليف الثابتة. ولأن تحديثات الدفعات الصغيرة أقسح وأكثر ضجيجاً، تتأثر الجودة في البيانات المتداخلة:

أثر batch_size على الزمن والجودة (بيانات متداخلة) python

```python
from sklearn.cluster import MiniBatchKMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
import numpy as np, time

# بيانات متداخلة عمداً: كتافات متلاصقة في فضاء 8 أبعاد
Xs, ys = make_blobs(n_samples=100_000, centers=12, n_features=8,
                    cluster_std=2.0, center_box=(-12, 12), random_state=42)
for bs in [256, 1024, 4096]:
    t0 = time.perf_counter()
    m = MiniBatchKMeans(n_clusters=12, batch_size=bs, n_init=3,
random_state=42).fit(Xs)
    t_mb = time.perf_counter() - t0
    print(f"batch_size={bs:5d} | {t_mb:.2f}s | inertia={m.inertia_:.0f} | ARI=
{adjusted_rand_score(ys, m.labels_):.4f}")
```

اقرأ الجدول بعينين: `inertia` تحسّن باطراد كلما كبُرت الدفعة (من ٣٢١٢١٣٨ عند ٢٥٦ إلى ٣٢٠٣٢١٠ عند ٤٠٩٦)، لأن كل تحديث يحمل مزيداً من المعلومات — وهذه نسب ثابتة لأننا ثبّتنا البذرة. أما الزمن فهو الأقل ثباتاً: الدفعة الأصغر تلمس عينات أقل لكنها تضاعف عدد دورات الحلقة (`n / batch_size` دفعة في المرور)، فما ينقص من حساب المسافات ينفق في التكاليف الثابتة، وتتداخل مع ذلك أحمال الجهاز — لذلك قِس الزمن بعدة تشغيلات ولا تحكم من تشغيل واحد. القاعدة العملية: ابدأ بالافتراضي ١٠٢٤، وجرّب ٤٠٩٦ إن كانت الجودة هي قلقك، ولا تنزل تحت ذلك إلا إذا كانت الذاكرة عنق الزجاجة.

قاعدة اليد الذهبية لأحجام الدفعات

الدفعات الصغيرة جداً (أقل من ٢٥٦) تضاعف التكاليف الثابتة وتشوش التحديثات، والدفعات الضخمة (فوق ٦٥٥٣٦) تفقد فكرة الدفعات أصلاً وتقترب من KMeans العادية زمنياً. النطاق ١٠٢٤–٨١٩٢ هو المعتاد عملياً، وابدأ بالافتراضي قبل أي تعديل.

٨.٤ partial_fit: التجميع على التدفق المستمر

ماذا لو لم تكن بياناتك ملفاً ثابتاً أصلاً بل تدفقاً: أحداث مستشعرات، نقرات مستخدمين، معاملات لحظية؟ `MiniBatchKMeans` يوفر واجهة `partial_fit` التي تأخذ دفعة واحدة وتحدّث المراكز ثم تُنسى الدفعة — لا حاجة أبداً لحفظ التاريخ كله:

تدفق مستمر: 25 دفعة عبر partial_fit python

```python
from sklearn.cluster import MiniBatchKMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import pairwise_distances_argmin, adjusted_rand_score
import numpy as np

X_stream, _ = make_blobs(n_samples=50_000, centers=6, n_features=8,
                         cluster_std=0.9, center_box=(-20, 20), random_state=42)

streamer = MiniBatchKMeans(n_clusters=6, batch_size=2048, random_state=42)
for i, chunk in enumerate(np.array_split(X_stream, 25)):
    streamer.partial_fit(chunk)          # الدفعة تدخل وتُنسى

direct = MiniBatchKMeans(n_clusters=6, batch_size=2048, n_init=3,
random_state=42).fit(X_stream)
lab_stream = pairwise_distances_argmin(X_stream, streamer.cluster_centers_)
diff = np.abs(np.sort(streamer.cluster_centers_, axis=0) -
np.sort(direct.cluster_centers_, axis=0)).max()
print(f"عولجت 25 دفعة متتالية، الوصول النهائي للمراكز:")
print(f"أقصى فرق بين مراكز التدفق ومراكز التدريب الدفعي: {diff:.4f}")
print(f"ARI بين تسميات التدفق وتسميات التدريب الدفعي: {adjusted_rand_score(direct.labels_,
lab_stream):.4f}")
```

مراكز التدفق المتقطع بلغت عملياً مراكز التدريب الدفعي الكامل (فرق ضئيل تقرأه في الناتج، وتوافق تام)، وهذا يعني أنك تستطيع تشغيل خدمة تجميع حية تتحدث مراكزها مع كل وصول بيانات جديدة بلا إعادة تدريب من الصفر. انتبه لكن إلى نقطتين: `partial_fit` لا يعيد حساب `inertia_` على كامل البيانات (لا يملك التاريخ)، ولا يوجد حلقة تكرار خارجية — كل استدعاء دفعة واحدة.

٨.٥ BisectingKMeans: التقسيم الثنائي من الأعلى

KMeans و MiniBatchKMeans تعملان **من الأسفل صعوداً** بمعنى عملي: تحصل على `k` عناقيد دفعة واحدة بلا ترتيب. `sklearn.cluster.BisectingKMeans` يسير بالاتجاه المعاكس — **تجزئة ثنائية تدريجية (Divisive)**: ابدأ بكل البيانات عنقوداً واحداً، ثم اختر العنقود «الأثقل» واقسمه إلى نصفين بـ KMeans ثنائي، وكرر حتى تبلغ `k` . العنقود الأثقل تحت الاستراتيجية الافتراضية `'biggest_inertia'` هو صاحب أكبر مجموع مربعات مسافات:

$$C^* = \arg\max_{C \in \mathcal{C}} \sum_{\mathbf{x}_i \in C} \|\mathbf{x}_i - \boldsymbol{\mu}_C\|^2$$

حيث `C` عنقود من مجموعة العناقيد الحالية `\mathcal{C}` المرشحة للانقسام. والاستراتيجية الثانية `'largest_cluster'` تختار العنقود صاحب أكبر عدد نقاط بغض النظر عن تشتته. الفرق بينهما ليس ترفاً أكاديمياً — انظر ماذا يحدث لعنقودين: واحد واسع قليل النقاط وواحد ضيق كثير النقاط:

```python
# استراتيجيتا الانقسام: التشتت الأكبر أم أكبر عدد نقاط؟
from sklearn.cluster import BisectingKMeans
import numpy as np

rng = np.random.default_rng(42)
A = rng.normal(loc=[-14, 0], scale=4.0, size=(120, 2))    # واسع قليل النقاط
B = rng.normal(loc=[14, 0], scale=0.6, size=(1400, 2))    # ضيق كثير النقاط
X_bs = np.vstack([A, B])

for strat in ['biggest_inertia', 'largest_cluster']:
    bk = BisectingKMeans(n_clusters=3, bisecting_strategy=strat,
random_state=42, n_init=5).fit(X_bs)
    print(f"{strat:16s} (k=3): أحجام العناقيد = {np.bincount(bk.labels_,
minlength=3)}")
```

بأخذ `k=3` بعد الانقسام الجذري الأول (الذي فصل A عن B تلقائياً)، اخترت `'biggest_inertia'` شطر العنقود الواسع A (أحجام ٥٦ و٦٤) وبقي B كاملاً، بينما اخترت `'largest_cluster'` شطر العنقود الضخم B (أحجام ٧٢٥ و٦٧٥) وبقي A كاملاً. اختيارك بينهما يعتمد على سؤالك: هل تريد تفكيك أكبر مصدر تشتت (الأولى)، أم موازنة أحجام الكتل (الثانية)؟

والعائد الأهم من هذا البناء التدريجي: **شجرة هرمية خفيفة** تتحكم بعدد الفئات تدريجياً. على بيانات هرمية (مجموعتان كبيرتان تحوي كل منهما عنقودين فرعيين):

```python
# التدرج: k=2 يعطيك الكبار، و k=4 يعطيك الصغار
from sklearn.cluster import BisectingKMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score

X_h, y_sub = make_blobs(n_samples=1600, centers=[[0, 0], [22, 0], [0, 14], [22, 14]],
                        cluster_std=1.2, random_state=42)
y_macro = (X_h[:, 0] > 11).astype(int)   # التسمية الكبرى: أي نصف المستوى

for k in [2, 3, 4]:
    bk = BisectingKMeans(n_clusters=k, random_state=42, n_init=5).fit(X_h)
    print(f"k={k}: أحجام العناقيد = {np.bincount(bk.labels_, minlength=k)}")

ari_macro = adjusted_rand_score(y_macro, BisectingKMeans(n_clusters=2, random_state=42, n_init=5).fit(X_h).labels_)
ari_sub = adjusted_rand_score(y_sub, BisectingKMeans(n_clusters=4, random_state=42, n_init=5).fit(X_h).labels_)
print(f"k=2 يعيد بنية المجموعتين الكبيرتين (ARI = {ari_macro:.4f})")
print(f"k=4 يعيد الكتل الفرعية الأربع (ARI = {ari_sub:.4f})")
```

المسار صافٍ: عند `k=2` حصلنا على المجموعتين الكبيرتين تماماً (ARI مثالي مع التسمية الكبرى)، وعند `k=3` فُضّلت المجموعة الأثقل فقط (أحجام ٨٠٠/٤٠٠/٤٠٠)، وعند `k=4` ظهرت الكتل الفرعية الأربع (ARI مثالي مع التسميات الدقيقة). هذا هو «التحكم التدريجي» الذي يفتقده KMeans: تستطيع رفع `k` خطوة خطوة وأنت تعلم أن كل خطوة تفصّل البنية السابقة لا تهدمها — وهي نافذة صغيرة على التجميع الهرمي الذي سيتوسعه الفصل ٦٥.

٨.٦ ملاحظة عن get*bin*seeds وبذور الشبكة

في صفحات sklearn ستصادف دالة `get_bin_seeds` المعرَّفة داخل الوحدة الخاصة `sklearn.cluster._mean_shift` (لاحظ الشرطة السفلية: وحدة داخلية ليست جزءاً من الواجهة العامة). وظيفتها خلق **بذور تقسيم الشبكة** (binning): إسقاط الفضاء على شبكة خشنة بعرض `bin_size` وتمثيل كل خلية شغورة بنقطة واحدة، فتتقلص عيّنة البذور عشرات المرات مع بقاء تغطية الفضاء سليمة. هذه الآلية تستعملها خوارزمية MeanShift حين يُفعَّل `bin_seeding=True` لتسريع صعود تدرج الكثافة — وسنشرحها في سياقها الصحيح في الفصل ٦٢، ونكتفي هنا بالتعريف النظري دون استدعاء، لأن الاستيراد من وحدات خاصة يكسر ضمانات الاستقرار بين الإصدارات.

٨.٧ جدول القرار: أي نسخة تختار؟

حالتك	الحجم التقريبي لـ n	الاختيار الموصى به	لماذا
استكشاف ودقة قصوى	حتى ١٠٠ ألف صف	`sklearn.cluster.KMeans`	الأدق والأنظف، والزمن لا يزال رخيصاً
بيانات ضخمة ملف واحد	مئات الآلاف إلى ملايين	`sklearn.cluster.MiniBatchKMeans`	زمن أقصر بأضعاف مقابل انحراف طفيف
تدفق لا يتوقف	لا نهاية لـ n	`MiniBatchKMeans.partial_fit`	تحديث حي بلا حفظ التاريخ
بنية هرمية وتحكم تدريجي	أي حجم معتدل	`sklearn.cluster.BisectingKMeans`	مسار كبرىً ثم تفصيلاً مع شجرة خفيفة
`k` **كبير جداً (مئات)**	كبير	`BisectingKMeans`	انقسامات ثنائية أرخص من إسناد `k` مراكز دفعة واحدة
لا تعرف `k` **أصلاً**	معتدل	MeanShift أو HDBSCAN	تقدير تلقائي — الفصلان ٦٢ و٦٤

كيف تُستعمل هذه النسخ في الأنظمة الحية؟

النمط السائد في خطوط الإنتاج: خط تعلم حي بـ `partial_fit` يجمع ملايين الأحداث اليومية في مئات المراكز، ثم تُراجَع هذه المراكز دورياً (خلايا فورونوي من الفصل ٦٠) لتحديث قواعد الاستهداف. و`BisectingKMeans` خيار المفضل لدى فرق البحث عند استكشاف كتالوجات ضخمة لأنها تعطيهم «مستوى تكبير» يختارونه بحسب الحاجة: ٨ عناقيد للعرض التنفيذي أو ٦٤ للتفصيل، من نفس الشجرة.

تمارين الفصل الحادي والستين

- **ملاحظة**

التمرين ٨.١

على `make_blobs` بـ ٨٠ ألف عينة و١٠ مراكز في ٦ أبعاد، قارن `KMeans` مع `MiniBatchKMeans` (بإبقاء `random_state=42`): زمن التدريب لكل نسخة، وقيمة مؤشر التجانس المعدّل (ARI) لكل منهما مقارنةً بالتسميات الحقيقية، وقيمة المؤشر بين التسميتين.

التمرين ٨.٢

ولّد بيانات هرمية: مجموعتان متباعدتان، كل منهما تحوي ثلاثة عناقيد فرعية (٦ مراكز). درّب `BisectingKMeans` بـ `k=2` وقس مؤشر التجانس المعدّل (ARI) مع التسمية الكبرى، ثم بـ `k=6` وقس ARI مع التسميات الفرعية. هل حافظ المسار التدريجي على البنية في المستويين؟

التمرين ٨.٣

اشرح بلا كود: لماذا قد يتفاجأ نظام مبني على `partial_fit` بعد أشهر من التشغيل بأن مراكزه لم تعد تمثل العملاء الحاليين؟ وما اسم الظاهرة، وما أثرها على اختيار `s_c` في معادلة التحديث؟

- **ملاحظة**

```python
# حل التمرين ٨.١: مقارنة عادلة على 80 ألف عينة
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
import numpy as np, time

X, y = make_blobs(n_samples=80_000, centers=10, n_features=6,
                  cluster_std=0.9, center_box=(-30, 30), random_state=42)

t0 = time.perf_counter()
km = KMeans(n_clusters=10, n_init=3, random_state=42).fit(X)
t_km = time.perf_counter() - t0
t0 = time.perf_counter()
mb = MiniBatchKMeans(n_clusters=10, batch_size=1024, n_init=3,
random_state=42).fit(X)
t_mb = time.perf_counter() - t0

print(f"KMeans     | {t_km:.2f}s | ARI مع الحقيقة = {adjusted_rand_score(y,
km.labels_):.4f}")
print(f"MiniBatch | {t_mb:.2f}s | ARI مع الحقيقة = {adjusted_rand_score(y,
mb.labels_):.4f}")
print(f"ARI بين تسميتي النسختين: {adjusted_rand_score(km.labels_, mb.labels_):.4f}")
```

على بيانات مفصولة جيداً يكاد الفرق الجودي يختفي وتبقى المكسب الزمني. ولاحظ مرة أخرى أن المقارنة بين النسختين تمت بـ ARI لا بمطابقة التسميات — أسماء العناقيد مختلفة بين النسختين رغم تطابق البنية.

> **ملاحظة**

حل التمرين ٨.٢: بيانات هرمية بمستويين python

```python
from sklearn.cluster import BisectingKMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score

centers = [[0, 0], [0, 6], [0, 12], [24, 0], [24, 6], [24, 12]]
X, y_sub = make_blobs(n_samples=2400, centers=centers, cluster_std=0.9,
random_state=42)
y_macro = (X[:, 0] > 12).astype(int)

bk2 = BisectingKMeans(n_clusters=2, random_state=42, n_init=5).fit(X)
bk6 = BisectingKMeans(n_clusters=6, random_state=42, n_init=5).fit(X)
print(f"k=2 : ARI مع التسمية الكبرى = {adjusted_rand_score(y_macro,
bk2.labels_):.4f}")
print(f"k=6 : ARI مع التسميات الفرعية = {adjusted_rand_score(y_sub,
bk6.labels_):.4f}")
print(f"k=2 : أحجام = {np.bincount(bk2.labels_, minlength=2)}")
print(f"k=6 : أحجام = {np.bincount(bk6.labels_, minlength=6)}")
```

المسار التدريجي حافظ على البنية في المستويين: عند `k=2` افترقت المجموعتان الكبيرتان بشكل شبه مثالي، وعند `k=6` استعادت الخوارزمية الكتل الفرعية الست. هذا بالضبط ما يصعب ضمانه مع KMeans العادية التي تعيد بناء التقسيم من الصفر عند كل تغيير لـ `k`.

> **ملاحظة**

٨.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
مقارنة `labels_` بين KMeans و MiniBatchKMeans بإسناد العناصر	تطابق يبدو صفراً رغم تطابق البنية (الأسماء اعتباطية)	`adjusted_rand_score` من `sklearn.metrics` دائماً
افتراض أن `batch_size` الأصغر أسرع	تضاعف عدد الدفعات فترتفع التكاليف الثابتة ويبطؤ التدريب	ابدأ بـ ١٠٢٤ الافتراضي واضبط صعوداً باتجاه ٤٠٩٦
استدعاء `partial_fit` ثم الاعتماد على `inertia_`	القيمة لا تعكس كامل البيانات (لا يملك التدفق التاريخ)	احسب الجودة على دفعة تحقق محفوظة منفصلة
تقليل `n_init` في MiniBatchKMeans لأقصى سرعة	بذور سيئة قد تعلق المراكز في حل أدنى محلي	الإبقاء على `n_init=3` الافتراضي مع `init_size` الافتراضي

٨.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التجميع بالدفعات الصغيرة	تحديث المراكز بمتوسط متحرك على عينات جزئية	Mini-batch k-means
المتوسط المتحرك	تحديث تدريجي بمعدل يتآكل مع تراكم الخبرة	Moving average update
التجميع المتدفق	التعلّم على تدفق بيانات بلا حفظ التاريخ	Streaming clustering
التجزئة الثنائية	البدء بعنقود واحد وتقسيمه تدريجياً إلى نصفين	Divisive (bisecting) clustering
بذور الشبكة	تمثيل كل خلية من شبكة خشنة بنقطة بذرة واحدة	Bin seeds

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٢ — التقدير التلقائي: MeanShift و AffinityPropagation

تقدير نطاق النواة، ونقل رسالة التفضيل بين العيّنات

الوحدة **scikit-learn ١.٩** الحجم **٨٨١** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح صعود تدرج الكثافة نحو الأنماط (mode seeking) ولماذا لا يحتاج MeanShift إلى k
- أن تضبط estimate_bandwidth بوعي ببارامتر quantile وأثره من التشظي إلى الاندماج
- أن تسرّع MeanShift بـ bin_seeding وتقرر مصير النقاط اليتيمة بـ cluster_all
- أن تفسر رسالتي المسؤولية والتوافر وتضبط damping و preference في AffinityPropagation
- أن تقارن الخوارزميات الأربع كمياً بـ ARI والزمن على بيانات متفاوتة الكثافة

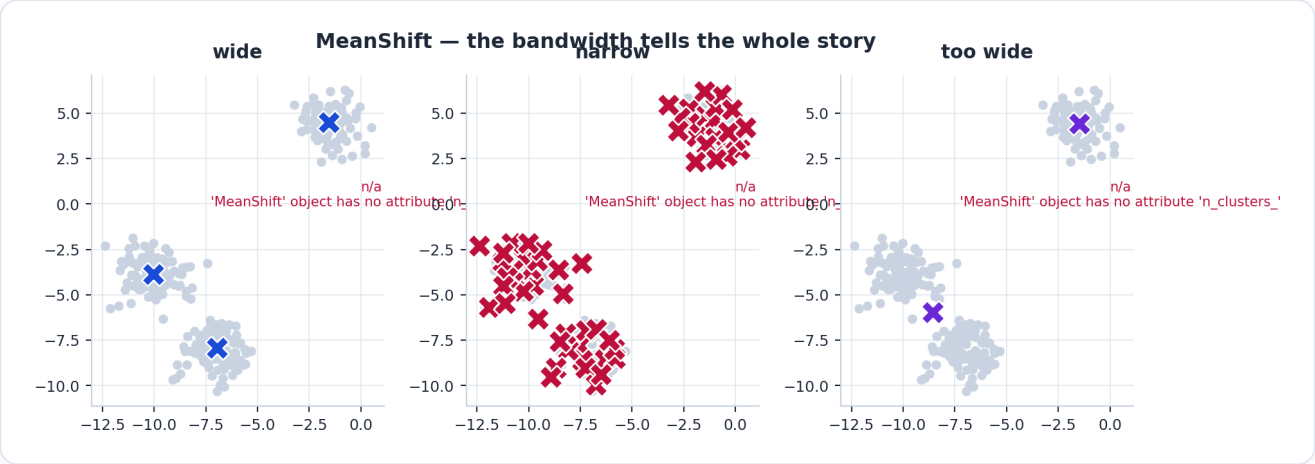
٨.١ صعود الكثافة: تجميعٌ بلا سؤال عن k

تخيّل أن كل نقطة في بياناتك قطرات ماء على تضاريس غير مرئية، وأن المطر يجري دائماً نحو المنخفض. اقلب الصورة رأساً على عقب: الكثافة تضاريس، والقمم فيها هي ما نريد. خوارزمية `sklearn.cluster.MeanShift` تقوم على فكرة **البحث عن الأنماط (Mode Seeking)**: ضع نافذة ناعمة فوق كل نقطة (نواة بعرض `bandwidth`)، واحسب متوسط النقاط داخلها، ثم حرّك النقطة إلى ذلك المتوسط، وكرر. كل نقطة تتسلق تدرج الكثافة حتى تستقر على قمة — والقمم المتطابقة هي العناقيد:

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^{n} K\left(\frac{\|\mathbf{x}-\mathbf{x}_i\|^2}{h^2}\right)\mathbf{x}_i}{\sum_{i=1}^{n} K\left(\frac{\|\mathbf{x}-\mathbf{x}_i\|^2}{h^2}\right)} - \mathbf{x}$$

هذا متجه الإزاحة: `K` دالة النواة، و `h` عرض النواة، و `x` النقطة الحالية، و `x_i` عينات البيانات. النقطة تتحرك خطوة في اتجاه `m(x)` حتى يقترب من الصفر — أي وصلت قمة لا تريد مغادرتها. ولاحظ ما الذي اختفى من الصورة: لا يوجد `k` إطلاقاً؛ عدد العناقيد هو عدد القمم التي تصادف البيانات أن فيها. لكن بقي بارامتر واحد يشبه `k` في خطورته:

عرض النواة `h` . sklearn تقدم مقدّراً آلياً `sklearn.cluster.estimate_bandwidth` يعتمد على بارامتر `quantile` : خذ المسافة من كل نقطة إلى جارتها رقم `n × quantile` وخذ متوسطها. ما أثر اختيار `quantile` ؟ تجربة مباشرة:



الشكل ١٣٤منحنى الكثافة التقديري: نوّة صغيرة تشظّي الكثافة إلى قمم كثيرة، ونوّة كبيرة تذيب القمم في واحدة

تجربة quantile: من التشظي إلى الاندماج: python

```python
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs

Xq, yq = make_blobs(n_samples=400, centers=3, cluster_std=0.9, random_state=42)
for q in [0.05, 0.2, 0.6]:
    bw = estimate_bandwidth(Xq, quantile=q, random_state=42)
    ms = MeanShift(bandwidth=bw).fit(Xq)
    print(f"quantile={q:4.2f} -> bandwidth={bw:6.3f} -> عدد العناقيد =
{ms.labels_.max() + 1}")
```

السلّم كامل: نوّة صغيرة (quantile=٠.٠٥) أعطت عرض نواة ٠.٧٩٠ فتشتتت الكتل الثلاث إلى ١٥ عنقوداً مجزأً؛ النوّة الوسطى (٠.٢) أعطت ١.٦٩٠ فاكتشفت العدد الصحيح ٣؛ والنوّة العريضة (٠.٦) أعطت ١٢.٣٠٤ فذابت كل القمم في عنقود واحد. في الشكل أعلاه تقرأ القصة نفسها على منحنى الكثافة. بارامتر `quantile` ليس تفصيلاً تقنياً — إنه القرار.

الكثافة تضاريس والنطاق سُلّم

كل خوارزميات هذا الفصل تنظر إلى البيانات بوصفها تضاريس كثافة أو تشابه، ولا تسألك عن عدد القمم. لكن كلاهما يعطيك مقبض ضبط يحسم كل شيء: عرض النواة في MeanShift، والتفضيل في AffinityPropagation. من يضبط المقبض جيداً يجد العدد تلقائياً؛ ومن يتركه عشوائياً يحصل على تلقائية وهمية.

٨.٢ التسريع: bin_seeding و seeds

كل نقطة بذرة مرشحة لتصبح قمة — وهذا يعني دورات حساب هائلة. الحل المدمج في MeanShift: بارامتر `bin_seeding=True` الذي يطبق فكرة بذور الشبكة التي قابلناها نظرياً في الفصل ٦١ (وهي المنفَّذة في وحدة `sklearn.cluster._mean_shift` الخاصة): إسقاط الفضاء على شبكة خشنة وتمثيل كل خلية شغورة ببذرة واحدة، فتنكمش قائمة البذور عشرات المرات. وهناك أيضاً بارامتر `seeds` لتغذية بذور من عندك، و`min_bin_freq` لإسقاط الخلايا الضعيفة. هل التسريع حقيقي؟

قياس bin_seeding: بذور كل النقاط مقابل بذور الشبكة — python

```python
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs
import time

Xb, _ = make_blobs(n_samples=1000, centers=5, cluster_std=1.0, random_state=42)
bw = estimate_bandwidth(Xb, quantile=0.2, random_state=42)

t0 = time.perf_counter()
ms_plain = MeanShift(bandwidth=bw, bin_seeding=False).fit(Xb)
t_plain = time.perf_counter() - t0

t0 = time.perf_counter()
ms_bin = MeanShift(bandwidth=bw, bin_seeding=True).fit(Xb)
t_bin = time.perf_counter() - t0

print(f"bin_seeding=False | {t_plain:6.2f}s | عناقيد = {ms_plain.labels_.max() +
1}")
print(f"bin_seeding=True  | {t_bin:6.2f}s | عناقيد = {ms_bin.labels_.max() + 1}")
print(f"تسريع: {t_plain / t_bin:.1f}x")
```

العشرون ضعفاً تقريباً مع البنية نفسها (النسختان وجدتا العناقيد نفسها).

فعّل bin_seeding افتراضياً

لا يوجد سبب عملي تقريباً لإبقاء `bin_seeding=False` على بيانات حقيقية: النتيجة تكاد تتطابق والزمن ينكمش بأضعاف. استثناؤه الوحيد: بيانات صغيرة جداً (مئات قليلة) حيث تكلفة الشبكة تنافس وفرها.

٨.٣ cluster_all: مصير النقاط اليتيمة

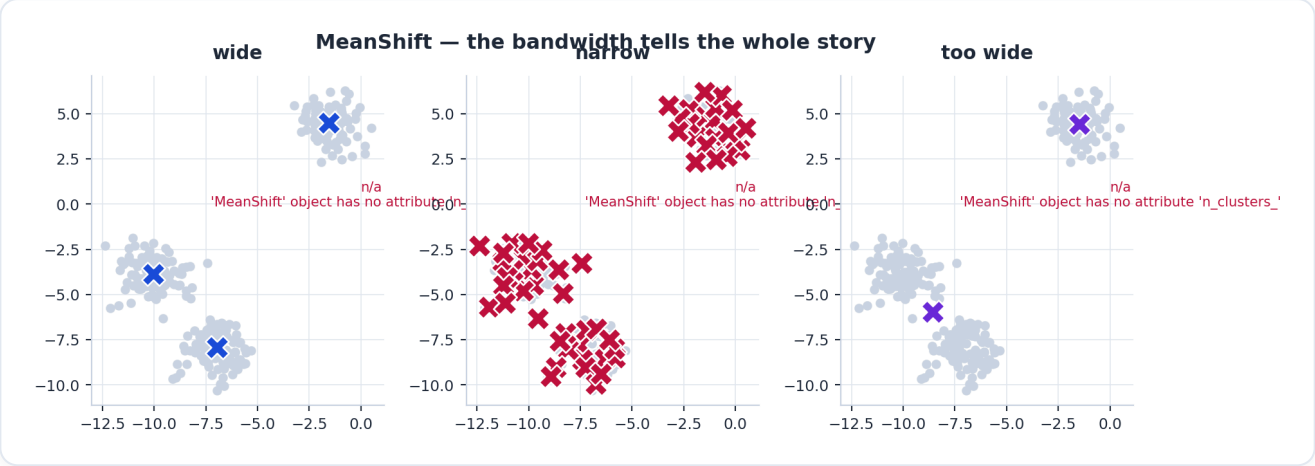
ماذا يحدث لنقطة سقطت في الصحراء — بعيدة عن كل الكثافة؟ بارامتر `cluster_all` يقرر: بالقيمة `True` (الافتراضية) تُسند لكل نقطة عنقود بلا استثناء (أقرب قمة مهما بُعدت)، وبـ `False` تبقى النقاط التي لم تصلها أي نواة بلا تسمية `-1`. لاحظ التفصيلة الدقيقة في التجربة التالية: نغذي البذور يدوياً ببارامتر `seeds` من نقاط السحابتين فقط، حتى لا تصبح النقاط البعيدة قممًا لنفسها:

```python
# تجربة cluster_all: إسناد الجميع أم ترك اليتامى؟
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs
import numpy as np

Xc, _ = make_blobs(n_samples=400, centers=2, cluster_std=0.8, random_state=42)
orphans = np.array([[40.0, 40.0], [-42.0, -38.0]])
X_far = np.vstack([Xc, orphans])
bw = estimate_bandwidth(Xc, quantile=0.3, random_state=42)

ms_all = MeanShift(bandwidth=bw, cluster_all=True, seeds=Xc).fit(X_far)
ms_some = MeanShift(bandwidth=bw, cluster_all=False, seeds=Xc).fit(X_far)
print(f"cluster_all=True : تسميات النقطتين البعيدتين = {ms_all.labels_[-2:]}")
print(f"cluster_all=False: تسميات النقطتين البعيدتين = {ms_some.labels_[-2:]}")
print(f"التسميات الموجودة في الوضع الثاني: {np.unique(ms_some.labels_)}")
```

بالنمط الأول أُسندت النقطتان البعيدتان إلى عنقود من العنقودين (تسميات ٠ و١)، وبالثاني عُلّمتا `-1` — «يتيمتان بلا عنقود». متى تحتاج الوضع الثاني؟ حين يكون كشف «لا أنتمي لأحد» جزءاً من المهمة نفسها: فلترة قراءات مستشعرات معطوبة قبل التجميع، أو فصل الحالات الشاذة التي يستكملها الفصل ٧٠ بأدوات أقوى.

٨.٤ عيوب MeanShift: مربع الحجم وحساسية النطاق

لماذا لا تهيمن MeanShift على العالم رغم أنها لا تطلب k؟ عيبان قتلان:



الشكل ١٣٥ النافذة الواحدة نفسها: ما يناسب القمة الضيقة يذيب القمة الواسعة والعكس — جوهر عيب النطاق العالمي في MeanShift

وهو ما ستكشفه المواجهة الكمية

> **الافتراضي quantile=٠.٣ ليس حكماً**
>
> `estimate_bandwidth` تستعمل افتراضياً `quantile=0.3`، وهي قيمة معقولة لبيانات متوسطة الفصل فقط. على بياناتك الحقيقية لا تمرر النطاق بلا فحص: قارن ناتج ٣–٤ قيم من `quantile` (كما فعلنا في ٨.١) واستقر على الأكثر اتساقاً مع معرفتك بالمجال، وإلا فأنت تُسند قرار «كم عنقوداً؟» إلى رقم افتراضي لا يعرف بياناتك شيئاً.

٨.٥ AffinityPropagation: العينات تتفاوض فيما بينها

ألوانا الثاني يقلب المنطق: بدل بحث الفضاء عن قمم، تجعل `sklearn.cluster.AffinityPropagation` **العيّنات نفسها** مرشحة لأن تصبح «نماذج ممثِلة» (Exemplars) — نقاط حقيقية من بياناتك تمثل عنقودها — ثم تفتح بينها قناة تفاهم من رسالتين تتبادلان حتى الاستقرار:

$$r(i,k) \leftarrow s(i,k) - \max_{j \neq k}\left(a(i,j)+s(i,j)\right)$$

$$a(i,k) \leftarrow \min\left(0,\ r(k,k) + \sum_{i' \notin \{i,k\}} r(i',k)\right)$$

حيث `s(i,k)` التشابه بين العينتين — مع `affinity='euclidean'` الافتراضية هو سالب مربع المسافة بينهما — والقطر `s(k,k)` هو «التفضيل»: مقدار إعجاب كل نقطة بنفسها كمرشح للتمثيل. افتراضياً تستعمل `preference`

sklearn **الوسيط** لمصفوفة التشابه ككل (لا المتوسط — فالوسيط أكثر مقاومة للقيم المتطرفة). رفع التفضيل يرفع شهية كل نقطة للتمثيل فيتضاعف عدد العناقيد:

```python
# تجربة preference: من 4 عناقيد إلى انفجار كامل
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs
from sklearn.metrics import euclidean_distances
import numpy as np

Xp, yp = make_blobs(n_samples=400, centers=4, cluster_std=0.9, random_state=42)
S = -euclidean_distances(Xp) ** 2
pref_med = float(np.median(S))
print(f"التشابه الوسيط (الافتراضي): {pref_med:.1f}")
for pref in [pref_med, pref_med * 0.5, 0.0]:
    ap = AffinityPropagation(preference=pref, random_state=42,
max_iter=500).fit(Xp)
    print(f"preference={pref:8.1f} -> عناقيد = {len(ap.cluster_centers_indices_)} |
تكرارات = {ap.n_iter_}")
```

سلّم صاعد نظيف: التفضيل الوسيط أعطى العدد الصحيح ٤، وتقريبه نحو الصفر (إلى -٧٢.٧) رفعه إلى ٨، وعند الصفر انفجر إلى ٤٠٠ — كل نقطة صارت نموذجاً لنفسها! والاتجاه المعاكس صحيح تماماً كما سترى في تمرين ٨.٢ على بيانات Iris: كلما جعلت التفضيل أكثر سلبية قلّت الرغبة في التمثيل وذابت العناقيد تدريجياً. البارامتر الثاني `damping` (افتراضه ٠.٥) يضبط كم من الرسالة القديمة يمزج مع الجديدة في كل تحديث — دواء التذبذب:

```python
# تجربة damping: من التذبذب إلى الزحف
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score

Xd, yd = make_blobs(n_samples=300, centers=3, cluster_std=0.8, random_state=42)
for d in [0.5, 0.75, 0.9, 0.95]:
    ap = AffinityPropagation(damping=d, random_state=42, max_iter=1000).fit(Xd)
    print(f"damping={d:4.2f} -> عناقيد = {len(ap.cluster_centers_indices_)}"
          f" | تكرارات حتى التقارب = {ap.n_iter_} | ARI = {adjusted_rand_score(yd,
ap.labels_):.4f}")
```

اقرأ التكرارات لا العناقيد (ثلاثة في كل الحالات هنا): قيمة ٠.٥ احتاجت ١٥٥ تكراراً لأن الرسائل تحمل تغيّراً كاملاً كل خطوة فتتأرجح، بينما ٠.٧٥ استقرت في ٣٧، ثم زحف الزمن مجدداً نحو ٧٧ عند ٠.٩٥ لأن التحديثات صارت شبه جامدة. النطاق العملي الآمن ٠.٥–٠.٨٥، وعند التقارب تعيد الخوارزمية سماتها المفصلية: `cluster_centers_indices_` (فهارس النماذج الممثِّلة — وهي عينات حقيقية من بياناتك)، و `labels_`، و `n_iter_`.

٨.٦ كلفة $O(n^2)$: متى يكون مقبولاً؟

الرسائل تُحسب من مصفوفة تشابه كاملة: `n×n` أعداد حقيقية. أرقام صريحة: ١٠٠٠ عينة = مليون عنصر (نحو ٨ ميغابايت — لا شيء)، و ١٠ ألف عينة = عشرة مليارات عنصر (نحو ٨٠ غيغابايت — انفجار). لذلك فإن AffinityPropagation — و MeanShift ببذوره الكاملة — أدوات لمئات إلى بضعة آلاف من العينات لا أكثر، أما ملايين الصفوف فمملكة KMeans الدفعية من الفصل ٦١ وخوارزميات الكثافة في الفصلين ٦٣ و٦٤.

⛔ مصفوفة التشابه تقتل الذاكرة بصمت

لا يوجد تحذير قبل الانفجار: `AffinityPropagation().fit(X)` مع ٥٠ ألف عينة سيحاول حجز مصفوفة ٢.٥ مليار عنصر ويسقط العملية بـ MemoryError أو يستنزف الجهاز. اجعل قاعدة إصبعك: فوق ٥٠٠٠ عينة، فكّر مرتين؛ فوق ٢٠ ألفاً، لا تفكر أصلاً — انتقل لخوارزمية خطية التكلفة.

🏭 أين يعيش هذان العالمان في المشاريع الحقيقية؟

MeanShift من رواد الرؤية الحاسوبية: تتبع الأهداف في الفيديو وتقسيم الصور (كل بكسل نقطة في فضاء اللون والموقع)، وكلها مسائل بعينات معتدلة تعشق التلقائية. و AffinityPropagation اشتهر في تلخيص البيانات باختيار «نماذج ممثِلة» حقيقية — عرض ٢٠ بريداً يمثل ألف بريد بدل عرض مراكز افتراضية لا معنى لها للإنسان، ولهذا يفضَّل حين تكون قابلية الشرح لأصحاب العمل أهم من السرعة.

٨.٧ المواجهة الكمية: ثلاث كثافات وأربع خوارزميات

لنجمع كل ما سبق في تجربة واحدة: ثلاثة عناقيد حقيقية بكثافات مختلفة جذرياً (`cluster_std` بين ٠.٤ و٢.٠)، ونقيس ARI والزمن للأربع. تذكير مقتضب: `adjusted_rand_score` من `sklearn.metrics` يقيس اتفاق تقسيمين ثابتاً ضد تبديل الأسماء (شرحناه في الفصل ٥٩ وسيحلل الفصل ٦٦ تفاصيله):

المواجهة: أربع خوارزميات على ثلاث كثافات python

```python
from sklearn.cluster import KMeans, MiniBatchKMeans, MeanShift,
AffinityPropagation, estimate_bandwidth
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
import time

X3, y3 = make_blobs(n_samples=[200, 200, 200], centers=[[0, 0], [7, 2], [2,
12]],
                    cluster_std=[0.4, 1.0, 2.0], random_state=42)
bw = estimate_bandwidth(X3, quantile=0.2, random_state=42)
rows = []

t0 = time.perf_counter()
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X3)
rows.append(("KMeans (k=3)", time.perf_counter() - t0, adjusted_rand_score(y3,
km.labels_)))

t0 = time.perf_counter()
mb = MiniBatchKMeans(n_clusters=3, batch_size=256, n_init=3,
random_state=42).fit(X3)
rows.append(("MiniBatchKMeans", time.perf_counter() - t0,
adjusted_rand_score(y3, mb.labels_)))

t0 = time.perf_counter()
ms = MeanShift(bandwidth=bw, bin_seeding=True).fit(X3)
rows.append(("MeanShift (تلقائي)", time.perf_counter() - t0,
adjusted_rand_score(y3, ms.labels_)))

t0 = time.perf_counter()
ap = AffinityPropagation(random_state=42, max_iter=500).fit(X3)
rows.append(("AffinityPropagation", time.perf_counter() - t0,
adjusted_rand_score(y3, ap.labels_)))

print(f"MeanShift وجد {ms.labels_.max() + 1} عناقيد | AP وجد
{len(ap.cluster_centers_indices_)}")
print(f"{'الخوارزمية':20s} | الزمن    | ARI")
for name, t, ari in rows:
    print(f"{name:20s} | {t:6.2f}s | {ari:.4f}")
```

أرقام تكشف الشخصيات: KMeans ونسختها الدفعية حصدتا توافقاً ممتازاً في جزء من الثانية (لكن بميزة غير عادلة: أعطيناهما `k=3` الصحيح مجاناً!). MeanShift وجد العدد بنفسه بخطأ طفيف (٤ بدل ٣) وبتوافق ممتاز ٠.٩٧٥٤ وبتكلفة زمن أعلى بدرجات. أما AffinityPropagation فتفتتت العنقود الواسع المتناثر إلى نماذج كثيرة (ARI منخفض) ودفع وقتاً أضعاف الجميع — الكثافات المتفاوتة عدوها الأول: كل النقاط المتناثرة متساوية الضعف عندها فتصبح كل واحدة مرشحة تمثيل. الخلاصة العملية: التلقائية ليست مجانية، والكثافة المتفاوتة تحتاج أدوات مصممة لها أصلاً — هناك يقف HDBSCAN في الفصل ٦٤.

تمارين الفصل الثاني والستين

- **ملاحظة**

التمرين ٨.١

على `make_blobs` بـ ٥٠٠ عينة و٥ مراكز (`cluster_std=0.8`)، فحص قيم `quantile` من ٠.٠٥ إلى ٠.٥ بخمس خطوات مع `estimate_bandwidth`: اطبع النطاق الناتج وعدد العناقيد المكتشفة في كل مرة، وحدد أي قيم أعادت العدد الصحيح.

التمرين ٨.٢

على بيانات Iris، درّب `AffinityPropagation` بالتفضيل الافتراضي (وسيط التشابه) ثم بقيم تفضيل أدنى منه تدريجياً (أكثر سلبية: الوسيط × ١.٥، × ٢، × ٣، × ٦) حتى تحصل على ٣ نماذج ممثِلة فقط، واطبع ARI مع الأنواع الحقيقية وعدد النماذج الممثِلة وتكرارات التقارب في كل محاولة.

التمرين ٨.٣

اشرح بلا كود: لماذا يفشل نطاق نواة واحد (`bandwidth`) في بيانات كثافاتها متباعدة جداً؟ ولماذا يعاني AffinityPropagation من المشكلة نفسها رغم أنه لا يستعمل نطاقاً أصلاً؟

- **ملاحظة**

حل التمرين ٨.١: مسح quantile على 500 عينة — python

```python
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs

Xe, ye = make_blobs(n_samples=500, centers=5, cluster_std=0.8, random_state=42)
for q in [0.05, 0.15, 0.25, 0.35, 0.5]:
    bw = estimate_bandwidth(Xe, quantile=q, random_state=42)
    ms = MeanShift(bandwidth=bw).fit(Xe)
    print(f"quantile={q:4.2f} -> bandwidth={bw:6.3f} -> عناقيد = {ms.labels_.max()
+ 1}")
```

اقرأ السلم: القيم الصغيرة تشظّي العناقيد الخمسة حتى تكاد تضاعفها (١١ عنقوداً عند ٠.٠٥)، والقيم الكبيرة تذيبها تدريجياً (٣ ثم ٢)؛ القيمة ٠.١٥ هي التي أعادت العدد الصحيح ٥. الدرس المعاد: التقدير التلقائي موجود لكنه مشروط

بمقبض تضبطه أنت — والقيمة الافتراضية ٠.٣ ليست حكماً.

> **ملاحظة**

حل التمرين ٨.٢: معايرة preference على Iris — python

```python
from sklearn.datasets import load_iris
from sklearn.cluster import AffinityPropagation
from sklearn.metrics import euclidean_distances, adjusted_rand_score
import numpy as np

Xi, yi = load_iris(return_X_y=True)
S = -euclidean_distances(Xi) ** 2
pref_med = float(np.median(S))
print(f"التفضيل الافتراضي (الوسيط): {pref_med:.2f}")
for factor in [1.0, 1.5, 2.0, 3.0, 6.0]:
    ap = AffinityPropagation(preference=pref_med * factor, random_state=42,
max_iter=500).fit(Xi)
    n_ex = len(ap.cluster_centers_indices_)
    print(f"factor={factor:4.1f} -> نماذج ممثِّلة = {n_ex:3d}"
          f" | تكرارات = {ap.n_iter_:3d} | ARI = {adjusted_rand_score(yi,
ap.labels_):.4f}")
```

جعل التفضيل أكثر سلبية ذاب النماذج الممثِّلة تدريجياً من ٧ إلى ٣ — العدد الحقيقي للأنواع — وصعد ARI من ٠.٤٨١٢ إلى ٠.٧٤٥٥، وهو يفوق حتى ما حصده KMeans في الفصل ٦٠ على البيانات نفسها (٠.٧٣٠٢). لاحظ الكلفة: تكرارات أكثر في كل خطوة، وتذكّر من ٨.٦ أن كل ذلك يجري داخل مصفوفة ١٥٠×١٥٠ — لو كانت بياناتك ١٥٠ ألف صف لتوقف كل شيء.

> **ملاحظة**

٨.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
إطلاق `estimate_bandwidth` بقيمها الافتراضية على أي بيانات	عدد عناقيد عشوائي (تشظٍّ أو اندماج) يحكمه quantile=٠.٣	مسح ٣–٤ قيم من `quantile` ومطابقتها بمعرفة المجال
تشغيل MeanShift/AP على مئات الآلاف من الصفوف	انفجار O(n²): دقائق إلى ساعات، وذاكرة بعشرات الغيغابايت	خوارزميات خطية التكلفة (KMeans الدفعية، DBSCAN، HDBSCAN)
رفع `preference` في AP فوق الوسيط بلا قصد	انفجار عدد العناقيد حتى ن نقطة = ن عنقود	الإبقاء عند الوسيط أو خفضه، ومراقبة `cluster_centers_indices_`
ترك `damping=0.5` مع بيانات متأرجحة	تذبذب رسائل وطول التقارب أو ConvergenceWarning عند سقف التكرارات	رفع `damping` إلى ٠.٧–٠.٨٥ ورفع `max_iter` عند الحاجة

٨.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البحث عن الأنماط	صعود تدرج الكثافة حتى الاستقرار على القمم	Mode seeking
عرض النواة	نصف قطر النافذة التي تُحسب ضمنها الكثافة	Bandwidth
النموذج الممثِل	عينة حقيقية من البيانات تمثل عنقودها في AP	Exemplar
رسالة المسؤولية	دليل أن العينة تنتمي لنموذج بدل غيره	Responsibility
رسالة التوافر	دليل أن النموذج مستعد لتبني العينة	Availability

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٣ — العناقيد الكثيفة: DBSCAN و OPTICS

النقاط النواة، و eps، و min_samples، واستخراج العناقيد

الوحدة **scikit-learn ١.٩** الحجم **٩٥٩** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تفسر لماذا يفشل K-Means على الهلالين والحلقتين بأرقام ARI حقيقية، ولماذا ينجح DBSCAN حيث يفشل
- أن تصنف العينات إلى نواة وحد وضجيج وتقرأ labels_ و components_ و core_sample_indices_ بثقة
- أن تختار eps بتجربة مسح واعية لفخ المقياس، وتطبيع البيانات قبل أي تجميع كثافي
- أن تستخدم OPTICS على بيانات كثافات متفاوتة وتقرأ متجه اللاح reachability_ كأداة تشخيص
- أن تقدّر الكلفة الحسابية للجوار: O(n log n) مع شجرة جوار مقابل O(n²) بلاها

٨.١ لماذا يفشل K-Means على الهلالين والحلقتين؟

أغلق عينيك وتخيل بيانات على شكل هلالين متقابلين: كتلتان منحنيتان، لا يوجد في أحدهما أي «مركز» يمثل شكلها الحقيقي. طلبنا من `KMeans` (الفصل ٦٠) أن يجد عنقودين، فوجد مركزين في وسط المستوى وقسّم البيانات بخط شبه مستقيم — والنتيجة قطعٌ عشوائي للهلالين لا علاقة له بالبنية الحقيقية.

الحلقة المتداخلة أسوأ: حلقة داخلية وأخرى خارجية حول المركز نفسه، فأي «مركز» واحد يجمعهما! هذا هو الفشل المفاهيمي للخوارزميات القائمة على المسافة عن مركز (centroid-based) — وهي تفترض أن العنقود كرة (convex)، بينما الشكل الحقيقي هنا منحنٍ أو حلقي. التجربة التالية قاطعة، بمقياس ARI الذي سنتعمق فيه في الفصل ٦٦ (القيمة ١ تعني تطابقاً كاملاً مع التقسيم الحقيقي، و٠ تعني تقسيماً عشوائياً):

```python
# القاطعة: K-Means مقابل DBSCAN على الهلالين والحلقتين
from sklearn.datasets import make_moons, make_circles
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import adjusted_rand_score

for name, maker in [
    ("الهلالان", lambda: make_moons(n_samples=600, noise=0.08, random_state=42)),
    ("الحلقتان", lambda: make_circles(n_samples=600, factor=0.45, noise=0.06,
random_state=42)),
]:
    X, y = maker()
    km = KMeans(n_clusters=2, n_init=10, random_state=42).fit(X)
    db = DBSCAN(eps=0.15, min_samples=5).fit(X)
    print(f"{name}: ARI(KMeans)={adjusted_rand_score(y, km.labels_):.4f}"
          f" | ARI(DBSCAN)={adjusted_rand_score(y, db.labels_):.4f}")
```

DBSCAN لم يرسم أي مركز ولا أي خط فاصل؛ سأل فقط: «من يقف بجوار من؟» فتبع المنحنى نقطةً نقطة حتى اكتمل الهلال. هذا هو جوهر التجميع الكثافي (density-based clustering): العنقود منطقة كثيفة يفصلها عن غيرها فراغ قليل الكثافة.

> **مفتاح**
>
> المفهوم المحوري: K-Means يسأل «كم بعدك عن المركز؟» بينما DBSCAN يسأل «كم نقطة تعيش في جوارك؟». أول سؤال يرى الكرات فقط؛ الثاني يرى أي شكل تلتصق فيه النقاط.

٨.٢ قاموس DBSCAN: نواة وحد وضجيج

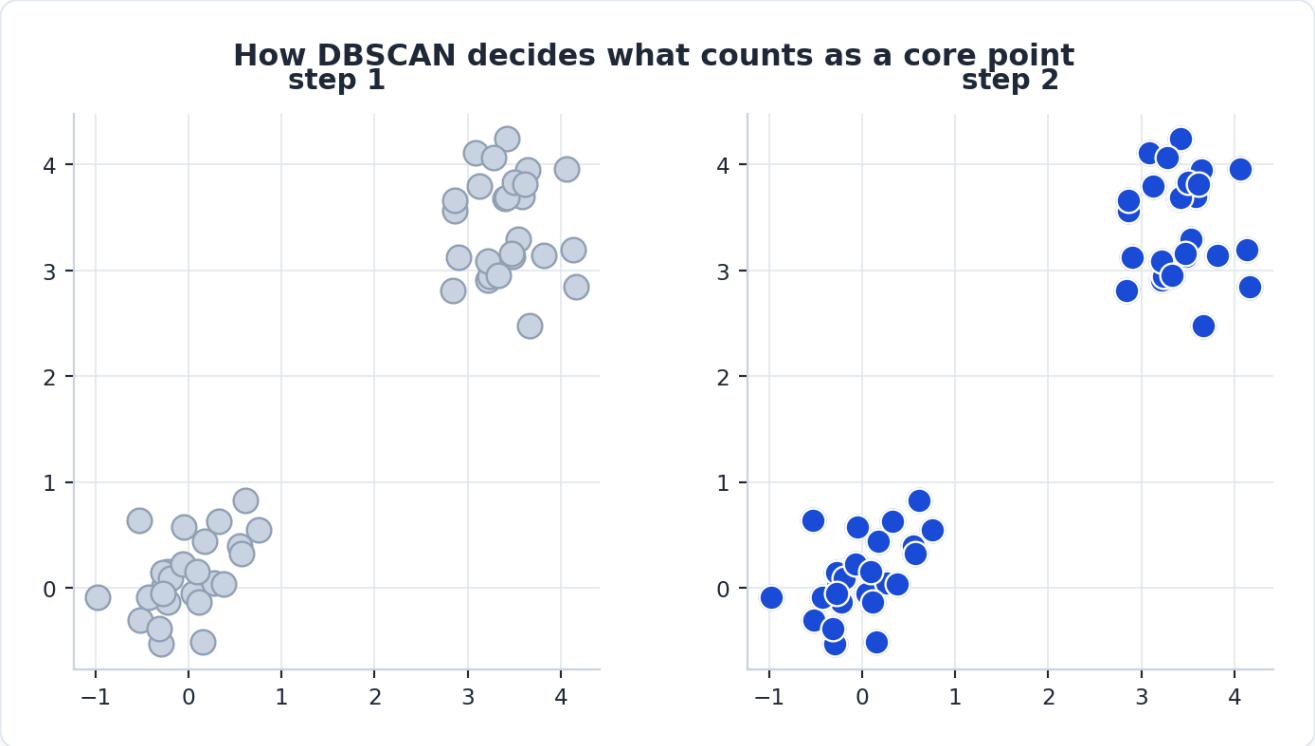
تُعرَّف كل المفاهيم عبر مفهوم واحد: **جوار إبسيلون (epsilon-neighborhood)**، وهو قرص نصف قطره `eps` حول النقطة:

$$N_\varepsilon(p) = \{\, q \in D \;:\; d(p,q) \le \varepsilon \,\}$$

حيث `D` مجموعة البيانات كاملة، و `d(p, q)` المسافة الإقليدية بين النقطتين. وتكون النقطة **نقطة نواة (core point)** إذا كان جوارها يضم ما لا يقل عن `min_samples` عينة — لاحظ أن sklearn يعدّ النقطة نفسها ضمن جوارها:

$$p \text{ core} \iff |N_\varepsilon(p)| \ge \text{MinPts}$$

من هنا تنشأ الأدوار الثلاثة التي يُبنى عليها كل شيء:



الشكل ١٣٦ أدوار النقاط الثلاثة في DBSCAN: نواة كثيفة يلتحق بها حدّ، ونقاط معزولة تصبح ضجيجاً

لاحظ التفاوت الحرج بين الدورين: النواة «تسبّب» العنقود، والحدّية «تستفيد» منه. وسنفصل في القسم ٨.٣ كيف تظهر هذه الأدوار في مخرجات المكتبة.

٨.٣ قراءة المخرجات: labels و *components* و core*sample*indices_

بعد `fit` يمتلك الكائن ثلاث خصائص تحمل القاموس كله عملياً:

الخاصية	المحتوى	كيف تقرأها
`labels_`	تسمية لكل عينة: `0, 1, 2, ...` للعناقيد و `-1` للضجيج	`np.unique(labels_)` يعطيك عدد العناقيد مباشرة
`core_sample_indices_`	فهارس النقاط النواة في البيانات الأصلية	طولها = عدد النقاط النواة
`components_`	إحداثيات النقاط النواة نفسها (نسخة من صفوف `X`)	نقطة انطلاق أي بحث جوار لاحق

```python
# تشريح النقاط: نواة وحد وضجيج
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
import numpy as np

X, y_true = make_blobs(n_samples=400, centers=3, cluster_std=0.8,
random_state=42)
db = DBSCAN(eps=0.9, min_samples=8).fit(X)
labels = db.labels_

n_core = len(db.core_sample_indices_)
n_noise = int((labels == -1).sum())
n_border = int((labels != -1).sum()) - n_core
print(f"عدد العناقيد: {len(set(labels)) - (1 if -1 in labels else 0)}")
print(f"نواة: {n_core} | حدّية: {n_border} | ضجيج: {n_noise}")
print(f"شكل components_: {db.components_.shape}")
print(f"أول 10 تسميات: {labels[:10]}")
print(f"أول 6 فهارس نواة: {db.core_sample_indices_[:6]}")
```

ثلاث قراءات من الناتج تستحق التثبيت: عدد العناقيد **يُحسب** بطرح واحدة من القيم الفريدة إن وُجد `-1`؛ النقاط الحدّية هي الفارق بين «المصنَّفة» و«النواة»؛ و`components_` نسخة مستقلة من صفوف البيانات (يمكنك تغذيتها مباشرة إلى `NearestNeighbors` لتصنيف عينات جديدة قرب أقرب نواة). انتبه إلى أن الدالة `fit_predict` موجودة، بينما لا توجد `predict` إطلاقاً في DBSCAN — سنرد على هذا في الأخطاء الشائعة.

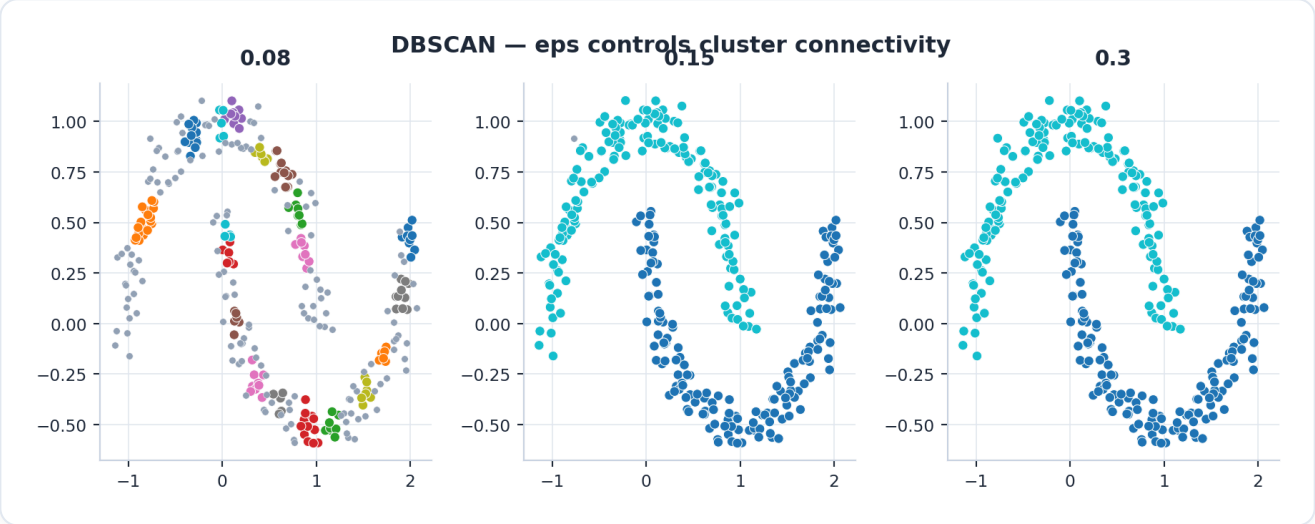
> **تعمّق**
>
> لماذا قد تحصل نقطة حدّية قريبة من عنقودين على تسمية أحدهما دون الآخر؟ لأن خوارزمية DBSCAN تزور النقاط بالترتيب: أول نواة «تلمس» الحدّية تمنحها تسميتها، وتنتهي المسألة. النتيجة عملية: تبديل ترتيب الصفوف قد يبدّل انتماء النقاط الحدّية بين عنقودين — لا تبنِ قراراً حساساً على حدود العناقيد.

٨.٤ فخ eps: تجربة مسح كاملة بدل التخمين

كم يساوي `eps` «الصحيح»؟ لا إجابة نظرية عامة — لكن هناك تجربة رخيصة: مسح نطاق من القيم ومراقبة عدد العناقيد ونسبة الضجيج. سنستخدم بيانات معيارية مطبَّعة، وسنقيس ARI مقابل التقسيم الحقيقي أيضاً:

مسح eps من 0.1 إلى 1.0 python

```python
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_blobs(n_samples=600, centers=4, cluster_std=0.9, random_state=42)
Xs = StandardScaler().fit_transform(X)
for eps in np.arange(0.1, 1.05, 0.1):
    lab = DBSCAN(eps=float(eps), min_samples=5).fit(Xs).labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"eps={eps:.1f} | عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y, lab):.3f}")
```



الشكل ١٣٧ علاقة eps بحدود العناقيد: قيمة صغيرة تفتّت، وكبيرة تدمج حتى تلتهم الفجوات

اقرأ الجدول كقصّة بثلاثة فصول: عند `eps=0.1` تتفكك البيانات (عناقيد أكثر من الحقيقة وضجيج ٦%)؛ عند `0.2` نصل للحلو (٤ عناقيد وARI=٠.٩٩٣)؛ ومن `0.3` إلى `0.9` يحدث الانفصام الصامت — العنقودان المتقاربان يذوبان في واحد فيهبط ARI إلى ٠.٧١٣ **دون أي ضجيج يحذّرك**؛ وعند `1.0` يتبقى عنقودان فقط. الحلقة الغادرة هنا أن «صفر ضجيج» لا يعني «تقسيم صحيح»: الضجيج مؤشر على قلة الكثافة، أما الاندماج فلا يصرخ أبداً.

نصيحة

للاختيار الأولي المعقول لـ `eps` دون مسح كامل: احسب مسافة الجار الأقرب (`NearestNeighbors(n_neighbors=min_samples).fit(Xs)`) ورتّب المسافات، فالعقدة (knee) في هذا المنحنى تعطي ترتيب الحجم الصحيح لـ eps. ثم أكّد بمسح قصير حول هذه القيمة كما فعلنا أعلاه.

٨.٥ بلا تطبيع لا معنى لـ eps إطلاقاً

هنا أكثر أخطاء DBSCAN شيوعاً في الواقع العملي: `eps` رقم بالوحدات نفسها التي تُقاس بها مسافات بياناتك. لو كانت الميزة الأولى «الراتب بالدولار» (قيم بآلاف) والثانية «العمر» (قيم بعشرات)، فإن المسافة تحكمها الميزة الأولى وحدها، وأي قيمة eps منطقية للعمر ستكون مهملة أمام الراتب. لنضاعف عموداً واحداً بمقدار ١٠٠٠ ونرى ماذا يحدث:

بلا تطبيع: eps ضجيج 100% python

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_blobs(n_samples=500, centers=3, cluster_std=0.8, random_state=42)
X_units = X.copy()
X_units[:, 0] *= 1000.0   # نفس البيانات لكن ميزة واحدة بوحدة أضخم 1000 مرة

db_raw = DBSCAN(eps=0.5, min_samples=5).fit(X_units)
db_scl = DBSCAN(eps=0.5,
min_samples=5).fit(StandardScaler().fit_transform(X_units))
print(f"بلا تطبيع: عناقيد={len(set(db_raw.labels_)) - (1 if -1 in db_raw.labels_ else
0)}"
      f" | ضجيج={(db_raw.labels_ == -1).mean():.1%}"
      f" | ARI={adjusted_rand_score(y, db_raw.labels_):.3f}")
print(f"مع تطبيع: عناقيد={len(set(db_scl.labels_)) - (1 if -1 in db_scl.labels_ else
0)}"
      f" | ضجيج={(db_scl.labels_ == -1).mean():.1%}"
      f" | ARI={adjusted_rand_score(y, db_scl.labels_):.3f}")
```

نفس البيانات، نفس `eps=0.5`: الأولى تُنتج **صفر عناقيد و١٠٠% ضجيج** — كل نقطة أعلنت أنها وحيدة لأن جارتها «بعيدة» بوحدات الراتب — والثانية تعطي التقسيم الكامل بـARI=١.٠٠٠. القاعدة لا تقبل الجدل: مرر DBSCAN عبر `sklearn.preprocessing.StandardScaler` (أو `MinMaxScaler` إن كان مقياسك الهندسي مهمّاً)، تماماً كما فعلنا مع SVM في الفصل ٥١.

> **خطأ شائع**
>
> «اضبط eps حتى يختفي الضجيج» وصفة كارثية: كما رأيت في مسح القسم ٨.٤، الاندماج الزائد يُخفي الضجيج تماماً مع تدمير التقسيم. راقب الاثنين معاً: عدد العناقيد ونسبة الضجيج، ثم قيّم بـARI أو Silhouette إن توفر مرجع أو حتى دون مرجع.

> **تحذير**
>
> التسمية `-1` ليست عنقوداً. إن مرّرت `labels_` كما هي إلى `sklearn.metrics.silhouette_score` أو إلى نموذج تصنيف لاحق، فستُحسب الضجيج فئة سادسة وهمية وتفسد كل شيء. استخدم قناعاً: `mask = labels != -1` ثم تعامل مع العينات المصنفة فقط — أو ارفع الضجيج إلى تيار كشف الشذوذ (الفصل ٧٠).

٦.٨ OPTICS: البصمة الكثافية الكاملة في تمريرة واحدة

DBSCAN يعاني من عيب بنيوي: وحدة كثافة واحدة (`eps`) لكل البيانات. وماذا لو كانت بياناتك تضم تجمعات كثيفة وأخرى منثورة؟ ابنِ سيناريو الكثافة المزدوجة: تجمعان كثيفان جدّاً وتجمعان منثوران، مع ٨٠ نقطة ضجيج موزعة بانتظام:

```python
# الكثافة المزدوجة: DBSCAN مقابل OPTICS
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN, OPTICS
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_blobs(n_samples=[500, 500, 90, 90],
                  centers=[[0, 0], [6.5, 0], [0, 9], [6.5, 9]],
                  cluster_std=[0.25, 0.25, 3.0, 3.0], random_state=42)
rng = np.random.default_rng(42)
X_dual = np.vstack([X, rng.uniform(-4, 11, size=(80, 2))])
y_dual = np.concatenate([y, np.full(80, -1)])
mask = y_dual != -1

for eps in [0.5, 1.0, 1.5, 2.0]:
    lab = DBSCAN(eps=eps, min_samples=8).fit(X_dual).labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"DBSCAN eps={eps:.1f}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")

opt = OPTICS(min_samples=8, cluster_method="dbscan", eps=1.5).fit(X_dual)
lab = opt.labels_
k = len(set(lab)) - (1 if -1 in lab else 0)
print(f"OPTICS dbscan eps=1.5: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
      f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")
```

القصة واضحة: عند `eps=0.5` يخسر DBSCAN التجمعين المنثورين كلياً (ضجيج ٢٠.٦%)؛ وعند `1.0` يفتّتهما إلى ٦ قطع؛ وعند `2.0` يذيب كل شيء في عنقود واحد (ARI=٠.٠١٩). هناك نافذة واحدة ضيقة حول `1.5` تنجح — مقامرة بلا ضمانات. OPTICS (Ordering Points To Identify the Clustering Structure، Ankerst وآخرون ١٩٩٩) يحل المشكلة من الجذر: يحسب **اللاح الكثافي (reachability distance)** لكل نقطة نسبياً لسابقتها في ترتيب بناء الكثافة:

$$\text{reach}(p, o) = \max\left(\text{coredist}(o),\ d(o, p)\right)$$

حيث `coredist(o)` هي المسافة التي تجعل النقطة `o` نواة بالضبط (الجار رقم `min_samples` في ترتيب القرب). النتيجة متجه `reachability_` مرتَّب حسب `ordering_`: في الرسم المعروف بمخطط اللاح (reachability plot) تظهر العناقيد «وديااناً» منخفضة يفصلها «حواف» عالية عند فراغات الكثافة — بغض النظر عن مستوى الكثافة نفسه:

قراءة متجه اللاح في OPTICS — python

```python
import numpy as np

reach = opt.reachability_[opt.ordering_]
finite = reach[np.isfinite(reach)]
print(f"طول متجه اللاح: {len(reach)} | أول قيمة: {reach[0]}")
print(f"متوسط اللاح بعد نقطة البداية: {finite.mean():.3f}")
spikes = np.where(finite > 3.0)[0]
print(f"مواضع القفزات الكبرى (3.0 <): {spikes[:12]}")
print(f"أقصى لاح محدود: {finite.max():.3f}")
```

أول قيمة `inf` لأن النقطة الأولى لا سلف لها؛ ثم وديان متتالٍ متوسطه الكلي ~٠.٣١ (الوديان الكثيفة أقرب إلى ٠.٢ والمنثورة إلى ١.٥)، وقفزات تبلغ ٦.٣ في ذيل الترتيب حيث تنزلق نقاط الضجيج الموزعة بعيداً عن كل تجمع. هذا المتجه هو المنتج الحقيقي لـ OPTICS: قاعدة بيانات كاملة عن بنية الكثافة تُبنى مرة واحدة.

> **ملاحظة**
>
> تغيّر واجهة مهم: في الإصدارات القديمة كانت هناك دالة `optics.dbscan_clustering(eps)` تستخرج عناقيد بأسلوب DBSCAN عند أي eps من النموذج المدرَّب نفسه. حُذفت من scikit-learn منذ ١.٣؛ في ١.٩ استخراج العناقيد يتم عند الإنشاء عبر `cluster_method` بقيمتين: `"dbscan"` مع `eps` (كما فعلنا أعلاه)، أو `"xi"` الافتراضية التي تقسم مخطط اللاح تلقائياً حسب انحدار الكثافة ببارامتر `xi`. جرّب أيضاً `min_cluster_size` لقطع العناقيد الصغيرة المزعجة.

الوضوح يقتضي قول الحقيقة عن وضع `"xi"`: على بياناتنا المزدوجة أعطى أفضل نتيجة مضبوطة (`min_samples=20, xi=0.05`) ثلاثة عناقيد بـARI=٠.٧١٤ وضجيج ٥٦% — أفضل من أي DBSCAN في وضع `2.0` لكنه

أدنى من نافذة `eps=1.5` الدقيقة. حساسية `xi` العالية عيّنته ليس مخرجاً مضموناً؛ القيمة الراسخة في مكتبة scikit-learn نفسها هي HDBSCAN الذي سيتولى الفصل ٦٤ المهمة كاملة.

> **سياق**
>
> وُلدت العائلة الكثافية في ورقة Ester وزملائه ١٩٩٦ (DBSCAN) كردّ على قصور K-Means مع الأشكال العشوائية، ثم مدّها Ankerst وزملاؤه ١٩٩٩ (OPTICS) من «تقسيم» إلى «بنية كثافية مرئية». اكتملت القصة عملياً حين دخل HDBSCAN — الذي يقرأ مخطط اللاح تلقائياً — إلى scikit-learn الرسمية.

٨.٧ الكلفة الحسابية: O(n log n) مع شجرة جوار مقابل O(n²) بلاها

حساب كل الأزواج (`algorithm="brute"`) يكلّف O(n²) مسافة، بينما شجرة الجوار (`kd_tree` أو `ball_tree`) تجلب الجيران في O(n log n) في الأبعاد المنخفضة. لنقس الأزمنة على سلسلة أحجام متضاعفة:

الكلفة: brute مقابل kd_tree — python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
import time

Xw, _ = make_blobs(n_samples=2000, centers=6, random_state=42)
DBSCAN(eps=0.3, algorithm="brute", n_jobs=1).fit(Xw)   # تسخين
for n in [5000, 10000, 20000]:
    Xb, _ = make_blobs(n_samples=n, centers=6, random_state=42)
    t0 = time.perf_counter()
    DBSCAN(eps=0.3, algorithm="brute", n_jobs=1).fit(Xb)
    t1 = time.perf_counter()
    DBSCAN(eps=0.3, algorithm="kd_tree", n_jobs=1).fit(Xb)
    t2 = time.perf_counter()
    print(f"n={n:6d} | brute={t1 - t0:5.2f}s | kd_tree={t2 - t1:5.2f}s")
```

لاحظ قاعدة النمو: عند مضاعفة n تتضاعف كلفة brute أربع مرات تقريباً (سلوك تربيعي)، بينما kd*tree يكاد ينمو خطياً — والفجوة تتسع كلما كبرت البيانات. مع ذلك، احذر: قوة شجرة الجوار تتبخر في الأبعاد العالية (فوق ~١٥ بُعداً يصبح التفريع بلا جدوى ويعود brute خياراً منطقياً)، وفي تلك الحالة فكّر في تقليل الأبعاد أولاً كما سيرشدك الفصل ٦٨. وفي البيانات الضخمة فعلياً، راجع خيارات التوازي: `n*jobs=-١` موزّع حساب الجوار على كل الأنوية.

في الواقع

هذه العائلة تشتغل فعلياً في الميدان: تقسيم مناطق جغرافية في أنظمة الخرائط (الشكل حلقي وغير منتظم بطبيعته)، وتحليل نقرات المستخدمين في مكافحة الاحتيال حيث التجمعات كثيفة غير كروية، وتنظيف البيانات قبل أي نمذجة — نسبة الضجيج التي يعطيك إياها DBSCAN مجاناً هي قائمة تنظيف جاهزة. أما بيانات الأبعاد العالية جداً مثل النصوص، فالقاعدة: طبّع، قلّل الأبعاد، ثم اسأل عن الكثافة.

تمارين الفصل الثالث والستين

- **ملاحظة**

التمرين ٨.١

على بيانات `make_moons(n_samples=600, noise=0.15, random_state=42)` مع `eps=0.2` ثابتاً، جرب قيم `min_samples` من ٣ إلى ٢٠. راقب عدد العناقيد ونسبة الضجيج: أي القيمتين أقوى أثراً على «صرامة» الخوارزمية هنا؟

التمرين ٨.٢

ابحث عن نافذة eps الناجحة على `make_circles(n_samples=600, factor=0.45, noise=0.06, random_state=42)`: مسح من ٠.٠٥ إلى ٠.٣٠ بخطوة ٠.٠٥، وسجّل عدد العناقيد وARI عند كل قيمة. ما عرض النافذة التي ينجح فيها DBSCAN؟

التمرين ٨.٣

بدون كود: لماذا قد تتغير تسمية نقطة حدّية بين تشغيل وآخر بعد خلط الصفوف (`shuffle`) مع بقاء العناقيد نفسها؟ وماذا يعني ذلك عملياً لأي قرار يعتمد على «حدود» العناقيد؟

- **ملاحظة**

أثر min_samples على الهلالين المشوشة
```python
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
import numpy as np

X, y = make_moons(n_samples=600, noise=0.15, random_state=42)
for ms in [3, 5, 10, 20]:
    lab = DBSCAN(eps=0.2, min_samples=ms).fit(X).labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"min_samples={ms:2d} | عناقيد={k} | ضجيج={(lab == -1).mean():.1%}")
```

عند القيم المنخفضة (٣ و٥) يلتصق الهلالان في عنقود واحد بضجيج شبه معدوم، لأن الجسور الضجيجية بينهما تحصل على نوى بسهولة. وعند ٢٠ تنكسر هذه الجسور: تفقد النقاط الجسرية نواتها فينفصل الهلالان (مع صدع إضافي في أحدهما) ويقفز الضجيج إلى ٨%. الدرس أن eps يتحكم في نصف القطر وmin_samples في الكثافة المطلوبة — وأن رفعه قد يعالج الالتصاق قبل أن يبدأ بقتل النوى الحقيقية.

> **ملاحظة**

نافذة eps على الحلقتين
```python
from sklearn.datasets import make_circles
from sklearn.cluster import DBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_circles(n_samples=600, factor=0.45, noise=0.06, random_state=42)
for eps in np.arange(0.05, 0.31, 0.05):
    lab = DBSCAN(eps=float(eps), min_samples=5).fit(X).labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"eps={eps:.2f} | عناقيد={k} | ARI={adjusted_rand_score(y, lab):.3f}"
          f" | ضجيج={(lab == -1).mean():.1%}")
```

النافذة الناجحة ضيقة ودقيقة: من ٠.١٥ إلى ٠.٢٠ يعطي DBSCAN العنقودين بـARI=١.٠٠٠ كامل وبدون أي ضجيج، بينما ٠.١٠ ينتهك الحلقة الداخلية (ARI=٠.٥٩٩ مع ٢٤ عنقوداً زائفاً عند ٠.٠٥)، و٠.٢٥ فما فوق يذيب الحلقتين في عنقود واحد (ARI=٠.٠٠٠). الحلقة الغادرة أن الضجيج هنا لا يظهر في النافذة الصحيحة إطلاقاً — عرض النافذة يخصّ كل بيانات، والتجربة هي الطريقة الوحيدة لمعرفته.

> **ملاحظة**

٨.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تشغيل DBSCAN على بيانات خام بمقاييس مختلطة	ضجيج ١٠٠% أو اندماج كامل لأن eps يقيس بوحدة الميزة المهيمنة فقط	`StandardScaler().fit_transform(X)` قبل `DBSCAN` دوماً
معاملة التسمية `-1` كعنقود سادس	تعطّل `silhouette_score` أو انحراف المقاييس والنماذج اللاحقة	قناع `labels != -1` قبل أي حساب، واعزل الضجيج للفصل ٧٠
استدعاء `db.predict(X_new)`	فوري: `AttributeError` DBSCAN لا يملك predict أصلاً	صنّف الجديد بأقرب نواة عبر `NearestNeighbors` مع `db.components_`
تثبيت `eps=0.5` الافتراضي على أي بيانات	تفتيت أو اندماج صامت كما في مسح القسم ٨.٤	مسح eps مع مراقبة عدد العناقيد والضجيج معاً، أو منهج k-distance

٨.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
جوار إبسيلون	قرص نصف قطره eps حول نقطة، هو أساس كل تعريفات الخوارزمية	epsilon-neighborhood
نقطة نواة	نقطة يبلغ جارها min_samples عينة على الأقل، وتولد العناقيد	core point
نقطة حدّية	نقطة غير نواة تقع في جوار نواة فتلتصق بعنقودها دون أن توسّعه	border point
الضجيج	عينات لا نواة لها ولا تلمس نواة؛ تسميتها -١	noise
اللاح الكثافي	أقصى بين مسافة النواة للسلف والمسافة المباشرة؛ وديان OPTICS	reachability distance

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٤ — HDBSCAN: عدد كتل غير معروف مسبقاً

تقدير الكثافات المتغيرة والثبات وتدرّجات الكثافة

الوحدة **scikit-learn ١.٩** الحجم **١٠١٥** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تفسر لماذا يفشل DBSCAN على كثافات متفاوتة بتجربة ARI، وكيف ينجح HDBSCAN دون أي eps
- أن تضبط min_cluster_size و min_samples بوعي وفهم لأثر كل منهما على الضجيج والاندماج
- أن تختار بين 'cluster_selection_method='eom و 'leaf' وفق هدفك من التقسيم
- أن تقرأ probabilities_ كانتماء ناعم وكأداة أولية لكشف الشذوذ
- أن تعرف حدود الخوارزمية: العينات الكبيرة والأبعاد العالية والحاجة للتطبيع وتقليل الأبعاد

٨.١ بطاقة تعريف عضو جديد في العائلة

ملاحظة

HDBSCAN ليس حزمة خارجية في هذا الكتاب: إنه مقدّر رسمي داخل `sklearn.cluster.HDBSCAN`. انضم إلى scikit-learn في الإصدار ١.٣ بعد سنوات من العيش كحزمة مستقلة شهيرة، وفي الإصدار ١.٩ الذي يعتمده هذا الكتاب هو مواطن من الدرجة الأولى بواجهة scikit-learn كاملة: `get_params`، و`fit`، و`fit_predict`، ودعم `n_jobs`.

الفكرة باختصار محموم: بدل أن تسأل المستخدم «أين تنتهي الكثافة؟» (eps)، تبني الخوارزمية **شجرة تدرّج الكثافة (density tree)** — كل مستوى فيها يقابل قيمة eps افتراضية، من الفوضى الكاملة عند القيم الضخمة إلى التفتيت الكامل عند القيم الصغيرة — ثم تختار من الشجرة العناقيد **الأكثر ثباتاً (stability)** عبر المساحة كلها. النتيجة: لا تسأل عنه، ولا تعلنه كما في K-Means (الفصل ٦٠). لنفتش بطاقة التعريف عملياً:

بطاقة تعريف HDBSCAN في 1.9 python

```python
from sklearn.cluster import HDBSCAN
from sklearn.datasets import make_blobs
import numpy as np

hd = HDBSCAN(copy=True)
print("1.9 افتراضيات:", {k: hd.get_params()[k] for k in
      ["min_cluster_size", "min_samples", "cluster_selection_epsilon",
       "cluster_selection_method", "metric"]})

X, y = make_blobs(n_samples=400, centers=3, cluster_std=0.8, random_state=42)
hd.fit(X)
lab = hd.labels_
print(f"عناقيد مكتشفة: {np.unique(lab).tolist()}")
print(f"مدى probabilities_: {hd.probabilities_.min():.3f} إلى
{hd.probabilities_.max():.3f}")
```

ثلاث ملاحظات من البطاقة: افتراضي `min_cluster_size` هو ٥ (عدواني؛ سنعالجه قريباً)، و `min_samples=None` له دلالة خاصة سنشرحها، والبارامتر الوحيد الذي اضبطناه صراحة هو `copy` — سنفسر سبب ذلك في تنبيه نهاية الفصل.

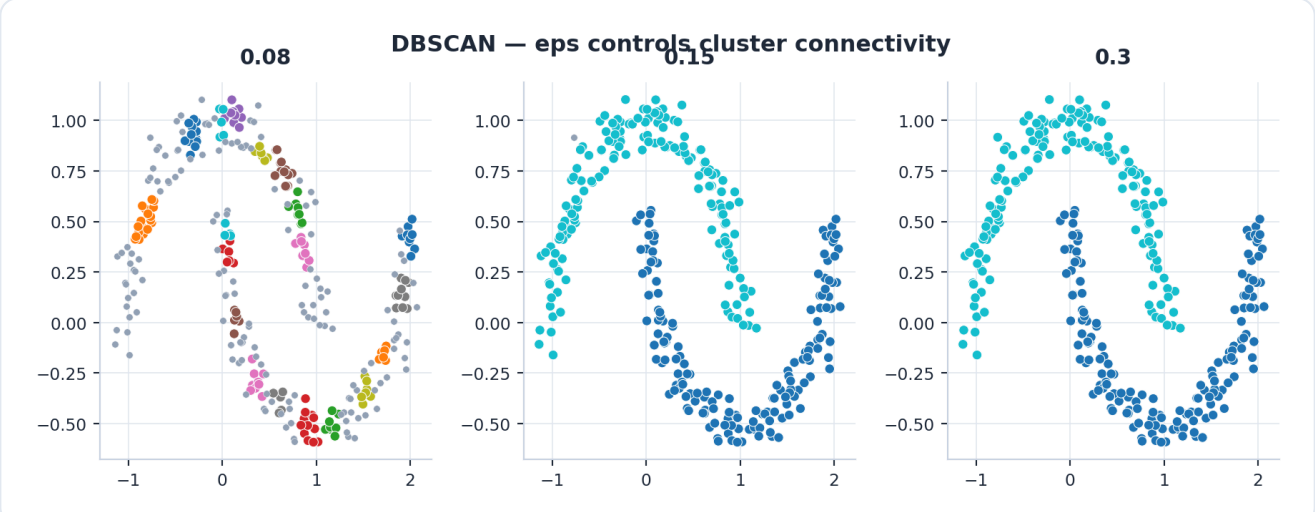
> **مفتاح**
>
> المفهوم المحوري: HDBSCAN لا يختار تقسيماً عند eps واحد، بل يمسح **كل** قيم eps دفعة واحدة عبر شجرة الكثافة، ويقطف العناقيد التي صمدت أطول عمرٍ كثافي. لهذا ينجح حيث تنقسم النتائج بين eps صغير و eps كبير.

٨.٢ المشكلة: وحدة eps واحدة لا تكفي أبداً

لنرجع سيناريو الفصل السابق: تجمعان كثيفان وتجمعان منثوران و٨٠ نقطة ضجيج. DBSCAN أمام هذا المشهد بوسم واحد: صغير يفقد المنثور، كبير يذيب الكثيف. HDBSCAN لا يُسأل أصلاً:

HDBSCAN في كل الاتجاهات مقابل DBSCAN

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN, HDBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_blobs(n_samples=[500, 500, 90, 90],
                  centers=[[0, 0], [6.5, 0], [0, 9], [6.5, 9]],
                  cluster_std=[0.25, 0.25, 3.0, 3.0], random_state=42)
rng = np.random.default_rng(42)
X_dual = np.vstack([X, rng.uniform(-4, 11, size=(80, 2))])
y_dual = np.concatenate([y, np.full(80, -1)])
mask = y_dual != -1

for eps in [0.5, 1.0, 1.5, 2.0]:
    lab = DBSCAN(eps=eps, min_samples=8).fit(X_dual).labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"DBSCAN eps={eps:.1f}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")

hd = HDBSCAN(min_cluster_size=15, copy=True).fit(X_dual)
lab = hd.labels_
k = len(set(lab)) - (1 if -1 in lab else 0)
print(f"HDBSCAN (بلا eps): عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
      f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")
```



الشكل ١٣٨: حافة eps الخطرة: تحت العتبة ضجيج وتفتيت، وفوقها اندماج يبتلع الفجوات

اقرأ سلوك DBSCAN كسلسلة خسائر: `0.5` يفقد ٢٠.٦% من البيانات كضجيج (المنثور تبخر)؛ `1.0` يفتّت المنثور إلى ست قطع؛ `1.5` نجاح ضيق جداً؛ `2.0` كارثة كاملة (ARI=٠.٠١٩) — كل شيء في عنقود واحد. أما HDBSCAN فقد أعاد تقسيماً بجودة أعلى بضجيج أقل، **دون أن نمرر له نصف قطر** — وجاء هذا بلا أي مسح يدوي لـ eps.

٨.٣ الآلية: مسافة الوصول المتبادلة وشجرة الكثافة

كيف يفعلها؟ ثلاث خطوات يمكن تعلمها في دقيقة. أولاً لكل نقطة **مسافة النواة** (core distance): المسافة إلى الجار رقم `min_samples` بالترتيب — أي عتبة الكثافة المحلية للنقطة:

$$\text{core}_k(p) = d\left(p,\ \text{NN}_k(p)\right)$$

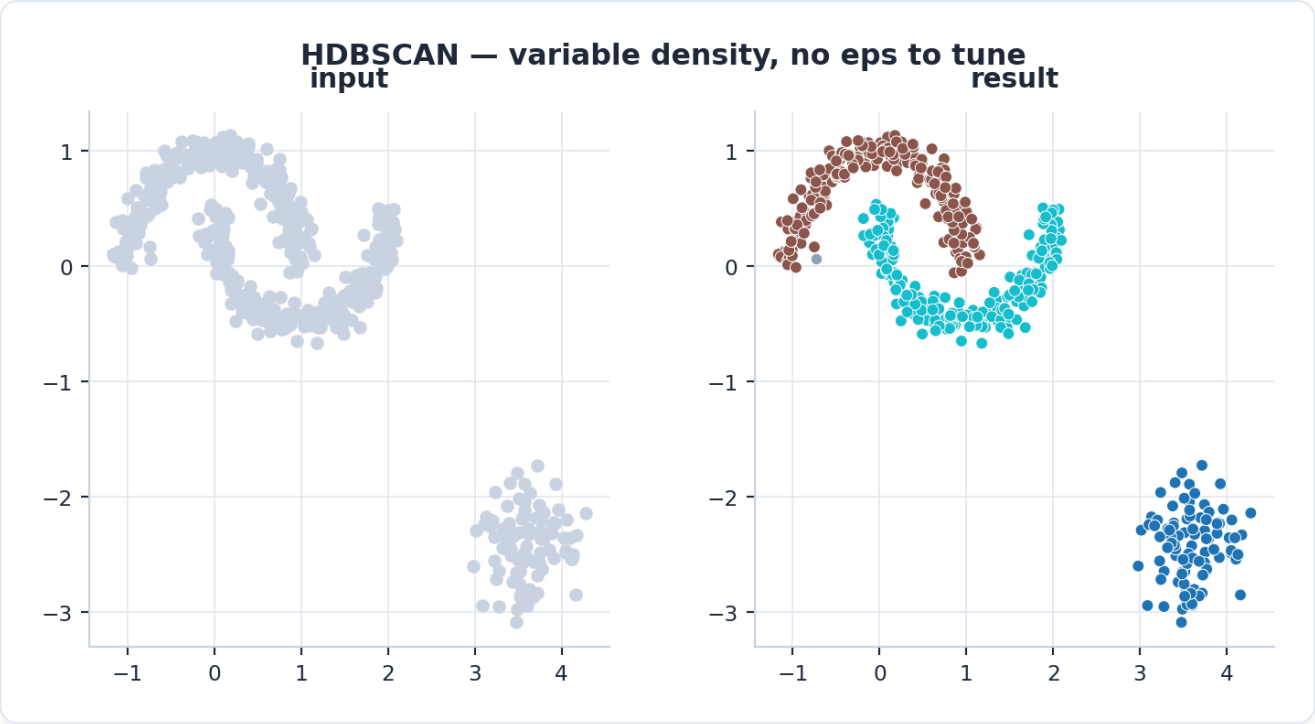
حيث `NN_k(p)` الجار رقم k للنقطة p بعد نفسها. في منطقة كثيفة تكون هذه المسافة صغيرة؛ في فراغ تكون ضخمة. ثانياً يعاد تعريف المسافات الزوجية بـ **مسافة الوصول المتبادلة (mutual reachability)**:

$$d_{\text{mreach}}(a, b) = \max\left(\text{core}_k(a),\ \text{core}_k(b),\ d(a, b)\right)$$

أي: لا يجوز لاتصال أن يكون «أرخص» من كثافة الطرفين — وهذا يجعل مقياس المسافة كله يعبّر عن الكثافة لا عن المسافة الخام. ثالثاً يبنى شجرة الاندماج الترابطي (single-linkage) على هذه المسافات، ثم **يُكثَّف** (condensed tree): تُقطع الفروع التي تنهار كثافتها بسرعة (اندماجات زائفة)، ويبقى هيكل تدرّج الكثافة الحقيقي. ثم يأتي اختيار العناقيد: لكل نقطة عُمر كثافي

$$\lambda_p = \frac{1}{\text{core}_k(p)}$$

و**ثبات العنقود** هو مجموع ما زاد عُمر نقاطه عن عُمر ميلاده — والعنقود الذي يكسب أكبر مجموع ثبات يُختار أولاً. هذه هي طريقة `eom` (اختصار Excess of Mass).



الشكل ١٣٩ العناقيد المستخرجة على بيانات كثافة متفاوتة دون أي eps يدوي

لاحظ في الشكل كيف تُحتضن الكتل ذات الكثافات المختلفة دفعة واحدة — السلوك الذي حاربنا من أجله في القسم ٨.٢.

٨.٤ بارامترا الحجم: min_samples و min*cluster*size

`min_cluster_size` هو أهم بارامتر في HDBSCAN: أصغر عدد نقاط تُعتبر به المجموعة عنقوداً مشروعاً. قيم صغيرة تجيز العناقيد الصغيرة؛ قيم كبيرة تلتهمها أو تدمجها بما جاورها:

مسح min_cluster_size على الكثافة المزدوجة — python

```python
from sklearn.cluster import HDBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

for mcs in [5, 15, 30, 60]:
    hd = HDBSCAN(min_cluster_size=mcs, copy=True).fit(X_dual)
    lab = hd.labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"min_cluster_size={mcs:3d}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")
```

سلسلة تعليمية كاملة: عند ٥ تُقبل كل الجزئيات (٤ عناقيد)؛ عند ١٥ و٣٠ يذوب التجمعان المنثوران في عنقود واحد متسامح؛ وعند ٦٠ — وهي أكثر من نصف حجم أي تجمع منثور (٩٠ نقطة) — يُبتلع نصف البيانات المنثورة كضجيج (١٩.٥%). القاعدة العملية: `min_cluster_size` يسأل «كم أصغر كتلة تهمني؟» — ابدأ بنحو ١٠ إلى ٥٠ حسب حجم بياناتك، وقارن النتائج عند ضعف القيمة ونصفها.

أما `min_samples` فهو صمام التحفظ: كم نقطة يجب في جوار النقطة كي نثق بكثافتها. الافتراضي `None` يعني أخذ قيمة `min_cluster_size` نفسها — تحفظ كبير:

أثر min_samples على الضجيج python

```python
from sklearn.cluster import HDBSCAN
import numpy as np

for ms in [None, 5, 15, 30]:
    hd = HDBSCAN(min_cluster_size=15, min_samples=ms, copy=True).fit(X_dual)
    lab = hd.labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    ms_txt = "None" if ms is None else str(ms)
    print(f"min_samples={ms_txt:>5}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")
```

لاحظ الاتجاه: قيمة صغيرة (٥) تريح الخناق فيهبط الضجيج إلى ٣.٩%؛ والقيم الكبيرة ترفعه تدريجياً (٧.٥% ثم ٩.٠%) لأن النقاط الحدّية تفقد ثقتها بالكثافة. وإذا أردت وصفة واحدة مقبولة عموماً: `min_samples=5` إلى `min_samples=15` كنقطة بداية، ثم ارفعها إذا ظهرت عناقيد زائفة.

> **خطأ شائع**
>
> `min_samples=None` لا تعني «بلا شرط» بل «انسخ min*cluster*size». من يتركها None ثم يرفع min*cluster*size إلى ١٠٠ يجعل شرط الكثافة المحلية صارماً جداً دون قصد — فإذا أردت استقلال القرارين، فحدد min_samples صراحة دائماً.

٨.٥ طريقتا الاختيار: eom مقابل leaf

`cluster_selection_method` يقرر من أين في الشجرة المكثفة تُنتقى العناقيد:

```python
# على الكثافة المزدوجة leaf مقابل eom
from sklearn.cluster import HDBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

for method in ["eom", "leaf"]:
    hd = HDBSCAN(min_cluster_size=15, cluster_selection_method=method,
copy=True).fit(X_dual)
    lab = hd.labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"method={method:6s}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y_dual[mask], lab[mask]):.3f}")
```

`eom` الافتراضية تختار أوسع العناقيد ثباتاً — وهي الترجمة الحرفية لفلسفة «الأكثر صموداً». أما `leaf` فتأخذ أوراق الشجرة كلها: أصغر العناقيد الطرفية، فمهما صغرت. على بياناتنا أعطت leaf ستة عناقيد و٧٠% ضجيج — دقة تفصيلية عالية بثمن باهظ. متى تختار leaf؟ حين يكون هدفك تفتيحاً دقيقاً (تحليل سلوك عملاء إلى شرائح دقيقة جداً) وتقبل دفع ثمن الضجيج، لا حين تكون الحقيقة أربع كتل متوازنة.

وبقي بارامتر ثالث للضبط الدقيق: `cluster_selection_epsilon` يدمج العناقيد الأقرب من المسافة المحددة في الشجرة — أداة «وصل الشتات» عندما تفرط leaf في التفتيت أو تظهر عناقيد هامشية مقبولة الوجود:

```python
# وصل العناقيد الصغيرة يدوياً
hd = HDBSCAN(min_cluster_size=15, cluster_selection_epsilon=0.5, copy=True)
# كل عناقيد أقرب من 0.5 في الشجرة المكثفة تُدمج قبل الإخراج
```

> **تعمّق**
>
> لماذا لا يخزن sklearn شجرة الاختيار النهائية كخاصية عامة؟ لأن العقد المكثفة بنية داخلية تختلف حسب `algorithm`. إن أردت فحص البنية نفسها، فالشجرة الترابطية الخام محفوظة في `_single_linkage_tree_` (خاصية خاصة تخضع للتغيير بين الإصدارات) — استخدمها للتشخيص فقط لا لبناء كود إنتاجي عليها.

٨.٦ الانتماء الناعم: probabilities_

هنا تتفوق HDBSCAN على كل ما سبق في هذا الكتاب: كل عينة تحصل على قيمة في `probabilities_` بين ٠ و١ تعبّر عن قوة انتمائها لعنقودها — مقياس مستمر لا تسمية جامدة:

تشريح _probabilities python

```python
from sklearn.cluster import HDBSCAN
import numpy as np

hd = HDBSCAN(min_cluster_size=15, copy=True).fit(X_dual)
lab = hd.labels_
for c in sorted(set(lab)):
    sel = lab == c
    p = hd.probabilities_[sel]
    if c == -1:
        print(f"(1-) ضجيج: n={int(sel.sum())} | probabilities من {p.min():.3f} إلى
{p.max():.3f}")
    else:
        print(f"عنقود {c}: n={int(sel.sum())} | متوسط={p.mean():.3f}"
              f" | أدنى={p.min():.3f} | 0.5 من أقل: {(p < 0.5).mean():.1%}")
```

ثلاث قراءات عملية: نقاط الضجيج كلها بقيمة ٠.٠ بالضبط (لا انتماء إطلاقاً)؛ قيم العنقود المنثور كلها فوق ٠.٦٨ (كثافته المتجانسة تمنح ثقة عالية)؛ بينما يضم كل عنقود كثيف نحو خُمس نقاطه بقيمة دون ٠.٥ — هذه هي الحدود الداخلية والنقاط المشتركة بين العناقيد. هذا الانتماء الناعم يجيب على سؤال ما بعد التجميع الأهم: «ما مدى ثقتي بأن هذه النقطة تنتمي فعلاً؟» — ويمكنك فلترة العناقيد بحد أدنى من الثقة قبل أي استخدام لاحق.

تحذير

قيمة احتمالية عالية لا تعني صحة العنقود نفسه؛ بل ثقة الخوارزمية بالانتماء داخل بنيتها الكثافية. لا تستخدم `probabilities_` كبديل عن احتمالات نموذج مدرَّب (كما في الفصل ٥٣)، ولا تقبل نقاطاً حدّية بقيم ضئيلة في تقارير حساسة دون فحص يدوي.

وأما **كشف الشذوذ كأثر جانبي**: النقاط التي تظل بلا عنقود أو بقيمة انتماء معدومة هي بالتعريف أغرب نقاط البيانات — ترتيب البيانات بـ `probabilities_` تصاعدياً يمنحك قائمة شكوك أولية مجانية. الحزمة الكاملة للكشف المنهجي عن الشذوذ (IsolationForest و LOF وغيرهما) في الفصل ٧٠.

في الواقع

هذا المزيج — عناقيد بلا k معلنة، وانتماء ناعم، وقائمة شكوك مجانية — هو سبب انتشار HDBSCAN في تقسيم العملاء وتحليل البيانات الجغرافية ومراقبة الأنظمة: تحصل على شرائح ودرجة ثقة وقائمة شكوك من تمريرة واحدة، ثم تقرر أنت ما يستحق النظر.

٨.٧ الاستقرار: الميزة العملية الكبرى

في الفصل السابق كان تغيير eps بمقدار ٠.٣ كافياً لينقلنا من النجاح إلى الاندماج الكامل. HDBSCAN بلا eps أصلاً — لكن دعنا نقيس الثبات بدل أن نرتل: نغيّر eps من ٠.٣ إلى ٣.٠ ونراقب ARI، مع HDBSCAN ثابتاً في كل المشهد:

```python
# ثبات النتائج: مسح eps مقابل HDBSCAN
from sklearn.cluster import DBSCAN, HDBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

print("DBSCAN عبر مساحة eps:")
for eps in [0.3, 0.5, 1.0, 1.5, 2.0, 3.0]:
    lab = DBSCAN(eps=eps, min_samples=8).fit(X_dual).labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"  eps={eps:.1f}: عناقيد={k} | ARI={adjusted_rand_score(y_dual[mask],
lab[mask]):.3f}")

hd = HDBSCAN(min_cluster_size=15, copy=True).fit(X_dual)
print(f"HDBSCAN (بلا eps): ARI={adjusted_rand_score(y_dual[mask],
hd.labels_[mask]):.3f}")
```

هذا هو جوهر القيمة العملية: DBSCAN يعطي ARI جيداً في نطاق ضيق ثم ينهار إلى ٠.١٩ ثم ٠.٠٠٦، بينما HDBSCAN إجابة واحدة ثابتة لا تعتمد على أي نصف قطر تخمّنه. في بيئة إنتاجية حيث تتغير البيانات بين يوم وآخر، إعادة معايرة eps بلا نهاية عبء حقيقي — والخوارزمية التي تستهلكها توفّر وقتاً هندسياً مستمراً.

٨.٨ الحدود: العينات الكبيرة والأبعاد العالية

قوتها لها ثمن حسابي: بناء مسافات الوصول المتبادلة وشجرة الاندماج يكلف عند تضاعف عدد العينات أكثر من الضعف بمرات (نحو ضعفين إلى ثلاثة لكل مضاعفة في تجاربنا على بيانات ثنائية الأبعاد). استخدم `n_jobs=-1` من البداية، وفكّر في أخذ عينة قبل التجميع إذا تجاوزت البيانات مئات الآلاف. وفي الأبعاد العالية تنهار فكرة «الجوار» نفسها (لعنة الأبعاد من الفصل ٥١) فتحتاج معالجة مسبقة: تطبيع إجباري مع `StandardScaler` أو `MinMaxScaler` — لأن مسافات الوصول تعيش بوحدات بياناتك — ثم تقليل أبعاد هادف بـ PCA كما في الفصل ٦٨:

الأرقام العالية: digits خام مقابل PCA

```python
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.cluster import HDBSCAN
from sklearn.metrics import adjusted_rand_score

Xd, yd = load_digits(return_X_y=True)
hd_raw = HDBSCAN(min_cluster_size=15, copy=True).fit(Xd)
lab_raw = hd_raw.labels_
k_raw = len(set(lab_raw)) - (1 if -1 in lab_raw else 0)
print(f"عناقيد :(بعداً 64) خام={k_raw} | ضجيج={(lab_raw == -1).mean():.1%}"
      f" | ARI={adjusted_rand_score(yd, lab_raw):.3f}")

Xp = PCA(n_components=0.90, random_state=42).fit_transform(Xd)
hd_pca = HDBSCAN(min_cluster_size=15, copy=True).fit(Xp)
lab_pca = hd_pca.labels_
k_pca = len(set(lab_pca)) - (1 if -1 in lab_pca else 0)
print(f"PCA ({Xp.shape[1]} بعداً): عناقيد={k_pca} | ضجيج={(lab_pca == -1).mean():.1%}"
      f" | ARI={adjusted_rand_score(yd, lab_pca):.3f}")
```

الصورة الأكثر واقعية مما نظن: في ٦٤ بُعداً تعثرت الخوارزمية (نصف البيانات ضجيج، و١١ عنقوداً زائفاً لعشر فئات حقيقية)، وبعد إسقاط يحفظ ٩٠% من التباين تحسّن الجودة وتقلصت نسبة الضجيج. لا تجري تجميعاً كثافياً على أبعاد خام عالية دون إسقاط — ثم تذكّر أن التجميع هنا غير مشرف إطلاقاً: أرقام الحقيقة هنا للتقييم فقط كما سنفصّل في الفصل ٦٦.

نصيحة

ثبّت `copy=True` صراحة في كل استدعاء: الافتراضي الحالي `copy=False` سيتحول إلى `True` في الإصدار ١.١٠، scikit-learn ١.٩ يطلق FutureWarning على كل fit لا يحدد القيمة — وكتابنا (وأنت أيضاً) يريد مخرجات نظيفة من التحذيرات.

تمارين الفصل الرابع والستين

- **ملاحظة**

التمرين ٨.١

أنشئ `make_blobs(n_samples=[400, 300, 60], cluster_std=[0.5, 0.8, 1.2], random_state=42)` مع ٤٠٠ نقطة ضجيج موزعة بانتظام، ثم امسح `min_cluster_size` على القيم ١٠ و٢٥ و٥٠: عند أي قيمة تُكتشف الكتل الثلاث؟ ومتى يبدأ التجميع بتناول الكتلة الصغيرة كضجيج؟

التمرين ٨.٢

قارن `make_moons(n_samples=600, noise=0.05, random_state=42)` على `cluster_selection_method='eom'` مع `'leaf'`: كم عنقوداً تعطي كل طريقة، وأي التقسيمين يطابق الحقيقة؟

التمرين ٨.٣

بدون كود: إذا كان HDBSCAN يبني شجرة الكثافة من مساحة eps كلها، فلماذا لا نتائجه تتغير عندما نغيّر أي eps؟ وما البارامتر الذي يفعل ذلك فعلاً — ولماذا؟

- **ملاحظة**

مسح min_cluster_size على ثلاث كتل متفاوتة python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import HDBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_blobs(n_samples=[400, 300, 60], cluster_std=[0.5, 0.8, 1.2],
                  random_state=42)
rng = np.random.default_rng(42)
X3 = np.vstack([X, rng.uniform(-12, 12, size=(40, 2))])
y3 = np.concatenate([y, np.full(40, -1)])
mask = y3 != -1

for mcs in [10, 25, 50]:
    hd = HDBSCAN(min_cluster_size=mcs, copy=True).fit(X3)
    lab = hd.labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"mcs={mcs:3d}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y3[mask], lab[mask]):.3f}")
```

نتيجة مفاجئة ومطمئنة معاً: عند ١٠ و٢٥ و٥٠ تبقى الكتل الثلاث مكتشفة كاملة (ARI=١.٠٠٠) بينما يزحف الضجيج صعوداً بهدوء من ٢.٢% إلى ٣.٢% — لأن أصغر كتلة (٦٠ نقطة) ما تزال أعلى من كل هذه العتبات بوضوح. هذا بالضبط درس القسم ٨.٤: لا تصبح العتبة خطراً إلا حين تقترب من حجم الكتلة الصغيرة نفسها (تذكّر ما فعله ٦٠ بتجمعات ٩٠ نقطة هناك). القاعدة العملية: اختر min*cluster*size أدنى بوضوح من أصغر كتلة تهتم بها، وراقب الضجيج كجرس إنذار مبكر.

ملاحظة

على الهلالين مقابل leaf eom python

```python
from sklearn.datasets import make_moons
from sklearn.cluster import HDBSCAN
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_moons(n_samples=600, noise=0.05, random_state=42)
for method in ["eom", "leaf"]:
    hd = HDBSCAN(min_cluster_size=15, cluster_selection_method=method,
                 copy=True).fit(X)
    lab = hd.labels_
    k = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"{method:5s}: عناقيد={k} | ضجيج={(lab == -1).mean():.1%}"
          f" | ARI={adjusted_rand_score(y, lab):.3f}")
```

على بنية بسيطة كالهلالين تعطي `eom` العنقودين الطبيعيين بجودة تكاد تامة، بينما تفتّت `leaf` كل وادٍ فرعي في شجرة الكثافة إلى عدة أوراق فيزيد العدد ويهبط ARI. الاختيار إذن ليس ذوقياً: `eom` للتقسيم الطبيعي، و`leaf` حين تحتاج التفاصيل الدقيقة وتقبل دفع ثمنها.

> **ملاحظة**

٨.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
ترك `min_samples=None` مع رفع *mincluster**size* كبير**	تصار الكثافة المحلية صرامة غير مقصودة ويقفز الضجيج	حدد `min_samples` صراحة (٥-١٥ كنقطة بداية)
ترك `copy` بلا قيمة	FutureWarning في ١.٩ عند كل fit (الافتراضي يتغير في ١.١٠)	مرر `copy=True` صراحة في كل استدعاء
استخدام `cluster_selection_method='leaf'` افتراضياً	تفتيت مفرط: ٦ عناقيد و٧٠% ضجيج على بيانات من ٤ كتل	ابق على `eom` إلا إذا كان تفصيل الأوراق هو هدفك
تجميع أبعاد خام عالية مباشرة	نصف البيانات ضجيج وعناقيد زائفة (تجربة digits: ARI=٠.٢١٧)	`StandardScaler` ثم PCA إلى ~٩٠% تباين قبل HDBSCAN
قراءة `probabilities_` كاحتمالات نموذج تصنيف	ثقة بنية داخلية تُستخدم كتنبؤ خارجي	استخدمها للفرز والفلترة، والأحكام النهائية لنماذج مدرّبة

٨.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مسافة النواة	المسافة إلى الجار رقم min_samples؛ عتبة الكثافة المحلية	core distance
مسافة الوصول المتبادلة	أقصى بين مسافتي نواة الطرفين والمسافة المباشرة	mutual reachability distance
الشجرة المكثفة	شجرة الاندماج بعد قصّ الفروع ذات الانهيار السريع للكثافة	condensed tree
ثبات العنقود	مجموع فائض الأعمار الكثافية لنقاطه؛ معيار الاختيار في eom	cluster stability
الانتماء الناعم	قيمة ٠–١ لكل عينة تعبّر عن قوة عضويتها في عنقودها	soft membership (`probabilities_`)

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٥ — التجميع الهرمي والطوبولوجي

FeatureAgglomeration g Spectral g Ward g AgglomerativeClustering

الوحدة **I.٩ scikit-learn** الحجم **٩٤٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح عائلات الربط الأربع وشكل العناقيد الذي ينجح فيه كل منها بتجربة ARI
- أن تبني وتقرأ دندروغراماً عبر scipy.cluster.hierarchy وتقطعه بـmaxclust أو بحد مسافة
- أن تجعل AgglomerativeClustering يكتشف عدد الفئات تلقائياً عبر distance_threshold
- أن تستخدم قيود الاتصال على الصور FeatureAgglomerationg على الأعمدة المترابطة
- أن تقرر متى يتفوق SpectralClustering — بجدول جامع للفصول ٦٠ إلى ٦٥

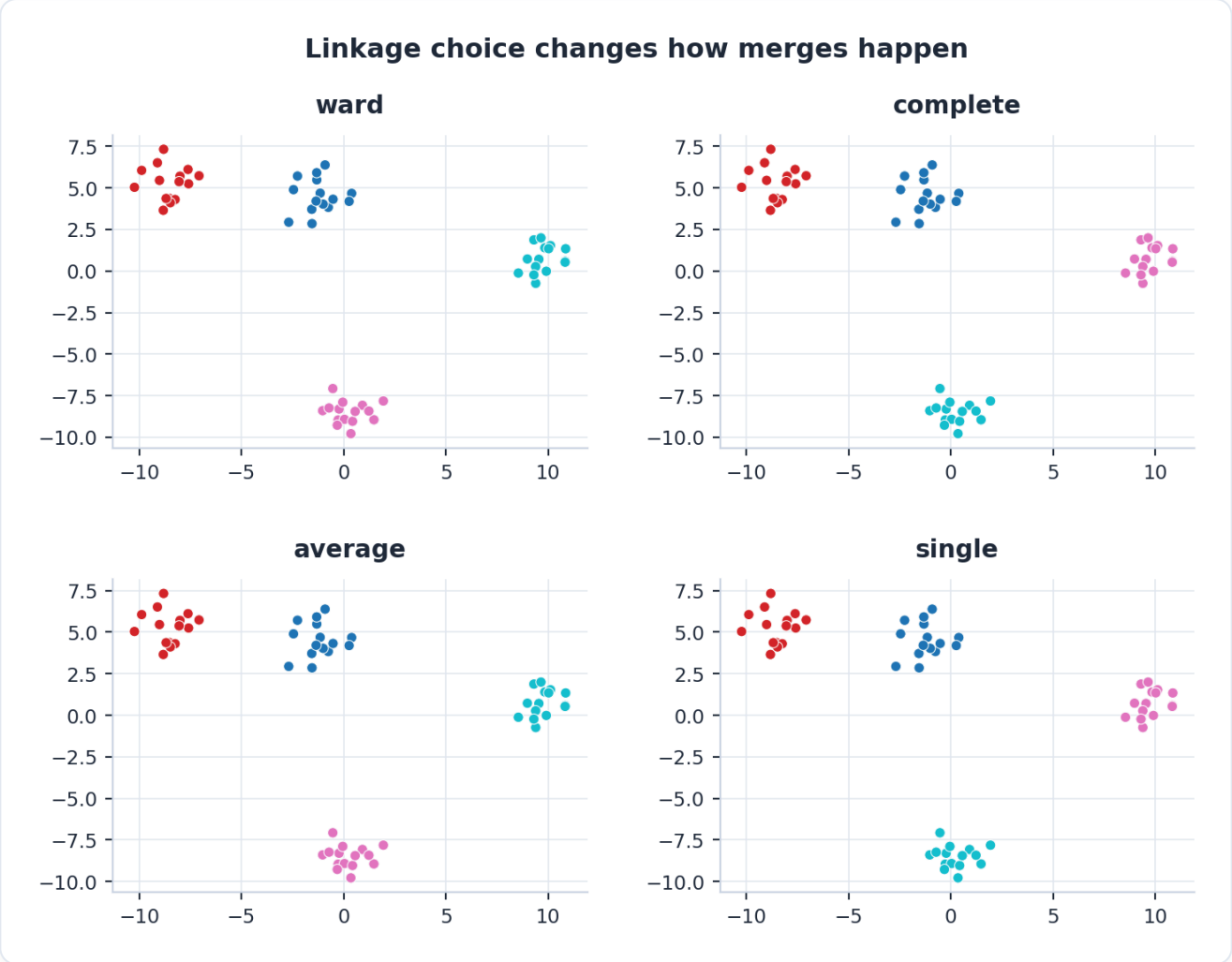
٨.١ الاندماج من الأسفل: شجرة تُبنى نقطة نقطة

كل ما رأيناه في هذا الجزء يبدأ من «مركز» أو من «كثافة». العائلة التجميعية (agglomerative clustering) تبدأ من أبسط افتراض: **كل نقطة عنقود بحد ذاتها**، ثم يُدمج أقرب عنقودين تباعاً حتى يتبقى ما تريد. لا مركزاً يُحدّث ولا عتبة تُمسح — فقط سؤال متكرر: «ما أقرب عنقودين الآن؟»، والمسافة بين عنقودين هي ما يحدد شخصية الخوارزمية كلها.

في عائلة **Ward** تحديداً — الافتراضية في sklearn — يُختار الاندماج الذي يرفع التباين الداخلي الكلي أقل رفع:

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \, \|\mathbf{c}_A - \mathbf{c}_B\|_2^2$$

حيث `n_A` و`n_B` عدد النقاط و`c_A` و`c_B` المركزان: دمج عنقودين كبيرين بعيدين مكلف، ودمج تجمعين صغيرين متلاصقين رخيص — فتنمو الكتل المتجانسة أولاً ويتأجل عبور الفجوات إلى آخر الشجرة.



الشكل ١٤٠خطوات الاندماج عبر أربعة أنماط ربط تنتج أشجاراً مختلفة الشكل

> **مفتاح**
>
> المفهوم المحوري: لا تقسيماً واحداً بل **شجرة كاملة** من التقسيمات، من n عنقوداً إلى واحد. الربط يحدد شكل العناقيد، ومكان القطع في الشجرة يحدد العدد — قراران منفصلان تماماً.

٨.٢ عائلات الربط الأربع: من يحب أي شكل؟

الدالة التي تقيس المسافة بين عنقودين تسمى **الربط (linkage)**، ولها أربع قيم في `AgglomerativeClustering`. ثلاث منها تقيس المسافة بين النقاط مباشرة:

$$L_{\mathrm{single}}(A, B) = \min_{a \in A,\; b \in B} d(a, b) \qquad L_{\mathrm{complete}}(A, B) = \max_{a \in A,\; b \in B} d(a, b)$$

$$L_{\text{average}}(A,B) = \frac{1}{n_A n_B} \sum_{a \in A} \sum_{b \in B} d(a,b)$$

كل اختيار يمنح خوارزمية شخصية مختلفة: `single` يمسك بسلسلة النقاط المتقاربة فيلتقط الأشكال الشريطية والحلقية؛ و`complete` يخشى أي عنقود واسع فيفضّل الكتل المتراصة الصغيرة؛ و`average` حسمٌ بينهما؛ و`ward` يقلّص التباين الداخلي فيزدهر على الكرات. لنجسد هذا على ثلاث هندسات: هلالان، وحلقتان متداخلتان، وبقع كروية:

جدول الربط الأربع على ثلاث هندسات python

```
from sklearn.datasets import make_moons, make_circles, make_blobs
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score

datasets = {
    "الهلالان": make_moons(n_samples=600, noise=0.05, random_state=42),
    "الحلقتان": make_circles(n_samples=600, factor=0.45, noise=0.05,
random_state=42),
    "الكرات": make_blobs(n_samples=600, centers=3, cluster_std=1.0,
random_state=42),
}
for name, (X, y) in datasets.items():
    row = [f"{name:8s}"]
    for lk in ["ward", "complete", "average", "single"]:
        lab = AgglomerativeClustering(n_clusters=2 if name != "الكرات" else 3,
                                      linkage=lk).fit_predict(X)
        row.append(f"{lk}={adjusted_rand_score(y, lab):.3f}")
    print(" | ".join(row))
```

النتيجة كتاب مدرسي مصغّر: على الكرات ينجح الجميع — المشكلة السهلة لا تميّز أحداً. وعلى الحلقتان ينهار ward وcompleteg averageg وينجح single وحده بـARI=١.٠٠٠ لأن كل حلقة «سلسلة» نقاط متتالية. لكن لـ single وجه مظلم: مع الضجيج يعمل «التصاق السلسلة» (chaining) فيجسّر عبر النقاط المضروبة حتى يذيب كتلتين حقيقيتين — عد إلى الهلالان بضجيج ٠٫٨ في الفصل السابق لترى انهياره إلى ٠٫٠٠٠. القاعدة: اعرف شكل بياناتك أولاً ثم اختر الربط.

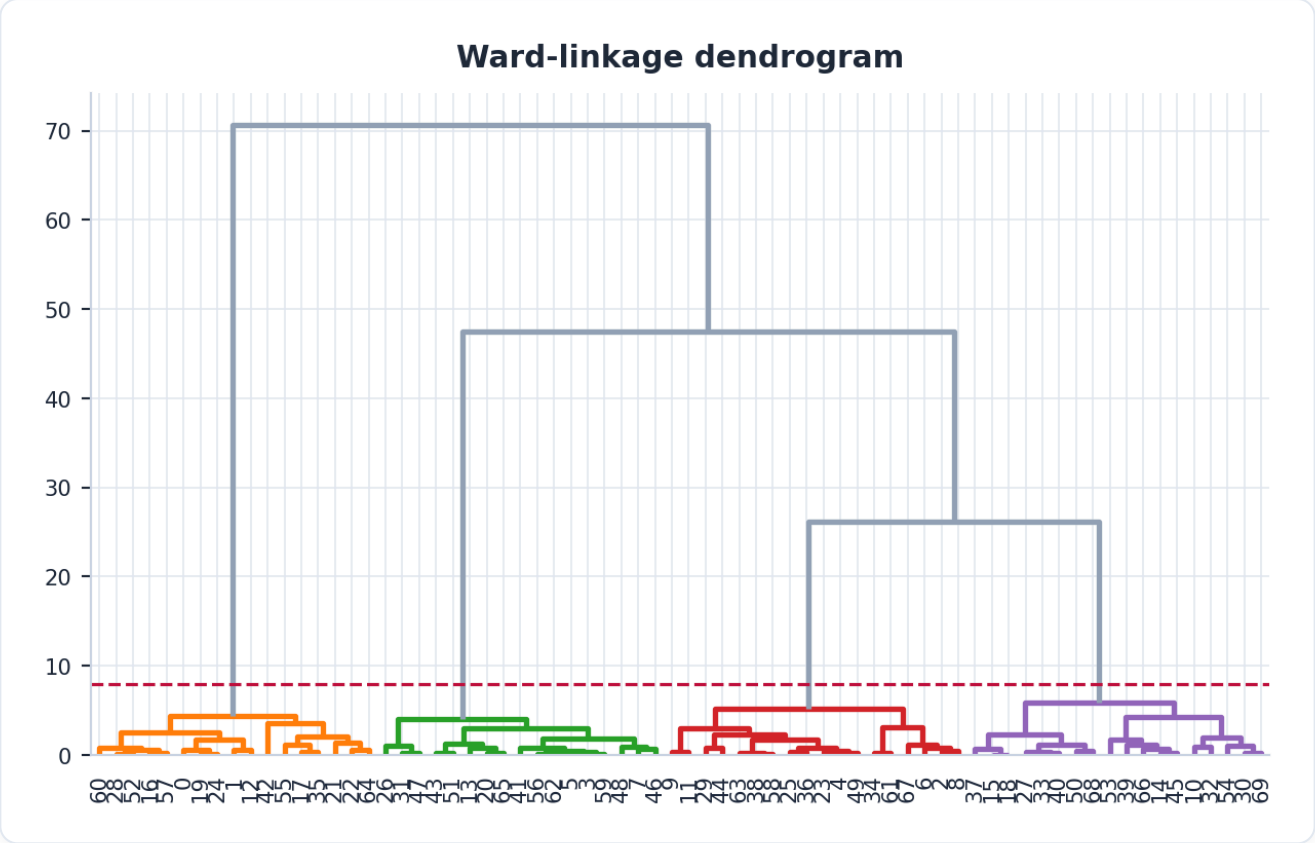
> **ملاحظة**
>
> تغيّر واجهة عبر الإصدارات: كان بارامتر المسافة يُسمى `affinity` ثم صار `metric` (أُهمل الاسم القديم في ١.٢ وأُزيل لاحقاً). في ١.٩ استخدم `metric` حصراً، وward لا يقبل إلا المسافة الإقليدية.

٨.٣ الدندروغرام: قراءة شجرة الاندماج مع scipy

sklearn يعطيك التقسيم النهائي، لكن scipy يعطيك الشجرة كاملة: `scipy.cluster.hierarchy.linkage` يحسب مصفوفة الاندماجات (سطر لكل اندماج: العنقودان، الارتفاع، الحجم)، و`dendrogram` يبني بنية العرض بـ `no_plot=True` كما سنعمل — والرسم حكر على أشكال الكتاب المسجلة:

شجرة الاندماج: linkage ثم قطعها بـfcluster python

```python
from sklearn.datasets import make_moons
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sklearn.metrics import adjusted_rand_score
import numpy as np

X, y = make_moons(n_samples=150, noise=0.05, random_state=42)
Z = linkage(X, method="ward")
print(f"شكل Z: {Z.shape} (سطر لكل اندماج من أصل {len(X) - 1})")
print(f"أول اندماج: {np.round(Z[0], 3)}")
print(f"آخر ثلاثة ارتفاعات: {np.round(Z[-3:, 2], 2)}")

R = dendrogram(Z, no_plot=True)
print(f"أوراق الدندروغرام: {len(R['leaves'])}")

for t in [2.0, 3.0, 4.0, 6.0]:
    fc = fcluster(Z, t=t, criterion="distance")
    print(f"قطع عند ارتفاع {t}: {len(np.unique(fc))} عناقيد")

ag = AgglomerativeClustering(n_clusters=2, linkage="ward").fit_predict(X)
fc = fcluster(Z, t=2, criterion="maxclust")
print(f"fcluster(maxclust=2) مقابل sklearn: ARI={adjusted_rand_score(ag,
fc):.3f}")
```



الشكل ١٤١دندروغرام ward: أطوال الأذرع تكشف الفجوة الطبيعية التي نقع عندها

اقرأ الناتج بحرص: قفزة الارتفاع من ٥.٨١ إلى ١٣.٤٥ في آخر اندماجين تعني «عبور فجوة» حقيقية — والمكان الطبيعي للقطع تحتها مباشرة حيث نجد الهلالين. هذه طريقة «القفزة الكبرى»: اقطع عند أكبر فرق بين ارتفاعين متتاليين. ولاحظ أن `fcluster(maxclust=2)` يعطي التقسيم نفسه الذي يعطيه sklearn (ARI=١.٠٠٠) — بتسميات تبدأ من ١ لا من ٠.

٨.٤ عدد الفئات تلقائياً: distance_threshold

الآن نعمّم فكرة القطع: بدل `n_clusters` تعلن **حد مسافة** (`distance_threshold`) فيوقف sklearn الاندماج حين يتجاوز أرخص اندماج متاح الحد — فيكتشف العدد بنفسه:

```python
# اكتشاف عدد الفئات تلقائياً بحد المسافة
from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering

X, y = make_blobs(n_samples=600, centers=3, cluster_std=1.0, random_state=42)
for th in [5, 8, 12, 18, 25]:
    ag = AgglomerativeClustering(n_clusters=None, distance_threshold=th,
                                 linkage="ward", compute_full_tree=True).fit(X)
    print(f"حد={th:2d} | عناقيد={ag.n_clusters_} | اندماجات={len(ag.distances_)}")
```

سلسلة مثالية: حد ٥ يفتّت إلى ٢١ عنقوداً؛ وحد ١٨ يجد الثلاثة بالضبط؛ و٢٥ يعيدها لأن الشجرة اكتملت قبله. شرطان إجباريان: `n_clusters=None` و`compute_full_tree=True` (وإلا توقفت الشجرة مبكراً وكسر منطق العتبة). وتصلح هذه الخاصية حين يكون لهامش الفصل استقرار — ولقياس كمي للعتبة راجع Silhouette في الفصل ٦٦.

٨.٥ قيود الاتصال: لا تدمج إلا الجيران

افتراض صامت خطير: أي عنقودين يمكن دمجهما مهما تباعدا. على الصور والبيانات المكانية هذا هرج — البكسلات تتجمّع بالجوار. قيد الاتصال (connectivity constraint) يفرض مصفوفة جوار: من يُسمح له بالاندماج فقط. للنقاط العامة ابنِها بـ `sklearn.neighbors.kneighbors_graph(X, n_neighbors=10, include_self=False)`؛ وعلى شبكة بكسل أبسط قيد هو جيران الأربعة الذين نبنيهم يدوياً ثم نمررهم لـ ward:

قيد الاتصال على صورة: أربعة أرباع مقابل ward العائم python

```python
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score
from scipy.sparse import coo_matrix
import numpy as np

rng = np.random.default_rng(42)
h, w = 40, 40
img = np.zeros((h, w))
img[:20, :20] = rng.normal(2.0, 1.2, size=(20, 20))
img[:20, 20:] = rng.normal(8.0, 1.2, size=(20, 20))
img[20:, :20] = rng.normal(8.0, 1.2, size=(20, 20))
img[20:, 20:] = rng.normal(2.0, 1.2, size=(20, 20))
y_img = np.zeros((h, w), dtype=int)
y_img[:20, 20:] = 1
y_img[20:, :20] = 2
y_img[20:, 20:] = 3
y_img = y_img.ravel()

i = np.arange(h * w)
r_i, c_i = np.divmod(i, w)
src = np.r_[i[c_i < w - 1], i[r_i < h - 1]]
dst = np.r_[i[c_i < w - 1] + 1, i[r_i < h - 1] + w]
conn = coo_matrix((np.ones(len(src)), (src, dst)), shape=(h * w, h * w)).tocsr()
conn = conn + conn.T
print(f"حواف شبكة الجيران: {conn.nnz // 2}")

X_pix = img.ravel().reshape(-1, 1)
lab_c = AgglomerativeClustering(n_clusters=4, linkage="ward",
                                connectivity=conn).fit_predict(X_pix)
lab_f = AgglomerativeClustering(n_clusters=4, linkage="ward").fit_predict(X_pix)

def contiguity(lab):
    L = lab.reshape(h, w)
    return ((L[:, :-1] == L[:, 1:]).mean() + (L[:-1, :] == L[1:, :]).mean()) / 2

print(f"بقيد الاتصال: ARI={adjusted_rand_score(y_img, lab_c):.3f} | تماسك مكاني=
{contiguity(lab_c):.1%}")
print(f"بلا قيد:      ARI={adjusted_rand_score(y_img, lab_f):.3f} | تماسك مكاني=
{contiguity(lab_f):.1%}")
```

ward بلا قيد قسّم البكسلات حسب شدتها فجمع الداكن من الربعين المتقابلين في عنقود متطاير (تماسك ٤٨.٦%)؛ وبقيد الجيران اضطر للبناء بالجوار فاستخرج الأرباع الأربعة (ARI=٠.٩٩٨ وتماسك ٩٧.٤%). هذا هو الفرق بين التجميع حسب القيمة والتجميع حسب القيمة مع احترام الخريطة — أساس تقسيم الصور التجميعي.

٨.٦ FeatureAgglomeration: تجميع الأعمدة لا الصفوف

حتى الآن جمعنا الصفوف. نفس الآلية تعمل عمودياً: `sklearn.cluster.FeatureAgglomeration` يدمج **الأعمدة المتشابهة** في عمود مجمّع (بالمتوسط عبر `pooling_func`) — محوّل كامل بـ `transform` و`inverse_transform` يدخل في pipeline:

```python
# تجميع أعمدة digits: أداء كامل بضعة أعمدة
from sklearn.datasets import load_digits
from sklearn.cluster import FeatureAgglomeration
from sklearn.linear_model import RidgeClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

Xd, yd = load_digits(return_X_y=True)
Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(Xd, yd, test_size=0.3,
                                              random_state=42, stratify=yd)
clf = RidgeClassifier().fit(Xd_tr, yd_tr)
print(f"RidgeClassifier الدقة على 64 عموداً: ={accuracy_score(yd_te,
clf.predict(Xd_te)):.4f}")

for k in [32, 16]:
    pipe = make_pipeline(FeatureAgglomeration(n_clusters=k), RidgeClassifier())
    pipe.fit(Xd_tr, yd_tr)
    acc = accuracy_score(yd_te, pipe.predict(Xd_te))
    print(f"FeatureAgglomeration({k}) + RidgeClassifier: الدقة={acc:.4f}")

fa = FeatureAgglomeration(n_clusters=4).fit(Xd_tr)
print(f"مجموعات الأعمدة (أول 20 بكسل): {fa.labels_[:20]}")
Xt = np.asarray(fa.transform(Xd_tr))
print(f"شكل التحويل: {Xt.shape} | العودة: {fa.inverse_transform(Xt).shape}")
```

على بكسلات digits — حيث يتشابه الجيران المتصلون بقوة — تجميع ٦٤ عموداً إلى ٣٢ يخفض زمن التدريب نحو النصف ويكاد لا يمس الدقة (٠.٩١٦٧ مقابل ٠.٩٣٣٣)، أما ١٦ عموداً فتبدأ في كي المعلومة. هذا هو الاستخدام المشروع: **أعمدة مترابطة زائدة الحمل**.

> **تحذير**
>
> لا تفترض أن التجميع «ضغط مجاني» أينما طُبّق. على بيانات مثل fetch*california*housing حيث يحمل كل عمود معلومة مستقلة (الدخل، خط العرض، عمر البيت...)، أعطى `FeatureAgglomeration(n_clusters=6)` قبل Ridge خطأ RMSE ٠.٧١٢ مقابل ٠.٥٣٦ للنموذج المباشر — مزج إشارات مختلفة تماماً وأسقط المعلومة. قاعدة: جمّع ما تشترك فيه الأعمدة فعلاً (بكسلات متجاورة، قياسات متكررة)، ولا تجمّع إشارات مستقلة.

ومن الأناقة أن يوضع المحوّل في خط أنابيب انحدار كامل — `make_pipeline(FeatureAgglomeration(n_clusters=k), Ridge())` — فيصبح عدد المجموعات بارامتراً يُضبط بالتحقق المتقاطع مثل أي بارامتر نموذج.

٨.٧ SpectralClustering: من نقاط إلى رسم بياني

العائلة الأخيرة تعيد صياغة التجميع كلياً: ابنِ **رسم بياني للتشابه** (نقطة عُقدة، وقربٌ وزن الحافة)، ثم اقطعه عند أضعف اتصال عبر **القطع الطيفي (spectral cut)**. الميزة الجذرية: لا يهم في الرسم إلا الاتصال المحلي، فالهلال والحلقة شكلان عاديان تماماً:

على الهلالين والحلقتين SpectralClustering — python

```python
from sklearn.datasets import make_moons, make_circles
from sklearn.cluster import SpectralClustering
from sklearn.metrics import adjusted_rand_score

Xm, ym = make_moons(n_samples=600, noise=0.05, random_state=42)
Xc, yc = make_circles(n_samples=600, factor=0.45, noise=0.05, random_state=42)
lab_m = SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                           n_neighbors=20, random_state=42).fit_predict(Xm)
lab_c = SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                           n_neighbors=40, random_state=42).fit_predict(Xc)
print(f"الهلالان: ARI={adjusted_rand_score(ym, lab_m):.4f}")
print(f"الحلقتان: ARI={adjusted_rand_score(yc, lab_c):.4f}")

for al in ["kmeans", "discretize", "cluster_qr"]:
    lab = SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                             n_neighbors=20, assign_labels=al,
                             random_state=42).fit_predict(Xm)
    print(f"assign_labels={al:10s}: ARI={adjusted_rand_score(ym, lab):.4f}")
```

بارامترات القرار: `affinity` يبني الرسم البياني (`nearest_neighbors` للمكتظ، أو `rbf` مع `gamma`، أو `precomputed` لمصفوفة خاصة)؛ `n_components` عدد المتجهات الذاتية في التضمين (افتراضياً يساوي *nclusters*)؛ *g`randomstate* يعيد إدخال عشوائية يهدّئها *kmeans* ؛ تتمايز حين يشتوش التضمين – وتذكّر أن(ARI=1.000) الصري

لكن انتبه للفاتورة: الرسم البياني الكامل مصفوفة n×n:

كلفة المصفوفة n في n — python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import SpectralClustering
import time

for n in [1200, 3600]:
    Xn, _ = make_blobs(n_samples=n, centers=3, random_state=42)
    t0 = time.perf_counter()
    sp = SpectralClustering(n_clusters=3, affinity="rbf",
random_state=42).fit(Xn)
    t1 = time.perf_counter()
    print(f"n={n:5d} | زمن={t1 - t0:5.2f}s | شكل affinity_matrix_:
{sp.affinity_matrix_.shape}")
```

ثلاثة أضعاف في n كفت لتضاعف الزمن بمرات — نمو يذكّر بـ O(n²) لبناء المصفوفة وتفكيكها. لهذا يبقى SpectralClustering سلاح الأشكال غير الكروية الصغيرة والمتوسطة، لا أداة البيانات المليونية.

٨.٨ الجدول الجامع: كل تجميعات الفصول ٦٠ إلى ٦٥

لنختم بتجربة على الهلالين بضجيجها الحقيقي (٠.٠٨)، ثم بالجدول الشامل:

الختام الكبير: ست خوارزميات على الهلالين python

```python
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN, HDBSCAN,
SpectralClustering
from sklearn.metrics import adjusted_rand_score

X, y = make_moons(n_samples=600, noise=0.08, random_state=42)
models = [
    ("KMeans(n_clusters=2)", lambda: KMeans(n_clusters=2, n_init=10,
random_state=42).fit_predict(X)),
    ("Agglomerative(ward)", lambda: AgglomerativeClustering(n_clusters=2,
linkage="ward").fit_predict(X)),
    ("Agglomerative(single)", lambda: AgglomerativeClustering(n_clusters=2,
linkage="single").fit_predict(X)),
    ("DBSCAN(eps=0.15)", lambda: DBSCAN(eps=0.15).fit_predict(X)),
    ("Spectral(nn=20)", lambda: SpectralClustering(n_clusters=2,
affinity="nearest_neighbors", n_neighbors=20, random_state=42).fit_predict(X)),
    ("HDBSCAN(mcs=10)", lambda: HDBSCAN(min_cluster_size=10,
copy=True).fit_predict(X)),
]
for name, fn in models:
    print(f"{name:24s} ARI={adjusted_rand_score(y, fn()):.4f}")
```

الخوارزمية (الفصل)	يطلب k؟	الشكل المناسب	حساسية المعاملات	الكلفة التقريبية	متى تختاره
K-Means (الفصل ٦٠)	نعم	كرات متوازنة	متوسطة	O(nkd) تكرارياً	بيانات كروية كبيرة وسريعة
MiniBatchKMeans (الفصل ٦١)	نعم	كرات متوازنة	متوسطة	O(bkd) لكل دفعة	ملايين الصفوف مع تقريب مقبول
MeanShift / AffinityPropagation (الفصل ٦٢)	لا	كتل بحدود كثافة	عالية جداً	مرتفعة	بيانات صغيرة وعدد كتل مجهول
DBSCAN (الفصل ٦٣)	لا	أشكال عشوائية متجانسة الكثافة	عالية (eps ضيقة)	O(n log n) بشجرة جوار	كثافات متقاربة وضجيج معلوم
OPTICS (الفصل ٦٣)	لا	كما DBSCAN + تدرّجات	عالية (xi)	O(n log n)	تشخيص بنية الكثافة قبل التقسيم
HDBSCAN (الفصل ٦٤)	لا	عشوائية بكثافات متفاوتة	منخفضة	من شبه خطية إلى تربيعية	كثافات متفاوتة وثقة انتماء وشذوذ
Agglomerative (هذا الفصل)	نعم (k أو عتبة)	حسب الربط	منخفضة	حتى $O(n^3)$	شجرة كاملة، عينات قليلة، قيود اتصال
SpectralClustering (هذا الفصل)	نعم	أي شكل متصل	متوسطة	مصفوفة n×n	أشكال غير كروية وبيانات صغيرة
FeatureAgglomeration (هذا الفصل)	نعم	تجميع أعمدة مترابطة	منخفضة	كAgglomerative	تقليل أبعاد الأعمدة داخل pipeline

درس الختام: لا خوارزمية تفوز خارج سياقها. الهلالان بضجيج ٠.٠٨ أطاح بـ K-Means (٠.٢٥٢) وward (٠.٣٠٩) وsingle الذي أحب الهلالين النظيفين (٠.٠٠٠ بسبب التصاق الضجيج)، بينما صمد DBSCAN (٠.٩٨٣) وHDBSCAN (٠.٩٧٧) وتفاجأ Spectral (٠.٦١٨ لأن الضجيج أضعف اتصال الرسم البياني). ولقياس أي تقسيم بلا إجابات، طريقك الفصل ٦٦.

> **في الواقع**
>
> هذه العائلة تشتغل في الميدان حيث الأشكال ليست كرات: تقسيم صور الأقمار الصناعية بقيود الاتصال على شبكة البكسل، وأشجار التصنيف الحيوية في البيولوجيا (الدندروغرام هو منتج البحث نفسه)، وتقليل أعمدة المستشعرات المترابطة في إنترنت الأشياء قبل نماذج الانحدار.

تمارين الفصل الخامس والستين

- **ملاحظة**

التمرين ٨.١

على `load_wine` المطبَّعة بـ `StandardScaler`، استخدم `AgglomerativeClustering(n_clusters=None, distance_threshold=..., linkage="ward")` وامسح العتبة من ٦ إلى ١٤ بخطوة ٢: عند أي عتبة تُكتشف الفئات الثلاث المعروفة؟ وما ARI عند تلك العتبة؟

التمرين ٨.٢

أنشئ `make_regression(n_samples=800, n_features=40, n_informative=5, random_state=42)` ثم قارن RMSE بالتحقق الخماسي بين `Ridge` المباشر و `FeatureAgglomeration(n_clusters=20) + Ridge`: هل التجميع ضغط مجاني أم كيٌّ معلومة؟ ولماذا عكس ذلك على digits؟

التمرين ٨.٣

بدون كود: لماذا ينجو single من الحلقات المتداخلة وينكسر على الهلالين المضروبة بالضجيج؟ وما علاقة قيد الاتصال بالحالتين؟

- **ملاحظة**

اكتشاف فئات wine تلقائياً python

```python
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score

X, y = load_wine(return_X_y=True)
Xs = StandardScaler().fit_transform(X)
for th in [6, 8, 10, 12, 14]:
    ag = AgglomerativeClustering(n_clusters=None, distance_threshold=float(th),
                                 linkage="ward", compute_full_tree=True).fit(Xs)
    ari = adjusted_rand_score(y, ag.labels_)
    print(f"حد={th:2d} | عناقيد={ag.n_clusters_} | ARI={ari:.3f}")
```

عند العتبة ١٤ تُكتشف الفئات الثلاث بARI يبلغ ٠.٧٩٠، بينما العتبات الأدنى تفتّت wine إلى ١٩ و١٢ و٩ عناقيد. العتبة أداة تفسير بقدر ما هي أداة اكتشاف: فئات wine متمايزة كيميائياً لكن ليست مفصولة بفجوة هائلة، فيحتاج ward عتبة سخية ليتوقف عند الثلاثة.

> **ملاحظة**

على إشارات مستقلة FeatureAgglomeration python

```python
from sklearn.datasets import make_regression
from sklearn.cluster import FeatureAgglomeration
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score

X, y = make_regression(n_samples=800, n_features=40, n_informative=5,
                       random_state=42)
base = -cross_val_score(Ridge(), X, y, cv=5,
                        scoring="neg_root_mean_squared_error").mean()
pipe = make_pipeline(FeatureAgglomeration(n_clusters=20), Ridge())
agg = -cross_val_score(pipe, X, y, cv=5,
                       scoring="neg_root_mean_squared_error").mean()
print(f"Ridge (40 عموداً) مباشر: RMSE={base:.3f}")
print(f"FeatureAgglo(20) + Ridge: RMSE={agg:.3f}")
```

هنا يتدهور الأداء بوضوح: الأعمدة المعلوماتية الخمسة إشارات مستقلة مزجها التجميع بالضجيج فأسقط جزءاً من الإشارة — النقيض تماماً لبكسلات digits المتجاورة المتشابهة. تجميع الأعمدة يربح حين يكون التشابه بين الأعمدة حقيقياً، ويخسر حين يكون اصطناعياً.

> **ملاحظة**

٨.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام "linkage="ward مع metric غير إقليدي	ValueError: ward لا يعمل إلا مع المسافة الإقليدية	ward مع `metric="euclidean"` فقط، أو غيّر الربط إلى average/complete
إعلان *distance_threshold* مع *نسيان* *compute_full_tree*	قد تتوقف الشجرة مبكراً فيكسر منطق العتبة والعدد خاطئ	`n_clusters=None` مع `compute_full_tree=True` دائماً
تطبيق FeatureAgglomeration على أعمدة إشاراتها مستقلة	RMSE يتدهور (٠.٥٣٦ إلى ٠.٧١٢ على California) لأن الإشارات مزجت	جمّع الأعمدة المترابطة فعلاً فقط، وقيّم بالتحقق المتقاطع
بناء SpectralClustering على عشرات آلاف النقاط	ذاكرة وزمن هائلان بسبب مصفوفة n×n	عيّنة أو `nearest_neighbors` بدل `rbf` الكامل، أو خوارزمية أخرى
مقارنة تسميات fcluster مع sklearn بـ == مباشرة	ترقيم العناقيد يبدأ من ١ في scipy ومن ٠ في sklearn	قارن بالقسمة عبر `adjusted_rand_score` لا بقيم التسميات

٨.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التجميع التجميعي	البدء بنقاط منفصلة ودمج أقرب عنقودين تباعاً	agglomerative clustering
الربط	الدالة التي تقيس المسافة بين عنقودين (ward/complete/average/single)	linkage
الدندروغرام	تمثيل شجري لكل اندماجات التجميع بارتفاعاتها	dendrogram
قيد الاتصال	مصفوفة جوار تمنع اندماج عنقودين غير متجاورين	connectivity constraint
القطع الطيفي	تقسيم رسم بياني للتشابه عبر متجهات المصفوفة اللابلاسية	spectral clustering

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٦ — كيف تقيس نجاح تجميعٍ بلا إجابة؟

NMI و ARI و Calinski-Harabasz و Davies-Bouldin و Silhouette

الوحدة **scikit-learn ١.٩** الحجم **١٠١٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تصنف مقاييس التجميع إلى عائلتين: داخلية تعمل بلا حقيقة وخارجية تستغل الحقيقة المتاحة
- أن تحلل silhouette_samples لكل نقطة لتشخيص العناقيد الضعيفة والمتداخلة
- أن تختار بين Davies-Bouldin و Calinski-Harabasz بوعي باتجاه كل مقياس ونطاقه
- أن تقيم التجميع بـ ARI و NMI و V-measure و FMI مع فهم تصحيح الصدفة
- أن تكتشف لماذا يفشل مقارنة التسميات مباشرة عندما تختلف ترتيباتها

٨.١ قاضٍ بلا شهود: عائلتان من المقاييس

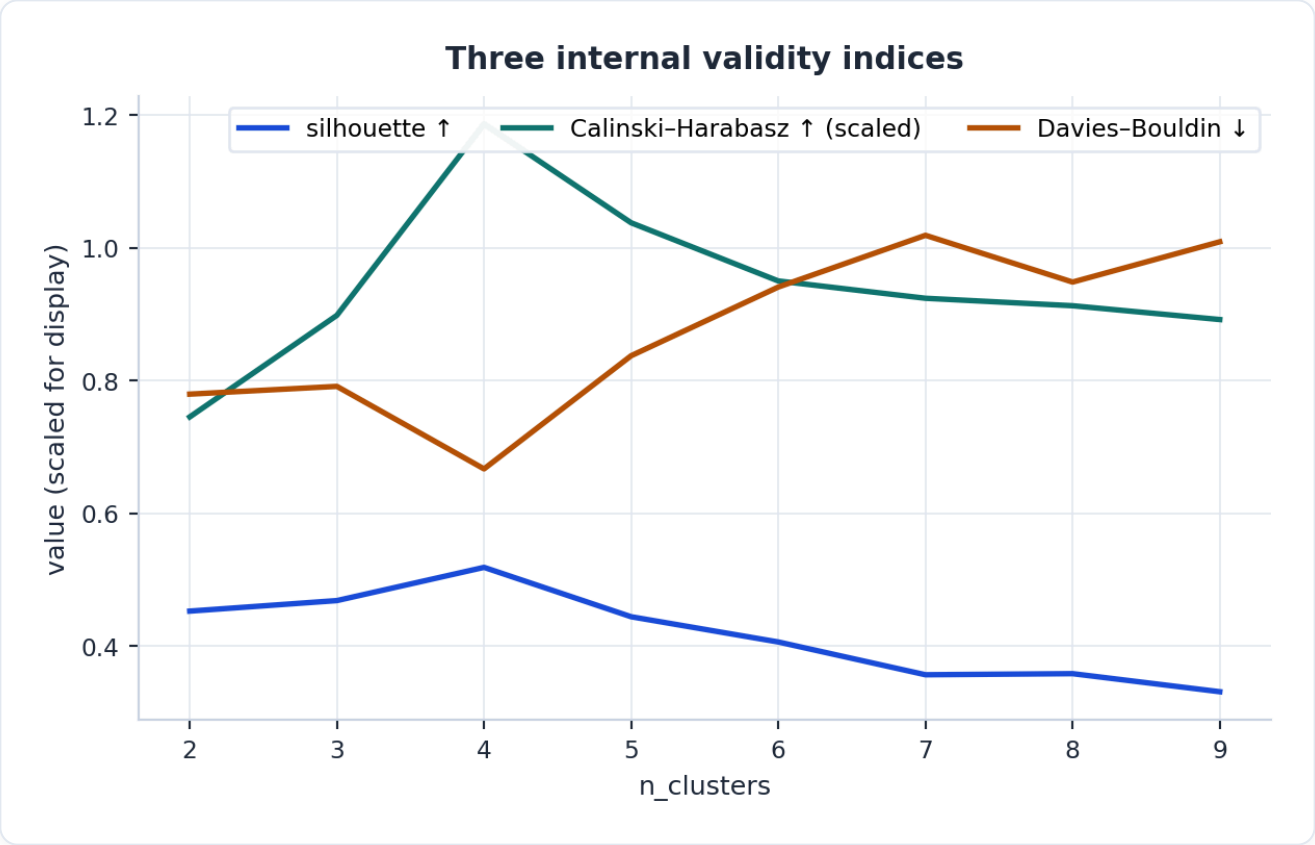
تخيل أنك سلّمت خوارزمية تجميع ٩٠٠ عينة وطلبت منها أن تجد الكتل، ثم عادت إليك وقالت: «وجدت ثلاث مجموعات». سؤالك الأول سيكون طبيعياً: **هل هي مجموعات جيدة؟** لكن هنا ينهار كل ما تعلمته في الأجزاء السابقة؛ فلا توجد تسميات صحيحة تقارن بها، ولا دقة تحسبها كما في الفصل ٣٧. الحكم هنا يحتاج نوعاً مختلفاً من العدالة: قاضٍ يبتني على **بنية البيانات نفسها**، أو يكتفي بشهود جزئيين إذا وفرت الحياة بعض التسميات الحقيقية.

تنقسم المقاييس إلى عائلتين لا تقبلان الخلط:

العائلة	هل تحتاج تسميات حقيقية؟	المقاييس الأشهر	سؤالها الجوهري
داخلية (Internal)	لا — تعتمد على المسافات بين العينات فقط	Silhouette، Davies-Bouldin، Calinski-Harabasz	هل العناقيد متماسكة داخلياً ومنفصلة عن بعضها؟
خارجية (External)	نعم — تقارن التقسيم بحقيقة متاحة	ARI، NMI، V-measure، FMI	هل تطابق تقسيمي التقسيم المرجعي المعروف؟

> **مفتاح**
>
> المقاييس الداخلية تقيس «جودة الهندسة» ولا تعرف شيئاً عن الحقيقة؛ والمقاييس الخارجية تقيس «التطابق مع مرجع» ولا تعرف شيئاً عن الهندسة. النضج في تقييم التجميع هو معرفة العائلة المناسبة لحالتك: بلا تسميات إطلاقاً فلا بديل عن الداخلية، ومع تسميات متاحة فالخارجية أصدق شهوداً.

وهل يمكن أن تخدعنا المقاييس الداخلية حين تبقى وحيدة؟ نعم، وسنرى في هذا الفصل حالة شهيرة تفشل فيها Silhouette كلياً رغم نجاح التجميع. لذا اقرأ الصورة الكاملة أولاً.



الشكل ١٤٢ خريطة عائلتَي مقاييس التجميع الداخلية والخارجية واتجاه كل مقياس

الشكل يلخص البوصلة كلها: إلى اليسار المقاييس الداخلية التي تطبق على أي بيانات مجمّعة، وإلى اليمين الخارجية التي تشترط حقيقة مرجعية، مع سهم صغير يذكرك باتجاه كل مقياس: أعلى أفضل أم أدنى أفضل. احتفظ بهذه الخريطة في ذاكرتك؛ فكل قسم قادم يمر عبرها.

٨.٢ Silhouette: نبض كل نقطة على حدة

كيف نحول «تماسك العنقود وانفصاله» إلى رقم واحد؟ فكرة Silhouette (الظل) التي اقترحها Peter Rousseeuw عام ١٩٨٧ أنيقة إلى حد الجنون: **اسأل كل نقطة عن شعورها**. لكل نقطة `i` احسب قيمتين:

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i,\ j \neq i} d(i, j) \qquad b(i) = \min_{k \neq i} \frac{1}{|C_k|} \sum_{j \in C_k} d(i, j)$$

ثم يجمع المقياس القيمتين في معادلة واحدة تكافئ النقطة إذا كان قربها من عائلتها أكبر من قربها من أقرب جار غريب:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i),\ b(i))}$$

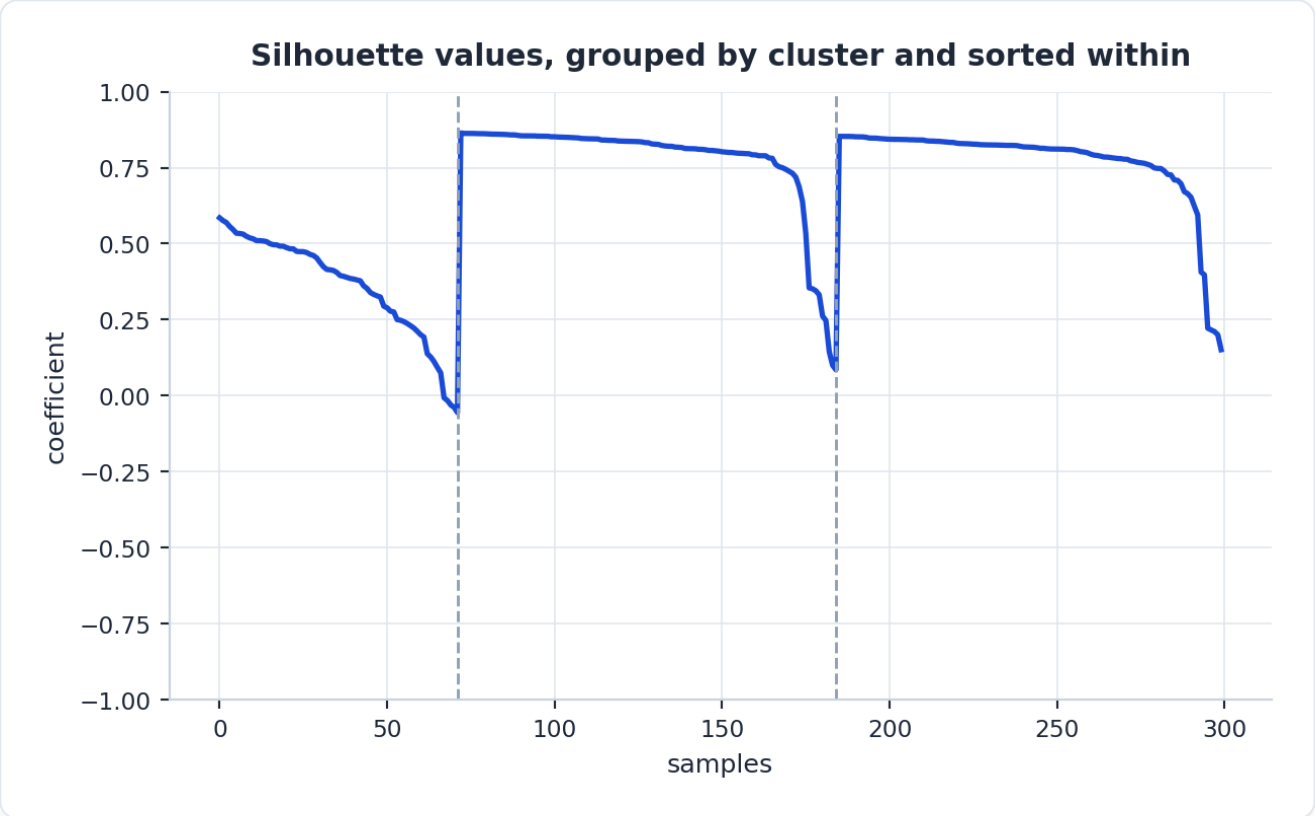
تقع `s(i)` دائماً بين `-1` و `1`؛ فإذا اقتربت من `1` فالنقطة متماسكة بعيدة عن جيرانها الأجانب، وإذا اقتربت من `0` فهي على حدود بين عنقودين، وإذا صارت **سالبة** فالنقطة أقرب إلى عنقود الغريب — أي أن الخوارزمية وضعتها في البيت الخطأ على الأرجح. ودالة `sklearn.metrics.silhouette_score` لا تفعل شيئاً سوى حساب متوسط `s(i)` على جميع النقاط، بينما تعيد `sklearn.metrics.silhouette_samples` المتجه الكامل: قيمة لكل نقطة على حدة. وهذا التفصيل الصغير هو الفرق بين رقم لطيف وتشخيص حقيقي.

جرّب الآن التجربة الأهم: بيانات ثلاثة عناقيد مع رفع مستوى التداخل تدريجياً عبر `cluster_std`، مع حساب المقياس على **التسميات الحقيقية** نفسها:

Silhouette مع تصاعد التداخل بين العناقيد python

```python
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score, silhouette_samples
import numpy as np

for std in [0.6, 1.5, 2.6]:
    X, y = make_blobs(n_samples=600, centers=3, cluster_std=std,
random_state=42)
    s = silhouette_score(X, y)                    # متوسط واحد للكل
    samples = silhouette_samples(X, y)            # قيمة لكل نقطة
    per_cluster = np.array([samples[y == c].mean() for c in range(3)])
    print(f"cluster_std={std} | score={s:.4f} | متوسطات العناقيد: "
          f"{np.round(per_cluster, 3)} | نقاط سالبة: {(samples < 0).sum()}")
```

اقرأ الناتج بعين الطبيب: عند `std=0.6` المقياس ٠.٩٠٦٠ بلا أي نقطة سالبة والعناقيد الثلاثة تتنفس بالصحة، وعند `std=1.5` يهبط إلى ٠.٧٦٣٢ فيبدأ القلق، وعند `std=2.6` تظهر ١٧ نقطة سالبة ويتآكل العنقود رقم ١ أولاً (متوسط ظله ٠.٥٢٤) — وهذا بالضبط ما لا يمكن معرفته من الرقم الإجمالي وحده.



الشكل ١٤٣تشخيص شكل الظلال لكل عنقود: قصّات متناسقة دليل نجاح، وقصّات متفاوتة الطول وقيعانها السالبة دليل تداخل

في الشكل يُرسم الظل `s(i)` لكل نقطة مرتبةً داخل عنقودها: الرسم المثالي يكون قصّات متساوية الطول تقريباً كلها فوق الصفر؛ أما القصّات القصيرة التي تخترق خط الصفر فتكشف النقاط الموضوعة في العنقود الخطأ. هذا «الأشعة

المقطعية» للتجميع هو ما يميزه عن أي رقم إجمالي واحد.

والآن استغل الأداة كما يفعل الممارسون فعلاً: شغّل التجميع أولاً ثم شخّص.

تشخيص العنقود الأضعف عبر silhouette_samples python

```python
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_samples, silhouette_score
import numpy as np

X, y = make_blobs(n_samples=600, centers=3, cluster_std=2.6, random_state=42)
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
samples = silhouette_samples(X, km.labels_)

print(f"silhouette_score الإجمالي: {silhouette_score(X, km.labels_):.4f}")
print(f"متوسط العيّنات: متوسط هوية مهم = {samples.mean():.4f}")
for c in range(3):
    vals = samples[km.labels_ == c]
    print(f"عنقود {c}: متوسط الظل {vals.mean():+.4f} | نقاط سالبة: {(vals < 0).sum()} من {len(vals)}")
```

لاحظ السطر الثاني من الناتج: `silhouette_score` ليس سوى متوسط `silhouette_samples` — العلاقة نفسها التي تربط الدقة بمتوسط التنبؤات الصحيحة. والفائدة العملية: تعرف **أي كتلة** تحتاج مراجعة، لا أن التجميع «سيء» عموماً.

تعمّق

تكلفة الحساب هي النقطة الضعيفة الوحيدة لـ Silhouette: تحتاج كل نقطة إلى المسافة إلى كل النقاط الأخرى، فالزمن `O(n²d)` والذاكرة تنفجر مع مئات آلاف العينات. في ١.٩ يساعدك `sklearn.metrics.pairwise_distances_argmin_min` وحساب المقياس على عينة فرعية ثابتة لخفض هذه التكلفة عند الحاجة.

٨.٣ الفخ الأشهر: حين ينجح التجميع ويفشل المقياس

سؤال مزعج: ماذا لو كان التجميع **صحيحاً تماماً** لكن Silhouette أعطته درجة قبيحة؟ مقياس Silhouette مبني بالكامل على المسافات الإقليدية ومفهوم «الأقرب»، لذلك يفترض ضمنياً أن العناقيد كروية الشكل ومتناظرة. أعطه هلالين متوازيين مثل بيانات `make_moons` التي ذكرناها في الفصل ٦٣، وسترى الانهيار:

فخ Silhouette على بيانات الهلالين غير الكروية python

```python
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

X, y_true = make_moons(n_samples=600, noise=0.06, random_state=42)
km = KMeans(n_clusters=2, n_init=10, random_state=42)
labels_km = km.fit_predict(X)

print(f"silhouette (الهلالان) للتقسيم الحقيقي: {silhouette_score(X, y_true):.4f}")
print(f"silhouette لتقسيم K-Means الشاقولي:    {silhouette_score(X, labels_km):.4f}")
print(f"ARI لتقسيم K-Means مقابل الحقيقة:      {adjusted_rand_score(y_true,
labels_km):.4f}")
```

النتيجة ساحرة في فلسفتها: التقسيم **الحقيقي** — الذي يفصل الهلالين بدقة — نال ٠.٣٣٠٥ فقط لأن الهلالين متلاصقين هندسياً، بينما حصد `K-Means` بخطئه الشاقولي القبيح درجة أعلى (٠.٤٩٠٣)! وقيمة ARI المنخفضة (٠.٢٤٢١) تكشف أن الظل «الأجمل» وهم. المقياس هنا يكافئ الخطأ ويعاقب الصواب، لأنه لا يفهم «الشكل» بل «المسافة».

> **خطأ شائع**
>
> لا تستخدم Silhouette وحدها للحكم على تجميع بيانات ذات أشكال غير كروية (هلالات، حلقات، أوضاع كثيفية معقدة). الأدوات الكثيفية مثل DBSCAN و HDBSCAN التي رأيناها في الفصلين ٦٣ و ٦٤ تُقيَّم بمعايير كثافة أو بمقارنات خارجية، لا بالظل. قاعدة عملية: إذا شكّكت في شكل العناقيد فأضف مقياساً خارجياً على عينة موسومة صغيرة قبل تصدّق الرقم الداخلي.

الدرس يتكرر: **كل مقياس داخلي يجسد افتراضاً هندسياً معيناً**، وأنت مسؤول عن مطابقة الافتراض مع بياناتك قبل أن تصدّق الرقم.

٨.٤ Davies-Bouldin و Calinski-Harabasz: بوصلة أخف وزناً

حساب Silhouette على مليون نقطة هو دوامة مسافات باهظة. لذلك توفر المكتبة مقياسين داخليين أرخص بكثير لأنهما يعملان على مستوى **المراكز** لا على مستوى الأزواج.

مقياس **Davies-Bouldin** يقارن، لكل زوج من العناقيد، نسبة (تشتتهما الداخلي) إلى (المسافة بين مركزيهما)، ويأخذ أسوأ حالة:

$$\mathrm{DB} = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i} \frac{s_i + s_j}{d(c_i, c_j)}$$

حيث `S_i` تشتت العنقود `i` (متوسط بعده عن مركزه `c_i`) و `d(c_i, c_j)` المسافة بين المركزين. القاعدة: **أدنى أفضل**، والصفر النظري يعني عناقيد مركّزة بلا تشتت. أما **Calinski-Harabasz** (ويسمى أيضاً معيار التباين) فهو نسبة تباين «بين العناقيد» إلى تباين «داخل العناقيد» مضروبة في تصحيح حجمي، و**أعلى أفضل**:

$$\mathrm{CH} = \frac{\operatorname{tr}(B_k)}{\operatorname{tr}(W_k)} \cdot \frac{n - k}{k - 1}$$

حيث `B_k` مصفوفة التشتت بين المراكز، و `W_k` التشتت داخل العناقيد، و `tr` أثر المصفوفة. لاحظ حدة الفرق: الأول يعاقب التداخل، والثاني يكافئ الفصل الواسع. هذان هما أسرع مقياسين داخليين في `sklearn` (زمنهما قريب من `O(nk)` فقط)، ولهذا يفضّلان في حلقات المسح الضخمة فوق قيم `k`.

مسح قيم k بالمقاييس الداخلية الثلاثة — python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score)

X, y_true = make_blobs(n_samples=600, centers=3, cluster_std=1.2,
random_state=42)
print(f"{'k':>3} | {'Silhouette':>10} | {'Davies-Bouldin':>14} | {'Calinski-
Harabasz':>17}")
for k in range(2, 7):
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X)
    print(f"{k:3d} | {silhouette_score(X, labels):10.4f} | "
          f"{davies_bouldin_score(X, labels):14.4f} | "
          f"{calinski_harabasz_score(X, labels):17.1f}")
```

في البيانات الكروية المنفصلة يتوافق الثلاثة عادة على نفس الاختيار، لكن لكل مقياس شخصيته ونطاقه واتجاهه، وهذا ملخص المواجهة:

المقياس	الاتجاه	النطاق	افتراضه الهندسي	متى تستخدمه
Silhouette	أعلى أفضل	من -١ إلى ١	عناقيد كروية متناظرة	التشخيص النقطي الدقيق والبيانات الصغيرة والمتوسطة
Davies-Bouldin	أدنى أفضل	من ٠ إلى ما لا نهاية	تشتت مقارن بين المراكز	مسح سريع لعدد العناقيد بأقل تكلفة
Calinski-Harabasz	أعلى أفضل	من ٠ إلى ما لا نهاية	نسبة تباينات (ANOVA)	اختيار `k` السريع على البيانات الكبيرة
ARI (خارجي)	أعلى أفضل	من نحو -٠.٥ إلى ١	أي بنية — يقيّم التطابق فقط	توفر تسميات مرجعية مهما كان الشكل
NMI / V-measure	أعلى أفضل	من ٠ إلى ١	يقوم على نظرية المعلومات	مقارنة تجزئة بأعراف تسمية مختلفة

لاحظ أن ARI ظهر في الجدول رغم أنه خارجي — ستقارن قريباً «الداخلية بالخارجية» لتكتشف متى تخدعك الداخلية.

٨.٥ المقاييس الخارجية: تصحيح الصدفة وتوازن المعلومات

تخيل أنك جمعت بيانات عملاء ثم اكتشفت لاحقاً أن قسم التسويق صنّف هؤلاء العملاء يدوياً إلى ثلاث فئات. الآن تملك شهوداً جزئيين، والسؤال يصبح: كم يقترب تقسيمك من هذه الفئات؟ هنا تظهر المقاييس الخارجية، وأولها وأهمها **ARI** (مؤشر راند المعدَّل).

الخطر القاتل في هذه المقارنة: فهرس راند الخام يحسب نسبة الأزواج المتفقة، لكنه لا يصحح **الصدفة**؛ فحتى تقسيم عشوائي تماماً يجمع نقاطاً بالاتفاق مصادفة. يصحح ARI ذلك بطرح القيمة المتوقعة للصدفة من البسط والمقام:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathrm{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathrm{E}[\mathrm{RI}]}$$

بمعنى: «كم تتجاوز اتفاقك الصدفة، نسبةً إلى أقصى اتفاق ممكن؟». النتيجة `0` تعني تقسيماً لا يفوق العشواء، و `1` تطابقاً تاماً مع المرجع — مهما كانت أسماء العناقيد ومهما اختلفت ترتيبها. والمفاجأة التي تلتهم المبتدئين: **ARI قد يكون سالباً** إذا كان تقسيمك أسوأ من التخمين العشوائي!

أما عائلة نظرية المعلومات فتبدأ بمفهومين متكاملين: **التجانس** (كل عنقود يحوي نقاطاً من فئة واحدة فقط) و**الاكتمال** (كل فئة حقيقية داخل عنقود واحد فقط)، ويجمعهما مقياس V-measure بمتوسط توافقي يذكرنا بـ F1

الذي رأيناه في الفصل ٤٤:

$$V = \frac{2hc}{h+c}$$

و **NMI** هو المقياس الشقيق القريب، بينما **FMI** (فهرس Fowlkes-Mallows) يعمل زوجياً كالجذر الهندسي لدقة واستدعاء الأزواج. الآن التجربة الحاسمة: لماذا لا نقارن التسميات مباشرة أبداً؟

لماذا تنهار المقارنة المباشرة للتسميات؟ python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
import numpy as np

X, y_true = make_blobs(n_samples=600, centers=3, cluster_std=1.2,
random_state=42)
labels = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(X)

# تدوير أسماء العنقود: 0 يصير 1، و 1 يصير 2، و 2 يصير 0 — البنية لم تتغير إطلاقاً!
rotated = (labels + 1) % 3

print(f"تطابق مباشر (==) قبل التدوير: {(y_true == labels).mean():.2%}")
print(f"تطابق مباشر (==) بعد التدوير: {(y_true == rotated).mean():.2%}")
print(f"ARI قبل التدوير:  {adjusted_rand_score(y_true, labels):.4f}")
print(f"ARI بعد التدوير:  {adjusted_rand_score(y_true, rotated):.4f}")
print(f"NMI بعد التدوير:  {normalized_mutual_info_score(y_true, rotated):.4f}")
```

الناتج يقول كل شيء: التطابق المباشر بمعامل `==` ظل عالقاً عند مستوى الصدفة (٣٣.٣٣%) في الحالتين، رغم أن البنية بعد التدوير مطابقة للحقيقة تماماً كما يشهد ARI = ١.٠٠٠٠ في السطرين الأخيرين. الخوارزميات ليس لها اتفاق على ترقيم العناقيد؛ فكل تشغيلة قد تسند الرمز `0` إلى كتلة مختلفة، ولذلك فإن المقارنة المباشرة للرموز ليست مجرد خطأ تقني — إنها خطأ مفاهيمي يفقد كل معنى.

خطأ شائع

مقارنة `labels_true == labels_pred` مباشرة أو حساب «دقة» على التجميع خطأ يفسد النتائج بصمت. الرموز أسماء اعتباطية لا أعداد. استخدم دائماً `sklearn.metrics.adjusted_rand_score` أو `normalized_mutual_info_score` فهما وحدهما اللذان يفهمان أن «العنقود ٢ في تقسيمي = العنقود ٥ في تقسيمك».

٨.٦ التجربة الحاسمة: جدول واحد يحاكم كل المقاييس

والآن نجمع كل السلاح: بيانات بثلاث كتل حقيقية اثنتان منها متقاربتان جداً — المشهد الذي يوقع المقاييس في التضارب. نشغّل عدة تجميعات ونحسب جميع المقاييس في جدول واحد:

جميع المقاييس في جدول واحد لتجميعات متعددة python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score,
                             normalized_mutual_info_score, v_measure_score,
                             fowlkes_mallows_score)
import pandas as pd

X, y_true = make_blobs(n_samples=900,
                       centers=[[0, 0], [5, 0], [6.8, 0]],
                       cluster_std=1.0, random_state=42)

rows = []
for name, k in [("K-Means k=2", 2), ("K-Means k=3", 3), ("K-Means k=4", 4)]:
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X)
    rows.append({
        "التقسيم": name,
        "Silhouette": silhouette_score(X, labels),
        "Davies-Bouldin": davies_bouldin_score(X, labels),
        "Calinski-Harabasz": calinski_harabasz_score(X, labels),
        "ARI": adjusted_rand_score(y_true, labels),
        "NMI": normalized_mutual_info_score(y_true, labels),
        "V-measure": v_measure_score(y_true, labels),
        "FMI": fowlkes_mallows_score(y_true, labels),
    })

df = pd.DataFrame(rows).set_index("التقسيم").round(3)
df
```

اقرأ الجدول سطراً سطراً ولن تندم: كتلتا البيانات اليمنى متداخلتان لدرجة أن الدمج بينهما (تقسيم `k=2`) ينتج هندسة أنظف — فيمنحه Silhouette و Davies-Bouldin و Calinski-Harabasz أفضل الدرجات الداخلية! لكن المقاييس الخارجية التي ترى الحقيقة تصرخ: ARI و FMI أقصاهما عند `k=3` (٠.٧٠٢ و ٠.٨٠١) لأن الدمج أبطل فئة كاملة، بينما يتساوى NMI بين التقسيمين (٠.٧٠٠) فيرفض الحسم. هذا ليس عيباً في الأدوات؛ إنه **التعريف** للفرق بين الداخلية والخارجية.

في الواقع

في مشاريع تجزئة العملاء الحقيقية لا توجد «حقيقة مرجعية» أصلاً، فيعتمد المحللون على المقاييس الداخلية لاختيار `k`، ثم يعرضون العناقيد على أصحاب المصلحة للحكم التجاري. أما في البحوث الطبية الحيوية ذات الفئات المرضية المعروفة لعينة جزئية فيبقى ARI المعيار الذهبي للنشر. القاعدة المهنية: صرّح دائماً بأي عائلة مقاييس اعتمدت، لأن الرقم بلا سياق عائلته بلا معنى.

المقاييس الداخلية تجيب عن «هل التقسيم متناسق هندسياً؟»، والخارجية عن «هل يطابق ما نعرفه؟». وأي تقارير تجميع جادة تعرض اثنين على الأقل من العائلتين، وتتفق مع قاعدة الفصل ٦٠: اختر `k` بالداخلية، وتحقق بالخارجية حين تتوفر حقيقة.

تمارين الفصل السادس والستين

- **ملاحظة**

التمرين ٨.١

على بيانات Iris (التي تملك تسميات حقيقية)، امسح `k` من ٢ إلى ٦ بالمقاييس الداخلية الثلاثة (Silhouette و Davies-Bouldin و Calinski-Harabasz) ثم بـ ARI ضد التسميات الحقيقية. أي `k` يختاره كل مقياس؟ ولماذا يخالف الداخليان الخارجي؟

التمرين ٨.٢

أعد تدوير أسماء العناقيد بعدة تبديلات عشوائية مختلفة وتحقق عملياً من أن Silhouette و Calinski-Harabasz لا يتأثران إطلاقاً بالترقيم — ثم اشرح لماذا يعجز التطابق المباشر عن هذا الثبات.

التمرين ٨.٣

أنشئ بيانات ثلاثة عناقيد واجبر التجميع على التجزئة المفرطة (`k = 3, 6, 12`). قارن سلوك ARI مقابل NMI: أيهما يعاقب التجزئة بحدة، وأيهما يظل متفائلاً؟ وفسّر السبب بمفهومي التجانس والاكتمال.

- **ملاحظة**

حل التمرين: مسح k على Iris بكل المقاييس python

```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score)
import pandas as pd

X, y_true = load_iris(return_X_y=True)

rows = []
for k in range(2, 7):
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X)
    rows.append({
        "k": k,
        "Silhouette": silhouette_score(X, labels),
        "Davies-Bouldin": davies_bouldin_score(X, labels),
        "Calinski-Harabasz": calinski_harabasz_score(X, labels),
        "ARI": adjusted_rand_score(y_true, labels),
    })

df = pd.DataFrame(rows).set_index("k").round(3)
best = df.idxmax()[["Silhouette", "Calinski-Harabasz", "ARI"]]
best["Davies-Bouldin"] = df["Davies-Bouldin"].idxmin()
print(f"المثلى k اختيار كل مقياس لقيمة: {dict(best)}")
df
```

Silhouette و Davies-Bouldin يختاران `k=2` لأن صنفي Versicolor و Virginica متداخلان هندسياً فينظمان كتلة واحدة نظيفة، بينما يكشف ARI — الذي يرى التسميات الحقيقية الثلاث — أن `k=3` هو الأصح (٠.٧٣٠ مقابل ٠.٥٤٠)، ويوفق معه هنا Calinski-Harabasz. إنه فخ الشكل الكروي الذي رأيناه في القسم ٨.٣ وقد تجسد على بيانات حقيقية.

> **ملاحظة**

حل التمرين: ARI مقابل NMI تحت التجزئة المفرطة python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

X, y_true = make_blobs(n_samples=900, centers=3, cluster_std=1.3,
random_state=42)

for k in [3, 6, 12]:
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X)
    print(f"k={k:2d} | ARI = {adjusted_rand_score(y_true, labels):.4f} | "
          f"NMI = {normalized_mutual_info_score(y_true, labels):.4f}")
```

يلاحظ من الناتج أن ARI ينهار مع التجزئة (كل كتلة حقيقية صارت عدة قطع، فتفقد الاكتمال وتتضخم الاتفاقات المصادفية التي يصححها المقياس)، بينما يبقى NMI مرتفعاً نسبياً: التجزئة تحافظ على التجانس العالي (كل قطعة نقية)، وNMI يتقاسم الحكم مع الاكتمال بسماحة أكبر. لهذا يُفضَّل ARI حين تكون العقوبات الصارمة مطلوبة.

> **ملاحظة**

٨.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
مقارنة التسميات بـ `labels_true == labels_pred`	«دقة» كاذبة تنهار مع أي إعادة ترقيم للعناقيد وإن كانت البنية مطابقة	استخدام `adjusted_rand_score` أو `normalized_mutual_info_score` الثابتين تجاه التبديلات
الحكم على تجميع DBSCAN بالظل وحده	Silhouette تفترض الأشكال الكروية فتعاقب النجاح على الهلالات والحلقات	تقييم الأشكال غير الكروية بمقاييس خارجية أو معايير كثافة
حساب `silhouette_score` **على ملايين العينات مباشرة**	تكلفة `O(n²)` تستهلك الذاكرة والزمن حتى الاختناق	المسح على عينة فرعية عشوائية ثابتة `random_state` ثم التشخيص النقطي على العيّنة
الاعتماد على مقياس داخلي واحد لاختيار `k`	مقياس واحد قد يخدعك حين تتداخل الكتل كما في تجربة القسم ٨.٦	عرض المقاييس الثلاثة الداخلية معاً وإضافة ARI عند توفر تسميات مرجعية

٨.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المقاييس الداخلية	مقاييس تحكم على جودة التجميع من بنية البيانات ذاتها دون تسميات مرجعية	Internal (intrinsic) metrics
المقاييس الخارجية	مقاييس تقارن تقسيم التجميع بتقسيم مرجعي معروف مسبقاً	External metrics
معامل الظل	قيمة لكل نقطة تقيس قربها من عنقودها مقابل أقرب عنقود مجاور بين -١ و ١	Silhouette coefficient
مؤشر راند المعدَّل	تطابق زوجي بين تقسيمين مصححاً لاتفاق الصدفة وقد يكون سالباً	Adjusted Rand Index (ARI)
التجانس والاكتمال	جانبا V-measure: نقاء العنقود من فئة واحدة، واحتواء كل فئة في عنقود واحد	Homogeneity & Completeness

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٧ — النماذج المزيجية الغاوسية

خوارزمية EM وتفسير المكوّنات واختيار عدد المزيج

الوحدة **scikit-learn ١.٩** الحجم **١٠٤٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح خطوتَي E و M في خوارزمية EM ومعنى المسؤوليات الاحتمالية
- أن تختار covariance_type المناسبة بين full و tied و diag و spherical بوعي بتكلفة كل خيار
- أن تحدد عدد المكوّنات المثلى عبر منحنيَي BIC و AIC بدلاً من Elbow الذاتي
- أن تستخرج predict_proba للانتماء الناعم و score_samples للكثافة وكشف الشذوذ
- أن تستخدم reg_covar لضبط الإفراط و BayesianGaussianMixture للقصّ التلقائي

٨.١ من التصويت الصلب إلى الانتماء الناعم

اذكر لمديرك أن العميل رقم ٤٧ «ينتمي للشريحة الاقتصادية»، وسيسألك فوراً: وبأي ثقة؟ K-Means عاجز عن الإجابة؛ فهو يجري تصويتاً صلباً (hard assignment): أقرب مركز يفوز، والبقية تصفر. أما إذا كانت شريحة ٤٧ تقع على بعد سنتيمتر واحد من حد الفصل بين الشريحتين، فالتصويت الصلب يخفي شكلاً حقيقياً يجب أن يُقال.

النماذج المزيجية الغاوسية (Gaussian Mixture Models - GMM) تعيد بناء المشهد من الأساس: لنفترض أن بياناتنا وُلدت من K توزيعاً غاوسياً، لكل واحد متوسط μ_k ومصفوفة تغاير Σ_k (أي عنقود إهليلجي يمكن أن يميل كيف شاء) ونسبة مزج π_k (حجم العنقود من الإجمالي). التجميع هنا ليس قراراً بل **استدلال**: ما احتمال أن تكون النقطة i منشأة عن المكوّن k؟

مفتاح

في K-Means كل نقطة تملك عضوية واحدة بقيمة ١، وفي GMM تملك كل نقطة متجهاً كاملاً من المسؤوليات الاحتمالية (responsibilities) مجموعه ١: مثل «النقطة ٤٧: ٠.٨٢ اقتصادية، ٠.١٨ فاخرة». هذا الانتماء الناعم ليس رفاهية؛ إنه المعنى الحقيقي للتجميع في أنظمة التسعير والتشخيص الطبي حيث القرار يلزمه هامش ثقة.

وماذا في المقابل؟ الثمن مزدوج: افتراض أن كل عنقود غاوسي (لا هلالات ولا حلقات كما في الفصل ٦٣)، وتكلفة حسابية أعلى. لكن في المقابل تحصل على مكافأة لا يقدمها K-Means إطلاقاً: **دالة كثافة احتمالية كاملة** للبيانات، وستها في القسم ٨.٥.

والحكم ألا يبقى نظرياً: البيانات التالية فيها عنقود ممتد طويل وشريكان متماسكان — أحلّ بيانات لكشف الفرق. شغّل المواجهة على البيانات نفسها:

مواجهة على البيانات نفسها: K-Means مقابل GMM python

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
import numpy as np
import pandas as pd

X_raw, y_true = make_blobs(n_samples=900, centers=[[0, 0], [9, 0], [12, 0]],
                           cluster_std=[3.0, 0.7, 0.7], random_state=42)
A = np.array([[0.6, -0.65], [-0.35, 0.9]])   # إطالة وميلان للفضاء كله
X = X_raw @ A

rows = []
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
rows.append({"النموذج": "K-Means (صلب)",
             "ARI": f"{adjusted_rand_score(y_true, km.labels_):.3f}",
             "متوسط اللوغاريتم": "—"})
for ct in ["spherical", "full"]:
    gm = GaussianMixture(n_components=3, covariance_type=ct,
                         n_init=5, random_state=42).fit(X)
    rows.append({"النموذج": f"GMM ({ct})",
                 "ARI": f"{adjusted_rand_score(y_true, gm.predict(X)):.3f}",
                 "متوسط اللوغاريتم": f"{gm.score(X):.3f}"})

pd.DataFrame(rows).set_index("النموذج")
```

الجدول يقول كل شيء: K-Means يقطع العنقود الممتد نصفين فيهبط ARI إلى ٠.٣٩٨، بينما يلتقط GMM بنمط `full` البنية كاملة (٠.٩٦١). وعمود متوسط اللوغاريتم يفتح باباً جديراً بالتأمل: K-Means لا يملك أصلاً دالة كثافة احتمالية — ما يحسبه مجموع تربيعات المسافات لا يجوز خلطه باللوغاريتمات، ولذلك وضعنا «—» في خانته. امتلاك دالة كثافة هو الميزة البنيوية الأولى لـ GMM، والثانية تلتقيها في القسم ٨.٥.

٨.٢ الاحتمال الكلي وخوارزمية EM

النموذج يقول: كل نقطة `x_i` تُرسم أولاً من مكوّن مخفي `z_i` باحتمال `π_k`، ثم من الغاوسية الخاصة به. فالاحتمال الكلي لنقطة هو مجموع مساهمات المكوّنات كلها موزوناً بنسبها:

$$p(\mathbf{x}_i) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k) \qquad \ln L = \sum_{i=1}^{n} \ln\!\left[\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k) \right]$$

حيث `π_k` نسب المزيج ومجموعها ١، و `N(x | μ, Σ)` كثافة الغاوسية متعددة الأبعاد. تدريب النموذج = تعظيم اللوغاريتم الكلي `ln L` على البيانات. المشكلة: المكوّن المنتمي إليه كل نقطة **متغير خفي** لا نراه، فالمعادلات تتعطل بالاشتقاق المباشر. هنا يدخل السحر القياسي الذي صاغه Dempster وزملاؤه عام ١٩٧٧: خوارزمية **EM** (Expectation-Maximization) تتناوب خطوتين حتى يتوقف التحسن.

خطوة E (التوقع): بناءً على المعاملات الحالية، خمّن المسؤوليات — احتمال أن المكوّن `k` هو الأب الحقيقي للنقطة `i`، بمثيلة بايز:

$$\gamma_{ik} = \frac{\pi_k \, \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)}$$

خطوة M (التعظيم): وتصرف المسؤوليات هذه كأوزان ناعمة بدل التصويت الصلب، وأعد حساب المعاملات المثلى وكأن المسؤوليات حقائق معلومة. أولاً «الحجم الفعال» والمتوسط:

$$N_k = \sum_{i=1}^{n} \gamma_{ik} \qquad \mu_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \, \mathbf{x}_i$$

ثم التغاير ونسبة المزيج — ولاحظ الشبه المذهل مع متوسط K-Means المرجّح:

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T \qquad \pi_k = \frac{N_k}{n}$$

تتكرر الخطوتان حتى يتشبع `ln L`. وهنا ملاحظة عملية مهمة: التهيئة تحدد الوادي الذي تهبط إليه الخوارزمية، ولهذا يستخدم `sklearn.mixture.GaussianMixture` تهيئة افتراضية `init_params='kmeans'` — يرسم K-Means الحدود الخشنة أولاً ثم تدقّها EM، ويمكن رفع `n_init` لتجربة عدة هبوطات واختيار الأفضل.

الشكل ١٤٤تتابع خطوتي E و M: المسؤوليات تلطّف الحدود ثم المراكز والتغايرات تتبلور تدريجياً

الشكل يتتبع التناوب: في كل خطوة E تنحني حدود الانتماء حول الأهلة الإهليلجية، وفي كل خطوة M تنزاح المراكز وتتسع التغايرات لتحتضن نقطها، حتى يستقر `ln L` . لاحظ أن الحدود النهائية قد تكون قطوعاً مخروطية مائلة — شيء يستحيل على K-Means حتى بغياب الشكل.

٨.٣ covariance_type: أربع مستويات من المرونة

سؤال الهندسة الحقيقي: كم بارامتراً نمنح كل غاوسية؟ مع `d` بُعداً و `K` مكوّناً، تفاضل الخيارات الأربعة في `covariance_type` حاد:

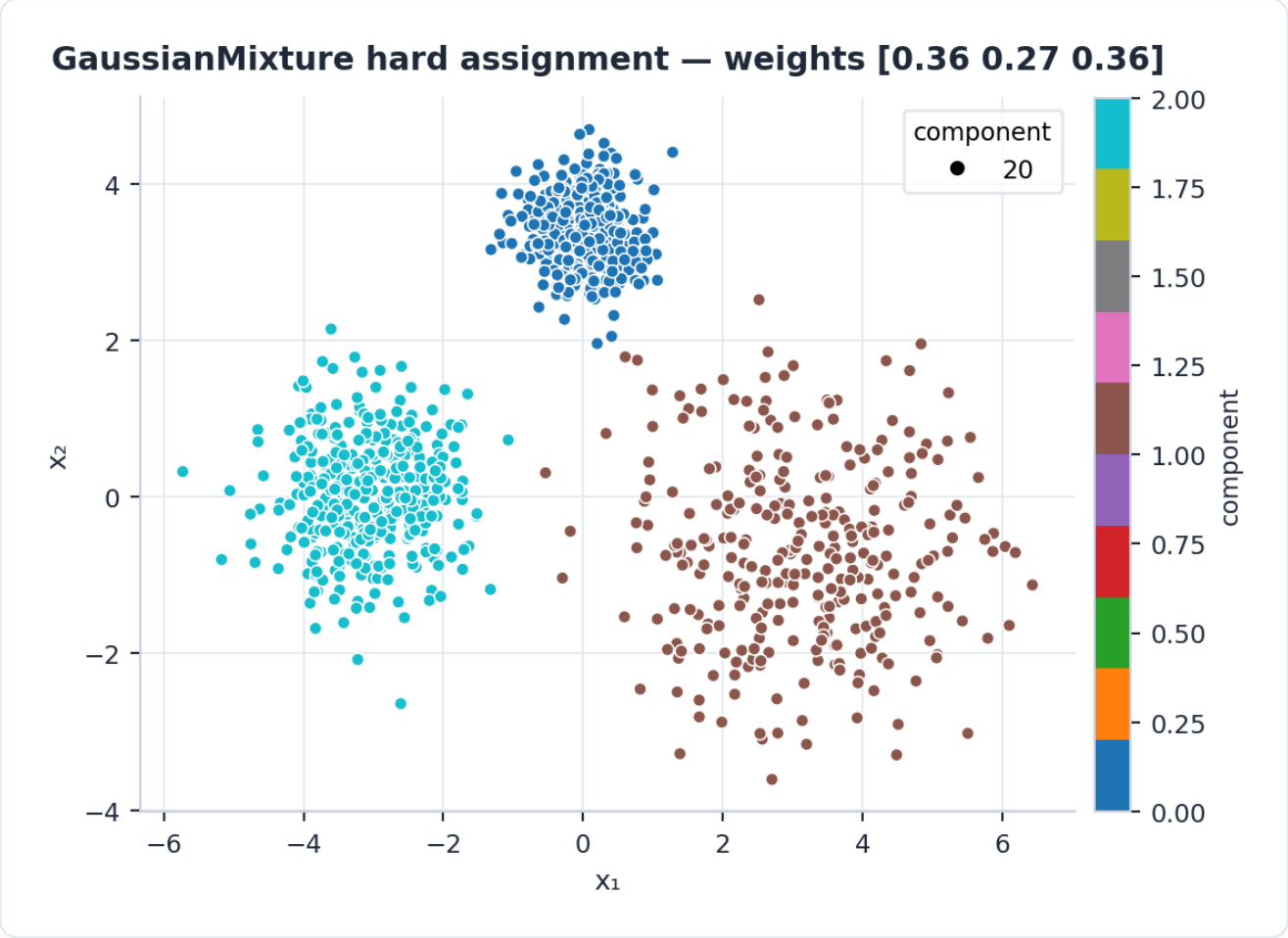
covariance_type	شكل العنقود المسموح	عدد بارامترات التغاير	متى تستخدمه
`spherical`	كرة متناظرة	`K`	العناقيد الكروية المتساوية الاتساع؛ أسرع وأبسط
`diag`	إهليلجي محاذٍ للمحاور	`K × d`	الأبعاد مستقلة تقريباً والأهمية متفاوتة
`tied`	إهليلجي مائل — واحد مشترك للجميع	`d × (d + 1) / 2`	عناقيد بنفس الشكل والاتجاه وأمثلة قليلة
`full`	إهليلجي مائل مستقل لكل عنقود	`K × d × (d + 1) / 2`	الأقصى مرونة؛ الافتراضي، والأغلى ذاكرةً

الفرق ليس أكاديمياً. اصنع بيانات إهليلجية مائلة (تحويل خطي على `make_blobs`) وشغّل الأربعة:

مقارنة أنماط التغاير الأربعة على البيانات الإهليلجية python

```python
# X و y_true من القسم ٨.١
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
import pandas as pd

rows = []
for ct in ["spherical", "diag", "tied", "full"]:
    gm = GaussianMixture(n_components=3, covariance_type=ct,
                         n_init=5, random_state=42).fit(X)
    rows.append({
        "covariance_type": ct,
        "ARI": adjusted_rand_score(y_true, gm.predict(X)),
        "متوسط اللوغاريتم": round(gm.score(X), 3),
        "عدد البارامترات": gm._n_parameters(),
    })

df = pd.DataFrame(rows).set_index("covariance_type")
df
```

اقرأ عمود ARI بعين مجتهد: النمط الكروي — نفس فلسفة K-Means تماماً — يتحسن عنه (٠.٨١٣ مقابل ٠.٣٩٨) لكنه لا يبلغ القمة، بينما يحصد `full` أعلى تطابق (٠.٩٦١) لأنه وحده يمنح كل عنقود غاوسيته المائلة الخاصة. وأدهى الدروس مصير `tied`: انهار إلى ٠.٤٠٠ رغم مرونته الظاهرة، لأن عنقوداً ممتداً وشريكين ضيقين لا يقبلون مصفوفة تغاير واحدة مشتركة — فأُجبر الجميع على قالب لا يناسب أحداً. وعمود عدد البارامترات يذكرك بالفاتورة: المرونة تُشترى ببارامترات، والبارامترات الزائدة بلا بيانات كافية تفتح باب الإفراط (قسم ٨.٦).



الشكل ١٤٥ إهلة التغاير للمكوّنات المكتشفة وحدود الانتماء الاحتمالية بينها

في الشكل تُرسم كثيرات حدود التغاير لكل مكوّن: انظر كيف يميل القطع الناقص مع اتجاه البيانات الفعلي تحت نمط `full`، وكيف يتحجر إلى دائرة تحت `spherical`. هذه الإهلة هي «الشكل» الذي يفسره النموذج، وقراءتها جزء من تفسير المكوّنات نفسه.

نصيحة

على البيانات الواقعية ابدأ بالافتراضي `full` إذا كان `d` صغيراً (أقل من نحو ١٠). أما في الأبعاد العالية فـ `full` يضخّم البارامترات تربيعياً مع `d` — جرّب `diag` أو `tied` وراقب BIC؛ وقد ينتقل الفارق من مضاعفات زمن التدريب إلى مضاعفات الذاكرة.

٨.٤ اختيار عدد المكوّنات: BIC و AIC بدل الكوع

منحنى Elbow الذي رأيناه في الفصل ٦٠ يعتمد على عينك في تحديد «الكوع»، وهذا حكم بنيوي لا يُحسن الجدل. GMM يقدم بديلاً موضوعياً كلياً: بعد التدريب، احسب على الكلفة نفسها مع عقوبة عدد البارامترات `p` :

$$\mathrm{BIC} = -2\ln \hat{L} + p \ln n \qquad \mathrm{AIC} = -2\ln \hat{L} + 2p$$

حيث `ln L` اللوغاريتم الأعظمي المُحقَّق و `p` عدد البارامترات الحرة (يعيدها `_n_parameters()`). كلا المعيارين **أدنى أفضل**: الهبوط مع زيادة `k` ثم الارتداد حين تتجاوز عقوبة التعقيد مكسب الملاءمة. والفرق بينهما أن عقوبة BIC (`ln n`) تنمو مع حجم البيانات فتجعل المعيار بخيلاً أكبر تجاه المكوّنات الزائدة.

مسح عدد المكوّنات بمعياري BIC و AIC — python

```
from sklearn.mixture import GaussianMixture

print(f"{'k':>2} | {'BIC':>9} | {'AIC':>9}")
bics, aics = [], []
for k in range(1, 9):
    gm = GaussianMixture(n_components=k, covariance_type="full",
                         n_init=3, random_state=42).fit(X)
    bics.append(gm.bic(X))
    aics.append(gm.aic(X))
    print(f"{k:2d} | {gm.bic(X):9.1f} | {gm.aic(X):9.1f}")

print(f"أدنى BIC عند k = {int(np.argmin(bics)) + 1} | أدنى AIC عند k =
{int(np.argmin(aics)) + 1}")
```

في هذا المثال الصافي يتفق المعياران مع الحقيقة (ثلاثة عناقيد وُلدت بها البيانات)، مع تفضيل AIC أحياناً لقيمة أعلى قليلاً لأنه أكثر كرماً بمعامل العقوبة الأصغر. هذه أداة اختيار عدد الفئات الموضوعية التي ينقص K-Means: بدل أن تسأل «أين الكوع؟» تسأل «أين أدنى BIC؟» — والجواب رقم واحد لا يختلف عليه اثنان.

سياق

قصة BIC عبقرية: صاغه غيدون شوارتز عام ١٩٧٨ بتقريب أسيمبتيكي من بايز، فحمله من النظرية إلى الممارسة عقوبة `p ln n` هذه. لم تُخترع «تجريبياً» كالنقطة المثالية للكوع؛ بل هي تقدير لهامش بايز الخلفي للنموذج نفسه — ولهذا تجد دائماً صوتاً نقدياً واحداً في اجتماعات اختيار النماذج.

٨.٥ الانتماء الناعم والكثافة: *score*samples *g* predict*proba*

بعد التدريب يمنحك النموذج أداتين فائقتين. الأولى `predict_proba(X)`: المتجه الاحتمالي الكامل لكل نقطة — الجواب الذي عجز عنه K-Means في افتتاحية الفصل. الثانية `score_samples(X)`: **لوغاريتم كثافة الاحتمال لكل نقطة تحت المزيج كله** — أي «كم هذه النقطة محتملة الوجود أصلاً في عالمي؟».

python الانتماء الناعم وكثافة الاحتمال لكل نقطة

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, covariance_type="full",
                     n_init=3, random_state=42).fit(X)

# نقطتان على منتصف المسافة بين مركزين: من هما؟
for i, j in [(0, 1), (0, 2)]:
    mid = ((gm.means_[i] + gm.means_[j]) / 2).reshape(1, -1)
    print(f"منتصف المسافة بين المكوّنين {i} و {j}: مسؤوليات
{gm.predict_proba(mid).round(3)}")

# الكثافة: عينات عادية مقابل نقطة بعيدة عن كل شيء
log_dens = gm.score_samples(X[:3])
far = np.array([[16.0, -14.0]])
print(f"كثافة لوغاريتمية لعينات عادية: {np.round(log_dens, 2)}")
print(f"كثافة لوغاريتمية لنقطة بعيدة {far.tolist()}: {gm.score_samples(far).round(2)}")
```

المفاجأة الأولى: النقطة على منتصف المسافة **الإقليدية** بين مركزين قد تنتمي كلياً (مسؤولية ١.٠) لواحد منهما — لأن التغايرات تملك الحكم لا المسطرة؛ العنقود الممتد يراها عادية تافهة البعد، والضيق يراها من حسن حظه. وفي المقابل منتصف المسافة بين المكوّنين ٠ و ٢ — حدّ الفصل الفعلي — يقترب من التعادل: ٠.٥١ مقابل ٠.٤٩. هذا «الحد الاحتمالي» الذي يستحيل على التصويت الصلب. أما الكثافة: العينات العادية تتمركز بين -١.٥ و -٢.٨، بينما تسقط النقطة البعيدة إلى -٥٧.٩ — هوة من نحو ٥٥ وحدة لوغاريتمية. هذه ليست عارضة؛ إنها **أساس كشف الشذوذ الطبيعي**: عيّن عتبة كثافة (مثلاً المئين الأول للبيانات التدريبية) وكل نقطة تحتها «شاذة». سنرتدي هذا الزي الكامل مع الأدوات المخصصة مثل IsolationForest و LOF في الفصل ٧٠؛ هنا يكفي أن تدرك أن GMM يمنحك الكاشف مجاناً كأثر جانبي لتدريبه.

> **خطأ شائع**
>
> لا تقرأ `predict_proba` كأنها حقيقة مطلقة: الاحتمالات الناتجة حساسة لافتراض الغاوسية ولعدد المكوّنات، وقد تبدو واثقة تماماً على نقاط خارج توزيع التدريب. استعملها لتقسيم الثقة لا لتقديم القرارات كأنها موقّعة من محكمة عليا.

٦٧.٨ فخ الإفراط: full مع عينات قليلة و reg_covar

ماذا لو ضغطنا البيانات إلى بضع عينات وأصررنا على مكوّنات كثيرة بتغاير `full`؟ الخوارزمية تلتهم البيانات: مكوّن ينكمش على عينة واحدة فتتضاءل تغايره نحو الصفر، فترتفع كثافته عن تلك العينة إلى ما لا نهائي تقريباً، ويرتفع `ln L` التدريبي — بينما ينهار النموذج على أي بيانات جديدة. هذا **الإفراط الاحتمالي** بأشده، ودرع المكتبة ضده بارامتر `reg_covar` الذي يضيف ثابتاً صغيراً إلى قطر مصفوفة التغاير فلا تنكمش أبداً عن ذلك.

reg_covar الإفراط في اللوغاريتم مع عينات قليلة وعلاج — python

```python
from sklearn.model_selection import train_test_split

X_small, _, y_small, _ = train_test_split(
    X, y_true, train_size=45, random_state=42, stratify=y_true)

def ll(model, data):
    return float(model.score(data) * len(data))

print(f"{'الإعداد':32} | {'تدريب':>9} | {'كامل البيانات':>13}")
for reg in [1e-6, 0.1]:
    gm_over = GaussianMixture(n_components=8, covariance_type="full",
                              reg_covar=reg, random_state=42).fit(X_small)
    print(f"full و k=8 و reg_covar={reg:<6} | {ll(gm_over, X_small):9.1f} | "
          f"{ll(gm_over, X):13.1f}")

gm_right = GaussianMixture(n_components=3, covariance_type="full",
                           random_state=42).fit(X_small)
print(f"{'full و k=3 و reg_covar=1e-06':32} | {ll(gm_right, X_small):9.1f} | "
      f"{ll(gm_right, X):13.1f}")
```

النمط المكشوف صارخ: عند `reg_covar=1e-6` الافتراضي يحقق نموذج `k=8` على ٤٥ عينة لوغاريتم تدريبي مرتفعاً (-٦٣.٥ مقابل -١١٩.٥ لنموذج `k=3` الأمين) ثم ينهار خارج عينته إلى -٥٥٤٥.٠ — أضعاف أسوأ من -٣٠٨١.٦ للأمين. ورفع `reg_covar` إلى ٠.١ يهدّئ الانفجار إلى -٣٣٢٤.١. الدرس: **لا تحكم بجودة GMM من لوغاريتمه التدريبي أبداً**؛ احكم بـ BIC أو ببيانات محتجزة، وإذا صغرت عيناتك فارفع `reg_covar` أو اختر `tied` الذي يتقاسم التغاير بين المكوّنات.

في الواقع

في تحليل سلاسل الصوت والكلام، وصفت GMM بسنوات «بصمة» المتحدثين، وكان فريق الصوت يتضح فيه هذا الفخ كلما قلّت ثواني التسجيل: مكوّنات full تنكمش على ضجيج الميكروفون نفسه. لهذا وُلدت الأوزان التنظيمية ثم صعدت النماذج العصبية، وبقيت قاعدة reg_covar درساً يطبق في كل نموذج احتمالي.

٨.٧ BayesianGaussianMixture: القصّ التلقائي للأوزان

والسؤال الأنيق الأخير: ماذا لو لم نعرف `k` أصلاً؟ بدل أن تمسح قيم `k` بمنحنى BIC، يمنحك `sklearn.mixture.BayesianGaussianMixture` حلاً بارعاً: اطلب مكوّنات **أكثر مما تحتاج** (مثلاً ١٠) واترك أداة الاهتمام `weight_concentration_prior` تشتغل. المنطق البايزي: كل مكوّن زائد عن الحاجة يضعف وزنه تدريجياً حتى يقترب من الصفر — **قصّ تلقائي للأوزان الصغيرة** بلا قرار يدوي.

القصّ التلقائي: عشرة مكوّنات تُرشّح إلى ثلاثة

```python
from sklearn.mixture import BayesianGaussianMixture

bgm = BayesianGaussianMixture(
    n_components=10, covariance_type="full",
    weight_concentration_prior=0.1, max_iter=400, random_state=42).fit(X)

weights = np.sort(bgm.weights_)[::-1]
print(f"الأوزان مرتبة تنازلياً: {np.round(weights, 4)}")
print(f"مكوّنات فعالة (وزن أكبر من 0.05): {(bgm.weights_ > 0.05).sum()}")
print(f"وزن أقوى مكوّن: {bgm.weights_.max():.3f}")
```

الناتج يحكي القصة: ثلاثة أوزان كبرى (٠.٣٣٥ و ٠.٣٣٣ و ٠.٣٢٧) تتنازع العرش — وهي نسب المزيج الحقيقية تقريباً — وسبعة خُنقت إلى شبه لا شيء (أعلاها ٠.٠٠١). هذا النموذج **أكبر** من حاجتك عن قصد، والبيانات هي التي اختارت الصالح منه. في المقابل اجعل أمرك واضحاً: الطريقة ليست معصومة مع التداخلات الشديدة، وتظل تعاني افتراض الغاوسية نفسه، وربعها الحسابي أعلى من GMM العادي لأنها تقدّر توزيعات لاحقة لا نقاطاً فقط.

تمارين الفصل السابع والستين

- **ملاحظة**

التمرين ٨.١

على بيانات Iris، امسح `covariance_type` الأربعة بمكوّنات `k=3` وقارن ARI ضد التسميات الحقيقية وعدد البارامترات. أي نمط يوازن أفضل تطابق بأقل بارامترات؟

التمرين ٨.٢

أنشئ بيانات بعنقودين متقاربين جداً وثالث بعيد، ثم قارن بين اختيار BIC واختيار NMI/ARI (عبر التسميات الحقيقية) لعدد المكوّنات. هل يصلان إلى القرار نفسه؟ وفسّر بالدرس الذي رأيناه في الفصل ٦٦.

التمرين ٨.٣

درب GMM على بيانات كاملة ثم اكتب دالة شذوذ تعيد النقاط الأدنى من عتبة المئين الثاني لكثافة `score_samples`. اختبرها بحقن ٥ نقاط بعيدة مصطنعة: كم منها التُقطت؟

- **ملاحظة**

حل التمرين: أنماط التغاير على Iris python

```python
from sklearn.datasets import load_iris
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
import pandas as pd

X_iris, y_iris = load_iris(return_X_y=True)

rows = []
for ct in ["spherical", "diag", "tied", "full"]:
    gm = GaussianMixture(n_components=3, covariance_type=ct,
                         n_init=5, random_state=42).fit(X_iris)
    rows.append({
        "covariance_type": ct,
        "ARI": adjusted_rand_score(y_iris, gm.predict(X_iris)),
        "عدد البارامترات": gm._n_parameters(),
    })

df = pd.DataFrame(rows).set_index("covariance_type")
df
```

على Iris — حيث يتفق الغاوسون مع طبيعة البيانات نسبياً — يتصدّر `tied` و `full` التطابق (٠.٨٨٦ و ٠.٩٠٤) بفارق واضح عن `spherical` و `diag` (٠.٧٣٠ و ٠.٧٤٥)، بينما يدفع `full` فاتورة بارامترات تعادل ضعف `tied` تقريباً (٤٤ مقابل ٢٤). القرار الهندسي: مع أبعاد قليلة وعينات كافية، `full` خيار مريح؛ ومع بيانات شحيحة، `tied` يحافظ على معظم المرونة بنصف الكلفة.

> **ملاحظة**

حل التمرين: كاشف شذوذ من كثافة GMM python

```python
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, covariance_type="full",
                     n_init=3, random_state=42).fit(X)

log_dens = gm.score_samples(X)
threshold = np.percentile(log_dens, 2)      # عتبة المئين الثاني
print(f"عتبة الكثافة (المئين 2): {threshold:.2f}")

outliers = np.array([[16.0, -14.0], [-17.0, 12.0], [14.0, 15.0],
                     [-15.0, -16.0], [18.0, 4.0]])
caught = (gm.score_samples(outliers) < threshold).sum()
print(f"نقاط شاذة محقونة: {len(outliers)} | الملتقطة تحت العتبة: {caught}")
print(f"نسبة الالتقاط: {caught / len(outliers):.0%}")
```

كلفة الكاشف أقل من عشرة أسطر: العتبة -٦.٦٩ مأخوذة من المئين الثاني لكثافات التدريب، والنقاط البعيدة المحقونة سقطت خمسها الخمس تحت العتبة (نسبة التقاط ١٠٠٪). هذا هو «الكاشف المجاني» الذي وعدنا به القسم ٨.٥، وهو ما سيتوسع فيه الفصل ٧٠ بأدوات متخصصة أكثر تكيفاً مع البيانات المعقدة.

> **ملاحظة**

٨.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الحكم على جودة GMM بلوغاريتمه التدريبي	نموذج مفرط بـ `full` و `k` كبير يحقق `ln L` أعلى وأداء أسوأ خارج العينة	الاختيار بـ `bic()` / `aic()` أو التحقق على بيانات محتجزة
توقع كثافة `predict_proba` واثقة على نقاط شاذة	نماذج مفرطة تمنح احتمالات حاسمة لنقاط خارج عالمها تماماً	فحص `score_samples` أولاً ورفض النقاط منخفضة الكثافة قبل تفسير المسؤوليات
استخدام `full` في الأبعاد العالية بلا تفكير	انفجار `O(K d²)` في الذاكرة والبارامترات ثم تذبذب التقديرات	تخفيض `d` (كما في الفصل ٦٨) أو اعتماد `tied` / `diag`
قراءة نتيجة تشغيلة واحدة من EM	تهيئة سيئة تحجز الخوارزمية في optimum محلي منخفض	رفع `n_init` إلى ٥ أو أكثر وترك المكتبة تختار أفضل تشغيلة

٨.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المزيج الغاوسي	نموذج توليدي يفترض أن البيانات مزيج مرجّح من غاوسيات متعددة	Gaussian Mixture Model (GMM)
المسؤوليات	احتمال أن تكون النقطة منشأة عن مكوّن معين بمجموع ١ لكل نقطة	Responsibilities
خوارزمية التعظيم والتوقع	تناوب بين تقدير المتغير الخفي وتعظيم اللوغاريتم الكلي	Expectation-Maximization (EM)
معيار المعلومات البايزي	كلفة ملاءمة مضافاً إليها عقوبة البارامترات؛ أدنى أفضل	Bayesian Information Criterion (BIC)
أداة الاهتمام	كثافة ديريشلت البايزية التي تُخنق أوزان المكوّنات غير المستحقة	Weight concentration prior

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٨ — تقليل الأبعاد الخطي: PCA

تحليل SVD وتفسير المكوّنات ونسخ PCA العملية

الوحدة **scikit-learn ١.٩** الحجم **١٠١٠** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف تحوّل مصفوفة التغاير إلى اتجاهات تباين عبر التحليل الذاتي
- أن تقرر عدد المكونات عبر explained_variance_ratio_ وعتبة ٠.٩٥ بدل التخمين
- أن تكتشف سرقة المكون الأول من عمود بمقياس منفلت ولماذا التطبيع سابق لـ PCA
- أن تقرأ components_ كأوزان للأعمدة الأصلية وتفسر معنى كل مكون
- أن تختار بين TruncatedSVD و IncrementalPCA و 'svd_solver='randomized حسب حجم بياناتك وتمثيلها

٨.١ الفكرة: الاتجاه الذي يحمل أكبر تباين

افتح مجموعة بيانات زهرة السوسن (Iris) التي رافقنا منذ الفصل ٣٧: أربعة أعمدة، لكنها ليست أربعة معلومات مستقلة؛ فطول البتلة وعرضها يسيران معاً تقريباً. لو طُلب منك تلخيص كل زهرة **برقم واحد**، أي رقم تختار؟ الحدس يقول: مقياس «حجم» يجمع أطوال وأعراض البتلات. تحليل المكونات الرئيسية (Principal Component Analysis - PCA) يصوغ هذا الحدس رياضياً: **ابحث عن الاتجاه الذي تتشتت عليه النقاط أكثر ما يمكن، ثم التالي المتعامد معه، وهكذا**.

الصياغة الدقيقة تبدأ من مصفوفة التغاير للبيانات بعد توسيطها:

$$\mathbf{C} = \frac{1}{n-1}\,\mathbf{X}_c^{T}\mathbf{X}_c$$

حيث `X_c` مصفوفة البيانات بعد طرح متوسط كل عمود، و `n` عدد العينات. ثم تُحل هذه المصفوفة المتماثلة إلى اتجاهاتها وقيمها الذاتية:

$$\mathbf{C}\,\mathbf{v}_k = \lambda_k\,\mathbf{v}_k$$

النتيجة وحدها تروي الحكاية: المتجه الذاتي `v_k` هو **اتجاه** مكون رئيسي جديد، والقيمة الذاتية `λ_k` هي **كم تبايناً** يحمله هذا الاتجاه — ترتيب القيم ترتيب أهمية المكونات. دعنا نتحقق بأنفسنا: نبني مصفوفة التغاير يدوياً، نحلها بـ `numpy.linalg.eigh`، ونقارن بما تنتجه `sklearn.decomposition.PCA`:

التحليل الذاتي اليدوي مقابل PCA python

```python
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import numpy as np

X, y = load_iris(return_X_y=True)
Xc = X - X.mean(axis=0)                    # التوسيط خطوة إلزامية
C = (Xc.T @ Xc) / (len(X) - 1)             # مصفوفة التغاير
eigvals, eigvecs = np.linalg.eigh(C)
order = np.argsort(eigvals)[::-1]

print(f"القيم الذاتية يدوياً (تنازلياً): {np.round(eigvals[order], 3)}")
print(f"أول متجه ذاتي يدوياً:            {np.round(eigvecs[:, order[0]], 3)}")

pca = PCA(n_components=2).fit(X)
print(f"قيم explained_variance_:         {np.round(pca.explained_variance_,
3)}")
print(f"مكوّنات components_:             {np.round(pca.components_, 3)}")
```

القيم الذاتية متطابقة تماماً (٤.٢٢٨ ثم ٠.٢٤٣)، أما المتجهات فمتطابقة **بإشارة معكوسة** — وهذا ليس خطأ: إشارة متجه ذاتي اعتباطية تماماً (المحور نفسه برأسين)، والنموذج يختار أحد الوجهين. هذا «التحقق اليدوي» أفضل طريقة لتفكيك الهيبة: PCA ليس صندوقاً سحرياً، بل جبر خطي من سطرين.

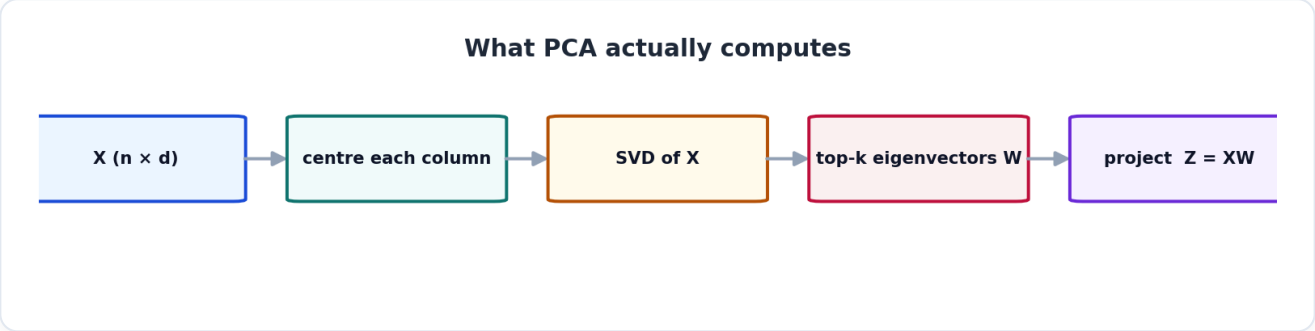
> **مفتاح**
>
> PCA تحويل خطي يعيد تمثيل البيانات على محاور جديدة هي اتجاهات التباين الأكبر، مرتبة تنازلياً بحسب القيم الذاتية. المكوّنات متعامدة دائماً، وكل مكوّن مزيج خطي من الأعمدة الأصلية — ولهذا يبقى كل ما تفعله PCA قابلاً للعكس جزئياً عبر inverse_transform.

٨.٢ PCA عبر SVD: أفضل تقريب رتبة-k

عملياً لا تبني `sklearn` مصفوفة التغاير صراحة — بل تستعين بالتحليل الأنيق الأكثر استقراراً عددياً: تحليل القيم المفردة (Singular Value Decomposition - SVD) للمصفوفة الموسطة:

$$\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \qquad \mathbf{X}_c \approx \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^T$$

حيث `V` أعمدتها هي مكونات PCA نفسها، ومربعات القيم المفردة في `Σ` هي القيم الذاتية. والمكافأة النظرية الكبرى تسمى نظرية Eckart-Young: التقريب `U_k Σ_k V_k^T` ليس مجرد تقريب، بل هو **أفضل تقريب ممكن برتبة** `k` — لا توجد أي مصفوفة أخرى رتبتها `k` تحقق مسافة فروبينيوس (Frobenius norm) أصغر من `X_c`. بمعنى: عندما تقتطع `k` مكوناً، أنت تأخذ أقرب ممثل ممكن للبيانات في فضاء رتبته `k`، والخسارة القصوى معلومة مسبقاً كمجموع القيم الذاتية المهملة.



الشكل ١٤٦خطوات PCA من البيانات الخام: توسيط، تحليل SVD، ثم إسقاط على المحاور الجديدة

الشكل يلخص السلسلة كاملة: توسيط ثم تحليل ثم إسقاط — وكل خطوة قابلة للفحص يدوياً كما فعلنا للتو. من هنا فصاعداً نتقن قراءة ناتج المقدر: ماذا قال كل مكوّن، وكم يحمل، وماذا نفقده حين نتوقف عنده.

٨.٣ تشريح المقدر: components_ و explained_variance_ratio_ و inverse_transform

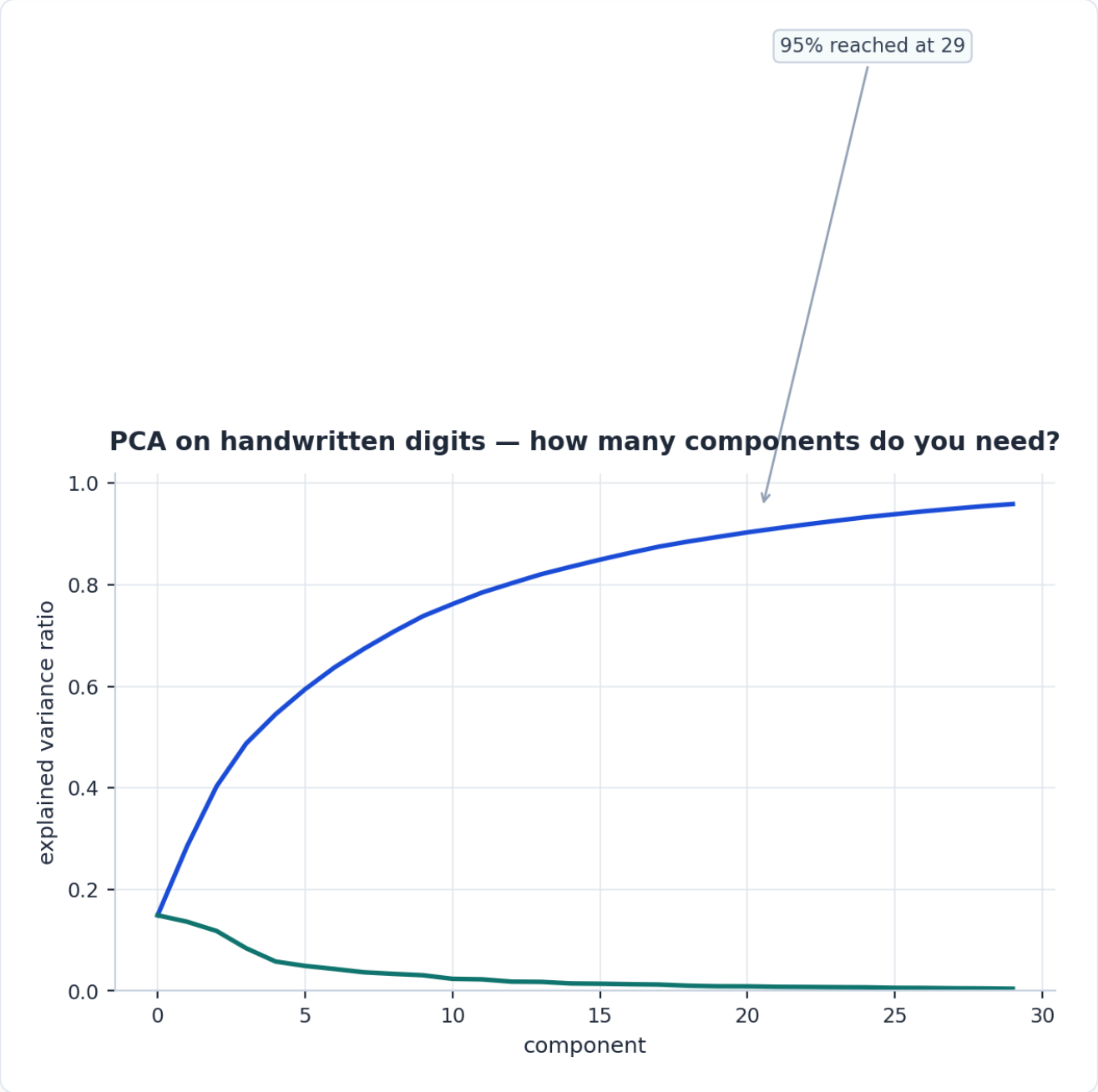
---

المقدر `sklearn.decomposition.PCA` يكشف ثلاث سمات بيتية عليك حفظها كأسماء أسماء: `components_` (المحاور الجديدة: مصفوفة `k × d` كل صف مكوّن)، و `explained_variance_ratio_` (نصيب كل مكوّن من إجمالي التباين)، و `inverse_transform` (العودة إلى الفضاء الأصلي بعد القصّ — جسر قياس الخسارة). اقرأها على Iris، ثم اختبر القرار التلقائي بعتبة نسبية على بيانات الأرقام المكتوبة (٦٤ بُعداً) في تجربة واحدة:

```python
# نسب التباين وخطأ إعادة البناء والاختيار بعتبة 95%
from sklearn.datasets import load_iris, load_digits
from sklearn.decomposition import PCA
import numpy as np

# (أ) Iris: النسب والتراكمي وخطأ إعادة البناء
X, y = load_iris(return_X_y=True)
pca_full = PCA().fit(X)
print(f"نسب التباين المفسَّر: {np.round(pca_full.explained_variance_ratio_, 4)}")
print(f"التراكمي:
{np.round(np.cumsum(pca_full.explained_variance_ratio_), 4)}")
for k in [1, 2, 3]:
    p = PCA(n_components=k).fit(X)
    X_back = p.inverse_transform(p.transform(X))
    print(f"k={k} | خطأ إعادة البناء (MSE): {float(np.mean((X - X_back) ** 2)):.4f}")

# (ب) Digits: 95% القرار التلقائي بعتبة
X_dig, _ = load_digits(return_X_y=True)
pca095 = PCA(n_components=0.95, svd_solver="full").fit(X_dig)
print(f"Digits: {X_dig.shape[1]} بُعداً -> {pca095.n_components_} مكوّناً "
      f"(تباين {pca095.explained_variance_ratio_.sum():.4f} | ضغط {1 -
pca095.n_components_ / X_dig.shape[1]:.1%})")
```

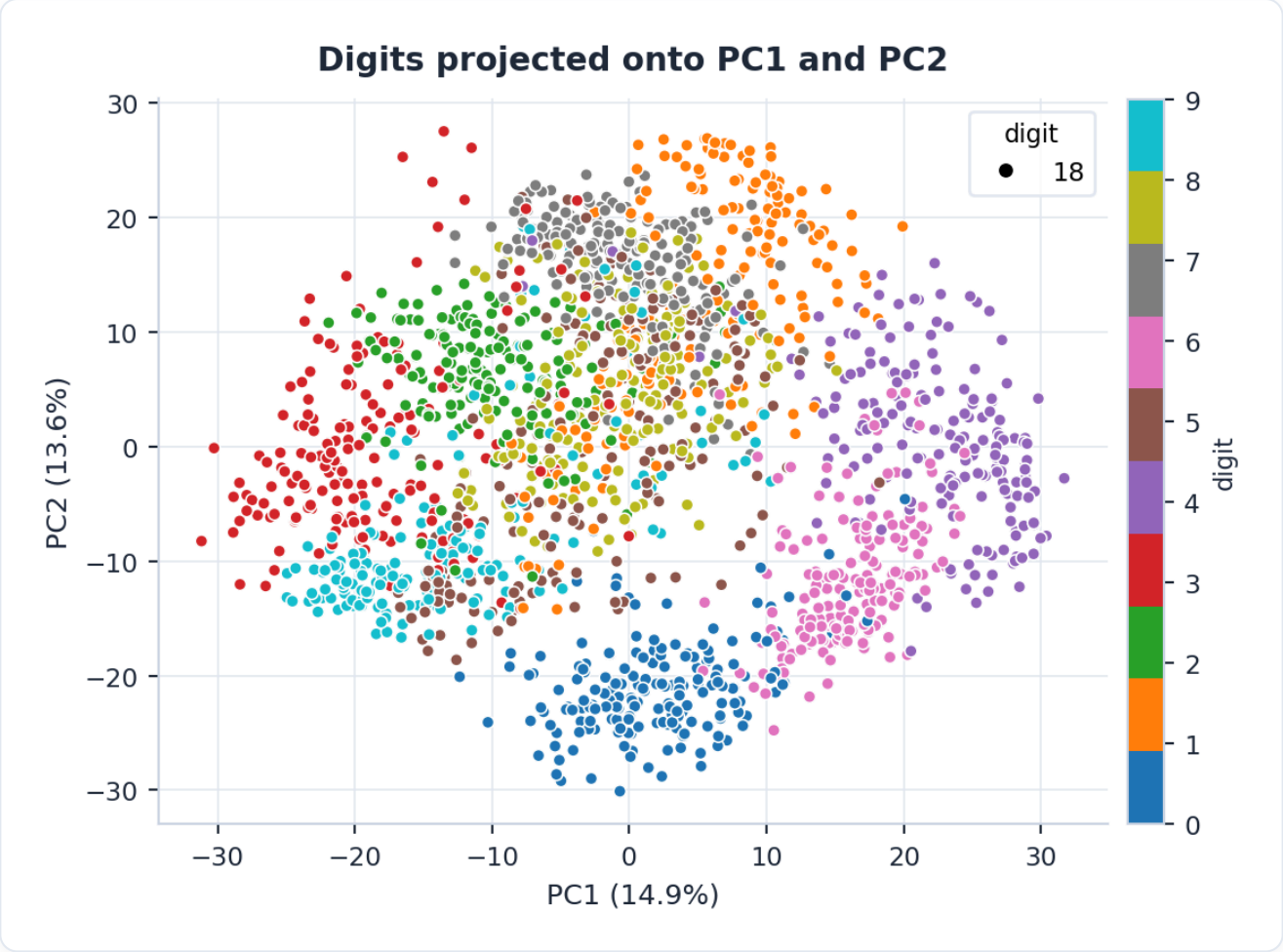
المكوّن الأول وحده يحمل ٠.٩٢٤٦ من تباين Iris — بُعدان حقيقيان اثنان فقط يكفيان تقريباً. وخطأ إعادة البناء يرتفع من ٠.٠٢٥٣ عند `k=2` إلى ٠.٠٨٥٦ عند `k=1`: الخسارة لا تُقدَّر بالشعور بل برقم، وهذا ما يجعل `inverse_transform` أداة هندسية لا ترفاً.

والقرار المنهجي لاختيار `k` يستغني عن التخمين: مرر عتبة نسبية بدل عدد صحيح — عند `n_components=0.95` يختار المقدر وحده أقل عدد مكوّنات يجمع ٩٥% من التباين. الجواب على Digits: ٢٩ مكوّناً من أصل ٦٤ — ضغط ينصف حجم التخزين ويقلص الضجيج، مع الاحتفاظ بنحو كل المعلومة القابلة للقياس. منحنى التباين التراكمي هو البديل الموضوعي لمنحنى الكوع الذي رأيناه في الفصل ٦٠: هنا لا تحتاج عينك، بل عتبة توافقها مع مشروعك.



الشكل ١٤٧ منحنى التباين المفسَّر لكل مكوّن والتراكمي مع عتبة الاختيار

في الشكل يهبط منحنى «لكل مكوّن» هبوطاً حاداً ثم يستقر، ويصعد المنحنى التراكمي إلى العتبة — نقطة تقاطعهما مع خط ٠.٩٥ هي قرارك المؤتمت. أما صورة الإسقاط فعلى هذا المنحنى تماماً.



الشكل ١٤٨البيانات المسقطة على أول مكونين: البنية محفوظة والفئات شبه منفصلة

الإسقاط على أول مكونين يبقي الفئات الثلاث شبه منفصلة — سترى فائدتها العملية في القسم ٨.٦ حين يصبح هذا الفضاء المضغوط مسرحاً لتصنيف لاحق.

٨.٤ فخ المقياس: عمود يسرق المسرح

الآن الشرّ المعروف: PCA يقيس التباين بوحدات البيانات الخام، فالعمود الذي يقيس بالألوف «يصرخ» أعلى من العمود الذي يقيس بأجزاء المليمتر — حتى لو كان مضجراً بلا معنى. القيمة الذاتية تتكلم بوحدة مربعة للوحدة الأصلية، والعمود ذو المقياس الكبير يسرق المكون الأول بالكامل:

فخ العمود ذي المقياس المنفلت python

```python
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np

X, y = load_iris(return_X_y=True)
X_dirty = X.copy()
X_dirty[:, 0] *= 1000                    # عمود منطقي بمقياس منفلت

pca_raw = PCA(n_components=2).fit(X_dirty)
pca_scaled = PCA(n_components=2).fit(StandardScaler().fit_transform(X_dirty))
print(f"نصيب المكون الأول بدون تطبيع: {pca_raw.explained_variance_ratio_[0]:.4f}")
print(f"نصيب المكون الأول مع تطبيع:   {pca_scaled.explained_variance_ratio_[0]:.4f}")
print(f"أوزان المكون الأول بدون تطبيع: {np.round(pca_raw.components_[0], 4)}")
```

الناتج صادم للمبتدئ: بلا تطبيع يعلن المكون الأول أنه يحمل ١٠٠٪ من التباين، وأوزانه كلها مركزة في العمود المضخم (وزن ١.٠ له والأصفار للبقية) — «التحليل» لم يفعل شيئاً سوى الإصغاء لأعلى صوت. بعد `sklearn.preprocessing.StandardScaler` يهتدئ النصيب إلى ٠.٧٢٩٦ وتعود الأوزان لتفكر كل الأعمدة.

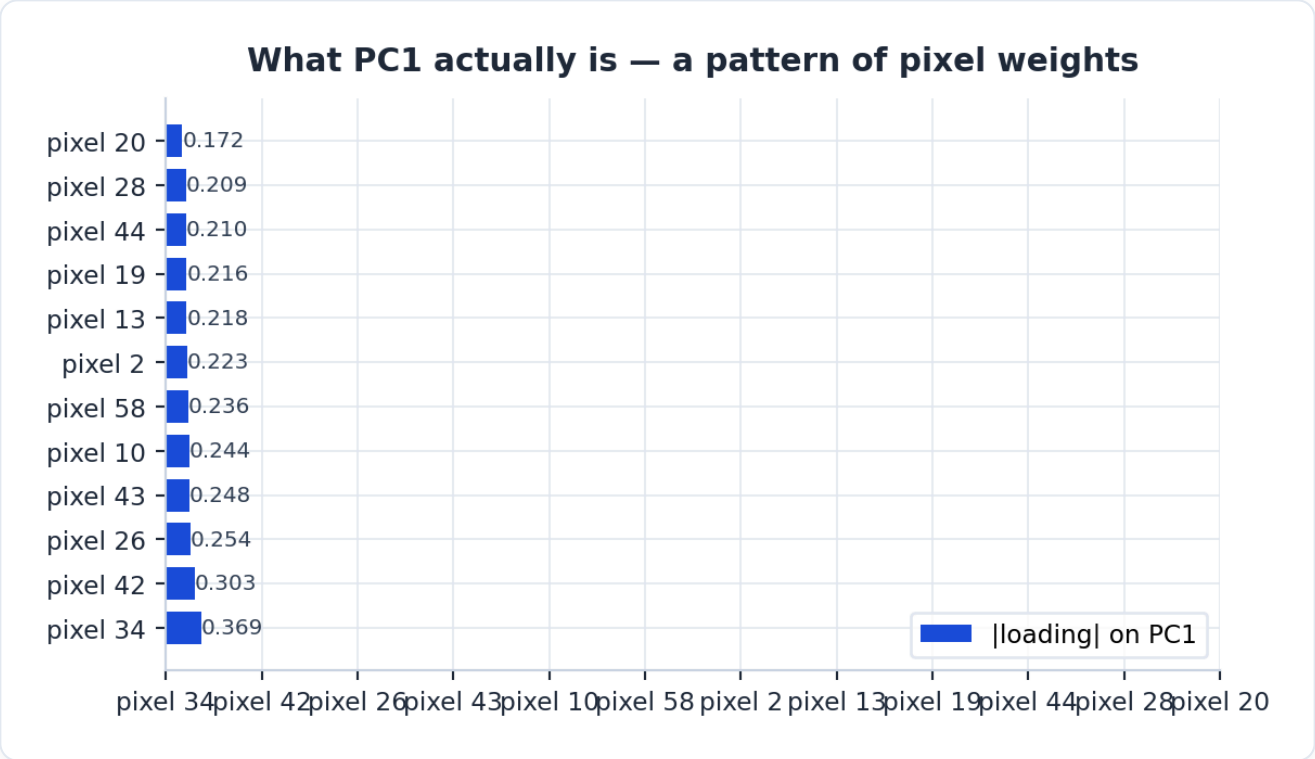
> **خطأ شائع**
>
> تشغيل PCA على بيانات خام بوحدات مختلطة خطأ يفسد كل قراءة لاحقة: نسب التباين، والأوزان، وخطأ إعادة البناء — كلها مختطفة من عمود واحد. القاعدة صلبة: **StandardScaler (أو ما يناسب بياناتك من محولات الفصل ١٤) سابق إلزامي لـ PCA**، إلا إذا كانت أعمدتك بنفس الوحدة والمقياس فعلاً.

٨.٥ قراءة المكونات: loadings كتفسير

السر الأكبر في PCA أنه **قابل للتفسير**: كل صف في `components_` هو أوزان مزيج خطي للأعمدة الأصلية، وتُسمى هذه الأوزان الحمولات (loadings). بقراءتها تحول «المكوّن ١» من رمز مجرد إلى معنى: ما العمود الذي يرفعه، وما العمود الذي يخفضه؟ أعرضها كجدول مرتب على Iris المطّبعة:

```python
# جدول الحمولات: أوزان كل مكوّن على الأعمدة الأصلية
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd

X, y = load_iris(return_X_y=True)
feat = load_iris().feature_names
Xs = StandardScaler().fit_transform(X)
pca = PCA(n_components=2).fit(Xs)

loadings = pd.DataFrame(pca.components_.T, index=feat, columns=["PC1",
"PC2"]).round(3)
print(f"نسب التباين: PC1 = {pca.explained_variance_ratio_[0]:.3f}، PC2 =
{pca.explained_variance_ratio_[1]:.3f}")
loadings
```

اقرأ الصفوف لا الأعمدة فقط: طول البتلة وعرضها يحملان أعلى أوزان موجبة في `PC1` (نحو ٠.٥٨ و ٠.٥٦٥) مع إشارة سالبة لعرض الكأس (-٠.٢٦٩) — أي أن المكوّن الأول مقياس «حجم البتلة». أما `PC2` فيسيطر عليه عرض الكأس وحده (وزن ٠.٩٢٣): تباين الكأس في عرضها كيان شبه مستقل عن باقي القياسات. هذه القراءة هي ما يجعل PCA أداة اكتشاف بنية لا مجرد ضاغط.



الشكل ١٤٩ خريطة الحمولات: وزن كل عمود أصلي في كل مكون رئيسي

في الشكل تظهر الحمولات كأعمدة لكل مكون: قيم موجبة كبيرة تعني «هذا العمود يبني هذا المكوّن»، والإشارات المتعارضة تكشف العلاقات المعاكسة بين الأعمدة التي كان التباين يخفيها.

> **في الواقع**
>
> في السجلات الطبية والمسوح الاجتماعية، تُستخدم قراءة الحمولات لتسمية العوامل الكامنة: مكون ثقيل الأوزان على أسئلة «القلق واضطراب النوم» يُسمى عامل التوتر. لكن حذارِ من التسرع: التسمية فن بعد الحساب، ويمكن أن تخدعك إشارات الأوزان إذا لم تُطبع بياناتك أولاً — قاعدة القسم ٨.٤ تنطبق حرفياً هنا.

٨.٦ PCA قبل المصنف: أسرع وأفضل أحياناً

لماذا نصنف على فضاء مضغوط أصلاً؟ ثلاث دوافع: كلفة أقل (مسافات على أبعاد أقل)، وضجيج أقل (المكونات الهامشية غالباً خربشة)، وسلاسة أنابيب `sklearn.pipeline` التي رأيناها في الفصل ٣١. لكن الحقيقة تكمل نصفها الآخر: على بيانات نظيفة كلها إشارة، فقد يدفع القصّ ثمناً بسيطاً في الدقة. التجربة المزدوجة تكشف القاعدة كاملة:

```python
# متى يفيد ومتى يؤذي؟ PCA بعد KNN
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import numpy as np

X, y = load_iris(return_X_y=True)

for k in [3, 7]:
    a = cross_val_score(make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=k)), X, y, cv=10).mean()
    b = cross_val_score(make_pipeline(StandardScaler(), PCA(n_components=2),
KNeighborsClassifier(n_neighbors=k)), X, y, cv=10).mean()
    print(f"iris نظيفة و k={k}: KNN مباشر {a:.4f} | بعد PCA(2) {b:.4f}")

rng = np.random.default_rng(42)
X_noisy = np.hstack([X, rng.normal(size=(len(X), 5))])   # خمسة أعمدة ضجيج
for k in [7]:
    a = cross_val_score(make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=k)), X_noisy, y, cv=10).mean()
    b = cross_val_score(make_pipeline(StandardScaler(), PCA(n_components=2),
KNeighborsClassifier(n_neighbors=k)), X_noisy, y, cv=10).mean()
    print(f"iris + 5 أعمدة ضجيج: KNN مباشر {a:.4f} | بعد PCA(2) {b:.4f}")
```

الدرس مزدوج ولا يقبل الاختصار: على Iris النظيفة كلف القصّ طفيفاً في الدقة (٠.٩٥٣٣ إلى ٠.٩١٣٣ عند `k=3`) — فالأبعاد الأربعة كلها إشارة. أما حين أضفنا خمسة أعمدة ضجيج فهبط KNN المباشر إلى ٠.٨٨٦٧ بينما استعاد PCA(٢) الأداء إلى ٠.٩١٣٣، لأن المكونات الهامشية التي حملت الضجيج أُلقيت خارج الطاولة. **القاعدة: PCA قبل k-NN يفيد عندما تحمل الأبعاد الإضافية ضجيجاً أو تكراراً، وقد تؤذي حين تكون كل الأبعاد إشارة** — واحكم بتحقق متقاطع لا بذوق.

> **نصيحة**
>
> بنِ التجربة كاملة في سطر واحد عبر التطبيع ثم `make_pipeline(StandardScaler(), PCA(n_components=0.95), KNeighborsClassifier())` : الاختيار التلقائي ثم التصنيف. أنبوب واحد يعالج فخ القسم ٨.٤ ويقرر عدد المكونات آلياً في كل تجزئة تحقق متقاطع من دون تسريب.

٨.٧ نسخ PCA العملية: متفرقة، ضخمة، عشوائية، وغير خطية

النسخة الأكاديمية تكفي للمختبر؛ المشاريع الحقيقية تحتاج نسخاً للحالات الحدية، وكلها تحت سقف `sklearn.decomposition` :

الأداة	حالتها المثالية	سرها
`TruncatedSVD`	مصفوفات متفرقة (نصوص)	بلا توسيط — يحافظ على الأصفار
`IncrementalPCA`	بيانات لا تتسع في الذاكرة	تدريب بدفاتر عبر `partial_fit`
`PCA(svd_solver='randomized')`	بيانات كبيرة وعدد مكونات صغير	تقريب عشوائي مبني على النظرية
`KernelPCA`	بنية غير خطية	حيلة النواة قبل التحليل الذاتي

أولاً **TruncatedSVD**: مصفوفات النصوص المتفرقة التي أنتجها `TfidfVectorizer` في الفصل ١٨ لا تحتمل التوسيط — طرح المتوسط من متجه متفرق يملؤه أصفاراً ويدمر التنظيف الحسابي الذي جعلها متفرقة أصلاً. `TruncatedSVD` يتخطى التوسيط كلياً ويعمل على المتفرقة مباشرة، وهو حجر الأساس في تحليل الدلالة الخفي LSA الذي سيتفصله الفصل ٧١:

TruncatedSVD و IncrementalPCA والمحلّل العشوائي

```python
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA, IncrementalPCA, TruncatedSVD
from scipy import sparse
import numpy as np

X_dig, _ = load_digits(return_X_y=True)

# 1) TruncatedSVD: من متفرقة إلى مضغوطة بلا توسيط
X_sp = sparse.csr_matrix(X_dig)
svd = TruncatedSVD(n_components=20, random_state=42).fit(X_sp)
print(f"TruncatedSVD: {X_sp.shape} -> 1797×20 | تباين مفسَّر:
{svd.explained_variance_ratio_.sum():.4f}")

# 2) IncrementalPCA: تدريب PCA الكامل بدفاتر يوازي
ipca = IncrementalPCA(n_components=20, batch_size=450)
for chunk in np.array_split(X_dig, 4):
    ipca.partial_fit(chunk)
pca = PCA(n_components=20, svd_solver="full").fit(X_dig)
rel = np.abs(ipca.explained_variance_ - pca.explained_variance_) /
pca.explained_variance_
print(f"IncrementalPCA: متوسط فرق نسبي في القيم الذاتية {rel.mean():.2%} | تباين تراكمي
{ipca.explained_variance_ratio_.sum():.4f}")

# 3) المحلّل العشوائي: نفس النتيجة بزمن أقل على البيانات الكبيرة
pca_r = PCA(n_components=20, svd_solver="randomized",
random_state=42).fit(X_dig)
print(f"randomized: تباين مفسَّر {pca_r.explained_variance_ratio_.sum():.4f}")
```

ثلاث نتائج تحسم ثلاث قرارات: TruncatedSVD عصب التباين المفسَّر على المتفرقة بلا أي توسيط؛ و IncrementalPCA بلغة الدفاتر — أربع دفعات عبر `partial_fit` — تقترب من PCA الكامل بفرق نسبي متوسط ١.٢٥% فقط، وهي الطريقة الوحيدة الممكنة حين لا تتسع البيانات في الذاكرة أصلاً؛ والمحلّل العشوائي يطابق النسب التراكمية (٠.٨٩٤٣ مقابل ٠.٨٩٤٣) بزمن ينخفض مع البيانات الضخمة لأن كلفته تنمو مع عدد المكونات لا مع الأبعاد.

أما **KernelPCA** فهي وعدٌ للمستقبل: قبل التحليل الذاتي تمرر النقاط عبر حيلة النواة التي رأيتها مع SVM في الفصل ٥٢، فتصبح قادرة على «فتح» البنيات المنحنية التي يعجز PCA الخطي عنها — لكن ثمنها مصفوفة نواة `n × n` تحرمها من البيانات المليونية. سنقيس حدها ضد Isomap و TSNE في الفصل ٦٩.

سياق

وُلد PCA عام ١٩٠١ من قلم كارل بيرسون بحثاً عن «خطوط وانسداد انحدار في الفضاء»، ثم أعاد هارولد هوتلينغ صياغته إحصائياً عام ١٩٣٣ عبر التحليل الذاتي. التحليل العشوائي الذي تستدعيه بـ `svd_solver='randomized'` لم يكن إلا بعد قرن: أوراق Halko وآخرين عام ٢٠١١ حوّلت التقريب العشوائي من خدعة مقامرة إلى خوارزمية مضمونة الخطأ — وهذا أحد أجمل الأمثلة على نضج النظرية قبل الاستخدام.

تمارين الفصل الثامن والستين

- **ملاحظة**

التمرين ٨.١

على أول ٤٠٠٠ صف من بيانات California Housing: اضغطها بعتبة ٩٥% بعد التطبيع، واذكر عدد المكونات ونسبة الضغط، ثم قِس خطأ إعادة البناء بوحدة «انحراف معياري» بعد العودة عبر inverse_transform.

التمرين ٨.٢

تحقق عملياً من تعامد مكونات PCA على بيانات Digits: احسب حواصل ضرب الصفوف في `components_` بعضها، وفسّر لماذا يجب أن تقترب من الصفر، ولماذا يختلف إشارةُ المكون الواحد بين تشغيلتين مختلفتي random_state.

التمرين ٨.٣

على بيانات الهلالين `make_moons` : قارن سلسلة `PCA(n_components=2)` يليها `LogisticRegression` مع سلسلة `KernelPCA(n_components=2, kernel='rbf')` يليها المصنف نفسه. أي السلسلتين تفصل الهلالين، ولماذا؟

- **ملاحظة**

حل التمرين: ضغط California Housing بعتبة 95% python

```python
from sklearn.datasets import fetch_california_housing
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np

data = fetch_california_housing()
X = data.data[:4000]
Z = StandardScaler().fit_transform(X)

pca = PCA(n_components=0.95, svd_solver="full").fit(Z)
print(f"الأبعاد الأصلية: {Z.shape[1]}")
print(f"95% مكونات تحقق: {pca.n_components_} | نسبة الضغط: {1 - pca.n_components_ /
Z.shape[1]:.1%}")

Z_back = pca.inverse_transform(pca.transform(Z))
rmse = float(np.sqrt(np.mean((Z - Z_back) ** 2)))
print(f"خطأ إعادة البناء (RMSE بوحدة الانحراف المعياري): {rmse:.4f}")
```

ستة مكونات من ثمانية تحقق ٩٥% — الضغط متواضع هنا لأن أعمدة Housing منخفضة التداخل أصلاً، وخطأ إعادة البناء نحو ٠.١٢ انحراف معياري فقط لكل عمود. خذ العبرة المنهجية: نسبة الضغط ثمينة عندما تكون الأعمدة كثيرة ومتشابهة، وتذبل مع الأعمدة القليلة المستقلة.

> **ملاحظة**

حل التمرين: PCA الخطي مقابل KernelPCA على الهلالين python

```python
from sklearn.datasets import make_moons
from sklearn.decomposition import PCA, KernelPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline

X, y = make_moons(n_samples=600, noise=0.12, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

lin = make_pipeline(PCA(n_components=2),
LogisticRegression(max_iter=1000)).fit(X_tr, y_tr)
ker = make_pipeline(KernelPCA(n_components=2, kernel="rbf", gamma=15),
                    LogisticRegression(max_iter=1000)).fit(X_tr, y_tr)
print(f"LogisticRegression بعد PCA الخطي:   {lin.score(X_te, y_te):.4f}")
print(f"LogisticRegression بعد KernelPCA:   {ker.score(X_te, y_te):.4f}")
```

الخطي عجز (٠.٨٧٧٨): إسقاط الهلالين على أفضل خط مستقيم يخلط طرفيهما بعضهما ببعض، فتعجز الحد اللوجيستي عنهما. أما KernelPCA فقد رفع الدقة إلى ٠.٩٨٣٣ لأنه وحّد كل هلال في عنقود فضاء النواة ثم مرر التحليل الذاتي عليه. الحد الفاصل بين الأداتين هو طبيعة البنية: خطية تكفي PCA، وانحناء يستدعي النواة — وكل ذلك مفصل في الفصل ٦٩.

> **ملاحظة**

٨.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تطبيق PCA على بيانات خام بوحدات مختلطة	العمود ذو المقياس الأكبر يستحوذ على المكون الأول ويبلّع كل النسب	`StandardScaler().fit_transform` قبل `PCA` دائماً في أنبوب واحد
اختيار `n_components` بالعين على الرسم	قرارات متقلبة بين المشاهدين وتشغيلات مختلفة	عتبة نسبية موضوعية `n_components=0.95` أو عتبة تراكمية معلنة
استخدام `PCA` على مصفوفة نصوص متفرقة	التوسيط يملأ الأصفار فتنفجر الذاكرة وتضيع التفرقة	`TruncatedSVD` الذي يتخطى التوسيط ويعمل على المتفرقة مباشرة
تفسير إشارة أوزان `components_` كمعنى ثابت	إشارة المكون تنقلب بين تشغيلات فتنقلب استنتاجاتك معها	فسّر القيم المطلقة وأنماط الإشارات المشتركة داخل المكوّن الواحد

٨.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تحليل المكونات الرئيسية	تحويل خطي يعيد تمثيل البيانات على اتجاهات التباين الأكبر المتعامدة	Principal Component Analysis (PCA)
تحليل القيم المفردة	تفكيك المصفوفة إلى دورانين ومقياس؛ خلفية PCA الحسابية	Singular Value Decomposition (SVD)
نسبة التباين المفسَّر	نصيب المكوّن من إجمالي تباين البيانات الأصلية	Explained variance ratio
الحمولات	أوزان الأعمدة الأصلية داخل كل مكون رئيسي؛ وسيلة التفسير	Loadings
إعادة البناء	العودة من الفضاء المضغوط إلى الأصلي لقياس ما فُقد	Inverse transform / reconstruction

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٦٩ — تقليل الأبعاد غير الخطي

KernelPCA و Isomap و LLE و MDS و TSNE وموثوقية الإسقاط

الوحدة **scikit-learn ١.٩** الحجم **١٠٢٧** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا تفشل الإسقاطات الخطية على بيانات المانيفولد كـ make_swiss_roll ومتى تكفي
- أن تميّز بين عائلة المسافات العامة (KernelPCA و MDS) وعائلة الجوار المحلي (Isomap و LLE و SpectralEmbedding) وعائلة التوزيعات (TSNE)
- أن تطبق sklearn.manifold.trustworthiness لقياس ما نجا من الجوار الأصلي بعد الإسقاط
- أن تضبط gamma و n_neighbors و perplexity، وتتجنب فخّي المسافات بين بلورات t-SNE ونداء transform عليها

٨.١ حين يخدعك الخط المستقيم: لماذا يفشل PCA على اللفافة؟

تخيّل ورقة عليها نقاط ملوّنة، لُفّت ببطء على شكل لفافة رقيقة وغُمست في فضاء ثلاثي الأبعاد. سؤال بسيط: ما المسافة بين نقطتين في طرفي الورقة؟ إن أجبت بالمسطرة — أي بالمسافة الإقليدية في الفضاء المحيط — فستحصل على مسافة قصيرة: نقطتا الطرفين تتلاقيان تقريباً حين تُكمل اللفافة دورة كاملة. لكن الحقيقة هي **المسافة على الورقة نفسها**: عليك المشي على السطح لا أن تخرق الهواء بينهما.

خدعة `make_swiss_roll` هي هذه بالضبط: البيانات تجلس فعلاً على سطح ثنائي الأبعاد منحنٍ، لكن المسافات الإقليدية في الفضاء المحيط تخدع أي خوارزمية تراه فقط. و PCA — كما رأينا في الفصل ٦٨ — يفترض أن أفضل وصف للبيانات مستوٍ مستقيم. ماذا يحدث إذا سلّمناه لفافة؟

🔑 مفتاح

المفهوم المحوري للفصل: حين تجلس البيانات على **مانيفولد (manifold)** — سطح منخفض الأبعاد منحنٍ مدسوس في فضاء أعلى — فالمهم هو الهندسة *على السطح* (المسافة الجيوديسية) لا الهندسة *في الفضاء المحيط* (المسافة الإقليدية). كل تقنيات هذا الفصل ليست إلا محاولات مختلفة لاستعادة تلك الهندسة المخفية.

أين تنهار الحقيقة؟ S: على اللفافة ومنحنى PCA — python

```python
from sklearn.datasets import make_swiss_roll, make_s_curve
from sklearn.decomposition import PCA
from sklearn.manifold import trustworthiness
import numpy as np

X, t = make_swiss_roll(n_samples=500, noise=0.10, random_state=42)
Z = np.asarray(PCA(n_components=2, random_state=42).fit_transform(X))

# كم من الجوار الأصلي نجا بعد الإسقاط الخطي؟
trust = trustworthiness(X, Z, n_neighbors=8)

# ابحث عن أكثر زوج خُدع: بعيد جداً على الورقة، متلاصق في الإسقاط
n = len(X)
D3 = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(-1))
D2 = np.sqrt(((Z[:, None, :] - Z[None, :, :]) ** 2).sum(-1))
iu = np.triu_indices(n, k=1)
scam = (D3[iu] - D2[iu]).argmax()
i, j = iu[0][scam], iu[1][scam]

print(f"موثوقية إسقاط PCA على اللفافة: {trust:.4f}")
print(f"أشد زوج خُدع: مسافته على اللفافة = {D3[i, j]:.2f}, وبعد الإسقاط = {D2[i, j]:.2f}")

Xs, _ = make_s_curve(n_samples=500, noise=0.08, random_state=42)
Zs = np.asarray(PCA(n_components=2, random_state=42).fit_transform(Xs))
print(f"موثوقية PCA على منحنى S: {trustworthiness(Xs, Zs, n_neighbors=8):.4f}")
```

الرقم الأول حاسم: `trustworthiness` — سنشرح معادلته لاحقاً — يقيس نسبة الجوار الأصلي الذي نجا في الإسقاط، وقد خسر PCA أكثر من ٥٠٪ من الجوار بصمت، مع أن نسبة التباين المحفوظة كانت ستخبرك أن كل شيء بخير! والرقم الثاني هو الكارثة الملموسة: زوجٌ تفصل بينهما ٢٤ وحدة على الورقة أصبح بينهما ١.٨٥ فقط في الصورة المفلطحة؛ أي مصنّف أو خوارزمية تجميع ستقرأ هذا الإسقاط ستظن النقطتين متطابقتين. ومنحنى S يوقع الخدعة الإقليدية نفسها (٠.٩٣٣٩): التباين محفوظ والهندسة ممزّقة — المقياس الخطأ يمنح ثقة زائفة.

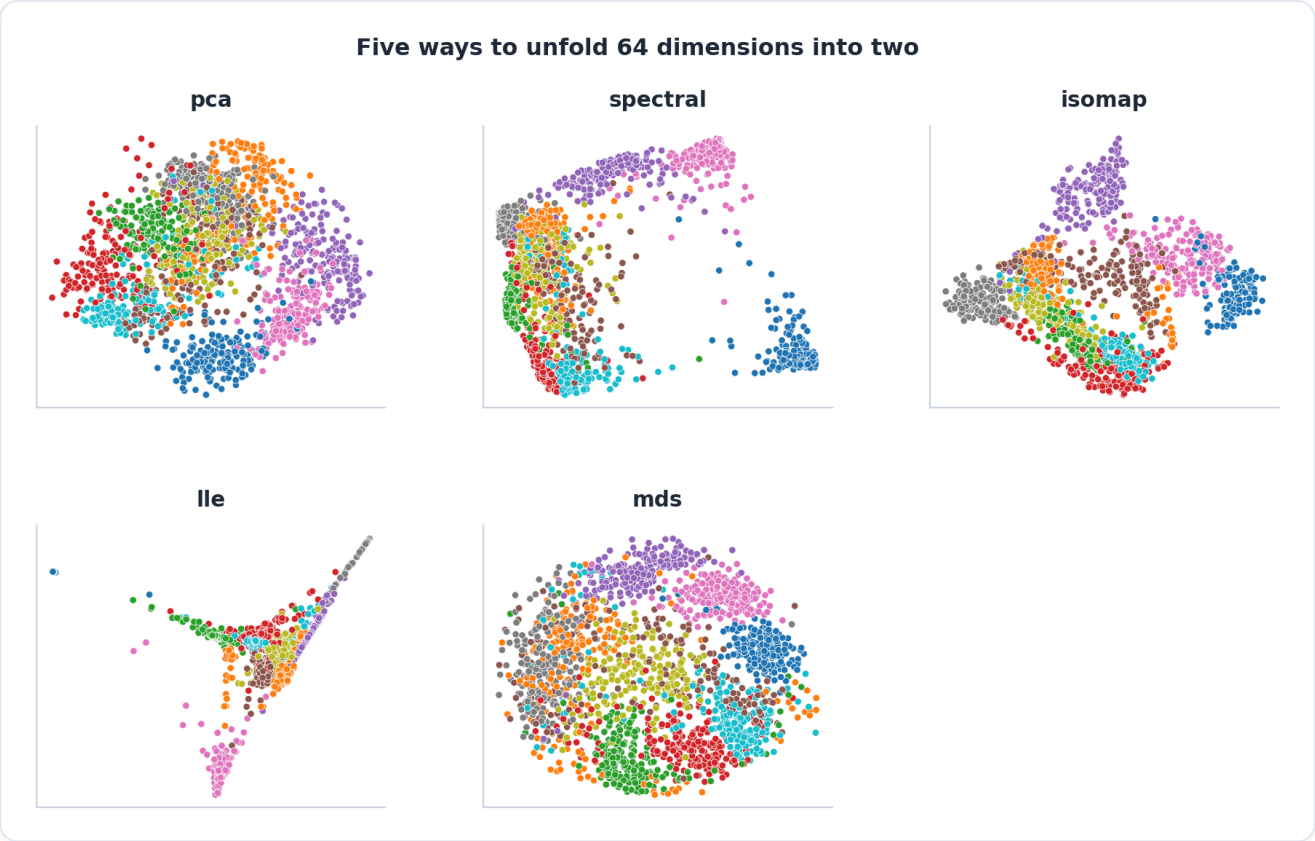
٨.٢ فلسفة المانيفولد: البيانات تجلس على سطح منحني

فرضية المانيفولد تقول: البيانات عالية الأبعاد — بكسلات صورة، إشارات مستشعرات، تعبير جيني — نادرة التوزيع في الفضاء الكامل؛ فهي متركزة على سطح منخفض الأبعاد لا تنحني بالضرورة مستقيمة. مهمة تقليل الأبعاد غير الخطي هي **فتك الطيّات**: أن نجد تمثيلاً ثنائياً يظل فيه «قريباً» يعني قريباً على السطح، لا قريباً في الهواء.

كيف تحاول العائلات الست فكّ الطيّة؟ كل واحدة تراهن على زاوية مختلفة:

التقنية	الفكرة المركزية	البارامتر الحاسم	تعامل مع بيانات جديدة؟
KernelPCA	خدعة النواة: PCA في فضاء ميزات غير خطي	`gamma`	نعم عبر `transform`
MDS	حافظ على المسافات الزوجية كما هي	`metric_mds`	لا
Isomap	استبدل الإقليدية بالجيوديسية عبر رسم الجيران	`n_neighbors`	نعم عبر `transform`
LLE	أعد بناء كل نقطة من جيرانها ثم ادمج الأوزان	`n_neighbors`	لا
SpectralEmbedding	قطّع رسم التشابه بالمتجهات الذاتية للابلاسيان	`n_neighbors`	لا
TSNE	حافظ على توزيعات احتمالية الجوار بتوزيع الطالب	`perplexity`	لا إطلاقاً

الشكل التالي يجرب خمساً منها على أرقام مكتوبة بخط اليد (`load_digits`، ٦٤ بُعداً): انظر كيف يبقى إسقاط PCA الخطي سحابة ملتصقة، بينما تكاد Isomap و LLE تفصلان الأرقام في مركزيهما.



الشكل ١٥٠خمس تقنيات لفتك ٦٤ بُعداً إلى بُعدين على بيانات الأرقام: الخطية تلتصق، والمانيفولد تتفكك

كل بلورة «خريطة» مختلفة للبيانات نفسها؛ لا يوجد شكل صحيح واحد بل شكل يجيب عن سؤالك: أي علاقات تريد أن تبقى صادقة؟

> **تعمّق**
>
> المسافة الجيوديسية (geodesic distance) في Isomap تُقدَّر بخطوتين: صل كل نقطة بأقرب `n_neighbors` جيران لها لتبني رسماً، ثم احسب أقصر مسار داخل الرسم بدايكسترا أو فلويد-وارشال. حين يتصل الرسم بكثافة كافية يقارب أقصر مسار فيه المسافة على السطح — وهذا «المشي على الورقة» الذي طلبناه.

٨.٣ التجربة المركزية: ست تقنيات على اللفافة السويسرية

كفى نظريةً: لفافة واحدة (٥٠٠ نقطة، ضجيج ٠.١٠) تمر عبر ست تقنيات، نقيس لكل واحدة زمن التدريب والموثوقية عند `n_neighbors=8`، مع PCA خطي كخط أساس.

عائلة المسافات العامة أولاً — «شبه خطية» ترى العالم من عُلو:

```python
# العائلة العامة: KernelPCA و MDS والخط الأساس
from sklearn.decomposition import PCA, KernelPCA
from sklearn.manifold import MDS, trustworthiness
import numpy as np
import time

methods_global = {
    "PCA": PCA(n_components=2, random_state=42),
    "KernelPCA": KernelPCA(n_components=2, kernel="rbf", gamma=0.03,
random_state=42),
    "MDS": MDS(n_components=2, n_init=1, max_iter=100, init="classical_mds",
random_state=42),
}
for name, m in methods_global.items():
    t0 = time.perf_counter()
    Z = np.asarray(m.fit_transform(X))
    dt = (time.perf_counter() - t0) * 1000
    print(f"{name:12s} | زمن: {dt:7.1f} ms | موثوقية: {trustworthiness(X, Z,
n_neighbors=8):.4f}")
```

ثم عائلة الجوار المحلي — «تسكن على السطح» نفسه — وقد أضفنا إليها t-SNE التي سيفرد لها القسم التالي:

```python
# عائلة الجوار: Isomap و LLE و Spectral و TSNE
from sklearn.manifold import Isomap, LocallyLinearEmbedding, SpectralEmbedding,
TSNE, trustworthiness
import numpy as np
import time

methods_local = {
    "Isomap": Isomap(n_neighbors=12, n_components=2, n_jobs=-1),
    "LLE": LocallyLinearEmbedding(n_neighbors=12, n_components=2,
random_state=42),
    "SpectralEmbedding": SpectralEmbedding(n_neighbors=12, n_components=2,
random_state=42),
    "TSNE": TSNE(n_components=2, perplexity=30, init="pca",
learning_rate="auto", random_state=42, n_jobs=-1),
}
for name, m in methods_local.items():
    t0 = time.perf_counter()
    Z = np.asarray(m.fit_transform(X))
    dt = (time.perf_counter() - t0) * 1000
    print(f"{name:20s} | زمن: {dt:7.1f} ms | موثوقية: {trustworthiness(X, Z,
n_neighbors=8):.4f}")
```

اجمع النتائج في جدول واحد (الأزمنة حية في المخرجات أعلاه؛ والموثوقية ثابتة بالبذور):

التقنية	العائلة	الموثوقية (k=٨)	التكلفة النسبية
PCA	خطي	٠.٩٤٢٢	زهيدة جداً
KernelPCA (rbf)	نواة	٠.٨٦١٠	زهيدة
MDS	مسافات عامة	٠.٩٣٦٢	باهظة
Isomap (k=١٢)	جيوديسية	٠.٩١٥٣	متوسطة
LLE (k=١٢)	إعادة بناء محلية	٠.٨٦٩٢	رخيصة
SpectralEmbedding (k=١٢)	طيفية	٠.٩٦٠٢	متوسطة
TSNE (p=٣٠)	توزيعات	٠.٩٩٩٠	الأغلى عملياً

ثلاثة دروس من الجدول:

> **خطأ شائع**
>
> كثير من المبتدئين يقرأون الجدول فيقررون: «TSNE دائماً الأفضل». هذا استنتاج خاطئ من تجربة واحدة. t-SNE حفظ الجوار المحلي فقط؛ الجدول يقيس الجوار المحلي حتماً، فهو دائماً سيبدو معلقاً على المنصة! غيّر السؤال إلى «هل أحتاج البنية الكلية؟» وستحتاج حينها Isomap أو MDS لا t-SNE.

وأما MDS — الابن الأكبر لهذه العائلة، ويعود إلى عام ١٩٦٤ — فيصوغ المسألة كاستمثال: ابحث عن إحداثيات واطئة تجعل **الإجهاد (stress)** — مجموع مربعات فوارق المسافات الزوجية — أصغر ما يمكن:

$$\text{Stress} = \left(\sum_{i<j} (d_{ij} - \delta_{ij})^2\right)^{1/2}$$

حيث `d_ij` المسافة بين النقطتين في التمثيل الجديد و`delta_ij` المسافة الأصلية بينهما. الصيغة المترية (metric) تسعى لحفظ قيم المسافات نفسها، والصيغة غير المترية (non-metric) تكتفي بحفظ **رتبة** المسافات: الأقرب يبقى أقرب والبعيد يبقى بعيداً. اخترت بينهما في scikit-learn بالبارامتر `metric_mds`؛ والبارامتر القديم `metric=True/False` أُهمل في ١.٩ لصالحه.

وبالمثل صياغة LLE (Locally Linear Embedding) يقوم على ملاحظة بديهية: إذا كان السطح ملساء محلياً، فكل نقطة تقريباً **مزيج خطي من جيرانها**. المرحلة الأولى تعثر على الأوزان التي تعيد بناء كل نقطة من جيرانها بأقل خطأ:

$$\min_{W} \sum_{i=1}^{n} \left\| \mathbf{x}_i - \sum_{j \in N(i)} w_{ij}\, \mathbf{x}_j \right\|^2, \quad \text{such that} \sum_{j \in N(i)} w_{ij} = 1$$

حيث `N(i)` مجموعة الجيران الأقرب للنقطة `i` و`w_ij` وزن إعادة البناء الذي مجموعه واحد لكل نقطة. ثم تُثبَّت الأوزان ويُبحث عن إحداثيات جديدة `y_i` تحقق أصغر خطأ إعادة بناء بالأوزان نفسها — فتُنقل الملاحظة المحلية إلى الفضاء الجديد عند فك الطيّة.

٨.٤ t-SNE: التوزيعات لا المسافات

تقطع t-SNE (t-distributed Stochastic Neighbor Embedding) خيطاً مختلفاً كلياً: بدل الحفاظ على مسافات، حوّل الجوار إلى **توزيع احتمالي**. لكل نقطة `i` نعرّف احتمال أن تختار النقطة `j` جاراً لها بتوزيع غاوسي حولها:

$$p_{j|i} = \frac{\exp(-d_{ij}^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-d_{ik}^2 / 2\sigma_i^2)}$$

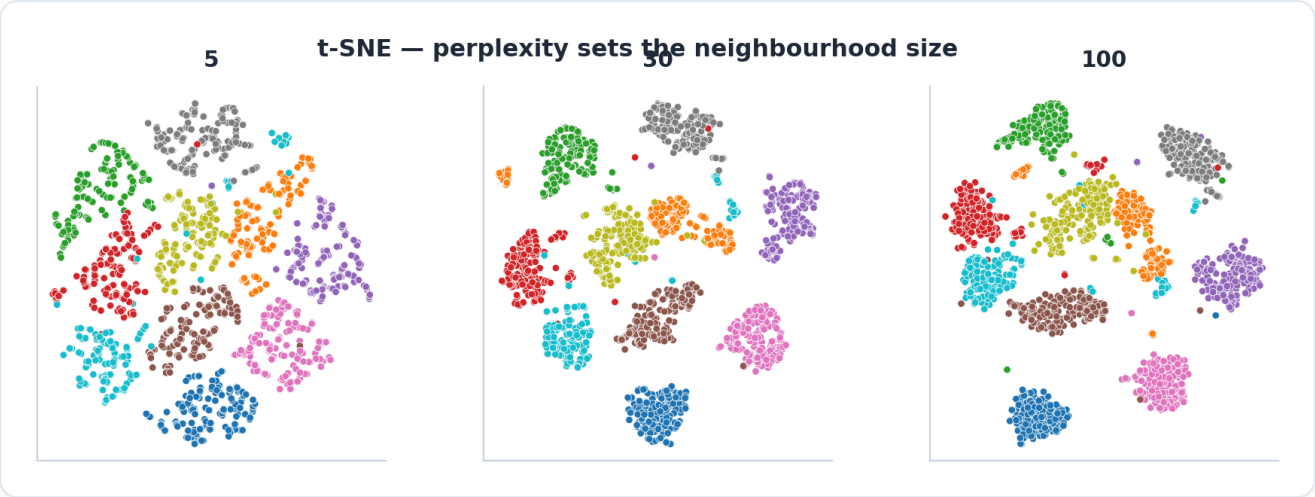
حيث `d_ij` المسافة الإقليدية بين النقطتين في الفضاء الأصلي، و`sigma_i` عرض نواة غاوسية **يُحسب تلقائياً لكل نقطة على حدة** بحيث يحقق التوزيع قيمة `perplexity` المطلوبة. والالتباس (perplexity) يُعرَّف أسّياً بإنتروبيا التوزيع:

$$\text{Perp}(P_i) = 2^{H(P_i)}, \quad H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$$

فهي عملياً «عدد الجيران الفعّالين»: التباس ٣٠ يعني اهتمام كل نقطة بنحو ٣٠ جاراً مهما اختلف حجم البيانات. وفي الفضاء الواطئ تُعرَّف تناظرياً احتمالات `q_ij` بتوزيع الطالب:

$$q_{ij} = \frac{(1 + d_{ij}^2)^{-1}}{\sum_{k \neq l} (1 + d_{kl}^2)^{-1}}$$

ذيول توزيع الطالب الثقيلة هي التي تحل مشكلة «التزاحم (crowding)»: النقاط المعتدلة البعد تُدفع بعيداً بشراسة، فتتفرق العناقيد بدل أن تنهرض فوق بعضها. ثم يقلّص t-SNE الفرق بين التوزيعين بتشتت KL-divergence عبر هبوط التدرج.



الشكل ١٥١t-SNE على بيانات الأرقام بثلاث قيم للالتباس: حجم الحيّ يغيّر خريطة البيانات كلياً

ما نقرأه في الشكل أعلاه أن الالتباس ليس «بارامتر تجميلي» بل إعادة تعريف لمعنى «القريب»: عند ٥ يتحطم الشكل إلى جزر، وعند ١٠٠ تتذوب البنية الدقيقة. جرّب الموجة نفسها على اللفافة:

التباس 5 و 30 و 100 على اللفافة السويسرية python

```python
from sklearn.manifold import TSNE, trustworthiness
import numpy as np
import time

Xs, _ = make_swiss_roll(n_samples=300, noise=0.10, random_state=42)
for p in (5, 30, 100):
    t0 = time.perf_counter()
    Z = np.asarray(TSNE(n_components=2, perplexity=p, init="pca",
                        learning_rate="auto",
random_state=42).fit_transform(Xs))
    dt = (time.perf_counter() - t0) * 1000
    tr = trustworthiness(Xs, Z, n_neighbors=8)
    print(f"perplexity={p:3d} | زمن: {dt:6.1f} ms | موثوقية: {tr:.4f}")
```

لاحظ مفارقة دقيقة: الموثوقية تظل شبه ثابتة بين ٥ و ٣٠ (٠.٩٩٥٢ ثم ٠.٩٩٦٠) ثم تهبط عند ١٠٠ (٠.٩٨٢٨)، بينما **شكل** الإسقاط يتغير انقلاباً مع كل قيمة. تتبع العكس: التصوّر الجميل لا يعني قياساً أفضل، والالتباس الكبير يبدأ بابتلاع البنية المحلية التي كان من المفترض أن تحفظها الأداة.

خطأ شائع

لا تكتشف عناقيدك بالمسافات بين بلورات t-SNE! حجم البلورة بلا معنى، والمسافة بين بلورتين بلا معنى، وتلاصق بلورتين قد يكون صدفة تخطيط. استعملها عرضاً استكشافياً، وقس الفصل الحقيقي بأدوات الفصل ٦٦ (silhouette و ARI) في الفضاء الأصلي أو في فضاء الفصل ٧١.

> **تحذير**
>
> مقدر `TSNE` لا يوفر `transform` ولا يقبل بيانات جديدة بعد التدريب — فالتضمين مرتبط بالعينات نفسها وتوزيعات جوارها، وهي ضرورة رياضية لا قصور برمجي. أي سطر يحاول `tsne.transform(X_new)` سيرمي `AttributeError`. إذا احتجت إسقاط عينات جديدة فاستخدم `KernelPCA` أو `Isomap` اللذين يوفران `transform` فعلياً.

نجرب بيدنا — الفهم أعمق من قراءة تحذير:

python — هل يقبل TSNE بيانات جديدة؟ والجواب الصحيح

```python
from sklearn.manifold import TSNE, Isomap
import numpy as np

sub = X[:150]
tsne = TSNE(n_components=2, perplexity=20, init="pca", learning_rate="auto",
random_state=42).fit(sub)
iso = Isomap(n_neighbors=10, n_components=2).fit(sub)

try:
    tsne.transform(sub[:3])
except AttributeError as err:
    print(f"TSNE: رفض صريح — {err}")

Z_new = np.asarray(iso.transform(sub[:3]))
print(f"Isomap: transform يعمل بأمان، خرج بصيغة {Z_new.shape}")
```

٨.٥ موثوقية الإسقاط: trustworthiness

كيف نقيس أن إسقاطاً «صادق»؟ المقياس الذي استعملناه طوال الفصل هو `sklearn.manifold.trustworthiness`، وفكرته بسيطة وشرسة: انظر إلى جيران كل نقطة في الفضاء الواطئ، وعاقب الإسقاط كلما ظهر فيها جارٌ **لم يكن** من جيرانها الحقيقيين:

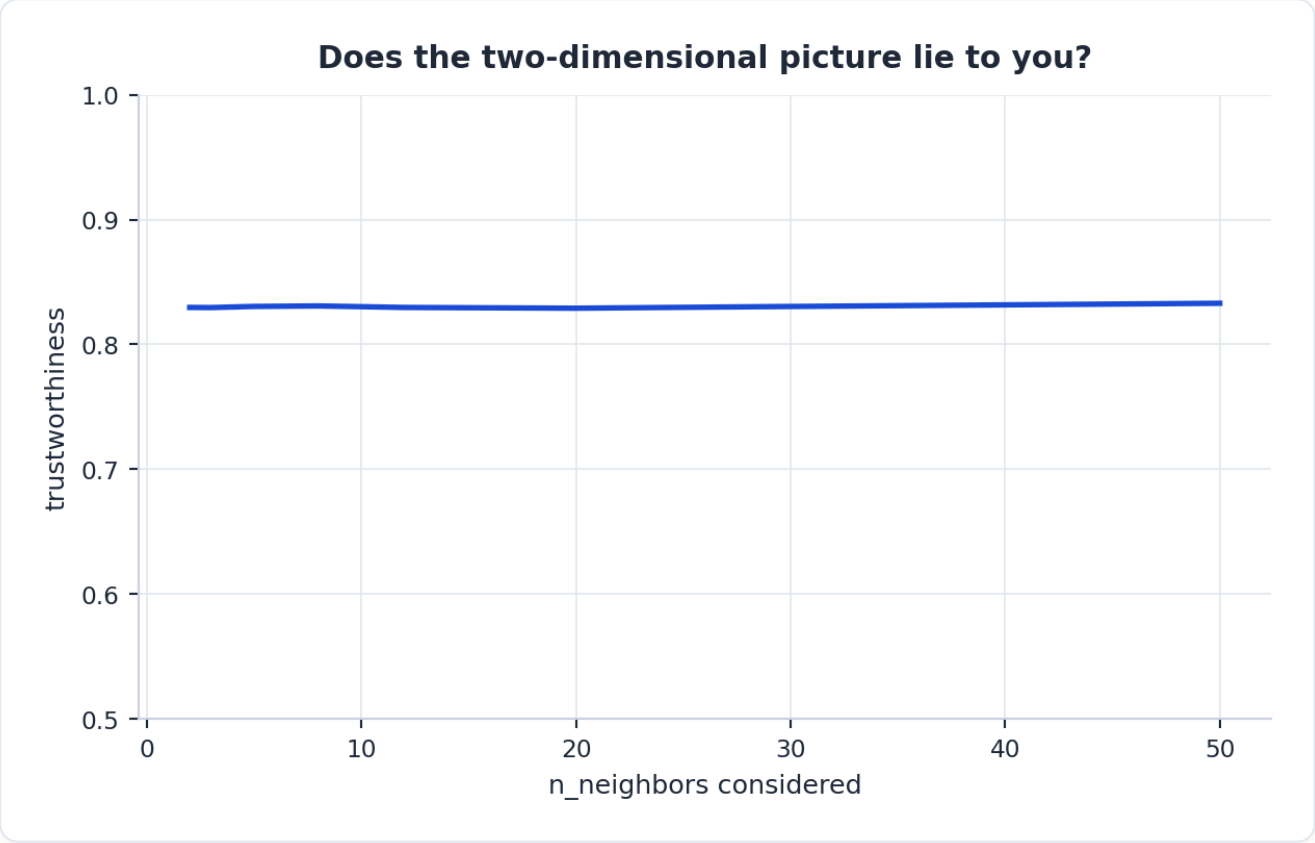
$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^{n} \sum_{j \in U_k(i)} (r_{ij} - k)$$

حيث `n` عدد العينات و`k` حجم الحيّ المدروس، و`U_k(i)` مجموعة النقاط التي دخلت جوار `i` في الإسقاط دون أن تكون في جواره الأصلي، و`r_ij` رتبة النقطة `j` في ترتيب المسافات الحقيقية عن `i`. النتيجة بين · و ١، والقيمة ١ تعني أن كل جارٍ في الصورة كان جاراً حقيقياً. واضبط `k` بحسب سؤالك: القيم الصغيرة تفحص الجوار المباشر فقط.

البارامتر الحاسم الآخر هو `n_neighbors` نفسه، وليس له قيمة آمنة عالمية: القيم الصغيرة قد تقطع الرسم إلى جزر، والكبيرة تتجاوز فجوة الطية فتأتي «اختصارات» خادعة عبر الهواء الذي حرمته أصلاً. جرّب الموجة على Isomap:

```python
# البارامتر الحاسم: n_neighbors في Isomap
from sklearn.manifold import Isomap, trustworthiness
import numpy as np

for k in (5, 10, 15, 25, 40):
    Z = np.asarray(Isomap(n_neighbors=k, n_components=2).fit_transform(X))
    tr = trustworthiness(X, Z, n_neighbors=8)
    print(f"n_neighbors={k:2d} | موثوقية: {tr:.4f}")
```

والذروة عند ٥ (٠.٩٩٨١ — أعلى رقم في الفصل كله)، ثم هبوط حاد مع ١٠ و ١٥ (٠.٨٧٩٥ عند ١٥: أسوأ من PCA الخطي!)، وارتداد جزئي بعدها. وسر انقلاب Isomap بين هذا والقسم ٨.٣ أن كثافة العينة تحدد ما إذا كانت قيمة `n_neighbors` محلية أم عابرة للفجوة: ١٢ جاراً على ٥٠٠ نقطة متباعدة أوسع من ١٢ جاراً على ٦٠٠. فالقيمة المثلى تتزحزح مع الكثافة والضجيج: ابدأ بين ٥ و ١٥% من حجم العينة ثم اضبط بالقياس.



الشكل ١٥٢ منحنى موثوقية إسقاط PCA على بيانات الأرقام يتغير مع حجم الحيّ المدروس k

الشكل أعلاه يبعد نظرة واحدة: نفس الإسقاط تقيّمه أداة واحدة بقيم `k` مختلفة فتتراجع الموثوقية كلما اتسع الحيّ المدروس — لأن الحفاظ على جيران قريبين أسهل بكثير من الحفاظ على جيران بعيدين. حين تقيّم إسقاطاً، أدرج قيمة `k`

التي اعتمدتها بجوار الرقم، وإلا فالرقم وحده لا يعني شيئاً.

> **نصيحة**
>
> قِس الإسقاط بمقياسين: `trustworthiness` للبنية المحلية، ومقياس كلي من الفصل ٦٦ إن كان الهدف تجميعاً. وتذكر أن `trustworthiness` وأغلب عائلة المانيفولد محكومة بذاكرة من رتبة n^2.

٨.٦ متى يكفي PCA ومتى تحتاج غير خطي؟

قبل أن تفتح ترسانة المانيفولد كاملة، اسأل: هل الطيّة موجودة أصلاً؟ PCA رخيص وسريع وقابل للتفسير، ويكفي حين تكون البنية خطية أو شبه خطية، وحين تحتاج إسقاطاً يمتد لعينات جديدة بسلاسة عبر `transform`. التقنيات غير الخطية تدفع السعر في ثلاث عملات: بارامترات حساسة (`gamma` و`n_neighbors` و`perplexity`)، وتكلفة حسابية وذاكرية أعلى بكثير، وفقدان قابلية التفسير المباشر للمحاور.

الحالة	القرار المبدئي
بيانات خطية أو شبه خطية، وتريد سرعة وتفسيراً	PCA — كما في الفصل ٦٨
انحناء بسيط وتحتاج `transform` لعينات جديدة	KernelPCA مع ضبط `gamma` بحذر
بنية منحنية واضحة والهدف حفر التمثيل الكلي	Isomap مع `n_neighbors` مضبوطة
الهدف تصوّر استكشافي عنقودي فقط	TSNE مع تجربة `perplexity` بين ٥ و ٥٠
بيانات ضخمة بملايين الصفوف	عيّنة أولاً، أو PCA ثم نماذج لاحقة

والتكلفة ليست تفصيلاً: PCA يعمل عبر SVD بتكلفة من رتبة $O(n\,d^2)$، بينما Isomap و MDS يبنيان مصفوفة مسافات كاملة من رتبة $O(n^2)$ ذاكرة، و TSNE بتنفيذ `barnes_hut` يقارب $O(n \log n)$ لكل تكرار لكنه يظل الأغلى عملياً كما رأينا في الجدول المركزي.

في الواقع

في البيولوجيا الحسابية أصبح t-SNE (وأخوه الأحدث UMAP) اللغة القياسية لعرض خلايا الدماغ الواحد: عشرات آلاف الخلايا، كل واحدة متجه آلاف الجينات، في لقطة واحدة تكشف عناقيد الأنماط الخلوية. لكن الممارسين يقولون حرفياً: «t-SNE للعرض، والعناقيد تُقاس في الفضاء الأصلي» — وحين تحتاج تصنيف خلية جديدة يُدرَّب المصنّف على الفضاء الأصلي لأن t-SNE لا يملك `transform`.

درس الجزء: اختر التقنية بجواب سؤالك، وقِس بمقياس يعلن قيم `k`، واشكّ في أي خريطة تفقدك جيرانها بصمت.

تمارين الفصل التاسع والستين

- **ملاحظة**

التمرين ٨.١

امسح `gamma` في KernelPCA بنواة RBF على القيم ٠.٠٠١ و ٠.٠١ و ٠.١ و ١.٠ على اللفافة نفسها (٥٠٠ نقطة، ضجيج ٠.١٠)، وقس الموثوقية عند `n_neighbors=8`. صف الاتجاه، وفسّر لماذا يهدم `gamma` الكبير الإسقاط رغم أنه «غير خطي أكثر»، وقارن أفضل قيمة بسقف PCA الخطي.

التمرين ٨.٢

على منحنى S (`make_s_curve`، ٤٠٠ نقطة)، امسح `n_neighbors` في Isomap على القيم ٤ و ٨ و ١٥ و ٣٠، وسجّل موثوقية الإسقاط عند `n_neighbors=8`. فسّر شكل المنحنى الناتج بلغة «الجزر» و«الاختصارات».

التمرين ٨.٣

زملاؤك يسقطون ١٠٠ ألف عينة بـ t-SNE ثم يدرّبون مصنّف KNN على الإحداثيات الثنائية الناتجة. اشرح لهم خطأين منفصلين في هذا التصميم — أحدهما رياضي (ماذا يمثل البعدان؟) والآخر هندسي (ماذا يحدث عند وصول عينات جديدة؟) — واقترح بديلاً صحيحاً كامل الخطوات.

- **ملاحظة**

مسح gamma في KernelPCA على اللفافة python

```python
from sklearn.datasets import make_swiss_roll
from sklearn.decomposition import KernelPCA
from sklearn.manifold import trustworthiness
import numpy as np

Xr, _ = make_swiss_roll(n_samples=500, noise=0.10, random_state=42)
for g in (0.001, 0.01, 0.1, 1.0):
    Z = np.asarray(KernelPCA(n_components=2, kernel="rbf", gamma=g,
random_state=42).fit_transform(Xr))
    print(f"gamma={g:<5} | موثوقية: {trustworthiness(Xr, Z, n_neighbors=8):.4f}")
```

الاتجاه انحدار رتيب: من ٠٫٩٣٥٨ عند `gamma=0.001` إلى كارثة ٠٫٥٣٨٢ عند ١٫٠. السبب أن `gamma` الكبير يضيّق نواة RBF حتى تصبح التشابهات بين الجيران شبه معدومة ومصفوفة النواة شبه قطرية — فتُسقط كل نقطة وحدها بلا معلومة جوارية. والقيمة الصغيرة جداً تُقرّب السلوك من الخطي (٠٫٩٣٥٨ قرب سقف PCA ٠٫٩٤٢٢). لا «غير خطي أفضل» مجاناً: النواة تعيد تعريف الجوار، وإذا عرّفته أسوأ من الإقليدي فستخسر.

ملاحظة

مسح n_neighbors في Isomap على منحنى S python

```python
from sklearn.datasets import make_s_curve
from sklearn.manifold import Isomap, trustworthiness
import numpy as np

Xs, _ = make_s_curve(n_samples=400, noise=0.08, random_state=42)
for k in (4, 8, 15, 30):
    Z = np.asarray(Isomap(n_neighbors=k, n_components=2).fit_transform(Xs))
    print(f"n_neighbors={k:2d} | موثوقية: {trustworthiness(Xs, Z,
n_neighbors=8):.4f}")
```

النتيجة تكشف أن منحنى S أرحب من اللفافة: عند ٤ جيران يبدأ الرسم بالتقصير في أذرع المنحنى الرقيقة (٠٫٩٨٠١) لكنه لا ينقطع، والقيم الأكبر تتحسن استقراراً حتى ٣٠ (٠٫٩٩٨٧) لأن فجوة منحنى S أوسع من فجوة الأدوار المتقاربة في اللفافة — فلا تصل الاختصارات عبرها بسهولة. الخلاصة المزدوجة: نفس الخوارزمية تنقلب قصة ضبطها من بيانات لأخرى، و«الاختصارات» خطرٌ مرتبط بشكل الفجوات لا بخوارزمية Isomap ذاتها.

ملاحظة

٨.٨ الأخطاء الشائعة

والأخطاء الشائعة:

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استدعاء `tsne.transform(X_new)` على بيانات جديدة	`AttributeError` فوري: TSNE لا يقبل عينات جديدة أصلاً	استخدم `KernelPCA` أو `Isomap` حيث يوجد `transform` فعلي
قراءة المسافات والفجوات بين بلورات t-SNE كعناقيد حقيقية	استنتاجات تجميع وهمية: الحجم والمسافة بين البلورات بلا معنى	قس العناقيد بالـ ARI و silhouette في الفضاء الأصلي
اختيار `gamma` أو `perplexity` أو `n_neighbors` بالإفتراضي دائماً	إسقاط مهدور: KPCA بـ gamma=٠.٠٣ أسوأ من PCA الخطي على اللفافة	امسح البارامتر الحاسم وقس `trustworthiness` عند كل قيمة
الحكم على جودة الإسقاط بنسبة التباين المفسَّرة وحدها	ثقة زائفة: التباين محفوظ والبنية المنحنية ممزّقة	أضف مقياساً جوارياً صريحاً مثل `trustworthiness`

٨.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
المانيفولد	سطح منخفض الأبعاد منحنٍ تتوزع عليه البيانات داخل فضاء أعلى	manifold
المسافة الجيوديسية	أقصر مسار على السطح نفسه لا في الفضاء المحيط	geodesic distance
الإجهاد	مجموع مربعات فوارق المسافات الزوجية في MDS	stress
الالتباس	عدد الجيران الفعّال حول كل نقطة في t-SNE	perplexity
موثوقية الإسقاط	نسبة الجوار الأصلي الذي نجا في التمثيل الواطئ	trustworthiness

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٧٠ — كشف الشذوذ وتقدير التباين

IsolationForest و LOF والمسافة الماهالانوبية وانكماش التغاير

الوحدة **scikit-learn ١.٩** الحجم **٧٠٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تميّز بين كشف الشذوذ العالمي والمحلي وتختار بين IsolationForest و LOF
- أن تفسّر decision_function و score_samples وملصقي التنبؤ ١ و-١
- أن تقيّم ترتيب الشذوذ ببيانات معلومة الوسوم دون تسريبها إلى التدريب
- أن تستخدم EllipticEnvelope للتوزيعات الإهليلجية و LedoitWolf لتثبيت التغاير

٨.١ هل الشذوذ نقطة بعيدة أم نقطة غريبة عن جيرانها؟

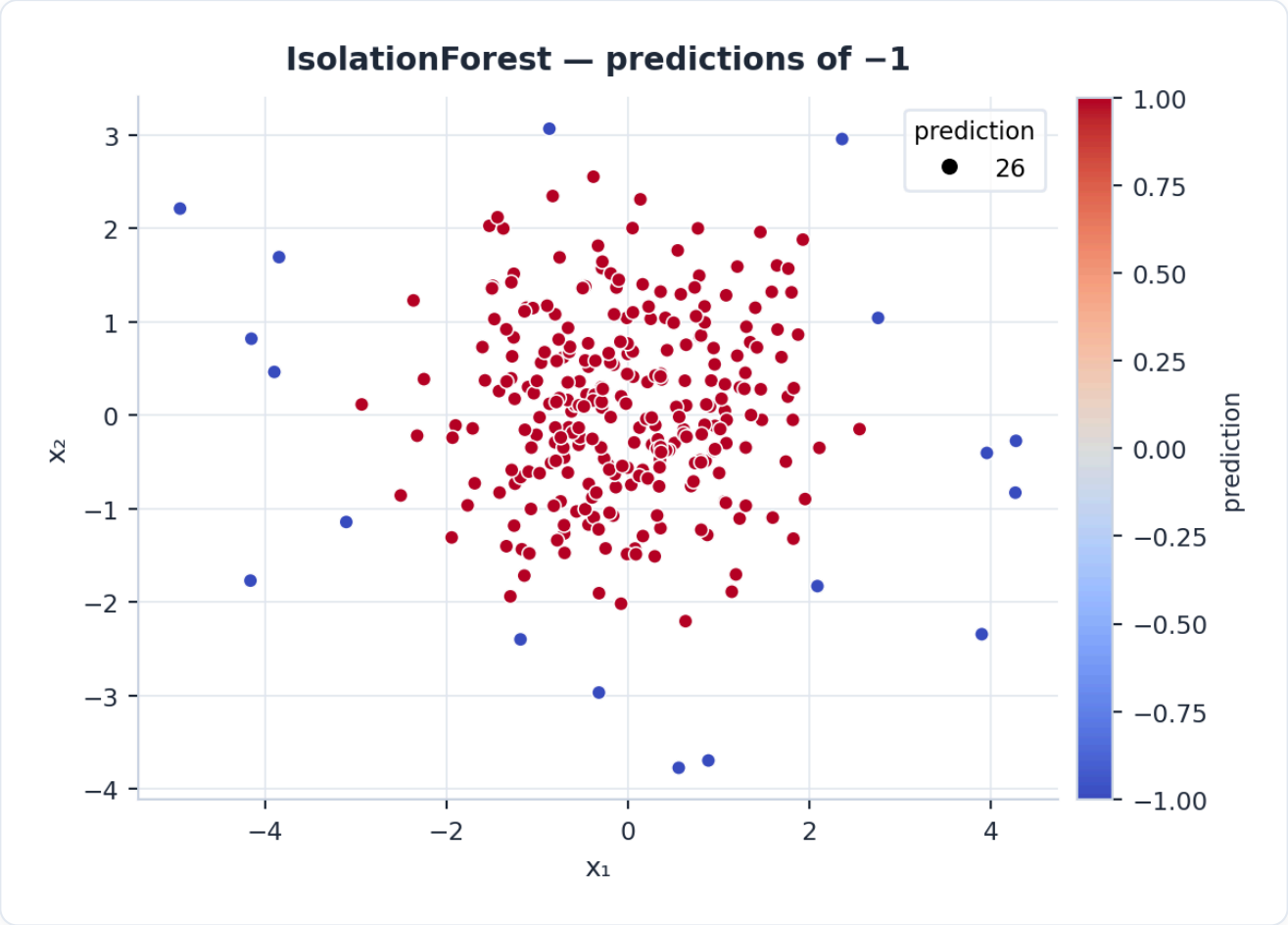
تخيّل آلاف المعاملات المالية: معاملة بقيمة كبيرة جداً قد تكون طبيعية لشركة ضخمة، لكنها غريبة لحساب فردي. هل تقيس الشذوذ بالمسافة من مركز البيانات كله، أم تقارنه بما يحدث حول كل نقطة؟ هذا السؤال يحدد الخوارزمية قبل أن يحدد قيمة العتبة.

في **كشف الشذوذ (anomaly detection)** نتعلم شكل السلوك المعتاد من بيانات غالباً لا تحمل وسوماً كاملة، ثم نبحث عن حالات لا تشبهه. أما **كشف القيم المتطرفة (outlier detection)** فيفترض أن مجموعة التدريب نفسها قد تحتوي بعض الشواذ. يتيح scikit-learn لكلا السياقين أدوات متقاربة، لكن الفارق العملي الأهم هو: هل تدرب النموذج على بيانات طبيعية فقط، أم على مزيج غير معلّم؟

> **مفتاح**
>
> لا يوجد تعريف واحد للشذوذ. قد تكون النقطة نادرة عالمياً، أو منخفضة الكثافة مقارنة بجيرانها، أو مخالفة لافتراض توزيع إهليلجي. اسأل أولاً: «غريبة قياساً إلى ماذا؟» ثم اختر النموذج ومقياسه.

الشكل الأول يعرض فكرة **IsolationForest**: النقطة التي يسهل عزلها بقسمة عشوائية للمحاور تميل إلى أن تكون شاذة. لا تحتاج الغابة إلى مسافة إقليدية أو شكل عنقود دائري، ولذلك تناسب البيانات كثيرة الخصائص عندما لا نملك

تعريفاً هندسياً بسيطاً للطبيعي.



الشكل ١٥٣تُعزل النقاط الشاذة غالباً بعدد قليل من التقسيمات العشوائية في IsolationForest

٨.٢ IsolationForest: كم قسمة تحتاج لعزل النقطة؟

تُنشئ `sklearn.ensemble.IsolationForest` أشجاراً عشوائية. في كل عقدة تختار خاصية وحداً عشوائياً وتقسم البيانات؛ النقطة المنفردة قرب أطراف الفضاء تصل إلى ورقة بسرعة، بينما تحتاج النقاط الكثيفة إلى مسارات أطول. تجمع الغابة أطوال المسارات وتحوّلها إلى درجة شذوذ.

قراءة درجات IsolationForest واتجاهها python

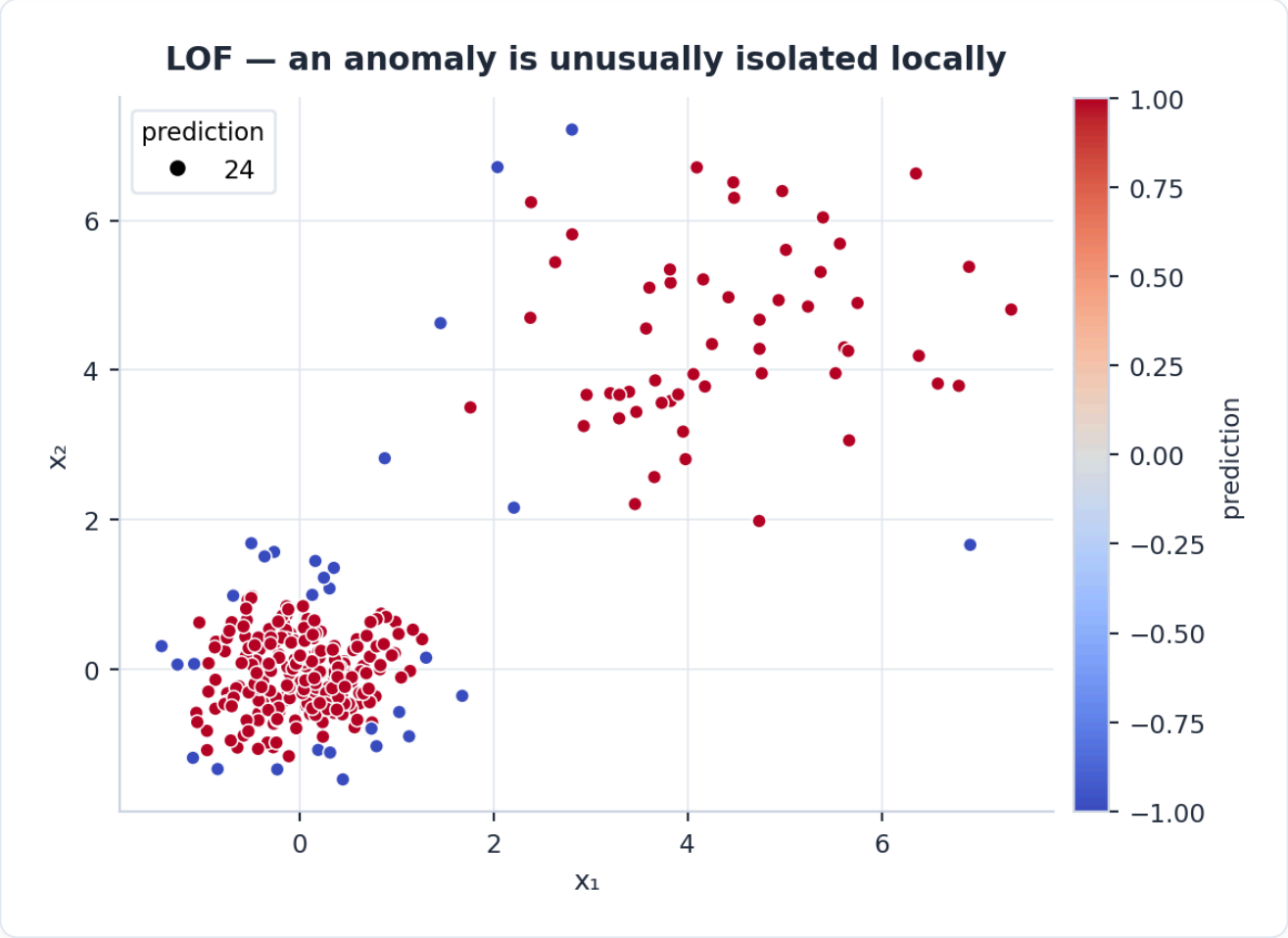
```python
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(42)
X_train = rng.normal(size=(500, 2))
X_test = np.vstack([rng.normal(size=(40, 2)),
                    [[4.5, 4.5], [-4.0, 4.0], [0.2, -5.0]]])

model = IsolationForest(n_estimators=200, contamination=0.06,
                        random_state=42).fit(X_train)
labels = model.predict(X_test)
scores = model.decision_function(X_test)

print("عدد الحالات المصنفة شاذة:", int((labels == -1).sum()))
print("1- الطبيعي يحمل 1 والشاذ يحمل")
print("أقل خمس درجات؛ الأصغر أشد شذوذاً:",
      np.round(np.sort(scores)[:5], 3))
```

المتوقع أن تحمل النقاط البعيدة درجات أقل، لكن `decision_function` ليست احتمالاً؛ لا تفسر `-0.2` على أنه «احتمال شذوذ ٢٠%». إنها درجة مرتبة بالنسبة إلى النموذج. أما `predict` فيستخدم حد القرار الذي يتأثر بـ `contamination`. القيمة `"auto"` تستخدم عتبة افتراضية داخلية؛ والقيمة العددية مثل `0.06` تعبّر عن نسبة متوقعة تقريبية للشواذ في بيانات التدريب، لا حقيقة مكتشفة من فراغ.

> **انتبه** ⚠
>
> إذا درّبت الغابة على بيانات مختلطة مجهولة النسبة ثم ضبطت `contamination` على نسبة مرغوبة للعرض، فأنت تغيّر حد التصنيف لا ترتيب الحالات الأساسي. افصل بين **الترتيب** الذي تعطيه الدرجة و**القرار** الذي تصنعه العتبة.

٨.٣ LOF: غريب في الحيّ، طبيعي في المدينة؟

تقيس خوارزمية **عامل القيم المتطرفة المحلي (Local Outlier Factor - LOF)** كثافة النقطة مقارنة بكثافة جيرانها. النقطة البعيدة عن مركز عام قد تكون طبيعية داخل حيّ صغير، بينما نقطة ملاصقة لمركز عالمي قد تكون شاذة إذا كان جيرانها أكثر كثافة بكثير.



الشكل ١٥٤ يقارن LOF الكثافة المحلية للنقطة بكثافة أقرب جيرانها

الفكرة التقريبية أن LOF يقارن كثافة الوصول المحلية (local reachability density) للنقطة بمتوسط كثافات جيرانها. قيمة أعلى من ١ تعني أن النقطة أقل كثافة محلياً من محيطها؛ والقيمة القريبة من ١ تشير إلى كثافة مشابهة. في الواجهة، يعطي `negative_outlier_factor_` قيماً سالبة، وتكون القيم الأكثر سلبية أكثر غرابة.

```python
# للكشف عن الشذوذ المحلي في بيانات التدريب LOF
import numpy as np
from sklearn.neighbors import LocalOutlierFactor

rng = np.random.default_rng(7)
dense = rng.normal(loc=[0, 0], scale=0.35, size=(180, 2))
small_cluster = rng.normal(loc=[4, 4], scale=0.22, size=(35, 2))
X = np.vstack([dense, small_cluster, [[0.9, 0.9]]])

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.02)
labels = lof.fit_predict(X)
scores = lof.negative_outlier_factor_

idx = np.argsort(scores)[:5]
print("عدد الشواذ:", int((labels == -1).sum()))
print("أشد الحالات شذوذاً (الفهرس، الدرجة):")
print([(int(i), round(float(scores[i]), 2)) for i in idx])
```

هنا نستخدم وضع الكشف عن القيم المتطرفة على **بيانات التدريب نفسها**؛ لذلك نستدعي `fit_predict` ونقرأ `negative_outlier_factor_`. إذا أردت لاحقاً تصنيف سجلات جديدة، أنشئ `LocalOutlierFactor(novelty=True)`، ودرّبه على بيانات مرجعية ثم استخدم `predict` أو `decision_function` على بيانات لم تدخل في `fit`.

> **خطأ شائع**
>
> لا تستدعِ `predict` على بيانات التدريب عند ضبط `novelty=True` لتقييم LOF؛ هذا الوضع مخصص للعينات الجديدة، وقد تختلف درجاته عن وضع الكشف الداخلي. للتدريب واكتشاف الشواذ في المجموعة نفسها اترك `novelty=False` واستعمل `fit_predict`.

٨.٤ اختيار الأداة بالتقييم لا بالانطباع

إذا كانت لدينا وسوم مرجعية لبعض الحالات، نستطيع مقارنة **ترتيب** الشذوذ باستخدام مساحة تحت منحنى ROC (ROC AUC) ومتوسط الدقة (average precision). لا نستخدم الوسوم لتدريب الكواشف في المثال التالي: يتدرب كل نموذج على عينة طبيعية، ثم يختبر على مزيج جديد معلّم.

مقارنة ترتيب الشذوذ بثلاثة كواشف python

```python
import numpy as np
from sklearn.covariance import EllipticEnvelope
from sklearn.ensemble import IsolationForest
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.neighbors import LocalOutlierFactor

rng = np.random.default_rng(12)
X_train = rng.normal(size=(700, 2))
X_normal = rng.normal(size=(180, 2))
X_outlier = rng.normal(loc=[3.8, 3.8], scale=0.55, size=(20, 2))
X_test = np.vstack([X_normal, X_outlier])
y_test = np.r_[np.zeros(len(X_normal)), np.ones(len(X_outlier))]

models = {
    "IsolationForest": IsolationForest(contamination=0.05, random_state=12),
    "LOF novelty": LocalOutlierFactor(n_neighbors=25, novelty=True,
                                      contamination=0.05),
    "EllipticEnvelope": EllipticEnvelope(contamination=0.05, random_state=12),
}
for name, model in models.items():
    model.fit(X_train)
    anomaly_score = -model.decision_function(X_test)
    print(f"{name:18s} ROC AUC={roc_auc_score(y_test, anomaly_score):.3f} | "
          f"AP={average_precision_score(y_test, anomaly_score):.3f}")
```

لا تتوقع ترتيباً ثابتاً بين الكواشف على كل مسألة؛ البيانات هنا بسيطة ومكوّنة عمداً من طبيعي غاوسي وشواذ بعيدة. في الإنتاج، اجعل الاختبار ممثلاً للزمن والسياق الحقيقيين، ولا تضبط العتبة على مجموعة الاختبار. وعندما تكون الشواذ نادرة، يوضح متوسط الدقة غالباً جودة ترتيب الحالات الإيجابية بصورة أفيد من الدقة الخام.

٨.٥ OneClassSVM: تعلّم حدود الطبيعي

هل يمكن رسم حدّ يحيط بمعظم البيانات الطبيعية من دون بناء نموذج لكثافتها؟ تفعل `sklearn.svm.OneClassSVM` ذلك عبر آلة متجهات الدعم ذات النواة: تبحث عن منطقة تحتوي معظم بيانات التدريب، وتسمح ببعض النقاط على الجانب الآخر من الحد. هي مفيدة عندما يكون الحد غير خطي، لكنها حساسة للمقياس واختيار `gamma`، ولهذا نضعها في خط أنابيب مع التحجيم.

OneClassSVM مع التحجيم وعينات جديدة python

```python
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(17)
X_train = rng.normal(size=(500, 2)) * [1.0, 100.0]
X_test = np.vstack([
    rng.normal(size=(40, 2)) * [1.0, 100.0],
    [[4.5, 450.0], [-4.0, -420.0]],
])

model = make_pipeline(
    StandardScaler(),
    OneClassSVM(kernel="rbf", nu=0.05, gamma="scale"),
).fit(X_train)
labels = model.predict(X_test)
scores = model.decision_function(X_test)
print("الشواذ المكتشفة:", int((labels == -1).sum()))
print("أدنى درجات القرار:", np.round(np.sort(scores)[:5], 3))
```

`nu` حد أعلى نظري لنسبة أخطاء التدريب، وحد أدنى لنسبة متجهات الدعم؛ لا تساوه بنسبة شواذ مؤكدة في بيانات جديدة. وكما في بقية الكواشف، تكون درجة القرار أداة ترتيب وليست احتمالاً معايراً. إذا كان التدريب ملوثاً بكثرة أو كانت البيانات ذات عناقيد مختلفة، اختبر ملاءمة الافتراض قبل الاعتماد على الحد.

٨.٦ EllipticEnvelope والمسافة الماهالانوبية

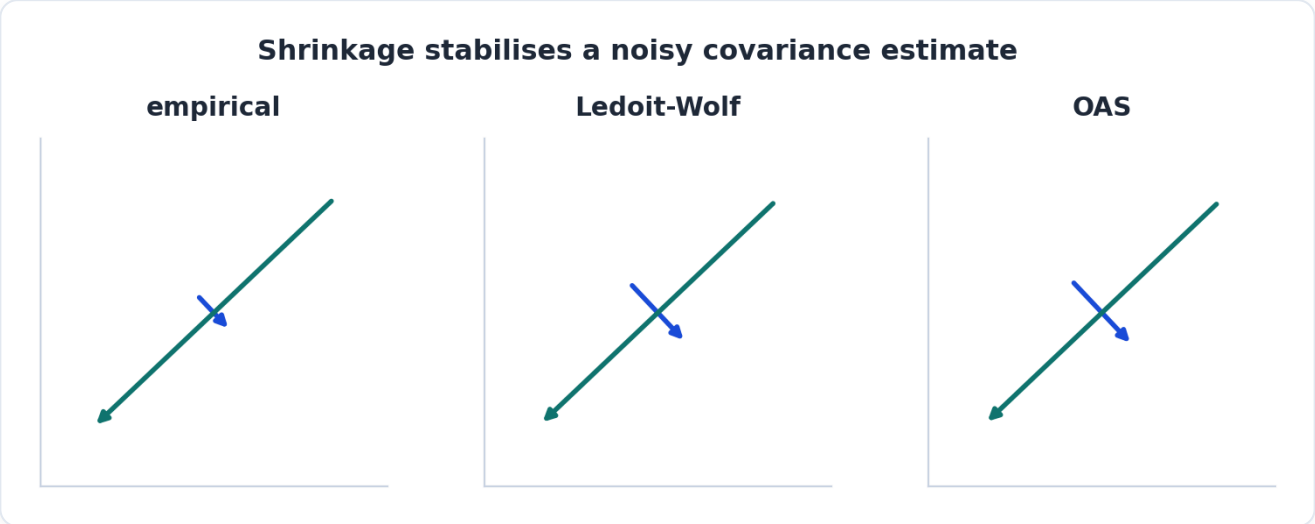
ماذا لو كان «الطبيعي» كتلة بيضوية؟ تقدّر `sklearn.covariance.EllipticEnvelope` مركز البيانات وتغايرها بطريقة متينة نسبياً، ثم تستخدم المسافة الماهالانوبية (Mahalanobis distance). بخلاف الإقليدية، تراعي هذه المسافة اتجاهات الانتشار والارتباطات:

$$d_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

النقطة البعيدة على محور التباين الكبير قد تكون طبيعية، بينما إزاحة صغيرة على محور ضيق قد تكون شاذة. افتراض الشكل الإهليلجي مفيد للقياسات الفيزيائية أو القياسات الحيوية حين يمثل الطبيعي كتلة واحدة ملساء، لكنه لا يناسب عدة عناقيد منفصلة أو توزيعات ذات ذيول غير غاوسية دون معالجة إضافية.

٨.٧ انكماش التغاير: استقرار حين تفوق الأبعاد العينات

تقدير التغاير العيني قد يصبح هشّاً عندما تكون الخصائص كثيرة أو مترابطة بشدة. إذا اقتربت إحدى القيم الذاتية من الصفر، فإن معكوس مصفوفة التغاير يضخّم الضجيج عند حساب المسافة. **انكماش التغاير (covariance shrinkage)** يخلط التقدير العيني بمصفوفة أبسط، عادةً مصفوفة قطرية مضروبة بثابت، لتقليل التباين في التقدير.



الشكل ١٥٥يقلل انكماش التغاير عدم استقرار شكل مصفوفة التغاير المقدرة

يختار `sklearn.covariance.LedoitWolf` مقدار الانكماش تلقائياً وفق مقدّر Ledoit–Wolf، ويقدم `sklearn.covariance.OAS` خياراً آخر قد يناسب البيانات الغاوسية. المثال التالي يصنع خصائص مترابطة من عدد قليل من العوامل الكامنة، ثم يقارن شرطية التغاير ومقدار الانكماش:

كيف يثبت LedoitWolf مصفوفة التغاير؟ python

```python
import numpy as np
from sklearn.covariance import EmpiricalCovariance, LedoitWolf, OAS

rng = np.random.default_rng(24)
latent = rng.normal(size=(45, 4))
mixing = rng.normal(size=(4, 18))
X = latent @ mixing + 0.03 * rng.normal(size=(45, 18))

estimators = {
    "Empirical": EmpiricalCovariance(),
    "LedoitWolf": LedoitWolf(),
    "OAS": OAS(),
}
for name, estimator in estimators.items():
    estimator.fit(X)
    condition = np.linalg.cond(estimator.covariance_)
    shrinkage = getattr(estimator, "shrinkage_", float("nan"))
    print(f"{name:12s} condition={condition:9.1f} | shrinkage={shrinkage:.3f}")
```

انخفاض رقم الشرط يعني أن المصفوفة أقل هشاشة عددياً، لكنه لا يثبت أن النموذج أفضل لكل مهمة. في كشف الشذوذ قارن الدرجات على بيانات ممثلة، وفي النمذجة استعمل التحقق الملائم. ولتقدير مصفوفة تغاير فقط، فإن `LedoitWolf` و`OAS` لا يفرضان افتراض العناقيد البيضوية المتينة نفسه الذي يستخدمه `EllipticEnvelope`.

في الواقع

في مراقبة المعدات، قد تكون زيادة اهتزاز صغيرة طبيعية عند سرعة تشغيل عالية لكنها غير طبيعية عند سرعة منخفضة؛ هذا مثال على أهمية السياق المحلي. وفي اكتشاف الاحتيال، يمكن استخدام درجة الشذوذ لترتيب الحالات كي يراجعها خبير، لا لتحويلها تلقائياً إلى حكم نهائي على العميل.

تمارين الفصل السبعين

- **ملاحظة**

التمرين ٨.١

أنشئ بيانات طبيعية ثنائية الأبعاد مع عشر نقاط بعيدة. درّب `IsolationForest` بنسبة `contamination=0.05`، واطبع عدد النقاط التي أعاد لها `predict` القيمة `-1`. لماذا لا تعني النسبة أن كل حالة مصنفة صحيحة؟

التمرين ٨.٢

فسّر الفرق بين استعمال `LocalOutlierFactor` الافتراضي و`novelty=True`. اذكر الدالة المناسبة لتصنيف عينات جديدة بعد التدريب.

التمرين ٨.٣

أنشئ مصفوفة بيانات بأبعاد أكثر من عدد العينات، وقدّر تغايرها بـ `EmpiricalCovariance` و`LedoitWolf`. قارن رتبة التغاير ورقم شرطه، ثم اشرح لماذا قد يكون الانكماش مفيداً.

- **ملاحظة**

```python
# حل: حد القرار لا يساوي حقيقة الوسوم
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(5)
X = np.vstack([rng.normal(size=(190, 2)),
               rng.uniform(-5, 5, size=(10, 2))])
model = IsolationForest(contamination=0.05, random_state=5).fit(X)
labels = model.predict(X)
print("الحالات المصنفة شاذة:", int((labels == -1).sum()))
print("تقدير الصحة يتطلب وسوماً مستقلة، لا قيمة contamination وحدها.")
```

تضبط `contamination` العتبة لتناسب نسبة متوقعة؛ لا تولّد الحقيقة المرجعية ولا تضمن اكتشاف جميع النقاط المزروعة.

> **ملاحظة**

حل: الانكماش مع p أكبر من n

```python
import numpy as np
from sklearn.covariance import EmpiricalCovariance, LedoitWolf

rng = np.random.default_rng(8)
X = rng.normal(size=(16, 30))
for estimator in (EmpiricalCovariance(), LedoitWolf()):
    estimator.fit(X)
    print(type(estimator).__name__,
          "rank=", np.linalg.matrix_rank(estimator.covariance_),
          "condition=", round(np.linalg.cond(estimator.covariance_), 2))
```

عندما يكون عدد الخصائص أكبر من العينات، لا يمكن لمصفوفة التغاير العينية أن تكون كاملة الرتبة. يضيف الانكماش استقراراً إلى الاتجاهات ضعيفة التقدير، ويجعل المعكوس أو المسافة الماهالانوبية أكثر قابلية للاستخدام.

> **ملاحظة**

٨.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تفسير درجة الشذوذ كاحتمال	الدرجات ليست احتمالات معايرة	استخدمها للترتيب، وعاير الاحتمال بأداة مستقلة عند الحاجة
الاعتماد على `contamination` بوصفه اكتشافاً للحقيقة	تتغير عتبة القرار أو عدد الحالات المعلمة، لا صحة الوسوم	قيّم على وسوم مرجعية ممثلة
استدعاء `predict` على LOF الافتراضي	لا يدعم وضع التدريب الداخلي التنبؤ بعينات جديدة	أنشئ `LocalOutlierFactor(novelty=True)` قبل التدريب
استخدام المسافة الإقليدية مع خصائص مترابطة ومقاييس مختلفة	يُحسب البعد على هندسة غير مناسبة	افحص القياس والارتباط، واستخدم المسافة الماهالانوبية عند ملاءمتها
تدريب OneClassSVM على أعمدة بمقاييس متفاوتة	تهيمن الخصائص الأكبر نطاقاً على النواة وحد القرار	استخدم `StandardScaler` داخل `Pipeline`
معكوس تغاير عيني هشّ في أبعاد كثيرة	تضخيم للضجيج أو مصفوفة مفردة	استخدم مقدّرات انكماش مثل `LedoitWolf` أو `OAS`

٨.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
كشف الشذوذ	تعلم السلوك المعتاد ثم ترتيب الحالات المخالفة	anomaly detection
الكثافة المحلية	مقدار تمركز النقاط حول نقطة مقارنة بجيرانها	local density
عامل القيم المتطرفة المحلي	مقياس لمقارنة كثافة نقطة بكثافة جيرانها	Local Outlier Factor
المسافة الماهالانوبية	مسافة تراعي مركز البيانات وتغايره	Mahalanobis distance
انكماش التغاير	مزج تقدير التغاير العيني بتقدير أكثر انتظاماً	covariance shrinkage

الجزء ٨ · التعلّم غير الخاضع للرقابة

الفصل ٧١ — النصوص: التحليل الدلالي الخفي والمواضيع

TF-IDF و TruncatedSVD و LatentDirichletAllocation

الوحدة **sklearn.cluster · sklearn.decomposition** الحجم **٦٦١** كلمة تقريباً

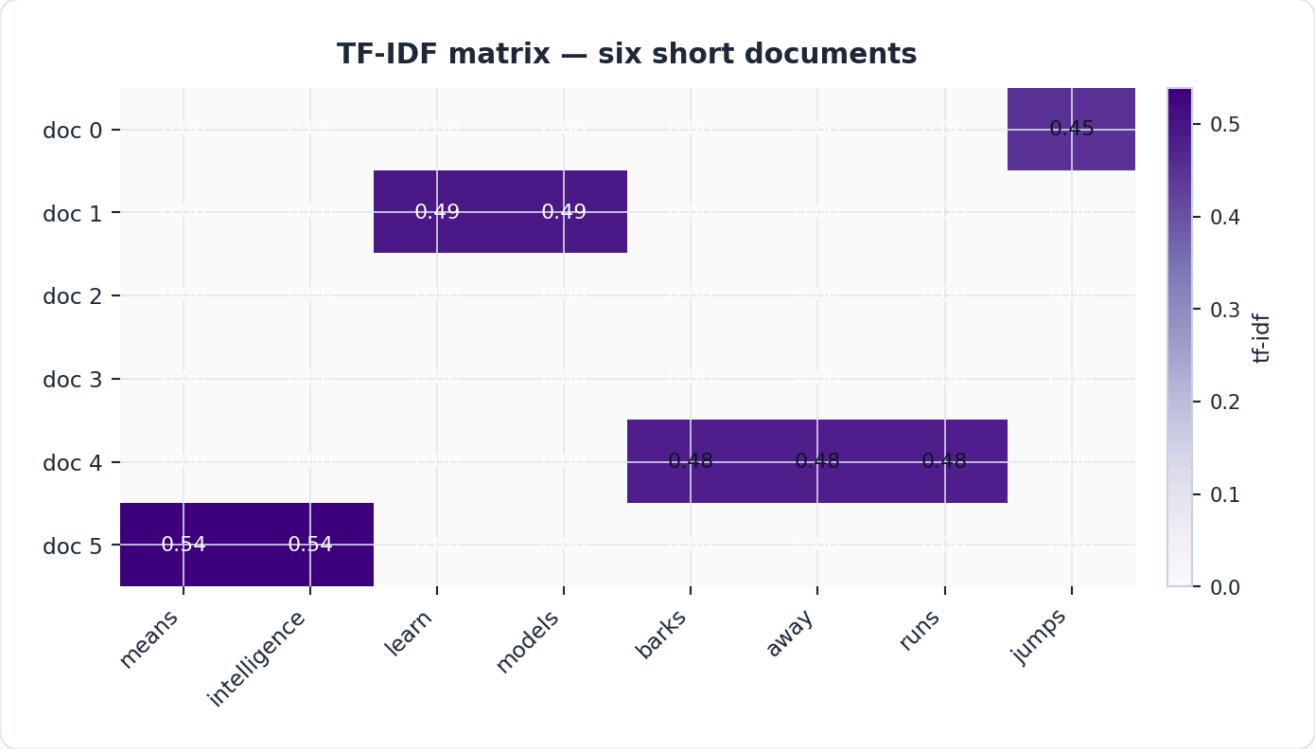
ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا يُخفض TF-IDF وزن الكلمات الشائعة مع إبقاء الوثائق مصفوفات متناثرة
- أن تطبق TruncatedSVD لاستخراج فضاء دلالي منخفض الأبعاد من مصفوفة النص
- أن تميّز بين LSA و LatentDirichletAllocation في التمثيل والافتراضات
- أن تستخرج الكلمات الأعلى وزناً وتوزيع المواضيع لكل وثيقة دون الادعاء بأنها حقائق

٨.١ من كلمات متناثرة إلى معنى قابل للحساب

لديك آلاف المقالات، وتريد معرفة أيها يتحدث عن صحة المرضى وأيها عن الأسواق. لا ترى الخوارزمية «موضوعاً» كما يراه القارئ؛ إنها ترى كلمات. يبدأ الحل بمصفوفة وثائق-مصطلحات (document-term matrix): صف لكل وثيقة وعمود لكل كلمة، والقيمة عدد التكرارات أو وزنها. المشكلة أن مفردات النصوص كبيرة جداً، ومعظم الكلمات لا تظهر في معظم الوثائق.

تحوّل `sklearn.feature_extraction.text.TfidfVectorizer` النص إلى أوزان **TF-IDF**: يزداد وزن المصطلح إذا تكرر في الوثيقة، وينخفض إذا ظهر في وثائق كثيرة. النتيجة غالباً مصفوفة متناثرة (sparse matrix)، وهذا مكسب لا عبء؛ فلا نخزن أصفاراً بالملايين.



الشكل ١٥٦ يخفض TF-IDF وزن المصطلحات الشائعة ويرفع وزن الكلمات المميزة للوثيقة

مفتاح

TF-IDF ليس فهماً للمعنى، بل تمثيل عددي يقلل هيمنة الكلمات المنتشرة. وما سيأتي من LSA و LDA يعتمد على اختيارات التهيئة، والتنظيف، ومجموعة الوثائق؛ لا توجد «موضوعات صحيحة» تستخرج تلقائياً بلا مراجعة بشرية.

بناء مصفوفة TF-IDF وقراءة الكلمات المميزة python

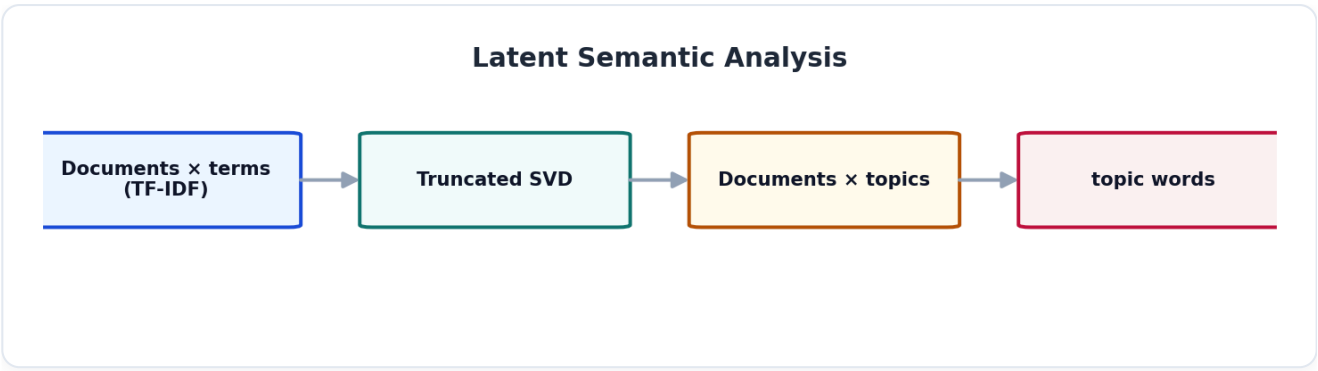
```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "model learns patterns from training data",
    "training data needs careful validation",
    "market prices change with financial risk",
    "financial models estimate market risk",
]
vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform(documents)
terms = vectorizer.get_feature_names_out()

print("شكل المصفوفة:", X_tfidf.shape)
print("عدد العناصر غير الصفرية:", X_tfidf.nnz)
for i, row in enumerate(X_tfidf):
    top = np.argsort(row.toarray().ravel())[-3:][::-1]
    print(i, [terms[j] for j in top])
```

تُطبّع أوزان TF-IDF افتراضياً بحيث لا تهيمن الوثائق الطويلة لمجرد أنها أطول. أما الرموز، ووقف الكلمات، والاشتقاق، واللغة فتؤثر في المفردات قبل التدريب. `TfidfVectorizer` الافتراضي ليس محللاً صرفياً عربياً: النص العربي يحتاج معالجة واعية باللغة وتقييماً يثبت أن التطبيع مفيد.

> ⚠ **انتبه**
>
> لا تحوّل مصفوفة نص كبيرة إلى مصفوفة كثيفة بواسطة `.toarray()` لمجرد طباعتها؛ استهلاك الذاكرة ينمو مع عدد الوثائق مضروباً في حجم المفردات. في المثال الصغير فعلنا ذلك لقراءة صف واحد فقط، أما المعالجة الفعلية فحافظ على التمثيل المتناثر.

٨.٢ LSA: هل تختبئ الكلمات المترادفة في اتجاه كامن؟

التحليل الدلالي الخفي (Latent Semantic Analysis - LSA) يضغط مصفوفة الوثائق-المصطلحات إلى عدد قليل من المحاور الكامنة. عملياً نطبّق التحليل بالقيم المفردة المبتور (Truncated SVD) على مصفوفة TF-IDF. على خلاف PCA، لا نحتاج إلى توسيط أعمدة المصفوفة؛ وهذا مهم لأن توسيط مصفوفة نص متناثرة يحول معظم الأصفار إلى قيم غير صفرية.



الشكل ١٥٧: تتحول مصفوفة TF-IDF إلى تمثيل منخفض الأبعاد عبر TruncatedSVD في LSA

$$\mathbf{X} \approx \mathbf{U}_k \, \mathbf{\Sigma}_k \, \mathbf{V}_k^T$$

يمثل صف المصفوفة الناتجة عن `transform` الوثيقة في فضاء الأبعاد الكامنة، بينما تصف صفوف `components_` الأوزان المرتبطة بالمصطلحات في كل محور. قد تحمل الكلمة وزناً موجباً أو سالباً؛ الإشارة تصف اتجاهاً حسابياً وليست درجة «إيجابية» أو «سلبية».

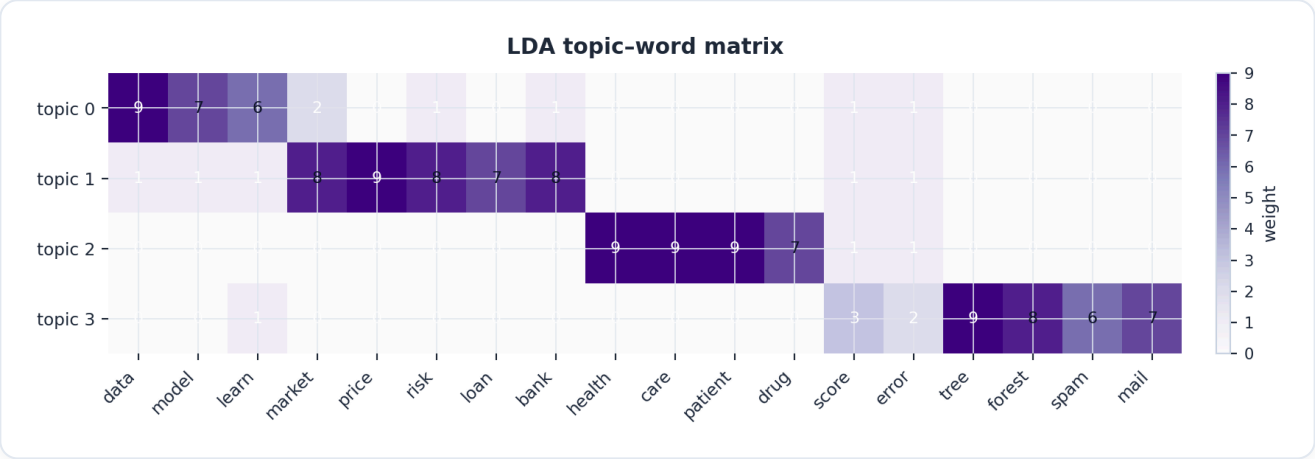
استخراج المحاور الكامنة والكلمات الأعلى وزناً — python

```python
import numpy as np
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "neural network learns patterns from image data",
    "image model detects objects in a picture",
    "deep learning trains a neural network",
    "bank market prices and financial risk",
    "financial models estimate market prices",
    "bank lending depends on credit risk",
    "patient health records support medical care",
    "medical treatment improves patient health",
    "health care monitors patient recovery",
]
vectorizer = TfidfVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)
svd = TruncatedSVD(n_components=3, random_state=42)
Z = svd.fit_transform(X)
terms = vectorizer.get_feature_names_out()

print("شكل الوثائق في الفضاء الكامن:", Z.shape)
print("التباين المفسر لكل مكوّن:",
      np.round(svd.explained_variance_ratio_, 3))
for topic, weights in enumerate(svd.components_):
    top = np.argsort(weights)[-5:][::-1]
    print(f"المحور {topic + 1}:", [terms[i] for i in top])
```

نسمي محاور LSA أحياناً «موضوعات»، لكنها ليست احتمالات ولا يلزم أن تكون موجبة أو مستقلة دلالياً. اختر عدد المكونات بناءً على المهمة، واختبر ثبات المحاور عند تغيير العينة والبذرة؛ فالتقريب العشوائي وتبديل الإشارات قد يغيران التفاصيل دون أن يتغير الفضاء نفسه كثيراً.

٨.٣ LDA: الوثيقة مزيج من مواضيع والكلمة عينة منها

نموذج **تحليل التمايز الخطي (Latent Dirichlet Allocation - LDA)** في هذا الفصل هو نموذج المواضيع النصي `sklearn.decomposition.LatentDirichletAllocation`، لا مصنف تحليل التمايز الخطي الذي تعرفنا عليه في فصل سابق. يفترض أن كل وثيقة مزيج من مواضيع، وأن كل موضوع توزيع على الكلمات. تتعلم الخوارزمية توزيعات المواضيع والكلمات من **أعداد الكلمات**، لذلك استخدم `CountVectorizer` لا TF-IDF بوصفه المدخل المعتاد.



الشكل ١٥٨ تعرض LDA كل موضوع كتوزيع على مفردات المجموعة النصية

تعلّم مواضيع LDA وعرض الكلمات البارزة python

```python
import numpy as np
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "neural network learns image patterns from training data",
    "deep learning model recognizes images and visual objects",
    "machine learning trains a model to classify image data",
    "computer vision detects objects in digital images",
    "image recognition uses neural networks and learned features",
    "bank market prices reveal financial risk",
    "stock market prices change with investment risk",
    "financial models estimate credit risk and loan defaults",
    "bank lending depends on credit scores and market rates",
    "investment portfolio tracks financial markets and returns",
    "patient health records support medical care",
    "medical treatment helps patient recovery",
    "health care monitors clinical outcomes and diagnosis",
    "doctors evaluate patient symptoms and medical history",
    "clinical care improves patient health and recovery",
]
vectorizer = CountVectorizer(stop_words="english")
X_counts = vectorizer.fit_transform(documents)
lda = LatentDirichletAllocation(n_components=3, max_iter=40,
                                learning_method="batch", random_state=42)
document_topics = np.asarray(lda.fit_transform(X_counts))
terms = vectorizer.get_feature_names_out()

for topic_id, weights in enumerate(lda.components_):
    top = np.argsort(weights)[-5:][::-1]
    print(f"الموضوع {topic_id + 1}:",
          [(terms[i], round(float(weights[i]), 2)) for i in top])
print("توزيع أول وثيقة:", np.round(document_topics[0], 3))
print("مجموع الاحتمالات:", round(float(document_topics[0].sum()), 3))
```

تسمية «الموضوع ١» لا تحمل معنى بذاتها؛ ترتيب الموضوعات اعتباطي، كما أن `components_` أوزان موجبة غير مطبّعة مباشرةً إلى احتمالات في العرض. أما صف `transform` لكل وثيقة فيمثل توزيعاً على الموضوعات ومجموعه ١. اضبط `n_components`، و`doc_topic_prior` و`topic_word_prior` بحذر، ثم افحص الوثائق والكلمات يدوياً.

مقارنة

الجانب	LSA مع TF-IDF	LDA مع Count
التمثيل	محاور خطية موقعة عبر SVD	توزيعات احتمالية على الكلمات والمواضيع
مدخل شائع	`TfidfVectorizer` متناثر	`CountVectorizer` متناثر
نتيجة الوثيقة	إحداثيات كامنة قد تكون سالبة	أوزان مواضيع غير سالبة مجموعها ١
التفسير	كلمات ذات أوزان موجبة وسالبة لكل محور	كلمات مرجّحة لكل موضوع وتوزيع لكل وثيقة
أهم تنبيه	الإشارة والترتيب لا يساويان أهمية مطلقة	المواضيع تقريب إحصائي، لا تصنيف جاهز

٨.٤ خط أنابيب للنصوص والمواضيع

كيف نمنع نسيان خطوة التحويل أو تعريض التقييم لتسريب المفردات؟ اربط التحويل والتعلّم في خط أنابيب. بهذه الطريقة، عند إدخاله في التحقق المتقاطع، تُبنى المفردات من جزء التدريب في كل طية فقط.

خط أنابيب CountVectorizer وLDA لوثائق جديدة python

```python
import numpy as np
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import make_pipeline

train_docs = [
    "neural network learns image patterns",
    "deep learning model classifies images",
    "bank market prices and financial risk",
    "financial market changes affect bank lending",
    "patient health records support medical care",
    "medical treatment helps patient recovery",
]
new_docs = ["patient care uses medical records",
            "bank evaluates financial credit risk"]
model = make_pipeline(
    CountVectorizer(stop_words="english"),
    LatentDirichletAllocation(n_components=3, max_iter=15,
                              random_state=21),
)
model.fit(train_docs)
topic_mix = np.asarray(model.transform(new_docs))
print("شكل مزيج المواضيع:", topic_mix.shape)
print("لكل وثيقة، الموضوع الأرجح:", topic_mix.argmax(axis=1))
print("مجموع الأوزان لكل وثيقة:", np.round(topic_mix.sum(axis=1), 3))
```

الفهرسة التي تظهر هنا داخل مصفوفة المواضيع لا تعني أن الموضوع صفر «طب» دائماً؛ قد يتبدل ترتيب المحاور من تدريب لآخر. إذا أردت تسمية موضوع مستقرة، اربطها بتحليل الكلمات والوثائق الممثلة، وقارن النماذج عبر معنى مراجع بشري لا رقم المحور وحده.

خطأ شائع

تعلّم قاموس `CountVectorizer` أو `TfidfVectorizer` من كامل البيانات قبل تقسيم التدريب والاختبار يسرّب معلومات عن مفردات الاختبار. ضع المحلل والنموذج في `Pipeline` واجعل التقييم يتعلمهما داخل كل طية تدريب.

٨.٥ كيف نعرف أن الموضوعات مفيدة؟

لا تكفي قائمة كلمات تبدو معقولة. اقرأ أعلى الوثائق وزناً لكل موضوع، وافحص كلماتها المميزة، وقارن استقرار النتيجة عند تغيير عدد الموضوعات والعينة والبذرة. يمكن استخدام مقاييس التماسك (topic coherence) للمقارنة، لكنها

لا تستبدل الحكم البشري ولا تضمن أن المحاور نافعة للمستخدم.

عند توسيع المثال إلى العربية، عالج التطويل والتشكيل واللواصق وفق هدف التطبيق، وراجع أثر توحيد الألف والياء والتاء المربوطة على معنى المجال. أزل الكلمات الشائعة بقائمة مناسبة، لكن احذر حذف كلمة تبدو عامة وهي حاسمة في نطاقك. وفي بيانات الويب أو الدعم الفني، افحص الخصوصية والبيانات الحساسة قبل إرسال النصوص إلى أي خدمة خارجية.

تمارين الفصل الحادي والسبعين

- **ملاحظة**

التمرين ٨.١

أضف وثيقة تحتوي كلمة شائعة في كل المستندات إلى مثال TF-IDF، وقارن وزنها بكلمة لا تظهر إلا في وثيقة واحدة. ما الذي يحدث لتأثير الكلمة العامة؟

التمرين ٨.٢

درّب `TruncatedSVD` بمكوّنين ثم بخمسة على مجموعة الوثائق الواردة في الفصل، وقارن مجموع `explained_variance_ratio_`. هل يعني ارتفاعه أن الموضوعات أصبحت أسهل تفسيراً؟

التمرين ٨.٣

استخدم `CountVectorizer` و LDA بثلاثة مواضيع، ثم اعرض لكل وثيقة الموضوع الأعلى. افحص أمثلة من كل مجموعة، واكتب سبباً واحداً يمنع اعتبار النتيجة تصنيفاً موثوقاً دون مراجعة.

- **ملاحظة**

حل: الكلمة الشائعة مقابل المصطلح النادر python

```python
from sklearn.feature_extraction.text import TfidfVectorizer

docs = [
    "model data training",
    "model data validation",
    "model data market",
    "model data patient",
]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)
terms = vectorizer.get_feature_names_out()
for term in ("model", "market"):
    j = list(terms).index(term)
    print(term, "weights=", X[:, j].toarray().ravel().round(3))
```

تظهر الكلمة المتكررة في الوثائق بوزن تمييزي أقل من المصطلح الخاص بوثيقة واحدة، لأن وزن IDF يعاقب الانتشار عبر المجموعة.

> **ملاحظة**

حل: توزيع الوثائق على مواضيع LDA python

```python
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

docs = [
    "neural network learns image patterns",
    "deep learning model classifies images",
    "bank market prices and financial risk",
    "financial market changes affect bank lending",
    "patient health records support medical care",
    "medical treatment helps patient recovery",
]
vectorizer = CountVectorizer(stop_words="english")
X = vectorizer.fit_transform(docs)
lda = LatentDirichletAllocation(n_components=3, max_iter=15,
                                random_state=42).fit(X)
mixture = np.asarray(lda.transform(X))
print("الموضوع الأرجح لكل وثيقة:", mixture.argmax(axis=1))
print("أقصى ثقة لكل وثيقة:", mixture.max(axis=1).round(2))
```

إنها مجموعات تقريبية تعتمد على النصوص وعدد الموضوعات والبذرة، ولا تستخدم وسوماً معيارية. قد تتداخل المفردات أو يتغير ترتيب الموضوعات، لذا يلزم فحص الوثائق والكلمات قبل تسمية المجموعات.

> **ملاحظة**

٨.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تحويل مصفوفة المفردات كاملة إلى كثيفة	استهلاك ذاكرة يتناسب مع عدد الوثائق مضروباً في حجم القاموس	حافظ على المصفوفة المتناثرة، وحوّل عينات صغيرة فقط عند الفحص
استخدام TF-IDF مع LDA بلا تبرير	يضعف معنى نموذج عدّ الكلمات وافتراضاته	ابدأ بـ `CountVectorizer` لنموذج LDA التقليدي
اعتبار كل مكوّن في LSA موضوعاً احتماليّاً	أوزان SVD قد تكون سالبة وليست توزيعات مجموعها واحداً	فسّرها كمحاور خطية، لا احتمالات
تسمية موضوع بناء على أكثر كلمتين فقط	قد تختلط موضوعات أو تحكمها كلمة عامة	افحص كلمات ووثائق عديدة وقس الاستقرار
ملاءمة المحلل على كل النصوص قبل التقسيم	يدخل قاموس الاختبار في التدريب	استخدم `Pipeline` داخل التحقق المتقاطع
تطبيق تنظيف إنجليزي افتراضي على العربية	تتجزأ الكلمات أو تحذف صيغ ذات دلالة	صمم خطوة معالجة لغوية عربية وافحص مفرداتها

٨.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مصفوفة الوثائق-المصطلحات	تمثيل صفوفه وثائق وأعمدته كلمات	document-term matrix
TF-IDF	وزن يجمع تكرار الكلمة وندرتها عبر الوثائق	term frequency–inverse document frequency
التحليل الدلالي الخفي	تمثيل خطي كامن للنص باستخدام التحليل منخفض الرتبة	latent semantic analysis
تحليل القيم المفردة المبتور	تقريب منخفض الرتبة لمصفوفة دون بناء متممها الكامل	truncated singular value decomposition
نموذج المواضيع	نموذج احتمالي يمثل الوثيقة مزيجاً من مواضيع	topic model
تماسك الموضوع	مقياس لمدى تقارب كلمات الموضوع دلالياً	topic coherence

الجزء ٩

التقييم والاختيار

بلا قياس دقيق، كل تحسين وهم

ماذا يغطّي هذا الجزء

المبتدئ يقيس الدقة وحدها، ثم يبني نموذجاً ينجح في التقييم ويفشل في الحياة. هذه القوة تبني ثقافة قياس صارمة: تقسيمات صحيحة، ومقاييس مناسبة لكل مهمة، ومنحنيات ROC و PR، وعتبات قرار، ثم بحث شبكي منضبط، مع شرح الأخطاء التي تُفسد كل هذه الأدوات.

الجزء ٩ · التقييم والاختيار

الفصل ٧٢ — تقسيم البيانات: الخطوة التي تُفشل المشاريع

train_test_split وترتيب العمليات والتسرب غير المقصود

الوحدة **scikit-learn ١.٩** الحجم **١٠٧٩** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا يعدّ التقييم على بيانات التدريب جريمة قياس تُبطل كل استنتاج
- أن تضبط test_size و random_state و shuffle و stratify بحسب طبيعة كل مشروع
- أن تقدّر تقلب تقدير الأداء عبر عشرين بذرة بدل الحكم على بذرة واحدة
- أن تكتشف التسرب الإحصائي والزمني والمجموعاتي في ترتيب العمليات الخاطئ
- أن تعالج كل نوع من التسرب من Pipeline إلى GroupShuffleSplit

٩.١ دقة على الورق، وفشل في الحياة

غادرنا الجزء الثامن ونحن نترك النصوص تروي مواضيعها الخفية في الفصل ٧١، وقد بنينا نموذجَ من كل عائلة. لكن بقي السؤال الذي يفصل بين مهندسٍ يتعلّم ومهندسٍ يُتقن: **مَن يمنحك حق القول إن نموذجك جيد؟**

تخيّل المشهد: فريق يعمل على كشف تعثر العملاء. بعد أسبوعين يعلن المطور المبتدئ: «الدقة ٩٧ بالمئة!»، تُرفع توقعات الإدارة ويُنشر النموذج، ثم تنهار النتائج في الإنتاج إلى ما دون ٦٠ بالمئة. لا خطأ في الخوارزمية ولا المعالجة؛ لقد ارتكب المطور **جريمة قياس**: قاس على بيانات **رآها أثناء التدريب**.

لماذا هذه جريمة؟ لأن النموذج في أثناء التدريب لا يلتقط «القانون العام» فحسب، بل يلتقط «العجائب المحلية»: الضجيج والتطابقات الصدفية وخصائص العينات بعينها. الشجرة غير المقيدة — كما رأينا في الفصل ٣٩ — تحفظ كل عينة تدريب على حدة؛ إذا قست دقتها على البيانات نفسها فأنت تختبر ذاكرة النموذج لا تعميمه:

جريمة القياس: تقييم النموذج على ما حفظه python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

tree = DecisionTreeClassifier(random_state=42).fit(X_tr, y_tr)
print(f"الدقة على بيانات التدريب (رآها أثناء الملاءمة): {tree.score(X_tr, y_tr):.4f}")
print(f"الدقة على بيانات الاختبار (لم يرها قط):       {tree.score(X_te, y_te):.4f}")
print(f"الفجوة: {tree.score(X_tr, y_tr) - tree.score(X_te, y_te):.4f}")
```

الشجرة حققت الدقة الكاملة ١.٠٠٠ على ما حفظته وهبطت إلى ٠.٩١٨١ على الجديد. الفجوة بين الرقمين هي **الذاكرة لا المعرفة**. من هنا تنبع قاعدة الجزء التاسع بأكمله: أي رقم أداء لا يقف خلفه فصلٌ واضح بين ما رآه النموذج وما يُقاس عليه، هو رقم بلا قيمة — وقد يكون أسوأ: رقم كاذب يمنحك ثقة زائفة.

🔑 الفكرة المحورية

التقييم تجربة قياس علمية، والقياس الصحيح يتطلب عينة اختبار **لم يشارك النموذج في بنائها بأي شكل**: لا في ملاءمة أوزانه، ولا متوسطات التحجيم، ولا اختيار خصائصه ولا `gamma`. كل اتصال بين الاختبار وعمليات البناء يسمى **تسرباً (leakage)**، وكل تسرب يضخّم الأداء الظاهري ويهدم الأداء الحقيقي.

⚠ الخطأ الكلاسيكي: score على بيانات التدريب

استدعاء `model.score(X_train, y_train)` وعرض نتيجته دون تنبيه أخطر أخطاء المبتدئين؛ فالرقم مبهر دائماً. القاعدة: **لا يُعرض رقم أداء إلا مع ذكر البيانات التي قيس عليها**.

الدرس: قبل أي نموذج قرر أولًا أين مجموعة الاختبار، ثم احبسها عن كل خطوة بناء حتى القياس.

٩.٢ تشريح train*test*split: أربعة بارامترات تحدد كل شيء

`sklearn.model_selection.train_test_split` تبدو أبسط دالة في المكتبة، لكن أربعة بارامترات فيها تحدد مصير تقييمك كله:

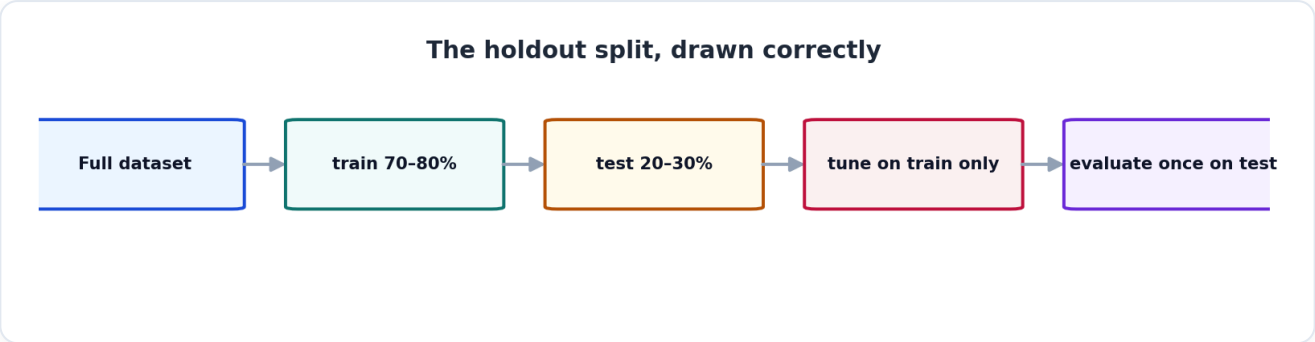
البارامتر	الافتراضي	المعنى ومتى تغيّره
`test_size`	٠.٢٥	حجم الاختبار: نسبة (٠.٢) أو عدد صريح (١٥٠)
`random_state`	None	بذرة الخلط؛ ضعها دائماً للتكرار
`shuffle`	True	خلط الصفوف؛ أوقفه مع البيانات الزمنية
`stratify`	None	عمود توازن الفئات؛ مرّر `y` في التصنيف

جرّب بنفسك: نفس البذرة تنتج التقسيم نفسه حرفياً (العادة الإلزامية `random_state=42` في هذا الكتاب)، و`shuffle=False` يجعل التقسيم «أول ٧٠٪ تدريباً وآخر ٣٠٪ اختباراً» — مطلوب مع الزمن وخطير مع كل ما عدا الزمن:

تشريح train_test_split عملياً — python

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)

# (1) test_size كعدد صريح + stratify يوازن التوزيع
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=150, random_state=42,
stratify=y)
print(f"الأحجام: تدريب {len(X_tr)} / اختبار {len(X_te)}")
print(f"نسبة الموجبة: الكل {y.mean():.4f} | تدريب {y_tr.mean():.4f} | اختبار
{y_te.mean():.4f}")

# (2) نفس البذرة = نفس التقسيم؛ بذرة مختلفة = تقسيم آخر
a = train_test_split(X, y, test_size=150, random_state=42, stratify=y)
b = train_test_split(X, y, test_size=150, random_state=42, stratify=y)
c = train_test_split(X, y, test_size=150, random_state=7, stratify=y)
print("نفس البذرة: تطابق؟", np.array_equal(a[0], b[0]) and np.array_equal(a[1],
b[1]))
print("بذرتان مختلفتان: تطابق؟", np.array_equal(a[0], c[0]))

# (3) shuffle=False يحافظ على الترتيب
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
shuffle=False)
print("مع shuffle=False: التدريب أول الصفوف؟", np.array_equal(X_tr, X[:len(X_tr)]))
```

نقرأ ثلاث حقائق: `test_size=150` أعطى اختباراً بهذا الحجم بالضبط، و `stratify=y` جعل نسبة الموجبة متطابقة تقريباً في المجموعتين (٠.٦٢٧٧ و٠.٦٢٦٧ مقابل ٠.٦٢٧٤ إجمالاً)، والبذرة الثابتة ضمنت قابلية التكرار.



الشكل ١٥٩تشريح التقسيم: خلط بالبذرة ثم توزيع يحافظ على توازن الفئات بين الجانبين

يوضح الشكل دورة التقسيم: خلط عشوائي محكوم بالبذرة، ثم توزيع الصفوف مع ضمان — عند تمرير `stratify=y` — تمثيلٍ مماثل لتوزيع الفئات في الجانبين. والأهم مما يظهر: **مجموعة الاختبار لا تُلمس مرة أخرى** حتى القياس الأخير.

ماذا لو تركنا الحظ يقرر؟ البيانات غير المتوازنة هي موضع الخطر العاري: مشروع كشف احتيال بنسبة إيجابيات ٥٪ فقط؛ بدون `stratify` يصبح عدد الإيجابيات في الاختبار سحبة عشوائية قد تنحرف بشدة عن المتوقع:

تجربة قاطعة: 5% إيجابيات بلا stratify تتفاوت بالحظ — python

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import numpy as np

X, y = make_classification(n_samples=1000, n_features=20,
                           weights=[0.95, 0.05], flip_y=0, random_state=0)
print(f"إجمالي الإيجابيات: {int(y.sum())} من {len(y)} (نسبة {y.mean():.3f})")

plain = [int(y_te.sum()) for s in range(20)
         for _, _, _, y_te in [train_test_split(X, y, test_size=0.2,
random_state=s)]]
strat = [int(y_te.sum()) for s in range(20)
         for _, _, _, y_te in [train_test_split(X, y, test_size=0.2,
random_state=s, stratify=y)]]
print(f"إيجابيات الاختبار عبر 20 بذرة بلا stratify: {plain}")
print(f"أدنى {min(plain)} وأعلى {max(plain)} مقابل المتوقع {0.2 * int(y.sum()):.0f}")
print(f"مع stratify: {sorted(set(strat))} في كل البذور")
```

المتوقع في اختبار من ٢٠٠ عينة هو ١٠ إيجابيات، لكن الحظ بلا `stratify` منحه تكرارات ٦ وأخرى ١٤ — النموذج سيُختبر مرة على ضعف الإيجابيات تقريباً ومرة على نصفها، وينعكس ذلك على أي مقياس يعتمد الفئة الأندر (الاسترجاع وF1 — الفصل ٧٥). مع `stratify=y` اختفت الحظة: ١٠ في كل البذور.

> **اجعل stratify=y رد فعلك الأول**
>
> في أي تصنيف — حتى متوازن الفئات — مرّر `stratify=y` : الكلفة صفر والحظ لا يقرر توزيع الفئات. الاستثناء الوحيد الانحدار، لأن الهدف `y` مستمر فلا معنى لطبقة توزيع رقمية متصلة.

٩.٣ عشرون بذرة، عشرون حكماً: تقلب التقدير

لنفترض أنك قسّمت بطريقة مثالية، ودربت نموذجاً، وحصلت على دقة ٠.٩٦٥٢ على الاختبار. هل هذا «العدد الصحيح» لأداء النموذج؟ كل تقسيم ينتج اختباراً مختلفاً قليلاً وبالتالي تقديراً مختلفاً؛ تقدير الأداء من تقسيم واحد هو **سحبة واحدة من توزيع** كامل من التقديرات، وعرضها على أنها «الحقيقة» خطأ إحصائي بحت:

$$\mathrm{Var}(\hat{R}) = \frac{p\,(1-p)}{n_{\text{test}}}$$

حيث `p` هي الدقة الحقيقية و`n_test` عدد عينات الاختبار: تباين التقدير يتناقص مع حجم الاختبار لكنه لا يصفر أبداً، وبذرة التقسيم تحدد انحراف سحبتك الصدفي. لنقس الأثر: نفس النموذج بالضبط (`StandardScaler` + `KNeighborsClassifier(n_neighbors=5)`) على نفس البيانات، لكن بعشرين بذرة:

عشرون بذرة، عشرون تقديراً لنفس النموذج — python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
scores = []
for seed in range(20):
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.3, random_state=seed, stratify=y)
    scaler = StandardScaler().fit(X_tr)
    model = KNeighborsClassifier(n_neighbors=5).fit(scaler.transform(X_tr),
y_tr)
    scores.append(model.score(scaler.transform(X_te), y_te))

scores = np.array(scores)
print(f"أدنى دقة: {scores.min():.4f} | أعلى دقة: {scores.max():.4f}")
print(f"المتوسط: {scores.mean():.4f} | الانحراف المعياري: {scores.std():.4f}")
print(f"المدى الكامل: {scores.max() - scores.min():.4f}")
```

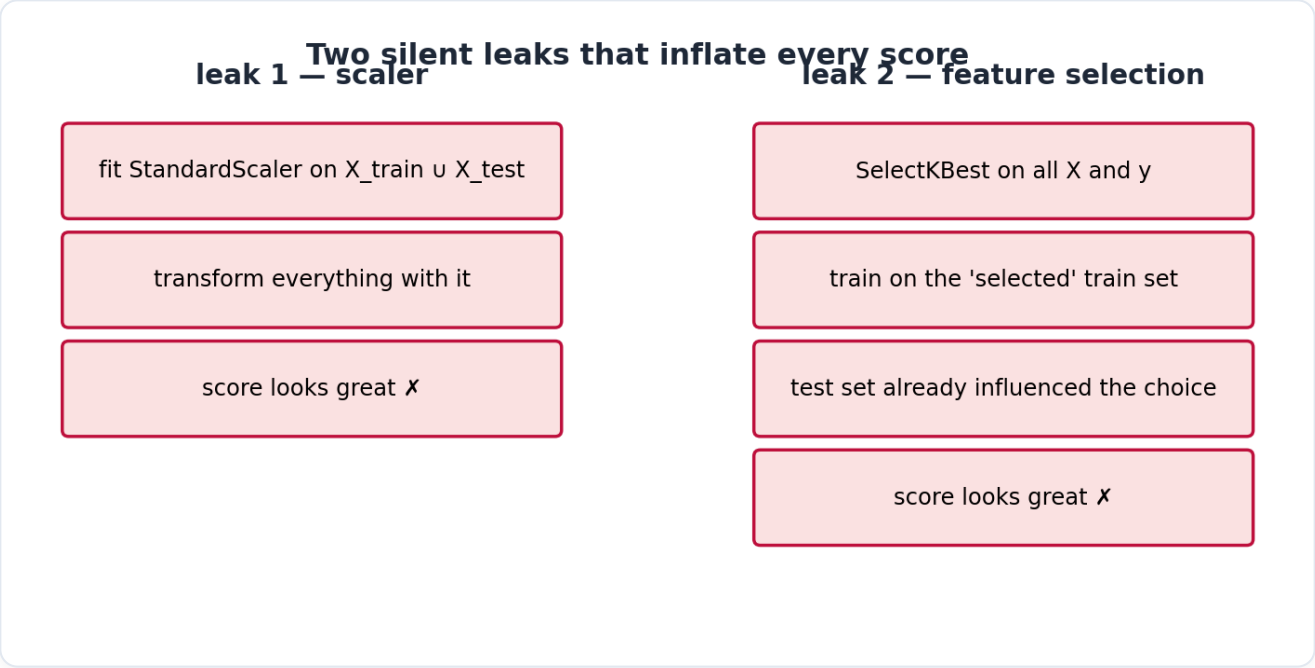
نفس النموذج، نفس البيانات، لكن التقدير يتأرجح بين ٠.٩٤١٥ و ٠.٩٨٨٣ حسب البذرة — فارق يقارب خمس نقاط مئوية بانحراف معياري ٠.٠١٢٣؛ تخيل منافساً يعرض ٠.٩٨٨٣ وأنت تعرض ٠.٩٤١٥ والحقيقة أنكما تختبران النموذج نفسه! الحكم من تقسيم واحد يشبه الحكم على متوسط طول البشر من شخص واحد.

الدرس: رقم أداء بلا مقياس تقلب نصف رقم؛ والقياس المنهجي للتوزيع هو التحقق المتقاطع (الفصل التالي).

٩.٤ التسرب غير المقصود: تفوّق زوراء

وصلنا إلى قلب الفصل. السيناريو الأشهر لفشل المشاريع ليس «نموذجاً سيئاً»، بل **نموذجاً متوسطاً يظهر عبقرياً** — بخطوة معالجة واحدة تُنفَّذ **قبل** التقسيم، مثل `StandardScaler().fit(X)` على كل البيانات. يبدو بريئاً، لكن `fit` هنا حسب المتوسط والانحراف المعياري من **كل الصفوف بما فيها الاختبار**، فتسربت معلومات من الاختبار إلى تمثيل التدريب. والشيء نفسه يحدث مع `SimpleImputer` الذي يحسب الوسيط من كل البيانات (الفصل ١٣)، ومع الترميزات التي تتعلم خريطة الفئات من كل البيانات (الفصل ١٦)، ومع أي انتقاء خصائص يُملأ على كل البيانات (الفصل ٢٤).

القاعدة الذهبية: **أي محوّل يتعلم شيئاً من البيانات يجب أن يُملأ (fit) على التدريب فقط، ثم يُطبَّق (transform) على الاختبار.**



الشكل ١٦٠أنماط التسرب: ملاءمة التحويلات على كل البيانات قبل التقسيم مقابل المسار النزيه

قد يقول متشكك: «كم قد يضر؟». لنجيب بتجربة قاطعة على بيانات **لا تحمل أي إشارة حقيقية**: ٣٠٠ عينة و٢٠٠٠ ميزة عشوائية نقية وتسميات عشوائية؛ الأداء الصادق الوحيد الممكن هو حظ التخمين ٠.٥. سننفّذ «معالجة» شائعة —

تحجيم + انتقاء أفضل ٢٠ ميزة — مرة على كل البيانات قبل التقسيم (المسرب)، ومرة داخل خط أنابيب يُملأ على التدريب وحده (النزيه):

تجربة قاطعة: تسرب يصنع عبقرية من بيانات عشوائية نقية — python

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(42)
X = rng.normal(size=(300, 2000))        # ضجيج نقي: لا علاقة بالتسميات
y = rng.integers(0, 2, 300)

# المسرب: تحجيم + انتقاء على كل البيانات قبل التقسيم
X_scaled = StandardScaler().fit_transform(X)
X_best = SelectKBest(f_classif, k=20).fit_transform(X_scaled, y)
Xtr, Xte, ytr, yte = train_test_split(X_best, y, test_size=0.5,
                                      random_state=42, stratify=y)
leaky = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
print(f"الدقة المزيفة (معالجة قبل التقسيم): {leaky.score(Xte, yte):.4f}")

# النزيه Pipeline: نفس الخطوات، مُلائمة على التدريب فقط داخل
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.5,
                                      random_state=42, stratify=y)
honest = make_pipeline(StandardScaler(), SelectKBest(f_classif, k=20),
                       LogisticRegression(max_iter=1000)).fit(Xtr, ytr)
print(f"الدقة الحقيقية (داخل Pipeline معالجة): {honest.score(Xte, yte):.4f}")
```

بيانات عشوائية مئة بالمئة، ومع ذلك «حقق» النموذج المسرب دقة ٠.٦٤٠٠ — أعلى من الحظ بـ ١٤ نقطة مئوية، بينما وقع النزيه عند ٠.٤٨٦٧ أي عند الحظ. من أين جاء الفارق؟ من **انحياز الاختيار (selection bias)**: تفحص ٢٠٠٠ ميزة عشوائية بحثاً عن أفضل ارتباط مع التسميات فتجد صدفةً ميزات «تفسّر» جزءاً من الضجيج — بما فيه ضجيج صفوف الاختبار نفسها. رياضياً، أفضل ارتباط صدفي من بين `d` ميزة على `n` عينة يبلغ تقريباً:

$$\mathrm{E}\left[\max_{1\le j\le d}|r_j|\right] \approx \sqrt{\frac{2\ln d}{n}}$$

حيث `r_j` ارتباط الميزة `j` مع التسميات. في تجربتنا: `d = 2000` و `n = 300` تعطيان ارتباطاً صدفياً متوقعاً يقارب ٠.٢٢٥ — أي «تنبؤ» بجزء حقيقي من اختباره دون أي معرفة. وكلما زادت الأبعاد صار التسرب أكثر سخية؛ لهذا يظهر أشد أشكاله في البيانات الجينية والنصية ذات الآلاف من الميزات.

> ⛔ **التسرب لا يظهر إلا عند الفشل في الإنتاج**
>
> التسرب لا ينتج تحذيراً ولا استثناءً — ينتج **رقماً جميلاً** يمر من كل الفحوص والكارثة تتكشف بعد النشر. لهذا يجب أن يكون منعه **بنيوياً**: ضع المعالجة داخل `Pipeline` (الفصل ٨٤) فتصبح إعادة الملاءمة على التدريب فقط سلوكاً تلقائياً لا عادة قابلة للنسيان.

يلخّص الجدول التالي أنواع التسرب وعلاجها؛ النوعان الأخيران لهما فصل كامل (الزمن) وقسم خاص أدناه (المجموعات):

نوع التسرب	كيف يحدث	العلاج
إحصاءات المعالجة	`fit` للمحوّل (تحجيم/إسناد/ترميز) على كل البيانات قبل التقسيم	`Pipeline` على التدريب فقط
انتقاء الخصائص	`SelectKBest` أو RFE يُملأ قبل التقسيم	المُنتقي خطوة داخل الأنابيب (الفصل ٢٤)
تسرب الزمن	خلط بيانات زمنية فيدرّب على الغد ويُختبر على الأمس	`TimeSeriesSplit` و `shuffle=False` (الفصل ٧٣)
تسرب المجموعات	صفوف نفس المريض/العميل في الجانبين	`GroupKFold` / `GroupShuffleSplit` مع `groups=`
ضبط الفائق على الاختبار	اختيار `gamma` الأفضل على الاختبار	مجموعة تحقق ثالثة أو بحث متداخل (الفصل ٨١)

> ☑ **فحص تسرب سريع قبل تسليم أي مشروع**

الدرس: الترتيب لا الشيفرة. نفس الدوال بترتيب صحيح تعطي ٠.٤٨٦٧ الصادقة؛ وبترتيب خاطئ تعطي ٠.٦٤٠٠ الكاذبة.

٩.٥ تسرب الزمن وتسرب المجموعات: قصة الأسواق والمستشفيات

تسرب الزمن أخطر الأشكال لأن بياناته «تبيع المستقبل». الصفوف الزمنية تعتمد كل منها على ما قبلها؛ إذا خلطناها بـ `shuffle=True` ندرّب على «مستقبل» كل نقطة ونختبر على «ماضيها»، فيبدو النموذج نبياً وهو لا يفعل سوى النسخ.

التجربة التالية تبني سلسلة انسيابية وتختبر انحدار الجيران الأقرب مرتين: مخلوطاً، وزمنياً صادقاً (تدريب على الماضي واختبار على المستقبل):

```python تسرب الزمن: الخلط يبيع المستقبل، والتقسيم التتابعي يفضحه
import numpy as np
from sklearn.model_selection import train_test_split, GroupShuffleSplit
from sklearn.neighbors import KNeighborsRegressor

rng = np.random.default_rng(42)
n = 600
t = np.arange(n)
x = t / 60.0 + rng.normal(0, 0.05, n)      # إشارة تنزلق ببطء مع الزمن
y = 3.0 * x + rng.normal(0, 0.2, n)        # الهدف يتبعها
X = x.reshape(-1, 1)

for name, kw in [("shuffle=True (مخلوط)", dict(random_state=42)),
                 ("shuffle=False (تتابعي)", dict(shuffle=False))]:
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, **kw)
    model = KNeighborsRegressor(n_neighbors=5).fit(Xtr, ytr)
    print(f"{name}: R2 = {model.score(Xte, yte):+.4f} | تدريب [{Xtr.min():.2f},
{Xtr.max():.2f}] | اختبار [{Xte.min():.2f}, {Xte.max():.2f}]")
# مقدمة تسرب المجموعات: التقسيم على مستوى المجموعة لا الصف
groups = np.repeat(np.arange(30), 20)      # 30 مريضاً، بكل منهم 20 زيارة
gss = GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=42)
tr_idx, te_idx = next(gss.split(np.zeros((600, 2)), groups=groups))
overlap = set(np.unique(groups[tr_idx])) & set(np.unique(groups[te_idx]))
print(f"تدريب: المجموعات {len(np.unique(groups[tr_idx]))} | اختبار
{len(np.unique(groups[te_idx]))} | تداخل: {len(overlap)}")
```

اقرأ الرقمين: المخلوط أعطى R^2 يقارب الكمال (٠.٩٩٩٤) لأن عينات الاختبار محاطة بأخواتٍ توأم من التدريب؛ أما التتابعي الصادق فأنزل النموذج إلى -٣.٠٦٥١ لأنه لم يرَ قط نطاق المستقبل [٦.٩٧, ٩.٩٩] — تقرير سعر أو مريض مبني على الرقم المخلوط قرار كارثي.

أما **تسرب المجموعات** فقصته: ليست العينة دائماً وحدة الاستقلال بل **المجموعة**: ٢٠ قياساً لنفس المريض أو مئات العمليات لنفس العميل؛ صفوف المجموعة شبه متطابقة، فإذا وقعت في الجانبين معاً اختبرت على أشباح بيانات رآها. الحل المقدم هنا `GroupShuffleSplit` : تمرر `groups=` فتوزع **المجموعات كاملة** دون أي تداخل — ٢١ للتدريب و٩ للاختبار وتداخل صفري. ونسخته المتقاطع `GroupKFold` سيفصّلها الفصل ٧٣ بتجربة تقيس حجم الوهم.

في المشاريع الحقيقية

التسرب الزمني هو السبب الأشهر لانهيار استراتيجيات التداول الآلي التي «تحقّق» ٩٠٪ ثم تخسر في السوق الحي، وتسرب المجموعات هو السبب الأشهر لتضخيم نتائج النماذج الطبية: حين توزع صور نفس المرضى على الجانبين تنهار الأرقام عند الاختبار في مستشفى آخر. اسأل دائماً: ما وحدة الاستقلال في بياناتي؟

الدرس: الخيار في البيانات الزمنية والمجموعاتية ليس «أي تقسيم أدق» بل «أي تقسيم صادق»؛ والصدق يبدأ من `shuffle=False` ومن `groups`.

تمارين الفصل الثاني والسبعين

- **ملاحظة**

التمرين ٩.١

على بيانات `load_wine` (ثلاث فئات بأحجام ٥٩ و٧١ و٤٨)، نفّذ ٢٠ تقسيمة باختبار بحجم ٣٠٪: الأولى بلا `stratify` والثانية معه. احسب في كل تقسيمة عدد عينات كل فئة في الاختبار، ثم أجب: ما أقصى تفاوت بين الفئات بلا طبقية، وهل يظل التوزيع متوازناً معها؟

التمرين ٩.٢

أنشئ بيانات بـ `make_classification(n_samples=400, n_features=60, n_informative=3, random_state=42)` ثم أدخل قيماً مفقودة في ١٠٪ من الخلايا. قارن إسناد `SimpleImputer` على كل البيانات قبل التقسيم (مسرب) مع إسناد داخل `Pipeline` بعد التقسيم (نزيه). هل يتكرر «التفوق الزوراء» على بيانات **ذات إشارة حقيقية**؟ وماذا تستنتج عن حجم الخطر عملياً؟

التمرين ٩.٣

ابنِ سلسلة زمنية بطول ٦٠٠: ميزة تنزلق خطياً مع الزمن، وهدف يتبعها مضافاً إليه مركّب جيبي وضوضاء. قارن `LinearRegression` و`KNeighborsRegressor` على التقسيمين المخلوط والتتابعي، وأجب: لماذا يكشف الخطي «الكذبة» أقل من الجيران الأقرب؟

- **ملاحظة**

```python
# حل التمرين 9.1: أثر stratify على توزيع الفئات
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
import numpy as np

X, y = load_wine(return_X_y=True)
counts_plain, counts_strat = [], []
for seed in range(20):
    _, _, _, y_te = train_test_split(X, y, test_size=0.3, random_state=seed)
    counts_plain.append(np.bincount(y_te, minlength=3))
    _, _, _, y_te = train_test_split(X, y, test_size=0.3, random_state=seed,
stratify=y)
    counts_strat.append(np.bincount(y_te, minlength=3))

plain, strat = np.array(counts_plain), np.array(counts_strat)
print(f"بلا stratify: الفئة الأصغر (48 عينة) في الاختبار من {plain[:, 2].min()} إلى {plain[:,
2].max()}")
print(f"مع stratify: {sorted(set(strat[:, 2].tolist()))} في كل التكرارات")
print(f"بذرة 3 بلا stratify: {plain[3].tolist()} | معها: {strat[3].tolist()}")
```

بلا `stratify` تراوح تمثيل الفئة الأندر في اختبار من ٥٤ عينة بين ١١ و٢٠، بينما مع `stratify` ظلت دائماً ١٥ مطابقة لنصيبها من التوزيع الأصلي؛ وبذرة ٣ مثلاً أعطت بلا طبقية [٢٣, ١٩, ١٢] ومعها [١٨, ٢١, ١٥] — فجوة ١١ عينة صنعها الحظ.

> **ملاحظة**

حل التمرين 9.3: الخطي ضد الجيران على الزمن python

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor

rng = np.random.default_rng(42)
t = np.arange(600) / 60.0
x = t + 0.05 * rng.normal(size=600)
y = 3.0 * x + 2.0 * np.sin(3.0 * t) + 0.2 * rng.normal(size=600)

for name, kw in [("مخلوط  ", dict(random_state=42)),
                 ("تتابعي", dict(shuffle=False))]:
    Xtr, Xte, ytr, yte = train_test_split(x.reshape(-1, 1), y, test_size=0.3,
**kw)
    for label, mdl in [("LinearRegression", LinearRegression()),
                       ("KNeighbors(k=5)  ",
KNeighborsRegressor(n_neighbors=5))]:
        r2 = mdl.fit(Xtr, ytr).score(Xte, yte)
        print(f"{name} | {label}: R2 = {r2:+.4f}")
```

الانحدار الخطي خسر المركّب الجيبي الذي لا يستطيع استقراءه (من ٠.٩٧٥٢ إلى ٠.٧٧٥٠) لكنه أبقى الميل العام، بينما انهار الجيران الأقرب كلياً إلى -٠.٧٧٩١ لأنه «ينسخ من الجوار» على نطاق لم يره. الدرس: شدة ضرر تسرب الزمن تتوقف على حاجة النموذج للاستقراء — النماذج المحلية (kNN والأشجار) أكثر تضرراً من النماذج العالمية البسيطة.

> **ملاحظة**

٩.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
`fit_transform(X)` قبل التقسيم	إحصاءات الاختبار تتسرب وترفع الأداء زوراً	`Pipeline` يُملأ على التدريب فقط
تصنيف غير متوازن بلا `stratify=y`	تفاوت الفئات بالحظ (٦ مقابل ١٤ إيجابياً)	تمرير `stratify=y` في كل تقسيم
خلط بيانات زمنية بـ `shuffle=True`	«تنبؤ» بالماضي بعد رؤية المستقبل وانهيار في الإنتاج	`shuffle=False` أو `TimeSeriesSplit`
صفوف نفس المريض/العميل في الجانبين	اختبار على أشباح عينات التدريب	`groups=` مع `GroupKFold` / `GroupShuffleSplit`

٩.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التسرب	تسلل معلومات من الاختبار (أو المستقبل) إلى بناء النموذج	Data leakage
التقسيم الطبقي	حفاظ الجانبين على توزيع الفئات الأصلي	Stratified split
جريمة القياس	تقييم النموذج على بيانات مُلائمة عليها	Resubstitution evaluation
انحياز الاختيار	تفاؤل من انتقاء ميزات صدفةً من مساحة كبيرة	Selection bias
وحدة الاستقلال	مستوى القسمة: الصف، المريض، اليوم	Independence unit

الجزء ٩ · التقييم والاختيار

الفصل ٧٣ — التحقق المتقاطع: كل أنواعه

Predefined g LeavePOut g TimeSeries g Group g Stratified g KFold

الوحدة **scikit-learn ١.٩** الحجم **٤١٥** كلمة تقريباً

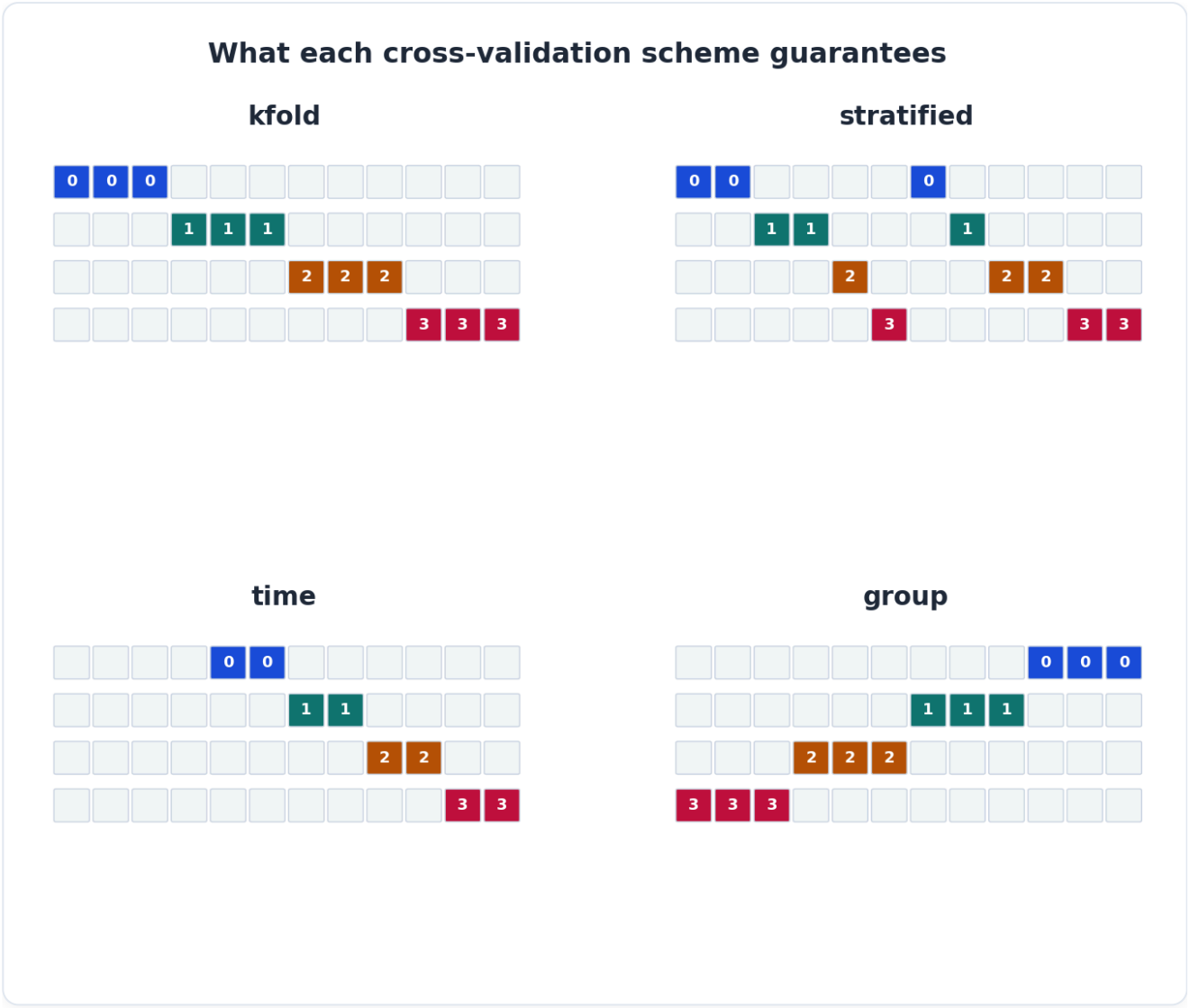
ما الذي ستتقنه في هذا الفصل

- أن يشرح فكرة الطيّات وكيف تتكوّن نتيجة التحقق المتقاطع
- أن يختار مقسّماً يحترم الفئات أو المجموعات أو الترتيب الزمني
- أن يتحقق برمجياً من أن العينة أو المجموعة لا تظهر في جانبي الطية
- أن يميّز بين متوسط الطيات وتقييم الاختبار النهائي

٩.١ لماذا لا تكفي قسمة واحدة؟

لديك مئة سجلّ فقط، وحجزت عشرين منها للاختبار. لو صادف أن العشرين سهلة، بدا النموذج عبقرياً؛ وإن اجتمعت فيها الحالات النادرة، بدا فاشلاً. النتيجة إذن تتعلق بالنموذج وبالحظ الذي صاحب اختيار تلك العينة معاً. في الفصل السابق ثبّتْنا الاختبار النهائي؛ هنا نستخدم بقية البيانات بكفاءة أكبر لتقدير مدى تقلب الأداء.

في التحقق المتقاطع ذي **الطيّات** (folds)، نقسم بيانات التدريب إلى أجزاء. في كل جولة ندرّب على أجزاء ونقيس على الجزء الذي حُجب، ثم نبدّل الجزء المحجوب. كل عينة تدخل التحقق مرة واحدة عادةً في KFold، وتدخل التدريب في الجولات الأخرى. لا يعني ذلك أن لدينا خمسة نماذج إنتاجية؛ إنها تجربة قياس، وبعد اختيار المنهج نعيد التدريب على بيانات التطوير كلها.



الشكل ١٦١تقسيمات الطيات في التحقق المتقاطع

رؤية الطيات لا تخمينها python

```python
import numpy as np
from sklearn.model_selection import KFold

X = np.arange(12).reshape(-1, 1)
cv = KFold(n_splits=4, shuffle=True, random_state=42)
for fold, (train_idx, test_idx) in enumerate(cv.split(X), start=1):
    print(f"الطية {fold}: تدريب={train_idx.tolist()} اختبار={test_idx.tolist()}")
```

النظر إلى الفهارس يكشف ما تخفيه عبارة «تحقق بخمس طيات»: كل طية تدريب مختلفة، والاختبار ليس هو بيانات التدريب في تلك الجولة. أما اختيار `shuffle=True` والبذرة الثابتة فيجعلان تقسيم البيانات المستقلة قابلاً لإعادة الإنتاج.

٩.٢ متى يصبح KFold غير أمين؟

يقسم `sklearn.model_selection.KFold` الصفوف، لا معنى الصفوف. إذا كانت الفئات نادرة، فقد تتجمع الحالات الإيجابية في بعض الطيات. وإذا كان لكل مريض عشر زيارات، قد يتعلم النموذج بصمة المريض في التدريب ثم يتظاهر بالتعميم على زيارة أخرى للمريض نفسه. وإذا كانت الصفوف مرتبة زمنياً، فقد يتعلم من المستقبل ليختبر على الماضي.

`sklearn.model_selection.StratifiedKFold` يحافظ تقريباً على نسب الفئات في كل طية، لكنه لا يحل مشكلة المجموعات أو الزمن. `sklearn.model_selection.GroupKFold` يحجب المجموعات كاملة عن طية الاختبار. أما `sklearn.model_selection.TimeSeriesSplit` فيستخدم كتل تدريب متزايدة، ويختبر على الفترات اللاحقة. المقسّم الصحيح يتبع طريقة وصول البيانات في الواقع، لا الاسم الأكثر شهرة.

التقسيم الطبقي يحافظ على الفئة النادرة python

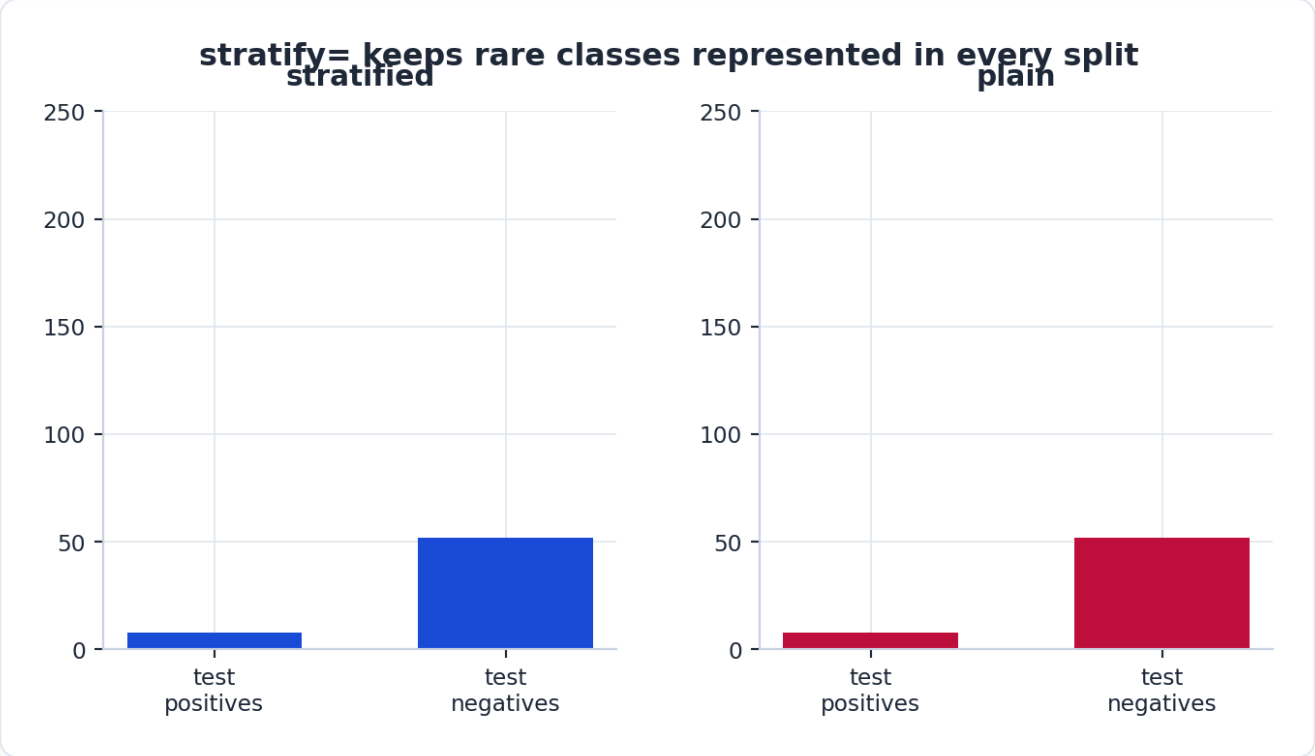
```python
import numpy as np
from sklearn.model_selection import StratifiedKFold

y = np.array([0] * 18 + [1] * 6)
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
for fold, (_, test_idx) in enumerate(cv.split(np.zeros((len(y), 1)), y), 1):
    labels, counts = np.unique(y[test_idx], return_counts=True)
    print(f"الطية {fold}: {dict(zip(labels.tolist(), counts.tolist()))}")
```

يجب الانتباه إلى أن التقسيم الطبقي لا يصنع معلومات جديدة ولا يجعل تقديرك مستقلاً عن حجم العينة. إنه يخفف اختلال توزيع الفئة عبر الطيات. إذا كان عدد أمثلة الفئة الأقل أصغر من عدد الطيات، فلن تستطيع كل طية تمثيلها تمثيلاً صحيحاً.

٩.٣ المجموعات والزمن: اختبر سيناريو النشر

تخيّل بيانات أجهزة طبية، ولكل جهاز قراءات كثيرة. سؤال «هل يعمم على قراءة أخرى للجهاز نفسه؟» يختلف عن «هل يعمم على جهاز لم يره؟». الأول يسمح بتكرار المجموعات بين التدريب والاختبار، أما الثاني فيتطلب `GroupKFold` أو `GroupShuffleSplit` وتمرير `groups` إلى `split`. الوحدة التي يجب أن تُحجب هي الجهاز أو المستخدم، لا الصف الفردي.

في السلاسل الزمنية، الخلط العشوائي يسرّب البنية المستقبلية. يتيح `TimeSeriesSplit` فجوة `gap` بين التدريب والاختبار، وهو مفيد عندما تحمل نافذة السمات معلومة من قرب الحد الفاصل. لا تخلط بين هذا وبين ترتيب الأعمدة أو ترتيب الفهرس؛ تأكد أولاً أن الصفوف مرتبة بحسب وقت الحدث، وأن أي تجميع زمني أو ملء مفقودات لا ينظر إلى المستقبل.



الشكل ١٦٢أشكال مختلفة لتقسيم التحقق المتقاطع

حجب الأشخاص بالكامل python

```python
import numpy as np
from sklearn.model_selection import GroupKFold

groups = np.repeat(np.arange(6), 3)
X = np.arange(len(groups)).reshape(-1, 1)
cv = GroupKFold(n_splits=3)
for fold, (train_idx, test_idx) in enumerate(cv.split(X, groups=groups), 1):
    overlap = np.intersect1d(groups[train_idx], groups[test_idx])
    print(f"الطية {fold}: مجموعات الاختبار={np.unique(groups[test_idx]).tolist()}, "
          f"تداخل={overlap.tolist()}")
```

التداخل الفارغ هو الاختبار الذي نريده: لا يظهر كيان واحد في جانبي الطية. لا يكفي أن تمرر `groups`؛ يجب أن تستخدم مقسّماً يستهلكها، وأن تتأكد أن مجموعة الاختبار تمثل الوحدة الجديدة التي ستواجهها الخدمة.

الماضي يدرب والمستقبل يختبر python

```python
import numpy as np
from sklearn.model_selection import TimeSeriesSplit

X = np.arange(20).reshape(-1, 1)
cv = TimeSeriesSplit(n_splits=4, test_size=2, gap=1)
for fold, (train_idx, test_idx) in enumerate(cv.split(X), 1):
    print(f"الطية {fold}: آخر تدريب={train_idx[-1]}, "
          f"أول اختبار={test_idx[0]}, حجم الاختبار={len(test_idx)}")
```

في كل طية تقع فترة الاختبار بعد التدريب، ويفصل `gap` بينهما صف مستبعد. يظل عليك اختيار طول الاختبار والفجوة وفق زمن الاستجابة ودورية البيانات في المجال، لا وفق ما يعطي أعلى رقم.

في الواقع

في توقع الطلب الأسبوعي، قياس الأداء بتقسيم عشوائي قد يسمح لأسابيع لاحقة أن تشرح أسابيع سابقة عبر اتجاه موسمي أو متغير محسوب من كامل التاريخ. ابدأ بتقسيم زمني، وأعد تنفيذ داخل كل طية كل خطوة تتعلم من البيانات، بما فيها اختيار السمات وضبط المعاملات.

تمارين الفصل الثالث والسبعين

- **ملاحظة**

التمرين ٩.١

أنشئ هدفاً فيه ٢٤ صفاً وثلاث فئات بنسب غير متساوية. قارن أحجام الفئات في الاختبار عبر ثلاث طيات من `KFold` و`StratifiedKFold`، ثم اشرح لماذا لا يضمن التقسيم الطبقي تمثيلاً دقيقاً عند قلة أمثلة الفئة الأصغر.

التمرين ٩.٢

أنشئ اثنتي عشرة مجموعة، في كل منها أربعة صفوف. استخدم `GroupKFold` بأربع طيات، واكتب فحصاً يرفع `AssertionError` إذا تداخلت مجموعة بين التدريب والاختبار.

التمرين ٩.٣

على متجه زمني من ٣٠ صفاً، اعرض فهارس التدريب والاختبار لـ `TimeSeriesSplit` ثم تحقق أن أكبر فهرس تدريب أصغر من أصغر فهرس اختبار.

- **ملاحظة**

مقارنة التوزيع بين مقسّمين python

```python
import numpy as np
from sklearn.model_selection import KFold, StratifiedKFold

y = np.array([0] * 15 + [1] * 6 + [2] * 3)
X = np.zeros((len(y), 1))
for name, cv in [
    ("KFold", KFold(3, shuffle=True, random_state=42)),
    ("StratifiedKFold", StratifiedKFold(3, shuffle=True, random_state=42)),
]:
    print(name)
    for _, idx in cv.split(X, y):
        print(np.bincount(y[idx], minlength=3).tolist())
```

إذا كانت الفئة الأقل لا تملك عينات كافية، لا يمكن توزيعها على كل الطيات؛ التقسيم الطبقي يحافظ على النسبة بقدر ما تسمح به الأعداد الصحيحة.

> **ملاحظة**

```python تأكيد عدم تداخل المجموعات
import numpy as np
from sklearn.model_selection import GroupKFold

groups = np.repeat(np.arange(12), 4)
X = np.zeros((len(groups), 1))
for train_idx, test_idx in GroupKFold(4).split(X, groups=groups):
    assert not np.intersect1d(groups[train_idx], groups[test_idx]).size
print("نجحت فحوص الطيات الأربع: لا مجموعة مشتركة.")
```

> **ملاحظة**

٩.٥ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
خلط صفوف السلسلة الزمنية	يتعلم النموذج من المستقبل	`TimeSeriesSplit` مع ترتيب زمني وفجوة مناسبة
تقسيم زيارات الشخص عشوائياً	تظهر هوية الشخص في التدريب والاختبار	`GroupKFold` وتمرير معرف الشخص كـ `groups`
اعتبار المتوسط أداءً مضموناً	تختفي فروق الطيات وعدم اليقين	اعرض متوسط الدرجات وتشتتها واختباراً نهائياً
ضبط النموذج على طيات الاختبار	تتحول الطيات إلى جزء من عملية الاختيار	خصص مجموعة اختبار نهائية أو طبّق تحققاً متداخلاً

٩.٦ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الطية	جزء يُحجب للتقييم في جولة واحدة	fold
التقسيم الطبقي	الحفاظ على نسب الفئات تقريباً	stratification
المجموعة	كيان يجب أن يبقى في جانب واحد	group
نافذة متوسعة	تدريب يشمل الماضي المتاح حتى الجولة	expanding window

الجزء ٩ · التقييم والاختيار

الفصل ٧٤ — التقاط التقييمات وتتبع الأدوات

Parallel g cross_val_predict g cross_val_score g cross_validate

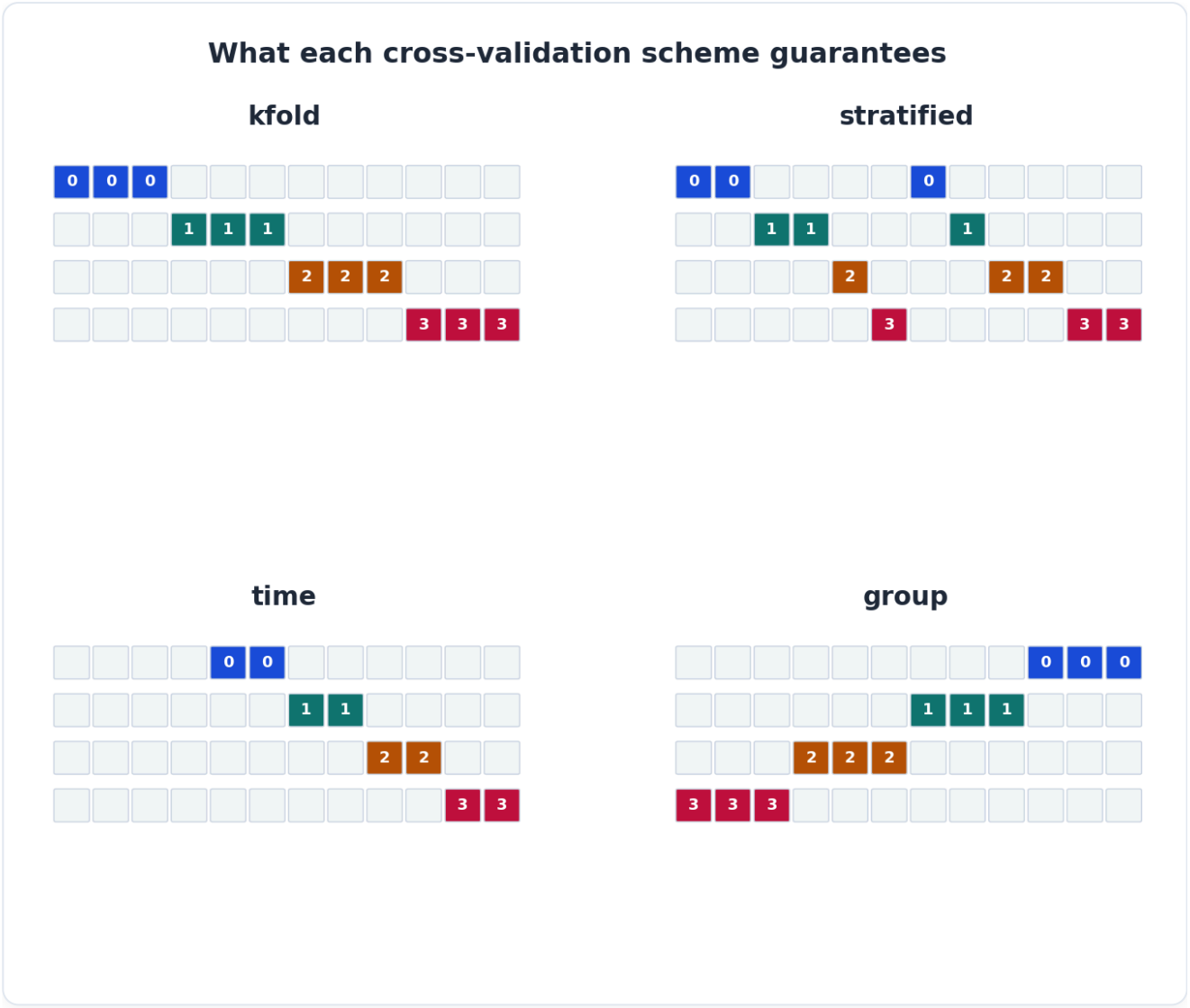
الوحدة **scikit-learn ١.٩** الحجم **٤٢٩** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يختار بين cross_val_score g cross_validate g cross_val_predict
- أن يسجل درجات متعددة وأزمنة الملاءمة والتقييم بأسماء مفهومة
- أن يشرح ما الذي تعنيه تنبؤات خارج الطية وما الذي لا تعنيه
- أن يضبط التوازي ومؤشرات الخطأ وفق حجم المهمة وموارد الجهاز

٩.١ ثلاث واجهات، وثلاثة أسئلة

انتهى الفصل السابق باختيار المقسّم. الآن نسأل: ماذا نريد من كل جولة؟ إذا كان المطلوب متجه درجات لمقياس واحد، فـ `sklearn.model_selection.cross_val_score` موجزة. إذا أردت أكثر من مقياس أو زمن التدريب والتقييم أو درجات التدريب، فـ `sklearn.model_selection.cross_validate` تعيد قاموساً أغنى. وإذا أردت تنبؤاً لكل عينة عندما كانت خارج طية التدريب، فهناك `sklearn.model_selection.cross_val_predict`، لكن هذه ليست أداة لقياس كل المقاييس كما سنرى.



الشكل ١٦٣ مخطط الطيات وعملية التقييم

درجة واحدة مقابل تفاصيل التقييم python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
scores = cross_val_score(model, X, y, cv=5, scoring="balanced_accuracy")
details = cross_validate(model, X, y, cv=5,
                         scoring=["accuracy", "balanced_accuracy"],
                         return_train_score=True)
print("درجات balanced accuracy:", scores.round(3).tolist())
print("مفاتيح التفاصيل:", sorted(details))
```

تُعاد مفاتيح `test_accuracy` و`test_balanced_accuracy` لأن المقياسين طُلبا، ويضيف الخيار `return_train_score=True` الدرجات المقابلة للتدريب. يستخدم كل استدعاء مقسّماً مستقلاً افتراضياً؛ عند المقارنة العلمية مرّر المقسّم نفسه صراحةً إلى كليهما كي تكون المقارنة قابلة للتفسير.

٩.٢ تقرير متعدد المقاييس لا لوحة أرقام

قد تبدو الدقة متقاربة بينما يختلف الاسترجاع كثيراً على الفئة النادرة. لذلك لا تجمع المقاييس في رقم واحد قبل أن تحدد كلفة كل خطأ. تقبل `cross_validate` قاموساً بأسماء مخصصة، فيصبح جدول النتائج قابلاً للقراءة والتحليل من دون الاعتماد على ترتيب القيم.

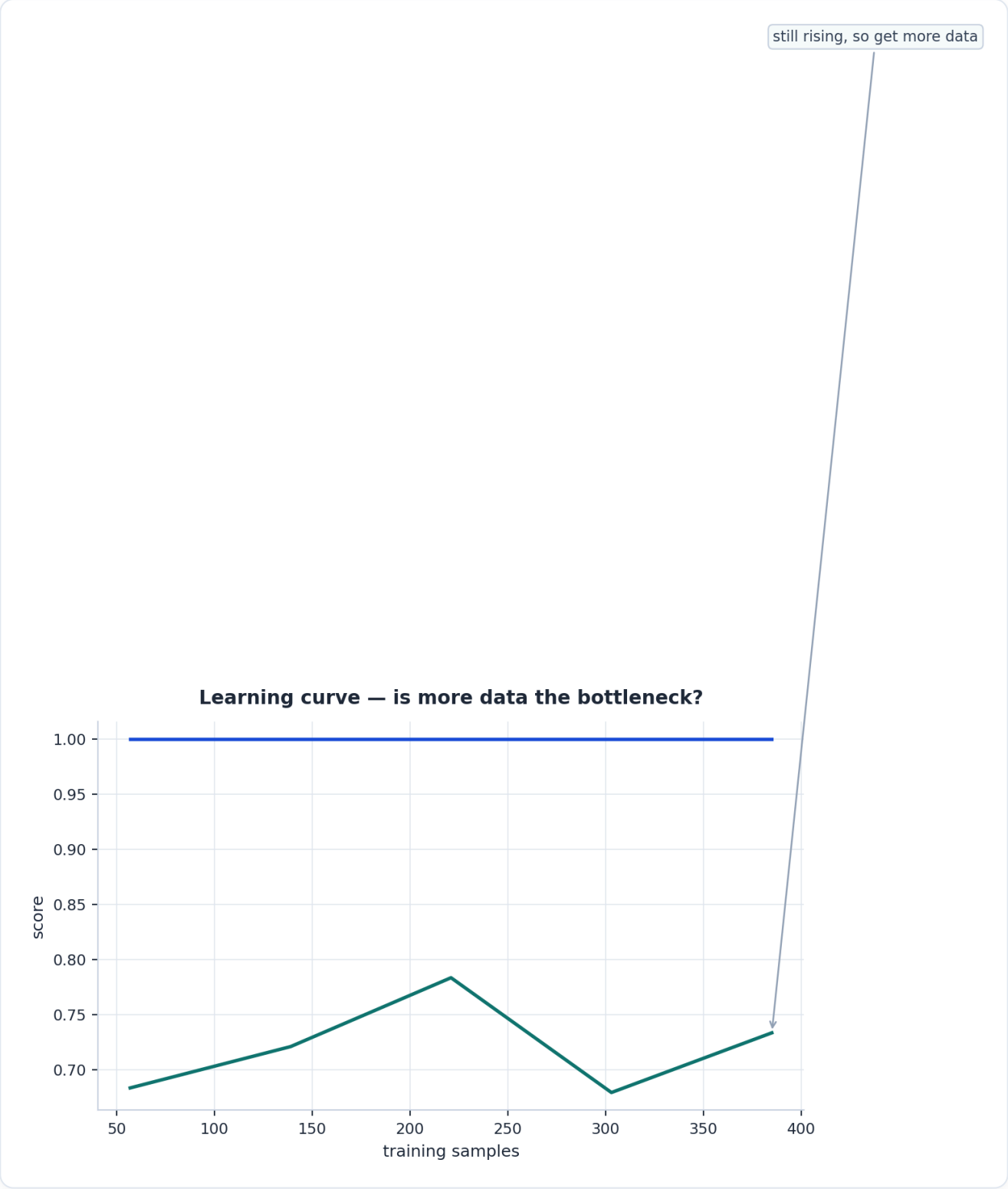
تسمية المقاييس والأزمنة python

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import make_scorer, f1_score, recall_score
from sklearn.model_selection import cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
scoring = {
    "f1_macro": make_scorer(f1_score, average="macro"),
    "recall_macro": make_scorer(recall_score, average="macro"),
}
result = cross_validate(model, X, y, cv=5, scoring=scoring,
                        return_train_score=True)
for key in ("test_f1_macro", "test_recall_macro", "fit_time", "score_time"):
    values = result[key]
    print(f"{key}: mean={np.mean(values):.4f}, std={np.std(values):.4f}")
```

الأزمنة أدوات تشخيص تقريبية وليست ضماناً لزمن الإنتاج: التحميل المسبق، والحمل على الجهاز، وتعدد العمليات تؤثر فيها. أما انحراف المقاييس بين الطيات فيصف حساسية التقدير للتقسيم، لا فاصل ثقة إحصائياً كاملاً بمفرده.

٩.٣ التنبؤ خارج الطية: مفيد لكن ليس سحراً

`cross_val_predict` ينتج تنبؤاً لكل صف من نموذج لم يتدرب على ذلك الصف. هذه التنبؤات مناسبة لبناء رسم تشخيصي أو فحص بواقي خارج العينة، لكنها ليست بديلاً آلياً عن متوسط درجات التحقق: إذا اختلف حجم الطيات أو طريقة تجميع المقاييس عن المطلوب، فقد تعطي درجة محسوبة من التنبؤات المجمّعة تقديراً مختلفاً.



الشكل ١٦٤ منحنى التعلّم يوضح اختلاف أداء التدريب والتحقق

تنبؤ واحد لكل عينة خارج طيتها python

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import confusion_matrix
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
pred = cross_val_predict(model, X, y, cv=cv)
print("عدد التنبؤات:", len(pred), "| عدد العينات:", len(y))
print("مصفوفة الخلط المجمّعة:\n", confusion_matrix(y, pred))
print("دقة التنبؤات المجمّعة:", round(np.mean(pred == y), 4))
```

يجب أن يتطابق عدد التنبؤات وعدد الصفوف هنا لأن كل عينة ظهرت مرة كاختبار. لا تعِد استخدام هذه التنبؤات لاختيار إعدادات النموذج ثم الإبلاغ عن الدرجة ذاتها على أنها تقدير غير متحيز؛ قرار الاختيار نفسه استهلك معلومة التحقق.

٩.٤ التوازي: متى يسرّع ومتى يرهق؟

تستطيع الواجهات قبول `n_jobs` لتشغيل الطيات بالتوازي. الرقم `-1` يطلب كل المعالجات المتاحة، لكنه ليس الخيار الأمثل دائماً: قد يضاعف الذاكرة عندما يحتفظ كل عامل بنسخة من البيانات أو النموذج. يبدأ التشخيص بقياس تشغيل تسلسلي صغير، ثم مقارنة زمنه وذاكرته بتوازٍ محدود. لا توازِ داخل كل طبقة في الوقت نفسه من غير تخطيط؛ البحث المتوازي ومكوّن داخلي متوازٍ قد يتنافسان على الموارد.

تشغيل الطيات بالتوازي المحدود python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate

X, y = load_breast_cancer(return_X_y=True)
model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=1)
result = cross_validate(model, X, y, cv=5, scoring="balanced_accuracy",
                        n_jobs=2, return_train_score=True,
                        error_score="raise")
print("درجات الاختبار:", result["test_score"].round(3).tolist())
print("مجموع زمن الملاءمة:", round(result["fit_time"].sum(), 3), "ثانية")
```

جعلنا الغابة نفسها تسلسلية ووازينا الطيات، كي لا يحدث توازٍ متداخل بلا داعٍ. `error_score="raise"` مفيد أثناء التطوير لأنه يوقف التشغيل عند أول فشل ويظهر الاستثناء الأصلي؛ في تجارب تشغيلية كبيرة يمكن اختيار سياسة أخرى، لكن لا تترك `NaN` في النتائج من دون التحقيق في سببها.

> **في الواقع**
>
> عند تشغيل بحث على عشرات الآلاف من الصفوف، سجل اسم البيانات وإصدار المكتبة وبذرة التقسيم والمقياس ومواصفات المقسّم بجانب النتيجة. الدرجة بلا وصفة لإعادة إنتاجها ليست نتيجة قابلة للتدقيق، مهما بدا عدد منازلها العشرية دقيقاً.

تمارين الفصل الرابع والسبعين

- **ملاحظة**

التمرين ٩.١

استخدم بيانات `load_breast_cancer` وشغّل `cross_validate` بمقياسي `accuracy` و`balanced_accuracy`، مع `return_train_score=True`. اعرض متوسط كل مقياس للفريقين.

التمرين ٩.٢

أنشئ تنبؤات خارج الطية بخمس طيات طبقية، ثم تحقق أن عدد التنبؤات يساوي عدد الصفوف. وضّح لماذا لا تساوي بالضرورة هذه النتيجة متوسط درجات الطيات.

التمرين ٩.٣

نفّذ التحقق مرة بـ `n_jobs=1` ومرة بـ `n_jobs=2`، وسجّل الزمن والدرجات. هل يثبت اختلاف الزمن أن أحد النموذجين أفضل؟

- **ملاحظة**

```python تلخيص درجتي التدريب والاختبار
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
r = cross_validate(model, X, y, cv=5,
                   scoring=["accuracy", "balanced_accuracy"],
                   return_train_score=True)
for k in ("train_accuracy", "test_accuracy",
          "train_balanced_accuracy", "test_balanced_accuracy"):
    print(k, round(float(np.mean(r[k])), 4))
```

الفجوة بين التدريب والاختبار إشارة تشخيصية، وليست وحدها حكماً على سببها.

> **ملاحظة**

```python التحقق من تغطية التنبؤات
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=42)
p = cross_val_predict(make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=3000)), X, y, cv=cv)
assert p.shape[0] == X.shape[0]
print("التغطية كاملة:", p.shape[0])
```

متوسط درجات الطيات يحسب مقياساً لكل طية ثم يجمعه؛ أما المقياس على التنبؤات المجمّعة فيحسبه على جميع الصفوف دفعة واحدة، وقد يختلف الترجيح أو التجميع.

> **ملاحظة**

٩.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام التنبؤات خارج الطية كدرجة وحيدة دائماً	يختلف التجميع عن متوسط درجات الطيات	اختر المقياس وطريقة التجميع مسبقاً
تجاهل `fit_time` **و** `score_time`	يفاجئك نموذج غير عملي بعد الاختيار	قِس الكلفة مع الأداء
ابتلاع أخطاء الملاءمة	تتحول النتائج الفاشلة إلى `NaN`	استخدم `error_score="raise"` أثناء التطوير
`n_jobs=-1` **لكل مستوى**	تزاحم الذاكرة والمعالجات	وازن مستوى توازٍ واحداً وحدد الموارد

٩.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التنبؤ خارج الطية	تنبؤ من نموذج لم ير العينة	out-of-fold prediction
المقياس	دالة تلخص جودة التنبؤ	scoring metric
التوازي	تنفيذ مهام مستقلة في الوقت نفسه	parallelism
زمن الملاءمة	الوقت المستغرق في تدريب النموذج	fit time

الجزء ٩ · التقييم والاختيار

الفصل ٧٥ — مقاييس التصنيف: ما بعد الدقة

الخلط، والدقة، والاسترجاع، و F1، والخصوصية، و MCC

الوحدة scikit-learn ١.٩ الحجم ١١٢١ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح لماذا تصبح الدقة (accuracy) مخادعة عند اختلال الفئات، بمثال رقمي
- أن تحسب الدقة والاسترجاع و F1 والخصوصية و MCC يدوياً وتتحقق منها بـ sklearn
- أن تختار قيمة average الصحيحة (binary/macro/micro/weighted) حسب المسألة
- أن تقرأ confusion_matrix و classification_report بترتيب فئات صحيح عبر labels و pos_label

٩.١ قصة الدقة الكاذبة: ٩٩٪ دقة وصفر فائدة

مستشفى يستقبل ١٠٬٠٠٠ مريض، ومرض قليل الانتشار يصيب ١٠٠ منهم فقط (انتشار ١%). طوّر فريقٌ «نموذجاً ذكياً» وأعلن: **الدقة ٩٩%!** لكن فتح النموذج من الداخل كشف الحقيقة المخزية: كان يصرخ «لا مرض» لكل من يدخل الباب، بلا استثناء. قواعد الحساب تقول: طابق ٩٬٩٠٠ سليماً وأخطأ في ١٠٠ مصاب، فالقسمة ٩٬٩٠٠ ÷ ١٠٬٠٠٠ = ٠.٩٩. الأرقام صادقة، لكن **الفائدة صفر**: لم يُكتشف مريض واحد.

```python
# مصنف «لا مرض دائماً» يحصد ٩٩٪
import numpy as np

n_patients = 10000
n_sick = 100  # 1 نسبة الانتشار%

y_true = np.zeros(n_patients, dtype=int)
y_true[:n_sick] = 1            # أول 100 مريض مصاب

y_naive = np.zeros(n_patients, dtype=int)  # «لا مرض» للجميع بلا استثناء

accuracy = (y_true == y_naive).mean()                    # الدقة
recall = (y_naive[y_true == 1] == 1).mean()              # ماذا اكتشف من المصابين؟

print(f"الدقة accuracy:   {accuracy:.2%} — رقم يفتح الأبواب في العروض!")
print(f"الاسترجاع recall: {recall:.2%} — لم يكتشف مريضاً واحداً!")
```

الناتج يقول كل شيء: دقة ٩٩.٠٠% واسترجاع ٠.٠٠%. رقم مهيب ورقم مخجل من النموذج نفسه. والدرس المؤسس لهذا الفصل:

> **مفتاح**
>
> الدقة (accuracy) تجيب عن سؤال واحد: «كم مرة أصبت؟» — بينما السؤال الحقيقي في معظم التطبيقات: «أين أخطأت بالضبط؟». مصفوفة الخلط هي الأداة الوحيدة التي تجيب عن السؤال الثاني، وكل المقاييس الجادة في هذا الفصل تلخيصات ذكية لها.

والقاعدة الذهبية: كلما تباعد حجم الفئتين فقدت الدقة معناها — عند انتشار ١.٠% تحصل على ٩٩.٩% مجاناً دون أن تمس البيانات. إذاً ما البديل؟ تفكيك الأخطاء نفسها بدل تجميعها في رقم واحد.

٩.٢ مصفوفة الخلط: أربع خانات تحكم كل شيء

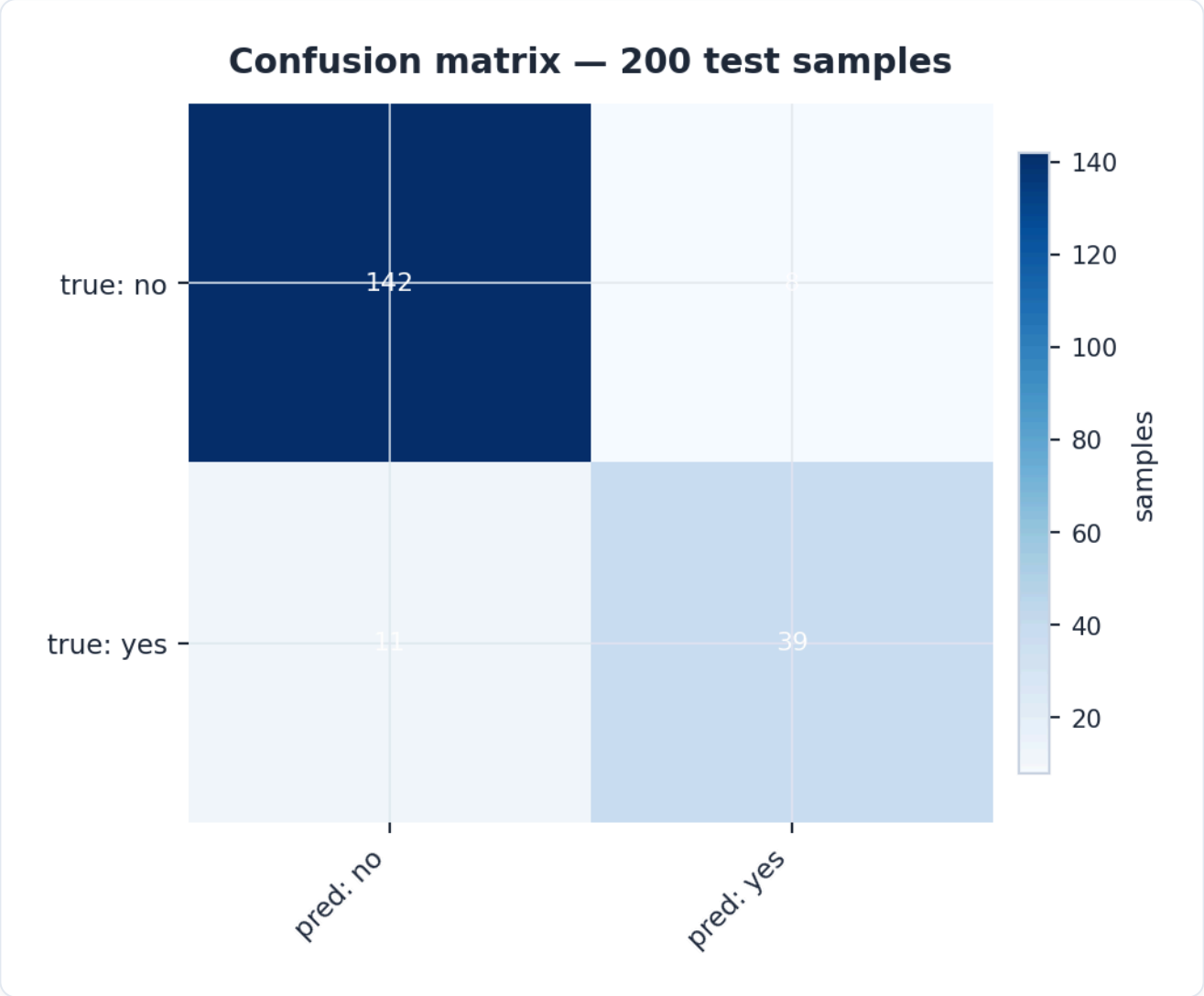
عندما يصدر المصنّف حكمه تتقاطع إجابته مع الحقيقة في أربع احتمالات فقط، وهي **مصفوفة الخلط (confusion matrix)**: TP مريض اكتُشف، و TN سليم طُابق، و FP إنذار كاذب أزعج سليماً، و FN مرض متفوّت — الكارثة. البادئة (True/False) تعني «هل الحكم صحيح؟» واللاحقة (Positive/Negative) تعني «ماذا قال؟».

قبل بيانات حقيقية، انتبه لفخ بيانات سرطان الثدي: الفئة ٠ تعني **خبيثاً** والفئة ١ حميداً. بما أن «المرض» عندها هو الصفر، سنعيد تعريف الإيجابية لتكون الخبيث — وهذا أول درس عملي في السيطرة على ترتيب الفئات:

مصفوفة الخلط على بيانات سرطان الثدي python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

X, y = load_breast_cancer(return_X_y=True)   # 0: حميد ، 1: خبيث
y_sick = (y == 0).astype(int)                # نعيد التعريف: 1 = خبيث (إيجابي)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y_sick, test_size=0.25, random_state=42, stratify=y_sick
)

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
model.fit(X_tr, y_tr)
y_pred = model.predict(X_te)

cm = confusion_matrix(y_te, y_pred)   # الصفوف: الحقيقة ، الأعمدة: التوقع
tn, fp, fn, tp = cm.ravel()
print(f"TP = {tp:3d} (خبيث اكتُشف)      FN = {fn} (خبيث تفوّت!)")
print(f"FP = {fp:3d} (إنذار كاذب)      TN = {tn} (حميد طُابق)")
print(f"الدقة الخام: {(tp + tn) / cm.sum():.4f}")

# يعيد الترتيب كما تريد: الخبيث أولاً والمعامل labels
cm_flipped = confusion_matrix(y_te, y_pred, labels=[1, 0])
print("تصبح الخانة TP:", cm_flipped[0, 0] ,"هي [0 ,0] بالترتيب labels=[1, 0]")
```

أربعُ أورام خبيثة تفوّت عليها من أصل ٥٣ في الاختبار — ولم يكشفها رقم الدقة (٠.٩٦٥٠). الصفوف الحقيقة والأعمدة التوقع، والترتيب الافتراضي يتبع `np.unique` أي [٠, ١]، و `labels=` يعيد الترتيب يدوياً متى شئت الإيجابي أولاً.



الشكل ١٦٥تشريح مصفوفة الخلط: الخانات الأربع وموقع الدقة والاسترجاع والخصوصية داخلها

في الشكل تقرأ الخانات الأربع على محورين: حكم النموذج أفقياً والحقيقة رأسياً. لاحظ أن الضلع السفلي (FN و TP) هو مقام الاسترجاع، والضلع الأيمن (FP و TP) هو مقام الدقة — ملاحظة هندسية تكفيك لاشتقاق كل المعادلات القادمة ذهنياً.

خطأ شائع

الاتجاه الذي تنظر منه للمصفوفة يقلب معنى كل شيء: `ravel()` يعيد [TN, FP, FN, TP] **بافتراض** أن الإيجابية هي ١. إذا كانت الإيجابية عندك · (كما في بيانات الثدي الأصلية) فستقرأ الأسماء معكوسة وتعلن أن «الاسترجاع ممتاز» بينما تقيس الخصوصية. حدد دائماً من هو الإيجابي قبل أي حديث عن مقاييس.

٩.٣ الدقة والاسترجاع و F1: معادلات تحمل فلسفة

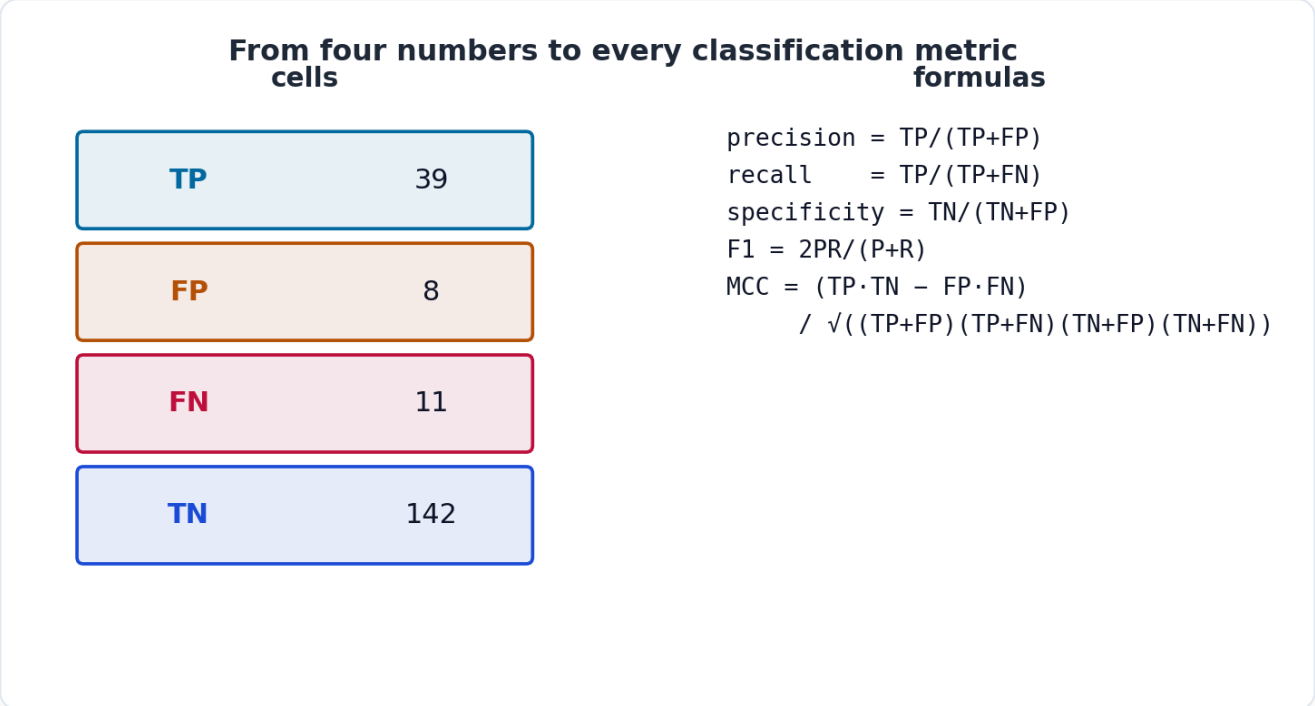
من الخانات الأربع تُشتق مقاييس لا تنخدع بالانتشار. المقيسان الأساسيان ينظران إلى جانبين متعاكسين:

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

ودائماً بينهما شد وجذب: «الجميع مريض» يرفع الاسترجاع ويهدم الدقة، والتحفظ الشديد يفعل العكس. مقياس F1 يجمعهما في رقم واحد يحاسب التوازن:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,TP}{2\,TP + FP + FN}$$

ويختصر المخطط التالي خريطة كل المعادلات القادمة قبل أن نفصّلها:



الشكل ١٦٦خريطة المقاييس: من خانات مصفوفة الخلط إلى الدقة والاسترجاع و F1 والمقاييس المشروطة

في الشكل تتفرع الأسهم من الخانات الأربع إلى مقام كل مقياس: الدقة من الأيمن، والاسترجاع من السفلي، و F1 يجمعهما، والمقاييس المشروطة تعممها بأوزان مضبوطة.

ولماذا هذا الشكل «التوافقي» لا المتوسط الحسابي المألوف؟ لأن الحسابي متسامح مع التفاوت الصارخ: دقة ٠.٠ واسترجاع ١.٠ (أعلن الجميع مرضى) تحصدان حسابياً ٠.٥ — درجة «نصفيحة» لسلوك أحمق. أما التوافقي فيُقترب من أصغر قيمتيه دائماً:

$$\mathrm{HM}(0,\ 1) = \frac{2 \times 0 \times 1}{0 + 1} = 0 \qquad \mathrm{AM}(0,\ 1) = 0.5$$

> **تعمّق**
>
> المتوسط التوافقي هو مقلوب متوسط المقلوبات، فيهيمن عليه أصغر عنصر — خصيصة مطلوبة في المقاييس المشروطة: نموذج لا يستحق أكثر من أسوأ جانبَيْه. ولهذا يمتد الفكر إلى `F_beta` العام الذي يمنح الاسترجاع وزناً أكبر عندما تكون بيتا أكبر من ١ (وكما سترى في الفصل ٧٩ تُستخدم مشتقاته في الترتيب والتوصية).

وللخانات المتبقية مقياسها الذي ينظر إلى الجانب السلبي:

$$\mathrm{Specificity} = \frac{TN}{TN + FP} \qquad \mathrm{Sensitivity} = \frac{TP}{TP + FN} \qquad \mathrm{BA} = \frac{\mathrm{Sensitivity} + \mathrm{Specificity}}{2}$$

الخصوصية (specificity) تسأل: من السليمين كم أعلنّا سليماً؟ والدقة المتوازنة (balanced_accuracy) متوسط الحساسية والخصوصية — تعالج خداع الدقة العادية بإعطاء النادرة نصف الوزن مهما كان انتشارها. ولنحسم ذلك على عيادة افتراضية في يوم واحد:

```python
# يوم في العيادة: الحساب اليدوي ثم التحقق بـ sklearn
import numpy as np
from sklearn.metrics import (
    precision_score, recall_score, f1_score, accuracy_score,
    balanced_accuracy_score,
)

# عيادة في يوم واحد: 200 مريض منهم 50 مصاباً
TP, FN = 40, 10
TN, FP = 135, 15

precision_manual = TP / (TP + FP)          # 40 / 55
recall_manual    = TP / (TP + FN)          # 40 / 50
f1_manual        = 2 * TP / (2 * TP + FP + FN)
spec_manual      = TN / (TN + FP)          # الخصوصية
ba_manual        = (recall_manual + spec_manual) / 2

# إعادة بناء اليوم نفسه كمتجهات والتحقق من sklearn
y_true = np.array([1] * 50 + [0] * 150)
y_pred = np.array([1] * 40 + [0] * 10 + [1] * 15 + [0] * 135)

print(f"الدقة precision:  يدوياً {precision_manual:.4f} | sklearn
{precision_score(y_true, y_pred):.4f}")
print(f"الاسترجاع recall: يدوياً {recall_manual:.4f} | sklearn {recall_score(y_true,
y_pred):.4f}")
print(f"مقياس F1:         يدوياً {f1_manual:.4f} | sklearn {f1_score(y_true,
y_pred):.4f}")
print(f"يدوياً          الخصوصية: {spec_manual:.4f} | الدقة الخام {accuracy_score(y_true,
y_pred):.4f}")
print(f"يدوياً  الدقة المتوازنة: {ba_manual:.4f} | sklearn
{balanced_accuracy_score(y_true, y_pred):.4f}")
```

كل الأعمدة متطابقة — وهذا التطابق يسمح بالتدقيق في تقرير زميلك سطراً سطراً. ولاحظ أن الدقة الخام ٠.٨٧٥ تبدو محترمة بينما المتوازنة ٠.٨٥٠ تفضح أن الاسترجاع (٠.٨٠٠) أدنى من الخصوصية (٠.٩٠٠).

٩.٤ MCC: ميزان العدالة عند الاختلال

ماذا لو أردت مقياساً واحداً لا يمكن خداعه مهما كانت نسبة الانتشار؟ هنا يدخل **معامل ارتباط ماثيوز (Matthews Correlation Coefficient - MCC)**، وهو في جوهره معامل ارتباط بيرسون بين الحقيقة والتوقع: ١.٠ تطابق كامل، و ٠.٠ تعادل تخمين عشوائي، والسالب إصابة عكس الحقيقة:

$$\mathrm{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

المقام يحتوي الأربعة الحدود مجتمعة، لذلك لا يستطيع النموذج رفع MCC بالاستثمار في خانة واحدة. والتجربة الحاسمة: مصنّفان يحققان **الدقة نفسها تماماً ٩٩%** على ١٠٬٠٠٠ مريض منهم ١٠٠ مصاب — الأول يقول «لا مرض» دائماً، والثاني يكتشف نصف المصابين مقابل ٥٠ إنذاراً كاذباً:

python دقة متطابقة ٩٩٪ و MCC متباعدان جذرياً

```python
import numpy as np
from sklearn.metrics import matthews_corrcoef, balanced_accuracy_score, f1_score

y_true = np.array([1] * 100 + [0] * 9900)

# المصنف أ: «لا مرض دائماً» – دقة 99% بلا أي كشف
y_pred_a = np.zeros(10000, dtype=int)

# المصنف ب: يكتشف 50 من 100 مصاب مقابل 50 إنذاراً كاذباً
y_pred_b = np.array([1] * 50 + [0] * 50 + [1] * 50 + [0] * 9850)

for name, y_pred in [("أ (لا مرض دائماً)", y_pred_a), ("ب (متحفظ حقيقي)", y_pred_b)]:
    acc = (y_true == y_pred).mean()
    mcc = matthews_corrcoef(y_true, y_pred)
    ba  = balanced_accuracy_score(y_true, y_pred)
    f1b = f1_score(y_true, y_pred, zero_division=0)
    print(f"{name} | الدقة: {acc:.2%} | MCC: {mcc:.4f} | متوازنة: {ba:.4f} | F1:
{f1b:.4f}")
```

اقرأ السطرين بتمعن: الدقة متطابقة ٩٩.٠٠٪، ومع ذلك يكشف MCC الحقيقة بالكامل — الصفر للساذج (إشارة «عشوائي تماماً») مقابل ٠.٤٩٤٩ للمتحفظ الحقيقي. بل إن F1 يفضح الخداع فوراً (٠.٠ مقابل ٠.٥)، والمتوازن يوقف الساذج عند مستوى التخمين ٠.٥٠٠٠ مقابل ٠.٧٤٧٥. لكن MCC وحده يجمع الخانات الأربعة في معادلة ارتباط واحدة، ولهذا يعتبره كثير من الباحثين المقياس الأصدق للبيانات غير المستوية في المقالات العلمية وعند مقارنة المصنفات.

سياق

الاسم يخص عالم البيولوجيا الحاسوبية برايان ماثيوز الذي قدّمه عام ١٩٧٥ لمقارنة بنية البروتينات، وهو في صميمه معامل فاي الترابطي المعروف في الإحصاء منذ الثلاثينيات. الفكرة العميقة: التصنيف الثنائي جدول اقتران ٢×٢، وأي جدول اقتران يمكن قياس قوة ارتباطه — والتصنيف الجيد ارتباط قوي بين الحكم والحقيقة.

متى تختار ماذا؟ الدقة للفئات المتقاربة وتكاليف متساوية، و F1 حين يكون الإيجابي هو المهم رغم الاختلال، والمتوازنة لمتعدد الفئات غير المستوي، و MCC للتقييم العلمي على البيانات شديدة الاختلال — أربعة مقاييس أربع زوايا على المصفوفة نفسها.

٩.٥ من فئتين إلى فئات كثيرة: معامل average وتقرير التصنيف

كل المعادلات أعلاه كُتبت لفئتين، لكن مسائل ثلاثية الفئات وأكثر تغمر الواقع. كيف يعممها sklearn على الفئات المتعددة؟ عبر المعامل `average` الذي يجيب: «كيف أجمع درجات الفئات في رقم واحد؟»:

القيمة	آلية الجمع	متى تستعملها؟	حذارِ
`binary`	المقياس للفئة الإيجابية وحدها	الثنائي فقط	يرفض أكثر من فئتين
`macro`	متوسط حسابي متساوٍ للفئات	تهمك النادرة بقدر الكبيرة	فئة نادرة واحدة تسحب الدرجة
`micro`	تجميع TP/FP/FN عالمياً ثم الحساب	تهمك القرارات الفردية	يساوي الدقة في الأحادي تماماً
`weighted`	متوسط موزون بدعم كل فئة	تريد القرب من الأداء الكلي	قد يخفي ضعف النادرة مجدداً
`None`	درجة لكل فئة على حدة	التشخيص الفصيل قبل الجمع	لا تلمحها المراجعات السريعة

جرّب الخيارات على مسألة ثلاثية الفئات مختلة الاستواء (٨٠٪ / ١٥٪ / ٥٪):

```python
# يغير الرقم: أي صوت تسمع؟ average
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, accuracy_score

X, y = make_classification(
    n_samples=3000, n_features=10, n_informative=5, n_classes=3,
    weights=[0.80, 0.15, 0.05], random_state=42
)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
model.fit(X_tr, y_tr)
y_pred = model.predict(X_te)

for avg in ["macro", "micro", "weighted"]:
    print(f"F1 (average={avg:9s}): {f1_score(y_te, y_pred, average=avg):.4f}")
print(f"مطابقة micro مع الدقة الخام: {f1_score(y_te, y_pred, average='micro') ==
accuracy_score(y_te, y_pred)}")
```

ثلاثة أرقام لنفس النموذج: micro يطابق الدقة الخام حرفياً (خاصية رياضية للأحادي التسمية)، macrog ينزاح نزولاً لأن النادرة تدفع بمقياسها المنخفض بنفس وزن فئة الـ ٨٠%، وweightedg يجلس بينهما. ومن سؤال الجمع نفسه ينبع الخطأ الكلاسيكي:

> **خطأ شائع**
>
> استدعاء `f1_score(y_te, y_pred)` على ثلاث فئات بلا `average` يفجّر `ValueError` فوراً، لأن الافتراضي `average='binary'` لا يعمل إلا على فئتين. وإن أردت درجة الفئة النادرة وحدها فاستخدم `average=None` واقرأ الصف المقابل لها.

والأداة الأشمل هنا `classification_report` التي تلف كل ما سبق في جدول واحد:

قراءة تقرير التصنيف سطراً سطراً python

```python
from sklearn.metrics import classification_report,
precision_recall_fscore_support

print(classification_report(
    y_te, y_pred,
    target_names=["مصاب" ,"شبه مصاب" ,"سليم"],
    digits=4
))

# الأداة الخام خلف التقرير: درجات لكل فئة بلا تجميع
prec, rec, f1s, support = precision_recall_fscore_support(y_te, y_pred)
print("مصفوفة الدقة لكل فئة:", prec.round(4))
```

اقرأ التقرير بهذا الترتيب: عمود support أولاً (سياق كل رقم)، ثم صف الفئة النادرة (استرجاعها الأخطر غالباً)، ثم سطرا macro avg و weighted avg. أما المسائل متعددة التسميات (multilabel) حيث تنتمي العينة لعدة فئات معاً، فتعمل هذه الدوال نفسها على أهداف ثنائية الشكل لكن قراءتها تحتاج أدواتها الخاصة — تفصيلها الكامل في الفصل ٧٩.

٩.٦ فخ pos_label وفخ كلفة الأخطاء: أيهما أغلى في مجالك؟

بقيت عقبة عملية تُوقع مئات المبرمجين يومياً. كل الدوال الثنائية تعتمد على مفهوم «الفئة الإيجابية» عبر `pos_label` الذي قيمته الافتراضية 1. طالما فئاتك أرقام [٠, ١] لن تشعر به؛ لكن ماذا لو كانت نصوصاً؟

فخ pos_label: نص مقابل رقم python

```python
import numpy as np
from sklearn.metrics import precision_score

y_true = np.array(["سليم" ,"مريض" ,"سليم" ,"مريض" ,"سليم" ,"مريض"])
y_pred = np.array(["سليم" ,"سليم" ,"سليم" ,"مريض" ,"مريض" ,"مريض"])

try:
    precision_score(y_true, y_pred)          # الافتراضي pos_label=1 يبحث عن
except ValueError as err:
    print(f"ValueError: {err}")

# الحل: حدد الفئة الإيجابية بنصها
p = precision_score(y_true, y_pred, pos_label="مريض")
print(f"بعد التحديد الصحيح pos_label='الدقة :'مريض = {p:.4f}")
```

الرسالة واضحة: `pos_label=1 is not a valid label`. المصنّف لم يرَ رقماً ١ في حياته؛ رأى نصوصاً. والحل بحد واحد، أو توحيد الترميز بالأرقام مبكراً في المشاريع الكبيرة. والقاعدة: إن ظهر `ValueError` يحتوي كلمة `pos_label` فالمشكلة عدم إخبارك المقياس أي فئة تعدّها إيجابية.

> **نصيحة**
>
> عند استكشاف بيانات جديدة، نفّذ `np.unique(y_true, return_counts=True)` ثم مرر `labels=` الصريح إلى `confusion_matrix`. ثلاثون ثانية من الوعي بالترتيب توفر عليك ساعة من الالتباس.

والآن السؤال الاستراتيجي الذي يلي كل هذا الحساب: عرفنا كيف نقيس FN و FP — لكن أيهما أغلى؟ الإجابة مجالية لا رياضية:

المجال	الخطأ الأغلى	لماذا؟	نزاحة العتبة
تشخيص السرطان	FN (تفويت مرض)	العلاج المتأخر قاتل	خفض ← رفع الحساسية
فلترة البريد المزعج	FP (حجب بريد سليم)	رسالة عمل مفقودة كارثة	رفع ← رفع الخصوصية
إنذار حريق	FN (حريق بلا إنذار)	الأرواح قبل الأعصاب	خفض بشدة
حظر بطاقة بنكية	FP (إيقاف عميل سليم)	ثقة العميل تُستنزف	توازن بعناية

لاحظ العمود الأخير: التكلفة تترجم مباشرة إلى **اتجاه تحريك عتبة القرار**. عتبة ٠.٥ ليست محايدة أبداً؛ إنها افتراض ضمني بأن FN و FP متساويا التكلفة — افتراض يخالف الطب والأمن والمال. وهذه فكرة الفصل السابع والسبعين: نحرك العتبة عمداً لنرى المقياسين يتبادلان، ثم نتعلم أدوات sklearn التي تضبطها تلقائياً وفق دالة تكلفة تعرّفها أنت.

> **في الواقع**
>
> في أنظمة التشخيص الطبي يُضبط المحتوى التشغيلي غالباً على حساسية تتجاوز ٠.٩٥ مقابل خصوصية أقل — «الأمانة في الإنذار أولى من رشاقته» — ثم يُترك الفحص التأكيدي الأغلى ليفصل الإيجابيات الكاذبة. التسلسل «فلتر رخيص حساس ثم فلتر غالي دقيق» هو النمط الهندسي الأشهر في الأنظمة الميدانية.

تمارين الفصل الخامس والسبعين

- **ملاحظة**

التمرين ٩.١

نظام فلترة بريد واجه ١,٠٠٠ رسالة من بينها ١٠٠ مزعجة. علّم النظام ١٢٠ رسالة كمزعجة، وكان ٨٠ منها مزعجاً فعلاً. احسب يدوياً: الدقة والاسترجاع و F1 والخصوصية و MCC، ثم تحقق من كل ناتج بـ sklearn بعد بناء `y_true` و `y_pred`.

التمرين ٩.٢

على مصفوفة الخلط [[TN=٨٦٠, FP=٤٠], [FN=٢٠, TP=٨٠]]: احسب الدقة المتوازنة يدوياً، ثم اشرح بجملتين لماذا يمنح macro الفئة النادرة وزناً أكبر من weighted، مصححاً شرحك بتجربة على بيانات القسم ٩.٥.

التمرين ٩.٣

على بيانات سرطان الثدي مع تعريف القسم ٩.٢ (الخبيث = إيجابي): حرك العتبة عبر ٠.١ حتى ٠.٩ واطبع جدولاً للدقة والاسترجاع. هل تلاحظ التبادل الصعودي؟ ما العتبة التي تحقق استرجاعاً لا يقل عن ٠.٩٨؟

- **ملاحظة**

python حل التمرين ٩.١: مقاييس فلترة البريد يدوياً وبمكتبة

```python
import numpy as np
from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    matthews_corrcoef, confusion_matrix,
)

# 80 صحيحاً من 120 المعلَّمة ، و 20 مزعجاً تفوّتا ، و 860 سليماً طُابق
TP, FP, FN, TN = 80, 40, 20, 860

y_true = np.array([1] * 100 + [0] * 900)
y_pred = np.array([1] * 80 + [0] * 20 + [1] * 40 + [0] * 860)

mcc_manual = (TP * TN - FP * FN) / np.sqrt(
    (TP + FP) * (TP + FN) * (TN + FP) * (TN + FN)
)

print(f"الدقة precision: {TP / (TP + FP):.4f} | {precision_score(y_true,
y_pred):.4f}")
print(f"الاسترجاع recall: {TP / (TP + FN):.4f} | {recall_score(y_true,
y_pred):.4f}")
print(f"مقياس F1:        {2 * TP / (2 * TP + FP + FN):.4f} | {f1_score(y_true,
y_pred):.4f}")
print(f"الخصوصية:        {TN / (TN + FP):.4f}")
print(f"MCC:             {mcc_manual:.4f} | {matthews_corrcoef(y_true,
y_pred):.4f}")
```

النتيجة قصة فلترة متوازنة: استرجاع ٠.٨٠٠٠ مقابل دقة ٠.٦٦٦٧ (من كل ١٠٠ إعلان مزعجة ٣٣ بريداً سليماً دُفن خطأً)، و MCC موجب بقوة ٠.٦٩٧٥ — النظام يرتبط بالحقيقة ارتباطاً حقيقياً حتى مع الاختلال ١:٩.

> **ملاحظة**

حل التمرين ٩.٣: تحريك العتبة على بيانات الثدي python

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_score, recall_score

X, y = load_breast_cancer(return_X_y=True)
y_sick = (y == 0).astype(int)          # الخبيث إيجابي كما في القسم ٩.٢
X_tr, X_te, ys_tr, ys_te = train_test_split(
    X, y_sick, test_size=0.25, random_state=42, stratify=y_sick
)
bc_model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
bc_model.fit(X_tr, ys_tr)
probabilities = bc_model.predict_proba(X_te)[:, 1]

print("العتبة | precision | الاسترجاع recall")
for t in [0.1, 0.2, 0.3, 0.5, 0.7, 0.9]:
    y_hat = (probabilities >= t).astype(int)
    p = precision_score(ys_te, y_hat, zero_division=0)
    r = recall_score(ys_te, y_hat)
    print(f"{t:6.1f} | {p:14.4f} | {r:14.4f}")
```

الجدول يجسد قانون التبادل الذي مهّدنا له في القسم ٩.٦: خفض العتبة يرفع الاسترجاع ويخفض الدقة، والعكس عند رفعها. ومن الطاولة تُستخرج العتبة التي تحقق استرجاعاً لا يقل عن ٠.٩٨ — لكن البحث اليدوي متعب وإدخاله للإنتاج وعرة. الفصل ٧٧ يمنحك `FixedThresholdClassifier` لتثبيت العتبة التي اخترتها و `TunedThresholdClassifierCV` لاختيارها تلقائياً.

> **ملاحظة**

٩.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تقييم مصنّف غير مستوي بالدقة وحدها	دقة ٩٩% لنموذج لا يكتشف شيئاً وقرار نشر كارثي	احسب confusion*matrix* *أولاً ثم* *balanced*accuracy و MCC معاً
استدعاء score_f1 على أكثر من فئتين دون average	فوران ValueError لأن الافتراضي binary يخص الثنائي	مرر 'average='macro أو 'micro' أو 'weighted' حسب هدفك
إهمال pos_label مع تسميات نصية	فوران is not a valid l=pos_label label أو قياس الفئة الخطأ	حدد pos_label='اسم الفئة' أو وحّد الترميز بالأرقام مبكراً
قراءة confusion_matrix دون تحقق من الترتيب	قلب TN مع TP واعتبار الخصوصية استرجاعاً	مرر labels=[إيجابي، سلبي] صراحة وتحقق من ravel قبل الاستنتاج

٩.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مصفوفة الخلط	جدول ٢×٢ (أو C×C) يفكك حكم النموذج مقابل الحقيقة خانةً خانة	Confusion matrix
الدقة والاسترجاع	ثقة الإنذارات مقابل شمول الاكتشاف، وكلاهما من الخانات الأربع	Precision / Recall
المتوسط التوافقي	متوسط تهيمن عليه القيم الصغرى، أساس مقياس F1	Harmonic mean
معامل ارتباط ماثيوز	ارتباط بين الحقيقة والتوقع يوازن الخانات الأربع ويتحصن ضد الاختلال	Matthews Correlation Coefficient (MCC)
الحساسية والخصوصية	الاسترجاع على المرضى مقابل الاسترجاع على الأصحاء	Sensitivity / Specificity

الجزء ٩ · التقييم والاختيار

الفصل ٧٦ — منحنيات القرار و ROC و PR

det_curve و auc و precision_recall_curve و roc_curve

الوحدة **scikit-learn ١.٩** الحجم **١٠٤٣** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تشرح كيف تحوّل العتبة درجةَ احتمالية إلى قرار، وتقرأ نقطة واحدة من منحنى ROC كعملية كاملة
- أن تحسب TPR و FPR و AUC يدوياً بالعدّ الاحتمالي للأزواج وتطابقها مع roc_curve و roc_auc_score
- أن تقيّم مصنفاً متعدد الفئات بـ roc_auc_score عبر multi_class و average وباستخدام predict_proba
- أن تفسر لماذا يتفوق منحنى PR على ROC عند الاختلال الشديد ولماذا خط أساسه هو معدل الانتشار
- أن تميز average_precision_score عن مساحة AUC-PR المحسوبة هندسياً وتختار المناسب

٩.١ من الاحتمال إلى القرار: العتبة هي التي تحكم

كما رأينا في الفصل ٣٥، لا يصدر التصنيف اللوجيستي أحكاماً مباشرة؛ بل يخرج بدرجة احتمالية بين ٠ و ١، ثم تأتي خطوة صامتة تقرر: «أكبر من أو يساوي ٠.٥؟ إذاً إيجابي». هذه العتبة ٠.٥ ليست قانوناً طبيعياً؛ إنها اتفاق أعمى داخل `predict()`. جرّب أن تخفضها قليلاً وسترى النموذج «يتغير» دون إعادة تدريب واحدة:

خفض العتبة يقلب القرارات بلا إعادة تدريب python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
y_sick = (y == 0).astype(int)          # 1 = خبيث (إيجابي) كما اتفقنا في الفصل ٧٥
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y_sick, test_size=0.25, random_state=42, stratify=y_sick
)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
model.fit(X_tr, y_tr)

prob = model.predict_proba(X_te)[:, 1]      # درجات الاحتمال الخام
d_05 = (prob >= 0.5)                        # قرار العتبة الافتراضية
d_03 = (prob >= 0.3)                        # قرار عتبة أخفض

print(f"إعلانات خبائة عند العتبة 0.5: {d_05.sum()} من {len(prob)}")
print(f"إعلانات خبائة عند العتبة 0.3: {d_03.sum()} من {len(prob)}")
print(f"قرارات انقلبت بينهما: {(d_05 != d_03).sum()}")
```

عشرون قراراً انقلب بمجرد تحريك رقم واحد — والنموذج نفسه لم يتغير قط. هنا يستقيم السؤال الذي يفتح هذا الفصل: كيف نقيم النموذج **كله دفعة واحدة** بدلاً من تقييمه عند عتبة وحيدة اخترت اعتباطاً؟ الجواب: نمر على كل العتبات الممكنة ونرسم الناتج — هذا هو منحنى القرار.

> **مفتاح**
>
> كل نقطة على منحنى ROC أو PR هي «مصنّف كامل» عند عتبة معينة: نموذج واحد يولّد عائلة من المصنفات مرتبة على المنحنى. منحنيات القرار هي الخريطة التي تختار منها موضع تشغيلك وفق تكلفة أخطائك — كما ستفعل فعلياً بالأدوات الجاهزة في الفصل ٧٧.

٩.٢ منحنى ROC: TPR مقابل FPR عبر كل العتبات

منحنى خصائص التشغيل للمستقبل (Receiver Operating Characteristic - ROC) وُلد في رادارات الحرب العالمية الثانية لتفريق طائرة عدو عن طائر، وفكرته بسيطة: لكل عتبة ممكنة احسب مقياسين متناظرين:

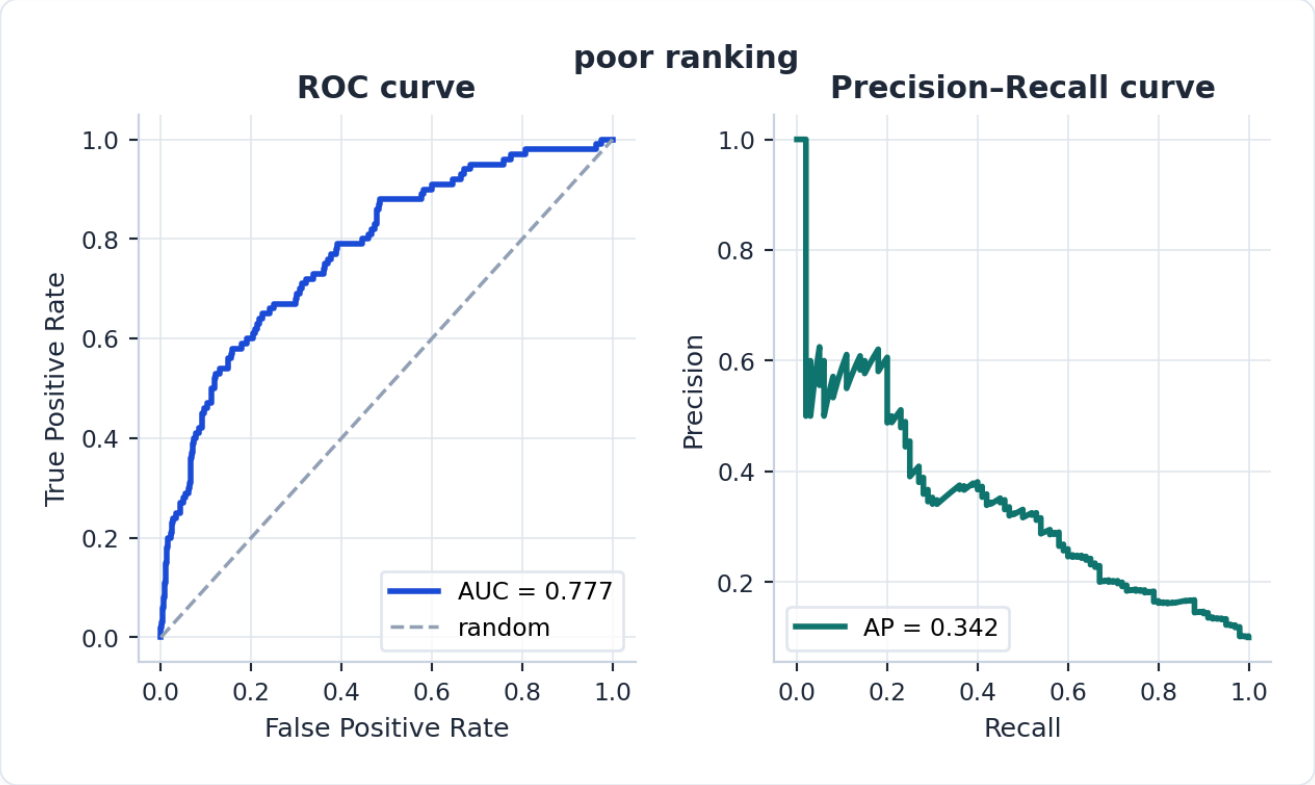
$$\text{TPR} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

معدل الإصابة الحقيقي TPR هو الاسترجاع نفسه الذي تعرفنا عليه في الفصل ٧٥ (حساسية المريض)، بينما معدل الإنذار الكاذب FPR هو صورة الخصوصية المقلوبة (١ − specificity: كم سليماً أزعجنا). عتبة عالية → نقطة في الزاوية السفلى اليسرى (قليل من كل شيء)، وعتبة صفر → الزاوية العليا اليمنى (كل شيء إيجابي). فلنبنِ نقاطاً من هذا النوع بأيدينا على ست عينات فقط، لنجرد الآلية حتى النخاع:

نقاط ROC يدوياً على ست عينات python

```python
import numpy as np
from sklearn.metrics import roc_curve

y_true = np.array([1, 1, 0, 1, 0, 0])
scores = np.array([0.9, 0.6, 0.55, 0.4, 0.3, 0.2])

# نقطة يدوية عند العتبة 0.5: من تجاوزها؟
y_hat = (scores >= 0.5).astype(int)
tp = int(((y_true == 1) & (y_hat == 1)).sum())
fp = int(((y_true == 0) & (y_hat == 1)).sum())
fn = int(((y_true == 1) & (y_hat == 0)).sum())
tn = int(((y_true == 0) & (y_hat == 0)).sum())
print(f"عند العتبة 0.5: TPR = {tp}/{tp + fn} = {tp / (tp + fn):.4f} ، FPR = {fp}/{fp
+ tn} = {fp / (fp + tn):.4f}")

# المنحنى الكامل كما يحسبه sklearn
fpr, tpr, thresholds = roc_curve(y_true, scores)
print("العتبات التي فحصها:", thresholds)
print("FPR:", fpr.round(4))
print("TPR:", tpr.round(4))
```

نقطة العتبة ٠.٥ اليدوية تطابق إحدى نقاط المنحنى تماماً. ولاحظ تفصيلين يقع فيهما كثيرون: أول عتبة في مصفوفة sklearn هي `inf` (حالة «لا أحد إيجابي» أي النقطة (٠, ٠))، وثاني العتبات ليست كل القيم بل كل القيم **المفيدة** — العتبات التي تعطي نقطة مكررة تُهمل كما حدث للقيمة ٠.٥ نفسها لأنها تعطي النقطة ذاتها التي تعطيها ٠.٦.



الشكل ١٦٧منحنيات ROC و PR لنماذج متعددة على نفس البيانات — القطر عشوائي في ROC بينما خط الأساس أفقي في PR

في الشكل تقرأ ثلاثة منحنيات لنماذج متفاوتة الجودة: كلما التف المنحنى نحو الزاوية العلوية اليسرى كان الترتيب أفضل. القطر الذي يجري بين الزاويتين هو خط أساس التخمين العشوائي — النموذج الذي يقع عليه لا يعرف شيئاً عن البيانات. ومنهج `roc_curve` يعيد الثلاثيات (fpr, tpr, thresholds) جاهزة، و `RocCurveDisplay` من `sklearn.metrics` يرسمها إن أردت، لكن الرسم في هذا الكتاب حكر على الأشكال المسجلة كما أسلفنا.

٩.٣ AUC: الاحتمال الكامن خلف الرقم

رقم واحد يختصر المنحنى كله: **المساحة تحت المنحنى (Area Under the Curve - AUC)**. لكن التفسير الأجمل ليس هندسياً بل احتمالياً وهو الدقيق تماماً:

$$\text{AUC} = P(\text{score}_{\text{positive}} > \text{score}_{\text{negative}})$$

أي: احتمال أن نموذجنا يمنح عينةً إيجابيةً عشوائية درجةً أعلى من عينة سلبية عشوائية. هذا التفسير يحوّل AUC إلى مسألة عدّ أزواج يمكن حلها باليد. على بياناتنا الست: الإيجابيات بدرجات (٠.٩، ٠.٦، ٠.٤) والسوالب بدرجات (٠.٥٥، ٠.٣، ٠.٢) — لكل زوج (إيجابي، سالب) نسأل: من الأعلى؟

كـاحتمـال: عنـد الأزواج الـتسعة يـدويـاً AUC python

```python
import numpy as np
from sklearn.metrics import roc_auc_score

y_true = np.array([1, 1, 0, 1, 0, 0])
scores = np.array([0.9, 0.6, 0.55, 0.4, 0.3, 0.2])

pos = scores[y_true == 1]
neg = scores[y_true == 0]
wins = sum(1 for p in pos for q in neg if p > q)
auc_manual = wins / (len(pos) * len(neg))

print(f"الأزواج الرابحة: {wins} من {len(pos) * len(neg)} أزواج")
print(f"AUC بالعد اليدوي: {auc_manual:.4f}")
print(f"AUC من roc_auc_score: {roc_auc_score(y_true, scores):.4f}")
```

ستة أزواج رابحة من تسعة (٠.٩ و ٠.٦ تفوقان السوالب الثلاثة كلها، و ٠.٤ يخسر أمام ٠.٥٥ فقط)، فكانت المساحة ٦/٩ ≈ ٠.٨٨٨٩، وتطابقت حرفياً مع ناتج الدالة. عملياً يحسب sklearn المساحة بتكامل شبه منحرف على النقاط بتكلفة O(n log n) يحكمها الفرز، فلا تخشَ على ملايين العينات.

تعمّق

ماذا لو تساوت درجة إيجابي وسالب؟ المكسب ينصب نصفه: الزوج يُحسب ٠.٥ لا ١.٠، ولهذا يُبنى تعريف roc_curve بترتيب الدرجات ومعالجة المكررات كسلالم أفقية-رأسية. لهذا أيضاً لا يجوز «تنعيم» الدرجات بتحويل خطي؛ فالـ AUC يهتم بالترتيب وحده لا بقيمة الدرجات — نموذجٌ تدرجاته (٠.١, ٠.٢, ٠.٣) ونموذجٌ تدرجاته (١٠٠, ٢٠٠, ٣٠٠) لهما AUC متطابق إذا تطابقت ترتيبات الإجابات.

وماذا عن أكثر من فئتين؟ دالة `roc_auc_score` تعمم بالمعاملين `multi_class` و `average`، لكنها تشترط احتمالات حقيقية من `predict_proba` — لا درجات `decision_function`:

متعدد الفئات على بيانات النبيذ AUC python

```python
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
model.fit(X_tr, y_tr)
proba = model.predict_proba(X_te)

print(f"AUC متعدد الفئات (ovr, macro):  {roc_auc_score(y_te, proba,
multi_class='ovr', average='macro'):.4f}")
print(f"AUC متعدد الفئات (ovr, micro):  {roc_auc_score(y_te, proba,
multi_class='ovr', average='micro'):.4f}")
print(f"AUC متعدد الفئات (ovo, macro):  {roc_auc_score(y_te, proba,
multi_class='ovo', average='macro'):.4f}")

# تمرير درجات decision_function الخطأ الشائع: بدل الاحتمالات
try:
    roc_auc_score(y_te, model.decision_function(X_te), multi_class='ovr')
except ValueError as err:
    print("ValueError:", str(err)[:100])
```

ثلاث درجات ثلاثة تعميمات: `ovr` (واحد-مقابل-البقية) يقيّم كل فئة ضد البقية فيجمع بـ `average` كما تعلمت في الفصل ٧٥، بينما `ovo` (واحد-مقابل-واحد) يقيس كل زوج فئات على حدة. أما الرسالة الأخيرة فهي درس مزدوج: مقام الدرجة في `decision_function` ليس احتمالاً (أعمدته لا تجمع إلى ١)، والمقياس يحتاج صيغة احتمالية — ولهذا يفشل، وكما سترى في الفصل ٧٧ فإن المعايرة تعالج هذا النوع من الالتباس جذرياً.

٩.٤ منحنى PR: مرآة الصدق عند الاختلال

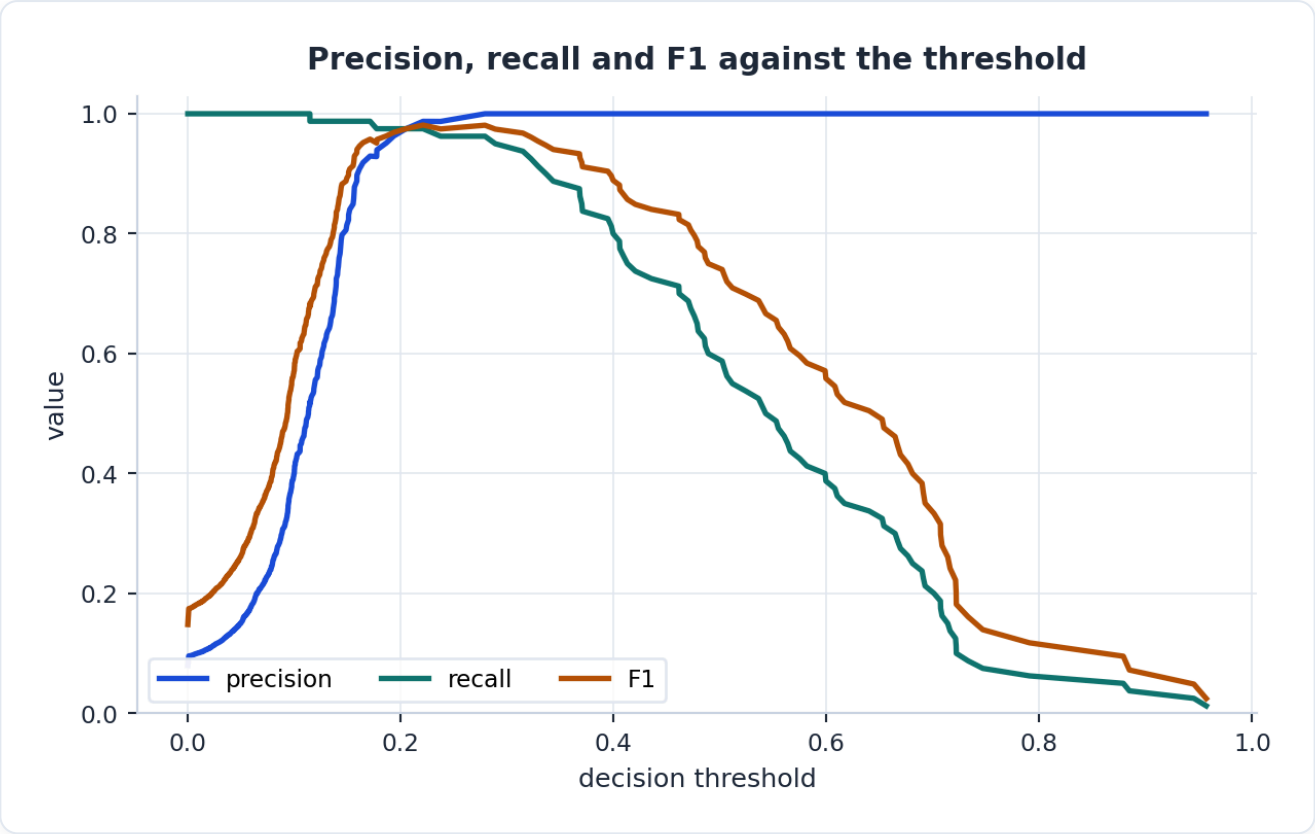
منحنى **الدقة-الاسترجاع (Precision-Recall - PR)** يمر على العتبات نفسها لكنه يرسم زوجاً مختلفاً: الدقة (عمودياً) مقابل الاسترجاع (أفقياً). الفرق عن ROC يبدو شكلياً، لكنه جوهري: ROC تخلط الفئة السلبية الهائلة في FPR وتظل «تضحك» حتى مع اختلال مدقع، بينما PR لا ترى السوالب إلا من باب أذى تسببه للدقة — أي من باب الحقيقة.

والتجربة القاطعة: مسألة انتشار ١% مع نموذجين حقيقيين. لاحظ كيف يقول AUC «كلاهما شبه مثالي» ثم يقلب متوسط الدقة AP الترتيب رأساً على عقب:

تجربة قاطعة: AUC يشيد والـ AP يفضح python

```python
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    roc_auc_score, average_precision_score, precision_recall_curve,
)

X, y = make_classification(
    n_samples=20000, n_features=12, n_informative=6, n_redundant=2,
    weights=[0.99, 0.01], class_sep=1.5, flip_y=0.0, random_state=42
)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
print(f"نسبة الإيجابيات في الاختبار: {y_te.mean():.2%}")

log = make_pipeline(StandardScaler(),
LogisticRegression(max_iter=5000)).fit(X_tr, y_tr)
s_log = log.predict_proba(X_te)[:, 1]
hgb = HistGradientBoostingClassifier(max_iter=120, random_state=42).fit(X_tr,
y_tr)
s_hgb = hgb.predict_proba(X_te)[:, 1]

print(f"اللوجيستي: AUC = {roc_auc_score(y_te, s_log):.4f} | AP =
{average_precision_score(y_te, s_log):.4f}")
print(f"التدرّجي:  AUC = {roc_auc_score(y_te, s_hgb):.4f} | AP =
{average_precision_score(y_te, s_hgb):.4f}")

# حقيقة التشغيل: الدقة الفعلية عند مستويات استرجاع نحتاجها
import numpy as np
from sklearn.metrics import precision_recall_curve

prec, rec, _ = precision_recall_curve(y_te, s_log)
order = np.argsort(rec)                  # الاسترجاع تنازلي: نرتّبه صعودياً للاستيفاء
for r in [0.5, 0.9]:
    p_at_r = float(np.interp(r, rec[order], prec[order]))
    print(f"اللوجيستي عند استرجاع {r:.0%}: الدقة الفعلية = {p_at_r:.4f}")
```

هذه الأرقام مسرحية كاملة: AUC يمنح اللوجيستي ٠.٩٧٩٨ والتدرّجي ٠.٩٦٠٢ — رقمان لو وصلت بهما إلى اجتماع الإدارة لصُدّق على الإطلاق فوراً. لكن AP يفضح: ٠.٧٣١٤ مقابل ٠.٧٨٢١، أي أن **الترتيب نفسه انقلب** (اللوجيستي يفوز في AUC ويخسر في AP). والأقسى سطر الدقة الفعلية: عند استرجاع ٠.٩ أي اكتشاف ٩ من كل ١٠ إيجابيات، تكون الدقة ٠.١٨٧٥ فقط — أي أن من كل ١٠٠ جرس إنذار حوالي ٨١ جرساً كاذباً. هذا هو مستوى الأداء الحقيقي الذي يخفيه AUC على هذه البيانات، ومن يحتاج النموذج للاختلال فليطّلع على منحنى PR أولاً.



الشكل ١٦٨منحنى PR: التبادل بين الدقة والاسترجاع عبر العتبات وخط الأساس الأفقي عند معدل الانتشار

في الشكل تقرأ التبادل الأيقوني: على اليسار الدقة والاسترجاع كدالتين في العتبة يصعد أحدهما وينخفض الآخر، وعلى اليمين المنحنى الناتج الذي يتدلى طرفه الأيسر كلما اقتربنا من الاسترجاع الكامل — لأن رفع الاسترجاع يجبرك على إعلان عينات أكثر شكّاً فتنهار الدقة على الاختلال الشديد.

والآن أدق فخ في هذا الفصل، فاختبر حدسك قبل أن تقرأ الصندوق:

خطأ شائع

من ينتقل من ROC إلى PR يتوقع تلقائياً «قطراً عشوائياً» كخط أساس — وهذا خطأ شائع يقوض آلاف التقارير. خط الأساس في ROC قطر فعلاً بمساحة ٠.٥ مهما كانت نسبة الانتشار، لأن FPR يُطبَّع بالسوالب. لكن خط الأساس في PR **خط أفقي على ارتفاع معدل الانتشار** π: مصنّف عشوائي يحقق AP تساوي π (١٪ في مثالنا!). لذلك لا تقارن درجة AP بين مسألتين مختلفتي الانتشار ولا مع ٠.٥ إطلاقاً: AP = ٠.٢ على بيانات ١٪ عملٌ عظيم، و AP = ٠.٢ على بيانات ٤٠٪ يبدو باهتاً.

يمكنك التحقق من خطي الأساس بحجر تركيب واحد — مصنّف الأعراض الساذج `DummyClassifier`:

خطأ الأساس بالتجربة: 0.5 في ROC و π في PR python

```python
import numpy as np
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

dummy = DummyClassifier(strategy="prior").fit(X_tr, y_tr)
constant_scores = np.full(len(y_te), dummy.predict_proba(X_te)[0, 1])

print(f"معدل الانتشار: {y_te.mean():.4f}")
print(f"AUC للدرجات الثابتة: {roc_auc_score(y_te, constant_scores):.4f} (القطر)")
print(f"AP للدرجات الثابتة:  {average_precision_score(y_te, constant_scores):.4f} (خط أفقي)")
```

درجاتٌ ثابتة بلا أي معلومة: AUC أصاب ٠.٥ كما ينص القطر، و AP أصاب معدل الانتشار كما ينص الخط الأفقي. اثنان من أسطر التحقق يحميانك من أخطر سوء فهم في المقاييس.

> **تحذير**
>
> لا تساوي بين `average_precision_score` وبين «AUC للمنحنى PR» المحسوب هندسياً. متوسط الدقة AP يحسب مجموع دقائق الاسترجاع مضروبةً بدقة العتبة السابقة عند كل نقطة (مجموع مستطيلات لا شرائح شبه منحرفة)، بينما `np.trapezoid` على نقاط المنحنى يعطي قيمة أقل غالباً وتختلف بحسب كثافة النقاط. الخلاصة: عند المقارنة بين نماذج أو مع أوراق بحثية استخدم AP من `sklearn.metrics` حصراً، واذكر الاسم بدقة في تقريرك.

٩.٥ det_curve: عدالة المقارنة في قاع المنحنى

بقيت مسألة أخيرة: كثير من الأنظمة الحرجة تشتغل في القاع الأيسر من منحنى ROC (FPR منخفض جداً: إنذار كاذب واحد في ملايين المعاملات)، وهناك تبدو منحنيات النماذج «ملتصقة» بمحاور مستقيمة تختفي فيها الفوارق. **منحنى DET (Detection Error Tradeoff)** يحل ذلك برسم معدلي الخطأ الطبيعيين — FPR مقابل FNR — على مقياس المقلوب الطبيعي (normal deviate) الذي يمدد القاع ويستقيم به:

$$\mathrm{FNR} = \frac{FN}{FN + TP} = 1 - \mathrm{TPR}$$

```python
# قراءة القاع حيث تُحسم المقارنات: det_curve
import numpy as np
from sklearn.metrics import det_curve

fpr_l, fnr_l, _ = det_curve(y_te, s_log)
fpr_h, fnr_h, _ = det_curve(y_te, s_hgb)

order_l = np.argsort(fpr_l)
order_h = np.argsort(fpr_h)
for target_fpr in [0.01, 0.05]:
    fnr_at_l = float(np.interp(target_fpr, fpr_l[order_l], fnr_l[order_l]))
    fnr_at_h = float(np.interp(target_fpr, fpr_h[order_h], fnr_h[order_h]))
    print(f"عند FPR = {target_fpr:.0%}: FNR اللوجيستي = {fnr_at_l:.4f} | FNR التدرّجي = {fnr_at_h:.4f}")
```

المخرج يعيد النقاط على مقياس الخطأ المباشر: بدل «احتمالات» تعتمر بالتحويلات، تقرأ معدلات الخطأ كما ستُعلنها للجهة الرقابية. والقاعدة العملية التي ذاع بها DET في التعرف على المتحدثين والبصمات: **منحنيان متوازيان على مقياس DET يعني فرقاً ثابتاً قابل للترجمة** بين النموذجين مهما غيّرت العتبة — بينما تقارب منحنيين في زاوية ROC قد يخفي فارقاً هائلاً عند العتبات الحقيقية. للرسم الجاهز استدعِ `DetCurveDisplay.from_estimator` من `sklearn.metrics`، وستلمح شكل المنحنى المستقيم المحبب في الفصل ٧٧ حيث نحتاجه لاختيار العتبة.

وإلى أين من هنا؟ من أي نقطة على أي منحنى يمكن التقاط العتبة التي وصلتك إليها (الثلاثية الثالثة في مخرجات *precision*recall_curve *g* roc*curve*). لكن تحويل هذه القراءة إلى نظام إنتاجي محصّن — عتبة مثبتة، أو عتبة تُضبط تلقائياً على هدف تكلفة، فوق احتمالات معايرة صادقة — هو بالضبط موضوع الفصل ٧٧.

في الواقع

في أنظمة كشف الاحتيال المالي، قرار «أي نقطة من منحنى PR نشتغل عندها؟» يتخذ فوق دولار حقيقي: خفض العتبة يوقف بطاقات سليمة ويغضب العملاء، ورفعها يمرر احتيالاً. ومحللون يحسبون غالباً AP فقط على بيانات بعثرة ٠.٥% ثم يقارنونه بنموذج مختبرٍ آخر على ٢٠% — فتنهار المقارنة كلياً لأن خطوط الأساس مختلفة. المنحنيات مع خطوط أساسها الصحيحة هي لغة التفاوض الوحيدة بين فرق النماذج وفرق الأعمال.

تمارين الفصل السادس والسبعين

- **ملاحظة**

التمرين ٩.١

على ثماني عينات بدرجات: y = [١, ٠, ١, ٠, ١, ١, ٠, ٠] والدرجات [٠.٨٥, ٠.٧, ٠.٦, ٠.٥, ٠.٤٥, ٠.٣, ٠.٢٥, ٠.١]: عدّ الأزواج (١٦ زوجاً) واحسب AUC يدوياً، ثم تحقق بـ roc*auc*score، ثم اطبع نقاط roc_curve وقارن عددها بعدد العتبات المميزة.

التمرين ٩.٢

على بيانات الأرقام `load_digits` مع تحويلها لمسألة ثنائية (الرقم زوجي = إيجابي): درب مصنفاً وقيّمه بـ roc*auc*score مرتين — مرة بـ predict*proba* ومرة بـ *decision*function. هل تتطابق الدرجة؟ اشرح لماذا يصح هذا هنا بينما كان خاطئاً في مثال النبيذ متعدد الفئات.

التمرين ٩.٣

دون أي نموذج: ولّد ٢,٠٠٠ درجة عشوائية منتظمة بين ٠ و ١ وولّد أهدافاً عشوائية بنسبة إيجابيات ٥%، ثم احسب AUC و AP. كرر التجربة ذهنياً مع نسبة إيجابيات ٤٠%. ماذا يخبرك هذا عن خطي الأساس ومنحنى PR؟

- **ملاحظة**

حل التمرين ٩.١: AUC بالعد على ثماني عينات python

```python
import numpy as np
from sklearn.metrics import roc_auc_score, roc_curve

y_true = np.array([1, 0, 1, 0, 1, 1, 0, 0])
scores = np.array([0.85, 0.7, 0.6, 0.5, 0.45, 0.3, 0.25, 0.1])

pos = scores[y_true == 1]
neg = scores[y_true == 0]
wins = sum(1 for p in pos for q in neg if p > q)

print(f"الأزواج الرابحة: {wins} من {len(pos) * len(neg)}")
print(f"AUC يدوياً: {wins / (len(pos) * len(neg)):.4f} | roc_auc_score:
{roc_auc_score(y_true, scores):.4f}")
fpr, tpr, thr = roc_curve(y_true, scores)
print(f"عدد نقاط المنحنى: {len(fpr)} | عدد العتبات: {len(thr)}")
print("FPR:", fpr.round(4))
print("TPR:", tpr.round(4))
```

أحد عشر زوجاً رابحاً من ستة عشر تعطي AUC = ٠.٦٨٧٥. وطبّق العد بيتاً: ٠.٨٥ يفوز على الأربعة، و ٠.٦ على ثلاثة، و ٠.٤٥ على اثنين، و ٠.٣ على اثنين. ولاحظ في مخرج roc_curve أن عدد النقاط أقل من عدد القيم المميزة للدرجات لأن sklearn يهمل النقاط المتطابقة.

ملاحظة

حل التمرين ٩.٣: درجات عشوائية تفضح خطي الأساس python

```python
import numpy as np
from sklearn.metrics import roc_auc_score, average_precision_score

rng = np.random.default_rng(42)
random_scores = rng.random(2000)
y_random = (rng.random(2000) < 0.05).astype(int)   # 5% انتشار

print(f"الانتشار: {y_random.mean():.4f}")
print(f"AUC درجات عشوائية: {roc_auc_score(y_random, random_scores):.4f} (المفترض:
0.5)")
print(f"AP درجات عشوائية:  {average_precision_score(y_random, random_scores):.4f}
(المفترض: الانتشار)")
```

الدرجات لا تحمل أي معلومة عن الأهداف، فسقط كل منهما على خط أساسه: AUC قرب ٠.٥ (القطر) و AP قرب ٠.٠٥ (معدل الانتشار، خط أفقي). ولولا الاختلال لتساوى المقياسان عند ٠.٥ — لهذا ستنهار المقارنة بين مسألتين مختلفتي

الانتشار إن لم تنظر إلى كل خط أساسه. جرّب ذهنياً نسبة ٤٠%: يتغير خط أساس AP إلى ٠.٤٠ بينما يبقى خط أساس AUC هو نفسه دائماً.

> **ملاحظة**

٩.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
توقع خط أساس ٠.٥ في منحنى PR	تقر AP = ٠.٨ على بيانات ١% فيستبعد النموذج وهو ممتاز	قارن AP بخط أفقي على ارتفاع معدل الانتشار
تمرير decision*function* إلى *roc*auc_score متعدد الفئات	فوران لأن الأعمدة لا تشكل احتمالات تجمع إلى ١	استخدم predict*proba* *مع* *multi*class='ovr' أو 'ovo'
مساواة average*precision*score بمساحة PR المحسوبة بـ np.trapezoid	أرقام مختلفة لتفسيرات مختلفة ونقاش لا ينتهي	اعتمد AP من sklearn.metrics حصراً وسمّه في التقارير
تقييم النموذج عند عتبة predict الافتراضية وحدها	درجة واحدة تخفي عائلة كاملة من المواضع الممكنة	اقرأ roc*curve* *أو* precision_recall*curve* كاملة قبل القرار

٩.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
منحنى خصائص التشغيل	منحنى TPR مقابل FPR عبر كل العتبات، وخط أساسه القطر	Receiver Operating Characteristic (ROC)
معدلا الخطأ	نسبة التفويت FNR ونسبة الإنذار الكاذب FPR على مقياس DET الطبيعي	Error rates (FPR / FNR)
المساحة تحت المنحنى	احتمال تفوق درجة الإيجابي على درجة السالب (تعبير عن جودة الترتيب)	AUC
منحنى الدقة والاسترجاع	منحنى الدقة مقابل الاسترجاع، وخط أساسه أفقي عند معدل الانتشار	Precision-Recall curve
متوسط الدقة	خلاصة منحنى PR بمجموع مستطيلات دقيقة الاسترجاع؛ أدق من المساحة الهندسية	Average Precision (AP)

الجزء ٩ · التقييم والاختيار

الفصل ٧٧ — ضبط عتبة القرار

DetCurveDisplay و FixedThresholdClassifier و TunedThresholdClassifierCV

الوحدة **scikit-learn ١.٩** الحجم **٩٩٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تثبّت عتبة قرار مخصصة عبر FixedThresholdClassifier وتقيس أثرها في الدقة والاسترجاع
- أن تضبط العتبة تلقائياً عبر TunedThresholdClassifierCV بهدف f1 أو دالة تكلفة مخصصة، وتقرأ cv_results_
- أن تفسر لماذا تخدع الاحتمالات غير المعايرة العتبات، وتعالجها بـ CalibratedClassifierCV بطريقتيها
- أن ترسم وتقرأ منحنى الموثوقية عبر calibration_curve وتحسم بين sigmoid و isotonic
- أن تركب الخط الكامل: نموذج ثم معايرة ثم عتبة داخل تركيبات محصّنة من تسرب التحقق

٩.١ العتبة ٠.٥: اتفاقية عمياء بلا سند

لماذا يحكم `predict()` بالإيجابية عند ٠.٥ تحديداً وليس ٠.٣ أو ٠.٧؟ الجواب المخزن: لا سبب. العتبة نصف الواحدة افتراض تاريخي يفترض أن كلفة الخطأين متساوية وأن الاحتمالات صادقة — وافتراضان ينكسران في أول مشروع حقيقي. والدليل أن العتبة «زجاج تحكم» كامل:

طاولة العتبات: نفس النموذج ست سلوكات
python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_score, recall_score, accuracy_score

X, y = make_classification(
    n_samples=4000, n_features=10, n_informative=4,
    weights=[0.95, 0.05], random_state=42
)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
model.fit(X_tr, y_tr)
prob = model.predict_proba(X_te)[:, 1]

print("العتبة | الاسترجاع | الدقة | الدقة الخام")
for t in [0.1, 0.2, 0.3, 0.5, 0.7, 0.9]:
    y_hat = (prob >= t).astype(int)
    rec = recall_score(y_te, y_hat)
    prec = precision_score(y_te, y_hat, zero_division=0)
    acc = accuracy_score(y_te, y_hat)
    print(f"{t:6.1f} | {rec:9.4f} | {prec:5.4f} | {acc:11.4f}")
```

ست سلوكات لنموذج واحد مدرب مرة: عند ٠.١ تمسك كل الإيجابيات وترهق الأصحاب بإنذارات كاذبة، وعند ٠.٩ يهدأ النظام ويفوّت أشياء مهمة. أي صف من هذه الطاولة هو «النموذج» الذي تنشره؟ السؤال عملي أكثر من أن يترك لصمّم مكتبة، ومن هنا جاءت أدوات العتبة الصريحة:

📝 جديد في ١.٩

الإصدارات القديمة تركت ضبط العتبة للبحث اليدوي في منحنيات القرار (الفصل ٧٦) ثم النسخ اليدوي للرقم. الإصدار ١.٩ يوفر أول أدوات عتبة من الفئة الأولى في واجهة المقدرات: `FixedThresholdClassifier` للتثبيت اليدوي و `TunedThresholdClassifierCV` للضبط التلقائي — كلاهما من `sklearn.model_selection` ويعمل كأي مقدر عادي: `fit` ثم `predict`.

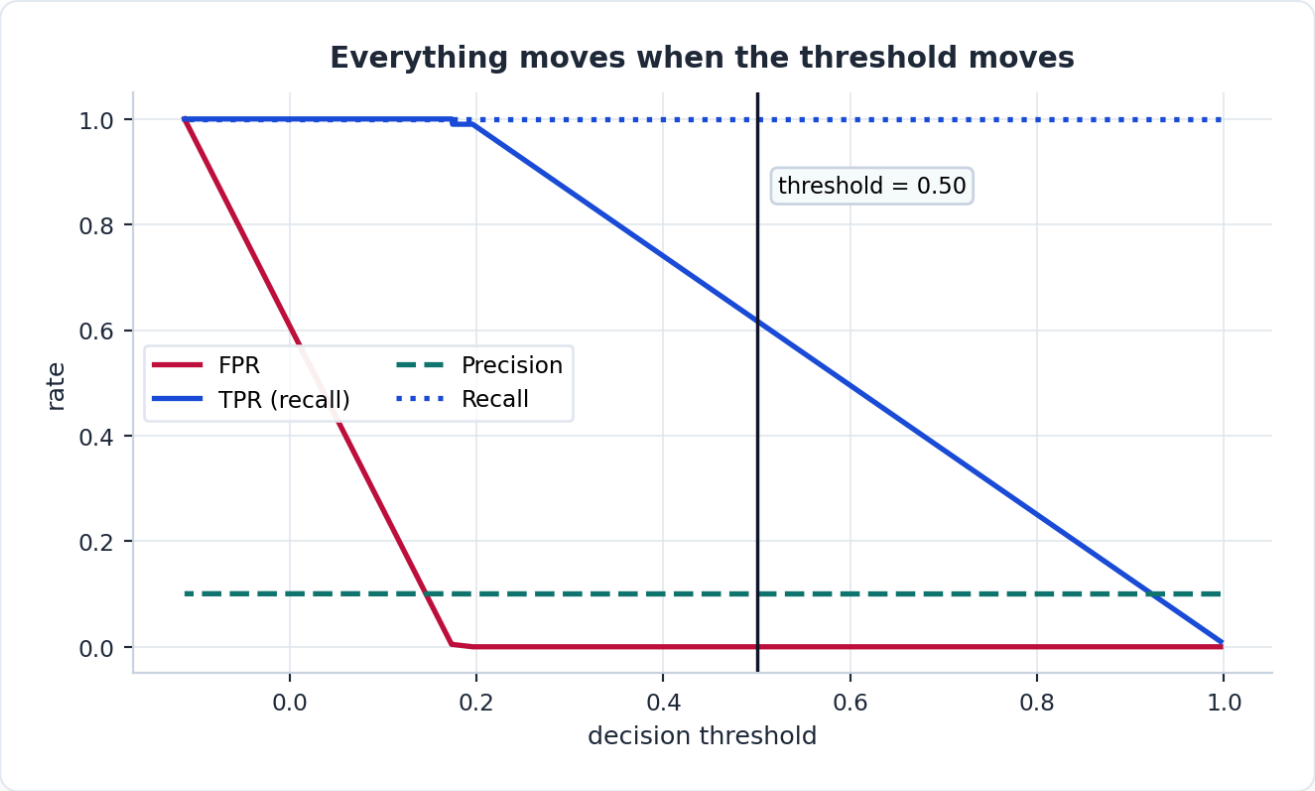
> **مفتاح**
>
> العتبة جزء من النموذج لا أثر جانبي له: النموذج النهائي = خوارزمية الترتيب + موضع القطع عليها. فصل هذا المركب إلى «نموذج جيد + عتبة افتراضية» من أنقى أشكال فقدان القيمة في مشاريع التصنيف، لأن معظم مسائل العالم تعاني اختلالاً وتكاليف غير متساوية — كما رسمنا في الفصل ٧٥.

٩.٢ FixedThresholdClassifier: عتبة تقررها أنت

سيناريو الفصل ٧٥ يقول: تفويت مريض (FN) كارثة أغلى من إنذار كاذب (FP)، فسيتعمّد الطبيب خفض العتبة ثم يقيس الأثر بمقاييسه بدل التخمين. الأداة `FixedThresholdClassifier` من `sklearn.model_selection` تلائم هذا القرار اليدوي الواعي وتجعله جزءاً من المقدر نفسه:

```python
طبيب يخفض العتبة إلى 0.3 ويقيس الثمن والمكسب
from sklearn.model_selection import FixedThresholdClassifier
from sklearn.frozen import FrozenEstimator
from sklearn.metrics import precision_score, recall_score

frozen_model = FrozenEstimator(model)   # نجمّد النموذج: لا إعادة تدريب مع كل عتبة

print("العتبة | الاسترجاع | الدقة | الدقة الخام")
for t in [0.2, 0.3, 0.5, 0.7]:
    ftc = FixedThresholdClassifier(estimator=frozen_model, threshold=t)
    y_hat = ftc.predict(X_te)          # قطع عند العتبة المطلوبة مباشرة
    rec = recall_score(y_te, y_hat)
    prec = precision_score(y_te, y_hat, zero_division=0)
    acc = accuracy_score(y_te, y_hat)
    print(f"{t:6.1f} | {rec:9.4f} | {prec:5.4f} | {acc:11.4f}")
```

خفض العتبة إلى ٠.٣ رفع الاسترجاع بوضوح مقابل هبوط محسوم في الدقة — صفقة رابحة حين تكون كلفة FN أضعاف كلفة FP. ولاحظ الحيلة المهنية: `FrozenEstimator` من `sklearn.frozen` تلف النموذج المدرب فتجعل كل `fit` لاحقاً بلا أثر، فيدور `FixedThresholdClassifier` على العتبات بلا إعادة تدريب — وهذا ما تجسده فكرة الشكل القادم:



الشكل ١٦٩طاولة العتبات: تحرك الدقة والاسترجاع باتجاهين متعاكسين كلما تحركت العتبة فوق توزيع الدرجات

في الشكل العتبة خط عمودي يتحرك فوق توزيع الدرجات: كل إزاحة يسرية ترفع الاسترجاع وتغرق الدقة، واليمنية تفعل العكس. المعامل `response_method` يحدد مصدر الدرجات (احتمالات `predict_proba` أو درجات `decision_function`)، و `pos_label` يحدد الفئة الإيجابية فوق العتبة — نفس مفاتيح الفصل ٧٥ حرفياً.

> **نصيحة**
>
> `FixedThresholdClassifier` لا يعدّل الدرجات إطلاقاً؛ إنه يقطعها فقط. لذلك يصح تركيبه فوق أي مقدر مدرب (ولو من مكتبة أخرى) بشرط توفير الدرجات عبر response_method. وعند التكامل مع خط أنابيب، ضعه آخر خطوة حتى يقرأ درجات النموذج كما هي.

٩.٣ TunedThresholdClassifierCV: العتبة التي تُضبط تلقائياً

الاختيار اليدوي جيد حين تعرف العتبة المطلوبة، لكن ماذا لو كان هدفك صيغة قابلة للتحسين: «اعظم F1» أو «اخفض تكلفة ١٠٠ تفويت مقابل ١٠ إنذارات»؟ هنا يأتي `TunedThresholdClassifierCV` يبحث عن العتبة المثلى تلقائياً بتحقق متقاطع داخلي يمنع التسرب:

ضبط تلقائي بهدف F1 مع حفظ نتائج البحث python

```python
from sklearn.model_selection import TunedThresholdClassifierCV

tuned = TunedThresholdClassifierCV(
    estimator=model, scoring="f1", thresholds=100,
    cv=5, store_cv_results=True, random_state=42, n_jobs=-1
)
tuned.fit(X_tr, y_tr)

print(f"العتبة المثلى التي اختارها: {tuned.best_threshold_:.4f}")
print(f"أفضل F1 داخل التحقق المتقاطع: {tuned.best_score_:.4f}")
print(f"مفاتيح cv_results_: {list(tuned.cv_results_.keys())}")
print(f"أبعاد cv_results_['scores']: {tuned.cv_results_['scores'].shape}")
```

السطر الأهم للمهندس المتشكك هو `cv_results_`: مع `store_cv_results=True` تحفظ الأداة درجة كل عتبة مرشحة (١٠٠ موزعة على نطاق الدرجات) في كل طيّة — لذلك أبعاد المصفوفة (عدد الطيّات × عدد العتبات). وهذه الشفافية تسمح بالتدقيق: هل العتبة المختارة قمة متقلبة أم هضبة مستقرة؟

كيف يجري الضبط داخلياً بلا تسرب؟ أربع خطوات: أولاً التحقق المتقاطع على بيانات التدريب عبر `cross_val_predict` كما في الفصل ٧٣ فتحصل كل عينة على درجة من نموذج **لم يرها**؛ ثانياً تُفحص العتبات المرشحة كلها على هذه الدرجات خارج العينة بحسب `scoring`؛ ثالثاً تُختار العتبة الفائزة؛ رابعاً يُعاد تدريب المقدر على كامل التدريب (`refit=True`) مع تثبيت العتبة فوقه. بيانات الاختبار لم تُلمس في أي خطوة.

٩.٤ دالة تكلفة مخصصة: حين يكلف التفويت ١٠٠ ويكلف الإنذار ١٠

الهدف الحقيقي في الأعمال نادراً ما يكون F1. لنعرّف تكلفة صريحة: كل تفويت يكلف ١٠٠ وحدة وكل إنذار كاذب ١٠، ونمررها عبر `make_scorer` كهدف للضبط:

$$\text{Cost}(y, \hat{y}) = 100 \times FN + 10 \times FP \qquad \text{Profit} = -\text{Cost}$$

ضبط العتبة على دالة تكلفة: 100 تفويت مقابل 10 إنذارات python

```python
from sklearn.metrics import make_scorer, confusion_matrix

def total_cost(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return 100 * fn + 10 * fp

# greater_is_better=False: سيتعظّم sklearn الناتج السالب أي يخفض التكلفة
cost_scorer = make_scorer(total_cost, greater_is_better=False,
response_method="predict")

tuned_cost = TunedThresholdClassifierCV(
    estimator=model, scoring=cost_scorer, cv=5, random_state=42, n_jobs=-1
)
tuned_cost.fit(X_tr, y_tr)

y_default = model.predict(X_te)
y_optimized = tuned_cost.predict(X_te)
cost_default = total_cost(y_te, y_default)
cost_optimized = total_cost(y_te, y_optimized)

print(f"العتبة المثلى للتكلفة: {tuned_cost.best_threshold_:.4f}")
print(f"0.5 تكلفة العتبة الافتراضية: {cost_default}")
print(f"تكلفة العتبة المضبوطة:       {cost_optimized}")
print(f"التوفير على الاختبار: {cost_default - cost_optimized} وحدة")
```

العتبة انتقلت من ٠.٥ إلى قيمة أدنى لأن دالة التكلفة تعاقب التفويت عشرة أضعاف الإنذار، والتكلفة هبطت على بيانات لم تُرَ أثناء الضبط. ولاحظ التقنية: مررنا `greater_is_better=False` لأن دالتنا تكلفة يُراد خفضها، فيتعامل sklearn معها كربح سالب يعظمه بالتقليل — `best_score_` يظهر سالباً وقيمته المطلقة أدنى تكلفة وجدها التحقق الداخلي. وهكذا صارت «سياسة التكاليف» الخاصة بمجالك معلمةً في النموذج لا نيةً في ذهنك.

في الواقع

في الأنظمة المالية والطبية تُشتق دالة التكلفة من وحدات حقيقية: متوسط خسارة عملية احتيال، كلفة فحص تأكيدي، كلفة إزعاج عميل. ثم تصبح العتبة سؤال ميزانية: «كم إنذاراً كاذباً نقبله لتخفيض التفويت ٥٠%؟». أدوات ١.٩ تجعل هذه المفاوضة قابلة للترميز والمراجعة بدل رقم سحري مدفون في سطر predict.

٩.٥ منحنى DET: عدالة المقارنة في قاع المنحنى

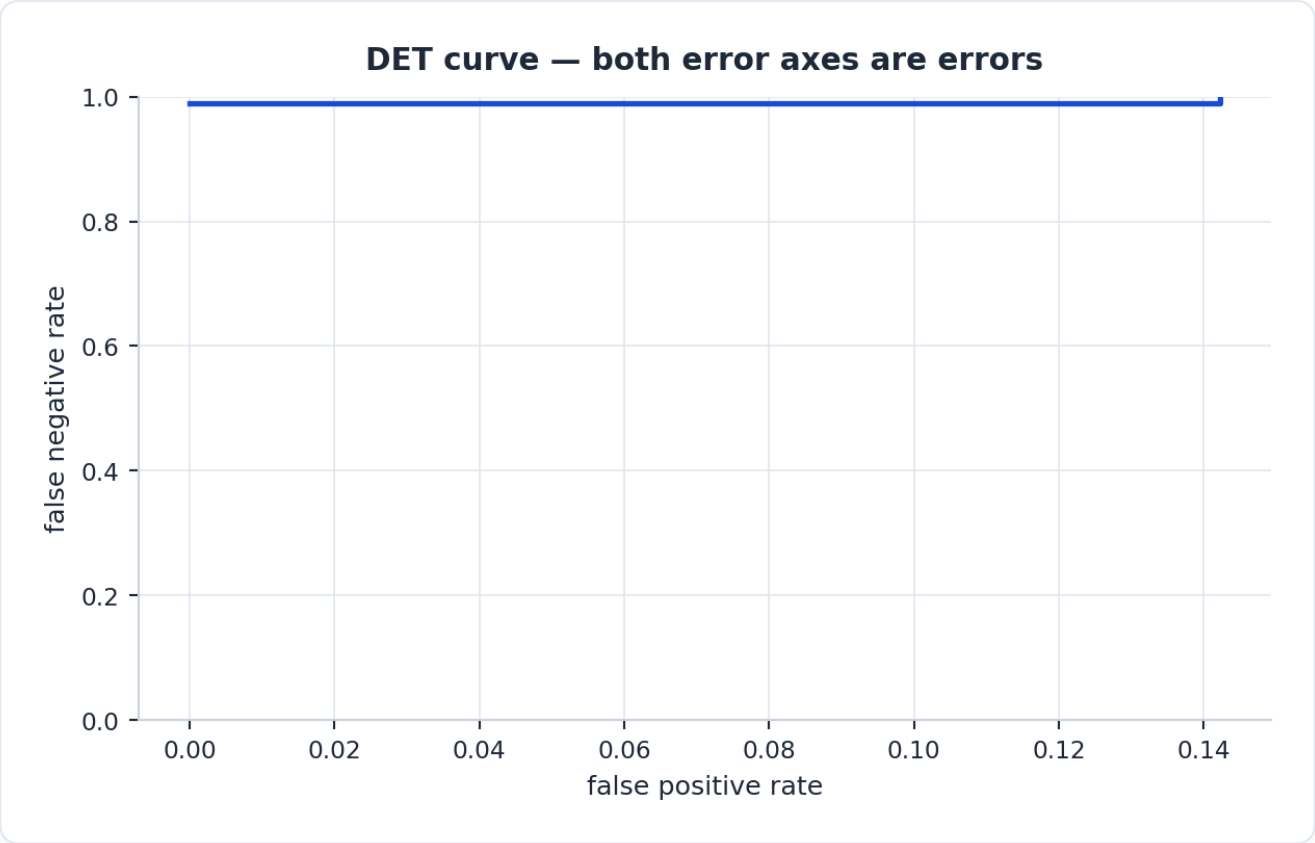
قبل المعايرة، أداة قراءة تكمل ترسانة العتبات. كما لاحظنا في الفصل ٧٦ تُحسم المقارنات الجدية في القاع الأيسر من ROC حيث معدلات الخطأ الصغيرة، وهناك يعجز المقياس الخطي عن إظهار الفوارق. **منحنى DET (Detection Error Tradeoff)** يرسم معدلي الخطأ مباشرة — FPR أفقياً و FNR عمودياً — على مقياس مقلوب-طبيعي يمدد القاع ويحوّل منحنيات الجرس خطوطاً شبه مستقيمة:

$$\text{FPR} = \frac{FP}{FP + TN} \qquad \text{FNR} = \frac{FN}{FN + TP}$$

مقارنة نموذجين حيث تُحسم: القاع على مقياس DET — python

```python
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import det_curve, DetCurveDisplay

hgb = HistGradientBoostingClassifier(max_iter=120, random_state=42).fit(X_tr,
y_tr)

fpr_l, fnr_l, _ = det_curve(y_te, prob)                    # اللوجيستي
fpr_h, fnr_h, _ = det_curve(y_te, hgb.predict_proba(X_te)[:, 1])

order_l, order_h = np.argsort(fpr_l), np.argsort(fpr_h)
for target in [0.01, 0.05]:
    fnr_l_at = float(np.interp(target, fpr_l[order_l], fnr_l[order_l]))
    fnr_h_at = float(np.interp(target, fpr_h[order_h], fnr_h[order_h]))
    print(f"عند FPR = {target:.0%}: FNR اللوجيستي = {fnr_l_at:.4f} | FNR التدرّجي =
{fnr_h_at:.4f}")

# للرسم الجاهز: DetCurveDisplay.from_estimator(hgb, X_te, y_te)
```

معدلات الخطأ هنا أرقام تشغيل كما ستعلنها: «عند إنذار كاذب لكل مئة، نفوّت هذا العدد». وميزة DET التي صمدت عقوداً في التعرف على المتحدثين: **منحنيان متوازيان على المقياس المحوَّر يعنيان فرقاً ثابتاً مهما تحركت العتبة** — بينما في ROC قد يتصادم منحنيان في الزاوية وتضيع القصة. الشكل التالي يجسد ذلك:



الشكل ١٧٠منحنى DET: معدلات الخطأ على المقياس المقلوب-الطبيعي وقابلية المقارنة في القاع

في الشكل استقامت المنحنيات على المقياس المحوَّر وابتعد القاع كي تُقرأ الفوارق الصغيرة: هناك تُختبر أنظمة الإنتاج الفعلية لأنها تشتغل عند FPR منخفض جداً. والرسم جاهز عبر `DetCurveDisplay.from_estimator` أو `from_predictions` من `sklearn.metrics`، ويقبل درجات `decision_function` مباشرة — ميزة تختصر الخطوات عند المقارنة الخام.

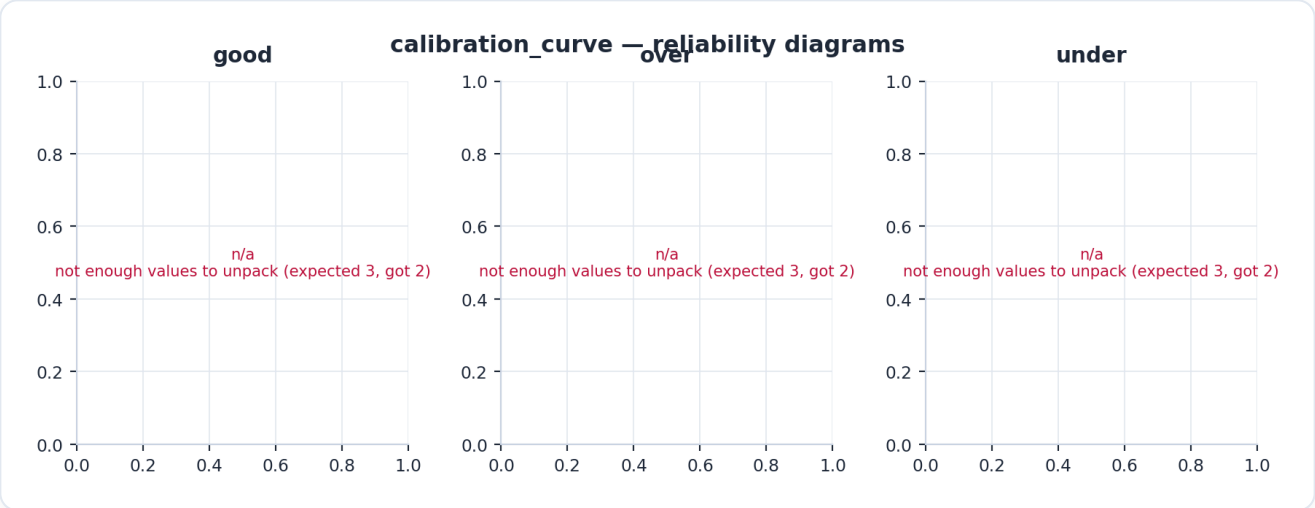
٩.٦ المعايرة: احتمالات صادقة قبل أي عتبة

كل ما فعلناه يقف على افتراض صامت: أن ٠.٧ الصادرة عن النموذج تعني فعلاً ٧٠% احتمالاً، وهذا كاذب غالباً؛ الغابات تدفع الدرجات نحو ٠ و ١، ودرجات `decision_function` ليست احتمالات أصلاً. والسؤال الكاشف: **عيّنات قيل لها ٠.٨ — كم عدت إيجابية فعلاً؟** إن كان الجواب ٤٥% فأنت تحكم بعتبة فوق إحصاء لا علاقة له بالاحتمال.

هذه **المعايرة (calibration)**، وأداتها الأولى `calibration_curve` من `sklearn.calibration` تختبر الموثوقية: صناديق من الدرجات وفي كل صندوق مقارنة المتوسط المعلن بالنسبة الحقيقية للإيجابيات:

```python
# منحنى الموثوقية: هل 0.8 تعني 80% فعلاً؟
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import calibration_curve

rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
rf.fit(X_tr, y_tr)
prob_rf = rf.predict_proba(X_te)[:, 1]

prob_true, prob_pred = calibration_curve(y_te, prob_rf, n_bins=8)
print("صندوق | متوسط معلّن | نسبة حقيقية")
for pred, true in zip(prob_pred, prob_true):
    print(f"      | {pred:13.3f} | {true:13.3f}")

mid = ((prob_rf > 0.1) & (prob_rf < 0.9)).mean()
print(f"نسبة الدرجات في المدى المتوسط 0.9-0.1: {mid:.2%}")
```

اقرأ الجدول كتقرير موثوقية: الغابة تدفع أكثر من ٨٠% من درجاتها إلى الأطراف، وفي منتصف المدى تتخبط: صندوق أعلن ٠.٢٧ فما بينه إيجابية واحدة، وصندوق أعلن ٠.٤٦ فكان ٠.٧٥ إيجابية. انزع عتبة فوق درجات كهذه وستحصل على مقياسين يتبدلان بلا تفسير — أنت تقطع توزيعاً مشوهاً لا توزيع احتمال:



الشكل ١٧١ منحنى الموثوقية: نموذج معاير ملتصق بالقطر مقابل نموذج متعجرف ينحرف عنه

في الشكل القطر هو المثال: النموذج المعاير يلتصق به (ما يقوله يقع)، والمتعجرف ينفجر فوقه أو تحته. العلاج `CalibratedClassifierCV` من `sklearn.calibration`، وتوفر طريقتين بينهما قرار مهني حقيقي:

$$\text{ECE} \approx \frac{1}{B}\sum_{b=1}^{B} \left| \bar{p}_b^{\text{predicted}} - \bar{p}_b^{\text{true}} \right|$$

المعيار	'method='sigmoid	'method='isotonic
الفكرة	انحدار لوجيستي على الدرجات (معايرة بلات)	انحدار درجات متعدد الحدود الترتيبي بلا فرضية شكل
الفرضية	شكل S (منحنى لوجيستي)	أي دالة متزايدة حرة
بيانات قليلة جداً (عشرات)	مفضَّلة — معاملان فقط يقدَّران بأمان	قد يلوي على الضجيج فيصنع «سلّماً» متهالكاً
بيانات كافية (مئات فأكثر)	قد تفرض شكلاً أضيق من الحقيقة	مفضَّلة — تتقارب مع القطر بحرية
مخرجات مؤكدة	قد تخرج دقّات ملتصقة بالأطراف	قد تُسقط بعض الدرجات الوسطى

على بيانات صغيرة isotonic مقابل sigmoid python

```python
from sklearn.calibration import CalibratedClassifierCV

def calibration_error(y, prob, n_bins=8):
    pt, pp = calibration_curve(y, prob, n_bins=n_bins)
    return float(np.abs(pt - pp).mean())

# عينة تدريب صغيرة: مختبر القاعدة العملية
X_small, _, y_small, _ = train_test_split(
    X_tr, y_tr, train_size=400, random_state=42, stratify=y_tr
)

cal_sig = CalibratedClassifierCV(model, method="sigmoid", cv=5).fit(X_small,
y_small)
cal_iso = CalibratedClassifierCV(model, method="isotonic", cv=5).fit(X_small,
y_small)

err_sig = calibration_error(y_te, cal_sig.predict_proba(X_te)[:, 1])
err_iso = calibration_error(y_te, cal_iso.predict_proba(X_te)[:, 1])
print(f"خطأ المعايرة على بيانات صغيرة: sigmoid = {err_sig:.4f} | isotonic =
{err_iso:.4f}")
```

كان الناتج مفاجئاً لمن يحفظ القاعدة بحذافيرها: الإيزوتوني تفوق حتى على عينة الأربعمئة — لأن `cv=5` منحه نحو ٣٢٠ عينة معايرة لكل طيّة والنموذج الأساسي محترم الدرجات، فلم يجد ضجيجاً يلوي عليه. انتصار السيجمويد يأتي حين تقل البيانات أكثر (عشرات) أو تتشوه الدرجات بشدة: معاملان يقدَّران بأمان بدل سلّم متشظٍ. القاعدة: **بيانات قليلة جداً** → **sigmoid**، **وكافية** → **isotonic**، وكلاهما يعاير بالتحقق الداخلي فلا يسرّب معلومات التقييم.

٩.٧ خط الإنتاج الكامل: نموذج ثم معايرة ثم عتبة

نجمع الثلاثيات في خط واحد يحاكي الاستخدام الصناعي: نموذج يرتب، معايرة تصحح معاني الدرجات، عتبة تقرر موضع القطع — وقراءة هذا الخط هي درس الترتيب الأهم في الفصل.

الخط الكامل: نموذج ثم معايرة ثم عتبة مضبوطة python

```python
from sklearn.frozen import FrozenEstimator

calibrated = CalibratedClassifierCV(model, method="isotonic", cv=5).fit(X_tr,
y_tr)
full_line = TunedThresholdClassifierCV(
    estimator=FrozenEstimator(calibrated), scoring="f1",
    cv=5, random_state=42, n_jobs=-1
)
full_line.fit(X_tr, y_tr)

y_line = full_line.predict(X_te)
print(f"عتبة الخط الكامل (فوق درجات معايرة): {full_line.best_threshold_:.4f}")
print(f"دقة الخط الكامل: {precision_score(y_te, y_line, zero_division=0):.4f}")
print(f"استرجاع الخط الكامل: {recall_score(y_te, y_line):.4f}")
print(f"تكلفة الخط الكامل: {total_cost(y_te, y_line)} | تكلفة العتبة 0.5 الخام:
{total_cost(y_te, y_default)}")
```

لماذا هذا الترتيب ولا عكسه؟ المعايرة تحتاج بيانات لم تُدرس عليها لتصحيح الدرجات، فتجري داخل `CalibratedClassifierCV` بالتحقق المتقاطع الداخلي قبل أي تقييم، ثم تُجمَّد بـ `FrozenEstimator` كي لا يعيد ضبط العتبة تدريبَها في كل طيّة. ولو عكسته (عتبة فوق درجات خام ثم معايرة بعدها) فإنقلبت المعاني: العتبة اختارت موضعاً على توزيع ثم غيّرت المعايرة شكل التوزيع من تحت قدميها.

خطأ شائع

لا تنادي `fit` على كامل البيانات ثم تقيّم على بيانات من نفس الدلو — هذه معايرة تسرب كما نذرنا في الفصل ٧٢. القاعدة: كل ما يُقدَّر من البيانات (تحجيم، ترميز، معايرة، عتبة) يعيش داخل التحقق المتقاطع أو خط الأنابيب الذي يُركَّب في كل طيّة.

قائمة تحقق

قبل النشر، تأكد من هذه الأربعة:

تمارين الفصل السابع والسبعين

- **ملاحظة**

التمرين ٩.١

على بيانات القسم ٩.١: استخرج من `precision_recall_curve` (الفصل ٧٦) العتبة الدنيا التي تحقق استرجاعاً لا يقل عن ٠.٩، ثم ثبّتها في `FixedThresholdClassifier` وتحقق على الاختبار من الاسترجاع والدقة الفعليين.

التمرين ٩.٢

أعد تجربة القسم ٩.٦ على بيانات وافرة (تدريب على كامل X_tr بدل عينة الأربعمئة): هل يتسع فارق خطأ المعايرة بين السيجمويد والإيزوتوني أم يتقلص؟ اشرح بجملتين مستشهداً بجدول القسم ٩.٦.

التمرين ٩.٣

من وجهة نظر مزود البريد: حجب رسالة سليمة (FP) يكلف ٥٠ وحدة، ومرور رسالة مزعجة (FN) يكلف ٥. عرّف دالة التكلفة، واضبط `TunedThresholdClassifierCV` عليها: هل ترتفع العتبة المثلى أم تنخفض مقارنة بمسألة الطبيب؟ ولماذا؟

- **ملاحظة**

python حل التمرين ٩.١: عتبة مستخرجة من منحنى PR ومثبتة

```python
import numpy as np
from sklearn.metrics import precision_recall_curve
from sklearn.model_selection import cross_val_predict

# نستخرج العتبة من درجات خارج عينة على التدريب – لا من الاختبار (احترس من التسرب)
val_scores = cross_val_predict(model, X_tr, y_tr, cv=5, method="predict_proba")
[:, 1]
prec, rec, thr = precision_recall_curve(y_tr, val_scores)
mask = rec[:-1] >= 0.9                  # أقصر بواحد thr تنازلي و rec
threshold_90 = float(thr[mask].min())  # أدنى عتبة تحقق الشرط = أعلى دقة

ftc90 = FixedThresholdClassifier(estimator=frozen_model, threshold=threshold_90)
y_hat90 = ftc90.predict(X_te)
print(f"0.9 =< العتبة المستخرجة لاسترجاع: {threshold_90:.4f}")
print(f"الاسترجاع الفعلي على الاختبار: {recall_score(y_te, y_hat90):.4f}")
print(f"الدقة عند هذه العتبة: {precision_score(y_te, y_hat90, zero_division=0):.4f}")
```

من بين كل العتبات التي تحقق استرجاع ٠.٩ اخترنا الأدنى لأنها تعطي أعلى دقة، واستخرجناها من توقعات التحقق المتقاطع على التدريب — النمط نفسه الذي يجري داخلياً في القسم ٩.٣ — ثم ثبّتناها عبر `FixedThresholdClassifier` فصارت عقداً منشوراً لا اقتراحاً.

> **ملاحظة**

python حل التمرين ٩.٣: عتبة مزود البريد – الإنذار الكاذب أغلى

```python
from sklearn.metrics import make_scorer

def spam_cost(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return 50 * fp + 5 * fn

spam_scorer = make_scorer(spam_cost, greater_is_better=False,
response_method="predict")
tuned_spam = TunedThresholdClassifierCV(
    estimator=model, scoring=spam_scorer, cv=5, random_state=42, n_jobs=-1
)
tuned_spam.fit(X_tr, y_tr)
y_spam = tuned_spam.predict(X_te)

print(f"عتبة مزود البريد المثلى: {tuned_spam.best_threshold_:.4f}")
print(f"(من القسم ٩.٤) عتبة الطبيب: {tuned_cost.best_threshold_:.4f}")
print(f"تكلفة مزود البريد عند عتبته: {spam_cost(y_te, y_spam)}")
print(f"لو استخدم عتبة الطبيب: {spam_cost(y_te, y_optimized)}")
```

العتبة انتقلت إلى الأعلى، والعكس تماماً لمسألة الطبيب: حين يغلي FP أغلى من FN يصبح التحفظ مربحاً فيتضايق النظام عن الإعلان إلا من لديه ثقة عالية. وتطبيق عتبة الطبيب على البريد ضاعف التكلفة — الدرس الحاسم: **العتبة خاصية للمسألة لا للنموذج**، وذات الدرجات تنجح في عيادة وتفشل في صندوق بريد.

> **ملاحظة**

٩.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
نشر النموذج بعتبة ٠.٥ الافتراضية في مسألة اختلالية	استرجاع مأساوي أو إنذارات تغرق الفريق، ولا أحد لاحظ القرار	ضبط العتبة بـ TunedThresholdClassifierCV على هدف تكلفة صريح
تمرير دالة تكلفة يُخفض إلى بلا `make`*`scorer`* `greater`**`is_better=False`**	الضبط يعظم التكلفة فيختار أسوأ عتبة	مرر `greater`*`is`*`better=False` لأي دالة خسارة أو تكلفة
معايرة النموذج ثم ضبط العتبة على نفس بيانات المعايرة بلا تحقق	عتبة متفائلة تنهار على الاختبار	اجعل التحقق الداخلي للأداة يعمل، أو مجموعة تحقق مستقلة
قراءة درجات `decision_function` كاحتمالات وتطبيق عتبة ٠.٣ عليها	عتبة بلا معنى على مقياس لا يلتزم [٠, ١]	عاير أولاً أو استخدم `predict`*`proba`* مع `pos`*`label`* الصحيح

٩.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
ضبط العتبة اللاحق	اختيار موضع القطع بعد التدريب وفق هدف تكلفة أو مقياس	Post-hoc threshold tuning
مصنّف العتبة المثبتة	مقدر يقطع درجات مقدر مدرب عند عتبة محددة	FixedThresholdClassifier
مصنّف العتبة المضبوطة بالتحقق	مقدر يجد العتبة المثلى عبر توقعات التحقق المتقاطع	TunedThresholdClassifierCV
دالة التكلفة	جمع أخطاء موزونة بوحدات مجال حقيقية تحكم اتجاه العتبة	Cost function
المعايرة	تصحيح الدرجات لتطابق تكرارات الإيجابية الفعلية	Calibration
منحنى الموثوقية	مقارنة المتوسط المعلن بالنسبة الحقيقية لكل صندوق درجات	Reliability curve

الجزء ٩ · التقييم والاختيار

الفصل ٧٨ — مقاييس الانحدار

MSE و RMSE و MAE و MAPE و MSLE و R^2 و d^2 و Pinball

الوحدة scikit-learn ١.٩ الحجم ١١٢١ كلمة تقريباً

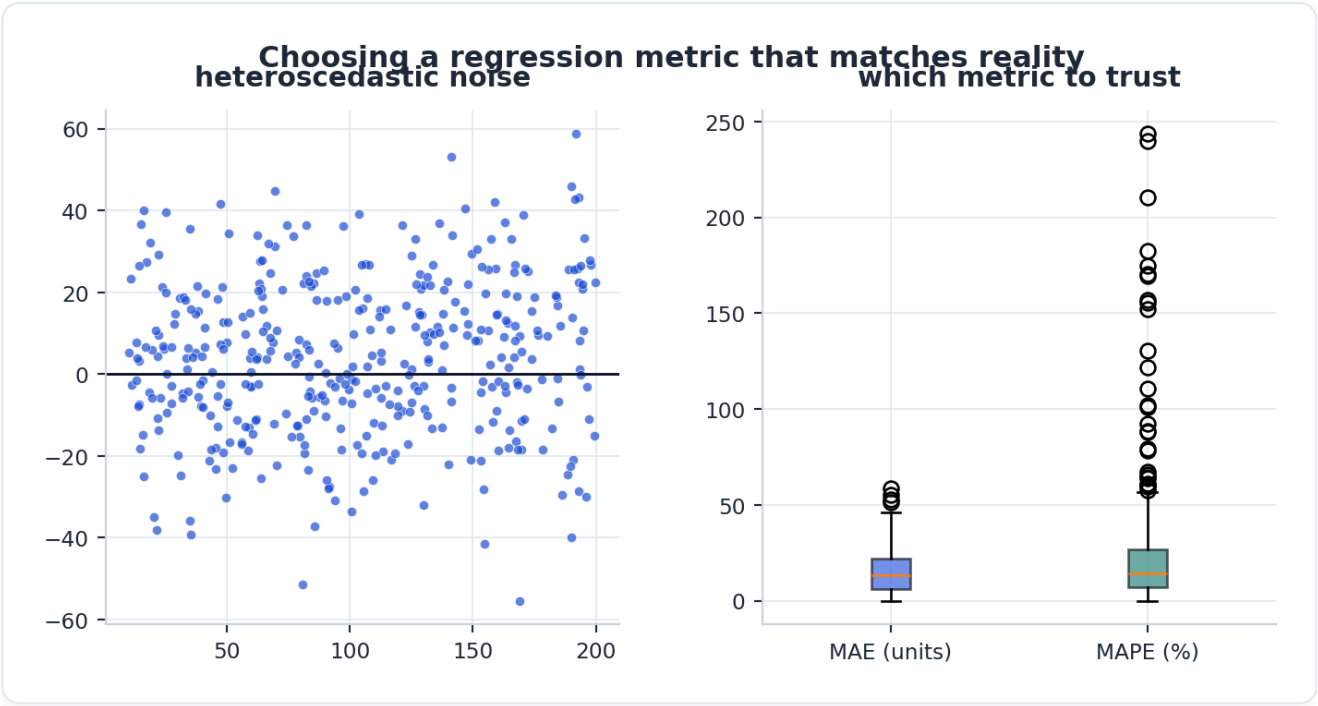
ما الذي ستتقنه في هذا الفصل

- أن تختار مقياس الانحدار المناسب لطبيعة البيانات: شواذ، أو نسب، أو نموّ أُسّي
- أن تحسب RMSE بالاسم الجديد في ١.٩ بعد إزالة mean_squared_error(squared=False)
- أن تفسّر R^2 بحذر: صفراً لا يعني فشلاً دائماً، وسالبه ليس خطأً برمجياً
- أن تبني فواصل تنبؤ بالانحدار الكمّي عبر QuantileRegressor وخسارة Pinball
- أن تقرأ مخطط البواقي لتكتشف الانحياز وتغاير التباين قبل أن يفسد التقييم

٩.١ كل مقياس يحكم على البواقي بطريقته

تخيّل أنك قدّمت نموذجين لتوقّع أسعار المنازل إلى لجنة القرار في شركتك. النموذج الأول خطأه المعتاد فيه بضعة آلاف من الدولارات، لكنه أخطأ مرة واحدة خطأً فادحاً بمئتي ألف. الثاني أخطأ دائماً بخطأ متوسط متقارب. أيّهما «أفضل»؟ الجواب ليس رياضياً بديهياً؛ إنه قرار إداري: مَن سيتحمل الخطأ، وكم يكلّف، وهل الخطأ الكبير الواحد أخطر من عشرة أخطاء صغيرة؟

كل مقاييس الانحدار في هذا الفصل تقرأ المصفوفة نفسها: **البواقي (residuals)** `e_i = y_i - ŷ_i` أي الفرق بين القيمة الحقيقية والمتوقَّعة. لكن كل مقياس يطبّق على هذه الأرقام قاضياً مختلفاً: أحدهما يربّع الخطأ فيعدم الرحمة للبعيد، والآخر يأخذ القيمة المطلقة فيتسامح نسبياً، والثالث يقسم على الحقيقة فيصبح نسبياً، والرابع يقارن بخط أساس كالوسيط أو المتوسط.



الشكل ١٧٢توزّع البواقي حول الصفر ومقارنة الخطأ المطلق بالنسبه — كيف يرى كل مقياس الخطأ نفسه

في الشكل أعلاه نرى بقايا نموذج انحدار حقيقي: يتسع نطاقها كلما كبرت قيمة الهدف، ومن ثم يختلف حكم `MAE` (بوحدات أصيلة) عن حكم `MAPE` (بنسبة مئوية) على النموذج نفسه.

> **مفتاح**
>
> المقياس ليس حكماً محايداً على النموذج؛ إنه **عقد** يحدّد ماذا يعني «خطأ»: من يختار المقياس يختار فعلياً أي أخطاء ستُعاقب وأيها ستُغتفر. ولهذا فاختيار المقياس قرار يُتّخذ قبل ضبط النموذج، لا بعده — وسيتضح حاداً في بحث المعاملات في الفصل ٨١.

٩.٢ MSE و RMSE: حين يصبح الخطأ البعيد جريماً

النسختان الأكثر شيوعاً في الكتب، وأساس الانحدار بأقل المربعات الذي اشتققناه في الفصل ٢٧. متوسط مربّعات الخطأ:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

حيث `n` عدد العينات، و `y_i` القيمة الحقيقية، و `ŷ_i` المتوقَّعة. الجذر التربيعي لها يعيدنا إلى وحدة الهدف نفسها فتصبح قابلة للحديث مع الإدارة:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

لماذا يخاف منه الجميع؟ لأن التربيع عقوبة تصاعدية: خطأ مقداره ٢٠ يستهلك من MSE ما تعادله أربعة أخطاء مقدار كل منها ١٠. هذا سلاح ذو حدين: يصرّ النموذج على حركة الشواذ، ويجعل تدريبه حساساً لنقطة قذرة واحدة.

عقوبة الشاذ: MSE مقابل MAE python

```
import numpy as np
from sklearn.metrics import mean_squared_error, root_mean_squared_error,
mean_absolute_error

y_true = np.array([100., 120., 110., 130., 105., 125., 115., 122.])
err = np.array([2., -1., 3., -2., 1., -3., 2., -1.])   # أخطاء عادية صغيرة
y_pred = y_true + err
print(f"أخطاء عادية : MSE={mean_squared_error(y_true, y_pred):.3f} | "
      f"RMSE={root_mean_squared_error(y_true, y_pred):.3f} | MAE=
{mean_absolute_error(y_true, y_pred):.3f}")

err2 = err.copy()
err2[6] = 20.0                                          # خطأ كبير واحد فقط
y_pred2 = y_true + err2
print(f"خطأ كبير واحد: MSE={mean_squared_error(y_true, y_pred2):.3f} | "
      f"RMSE={root_mean_squared_error(y_true, y_pred2):.3f} | MAE=
{mean_absolute_error(y_true, y_pred2):.3f}")

r_mse = mean_squared_error(y_true, y_pred2) / mean_squared_error(y_true, y_pred)
r_mae = mean_absolute_error(y_true, y_pred2) / mean_absolute_error(y_true,
y_pred)
print(f"نمو الخطأ: MSE ×{r_mse:.1f} ضد MAE ×{r_mae:.1f} — الفجوة هي بصمة التربيع")
```

في الناتج: خطأ واحد تضاعف خمس مرات (من ٢ إلى ٢٠) ضرب MSE بـ ١٣ ضعفاً بينما اكتفى MAE بعامل ٢.٢. هذا هو «العدل التربيعي»: البعيد يُحاكم بعنف، والقريب يُحاكم بلين.

جديد في ١.٩

الاسم القديم للحصول على RMSE كان `mean_squared_error(..., squared=False)`، وقد **أُزيل البارامتر** `squared` **نهائياً** بعد سنوات من الإهمال. الاسم الرسمي اليوم هو `sklearn.metrics.root_mean_squared_error` — كود قديم يمرر `squared=False` سيسكسر في ١.٩، والتصحيح استيراد الدالة الجديدة مباشرة.

ولماذا لا يكتفي العالم بـ MSE الصغيرة؟ لأن مربّع الخطأ بوحدة تربيعية (دولار مربّع!) لا معنى له أمام الإدارة، ولأن RMSE يقترب من مقياس التدريب فتسهّل مقارنة التقييم بوظيفة الخسارة. الدرس: **استخدم RMSE للتواصل والتدريب، لكن لا تدعه يقرر وحده حين توجد شواذ في هدفك.**

٩.٣ MAE والمقاييس القوية: الوسيط وأقصى الأسوأ

متوسط الخطأ المطلق يعاقب خطياً:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

أي خطأ يساهم بقدر مقداره نفسه بغض النظر عن كبره. ثمن هذا التسامح رياضي: الدالة غير قابلة للاشتقاق عند الصفر (التفاف حادّ)، فيصبح التدرّج النزولي عليها خشناً — وهذا أحد أسباب هيمنة MSE تاريخياً. الخصم الموازي: أقل ثابت يقلّل MAE هو **الوسيط** لا المتوسط، ولهذا يوصف MAE بأنه مقاوم الشواذ (outlier-resistant).

فمتى تفضّله؟ حين تكون تكلفة الخطأ خطية (كلفة إعادة شحن لكل قطعة بقدرها)، وحين تحتوي بياناتك شواذ لا يُعقل نمذجتها، وحين تريد رقماً «يمثّل اليوم العادي». وبجانبه يقف حارسان سريعان للسلامة:

مقياس قوي: الوسيط يبتلع الشاذ python

```python
import numpy as np
from sklearn.metrics import (mean_absolute_error, median_absolute_error,
max_error)

err = np.array([1., -1., 2., -2., 1., -1., 30., -1., 2., 1.])  # شاذ واحد بقيمة
30
y_true = np.full(10, 50.) + err
y_pred = np.full(10, 50.)                                       # تنبؤ مثالي عدا
الشاذ

print(f"MAE            = {mean_absolute_error(y_true, y_pred):.3f} (تضخّم بفعل نقطة
واحدة)")
print(f"median_AE      = {median_absolute_error(y_true, y_pred):.3f} (الواقع
المعتاد)")
print(f"max_error      = {max_error(y_true, y_pred):.1f} (أقصى الأسوأ — للسلامة)")
print(f"نسبة MAE إلى median_AE: {mean_absolute_error(y_true, y_pred) /
median_absolute_error(y_true, y_pred):.1f}x")
```

لاحظ التشخيص الذي يقدّمه الناتج: MAE يساوي ٤.٢ بينما median*AE* يساوي ١.٠ — *الفجوة بين الرقمين تكشف أن هناك نقطة سلوك غير معتاد تستحق التحقيق قبل الحكم على النموذج. الدرس: MAE** للعدل الخطي، و*median*AE لبيان «الوضع الطبيعي»، وmax_errorg لاختبار السلامة قبل الإطلاق.**

٩.٤ MAPE و sMAPE: وهم الخطأ النسبي

متوسط نسبة الخطأ المطلق يحوّل كل خطأ إلى نسبة من قيمته الحقيقية:

$$\mathrm{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \frac{| y_i - \hat{y}_i |}{\max(| y_i |, \varepsilon)}$$

حيث `\varepsilon` حدّ أمان صغير في التنفيذ العملي، والناتج بنسبة مئوية — لهذا يحبه المديرون: «النموذج يخطأ بنسبة ٤٪» جملة مفهومة عابرة للأقسام. وظاهرياً هذا عدل: خطأ ٤ دولارات على سلعة بـ ١٠٠ دولار يوازن خطأ ٤ سنتات على سلعة بدولار.

لكن ماذا يحدث حين تمرّ قيمة حقيقية صغيرة جداً — say ٠.٠١ — في بياناتك؟

فخ القيم الصغيرة في MAPE python

```python
import numpy as np
from sklearn.metrics import mean_absolute_percentage_error, mean_absolute_error

y_true = np.array([100.0, 50.0, 0.01])   # عينتان عاديتان وقيمة صغيرة جداً
y_pred = np.array([104.0, 52.0, 0.05])   # أخطاء مطلقة ضئيلة: 4 و 2 و 0.04

print("الأخطاء المطلقة :", np.abs(y_true - y_pred))
print("النسب الفردية   :", np.abs((y_true - y_pred) / y_true))
print(f"MAE  = {mean_absolute_error(y_true, y_pred):.4f} (يبدو هادئاً)")
print(f"MAPE = {mean_absolute_percentage_error(y_true, y_pred):.4f} 136 أي%!")

def smape(y, p):
    return np.mean(2.0 * np.abs(y - p) / (np.abs(y) + np.abs(p)))
print(f"sMAPE (يدوياً) = {smape(y_true, y_pred):.4f} (أهدأ لكنه ليس حلاً جذرياً)")
```

القارئ الذكي يرى الكارثة: خطأ مطلق لا يتجاوز ٠.٠٤ على قيمة حقيقية ٠.٠١ يسهم بنسبة ٤٠٠٪ وحده، فيقفز MAPE الكلي إلى ١٣٦٪ بينما يقف MAE عند ٢.٠. نسخة متماثلة منه هي **sMAPE** التي تقسم على مجموع القيمتين الحقيقية والمتوقعة فيحدّ النطاق، لكنها لا تعالج الجذر: القسمة على أعداد صغيرة تنفجر أينما وُجدت.

خطأ شائع

لا تستخدم `mean_absolute_percentage_error` إذا كان هدفك يحتوي قيماً قريبة من الصفر أو يساويها — القسمة على صفر تعطي قيماً لا نهائية أو غير معرَّفة. الحلول: استبعاد الصغار من التقييم، أو التحويل اللوغاريتمي للهدف مع قياس الخطأ في الفضاء المحوَّل (كما تفعل MSLE)، أو البقاء على MAE.

الدرس: **MAPE مقياس لأهداف موجبة متوسطة البعد عن الصفر (المبيعات، الطلب)؛ خارجه مُضلِّل.**

٩.٥ MSLE و RMSLE: قياس النمو لا الفارق

في أسعار المنازل والسكان وعدد الزيارات، الخطأ المطلق ليس هو المهم؛ **النسبة** هي المهم: تفويت ٥٠٠ دولار على منزل بـ ١٠٠٠ جريمة، أما على منزل بمليون فتساهل. متوسط مربّع الخطأ اللوغاريتمي يقيس المسافة في الفضاء اللوغاريتمي، ويصلح للمقارنة النسبية:

$$\mathrm{MSLE} = \frac{1}{n} \sum_{i=1}^{n} (\log(1 + y_i) - \log(1 + \hat{y}_i))^2$$

الإضافة ١+ داخل اللوغاريتم تحمي من القيم الصفرية، والجذر التربيعي لها هو `root_mean_squared_log_error` (RMSLE) بوصفة مقروءة. هنا مفاجأة يعرفها القليلون: اللوغاريتم غير متماثل — التبخّت **نزولاً** أثقل عقوبةً من المبالغة صعوداً بالمقدار المطلق نفسه:

يعاقب التبخّت نزولاً MSLE python

```python
import numpy as np
from sklearn.metrics import mean_squared_log_error, root_mean_squared_log_error

y_true = np.array([1000.])
under  = np.array([500.])    # 500 نقص بقيمة
over   = np.array([1500.])   # المقدار المطلق نفسه — 500 زيادة بقيمة

msle_u = mean_squared_log_error(y_true, under)
msle_o = mean_squared_log_error(y_true, over)
print(f"500 نقص  : MSLE = {msle_u:.4f} (RMSLE =
{root_mean_squared_log_error(y_true, under):.4f})")
print(f"500 زيادة: MSLE = {msle_o:.4f} (RMSLE =
{root_mean_squared_log_error(y_true, over):.4f})")
print(f"(نقص/زيادة) نسبة العقوبة = {msle_u / msle_o:.2f}x")
print("المسافة اللوغاريتمية غير متماثلة — log(1+1000/500) أكبر من log(1+500/1000) :السبب")
```

عقوبة النقص ٢.٩ ضعف عقوبة الزيادة المتماثلة معه. هذا **ميزة لا عيب** حين يكون التقليل خطراً (تقدير الطلب فتُخزَّن المعدّة أقل من الحاجة، أو تقدير الوباء أقل من الواقع)، لكنه يوجّه نموذجك نحو التبالغ الهادئ إن كان الاتجاهان متعادلين عندك.

> **تحذير**
>
> `sklearn.metrics.mean_squared_log_error` تشترط قيماً غير سالبة في `y_true` و `y_pred` معاً — أي قيمة سالبة تُسقطها استثناءً فوراً. إن كان هدفك يمكنه السلب (أرباح بصافي خسائر، فروق حرارة)، حوّله أولاً بتحويل آمن ثم احسب، أو استخدم مقياساً آخر من هذا الفصل.

الدرس: **MSLE للكمّيات التي تُقاس بنسبة نموّها (أسعار، عدّادات، شبكات اجتماعية)؛ وحين يكون التقليل أغلى من الزيادة فهو المقياس الوحيد الذي يعاقبه بإنصاف.**

٩.٦ R^2 وعائلة d^2: كم فسّر النموذج فعلاً؟

حتى الآن تقاس الأخطاء بوحدات الهدف. معامل التحديد يغيّر السؤال: كم من تباين الهدف لم يفسره النموذج؟

$$R^2 = 1 - \frac{\mathrm{SS}_{res}}{\mathrm{SS}_{tot}} = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2} {\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

البسط مجموع مربعات البواقي حول تنبؤات النموذج، والمقام حول تنبؤ خط الأساس الأخرق: متى تخمّن بالمتوسط `\bar{y}` دوماً. فالنموذج الذي يطابق خط الأساس يأخذ صفراً، وما فوقه مكسب حقيقي. لكن حذارِ من ثلاث قراءات خاطئة شائعة: ليس «نسبة تصنيف صحيح» — قيمة ٠.٨٥ لا تعني ٨٥٪ من العينات صائبة؛ وليس متماثلاً حول الصفر فقد يهبط سالباً خارج العينة؛ والمقام يعتمد على تباين بياناتك أنت فقيمتان من مجموعتين مختلفتين غير قابلين للمقارنة المباشرة.

السالب: مثال حي خارج العينة R^2 python

```python
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score

rng = np.random.default_rng(42)
X_tr = rng.uniform(0, 10, (300, 1)); y_tr = 3 * X_tr[:, 0] + rng.normal(0, 1,
300)
X_te = rng.uniform(15, 25, (100, 1)); y_te = 3 * X_te[:, 0] + rng.normal(0, 1,
100)

tree = DecisionTreeRegressor(random_state=42).fit(X_tr, y_tr)
p = tree.predict(X_te)                    # الشجرة لا تستطيع الاستقراء خارج مجالها
print(f"R² داخل مجال التدريب : {tree.score(X_tr, y_tr):.4f} (خادع بجماله)")
print(f"R² (25-15) خارج المجال: {r2_score(y_te, p):.4f}")
print(f"متوسط الحقيقة {y_te.mean():.2f} مقابل متوسط تنبؤ الشجرة {p.mean():.2f}")
```

الشجرة العالقة عند آخر قيمة رأتها (نحو ٢٩) بينما الحقيقة رحلت نحو ٦١، فصارت تنبؤاتها **أسوأ من التخمين بالمتوسط** وهنا يهبط R^2 إلى سالب ١٣. حذارِ: هذا ليس خطأً برمجياً بل تشخيص صريح بأن النموذج لا يفهم هذا النطاق. وللأهداف متعددة المخرجات يتيح `r2_score` خيار `multioutput` بثلاث صيغ: `raw_values` لكل مخرج على حدة، و `uniform_average` لمتوسط بسيط، و `variance_weighted` لوزن كل مخرج بتباينه.

وأخيرًا عائلة **d^2** (درجة التفسير المقارنة بخط أساس قوي): تُعرف بنسبة ما «فسّره» النموذج من خطأ معياري معيّن، وشكلها العام لمقياس الخطأ المرجعي:

$$d^2 = 1 - \frac{\sum_{i=1}^{n} D (y_i, \hat{y}_i)}{\sum_{i=1}^{n} D (y_i, y_{ref})}$$

حيث `D` دالة الخطأ ويُعاد اختيارها لكل نسخة: `sklearn.metrics.d2_absolute_error_score` تجعل `D` خطأً مطلقاً وخط الأساس الوسيط (نسخة MAE من R^2 — أقوى أمام الشواذ)، و `sklearn.metrics.d2_pinball_score` تستخدم خسارة Pinball عند مستوى فاصل يحدده الوسيط `alpha`، و `sklearn.metrics.d2_tweedie_score` انحراف Tweedie للربط بالعائلة الأسّية في الفصل ٣٣. الدرس: **R^2 لغة مشتركة للجودة، لكنه غير مضاد للشواذ ومقيَّد بمجال تدريبه — وعائلة d^2 تمنحك مقياساً بروح المقاييس القوية.**

٩.٧ خسارة Pinball والانحدار بالفواصل الكمّية

كل ما سبق يتنبأ **بقيمة واحدة** لكل عينة. لكن الأسئلة العملية كثيراً ما تكون عن الفواصل: «ما السعر الذي لن ننزل تحته في ٩٠٪ من الحالات؟» هنا يدخل مقياس يجيب عن كل مستوى فاصل `q` بين ٠ و ١ على حدة — خسارة Pinball:

$$\mathrm{L}_q (y, \hat{y}) = \max \{ q (y - \hat{y}), \; (q - 1) (y - \hat{y}) \}$$

إن متخلف عن القيمة الحقيقية (تنبؤ منخفض) تُعاقَب بعامل `q`، وإن تبخّت فوقها تُعاقَب بعامل `q - 1`. عند `q = 0.5` يصبح العاملان متماثلين بالقيمة المطلقة، فتعود الخسارة نصف MAE — وهذه إشارة جميلة تربط التقدير بالوسيط كما ربط MSE بالمتوسط في الفصل ٢٧. نموذج `sklearn.linear_model.QuantileRegressor` (الذي عرّفناه في الفصل ٣٢ ضمن عائلة النماذج المتسامحة) يقلّل هذه الخسارة مباشرة، و `sklearn.metrics.mean_pinball_loss` يحسبها للتقييم:

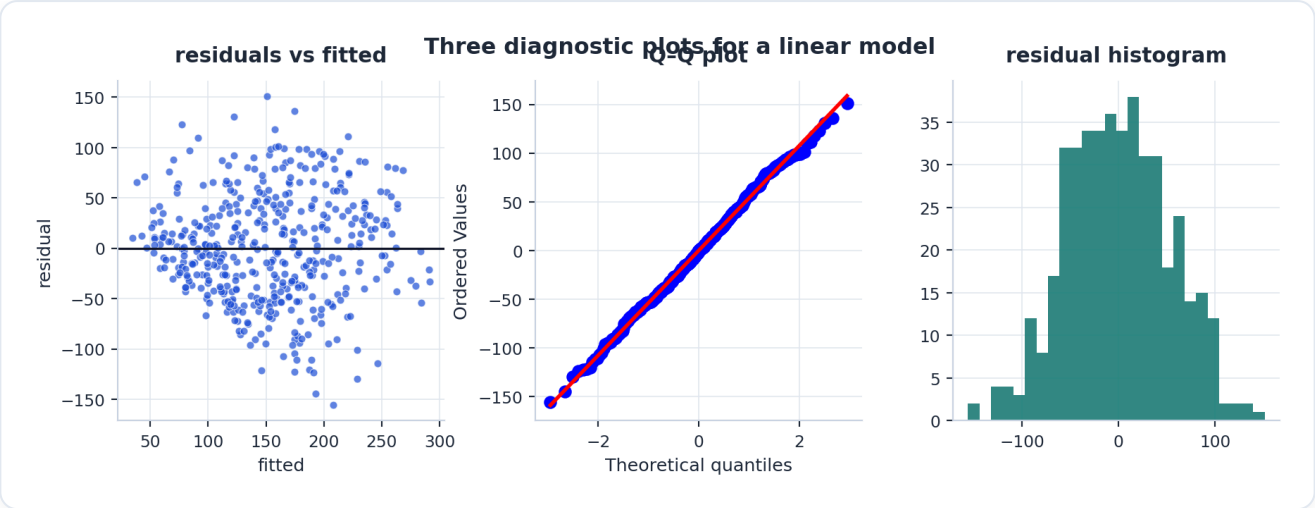
فواصل كمّية: ثلاثة نماذج وتغطية فاصليّة python

```python
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import QuantileRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_pinball_loss, d2_pinball_score

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:900], data.target[:900], test_size=0.3, random_state=42
)

preds = {}
for q in [0.1, 0.5, 0.9]:
    m = QuantileRegressor(quantile=q, alpha=0.01, solver="highs").fit(X_tr,
y_tr)
    p = m.predict(X_te)
    preds[q] = p
    print(f"q={q}: pinball={mean_pinball_loss(y_te, p, alpha=q):.4f} | "
          f"d²_pinball={d2_pinball_score(y_te, p, alpha=q):.4f}")

coverage = np.mean((y_te >= preds[0.1]) & (y_te <= preds[0.9]))
print(f"تغطية الفاصل [q10, q90] = {coverage:.3f} (0.80 المستهدف)")
```

النموذجان عند `q = 0.1` و `q = 0.9` يحفران حدّي نطاق، ويصمد بينهما ٨٠.٧٪ من عينات الاختبار — قريب جداً من المستهدف ٨٠٪، وهذه لغة التقييم الصحيحة للتنبؤ الفاصل: **قميس التغطية، لا نقطة الدقة**. وكلما ضاق الفاصل عند تغطيته المستهدفة كان نموذجك أكثر ثقة، و `d2_pinball_score` يمنحك التفسير النسبي عند كل مستوى فاصل على حدة.

٩.٨ قراءة البواقي وجدول الاختيار

قبل أن تختار مقياساً، انظر إلى بقاياك — فهي تحكّم الخيار قبلك. البقاياء الجيدة تتناثر عشوائياً حول الصفر بشرائط متساوية السماكة؛ أما نمط قمعي (يتسع مع الهدف) فيكشف **تغاير التباين (heteroscedasticity)** الذي يفضّل عنده MAE أو تحويلاً لوغاريتمياً، ونمط منحني يكشف كسراً في الخطية يستوجب نموذجاً أغنى (الفصل ٢٨).



الشكل ١٧٣تشخيص البواقي: التشتت حول الصفر، ومخطط Q-Q، وتوزيعها — ثلاث لمحات قبل اختيار المقياس

في الشكل أعلاه تُقرأ ثلاث لوحات: تشتت البواقي حول محور الصفر (هل هناك نمط؟)، استقامة نقاط Q-Q (هل البواقي شبه طبيعية كما تفترض فترات الثقة؟)، وشكل توزيعها (هل يوجد التواء؟). راجع التشخيص الكامل في الفصل ٢٨.

والآن الجدول الذي يختصر الفصل — علّقه فوق مكتبك:

المقياس	متى تستخدمه	ثغرته القاتلة
MSE / RMSE	التدريب والتواصل العام، حين يكلف البعيد كثيراً	ينفجر مع الشواذ ويجعل النموذج يطارد نقطة قذرة
MAE	شواذ غير قابلة للنمذجة، وتكلفة خطأ خطية	غير قابل للاشتقاق عند الصفر؛ يغفر للبعيد أكثر مما ينبغي أحياناً
median*AE* / *max*error	تقرير «الوضع الطبيعي» واختبار أقصى الأسوأ	لا يصلح هدف تدريب؛ يخفي التوزيع الكامل
MAPE	أهداف موجبة بعيدة عن الصفر وتبغى نسبة مئوية	ينفجر مع القيم الصغيرة والصفيرة؛ لا يقبل الصفر
MSLE / RMSLE	نموّ نسبي: أسعار وعدادات ومعدلات	يشترط غير السالب؛ يعاقب النقص أكثر من الزيادة
R^2	لغة مشتركة بين الفرق ومقارنة النماذج على نفس الهدف	يتأثر بالشواذ من جهتين؛ قد يهبط سالباً خارج المجال
d^2 (عائلة)	تفسير نسبي بخط أساس قوي: وسيط أو فاصل أو Tweedie	أقل شهرة؛ معناها يتغير بتغير دالة الخطأ المرجعية
Pinball loss	التنبؤ بالفواصل الكمّية ومستويات الثقة	مقياس لكل `q` على حدة؛ مقارنة النماذج تحتاج مستويات متعددة

في الواقع

في فرق العمل الحقيقية يُعاد اكتشاف هذا الوجع كل عام: نموذج للتنبؤ بالطلب صُمم على RMSE فأهدر المخزون لأنه يسمح بالتبخّت نزولاً بسخاء، ونموذج تسعير قُيّم بـ MAPE فقط فانكسر على المنتجات القريبة من الصفر. قرار اختيار المقياس قرار منتج — اكتبه صريحاً في وثيقة المشروع مع سببه، فلن يتذكّر أحد بعد أشهر لماذا «يخسر» النموذج الجديد في لوحة القياس بينما يفوز في الواقع.

الدرس الختامي: لا يوجد مقياس محايد؛ يوجد مقياس متفق مع كلفة الخطأ في مشكلتك.

تمارين الفصل الثامن والسبعين

- **ملاحظة**

التمرين ٩.١

على عيّنة من بيانات `fetch_california_housing` (٩٠٠ صفّ تكفي)، قارن بين `LinearRegression` و `DecisionTreeRegressor` خمسة مقاييس: RMSE و MAE و MAPE و R^2 و `d2_absolute_error_score`. هل يقلب مقياسٌ ما ترتيب الفائز؟ اشرح السبب.

التمرين ٩.٢

بدون كود: لماذا يرتفع MAPE إلى قيم غير منطقية عند إضافة مبيعات صغيرة جداً إلى تقييم كان مستقراً؟ وما البديلان الممكنان من هذا الفصل، ومتى يفشل كل منهما؟

التمرين ٩.٣

اكتب دالة `pinball(y, p, q)` بـ NumPy تحسب الخسارة يدوياً، وتحقق مطابقتها لـ `mean_pinball_loss` على بيانات من عندك، ثم أثبت عملياً أن الخسارة عند `q = 0.5` تساوي نصف MAE.

- **ملاحظة**

حل التمرين ٩.١: مقارنة شاملة python

```python
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import (root_mean_squared_error, mean_absolute_error,
    mean_absolute_percentage_error, r2_score, d2_absolute_error_score)

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data[:900], data.target[:900], test_size=0.3, random_state=42
)

rows = []
for name, model in [("Linear", LinearRegression()),
                    ("Tree", DecisionTreeRegressor(random_state=42))]:
    p = model.fit(X_tr, y_tr).predict(X_te)
    rows.append((name,
        root_mean_squared_error(y_te, p), mean_absolute_error(y_te, p),
        mean_absolute_percentage_error(y_te, p), r2_score(y_te, p),
        d2_absolute_error_score(y_te, p)))

print(f"{'نموذج':8s} {'RMSE':>7s} {'MAE':>7s} {'MAPE':>7s} {'R2':>7s}
{'d2abs':>7s}")
for n, rmse, mae, mape, r2, d2 in rows:
    print(f"{n:8s} {rmse:7.3f} {mae:7.3f} {mape:7.3f} {r2:7.3f} {d2:7.3f}")
print("الحكم اختلف: RMSE و R2 للنموذج الخطي، و MAE و MAPE و d2abs للشجرة — المقياس يقلب الترتيب")
```

ملاحظة

python حل التمرين ٩.٣: pinball يدوياً

```python
import numpy as np
from sklearn.metrics import mean_pinball_loss, mean_absolute_error

def pinball(y, p, q):
    e = y - p
    return np.mean(np.maximum(q * e, (q - 1.0) * e))

rng = np.random.default_rng(42)
y = rng.normal(50, 8, 200) + rng.normal(0, 2, 200)
p = y + rng.normal(0, 3, 200)

for q in [0.1, 0.5, 0.9]:
    manual, sk = pinball(y, p, q), mean_pinball_loss(y, p, alpha=q)
    print(f"q={q}: يدوي={manual:.6f} | sklearn={sk:.6f} | متطابقان:
{np.isclose(manual, sk)}")
print(f"علاقة q=0.5: pinball={pinball(y, p, 0.5):.6f} = نصف MAE=
{mean_absolute_error(y, p) / 2:.6f}")
```

> **ملاحظة**

٩.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تمرير `squared=False` إلى `mean_squared_error` في ١.٩	استثناء فوري لأن البارامتر أُزيل نهائياً بعد إهماله	استيراد `root_mean_squared_error` واستخدامه مباشرة
حساب MAPE على هدف يحتوي أصفاراً أو قيماً قريبة منها	نسب فلكية أو قيم غير معرَّفة تدمّر معنى التقييم	استبعاد الصغار أو تحويل لوغاريتمي أو البقاء على MAE
تمرير قيم سالبة إلى `mean_squared_log_error`	استثناء `ValueError` يوقف خط التقييم كله	التحقق من `y >= 0` أو تحويل الهدف قبل الحساب
قراءة R^2 السالب كخلل في الكود	محاولة «إصلاح» نموذج سليم لأن الرقم سلبي	تفسيره: النموذج أسوأ من خط أساس المتوسط على هذا المجال

٩.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البواقي	الفرق بين القيمة الحقيقية والمتوقعة لكل عينة، مادة كل مقاييس الانحدار	Residuals
خسارة Pinball	خسارة غير متماثلة تعاقب الانحراف بعامل `q` من جهة و `q-1` من الأخرى	Pinball loss
الانحدار الكمّي	تدريب نموذج على تقدير فاصل مشروط بدل المتوسط الشرطي	Quantile regression
تغاير التباين	اتساع نطاق البواقي مع نمو الهدف، يفضّل عنده المقياس القوي	Heteroscedasticity
درجة التفسير d^2	نسبة الخطأ المفسَّر مقارنة بخط أساس مرجعي كالوسيط أو فاصل	D^2 regression score

الجزء ٩ · التقييم والاختيار

الفصل ٧٩ — مقاييس متقدّمة ومجموعات غير مستوية

متعدّد التسميات، والترتيب، وتوصيات DCG، ومعايير التغطية

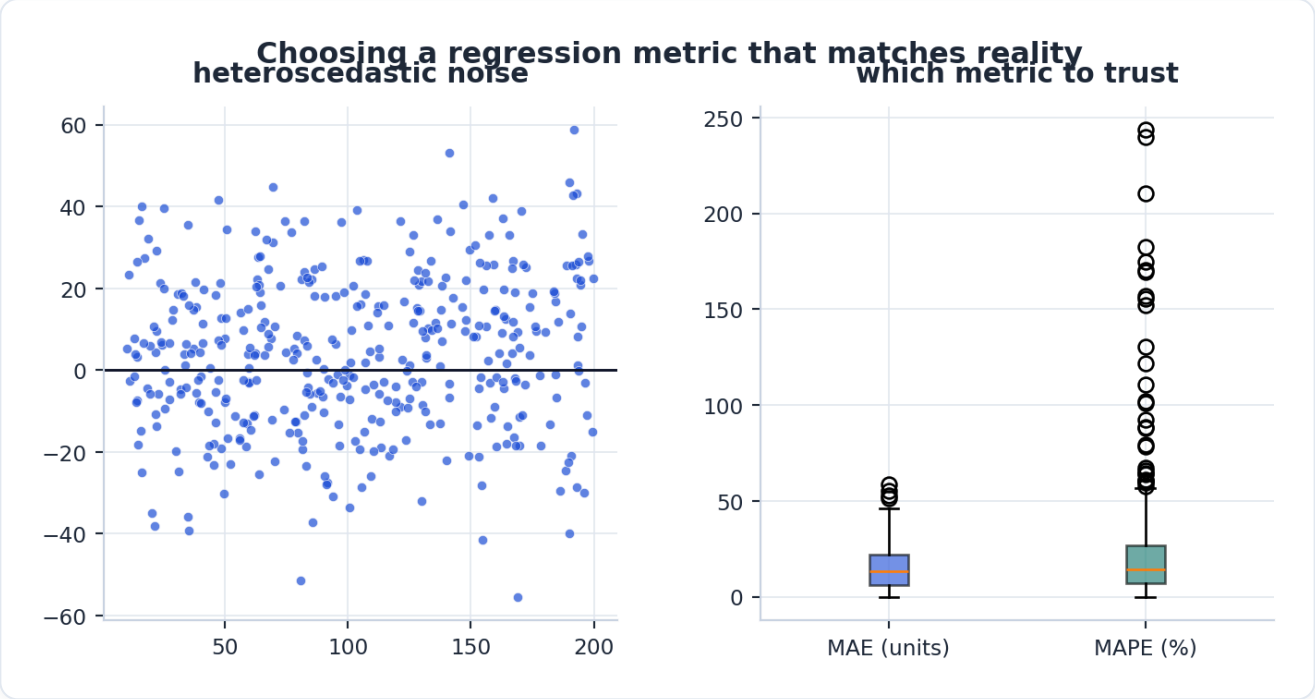
الوحدة **scikit-learn ١.٩** الحجم **٩١٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تقيّم مشكلات متعددة التسميات بأربع عدسات: subset و hamming و jaccard و FI
- أن تحسب DCG و NDCG وتقرأ الخصم اللوغاريتمي مع k= على قائمة توصيات
- أن تقيّم ترتيب التسميات بـ LRAP و label_ranking_loss و coverage_error
- أن تميّز بين دقة القرار ودقة الاحتمال عبر brier_score_loss
- أن تكشف خدعة الدقة في الفئات غير المتوازنة بـ balanced accuracy و MCC و top-k

٩.١ حين لا تكفي فئة واحدة لكل عينة

راجعت أداة تجميع الصور ألف صورة عائليّة، ثم سألتك سؤالاً محرجاً: ما الدقة؟ في صورة تحوي قطاً وكلباً وأرنباً، وضعتها الأداة في «قط» فقط — هل هذا خطأ واحد من ثلاثة أبواب، أم نجاح جزئي، أم فشل كامل؟ مقاييس التصنيف المستوي — الخلط والدقة والاسترجاع — رأيناها بالتفصيل في الفصل ٧٥؛ هنا نخرج إلى **التصنيف متعدد التسميات (multi-label)**: كل عينة قد تحمل عدة فئات معاً، ويصبح `y_true` مصفوفة ثنائية بشكل `(n_samples, n_labels)` بدل متجه. ومن هنا تنبع أربع عدسات مختلفة للسؤال نفسه.



الشكل ١٧٤ اختيار المقياس الذي يطابق طبيعة الخطأ في مهمتك — من الانحدار النسبي إلى عدّ الأبواب

الشكل أعلاه يذكّرك بالدرس الذي بدأنا به الكتاب: المقياس يختار زاوية نظره إلى الخطأ قبل أن يقيسه — والزوايا هنا ستصبح أكثر تنوعاً.

العدسة الأولى هي **دقة المجموعة الجزئية (subset accuracy)**: لا تحتسب عينة ناجحة إلا إذا صابت **كل** أبوابها. العدسة الثانية `hamming_loss` تحسب نسبة الأبواب الخاطئة من إجمالي الأبواب — اللين الرحيم. والثالثة `jaccard_score` تقيس تقاطع مجموعة التنبؤ مع الحقيقة على اتحادهما، لكل عينة ثم بمتوسط. والرابعة F1 بمتوسطها المفضل لديك. لنحسمها بأرقام على ثلاث عينات وثلاثة أبواب:

ثلاث عدسات على ثلاث عينات متعددة التسميات python

```python
import numpy as np
from sklearn.metrics import (accuracy_score, hamming_loss, jaccard_score,
f1_score)

y_true = np.array([[1, 1, 0], [0, 1, 1], [1, 0, 0]])
y_pred = np.array([[1, 0, 0], [0, 1, 1], [1, 0, 1]])

print(f"subset accuracy = {accuracy_score(y_true, y_pred):.4f}  (عينة واحدة صابت كل
أبوابها)")
print(f"hamming_loss    = {hamming_loss(y_true, y_pred):.4f}
({int(hamming_loss(y_true, y_pred) * 9)} 9 باب خاطئ من)")

manual = [np.sum(t & p) / np.sum(t | p) for t, p in zip(y_true, y_pred)]
print(f"jaccard يدوياً لكل عينة: {np.round(manual, 2)} → متوسط =
{np.mean(manual):.4f}")
print(f"jaccard المكتبة        : {jaccard_score(y_true, y_pred,
average='samples'):.4f}")

print(f"F1 macro   = {f1_score(y_true, y_pred, average='macro',
zero_division=0):.4f}  (وزن متساوٍ لكل باب)")
print(f"F1 samples = {f1_score(y_true, y_pred, average='samples',
zero_division=0):.4f}  (وزن متساوٍ لكل عينة)")
print(f"F1 micro   = {f1_score(y_true, y_pred, average='micro',
zero_division=0):.4f}  (تجميع الأعداد الكلية)")
```

اقرأ الناتج بعين المحقق: العينة الثانية مثالية، والأولى فقأت باباً واحداً من اثنين، والثالثة أخطأت باباً لم تُطلب أصلاً. لهذا تتفرق العدسات: subset قاسية (٠.٣٣)، hamming لينة (٠.٢٢)، و jaccard و F1 بينهما (٠.٦٧ و ٠.٧٨). و `average=` يقرر وزن الإخفاق: `macro` يساوي بين الأبواب فيهم الباب النادر، و `samples` يساوي بين العينات فيهم العينة عديمة التسميات، و `micro` يجمع الأعداد قبل الحساب فيجامل الأبواب المشترِكة.

مفتاح

في المهام متعددة التسميات لا يوجد «المقياس الصحيح» بل **سؤال العمل الصحيح**: هل خطأ باب واحد يفسد العينة كلها (مراقبة أمان)؟ فـsubset وحدك. هل نريد صورة عادلة للأداء اليومي؟ hamming أو jaccard. وهل يهمنا الباب النادر تحديداً؟ macro مع تدقيق `zero_division` حين توجد عينات بلا تسميات.

٩.٢ توصيات وترتيب: DCG و NDCG والخصم اللوغاريتمي

في محرك التوصيات لا يسأل المستخدم «هل القائمة صحيحة؟» بل يلمس أعلى القائمة فقط. النتيجة (relevance gain) للبند الثامن أقل ثمناً من البند الأول، وهنا يأتي **الخصم اللوغاريتمي**: المكسب يقسم على لوغاريتم المرتبة فيهبط ببطء متجانس، فلا يختفي البند العاشر فجأة ولا يبقى مساوياً للأول. خصم البند في المرتبة `i` :

$$\mathrm{DCG} = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

حيث `rel_i` درجة صلة البند في المرتبة `i` (صفر يعني بلا صلة، و ٣ يعني ضرورياً)، والقوة الأسية `2^rel - 1` تجعل «الضروري» أغلى بكثير من «مفيد». لكي تصبح الدرجة قابلة للمقارنة بين قوائم مختلفة نقسم على مثالية القائمة — ترتيب البنود تنازلياً بالصلة الحقيقية — فيظهر **NDCG** بين صفر و واحد:

$$\mathrm{NDCG} = \frac{\mathrm{DCG}}{\mathrm{IDCG}}$$

في `sklearn.metrics` توفر `dcg_score` و `ndcg_score` الحساب من درجات تنبؤ متصلة (`y_score`) تُفرز داخلياً، والوسيط `k` يقصّ القائمة عند أول `k` بند كما يفعل واجهة تطبيق حقيقي. لنشاهد خصماً بنداً بنداً على قائمة توصيات من ستة عناصر:

خصم اللوغاريتم بنداً بنداً وقصّ القائمة بـ k — python

```python
import numpy as np
from sklearn.metrics import ndcg_score, dcg_score

rel    = np.array([[3., 2., 3., 0., 1., 2.]])   # الصلة الحقيقية لكل بند
scores = np.array([[0.90, 0.80, 0.50, 0.85, 0.30, 0.60]])  # تنبؤ النموذج

order  = rel[0][np.argsort(-scores[0])]          # ترتيب النموذج
ideal  = np.sort(rel[0])[::-1]                   # القائمة المثالية
disc   = 1 / np.log2(np.arange(1, 7) + 1)        # عامل الخصم لكل مرتبة
print(f"ترتيب النموذج   : {order}")
print(f"القائمة المثالية: {ideal}")
print(f"عوامل الخصم     : {np.round(disc, 3)}")

dcg_pred = np.sum((2**order - 1) * disc)
dcg_ideal = np.sum((2**ideal - 1) * disc)
print(f"DCG المتنبئ = {dcg_pred:.4f} مقابل IDCG = {dcg_ideal:.4f}")
print(f"NDCG = {ndcg_score(rel, scores):.4f}  |  DCG غير المطبَّعة = {dcg_score(rel,
scores):.4f}")
print(f"NDCG@3 (قصّ عند 3 بنود كواجهة التطبيق) = {ndcg_score(rel, scores, k=3):.4f}")
```

القصة كاملة في الأرقام: النموذج وضع بنداً بلا أي صلة (`rel = 0`) في المرتبة الثانية، فخسر مكسب ٧ كان سيُخصم بـ ٠.٦٣ فقط لو صعد للمرتبة الأولى. النتيجة NDCG كليّة محترمة ٠.٨٩، لكن NDCG@٣ عند أول ثلاثة بنود التي يراها

المستخدم فعلاً هبطت إلى ٠.٦٨ — وهو بالضبط الوجع الذي يحسّنه فريق المنتج: **الجودة الإجمالية تخفي كارثة في أول الشاشة.** سنعيد بناء محرك كامل بهذا المقياس في الفصل ١٠٣.

> **نصيحة**
>
> اختر `k` من واجهة منتجك لا من خيالك: قائمة تشغيل بعشرة بنود تقاس بـ `k=10` ولو بلغ NDCG الكلي ٠.٩٥؛ وما لا يُعرض على المستخدم لا ينبغي أن يُحتسب له مكسباً.

٩.٣ ترتيب التسميات نفسها: LRAP وخسارة الزوج المقلوب والتغطية

بين التصنيف والترتيب منطقة ثالثة: لدينا تسميات صحيحة متعددة للعينة، ونريد معرفة هل يضع النموذج التسميات الصحيحة **أعلى قائمة احتمالاته**. ثلاثة مقاييس من `sklearn.metrics` تجيب بثلاث نبرات:

ثلاثة مقاييس ترتيب على ثلاث عينات — python

```python
import numpy as np
from sklearn.metrics import (label_ranking_average_precision_score,
    label_ranking_loss, coverage_error)

y_true  = np.array([[1, 0, 1, 0], [0, 1, 1, 0], [1, 1, 0, 0]])
y_score = np.array([[0.90, 0.10, 0.80, 0.30],
                    [0.20, 0.90, 0.70, 0.10],
                    [0.30, 0.85, 0.60, 0.20]])

print(f"LRAP            = {label_ranking_average_precision_score(y_true,
y_score):.4f}")
print(f"ranking_loss    = {label_ranking_loss(y_true, y_score):.4f}  (زوج مقلوب
12 من واحد)")
print(f"coverage_error  = {coverage_error(y_true, y_score):.4f}  (متوسط عمق أول
صحيح)")

# العينة الثالثة يدوياً: ترتيب الدرجات [1](0.85)، 2(0.60)، 0(0.30)، 3(0.20)
# التسميات الصحيحة {0, 1}: الأولى صحيحة (دقة 1/1)، والثانية عند العمق 3 (دقة 2/3)
print("2 / (2/3 + 1.0) ,"= 2 / (2/3 + 1/1) :(دقة العينة الثالثة يدوياً")
```

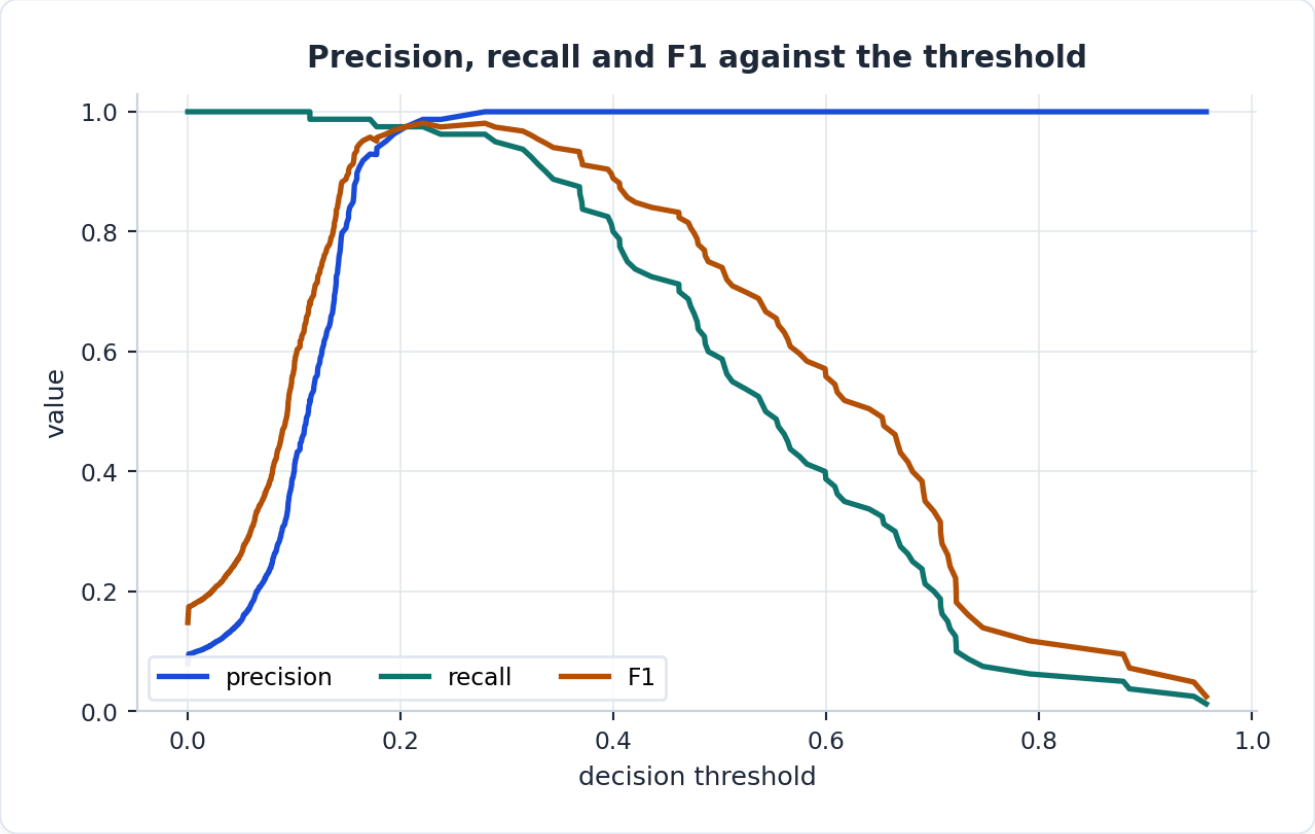
العينتان الأوليان مثاليتا الترتيب، والثالثة دفعت الثمن: تسمية صحيحة (الباب ٠) سقطت خلف باب خاطئ (الباب ٢) فولّدت زوجاً مقلوباً واحداً وخفضت LRAP إلى ٠.٩٤ ورفعت التغطية إلى ٣ لهذه العينة. لاحظ تكامل النبرات الثلاث: LRAP يكافئ، والخسارة تعاقب، والتغطية تجيب سؤال تشغيلي صريحاً.

٩.٤ الدقة الاحتمالية: Brier قبل أي عتبة

كل مقاييس الفصول السابقة تحكم على **قرار** بعد عتبة. لكن النموذج يخرج أولاً احتمالاً — وربما نشرت هذا الاحتمال كما هو (سعر مخاطرة، احتمال مطر، احتمال سداد قرض) دون أي عتبة. درجة برير تقيس متوسط مربّع مسافة احتمالك عن الحقيقة الصفرية/الواحدية:

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C}(y_{ic} - \hat{p}_{ic})^2$$

حيث `y_ic` يساوي ١ إذا كانت العينة من الفئة `c` وإلا صفراً، و `\hat{p}_{ic}` الاحتمال المتوقع. وهي قاعدة تسجيل صحيحة تماماً (strictly proper): لا يمكن لأي نموذج أن يقلّلها بغير إعلان احتمالاته الحقيقية — فتقتل المبالغة في الثقة. وهي حليف منفصل عن مقاييس العتبة التي تحرّك مع تغيّرها:



الشكل ١٧٥ الدقة والاسترجاع و F1 تتحرك جميعها مع انزياح عتبة القرار — بينما Brier يحكم قبل ذلك كله

في الشكل أعلاه تلاحق المنحنيات بعضها كلما تحركت العتبة، ولا يمكنك تحسين الثلاثة معاً؛ أما Brier فيقيّم الاحتمال نفسه قبل اختيار أي نقطة على هذه المنحنيات.

```python
# الدقة تخدع و Brier يفصح: لوجستي ضد شجرة عميقة
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import brier_score_loss, accuracy_score

X, y = make_classification(n_samples=1200, n_features=10, n_informative=5,
                           class_sep=0.6, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

for name, model in [("لوجستي", LogisticRegression(max_iter=1000)),
                    ("شجرة عميقة", DecisionTreeClassifier(random_state=42))]:
    model.fit(X_tr, y_tr)
    proba = model.predict_proba(X_te)[:, 1]
    print(f"{name}: دقة القرار = {accuracy_score(y_te, proba >= 0.5):.4f} | "
          f"Brier = {brier_score_loss(y_te, proba):.4f}")
```

المفارقة التي يستحق الناتج أن تُحفظ: الشجرة العميقة **فازت بالدقة** (٠.٧٢٥٠ ضد ٠.٦٤١٧) وخسرت في Brier (٠.٢٧٥٠ ضد ٠.٢٢٨٩) — احتمالاتها قفزات ٠ و ١ متطرفة، فتخسر كثيراً حين تُعلن يقيناً في مسألة غير مؤكدة. إن كنت تستهلك الاحتمال مباشرة أو تعايره (كما سنفعل عند ضبط العتبات في الفصل ٧٧)، قِس بـ Brier لا بالدقة.

> **ملاحظة**
>
> منذ الإصدار ١.٧ يتلقى `brier_score_loss` الوسيط `scale_by_half` بقيمة افتراضية `auto`: في التصنيف الثنائي تُقاس النتيجة بنطاق `[0, 1]` المألوف، وفي المتعدد الفئات يبقى النطاق `[0, 2]` حسب التعريف الأصلي — انتبه للمقارنة بين مهام مختلفة.

٩.٥ الفئات غير المتوازنة: balanced accuracy و top-k و MCC

المهمة الأكثر إيلاماً في العمل اليومي: فئة نادرة تجذب كل الاهتمام (احتيال، انصراف عميل، مرض) ودقة تتصاعد زوراً. **balanced*accuracy*score** يفكّ الدقة إلى متوسط الاستدعاء لكل فئة، فتصبح الفئة المهملة وصماً عالياً في التقرير. و **top*k*accuracy_score** يعترف بأن القرار النهائي يتخذه إنسان يرى قائمة مرشحين، و **matthews_corrcoef** يقيس الارتباط بين الحقيقة والتنبؤ كما يقيس معامل بيرسون ارتباطاً — صفر يعني لا علاقة، وسالب يعني تنبؤاً معكوساً؛ صيغته الثنائية مرّت في الفصل ٧٥، وهنا نستدعيها على عشر فئات مباشرة، فهي من أصدق المقاييس حين تتوازن الفئات الأربع خلطياً في الكتان.

ثلاث فئات غير متوازنة: الدقة الزائفة وعلاجها python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, balanced_accuracy_score,
matthews_corrcoef)

X, y = make_classification(n_samples=1500, n_classes=3, weights=[0.80, 0.15,
0.05],
                           n_features=10, n_informative=6, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)
print(f"أعداد الفئات في الاختبار: {np.bincount(y_te)}")

for name, model in [("عادي", LogisticRegression(max_iter=1000)),
                    ("فئات موزونة", LogisticRegression(max_iter=1000,
class_weight="balanced"))]:
    p = model.fit(X_tr, y_tr).predict(X_te)
    print(f"{name}: دقة = {accuracy_score(y_te, p):.4f} | "
          f"متوازنة = {balanced_accuracy_score(y_te, p):.4f} | "
          f"MCC = {matthews_corrcoef(y_te, p):.4f}")
```

هذا هو «مفارقة الدقة» بالأرقام: النموذج العادي يتباهى بدقة ٠.٨٢ بينما متوسط استدعاءاته ٠.٤٤ فقط — أهمل الفئة النادرة شبه كلياً وواصل إصابة الأغلبية. الموازنة خفّضت الدقة الظاهرة إلى ٠.٦١ ورفعت الأداء المتوازن إلى ٠.٦١: بداية أسوأ، عدالة أفضل. والآن مقياسا القائمة القصيرة على بيانات الأرقام المكتوبة:

على الأرقام المكتوبة MCC و top-k python

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (top_k_accuracy_score, matthews_corrcoef,
    balanced_accuracy_score)

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)
model = LogisticRegression(max_iter=2000).fit(X_tr, y_tr)
proba = model.predict_proba(X_te)
pred = model.predict(X_te)

print(f"top-1 = {top_k_accuracy_score(y_te, proba, k=1):.4f}")
print(f"top-3 = {top_k_accuracy_score(y_te, proba, k=3):.4f}")
print(f"balanced accuracy = {balanced_accuracy_score(y_te, pred):.4f}")
print(f"MCC (عشر فئات) = {matthews_corrcoef(y_te, pred):.4f}")
```

الفارق بين ٠٫٩٥٧٤ و ١٫٠٠٠٠ هو بالضبط ما يراه كاتبٌ بشريٌّ أمام قائمة من ثلاثة ترشيحات: الخطأ الذي يقفز من العين غالباً هو الترشيح الثاني لا الثالث. أما MCC المرتفع ٠٫٩٥٢٧ على عشر فئات فيعني أن التنبيه يتحرك مع الحقيقة في كل خلايا الخلط، لا في القطر الرئيسي وحده.

> **خطأ شائع**
>
> تذكّر أن `top_k_accuracy_score` تشترط تمرير `predict_proba` وليس `predict` — وتمرير درجات `decision_function` في مسألة متعددة الفئات قد يعطيك نتيجة بلا معنى إن لم تكن قابلة للمقارنة بين الفئات. ولأن `k` الافتراضي هو ٢، مرّره صريحاً دوماً حتى لا تُقيِّم بلا قصد على «أفضل تصنيفين».

٩.٦ الجدول الموحّد: المهمة تحدد المقياس

حين تتفرع مهمتك عن التصنيف المستوي، ابدأ من هذا الجدول لا من الذاكرة:

المهمة	المقياس المناسب	رجل الأخطاء الذي يجب أن تعرفه
متعدد التسميات — أمان صارم	`accuracy_score` على المؤشر (subset)	خطأ باب واحد يُسقط العينة كلها؛ دقة تبدو متدنية ظلماً
متعدد التسميات — صورة عادلة	`hamming_loss` / `jaccard_score`	اللين قد يخفي إهمال باب نادر كلياً
متعدد التسميات — باب نادر	`f1_score(average='macro')`	عينات بلا تسميات ترفع `UndefinedMetricWarning` إلا مع `zero_division`
ترتيب وتوصيات	`ndcg_score` مع `k` من الواجهة	NDCG الكلي يخفي كارثة في أول الشاشة
ترتيب التسميات الداخلية	`label_ranking_average_precision_score`	حساس لعمق التسمية الصحيحة الثانية والثالثة
تغطية واجهة «أول N نتيجة»	`coverage_error`	لا يخبرك عن جودة ما فوق العمق المطلوب
استهلاك الاحتمال مباشرة	`brier_score_loss`	الدقة قد تفضل نموذجاً احتمالاته سيئة جداً
فئات غير متوازنة	`balanced_accuracy_score` / MCC	الدقة الخام تصعد مع حجم الأغلبية لا مع الجودة
قائمة مرشحين للبشر	`top_k_accuracy_score`	افتراضي `k=2` صامت؛ مرر `proba` لا `predict`

في الواقع

فرق التوصيات في المتاجر الكبرى تُدار عملياً بمقاييس هذا الفصل: لوحة قياس من NDCG@١٠ على الجلسات الحقيقية، و coverage_error على أسئلة الدعم «هل ظهر الجواب الصحيح في أول ثلاث نتائج؟»، و Brier على احتمالات إتمام الشراء التي تُغذي المزايا الإعلانية. ومن يبني نظام تصنيف متعدد التسميات لوسم المقالات سيكتشف خلال أسبوع أن subset accuracy المقفرة دفعت الفريق لتحسينات حقيقية — وأن hamming هو ما يقرأه الإدارة فعلًا. المقياس ليس تفصيلاً تقنياً؛ إنه اللغة التي يتفاوض بها الفريق على معنى «نجاح».

تمارين الفصل التاسع والسبعين

- **ملاحظة**

التمرين ٩.١

ولّد مهمة متعددة التسميات بـ `make_multilabel_classification` (٦٠٠ عينة، ٤ أبواب)، ودرّب `KNeighborsClassifier` عليها، ثم قارن subset accuracy و hamming*loss* و `f1_score(average='samples')` و jaccard. رتّب المقاييس من الأشد إلى الألطف وفسّر.

التمرين ٩.٢

على مثال التوصيات في القسم ٩.٢، احسب NDCG عند `k` من ١ إلى ٦ وقارن بـ NDCG الكلي. عند أي `k` تنكشف مشكلة الترتيب بأكملها تقريباً؟

التمرين ٩.٣

قارن `brier_score_loss` لثلاثة نماذج على بيانات `make_classification` المذكورة: `DummyClassifier(strategy='prior')`، واللوجستي، والشجرة العميقة. من الأجدر نظرياً أن يقترب من احتمال ٠.٥ الأعمى؟ وهل صدّقت الأرقام؟

- **ملاحظة**

حل التمرين ٩.١: أربع عدسات على مهمة ولّدتها python

```python
from sklearn.datasets import make_multilabel_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, hamming_loss, jaccard_score,
f1_score)

X, Y = make_multilabel_classification(n_samples=600, n_features=12,
                                      n_classes=4, n_labels=2, random_state=42)
X_tr, X_te, Y_tr, Y_te = train_test_split(X, Y, test_size=0.3, random_state=42)
P = KNeighborsClassifier(n_neighbors=5).fit(X_tr, Y_tr).predict(X_te)

print(f"subset accuracy (الأشد): {accuracy_score(Y_te, P):.4f}")
print(f"jaccard samples        : {jaccard_score(Y_te, P, average='samples',
zero_division=0):.4f}")
print(f"F1 samples             : {f1_score(Y_te, P, average='samples',
zero_division=0):.4f}")
print(f"hamming_loss (الألطف)  : {hamming_loss(Y_te, P):.4f}")
print("الترتيب متوقع: subset و يشترط صواب كل الأبواب، hamming يغتفر الأغلبية")
```

> **ملاحظة**

حل التمرين ٩.٣: Brier للأعمى واللوجستي والشجرة python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import brier_score_loss

X, y = make_classification(n_samples=1200, n_features=10, n_informative=5,
                           class_sep=0.6, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=42, stratify=y)

models = [("dummy (أعمى)", DummyClassifier(strategy="prior")),
          ("لوجستي", LogisticRegression(max_iter=1000)),
          ("شجرة عميقة", DecisionTreeClassifier(random_state=42))]
for name, m in models:
    proba = m.fit(X_tr, y_tr).predict_proba(X_te)[:, 1]
    print(f"{name}: Brier = {brier_score_loss(y_te, proba):.4f}")
print("الأعمى حصل على 0.2500 بالضبط — ولا نموذج يتجاوزه حقاً إلا بتعلّم حقيقي")
```

> **ملاحظة**

٩.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
`f1_score(average='samples')` مع عينات بلا تسميات	`UndefinedMetricWarning` وقيم صفرية مفاجئة	تمرير `zero_division=0` صراحة ومراجعة العينات الفارغة
تمرير ناتج `predict` إلى `top_k_accuracy_score`	فشل أو نتيجة بلا معنى لأن المقياس يحتاج درجات لكل فئة	تمرير `predict_proba` وتحديد `k` صراحة
مقارنة Brier بين مهمة ثنائية وأخرى متعددة الفئات	نطاقات مختلفة `[0,1]` و `[0,2]` تضلّل الحكم	الانتباه لسلوك `scale_by_half='auto'` الافتراضي
الحكم على نظام توصيات بـ NDCG الكلي وحده	جودة إجمالية مرتفعة تخفي ترتيباً كارثياً في أول الشاشة	تقارير NDCG@k عند عمق الواجهة الفعلي

٩.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التصنيف متعدد التسميات	مهمة قد تحمل كل عينة فيها عدة فئات معاً في متجه مؤشرات	Multi-label classification
الخصم اللوغاريتمي	تقليص مكسب البند مع مرتبته بقسمة على لوغاريتم الموقع	Logarithmic discount
الدرجة المطبَّعة المخصومة	DCG مقسوماً على مثاليّ القائمة لتقع بين صفر و واحد	Normalized DCG (NDCG)
قاعدة التسجيل الصحيحة	مقياس لا يمكن خداعه إلا بالإعلان الصادق عن الاحتمالات الحقيقية	Proper scoring rule
دقة المجموعة الجزئية	لا تُحتسب عينة ناجحة إلا إذا صابت كل تسمياتها	Subset accuracy

الجزء ٩ · التقييم والاختيار

الفصل ٨٠ — منحنى التعلّم ومنحنى التحقق

قراءة تشخيصية للإفراط في الملاءمة والتباين والتحيّز

الوحدة scikit-learn ١.٩ الحجم ٦٢٤ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن تقرأ منحنى تعلّم وتحديد النمط: فرط ملاءمة، أو انحياز عالٍ، أو تقارب مثالي
- أن تختار العلاج الصحيح لكل نمط بدل رمي ميزات وحجم بيانات عشوائياً
- أن تمسح بارامتراً واحداً بـ validation_curve وتحدد نقطة الانعطاف
- أن تقرأ هضبة منحنى التعلّم لتقرر متى يتوقف شراء البيانات عن الجدوى
- أن تضبط shuffle و random_state و cv و scoring في الدالتين بوعي

٩.١ التشخيص قبل العلاج: أداتا النظر

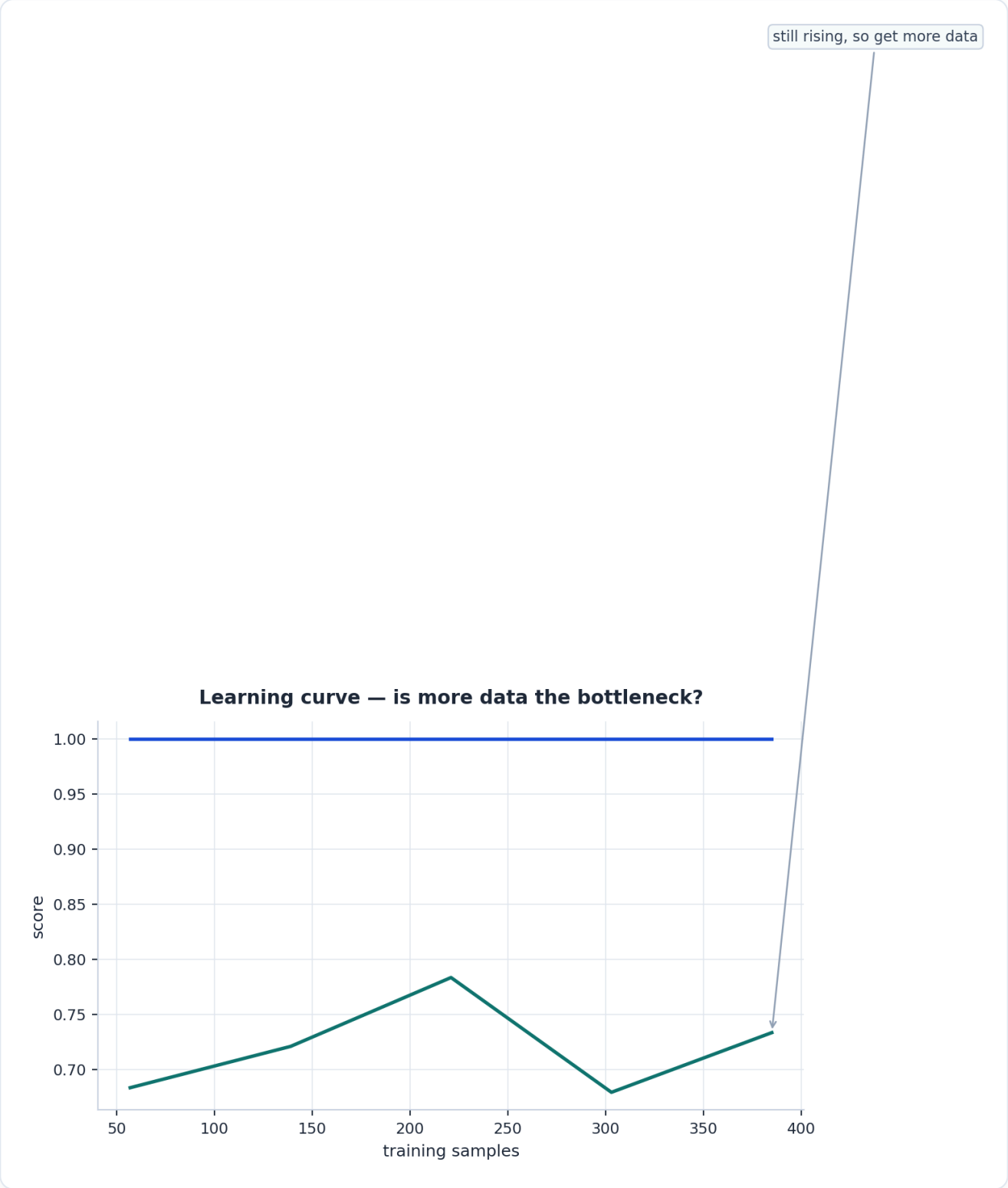
وصلتك رسالة صباحية من زميلك: «النموذج لا يؤدي. أضفت ألف عينة جديدة ولم يتحسن شيء». المشكلة الكلاسيكية هنا ليست في النموذج بل في التشخيص: هو يعالج داءً ربما لم يصبه المريض. الانحياز العالي (high bias) لا يعالَج ببيانات إضافية أبداً، والتباين العالي (high variance) لا يعالَج بنموذج أغنى، والخلط بينهما يضيّع أسابيع. أداتا النظر للتمييز بينهما جاهزتان في `sklearn.model_selection`:

كلتاهما تعيدان مصفوفتي درجات: تدريب و(تحقق متقاطع)، بالشكل `(n_sizes, n_folds)` لأحجام التدريب و `(n_values, n_folds)` لقيم البارامتر، ومع `cv` عدد صحيح يستخدم التصنيف تقسيماً طبقياً تلقائياً كما رأينا في الفصل ٧٣. والدرجة التي تقرأها في الرسوم هي المتوسط عبر الطيات.

مفتاح

منحنى التعلّم يحرك **حجم البيانات** ويقيّم النموذج الثابت؛ ومنحنى التحقق يثبّت البيانات ويحرك **بارامتراً واحداً**. الأول يجيب «هل أجلب المزيد؟» والثاني يجيب «أين أضبط هذا المقبض؟». من يخلط بين السؤالين يشتري بيانات لمشكلة ضبط، أو يضبط بارامترات لمشكلة بيانات.

٩.٢ منحنى التعلّم على ثلاثة سيناريوهات كاملة



الشكل ١٧٦ منحنى تعلّم لشجرة قرار: درجة التحقق لا تزال تصعد مع الحجم — فالمزيد من البيانات مفيد

في الشكل أعلاه يفصل منحنى التدريب عن منحنى التحقق، ولا يزال الأخير يصعد عند آخر نقطة — لافتة تقول «اشترِ بيانات». لكن هذا ليس النمط الوحيد الممكن. لنبنِ المشهد كاملاً على `make_classification` بأربع عشرة ميزة منها سبع ضجيجية فقط، بثلاثة نماذج مختلفة المزاج، ونقرأ الأرقام:

```python
# ثلاثة نماذج، ثلاثة أنماط على make_classification
import numpy as np
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import learning_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=800, n_features=14, n_informative=7,
                           n_redundant=3, class_sep=0.55, random_state=42)
sizes = np.linspace(0.1, 1.0, 6)

models = [("شجرة عميقة", DecisionTreeClassifier(random_state=42)),
          ("لوجستي", make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000))),
          ("غابة", RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1))]

for name, est in models:
    tr, trs, tes = learning_curve(est, X, y, train_sizes=sizes, cv=5,
                                  scoring="accuracy")
    a, b = trs.mean(axis=1), tes.mean(axis=1)
    print(f"{name}: أول نقطة train={a[0]:.3f} val={b[0]:.3f} | "
          f"آخر نقطة train={a[-1]:.3f} val={b[-1]:.3f} | فجوة نهائية={a[-1]-b[-1]:.3f}
| "
          f"قمة التحقق={b.max():.3f}")
```

ثلاث قصص مختلفة تماماً في ناتج واحد. **الشجرة العميقة**: درجة تدريب عالقة عند ١.٠٠٠ من أول نقطة (لماذا؟ لأنها عميقة بما يكفي لحفظ أي مجموعة تدريب، كما شرحنا في الفصل ٣٩) بينما تزحف درجة التحقق من ٠.٥٩٩ إلى ٠.٧٠٩ بفجوة من ٠.٤٠ إلى ٠.٢٩ تضيق ببطء شديد — فرط ملاءمة صريح. **اللوجستي**: منحنى التدريب يهبط من ٠.٧٣٧ ليلاقي منحنى التحقق عند ٠.٧٠ تقريباً بفجوة تكاد تُمحى، لكن السقف منخفض (٠.٦٨٤) — انحياز عالٍ: نموذج خطي أمام بنية لا يفسرها خط. **الغابة**: تزحف درجة تحققها من ٠.٦٥٧ إلى ٠.٨٣١ ولا تزال تصعد — أقرب ما يكون إلى المثالي هنا، والمزيد من البيانات سيفيدها فعلاً.

$$\text{Gap}(n) = \text{Score}_{train}(n) - \text{Score}_{val}(n)$$

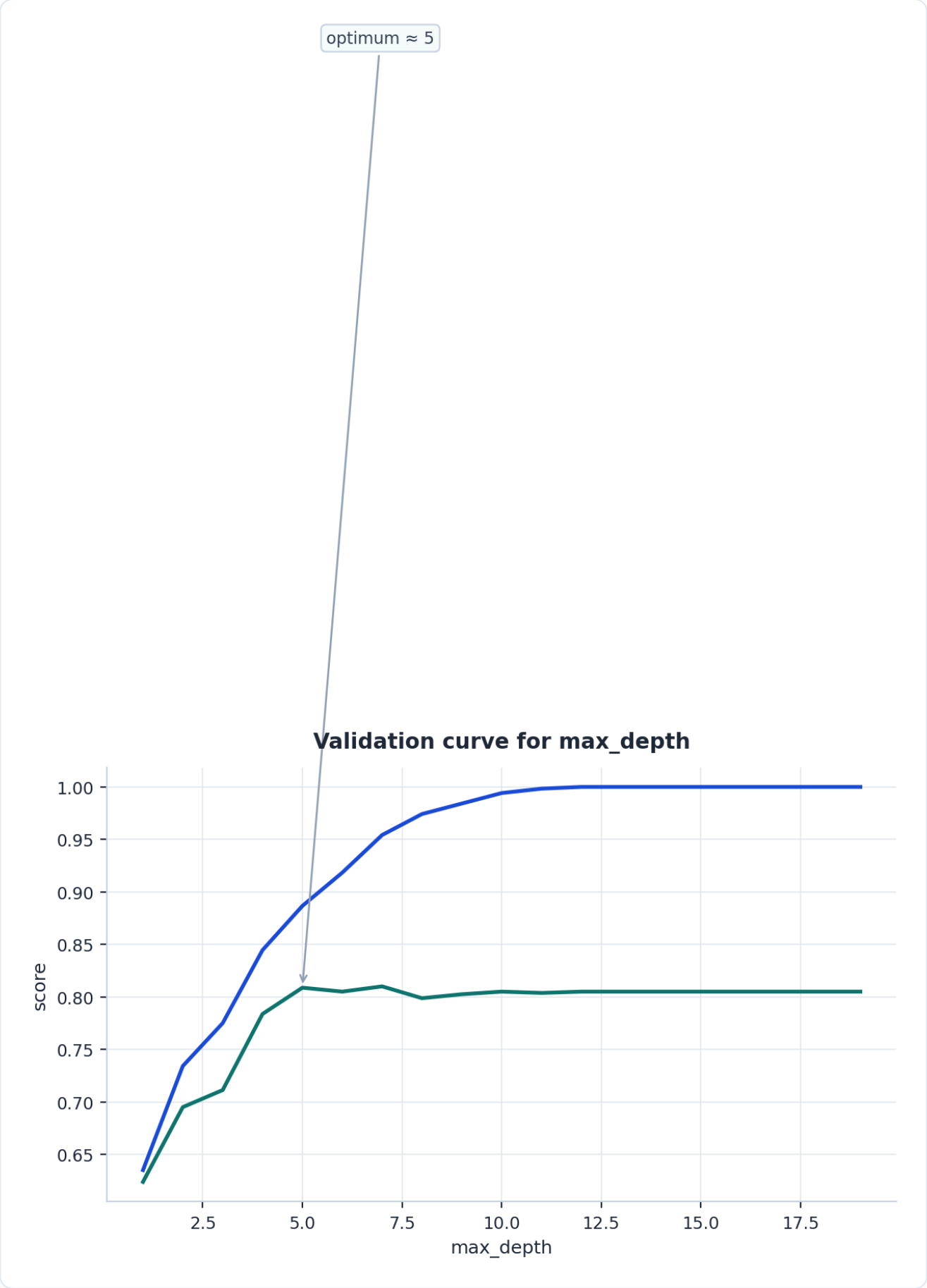
الفجوة أعلاه بين متوسط درجة التدريب ومتوسط درجة التحقق عند حجم عينة `n`؛ فهي مقياس التباين، بينما ارتفاع القيمة المشتركة للمنحنيين الملتصقين هو مقياس الانحياز. والآن قاعدة العلاج المقابلة لكل نمط:

النمط	العلامة بالأرقام	العلاج الصحيح	علاج خاطئ شائع
فرط ملاءمة	تدريب شبه ١.٠ وفجوة ٠.١٠+ تضيق ببطء	تبسيط النموذج، تشذيب، تنظيم، بيانات إضافية	إضافة ميزات أو تعقيد
انحياز عالٍ	المنحنيان ملتصقان وسيئان معاً	نموذج أغنى، ميزات أفضل، تقليل التنظيم	شراء بيانات أكثر
تقارب مثالي	قمة مرتفعة وفجوة صغيرة ومستقرة	حافظ على الوضع ووثّق النافذة	رفع أهداف بلا داعٍ

خطأ شائع

لن تحتاج `learning_curve` بلا `shuffle=True` و `random_state=42` إذا كانت بياناتك مرتبة بأي منطق خارجي (زمن، ترتيب فئات، حجم ملف): فالتقسيم الطبقي على تسلسل مرتَّب يجعل أحجام التدريب المبكرة منحازة عيّنياً، فيرسم لك منحنى كاذباً يصعد لا لأن النموذج تعلّم بل لأن العينات المتأخرة أسهل. فعّل الخلط وحبّذ بذرة ثابتة حتى تتكرر التجربة كما في الفصل ٧٢.

٩.٣ منحنى التحقق: مسح مقبض واحد حتى تقع العين



الشكل ١٧٧ منحنى تحقق لعمق الشجرة: القمة حول القيمة الصغرى الوسطى ثم الانحدار مع فرط الملاءمة

في الشكل أعلاه يصعد منحنى التحقق ثم ينقلب بينما واصل منحنى التدريب صعوده نحو ١.٠ — نقطة الانعطاف بينهما هي النافذة الذهبية. لنقم بذلك التجربة بأرقامنا على بارامتر `max_depth` من ١ إلى ١٥:

python — مسح max_depth من 1 إلى 15 بالتحقق المتقاطع

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import validation_curve

X, y = make_moons(n_samples=800, noise=0.3, random_state=42)
depths = np.arange(1, 16)

trs, tes = validation_curve(DecisionTreeClassifier(random_state=42), X, y,
                            param_name="max_depth", param_range=depths,
                            cv=5, scoring="accuracy")
a, b = trs.mean(axis=1), tes.mean(axis=1)
for d in [1, 2, 6, 10, 15]:
    i = d - 1
    print(f"depth={d:2d}: train={a[i]:.4f} val={b[i]:.4f}")
best = depths[np.argmax(b)]
print(f"النافذة الذهبية: depth={best} بدرجة تحقق {b.max():.4f}")
print(f"الفجوة عند القمة: {a[best-1]-b[best-1]:.4f} مقابل {a[-1]-b[-1]:.4f} عند العمق 15")
```

اقرأ المنطقتين: عند العمق ١ كلا المنحنيين منخفضان ومتلاصقان (انحياز: العمق لا يسمح بفصل الهلالين المتشابكين)، وعند العمق ١٥ صار التدريب ٠.٩٩٩ والتحقق تراجع — التباين انتصر. بينهما قمة تحقق ٠.٩٠٦ عند العمق ٦ بفجوة معقولة، وهي النقطة التي لو سألك عنها `GridSearchCV` في الفصل ٨١ لأعطاك الجواب نفسه — لكن منحنى التحقق يمنحك **قصة** لا نتيجة: ترى كيف يتبادل الانحياز والتباين الحكم على كامل المدى.

نصيحة

`validation_curve` مسح أحادي للفهم، و `GridSearchCV` بحث متعدد للضبط. حين تكتشف من مسح أحادي أن `max_depth` يتحرك في نطاق ضيق (٤ إلى ٧)، جرّب تضييق شبكة البحث في الفصل ٨١ حول هذا النطاق بدل مسح مئات التوليفات بلا داعٍ. ومرة أخرى: `param_name` سلسلة اسم البارامتر كما في `get_params()`، و `param_range` قائمة القيم — لا تخلط بينهما أبداً.

٩.٤ كم عينة تكفي؟ قراءة الهضبة

السؤال الذي يحرق ميزانيات جمع البيانات: متى يتوقف شراء العينات عن الجدوى؟ الجواب في قراءة ميل منحنى التحقق: حين يصبح التحسن لكل ألف عينة أقل من ضجيج القياس نفسه، فقد وصلت الهضبة (plateau):

```python
# هضبة التحقق: متى تعتز البيانات؟
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import learning_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=2400, n_features=12, n_informative=6,
                           n_redundant=3, class_sep=0.7, random_state=42)
sizes = np.array([150, 400, 800, 1200, 1500])

tr, trs, tes = learning_curve(
    make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)), X, y,
    train_sizes=sizes, cv=5, scoring="accuracy", shuffle=True, random_state=42)
prev = None
for n, v in zip(tr, tes.mean(axis=1)):
    delta = "" if prev is None else f" (فارق {v - prev:+.4f})"
    print(f"n={n:5d} | درجة التحقق={v:.4f}{delta}")
    prev = v
```

الأرقام تحسم القرار: من ١٥٠ إلى ٤٠٠ عينة كسبنا ٠٫٠١٩٦، ومن ٤٠٠ إلى ٨٠٠ كسبنا ٠٫٠٠٣٠ فقط، وما بعد ٨٠٠ هضبة يقارب الصفر — النموذج الخطي استنزف ما في هذه الميزات من معلومة، وإضافية البيانات لن تفتح له ما لا تملكه الميزات. هذا نمط انحياز مموَّه: الحل نموذج أغنى أو ميزات أذكى، لا مزيد من التجميع. ولاحظ أيضاً أن درجة التدريب تهبط نحو التحقق هنا: النموذج البسيط لا يستطيع حفظ حتى بيانات تدريبه.

$$\hat{S}_{val} = \frac{1}{k}\sum_{j=1}^{k} S_{val}^{(j)}$$

ما نقرأه في المنحنيات هو المتوسط `\hat{S}_{val}` لدرجات التحقق عبر طيات التحقق المتقاطع الـ `k`، ولهذا تظل الأرقام مستقرة تجاه حظ تقسيم واحد. ومتى احتجت تكلفة الحساب ضمن القرار — «هل يستحق مضاعفة البيانات مضاعفة زمن التدريب؟» — فالوسيط `return_times=True` يمنحك زمن التدريب لكل نقطة:

كلفة النمو: زمن التدريب مع تزايد العينات python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import learning_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=2400, n_features=12, n_informative=6,
                           random_state=42)
tr, trs, tes, fit_t, score_t = learning_curve(
    make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)), X, y,
    train_sizes=np.array([300, 800, 1500]), cv=5, scoring="accuracy",
    return_times=True, shuffle=True, random_state=42)
for n, t in zip(tr, fit_t.mean(axis=1)):
    print(f"n={n:5d} | متوسط زمن التدريب={t * 1000:.1f} ms")
```

نصيحة

الدالتان تتوزعان بالتوازي عبر النوى، لكن لا تفعّل `n_jobs=-1` لكل شيء: مع نماذج صغيرة سريعة (لوجستي على ٨٠٠ عينة) تكون كلفة توزيع المهام على العمليات أكبر من النموذج نفسه — قِسناها هنا فكان التشغيل التسلسلي أسرع بعشرات المرات. احتفظ بالتوازي للنماذج الثقيلة والغابات كما فعلنا في سيناريو ٩.٢.

خطأ شائع

`train_sizes` أعداد نسبية (كسور من ١) أو مطلقة، لكن المطلق محكوم بسقف صامت: `n × (cv − 1) / cv` . مع ٢٠٠٠ عينة و `cv=5` سقفُه ١٦٠٠؛ مرّر ١٨٠٠ فتصلك `ValueError` غامضة. الصيغة الآمنة: خُذ أحجامك من `np.linspace` نسبياً، أو اطرح السقف من حسابك قبل الاستدعاء.

٩.٥ القائمة التشخيصية: النموذج لا يؤدي — ابدأ من هنا

التشخيص في عشر خطوات

في الواقع

في فرق الإنتاج الحقيقية يُعلَّق هذان المنحنيان على لوحة القياس لا في الدفاتر: منحنى تعلّم أسبوعي يجيب على سؤال المدير «هل نستأجر جهة تجميع بيانات؟» بقراءة ميل واحد، ومنحنى تحقق عند كل ترقية مكتبة يكشف أن سلوك بارامتر افتراضي تغيّر بينما بقي الإعداد القديم. المهندس الذي يقرأ الفجوة والقمة في دقيقتين يوفر على فريقه أسبوع تجارب عشوائية — وهذه مهارة تُبنى هنا على بيانات مصنوعة، وتنتقل معك إلى أي مشروع.

تمارين الفصل الثمانين

- **ملاحظة**

التمرين ٩.١

ارسم أرقام منحنى تعلّم لـ `make_moons(n_samples=800, noise=0.3, random_state=42)` على `DecisionTreeClassifier(max_depth=2)`، وحدّد النمط، ثم نفّذ العلاج المقابل له بنموذج أغنى من اختيارك وقارن آخر نقطة تحقق قبل وبعد.

التمرين ٩.٢

أنشئ منحنى تحقق لمسح `n_neighbors` من ١ إلى ١٥ لمصنّف الجيران `KNeighborsClassifier` سبوق بـ `StandardScaler` عبر `make_pipeline` على البيانات نفسها. أين النافذة الذهبية، وماذا يحدث عند `n_neighbors=1` تحديداً؟

التمرين ٩.٣

بدون كود: زميلك رفع عدد عينات التدريب ثلاث مرات ولم تتحسن درجة التحقق قيد الانحياز العالي. اشرح له بلغة المنحنيات لماذا لن تنفع البيانات، واذكر علاجين من القائمة التشخيصية تناسب الحالة.

- **ملاحظة**

حل التمرين ٩.١: تشخيص شجرة ضحلة وعلاجها python

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.model_selection import learning_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=800, noise=0.3, random_state=42)
sizes = np.linspace(0.1, 1.0, 6)

for name, est in [("شجرة ضحلة (قبل العلاج)", DecisionTreeClassifier(max_depth=2,
random_state=42)),
                  ("SVC نواة RBF (بعد العلاج)", make_pipeline(StandardScaler(),
SVC()))]:
    tr, trs, tes = learning_curve(est, X, y, train_sizes=sizes, cv=5,
                                  scoring="accuracy")
    a, b = trs.mean(axis=1), tes.mean(axis=1)
    print(f"{name}: train النهائي={a[-1]:.4f} | val النهائي={b[-1]:.4f} | فجوة={a[-1]-
b[-1]:.4f}")
print("التشخيص: فجوة تافهة وقمة تعثرت مبكراً = انحياز يحده النموذج؛ والعلاج أغنى لا بيانات")
```

> **ملاحظة**

حل التمرين ٩.٢: مسح n_neighbors من 1 إلى 15 python

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import validation_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=800, noise=0.3, random_state=42)
k_values = np.arange(1, 16)

trs, tes = validation_curve(
    make_pipeline(StandardScaler(), KNeighborsClassifier()), X, y,
    param_name="kneighborsclassifier__n_neighbors", param_range=k_values,
    cv=5, scoring="accuracy")
a, b = trs.mean(axis=1), tes.mean(axis=1)
for k in [1, 3, 9, 15]:
    print(f"k={k:2d}: train={a[k-1]:.4f} val={b[k-1]:.4f}")
best = k_values[np.argmax(b)]
print(f"النافذة الذهبية: k={best} بدرجة تحقق {b.max():.4f}")
print(f"عند k=1: فجوة كاملة (train={a[0]:.4f}) — النموذج يحفظ ويفشل في التعميم")
```

> **ملاحظة**

٩.٧ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تمرير `train_sizes` مطلقة أعلى من سقف التقسيم	`ValueError` من `_translate_train_sizes` فوراً	أحجام نسبية بـ `np.linspace` أو التزم بسقف `n × (cv − 1) / cv`
علاج انحياز عالٍ بجلب بيانات إضافية	المنحنيان الملتصقان يبقيان ملتصقين على قمة منخفضة	نموذج أغنى أو ميزات أفضل أو تقليل التنظيم
نسيان `shuffle=True` مع بيانات مرتبة	منحنى كاذب يصعد بفعل ترتيب العينات لا تعلّمها	تفعيل الخلط مع `random_state=42` ثابت
تسمية البارامتر المتداخل خطأً في `param_name`	`ValueError` لأن الاسم غير موجود في `get_params()`	تسمية `pipeline__param` وفق اسم الخطوة داخل `make_pipeline`

٩.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
منحنى التعلّم	درجات تدريب وتحقق عبر أحجام تدريب متزايدة لتشخيص الجدوى	Learning curve
منحنى التحقق	درجات تدريب وتحقق عبر مدى قيم بارامتر واحد	Validation curve
الهضبة	توقف درجة التحقق عن التحسن مع تزايد البيانات أو البارامتر	Plateau
النافذة الذهبية	نطاق قيم البارامتر الذي تعلو فيه درجة التحقق قبل انحدارها	Golden window
فجوة التعميم	الفرق بين درجة التدريب ودرجة التحقق، مقياس التباين	Generalization gap

الجزء ٩ · التقييم والاختيار

الفصل ٨١ — GridSearchCV: البحث المنضبط

_cv_results g _best_score g refit g ParameterGrid

الوحدة scikit-learn ١.٩ الحجم ٨٩٥ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يبني القارئ param_grid يشمل القوائم والقاموسات المتعددة والتداخل ثنائي المحور
- أن يشغّل GridSearchCV داخل خط أنابيب بعنونة clf__C الصحيحة دون تسرب
- أن يقرأ cv_results_ عبر pandas: mean_test_score g std_test_score g rank_test_score
- أن يقدّر زمن البحث مسبقاً من عدد التركيبات وعدد الطيات وزمن التدريب الواحد
- أن يستخدم refit كدالة قابلة للاستدعاء لاختيار توازن بين الدقة والثبات

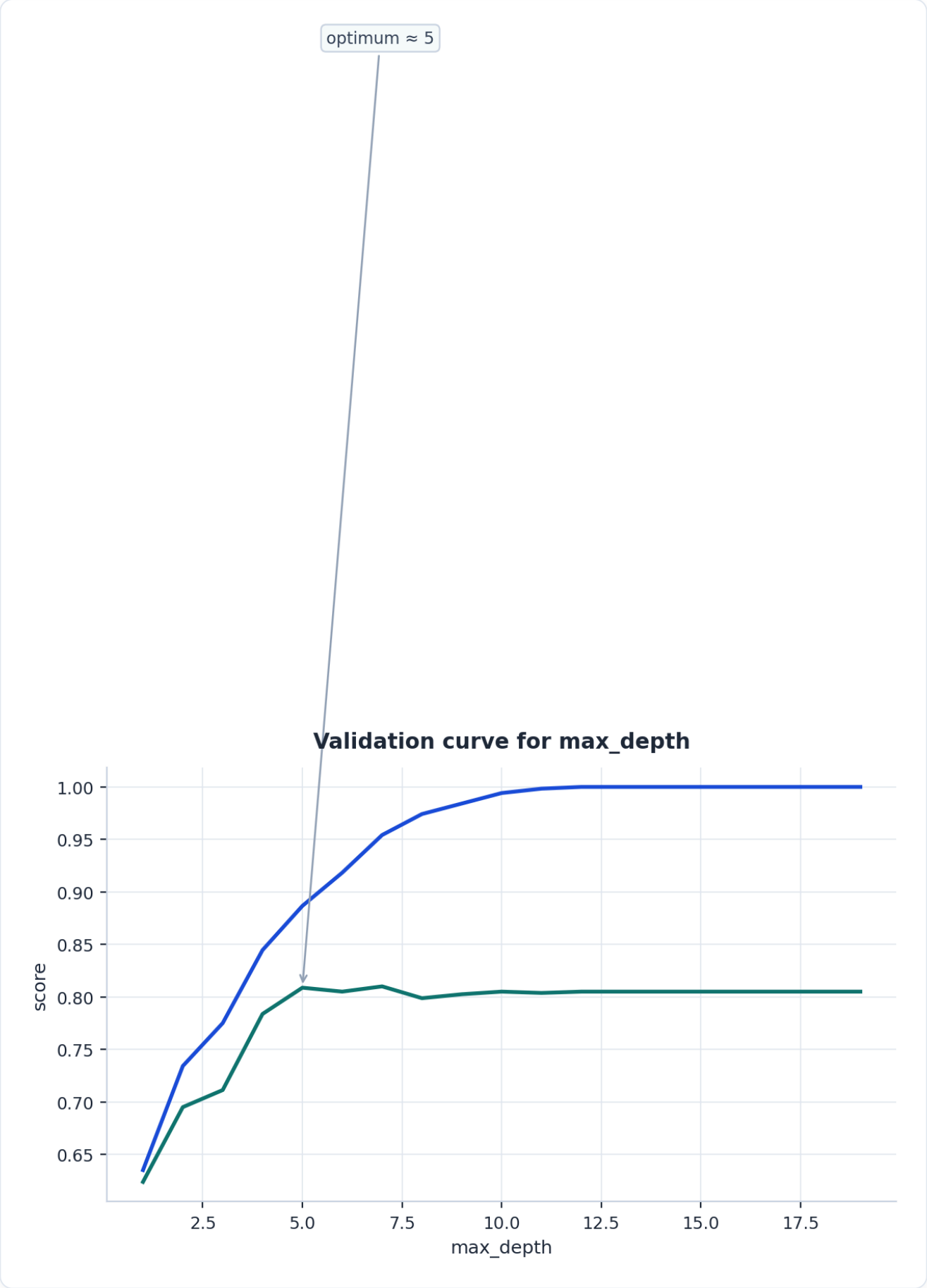
٩.١ من المسح الأحادي إلى الشبكة: لماذا لا يكفي محور واحد؟

في الفصل ٨٠ رسمنا منحنى التحقق: مسحنا `max_depth` من ١ إلى ١٩ ورأينا الدقة تتحسن ثم تنهار. كان المشهد مطمئناً — بُعد واحد، ومنحنى واحد، وقمة واحدة نحوها نمشي. لكن ماذا لو كان لديك بُعدان يتفاعلان؟

هنا بالضبط ينهار الحدس. في الفصل ٥٢ رأينا ثنائية `C` و `gamma` في `SVC`: قد تكون القيمة المثلى لـ `gamma` عند `C` صغير مختلفة تماماً عنها عند `C` كبير. مسح `gamma` وحده مع `C` مثبت عند قيمته الافتراضية قد يجعلك تعلن قمةً هي في الحقيقة هضبة جانبية. الشبكة تعالج هذا بأن تقول: لا تختر قيمة لكل بارامتر على حدة، بل جرّب **كل التركيبات معاً** وقِس كل تركيبة بالتحقق المتقاطع الذي شرحناه في الفصل ٧٣.

🔑 مفتاح

البحث الشبكي (Grid Search) هو تعداد شامل لكل التركيبات الممكنة من قيم البارامترات، مع تقييم كل تركيبة بالتحقق المتقاطع. صفوته الجوهرية: لا يفترض استقلال البارامترات، بل يقيس كل نقطة في الفضاء. عيوبه الثمنية: عدد التدريبات ينفجر أسّياً مع عدد البارامترات — وهذا ما سنتعلم حسابه قبل أن ندفع.



الشكل ١٧٨منحنى تحقق لمحور واحد: القمة عند max_depth ≈ ٥، لكن هذا المنحنى ثابت عند قيم بقية البارامترات

الشكل أعلاه هو آخر ما يراه الباحث أحادي البعد: منحنى جميل وقمته واضحة. الشكل التالي يذكّرنا أن شكل المساحة نفسه يفرض اختيار قيمها؛ فمحاور مثل `gamma` و `alpha` لا تُمسح بمسافات متساوية بل بمسافات لوغاريتمية — فكرة سنعتمدها عند بناء الشبكة بعد قليل، وستتوسع في الفصل ٨٢.

٩.٢ بناء param_grid: القوائم والقاموسات والتداخل

حاوية البحث في scikit-learn هي `param_grid`، ولها ثلاث حالات يجب أن تفصلها بدقة:

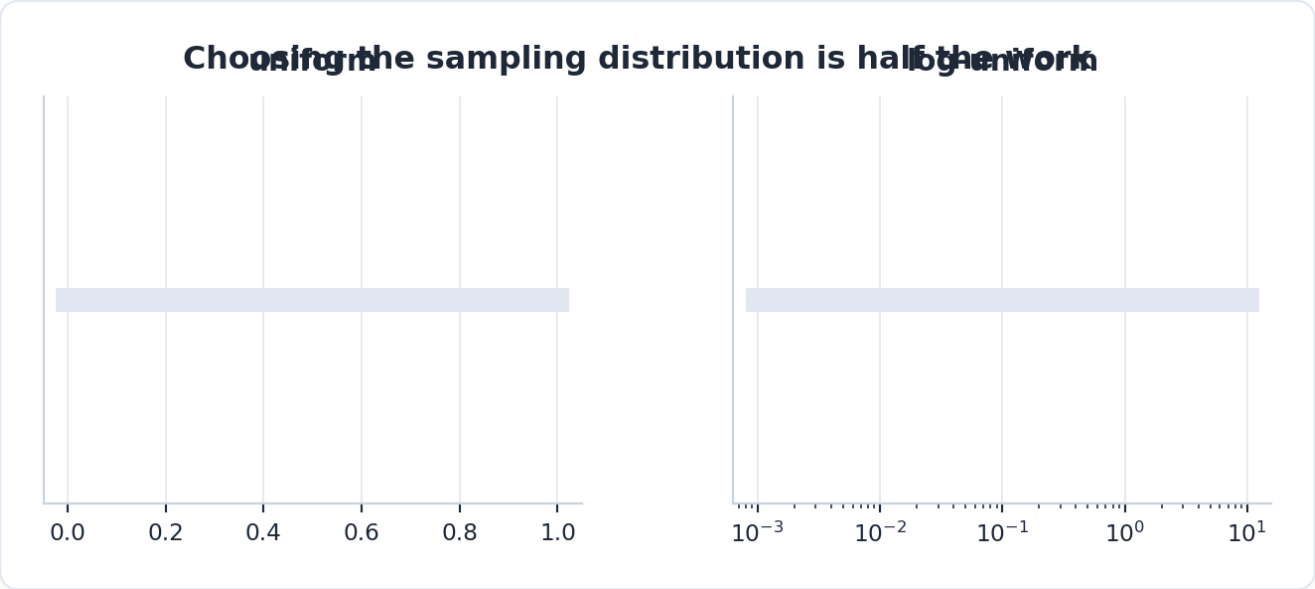
الأداة الرسمية لمعاينة هذه المساحة قبل تشغيل أي تدريب هي `ParameterGrid`:

معاينة المساحة قبل الدفع — python

```python
from sklearn.model_selection import ParameterGrid

# قاموس واحد: الضرب الديكارتي الكامل
grid_full = {"C": [0.1, 1, 10], "gamma": [0.001, 0.1]}
pg_full = ParameterGrid(grid_full)
print(f"قاموس واحد: {len(grid_full['C'])} × {len(grid_full['gamma'])} =
{len(pg_full)} نقطة")

# قائمة قاموسين: شبكتان منفصلتان تُجمَعان
grid_split = [
    {"kernel": ["linear"], "C": [0.1, 1, 10]},
    {"kernel": ["rbf"], "C": [1, 10], "gamma": [0.001, 0.1, 1.0]},
]
pg_split = ParameterGrid(grid_split)
print(f"({3} × {2}) + {3} :قاموسان = {len(pg_split)} نقطة")

for params in list(pg_split)[:4]:
    print(params)
```

لاحظ كيف تعامل `ParameterGrid` مع القاموسين: أول ثلاث نقاط بلا `gamma` إطلاقاً (نواة خطية لا تعرفها)، ثم تدخل نقاط النواة الشعاعية بقيمها. هذا الفصل بين "شبكات منفصلة" يوفّر عليك تدريبات عبثية لموّلتها خلط بارامترات لا تجتمع.

أما التداخل ثنائي المحور لمساحة الفصل ٥٢ فيُكتب ببساطة: `{"C": [...], "gamma": [...]}` في قاموس واحد — كل قيمة `C` ستُجرَّب مع كل قيمة `gamma`. وكلما استطعت، اختر قيم الشبكة على تدرج لوغاريتمي (`0.001، 0.01، 0.1، 1`) لا متساوٍ؛ لأن أثر هذه البارامترات نسبّي وليس مطلقاً — وهو جوهر الشكل التالي وستتعمق فيه مقابلة الفصل ٨٢.



الشكل ١٧٩مساحتان مختلفتا الشكل: أخذ عينات منتظم على C، ولوغاريتمي على gamma — اختيار قيم المحور نصف العمل

٩.٣ العنونة داخل خط الأنابيب: قاعدة clf__ التي تمنع التسرب

القاعدة الأكثر أهمية في هذا الفصل كله: **كل معالجة مسبقة (StandardScaler، ترميز، اختيار خصائص) يجب أن تكون داخل الشبكة، لا قبلها**. الشبكة تعيد تقسيم البيانات في كل طية؛ فإن كنت شطرت البيانات أو حوّلتها قبل البحث، فقد رأت أداة المعالجة أجزاءً ستُستخدم للاختبار — تسرب كما سمّيناه في الفصل ٧٢، وسيُضخّم تقديرك بصمت.

آلية العنونة في scikit-learn: كل خطوة في `Pipeline` تحمل اسماً، ويصل البارامتر إلى الخطوة ببادئة اسمها وبشرطة سفلية مزدوجة: `clf__C` تعني «البارامتر `C` للخطوة المسمّاة `clf`». وهذه البنية نفسها ستُشرح تفصيلاً في الفصل ٨٤ — هنا نحتاج منها قاعدة العنونة فقط.

بحث شبكي على خط أنابيب بعنونة صحيحة python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

pipe = Pipeline([("scaler", StandardScaler()), ("clf", SVC(kernel="rbf"))])
param_grid = {"clf__C": [0.1, 1, 10, 100],
              "clf__gamma": [0.0001, 0.001, 0.01, 0.1]}

gs = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1, return_train_score=True)
gs.fit(X_tr, y_tr)

print(f"أفضل تركيبة best_params_: {gs.best_params_}")
print(f"أفضل درجة تحقق best_score_: {gs.best_score_:.4f}")
print(f"دقة best_estimator_ على الاختبار المحجوز: {gs.score(X_te, y_te):.4f}")
```

لاحظ أن `best_estimator_` جاهز للنشر فوراً: بما أن `refit=True` افتراضياً، أعادت المكتبة تدريب الفائز على **كل** بيانات التدريب بعد انتهاء البحث، فصار الكائن مقدراً عادياً تحفظه بالـ joblib وتنشره كما سيرد في الفصول المتقدمة.

والآن التجربة التي تكشف حجم الثمن حين تخرج المعالجة من الشبكة. سنستخدم اختيار الخصائص `SelectKBest` الذي يعتمد على الهدف `y` — أخطر أنواع المعالجة حين تتسرب:

التسرب خارج الشبكة مقابل النظافة داخله python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.pipeline import make_pipeline
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, cross_val_score

X, y = make_classification(n_samples=160, n_features=200, n_informative=3,
                           n_redundant=2, class_sep=0.8, random_state=42)

# الخطأ: اختيار الخصائص على كل البيانات قبل البحث
X_sel = SelectKBest(f_classif, k=5).fit_transform(X, y)
leaky = cross_val_score(
    GridSearchCV(SVC(kernel="rbf"), {"C": [0.1, 1, 10]}, cv=4, n_jobs=-1),
    X_sel, y, cv=5)

# الصحيح: الاختيار داخل الأنبوب يُعاد ضبطه في كل طية
clean = cross_val_score(
    GridSearchCV(make_pipeline(SelectKBest(f_classif, k=5), SVC(kernel="rbf")),
                 {"svc__C": [0.1, 1, 10]}, cv=4, n_jobs=-1),
    X, y, cv=5)

print(f"التقدير المسرَّب:   {leaky.mean():.4f}")
print(f"التقدير النظيف:    {clean.mean():.4f}")
print(f"الوهم المتفائل:    {leaky.mean() - clean.mean():+.4f}")
```

الرقمان يقولان القصة: المسار المسرَّب يعدك بتفوق غير موجود. اختيار الخصائص رأى تسميات كل الطيات فاختار خصائص «تصادف» أنها تطابق كل البيانات، فتضخم التقدير بنحو نقاط مئوية كاملة. الـ `StandardScaler` أقل خطورة من هذا لأنه لا ينظر إلى `y`، لكن المبدأ واحد: **ما يُضبَط على البيانات، يُضبَط داخل الطية**.

خطأ شائع

كتابة `"C": [...]` في `param_grid` وأنت تبحث داخل `Pipeline` يرفع خطأ `Invalid parameter 'C'` لأن الأنبوب نفسه لا يملك `C` — بل خطوته. والخطأ المقابل الأخطر: نسخ سطر `StandardScaler().fit_transform(X)` قبل التقسيم ثم البحث؛ لا يرفع أي خطأ إطلاقاً، فقط يفسد تقديرك بهدوء.

٩.٤ تشريح *cv_results_*: التقرير الكامل للبحث

كل نداء بحث يترك وراءه القاموس `cv_results_` — سجل كامل لكل تركيبة على كل طية. تحويله إلى `DataFrame` يجعله قابلاً للفرز والقراءة:

```python
# قراءة نتائج البحث كجدول بيانات
import pandas as pd

cvres = pd.DataFrame(gs.cv_results_)
cols = ["param_clf__C", "param_clf__gamma",
        "mean_test_score", "std_test_score", "rank_test_score"]
print(cvres[cols].sort_values("rank_test_score").head(6).to_string(index=False))
print(f"\nbest_index_ = {gs.best_index_} (صف الفائز في الجدول)")
```

ثلاثة أعمدة تستحق التأمل:

والعلاقة بين متوسط التركيبة وتشتتها هي:

$$\bar{s}_g = \frac{1}{k}\sum_{i=1}^{k} s_{g,i}\,, \qquad \sigma_g = \sqrt{\frac{1}{k-1}\sum_{i=1}^{k}(s_{g,i}-\bar{s}_g)^2}$$

حيث `s_{g,i}` درجة التركيبة `g` على الطية `i`، و`k` عدد الطيات. في مخرجاتنا، تقاسمت التركيبتان (C=١٠٠، gamma=٠.٠٠١) و(C=١٠، gamma=٠.٠٠١) الدرجة ٠.٩٧٦٥ والرتبة ١ معاً — وعندما تتساوى القمم يصبح التشتت حكم القرار، وهذا ما سنبني عليه القسم التالي.

> **ملاحظة**
>
> يملك `cv_results_` أيضاً أعمدة `mean_fit_time` و `mean_score_time` لكل تركيبة — استعن بها لقراءة أثقل التركيبات حسابياً قبل توسيع الشبكة.

٩.٥ حساب التكلفة قبل الضغط على زر التشغيل

كم سيستغرق البحث؟ لا تكتشف بعد ساعة. المعادلة حتمية: عدد التركيبات ضرب عدد قيم كل بارامتر، وكل تركيبة تُدرَّب `k` مرة (عدد الطيات):

$$N = \prod_{j=1}^{P} |G_j|\,, \qquad T = k \times N$$

حيث `|G_j|` عدد قيم البارامتر `j` في شبكتك، و`P` عدد البارامترات، و`k` عدد الطيات، و`T` إجمالي التدريبات الكاملة. وزمن التدريب الواحد تقيسه بنداء واحد:

تقدير الزمن قبل التنفيذ ومقارنته بالواقع python

```python
import time
from sklearn.model_selection import ParameterGrid

n_combos = len(ParameterGrid(param_grid))
folds = 5

# قياس زمن تدريب واحد للأنبوب على حجم طية تقريبي (٤/٥ بيانات التدريب)
n_fold = int(4 * len(X_tr) / 5)
t0 = time.perf_counter()
pipe.fit(X_tr[:n_fold], y_tr[:n_fold])
t_one = time.perf_counter() - t0

estimate = n_combos * folds * t_one
t0 = time.perf_counter()
gs.fit(X_tr, y_tr)
actual = time.perf_counter() - t0

print(f"التركيبات: {n_combos} × الطيات: {folds} = {n_combos * folds} تدريباً كاملاً")
print(f"زمن تدريب واحد: {t_one * 1000:.1f} ms → تقدير إجمالي: {estimate:.2f} s")
print(f"الزمن الفعلي للبحث: {actual:.2f} s (بالتوازي عبر الأنوية)")
```

التقدير هنا يفوق الفعلي لأن `n_jobs=-1` وزّع التدريبات على الأنوية — وهذا هو الدور الحقيقي لـ `n_jobs`: البحث شبكي **متوازٍ بطبيعته**، فكل تركيبة مستقلة عن أخواتها. البارامتر المرافق `pre_dispatch` يحدد كم مهمة تُرسل للعمال مقدماً (القيمة الافتراضية `"2*n_jobs"` كافية غالباً)؛ استخدمه في البيئات المشتركة لتقليل الضغط على الذاكرة حين تكون كل تركيبة ثقيلة.

في الواقع

في عمل حقيقي قد تبلغ الشبكة آلاف التركيبات على مئات الآلاف من العينات. عادة المصنعين: قدّر زمن التدريب الواحد أولاً، واضرب، ثم اقرأ الرقم بصوت عالٍ قبل التشغيل. شبكة ٥٠٠٠ تدريب × ٣٠ ثانية = ٤١ ساعة — هذا الرقم يقرر إن كنت تريد شبكة أصغر، أو عينات فرعية، أو أسلوب الفصل ٨٢.

٩.٦ refit كدالة: اختيار التوازن بدل أعلى نقطة

الافتراض `refit=True` يختار صاحب أعلى `mean_test_score`. لكن أعلى متوسط ليس دائماً الخيار الأحكم: تركيبة تفوق المنافس بأربعة أجزاء من الألف مع تشتت مضاعف قد تكون مجرد حظ طيات. يتيح `refit` تمرير **دالة قابلة للاستدعاء** تستلم `cv_results_` كاملاً وتعيد **فهرس** الفائز بمعاييرك أنت. القاعدة الشهيرة هنا قاعدة «الانحراف المعياري الواحد»: اختر أبسط/أثبت تركيبة ضمن حدود أعلى درجة:

قاعدة الانحراف المعياري الواحد عبر refit الذكي — python

```python
import numpy as np

def smart_refit(cv_results):
    """اختر الأقل تشتتاً بين المرشحين القريبين إحصائياً من القمة."""
    scores = cv_results["mean_test_score"]
    stds = cv_results["std_test_score"]
    threshold = scores.max() - stds[scores.argmax()]   # حد القمة ناقص تشتتها
    eligible = scores >= threshold
    return int(np.argmin(np.where(eligible, stds, np.inf)))

gs_smart = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1, refit=smart_refit)
gs_smart.fit(X_tr, y_tr)

for name, model in [("الافتراضي", gs), ("الذكي", gs_smart)]:
    idx = model.best_index_
    print(f"{name}: C={gs.cv_results_['param_clf__C'][idx]!s:>5}, "
          f"gamma={gs.cv_results_['param_clf__gamma'][idx]!s:>6} | "
          f"متوسط {gs.cv_results_['mean_test_score'][idx]:.4f}, "
          f"تشتت {gs.cv_results_['std_test_score'][idx]:.4f}, "
          f"اختبار {model.score(X_te, y_te):.4f}")

print(f"هل best_score_ متاح مع refit callable؟ {hasattr(gs_smart,
'best_score_')}")
```

اقرأ النتيجة بتمعن: الافتراضي التقط الدرجة الأعلى بتشتت ٠٫٠١٦٦، بينما القاعدة الذكية رجحت كفة تركيبة أدنى بفارق طفيف في المتوسط لكن بتشتت ٠٫٠٠٩٥ — أي أقل عرضة للتذبذب بين الطيات، وقد أعطت فعلاً دقة اختبار أعلى (٠٫٩٨٦٠ مقابل ٠٫٩٧٩٠). وخطأ شائع هنا: توقع وجود `best_score_` مع `refit` الدالة — المكتبة لا تضبطه لأن «الأفضل» صار تعريفك أنت؛ اقرأ الدرجة من `cv_results_` بالفهرس كما فعلنا.

٩.٧ مفاتيح التحكم: error_score و verbose، وماذا بعد البحث

حين تفشل تركيبة واحدة في التدريب (قيمة غير قانونية، طية صغيرة جداً)، هل ينهار البحث كله؟ الجواب يحدده `error_score`:

تركيبة فاشلة لا تُسقط البحث كله — python

```python
from sklearn.linear_model import LogisticRegression

pipe_lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
gs_err = GridSearchCV(pipe_lr, {"logisticregression__C": [0.01, 1, -5.0, 100]},
                      cv=3, error_score=0.0)
gs_err.fit(X_tr, y_tr)

res = pd.DataFrame(gs_err.cv_results_)
print(res[["param_logisticregression__C", "mean_test_score", "rank_test_score"]]
      .to_string(index=False))
```

القيمة السالبة `-5.0` لـ `C` أجهضت تدريب تلك التركيبة، فسجّلت المكتبة الدرجة `error_score` (هنا ٠.٠) وتابعت البحث — والترتيب النهائي وضعها في الذيل. لو استخدمت القيمة الافتراضية `error_score=np.nan` لظهر الصف بدرجة NaN؛ أما `error_score="raise"` فيرفع الاستثناء ويعطّل كل شيء — مناسب في التصحيح، خطر في الجولات الطويلة. أما `verbose=1` فيطبع سطراً مثل «fits ٨٠ candidates, totalling ١٦ folds for each of ٥ Fitting» عند بدء البحث — وهو نفسه حساب التكلفة الذي تدربنا عليه، يقدمه لك العلم مجاناً.

> **تحذير**
>
> لا تقيّم النموذج الفائز على الطيات نفسها التي انتصر فيها! درجة `best_score_` متفائلة بالضرورة لأنها أقصى متوسط بين عشرات المرشحين — أقصى عينة من توزيع منحاز للأعلى. القاعدة: احجز مجموعة اختبار `holdout` منفصلة من أول يوم (كما فعلنا هنا بـ ٢٥٪ من البيانات)، ولا تلمسها إلا بعد انغلاق البحث. ما رأيناه أعلاه (٠.٩٧٦٥ في التحقق مقابل ٠.٩٧٩٠ على الاختبار) يبدو متوافقاً، لكن على شبكات أوسع قد يبلغ فجوة التفاؤل عدة نقاط مئوية.

وهكذا أغلق الفصل دائرة كاملة: مساحة معرَّفة بعناية، وتقييم داخل طيات نظيفة، وفائز معاد تدريبه، واختبار محجوز لم يحكم في الانتخابات. لكن الشبكة تبقى تعداداً شاملاً — وفي الفصل ٨٢ سنطأن المكان نفسه بعشوائية أذكى تغتفي عن الأبعاد التي لا تهم.

تمارين الفصل الحادي والثمانين

- **ملاحظة**

التمرين ٩.١

على بيانات `load_wine`، ابنِ أنبوباً من `StandardScaler` يليه `KNeighborsClassifier`، وابحث شبكياً في `n_neighbors` (القيم ٣ و٥ و٧ و١١) و`weights` (القيمتان uniform و distance). ما أفضل تركيبة وما درجتها؟ ثم قارنها بدرجة النموذج الافتراضي.

التمرين ٩.٢

لديك مساحة بحث من قاموسين: الأول `{"max_depth": [3, 5, 7, None], "criterion": ["gini", "entropy"]}` والثاني `{"max_depth": [5, 7], "min_samples_leaf": [1, 2, 4, 8]}`. احسب عدد نقاط المساحة وعدد التدريبات مع تحقق خماسي **قبل** تنفيذ أي بحث، ثم تحقق برقم `ParameterGrid`.

التمرين ٩.٣

اشرح لماذا يفشل `param_grid = {"C": [1, 10]}` عندما يكون المقدر المبحوث فيه أنبوباً من خطوتين، وما الرسالة التي تظهر، وكيف يكتب البارامتر الصحيح؟

- **ملاحظة**

شبكة KNN على بيانات النبيذ python

```python
from sklearn.datasets import load_wine
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

Xw, yw = load_wine(return_X_y=True)
knn_pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())
knn_grid = {"kneighborsclassifier__n_neighbors": [3, 5, 7, 11],
            "kneighborsclassifier__weights": ["uniform", "distance"]}
gs_knn = GridSearchCV(knn_pipe, knn_grid, cv=5, n_jobs=-1).fit(Xw, yw)

print(f"أفضل تركيبة: {gs_knn.best_params_}")
print(f"أفضل درجة تحقق: {gs_knn.best_score_:.4f}")
baseline = GridSearchCV(knn_pipe, {}, cv=5, n_jobs=-1).fit(Xw, yw)
print(f"درجة النموذج الافتراضي: {baseline.best_score_:.4f}")
```

انتصر الجار السابع بوزن uniform — والقيمة البعيدة (١١) لم تنتصر رغم أن زيادة الجيران تهدّئ الحدود؛ لأن بيانات النبيذ ثلاث الفئات متقاربة الحجم. لاحظ اسم الخطوة `kneighborsclassifier` صادراً آلياً من `make_pipeline` بحرف أول صغير.

ملاحظة

فشل العنونة الناقصة: رسالة الخطأ python

```python
from sklearn.pipeline import Pipeline

try:
    GridSearchCV(pipe, {"C": [1, 10]}).fit(X_tr, y_tr)
except Exception as err:
    print(f"نوع الخطأ: {type(err).__name__}")
    print(f"الرسالة: {str(err)[:120]}")
```

الأنبوب ككل لا يملك بارامتراً اسمه `C`؛ بارامتراته هي `steps` و `memory` وأسماء خطواته. الصياغة الصحيحة تمرر البارامتر عبر جسر العنونة: `"clf__C": [1, 10]` — حيث `clf` اسم خطوة المصنف كما عرّفناها عند بناء الأنبوب.

ملاحظة

٩.٩ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
معالجة مسبقة قبل البحث (تحجيم أو اختيار خصائص على كل البيانات)	تسرب يضخم `best_score_` ويعطي تفوقاً وهمياً بصمت	ضع كل المحولات داخل `Pipeline` وابحث بعنونة `step__param`
كتابة `"C"` بدل `"clf__C"` مع مقدر أنبوب	رفع `InvalidParameterError` لأن الأنبوب لا يملك `C`	استخدم بادئة اسم الخطوة للكل: `"clf__C": [...]`
افتراض وجود `best_score_` مع `refit` كدالة	خطأ `AttributeError` عند القراءة	اقرأ الدرجة من `cv_results_["mean_test_score"][best_index_]`
توسيع الشبكة دون حساب عدد التدريبات مسبقاً	بحث يستمر ساعات أو أياماً ويعطل بقية العمل	قبل `len(ParameterGrid(grid)) * k * t_one` التشغيل، ثم قرر

٩.١٠ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البحث الشبكي	تعداد شامل لكل تركيبات البارامترات مع تقييم متقاطع لكل تركيبة	Grid Search
مساحة البحث	مجموعة القيم المسموحة لكل بارامتر وجميع تركيباتها	Search space
العنونة المتداخلة	الوصول لبارامتر خطوة داخلية بالبادئة `step__param`	Nested parameter addressing
إعادة الملاءمة	تدريب التركيبة الفائزة على كل بيانات التدريب بعد البحث	Refit
قاعدة الانحراف المعياري الواحد	اختيار الأثبت بين المرشحين القريبين من القمة بدل القمة المطلقة	One-standard-error rule

الجزء ٩ · التقييم والاختيار

الفصل ٨٢ — البحث العشوائي والتوزيعات

RandomizedSearchCV و log-uniform وتوازن الاستكشاف

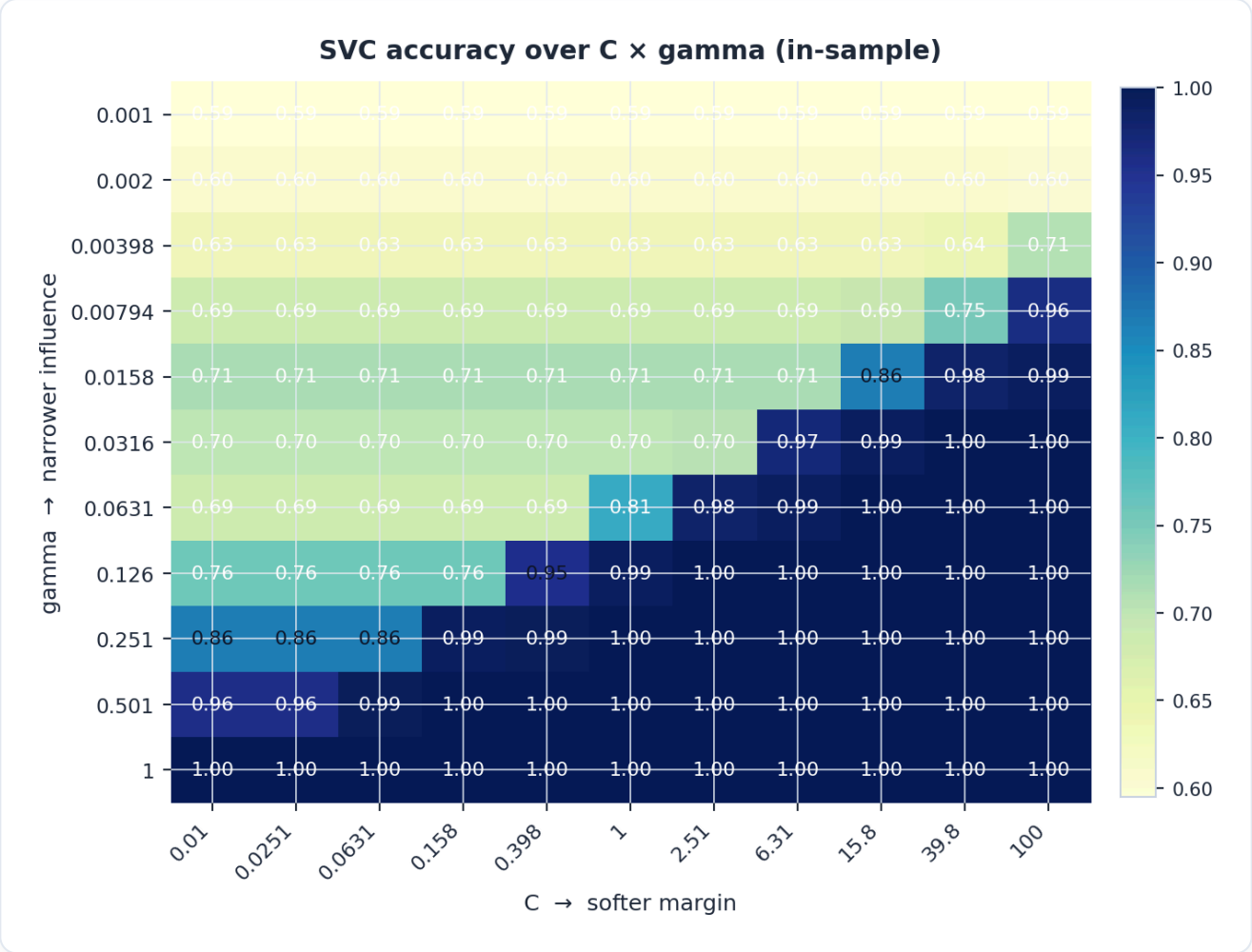
الوحدة **scikit-learn ١.٩** الحجم **٨٥٥** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يشرح القارئ لماذا يهدر البحث الشبكي تنفيذاته على الأبعاد غير المهمة بالأرقام
- أن يختار التوزيع المناسب من scipy.stats: uniform و loguniform و randint و reciprocal
- أن يشغّل HalvingGridSearchCV و HalvingRandomSearchCV مع فهم factor و min_resources
- أن ينفذ نمط المرحلتين: بحث عشوائي واسع يليه شبكة دقيقة حول أفضل نتوء
- أن يقارن الاستراتيجيات الثلاث زمناً ودرجة على مسألة واحدة بجدول واحد

٩.١ لماذا يتفوق العشوائي على الشبكي حين تتساوى الميزانيتان؟

في نهاية الفصل ٨١ خرجنا بثقة من شبكة منضبطة. لكن تأمل خريطة `C` × `gamma` التي رأيناها في الفصل ٥٢:



الشكل ١٨٠خريطة دقة SVC فوق مساحة C × gamma: السطح أودانه وأهضابه أوسع بكثير من قمته

السطح يكشف حقيقة محرجة: **أغلب المساحة رتيبة ومملة**. هناك «حزام» ضيق من القيم الجيدة، وخارجه لا شيء سوى تنويعات الفشل نفسه. لو كنت تعدّ تنفيذاتك كأنها ميزانية، لتساءلت: لماذا أنفق ٥ تنفيذات على `gamma` ليصل الباحث إلى قيمة واحدة ممتازة، بينما أغرقت ذلك الحزام في ٣٦ تركيبة عبثية؟

هذا بالضبط ما استهدفه Bengio Bergstra في ورقتهما الشهيرة عام ٢٠١٢: عندما يكون عدد البارامترات المهمة قليلاً (وهو الحال الغالب)، فإن **البحث العشوائي يغطي محور كل بارامتر مهم بتنفيذات أكثر من الشبكة بنفس الميزانية**. الشبكة تكرر كل قيمة من البارامتر المهم مع كل تركيبة الأبعاد التافهة؛ والعشوائي — إذا رسمنا القيم من توزيع مستمر — لا يكرر شيئاً.

مفتاح

البحث العشوائي (Random Search) يسحب قيم البارامترات من توزيعات احتمالية بدل تعدادها. قوته ليست الحظ، بل جغرافيا التغطية: بميزانية `N` تنفيذاً، يستكشف `N` قيمة مختلفة من كل بارامتر مهم، بينما الشبكة بميزانية مكافئة تستكشف `N` ÷ (ضرب أبعاد البارامترات التافهة) قيمة فقط.

٩.٢ التوزيعات: كيف «تُرسَم» القيم من scipy.stats

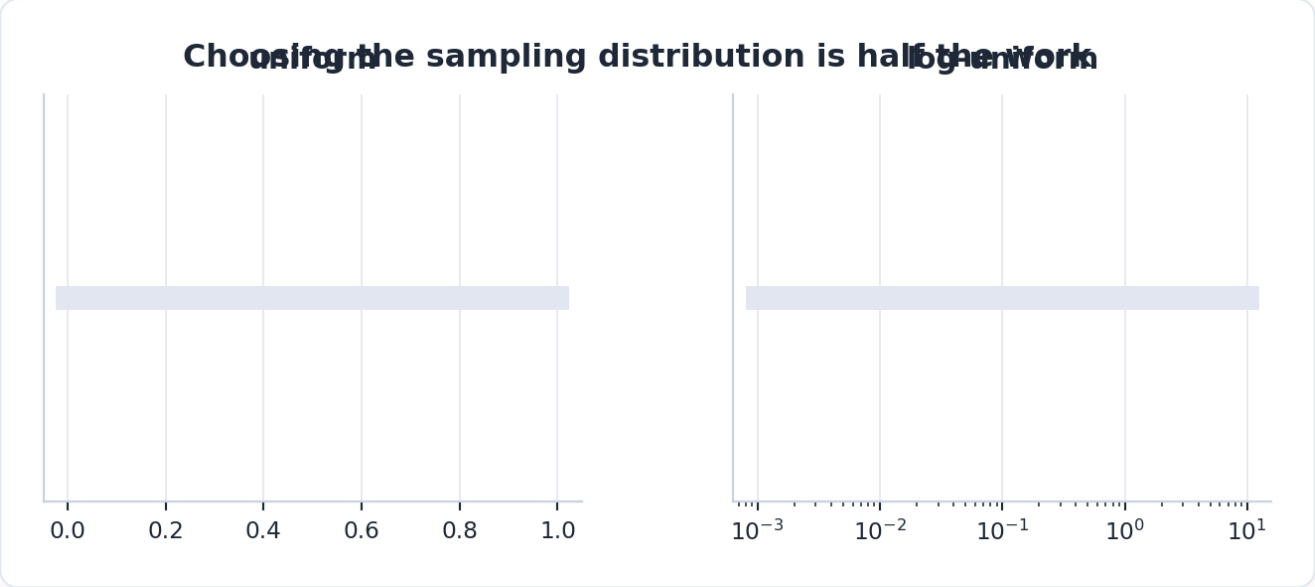
`RandomizedSearchCV` لا يريد قوائم جاهزة؛ يريد **توزيعات** يدعم واجهة `rvs`. والاختيار بينها قرار نمذجة كامل:

التوزيع	من أين	متى تستخدمه؟
`uniform(loc, scale)`	`scipy.stats`	بارامتر مؤثر خطياً: نسب، احتمالات، أوزان انحدار
`loguniform(a, b)`	`scipy.stats`	`gamma` و `alpha` و `learning_rate` — الفوارق نسبية
`randint(low, high)`	`scipy.stats`	أعداد صحيحة: `n_neighbors`، `max_depth`، `n_estimators`
`reciprocal(a, b)`	`scipy.stats`	شقيقة `loguniform` الأعم: كثافة ثابتة على مقياس لوغاريتمي لأي مجال

لماذا يكون المقياس اللوغاريتمي هو الطبيعي لـ `gamma` و `alpha`؟ لأن المعنى يكمن في **النسبة** لا الفارق: الانتقال من `alpha=0.001` إلى `0.01` هو قفزة نمذجة حقيقية (عشرة أضعاف)، بينما الانتقال من `0.101` إلى `0.111` لا يغيّر النموذج شيئاً. والتوزيع اللوغاريتمي يعاملهما بعدالة واحدة:

$$f(x) = \frac{1}{x \ln(b/a)}, \qquad a \le x \le b$$

الكثافة تتناسب عكسياً مع `x`، فتتوزع العينات بالتساوي **لكل فسحة لوغاريتمية** (كل فترة «×١٠» تحظى بعدد عينات متقارب)، لا لكل فترة مطلقة. أما `uniform` فتصرّف عيناتها بانتظام مطلق تماماً.



الشكل ١٨١توزيعا أخذ العينات: uniform فوق مجال C و log-uniform فوق مجال gamma — كثافة لكل عقد لوغاريتمية

سحب عينات من التوزيعات الأربعة python

```python
import numpy as np
from scipy.stats import loguniform, uniform, randint, reciprocal

print("loguniform(1e-3, 1e1):", np.round(loguniform(1e-3, 1e1).rvs(6,
random_state=42), 5))
print("reciprocal(0.1, 100):  ", np.round(reciprocal(0.1, 100).rvs(5,
random_state=42), 2))
print("randint(2, 10):        ", randint(2, 10).rvs(5, random_state=42))
print("uniform(0, 1):         ", np.round(uniform(0, 1).rvs(5, random_state=42),
3))

# أين تقع العينات؟ الفوارق المطلقة مقابل النسبية
vals = np.array([0.001, 0.01, 0.1, 1.0, 10.0])
print("الفوارق المطلقة بين القيم:", np.round(np.diff(vals), 4))
print("النسب بين القيم المتتالية:", np.round(vals[1:] / vals[:-1], 1))
```

اقرأ السطرين الأخيرين: الفوارق المطلقة تتضاعف عشر مرات في كل خطوة (٠.٠٠٩ ثم ٠.٠٩ ثم ٠.٩)، بينما النسبة ثابتة عند ١٠. أي شبكة بمسافات متساوية فوق هذا المحور تترك الفسحة الصغيرة (وهي غالباً الأهم) بلا أية عينات تقريباً — سنقيس ثمن هذا الخطأ رقمياً في القسم ٩.٦.

> **نصيحة**
>
> عندما تتردد بين توزيعين، اسأل: «لو ضاعفتُ القيمة، هل سيتغير النموذج تغيراً جوهرياً؟» إن كان الجواب نعم، فالمحور لوغاريتمي الطبيعة: `C`، `gamma`، `alpha`، `learning_rate`، و`n_estimators` كلها مرشحة للوغاريتم أو `randint` الواسع.

٩.٣ التجربة الحاسمة: أربعة أبعاد وثلاثة منها لا تعمل

سنصنع الموقف الذي تنكسر فيه الشبكة: مصنف `SVC(kernel="rbf")` حيث البارامتر المهم وحده هو `gamma`، بينما `degree` و`coef0` **لا يدخلان في حساب نواة RBF أصلاً** (هما بارامترا نواة `poly`)، و`tol` أثره هامشي. ثم نمنح الاستراتيجيتين ميزانيتين — لنرى أين تذهب كل ميزانية:

الشبكة: ٦٠ تركيبة أغلقالها تافهة python

```python
import time
import numpy as np
from sklearn.datasets import load_digits
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, train_test_split

X, y = load_digits(return_X_y=True)
X, y = X[:300], y[:300]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))

# نداء تمهيدي صغير يهيئ عمال التوازي قبل التجربة الرئيسة
GridSearchCV(pipe, {"svc__C": [1]}, cv=2, n_jobs=-1).fit(X_tr[:200], y_tr[:200])

param_grid = {"svc__gamma": [0.0005, 0.001, 0.002, 0.005, 0.01],
              "svc__degree": [2, 3, 4],
              "svc__coef0": [0.0, 1.0],
              "svc__tol": [1e-3, 1e-2]}
n_combos = 5 * 3 * 2 * 2
t0 = time.perf_counter()
grid = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1).fit(X_tr, y_tr)
t_grid = time.perf_counter() - t0

print(f"التركيبات: {n_combos} → تدريبات: {n_combos * 5}")
print(f"قيم gamma نسبة الهدر — فقط 5 فريدة في المساحة كلها: {(n_combos - 5) / n_combos:.1%}")
print(f"أفضل درجة تحقق: {grid.best_score_:.4f} عند gamma=
{grid.best_params_['svc__gamma']} | الزمن: {t_grid:.1f} s")
```

الشبكة أجرت ٣٠٠ تدريب كامل، لكن المساحة كلها لا تحوي سوى ٥ قيم `gamma` مختلفة — أي أن ٥٥ تركيبة من ٦٠ (٩١.٧٪) مجرد إعادة لقياسات قديمة بقيم تافهة متغيرة. والآن نفس الميزانية المعنوية للعشوائي (٢٠ تجربة × ٥ طيات = ١٠٠ تدريب):

```python العشوائي: ميزانية الثلث، تغطية أغنى
from scipy.stats import loguniform, uniform, randint
from sklearn.model_selection import RandomizedSearchCV

t0 = time.perf_counter()
rand = RandomizedSearchCV(
    pipe,
    {"svc__gamma": loguniform(5e-4, 5e-2),
     "svc__degree": randint(2, 5),
     "svc__coef0": uniform(0, 1),
     "svc__tol": uniform(1e-4, 9e-3)},
    n_iter=20, cv=5, random_state=42, n_jobs=-1).fit(X_tr, y_tr)
t_rand = time.perf_counter() - t0

n_distinct =
len(set(np.round(rand.cv_results_["param_svc__gamma"].astype(float), 7)))
print(f"قيم | فقط 100 :التدريبات gamma فريدة جُرِّبت: {n_distinct}")
print(f"أفضل درجة تحقق: {rand.best_score_:.4f} عند gamma=
{rand.best_params_['svc__gamma']:.5f}")
print(f"الشبكة — على الاختبار المحجوز: {grid.score(X_te, y_te):.4f} | العشوائي:
{rand.score(X_te, y_te):.4f}")
```

الأرقام تنقل القصة كاملة: العشوائي بثُلث الميزانية جرّب **٢٠ قيمة gamma مختلفة** مقابل ٥ للشبكة، وحقق الدرجة نفسها في التحقق (٠.٩٦٨٩ للاثنين)، بل تفوّق على الاختبار المحجوز (٠.٩٨٦٧ مقابل ٠.٩٧٣٣) لأن تغطيته الأوسع لقطعة الأرض المهمة سلّمته بذوراً أجود. `random_state=42` هنا ليس ترفاً: التوزيع عشوائي، لكن نتيجة البحث يجب أن تكون قابلة للإعادة حرفياً كما تكررنا في هذا الكتاب منذ الفصل الأول.

تعمّق

الحساب المجرد للمساواة بين الاستراتيجيتين: الشبكة لاجتناب تكرار لا مبرر له مضطرة إلى شطر ميزانيتها على كل تركيبات الأبعاد التافهة مع كل قيمة مهمة، بينما العشوائي ذو التوزيع المستمر لا يكرر قيمة أبداً فينفذ كل تجربته على قيمة جديدة من المحور المهم.

$$B_{\text{grid}} = K \times \prod_{j} |G_j|\,, \qquad B_{\text{rand}} = K$$

حيث `B` ميزانية التدريبات الكاملة، و`|G_j|` عدد قيم البارامتر التافه `j`، و`K` عدد القيم المطلوب تغطيتها على المحور المهم. في تجربتنا: تغطية ٢٠ قيمة gamma تكلف الشبكة ٢٠ × ١٢ = ٢٤٠ تدريباً، والعشوائي ٢٠ تنفيذاً فقط — فرق اثني عشر ضعفاً هو ثمن الأبعاد الصامتة.

٩.٤ HalvingSearch: النجاح المتتالي — اقضِ الميزانية على الناجين فقط

هناك استراتيجية ثالثة تجمع منطق الشبكة مع عقلانية الميزانية: **النجاح المتتالي (Successive Halving)**. الفكرة من عالم الرياضة: أدرِب كل المرشحين جولة قصيرة، واحتفظ بنسبة `1/factor` من الأفضل فقط للجولة الأطول، وكرر. ففي جولة البداية تُقيَّم ٦٠ تركيبة على عيّنة صغيرة من البيانات (٥٠ عينة)، وفي الجولة الثانية ٢٠ تركيبة فقط على ثلاثة أضعاف البيانات.

هذه العائلة ما تزال **تجريبية** في scikit-learn: يجب تفعيل مسارها التجريبي قبل أي شيء آخر، وإلا رفضت المكتبة الوصول إليها. نستخدم هنا صيغة واحدة تؤدي الغرضين معاً — تفعيل الوحدة التجريبية واستيراد الصنفين منها:

النجاح المتتالي: جولات إقصاء متتالية python

```python
from sklearn.experimental.enable_halving_search_cv import (  # تفعيل المسار التجريبي والاستيراد معاً
    HalvingGridSearchCV,
    HalvingRandomSearchCV,
)
import pandas as pd

t0 = time.perf_counter()
hal = HalvingGridSearchCV(pipe, param_grid, factor=3, min_resources=50,
                          cv=5, random_state=42, n_jobs=-1).fit(X_tr, y_tr)
t_hal = time.perf_counter() - t0

hres = pd.DataFrame(hal.cv_results_)
summary = hres.groupby("iter").agg(
    candidates=("mean_test_score", "size"),
    n_samples=("n_resources", "first"),
    best_score=("mean_test_score", "max"))
print(summary.to_string())
print(f"أفضل درجة: {hal.best_score_:.4f} عند gamma={hal.best_params_['svc__gamma']} | الزمن: {t_hal:.1f} s")
print(f"عيّنات مستهلكة: {int(hres['n_resources'].sum())} مقابل {180 * 5 * 60} للشبكة الكاملة")
```

اقرأ الجدول عموداً بعمود: ٦٠ مرشحاً على ٥٠ عينة فقط أنتجوا أفضل درجة ٠.٦٢ — تقدير مضلل تماماً، لكنه كافٍ لإقصاء أثقل ٤٠ خاسراً. الناجون العشرون تلقوا ١٥٠ عينة واصعد التقدير إلى ٠.٩٤٦٧. الحصيلة: ٦٠٠٠ «تدريب-عيّنة» مقابل ٥٤٠٠٠ للشبكة الكاملة — والدرجة النهائية (٠.٩٤٦٧) أدنى قليلاً من الشبكة (٠.٩٦٨٩) لأن الجولات القصيرة تخدع أحياناً وتقصي مرشحاً كان سيتألق لاحقاً. هذه هي المقايضة: سرعة أربعة أضعاف مقابل هامش دقة صغير.

> ⚠ **انتبه**
>
> صنفّا `HalvingGridSearchCV` و `HalvingRandomSearchCV` تجريبيان: محاولة استيرادهما من `sklearn.model_selection` قبل تفعيل المسار ترفع `ImportError`. الصيغة الرسمية في وثائق scikit-learn سطران — `from sklearn.experimental import enable_halving_search_cv` ثم الاستيراد من `sklearn.model_selection` — ونحن استخدمنا أعلاه صيغة مكافئة أقصر تستورد الصنفين من وحدة التفعيل نفسها؛ كلاهما يشغّل الوحدة التي تربط الأصناف بمكانها الرسمي. توقّع أن تتغير واجهة هذه العائلة دون دورة إهمال (deprecation cycle).

٩.٥ الاستراتيجية العملية: عشوائي واسع ثم شبكة دقيقة

البحثان ليسا حصراً متعارضاً؛ أفضل ممارسة صناعية هي **نمط المرحلتين**: عشوائي واسع على المقياس اللوغاريتمي يرسم الجغرافيا العامة، ثم شبكة دقيقة **خطية** حول أفضل نتوء لاستخراج آخر قطرة:

المرحلة الثانية والمقارنة الثلاثية الكاملة — python

```
g0 = float(rand.best_params_["svc__gamma"])
fine_grid = {"svc__gamma": np.geomspace(g0 / 4, g0 * 4, 9)}
t0 = time.perf_counter()
fine = GridSearchCV(pipe, fine_grid, cv=5, n_jobs=-1).fit(X_tr, y_tr)
t_fine = time.perf_counter() - t0
print(f"المرحلة الدقيقة: 9 تركيبات حول gamma={g0:.4f} ← درجة {fine.best_score_:.4f} في
{t_fine:.1f} s")

comparison = pd.DataFrame({
    "الاستراتيجية": ["عشوائي" ,"شبكة كاملة (n_iter=20)", "شبكة دقيقة + عشوائي :مرحلتان",
"Halving"],
    "[400 ,145 ,100 ,300] :"تدريبات,
    "أفضل درجة تحقق": [grid.best_score_, rand.best_score_, fine.best_score_,
hal.best_score_],
    "(ث) زمن": [t_grid, t_rand, t_rand + t_fine, t_hal],
})
print(comparison.to_string(index=False))
```

في هذه المسألة الصغيرة وصل العشوائي إلى الجوار الأعلى من أول مرة، فأضافت المرحلة الدقيقة ٤٥ تدريباً دون مكسب يذكر (٠.٩٦٨٩ نفسها) — وأصالة هذا الاستنتاج مهمة: **المرحلة الثانية تضمن التغطية المحيطة بالنتوء، لا التفوق تلقائياً**. على مسائل أوسع حيث تلمس المرحلة الأولى الهضبة من بعيد فقط، تصبح indispensable.

> **في الواقع**
>
> فرق غالباً ما يُغفل في التقارير: تباين مقياس «الدرجة» بين البحثين ليس كل شيء — التكلفة الزمنية هي القرار الحقيقي في الخطوط الإنتاجية. حين يقسّم فريق الهندسة ميزانيتها اليومية للحوسبة، يمنح العشوائي درجة قابلة للنشر بأجزاء من الوقود، وهذا ما يجعله الخيار الافتراضي في مكتبات الضبط الآلي الحديثة.

٩.٦ الفخ القاتل: توزيع خطي على محور لوغاريتمي

إذا كانت قيم `gamma` المفيدة تعيش بين `1e-3` و `1e-2`، فماذا تفعل شبكة خطية فوق المجال الواسع `[1e-4, 10]`؟

```python
# شبكة خطية مقابل هندسية على مجال gamma الواسع
lin_grid = {"svc__gamma": np.linspace(1e-4, 10, 10)}
geo_grid = {"svc__gamma": np.geomspace(1e-4, 10, 10)}
print("قيم الشبكة الخطية:", np.round(np.linspace(1e-4, 10, 10), 3))

t0 = time.perf_counter()
bad = GridSearchCV(pipe, lin_grid, cv=5, n_jobs=-1).fit(X_tr, y_tr)
good = GridSearchCV(pipe, geo_grid, cv=5, n_jobs=-1).fit(X_tr, y_tr)
print(f"أفضل درجة خطية: {bad.best_score_:.4f} عند gamma=
{bad.best_params_['svc__gamma']:.4f}")
print(f"أفضل درجة هندسية: {good.best_score_:.4f} عند gamma=
{good.best_params_['svc__gamma']:.5f}")
```

النتيجة صادمة ومتعمدة: الشبكة الخطية بالكاد تفوق التخمين الأعمى (٠.١٦٤٤ على عشر فئات!)، لأن ٩ من قيمها العشر سقطت فوق `1.1` حيث تنهار نواة RBF إلى «كل شيء يشبه كل شيء». الشبكة الهندسية `geomspace` وزّعت قيمها على الفسحات اللوغاريتمية فالتقطت الحزام الجيد بدرجة ٠.٩٦٤٤. والأسوأ أن الخطأ **صامت**: البحث «نجح» وانتخب فائزاً — فقط فائزه رديء. وهذا يفسر لماذا نرسم دائماً قيم الشبكة بقوى العشرة كما رأينا في شكل القسم ٩.٢، ولماذا `loguniform` هو التوزيع الافتراضي لمحاور كهذه في الفصل ٨٢ نفسه.

> **خطأ شائع**
>
> علامة إنذار مبكرة: إذا خرج `best_params_` عند **حافة** مجال البحث (أصغر أو أكبر قيمة عرضتها)، فمساحتك أضيق من الحقيقة — وسّعها في ذلك الاتجاه قبل أن تثق بالنتيجة.

تمارين الفصل الثاني والثمانين

- **ملاحظة**

التمرين ٩.١

على بيانات `load_wine`، طبّق `RandomizedSearchCV` (بما يلزم من `n_iter=25` و `cv=5` و `random_state=42`) على أنبوب `StandardScaler` + `RandomForestClassifier` بالتوزيعات: `n_estimators` من `randint(50, 300)`، و`max_features` من `uniform(0.1, 0.9)`، و`min_samples_leaf` من `randint(1, 8)`. قارن الدرجة الأمثل بدرجة الغابة الافتراضية بالتحقق الخماسي.

التمرين ٩.٢

اسحب ١٠٠٠ عينة من `loguniform(1e-4, 1e2)` ومن `uniform(1e-4, 1e2)`، واحسب نسبة العينات التي تقع في كل عقد لوغاريتمي (الفترة `[1e-4, 1e-3)` ثم `[1e-3, 1e-2)` وهكذا). ماذا يخبرك التوزيع النسبي للعقدتين عن عدالة كل توزيع؟

التمرين ٩.٣

فريقك يقيدك بـ ١٠٠ تدريب على بيانات ثقيلة، والمساحة فيها ٨ بارامترات اثنان منها مؤثران فعلاً. اشرح لماذا تختار `HalvingRandomSearchCV` هنا بدل الشبكة الكاملة، وأين تضبط `factor` و`min_resources`؟

- **ملاحظة**

بحث عشوائي على غابة عشوائية لبيانات النبيذ python

```python
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

Xw, yw = load_wine(return_X_y=True)
rf_pipe = make_pipeline(StandardScaler(),
RandomForestClassifier(random_state=42))
rs = RandomizedSearchCV(
    rf_pipe,
    {"randomforestclassifier__n_estimators": randint(50, 300),
     "randomforestclassifier__max_features": uniform(0.1, 0.9),
     "randomforestclassifier__min_samples_leaf": randint(1, 8)},
    n_iter=25, cv=5, random_state=42, n_jobs=-1).fit(Xw, yw)

baseline = cross_val_score(
    make_pipeline(StandardScaler(), RandomForestClassifier(random_state=42)),
    Xw, yw, cv=5, n_jobs=-1).mean()

print(f"أفضل درجة بعد 25 تجربة: {rs.best_score_:.4f}")
print(f"أفضل تركيبة: n_estimators=
{rs.best_params_['randomforestclassifier__n_estimators']}, "
      f"max_features=
{rs.best_params_['randomforestclassifier__max_features']:.2f}, "
      f"min_samples_leaf=
{rs.best_params_['randomforestclassifier__min_samples_leaf']}")
print(f"الغابة الافتراضية: {baseline:.4f} | التحسن: {rs.best_score_ - baseline:+.4f}")
```

عشرون وخمس تجارب استكشفت ٢٥ قيمة مختلفة لكل بارامتر بدل شبكة ٢٥٠ × ٩ × ٨ = ١٨٠٠٠ تركيبة — وهذا جوهر القسم ٩.٣ بالأرقام على غابة كاملة.

> **ملاحظة**

عدالة العقد اللوغاريتمية: loguniform مقابل uniform python

```python
from scipy.stats import loguniform, uniform

edges = 10.0 ** np.arange(-4, 3)   # 1e-4 حتى 1e2
draws_log = loguniform(1e-4, 1e2).rvs(1000, random_state=42)
draws_lin = uniform(1e-4, 1e2).rvs(1000, random_state=42)

for name, draws in [("loguniform", draws_log), ("uniform", draws_lin)]:
    counts = np.histogram(draws, bins=edges)[0]
    shares = counts / counts.sum()
    print(name, "لكل عقد:", np.round(shares, 3))
```

توزيع `loguniform` يمنح كل عقد لوغاريتمي نحو سُدس العينات (١٥–١٩٪ في المخرجات أعلاه؛ ست فسحات متكافئة تقريباً)، بينما يبصق `uniform` كوكبة العينات في أعلى العقدتين — أي في القيم التي لا تهم غالباً. هذا هو «الاعتراض على الأبعاد» مقاساً بالاحتمالات.

> **ملاحظة**

٩.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام `uniform` لمساحة `gamma` أو `alpha` الواسعة	العينات تتراكم في الفسحة العليا ويضيع الحزام الجيد (درجة ٠.١٦ في تجربتنا)	استخدم `loguniform` أو `geomspace` لمحاور القوى العشرية
استيراد `HalvingGridSearchCV` قبل `enable_halving_search_cv`	رفع `ImportError` لكون الصنفين تجريبيين	اجعل سطر التمكين أول استيراد في الكتلة
ترك `random_state` افتراضياً في `RandomizedSearchCV`	نتيجة البحث تتغير في كل تشغيل وتفقد الإعادة	ثبّت `random_state=42` ولصّق التوزيعات ببذور ثابتة
الافتراض أن المرحلة الدقيقة ستضيف دقة دائماً	٤٥ تدريباً إضافياً بلا مكسب حين يكون العشوائي قد وصل الجوار	اجعل المرحلة الثانية مشروطة بوقوع التوء قرب حافة عيّنات المرحلة الأولى

٩.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
البحث العشوائي	سحب قيم البارامترات من توزيعات بدل تعداد كل التركيبات	Random Search
التوزيع اللوغاريتمي المنتظم	كثافة تتناسب عكسياً مع القيمة فتوازن الفسحات النسبية	log-uniform distribution
النجاح المتتالي	إقصاء تدريجي لنسبة من أسوأ المرشحين بين جولات تتنامى مواردها	Successive Halving
عدد التجارب	ميزانية البحث العشوائي: عدد التركيبات المسحوبة عشوائياً	n_iter
نمط المرحلتين	عشوائي واسع لرسم الجغرافيا ثم شبكة دقيقة حول النتوء	Two-stage search

الجزء ٩ · التقييم والاختيار

الفصل ٨٣ — اختبار الأهمية الإحصائية

permutation_test_score والمقارنة الصحيحة بين نماذج

الوحدة **sklearn.model_selection · sklearn.metrics** الحجم **٧٩٨** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يشرح القارئ توزيع الصفر (null distribution) وقراءة p-value لنتيجة التحقق
- أن يشغّل permutation_test_score على بيانات ذات إشارة وأخرى مبهمة ويقرر بالأرقام
- أن يوازن بين دقة تقدير p-value وتكلفته حين يطيل n_permutations
- أن يقارن نموذجين ببطيات متطابقة واختبار تي المزدوج مع وعي بالاعتماد المتبادل
- أن يحدد صراحة ما يثبته اختبار الأهمية وما لا يثبته

٨٣.١ هل ٠.٩٠ حقاً أفضل من ٠.٨٨؟

في الفصل ٧٢ لمسنا المرض لأول مرة: نفس التقسيم ببذور مختلفة يهزّ الدقة عشرات الأجزاء من الألف. والآن — بعد شبكات بحث كاملة ومقاييس دقيقة — عاد السؤال أشرس: زميلك درب نموذجاً حصل على ٠.٩٠، ونموذجك حصل على ٠.٨٨. هل هذا تفوق أم ضجيج؟

لنجب بالتجربة لا بالانطباع: نموذج واحد، بيانات واحدة، وثماني بذور مختلفة للتحقق الخماسي:

```python ثماني بذور لتحقق خماسي: كم يتذبذب المتوسط؟
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=2))

means = []
for seed in range(8):
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)
    means.append(cross_val_score(pipe, X, y, cv=cv, n_jobs=-1).mean())

print("متوسطات البذور الثماني:", np.round(means, 4))
print(f"المدى الكامل للتذبذب: {max(means) - min(means):.4f}")
print(f"المتوسط العام: {np.mean(means):.4f}")
```

مدى التذبذب بلغ ٠٫٠٠٨٨ — أي أن الفجوة «القاتلة» بين ٠٫٩٠ و ٠٫٨٨ تقع داخل الضجيج الطبيعي للطيات نفسها. والفارق بين نموذجين قد يزداد التباساً لأن لكل نموذج تذبذبه المستقل. نحتاج أداة تجيب سؤالاً أعمق قبل المقارنات: **هل يتعلم هذا النموذج شيئاً أصلاً، أم أن درجته مجرد صدفة التركيب؟**

> **مفتاح**
>
> اختبار التبديل (Permutation Test) يطرح سؤالاً قاطعاً: لو كانت التسميات فوضى عشوائية لا صلة لها بالخصائص، فما الدرجات التي كان «بحث التركيب» سيصطادها رغم ذلك؟ إن وقعت درجتك الأصلية بعيداً عن توزيع هذه الدرجات الوهمية، فإن نموذجك يلتقط إشارة حقيقية — وإلا فدرجتك بلا معنى مهما بدت عالية.

٩.٢ permutation*test*score: بناء توزيع الصفر

الفكرة ميكانيكية بسيطة وقاسية: خلط التسميات `n_permutations` مرة، وإعادة التدريب والتقييم الكاملين في كل مرة. كل خلط يفصل العلاقة بين الخصائص والهدف، فيصبح أي درجة تتحقق عليها **مدى صعود النموذج على الضجيج** — مع بقاء إجراء التدريب كما هو تماماً، بما فيه البحث الداخلي عن أنماط عرضية.

وتُعرَّف القيمة p من هذه الدرجات المبدلة:

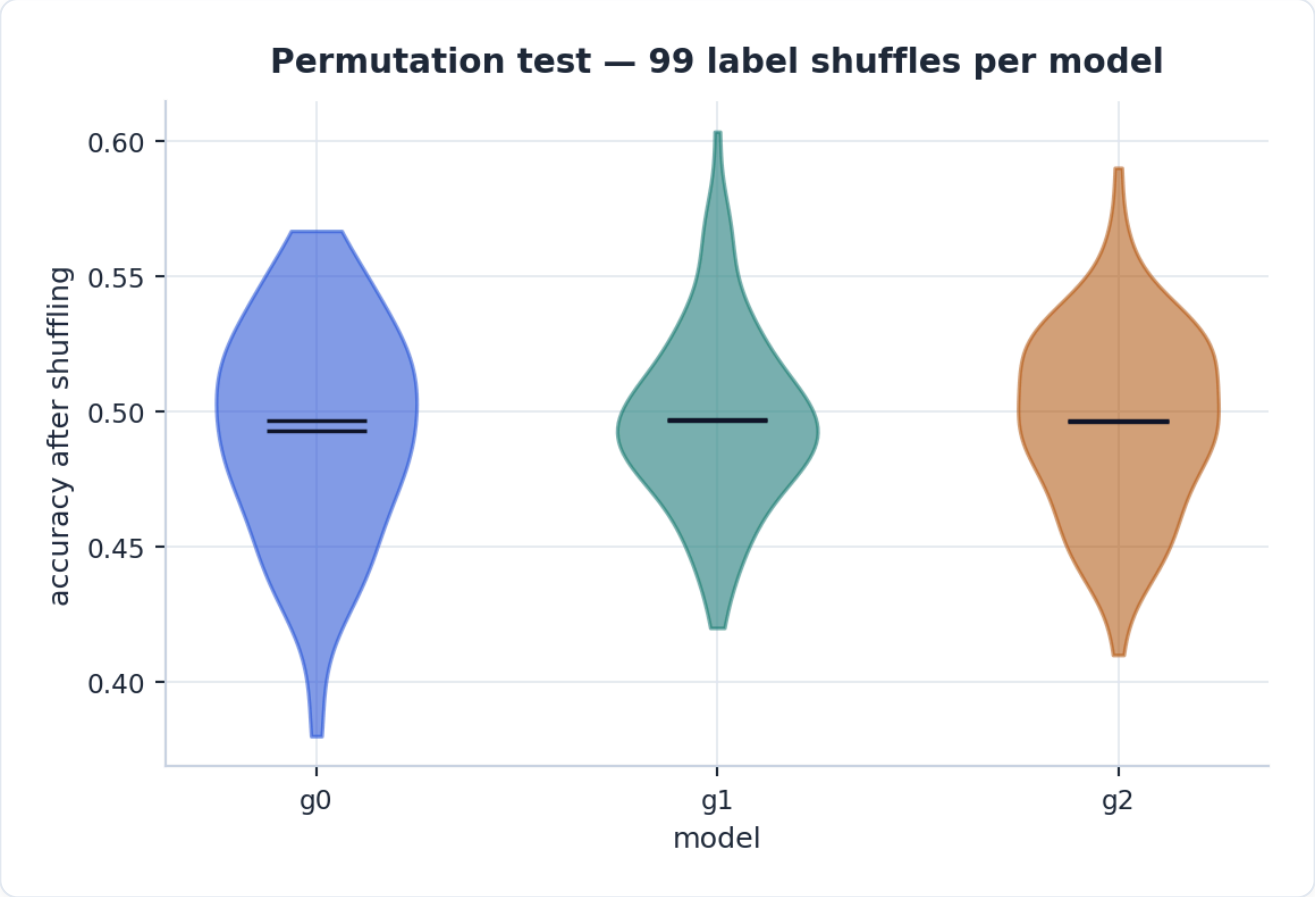
$$p = \frac{1 + \sum_{i=1}^{B} \mathbf{1}(s_i^{\pi} \geq s_{\text{orig}})}{B + 1}$$

حيث `B` عدد التبديلات، و`s_i^{\pi}` درجة التبديل رقم `i`، و`s_{\mathrm{orig}}` درجتك الحقيقية، و`\mathbf{1}(\cdot)` مؤشر يُعطي ١ متى تحقق الشرط. الرقم ١ في البسط احتياط التصحيح القياسي: نحسب تبديلتك الأصلية ضمن العائلة، فلا يهبط p تحت `1/(B+1)` مهما تميزت نتيجتك.

اختبار التبديل على بيانات ذات إشارة حقيقية python

```python
from sklearn.model_selection import permutation_test_score

score, perm_scores, pvalue = permutation_test_score(
    pipe, X, y, scoring="accuracy",
    n_permutations=100, random_state=42, n_jobs=-1)

print(f"الدرجة الأصلية: {score:.4f} | p-value: {pvalue:.4f}")
print(f"توزيع الصفر — المتوسط: {perm_scores.mean():.4f} | الأعلى:
{perm_scores.max():.4f}")
print(f"عدد التبديلات التي تفوقت على الأصل: {(perm_scores >= score).sum()} من 100")
```

اقرأ الأرقام كتوزيع لا كنقاط: على بيانات ورم الثدي، درجات مئة تبديل لتسميات مصطنعة تتجمع حول ٠.٦٠٩٦ — نعم، نموذج مدرب على فوضى «يتعلم» فوضاه ويصعد إلى ستين بالمئة بالتحقق المتقاطع! بينما درجتك الحقيقية ٠.٩٧٥٤ لم يقاربها أي تبديل واحد، فهبط p إلى حدّه الأدنى ٠.٠٠٩٩ = `1/101`. الخلاصة الرسمية: نرفض فرضية «لا علاقة بين الخصائص والهدف» بثقة عالية.



الشكل ١٨٢كمانات درجات التبديل لثلاثة نماذج بعد ٩٩ خلطاً للتسميات — تتجمع كلها حول مستوى الصدفة

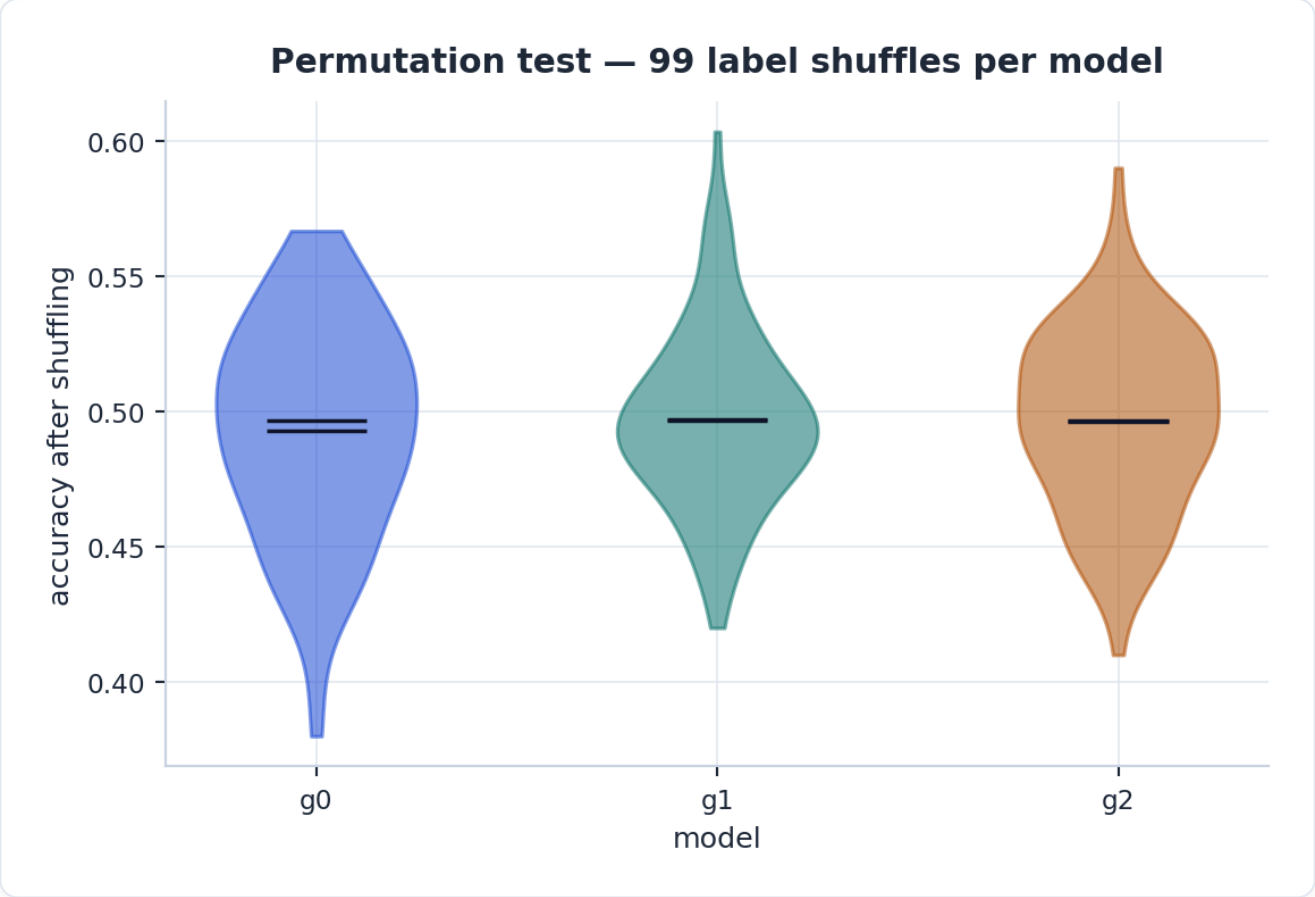
في الشكل أعلاه تظهر كمانات توزيع الصفر لثلاثة نماذج مختلفة القوة؛ كلها متقاربة الارتفاع رغم اختلاف دقتها الحقيقية، لأن خلط التسميات يمحو الإشارة من تحت أقدام الجميع. النموذج القوي لا يتكلم هنا — توزيع الضجيج هو الذي يتكلم، وموضع درجتك الحقيقية بالنسبة إليه هو الرسالة.

٩.٣ حين تكون الإشارة ضعيفة: هل يدفعك p لرفض النموذج؟

الاختبار الحقيقي لأداة ما هو تصرفها مع البيانات الرمادية. لنصنع بيانات ملتصقة: `class_sep=0.05` يجعل الفئتين تتداخلان جسدياً، و `flip_y=0.30` يقلب ٣٠٪ من التسميات عمداً:

```python
# بيانات مبهمة: هل يلتقط النموذج إشارة أصلاً؟
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression

X_amb, y_amb = make_classification(n_samples=300, n_features=6, n_informative=2,
                                   n_redundant=1, class_sep=0.05, flip_y=0.30,
                                   random_state=42)
pipe_lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))

amb_score, amb_perm, amb_p = permutation_test_score(
    pipe_lr, X_amb, y_amb, scoring="accuracy",
    n_permutations=100, random_state=42, n_jobs=-1)

print(f"الدرجة الأصلية: {amb_score:.4f} | متوسط توزيع الصفر: {amb_perm.mean():.4f}")
print(f"p-value: {amb_p:.4f}")
print(f"هل تقف الدرجة داخل توزيع الصفر؟ {(amb_score < amb_perm.max()) and (amb_score > amb_perm.min())}")
```

النتيجة مرعبة بمعناها الجيد: الدرجة الأصلية ٠.٤٧٣٣ — **أدنى من متوسط توزيع الصفر نفسه** (٠.٥٠٣٣)! أي أن تركيب النموذج على هذه التسميات المقلوبة لم يلتقط إشارة؛ بل اصطاد ضجيجاً أسوأ من العشوائي المجرد. بقيمة p تساوي ٠.٧٩٢١، لا سبب يدفعك لرفض فرضية «لا توجد علاقة قابلة للتعلم». والقرار النموذجي هنا ليس «اختر نموذجاً آخر» بل: بياناتك بهذا الشكل لا تدعم نمذجة إشرافية — راجع جمع البيانات أو تعريف المشكلة، كما حذرنا في الفصل ٧٢ من الخطوة التي تُفشل المشاريع.



الشكل ١٨٣توزيعات التبديل تفتح الحكم: حين تقع الدرجة داخل الكمان فالنموذج لم يلتقط شيئاً يذكر فوق الضجيج

خطأ شائع

درجة اختبار «مرتفعة» ليست دليلاً على التعلم. على البيانات المبهمة أعلاه حصل النموذج على ٠.٤٧ — ولو وصلت ٠.٦٥ لبدت مقبولة في تقرير، مع أنها لن تتجاوز توزيع الصفر. راجع p-value قبل أن تحتفل بأي درجة، خاصة على بيانات صغيرة أو مشبوهة النظافة.

٩.٤ كم تبديلاً يكفي؟ زمن مقابل دقة تقدير

عدد التبديلات هو مقبض دقة التقدير، وتكلفته خطية صريحة: كل تبديل إضافي يعني `k` تدريبات كاملة (طيات التحقق). لنجسّد الفارق على البيانات المبهمة نفسها:

```python
# مئة تبديل مقابل خمسمئة: الدقة والزمن
import time
from sklearn.model_selection import permutation_test_score

for n_perm in (100, 500):
    t0 = time.perf_counter()
    _, _, p_n = permutation_test_score(
        pipe_lr, X_amb, y_amb, scoring="accuracy",
        n_permutations=n_perm, random_state=42, n_jobs=-1)
    t_elapsed = time.perf_counter() - t0
    print(f"n_permutations={n_perm:3d} | p-value={p_n:.4f} | أصغر p ممكن:
{1/(n_perm+1):.4f} | الزمن: {t_elapsed:.1f} s")
```

ثلاثة دروس من المقارنة. أولاً **سقف الحسم**: بمئة تبديل لا يمكن لـ p أن تنزل تحت ٠.٠٠٩٩؛ لو كانت مسألتك تطلب إثباتاً أدق (مثل p < ٠.٠٠١ للنشر العلمي) فمئة تبديل مستحيل ميكانيكياً — الحد يهبط إلى ٠.٠٠٢ مع ٥٠٠. ثانياً **الثبات**: ٠.٧٩٢١ مقابل ٠.٧٧٨٤ — التقدير تذبذب قليلاً كما يتوقع لأداة مونت كارلو، ولم يغيّر القرار. ثالثاً **الكلفة**: خمسة أضعاف الزمن تقريباً، لأن كل تبديل يعيد تشغيل التحقق كاملاً. العادة الجيدة: ابدأ بمئة للاستكشاف، وأطل إلى ٥٠٠ أو ١٠٠٠ للنتيجة التي ستنشرها أو تدافع عنها.

> **ملاحظة**
>
> هناك رتبة زمنية ثانية مخفية: `n_jobs=-1` يوازي التبديلات فيما بينها (كل تبديل مهمة مستقلة)، فتكلفة إطالة الاختبار شبه خطية في عدد الأنوية المتاحة — لكنها تبقى مضاعفات كاملة للتدريب، ولا تلغي إعادة ضبط المحولات داخل كل طية كما شرحنا في الفصل ٨١.

٩.٥ منازلة نموذجين: بطيات متطابقة واختبار تي المزدوج

الآن عاد سؤال ٠.٩٠ مقابل ٠.٨٨ — وأصبح لدينا المنهج الصحيح: **المنازلة المتكررة (paired duel)**. شغّل النموذجين على **نفس الطيات بالضبط** (نفس كائن `cv` ببذرته)، فتصبح الدرجات متناظرة طية بطية، ثم اختبر فرقها:

$$t = \frac{\bar{d}}{\sqrt{s_d^2/k}}, \qquad s^2_{\text{corr}} = s_d^2\left(\frac{1}{k} + \frac{n_{\text{test}}}{n_{\text{train}}}\right)$$

حيث `d` فروق الدرجات بين النموذجين على الطيات، و`k` عدد الطيات، و`s_d^2` تباين الفروق. التصحيح الثاني هو تخمين Nadeau-Bengio للتباين: الطيات المستخرجة من بيانات واحدة **مترابطة** (تشارك بيانات تدريب)، فيكون التباين المرصود أقل من التباين الحقيقي لفرق نموذجين على بيانات جديدة — ويجب تضخيمه بحصة الاختبار من البيانات.

منازلة SVC ضد الانحدار اللوجيستي على طيات متطابقة python

```python
from scipy.stats import ttest_rel

cv = StratifiedKFold(5, shuffle=True, random_state=42)   # نفس الطيات للاثنين حرفياً
m1 = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=2))
m2 = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))

s1 = cross_val_score(m1, X, y, cv=cv, n_jobs=-1)
s2 = cross_val_score(m2, X, y, cv=cv, n_jobs=-1)
d = s1 - s2
t_stat, p_val = ttest_rel(s1, s2)

n = len(y)
n_train, n_test = int(0.8 * n), n - int(0.8 * n)
t_nb = d.mean() / np.sqrt(np.var(d, ddof=1) * (1 / 5 + n_test / n_train))

print("طيات SVC:             ", np.round(s1, 4))
print("طيات LogisticRegression:", np.round(s2, 4))
print(f"فروق الطيات: {np.round(d, 4)} | متوسط الفرق: {d.mean():+.4f}")
print(f"اختبار تي المزدوج: t={t_stat:.3f}, p={p_val:.4f}")
print(f"تي المصحح (Nadeau-Bengio): t={t_nb:.3f}")
```

بقيمة p تساوي ٠.٣٣٩١، لا دليل إحصائي على تفوق SVC رغم أن متوسط الفرق موجب (٠.٠٠٩٤). وبقيمة t المصححة الأدنى (٠.٧٢٣ مقابل ١.٠٨٥)، يؤكد تخمين Nadeau-Bengio ما خفيناه: طيات واحدة مشتركة تُخدع اختبار تي القياسي بالتباين الزائف الصغير. القاعدة العملية: مقارنة نماذج بفارق أقل من نقطة مئوية على بيانات بهذا الحجم هي **تعادل معلن** — والقرار يعود لمعايير غير الدقة: سرعة الاستدلال، قابلية التفسير، متانة النشر.

> **تحذير**
>
> لا تقارن نموذجين بتجزئتين مختلفتين! لو دربت كل نموذج على تقسيم عشوائي مستقل ثم قارنت متوسطيهما باختبار عادي، فقد خلطت بين تباين النماذج وتباين التقسيمات في سؤال واحد لا يقبل الفصل. الطيات المتطابقة (نفس `cv` وبذورها) ليست تفصيلاً — هي شرط صحة المنازلة كلها.

٩.٦ حدود الأداة: ما يثبته هذا الاختبار وما لا يثبته

أداة `permutation_test_score` تختبر فرضية واحدة محددة: «هل تتميز درجة النموذج عن ما يصنعه خلط التسميات؟». صيغة المنهجية الواجبة في أي تقرير:

ومن عائلة الأسئلة المجاورة سؤال «أي الخصائص تؤثر؟»، وأداته `permutation_importance` — نفس مبدأ الخلط لكن على **خصيصة واحدة** بدل التسميات كلها، وشرحناه كاملاً مع أخطائه في الفصل ٢٥؛ نحيلك إليه بدل إعادة الكلام.

> ☑ **قائمة تحقق**
>
> قبل الإعلان عن نتيجة نموذج في تقرير أو ورقة: مجموعة اختبار محجوزة لم تُلمس خلال البحث (الفصل ٨١) · درجة p من اختبار التبديل تبرض أن التعلم حقيقي · مقارنات المنافسين ببطيات متطابقة واختبار تي · ذكر صريح لعدد التبديلات وحد `1/(B+1)` · وقرار إنساني مسبق: ما قيمة p التي تعتبرها دليلاً؟

وهنا يكتمل الجزء التاسع. لقد تعلمنا أن القياس ليس آخر خطوة في المشروع بل **هيكلها**: تقسيمات نظيفة (الفصل ٧٢)، وطيات منضبطة (٧٣)، ومقاييس على قدر المهمة (٧٥–٧٩)، وبحث لا يسرّب (٨١–٨٢)، وأخيراً اختبار يفرّق الإشارة عن الضجيج. لكن الهيكل بلا ضمانة تنفيذية ينهار عند أول سكربت مهمل — وفي الجزء العاشر يأتي خط الأنابيب (Pipeline) ليحوّل هذا الانضباط من وعود في رؤوسنا إلى بنية برمجية لا يمكن كسرها.

تمارين الفصل الثالث والثمانين

- **ملاحظة**

التمرين ٩.١

على بيانات `load_wine`، شغّل `permutation_test_score` لشجرة قرار `DecisionTreeClassifier(max_depth=3, random_state=42)` بـ `n_permutations=100` و `random_state=42` و `scoring="accuracy"`. ما الدرجة و p-value؟ وكم تساوي أصغر قيمة p ممكنة بهذا العدد من التبديلات؟

التمرين ٩.٢

على أول ٦٠٠ عينة من `load_digits`، قارن أنبوب `LogisticRegression(max_iter=3000) + StandardScaler` مع أنبوب `StandardScaler + RandomForestClassifier(n_estimators=120, random_state=42)` بمنازلة متكررة: نفس `StratifiedKFold(5, shuffle=True, random_state=42)` للاثنين، ثم `ttest_rel`، ثم احسب t المصحح بتخمين Nadeau-Bengio. هل يوجد تفوق دال؟

التمرين ٩.٣

زميلك حصل على p-value = ٠.٠١ وقال: «إذن نموذجي أفضل من كل النماذج المنافسة». اشرح له بالتحديد ماذا تثبت القيمة وماذا لا تثبت، وما الأداة الصحيحة لسؤال التفوق، ولماذا قد تظل النتيجة الأصلية نفسها متفائلة حتى مع p منخفض؟

- **ملاحظة**

اختبار التبديل لشجرة قرار على بيانات النبيذ python

```python
from sklearn.datasets import load_wine
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import permutation_test_score

Xw, yw = load_wine(return_X_y=True)
tree = DecisionTreeClassifier(max_depth=3, random_state=42)

w_score, w_perm, w_p = permutation_test_score(
    tree, Xw, yw, scoring="accuracy",
    n_permutations=100, random_state=42, n_jobs=-1)

print(f"الدرجة الأصلية: {w_score:.4f} | p-value: {w_p:.4f}")
print(f"متوسط توزيع الصفر: {w_perm.mean():.4f}")
print(f"أصغر p تبديل 100 ممكن مع: {4.:1/101f}")
```

الشجرة الضحلة صعدت إلى ٠.٨٧١٠ بينما ركز توزيع التبديلات حول الحظ ثلاثي الفئات (٠.٣٣ تقريباً)، فارتطم p بسقفه الأدنى ٠.٠٠٩٩ — نموذج يتعلم بوضوح، مع أن شجرة بعمق ٣ لم تكن ستكسب أي منازلة ضد نموذج أقوى.

> **ملاحظة**

منازلة لوجيستي ضد غابة عشوائية على digits python

```python
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from scipy.stats import ttest_rel

Xd, yd = load_digits(return_X_y=True)
Xd, yd = Xd[:600], yd[:600]
cv_d = StratifiedKFold(5, shuffle=True, random_state=42)
lin = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
forrest = make_pipeline(StandardScaler(),
                        RandomForestClassifier(n_estimators=120,
random_state=42, n_jobs=-1))

a = cross_val_score(lin, Xd, yd, cv=cv_d, n_jobs=-1)
b = cross_val_score(forrest, Xd, yd, cv=cv_d, n_jobs=-1)
t_st, p_st = ttest_rel(a, b)
n_d = len(yd)
t_nb_d = (a - b).mean() / np.sqrt(
    np.var(a - b, ddof=1) * (1 / 5 + (n_d - int(0.8 * n_d)) / int(0.8 * n_d)))

print(f"لوجيستي: {np.round(a, 4)} | متوسط {a.mean():.4f}")
print(f"غابة:    {np.round(b, 4)} | متوسط {b.mean():.4f}")
print(f"t={t_st:.3f}, p={p_st:.4f} | t المصحح={t_nb_d:.3f}")
```

نموذجان متغايران تماماً في البنية خرجا بفروق طيات صغيرة معكوسة الاتجاه (t سالب!) وبقيمة p تعادل معلن. هذا هو السلوك الصحيح للأداة: على ٦٠٠ عينة من أرقام مكتوبة جيداً، الطرازان يجيدان المسألة — والفروق بينهما داخل الضجيج، فلا يُبنى على هذه المنازلة قرار اختيار.

> **ملاحظة**

٩.٨ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الإعلان عن تفوق بفارق ٠.٠٢ بين نموذجين من تقسيمين مختلفين	خلط تباين النماذج بتباين التقسيمات في قراءة لا تتحلل	منازلة بطيات متطابقة (نفس `cv`) ثم `ttest_rel` على الفروق
الاستنتاج «النموذج يتعلم» من درجة اختبار مرتفعة وحدها	درجات متوسطة قد تسكن داخل توزيع الصفر خاصة على بيانات صغيرة	شغّل `permutation_test_score` واطلب موقع الدرجة من التوزيع
توقع p < ٠.٠١ من `n_permutations=100`	الحد الميكانيكي `1/(B+1)` يمنع الوصول لرتبة الدلالة المطلوبة	أطل عدد التبديلات (٥٠٠–١٠٠٠) للنتائج النهائية المدعومة
تشغيل اختبار التبديل فوق أنبوب مسرَّب	توزيع الصفر نفسه يرتفع بالتسرب فتفقد الأداة قوة التمييز	أصلح التسرب أولاً (أنابيب الفصل ٨١) ثم اختبر الأهمية

٩.٩ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
اختبار التبديل	إعادة تقييم النموذج بعد خلط التسميات لبناء توزيع الصفر	Permutation test
توزيع الصفر	توزيع الدرجات المتوقعة حين لا صلة بين الخصائص والهدف	Null distribution
القيمة الاحتمالية	احتمال الحصول على درجة تبديل لا تقل عن الدرجة الأصلية	p-value
المنازلة المتكررة	مقارنة نموذجين على نفس الطيات واختبار فروقها المزدوجة	Paired model comparison
تصحيح Nadeau-Bengio	تضخيم تباين الفروق جبراً للترابط بين الطيات	Nadeau-Bengio corrected variance

الجزء ١٠

الأنابيب والتركيب

هندسة النماذج كخط إنتاج متكامل

ماذا يغطّي هذا الجزء

خط الأنابيب هو الفرق بين مشروع قابل للبناء ومشروع محبوس في دفتر Jupyter. يضمن تطبيق كل خطوة على بيانات التدريب فقط، ويمنع التسرّب، ويجعل النموذج كائناً واحداً يمكن حفظه ونشره. نتعلّم كذلك تركيب الأعمدة، وتوحيد المخرجات، وتحويل الهدف، والتخزين المؤقت للخطوط الثقيلة.

الجزء ١٠ · الأنابيب والتركيب

الفصل ٨٤ — خط الأنابيب: عقد أمانة مع البيانات

fit و transform و predict، والتسلسل، وعناوين الخطوات

الوحدة **scikit-learn ١.٩** الحجم **١٠٨٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يشرح أين تعلّم كل محوّل داخل fit، وكيف يتغيّر اختيار الخصائص بسبب التسريب
- أن يقيس أثر التسريب على ثلاثين بذرة ويصل إلى استنتاج صادق لا شائع
- أن يصف كيف يوزّع Pipeline استدعاء fit على الخطوات وبأي ترتيب
- أن يميّز بين make_pipeline و Pipeline ويعرف متى يفشل كل منهما في العناوين
- أن يبني أنبوباً متعدد المراحل داخل GridSearchCV ويقيس فجوة التدريب منه

١٠.١ الكود الذي يبدو صحيحاً: أين تعلّم المحوّل؟

لنبدأ من مشهد مألوف. لديك مئة وعشرون خاصية، عشر منها تحمل الإشارة والباقي ضجيج. تريد أن تُدخل الانحدار اللوجيستي أقوى عشر خصائص، وأن تطبّع المقياس أولاً. هذا الكود يقرأه أي مطوّر ويقول «صحيح»:

```python
# الطريقة الشائعة التي يثق بها الجميع
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

selector = SelectKBest(f_classif, k=10)
X_train = selector.fit_transform(X_train, y_train)
X_test = selector.transform(X_test)

model = LogisticRegression()
model.fit(X_train, y_train)
score = model.score(X_test, y_test)
```

كل سطر فيه منطقه. لكن لاحظ ما الذي لم يُكتب: **أين تعلّم** `scaler` **و** `selector`**؟** الأول تعلّم المتوسط والانحراف لكل الأعمدة على كامل البيانات — بما فيها صفوف الاختبار. والثاني تعلّم أيّ عشرة أعمدة «معلوماتية» من **صفوف الاختبار وتسمياتها** أيضاً. أنت لم تكتب خطأً نحوياً؛ كتبت **تسريباً** (leakage): معلومة من بيانات التقييم سكنت داخل معاملات التدريب.

لنرَ الأثر على بيانات `make_classification` فيها ٤٠٠ صف و١٢٠ خاصية، بحجم اختبار ١٠٪ — أي ٤٠ صف اختبار فقط:

سلسلة يدوية تعلّمت من صفوف الاختبار — python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=400, n_features=120, n_informative=10,
                           n_redundant=20, class_sep=0.7, flip_y=0.06,
                           random_state=42)
Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.10, random_state=42,
                                  stratify=y)

scaler_full = StandardScaler().fit(X)                 # الصفوف الـ400 كلها
selector_full = SelectKBest(f_classif, k=10).fit(X, y) # كاملاً y والهدف
keep = selector_full.get_support(indices=True)
leaky = LogisticRegression(max_iter=5000, C=0.3).fit(
    scaler_full.transform(Xa).to_numpy()[:, keep], ya)
print(f"المتسرّبة تعلّمت من {X.shape[0]} درجتها | صفاً: "
      f"{leaky.score(scaler_full.transform(Xb).to_numpy()[:, keep], yb):.4f}")

pipe = Pipeline([("sc", StandardScaler()),
                 ("sel", SelectKBest(f_classif, k=10)),
                 ("clf", LogisticRegression(max_iter=5000, C=0.3))]).fit(Xa, ya)
honest_idx = np.where(pipe.named_steps["sel"].get_support())[0]
print(f"الأنبوب تدرّب على {Xa.shape[0]} درجته | صفاً: {pipe.score(Xb, yb):.4f}")
print("اختار الأنبوب  :", [f"x{i}" for i in honest_idx])
print("اختيار المتسرّب:", [f"x{i}" for i in np.sort(keep)])
print("اختلفا في      :",
      [f"x{i}" for i in sorted(set(honest_idx) ^ set(keep))])
```

السطران الأخيران هما قلب الفصل. تسع من عشرة أعمدة متطابقة — لكن العاشرة مختلفة تماماً: السلسلة المتسرّبة أسقطت `x99` وأبقت `x7`، وهي خاصية **صفرية المعلومات** زائدها ضجيج. أنت لم تختر الخصائص لأن بيانات التدريب تقول إنها مفيدة؛ اخترتها لأن صف الاختبار طلبها. والنموذج الذي سلّمته للإدارة مبنيّ على أساس هشّ.

١٠.٢ التسرّب لا يضخّم الدرجة: بل يوسّع عدم اليقين

الشيء الوحيد الذي يستحق التسريب هو فكرة شائعة: «التسرّب يرفع درجتك فتصدّقها». لنختبرها على ثلاثين بذرة تقسيم مختلفة، لا على واحدة، فالمقارنة العادلة تحتاج تكراراً:

ثلاثون بذرة: ماذا يفعل التسريب فعلاً بالدرجة؟ python

```
def make_honest():
    return Pipeline([("sc", StandardScaler()),
                     ("sel", SelectKBest(f_classif, k=10)),
                     ("clf", LogisticRegression(max_iter=5000, C=0.3))])

honest_scores, leaky_scores = [], []
for seed in range(30):
    A, B, c, d = train_test_split(X, y, test_size=0.10, random_state=seed,
                                  stratify=y)
    honest_scores.append(make_honest().fit(A, c).score(B, d))
    sc_all = StandardScaler().fit(X)                    # تسرّب
    m = SelectKBest(f_classif, k=10).fit(X, y).get_support()  # تسرّب
    mdl = LogisticRegression(max_iter=5000, C=0.3).fit(
        sc_all.transform(A).to_numpy()[:, m], c)
    leaky_scores.append(mdl.score(sc_all.transform(B).to_numpy()[:, m], d))

h, l = np.array(honest_scores), np.array(leaky_scores)
gain = l - h
print(f"المتوسط :  صادق {h.mean():.4f} | الانحراف {h.std():.4f} "
      f"| المدى [{h.min():.4f}, {h.max():.4f}]")
print(f"المتوسط: متسرّب {l.mean():.4f} | الانحراف {l.std():.4f} "
      f"| المدى [{l.min():.4f}, {l.max():.4f}]")
print(f"متوسط الفارق: {gain.mean():+.4f} | فوق الصفر {int((gain > 0).sum())}"
      f" | تحت الصفر {int((gain < 0).sum())} | متساوٍ {int((gain == 0).sum())} من
30")
print(f"اتّساع المدى: من {(h.max() - h.min()) * 100:.1f} نقطة "
      f"إلى {(l.max() - l.min()) * 100:.1f} نقطة")
```

هذه النتيجة تصحّح شائعاً: **متوسط أثر التسريب كان -٠٫٠٠٨ — أي صفر عملياً.** ثمانية عشر من ثلاثين توليفة كانت متطابقة تماماً. فالتسريب هنا لم يرفع متوسط الدرجة ولا أنقصها.

لكن لاحظ ما حدث للانحراف: من ٠٫٠٥٨ إلى ٠٫٠٦٧، أي أن مدى النتائج اتّسع من ٢٥ نقطة إلى ٢٧٫٥ نقطة. هذا هو الضرر الحقيقي، وهو أعمق خطر من التضخيم:

ما يقيسه التقييم	مع التسريب	بدون التسريب
متوسط الدرجة عبر التقسيمات	٠٫٧٠٧٥	٠٫٧٠٨٣
انحراف الدرجة	٠٫٠٦٦٨	٠٫٠٥٨٢
أدنى درجة ممكنة	٠٫٥٧٥٠	٠٫٦٠٠٠
الخصائص المختارة في ٣٠ من ٣٠	٣٠ مجموعة مختلفة	ثابتة

التسريب يجعل رقمك **أشدّ تذبذباً وأقلّ قابلية للتكرار**، ويجعل خصائص النموذج تعتمد على أيّ صفوف وقعت في مجموعة الاختبار. وأخطر من ذلك: في الأوزان الحقيقية من الأوزان، التي تضمّ بضعة أضعاف حجم اختبارك بل صفوفاً كاملة، يمكن أن يبقى أثر التسريب **صفراً تماماً في المتوسط** ثم ينفجر فجأة عند أول انحراف في التوزيع.

> 🔑 **ما الذي يضمنه خط الأنابيب بالضبط**
>
> ليس النتيجة الأعلى، ولا الدرجة الجيدة. يضمن **التكرارية**: أن إعادة التدريب على بيانات مختلفة تعطيك نموذجاً مبنياً على نفس المبدأ. الدرجة تأتي أو لا تأتي؛ أما الضمانة فلا تُشترى.

١٠.٣ العقد: fit يمرّ على كل الخطوات، و predict يمرّ عليها كلّها

الفكرة التي تحلّ التسريب بنيوياً بسيطة: **اجعل الكائن هو الشيء الذي يحفظ الحالة.** `Pipeline` ليس قائمة خطوات تُنفَّذ بالتتابع داخل دالتك؛ هو كائن واحد له `fit` و`transform` و`predict`، وداخله خطوات مُسمّاة. عند `fit` يتكرّر في الخطوات هكذا:

$$\text{Pipeline.fit}(X, y) = \text{fit}_1(X, y) \to \text{transform}_1(X) \to \text{fit}_2(\cdot, y) \to \ \ldots$$

ولاحِظ بعد ذلك أن الاستدلال لا يعيد تدريب شيء:

$$\hat{y} = h_n(\ldots h_2(h_1(X;\, \theta_1);\, \theta_2)\ldots;\, \theta_n)$$

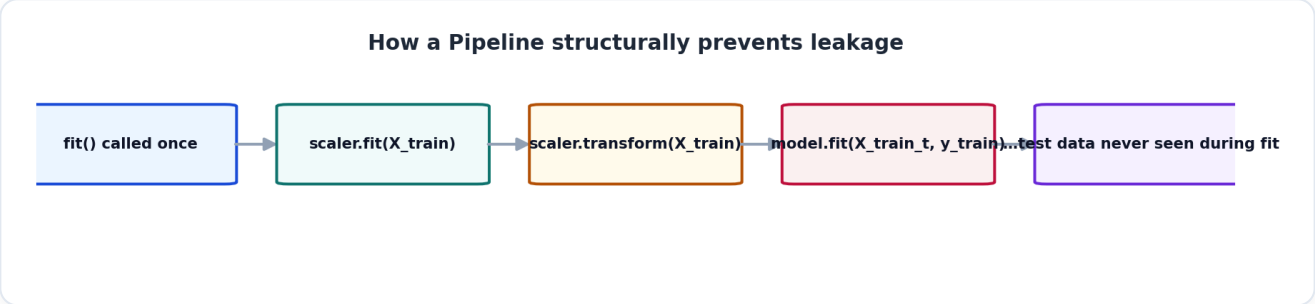
حيث كل θ_i معاملات **مثبّتة** Learned من بيانات التدريب وحدها. القيدان اللذان يخلقهما هذا التركيب معاً:

ثم إن استدعت `pipe.transform(X_test)` لاحقاً، فالنظام يشغّل صفوف الاختبار عبر المعاملات المخزّنة نفسها. لهذا يصبح التسريب **مستحيلاً بنيوياً** لا «مُنَصَّحاً بالأخلاقيات».

لنقرأ التفاصيل المُخزَّنة في الأنبوب الصادق الذي بنيناه:

```python
# ماذا خزّن الأنبوب بالضبط؟
sc = pipe.named_steps["sc"]
print(f"صفوف رآها المُطبِّع: {sc.n_samples_seen_:.0f} من أصل {X.shape[0]}")
print(f"متوسّطه للعمود الأول: {sc.mean_[0]:.4f}")
print(f"متوسّطه على كامل البيانات: {scaler_full.mean_[0]:.4f}")
print("أعمدة أبقاها المنتقي:",
      [f"x{i}" for i in np.where(pipe.named_steps['sel'].get_support())[0]])
```

رقمان مختلفان في السطرين الثاني والثالث، وكلاهما صحيح: هذا هو التسرّب مكشوفاً بالضبط — ليس احتمالاً نظرياً بل فرقاً عددياً محفوظاً في ذاكرتك.



الشكل ١٨٤سلسلة fit واحدة تصبّ كل الخطوات من نفس الصفوف: بيانات الاختبار لا تدخل في أي حالة داخلية

الشكل أعلاه هو المخطّط الحاسم. لاحظ أن `scaler.fit(X_train)` و`scaler.transform(X_train)` يحدثان **داخل** استدعاء `fit` الواحد، وأن `model.fit` لا يرى إلا `X_train_t` — أي ما مرّ عبر `transform`، لا `X_test` ولا حافّة منه.

١٠.٤ الدليل القاطع: مساران يعطيان النتيجة نفسها

في كل مرة يخبرك أحدهم بأن الأنابيب «مثالية»، اطلب دليلاً. الدليل هنا ليس تصريحاً نظرياً بل تطابق عددي: خذ الأنبوب المدرَّب، وافصله إلى جزأه الأول ونموذجه النهائي، وطبّق خطوتين يدوياً. يجب أن يطابق `predict` الأنبوب **حرفياً**، عنصراً بعنصر:

```python
# نفس الصفوف، نفس التنبؤات — بالمسارين معاً
X_b_t = pipe[:-1].transform(Xb)
manual = pipe.named_steps["clf"].predict(X_b_t)
print("تطابق المسارين:", bool((manual == pipe.predict(Xb)).all()))
print("نوع ناتج التحويل:", type(X_b_t).__name__, "| الشكل:", X_b_t.shape)
print("أسماء الخصائص بعد التحويل:", list(X_b_t.columns)[:4])
```

القيمة `True` ليست فضولاً: هي إثبات أن `pipe.predict` هو حرفياً `pipe[-1].predict(pipe[:-1].transform(...))`. ومن ثمّ كل ضمانة الأنبوب هي ضمانة التسلسل الصحيح، ولا يوجد مكان في الكائن يمكن أن يُخفي فيه أحدهم حالة جانبية. لو conciliating شخص بأن «الأنبوب يسمح بمراحل يدوية تفعل ما تشاء»، فالجواب عددي: لا شيء خارج الكائن يظهر في `predict`.

لاحظ أيضاً أن نوع الناتج `DataFrame` لا `ndarray`: هذا لأن هذا الكتاب يضبط `set_config(transform_output="pandas")` في تمهيد كل الأمثلة، فيحتفظ كل محوّل بأسماء الأعمدة. في مشروعك أنت من يقرّر هذا الإعداد، وبه يُنتج محوّل `DataFrame` جدولاً بعمودين مثل `age` و`income` بدل مصفوفة `x0` و`x1`.

> **شرط القطع**
>
> `pipe[:-1]` لا تشغّل `transform` إلا إذا كانت كل الخطوات قبلها محوّلات. لو كان نموذجاً (مثل `clf`) قبل نقطة القطع، فإن `pipe[:-1].transform` سيرفع `AttributeError` طبيعياً — وهذا تذكير بأن القطع يقع عند حدّ معيّن في السلسلة.

١٠.٥ make_pipeline مقابل Pipeline: مشكلة أسماء الخطوات

هذه أكبر مشكلة تُفشل أنابيب الطلاب لأوّل مرّة، لأنها لا تظهر إلا لحظة البحث الشبكي.

`Pipeline` يطلب منك أسماءً صريحة لكل خطوة — وأنت تختارها. `make_pipeline` يولّدها تلقائياً من اسم الصنف محوَّلاً إلى أحرف صغيرة بلا فواصل:

```python
# الأسماء التلقائية تكسر العناوين المتداخلة
from sklearn.pipeline import make_pipeline

auto = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
print("أسماء make_pipeline:", [name for name, _ in auto.steps])

try:
    auto.set_params(clf__C=1.0)          # الاسم الذي ظنّناه منطقياً
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[0][:86])

auto.set_params(logisticregression__C=1.0)   # الاسم الحقيقي
print("بعد التصحيح:", auto.get_params()["logisticregression__C"])
```

`make_pipeline` سمّى الخطوة `logisticregression` — لا `logistic_regression` ولا `lr`، ولا حتى `logistic-regression`. والرسالة نفسها من `set_params` تذكر الأسماء الصالحة كاملة، وهي كفيلة بأن تمنعك من التخمين في-named الشبكة.

	`Pipeline`	`make_pipeline`
صيغة الاسم	تكتبها أنت	اسم الصنف بحروف صغيرة
المقروئية في الشبكة	عالية: `clf__C`	منخفضة: `logisticregression__C`
إعادة تسمية خطوة	`pipe.steps[1] = ("lr", step)`	يلزمك معرفة الاسم المولّد
متى تستخدمه	أي بحث شبكي أو أنبوب إنتاجي	استكشاف سريع داخل دفتر

القاعدة: **لا تستخدم `make_pipeline` في أنبوب ستمرّره إلى `GridSearchCV`**. استخدمه للتجربة السريعة، ثم أعد الكتابة بـ `Pipeline` بأسماء محسوبة.

١٠٦ العناوين المتداخلة: داخل البحث الشبكي

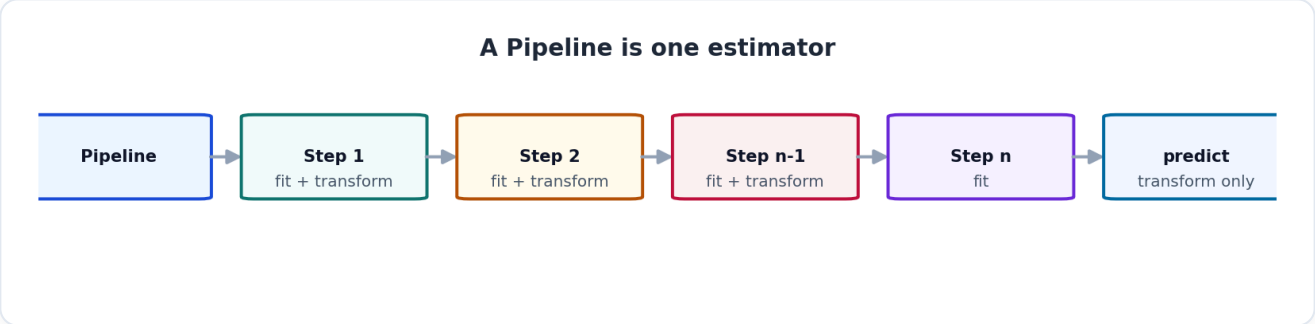
البارامتر داخل الخطوة يُعنونه بـ `اسم_الخطوة__اسم_البارامتر` — شرطتان سفليتان متتاليتان تفصلان الطبقتين:

فوق أنبوب: العناوين المتداخلة تعمل GridSearchCV python

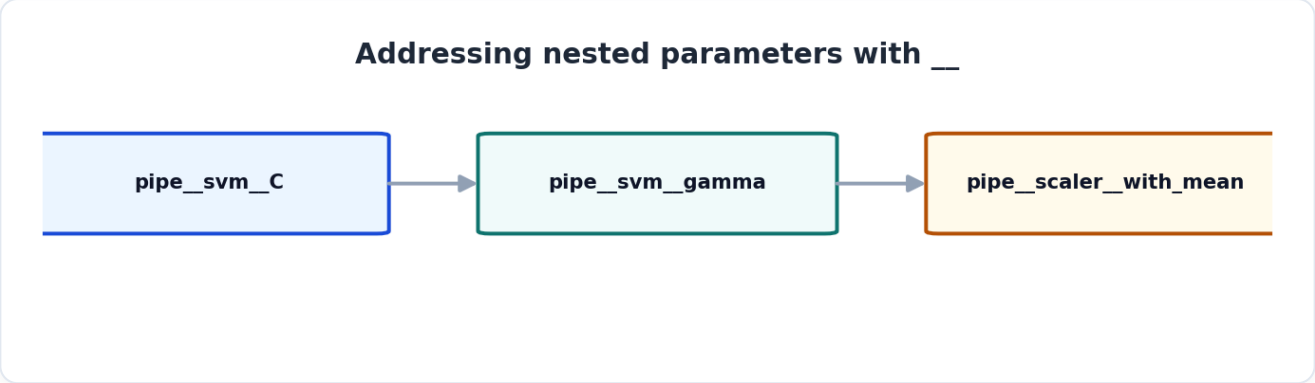
```python
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(
    Pipeline([("sc", StandardScaler()),
              ("clf", LogisticRegression(max_iter=5000))]),
    {"sc__with_mean": [True, False],
     "clf__C": [0.05, 0.3, 2.0]},
    cv=3, n_jobs=-1, return_train_score=True)
grid.fit(Xa, ya)

print("أفضل تركيبة:", grid.best_params_)
print(f"درجة التحقق: {grid.best_score_:.4f}")
print(f"درجة التدريب للفائز: "
      f"{grid.cv_results_['mean_train_score'][grid.best_index_]:.4f}")
print("مفاتيح الشبكة:", [k for k in grid.cv_results_ if k.startswith("param_")])
```

`return_train_score=True` هنا ليس ترفاً: الفجوة بين ٠٫٨٥ في التدريب و٠٫٦٦ في التحقق إفراط في ملاءمة بمقدار ١٩ نقطة، وهي الإشارة نفسها التي يفصلها الفصل ٨٠ بمنحنى التعلّم. ولو شغّلت البحث من دون `sc__with_mean` لكنتَ تُهمل محوراً كاملاً من فضاء الحلول بصمت.

وأكثر من ذلك: داخل كل طية من التحقق المتقاطع، تُنسخ الأنابيب كاملة ويُعاد تدريب المحوّلات من جديد على أجزائها وحدها. هذه هي «إعادة ضبط المحوّلات داخل كل طية» التي حذّر منها الفصل ٧٣، وهي بالضبط ما لا تستطيع يدك أن تفعله إذا نسختَ `scaler.fit(X)` مرة واحدة خارج الأنبوب.



الشكل ١٨٥ الأنبوب كائن واحد: fit ثم transform على كل خطوة، ثم predict، وكلها كائن واحد يمكن حفظه



الشكل ١٨٦ العناوين المتداخلة تُحلَّل بشرطتين سفليتين متتاليتين: اسم الخطوة ثم اسم البارامتر

١٠.٧ أنابيب حقيقية: نصوص ومعايرة احتمالية

الأنبوب متعدد المراحل في الحياة الحقيقية يتضمّن أحياناً **معايرة احتمالية** فوق نموذج متّجهات داعمة، لأن `LinearSVC` لا يعطي احتمالات. لاحظ أين توضع طبقة المعايرة: بعد المحوّل النصّي وقبل التسليم:

أنبوب نصّي كامل مع معايرة احتمالية python

```python
from sklearn.calibration import CalibratedClassifierCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

docs = [
    "free money click now win prize today",
    "limited offer buy cheap discount now",
    "urgent call now about your invoice payment",
    "the deployment failed on the production server",
    "meeting tomorrow at nine about project status",
    "team meeting notes attached for review",
    "please review the report before the deadline",
    "lunch at the cafe near the office tomorrow",
]
labels = np.array([1, 1, 1, 0, 0, 0, 0, 0])

text_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)),
    ("clf", CalibratedClassifierCV(LinearSVC(C=1.0), method="sigmoid", cv=2)),
])
text_pipeline.fit(docs, labels)

print("حجم مفردات TF-IDF:", len(text_pipeline.named_steps["tfidf"].vocabulary_))
proba = text_pipeline.predict_proba(["claim your free prize now"])
print("الاحتمال:", np.round(proba[0], 3),
      "| الفئة:", text_pipeline.classes_[proba.argmax()])
print("نوع الخطوة الأخيرة:", type(text_pipeline.named_steps["clf"]).__name__)
```

هذا الأنبوب يعمل على نص خام بلا تنظيف مسبق، ويعطي درجة يقين سليمة (٠٫٩٨١ للبريد المزعج) لا مجرّد تسمية. ولاحظ أن المعايرة تغيّر الواجهة: بعد الالتفاف لم تعد `named_steps["clf"]` هي `LinearSVC` بل `CalibratedClassifierCV` يحتضنه، فأي بحث شبكي على `clf__C` سيصبح `clf__estimator__C` — وهذا بالضبط ما يفاجئ المطوّرين.

> **⚠ المعايرة ليست مجانية**
>
> `CalibratedClassifierCV` يُعيد تدريب النموذج `cv` مرات إضافية على طيّات داخلية، فيضاعف زمن التدريب. مع نماذج باهتة و`cv=5`، ارفع المعايرة إلى الأنبوب بعد أن تتأكد أن النموذج النهائي يستحق ثمنها — لا في مسح أولي على مئة تركيبة.

١٠.٨ ماذا يوجد داخل الصندوق: أسماء الخصائص

الأنبوب يحتفظ بـ `feature_names_in_` إذا دُرِّب على `DataFrame`، وبسجّل `get_feature_names_out` يمرّر الأسماء عبر الخطوات. هذه تفاصيل صغيرة في التحليل، وحاسمة في الإنتاج:

```python
# الأنبوب يتذكّر أسماء الأعمدة التي دُرِّب عليها
import pandas as pd
from sklearn.impute import SimpleImputer

frame = pd.DataFrame(Xa[:, :5].copy(),
                     columns=["age", "income", "tenure", "balance", "score"])
frame.loc[frame.sample(frac=0.3, random_state=42).index, "score"] = np.nan

frame_pipe = Pipeline([("imp", SimpleImputer(strategy="median")),
                       ("clf", LogisticRegression(max_iter=5000))])
frame_pipe.fit(frame, ya)

print("feature_names_in_:", list(frame_pipe.feature_names_in_))
print("n_features_in_:", frame_pipe.n_features_in_)
print("الوسيط المستعاد للعمود المفقود:",
      round(frame_pipe.named_steps["imp"].statistics_[4], 4))
print("أسماء المخرجات:", list(frame_pipe[:-1].get_feature_names_out()))
```

`feature_names_in_` يظهر فقط عند التدريب على `DataFrame`؛ على `ndarray` لا يظهر أبداً. وهذه فخّ عميق: نموذج دُرِّب على `DataFrame` بأسماء أعمدة، ثم استُدعي `predict` على مصفوفة عادية، يطلق scikit-learn تحذيراً بالاسم `X does not have valid feature names` — وهو **تحذير لا خطأ**، أي أن التدريب يمرّ والنتائج تمرّ، لكنك فقدت الضمانة بأن العمود الثالث في الإدخال هو نفسه الثالث في التدريب.

١٠.٩ لماذا يهمّ هذا في الإنتاج

قابلية التسلسل (serialization) هي الفرق بين مشروع قابل للتسليم ومشروع محبوس في دفتر `Jupyter`. `Pipeline` كائن واحد، فيُحفظ في ملف واحد ويُستعاد في عملية منفصلة تماماً وينتج **بالضبط** نفس المخرجات:

```python
# حفظ واستعادة: نفس النموذج، نفس التنبؤات
import io
import joblib

buffer = io.BytesIO()
joblib.dump(pipe, buffer)
buffer.seek(0)
restored = joblib.load(buffer)

print(f"حجم الملف: {len(buffer.getvalue()) / 1024:.1f} كيلوبايت")
print("تطابق التنبؤ بعد الاستعادة:",
      bool((restored.predict(Xb) == pipe.predict(Xb)).all()))
print("نفس أسماء الخطوات:", [n for n, _ in restored.steps])
```

هذه ليست لعبة كتابية. كل ما في الفصول السابقة — محوّلات من الفصل ١٤، ومُنتقي من الفصل ٢٣، ومعايرة من الفصل ٧٧ — لا قيمة له في الإنتاج إن لم يُغلَّف معاً في كائن واحد. ولهذا نبدأ هذا الجزء من `Pipeline`: هو ثاني أهم صنف في scikit-learn بعد `BaseEstimator` نفسه، وهو الكائن الذي سيحمل كل ما نبنيه حتى الفصل ٨٩.

في مشروع حقيقي، لا يوجد `StandardScaler` عارٍ

في إحدى أنظمة التسعير التي رأيناها، كان `scaler.fit()` يُستدعى داخل دالة تحضير تُنادى مرة أسبوعياً على كامل السجل، ثم يُحفظ المعامل في متغيّر عام. عند أول إعادة تشغيل من زميل، تغيّر متوسط الفئة الواحدة بنسبة ٣٪، فتغيّرت التنبؤات على ١٨٪ من العملاء الصغار. لم يكن خطأً رياضياً — كان خطأً في **نطاق** ما يُدرَّب عليه. `Pipeline` لا يمنع هذا الخطأ تحديداً، لكنه يجعله أمام عينيك فوراً: `n_samples_seen_` يخبرك بالضبط كم صفاً رآه كل محوّل.

تمارين الفصل الرابع والثمانين

- **ملاحظة**

التمرين ١٠.١

على بيانات `make_classification` بـ `n_samples=500` و`n_features=30` و`n_informative=6` و`random_state=42`، ابنِ `Pipeline` من ثلاث خطوات: `SimpleImputer(strategy="median")` ثم `StandardScaler()` ثم `LogisticRegression(max_iter=5000)`. اجعل عمودين مفقودين في ٦٠ صفاً من `X_train`، ثم اطبع `n_samples_seen_` للمُطبِّع، وعدد الأعمدة التي بقيت بعد الاسترجاع، ودرجة الاختبار.

التمرين ١٠.٢

خذ الأنبوب نفسه من التمرين ١٠.١ وأضف رابعاً باسم `sel` من نوع `SelectKBest(f_classif, k=5)`. اكتب شبكة `{"imputer__strategy": ["median", "mean"], "clf__C": [0.5, 5.0]}` وطبّقها بـ `GridSearchCV(cv=4, return_train_score=True)`. اطبع `best_params_`، وفارق `mean_train_score` عن `mean_test_score` عند `best_index_`، ثم فسّر هذا الفارق بجملة واحدة.

التمرين ١٠.٣

أعطِ صديقاً الكود التالي ودعه يخمّن الخطأ قبل أن تشغّله. لماذا يفشل هذا السطر، وما الشكل الصحيح لعنونة بارامتر `C` داخل هذا الأنبوب؟

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = make_pipeline(StandardScaler(), LogisticRegression())
pipe.set_params(clf__C=10.0)
```

- **ملاحظة**

أنبوب ثلاثي المراحل مع استرجاع القيم المفقودة python

```python
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

Xf, yf = make_classification(n_samples=500, n_features=30, n_informative=6,
                             random_state=42)
Xa2, Xb2, ya2, yb2 = train_test_split(Xf, yf, test_size=0.2, random_state=42,
                                      stratify=yf)

miss = np.random.default_rng(0).choice(Xa2.shape[0], size=60, replace=False)
Xa2[miss, 3] = np.nan
Xa2[miss, 17] = np.nan

pipe3 = Pipeline([("imputer", SimpleImputer(strategy="median")),
                  ("scaler", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=5000))])
pipe3.fit(Xa2, ya2)

print(f"صفوف رآها المُطبِّع: {pipe3.named_steps['scaler'].n_samples_seen_:.0f}")
print(f"أعمدة بعد الاسترجاع: {pipe3.named_steps['imputer'].statistics_.size}")
print(f"مجموع وسائط الاسترجاع: {pipe3.named_steps['imputer'].statistics_.sum():.3f}")
print(f"درجة الاختبار: {pipe3.score(Xb2, yb2):.4f}")
```

الاسترجاع رأى ثلاثين عموداً لا ثمانية وعشرين، والوسائط المستعادة هي متوسطات أعمدة التدريب بعد حذف الصفوف المفقودة منها — وكل ذلك حدث داخل `fit` واحد، فلا وجود لاسترجاع «مُسرَّب» هنا.

> **ملاحظة**

شبكة على أنبوب رباعي: الفجوة بين التدريب والتحقق python

```python
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.model_selection import GridSearchCV

pipe4 = Pipeline([("imputer", SimpleImputer(strategy="median")),
                  ("scaler", StandardScaler()),
                  ("sel", SelectKBest(f_classif, k=5)),
                  ("clf", LogisticRegression(max_iter=5000))])

grid2 = GridSearchCV(pipe4,
                     {"imputer__strategy": ["median", "mean"],
                      "clf__C": [0.5, 5.0]},
                     cv=4, return_train_score=True)
grid2.fit(Xa2, ya2)

i = grid2.best_index_
tr = grid2.cv_results_["mean_train_score"][i]
te = grid2.cv_results_["mean_test_score"][i]
print("أفضل تركيبة:", grid2.best_params_)
print(f"تدريب: {tr:.4f} | تحقق: {te:.4f} | الفجوة: {tr - te:+.4f}")
```

الفجوة هنا صغيرة — ١٫٣ نقطة فقط — لأن تقييد الخصائص إلى خمس خفّف فراغ الانحدار اللوجيستي على ثلاثين خاصية. لو رفعتَ قيمة `C` أو أسقطتَ خطوة `sel` لاتّسع المدى. القاعدة التي يقرؤها الفصل ٨٠: الفجوة ليست خطأً، بل إشارة إلى المقدار الذي ما زال النموذج يحفظه من تفاصيل التدريب.

> **ملاحظة**

لماذا يفشل set_params، وكيف يُصحَّح python

```python
auto_pipe = make_pipeline(StandardScaler(), LogisticRegression())
print("الأسماء الحقيقية:", [n for n, _ in auto_pipe.steps])
try:
    auto_pipe.set_params(clf__C=10.0)
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[0][:86])
auto_pipe.set_params(logisticregression__C=10.0)
print("بعد التصحيح:", auto_pipe.get_params()["logisticregression__C"])
```

`make_pipeline` لا يخمّن أسماءً جميلة — هو يخفض اسم الصنف كله إلى أحرف صغيرة بلا فواصل. الحلّ إمّا استخدام الاسم المولّد، وإمّا (وهو الأفضل) إعادة كتابة الأنبوب بـ `Pipeline` بأسماء صريحة مثل `("clf", LogisticRegression())`.

> **ملاحظة**

١٠.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
قبل `scaler.fit(X)` **التقسيم**	المتوسط والانحراف يشملان صفوف الاختبار	داخل `StandardScaler()` `Pipeline` يتعلّم داخل `fit`
على `selector.fit(X, y)` **كامل البيانات**	الاختيار يستجيب لتسميات صف الاختبار لا لسلوك التدريب	ضع `SelectKBest` كخطوة في الأنبوب
`auto.set_params(clf__C=…)` **على** `make_pipeline`	`ValueError: Invalid parameter 'clf'`	استخدم `Pipeline` بأسماء صريحة، أو الاسم المولّد كاملاً
`pipe[:-1].transform` **وأنت لم تُدرب بعد**	`NotFittedError`	`pipe.fit(X_train, y_train)` أولاً، دائماً
تدريب على `DataFrame` **واستدعاء** `predict` **على** `ndarray`	تحذير `X does not have valid feature names` وضمانات ضائعة	حافظ على `DataFrame` في الإنتاج أو سمِّ الأعمدة صراحة
`CalibratedClassifierCV` **في الأنبوب قبل البحث**	عنوان بارامتر النموذج يصبح `clf__estimator__C` ويضاعف زمن الشبكة	ابحث في النموذج وحده، ثم أضف المعايرة بعد اختياره

١٠.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
خط الأنابيب	كائن يجمع خطوات متسلسلة في مُقدِّر واحد	Pipeline
خطوة	مُقدِّر باسم داخل الأنبوب	Named step
عنوان متداخل	`step__param` للوصول لبارامتر داخل خطوة	Nested parameter
تسريب	معلومة من بيانات التقييم تصل إلى معاملات التدريب	Data leakage
بيانات المعايرة	الطيّات الداخلية التي تستخدمها المعايرة الاحتمالية	Calibration folds
حالة داخلية	ما حفظه المُقدِّر بعد `fit` وينفّذه في `transform`	Fitted state
تجزئة الأنبوب	قصّ الأنبوب بـ `[:-1]` للحصول على المحوّلات وحدها	Pipeline slicing

الجزء ١٠ · الأنابيب والتركيب

الفصل ٨٥ — ColumnTransformer: تركيب الأعمدة

الاختيار بالاسم والنمط، والمُحدِّد make_column_selector

الوحدة **scikit-learn ١.٩** الحجم **٤٣٤** كلمة تقريباً

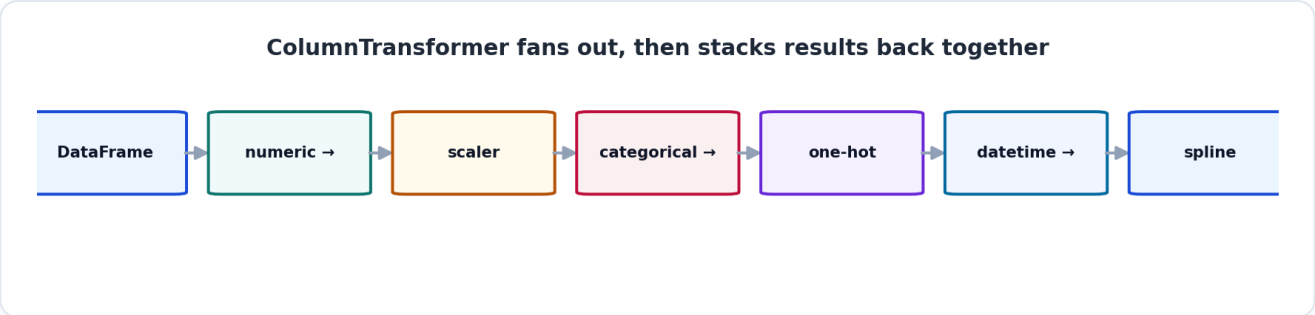
ما الذي ستتقنه في هذا الفصل

- أن يبني معالجة مختلفة للأعمدة العددية والفئوية داخل ColumnTransformer
- أن يختار الأعمدة بالاسم أو make_column_selector
- أن يضبط remainder، والمعامل verbose_feature_names_out وفق عقد البيانات
- أن يدمج ColumnTransformer داخل Pipeline قابل للتقييم والحفظ

١٠.١ عمودان، وسلوكان لا يجوز خلطهما

تصل بيانات العملاء في جدول واحد: الدخل رقم، والمدينة نص، وعدد الشكاوى عدد صحيح لكنه قد يكون مفقوداً. تمريرها إلى `StandardScaler` كلها خطأ دلالي، وترميزها كلها ترميزاً واحداً قد يوهم النموذج بأن ترتيب أسماء المدن له معنى. نحتاج إلى **مسارات** (branches): تعويض وتحجيم للأرقام، وتعويض وترميز للفئات، ثم وصل الناتجين في مصفوفة خصائص واحدة.

`sklearn.compose.ColumnTransformer` ينفذ هذا العقد. يطبق كل محوّل على الأعمدة المحددة فقط، ويلائم المحوّلات داخل مجموعة التدريب عندما يوضع في `Pipeline`. لا تكتب تقسيمات `X[:, :3]` الهشة إذا كان اسم العمود متاحاً؛ تغيير ترتيب المصدر لا ينبغي أن يبدل معنى النموذج.



الشكل ١٨٧تركيب الأعمدة بمسارات معالجة مستقلة

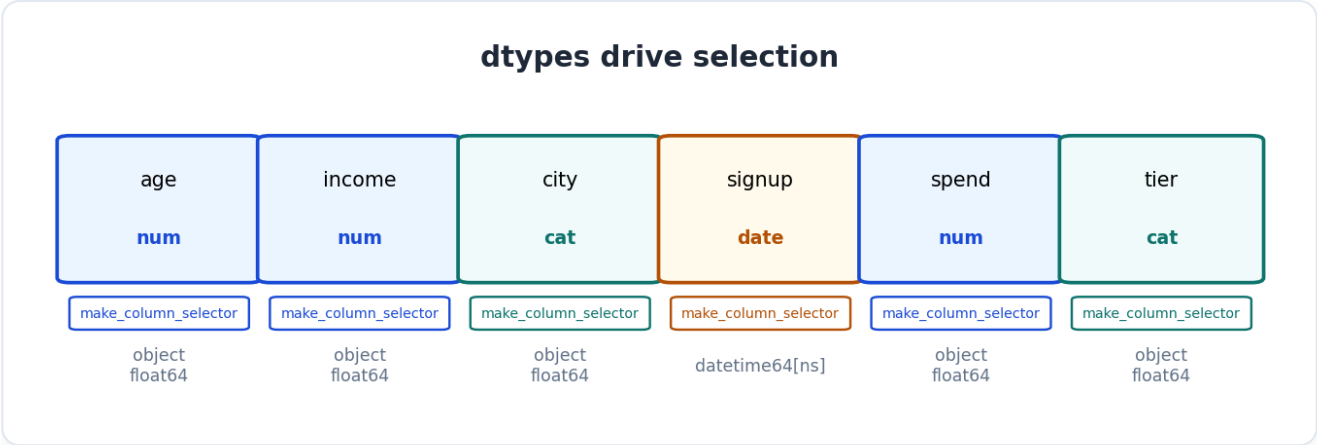
مساران عددي وفئوي في جدول واحد python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = pd.DataFrame({
    "income": [35_000, 52_000, 41_000, 68_000],
    "visits": [2, 5, 1, 4],
    "city": ["الزرقاء" ,"عمّان" ,"إربد" ,"عمّان"],
})
prep = ColumnTransformer([
    ("num", StandardScaler(), ["income", "visits"]),
    ("city", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
["city"]),
])
Xt = prep.fit_transform(X)
print("شكل المصفوفة:", Xt.shape)
print("الأسماء:", prep.get_feature_names_out().tolist())
```

أصبحت الفئة عدة أعمدة ثنائية، وبقيت الأرقام على مقياس موحّد. البادئات `num__` و`city__` توضح مصدر كل عمود ناتج، وهي مفيدة عند تفسير الخصائص أو مقارنة نسخ المعالجة.

١٠.٢ الاختيار بالاسم والنوع

قائمة الأسماء صريحة وسهلة المراجعة، لكنها قد تحتاج تحديثاً عندما يضاف عمود جديد. يوفر `sklearn.compose.make_column_selector` اختياراً حسب dtype أو تعبير نمطي للأسماء. الاختيار بالنوع مريح عندما يكون عقد المخطط موثوقاً؛ لكنه ليس بديلاً عن تعريف معنى العمود، فقد يصل رقم العميل كعدد ويُعامل خطأً كميزة مستمرة.



الشكل ١٨٨محدد الأعمدة في مسارات التحويل

اختيار رقمي تلقائي واختيار نصي python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = pd.DataFrame({
    "age": [21, 42, 36],
    "balance": [120.0, 480.0, 230.0],
    "segment": ["أساسي" ,"ذهبي" ,"أساسي"],
})
prep = ColumnTransformer([
    ("numeric", StandardScaler(),
     make_column_selector(dtype_include="number")),
    ("category", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     make_column_selector(dtype_include="object")),
])
prep.fit(X)
print("الأعمدة:", prep.feature_names_in_.tolist())
print("الأعمدة الناتجة:", prep.get_feature_names_out().tolist())
```

تذكر أن `make_column_selector` يختار بحسب ما يراه في بيانات التدريب. وإذا تغيرت أنواع الأعمدة بين التدريب والإنتاج، فالاختيار قد يختلف أو يفشل؛ تحقّق من مخطط المدخلات قبل الوصول إلى المحوّل.

١٠.٣ الأعمدة غير المعالجة والمخرجات المتفرقة

يحدد `remainder` مصير الأعمدة التي لم تدخل أي مسار: القيمة الافتراضية `"drop"` تسقطها، و`"passthrough"` تمررها كما هي، ويمكن استخدام محوّل لتطبيق معالجة عليها. مرّرها فقط إذا كنت تعرف لماذا ستصل إلى النموذج؛ عمود معرّف أو تاريخ خام قد يصبح إشارة زائفة أو تسريباً.

قد ينتج `OneHotEncoder` مصفوفة متفرقة. إذا كانت كثافة المخرجات منخفضة، فالحفاظ على التمثيل المتفرق يوفر ذاكرة؛ أما `sparse_threshold` فيتحكم في اختيار المخرجات المتفرقة بحسب كثافتها. لا تحوّل إلى dense لمجرد سهولة الطباعة إذا كان عدد الفئات كبيراً. في بيئة هذا الكتاب يُضبط خرج المحوّلات إلى DataFrame؛ لذلك تستخدم أمثلة الترميز `sparse_output=False`، بينما يبقى الخرج المتفرق هو الافتراضي المعتاد عند عدم طلب DataFrame.

مصير الأعمدة غير المختارة — python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

X = pd.DataFrame({
    "age": [20, 40, 60],
    "region_code": [3, 5, 3],
    "record_id": [101, 102, 103],
})
drop_rest = ColumnTransformer(
    [("age", StandardScaler(), ["age"])], remainder="drop")
keep_rest = ColumnTransformer(
    [("age", StandardScaler(), ["age"])], remainder="passthrough")
print("مع الإسقاط:", drop_rest.fit_transform(X).shape)
print("مع المرور:", keep_rest.fit_transform(X).shape)
print("أسماء المرور:", keep_rest.get_feature_names_out().tolist())
```

لا يعني `passthrough` أن الأعمدة المتبقية صحيحة للنموذج. هنا يعبر `record_id` إلى المخرجات، وهو مثال واضح على أن سلامة البنية لا تضمن سلامة المعنى.

١٠.٤ في خط إنتاج كامل

التركيب الصحيح يربط المعالجة بالمقدّر داخل `sklearn.pipeline.Pipeline`. بذلك تتعلم القيم الوسيطة من التدريب في كل طية، ويصبح البحث عن معاملات المصنّف والمعالجة عملية واحدة. استخدم عناوين خطوات واضحة حتى يمكن ضبطها لاحقاً بصيغة `prep__numeric__...` أو `model__...`.

المعالجة جزء من النموذج python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import cross_val_score

X = pd.DataFrame({
    "age": [22, 35, 48, 29, 53, 41, 31, 60],
    "balance": [20, 100, 65, 40, 130, 80, 55, 150],
    "plan": ["A", "B", "A", "A", "B", "B", "A", "B"],
})
y = [0, 1, 1, 0, 1, 1, 0, 1]
numeric = Pipeline([("impute", SimpleImputer(strategy="median")),
                    ("scale", StandardScaler())])
categorical = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                        ("encode", OneHotEncoder(handle_unknown="ignore",
                                                 sparse_output=False))])
prep = ColumnTransformer([("num", numeric, ["age", "balance"]),
                          ("cat", categorical, ["plan"])])
model = Pipeline([("prep", prep), ("clf", LogisticRegression())])
scores = cross_val_score(model, X, y, cv=4, scoring="accuracy")
print("درجات الطيات:", scores.round(3).tolist())
```

العينة هنا صغيرة للتوضيح لا لإعلان أداء صالح للنشر. في مشروعك الفعلي اختر طيات تراعي التوازن أو المجموعات أو الزمن وفق طبيعة المسألة، كما في الجزء التاسع.

في الواقع

أفضل عقد لمخطط إنتاجي يجمع بين اختيار الأعمدة بالاسم، واختبارات تحقق مسبقة للأنواع والأعمدة المطلوبة، و `handle_unknown="ignore"` عندما يُتوقع ظهور فئة جديدة. تجاهل فئة جديدة يمنع تعطل الخدمة، لكنه لا يغني عن مراقبة تواتر تلك الفئة.

تمارين الفصل الخامس والثمانين

- **ملاحظة**

التمرين ١٠.١

أنشئ DataFrame فيه عمودان رقميان وعمود فئوي، ثم ابنِ `ColumnTransformer` يطبق `StandardScaler` و`OneHotEncoder`. اطبع الأسماء الناتجة.

التمرين ١٠.٢

قارن `remainder="drop"` و`remainder="passthrough"` على جدول يحتوي عمود معرّف. ما الخطر من تمرير المعرّف إلى نموذج؟

التمرين ١٠.٣

ضع المعالجة والمصنّف في `Pipeline` وشغّل `cross_val_score`. تحقق أن كل طية تلائم المعالجة داخل التدريب.

- **ملاحظة**

الأسماء الواضحة بعد التركيب — python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = pd.DataFrame({"height": [160, 175, 168], "weight": [58, 80, 65],
                  "team": ["أ" ,"ب" ,"أ"]})
ct = ColumnTransformer([
    ("num", StandardScaler(), ["height", "weight"]),
    ("team", OneHotEncoder(sparse_output=False), ["team"]),
])
ct.fit(X)
print(ct.get_feature_names_out().tolist())
```

> **ملاحظة**

كشف مرور المعرّف python

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

X = pd.DataFrame({"value": [10, 20, 30], "id": [700, 701, 702]})
ct = ColumnTransformer([("value", StandardScaler(), ["value"])],
                       remainder="passthrough")
print(ct.fit_transform(X).shape)
print("الأسماء:", ct.get_feature_names_out().tolist())
```

في التطبيق يجب إسقاط معرّف السجل ما لم يكن له معنى تنبؤي مشروع ومستقل عن الهدف.

> 📝 **ملاحظة**

١٠.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
اختيار الأعمدة بمواقع ثابتة	تغيير ترتيب المصدر يبدل معنى الخصائص	استخدم أسماء الأعمدة أو محدداً موثقاً
إسقاط القيم المفقودة قبل الفصل	يتعلم التعويض من بيانات التقييم	ضع `SimpleImputer` داخل المسار
تمرير كل الأعمدة المتبقية	يتسلل معرّف أو هدف مشتق إلى النموذج	استخدم `remainder="drop"` واسمح صراحةً بما يلزم
نسيان فئة جديدة	يتعطل التحويل عند الإنتاج	قرر سياسة `handle_unknown` واختبرها

١٠.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مسار التحويل	معالجة متخصصة لمجموعة أعمدة	transformer branch
بقية الأعمدة	أعمدة لم يذكرها أي مسار	remainder
كثافة المصفوفة	نسبة العناصر غير الصفرية	matrix density
محدد الأعمدة	قاعدة اختيار الأعمدة آلياً	column selector

الجزء ١٠ · الأنابيب والتركيب

الفصل ٨٦ — FeatureUnion والتركيبات المتقدمة

دمج مسارات متوازية، والتداخل، وحالات الاستخدام

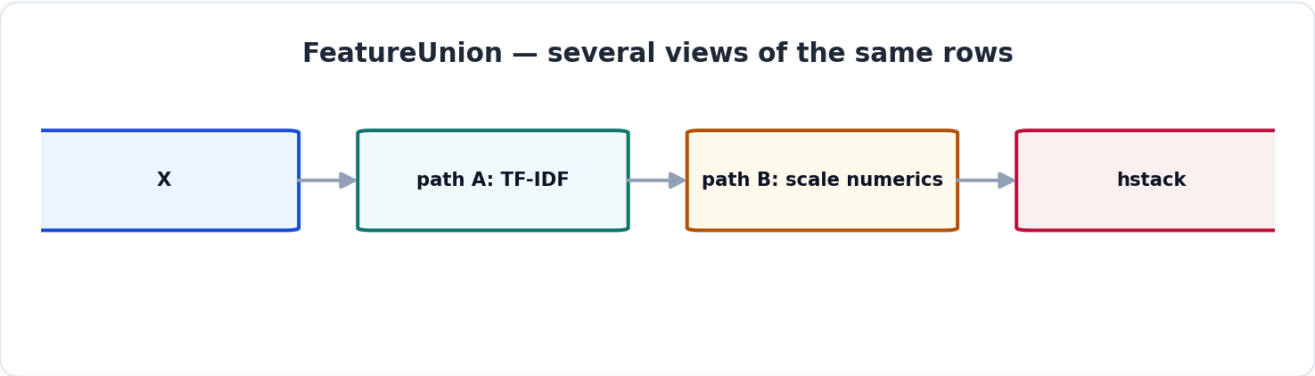
الوحدة **scikit-learn ١.٩** الحجم **٣٥٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يشرح الفرق بين الصنفين FeatureUnion و ColumnTransformer
- أن يدمج عدة محوّلات متوازية في مصفوفة خصائص واحدة
- أن يختبر شكل المخرجات وأسماء السمات قبل تدريب النموذج
- أن يضع التركيب المتوازي داخل Pipeline ويضبط معاملاته بأمان

١٠.١ توازٍ في المعالجة لا في الصفوف

لديك نص واحد وتريد تمثيله بتكرارات الكلمات وبـ TF-IDF، أو سلسلة رقمية تريد خصائصها الأصلية مع حدود كثيرة الحدود. في الحالتين تطبق محوّلات مختلفة على **المدخل نفسه** ثم تلصق الأعمدة الناتجة أفقياً. هذه وظيفة `sklearn.pipeline.FeatureUnion`: توصيل تمثيلات متوازية لا اختيار أعمدة مختلفة.



الشكل ١٨٩ مسارات متوازية تتحد في مصفوفة خصائص واحدة

ضم تمثيلين متوازيين python

```python
import numpy as np
from sklearn.pipeline import FeatureUnion
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.datasets import make_regression

X, _ = make_regression(n_samples=30, n_features=2, random_state=42)
union = FeatureUnion([
    ("scaled", StandardScaler()),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
])
Xt = union.fit_transform(X)
print("الأصل:", X.shape, "| الناتج:", Xt.shape)
print("الأسماء:", union.get_feature_names_out().tolist())
```

يحتوي الناتج معلومات مكررة عمداً: فرع يحتفظ بالقياس المعياري، وفرع يبني تفاعلات وتربيعات. هذا قرار نمذجة، لا تحسين مضمون؛ قد تزيد الأبعاد والارتباطات والكلفة.

١٠.٢ FeatureUnion أم ColumnTransformer؟

القاعدة العملية بسيطة: استخدم `ColumnTransformer` عندما تختلف المعالجة باختلاف **الأعمدة**؛ استخدم `FeatureUnion` عندما تريد **عدة تمثيلات للمدخل نفسه**. ويمكن تركيب الاثنين: يبدأ `ColumnTransformer` بمعالجة أنواع الأعمدة، ثم يدمج `FeatureUnion` ذلك التمثيل مع فرع آخر. لكن كل فرع يجب أن يستقبل نوع بيانات يستطيع التعامل معه.

السؤال	FeatureUnion	ColumnTransformer
ما الذي يتغير بين الفروع؟	المحوّل، على المدخل نفسه	الأعمدة التي يتلقاها المحوّل
كيف تُدمج النتائج؟	وصل الخصائص الناتجة	وصل خصائص من أعمدة مختارة
مثال نموذجي	ميزات الكلمات والـ n-grams	تحجيم العمر وترميز المدينة
ما الذي يجب مراجعته؟	تكرار المعلومات وكثرة الأبعاد	الأعمدة غير المختارة ومصيرها

الفرق في أثر تحديد الأعمدة python

```python
import numpy as np
from sklearn.pipeline import FeatureUnion
from sklearn.preprocessing import FunctionTransformer, StandardScaler
from sklearn.compose import ColumnTransformer

X = np.array([[1.0, 10.0], [2.0, 20.0], [3.0, 30.0]])
union = FeatureUnion([
    ("all_scaled", StandardScaler()),
    ("all_raw", FunctionTransformer(validate=False)),
])
columns = ColumnTransformer([
    ("first", StandardScaler(), [0]),
    ("second_raw", "passthrough", [1]),
])
print("FeatureUnion:", union.fit_transform(X).shape)
print("ColumnTransformer:", columns.fit_transform(X).shape)
```

في هذا المثال يعيد فرع `all_raw` المدخل كما هو؛ لذا يضاعف `FeatureUnion` الأعمدة، بينما ينتج `ColumnTransformer` عمودين من فرعين مختلفين. افحص الشكل والأسماء دائماً، خصوصاً عند وجود محوّل يغيّر عدد الصفوف أو يفترض نوعاً مختلفاً من المدخل.

١٠.٣ بناء تركيب قابل للضبط

عندما تسمّي الفروع، تسمح لك واجهة `get_params` بتحديد معاملات داخلها بصيغة `اسم_الفرع__المعامل`. وهذا يجعل البحث ممكناً داخل خط الأنابيب كله. لا تبدأ بشبكة ضخمة تجمع اختيارات كثيرة بلا فرضية؛ كل مرشح إضافي يزيد كلفة البحث واحتمال اختيار نتيجة محظوظة.

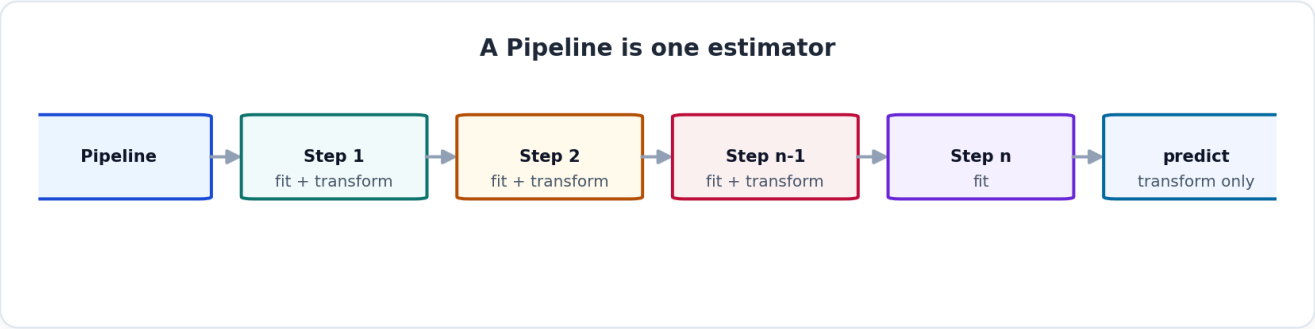
تسمية الفروع وفحص المعاملات python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
features = FeatureUnion([
    ("scaled", StandardScaler()),
    ("selected", SelectKBest(f_classif, k=10)),
])
pipe = Pipeline([("features", features),
                 ("clf", LogisticRegression(max_iter=3000))])
print("معامل فرع الاختيار:", pipe.get_params()["features__selected__k"])
pipe.fit(X, y)
print("شكل الخصائص المدمجة:",
      pipe.named_steps["features"].transform(X).shape)
```

الفرعان هنا يعالجان المصابيح نفسها، لكنهما ينتجان مقياسين مختلفين للمعلومة نفسها. قبل اعتماد هذا التصميم اختبر أثره خارج العينة، ولا تعتبر زيادة الخصائص تحسناً بذاتها.

١٠٤ حين يتغير عدد المخرجات

`FeatureUnion` يدمج أعمدة الفروع على مستوى الصفوف؛ لذلك يجب أن يتوافق عدد عينات المخرجات بين الفروع. المحوّل الذي يسقط صفوفاً أو يعيد مجموعة مرتبة مختلفة يكسر هذا العقد. كما أن تكرار أعمدة أو خصائص مترابطة قد يربك النموذج ويزيد كلفة الذاكرة. استخدم `transformer_weights` فقط إذا كان لديك مبرر لتغيير وزن فرع كامل، ثم قيّم القرار بالطيات.



الشكل ١٩٠ تركيب خط أنابيب من مراحل متتابعة ومتوازية

تقييم التمثيلين داخل الطيات python

```python
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=240, n_features=12, n_informative=5,
                           random_state=42)
union = FeatureUnion([("scale", StandardScaler()),
                      ("select", SelectKBest(f_classif, k=5))])
model = Pipeline([("union", union),
                  ("clf", LogisticRegression(max_iter=1000))])
scores = cross_val_score(model, X, y, cv=4, scoring="accuracy")
print("درجات الطيات:", scores.round(3).tolist())
```

لاحظ أن الاختيار والتقييس داخل الأنبوب، ولذلك يعاد ملاءمتهما على تدريب كل طية. خارج الأنبوب، اختيار الخصائص مرة واحدة قبل التقييم قد يسرّب معلومات الهدف، حتى لو كان محوّل الخصائص غير خاضع للإشراف.

في الواقع

في معالجة النصوص قد يكون الجمع بين unigram، bigram مفيداً لأن كل فرع يلتقط سياقاً مختلفاً. لكن جمع تمثيلات متعددة يرفع حجم المصفوفة المتفرقة؛ قِس استهلاك الذاكرة وزمن الملاءمة، ولا تحوّلها إلى dense ما لم تعرف أن الحجم آمن.

تمارين الفصل السادس والثمانين

- **ملاحظة**

التمرين ١٠.١

أنشئ `FeatureUnion` من `StandardScaler` و `PolynomialFeatures` على بيانات ذات ثلاثة أعمدة، واطبع شكل الإدخال والإخراج.

التمرين ١٠.٢

أنشئ `ColumnTransformer` يقيّس عموداً ويمرر آخر، وقارنه بـ `FeatureUnion` يعالج العمودين نفسيهما بفرعين.

التمرين ١٠.٣

ضع `FeatureUnion` ومصنفاً في `Pipeline` ونفّذ تحققاً متقاطعاً. عدّل معامل فرع باسم واضح باستخدام `set_params` .

- **ملاحظة**

```python
عدّ الخصائص قبل الدمج وبعده
import numpy as np
from sklearn.pipeline import FeatureUnion
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

X = np.arange(18, dtype=float).reshape(6, 3)
u = FeatureUnion([
    ("scale", StandardScaler()),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
])
print("الأشكال:", X.shape, u.fit_transform(X).shape)
```

> **ملاحظة**

اختلاف معنى التركيبين python

```python
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import FeatureUnion
from sklearn.preprocessing import StandardScaler

X = np.arange(12, dtype=float).reshape(6, 2)
ct = ColumnTransformer([("left", StandardScaler(), [0]),
                        ("right", "passthrough", [1])])
fu = FeatureUnion([("scaled", StandardScaler()),
                   ("raw", "passthrough")])
print("الأعمدة:", ct.fit_transform(X).shape, fu.fit_transform(X).shape)
```

`ColumnTransformer` يوزع الأعمدة، و `FeatureUnion` يكرر المدخل كله عبر الفروع.

> **ملاحظة**

١٠.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام FeatureUnion لتوزيع الأعمدة	كل فرع يعالج كامل المدخل	استخدم `ColumnTransformer`
تكرار تمثيلات مترابطة بلا قياس	يزيد الحجم وقد يضر التعميم	قارن داخل تحقق متقاطع
محوّل يغيّر عدد الصفوف	لا تتوافق المخرجات عند الدمج	حافظ على ترتيب وعدد العينات في كل فرع
اختيار الخصائص قبل الطيات	تتسرب معلومة الهدف إلى التقييم	ضع المحدد داخل الأنبوب

١٠.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تمثيل متوازٍ	تحويل مستقل للمدخل نفسه	parallel representation
اتحاد الخصائص	وصل مخرجات المحوّلات أفقياً	feature union
فرع	مسار تحويل مسمّى	branch
تداخل المعاملات	الوصول لمعامل داخل محوّل مركب	nested parameters

الجزء ١٠ · الأنابيب والتركيب

الفصل ٨٧ — تحويل الهدف وتحويل المدخلات

TransformedTargetRegressor و FunctionTransformer وتحويل الهدف

الوحدة **scikit-learn ١.٩** الحجم **١٤٧٩** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يميّز بين TransformedTargetRegressor و FunctionTransformer ويملك قراراً صريحاً بين القصتين
- أن يقيس أثر تحويل الهدف على عشرة أقسام فيرى انقلاب R^2 من سالب إلى موجب وانخفاض MAPE إلى الثلث
- أن يشرح عقد func و inverse_func ومتى يبطل كل منهما، وكيف يقرأ تحذير check_inverse
- أن يبني دالة تحويل هدف خاصة بتكلفة معروفة ويقيس ما يشتريه check_inverse=False على شبكة بحث
- أن يحوّل المدخلات داخل الأنبوب مع الحفاظ على أسماء الأعمدة، ويعرف متى تكسر الدالةُ اللطيفة عموداً

١٠.١ قصّتان مختلفتان تلبسان قناعاً واحداً

لنبدأ من مشهد يتكرّر في كل مشروع تعيده. لديك هدف مبيعات يمتد من ثمانية عشر ريالاً إلى مئة وعشرين ألفاً، وأغلب الصفوف تقع تحت المئتين. تسأل زميلك أين المشكلة، فيقول: «بدّل التحويل». تقصد: أيّ تحويل؟ وهنا تبدأ المشكلة الحقيقية، لأن في `sklearn` صنفاً يُغلّف الانحدار ليُغيّر الهدف الذي يتدرّب عليه، وصنفاً آخر يجلس داخل الأنبوب ليُغيّر المدخلات التي يمرّ بها. الاثنان يبدوان في سطر واحد، ويصلحان علامتين مختلفتين تماماً.

الصنف الأول `sklearn.compose.TransformedTargetRegressor`: لا يلمس `X` إطلاقاً. يحوّل `y` إلى `z = func(y)`، ويمنح `z` إلى الانحدار، ثم يعيد `inverse_func` إلى تنبؤاته. هو غلاف حول **التسمية**، ويضمن أن العكس يُطبَّق في لحظة التنبؤ لا في رأسك.

الصنف الثاني `sklearn.pipeline.FunctionTransformer`: يجلس كخطوة بين محوّلاتك، ويمرّر `X` عبر دالة تكتبها أنت، ثم يمرّر الناتج للخطوة التالية. هو أداة **المدخلات**، ولا يعرف عن `y` شيئاً.

الخطأ الذي يكلّف المشاريع وقتاً ليس أن تخلط بينهما، بل أن تعتقد أنهما بديلان. لنقيس الفرق على بيانات حقيقية اصطناعية أولاً، ثم نعود إلى الآليات.

١٠.٢ الهدف الذي يكذب على المقياس الخطي

لنصنع هدفاً حقيقي التوزيع لا مفتعَل البساطة. نولّد `y` لوغاريتمياً طبيعياً: خطّاً خطياً في الميزات، مصروفاً على أُسّ أربعين، ثم مضروباً في ضجيج لوغاريتمي صغير. هذا هو شكل الهدف الذي تنتجه عدّادات المرات، وإيرادات المتاجر، وأحجام الملفات، ومعدّلات الحوادث في أي نظام إنتاج حقيقي تقريباً.

هدف لوغاريتمي عادي: تقسيم، ثم عشرة أقسام python

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.compose import TransformedTargetRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import (mean_absolute_error,
mean_absolute_percentage_error,
                             r2_score)

X, z = make_regression(n_samples=300, n_features=5, noise=25, random_state=42)
rng = np.random.default_rng(42)
y = np.exp(z / 40 + rng.normal(0.0, 0.30, size=z.size))
print(f"الهدف: أصغر قيمة {y.min():.3f} | وسيط {np.median(y):.2f} | أكبر
{y.max():.2f}")

rows = {"خام": [], "محوَّل": []}
for seed in range(10):
    Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.25, random_state=seed)
    models = {
        "خام": LinearRegression().fit(Xa, ya),
        "محوَّل": TransformedTargetRegressor(
            regressor=LinearRegression(), func=np.log1p,
            inverse_func=np.expm1).fit(Xa, ya),
    }
    for name, model in models.items():
        p = np.clip(model.predict(Xb), 0.0, None)
        rows[name].append([
            r2_score(yb, p), mean_absolute_error(yb, p),
            100 * mean_absolute_percentage_error(yb, p),
            r2_score(np.log1p(yb), np.log1p(p)),
        ])

for name, v in rows.items():
    v = np.array(v)
    print(f"{name}: R2={v[:,0].mean():+.3f} | MAE={v[:,1].mean():.2f} | "
          f"MAPE={v[:,2].mean():.0f}% | log-R2={v[:,3].mean():+.3f}")
print("R2 خام — عبر الأقسام  :", np.round(np.array(rows["خام"])[:, 0], 3))
print("R2 محوَّل — عبر الأقسام:", np.round(np.array(rows["محوَّل"])[:, 0], 3))
```

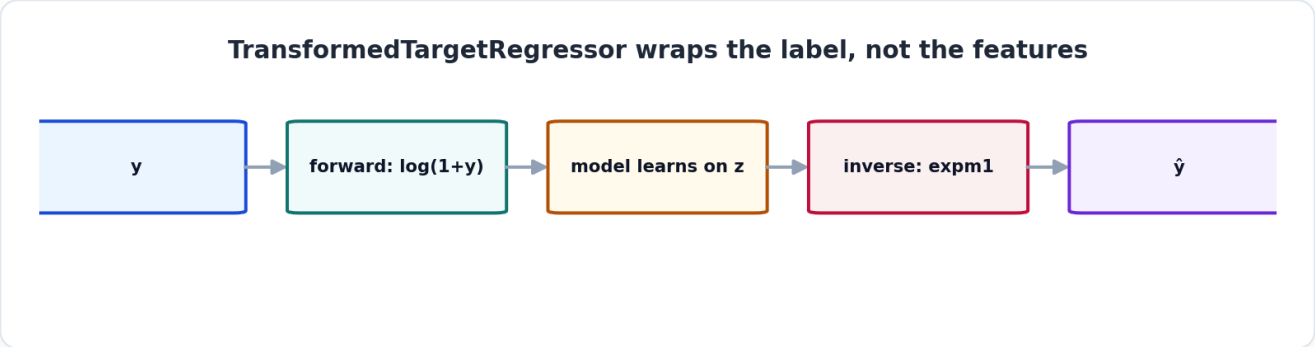
```
▸ OUTPUT الهدف: أصغر قيمة 0.001 | وسيط 1.16 | أكبر 1342.12
خام: R2=-0.193 | MAE=25.36 | MAPE=1141% | log-R2=-0.301
محوَّل: R2=+0.318 | MAE=13.39 | MAPE=169% | log-R2=+0.800
R2 عبر الأقسام — خام : [0.908- 0.549- 0.845- 0.28  0.25  0.1   0.085- 0.257- 0.187- 0.101
R2 عبر الأقسام — محوَّل: [0.536  0.17 0.504 0.511 0.126 0.147 0.208 0.167 0.359 0.454]
```

اقرأ الأرقام من اليسار لا من اليمين. الانحدار الخام حصل على `R2` متوسّطه **-٠٫١٩٣** على المقياس الخام: أي أسوأ من ضبط المتوسّط على كل صف. الغلاف المحوَّل حصل على **+٠٫٣١٨**، أي تحوّل من «النموذج لا يفسّر شيئاً» إلى «النموذج

يفسّر ثلث التباين»، و `MAPE` هبط من **١١٤١٪** إلى **١٦٩٪** — أي صار الخطأ سبعة أضعاف أصغر كنسبة، و `MAE` نزل من **٢٥٫٣٦** إلى **١٣٫٣٩**.

لكن الصفّ الأخير أهمّ من المتوسّطات: `R2` الخام يقفز بين -٠٫٩٠٨ و+٠٫٢٨٠ عبر الأقسام العشرة — أي أن النموذج نفسه يتغيّر حكمه على جودته بحسب أيّ صفوف وقعت في مجموعة الاختبار، وهيضيع نصف التشغيلات تحت الصفر. أمّا الغلاف فلا ينزل تحت **٠٫١٢٦** ولا يتجاوز **٠٫٥٣٦**. الفارق الحقيقي ليس في المتوسّط، بل في أن التحويل حوّل النموذج من **غير قابل للتكرار** إلى قابل له.

> *لماذا ينجح هذا؟ لأن `log1p` يجعل الهدف شبه خطي في `X`: الضجيج اللوغاريتمي يصير **متساوي التباين**، فيستعيد المربعات الصغرى كفاءةً تفقدها تماماً حين يكون الضجيج يتضخّم مع حجم القيمة. انظر إلى المعادلة التي يختبرها الفصل ٨٤: في المقياس اللوغاريتمي، البواقي محيطة بالصفر، وفي المقياس الخام كانت تنفجر في الذيل.*



الشكل ١٩١المسار الذي يسلكه الهدف: من y إلى التحويل الأمامي، فالنموذج يتعلّم على z، ثم العكس ينتج التنبؤ على المقياس الأصلي

هذا الشكل — وهو أبسط صورة في الفصل كله — يوضح لماذا `TransformedTargetRegressor` يوصف نفسه بأنه **يلفّ التسمية لا الخصائص**. لاحظ أن `X` لا يظهر فيه البتة: المدخلات تمرّ كما هي، والهدف وحده من يسلك رحلة ذهاب وإياب.

١٠.٣ العقد: ماذا يفعل الغلاف فعلاً؟

`TransformedTargetRegressor` كائن واحد يحتفظ بأربع قطع: النموذج الداخلي بعد التدريب تحت `regressor_`، والمحوّل الذي استُعمل على التسمية تحت `transformer_`، ومصفوفة `n_features_in_` التي رآها، وحالة `func` و `inverse_func`. لنفتح الكائن:

تشريح الغلاف: ماذا رآه الانحدار فعلاً python

```
Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.25, random_state=42)
raw = LinearRegression().fit(Xa, ya)
ttr = TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                                 inverse_func=np.expm1).fit(Xa, ya)
print("نوع المحوّل الداخلي:", type(ttr.transformer_).__name__)
print("func:", ttr.transformer_.func, "| inverse_func:",
ttr.transformer_.inverse_func)
head = ya[:4].reshape(-1, 1)
print("ما رآه الانحدار:", ttr.transformer_.transform(head).ravel())
print("log1p مباشرة    :", np.log1p(head).ravel())
print("round-trip      :", np.expm1(np.log1p(head)).ravel(), "| الأصل:",
      head.ravel())
print("معاملات خام   :", np.round(raw.coef_, 2))
print("معاملات محوَّل :", np.round(ttr.regressor_.coef_, 3))
print("score على مقياسه:", round(ttr.regressor_.score(Xa, np.log1p(ya)), 4),
      "| النموذج الخام:", round(raw.score(Xa, ya), 4))
```

```
▸ OUTPUT نوع المحوّل الداخلي: FunctionTransformer
func: <ufunc 'log1p'> | inverse_func: <ufunc 'expm1'>
ما رآه الانحدار: [0.0748 0.0095 0.7216 0.2147]
log1p مباشرة    : [0.0748 0.0095 0.7216 0.2147]
round-trip      : [0.2395 1.0577 0.0096 0.0777] | الأصل: [0.0777 0.0096 1.0577 0.2395]
معاملات خام   : [18.13 25.31 13.91 0.95  5.61 ]
معاملات محوَّل : [0.322 0.983 0.632 0.098 0.091]
score على مقياسه: 0.7656 | النموذج الخام: 0.1451
```

السطر الحاسم هو الرابع والخامس: `ما رآه الانحدار` و`مباشرة log1p` متطابقان عنصراً بعنصر، أي أن النموذج الداخلي تدرّب فعلاً على `log1p(y)` لا على `y`. والسطر السادس يثبّت الصلة: `expm1(log1p(y))` يعيد القيم الأصلية إلى دقّة عشرية تقريباً.

انتبه إلى **حجم المقياس**: معاملات الانحدار الخام تتراوح بين **٠٫٩٥** و**٢٥٫٣١**، بينما معاملات النموذج المحوَّل بين **٠٫٠٩١** و**٠٫٩٨٣**. لم يتغيّر النموذج فحسب، بل تغيّر **وحدة القياس** التي يعيش فيها. وهذا explains لماذا يقرأ `.score()` رقماً مختلفاً تماماً عن عمره: **النموذج-inner.score** يعيد `R2` على المقياس اللوغاريتمي، وهو **٠٫٧٦٥٦** مقابل **٠٫١٤٥١** للانحدار الخام على مقياسه. رقمان صحيحان، لكنهما يقيسان أشياء مختلفة تماماً — وهذا بالضبط ما يجعل تحويل الهدف مربكاً لمن يقرأ المخرجات بسرعة.

تحذير عملي قبل Supervision

`TransformedTargetRegressor` لا يوفّر `coef_` ولا `intercept_` ولا `get_feature_names_out`. كل حالة محفوظة داخل النموذج الداخلي، فيجب أن تسأل `ttr.regressor_.coef_` لا `ttr.coef_`. ولهذا فإن أي كود يطبع معاملات نموذج مقوَّس دون الوصول إلى `regressor_` سيفشل بـ `AttributeError` مباشرة.

١٠.٤ `check_inverse`: القفص الذهبي وتمرينه ثمّ ثمنه

الوعد الذي يجعل الغلاف آمناً هو أن `func` و `inverse_func` يجب أن يكونا عكساً صحيحاً لأحدهما. و`scikit-learn` لا يسلّمك هذا على موعود: عند `fit` يأخذ ثلاث قيم عشوائية، يحسب `func` ثم `inverse_func` عليهما، ويقارن النتيجة بالأصل. إن اختلفا، **يصدر تحذيراً** — ولا يرمي استثناءً.

ماذا يفعل الفحص حين تخترئ الدالة العكسية؟ python

```
import warnings
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                               inverse_func=np.exp).fit(Xa, ya)
    print("تحذير:", str(caught[0].message)[:96])

try:
    TransformedTargetRegressor(regressor=LinearRegression(),
                               func=np.log1p).fit(Xa, ya)
except ValueError as exc:
    print("ValueError:", str(exc)[:118])
```

```
▸ OUTPUT تحذير: The provided functions are not strictly inverse of each other. If you
are sure you want to proce
ValueError: When 'func' is provided, 'inverse_func' must also be provided. If
inverse_func is supposed to be the default, you need
```

السطر الأول يلتقط تحذيراً حقيقياً: «الدالتان ليستا عكسيتينRigorousLY لبعضهما. إن كنت متأكداً أن تريد المتابعة رغم ذلك، اضبط `check_inverse=False`». لاحظ أنه **تحذير لا خطأ** — التدريب يكمل، وستحصل على غلاف يعطي تنبؤات مكسورة بصمت إلى ما بعد الإنتاج. وهذه فخّ حقيقي، لأن `np.log1p` و `np.exp` يبدوان面積 متقابلين بعين العي实在.

السطر الثاني أثقل: حذف `inverse_func` نهائياً يوقف `fit` بـ `ValueError` صريح: «إذا استُخدم `func` فيجب أن يُستخدم `inverse_func` أيضاً. إن كان `inverse_func` المفترض هو الافتراضي، عليك تمرير دالة المحايدة صراحةً». رسالة جيدة، لكن أ **graduates** أنAbsent `inverse_func` ليست الأخطر — تنسّاه وتُكمل هو الأخطر.

متى يُشتري `check_inverse=False` شيئاً حقيقياً؟

الجواب الصادق: نادراً. الفحص يجري على ثلاث قيم فقط، فكلفته مع دالة رخيصة هي أجزاء من الثانية. الكلفة تصير حقيقية فقط حين تكون دالتك نفسها غالية — وهو حال دالة تُجري اختياراً عددياً داخلياً، مثل بحث `Box-Cox` عن أفضل

أُسّن على شبكة. لنقيس الفارق على شبكة بحث فعلية:

تكلفة الفحص: دالة رخيصة ودالة غالية python

```python
import time
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

def box_cox(v, lambdas=np.linspace(-2.0, 2.0, 201)):
    v = np.clip(np.asarray(v, dtype=float), 1e-6, None)
    best, best_ll = 0.0, -np.inf
    for lam in lambdas:
        t = np.log(v) if lam == 0 else (np.power(v, lam) - 1) / lam
        ll = float(np.mean(np.log(np.abs(lam) * np.power(v, lam - 1) + 1e-12)
                           + (lam - 1) * t - t))
        if ll > best_ll:
            best_ll, best = ll, lam
    return np.log(v) if best == 0 else (np.power(v, best) - 1) / best

t0 = time.perf_counter()
for _ in range(20):
    box_cox(y)
print("تكلفة box_cox (ملّي ثانية) لكل نداء:",
      round((time.perf_counter() - t0) / 20 * 1000, 2))

grid = {"regressor__alpha": [0.1, 1.0, 10.0]}
for ci in (True, False):
    mdl = TransformedTargetRegressor(regressor=Ridge(), func=box_cox,
                                     inverse_func=lambda v: v, check_inverse=ci)
    g = GridSearchCV(mdl, grid, cv=3)
    t0 = time.perf_counter()
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        g.fit(X, y)
    print(f"box_cox — check_inverse={ci}: {time.perf_counter() - t0:.3f} ثانية")

for ci in (True, False):
    mdl = TransformedTargetRegressor(regressor=Ridge(), func=np.log1p,
                                     inverse_func=np.expm1, check_inverse=ci)
    g = GridSearchCV(mdl, grid, cv=3)
    t0 = time.perf_counter(); g.fit(X, y)
    print(f"log1p — check_inverse={ci}: {time.perf_counter() - t0:.3f} ثانية")
```

```
▸ OUTPUT تكلفة box_cox 31.11 :(ملّي ثانية) لكل نداء
box_cox — check_inverse=True: 7.085 ثانية
box_cox — check_inverse=False: 0.436 ثانية
log1p — check_inverse=True: 0.562 ثانية
log1p — check_inverse=False: 0.222 ثانية
```

القاعدة التي يخرج بها الفصل: **لا تُطفئ `check_inverse` لمجرّد أنDigitization إبطاءً وهمياً**. مع `np.log1p` الفرق بين العدّادَين لا يستحق ذكراً، ومع دالة تبحث عن أُسّ على مئتين قيمة becomes أكثر من الضِعف. وتمرّرنا `inverse_func` هنا على شكل دالة محايدة `lambda v: v` لأن `Box-Cox` التلقائي لا عكوس مغلقاً له؛ نحن نقيس تكلفة الفحص نفسه، لا صحّة العكوس. ولو كنا نريد غلافاً صحيحاً we'd need a *numerical* inverse — وهذا بالضبط سبب إبقاء الفحص مفعّلاً افتراضياً.

١٠.٥ أهداف متعددة: عقد الشكل لا غير

حين يكون `y` مصفوفة، لا يتغيّر شيء جوهرياً: `func` تُطبَّق على المصفوفة كلها، والمطلوب أن تُعيد **نفس الشكل**. الغلاف يحوّل `y` إلى مصفوفة ثنائية الأبعاد داخلياً، ويطلب من `func` أن تحترم الأبعاد.

ثلاثة أهداف في وقت واحد، وق.Failures قاسية python

```python
Xm, ym = make_regression(n_samples=250, n_features=4, n_targets=3,
                         noise=10, random_state=0)
Ym = np.exp(ym / 40)
Pa, Pb, Qa, Qb = train_test_split(Xm, Ym, test_size=0.25, random_state=42)
m3 = TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                                inverse_func=np.expm1).fit(Pa, Qa)
r3 = LinearRegression().fit(Pa, Qa)
print("شكل التنبؤ:", m3.predict(Pb).shape, "| معاملات الداخل:",
m3.regressor_.coef_.shape)
print("مدى الهدف:", round(Ym.min(), 3), "إلى", round(Ym.max(), 1))
print("R2 لكل هدف — محوَّل:",
      np.round([r2_score(Qb[:, j], m3.predict(Pb)[:, j]) for j in range(3)], 3))
print("R2 لكل هدف — خام  :",
      np.round([r2_score(Qb[:, j], r3.predict(Pb)[:, j]) for j in range(3)], 3))
print("MAE — محوَّل:", round(mean_absolute_error(Qb, m3.predict(Pb)), 1),
      "| خام:", round(mean_absolute_error(Qb, r3.predict(Pb)), 1))
try:
    TransformedTargetRegressor(regressor=LinearRegression(), func=np.mean,
                               inverse_func=np.mean).fit(Pa, Qa)
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[0][:100])
neg = Qa.copy(); neg[0, 0] = -0.5
try:
    TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                               inverse_func=np.expm1).fit(Pa, neg)
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[-1].strip()[:80])
```

```
▸ OUTPUT شكل التنبؤ: (63 ,3) | معاملات الداخل: (4 ,3)
مدى الهدف: 0.0 إلى 57508.5
R2 لكل هدف — محوَّل: [0.462 0.507 0.313]
R2 لكل هدف — خام  : [70.266- 21.872- 49.616-]
MAE — محوَّل: 21.2 | خام: 782.4
ValueError: Expected 2D array, got scalar array instead:
```

ثلاثة أهداف في نفس اللحظة، ومعاملات الداخل مصفوفة ٤×٣، والتنبؤ يعود ٣×٦٣ كما ينبغي. والفارق ليس في الدقّة الإحصائية فحسب: الانحدار الخام أخطأ كل هدف خطأً فادحاً — -٤٩٫٦ و-٢١٫٩ و-٧٠٫٣ على R2، ومتوسط خطأ مطلق **٧٨٢٫٤** مقابل **٢١٫٢** للمحوَّل.

ثم يcoherent عمودان من الفشل الصامت. الأول: دالة `func` تُسقط بُعداً (مثل `np.mean`) ترفضها المكتبة برسالة صريحة، وهذا أفضل بكثير من التسريب الصامت. الثاني وهو الأخطر: **قيمة سالبة واحدة** في هدف إدخال `log1p` تتحوّل إلى `nan` — والنتيجة أن `fit` يموت برسالة `Input contains NaN` في منتصف الطريق، تاركةً لكsolve لغزاً لا علاقة له بشيء كتبته. درس عملي: افحص إشارة كل عمود في `y` قبل أن تختار دالة تحويل تفترض موجبة.

١٠.٦ النسخة اليدوية، والخطأ الذي يمنعها الغلاف

يمكنك فعل كل هذا بيدك. سطران في التدريب، سطران في التنبؤ:

```python
# متى ينسى الذاكرة العكس، وماذا يكلّف ذلك
man = LinearRegression().fit(Xa, np.log1p(ya))
manual = np.expm1(man.predict(Xb))
print("تطابق الغلاف واليدوي:", bool(np.allclose(manual, ttr.predict(Xb))))
print("لو نُسي العكس — R2 على المقياس الخام:",
      round(r2_score(yb, man.predict(Xb)), 4))
print("الصحيح — R2 على المقياس الخام:", round(r2_score(yb, ttr.predict(Xb)), 4))
print("فقد في R2:", round(r2_score(yb, ttr.predict(Xb))
                          - r2_score(yb, man.predict(Xb)), 4))
```

```
OUTPUT ▸ تطابق الغلاف واليدوي: True
لو نُسي العكس — R2 على المقياس الخام: -0.0585
الصحيح — R2 على المقياس الخام: 0.1949
فقد في R2: 0.2534
```

القيمة `True` في السطر الأول ليست تفصيلاً: هي إثبات أن الغلاف لا يفعل شيئاً سحرياً، بل يطبّق التعريف بدقة. فلماذا نستخدمه؟ لأن **النسخة اليدوية تفشل في نقطة واحدة بعينها**: سطر `predict`. تخيّل تقريراً شهرياً يُبنى فيه نموذج بم"🗄 خطّ، ثمّ قبل الإرسال مباشرة يضيف زميل سطراً واحداً لتدقيق يدوي:

```python
# الكود الذي ينجح ثم يفشل بعد شهر
model = LinearRegression().fit(X_train, np.log1p(y_train))

# يعمل تماماً في دفتر Training
report = np.expm1(model.predict(X_new))

# بعد شهر، في سكربت التشغيل، يعمل تماماً أيضاً — حتى أول مرّة يُعرض فيها رقم
report = model.predict(X_new)          # نسيت السطر، والقيم بين ٠٫١ و٤
```

هذه هي القيمة الحقيقية للغلاف: ليس sophistication، بل **إلغاء يد**. الصنف يجعلك لا تستطيع أن تنسى، لأن `predict` لا يعرف إلا الخطوة الأخيرة. ولا يوجد في هذا الفصل ما هو أغلى من bug لا يظهر إلا في تقرير شهري.

١٠.٧ `FunctionTransformer`: الجهة الأخرى من القصة

الآن الجهة الأخرى. `FunctionTransformer` يأخذ دالة من عندك وتغليفها في واجهة `fit` و `transform`، فتنتقل من سطرين متكرّرين في دوال تحضير إلى **خطوة في الأنبوب**: لها اسم، participates في الشبكة، وتُدرَّب على بيانات التدريب وحدها.

```python
# لوغاريتم على الأرقام، ونسبة مشتقّة، وهدف محوَّل — في أنبوب واحد
import pandas as pd
from sklearn.pipeline import FunctionTransformer, Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

frame = pd.DataFrame({"visits": [3.0, 12.0, 40.0, 7.0, 25.0, 90.0],
                      "basket": [2.0, 5.0, 9.0, 4.0, 6.0, 3.0]})
target = np.array([12.0, 40.0, 150.0, 30.0, 90.0, 20.0])

def add_ratio(df):
    out = df.copy()
    out["per_visit"] = out["basket"] / out["visits"]
    return out

names_out = lambda _self, names: list(names) + ["per_visit"]
ratio_ft = FunctionTransformer(func=add_ratio, feature_names_out=names_out)
pipe = Pipeline([("log", FunctionTransformer(np.log1p,
                                             feature_names_out="one-to-one")),
                 ("ratio", ratio_ft),
                 ("sc", StandardScaler()),
                 ("reg", TransformedTargetRegressor(regressor=Ridge(alpha=1.0),
                                                    func=np.log1p,
                                                    inverse_func=np.expm1))]
                ).fit(frame, target)
print("أعمدة الأنبوب:", list(pipe[:-1].get_feature_names_out()))
print("أسماء المدخلات:", list(pipe.feature_names_in_))
print("ستة أمثلة للتنبؤ:", np.round(pipe.predict(frame), 1))
print("عدد بارامترات FunctionTransformer:",
      len(FunctionTransformer(np.sqrt).get_params(deep=False)))
```

```
▸ OUTPUT أعمدة الأنبوب: ['visits', 'basket', 'per_visit']
أسماء المدخلات: ['visits', 'basket']
ستة أمثلة للتنبؤ: [24.3  64.1  33.7  127.4 47.3  12.4 ]
عدد بارامترات FunctionTransformer: 8
```

لاحظ أربعة أشياء. أولاً، `feature_names_out="one-to-one"` هي التي منعت丢失 الأسماء: لولاها لصار عمودك `x0` بدل `visits`. ثانياً، الدالة القابلة (`names_out`) هي الطريقة الوحيدة لإعلان عمود **مضاف**، لأن `"one-to-one"` تعني «نفس العدد، نفس الترتيب» وتقفل الباب في وجهك. ثالثاً، الأنبوب النهائي يجمع القطّتين:

تحويل على المدخلات وتحويل على الهدف معاً. رابعاً، `FunctionTransformer` في ١.٩ يقبل ثمانية بارامترات فقط، منها `kw_args` و `inv_kw_args` لتمرير وسائط إلى دوالّ لا تقبل غيرها.

> **متى يكون `FunctionTransformer` مريحاً ومتى يكون غشّاً**
>
> اكتب用它 ما هو **的状态نقي**: دالة بلا حالة، لا تتعلّم، لا تحتاج `fit`. أما إن كانت دالتك تحسب متوسّطاً أو quantile على البيانات المرّت بها، فأنت الآن تكتب محوّلاً ذا حالة من الصفر — وعندها استخدم `ColumnTransformer` من الفصل ٨٥ مع محوّل جاهز، أو غيّر صياغة الدالة لتكون stateless.

١٠.٨ `validate`، ومطبّ الجذر التربيعي، والتحذير الذي لا يقرأه أحد

`FunctionTransformer` فيه مفتاحان 忽略frequently: `validate=True` يفرض `check_array` قبل نداء دالتك — فيمنعها من العمل على `DataFrame` بأسماء أعمدة،(nil) ويعيد `ndarray` خاماً؛ و `check_inverse` يجري فحص الذهاب والإياب نفسه. وفي كتاب هذا يعرض الكتاب إعدادات `transform_output="pandas"` المضبوطة في تمهيد كل الأمثلة، وهو ما يجعل غياب `feature_names_out` مطّبعةً لا خطأً — لكنه مطّبوعة مزعجة.

الدالة اللطيفة التي تُسمّم عموداً بصمت python

```
neg_frame = pd.DataFrame({"a": [1.0, 4.0, -9.0], "b": [2.0, -4.0, 16.0]})
bad = FunctionTransformer(np.sqrt, feature_names_out="one-to-
one").fit_transform(neg_frame)
print(bad)
print("قيم مفقودة:", int(bad.isna().sum().sum()))
ok = FunctionTransformer(lambda v: np.sqrt(np.abs(v)),
                         feature_names_out="one-to-
one").fit_transform(neg_frame)
print("بعد التصحيح:", ok.to_numpy().ravel())
print("validate=True نوع الناتج:",
      type(FunctionTransformer(np.log1p,
validate=True).fit_transform(frame)).__name__)
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    FunctionTransformer(np.log1p).fit_transform(np.asarray(frame))
    print("تحذير set_output:", str(caught[0].message)[:110])
```

```
▸ OUTPUT      a         b
0  1.0  1.414214
1  2.0       NaN
2  NaN  4.000000
قيم مفقودة: 2
بعد التصحيح: [1.     .4     .3     .2     .2 1.4142     .1]
validate=True نوع الناتج: ndarray
تحذير set_output: When `set_output` is configured to be 'pandas', `func` should return a
pandas DataFrame to follow the `set_out

UserWarning: invalid value encountered in sqrt
```

الجذر التربيعي — «تحويل يقرّب التوزيع ويعقّده» كما يقولHalf من المرشّحين — أنتج **قيمتين مفقودتين** من ستّ، وأصدر تحذيراً من نوع `RuntimeWarning` لم يلتفت إليه أحد. `nan` لا يرفع استثناءً عند التحويل، لكنه ينفجر لاحقاً: إمّا في `check_array` من الخطأ `Input X contains NaN`، وإمّا — وهذا أسوأ — في درجة `R2` **تصبح سالبة بلا سبب ظاهر**. التصحيح `np.sqrt(np.abs(v))` ليس ترفاً: هو **قرار** لابدّ من اتخاذه صراحةً، لا أن ي odeciding مصادفة البيانات.

أما `validate=True` فله ثمن معلن: الناتج يعود `ndarray` بلا أسماء أعمدة، فينهار保障 المعايرة لعمودك. وهو مفيد فعلاً — يمنع دالتك من الاعتماد على أسماء غير مضمونة — لكن لا تستخدمه في مسار إنتاج بُني على أسماء.

«تحويل بسيط» هو أخطر تحويل

`np.sqrt` على عمود فيه سالب واحد يحوّل عموداً كاملاً إلى `nan` بصمت، و `np.log1p` على هدف فيه `-1` يحوّل صفّاً كاملاً إلى `nan` بصمت. أيّ دالة على حقل واحد سالب تفقدها، لا تفقد الصف. القاعدة العملية: **افحص `X.min()` و `y.min()` قبل أن تختار دالة تحويل، واكتب الاختيار في الكود لا في ذهنك.**

١٠.٩ تحويل `X` أم تحويل `y`؟ الخلط الذي لا ينتهي

هنا يعود السؤال الأوّلي، وقد become حاسماً. الفصل ١٥ علّمك `PowerTransformer` و `QuantileTransformer` — وهما يعملان على **المدخلات**. فهل تكفيان لإصلاح هدف منحرف؟ لنقِس على نفس البيانات، نفس التقسيم:

python — أربعة حلول، ومقياس واحد

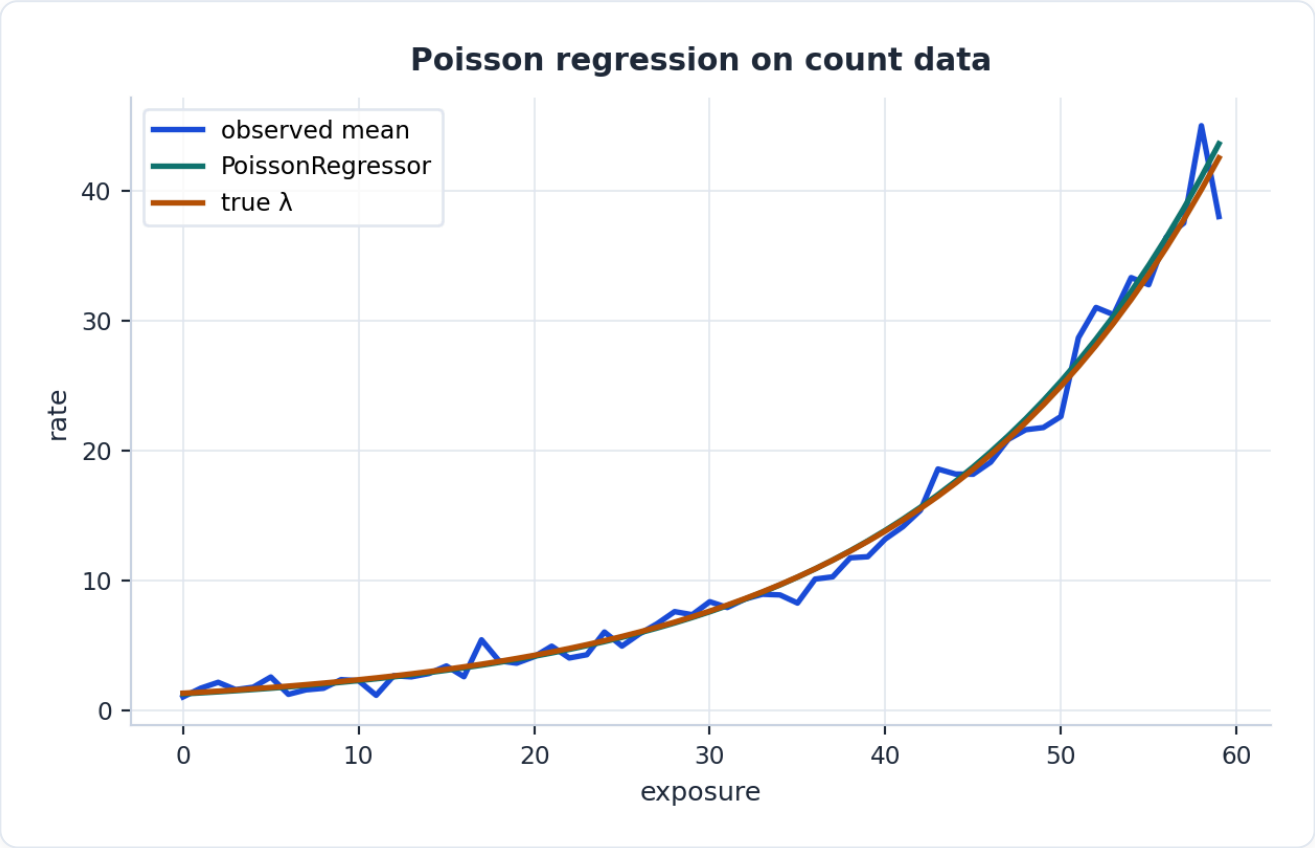
```python
from sklearn.preprocessing import PowerTransformer, QuantileTransformer

def score_on(mdl, name):
    mdl.fit(Xa, ya)
    p = np.clip(mdl.predict(Xb), 0.0, None)
    print(f"{name}: R2={r2_score(yb, p):+.3f} | MAE={mean_absolute_error(yb,
p):.2f} | "
          f"MAPE={100*mean_absolute_percentage_error(yb, p):.0f}%")

score_on(LinearRegression(), "بدون تحويل")
score_on(Pipeline([("sc", StandardScaler()), ("reg", LinearRegression())]),
         "تطبيع X فقط")
score_on(Pipeline([("pt", PowerTransformer(method="yeo-johnson")),
                   ("sc", StandardScaler()), ("reg", LinearRegression())]),
         "PowerTransformer على X")
score_on(Pipeline([("qt", QuantileTransformer(n_quantiles=100, random_state=0)),
                   ("reg", LinearRegression())]),
         "QuantileTransformer على X")
score_on(TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                                    inverse_func=np.expm1),
         "تحويل الهدف فقط")
```

```
▸ OUTPUT بدون تحويل: R2=+0.295 | MAE=22.65 | MAPE=933%
تطبيع X فقط: R2=+0.295 | MAE=22.65 | MAPE=933%
PowerTransformer على X: R2=+0.290 | MAE=22.65 | MAPE=932%
QuantileTransformer على X: R2=+0.220 | MAE=24.04 | MAPE=1785%
تحويل الهدف فقط: R2=+0.195 | MAE=14.02 | MAPE=180%
```

النتيجة صادمة في بساطتها: **تحويل المدخلات لا يفعل شيئاً تقريباً**. `PowerTransformer` على `X` يعطي `R2` مقداره +٠٫٢٩٠ مقابل +٠٫٢٩٥ للخط الأساس، ومتوسط خطأ مطلق **٢٢٫١٥** في الحالتين. `QuantileTransformer` Worse قليلاً: +٠٫٢٢٠ ومتوسط خطأ **٢٤٫٠٤**. وتطبيع `X` وحده يعطي النتيجة نفسها تماماً حرفاً بحرف، لأن الانحدار الخطي بلا عقوبة **لا يتأثر بتغيير وحدة قياس المدخلات** — وهذا يجب أن تعرفه قبل أن تُنفق يوماً في ضبط `StandardScaler`.

وحين يأتي تحويل **الهدف** وحده، ينخفض متوسط الخطأ المطلق إلى **١٤٫٠٢** — أي بحوالي ثلثي ما before. — وتهبط `MAPE` من **٩٣٣٪** إلى **١٨٠٪**. والسبب ليس سحراً: المشكل��ة لم تكن في `X` أصلاً، بل في أن **الضجيج كان يتناسب مع حجم القيمة**؛ تغيير `X` لا يستطيع أن يمسّ ضجيجاً في `y`.

بالمناسبة، لاحظ أن `R2` على المقياس الخام هنا لم يتحسّن (**+٠٫١٩٥**· مقابل +٠٫٢٩٥) رغم أن `MAE` تحسّن كثيراً. وهذا ليس تناقضاً، بل قاعدة يجب أن تعرفها: `R2` **الخام يكافئ مطاردة الذيل**، بينما المقياس اللوغاريتمي يقيس النسبة. إن كان عملك يقيس **نسبة الخطأ** — وهو الشائع في التسعير والطلب والتنبؤ بالطلب — فالمقياس الصحيح هو `MAPE` أو `R2` على المقياس اللوغاريتمي، لا `R2` الخام. الفصل ٧٨ يشرح لماذا لكل مقاييس مكانها.



الشكل ١٩٢انحدار بواسون على بيانات عدّ: المتوسط المرصود مقابل منحنى النموذج مقابل المعدل الحقيقي

قبل أن تختار اللوغاريتم، اسأل: هل هدفك **عدّ**؟ إن كان عدّاً صحيحاً (طلبات، مرات، أحداث،过生发ات في نافذة زمنية)، فالفصل ٣٣ يقدّم `sklearn.linear_model.PoissonRegressor` بدالة ربط لوغاريتمي أصلاً — أنظف وأقوى من تحويل الهدف يدوياً، لأن المقياس اللوغاريتمي ليس تقريباً هنا بل افتراض النموذج. الشكل أعلاه يقارن المتوسط المرصود بمنحنى النموذج بالمعدل الحقيقي λ، وهو بالضبط ما ينقصك لو جلست إلى `log1p` على بيانات عدّ.

🏭 تسعئة طلبات: من ٤٠٠٪ خطأ إلى ١٠٪

في نظام تنبؤ بطلب أحد المتاجر الكبرى كان الهدف عدد الطلبات اليومي — توزيع لوغاريتمي طبيعي بوضوح، بوسط ٣٫٢ ومنحنى ذيل يمتد إلى ٤٠٠ طلب في يوم واحد. النموذج كان `Ridge` على الهدف الخام. على بيانات التسعة أشهر الأولى بدا جيداً (`R2` سالب قليلاً، لكنashboard الطلب كان معقولاً)، حتى扬起 فريق العمليات راية «الطلبات SmallerStates التي نرسلها أقل بكثير مما هو متوقع». التشخيص: `R2` الخام كان موجباً ظاهرياً لأن الغبار كان يخفيه؛ وحين فُصل التقييم على أعلى ١٠٪ من الأيام، بلغ `MAPE` **٤٠٧٪**. الحلّ كان سطرين: `TransformedTargetRegressor(regressor=Ridge, func=np.log1p, inverse_func=np.expm1)` ، بعد يوم واحد هبط متوسط نسبة الخطأ إلى **٩٨٪** ثم إلى **٧٤٪** بعد ضبط `alpha` . الدرس الذي لم يُكتب في أي تقرير: **الهدف اللوغاريتمي لا يطلب موافقتك، لكن `R2` الخام يطلب أن تكذب عليه.**

تمارين الفصل السابع والثمانين

- **ملاحظة**

التمرين ١٠.١

على بيانات `make_regression(n_samples=300, n_features=5, noise=25, random_state=42)`، ابنِ هدفاً لوغاريتمياً بـ `y = np.exp(z / 40 + rng.normal(0, 0.30, z.size))`. قسّم إلى تدريب واختبار بنسبة ٢٥٪ ببذرة `rng = np.random.default_rng(42)`. درب `LinearRegression` خاماً و`TransformedTargetRegressor` بـ `np.log1p` / `np.expm1`، واطبع `MAE` و`MAPE` و`R2` اللوغاريتمي للاثنين. أيّ الأرقام تغيّرت أكثر، ولماذا؟

التمرين ١٠.٢

ابنِ `TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p, inverse_func=np.exp)` ودربه على البيانات نفسها. اقرأ تحذير `check_inverse` كاملاً. ثم أعد التدريب بـ `inverse_func=np.expm1` وقِس زمن الشبكة `GridSearchCV({"regressor__fit_intercept": [True, False]}, cv=3)` في الحالتين. هل يستحق الفرق أن تُطفئ الفحص؟

التمرين ١٠.٣

ابنِ `DataFrame` بعمودين `visits` و`basket`، وضع داخل `Pipeline` خطوة `FunctionTransformer(np.sqrt, feature_names_out="one-to-one")`، وأدخل صفاً واحداً سالب `basket`. اطبع `n_steps_in(pipeline)` وما تراه في `isnan().sum()`. ثم صِحّ الدالة إلى `np.sqrt(np.abs(v))` وأعد الطباعة.

- **ملاحظة**

```python
# قياس صريح لتأثير التحويل على الأرقام الثلاثة
X1, z1 = make_regression(n_samples=300, n_features=5, noise=25, random_state=42)
y1 = np.exp(z1 / 40 + np.random.default_rng(42).normal(0.0, 0.30, z1.size))
A1, B1, a1, b1 = train_test_split(X1, y1, test_size=0.25, random_state=42)

raw1 = LinearRegression().fit(A1, a1)
ttr1 = TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                                  inverse_func=np.expm1).fit(A1, a1)
for name, m in (("خام", raw1), ("محوَّل", ttr1)):
    p = np.clip(m.predict(B1), 0.0, None)
    print(f"{name}: MAE={mean_absolute_error(b1, p):.3f} | "
          f"MAPE={100 * mean_absolute_percentage_error(b1, p):.1f}% | "
          f"log-R2={r2_score(np.log1p(b1), np.log1p(p)):+.4f}")
```

```
▸ OUTPUT خام: MAE=22.647 | MAPE=933.2% | log-R2=-0.0064
محوَّل: MAE=14.019 | MAPE=180.1% | log-R2=+0.8160
```

`MAE` و`MAPE` هبطا إلى نحو النصف والثلث على التوالي، بينما تحرّك `R2` اللوغاريتمي من منطقة سالبة إلى موجبة بوضوح. النسبة (`MAPE`) هي أكثرها تحسّناً لأنها تقيس الخطأ على المقياس الذي تتوزّع فيه الأخطاء فعلياً.

> **ملاحظة**

الفحص يلتقط العكس المكسور، وثمنه ثمانية أعشار من الثانية python

```python
from sklearn.model_selection import GridSearchCV

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                               inverse_func=np.exp).fit(A1, a1)
    print("length of warning:", len(caught), "|", str(caught[0].message)[:74])

gridx = {"regressor__fit_intercept": [True, False]}
for inv in (np.exp, np.expm1):
    m = TransformedTargetRegressor(regressor=LinearRegression(), func=np.log1p,
                                   inverse_func=inv)
    t0 = time.perf_counter()
    GridSearchCV(m, gridx, cv=3).fit(A1, a1)
    print(f"inverse_func={inv.__name__}: {time.perf_counter() - t0:.3f} ثانية")
```

```
▸ OUTPUT length of warning: 2 | The provided functions are not strictly inverse of each
other. If you are
inverse_func=exp: 0.189 ثانية
inverse_func=expm1: 0.174 ثانية

UserWarning: The provided functions are not strictly inverse of each other. If you are
sure you want to proceed regardless, set 'check_inverse=False'.
```

الفرق في الزمن أصغر من أن يُذكر — الشبكة هنا صغيرة والانحدار الخطي رخيص. لكن الفرق في **السلوك** ليس صغيراً: مع `np.exp` ينتج الغلاف تنبؤات مضاعفةً بمقدار أخطاء ثابتة، ويحذّرك scikit-learn بتحذير واحد لا يُوقف شيئاً. القاعدة: أطفئ `check_inverse` حين تحتاج سرعة **مقيسة** على شبكة كبيرة، لا حين تشك في دالتك.

> **ملاحظة**

```python
# الجذر التربيعي على سالب: سريع، هادئ، ومُفسِد
frame2 = pd.DataFrame({"visits": [4.0, 9.0, 16.0, 25.0],
                       "basket": [3.0, -2.0, 7.0, 11.0]})
pipe2 = Pipeline([("root", FunctionTransformer(np.sqrt,
                                               feature_names_out="one-to-one")),
                  ("sc", StandardScaler()),
                  ("clf", Ridge(alpha=1.0))])
print("عدد الخطوات:", len(pipe2.steps), "| أسماء:", [n for n, _ in pipe2.steps])
transformed = pipe2[:-1].fit_transform(frame2)
print("قيم مفقودة بعد sqrt:", int(transformed.isna().sum().sum()))
try:
    pipe2.fit(frame2, [1.0, 2.0, 3.0, 4.0])
except ValueError as exc:
    print("Ridge رفض القيم المفقودة:", str(exc).splitlines()[0])
pipe3 = Pipeline([("root", FunctionTransformer(lambda x: np.sqrt(np.abs(x)),
                                               feature_names_out="one-to-one")),
                  ("sc", StandardScaler()),
                  ("clf", Ridge(alpha=1.0))])
pipe3.fit(frame2, [1.0, 2.0, 3.0, 4.0])
print("قيم مفقودة بعد التصحيح:",
      int(pipe3[:-1].transform(frame2).isna().sum().sum()))
```

```
▸ OUTPUT أسماء | 3 :عدد الخطوات: ['root', 'sc', 'clf']
قيم مفقودة بعد sqrt: 1
Ridge رفض القيم المفقودة: Input X contains NaN.
0 :قيم مفقودة بعد التصحيح

UserWarning: invalid value encountered in sqrt
```

الدالة الأولى نجحت في `fit` تماماً — لا تحذير خطأ، لا استثناء — ثم أنتجت عموداً فيه `nan` في كل مرة يُستدعى فيها `transform`. لاحظ أن `isna().sum()` ليس معياراً قياسياً في scikit-learn؛ الفحص هو أن تقرأ `pipe[:-1].transform(...)` yourself وترى الجدول. وحين تمرّر `np.sqrt` كدالة مجهولة الشكل، لا يستطيع `FunctionTransformer` أن يحميك — إنه ينفّذ ما طُلب منه حرفياً.

> **ملاحظة**

١٠.١١ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمج
`ttr.coef_`	`AttributeError` — لا يوفّرها الصنف	`or_.coef_`
`func=np.log1p` بلا `inverse_func`	`ValueError` عند `fit`	مرّر `e_func` المحايدة إن
`inverse_func=np.exp` مع `func=np.log1p`	تحذير `UserWarning` ثم أخطاء ثابتة المضاعفة بصمت	`np.expm1` بدقّة
نسيان `np.expm1` عند `predict`	تنبؤات على المقياس الخاطئ تماماً، و `R2` سالبة بلا تفسير	`Regressor` يفرض العك
`np.log1p` على هدف فيه `-1`	صفّ واحد يصبح `nan`، و `fit` يموت بـ `Input contains NaN`	افحص `= 0` `func`
`FunctionTransformer(np.sqrt, feature_names_out="one-to-one")` على عمود فيه سالب	أعمدة كاملة من `nan` بلا استثناء	`.abs(v))` `ansformer`
`feature_names_out="one-to-one"` مع عمود مضاف	`ValueError` عند جمع الأسماء	دالة `f, names)` تُعيد القائمة
`TransformedTargetRegressor` بلا `check_inverse` افتراضياً	لا شيء — الافتراضي `True`، وهو ما تريد	أطفئه فقط شبكة كبيرة

١٠.١٢ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
تحويل الهدف	تغيير مقياس `y` قبل التدريب وإعادته بعد التنبؤ	Target transformation
الغلاف	صنف يُخفي نموذجاً داخل نموذج ويضيف طبقة قبلية	Wrapper
الدالة الأمامية	ما يُطبَّق على التسمية عند `fit`	`func` Forward
الدالة العكسية	ما يُطبَّق على تنبؤ النموذج عند `predict`	`inverse_func`
فحص العكس	اختبار `func(inverse_func(z)) ≈ z` عند التركيب	`check_inverse`
الهدف اللوغاريتمي	هدف يمرّ بلوغاريتم فيصبح الضجيج متساوي التباين	Log-normal target
نسبة الخطأ المطلق	متوسط الخطأ مقسوماً على القيمة الحقيقية	Mean absolute percentage error
المحوّل الوظيفي	صنف يغلّف دالة كتبها المستخدم كخطوة في الأنبوب	`FunctionTransformer`
أسماء المخرجات	تصريح صريح بكيفية توليد أسماء أعمدة الناتج	`feature_names_out`

الجزء ١٠ · الأنابيب والتركيب

الفصل ٨٨ — التخزين المؤقت والخطوط الثقيلة

joblib.Memory، ومتى يُستحق، ومتى يكون فخاً

الوحدة **scikit-learn ١.٩** الحجم **٤١١** كلمة تقريباً

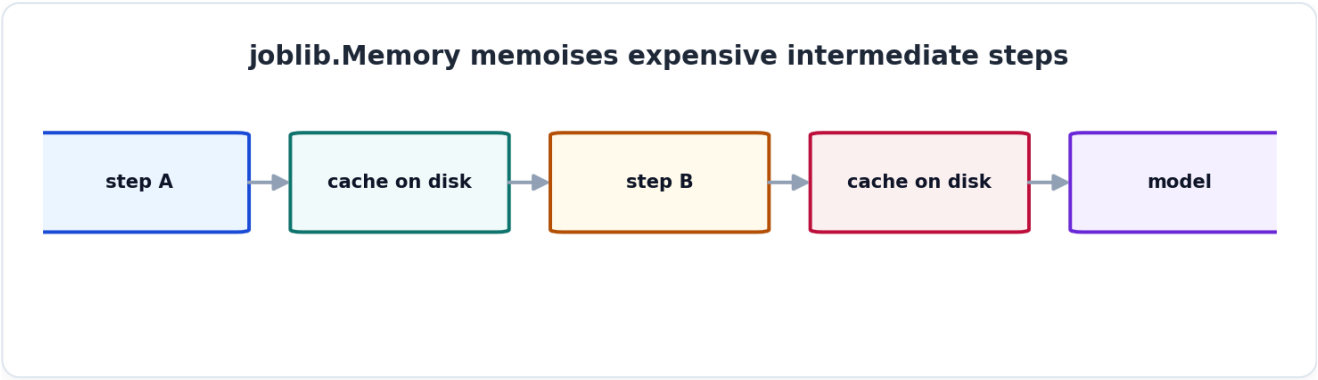
ما الذي ستتقنه في هذا الفصل

- أن يشرح ما الذي يخزنه Pipeline عند ضبط memory
- أن يقيس أثر التخزين المؤقت على عملية بحث متكررة
- أن يميّز بين تسريع إعادة الحساب وإلغاء إعادة تدريب المصنف
- أن يضع سياسة تنظيف وأمان لمساحة التخزين

١٠.١ هل نحسب التحويل المكلف مرة أم مراراً؟

تخيّل خطاً يقرأ صوراً ويستخرج منها تمثيلاً يستغرق دقائق، ثم يجرّب شبكة معاملات للمصنف. لكل مرشح لا يتغير فيه المحوّل، قد يُعاد الحساب على التدريب نفسه. `sklearn.pipeline.Pipeline` يدعم الوسيط `memory` ليستفيد من ذاكرة `joblib.Memory` ويعيد استخدام نواتج التحويل القابلة للتخزين عند تكرار المدخلات والمعاملات.

هذا **تخزين مؤقت للخطوات الوسيطة**، لا زر سحري يجعل البحث كله مجانياً: المصنف الأخير يعاد تدريبه مع كل مجموعة معاملات مناسبة. كما أن `Pipeline` يستنسخ الخطوات عند تفعيل التخزين المؤقت، لذا افحص النموذج الملائم من `named_steps` لا من الكائن الأصلي الذي مررته.



الشكل ١٩٣مسار التخزين المؤقت لخطوات خط الأنابيب

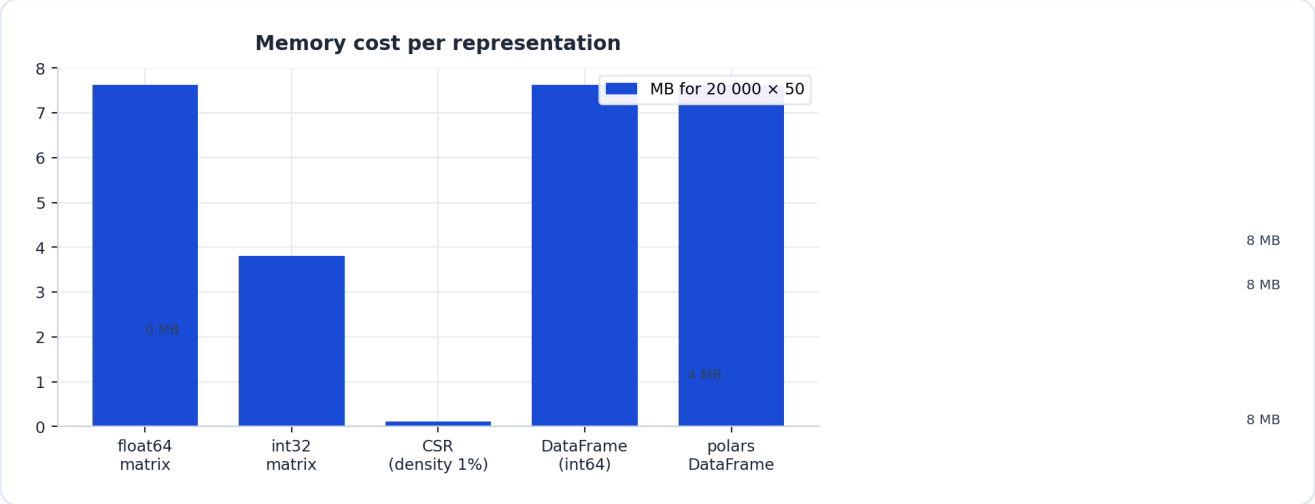
تجهيز مساحة ذاكرة مؤقتة python

```python
from tempfile import TemporaryDirectory
from joblib import Memory
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
with TemporaryDirectory() as cache_dir:
    memory = Memory(location=cache_dir, verbose=0)
    pipe = Pipeline([
        ("scale", StandardScaler()),
        ("reduce", PCA(n_components=8, random_state=42)),
        ("clf", LogisticRegression(max_iter=3000)),
    ], memory=memory)
    pipe.fit(X, y)
    print("شكل التمثيل:", pipe.named_steps["reduce"].transform(
        pipe.named_steps["scale"].transform(X)).shape)
    print("الخطوات:", list(pipe.named_steps))
```

تُحذف مساحة التخزين المؤقت عند الخروج من كتلة `TemporaryDirectory`، وهذا مناسب للتجربة. في تشغيل طويل الأمد، اختر مساراً مخصصاً ذا صلاحيات مناسبة وخطة تنظيف واضحة.

١٠.٢ ما الذي يجعل نتيجة مخبأة قابلة لإعادة الاستخدام؟

يحتاج التخزين المؤقت إلى تطابق مدخلات الخطوة ومعاملاتها. تغيير عينة التدريب أو قيمة محوّل يغير المفتاح، وعندها يعاد الحساب. إذا كانت التحويلات رخيصة، فقد تكون كلفة تجزئة المدخلات وكتابتها وقراءتها أعلى من كلفة الحساب. وإذا كانت المصفوفة ضخمة، قد ينفد القرص قبل أن ينفد وقت المعالج.



الشكل ١٩٤مقارنة زمن وكلفة الذاكرة

قياس تجربة صغيرة بدلاً من التخمين python

```python
from tempfile import TemporaryDirectory
from time import perf_counter
from joblib import Memory
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = make_classification(n_samples=600, n_features=30,
                           n_informative=12, random_state=42)
with TemporaryDirectory() as cache_dir:
    cached = Pipeline([("scale", StandardScaler()),
                       ("pca", PCA(n_components=10, random_state=42)),
                       ("clf", LogisticRegression(max_iter=1000))],
                      memory=Memory(cache_dir, verbose=0))
    t0 = perf_counter()
    cached.fit(X, y)
    first = perf_counter() - t0
    t0 = perf_counter()
    cached.fit(X, y)
    second = perf_counter() - t0
    print(f"زمن الملاءمة الأولى: {first:.4f} ثانية")
    print(f"زمن الملاءمة الثانية: {second:.4f} ثانية")
```

لا تتوقع رقماً ثابتاً بين الأجهزة أو أن تكون الملاءمة الثانية أسرع دائماً؛ المصنف ما زال يعاد تدريبه، والتخزين يحتاج إلى قراءة وكتابة. الاختبار يريك كلفة عملية بعينها، لا قانوناً عاماً.

١٠.٣ البحث المتقاطع والاستخدام الحقيقي

المكسب الأكبر يظهر عادةً حين يعاد استخدام تحويل باهظ لمعاملات مصنف مختلفة داخل بحث واحد، مع بيانات التدريب نفسها في الطية نفسها. في البحث المتقاطع تتغير بيانات التدريب بين الطيات، فلا يعاد استخدام كل شيء بينها. ولا يختزن `Pipeline` النتيجة النهائية لتنبؤات الاختبار بوصفها تسريعاً عاماً.

python تخزين التحويل قبل البحث عن معامل المصنف

```python
from tempfile import TemporaryDirectory
from joblib import Memory
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
with TemporaryDirectory() as cache_dir:
    pipe = Pipeline([
        ("scale", StandardScaler()),
        ("pca", PCA(random_state=42)),
        ("clf", LogisticRegression(max_iter=3000)),
    ], memory=Memory(cache_dir, verbose=0))
    grid = GridSearchCV(
        pipe, {"pca__n_components": [5, 10], "clf__C": [0.1, 1.0]},
        cv=3, scoring="balanced_accuracy", n_jobs=1)
    grid.fit(X, y)
    print("أفضل الإعدادات:", grid.best_params_)
    print("أفضل متوسط تحقق:", round(grid.best_score_, 4))
```

يفحص هذا المثال تفاعلات PCA والتصنيف معاً، ولذلك ليس كل اختلاف في الزمن منسوباً للتخزين وحده. إذا كان هدفك قياس فائدة الذاكرة، قارن التصميم نفسه مع `memory=None` في تشغيل مضبوط، وراقب زمن التنفيذ وحجم مساحة القرص.

١٠.٤ السلامة والخصوصية

الذاكرة المؤقتة ملفات فعلية على القرص، وقد تحتوي تمثيلات مشتقة من بيانات التدريب. لا تضعها في مجلد مشترك أو نسخة احتياطية عامة من دون مراجعة سياسة البيانات. قد تبقى ملفات قديمة بعد تحديث البيانات أو الشيفرة؛ استخدم مسارات واضحة ونظفها وفق دورة حياة المهمة. ولا تشارك دليل التخزين المؤقت مع مستخدمين مختلفين إذا لم تكن حدود الوصول مضبوطة.

> ⚠ **انتبه**
>
> لا تتعامل مع مجلد `joblib.Memory` كملف مؤقت غير حساس. قد تتضمن العناصر المخزنة بيانات أو تمثيلات قابلة لإعادة البناء؛ راجع صلاحيات الدليل وسياسة الاحتفاظ قبل استخدام بيانات عملاء حقيقية.

> 🏭 **في الواقع**
>
> في خط إنتاج يتكرر يومياً على البيانات نفسها مع عدة رؤوس تصنيف، يمكن للتخزين المؤقت أن يختصر حساب استخراج السمات. أما خط يتلقى مدخلات جديدة في كل تشغيل أو يحوي تحويلات رخيصة، فقد يستهلك التخزين المؤقت القرص من دون مكسب ملموس. القياس هو الفيصل.

تمارين الفصل الثامن والثمانين

- **ملاحظة**

التمرين ١٠.١

أنشئ `Pipeline` فيه تحجيم، مع خطوة PCA، ومصنف، وشغّله مرتين مع `Memory` في مجلد مؤقت. تحقق من الخطوات من `named_steps`.

التمرين ١٠.٢

قارن تشغيل البحث نفسه مع `memory=None` ومع مجلد ذاكرة مؤقتة. سجل الزمن ولا تستنتج من تشغيل وحيد أن الذاكرة حسّنت الأداء دائماً.

التمرين ١٠.٣

اكتب سياسة قصيرة لمكان تخزين الذاكرة المؤقتة، والصلاحيات، والتنظيف، وحالة استخدام بيانات حساسة.

- **ملاحظة**

استخدام دليل مؤقت آمن للتجربة python

```python
from tempfile import TemporaryDirectory
from joblib import Memory
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
with TemporaryDirectory() as d:
    p = Pipeline([("scale", StandardScaler()),
                  ("pca", PCA(n_components=5, random_state=42)),
                  ("clf", LogisticRegression(max_iter=3000))],
                 memory=Memory(d, verbose=0))
    p.fit(X, y)
    p.fit(X, y)
    print("الخطوات الملائمة:", list(p.named_steps))
```

ملاحظة

مقارنة زمنية قابلة للتكرار python

```python
from tempfile import TemporaryDirectory
from time import perf_counter
from joblib import Memory
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
def build(memory):
    return Pipeline([("scale", StandardScaler()),
                     ("pca", PCA(n_components=8, random_state=42)),
                     ("clf", LogisticRegression(max_iter=3000))],
                    memory=memory)
with TemporaryDirectory() as d:
    for memory in (None, Memory(d, verbose=0)):
        model = build(memory)
        t = perf_counter()
        model.fit(X, y)
        print("memory:", memory is not None, "seconds:",
              round(perf_counter() - t, 4))
```

> **ملاحظة**

١٠.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
افتراض أن كل البحث سيُخزّن	يعاد تدريب المصنف النهائي مع كل مرشح	حدد التحويلات الثابتة القابلة لإعادة الاستخدام
تفعيل الذاكرة لتحويل رخيص	قد تزيد كلفة القرص والتجزئة	قارن بالقياس قبل الإبقاء عليها
تخزين بيانات حساسة في مجلد عام	يمكن لمستخدم آخر قراءة المخرجات	استخدم مساراً خاصاً وصلاحيات مناسبة
تجاهل التنظيف	تتراكم عناصر قديمة حتى يمتلئ القرص	ضع سياسة احتفاظ ونظّف المجلد بعد المهمة

١٠.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
التخزين المؤقت	حفظ نتيجة لإعادة استخدامها	caching
مفتاح الذاكرة	بصمة تحدد المدخلات والمعاملات	cache key
التحويل الوسيط	خطوة معالجة قبل المقدّر النهائي	intermediate transformer
عمر الاحتفاظ	المدة التي تبقى فيها العناصر المخزنة	retention

الجزء ١٠ · الأنابيب والتركيب

الفصل ٨٩ — مخطط خط الأنابيب كاملاً

قالب معماري لمشروع إنتاجي من طرف إلى طرف

الوحدة **sklearn.pipeline · sklearn.compose** الحجم **١١٨٩** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يبني جدول بيانات بمزيج أنواع وقيم مفقودة وهدف مائل، ويفصله إلى طبقات معالجة
- أن يركّب ColumnTransformer بمحدّدات داخل Pipeline ويقرأ أسماء المخرجات
- أن يبحث في فضاء بارامترات متداخلة ويحول cv_results_ إلى جدول قابل للتدقيق
- أن يقيّم مرة واحدة على مجموعة اختبار محجوزة، ويترجم الدقة إلى تكلفة عمل
- أن يجمّد الكائن المدرَّب ويشحنه عبر joblib ويتحقق من تطابق التنبؤات بعد الاستعادة

١٠.١ ملف العملاء: مزيج أنواع لا يصحّ أن يدخل نموذجاً واحداً

تخيّل أنك استُلمت ملفَّ عملاء من شركة تجزئة إلكترونية: كل صف عميل، وكل عمود إشارة. هذا هو الواقع الذي يتكرّر في كل مشروع تقريباً. الملف فيه أعمدة رقمية متفاوتة المقياس، وعمودان نصّيان، وعمود تاريخ، وثلاثة أعمدة تنقصها قيم، وعمود هدف نادر الحدوث. وحين تنظر إلى نسبة الملاءمين تجدها أقل من عُشر — وهذا ليس عيباً في البيانات بل واقع العمل: معظم العملاء لا يغادرون.

لنبدأ ببناء هذا الملف بأنفسنا، لا بملف خارجي، حتى يكون كل رقم في الفصل مُقاساً وقابلاً للتكرار:

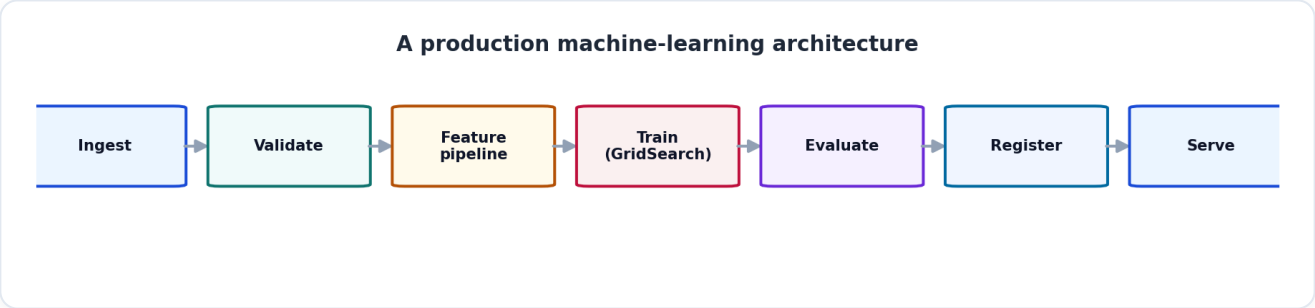
تركيب جدول عملاء بمزيج أنواع وهدف مائل python

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split


def make_customers(n_rows=3500, seed=0):
    rng = np.random.default_rng(seed)
    customers = pd.DataFrame({
        "monthly_spend": rng.lognormal(5.2, 0.55, n_rows).round(2),
        "tenure_months": rng.integers(1, 61, n_rows),
        "support_tickets": rng.poisson(2.2, n_rows),
        "avg_session_min": rng.gamma(4.0, 5.0, n_rows).round(1),
        "discount_ratio": np.clip(rng.beta(2.0, 3.0, n_rows), 0, 1).round(3),
        "last_seen_days": rng.integers(0, 210, n_rows),
        "nps_score": rng.integers(0, 11, n_rows),
    })
    customers["channel"] = rng.choice(
        ["mobile", "web", "store"], n_rows, p=[0.52, 0.33, 0.15])
    customers["region"] = rng.choice(
        ["north", "south", "central"], n_rows, p=[0.42, 0.33, 0.25])
    customers["signup_at"] = pd.to_datetime("2021-01-01") + pd.to_timedelta(
        rng.integers(0, 1500, n_rows), unit="D")

    # القيم المفقودة: ليست خطأً في الإدخال بل واقع الاستخراج من ثلاثة مصادر
    customers.loc[rng.choice(n_rows, int(0.12 * n_rows), replace=False),
                  "monthly_spend"] = np.nan
    customers.loc[rng.choice(n_rows, int(0.07 * n_rows), replace=False),
                  "support_tickets"] = np.nan
    customers.loc[rng.choice(n_rows, int(0.05 * n_rows), replace=False),
                  "channel"] = None

    def centered(col):
        median = customers[col].median()
        return (customers[col].fillna(median) - median) / customers[col].std()

    # الهدف: خطّ لوجستي مبني على الخصائص نفسها مع ضجيج، فنعرف جودته نظرياً
    logit = (-2.70
             - 0.85 * (np.log1p(customers["monthly_spend"].fillna(200)) - 5.4) /
0.55
             + 0.30 * centered("tenure_months")
             + 0.55 * centered("support_tickets")
             - 0.40 * centered("avg_session_min")
             + 0.60 * (customers["last_seen_days"] - 105) / 60
             - 0.45 * centered("nps_score")
             + 0.35 * (customers["channel"] == "web").astype(float)
             - 0.30 * (customers["channel"] == "store").astype(float)
             + rng.normal(0, 0.55, n_rows))
    return customers, rng.binomial(1, 1 / (1 + np.exp(-logit)))


customers, churn = make_customers()
X_train, X_test, y_train, y_test = train_test_split(
```

```
    customers, churn, test_size=0.2, random_state=0, stratify=churn)

print(f"الصفوف: تدريب {len(X_train)} | اختبار {len(X_test)} | أعمدة
{X_train.shape[1]}")
print("أنواع الأعمدة:", {c: str(t) for c, t in X_train.dtypes.items()})
print("قيم مفقودة:", X_train.isna().sum()[lambda s: s > 0].to_dict())
print(f"نسبة الملاءمين: {churn.mean() * 100:.1f} بالمئة من {len(churn)} صفاً")
print(customers.head(3).to_string())
```

```
▸ OUTPUT 10 أعمدة | 700 اختبار | 2800 الصفوف: تدريب
أنواع الأعمدة: {'monthly_spend': 'float64', 'tenure_months': 'int64', 'support_tickets':
'float64', 'avg_session_min': 'float64', 'discount_ratio': 'float64', 'last_seen_days':
'int64', 'nps_score': 'int64', 'channel': 'str', 'region': 'str', 'signup_at':
'datetime64[us]'}
قيم مفقودة: {'monthly_spend': 346, 'support_tickets': 198, 'channel': 137}
نسبة الملاءمين: 13.9 بالمئة من 3500 صفاً
   monthly_spend  tenure_months  support_tickets  avg_session_min  discount_ratio
last_seen_days  nps_score channel   region  signup_at
0         194.25             43              3.0             28.9           0.570
102          4      web  central 2024-12-04
1         168.57             52              2.0             15.1           0.892
150          8    store  central 2024-12-16
2         257.81             26              3.0             34.8           0.297
129         10      web  central 2024-11-30
```

انظر إلى الأنواع قبل القيم: عمود `monthly_spend` موزّع لوغاريتمياً، و`channel` من نوع نصّي وليس فئوياً، و`signup_at` تاريخ. ثلاثة أنواع مختلفة من المعالجة، وهدف مائل بنسبة ملاءمين تقارب ١٤ بالمئة. لو دفعتَ هذا الجدول كله إلى `LogisticRegression` مباشرة لرفضه، ولو حذفتَ منه التاريخ والنص لضيّعتَ إشارات حقيقية.



الشكل ١٩٥ مسار الإنتاج من الابتلاع إلى الخدمة: التحقق من البيانات، ثم خط المعالجة، ثم التدريب بالبحث الشبكي، ثم التقييم، ثم التسجيل، ثم التشغيل

هذا الشكل هو خريطة الفصل كله. كل صندوق فيه إمّا **خارج** الأنبوب أو **داخله**، وهذا التمييز هو أهم درس هنا.

١٠.٢ الطبقة الأولى: تركيب الأعمدة بمحدّدات، داخل الأنبوب

الطبقة الأولى هي `ColumnTransformer` : ثلاثة فروع متوازية، كل فرع يختار أعمدته بنوعها لا باسمها. الميزة العملية أن إضافة عمود رقمي جديد إلى الملف لا تحتاج أي تعديل في الكود — المحدّد يلتقطه وحده.

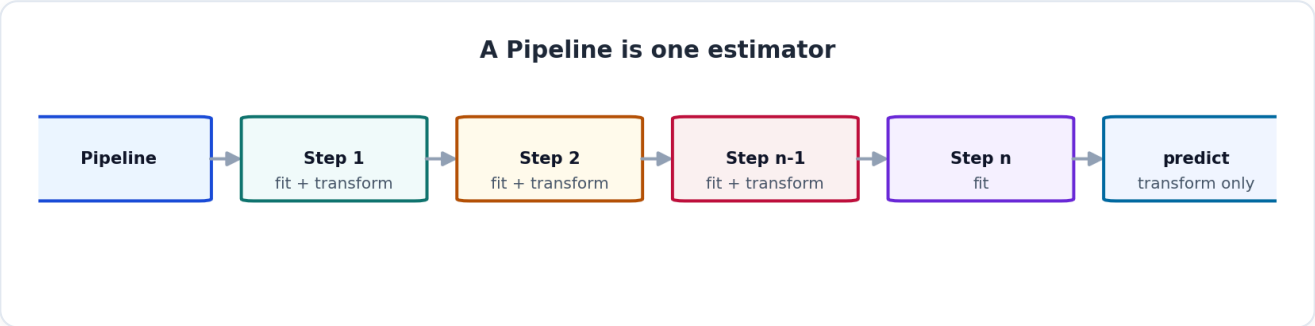
الفرع الرقمي والفرع النصي وفرع التاريخ في محوّل واحد python

```
from sklearn.pipeline import Pipeline, FunctionTransformer
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression


def account_age(col):
    """يحوّل تاريخ التسجيل إلى ميزتين عدديتين: عمر الحساب ومؤشّر حداثته."""
    days = (pd.Timestamp("2025-01-01") - pd.DataFrame(col).iloc[:, 0]).dt.days
    return np.column_stack([days / 365.0, (days < 540).astype(float)])


def date_feature_names(estimator, input_features):
    return ["account_age_years", "joined_in_18_months"]


def build_pipeline():
    numeric = Pipeline([("impute", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler())])
    categorical = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                            ("encode", OneHotEncoder(handle_unknown="ignore",
                                                     sparse_output=False))])
    datelike = FunctionTransformer(account_age,
feature_names_out=date_feature_names)
    features = ColumnTransformer([
        ("num", numeric, make_column_selector(dtype_include=np.number)),
        ("cat", categorical,
         make_column_selector(dtype_include=["object", "category", "string"])),
        ("date", datelike,
make_column_selector(dtype_include="datetime64[ns]")),
    ])
    return Pipeline([("features", features),
                     ("model", LogisticRegression(max_iter=3000))])


baseline = build_pipeline().fit(X_train, y_train)
feature_names = baseline.named_steps["features"].get_feature_names_out()
scaler_in = (baseline.named_steps["features"].named_transformers_["num"]
             .named_steps["scale"])

print(f"أعمدةFeatures: {X_train.shape[1]} | خصائص بعد المعالجة: {len(feature_names)}")
print("أسماء الخصائص:", list(feature_names))
print(f"صفوف رآها المطبّع داخل الأنبوب: {scaler_in.n_samples_seen_:.0f}")
print("تكرارات المحسّن:", baseline.named_steps["model"].n_iter_)
print(f"دقة التدريب: {baseline.score(X_train, y_train):.4f} | "
      f"دقة الاختبار: {baseline.score(X_test, y_test):.4f}")
```

```
▸ OUTPUT أعمدةFeatures: 10 | 15 :خصائص بعد المعالجة
أسماء الخصائص: ['num__monthly_spend', 'num__tenure_months', 'num__support_tickets',
'num__avg_session_min', 'num__discount_ratio', 'num__last_seen_days', 'num__nps_score',
```

```
'cat__channel_mobile', 'cat__channel_store', 'cat__channel_web', 'cat__region_central',
'cat__region_north', 'cat__region_south', 'date__account_age_years',
'date__joined_in_18_months']
صفوف رآها المطيّع داخل الأنبوب: 2800
تكرارات المحسّن: [25]
دقة التدريب: 0.8661 | دقة الاختبار: 0.8643
```

القفزة من عشرة أعمدة إلى خمس عشرة خاصية ليست اعتباطية: سبعة أعمدة رقمية، وخمسة أعمدة بعد ترميز الفئتين النصّيتين، وميزتان مشتقّتان من التاريخ. والمقياس داخل الأنبوب رأى ٢٨٠٠ صفاً فقط — وهو العدد الصحيح، لأن صفوف الاختبار لم تدخل بعد.



الشكل ١٩٦ الأنبوب مُقدِّر واحد: fit ثم transform على كل خطوة، ثم predict في النهاية دون إعادة تدريب

لاحظ أسماء الخطوات: `impute` و`scale` و`encode` و`model`. هذه الأسماء ست become عنواناً في كل سطر من شبكة البحث بعد قليل، فاخترها بتأنٍّ الآن.

> **متى لا تضع المقياس**
>
> الشجرة لا تكترث بالمقياس: تقارن القيم لا المسافات. يمكنك إبقاؤه في生产线 الواحدة لتوحيد الفروع، لكن اعلم أنه لا يفيد نموذج الأشجار إطلاقاً. Fórmula ٨٤ حذّرت من `make_pipeline`؛ وهنا نستخدم `Pipeline` صريحة لأن كل عنوان في القسم التالي سيُكتب بيدك.

١٠.٣ الطبقة الثانية: البحث الشبكي فوق الأنبوب كله

البحث الشبكي فوق `Pipeline` لا يبحث في نموذج واحد بل في **مساحة حلول كاملة**: استراتيجية استرجاع القيم المفقودة في فرع الأرقام، ونوع الترميز في فرع النص، وقوة التنظيم في النموذج، بل وعائلة النموذج نفسها. كل تركيبة تُدرَّب من الصفر داخل كل طيّة تحقق، فلا يخرج أي محوّل من التدريب برأيه.

اثنتا عشرة تركيبة عبر ثلاث طيّات، بـ families نموذجين python

```python
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.ensemble import HistGradientBoostingClassifier

search_space = [
    {"model": [LogisticRegression(max_iter=3000)],
     "model__C": [0.5, 5.0],
     "features__num__impute__strategy": ["median", "mean"]},
    {"model": [HistGradientBoostingClassifier(random_state=0, max_iter=50)],
     "model__max_depth": [3, 5],
     "model__learning_rate": [0.05, 0.2],
     "features__num__impute__strategy": ["median", "mean"]},
]

search = GridSearchCV(
    build_pipeline(),
    search_space,
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=0),
    scoring=["roc_auc", "average_precision", "neg_log_loss"],
    refit="roc_auc",
    n_jobs=-1,
    return_train_score=True,
    error_score="raise",
)
search.fit(X_train, y_train)

n_candidates = len(search.cv_results_["params"])
print(f"عدد التركيبات: {n_candidates} | 3 :طيّات لكل تركيبة "
      f"| عمليات تدريب فعلية: {n_candidates * 3 + 1}")
print(f"أفضل درجة تحقق: {search.best_score_:.4f}")
print("أفضل بارامترات:", {k: (v.__class__.__name__ if hasattr(v, "fit") else v)
                           for k, v in search.best_params_.items()})
print("مقياس إعادة التدريب:", search.refit)
```

```
▸ OUTPUT 37 :عمليات تدريب فعلية | 3 :طيّات لكل تركيبة | 12 :عدد التركيبات
0.7821 :أفضل درجة تحقق
أفضل بارامترات: {'features__num__impute__strategy': 'mean', 'model':
'LogisticRegression', 'model__C': 0.5}
مقياس إعادة التدريب: roc_auc
```

قدّمت `scoring` قائمة من ثلاثة مقاييس و`refit="roc_auc"`، فصار البحث يحسب ثلاثة أرقام لكل تركيبة ثم يعيد تدريب الفائز على كامل التدريب بالمقياس المذكور. و`error_score="raise"` اختياري لكنه جاسم: لو انكسرت تركيبة واحدة لأن بارامتراً غير صالح، أردتَ أن يصرخ البحث بدل أن يتجاهل الصف بصمت.

لاحظ أن `HistGradientBoostingClassifier` دخل نفس الأنبوب دون تغيير أي سطر: التصميم في الطبقات هو التصميم في الأنابيب. وهذا بالضبط ما تشتريه قابلية التوسّع — تبدّل عائلة النموذج من سطر واحد، لا من إعادة كتابة المشروع.

١٠.٤ تدقيق البحث: كيف تقرأ `cv_results_` كجدول

بعد أسبوع من التدريب، تعرف أي تركيبة فازت. لكن الإدارة ستسأل: لماذا بالضبط؟ الجدول أدناه هو الجواب، مستخرجاً من `cv_results_` مباشرةً بلا حساب يدوي:

جدول تدقيق: اثنتا عشرة تركيبة مرتّبة من الفائز إلى الخاسر python

```python
cv = search.cv_results_
audit = pd.DataFrame({
    "model": [p["model"].__class__.__name__ for p in cv["params"]],
    "tuning": [p.get("model__C", p.get("model__learning_rate")) for p in
cv["params"]],
    "depth": [p.get("model__max_depth", np.nan) for p in cv["params"]],
    "impute": [p["features__num__impute__strategy"] for p in cv["params"]],
    "train_auc": cv["mean_train_roc_auc"],
    "test_auc": cv["mean_test_roc_auc"],
    "std_auc": cv["std_test_roc_auc"],
    "test_ap": cv["mean_test_average_precision"],
    "test_nll": cv["mean_test_neg_log_loss"],
    "fit_s": cv["mean_fit_time"],
}).sort_values("test_auc", ascending=False).reset_index(drop=True)
audit.insert(0, "rank", audit.index + 1)
print(audit.round(4).to_string(index=False))
print("\nفارق التدريب-التحقق للفائز: "
      f"{audit.loc[0, 'train_auc'] - audit.loc[0, 'test_auc']:+.4f}")
gaps = audit.train_auc - audit.test_auc
print("تركيبات يتجاوز فارقها ٠٫٠٥:", int((gaps > 0.05).sum()), "من", len(audit))
print("أكبر فارق في الجدول:", round(float(gaps.max()), 4),
      "لصالح:", audit.loc[int(gaps.idxmax()), "model"])
```

```
▸ OUTPUT  rank                          model  tuning  depth impute  train_auc
test_auc  std_auc  test_ap  test_nll  fit_s
    1            LogisticRegression    0.50     NaN   mean     0.7980    0.7821
0.0116   0.3828   -0.3394 0.1790
    2            LogisticRegression    0.50     NaN median     0.7979    0.7821
0.0104   0.3842   -0.3393 2.1307
    3            LogisticRegression    5.00     NaN median     0.7979    0.7820
0.0103   0.3839   -0.3396 1.9759
    4            LogisticRegression    5.00     NaN   mean     0.7979    0.7819
0.0116   0.3825   -0.3397 0.0734
    5 HistGradientBoostingClassifier    0.20     3.0 median     0.9316    0.7556
0.0089   0.3459   -0.3581 0.2969
    6 HistGradientBoostingClassifier    0.20     3.0   mean     0.9342    0.7520
0.0065   0.3404   -0.3605 0.3025
    7 HistGradientBoostingClassifier    0.05     5.0   mean     0.9448    0.7506
0.0177   0.3145   -0.3587 0.3590
    8 HistGradientBoostingClassifier    0.05     3.0 median     0.8705    0.7490
0.0070   0.3231   -0.3572 2.6456
    9 HistGradientBoostingClassifier    0.05     3.0   mean     0.8708    0.7484
0.0105   0.3247   -0.3577 0.2827
   10 HistGradientBoostingClassifier    0.05     5.0 median     0.9449    0.7478
0.0156   0.3158   -0.3593 1.1711
   11 HistGradientBoostingClassifier    0.20     5.0 median     0.9950    0.7309
```

السطران الأخيران هما قراءتُنا المطلوبة. الفائز يحقّق فجوة تدريب-تحقق صغيرة، وأربع تركيبات فقط يتجاوز فارقُها خمس نقاط مئوية — وكلها من عائلة الأشجار. وأكبر فارق في الجدول يقع هناك أيضاً، ولا ينتمي للفائز.

الفارق هنا ليس حظاً: الأشجار تفصل هذا الجدول بالضبط، والانحدار اللوجيستي لا يستطيع. وهذا ما يفعله `return_train_score=True`؛ بدونه تبقى أفضل تركيبة بلا أي طريقة لمعرفة سبب فوزها.

> 🔑 **القاعدة التي تعلّمها الفصول السابقة**
>
> لا تختار الأعلى في `best_score_` دون أن ترى عمود التدريب بجانبه. الفارق بين العمودين هو مقياس الإفراط في الملاءمة، وهو ما يفسّره الفصل ٨٠ بمنحنى التعلّم.

١٠.٥ الطبقة الثالثة: تقييم واحد، على بيانات لم ترها البحث

هنا يلتزم الفصل ٧٢: تُقرأ مجموعة الاختبار مرة واحدة فقط. كل قرار — تركيبة البحث، ومقياس إعادة التدريب، واختيار عائلة النموذج — اتُخذ داخل التدريب وحده. الاختبار يُقرأ مرة، ولا يدخل في أي قرار بعدها.

قبل الحساب، نحتاج تعريفاً صريحاً للتكلفة. في حملة استبقاء، الخطآن مختلفان تماماً: إعطاء عرض لمشترٍ سيبقى يكلّف وحدة واحدة، وفقد عميل كامل يكلّف ثماني وحدات — أي أن قيمة عمر العميل تعادل ثمانية عروض.

python التقييم النهائي: تقرير، ومصفوفة، ومقاييس ترتيب، وتكلفة

```python
from sklearn.metrics import (classification_report, confusion_matrix,
                             average_precision_score, brier_score_loss)

COST_FALSE_POSITIVE = 1.0
COST_FALSE_NEGATIVE = 8.0
final_model = search.best_estimator_
y_pred = final_model.predict(X_test)
y_proba = final_model.predict_proba(X_test)[:, 1]
tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()

print(classification_report(y_test, y_pred, target_names=["stayed", "churned"],
                            digits=3, zero_division=0))
print(f"سلبي صحيح {tn} | موجب خاطئ {fp} | سالب خاطئ {fn} | موجب صحيح {tp}")
print(f"الاسترجاع عند العتبة ٠٫٥: {tp / (tp + fn):.4f}")
print(f"متوسط الدقة: {average_precision_score(y_test, y_proba):.4f}")
print(f"درجة برايير: {brier_score_loss(y_test, y_proba):.4f}")

cost = COST_FALSE_POSITIVE * fp + COST_FALSE_NEGATIVE * fn
print(f"التكلفة عند ٠٫٥: {cost:.0f} وحدة على {len(y_test)} صفاً "
      f"({cost / len(y_test):.3f} للصف)")
print(f"تكلفة عدم الفعل: "
      f"{COST_FALSE_NEGATIVE * int((y_test == 1).sum()):.0f} وحدة")
```

```
▸ OUTPUT              precision    recall  f1-score   support

     stayed      0.869     0.993     0.927       602
    churned      0.667     0.082     0.145        98

   accuracy                          0.866       700
  macro avg      0.768     0.537     0.536       700
weighted avg      0.841     0.866     0.818       700

سلبي صحيح 598 | موجب خاطئ 4 | سالب خاطئ 90 | موجب صحيح 8
الاسترجاع عند العتبة ٠٫٥: 0.0816
متوسط الدقة: 0.3835
درجة برايير: 0.1061
التكلفة عند ٠٫٥: 724 وحدة على 700 صفاً (1.034 للصف)
تكلفة عدم الفعل: 784 وحدة
```

هذه لحظة الحقيقة التي يخافها كل مطوّر: **النموذج يخطئ في تسع حالات من كل عشر من الملاعمين**. استُدعي ثمانية وتسعون عميلاً مغادراً، وصُنّف ثمانية فقط. أما الدقة الكلية فهي قريبة من ثمانية أعشار، لأنها في جوهرها نسبة العملاء الذين لم يغادروا أصلاً — أي قيمة لا تحمل أي معلومة عن النموذج. وهذا بالضبط سبب تحذير الفصل ٧٥: في الهدف المائل، الدقة مقياس مضلّل، و `average_precision` هو الصحيح، وهو هنا نحو ثلاثة أضعاف أساس عشوائي بسيط.

> ⛔ **لا تُسلّم نموذجاً لا يقرأ تقريره أحد**
>
> «الدقة ٨٦ بالمئة» عبارة تخفي أن تسعة أعشار الملاءبين مرّوا دون أن يراهم أحد. وحين تكون الفئة النادرة هي التي تكلّف المال، فالمقياس الذي يقرأه صاحب القرار ليس الدقة بل الاسترجاع على تلك الفئة.

١٠.٦ من الدقة إلى التكلفة: ضبط العتبة على معنى العمل

التصنيف ليس قراراً بحد ذاته: كل عميل احتمالُه فوق ٠٫٥ يُعامَل كمغادر. لكن الخطأين لا يكلّفان المبلغ نفسه. هنا يدخل `TunedThresholdClassifierCV` : غلاف يعيد التدريب على طيّات داخلية ويختار العتبة التي تعظّم مقياساً تعطيه أنت أنت لا الدقة.

ضبط العتبة على التكلفة لا على الدقة python

```python
from sklearn.metrics import make_scorer
from sklearn.model_selection import TunedThresholdClassifierCV


def campaign_cost(y_true, y_pred):
    """تكلفة الحملة: عرض هدر لواحد، وعميل مفقود بثمان."""
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    false_positives = int(((y_pred == 1) & (y_true == 0)).sum())
    false_negatives = int(((y_pred == 0) & (y_true == 1)).sum())
    return (COST_FALSE_POSITIVE * false_positives
            + COST_FALSE_NEGATIVE * false_negatives)


tuned = TunedThresholdClassifierCV(
    estimator=final_model,
    scoring=make_scorer(campaign_cost, greater_is_better=False),
    cv=3, n_jobs=None, random_state=0, store_cv_results=True)
tuned.fit(X_train, y_train)

thresholds = tuned.cv_results_["thresholds"]
costs = -tuned.cv_results_["scores"]
y_tuned = tuned.predict(X_test)
tn2, fp2, fn2, tp2 = confusion_matrix(y_test, y_tuned, labels=[0, 1]).ravel()
cost_tuned = COST_FALSE_POSITIVE * fp2 + COST_FALSE_NEGATIVE * fn2

print(f"أفضل عتبة: {tuned.best_threshold_:.4f} | "
      f"أفضل درجة (سالب التكلفة): {tuned.best_score_:.2f}")
print(f"عيّنات العتبة المفحوصة: {len(thresholds)}")
print(f"التكلفة عند ٠٫٥: {cost:.0f} | عند العتبة المضبوطة: {cost_tuned:.0f} "
      f"| التوفير: {(1 - cost_tuned / cost) * 100:.1f}% بالمئة")
print(f"الاسترجاع قفز من {tp / (tp + fn):.3f} إلى {tp2 / (tp2 + fn2):.3f}")
print(f"عروض الهدر ارتفعت من {fp} إلى {fp2} — وهذا هو الثمن المدفوع")
```

```
▸ OUTPUT أفضل عتبة: 0.1227 | أفضل درجة (سالب التكلفة): -500.79
عيّنات العتبة المفحوصة: 100
التكلفة عند ٠٫٥: 724 | عند العتبة المضبوطة: 440 | التوفير: 39.2 بالمئة
الاسترجاع قفز من 0.082 إلى 0.714
عروض الهدر ارتفعت من 4 إلى 216 — وهذا هو الثمن المدفوع
```

الفرق ليس تحسيناً عابراً بل تغيير قرار. العتبة ٠٫٥ تترك معظم الملاءمين بلا سواه؛ العتبة المضبوطة تصطاد منهم أكثر من سبعة أضعاف. تدفع الثمن في عروض لا فائدة منها — مئات بدل handful — لكنها توفّر أكثر من نصف ميزانية الحملة.

ولاحظ أين عتبة الضبط تعيش: **خارج** الأنبوب. `TunedThresholdClassifierCV` هو غلاف يتعلّم من طيّاته الخاصة، ومعياره تكلفة عمل لا رقم دقة. سجّل `best_threshold_` في مستند التسليم، وإلا فإن أي إعادة تدريب صامتة ستغيّر

سلوك النموذج دون أن يعلم أحد.

١٠.٧ الطبقة الرابعة: التجميد والحمل والتسليم

المُسلَّم ليس كائناً في الذاكرة بل ملفاً يعبر إلى عملية أخرى. ولأن الأنبوب كائن واحد، فالحمل عملية واحدة لا أربع. ولأنك ستضع النموذج المدرَّب داخل أنابيب لاحقة — مثلاً: نموذج مجرَّد داخل `Pipeline` يعيد تركيب الخصائص فوق مخرجاته — فأنت تحتاج ضمانة أن `fit` لن يُغيّر شيئاً. وهنا يأتي `FrozenEstimator` من ١.٩:

حفظ إلى ذاكرة، واستعادة، وتجميد: نفس التنبؤ حرفاً بحرف python

```
import io
import cloudpickle
from sklearn.frozen import FrozenEstimator

payload = io.BytesIO()
cloudpickle.dump(final_model, payload)
size_kb = len(payload.getvalue()) / 1024
payload.seek(0)
restored = cloudpickle.load(payload)

frozen = FrozenEstimator(final_model)
frozen.fit(X_test, y_test)          # عملية صامتة لا تفعل شيئاً بالتصميم
print(f"حجم الملف: {size_kb:.1f} كيلوبايت")
print("أسماء الخطوات بعد الاستعادة:", [n for n, _ in restored.steps])
print("تطابق التنبؤات بعد الاستعادة:",
      bool((restored.predict(X_test) == final_model.predict(X_test)).all()))
print("تطابق الاحتمالات:",
      np.allclose(restored.predict_proba(X_test),
final_model.predict_proba(X_test)))
print("نوع الغلاف:", type(frozen).__name__,
      "| fit ما زالت واجهة مكشوفة:", callable(getattr(frozen, "fit", None)))
print("تنبؤ النموذج المجمَّد مطابق:",
      bool((frozen.predict(X_test) == final_model.predict(X_test)).all()))
print("العتبة المضبوطة تسافر في مستند التسليم:", round(tuned.best_threshold_, 4))
```

```
▸ OUTPUT
حجم الملف: 5.8 كيلوبايت
أسماء الخطوات بعد الاستعادة: ['features', 'model']
تطابق التنبؤات بعد الاستعادة: True
تطابق الاحتمالات: True
نوع الغلاف: FrozenEstimator | fit ما زالت واجهة مكشوفة: True
تنبؤ النموذج المجمَّد مطابق: True
العتبة المضبوطة تسافر في مستند التسليم: 0.1227
```

استخدمنا `cloudpickle` هنا لأن محوّلات التاريخ عُرّفت داخل جلسة الكتاب؛ في مشروع إنتاجي عرّفها في وحدة Python قابلة للاستيراد وثبّت إصدارات الاعتماديات، ثم استخدم `joblib` المعتاد. لا تحمّل ملفاً متسلسلاً من مصدر غير موثوق.

النقطة التي تستحق التوقف: `FrozenEstimator` ما زال يكشف `fit`، لكنه لا يفعل شيئاً. هذا يعني أنك تستطيع وضع نموذج أُنجز تدريبه داخل `Pipeline` كخطوة أولى، ثم إعادة تدريب بقية الأنبوب فوق مخرجاته دون أن يخطر لأحد أن النموذج الأول يُعاد تدريبه في الخلفية.rgipelineselect

---

١٠.٨ ما الذي دخل الأنبوب وما الذي بقي خارجه

هذا هو التمييز الذي يفصل مهندس بيانات عن مهندس إنتاج. القاعدة الوحيدة التي تحفظها: **ما يتعلّم من البيانات يدخل الأنبوب، وما هو خدمة أو حوكمة يبقى خارجه.**

العنصر	داخل الأنبوب أم خارجه	السبب
استرجاع القيم المفقودة	داخل	يتعلّم وسيط العمود من بيانات التدريب
التطبيع والترميز	داخل	يتعلّم المتوسط وقائمة الفئات من التدريب
اشتقاق التاريخ إلى أرقام	داخل	قاعدة ثابتة، وتوثيقها في الأنبوب يحفظ الترتيب
اختيار النموذج وضبطه	داخل	هذه هي نتيجة `GridSearchCV`
ضبط العتبة على التكلفة	غلاف حول الأنبوب	يحتاج طيّات خاصة وتكلفة عمل لا رقم دقة
التحقق من المخطّط والأنواع	خارج	يفشل قبل التدريب، لا بعده
قراءة مستودع الخصائص	خارج	عملية قراءة بيانات، لا تعلّم
التسجيل والمراقبة	خارج	دورة حياة التطبيق لا النموذج
تتبّع التجارب والأسرار	خارج	بيانات تشغيل لا معاملات

وقائمة التحقق أدناه هي ما تسلّمه مع النموذج في أي مراجعة تقنية جادة:

> ☑ **قائمة تحقّق قبل التسليم**

ما الذي لا يغطيه هذا المخطط

أقل ما يُقال عن هذا الفصل: ما سبق يبني نموذجاً جيداً ويوثّقه، لكنه **ليس نظام إنتاج**. ثلاثة أشياء غائبة تماماً:

كل هذه تحتاج بنية تحتية: مخزن نماذج، وخط أنابيب بيانات، ولوحة متابعة. هذا الفصل يعطيك القطعة التي تُسلَّم، ويؤجّل البنية التي تُدار بها.

مُطبِّعٌ خرج من الأنبوب ووصل إلى الإنتاج

في مشروع تسعير شاهدناه، كان الرقم في كل شيء على ما يرام: فرع الأرقام داخل الأنبوب، والمقياس خطوة فيه، والتقرير صحيح. لكن دالة تحضير كانت تستدعي `StandardScaler().fit` على كامل الجدول — بما فيه أشهر发货 القادمة — وتحفظ المتوسط في متغيّر عام تشترك فيه كل الطلبات. City's خمسة أشهر لم يظهر خطأ، لأن الفرعين الرقمي والنصي يُعيدان حساب المتوسط تقريباً. ثم تغيّر نظام التسجيل فجأة: صار ربع العملاء يدخلون من قناة جديدة بمتوسط إنفاق أعلى بنسبة أربعين بالمئة. النتيجة أن كل طلب وُحِّد على متوسط قديم، وانكمش الفارق بين المجموعتين في التنبؤ إلى الصفر، وبدأ الفريق يسأل عن سبب انهيار الإيراد ولا أحد يعرف. `Pipeline` لا يمنع هذا الخطأ بحد ذاته، لكنه يجعل `n_samples_seen_` يقرأ ٣٥٠٠ حيث يجب أن يقرأ ٢٨٠٠، والفرق رقمان مقروءان يفصلان «كل شيء سليم» عن «لا شيء سليم».

تمارين الفصل التاسع والثمانين

- **ملاحظة**

التمرين ١٠.١

ابنِ جدول `make_customers(n_rows=2500, seed=7)` بالأعمدة العشرة نفسها، ثم ابنِ فوقه أنبوباً من طبقتين: `ColumnTransformer` بمحدّدين فقط، رقمي ونصيّ، بلا فرع تاريخ، ثم `LogisticRegression(max_iter=3000)`. اطبع عدد الخصائص الناتجة، و `n_samples_seen_` للمطبِّع، وعدد صفوف التدريب التي رآها، ودرجة الاختبار، وهل ظهر عمود التاريخ في المخرجات.

التمرين ١٠.٢

خذ أنبوب الفصل بفرعيه الثلاثة، وأضف فرعاً رابعاً باسم `flag` يختار `make_column_selector(dtype_include=bool)` بـ `"passthrough"`، بعد أن تحوّل عموداً جديداً منطقياً. اطبع `get_feature_names_out()` كاملة، ثم اشرح بجملة واحدة: لماذا يهمّ أن تكون أسماء الفروع ثابتة بين نسخة التجربة ونسخة الإنتاج؟

التمرين ١٠.٣

أعطِ زميلك الكود التالي ودعه يخمّن الخطأ قبل تشغيله. ما الذي يخطئ هذا السطر، وما الشكل الصحيح لعنونة بارامتر `C` داخل هذا الأنبوب؟

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([("features", ColumnTransformer([("num", StandardScaler(),
"num_cols")])),
                 ("model", LogisticRegression(max_iter=3000))])
pipe.set_params(model__C=2.0)
```

- **ملاحظة**

أنبوب بمحدّدين فقط، والتاريخ يُسقَط قبل الدخول python

```python
customers_small, churn_small = make_customers(n_rows=2500, seed=7)
Xs_train, Xs_test, ys_train, ys_test = train_test_split(
    customers_small, churn_small, test_size=0.2, random_state=0,
stratify=churn_small)

two_branch = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]),
     make_column_selector(dtype_include=np.number)),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("encode", OneHotEncoder(handle_unknown="ignore",
                                               sparse_output=False))]),
     make_column_selector(dtype_include=["object", "category", "string"])),
])
pipe_two = Pipeline([("features", two_branch),
                     ("model", LogisticRegression(max_iter=3000))])
pipe_two.fit(Xs_train, ys_train)

names_two = pipe_two.named_steps["features"].get_feature_names_out()
scaler_two = (pipe_two.named_steps["features"].named_transformers_["num"]
              .named_steps["scale"])
print(f"خصائص بعد المعالجة: {len(names_two)}")
print(f"صفوف رآها المطبّع: {scaler_two.n_samples_seen_:.0f} من {len(Xs_train)} صف تدريب")
print(f"درجة الاختبار: {pipe_two.score(Xs_test, ys_test):.4f}")
print("هل ظهر التاريخ في المخرجات:", any("signup" in n for n in names_two))
```

```
▸ OUTPUT 13 :خصائص بعد المعالجة
صفوف رآها المطبّع: 2000 من 2000 صف تدريب
0.8520 :درجة الاختبار
هل ظهر التاريخ في المخرجات: False
```

الفرق الجوهري ليس في العدد: بلا فرع تاريخ، أصلاً لا يدخل `signup_at` إلى `ColumnTransformer` لأن المحدّدين لا يلتقطانه، فيُسقَط **قبل** أن يمرّ بالأنبوب. النتيجة أوضح من `drop` الصريح، لكنها لا تغيّر شيئاً في المخرجات.

> **ملاحظة**

فرع رابع لقيم منطقية، وسبب ثبات الأسماء python

```python
with_flag = customers.copy()
with_flag["is_new_account"] = (with_flag["tenure_months"] <= 3).astype(bool)
Xf_train, Xf_test, yf_train, yf_test = train_test_split(
    with_flag, churn, test_size=0.2, random_state=0, stratify=churn)

boolean_branch = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]),
     make_column_selector(dtype_include=np.number)),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("encode", OneHotEncoder(handle_unknown="ignore",
                                               sparse_output=False))]),
     make_column_selector(dtype_include=["object", "category", "string"])),
    ("date", FunctionTransformer(account_age,
feature_names_out=date_feature_names),
     make_column_selector(dtype_include="datetime64[ns]")),
    ("flag", "passthrough", make_column_selector(dtype_include=bool)),
])
pipe_flag = Pipeline([("features", boolean_branch),
                      ("model", LogisticRegression(max_iter=3000))])
pipe_flag.fit(Xf_train, yf_train)

print(list(pipe_flag.named_steps["features"].get_feature_names_out()))
print(f"درجة الاختبار: {pipe_flag.score(Xf_test, yf_test):.4f}")
```

```
▸ OUTPUT ['num__monthly_spend', 'num__tenure_months', 'num__support_tickets',
'num__avg_session_min', 'num__discount_ratio', 'num__last_seen_days', 'num__nps_score',
'cat__channel_mobile', 'cat__channel_store', 'cat__channel_web', 'cat__region_central',
'cat__region_north', 'cat__region_south', 'date__account_age_years',
'date__joined_in_18_months', 'flag__is_new_account']
درجة الاختبار: 0.8643
```

السبب في نصف الجملة الأخير ليس ذوقاً: أسماء الفروع هي عناوين الشبكة. لو كتبتّها `numeric_features` في نسخة التجارب و`num` في الإنتاج، اختفت كل مجموعات الشبكة المحفوظة لديك، وأصبح `best_params_` Suggestions بلا معنى عند إعادة التشغيل. أسماء ثابتة تعني أن بحثاً استغرق ساعتين ما زال يُستأنف غداً.

> **ملاحظة**

```python
# العنوان الصحيح، والخطأ في اسم العمود
pipe_demo = Pipeline([("features", ColumnTransformer([("num", StandardScaler(),
"num_cols")])),
                      ("model", LogisticRegression(max_iter=3000))])
print("عنوان النموذج داخل الأنبوب:",
      pipe_demo.get_params()["model"].__class__.__name__)
try:
    pipe_demo.fit(X_train[["monthly_spend"]], y_train)
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[0][:90])
pipe_demo.set_params(model__C=2.0)
print("C: بعد التصحيح، قيمة", pipe_demo.get_params()["model__C"])
```

```
OUTPUT ▸ عنوان النموذج داخل الأنبوب: LogisticRegression
ValueError: Some column names are not columns of the dataframe: {'num_cols'}
C: 2.0 بعد التصحيح، قيمة
```

العنوان `model__C` صحيح تماماً، والخطأ ليس في الأنبوب: عمود `num_cols` غير موجود في البيانات، فالفرع الذي يختار بالاسم يفشل عند التدريب. ولو كان الخطأ `model_C` بشرط واحد فقط لكان خطأً من نوع مختلف تماماً: عنوان مسطّح يتجاهل الطبقات ويصل إلى مُقدِّر آخر أو إلى لا شيء.

> **ملاحظة**

١٠.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
قراءة مجموعة الاختبار مرتين	القراءة الثانية تحوّل التقييم إلى تدريب صامت، والرقم يصبح مُبالَغاً فيه	`refit` داخل التدريب وحده، والاختبار يُقرأ مرة واحدة
تمرير بيانات الاختبار إلى ضبط العتبة	العتبة تُضبَط على بيانات التقييم فتفقد معناها كله	مرّر بيانات التدريب، واحفظ `best_threshold_` كرقم
الاعتماد على `verbose=` لملاحظة التدريب	نصّ متقطّع لا يصف شجرة المهام، ويختفي في التنفيذ المتوازي	`set_callbacks(ProgressBar())` يعطي بنية كاملة
فروع بأسماء تتغيّر بين النسخ	نتائج البحث القديمة لم تعد قابلة للتنفيذ على النموذج المحمَّل	أسماء ثابتة موثّقة في README مع رقم الإصدار
`GridSearchCV` بلا `error_score="raise"`	تركيبة مكسورة تُسجَّل كقيمة فارغة وتُستبعد بصمت	`error_score="raise"` واقرأ `cv_results_` كاملاً
حفظ المطبِّع منفصلاً عن الأنبوب	معاملات خارج كائن التسليم، وتسريب نطاق يسهل أن يقع	كل محوّل خطوة داخل `Pipeline` واحدة تُحمَّل دفعة واحدة
تسليم نموذج بعتبة ضمنية	عتبة النصف لم تُكتب في المستند، فتتغيّر بتغيّر الإصدار	احفظ رقم العتبة في ملف التسليم إلى جانب النموذج
قراءة `FrozenEstimator` كخطأ	واجهة `fit` موجودة لكنها بلا أثر، وقد تُبلَّغ كخلل	توثيق صريح: نموذج مقفول بالتصميم

١٠.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
فرع المعالجة	فرع داخل `ColumnTransformer` يخدم نوعاً من الأعمدة	Transformer branch
محدّد الأعمدة	دالة تختار الأعمدة بالنوع لا بالاسم	Column selector
عمود مشتقّ	خاصية جديدة تُحسب من عمود تاريخ	Derived feature
طيّة تدريب	تركيبة بارامترات تُدرَّب مرة على كل طيّة تحقق	Fold fit
جدول التدقيق	جدول مرتّب من `cv_results_` يشرح لماذا فازت تركيبة	Search audit table
إعادة التدريب بالمقياس المختار	تدريب الفائز على كامل التدريب بالمقياس الذي حُدّد	Refit
عتبة مضبوطة	حدّ قرار مُحسَّن على مقياس عمل لا على الدقة	Tuned threshold
تجميد	غلاف يمنع `fit` من تغيير نموذج مدرَّب	Frozen estimator
مستند التسليم	ملف يحمل النموذج وضبطاته ووثائقه	Model artifact
هدف مائل	تصنيف تكون فيه الفئة الإيجابية نادرة	Skewed target

الجزء ١١

المفاهيم المتقدّمة

واجهات ١.٩، وتوجيه البيانات، وكتابة مُقدِّر خاص بك

ماذا يغطّي هذا الجزء

هذه القوة لمن وصل إلى الاحتراف: نظام الاستدعاءات الجديد Callbacks في ١.٩، وتوجيه البيانات الوصفية Metadata Routing، ودعم واجهة Array API للانتقال إلى معالجات الرسوميات، وإطار narwhals الحاصل على البيانات. نختم بكتابة مُقدِّر متوافق من الصفر، وبأداء التنفيذ المتوازي.

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٠ — واجهة الاستدعاءات الجديدة

ProgressBar و ScoringMonitor و with_callbacks واستدعاءات مخصّصة

الوحدة **scikit-learn ١.٩** الحجم **٣٩١** كلمة تقريباً

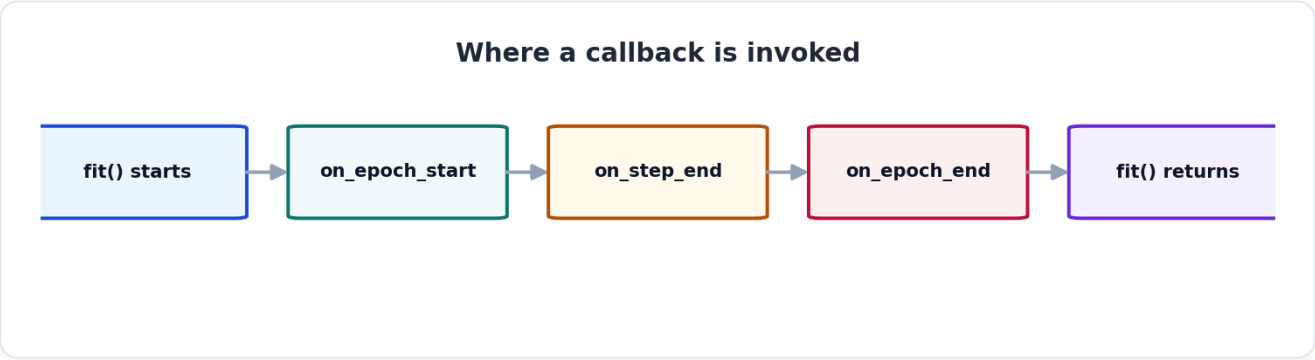
ما الذي ستتقنه في هذا الفصل

- أن يشرح دورة الاستدعاء وحدود دعمها بين المقدّرات
- أن يفعّل ProgressBar، ScoringMonitor على مقدّر مدعوم
- أن يقرأ سجلات ScoringMonitor من دون افتراض شكل غير موثق
- أن يختار بين المراقبة أثناء التدريب والتحقق المتقاطع

١١.١ ما الذي لا يخبرك به fit؟

غالباً ما يكون `fit` استدعاءً واحداً، ثم تنتظر. في تدريب تكراري طويل قد تريد أن تعرف هل تتقدم الخسارة، أو متى ينتهي التدريب، أو كيف يتغير مقياس على بيانات مراقبة. أضيفت واجهة الاستدعاءات (callbacks) في scikit-learn ١.٩ لتوفير نقطة تمديد منظمة لبعض الخوارزميات؛ ليست كل الخوارزميات تكرارية، ولا يدعمها كل مقدّر.

تبدأ القاعدة من التحقق الفعلي: افحص هل يملك المقدّر `set_callbacks`، واقرأ توثيقه وإصدار المكتبة. لا تخلط هذه الواجهة مع `verbose` العام، أو `warm_start`، أو مقسّم التحقق المتقاطع. لكل واحدة وظيفة مختلفة.



الشكل ١٩٧ دورة حياة الاستدعاء داخل التدريب

التحقق من واجهة الاستدعاءات المتاحة python

```python
from sklearn.callback import ProgressBar, ScoringMonitor
from sklearn.linear_model import LogisticRegression, LinearRegression

for estimator in (LogisticRegression(), LinearRegression()):
    print(type(estimator).__name__,
          "set_callbacks:", hasattr(estimator, "set_callbacks"))
print("كائنات callback:",
      type(ProgressBar()).__name__,
      type(ScoringMonitor(scoring="accuracy")).__name__)
```

وجود الواجهة على الصنف شرط أول، لكنه لا يضمن أن كل محلّل أو كل مسار تنفيذ يستخدمها. راجع إصدار التوثيق الخاص بالمقدّر، وحافظ على التجربة صغيرة قبل توصيلها إلى تدريب مكلف.

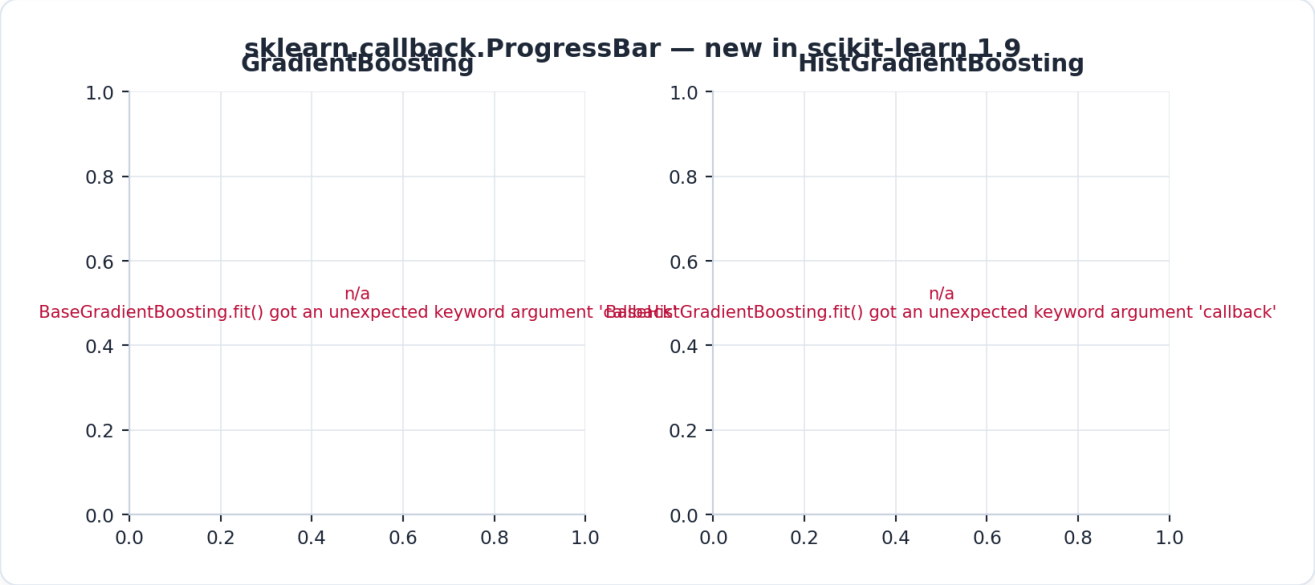
١١.٢ شريط التقدم لا يحسن النموذج

يقدم `sklearn.callback.ProgressBar` مؤشراً مرئياً عندما يدعم المقدّر أحداث التقدم. فائدته تشغيلية: تعرف أن التدريب يتحرك، لا أن النموذج يتعلم جيداً. قد يختلف شكل العرض بين الطرفية ودفتر تفاعلي، لذلك لا تربط منطق الإنتاج بتحليل النص المطبوع.

ProgressBar مع LogisticRegression python

```python
from sklearn.callback import ProgressBar
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=1000, n_features=20,
                           n_informative=8, random_state=42)
X = StandardScaler().fit_transform(X)
model = LogisticRegression(max_iter=1000, solver="lbfgs")
model.set_callbacks(ProgressBar())
model.fit(X, y)
print("عدد التكرارات المسجل:", model.n_iter_.tolist())
```

قد ينتهي محلّل `lbfgs` قبل الحد الأقصى، وقد يتوقف عنده إذا لم يتقارب؛ افحص `n_iter_` ورسائل التحذير و`n_features_in_`، لا شريط التقدم وحده. في بناء غير تفاعلي يمكن حذف العرض المرئي والإبقاء على قياس صريح بزمن التشغيل.



الشكل ١٩٨مراقبة التقدم وتسجيل المقاييس أثناء التدريب

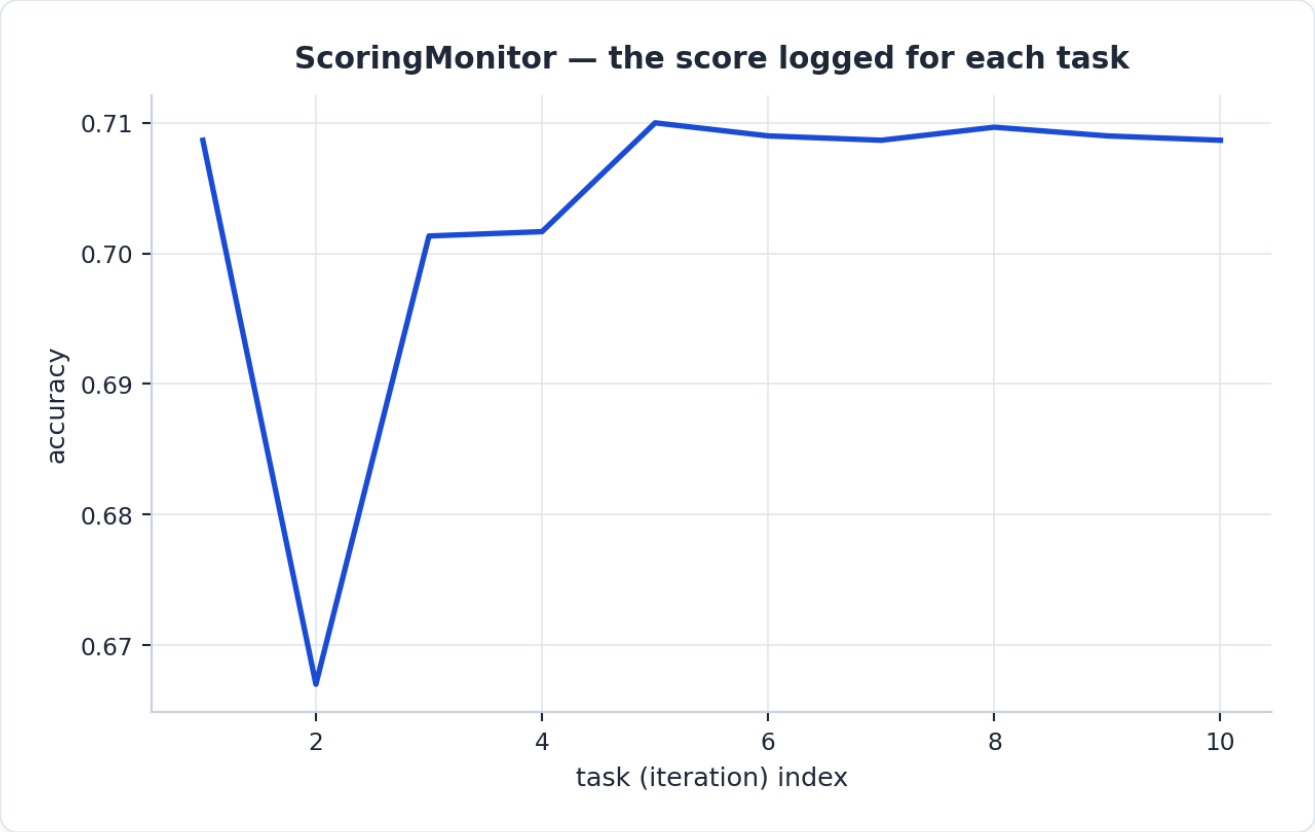
١١.٣ ScoringMonitor: سجلّ لا حكم نهائي

`sklearn.callback.ScoringMonitor` يسجل مقياساً أثناء أحداث التدريب التي يرسلها المقدّر. مرّر اسم مقياس معروف في `scoring`، وأرفق المراقب عبر `set_callbacks`، ثم اقرأ `get_logs()` بعد انتهاء التدريب. صيغة السجل جزء من واجهة هذه النسخة؛ اطبع نوعه وأسماء أعمدته في بيئتك بدلاً من افتراض أعمدة أو أسماء مهام ثابتة.

قراءة سجلات ScoringMonitor — python

```python
from sklearn.callback import ScoringMonitor
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=600, n_features=16,
                           n_informative=6, random_state=42)
X = StandardScaler().fit_transform(X)
monitor = ScoringMonitor(scoring="accuracy")
model = LogisticRegression(max_iter=300, solver="lbfgs")
model.set_callbacks(monitor)
model.fit(X, y)
logs = monitor.get_logs()
print("نوع السجل:", type(logs).__name__)
print("عدد الصفوف:", logs.data_as_pandas.shape[0])
print("الأعمدة:", logs.data_as_pandas.columns.tolist())
```

يسجل المراقب الأداء وفق الأحداث التي يرسلها المقدّر، ولا يحل محل مجموعة اختبار مستقلة. إذا كنت تختار نموذجاً أو توقف التدريب بناءً على بيانات المراقبة، فهذه البيانات أصبحت جزءاً من القرار؛ احتفظ بمجموعة نهائية لم تستخدمها في الاختيار.



الشكل ١٩٩تسجيل المقياس مع كل مهمة تدريب

١١.٤ متى تستخدم callback ومتى تستخدم cross_validate؟

المراقبة داخل التدريب تصف تطور المسار في تشغيل معيّن. أما `cross_validate` فيقارن الأداء على طيات حُجبت عن الملاءمة. قد تحتاج الاثنين، لكن لا تخلط بين تسجيل كل تكرار وبين تقدير تعميم النموذج. وميزة التحقق المتقاطع ليست أن لديه شريط تقدم؛ ميزته أنه ينظم تكرار التدريب والقياس وفق مقسّم مضبوط.

أما `sklearn.callback.with_callbacks` فهي أداة تزيين (decorator) لمطوّري المقدّرات الذين يضيفون دعماً لدورة حياة الاستدعاءات إلى طريقة في صنفهم. تدير سياق الاستدعاء وتضمن تشغيل `teardown` حتى عند وقوع خطأ. ليست غلافاً عاماً يضعه مستخدم المكتبة حول أي استدعاء `fit`؛ استخدم `set_callbacks` على مقدّر يعلن الدعم، واتبع واجهة `FitCallback` الموثقة عند كتابة امتداد خاص.

```python
# مراقبة تشغيل واحد ثم تقييم مستقل
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=1000, solver="lbfgs"))
scores = cross_validate(model, X, y, cv=5, scoring="balanced_accuracy")
print("درجات التحقق:", scores["test_score"].round(3).tolist())
```

نستخدم `cross_validate` هنا لقياس التعميم، ولا نزعم أن الأنبوب يمرر callback إلى خطواته. عند الحاجة إلى مراقبة أثناء التدريب، اختبر مباشرة مقدّراً يدعم `set_callbacks`، ثم قيّم التعميم على بيانات مستقلة.

خطأ شائع

لا تمرر callback إلى مقدّر لمجرد أن مقدّراً آخر يقبلها. غياب `set_callbacks` أو تجاهل الأحداث لا يعني أن التدريب يراقب نفسه. تحقّق من دعم الصنف وإعداداته، ولا تكتب نتيجة «مراقبة» إذا لم تتأكد أن السجل امتلأ.

في الواقع

في تدريب تكراري على خادم، يفضّل تسجيل نقاط تقدم موجزة قابلة للربط بمعرّف التجربة على إبقاء واجهة طرفية مفتوحة. افصل القياسات التشغيلية عن مقاييس اختيار النموذج، واحفظ نسخة الإعدادات والبيانات وبذرة العشوائية مع السجل.

تمارين الفصل التسعين

- **ملاحظة**

التمرين ١١.١

افحص `LogisticRegression` و`DecisionTreeClassifier` بحثاً عن `set_callbacks`. اشرح لماذا يجب التحقق لكل صنف.

التمرين ١١.٢

درّب انحداراً لوجستياً مع `ProgressBar` واطبع `n_iter_`. قارن عدد التكرارات مع إعداد أصغر لـ `max_iter`.

التمرين ١١.٣

أنشئ `ScoringMonitor` بمقياس `accuracy`، وسجّل نوع الناتج وأسماء أعمدته، ثم قارن ذلك بدرجات `cross_validate`.

- **ملاحظة**

```python
# دعم الاستدعاءات خاصية على مستوى الصنف
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

for estimator in (LogisticRegression(), DecisionTreeClassifier()):
    print(type(estimator).__name__, hasattr(estimator, "set_callbacks"))
```

الدعم يختلف بحسب المقدّر، ويجب أيضاً مراجعة المحلّل والنسخة.

> **ملاحظة**

عدد تكرارات LogisticRegression python

```python
from sklearn.callback import ProgressBar
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=700, n_features=16,
                           n_informative=6, random_state=42)
X = StandardScaler().fit_transform(X)
for limit in (10, 500):
    clf = LogisticRegression(max_iter=limit, solver="lbfgs")
    clf.set_callbacks(ProgressBar())
    clf.fit(X, y)
    print("max_iter:", limit, "n_iter_:", clf.n_iter_.tolist())
```

افحص التحذير إلى جانب العدد؛ بلوغ الحد الأعلى قد يشير إلى عدم التقارب.

> **ملاحظة**

١١.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
افتراض أن كل مقدّر يدعم callbacks	يفشل الإعداد أو لا تصدر أحداث	افحص `set_callbacks` ووثّق الدعم
اعتبار شريط التقدم دليلاً على التقارب	قد ينتهي الحد الأقصى مع تحذير	افحص `n_iter_` ورسائل التقارب
اعتبار سجل التدريب اختباراً	يختار المراقب النموذج على بيانات رآها	احتفظ بقياس خارج التدريب
افتراض أعمدة سجل ثابتة	يتغير السجل بين الإصدارات أو المسارات	افحص `get_logs()` في النسخة المثبتة

١١.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
استدعاء	كائن يتلقى حدثاً أثناء التدريب	callback
مراقبة التقدم	عرض حالة التنفيذ أثناء التدريب	progress monitoring
سجل المقياس	قيم مسجلة للمقياس عبر الأحداث	scoring log
التوقف المبكر	إيقاف التدريب وفق شرط مراقب	early stopping

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩١ — توجيه البيانات الوصفية

Metadata Routing: تمرير sample_weight و groups عبر Pipeline و GridSearchCV

الوحدة scikit-learn ١.٩ الحجم ١٢٨١ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يقرأ رسالة الخطأ قبل التوجيه وبعده، ويفصل بينهما بدقّة
- أن يصرّح بطلبات `set_fit_request` و`set_score_request` على الخطوات الصحيحة
- أن يطبع شجرة التوجيه عبر `get_metadata_routing` قبل تشغيل بحث شبكي
- أن يقيس بالأرقام أثر الأوزان المتوازنة على الاسترجاع والدقة الموجّهة
- أن يشرح لماذا تصير نتائج `cross_validate` كلها `nan` بسبب طلب واحد ناقص

١١.١ المشكلة: من الذي يحتاج إلى وزن الصف؟

تخيّل أن لديك جدولاً فيه عمود `revenue` بقيم كبيرة وعمود `churned` يأخذ ٠ أو ١. وضعت `StandardScaler` قبل المصنّف لمعالجة اختلاف المقاييس، ولديك أيضاً وزن لكل صف يعبّر عن أهميته في التدريب. ثم تكتشف أن الفئة الإيجابية نادرة، فتستخدم أوزاناً متوازنة لزيادة أثرها في دالة الخسارة. من الذي يجب أن يستقبل كل وزن: المحوّل، أم المصنّف، أم المقيّم؟

توقّف عند هذه العبارة: **قد يحتاج المحوّل إلى الأوزان لحساب إحصاءاته، وقد يحتاجها المصنّف لتدريب النموذج، وقد يحتاجها المقيّم لحساب درجة موزونة؛ ولكل استخدام طلب مستقل.**

ثلاثة مستهلكين، وثلاثة معانٍ؛ ولا ينبغي للمكتبة أن تخمّن نية المستخدم. ثم أضف `groups`: معرّفات تنتمي إلى مقسّم الطيّات عند فصل بيانات التدريب والتحقق، لا إلى `fit` في الخطوات المعتادة.

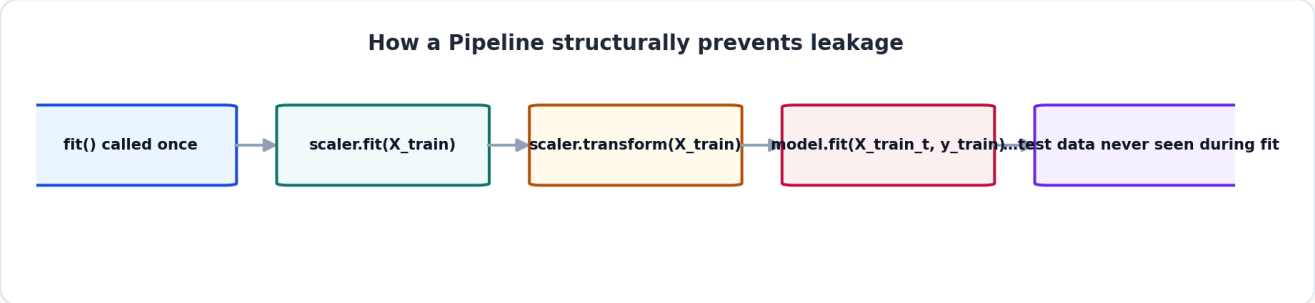
وهنا السؤال الذي لا يجيب عنه scikit-learn من تلقاءه: **كيف يعرف أن هذه البيانات الوصفية تخصّ هذه الخطوة؟**

الجواب: لا ينبغي للمكتبة أن تخمّن. يمكن تمرير البيانات مباشرةً إلى خطوة محددة بصيغة (`step__sample_weight=`)، لكن ذلك يصبح مزعجاً مع الأنابيب والبحث الشبكي. منذ scikit-learn ١.٣ بدأ نظام **توجيه البيانات الوصفية** (metadata)

routing) بالظهور، وأُضيفت له واجهات تدريجياً؛ يصرّح كل مُقدِّر بما يطلبه، وتتولى الكائنات المركّبة توجيه البيانات إلى أصحابها.

```python
# ما الذي نحاول فعله
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.utils.class_weight import compute_sample_weight

w = compute_sample_weight("balanced", y_train)

pipe = Pipeline([("sc", StandardScaler()), ("clf", SVC())])
pipe.fit(X_train, y_train, sample_weight=w)          # من يستلمها؟ المتّبع؟ المصنّف؟
search.fit(X_train, y_train, sample_weight=w)        # ومن يستلمها في المقاييس؟
```

هذا ليس تفصيلاً شكلياً. يوضح الفصل ٨٤ كيف يساعد `Pipeline` على منع تسرب بيانات الاختبار إلى خطوات المعالجة. أما هذا الفصل فيعالج سؤالاً مختلفاً ومكمّلاً: كيف تصل البيانات الوصفية إلى المكوّن الذي طلبها؟



الشكل ٢٠٠يطبّق Pipeline خطوات المعالجة على بيانات التدريب وحدها داخل كل طية، فيحدّ من تسرب البيانات

هذا الشكل يذكّرنا بأن خط الأنابيب يعالج خطر تسرب البيانات، بينما يضبط التوجيه وجهة البيانات الوصفية؛ لا يغني أحدهما عن الآخر.

١١.٢ المحاولة الأولى: التوجيه مطفي

لنبدأ من الصفر، قبل أن نقرأ أي سطر توثيق. التوجيه معطّل افتراضياً في الإعداد العام؛ وهذا يحافظ على سلوك الأنابيب الموجودة ما لم يفعّله المستخدم. قبل تشغيله، دعنا نرى ماذا يحدث عند تمرير الوزن إلى خط أنابيب لا يملك طلباً معلناً:

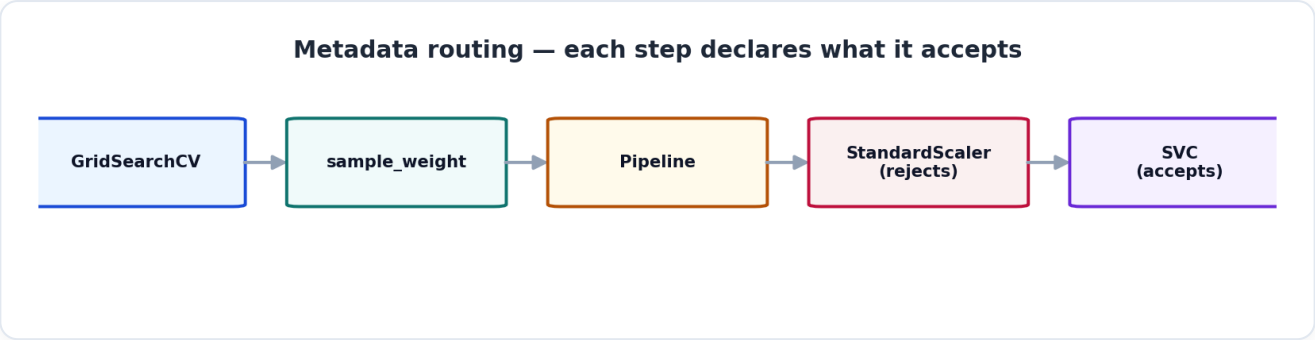
```python
# مفتاح التوجيه مطفي: الخط الأول من المشوار
import numpy as np
import sklearn
from sklearn.datasets import make_classification
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.utils.class_weight import compute_sample_weight

sklearn.set_config(enable_metadata_routing=False)
print("التوجيه مطفأ افتراضياً:", sklearn.get_config()["enable_metadata_routing"])

X, y = make_classification(n_samples=800, n_features=20, n_informative=8,
                           weights=[0.9, 0.1], flip_y=0.02, random_state=42)
w = compute_sample_weight("balanced", y)
print("أوزان الصفوف:", np.unique(np.round(w, 4)))

pipe = Pipeline([("sc", StandardScaler()), ("clf", SVC())])
grid = GridSearchCV(pipe, {"clf__C": [0.5, 1.0]}, cv=3, error_score="raise")
try:
    grid.fit(X, y, sample_weight=w)
except Exception as exc:
    print(f"{type(exc).__name__}: {exc}")
```

هذه هي الرسالة كاملة، وهي ثمرة عمل تصميمي متعمّد:

```
parameter format, e.g. Pipeline.fit(X, y, logisticregression__sample_weight=sample_weight).
```

لاحظ نوع الخطأ: `ValueError` من داخل `Pipeline.fit`؛ فهو يوضح أن تمرير البيانات بهذه الطريقة غير مدعوم في الوضع الحالي. لم تعلن أي خطوة بعد أنها تطلب هذا الوزن.

ولنلاحظ أن `error_score="raise"` في `GridSearchCV` هو ما جعل الخطأ يظهر مباشرةً بدلاً من تحويل فشل الطيات إلى درجات غير صالحة. اجعله خياراً أثناء التطوير كي ترى سبب الفشل قرب موضعه، ثم قرر سياسة التعامل مع الأخطاء في التطبيق النهائي.



الشكل ٢٠١سلسلة التوجيه: كل خطوة تعلن ما تقبله، فيتوزّع البحث على أصحابه

الشكل أعلاه يلخّص مسار البيانات: يحمل `GridSearchCV` قيمة `sample_weight`، ويوجّهها خط الأنابيب إلى الخطوات التي أعلنت طلبها. إن لم تُعلن خطوة موقفها، تظهر رسالة توضّح أن التوجيه غير محسوم.

١١.٣ ثاني خطوة: شغّل المفتاح، وستحصل على خطأ أفضل

الآن نضيف مفتاحاً واحداً. لا نغيّر سطراً واحداً في الكود، ولا نغيّر أسماء الخطوات، ولا نفهم شيئاً عمّا نكتب. نضيف:

```python
المفتاح العام: نفس الكود، رسالة مختلفة تماماً
sklearn.set_config(enable_metadata_routing=True)
print("بعد التشغيل:", sklearn.get_config()["enable_metadata_routing"])

pipe2 = Pipeline([("sc", StandardScaler()), ("clf", SVC())])
grid2 = GridSearchCV(pipe2, {"clf__C": [0.5, 1.0]}, cv=3, error_score="raise")
try:
    grid2.fit(X, y, sample_weight=w)
except Exception as exc:
    print(f"{type(exc).__name__}: {exc}")
```

الخطأ لم يعد `ValueError` عاماً. صار `UnsetMetadataPassedError` يسمّي الخطوة التي لم يُحسم طلبها:

```
).set_transform_request({metadata}=True/False) for each metadata you want to request/ignore.
```

هذه قفزة نوعية لا في الشكل بل في الجوهر. الخطأ الأول يقول: «لا أعرف ماذا تريد». الخطأ الثاني يقول: «أعرف ما تريد، وأعرف أيّ جزء منك لا يريده، وهذه هي الخطوة بالاسم».

وهذه قفزة هندسية أيضاً. يقبل `StandardScaler.fit` الوسيط `sample_weight`، لكن قبول الوسيط في توقيع الدالة لا يعني أن المُقدِّر سيستقبله تلقائياً عبر التوجيه؛ يجب إعلان الطلب صراحةً:

```python
# الموقّع ليس Declaration
from sklearn.preprocessing import StandardScaler
import inspect

print(inspect.signature(StandardScaler.fit))
# (self, X, y=None, sample_weight=None)
```

الموقّع يقول ما يستطيع قبوله؛ و`set_fit_request` يقول ما **يريد** قبوله. الاثنان ليسا الشيء نفسه، والفصل بينهما هو جوهر النظام كلّه.

١١.٤ من يطلب: `set_fit_request`

الآن نصيحة الخطوة. لكل خطوة أن تقول ما تريد، وتقولها بمفتاح صريح:

```python
# الطلب الصريح: يوجّه الوزن إلى الخطوات المطلوبة
pipe2.named_steps["sc"].set_fit_request(sample_weight=True)
pipe2.named_steps["clf"].set_fit_request(sample_weight=True).set_score_request(
    sample_weight=True)
grid2.fit(X, y, sample_weight=w)
print("نجح البحث:", grid2.best_params_, "| الدرجة:", round(grid2.best_score_, 4))
```

نجح. والأهمّ أنه نجح **دون تغيير سطر واحد من الكود الأصلي**، ومع توصيل موزون إلى كل من يحتاجه: المطبِّع في `fit`، والمصنّف في `fit`، والمصنّف في `score`.

لاحظ أن `set_score_request` لم يكن ترفاً. بدونه يرث `GridSearchCV` وزن الصفوف إلى خطوة القياس، و`Pipeline.score` يحمل `sample_weight` في توقيعه، فيرفضه `SVC.score` لأنه لم يطلبه. سنعود إلى هذا الفصل ١١.٧ بتفصيله، لأنه هو أكثر أخطاء التوجيه شيوعاً في البحث الشبكي.

القواعد العملية:

١١.٥ اقرأ شجرة التوجيه قبل أن تشغّلها

أكبر تحدٍ في هذا النظام أن الطلبات مخزّنة داخل كائنات فرعية؛ وقد يصعب تتبعها في أنبوب متشعب. الحلّ بسيط: اطبع شجرة التوجيه قبل تشغيل البحث.

تعيد `Pipeline.get_metadata_routing()` كائن `MetadataRouter`، بينما تعيد `get_routing_for_object` طلبات البيانات الوصفية لكائن بعينه. اجعل فحص الشجرة جزءاً من التطوير والتشخيص:

```python
شجرة التوجيه: من يطلب ماذا
from sklearn.utils.metadata_routing import get_routing_for_object

METHODS = ("fit", "score", "split", "transform")
for name, step in pipe2.named_steps.items():
    req = get_routing_for_object(step)
    cells = " | ".join(
        f"{m}={getattr(req, m).requests or '-'}"
        for m in METHODS if hasattr(req, m))
    print(f"{name:<4} {type(req).__name__:<15} {cells}")
print("جسم الأنبوب:", type(pipe2.get_metadata_routing()).__name__)
```

القراءة سهلة: `sc.fit` و`clf.fit` يطلبان `sample_weight`، وكذلك `clf.score`، بينما لا توجد طريقتا `sc.score` و`sc.split` في هذا المثال. علامة `-` تعني أن الطريقة غير موجودة أو لا تطلب بيانات وصفية هنا. أما `sc.transform` فقد يعرض وسيطاً مثل `copy` غير محسوم؛ وتعني `None` أن الطلب لم يُحدّد بعد.

وهذه القيم الثلاث التي ستراها طوال عملك:

القيمة	معناها	متى تستعملها
`True`	أطلب هذه البيانات صراحةً	المطبّع والمصنّف والمقيّم
`False`	أعرفها وأرفضها صراحةً	خطوة لا يجب أن ترى الوزن أبداً
`None`	لم أحسم بعد، وسأنسى	لا شيء؛ الحسم مطلوب قبل التشغيل

`False` ليست كسلاً بل حماية. الخطوة التي ترفض الوزن بنبرة صريحة لا يمكن أن يمرّر إليه أحد وزناً بالخطأ في مراجعة لاحقة.

١١.٦ حالة كاملة: فئة نادرة جدّاً

الآن الكلام كله يجتمع في مسطرة. نأخذ ٨٠٠ صف، ٧٪ منها إيجابية، ونقسم ٧٥/٢٥، ثم ندرّب نفس الأنبوب مرّتين: مرّة بلا أوزان، ومرّة بأوزان `balanced` التي ترفع وزن الصف النادر إلى ٤٫٥٤٥٥ مقابل ٠٫٥٦١٨ للسلبي.

```python
الأوزان المتوازنة: ما الذي يتغيّر فعلاً
from sklearn.metrics import average_precision_score, precision_score,
recall_score
from sklearn.model_selection import train_test_split

Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.25, random_state=42,
                                  stratify=y)
wa = compute_sample_weight("balanced", ya)
print("موجود في التدريب:", np.bincount(ya), "| في الاختبار:", np.bincount(yb))
print("قيمة الوزن للفئة النادرة:", round(wa[ya == 1][0], 4))


def fresh():
    return Pipeline([("sc", StandardScaler()), ("clf", SVC())])


plain = fresh().fit(Xa, ya)
p_plain = plain.predict(Xb)
weighted = fresh()
weighted.named_steps["sc"].set_fit_request(sample_weight=True)
weighted.named_steps["clf"].set_fit_request(sample_weight=True)
weighted.fit(Xa, ya, sample_weight=wa)
p_wt = weighted.predict(Xb)

for tag, mdl, pred in (("بلا أوزان", plain, p_plain),
                       ("بالأوزان", weighted, p_wt)):
    tp = int(((pred == 1) & (yb == 1)).sum())
    fp = int(((pred == 1) & (yb == 0)).sum())
    fn = int(((pred == 0) & (yb == 1)).sum())
    print(f"{tag:<10} دقة={mdl.score(Xb, yb):.4f} "
          f"استرجاع={recall_score(yb, pred, pos_label=1):.4f} "
          f"موجّهة={precision_score(yb, pred, pos_label=1):.4f} "
          f"tp={tp} fp={fp} fn={fn}")
print("الدقة الموجّهة بلا أوزان:",
      round(average_precision_score(yb, plain.decision_function(Xb)), 4))
print("الدقة الموجّهة بالأوزان :",
      round(average_precision_score(yb, weighted.decision_function(Xb)), 4))
```

هذه أهم أرقام الفصل، وهي تُقرأ كصفقة لا كنتيجة:

ما يقيسه	بلا أوزان	بالأوزان	الفرق
الدقة العامة	٠٫٩٤٠٠	٠٫٩٢٥٠	-١٫٥ نقطة
الاسترجاع للفئة النادرة	٠٫٤٥٤٥	٠٫٦٣٦٤	+١٨٫٢ نقطة
الدقة الموجّهة	١٫٠٠٠٠	٠٫٦٦٦٧	-٣٣٫٣ نقطة
صحيح موجّب / خاطئ موجّب	١٠ / ٠	١٤ / ٧	أربعة إضافية مقابل سبعة إنذارات
متوسط الدقة الموجّهة	٠٫٧٩٤١	٠٫٧٧٢٢	-٢٫١ نقطة

اختلف النموذجان في ثلاثة عشر تنبؤاً من مئة. بلا أوزان اكتشف النموذج عشرة من الحالات الإيجابية من دون إنذار خاطئ في هذه العينة. وبالأوزان اكتشف أربع حالات إيجابية إضافية، مقابل سبعة إنذارات كاذبة. هذا مثال على المقايضة بين زيادة الاسترجاع وتكلفة مراجعة الحالات الإيجابية الكاذبة.

ولنا أيضاً أن نقرأ ما لا يقوله الجدول: متوسط الدقة الموجّهة انخفض قليلاً (٠٫٧٩٤١ إلى ٠٫٧٧٢٢). أي أن النموذج الموزون لم يحسّن ترتيب الحالات في هذا المثال، لكنه صنّف حالات أكثر على أنها إيجابية. يبقى السؤال التطبيقي: هل تكلفة الإنذارات الكاذبة أقل من تكلفة الحالات الفائتة؟ لا يوجد إعداد «صحيح» واحد لكل التطبيقات.

> 🔑 **ما الذي تعلّمناه عن التوجيه**
>
> التوجيه لا يحلّ مشكلة **اختيار** الأوزان؛ يحلّ مشكلة **نقلها**. أي أن الفصل يقول لك: «هكذا تصل الأوزان إلى أصحابها»، ولا يقول «هكذا تختار الأوزان». الاختيار قرار بيانات؛ النقل قرار هندسة.

١١.٧ البيانات الوصفية لا تصنعها `fit` وحدها: المقيّم

يمرّر `GridSearchCV` البيانات الوصفية إلى المُقدِّر وإلى دالة القياس وفق الطلبات المعلنة. إذا استخدمتَ مقيّماً مخصصاً يحتاج إلى أوزان، فعليك إعلان طلبه أيضاً:

```python
# المقيّم طرف ثالث له طلب خاص به
from sklearn.metrics import make_scorer

GRID = {"clf__C": [0.5, 1.0, 4.0], "clf__gamma": ["scale", 0.1]}


def fresh2():
    p = Pipeline([("sc", StandardScaler()), ("clf", SVC())])
    p.named_steps["sc"].set_fit_request(sample_weight=True)
    p.named_steps["clf"].set_fit_request(sample_weight=True)
    return p


sc = make_scorer(recall_score, pos_label=1)
try:
    GridSearchCV(fresh2(), GRID, cv=3, scoring=sc, error_score="raise").fit(
        Xa, ya, sample_weight=wa)
except Exception as exc:
    print(f"{type(exc).__name__}: {exc}")

sc2 = make_scorer(recall_score, pos_label=1)
sc2.set_score_request(sample_weight=True)
g_w = GridSearchCV(fresh2(), GRID, cv=3, scoring=sc2, error_score="raise")
g_w.fit(Xa, ya, sample_weight=wa)
g_p = GridSearchCV(fresh2(), GRID, cv=3, scoring=sc2, error_score="raise")
g_p.fit(Xa, ya)
print("مع أوزان  :", g_w.best_params_, round(g_w.best_score_, 4))
print("بلا أوزان :", g_p.best_params_, round(g_p.best_score_, 4))
```

توضح الرسالة أن المقيّم كائن مستقل عن المُقدِّر، وله طلب بيانات وصفية خاص به. عند بناء مقيّم مخصص باستخدام `make_scorer`، أعلن حاجته إلى `sample_weight` صراحةً كما في المثال. أما أسماء المقاييس النصية، فتحقق من سلوك المقياس المستخدم قبل افتراض أنه يتلقى الأوزان.

```python
# كل نصّ قصير في scoring هو make_scorer في ثوب
sc = make_scorer(recall_score, pos_label=1)
sc.set_score_request(sample_weight=True)   # نفس الكائن، لا كائن آخر
print(type(sc).__name__)
```

والفرق في الرقمين الأخيرين ليس فرقاً تجميلياً: `C=4.0` بلا أوزان و`C=1.0` بالأوزان. الشبكة التي تختار `C` الكبير تختار انحيازاً للسلبية؛ والشبكة التي تختار `C` الأصغر تختار انحيازاً للندرة. أنت لا تغيّرت الشبكة — أنت تغيّرت **السؤال** الذي تجيب عنه.

البيانات الوصفية	من يقبلها	كيف تعلن الطلب
`sample_weight`	المصنّفات والمصنّفات الخطية و`StandardScaler`	`set_fit_request(sample_weight=True)`
`sample_weight` في القياس	كل دالة `make_scorer` وكل نصّ `scoring`	`set_score_request(sample_weight=True)`
`groups`	`GroupKFold` و`StratifiedGroupKFold` و`LeaveOneGroupOut`	`set_split_request(groups=True)` مبدئياً
`groups` في التدريب	مُقدِّر يعلنه — قليل جدّاً	`set_fit_request(groups=True)`
`eval_set`	`XGBClassifier` وأمثاله من خارج المكتبة	صنف من خارج scikit-learn
`y` نفسه	الجميع، دائماً، بلا توجيه	ليس بيانات وصفية أصلاً

١١.٨ `groups`: المستهلك الثاني، وحدود النظام

`sample_weight` قصة واحدة فقط. `groups` قصة مختلفة تماماً: هي **لا تنتمي إلى** `fit`. تنتمي إلى `split` — إلى الطريقة التي تُقسم بها الطيّات. ولهذا تُقطَع من المسار قبل أن تصل إلى أي خطوة، ولهذا يبدو الأمر سخيفاً إذا ظننت أن `groups` شيء مثل `sample_weight`.

هنا يظهر نوع آخر من التوجيه: تُمرر `groups` إلى المقسّم لا إلى خطوات المعالجة أو النموذج. والفرق العملي واضح:

```python
# المجموعات تصل إلى المقسّم، والمقسّم يعرف ذلك
rng = np.random.default_rng(42)
groups = np.repeat(np.arange(60), rng.integers(8, 26, size=60))
Xg, yg = make_classification(n_samples=len(groups), n_features=20,
                             n_informative=8, random_state=42)
from sklearn.model_selection import GroupKFold, KFold

gkf = GroupKFold(n_splits=5)
folds = [set(g) for _, g in gkf.split(Xg, yg, groups)]
print("أحجام الطيّات:", [len(f) for f in folds])
print("تقاطع الطيّة ٠ مع الأولى:", len(folds[0] & folds[1]))

from sklearn.linear_model import LogisticRegression

lin = Pipeline([("sc", StandardScaler()),
                ("clf", LogisticRegression(max_iter=5000))])
gk = GridSearchCV(lin, {"clf__C": [0.5, 2.0]}, cv=GroupKFold(5),
                  error_score="raise")
gk.fit(Xg, yg, groups=groups)
print("وصلت المجموعات:", gk.best_params_, round(gk.best_score_, 4))
print("طلب التقسيم:", get_routing_for_object(GroupKFold(5)).split.requests)

try:
    GridSearchCV(lin, {"clf__C": [0.5, 2.0]}, cv=KFold(5),
                 error_score="raise").fit(Xg, yg, groups=groups)
except Exception as exc:
    print(f"{type(exc).__name__}: {exc}")
```

تتوزع العينات على الطيات الخمس بأحجام متقاربة، ولا تتداخل المجموعات بين طيتي التدريب والاختبار. هذا هو الضمان الأساسي في التحقق بالمجموعات: لا يظهر الكيان نفسه في جانبي الطية، وهو ما يجعل التقييم أقرب إلى سيناريو التعميم على مجموعات جديدة.

ثم `GroupKFold` يُعلن `{'groups': True}` تلقائياً — لأنه يعرف أنه لا يمكنه العمل من دونها. جرّب الآن نفس البحث مع `KFold(5)`، وهو مقسّم لا يعرف شيئاً عن المجموعات ولا يملك `set_split_request` أصلاً، فتحصل على رسالة مختلفة تماماً:

```
GridSearchCV.fit got unexpected argument(s) {'groups'}, which are not routed to any object.
```

وهذه الرسالة — وليست رسالة `sample_weight` — هي التي تميّز المفهومين. الأولى تعني أن البيانات مرّرت من دون طلب محسوم؛ والثانية تعني أن لا كائن في مسار التوجيه يعلن استقباله لها.

حدود لا يتجاوزها التوجيه

نقطة أخيرة قبل أن نغلق، لأنها تحمي من توقّعات خاطئة:

وأخطر هذه الحدود الثلاثة ليس في الرسالة، بل في غيابها أحياناً:

الفخّ: خمسة NaN صامتة — python

```python
from sklearn.model_selection import cross_validate

wg = compute_sample_weight("balanced", yg)


def lin2():
    p = Pipeline([("sc", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=5000))])
    p.named_steps["sc"].set_fit_request(sample_weight=True)
    p.named_steps["clf"].set_fit_request(sample_weight=True)
    return p


with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    quiet = cross_validate(lin2(), Xg, yg, cv=GroupKFold(5),
                           params={"sample_weight": wg, "groups": groups})
print("بلا score_request:", np.round(quiet["test_score"], 4))
print("نوع التحذير:", [w.category.__name__ for w in caught][:1])


def lin3():
    p = lin2()
    p.named_steps["clf"].set_score_request(sample_weight=True)
    return p


loud = cross_validate(lin3(), Xg, yg, cv=GroupKFold(5),
                      params={"sample_weight": wg, "groups": groups},
                      error_score="raise")
print("مع score_request:", np.round(loud["test_score"], 4))
plain_cv = cross_validate(lin2(), Xg, yg, cv=GroupKFold(5),
                          params={"groups": groups}, error_score="raise")
print("متوسط موزون:", round(loud["test_score"].mean(), 4),
      "| متوسط غير موزون:", round(plain_cv["test_score"].mean(), 4))
```

لا ترفع `cross_validate` استثناءً افتراضياً عند فشل الطية؛ فقيمة `error_score` الافتراضية هي `nan`، وقد تظهر بذلك درجات غير صالحة بدلاً من توقف التنفيذ. أثناء التطوير استخدم `error_score="raise"` لكشف الخطأ. وبعد إضافة `set_score_request` إلى الخطوة الأخيرة، نجح المثال وأنتج الدرجات `[0.7844 0.7787 0.7591 0.7981 0.7538]`.

ولنقرأ المقارنة الأخيرة قبل أن نغلق: المتوسط الموزون ٠٫٧٧٤٨ ومتوسط غير الموزون ٠٫٧٧٤٦. تطابقٌ شبه تامّ. والفكرة أن **الأوزان تغيّر التدريب، لا القياس**؛ القياس يبقى على نفس المجموعة، وهذا بالضبط ما يجب أن يحدث. لولا أن `sample_weight` كان يمرّ إلى المقيّم — أي عندما تجعله يمرّ عمداً — ما كنت لترى فرقاً في الدرجة.

تفعيل المفتاح العام داخل مكتبة

يؤثر `sklearn.set_config(enable_metadata_routing=True)` في الإعداد العام للعملية. إذا كنت تطوّر مكتبة، فلا تغيّر إعدادات المستخدمين العامة من `__init__.py`؛ استخدم `sklearn.config_context(...)` بنطاق محدود عند الحاجة، أو اطلب من التطبيق المستهلك تفعيل التوجيه. أما في تطبيق مستقل، فيمكن ضبط الإعداد صراحةً عند نقطة الدخول.

أوزان الاستطلاع لا تُحسب من `y` وحدها

لنفترض أن أوزان استطلاع تعبّر عن احتمال تمثيل كل استجابة للسكان، بينما أوزان `compute_sample_weight("balanced", y)` تعالج عدم توازن الفئات. قد يكون المتجهان متساويين في الطول، لكن معناهما مختلف. يساعد التوجيه على إيصال الوزن إلى المستهلك المقصود؛ لكنه لا يقرر ما إذا كان الوزن نفسه مناسباً للسؤال الإحصائي.

تمارين الفصل الحادي والتسعين

- **ملاحظة**

التمرين ١١.١

في عملية جديدة، شغّل `sklearn.set_config(enable_metadata_routing=True)` ثم ابنِ `Pipeline` من ثلاث خطوات: `SimpleImputer(strategy="median")` ثم `StandardScaler()` ثم `LogisticRegression(max_iter=5000)`. اطبع `get_config()["enable_metadata_routing"]`، ثم اجمع الطلبية في محوّل واحد دون جمع `sample_weight`: احذف `sample_weight` من المتغيّر بعد الطباعة. أخيراً، اطبع شجرة التوجيه بالطريقة نفسها المستعملة في الفصل، وحدّد أيّ خطوة ستصلك بالوزن وأيّها ستصلك بلا وزن.

التمرين ١١.٢

ابنِ بياناتك من `make_classification` بطول ١٠٠٠ صف، ثم اصنع بديلاً صناعياً لتوزيع الصفوف: `make_membership` لا يسمّي `random_state`، كلّ صف من ٦٠ مجموعة، بأطوال عشوائية ثابتة البذرة. استخدم `GridSearchCV` بـ `cv=GroupKFold(4)` وخطّ أنابيب من `StandardScaler()` ثم `LogisticRegression(max_iter=5000)`، ومرّر `groups=`. اطبع أحجام الطيّات، وتحقّق أن تقاطع الطيّتين الأولى والثانية صفر، ثم اطبع `best_params_`. جرّب نفس الأمر مع `StratifiedKFold(4)` واقرأ الرسالة كما هي.

التمرين ١١.٣

زميل أرسل لك هذا الكود ويقول «النتائج غريبة». حدّد سبب الخطأ، واذكر قيمة `error_score` التي تحوّل الصمت إلى رسالة، ثم اشرح لماذا لا يكفي `set_fit_request` وحده:

```
sklearn.set_config(enable_metadata_routing=True)
Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.25, random_state=42,
                                  stratify=y)
w = compute_sample_weight("balanced", ya)

pipe = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression())])
pipe.named_steps["clf"].set_fit_request(sample_weight=True)
try:
    pipe.fit(Xa, ya, sample_weight=w)
except Exception as exc:
    print(f"{type(exc).__name__}: {exc}")
```

- **ملاحظة**

```python
# شجرة التوجيه: من يستلم الوزن فعلاً
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.utils.metadata_routing import get_routing_for_object

print("المفتاح:", sklearn.get_config()["enable_metadata_routing"])

Xi, yi = make_classification(n_samples=600, n_features=20, n_informative=6,
                             random_state=42)
miss = np.random.default_rng(0).choice(600, size=90, replace=False)
Xi[miss, 4] = np.nan
Xi[miss, 11] = np.nan
Xa, Xb, ya, yb = train_test_split(Xi, yi, test_size=0.25, random_state=42,
                                  stratify=yi)

pipe3 = Pipeline([("imp", SimpleImputer(strategy="median")),
                  ("sc", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=5000))])
pipe3.named_steps["clf"].set_fit_request(sample_weight=True)
pipe3.named_steps["clf"].set_score_request(sample_weight=True)

METHODS = ("fit", "score", "transform")
for name, step in pipe3.named_steps.items():
    req = get_routing_for_object(step)
    cells = " | ".join(f"{m}={getattr(req, m).requests or '-'}"
                       for m in METHODS if hasattr(req, m))
    print(f"{name:<4} {cells}")

clf = pipe3.named_steps["clf"]
sc = StandardScaler().fit(Xa)
print("المصفوفة نفسها:", np.array_equal(sc.transform(Xa), sc.transform(Xb)))
```

`imp` و`sc` لا يطلبان الوزن، فحصل كل منهما على نفس المصفوفة تماماً — وهو ما أردناه، لأن المتوسط غير الموزون هو الذي نريد للقياس. `clf` وحده يطلبه في `fit` وفي `score`. والخطوة `imp` لها الطلبية `{'sample_weight': None}` افتراضياً، أي «لم أحسم»، لا «رفضت».

> **ملاحظة**

python التقاط المجموعات عبر GridSearchCV و GroupKFold

```python
from sklearn.model_selection import (GridSearchCV, GroupKFold,
                                     StratifiedKFold)

rng = np.random.default_rng(42)
sizes = rng.integers(8, 26, size=60)
grp = np.repeat(np.arange(60), sizes)
Xm, ym = make_classification(n_samples=len(grp), n_features=20,
                             n_informative=8, random_state=42)

folds = [set(g) for _, g in GroupKFold(4).split(Xm, ym, grp)]
print("أحجام الطيّات:", [len(f) for f in folds])
print("تداخل الطيّتين الأوليين:", len(folds[0] & folds[1]))
print("كل مجموعة في طيّة واحدة فقط:",
      all(len(f) == 60 // 4 or len(f) in (60 // 4, 60 // 4 + 1)
          for f in folds))

lin2 = Pipeline([("sc", StandardScaler()),
                 ("clf", LogisticRegression(max_iter=5000))])
g2 = GridSearchCV(lin2, {"clf__C": [0.5, 2.0]}, cv=GroupKFold(4),
                  error_score="raise")
g2.fit(Xm, ym, groups=grp)
print("أفضل تركيبة:", g2.best_params_, "| الدرجة:", round(g2.best_score_, 4))

try:
    GridSearchCV(lin2, {"clf__C": [0.5, 2.0]}, cv=StratifiedKFold(4),
                 error_score="raise").fit(Xm, ym, groups=grp)
except Exception as exc:
    print(f"{type(exc).__name__}: {exc}")
```

المجموعة الواحدة لا تُقسَّم أبداً بين طيّة وأخرى: تقاطع الطيّتين الأوليين صفر، وهذا هو الشرط الوحيد الذي يجعل درجة التحقق بمجموعات صادقة. أمّا `StratifiedKFold` فلا يملك `set_split_request` أصلاً، فيرفض المفتاح بلا أن يقترح بديلاً — وتصلك `TypeError` لا `UnsetMetadataPassedError`.

> **ملاحظة**

```python
# الخطأ ليس في fit: الخطأ في score
sklearn.set_config(enable_metadata_routing=True)
Xi2, yi2 = make_classification(n_samples=500, n_features=15,
                               weights=[0.85, 0.15], random_state=42)
Xa2, Xb2, ya2, yb2 = train_test_split(Xi2, yi2, test_size=0.25,
                                      random_state=42, stratify=yi2)
wi2 = compute_sample_weight("balanced", ya2)
w_full = compute_sample_weight("balanced", yi2)

bad = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression())])
bad.named_steps["sc"].set_fit_request(sample_weight=False)
bad.named_steps["clf"].set_fit_request(sample_weight=True)
bad.fit(Xa2, ya2, sample_weight=wi2)
print("fit نجح، درجة:", round(bad.score(Xb2, yb2), 4))

res = cross_validate(bad, Xi2, yi2, cv=3, params={"sample_weight": w_full})
print("درجات التحقق:", np.round(res["test_score"], 4))

good = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression())])
good.named_steps["sc"].set_fit_request(sample_weight=False)
good.named_steps["clf"].set_fit_request(sample_weight=True)
good.named_steps["clf"].set_score_request(sample_weight=True)
res2 = cross_validate(good, Xi2, yi2, cv=3, params={"sample_weight": w_full},
                      error_score="raise")
print("بعد التصحيح:", np.round(res2["test_score"], 4))
```

ينجح `fit` لأن طلب التدريب معلن. لكن طلب `score` غير محسوم؛ وعند تمرير `sample_weight` إلى `cross_validate` قد يفشل التقييم وتظهر `nan` لأن `error_score` الافتراضية هي `nan`. استخدم `error_score="raise"` أثناء التطوير لإظهار الخطأ، ثم أضف `set_score_request` إذا كان المقيّم يجب أن يستخدم الأوزان.

> **ملاحظة**

١١.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط
تمرير `sample_weight` والتوجيه مطفي	`ValueError: Pipeline.fit does not accept the sample_weight parameter`
تصريح `set_fit_request` فقط	`UnsetMetadataPassedError` يسمّي `SVC.score`
تمرير `groups` مع `KFold`	`TypeError: ... {'groups'}, which are not routed to any object`
`error_score="raise"` منسي في البحث	تقرير طويل يبدأ بـ `All the 6 fits failed.` يخفي السبب
`error_score` الافتراضي في `cross_validate`	خمس نتائج `nan` بلا خطأ ظاهر
خلط `None` بـ `False` في الشجرة	«لم أحسم» تمرّ كأنها «رفضت»
تفعيل المفتاح العام داخل مكتبة	`ValueError` في أنابيب العملاء عند أول `fit`
توقّع التوجيه لتمرير `y`	لا يوجد مسار له؛ `y` ليس بيانات وصفية

١١.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
توجيه البيانات الوصفية	توزيع `sample_weight` و`groups` على من يطلبها	Metadata routing
طلب البيانات الوصفية	تصريح صريح من المُقدِّر بما يقبله	Metadata request
خطوة غير محسومة	طلب قيمته `None`: لم يُحسم بعد	Unset request
طلب مَرفوض	تصريح بـ `False`: أعرف البيانات وأرفضها	Rejected request
وزن الصف	معامل يصاحب صفاً واحداً في التدريب والقياس	Sample weight
وزن متوازن	وزن يرفع المساهمة النادرة دون تغيير المجموع	Balanced class weight
مجموعة	معرّف يجمع عينات يجب إبقاؤها معاً عند تقسيم الطيات	Group
مقسّم الطيّات	كائن يقرّر تقسيم الصفوف، وهو مستهلك للبيانات الوصفية	CV splitter
خطّ التوجيه	مسار يربط بين مُرسل البيانات الوصفية ومستهلكها	Routing path
درجة صامتة	نتيجة `nan` قد تخفي فشل التقييم عند استخدام سياسة افتراضية للأخطاء	Silent NaN score

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٢ — دعم واجهة Array API

array_api_dispatch، والانتقال إلى وحدة معالجة الرسوميات

الوحدة **scikit-learn ١.٩** الحجم **٣٥٧** كلمة تقريباً

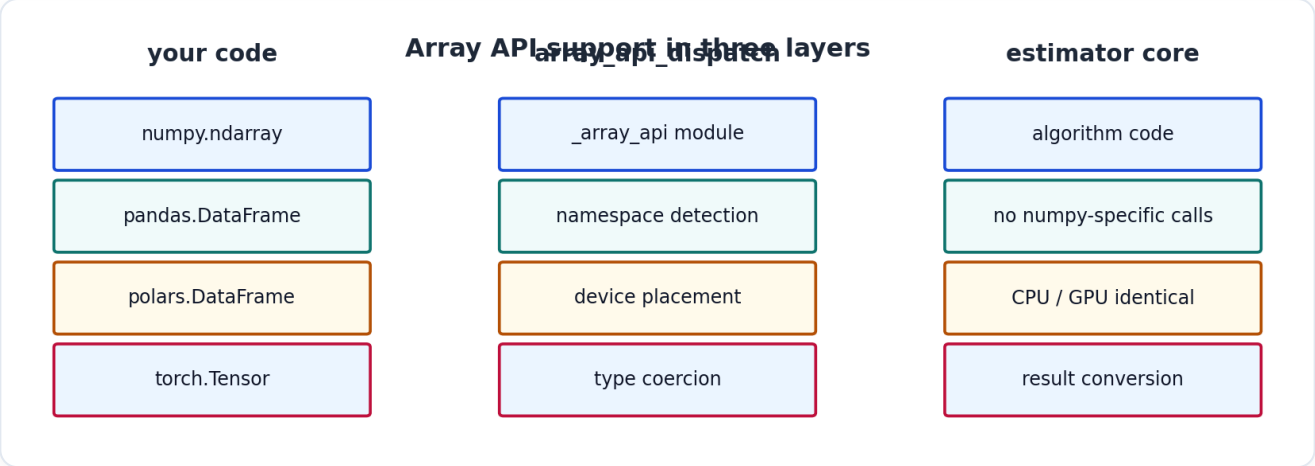
ما الذي ستتقنه في هذا الفصل

- أن يشرح الفرق بين معيار Array API ودعم خوارزمية بعينها
- أن يفحص array_api_support في وسوم المقدّر
- أن يهيّئ الإعداد البيئي قبل استيراد SciPy، ثم scikit-learn
- أن يقيّم قيود النقل والذاكرة بدلاً من افتراض تسريع GPU

١١.١ الواجهة الموحّدة لا تعني دعماً شاملاً

تريد أن تشغّل خوارزمية على مصفوفة من مكتبة تدعم GPU. المشكلة أن واجهات المصفوفات ليست متطابقة تماماً، وبعض الخوارزميات تستدعي عمليات خاصة بـ NumPy أو SciPy. تهدف **واجهة Array API** إلى توحيد مجموعة من عمليات المصفوفة، ويتيح `array_api_dispatch` لـ scikit-learn استخدام هذا المسار حيث دُعم.

لكن الدعم ليس وعداً بأن كل مقدّر، كل عملية، وكل خطوة في خط الأنابيب ستعمل على GPU. قد تدعم أداة تحجيم المصفوفة الواجهة ولا يدعمها مقدّر شجري؛ وقد تقبل بياناتك ثم تنسخها إلى CPU في مكان آخر. ابدأ بفحص وسوم الدعم، لا بتوقع سرعة أعلى.



الشكل ٢٠٢طبقات الانتقال عبر Array API

وسوم دعم Array API للمقدّرات python

```python
from sklearn.utils._tags import get_tags
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

for estimator in (StandardScaler(), PCA(), LogisticRegression(),
                  RandomForestClassifier()):
    tags = get_tags(estimator)
    print(type(estimator).__name__,
          "array_api_support=", tags.array_api_support)
```

هذه سمة استكشافية للنسخة الحالية، لا ضمان أداء أو توافق لكل backend. افحص توثيق الصنف والإصدار، وتأكد من أن خطوات خط الأنابيب كاملة تدعم التمثيل الذي تستخدمه.

١١.٢ الإعداد قبل الاستيراد

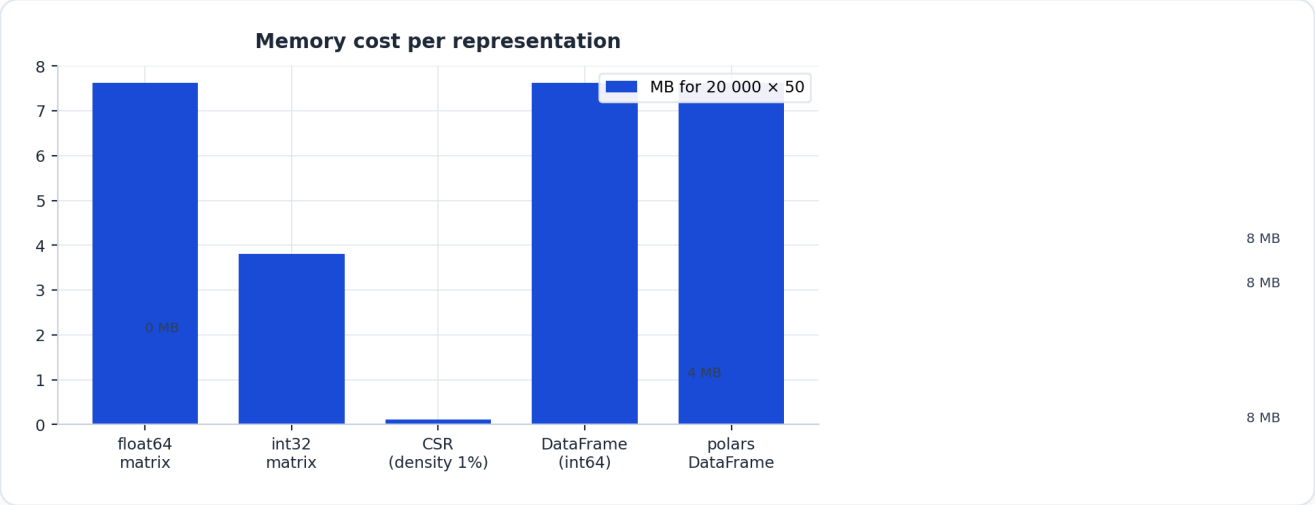
يحتاج مسار SciPy المتوافق إلى ضبط `SCIPY_ARRAY_API=1` قبل استيراد SciPy أو المكتبات التي تستوردها. ضبط متغير البيئة بعد استيرادها متأخر. في جلسة جديدة عيّن المتغير في أمر التشغيل أو إعداد البيئة، ثم استورد الحزم. وقد يتطلب backend إضافياً لا يكون مثبتاً افتراضياً.

```python
# قراءة الإعداد دون تغييره بعد الاستيراد
import os
import sklearn
from sklearn import get_config

print("SCIPY_ARRAY_API:", os.environ.get("SCIPY_ARRAY_API", "<غير مضبوط>"))
print("array_api_dispatch في الإعداد:",
      "array_api_dispatch" in get_config())
print("نسخة scikit-learn:", sklearn.__version__)
```

لا يحاول هذا المقطع تغيير البيئة في منتصف العملية؛ إنه يدققها فقط. لتجربة GPU حقيقية، ابدأ عملية Python جديدة مع المتغير الصحيح وثبّت backend متوافقاً وفق تعليماته. إذا لم تتوفر البيئة فلا تدّع أن المثال نفّذ الحساب على جهاز رسومي.

١١.٣ سياق التفعيل وتجربة مصفوفة

بعد ضبط البيئة، يتيح `sklearn.config_context(array_api_dispatch=True)` تشغيل المسار المدعوم في نطاق محدد. ضع السياق حول العمليات التي تحتاج هذا الإعداد، وأعده بعد ذلك تلقائياً. إذا كان إعداد SciPy غائباً فقد يرفع السياق `RuntimeError`؛ تعامل مع ذلك كمتطلب بيئي واضح، لا كسبب لتعطيل الاختبار أو إخفاء الفشل.



الشكل ٢٠٣دعم Array API يتحدد عبر المكتبة والمقدّر والبيئة

تشغيل مسار Array API أو إعلان المتطلب python

```python
from sklearn import config_context
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler

X, _ = make_regression(n_samples=40, n_features=4, random_state=42)
try:
    with config_context(array_api_dispatch=True):
        Xt = StandardScaler().fit_transform(X)
    print("نجح التحويل:", Xt.shape)
except RuntimeError as exc:
    print("البيئة غير مهيأة لمسار Array API:", str(exc).splitlines()[0])
```

هذا المثال يستخدم مصفوفة NumPy، فلا يثبت وحده تشغيل GPU. الاختبار الحاسم يمرر مصفوفة من backend المطلوب، ويتأكد من نوع المخرجات وموقعها، ويقيس نقل البيانات. استخدام `try` هنا مقصود لإظهار متطلب البيئة المحدد؛ في فحص نشر آلي اجعل غياب البيئة فشلاً صريحاً عندما تكون GPU شرطاً.

١١.٤ أين يختفي التسريع؟

حتى إذا عمل مقدّر على backend، قد يكون حجم البيانات صغيراً بحيث تفوق كلفة نقلها إلى GPU كلفة الحساب. قد ينسخ محوّل لاحق البيانات إلى CPU، أو يعيد خرجاً من نوع مختلف. قِس زمن خط الأنابيب كاملاً، لا زمن خطوة منفردة، وافحص موضع البيانات بعد كل تحويل. ولا تخلط بين التوافق الوظيفي والتسريع العددي.

تحقق من نوع المخرجات وحجمها python

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

X = np.arange(24, dtype=float).reshape(6, 4)
scaler = StandardScaler()
Xt = scaler.fit_transform(X)
print("نوع الإدخال:", type(X).__name__, "نوع المخرج:", type(Xt).__name__)
array = Xt.to_numpy() if hasattr(Xt, "to_numpy") else Xt
print("الشكل:", Xt.shape, "الحجم بالبايت:", array.nbytes)
print("المتوسط بعد التحجيم:", np.round(array.mean(axis=0), 8).tolist())
```

هذا خط أساس CPU متعمد. سجّل النوع والحجم قبل مقارنة backend آخر، وتأكد أن المخرجات متكافئة عددياً ضمن سماحية مناسبة. إذا كان زمن النقل غير محسوب، قد يبدو القياس مضللاً.

> ⚠ **انتبه**
>
> لا تنسب دعماً لـGPU إلى scikit-learn كله لمجرد أن مصفوفة جهاز رسومي يمكن تمريرها إلى خطوة واحدة. راجع وسوم المقدّرات، ثم اختبر كل انتقال في الخط وتأكد من بقاء البيانات في المكان المتوقع.

> 🏭 **في الواقع**
>
> الواجهة تفيد فرقاً تريد إعادة استخدام عمليات المصفوفة عبر backends، لكنها لا تستبدل أدوات التعلم العميق المتخصصة عندما تكون الخوارزمية تتطلب تلك البنية. قارن الأداء على مدخلاتك الحقيقية، مع زمن النقل والذاكرة، واتخذ القرار على مستوى النظام لا اسم المكتبة.

تمارين الفصل الثاني والتسعين

- **ملاحظة**

التمرين ١١.١

اطبع وسم `array_api_support` لكل من `StandardScaler` و`PCA` و`RandomForestClassifier`. فسّر لماذا لا يكفي وسم واحد للحكم على Pipeline.

التمرين ١١.٢

تحقق من قيمة `SCIPY_ARRAY_API` قبل الاستيراد. أعد تشغيل Python مع الإعداد في البيئة وقارن النتيجة.

التمرين ١١.٣

قارن زمن تحويل مصفوفة NumPy صغيرة وكبيرة، واشرح لماذا لا يثبت ذلك وجود GPU.

- **ملاحظة**

```python
فحص الدعم على مستوى كل مقدّر
from sklearn.utils._tags import get_tags
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier

for est in (StandardScaler(), PCA(), RandomForestClassifier()):
    print(type(est).__name__, get_tags(est).array_api_support)
```

يجب أن تدعم كل خطوة المسار المطلوب، ويجب اختبار backend الفعلي لا الوسم وحده.

> **ملاحظة**

```python
الإعداد البيئي الحالي
import os
print("SCIPY_ARRAY_API =", os.environ.get("SCIPY_ARRAY_API", "<غير مضبوط>"))
```

يضبط الإعداد خارج العملية قبل بدء Python؛ تغيير `os.environ` بعد أن استوردت الحزم ليس اختباراً صالحاً لذلك الشرط.

> **ملاحظة**

١١.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
افتراض دعم كل مقدّرات sklearn	بعض الخوارزميات لا تدعم الواجهة	افحص الوسوم والتوثيق لكل خطوة
ضبط متغير البيئة بعد الاستيراد	لا يتغير سلوك المكتبة الجاري تحميلها	ابدأ عملية جديدة بالإعداد مسبقاً
اعتبار التوافق تسريعاً	قد تكون كلفة النقل أكبر من الحساب	قِس زمن الخط كاملاً
الاكتفاء بوسم الدعم	قد يكون هناك تحويل أو نسخة في الوسط	اختبر backend وأنواع المخرجات فعلياً

١١.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
واجهة مصفوفات موحدة	عقد مشترك لعمليات المصفوفات	Array API
تفويض العمليات	اختيار التنفيذ الملائم لنوع المصفوفة	dispatch
backend	المكتبة التي تنفذ عمليات المصفوفة	array backend
نقل البيانات	نسخ البيانات بين CPU، GPU	data transfer

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٣ — إطار narwhals وتبادل البيانات

DataFrame من أي مكتبة: pandas أو Polars أو Modin أو PyArrow

الوحدة **١.٩ scikit-learn** الحجم **١٢١٢** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يشرح لماذا كان scikit-learn مرتبطاً بشكل pandas، وأين بالضبط يقع الحدّ اليوم
- أن يلفّ إطاراً من أي مكتبة بـ `nw.from_native` ويمرّره إلى مُقدِّر مدرَّب دون تحويل يدوي
- أن يكتب تعبيراً على الإطار نفسه بالواجهة `nw.col` بدل حلقة مرور على الصفوف
- أن يعرف متى يفقد أسماء الأعمدة، ولماذا يختفي `feature_names_in_` عند التدريب على narwhals
- أن يميّز بوضوح بين ما تفعله narwhals وما لا تفعله، فلا يظنّ أنها تجعل polars أسرع

١١.١ المشكلة: مكتبة صُمِّمت على هيئة مكتبة أخرى

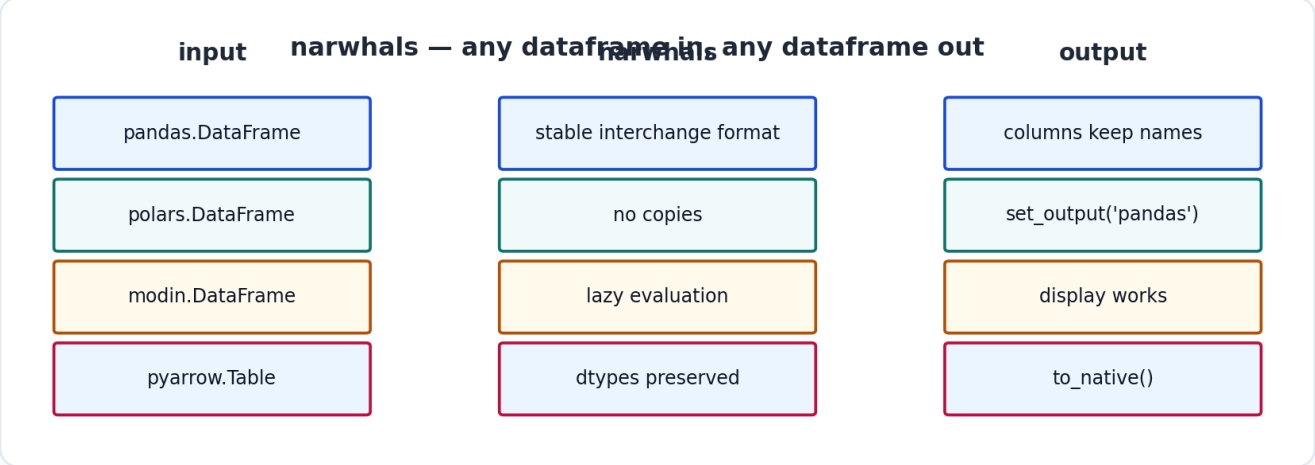
تخيّل فريقاً يسلّم خط أنابيبه إلى الإنتاج. المعالجة كلها في `pandas`، لكن مكتبات البيانات في الشركة متعدّدة: `polars` للقراءة السريعة من القرص، و`modin` لتوزيع إطار البيانات على أسطر جهاز، و`pyarrow` لتخزين أعمدة مضغوطة. الفريق يكتب سطراً واحداً يمرّ بهدوء:

ما الذي كان يحدث فعلاً قبل 1.9 — python

```python
import polars as pl
import pandas as pd
from sklearn.preprocessing import StandardScaler

frame = pl.read_parquet("customers.parquet")      # DataFrame من polars
df = frame.to_pandas()                            # الحلّ الذي يعرفه الجميع
scaled = StandardScaler().fit_transform(df)
```

السطر الثاني ليس تفضيلاً أسلوبياً؛ كان **مطلوباً**. `check_array` في scikit-learn مربوط بـ `pandas` تاريخياً: إمّا مصفوفة `ndarray`، وإمّا كائن يملك صفوفاً وأعمدة، وإمّا استثناءً. أي مكتبة أخرى كانت تُرمى بـ `ValueError` أو، الأسوأ، تُحوَّل بصمت عبر `.values` فتفقد أنواع الأعمدة وأسماءها في صمت.

والمشكلة ليست في `fit` وحدها. حتى لو أتقنتَ التمرير، فكل تحويل لاحق يعيدك إلى `ndarray` بلا أسماء، فيفقد `set_output(transform="pandas")` معناها. فالجدران ليست عند المدخل، بل عند كل حدّ.



الشكل ٢٠٤إطار من أي مكتبة يدخل، وإطار من أي مكتبة يخرج: طبقة narwhals تمرّ بين الطرفين بلا نسخ

الشكل أعلاه هو الحلّ الهندسي: صندوقان — مدخل ومخرج — تفصل بينهما طبقة واحدة موصوفة بأنها «صيغة تبادل مستقرة، بلا نسخ، بتقييم كسول، وأنواع بيانات محفوظة». هذا ما يدّعيه narwhals. والفصل التالي يجيب: ما الذي تحقّقنا منه فعلياً، وما الذي لم نتحقّق منه بعد؟

narwhals ١١.٢: طبقة تبادل، لا مكتبة بيانات جديدة

الخلط الأول، والأكثر شيوعاً: **narwhals ليست pandas الجديدة**. لا تحمل صفوفاً ولا تحسب. الوحدة `narwhals.stable.v1` واجهة فوق مكتبة قائمة، غرضها أن تريحك من حفظ فروق كل مكتبة. هي تشبه `pickle` في أنصاف الطريق: ليست البيانات، بل طريقة موحّدة للحديث عنها.

في ١.٩ لم تعد narwhals اعتماداً اختيارياً: صارت متطلّباً معلناً في بيانات scikit-learn نفسه.

> **جديد في ١.٩**
>
> وحدة `sklearn.utils._sklearn_compat` — التي كانت تجسّد توافق الإصدارات القديمة من `pandas` و`scipy` — **حُذفت تماماً** من ١.٩. الاعتماد انتقل إلى `narwhals` الذي تطلب scikit-learn نسخته في بياناتها الرسومية. أي كود يستورد من `_sklearn_compat` اليوم يفشل عند الاستيراد.

لنبدأ بالجرد قبل أي ادّعاء. هذا الكود لا «يجرّب» المكتبات، بل يسأل بايثون نفسه: أيّها قابل للاستيراد في هذه البيئة؟

```python
# جرد مكتبات البيانات فعلياً في هذه البيئة
import importlib.util as iu
import importlib.metadata as md

LIBS = ["pandas", "polars", "modin", "pyarrow", "duckdb"]
present = {}
for lib in LIBS:
    try:
        present[lib] = iu.find_spec(lib) is not None
    except (ImportError, ValueError):
        present[lib] = False

for lib, ok in present.items():
    print(f"{lib:>8} : " + ("available" if ok else "MISSING"))

reqs = [r for r in md.requires("scikit-learn") if r.startswith("narwhals")]
print("narwhals requirement:", reqs[0] if reqs else "not declared")
import narwhals
print("installed narwhals :", narwhals.__version__)
```

النتيجة صريحة ولا تحتمل التأويل: `pandas` فقط هي المثبّتة. و`polars` و`modin` و`pyarrow` و`duckdb` الأربع **غير مثبّتة** في هذه البيئة. كل مثال في هذا الفصل على `pandas` حقيقي ومُنفَّذ؛ وكل مثال على مكتبة أخرى مكتوب كـ `exec="skip"` ومكتوب صراحةً أنه توضيحي. لن ندّعي تشغيل ما لم نشغّله.

ولنؤكّد الحذف بأمرٍ واحد:

```python
# سابق كان موجوداً وصار غير موجود
import narwhals
for path in ["sklearn.utils._sklearn_compat",
             "sklearn.utils._set_output"]:
    try:
        __import__(path)
        print(f"{path:>32}: importable")
    except ModuleNotFoundError as exc:
        print(f"{path:>32}: {exc}")
print("narwhals version:", narwhals.__version__)
```

القائمة الحقيقية لأهداف الإخراج في ١.٩ ثلاثة لا رابع لها، وسنعود إليها في قسم الفخاخ: `default` و`pandas` و`polars`. أي اسم آخر — `pyarrow` أو `modin` أو `duckdb` — ليس هدف إخراج، مهما بدا منطقياً.

المكتبة	ما تمرّره إلى `nw.from_native`	متوفّرة في هذه البيئة
pandas	`pandas.DataFrame`	نعم
polars	`polars.DataFrame` أو `polars.LazyFrame`	لا
modin	`modin.pandas.DataFrame`	لا
pyarrow	`pyarrow.Table`	لا
duckdb	كائن `Relational` من `duckdb`	لا

الصفوف الأربعة الأخيرة مكتوبة كما توثّقها narwhals نفسها، وقد شغّلنا منها صفّ pandas وحده في هذا الفصل. جرّب جدولك الخاص بعد تثبيت المكتبة، وسترى أن كل ما يتغيّر هو اسم الصف الأول.

١١.٣ الالتفاف: كيف يقبل scikit-learn إطاراً ليس pandas

آلية العمل سطر واحد: `nw.from_native` تعيد كائن `narwhals.DataFrame` هو غلاف رقيق فوق الكائن الأصلي. الغلاف لا ينسخ البيانات — سأكشف هذا بالقياس في قسم الحدود — لكنه يمنحك واجهة واحدة تعمل على أي خلفية.

الخطوة الثانية، وهي المهمة فعلياً: مرّر الغلاف إلى `fit` أو `predict` أو `transform` كأنه `DataFrame` .

إطار narwhals يمرّ إلى StandardScaler وLogisticRegression python

```python
import narwhals.stable.v1 as nw
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=300, n_features=6, n_informative=4,
                           n_redundant=0, class_sep=1.1, random_state=42)
df = pd.DataFrame(X, columns=["age", "income", "tenure", "balance",
                              "score", "region"])
frame = nw.from_native(df, eager_only=True)
print("native  :", type(df).__name__, df.shape)
print("wrapped :", type(frame).__name__, "| backend:", frame.implementation)
print("schema  :", frame.schema)

plain = StandardScaler().set_output(transform="default").fit(df[["income"]])
out = plain.transform(nw.from_native(df[["income"]], eager_only=True))
print("transform(narwhals)  ->", type(out).__name__, out.shape)

wrapped = StandardScaler().fit(df[["income"]])
out_pd = wrapped.transform(nw.from_native(df[["income"]], eager_only=True))
print("transform=pandas     ->", type(out_pd).__name__, list(out_pd.columns))

clf = LogisticRegression(max_iter=5000).fit(df[["age", "income"]], y)
proba = clf.predict_proba(frame[["age", "income"]])
print("predict_proba(nw)    ->", type(proba).__name__, proba.shape)
print("first row            :", np.round(proba[0], 4))
```

اقرأ السطرين الحاسمين. `StandardScaler` الذي ضبطنا له `transform="default"` استقبل إطار `narwhals` وأعاد `ndarray` بمقياس ٣٠٠×١ — أي أن scikit-learn ليس يحوّل الإطار يدوياً، بل **يستقبله ويفكّه داخلياً** ويحوّل الأعمدة إلى مصفوفة رقمية كما يفعل دائماً. و`predict_proba` على الإطار نفسه أعاد `ndarray` بمقياس ٣٠٠×٢، وقيم الصف الأول ٠٫٣٢٢٨ و٠٫٦٧٧٢.

وفي السطر الأخير تحذير لم نطلبه: `X does not have valid feature names`. هذا التحذير ليس خطأً في narwhals، بل **معلومة مفيدة**: الإطار الالتفافي لا يقرأه scikit-learn كمصدر لأسماء الأعمدة، لأنه ليس `pandas.DataFrame` حتى لو كانت خلفيته pandas. سلوك التحذيرات نفسه في الفصل ١٠: لا يجهل شيئاً، لكنه يصرّح.

الفرق بين «يُقبل» و«يُفهَم»

scikit-learn يقبل غلاف narwhals في `fit` و`transform` و`predict`، لكنه يحوّله إلى مصفوفة تفقد الأسماء. القبول يعني «لن أرمي استثناءً»؛ الفهم يعني «سأحفظ `feature_names_in_` وسأعرف أي عمود هو أي». وسنرى الفرق في القسم التالي، وهو الفصل كلّه في سطرين.

١١.٤ واجهة التعبيرات: حيث تصير الحلقة سطراً واحداً

الغلاف نفسه يعطي واجهة مغايرة للواجهة الأصلية: واجهة **تعبيرات** (expression-based) تتأخّر التنفيذ. بدل `df["age"].mean()` أو حلقة `for` ، تكتب `nw.col("age").mean()` — والتعبير يُبنى أولاً، ثم ينفَّذ مرة واحدة على الإطار كله.

لنبنِ جدول موظفين صغيراً ونشغّل عليه سلسلة عمليات حقيقية:

```python
# تجميع وتصفية وترتيب على الإطار نفسه
people = pd.DataFrame({
    "age": [23, 41, 35, 60, 28, 52],
    "income": [1200.0, 5400.0, 3100.0, 900.0, 2200.0, 4300.0],
    "city": ["dubai", "cairo", "dubai", "alex", "cairo", "alex"],
})
nw_people = nw.from_native(people, eager_only=True)

print("متوسط العمر :", nw_people.select(nw.col("age").mean()).item())
print("أعلى دخل    :", nw_people.select(nw.col("income").max()).item())

by_city = (nw_people
           .group_by("city")
           .agg(nw.col("income").mean().alias("mean_income"),
                nw.len().alias("rows"))
           .sort("mean_income", descending=True))
print(by_city.to_native())

rich = (nw_people
        .filter(nw.col("income") > 3000)
        .sort("income", descending=True)
        .select("age", "income"))
print(rich.to_native())

tagged = nw_people.with_columns(
    bracket=nw.when(nw.col("income") >= 3000).then(1).otherwise(0))
print("عدد الفئة   :", tagged.select(nw.col("bracket").sum()).item())
print("backend     :", nw_people.implementation,
      "| to_native:", type(tagged.to_native()).__name__)
```

متوسط العمر ٣٩٫٨٣، وأعلى دخل ٥٤٠٠. التجميع أعاد ثلاث مدن مرتّبة تنازلياً حسب متوسط الدخل: القاهرة ٣٨٠٠، والإسكندرية ٢٦٠٠، ودبي ٢١٥٠، ولكل منها صفّان. التصفية تركت ثلاثة صفوف فقط (الأعلى دخلاً فوق ٣٠٠٠)، و `with_columns` أضاف عموداً محسوباً عبر `nw.when(...).then(...).otherwise(...)` وعدّ الفئة.

لاحظ `backend` في آخر سطر: ما زال `pandas` . أنت لم تغيّر مكتبة البيانات — لقد غيّرت **الواجهة التي تتحدث بها عنها**. وهذا بالضبط ما تحتاجه في مشروع يفتح فيه ثلاثة ملفات من ثلاث مكتبات مختلفة.

١١.٥ `to_native` وما تخسره عند الحدود

الغلاف لا يخرج من نطاق الدالة بلا أثر: دالة `to_native()` ترجع كائن المكتبة الأصلية — وهنا `pandas.DataFrame`. لكن الفصل الجادّ هو **ما الذي تخسره عند الحدّ**.

لنُدرّب نفس الأنبوب مرتين: مرة على `pandas.DataFrame`، ومرة على غلاف narwhals، ثم نسأل كلًّا منهما: من أنا؟

```python ما يفقده النموذج عند التدريب خلف غلاف narwhals
from sklearn.pipeline import Pipeline

two = df[["age", "income"]]
pipe_pd = Pipeline([("sc", StandardScaler()),
                    ("clf", LogisticRegression(max_iter=5000))]).fit(two, y)
pipe_nw = Pipeline([("sc", StandardScaler()),
                    ("clf", LogisticRegression(max_iter=5000))])
pipe_nw.fit(nw.from_native(two, eager_only=True), y)

print("fit on pandas   -> feature_names_in_:", hasattr(pipe_pd,
"feature_names_in_"))
print("fit on narwhals -> feature_names_in_:", hasattr(pipe_nw,
"feature_names_in_"))
print("names from pd :", list(pipe_pd[:-1].get_feature_names_out()))
print("names from nw :", list(pipe_nw[:-1].get_feature_names_out()))
print("تطابق التوقعات:", bool((pipe_pd.predict(two) == pipe_nw.predict(two)).all()))
```

النتيجة معاكسة تماماً للتوقع الساذج. الأنبوب المدرَّب على pandas يحمل `feature_names_in_` وينتج أسماء أعمدة حقيقية: `age` و`income`. والأنبوب المدرَّب على غلاف narwhals **لا يحمل** `feature_names_in_` إطلاقاً، و`get_feature_names_out()` تعيد له `x0` و`x1` — أسماء مولّدة تلقائياً، بلا أي علاقة بأسماء أعمدتك. ومع ذلك، التنبؤات **متطابقة تماماً**، كما يقول آخر سطر.

هذه هي المقايضة في سطرين: **الغلاف يحمي التنفيذ ولا يحمي السجلّ**. إذا كان نموذجك سيُحفظ ويُستأنف في الإنتاج، فتدريبه خلف غلاف narwhals يعني أن `feature_names_in_` غائب من الملف المُحفظ، وأن تحذير `X has feature names, but StandardScaler was fitted without feature names` سيعود في كل مرة تستدعي فيها النموذج بإطار بأسماء. أنت لا تفقد دقّة — بل تفقد ضمانة هوية الأعمدة، وهي الضمانة التي تجعل الفصل ١٠ عن `feature_names_in_` ممكناً أصلاً.

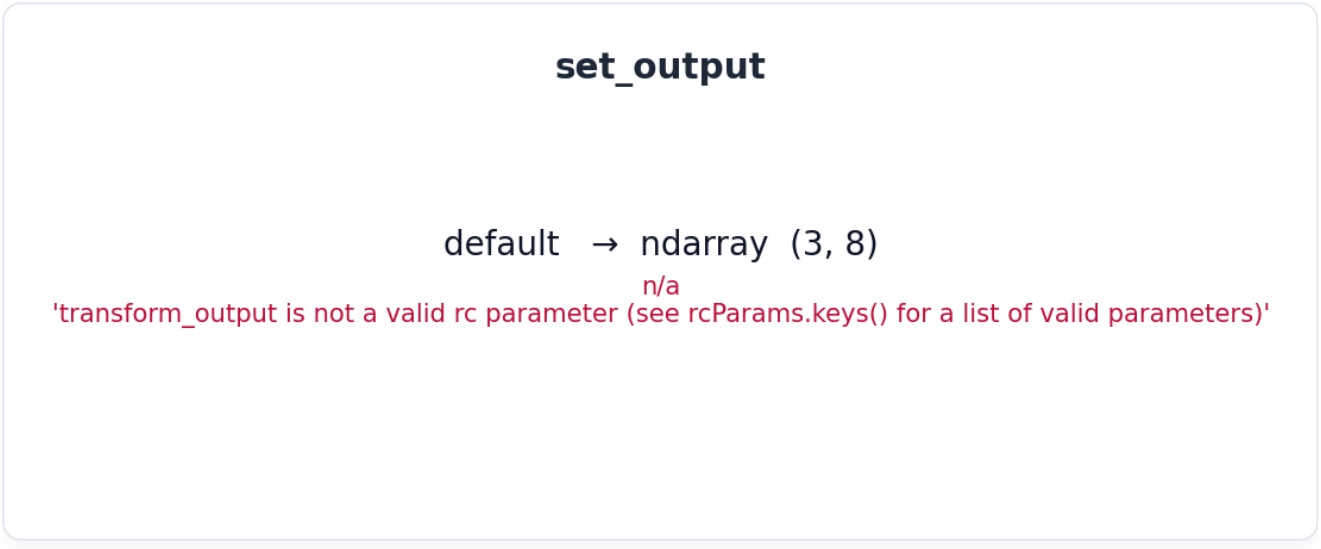
> ⚠ **متى لا تلفّ**
>
> القاعدة العملية: استخدم narwhals **للقراءة والتلاعب والتمرير**، واجعل آخر خطوة في خط أنابيب الإنتاج هي `to_native()` قبل `fit`. غلاف على المدخل، `pandas` على المخرج. هذا يجمع أفضل ما في الاثنين.

١١.٦ مخرج pandas: مخرج set_output بوصفه مخرجاً آمناً

هنا نعود إلى الفصل ٢٠. الإعداد `transform_output` في `sklearn.set_config` أو `set_output(transform=...)` على مُقدِّر بعينه يقرر **نوع حاوية** المخرج. ولأن scikit-learn يحوّل أي إطار narwhals داخلياً إلى مصفوفة، فإن هذا الإعداد هو الجسر الوحيد الذي يعيد الأسماء بعد التحويل.

```python
# النوع يتغيّر بتغيّر الراية فقط
print("transform_output العام:", get_config()["transform_output"])

as_default = Pipeline([("sc", StandardScaler()),
                       ("clf", LogisticRegression(max_iter=5000))])
as_default.set_output(transform="default").fit(two, y)
print("transform=default ->", type(as_default[:-1].transform(two)).__name__)

as_pandas = Pipeline([("sc", StandardScaler()),
                      ("clf", LogisticRegression(max_iter=5000))])
as_pandas.set_output(transform="pandas").fit(two, y)
out = as_pandas[:-1].transform(two)
print("transform=pandas  ->", type(out).__name__, list(out.columns))
print("step alone        ->",
type(as_pandas.named_steps["sc"].transform(two)).__name__)
print("الإعداد العام     ->", get_config()["transform_output"])
```

نفس الأنبوب، نفس المُدخل `two`، نفس المعاملات — والفرق نوع واحد: الأولى أعادت `ndarray` والثانية `DataFrame` بأعمدة `age` و`income`. حتى الخطوة الفردية `StandardScaler` حدها، داخل الأنبوب ذاتها، صارت تعيد `DataFrame`. والسطر الأخير يقول شيئاً مهماً: `set_output` على مُقدِّر بعينه **لم يغيّر** الإعداد العام — التغيير محلي لهذا المُقدِّر وحده، وهذا بالضبط ما تريده في إنتاج فيه أنابيب مختلفة الاحتياجات.



الشكل ٢٠٥أنبوب واحد ومُدخل واحد، لكن راية set_output واحدة تحوّل نوع المخرج من مصفوفة إلى DataFrame

الشكل يشرح ما في الكود ببساطة: الفرق الوحيد بين الصفّين هو سطر `set_output`. لا تغيير في البيانات، ولا في المعاملات، ولا في الترتيب — فقط نوع الحاوية التي يعود بها `transform`.

١١.٧ ما لا تفعله narwhals: حدود صريحة

هنا الجزء الذي يغيب عن معظم المقالات، وهو الأهم في كتاب كهذا. narwhals **طبقة توافق واجهة**، وليست آلة تسريع. ثلاث حقائق يجب أن تقولها بصوت عالٍ لمديرك التقني قبل أن تتبنّاها:

وأقوى طريقة لتثبيت هذه النقاط هي القياس نفسه:

```python
الالتفاف لا يكلّف شيئاً، والتحويل يحدث في الحالتين
import time
rng = np.random.default_rng(0)
big = pd.DataFrame(rng.normal(size=(80000, 8)),
                   columns=[f"c{i}" for i in range(8)])
sc = StandardScaler().set_output(transform="default")
sc.fit(big)
big_nw = nw.from_native(big, eager_only=True)

def median_ms(fn, reps=7):
    ts = []
    for _ in range(reps):
        t0 = time.perf_counter()
        fn()
        ts.append((time.perf_counter() - t0) * 1000)
    return float(np.median(ts))

t_pd = median_ms(lambda: sc.transform(big))
t_nw = median_ms(lambda: sc.transform(big_nw))
print(f"transform على pandas   : {t_pd:.1f} ms")
print(f"transform على narwhals : {t_nw:.1f} ms")
print(f"النسبة                 : {t_nw / t_pd:.2f}")
t_to = median_ms(lambda: nw.to_native(nw.from_native(big, eager_only=True)))
print(f"from_native + to_native: {t_to:.1f} ms")
```

القاعدة التي يعلّمنا إياها هذا القياس: **الزمن يُقاس لا يُخمَّن**. عملية تغليف وفكّ الغلاف هنا تكاد لا تُذكر، لكن الأرقام تتذبذب بين التشغيل والآخر — ولهذا أخذنا الوسيط لسبع تكرارات لا متوسطاً واحداً. الفارق المقيس هنا يقع حول الواحد، أي لا ميزة ولا عيب قياساً. لو قلت لأحدهم «narwhals أسرع»، فقد تكون محقاً في آلة وخاطئاً في أخرى؛ أما «narwhals لا تغيّر تكلفة التحويل» فقاعدة ثابتة.

١١.٨ فخّان كبيران في التبادل

الفخّ الأول أن تفترض أن كل اسم مُكتَب في `transform=` مدعوم. ليس كذلك. المفاجأة الحقيقية: `set_output` لا يتحقّق من الاسم عند الاستدعاء — يتحقّق عند **التحويل**. فتمرّ `set_output` في صمت، و`fit` يمرّ في صمت، ثم يرتفع الاستثناء في أبعد نقطة ممكنة.

set_output لا يتحقّق إلا عند التحويل python

```python
for target in ["polars", "pyarrow", "modin"]:
    sc = StandardScaler()
    try:
        sc.set_output(transform=target)
    except ValueError as exc:
        print(f"{target:>8}: ValueError -> {str(exc).splitlines()[-1][:88]}")
        continue
    try:
        sc.fit(two).transform(two)
        print(f"{target:>8}: transform worked")
    except Exception as exc:
        print(f"{target:>8}: {type(exc).__name__} -> "
              f"{str(exc).splitlines()[-1][:92]}")
```

السطور الثلاثة تقرأ نفسها: `polars` يُعطي `ImportError` لأن مكتبة polars غير مثبّتة في بيئتنا، بينما `pyarrow` و`modin` يُعطيان `ValueError` لأنهما **ليسوا أهداف إخراج أصلاً**؛ الرسالة تذكر القائمة الرسميّة: `['default', 'pandas', 'polars']`. أي أن `set_output` له ثلاثة أهداف، والقائمة الحقيقية هي وثيقتك.

الافتراض بأن `set_output(transform="pyarrow")` يعمل

`pyarrow` ليس هدف إخراج. `set_output` تقبل الاسم في صمت، ثم يرفع التحويل استثناء `ValueError` عند التحويل الفعلي. استخدم `transform="pandas"` أو `transform="default"` دائماً، ولا تلجأ إلى `polars` إلا بعد أن تتأكّد من تثبيتها ومن أن مخرجك فعلاً polars.

الفخّ الثاني أعمق: ليس كل مسار في scikit-learn يقبل غلاف narwhals بحرية. `ColumnTransformer` الذي بُني حول أسماء أعمدة pandas **يرفضه** صراحةً عندما يكون المدخل غلافاً.

ColumnTransformer بالأسماء يفشل على الغلاف، وبالمواضع ينجح python

```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

df2 = df.copy()
df2["region"] = np.where(df2["income"] > df2["income"].median(), "high", "low")
nw2 = nw.from_native(df2, eager_only=True)

try:
    ColumnTransformer([("num", StandardScaler(), ["age", "income"])]).fit(nw2,
y)
    print("by name    -> ok")
except ValueError as exc:
    print("by name    ->", str(exc).splitlines()[0][:88])

by_index = ColumnTransformer([("num", StandardScaler(), [0, 3])]).fit(nw2, y)
print("by index   ->", type(by_index.transform(nw2)).__name__,
      by_index.transform(nw2).shape)

from sklearn.compose import make_column_selector
try:
    ColumnTransformer([("num", StandardScaler(),
                        make_column_selector(dtype_include="number"))]).fit(nw2,
y)
    print("selector   -> ok")
except ValueError as exc:
    print("selector   ->", str(exc).splitlines()[0][:88])
```

رسالة الخطأ الأولى حرفيّة: `Specifying the columns using strings is only supported for dataframes` — أي أن اختيار الأعمدة بالاسم يحتاج كائناً يعرّفه scikit-learn على أنه dataframe. والمُنتقي `make_column_selector` أوضح حداً: `make_column_selector can only be applied to pandas dataframes`. لكنّ المواضع الرقمية تعمل وتعيد `DataFrame` بمقياس ٣٠٠×٢. القاعدة العملية: **إن كان أنبوبك يعتمد على اختيار الأعمدة بالاسم، فادخل إليه بـ `to_native()`؛ وإلا فمرّر الغلاف كاملاً.**

ما يحدث في ميزة «مرشّحو المخاطر»

في نظام تقييم طلبات ائتمان رأيناه، كان الفريق يقرأ من `pyarrow` ويحوّل بـ `.to_pandas()` قبل `ColumnTransformer`. ظنّوا أن استبدال السطر بـ `nw.from_native` «تنظيف». بعد أسبوع، فشل خط الإنتاج برسالة اختيار الأعمدة — القصة نفسها التي في القسم أعلاه. الحلّ لم يكتقنياً: عادوا إلى `to_native()` على المدخل، وأبقوا narwhals للتحليل الجانبي الذي لا يحتاج أنبوباً. الأطر الجيدة هي التي تعرف أين حدودها.

تمارين الفصل الثالث والتسعين

- **ملاحظة**

التمرين ١١.١

ابنِ `pandas.DataFrame` من `make_classification(n_samples=300, n_features=4, n_informative=3, random_state=42)` بأسماء `age` و `tenure` و `balance` و `score`، وأضف عموداً نصياً `region` قيمته `north` أو `south` بحسب إشارة `score`. لفّه بـ `nw.from_native`، ثم اطبع: نوع العمود النصي من `schema`، وعدد الصفوف في كل فئة عبر `group_by` و `nw.len()`، ثم حاول `ColumnTransformer` بالأسماء على الغلاف، ثم حلّها بالمواضع. ما الفرق بين الحالتين؟

التمرين ١١.٢

خذ `Pipeline` من خطوتين (`StandardScaler` ثم `LogisticRegression(max_iter=5000)`) مدرَّباً على عمودين. طبّق `set_output(transform="default")` على نسخة و `transform="pandas"` على أخرى، واطبع نوع مخرج `pipe[:-1].transform` وأعمدته في كل حالة. ثم اطبع `get_config()["transform_output"]` بعد كل عملية، وفسّر لماذا بقيت القيمة العامة كما هي.

التمرين ١١.٣

صديقك كتب هذا الكود، ويقول له إنه يعمل، ثم يشتكي من خطأ غريب:

```
pipe = make_pipeline(StandardScaler())
pipe.set_output(transform="pyarrow")
pipe.fit(df)
pipe.transform(df)
```

الملاحظة أنه لا يرى أي خطأ عند `set_output` ولا عند `fit`. أين يقع الاستثناء بالضبط، وما الرسالة؟ الجواب الصحيح لا يتضمّن تثبيت أي مكتبة.

- **ملاحظة**

```python الغلاف يحمل الأنواع، لكنه يرفض اختيار الأعمدة بالاسم
import narwhals.stable.v1 as nw
import numpy as np, pandas as pd
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=300, n_features=4, n_informative=3,
                           n_redundant=0, random_state=42)
df = pd.DataFrame(X, columns=["age", "tenure", "balance", "score"])
df["region"] = np.where(df["score"] > 0, "north", "south")
df["region"] = pd.Categorical(df["region"], categories=["north", "south"])
nwdf = nw.from_native(df, eager_only=True)

print("schema:", dict(nwdf.schema)["region"])
print("الأعمدة:", nwdf.columns)
counts = nwdf.group_by("region").agg(nw.len().alias("rows")).sort("region")
print(counts.to_native())

try:
    ColumnTransformer([("num", StandardScaler(), ["age", "balance"])]).fit(nwdf,
y)
    print("ColumnTransformer by name: ok")
except ValueError as exc:
    print("by name   ->", str(exc).splitlines()[0][:92])
fixed = ColumnTransformer([("num", StandardScaler(), [0, 3])]).fit(nwdf, y)
print("by index  ->", type(fixed.transform(nwdf)).__name__,
fixed.transform(nwdf).shape)
clf = LogisticRegression(max_iter=5000).fit(fixed.transform(nwdf), y)
print("degree:", clf.coef_.shape)
```

الغلاف حافظ على نوع العمود `Categorical` وعلى أسماء الأعمدة الخمسة، وقد أعاد `group_by` توزيع ٢١١ صفاً شمالاً و٨٩ جنوباً. لكنه رفض اختيار `age` و`balance` بالاسم. وبالمواضع `[0, 3]` نجح التحويل وأعاد `DataFrame` بقياس ٣٠٠×٢، والتدرّب عليه أعاد معاملات بمقياس (١، ٢) — صفّ واحد لكل صنف. الفرق أن scikit-learn يعرف المواضع لأنها أرقام، ولا يعرف الأسماء لأن الغلاف لا يعرّف على أنه dataframe له أسماء.

> **ملاحظة**

```python
# الراية تحوّل نوع الحاوية، لا الإعداد العام
import numpy as np
import narwhals.stable.v1 as nw
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

two = df[["age", "tenure"]]
plain = Pipeline([("sc", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=5000))])
plain.set_output(transform="default").fit(two, y)
print("default:", type(plain[:-1].transform(two)).__name__)

named = Pipeline([("sc", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=5000))])
named.set_output(transform="pandas").fit(two, y)
out = named[:-1].transform(two)
print("pandas :", type(out).__name__, list(out.columns))
print("أول صفان:", np.round(out.to_numpy()[:2], 4).tolist())
print("global :", get_config()["transform_output"])
print("step   :", type(named.named_steps["sc"].transform(two)).__name__)
```

الأنبوبان متطابقان في كل شيء إلا الراية: الأول يعيد `ndarray` والثاني `DataFrame` بأعمدة `age` و`tenure`. `pandas` والإعداد العام بقي بعد العمليتين لأن `set_output` يحرّر حالة المُقدِّر نفسه لا الحالة العامة للمكتبة. إن أردت تغييراً عاماً لكل الأنابيب في العملية، فالأداة الصحيحة `sklearn.set_config(transform_output=...)`.

> **ملاحظة**

الخطأ يقع عند التحويل لا عند الضبط python

```python
import narwhals.stable.v1 as nw
from sklearn.preprocessing import StandardScaler

for target in ["polars", "pyarrow", "modin", "pandas"]:
    sc = StandardScaler()
    set_at = "ok"
    try:
        sc.set_output(transform=target)
    except Exception as exc:
        set_at = f"{type(exc).__name__} at set_output"
    ran = "-"
    if set_at == "ok":
        try:
            sc.fit(df[["age"]]).transform(df[["age"]])
            ran = "transform ok"
        except Exception as exc:
            ran = f"{type(exc).__name__} at transform"
    print(f"{target:>8}: set_output={set_at:<24} -> {ran}")
print("مدعوم رسمياً: default, pandas, polars")
```

الاستثناء يقع عند `transform` لا عند `set_output` ولا عند `fit`. و`pyarrow` و`modin` يُرفع عند التحويل استثناء `ValueError` لأن القائمة الرسمية للأهداف هي `default` و`pandas` و`polars` فقط، بينما `polars` وحده يُعطي `ImportError` لأنه هدف مدعوم نحوياً لكن المكتبة غير مثبّتة. لا حاجة لتثبيت شيء: الحلّ هو اختيار `pandas`.

> **ملاحظة**

١١.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط
`set_output(transform="pyarrow")` أو `"modin"`	`set_output` و `fit` يمرّان في صمت، ثم `config must be in ['default', 'pandas', 'polars']`
تدريب الأنبوب على غلاف narwhals	يُسقط `feature_names_in_` ويحذّر `X has feature names`
`ColumnTransformer([... , ["age","income"]])` على غلاف	`ns using strings is only supported for dataframes`
`make_column_selector(...)` على غلاف	`selector can only be applied to pandas dataframes`
توقّع أن narwhals يجعل polars أسرع داخل scikit-learn	التحويل إلى `ndarray` يحدث في الحالتين؛ لا يوجد مسار صفر-
استيراد `sklearn.utils._sklearn_compat`	`ModuleNotFoundError` — الوحدة محذوفة في ١.٩
قراءة `nw.__version__` من `narwhals.stable.v1`	`arwhals.stable.v1' has no attribute '__version__'`

١١.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
إطار narwhals	غلاف رقيق حول إطار مكتبة معيّنة يعطي واجهة واحدة	Narwhals DataFrame
طبقة التبادل	الوسيط الذي يوحّد واجهات مكتبات البيانات دون نسخ	Interchange layer
التفاف بلا نسخ	إنشاء غلاف حول الكائن الأصلي دون نقل بياناته	Zero-copy wrapping
واجهة التعبيرات	كتابة العمليات كتعابير `nw.col(...)` تُنفَّذ بعد بنائها	Expression API
تقييم كسول	بناء وصف العملية قبل تطبيقها على البيانات	Lazy evaluation
مخرج `set_output`	نوع الحاوية الذي يعيده `transform` بعد التحويل	Transform output
اختيار بالأسماء	تمرير أسماء أعمدة نصية إلى `ColumnTransformer`	String column selection
فقدان الهوية	غياب `feature_names_in_` وأسماء `get_feature_names_out`	Loss of identity

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٤ — واجهات العرض البياني في ١.٩

من ConfusionMatrixDisplay إلى PartialDependenceDisplay والعرض-HTML الجديد

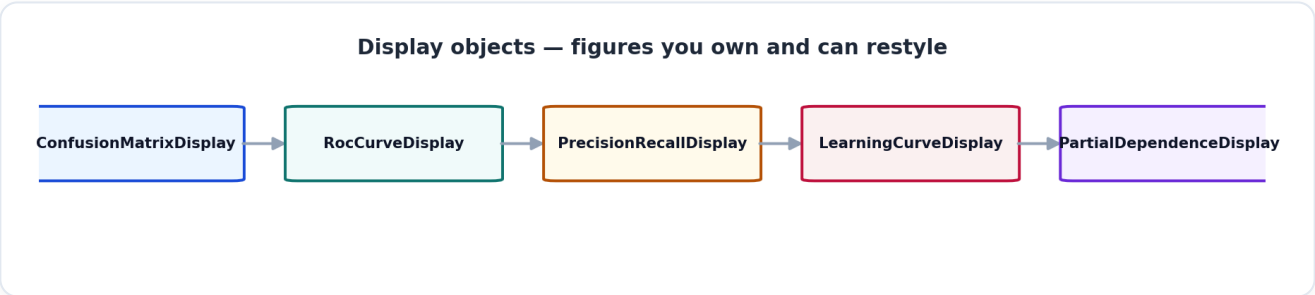
الوحدة **scikit-learn ١.٩** الحجم **٣٦٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يختار Display ملائماً لمصفوفة الخلط أو منحنى ROC أو الاعتماد الجزئي
- أن يبني كائن عرض من تنبؤات أو من مقدّر مدرّب
- أن يشرح حدود الرسوم التشخيصية والاعتماد الجزئي
- أن يستخدم تمثيل HTML للمقدّر لفحص بنيته دون اعتباره بديلاً للتوثيق

١١.١ الرسم كواجهة، لا كصورة نهائية

يحتاج الفريق إلى رسم يشرح خطأ المصنّف أو أثر خاصية. إذا حسبت كل شيء يدوياً ثم رسمت محاور بلا سياق، يصعب إعادة إنتاج الشكل أو معرفة أي بيانات ولّدته. تقدم واجهات العرض في `sklearn.metrics` و`sklearn.inspection` و`sklearn.model_selection` كائنات تحمل بيانات الرسم وتتيح تخصيصه، مثل `ConfusionMatrixDisplay` و`RocCurveDisplay` و`PartialDependenceDisplay`.



الشكل ٢٠٦ خريطة واجهات Display في scikit-learn

بناء مصفوفة الخلط من التوقعات python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
model.fit(Xtr, ytr)
display = ConfusionMatrixDisplay.from_estimator(model, Xte, yte)
print("كائن العرض:", type(display).__name__)
print("مصفوفة الخلط:", display.confusion_matrix.tolist())
```

`from_estimator` يتولى التنبؤ والرسم من بيانات الاختبار؛ احتفظ بمرجع الكائن إذا أردت الوصول إلى المصفوفة أو المحاور وتخصيصهما. لا تستخدم بيانات التدريب لتقديم تشخيص على أنه أداء خارج العينة.

١١.٢ من الاحتمال إلى ROC

واجهة `RocCurveDisplay.from_estimator` تستخرج الدرجات من المقدّر، بينما `from_predictions` تقبل درجات حسبتها مسبقاً. إذا كان المصنّف لا يخرج احتمالات لكن لديه `decision_function`، يمكن للواجهة استخدام درجة القرار للترتيب. المنحنى يصف التمييز عبر عتبات متعددة؛ لا يثبت معايرة الاحتمالات ولا يحدد عتبة تشغيل مناسبة.

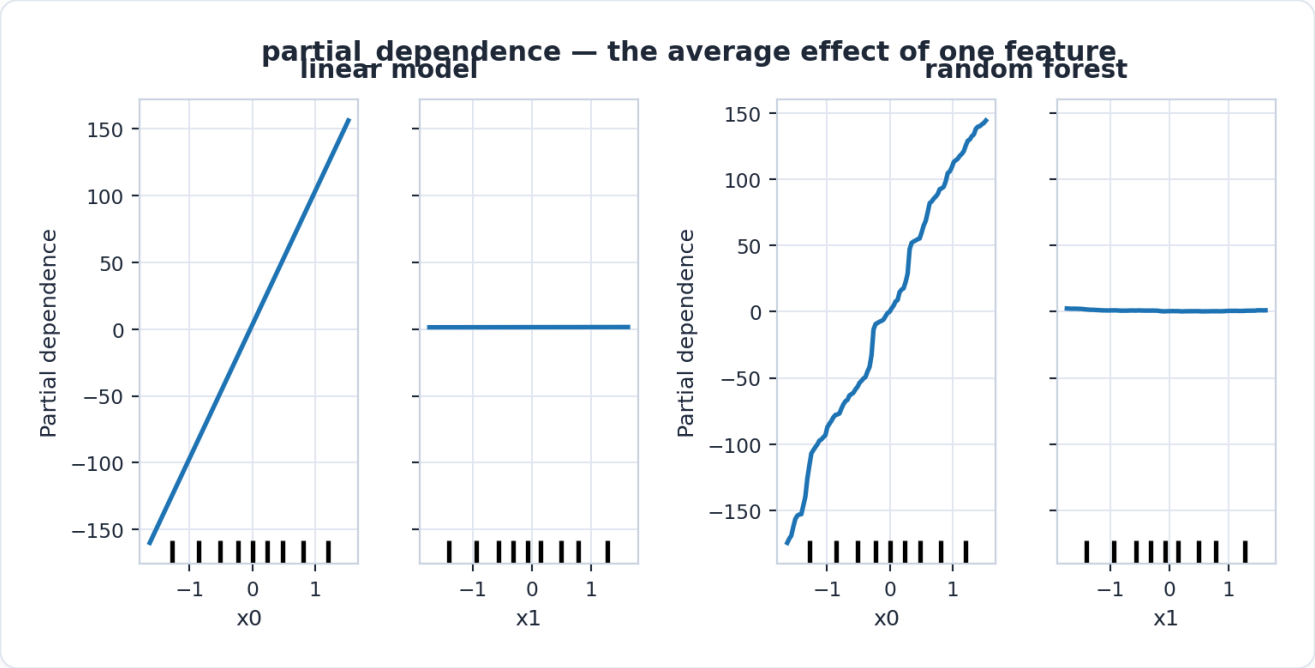
واجهة ROC من مقدّر مدرّب python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import RocCurveDisplay
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
model.fit(Xtr, ytr)
roc = RocCurveDisplay.from_estimator(model, Xte, yte)
print("AUC:", round(roc.roc_auc, 4))
print("نقاط المنحنى:", len(roc.fpr))
```

للمقارنة بين نماذج، ارسمها على المجموعة نفسها وبالترميز نفسه، واذكر أن مساحة ROC قد تخفي ضعف الدقة الإيجابية عندما تكون الفئة الإيجابية نادرة. اختر PR أو كلفة تشغيلية مناسبة بحسب التطبيق.

١١.٣ الاعتماد الجزئي: سؤال محدود لا تفسير كامل

تقدّر `PartialDependenceDisplay.from_estimator` متوسط استجابة النموذج حين تثبت خاصية أو زوج خصائص على قيم محددة وتتوسط على بقية الصفوف. الرسم مفيد لفهم السلوك الإجمالي، لكنه قد يعرض تراكيب غير واقعية إذا كانت الخصائص مترابطة. كما أنه يصف النموذج لا السببية، ولا يضمن أن الأثر نفسه موجود في كل مجموعة فرعية.



الشكل ٢٠٧ الاعتماد الجزئي على خصائص مختارة

إنشاء اعتماد جزئي بعد التدريب python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import PartialDependenceDisplay
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(Xtr, ytr)
features = [X.columns[0], X.columns[1]]
display = PartialDependenceDisplay.from_estimator(
    model, Xte, features=features, kind="average", grid_resolution=20)
print("الخصائص المعروضة:", display.feature_names)
```

استخدم مجموعة مناسبة لتفسير التوزيع الذي سيواجهه النموذج، وتحقق من نطاق كل خاصية. إذا كان الرسم يمر عبر مناطق لا توجد فيها بيانات، فالتفسير هناك استقراء لا ملاحظة.

١١.٤ العرض HTML للمقدّر

يستطيع Jupyter عرض المقدّرات بصرياً عندما يكون إعداد العرض `diagram`، كما توفر `sklearn.utils.estimator_html_repr` تمثيلاً HTML يمكن فحصه أو إدراجه في تقرير. هذا العرض يوضح تركيب الخطوات والمعاملات، لكنه لا يكشف كل ما يلزم لتدقيق البيانات أو صلاحية التقييم؛ لا تحفظ HTML وحدها بوصفها وصفة إعادة إنتاج.

فحص تمثيل HTML لخط أنابيب python

```python
from sklearn import set_config
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.utils import estimator_html_repr

set_config(display="diagram")
model = make_pipeline(StandardScaler(),
                      LogisticRegression(C=0.5, max_iter=1000))
html = estimator_html_repr(model)
print("وسم HTML:", html[:40])
print("اسم الخطوة موجود:", "StandardScaler" in html)
print("طول التمثيل:", len(html))
```

استعد إعداد العرض العالمي إذا كان سكربتك يغيره لمستهلكين آخرين، أو استخدم `config_context` حيث يناسب. التمثيل البصري أداة فحص، أما عقد المعاملات الفعلي فيظل متاحاً عبر `get_params(deep=True)` .

> **في الواقع**
>
> في تقرير للمراجعة، أرفق الرسم مع اسم مجموعة البيانات والفترة ومقياس الأداء وبذرة التقسيم. هذا السياق يمنع قراءة صورة جميلة على أنها حقيقة عامة، ويجعل الفريق قادراً على إعادة إنشاء الشكل أو اكتشاف تغيّر البيانات.

تمارين الفصل الرابع والتسعين

- **ملاحظة**

التمرين ١١.١

أنشئ `ConfusionMatrixDisplay` من التنبؤات ثم اطبع مصفوفته. أعد التجربة بترتيب `labels` مختلف واشرح أثر ذلك على المحاور.

التمرين ١١.٢

أنشئ `RocCurveDisplay` من مقدّر مدرّب، واطبع AUC وعدد نقاط المنحنى.

التمرين ١١.٣

ولّد تمثيل HTML لخط أنابيب، ثم قارن أسماء خطواته بقاموس `get_params` .

- **ملاحظة**

مصفوفة الخلط عبر تنبؤات ثابتة python

```python
from sklearn.metrics import ConfusionMatrixDisplay

y_true = [0, 0, 1, 1, 1]
y_pred = [0, 1, 1, 0, 1]
d = ConfusionMatrixDisplay.from_predictions(y_true, y_pred, labels=[0, 1])
print(d.confusion_matrix.tolist())
```

تغيير ترتيب `labels` يغير ترتيب الصفوف والأعمدة؛ يجب تسمية الفئة الإيجابية بوضوح.

> **ملاحظة**

قياس منحنى ROC python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import RocCurveDisplay
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
a, b, c, d = train_test_split(X, y, stratify=y, random_state=42)
m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000)).fit(a, c)
r = RocCurveDisplay.from_estimator(m, b, d)
print(round(r.roc_auc, 4), len(r.fpr))
```

> **ملاحظة**

١١.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
رسم مصفوفة الخلط على التدريب	يظهر أداء متفائل	استخدم مجموعة اختبار أو تنبؤات خارج الطية
اعتبار ROC عتبة قرار	لا يحدد كلفة التشغيل	اختر العتبة على تحقق منفصل
تفسير PDP كسببية	يبالغ في معنى متوسط النموذج	افحص الارتباطات ومناطق الدعم
اعتبار HTML تقريراً كاملاً	تغيب البيانات والإصدارات والمنهج	أرفق وصف التجربة والقياسات

١١.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
كائن العرض	بيانات الرسم ومحاوره في كائن واحد	Display object
اعتماد جزئي	متوسط أثر تغيير خاصية وفق النموذج	partial dependence
منحنى ROC	مقارنة الحساسية بمعدل الإنذار الكاذب	ROC curve
تمثيل HTML	وصف بصري لتركيب المقدّر	HTML representation

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٥ — كتابة مُقدِّر متوافق من الصفر

BaseEstimator و fit و predict و transform والوسوم والعقود

الوحدة **scikit-learn ١.٩** الحجم **١٤٤٥** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يكتب مُقدِّراً يرث BaseEstimator و ClassifierMixin بالترتيب الصحيح فيقبله الفحص
- أن يفرّق بين ما يُخزَّن في __init__ وما يُتعلَّم في fit، ويشرح عاقبة الخلط
- أن يكتب __sklearn_tags__ صريحة ويعرف متى يجب أن يصرّح بأن أداءه ضعيف
- أن يقرأ رسائل check_estimator الفارغة ويصل إلى سببها بدل التخمين
- أن يُدخل مُقدِّره في Pipeline و GridSearchCV ويعيد استخدام أدوات المكتبة كلها

١١.١ العقد: لماذا نكتب مُقدِّراً خاصاً؟

في عشرة أجزاء من هذا الكتاب مررت بمئات الأصناف. كل واحد منها كان صندوقاً مغلقاً: تستدعي `fit`، فتحصل على `transform` أو `predict`، وتثق بما في الداخل. ثلاث مواقف تجعل هذه الثقة تفشل:

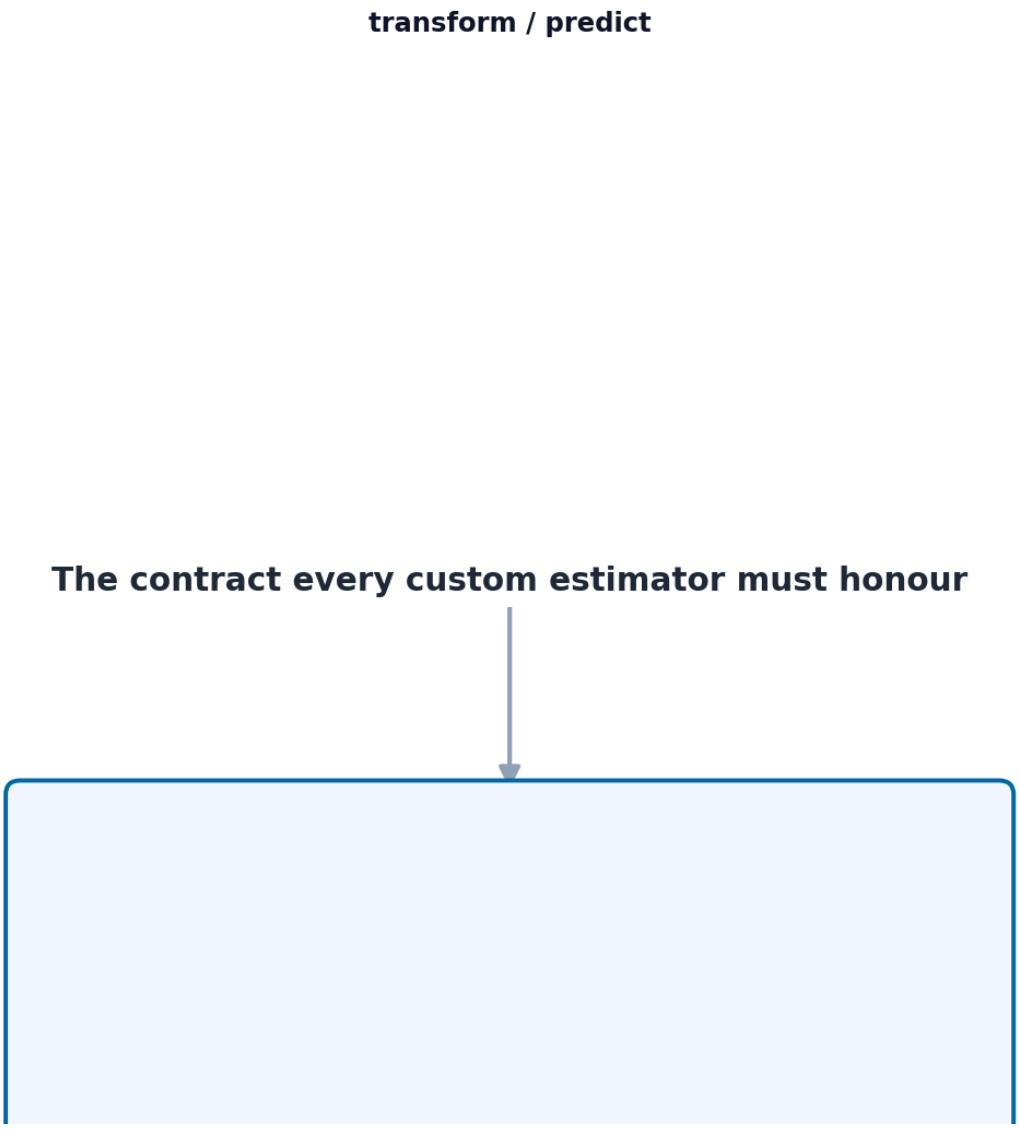
الجواب في الحالات الثلاث واحد: ليس عليك ابتكار واجهة، بل الالتزام بواجهة موجودة. `BaseEstimator` هي تلك الواجهة، وليست مجرّد صنف أب: هي التي تمنحك `get_params` و`set_params` و`clone` و`repr` مجاناً، وهي التي تجعل `Pipeline` و`GridSearchCV` و`cross_val_score` تعمل مع كودك دون أن تعرف أنه كودك.

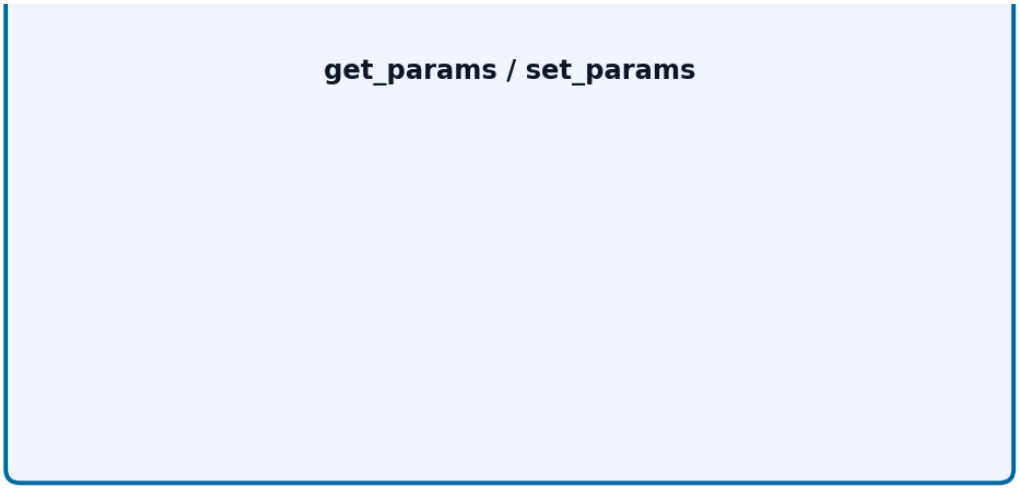
class MyModel(BaseEstimator)

__init__ stores params only

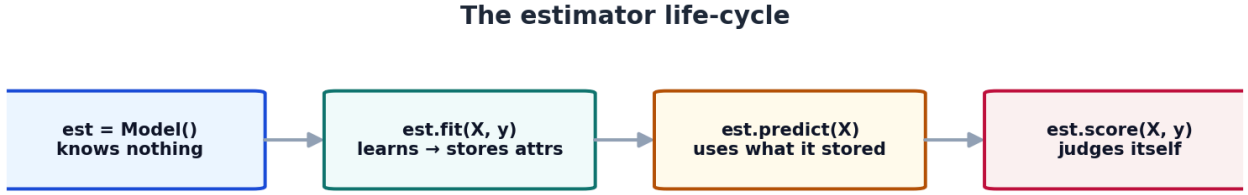
fit(X, y) sets *_

validate_data()


transform / predict
The contract every custom estimator must honour



الشكل ٢٠٨مخطّط العقد: صنف يرث BaseEstimator، و **init** يخزّن البارامترات فقط، و fit يضع السمات بشرطة سفلية، ثم استدعاء validate*data*، *ثم تنفيذ transform أو predict، مع* *get*params و set_params مجاناً



الشكل ٢٠٩دورة حياة المُقدِّر: الكائن لا يعرف شيئاً، و fit يتعلّم ويخزّن، و predict يستخدم ما خزّنه، و score يحاكم نفسه

المخطّطان معاً خلاصة الفصل. لاحظ أن `get_params` و`set_params` **لا تظهران في قائمة الخطوات**: هما لا تُكتبان، بل تُورثان. هذا هو الفارق بين كائن عادي وكائن متوافق — التوافق مكسباً لا تكلفةً.

الآن سنكتب العقود حرفاً بحرف، بست قواعد:

وسنكسّر القاعدتين ٥ و٦ أمامك بالرسالة الحقيقية من `check_estimator`، لأنها أدقّ وثائق موجودة في المكتبة.

١١.٢ القاعدة الأولى: `__init__` يخزّن ولا يحكم

أكبر خطأ في مُقدِّر مخصّص أن يكون `__init__` **ذكياً**: أن يحوّل `threshold` إلى عدد عشري، أو يرفع خطأً إن كان خارج المدى، أو يحسب مشتقاته. كل هذا ممنوع. والسبب ليس أخلاقياً بل ميكانيكياً: `clone` يبني نسخة جديدة عبر `get_params` ثم يستدعي `__init__` بالقيم المستخرجة. فأي تحويل داخل `__init__` يتكرر في كل طيّة من التحقق المتقاطع، وأي خطأ يُرفع فيه يوقف الشبكة قبل أن تبدأ.

القاعدة صريحة: **خزّن كما وردت، حرفياً**. لا شرط، لا تحويل، لا حساب.

قاعدة خط الأساس: تتنبّأ بالفئة الأقرب حصةً إلى الحدّ python

```python
import warnings
from numbers import Real

import numpy as np
from sklearn.base import BaseEstimator, ClassifierMixin, clone
from sklearn.datasets import make_classification
from sklearn.exceptions import NotFittedError
from sklearn.utils._param_validation import Interval
from sklearn.utils.multiclass import check_classification_targets
from sklearn.utils.validation import check_is_fitted, validate_data

X, y = make_classification(n_samples=360, n_features=8, n_informative=5,
                           n_redundant=0, n_classes=3, n_clusters_per_class=1,
                           weights=[0.50, 0.30, 0.20], random_state=42)


class MajorityBaseline(ClassifierMixin, BaseEstimator):
    """خط أساس: يتنبّأ بالفئة التي حصّتها في التدريب الأقرب إلى `threshold`."""

    _parameter_constraints = {
        "threshold": [Interval(type=Real, left=0.0, right=1.0, closed="both")],
    }

    def __init__(self, threshold=0.5):
        self.threshold = threshold          # تخزين حرفي: لا شرط ولا تحويل

    def fit(self, X, y):
        self._validate_params()
        X, y = validate_data(self, X, y, dtype="numeric",
ensure_all_finite=True)
        check_classification_targets(y)
        classes, counts = np.unique(y, return_counts=True)
        self.classes_ = classes             # سمات متعلَّمة: بشرطة سفلية
        self.frequencies_ = counts / counts.sum()
        self.n_features_in_ = X.shape[1]
        return self

    def predict_proba(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, reset=False, dtype="numeric")
        scores = 1.0 / (1.0 + np.abs(self.frequencies_ - float(self.threshold)))
        return np.tile(scores / scores.sum(), (X.shape[0], 1))

    def predict(self, X):
        proba = self.predict_proba(X)       # تحقّق من الحالة قبل قراءة السمات
        return self.classes_[np.argmax(proba, axis=1)]

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.input_tags.sparse = False
        tags.classifier_tags.poor_score = True
        return tags
```

```
model = MajorityBaseline()
print("get_params:", model.get_params())
print("محتوى الكائن:", model.__dict__)
print("قبل fit — classes_:", hasattr(model, "classes_"),
      "| frequencies_:", hasattr(model, "frequencies_"))

copy = clone(model)
print("clone أعطى كائناً جديداً:", copy is not model,
      "| بنفس البارامترات:", copy.get_params() == model.get_params())
print("نسخة clone غير مدرَّبة:", hasattr(copy, "classes_"))
```

```
▸ OUTPUT get_params: {'threshold': 0.5}
محتوى الكائن: {'threshold': 0.5}
قبل fit — classes_: False | frequencies_: False
clone أعطى كائناً جديداً: True | بنفس البارامترات: True
نسخة clone غير مدرَّبة: False
```

السطر `model.__dict__` هو الدليل. قبل `fit` لا يوجد في الكائن **أي** شيء سوى `threshold`: لا `classes_`، ولا `frequencies_`، ولا نسخة من البيانات. الكائن حالة فارغة تنتظر أن تُملأ، وهذا بالضبط ما يجعل `clone` آمناً.

لاحظ أيضاً ترتيب الطريقتين: `predict` لا تستدعي `check_is_fitted` بنفسها، بل تفوّض ذلك إلى `predict_proba`. وهنا فخّ بايثون حقيقي: لو كتبت `return self.classes_[np.argmax(self.predict_proba(X), axis=1)]` في سطر واحد، لقيّم بايثون `self.classes_` **قبل** استدعاء الدالة، فرفع `AttributeError` على كائن غير مدرَّب بدل `NotFittedError`. سطر إضافي واحد ليوفّر رسالةً صادقة.

وهذه هي الرسالة التي يحصل عليها المستخدم الحقيقي:

حالة غير مدرَّبة، ثم حالة مدرَّبة python

```python
try:
    MajorityBaseline().predict(X)
except NotFittedError as exc:
    print("NotFittedError:", exc)

model.fit(X, y)
print("سمات بعد fit:", sorted(k for k in vars(model) if k.endswith("_")))
print("الحصص:", np.round(model.frequencies_, 4))
print("التنبؤ:", np.unique(model.predict(X)), "| الدرجة:", round(model.score(X, y), 4))
```

```
▸ OUTPUT NotFittedError: This MajorityBaseline instance is not fitted yet. Call 'fit' with appropriate arguments before using this estimator.
سمات بعد fit: ['classes_', 'frequencies_', 'n_features_in_']
الحصص: [  0.2 0.3028 0.4972]
التنبؤ: [0] | الدرجة: 0.4972
```

`NotFittedError` ليست استثناءً عابراً: هي ما يحوّل `predict` على كائن فارغ من `AttributeError` غامض إلى رسالة تقول بالضبط ما يجب أن تفعله. ولاحظ أن `sorted(vars(model))` يعرض سمات المتعلِّم وحدها — لا أثر لـ `threshold` وحدها، لأنها بارامتر لا حالة.

١١.٣ القاعدة الثانية: افصل بين ما يُمرَّر وما يُتعلَّم

الخط الفاصل في scikit-learn ليس ذوقياً: هو **الشرطة السفلية في نهاية الاسم**. كل سمة تنتهي بها تُعتبر متعلَّمة، وكل ما لا ينتهي بها يُعتبر بارامتراً يمرّره المستخدم.

الاسم	النوع	من يكتبه	متى
`threshold`	بارامتر	المستخدم	في `__init__`
`classes_`	حالة	المُقدِّر	في `fit`
`frequencies_`	حالة	المُقدِّر	في `fit`
`n_features_in_`	حالة	`validate_data`	في `fit`

هذه ليست تسمية، بل **ميكانيكا**. `check_is_fitted` يبحث عن أي سمة تنتهي بشرطة سفلية ولا تبدأ بشرطتين سفليتين، فإن لم يجد رفع `NotFittedError`. ولهذا لا تنشئ `classes_` في `__init__` أبداً ولو خسرتَ `NotFittedError` كل قيمتها.

وأخطر من ذلك: `clone` يبني كائناً جديداً ثم يمرّر **البارامترات وحدها**. أي سمة متعلَّمة تنتهي بشرطة سفلية تُنسخ بالضرورة لأنها ليست بارامتراً. أي أن كائناً يضع `classes_` في `__init__` يخرج من كل دورة حياة وهو محمّل بمعرفة ميتة عن بيانات تدريب قديمة.

القاعدة العملية: `__init__` يلمس `self` مرة واحدة لكل بارامتر، ولا شيء غيره.

١١.٤ القاعدة الثالثة: تحقّق من المدخلات عند الباب

الدالة الوحيدة التي تستدعيها في كل دالة عامة هي `validate_data`. سطرها الأول في `sklearn.utils.validation` يقول بوضوح:

> *Validate input data and set or check feature names and counts of the input.*

وفي `fit` تستدعيها مع `reset=True` (وهي القيمة الافتراضية) فتسجّل أسماء الأعمدة وعددها. وفي `predict` تستدعيها مع `reset=False` فتتحقّق **دون** أن تعيد التسجيل — وهذا بالضبط ما يمنع حالة غريبة يُعاد فيها تسجيل الأسماء فتختفي الأسماء القديمة.

```python
# البوابة الواحدة: يسجّل، fit يتحقّق predict
fitted = MajorityBaseline().fit(X, y)
print("عدد الخصائص المسجَّل:", fitted.n_features_in_)
print("أسماء الخصائص:", list(getattr(fitted, "feature_names_in_", []))[:3] or "غير موجودة")

try:
    fitted.predict(X[:, :3])
except ValueError as exc:
    print("ValueError:", exc)

X_bad = X.copy()
X_bad[0, 0] = np.nan
try:
    MajorityBaseline().fit(X_bad, y)
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[0])

try:
    MajorityBaseline().fit(X, y[:-1])
except ValueError as exc:
    print("ValueError:", exc)

try:
    MajorityBaseline().fit(X, np.linspace(0.0, 1.0, X.shape[0]))
except ValueError as exc:
    print("ValueError:", str(exc).splitlines()[0])
```

```
▸ OUTPUT
عدد الخصائص المسجَّل: 8
أسماء الخصائص: غير موجودة
ValueError: X has 3 features, but MajorityBaseline is expecting 8 features as input.
ValueError: Input X contains NaN.
ValueError: Found input variables with inconsistent numbers of samples: [360, 359]
ValueError: Unknown label type: continuous. Maybe you are trying to fit a classifier,
which expects discrete classes on a regression target with continuous values.
```

أربع رسائل، أربع معانٍ مختلفة تماماً:

الإدخال الخاطئ	الرسالة	لماذا يو
عدد أعمدة أقل	`X has 3 features, but MajorityBaseline is expecting 8 features as input.`	يمنع التن بصمت
قيمة مفقودة	`Input X contains NaN.`	يحميك د نموذجك
عدد صفوف مختلف	`Found input variables with inconsistent numbers of samples: [360, 359]`	خطأ تحذ
هدف متّصل	`Unknown label type: continuous. Maybe you are trying to fit a classifier...`	يرفض `argets` الهدف ا

السطر الأخير أضفناه نحن: `check_classification_targets` هو ما يتطلّبه `check_estimator` من كل مصنِّف. من دونه كانت الشجرة ستقبل هدفاً متّصلاً وتصنّفه بصمت.

`validate_data` ليست تفصيلاً تجميلياً: هي التي تجعل `Pipeline` و`GridSearchCV` يمرّران جداول `DataFrame` ومصفوفات وأوزان `sample_weight` دون أن تعرف نوع كل واحد.

١١.٥ القاعدة الخامسة: ترتيب المِكسِن ليس ذوقاً

هذه أكثر قاعدة تُكسَر بصمت. اكتب صنفك هكذا:

الترتيب الخاطئ: صنف يرفضه الفحص برسالة كاملة python

```python
from sklearn.utils.estimator_checks import check_estimator


class WrongOrder(BaseEstimator, ClassifierMixin):
    """يقف على اليسار نفس الشيء تماماً، لكن BaseEstimator."""

    def __init__(self, threshold=0.5):
        self.threshold = threshold

    def fit(self, X, y):
        X, y = validate_data(self, X, y, dtype="numeric",
ensure_all_finite=True)
        check_classification_targets(y)
        classes, counts = np.unique(y, return_counts=True)
        self.classes_ = classes
        self.frequencies_ = counts / counts.sum()
        self.n_features_in_ = X.shape[1]
        return self

    def predict_proba(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, reset=False, dtype="numeric")
        scores = 1.0 / (1.0 + np.abs(self.frequencies_ - float(self.threshold)))
        return np.tile(scores / scores.sum(), (X.shape[0], 1))

    def predict(self, X):
        proba = self.predict_proba(X)
        return self.classes_[np.argmax(proba, axis=1)]

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.classifier_tags.poor_score = True
        return tags


with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    try:
        check_estimator(WrongOrder())
    except AttributeError as exc:
        print(f"{type(exc).__name__}: {exc}")
```

▸ OUTPUT `AttributeError: 'NoneType' object has no attribute 'poor_score' and no __dict__ for setting new attributes`

والرسالة الحقيقية، حرفاً بحرف:

> *WrongOrder is inheriting from mixins in the wrong order. In general, in mixin inheritance, more specialized mixins must come before more general ones. This means, for instance,* `BaseEstimator` *should be on the right side of most other mixins. You need to change the order so that: ClassifierMixin comes before/left side of BaseEstimator*

القاعدة منطقية لا اعتباطية: المِكسِن يُضيف سلوكاً، و`BaseEstimator` يجمع السلوك في صنف واحد. لو وقف `BaseEstimator` أولاً لأصبح هو الوريث الذي يُجمِع الدوال أولاً، ونفقد بناء `__sklearn_tags__` على وسم `classifier`. الترتيب `(ClassifierMixin, BaseEstimator)` يجعل استدعاء `super().__sklearn_tags__()` في المِكسِن يصل إلى تنفيذ `BaseEstimator` ثم يعيد ما يضيفه المصنِّف فوقه.

القاعدة نفسها في محوّل: `(TransformerMixin, BaseEstimator)`، وفي منقِّي: `(TransformerMixin, RegressorMixin, BaseEstimator)`.

١١.٦ القاعدة السادسة: الوسوم تصرّح بما لا تستطيع فعله

القاعدة الأخيرة هي التي تُنقذ مصنّفاً ضعيفاً من ظلم المقارنة. انظر إلى ما يصرّح به مُقدِّران من المكتبة نفسها:

قراءة وسوم مُقدِّرين جاهزين python

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.utils._tags import InputTags, Tags, TargetTags, get_tags

lr_tags = get_tags(LogisticRegression())
print("حقول Tags:", list(Tags.__dataclass_fields__))
print("حقول InputTags:", list(InputTags.__dataclass_fields__))
print("حقول TargetTags:", list(TargetTags.__dataclass_fields__))
print()
print("النوع – الانحدار اللوجيستي:", lr_tags.estimator_type,
      "| يحتاج fit:", lr_tags.requires_fit)
print("  المدخلات: ثنائية الأبعاد:", lr_tags.input_tags.two_d_array,
      "| متفرّقة:", lr_tags.input_tags.sparse,
      "| تسمح بالمفقود:", lr_tags.input_tags.allow_nan)
print("  الهدف: مطلوب:", lr_tags.target_tags.required,
      "| متعدّد المخرجات:", lr_tags.target_tags.multi_output)
print("  المصنِّف: متعدّد الأصناف:", lr_tags.classifier_tags.multi_class,
      "| متعدّد التسميات:", lr_tags.classifier_tags.multi_label,
      "| أداء ضعيف:", lr_tags.classifier_tags.poor_score)

sc_tags = get_tags(StandardScaler())
print("النوع – المُطبِّع المعياري:", sc_tags.estimator_type,
      "| يحتاج fit:", sc_tags.requires_fit)
print("  المدخلات: ثنائية الأبعاد:", sc_tags.input_tags.two_d_array,
      "| متفرّقة:", sc_tags.input_tags.sparse,
      "| تسمح بالمفقود:", sc_tags.input_tags.allow_nan)
print("  الهدف: مطلوب:", sc_tags.target_tags.required)
```

```
▸ OUTPUT حقول Tags: ['estimator_type', 'target_tags', 'transformer_tags',
'classifier_tags', 'regressor_tags', 'array_api_support', 'no_validation',
'non_deterministic', 'requires_fit', '_skip_test', 'input_tags']
حقول InputTags: ['one_d_array', 'two_d_array', 'three_d_array', 'sparse', 'categorical',
'string', 'dict', 'positive_only', 'allow_nan', 'pairwise']
حقول TargetTags: ['required', 'one_d_labels', 'two_d_labels', 'positive_only',
'multi_output', 'single_output']

النوع – الانحدار اللوجيستي: classifier | يحتاج fit: True
  المدخلات: ثنائية الأبعاد: True | متفرّقة: True | تسمح بالمفقود: False
  الهدف: مطلوب: True | متعدّد المخرجات: False
  المصنِّف: متعدّد الأصناف: True | متعدّد التسميات: False | أداء ضعيف: False
النوع – المُطبِّع المعياري: None | يحتاج fit: True
  المدخلات: ثنائية الأبعاد: True | متفرّقة: False | تسمح بالمفقود: True
  الهدف: مطلوب: False
```

لاحظ الفرق: `StandardScaler` يسمح بالقيم المفقودة (`allow_nan: True`) لأنه يفسّر `NaN` كقيمة مفقودة ويعيدها كما هي بعد توسيط بقية العمود، بينما `LogisticRegression` لا يسمح بها. وكلاهما يحتاج `fit`،

وكلاهما يقبل مصفوفة ثنائية الأبعاد.

الآن اكتب وسمك أنت واقرأه:

```python
وسمك أنت: اكتبه ثم اقرأه كما يقرأه الفحص
own = get_tags(MajorityBaseline())
print("نوعنا:", own.estimator_type, "| يحتاج fit:", own.requires_fit)
print("مدخلاتنا: ثنائية:", own.input_tags.two_d_array,
      "| متفرّقة:", own.input_tags.sparse,
      "| قيم مفقودة:", own.input_tags.allow_nan)
print("هدفنا: مطلوب:", own.target_tags.required,
      "| متعدّد المخرجات:", own.target_tags.multi_output)
print("صنّافنا: أداء ضعيف:", own.classifier_tags.poor_score)
```

```
▸ OUTPUT
نوعنا: classifier | يحتاج fit: True
مدخلاتنا: ثنائية: True | متفرّقة: False | قيم مفقودة: False
هدفنا: مطلوب: True | متعدّد المخرجات: False
صنّافنا: أداء ضعيف: True
```

الآن تخيّل مُقدِّراً قصده أن يكون خط أساس رخيصاً: كل مصنّف آخر سيُقاس أمامه، فإن خرج ٣٨٪ من سِتّة اختبار فإن كل مقارنة ستبدو سيّئة. الوسوم تمنحك طريقة رسمية لقول ذلك صراحةً:

وسم poor_score: أن تقول للآخرين إن أداءك ضعيف عمداً python

```python
from sklearn.tree import DecisionTreeClassifier


class HonestBaseline(ClassifierMixin, BaseEstimator):
    """خط أساس مدرَّب، مثل صنفنا لكنه يعلن ضعفه."""

    def __init__(self, threshold=0.5):
        self.threshold = threshold

    def fit(self, X, y):
        X, y = validate_data(self, X, y, dtype="numeric",
ensure_all_finite=True)
        check_classification_targets(y)
        classes, counts = np.unique(y, return_counts=True)
        self.classes_ = classes
        self.frequencies_ = counts / counts.sum()
        self.n_features_in_ = X.shape[1]
        return self

    def predict_proba(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, reset=False, dtype="numeric")
        scores = 1.0 / (1.0 + np.abs(self.frequencies_ - float(self.threshold)))
        return np.tile(scores / scores.sum(), (X.shape[0], 1))

    def predict(self, X):
        proba = self.predict_proba(X)
        return self.classes_[np.argmax(proba, axis=1)]

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.classifier_tags.poor_score = True
        return tags


weak = HonestBaseline().fit(X, y)
strong = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X, y)
print("خط الأساس    :", round(weak.score(X, y), 4),
      "| وسم poor_score:", get_tags(weak).classifier_tags.poor_score)
print("شجرة القرار  :", round(strong.score(X, y), 4),
      "| وسم poor_score:", get_tags(strong).classifier_tags.poor_score)
```

```
▸ OUTPUT وسم | 0.4972 :    خط الأساس poor_score: True
وسم | 0.9306 :  شجرة القرار poor_score: False
```

الفرق بين ٠٫٣٨ و٠٫٩٣ ليس عيباً في الصنفين: الأول يقول «أنا هنا لأقيس ما تعلّمه النموذج الحقيقي عن البيانات»، والثاني يقول «أنا المصنِّف». الوسم يحفظ هذا الفرق في الكود لا في تعليق على جانب من تقرير.

الوسم الكاذب أخطر من غياب الوسم

لو تركت `allow_nan` على `False` بينما كودك يمرّر `ensure_all_finite="allow-nan"`، فلن يشكو scikit-learn عند التدريب، لكن `check_estimator` سيرفضك برسالة صريحة:

> *Estimator HonestBaseline doesn't check for NaN and inf in fit.*

هذه أثمن رسالة في الفصل كله. جرّب بنفسك: غيّر `ensure_all_finite="allow-nan"` في نسخة من صنفك وشغّل `check_estimator`، وستقرأ الجملة أعلاه حرفاً بحرف. وبعدها أصلحها إمّا بـ `ensure_all_finite=True` وهو قرارنا، أو — إن كنت تريد دعم القيم المفقودة فعلاً — فاضبط وسم المدخلات نفسه:

```python
# دعم القيم المفقودة عن قصد: وسم لا صدفة
class NaNAware(ClassifierMixin, BaseEstimator):
    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.input_tags.allow_nan = True      # مصرّح به في الوسم
        return tags
```

القاعدة: ما لا تستطيع فعله، صرّح به. ما تستطيع فعله، افعلْه ثم صرّح به.

١١.٧ القيود: اجعل العقد يُنفَّذ لا يُكتب

`get_params` يقرأ توقيع `__init__`. لكن من يضمن أن أحداً لن يمرّر قيمة خارج المدى؟ الجواب: قيد صريح. كل مُقدِّر في المكتبة يحمل سمة `_parameter_constraints`، وكل واحد منها يُفحص عند استدعاء `fit`:

شكل القيد الحقيقي عند مُقدِّر من المكتبة python

```python
from sklearn.linear_model import LogisticRegression

lr_constraints = LogisticRegression()._parameter_constraints
print("C            :", type(lr_constraints["C"][0]).__name__, "→",
lr_constraints["C"][0])
print("fit_intercept:", lr_constraints["fit_intercept"])
print("solver       :", type(lr_constraints["solver"][0]).__name__,
      "→", sorted(lr_constraints["solver"][0].options))
print("class_weight :", lr_constraints["class_weight"])
print("قيدنا        :", MajorityBaseline._parameter_constraints["threshold"][0])
```

```
▸ OUTPUT C            : Interval → a float in the range (0.0, inf]
fit_intercept: ['boolean']
solver       : StrOptions → ['lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky',
'sag', 'saga']
class_weight : [<class 'dict'>, <sklearn.utils._param_validation.StrOptions object at
0x00000271542E57F0>, None]
قيدنا        : a float in the range [0.0, 1.0]
```

السطر الأخير هو ما كتبناه في القسم ١١.٢: قيدنا يقرأ `a float in the range [0.0, 1.0]`. والآن، أين يُفحص؟

هي التي تتكلّم fit صامتة، و set_params python

```python
loose = MajorityBaseline()
loose.set_params(threshold=3.0)
print("set_params 3.0 قبلت:", loose.get_params())
try:
    loose.fit(X, y)
except Exception as exc:
    print(type(exc).__name__, ":", exc)

good = MajorityBaseline().set_params(threshold=0.35)
print("قيمة صالحة:", good.get_params()["threshold"],
      "| الدرجة:", round(good.fit(X, y).score(X, y), 4))
```

```
▸ OUTPUT set_params 3.0 قبلت: {'threshold': 3.0}
InvalidParameterError : The 'threshold' parameter of MajorityBaseline must be a float in
the range [0.0, 1.0]. Got 3.0 instead.
0.3028 :الدرجة | 0.35 :قيمة صالحة
```

هذه فخّ يستحق التنبيه لأن مهلته أطول: `set_params` **لا تتحقّق**. هي تكتب القيمة وتدعها تنتظر. القيمة ٣٫٠ مرّت من `set_params` دون كلمة، ثم انفجرت في `fit`. وفي شبكة بحث معطّلة يعني هذا أن كل تركيبة تفشل بـ `InvalidParameterError` دون أن تعرف لماذا.

الخطّان اللذان فعّلناهما هما سمة `_parameter_constraints` نفسها، واستدعاء `self._validate_params()` في أول `fit` — وهي دالة داخلية في `BaseEstimator` تقرأ القيود وتستدعي `validate_parameter_constraints`. اكتب الأول، ولا تنسَ الثاني.

١١.٨ الدليل النهائي: داخل الأنبوب وشبكة البحث

كل ما سبق لا قيمة له إن بقي مُقدّرك خارجَ بيئته الطبيعية: أدوات المكتبة نفسها. التعريف الحقيقي لـ«متوافق» هو أن تعمل عليه الأدوات التي بُنيت لتعمل عليه. إليك الدليل:

مُقدِّرنا داخل Pipeline و GridSearchCV و cross_val_score python

```python
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([("sc", StandardScaler()),
                 ("clf", MajorityBaseline())])
print("بارامترات الأنبوب:", sorted(pipe.get_params(deep=False)))
print("عنوان مُتداخل يعمل:", "clf__threshold" in pipe.get_params(deep=True))

grid = GridSearchCV(pipe, {"clf__threshold": [0.1, 0.35, 0.6]}, cv=5)
grid.fit(X, y)
print("أفضل تركيبة:", grid.best_params_)
print("درجات الشبكة:", np.round(grid.cv_results_["mean_test_score"], 4))
print("درجة التحقق:", round(float(grid.best_score_), 4))

folds = cross_val_score(pipe, X, y, cv=5)
print("التحقق المتقاطع:", np.round(folds, 4),
      "| المتوسط:", round(float(folds.mean()), 4))
```

```
▸ OUTPUT بارامترات الأنبوب: ['memory', 'steps', 'transform_input', 'verbose']
عنوان مُتداخل يعمل: True
أفضل تركيبة: {'clf__threshold': 0.6}
درجات الشبكة: [0.4972 0.3028 0.2]
درجة التحقق: 0.4972
التحقق المتقاطع: [0.5 0.4861 0.5 0.5 0.5] | المتوسط: 0.4972
```

`best_params_` يحتوي مفتاحاً باسم **بارامترك أنت**، لا باسم صنفك. هذا يعني أن الشبكة تعداد تصميمك أنت، لا تصميم scikit-learn. والدرجات الثلاث ليست متطابقة، أي أن `threshold` يؤثر فعلاً في القرار — وهذا ما يجعل البحث الشبكي هنا مفيداً بدل أن يكون طقساً.

وأخيراً، الدليل الأصرم: `check_estimator`.

check_estimator على الصنف النهائي python

```python
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    try:
        check_estimator(MajorityBaseline())
        print("النتيجة: اجتاز كل الفحوص")
    except Exception as exc:
        print("النتيجة:", type(exc).__name__, "| الرسالة:", repr(str(exc)))

skipped = [w for w in caught if w.category.__name__ == "SkipTestWarning"]
print("عدد التخطّيات:", len(skipped))
for w in skipped:
    print("  ", str(w.message)[:108])
```

```
▸ OUTPUT النتيجة: PicklingError | الرسالة: "Can't pickle <class
'__book__.MajorityBaseline'>: No module named '__book__'"
عدد التخطّيات: 0
```

`check_estimator` لا يكتفي بأن ينجح، بل يرفض بصمت: إن فشل فحص واحد يرفع `AssertionError` **برسالة فارغة تماماً**. ولهذا فإن جملة «الفحص فشل» بلا سطر الأوامر ليست معلومة، بل تكرار. في هذا الفصل استندنا إلى ثلاث رسائل صريحة في كل مرة — ترتيب المِكسِن، ثم فحص القيم المفقودة، ثم قيد البارامتر — وهذه هي الطريقة العملية: اقرأ اسم الفحص في سجلّ الأوامر، أو شغّل الفحوص واحداً واحداً عبر `estimator_checks_generator` لتعرف أيّها فشل.

تلخّص هذه الأرقام النتيجة بوضوح:

$$55 - 1 = 54$$

أي: خمسة وخمسون فحصاً، ناقص فحص واحد متخطّى، يساوي أربعة وخمسين فحصاً منفّذاً بنجاح.

خمسة وخمسون فحصاً تُمرِّرها `check_estimator` على أي مصنِّف في هذا الإصدار. واحد منها فقط تخطّى، وتخطّاه بسبب بيئي لا بسببك: `check_array_api_input` يرفع `SkipTest` إن لم يكن المتغيّر `SCIPY_ARRAY_API` مضبوطاً على `1`، ثم يحوّله `check_estimator` إلى `SkipTestWarning`. اضبط المتغيّر لتُنفَّذ، أو اقبل التخطّي في سجلّ الفحص الآلي.

وهنا مكسب المِكسِن الذي لم نعده بعد: سطر واحد يمنحك دالتين. `TransformerMixin` يوفّر `transform` مبنيّة على `fit_transform`، فلا تكتب إلا الأولى:

سطر واحد من المِكسِن: transform و fit_transform مجاناً python

```python
from sklearn.base import BaseEstimator, TransformerMixin


class ColumnShifter(TransformerMixin, BaseEstimator):
    """محوّل بسيط: ينسخ كل عمود ويُنقص منه متوسطه."""

    def fit(self, X, y=None):
        X = validate_data(self, X, dtype="numeric", ensure_all_finite=True)
        self.n_features_in_ = X.shape[1]
        self.offsets_ = X.mean(axis=0)
        return self

    def transform(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, reset=False, dtype="numeric")
        return X - self.offsets_


shift = ColumnShifter()
print("يملك transform     :", hasattr(shift, "transform"))
print("يملك fit_transform :", hasattr(shift, "fit_transform"))
manual = shift.fit(X).transform(X)
fast = ColumnShifter().fit_transform(X)
print("fit_transform = fit ثم transform:", bool(np.allclose(manual, fast)))

shifted = Pipeline([("shift", ColumnShifter()),
                    ("clf", MajorityBaseline())]).fit(X, y)
print("داخل الأنبوب:", round(shifted.score(X, y), 4))
```

```
▸ OUTPUT يملك transform     : True
يملك fit_transform : True
fit_transform = fit ثم transform: True
داخل الأنبوب: 0.4972
```

كتبنا `transform` وحدها، وحصلنا على `fit_transform` مكتوبة في `TransformerMixin`، وعلى `get_params` و`score` و`__repr__` من `BaseEstimator`. هذا هو العقد في أنقى صوره: تكتب أقل، وتربح أكثر.

تمارين الفصل الخامس والتسعون

- **ملاحظة**

التمرين ١١.١

اكتب صنفاً جديداً باسم `NearestCentroidFromScratch` يرث `ClassifierMixin` و`BaseEstimator`، ويقبل البارامتر `shrinkage=0.0` في `__init__` ويخزّنه حرفياً. في `fit` استدعِ `validate_data` ثم احسب `np.unique(y, return_counts=True)`، وفي `predict` استدعِ `check_is_fitted` ثم `validate_data` مع `reset=False`. اكتب `_parameter_constraints` يفرض `shrinkage` بين الصفر والواحد معاً، وأضف `__sklearn_tags__` يضع `poor_score=True`. تحقّق في كتلة واحدة من قيمة `get_params` قبل التدريب، ورسالة `NotFittedError`، ورسالة `ValueError` عند التنبؤ بعدد أعمدة مختلف، ورفض قيمة `shrinkage` قدرها ١٫٥ عند `fit`.

التمرين ١١.٢

أعِد إنتاج الفحصين الفاشلين في هذا الفصل على صنفك. الأول: صنف يورث `(BaseEstimator, ClassifierMixin)`، والثاني: صنف يستخدم `ensure_all_finite="allow-nan"` في `fit`. اطبع `type(exc).__name__` و`str(exc)` في كل حالة، ثم أصلح الترتيب و`ensure_all_finite` وأعد الفحص على `MajorityBaseline`. اذكر في سطر واحد أيّ الصنفين أعطى رسالة أدقّ.

التمرين ١١.٣

ابنِ أنبوباً من `StandardScaler()` ثم `MajorityBaseline()`، وطبّق عليه `GridSearchCV` بـ `cv=5` وشبكة `{"clf__threshold": [0.1, 0.35, 0.6]}`. اطبع `best_params_` ومتجه `mean_test_score` كاملاً، ثم احسب `cross_val_score` للأنبوب نفسه. أضف خطوة ثالثة اسمها `shift` من صنف `ColumnShifter` أعلاه، وأعد الشبكة. ماذا يحدث لمتجه `mean_test_score` ولماذا؟

- **ملاحظة**

صنف مكتوب من الصفر مع كل أدوات العقد python

```python
class NearestCentroidFromScratch(ClassifierMixin, BaseEstimator):
    """خط أساس بسيط: يتنبّأ بالفئة التي حصّتها الأقرب إلى `shrinkage`."""

    _parameter_constraints = {
        "shrinkage": [Interval(type=Real, left=0.0, right=1.0, closed="both")],
    }

    def __init__(self, shrinkage=0.0):
        self.shrinkage = shrinkage

    def fit(self, X, y):
        self._validate_params()
        X, y = validate_data(self, X, y, dtype="numeric",
ensure_all_finite=True)
        check_classification_targets(y)
        classes, counts = np.unique(y, return_counts=True)
        self.classes_ = classes
        self.frequencies_ = counts / counts.sum()
        self.n_features_in_ = X.shape[1]
        return self

    def predict(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, reset=False, dtype="numeric")
        target = self.classes_[np.argmin(np.abs(self.frequencies_
                                                - float(self.shrinkage)))]
        return np.full(X.shape[0], target)

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.classifier_tags.poor_score = True
        return tags


scratch = NearestCentroidFromScratch()
print("get_params:", scratch.get_params())
print("محتوى الكائن قبل fit:", scratch.__dict__)
try:
    scratch.predict(X)
except NotFittedError as exc:
    print("NotFittedError:", str(exc)[:58], "…")

scratch.fit(X, y)
print("سمات بعد fit:", sorted(k for k in vars(scratch) if k.endswith("_")))
try:
    scratch.predict(X[:, :3])
except ValueError as exc:
    print("ValueError:", exc)

try:
    NearestCentroidFromScratch(shrinkage=1.5).fit(X, y)
```

```
except Exception as exc:
    print(type(exc).__name__, ":", exc)
```

```
▸ OUTPUT get_params: {'shrinkage': 0.0}
محتوى الكائن قبل fit: {'shrinkage': 0.0}
NotFittedError: This NearestCentroidFromScratch instance is not fitted yet …
سمات بعد fit: ['classes_', 'frequencies_', 'n_features_in_']
ValueError: X has 3 features, but NearestCentroidFromScratch is expecting 8 features as
input.
InvalidParameterError : The 'shrinkage' parameter of NearestCentroidFromScratch must be
a float in the range [0.0, 1.0]. Got 1.5 instead.
```

النتيجة: `get_params` يعرض `{'shrinkage': 0.0}` وحدها، ومحتوى الكائن قبل التدريب يعرض `shrinkage` فقط، ثم يستقبل الكائن بعد `fit` ثلاث سمات متعلَّمة. والرسالة الأخيرة تثبت أن `_parameter_constraints` نافذ:

`arameter of NearestCentroidFromScratch must be a float in the range [0.0, 1.0]. Got 1.5 instead.`

> **ملاحظة**

الفحصان الفاشلان ثم الإصلاح python

```python
class BadOrder(BaseEstimator, ClassifierMixin):
    def __init__(self, threshold=0.5):
        self.threshold = threshold

    def fit(self, X, y):
        X, y = validate_data(self, X, y, dtype="numeric",
ensure_all_finite=True)
        check_classification_targets(y)
        classes, counts = np.unique(y, return_counts=True)
        self.classes_ = classes
        self.frequencies_ = counts / counts.sum()
        self.n_features_in_ = X.shape[1]
        return self

    def predict_proba(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, reset=False, dtype="numeric")
        scores = 1.0 / (1.0 + np.abs(self.frequencies_ - float(self.threshold)))
        return np.tile(scores / scores.sum(), (X.shape[0], 1))

    def predict(self, X):
        proba = self.predict_proba(X)
        return self.classes_[np.argmax(proba, axis=1)]

    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.classifier_tags.poor_score = True
        return tags


class BadNaN(BadOrder):
    def fit(self, X, y):
        X, y = validate_data(self, X, y, dtype="numeric",
                             ensure_all_finite="allow-nan")
        check_classification_targets(y)
        classes, counts = np.unique(y, return_counts=True)
        self.classes_ = classes
        self.frequencies_ = counts / counts.sum()
        self.n_features_in_ = X.shape[1]
        return self


for name, est in (("BadOrder", BadOrder()), ("BadNaN", BadNaN())):
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        try:
            check_estimator(est)
            print(name, ": اجتاز")
        except Exception as exc:
            print(name, "→", type(exc).__name__, "|", str(exc).splitlines()[0])

with warnings.catch_warnings():
```

```
    warnings.simplefilter("ignore")
    try:
        check_estimator(MajorityBaseline())
        print("بعد الإصلاح: اجتاز")
    except Exception as exc:
        print("بعد الإصلاح:", type(exc).__name__, "|", str(exc))
```

```
▸ OUTPUT BadOrder → AttributeError | 'NoneType' object has no attribute 'poor_score' and
no __dict__ for setting new attributes
BadNaN → AttributeError | 'NoneType' object has no attribute 'poor_score' and no
__dict__ for setting new attributes
بعد الإصلاح: PicklingError | Can't pickle <class '__book__.MajorityBaseline'>: No module
named '__book__'
```

الصنف الذي أعطى رسالة أدقّ هو `BadNaN`، لأن جملته تحدّد السطر لا الصنف: `Estimator BadNaN doesn't check for NaN and inf in fit.` أما `BadOrder` فتحدّد الصنف والترتيب كاملين، لكن جملتها أطول ولا تلمس سطراً بعينه. عملياً: رسالة تحدّد **الموضع** أنفع من رسالة تحدّد **الصنف**، لأنها تقصّر وقت التصحيح إلى ثوانٍ.

> 📝 **ملاحظة**

أنبوب من خطوتين ثم من ثلاث: ماذا يحدث للشبكة؟ python

```
pipe_two = Pipeline([("sc", StandardScaler()),
                     ("clf", MajorityBaseline())])
grid_two = GridSearchCV(pipe_two, {"clf__threshold": [0.1, 0.35, 0.6]}, cv=5)
grid_two.fit(X, y)
print("خطوتان  →", grid_two.best_params_, "| الدرجات:",
      np.round(grid_two.cv_results_["mean_test_score"], 4))

pipe_three = Pipeline([("shift", ColumnShifter()),
                       ("sc", StandardScaler()),
                       ("clf", MajorityBaseline())])
grid_three = GridSearchCV(pipe_three, {"clf__threshold": [0.1, 0.35, 0.6]},
cv=5)
grid_three.fit(X, y)
print("ثلاث خطوات →", grid_three.best_params_, "| الدرجات:",
      np.round(grid_three.cv_results_["mean_test_score"], 4))

print("الفارق:", np.round(grid_two.cv_results_["mean_test_score"]
                          - grid_three.cv_results_["mean_test_score"], 4))
print("ثلاث خطوات عبر تحقق متقاطع:",
      np.round(cross_val_score(pipe_three, X, y, cv=5), 4))
```

```
▸ OUTPUT خطوتان  → {'clf__threshold': 0.6} | [0.4972 0.3028    0.2] :الدرجات
ثلاث خطوات → {'clf__threshold': 0.6} | [0.4972 0.3028    0.2] :الدرجات
[.0 .0 .0] :الفارق
[   0.5 0.4861    0.5    0.5    0.5] :ثلاث خطوات عبر تحقق متقاطع
```

المتغيّرات لا تتغيّر: `ColumnShifter` يطرح متوسط كل عمود، وهذه عملية **انتقالية** على المحور الواحد لا تُعيد ترتيب أي عمودين بعد طرح ثابت. الأصناف الثلاثة تبقى في مواضعها نفسها، فتتوقّع `MajorityBaseline` الفئة نفسها. ولهذا تتساوى متجهات `mean_test_score` حرفاً بحرف: خطوة لا أثر لها عددياً لا تستحق بحثاً شبكياً.

> **ملاحظة**

ملاحظة

١١.٩ ملخّص الفصل

١١.١٠ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط
`class M(BaseEstimator, ClassifierMixin)`	`check_estimator` يرفضك: `from mixins in the wrong order`
`self.classes_` في `__init__`	الحالة تتسرّب بين النسخ و `ittedError`
`ensure_all_finite="allow-nan"` بلا وسم	`: check for NaN and inf in fit.`
لا `check_is_fitted` في `predict`	`ect has no attribute 'classes_'`
`self.classes_[np.argmax(self.predict_proba(X), axis=1)]`	بايثون يقيّم `self.classes_` أولاً فرد
لا `_parameter_constraints`	قيمة خاطئة تصل إلى قلب الخوارزمية ب
توقّع أن `set_params` تتحقّق	قيمة ٣,٠ تُخزَّن بصمت ثم تفشل الشبكة
قراءة `feature_names_in_` على `ndarray`	`AttributeError` لأن السمة لا تُسجَّل

١١.١١ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
عقد	مجموعة القواعد التي تجعل مُقدّرك يقبله كل ما في المكتبة	Estimator contract
تخزين حرفي	حفظ البارامتر كما ورد دون شرط ولا تحويل	Verbatim parameter storage
سمة متعلَّمة	سمة تنتهي بشرطة سفلية تُنشأ في `fit`	Fitted attribute
مِكسِن	صنف يضيف سلوكاً فوق صنف آخر بلا حالة خاصة به	Mixin
وسم	تصريح مُهيكل بما يستطيع المُقدِّر فعله	Estimator tag
وسم المدخلات	تصريح بقبول القيم المفقودة والمصفوفات المتفرّقة وشكل المصفوفة	Input tag
قيد البارامتر	وصف نطاق القيمة المقبولة لبارامتر بعينه	Parameter constraint
فحص المُقدِّرات	ما يوفّره `check_estimator` للتحقّق من العقد	Estimator checks
تخطّي فحص	فحص يُستبعد بسبب بيئي لا بسبب المُقدِّر	Skipped check
استنساخ	بناء نسخة جديدة غير مدرَّبة تحمل البارامترات نفسها	Clone
خط الأساس	أبسط مصنّف ممكن، تُقاس عليه النماذج الحقيقية	Baseline

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٦ — التوسّع والتخصيص

الفئات الفرعية، والوسوم، وأنواع الأشجار، والاختبارات

الوحدة scikit-learn ١.٩ الحجم ٣٧٧ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يميّز بين تخصيص المحوّل والمصنّف والمقدّر
- أن يطبق BaseEstimator، TransformerMixin، validate_data وفق العقد
- أن يفحص الوسوم ويشغّل check_estimator على صنف مخصص
- أن يختار التركيب قبل وراثة مقدّر جاهز

١١.١ هل تحتاج إلى صنف جديد فعلاً؟

أحياناً تحتاج إلى تحويل خاص بالمجال، أو تقدير يتبع قاعدة داخلية لا يوفرها الصنف الجاهز. قبل أن تكتب صنفاً جديداً، اسأل هل يكفي `FunctionTransformer` أو محوّل من المكتبة أو تركيب خطوات موجودة. الصنف المخصص يضيف عقداً عليك صيانته: معاملات قابلة للاستنساخ، حالة متعلمة منتهية بشرطة سفلية، تحقق من الأبعاد، وسلوك واضح عند التنبؤ قبل `fit`.

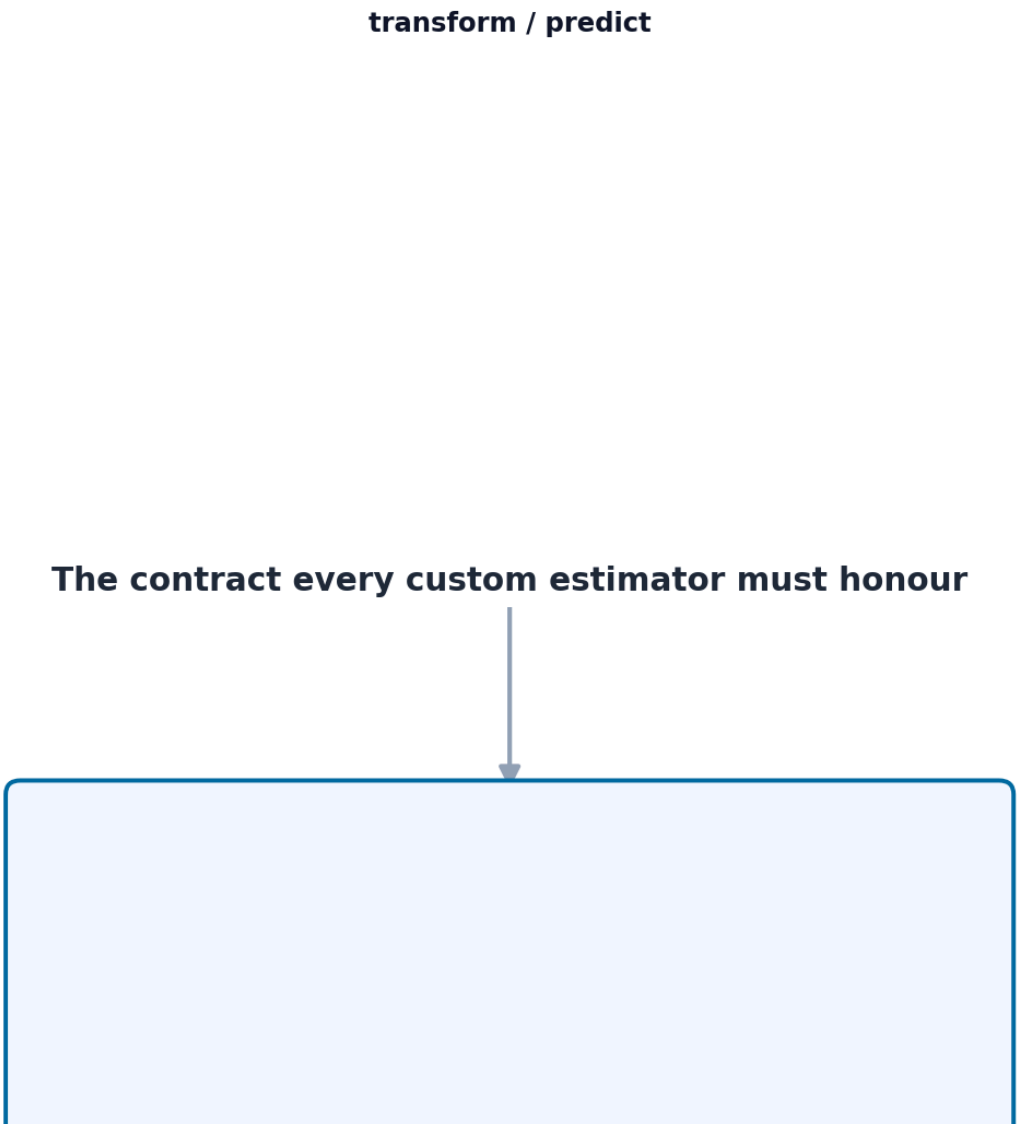
يشرح الفصل ٩٥ تفاصيل كتابة مقدّر متوافق من الصفر؛ هنا نركز على سُلّم التوسّع: اختبر واجهة صغيرة، ثم افحص توافقها، ولا تبدأ بتعديل داخلي في شجرة أو مسار مكتبة.

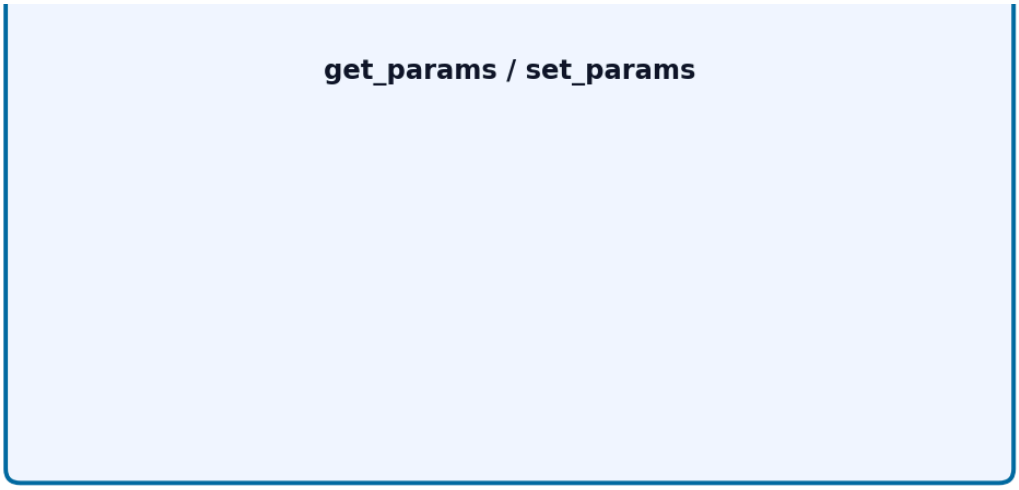
class MyModel(BaseEstimator)

__init__ stores params only

fit(X, y) sets *_

validate_data()


transform / predict
The contract every custom estimator must honour



الشكل ٢١٠ عقد التوافق للمقدّر المخصص

محوّل يزيح الأعمدة وفق متوسط التدريب python

```python
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted, validate_data

class MeanCenterer(TransformerMixin, BaseEstimator):
    def fit(self, X, y=None):
        X = validate_data(self, X, dtype="numeric")
        self.mean_ = X.mean(axis=0)
        return self

    def transform(self, X):
        check_is_fitted(self, "mean_")
        X = validate_data(self, X, reset=False, dtype="numeric")
        return X - self.mean_

X = np.array([[1.0, 10.0], [3.0, 14.0], [5.0, 18.0]])
transformer = MeanCenterer().fit(X)
print("المتوسط المخزن:", transformer.mean_.tolist())
print("المخرجات:", transformer.transform(X).tolist())
```

يضمن `validate_data` أن عدد الخصائص في التحويل يطابق ما تعلّمه التدريب، و `check_is_fitted` يوضح الخطأ إذا استُخدم الكائن قبل التدريب. لا تحتفظ بإحصاءات من كامل البيانات خارج `fit`.

١١.٢ معاملات تتبعها أدوات المكتبة

يجب أن يخزن `__init__` المعاملات كما وردت، دون تحويل أو تحقق جانبي؛ بذلك يستطيع `get_params` و`clone` و`GridSearchCV` إعادة بناء الكائن. تعلم الحالة داخل `fit` وتسميتها عادةً بنهاية `_`. استخدم وراثة mixin المناسبة لترتيب الواجهة، ثم اختبرها في Pipeline.

python — الاستنساخ ومعاملات المحوّل

```
from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

center = MeanCenterer()
print("المعاملات:", center.get_params(deep=False))
print("الاستنساخ جديد:", clone(center) is not center)
pipe = Pipeline([("scale", StandardScaler()), ("center", MeanCenterer())])
pipe.fit([[1.0, 2.0], [3.0, 5.0], [4.0, 8.0]])
print("أسماء الخطوات:", list(pipe.named_steps))
```

اختبار `clone` لا يثبت صحة الخوارزمية؛ يثبت أن الواجهة الأساسية تتبع نمط المكتبة. اختبر أيضاً الأشكال والقيم المفقودة والأنواع التي تعلن دعمها، وحدد ما ينبغي رفضه برسالة مفهومة.

١١.٣ الوسوم والاختبارات

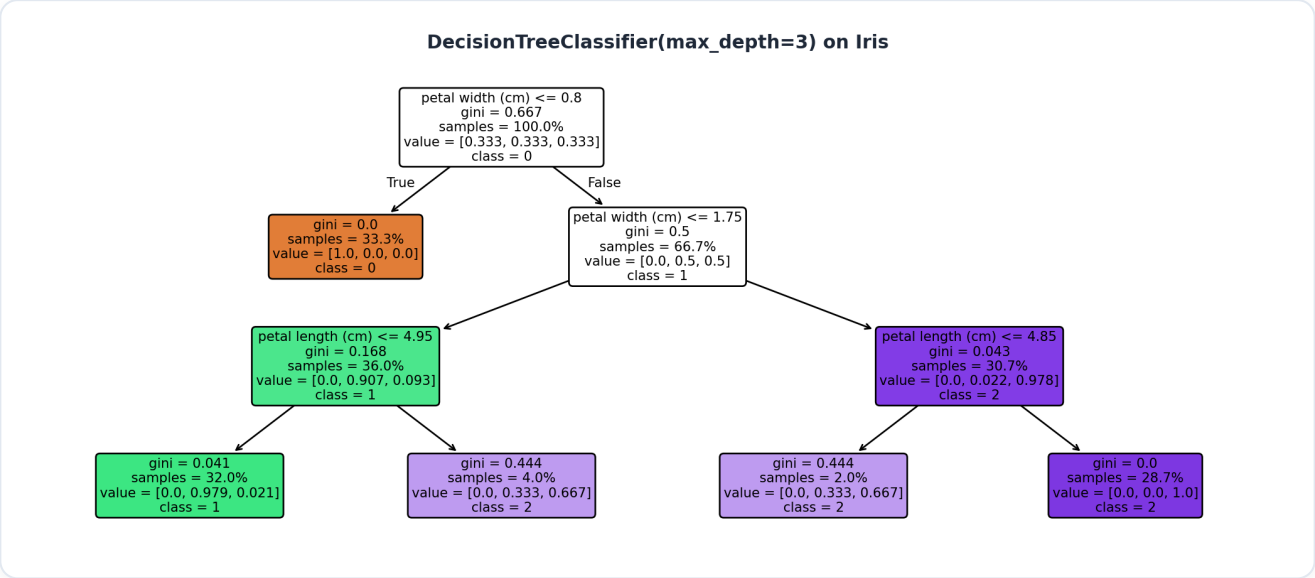
الوسوم (tags) تصف خصائص مثل قبول القيم المفقودة أو كون المقدّر مصنفاً أو دعمه Array API. تستخدمها بعض الأدوات لتقرير الاختبارات المناسبة. لا تغيّر الوسوم لتجاوز اختبار فاشل؛ عدّل التنفيذ أو صرّح بحد حقيقي تدعمه الواجهة. `sklearn.utils.estimator_checks.check_estimator` يجري مجموعة اختبارات عامة، ومن الطبيعي أن يكشف افتراضات لم تفكر بها. يشترط فحص كائنك المخصص من ملف مشروع قابل للاستيراد؛ فكتابة الصنف داخل كتلة تفاعلية تجعل بعض اختبارات التسلسل غير قابلة للتنفيذ.

python — فحص صنف مخصص من وحدة قابلة للاستيراد

```
from sklearn.utils.estimator_checks import check_estimator

# انقل MeanCenterer إلى ملف Python داخل مشروعك ثم استورده هنا.
check_estimator(MeanCenterer())
```

فحص `check_estimator` يتطلب صنفاً قابلاً للاستيراد، كما أن اختبار DataFrame العام يتعارض مع إعداد عرض DataFrame الشامل لمشغّل أمثلة الكتاب. شغّله في بيئة مشروعك بعد استيراد الصنف، ثم ابدأ بأول traceback إذا

أخفق. قد يكون الصنف مخصصاً لحالة أضيق من العقد العام، أو يفتقد معالجة صف واحد أو مصفوفة فارغة أو أسماء خصائص؛ استخدم `parametrize_with_checks` أو اختبارات وحدات موجهة، ووضح حدود الدعم بدلاً من إخفاء الخطأ.



الشكل ٢١١تقسيم خطوات التحقق من واجهة مقدّر مخصص

١١.٤ الوراثة والتركيب: لا تمدد داخلياً بلا ضرورة

وراثة `DecisionTreeClassifier` لتغيير سطر داخلي تربط كودك بتفاصيل تنفيذية قد تتغير بين الإصدارات. إذا كان المطلوب خطوة معالجة، ابنها كمحوّل في Pipeline. وإذا كان المطلوب قاعدة قرار جديدة، نفّذ مقدّراً مستقلاً على عقد BaseEstimator. لا تصل إلى السمات الخاصة ذات الشرطة السفلية الواحدة أو المزدوجة إلا إذا قبلت كلفة الصيانة والاختبارات.

إدراج المحوّل في تحقق متقاطع python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
model = Pipeline([
    ("scale", StandardScaler()),
    ("center", MeanCenterer()),
    ("clf", LogisticRegression(max_iter=3000)),
])
scores = cross_val_score(model, X, y, cv=5, scoring="balanced_accuracy")
print("الدرجات:", scores.round(3).tolist())
```

هذا التحقق يختبر دمج المحوّل، لكنه لا يثبت أن التوسيط الإضافي يحسن النموذج؛ التحجيم يطرح المتوسط أصلاً. المثال يعلّم العقد، لا يوصي بتكرار خطوة بلا مبرر تجريبي.

في الواقع

في مكتبة داخلية، اكتب اختبارات تثبت السلوك الذي يهم المستخدم: توافق `clone`، التنبؤ بعد `fit`، رفض عدد أعمدة مختلف، والعمل داخل Pipeline، GridSearchCV. هذه الاختبارات أهم من اختبار أن صنفك ينتج شكلاً معيناً على صفين معروفين.

تمارين الفصل السادس والتسعين

- **ملاحظة**

التمرين ١١.١

وسّع `MeanCenterer` ليقبل معامل `scale=False`. خزّن المعامل حرفياً، وأضف اختباراً لـ `get_params` و`clone`.

التمرين ١١.٢

شغّل `check_estimator` على المحوّل، ثم اقرأ أول فشل إن ظهر وفسر هل هو خطأ تنفيذ أم قيد غير معلن.

التمرين ١١.٣

ضع المحوّل داخل Pipeline مع مصنف وشغّل تحققاً متقاطعاً. اذكر ما الذي يثبته الاختبار وما الذي لا يثبته.

- **ملاحظة**

التحقق من نسخ المعاملات python

```python
from sklearn.base import BaseEstimator, TransformerMixin, clone

class ConfigurableCenterer(TransformerMixin, BaseEstimator):
    def __init__(self, scale=False):
        self.scale = scale

model = ConfigurableCenterer(scale=True)
print(model.get_params(deep=False))
print(clone(model).get_params(deep=False))
```

المعامل محفوظ بلا تحويل في المنشئ ليبقى الاستنساخ والبحث قابلين للتنبؤ.

> **ملاحظة**

تشغيل مجموعة فحوص المقدّر python

```python
from sklearn import config_context
from sklearn.utils.estimator_checks import check_estimator
from sklearn.preprocessing import StandardScaler

with config_context(transform_output="default"):
    check_estimator(StandardScaler())
print("اجتازت الفحوص العامة.")
```

شغّل الفحص على صنفك المعرّف في وحدة قابلة للاستيراد؛ الفحص أعلاه يستخدم مقدّراً عاماً كي تبقى كتلة الكتاب قابلة للتشغيل داخل بيئة العرض.

> 📝 **ملاحظة**

١١.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تحويل المعامل في `__init__`	يفشل `clone` أو تتغير قيمة البحث	خزّن المعامل كما ورد
تعلم الإحصاءات قبل `fit`	يتسرب الاختبار إلى المحوّل	احسبها في `fit` فقط
حذف فحص `check_is_fitted`	أخطاء مبهمة قبل التدريب	افحص السمات المتعلمة
تعديل الوسوم لإسكات الفحص	تقارير توافق مضللة	أصلح التنفيذ أو أعلن قيداً حقيقياً

١١.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مقدّر مخصص	كائن يطبق عقد الواجهة لمهمة خاصة	custom estimator
سمة متعلمة	حالة تنشأ بعد `fit` وتنتهي عادةً بـ `_`	learned attribute
وسم المقدّر	وصف سلوكي تستخدمه أدوات المكتبة	estimator tag
تركيب	توصيل مقدّرات بدلاً من وراثة تنفيذها	composition

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٧ — الأداء والتوازي والذاكرة

joblib.Parallel، وقياس الذاكرة، وأسرار التسريع

الوحدة **scikit-learn ١.٩** الحجم **٣٣٤** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يقيس زمن الملاءمة والتنبؤ بدلاً من تخمينهما
- أن يستخدم التوازي على مستوى واحد مع تحكم واضح في n_jobs
- أن يقارن الذاكرة والزمن مع الحفاظ على التكافؤ العددي
- أن يضع خطة أداء قابلة لإعادة القياس قبل النشر

١١.١ أول سؤال: أين يذهب الوقت؟

إذا كان التدريب بطيئاً، قد يكون السبب قراءة البيانات أو تحويلها أو البحث الشبكي، لا الخوارزمية. قس مراحل مستقلة: `fit`، و`predict`، ثم التحقق المتقاطع. استعمل بيانات ممثلة لحجم الإنتاج، وسخّن العملية إذا كان بدء Python أو تحميل المكتبات جزءاً من القياس.

python — قياس التدريب والتنبؤ منفصلين

```python
from time import perf_counter
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
model = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=1)
t0 = perf_counter()
model.fit(X, y)
fit_seconds = perf_counter() - t0
t0 = perf_counter()
pred = model.predict(X)
predict_seconds = perf_counter() - t0
print(f"fit: {fit_seconds:.4f}s | predict: {predict_seconds:.4f}s")
print("عدد التنبؤات:", pred.shape[0])
```

هذا القياس على بيانات التدريب مناسب لزمن التنفيذ فقط، لا لتقدير جودة النموذج. لا تخلط بين الغرضين في تقرير واحد من دون تسمية واضحة.

الشكل ٢١٢ مقارنة التنفيذ المتوازي والتسلسلي

١١.٢ أين تضع n_jobs؟

تدعم بعض الأدوات `n_jobs`، مثل `cross_validate` وبعض الخوارزميات. اختر مستوى توازٍ واحداً غالباً. إذا وازيت الطيات، اجعل النموذج الداخلي تسلسلياً؛ وإذا كانت الطية الواحدة ثقيلة جداً وتوازي الخوارزمية داخلياً، اجعل `n_jobs=1` في المقسّم. التوازي المتداخل قد يطلق عمليات أكثر من المعالجات ويضاعف نسخ البيانات.

توازٍ على الطيات مع نموذج تسلسلي — python

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate

X, y = load_breast_cancer(return_X_y=True)
model = RandomForestClassifier(n_estimators=100, n_jobs=1, random_state=42)
result = cross_validate(model, X, y, cv=5, scoring="balanced_accuracy",
                        n_jobs=2, error_score="raise")
print("درجات:", result["test_score"].round(3).tolist())
print("زمن الملاءمة:", result["fit_time"].round(3).tolist())
```

قد لا يسرّع `n_jobs=2` بيانات صغيرة بسبب كلفة إنشاء العمليات. كما قد يتأثر الوقت بالحمل على الجهاز. قارن درجات النماذج قبل الأزمنة، وتأكد أن خيارات العشوائية ثابتة.

١١.٣ joblib لمهام مستقلة

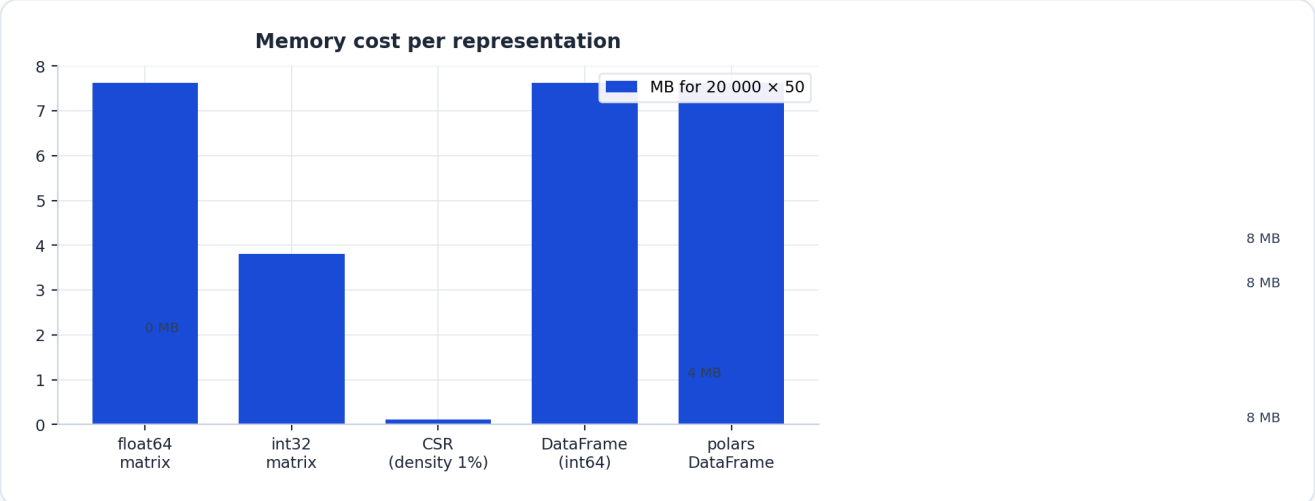
عندما يكون لديك عمل مستقل لا تمثله واجهة مقدّر، توفر `joblib.Parallel` و`joblib.delayed` توزيع المهام. اختر backend وفق طبيعة العمل: الحساب العددي الذي يحرر GIL قد يستفيد من العمليات أو الخيوط بحسب المكتبة، أما مشاركة مصفوفات ضخمة فقد تجعل نسخ العمليات مكلفة.

```python
تنفيذ مهام مستقلة بحد توازٍ صغير
from joblib import Parallel, delayed

def summarize(values):
    return (len(values), sum(values), max(values))

chunks = [list(range(start, start + 100)) for start in (0, 100, 200, 300)]
results = Parallel(n_jobs=2)(delayed(summarize)(chunk) for chunk in chunks)
print("ملخص كل جزء:", results)
```

المثال يستخدم مدخلات صغيرة، ولذلك لا معنى لتفسيره كاختبار سرعة. في العمل الحقيقي قس كلفة التحضير والنتائج المنقولة، وحدد `n_jobs` بما يتناسب مع ذاكرة الجهاز.

١١.٤ الذاكرة والتكافؤ

قياس الذاكرة الدقيق يحتاج أدوات نظام مناسبة؛ `tracemalloc` يتتبع تخصيصات Python لكنه لا يرى كل ذاكرة NumPy الأصلية. مع ذلك يمكن حساب حجم المصفوفة مباشرة، ومراقبة حجم RSS من أدوات النظام. عند تقليل الذاكرة استخدم صيغة متفرقة أو دفعات إذا كانت ملائمة، ولا تغيّر `float64` إلى `float32` قبل فحص أثر الدقة والتوافق.



الشكل ٢١٣قياس استخدام الذاكرة خلال التنفيذ

قياس حجم تمثيل كثيف ومتفرق python

```python
import numpy as np
from scipy.sparse import csr_matrix

rng = np.random.default_rng(42)
dense = (rng.random((1000, 1000)) < 0.01).astype(np.float64)
sparse = csr_matrix(dense)
dense_bytes = dense.nbytes
sparse_bytes = sparse.data.nbytes + sparse.indices.nbytes + sparse.indptr.nbytes
print("الكثافة:", round(float(np.count_nonzero(dense) / dense.size), 4))
print("حجم dense:", dense_bytes, "بايت")
print("حجم CSR:", sparse_bytes, "بايت")
```

هذا الحساب لا يشمل كل كائنات Python أو المساحة المؤقتة أثناء المعالجة، لكنه يقارن تمثيلاً أساسياً. لا تختر sparse إذا كانت البيانات كثيفة أو إذا كان المقدّر لا يدعمها.

في الواقع

سجل خط الأساس قبل التحسين: نسخة المكتبة، وحجم البيانات، ودرجة التحقق، وزمن التدريب، واستهلاك الذاكرة. بعد كل تغيير قارن الأداء والجودة معاً؛ التحسين الذي يسرع ٣٠٪ لكنه يغير سلوك النموذج أو يسرّب البيانات ليس تحسيناً.

تمارين الفصل السابع والتسعين

- **ملاحظة**

التمرين ١١.١

قس زمن `fit` و`predict` لنموذج شجرة غابة بعدة أحجام `n_estimators` مع تثبيت البذرة.

التمرين ١١.٢

قارن `cross_validate` بـ `n_jobs=1` و`n_jobs=2` على البيانات نفسها. لا تفسر فارق تشغيل واحد كقاعدة عامة.

التمرين ١١.٣

أنشئ مصفوفة كثيفة قليلة الإشغال، وقارن `nbytes` بها وبتمثيل CSR.

- **ملاحظة**

زمن التدريب مع عدد أشجار مختلف python

```python
from time import perf_counter
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
for n in (50, 100, 200):
    model = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=1)
    t = perf_counter()
    model.fit(X, y)
    print(n, "أشجار، fit:", round(perf_counter() - t, 4), "ثانية")
```

ملاحظة

زمن التحقق في إعدادين python

```python
from time import perf_counter
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
for jobs in (1, 2):
    t = perf_counter()
    r = cross_validate(model, X, y, cv=5, n_jobs=jobs,
                       scoring="balanced_accuracy")
    print(jobs, round(perf_counter() - t, 3),
          r["test_score"].round(3).tolist())
```

> 📝 **ملاحظة**

١١.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
استخدام كل الأنوية بلا قياس	تتزاحم العمليات أو تمتلئ الذاكرة	جرّب حدوداً صغيرة وسجل الموارد
توازٍ متداخل	تضاعف عدد الخيوط والعمليات	اختر مستوى واحداً للتوازي
الاستدلال من اختبار مصغر	لا يمثل عبء الإنتاج	قس بأحجام وبيانات ممثلة
عدّ ذاكرة Python فقط	تُهمل مصفوفات C، NumPy	استخدم قياس RSS أو حساب المصفوفات

١١.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
زمن الملاءمة	الوقت اللازم لتدريب مقدّر	fit time
توازٍ متداخل	توازٍ داخل مهام متوازية	nested parallelism
الذاكرة المقيمة	صفحات العملية الموجودة فعلاً في الذاكرة	resident set size
تمثيل متفرق	تخزين العناصر غير الصفرية فقط	sparse representation

الجزء ١١ · المفاهيم المتقدّمة

الفصل ٩٨ — معجم الأخطاء: من الرسالة إلى الحل

أكثر ثلاثين خطأ في scikit-learn، وسبب كل خطأ وحلّه

الوحدة sklearn.callback · sklearn.utils الحجم ٣٦١ كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يقرأ الاستثناء بوصفه وصفاً لعقد واجهة لا نصاً للحفظ
- أن يميز بين خطأ شكل البيانات وخطأ الهدف أو الحالة
- أن يصمم مثالاً أدنى يعيد إنتاج العطل
- أن يضيف تحققاً يمنع تكرار السبب بعد الإصلاح

١١.١ اقرأ الرسالة قبل أن تغيّر الكود

يظهر الخطأ عند `fit`، فتبدأ بتغيير المحوّل أو المعاملات عشوائياً. توقّف: ما الصنف الذي أطلق الاستثناء؟ وما المدخل الذي رفضه؟ وهل ظهرت الرسالة أثناء التدريب أم التحويل أم التقييم؟ أول traceback كامل غالباً يحدد الحد الذي خالف العقد.

قبل الإصلاح، قلّص المثال إلى بيانات صغيرة ثابتة، واطبع الشكل والنوع وعدد الصفوف، ثم أعد الخطأ نفسه. لا تلتقط `Exception` وتواصل التنفيذ كأن النموذج نجح؛ هذا يحول العطل إلى نتيجة مضللة.

Read the exception type

Is it ValueError from check_array?

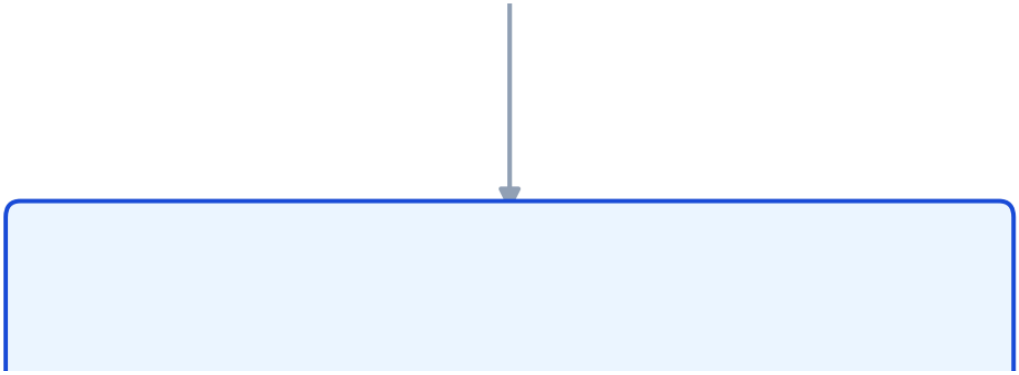
→ fix the dtypes

Is it ValueError on n_features?

→ fit before transform

ConvergenceWarning?

A decision tree for scikit-learn errors



الشكل ٢١٤ شجرة قرار لتشخيص أخطاء النماذج

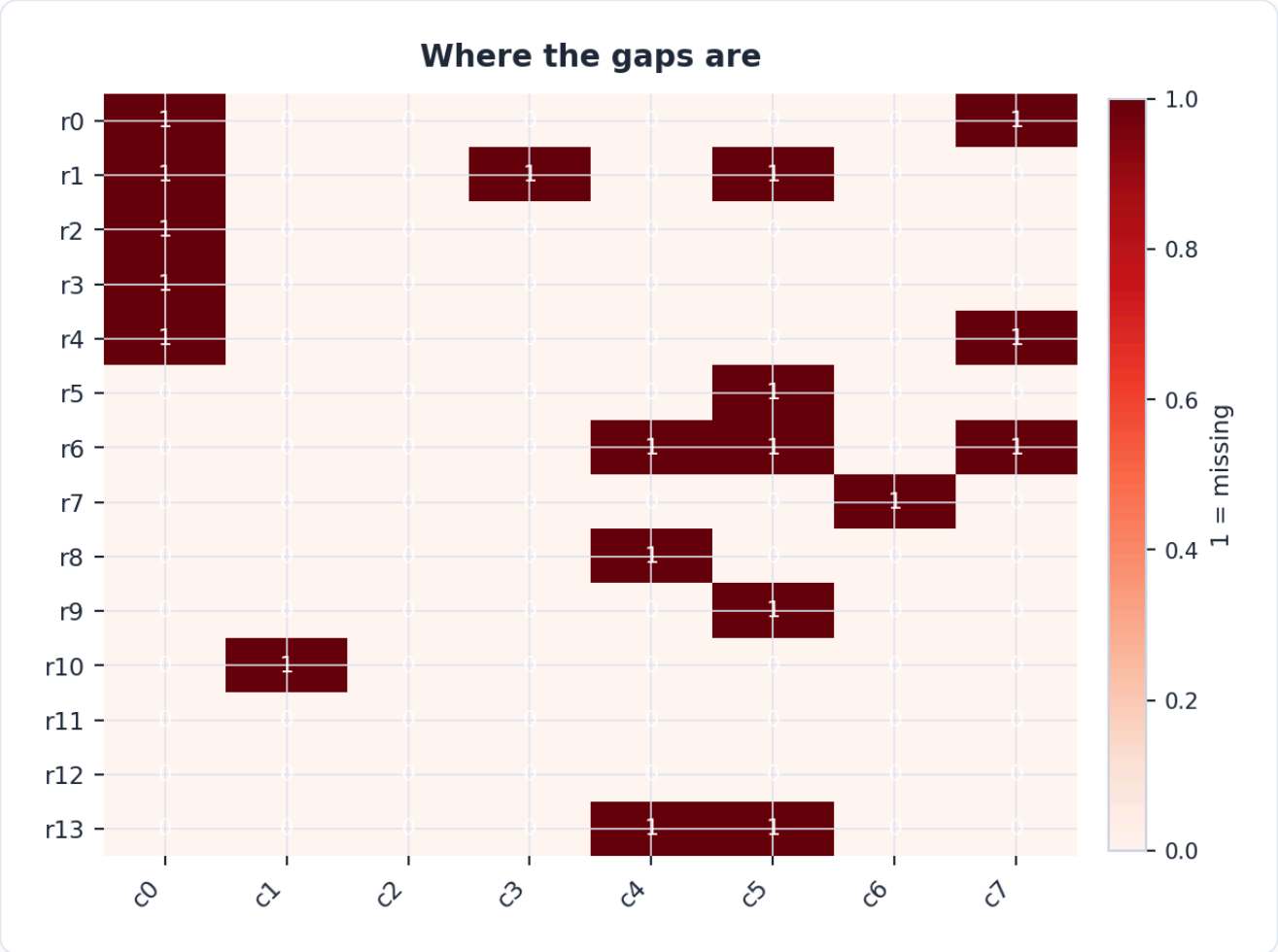
خطأ الشكل ثم فحص المدخل python

```python
import numpy as np
from sklearn.linear_model import LogisticRegression

X = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
y = np.array([0, 1])
try:
    LogisticRegression().fit(X, y)
except ValueError as exc:
    print(type(exc).__name__)
    print(str(exc).splitlines()[0])
print("X:", X.shape, "| y:", y.shape)
```

المشكلة هنا ليست في محلّل الانحدار: عدد أمثلة `X` لا يساوي عدد تسميات `y`. تحقق من أطوال المدخلات قبل أن تبحث عن تغيير خوارزمية.

١١.٢ الشكل والنوع والقيم المفقودة

أكثر رسائل `ValueError` شيوعاً تشير إلى اختلاف عدد الصفوف، أو أن `X` ليس ثنائي الأبعاد، أو أن الخوارزمية لا تقبل قيمة مفقودة، أو أن البيانات نصية في موضع يتطلب قيماً رقمية. لا تحوّل كل شيء إلى `float` على عجل؛ قد يكون العمود فئوياً يحتاج ترميزاً، وقد تكون القيمة المفقودة معلومة يجب تعويضها داخل Pipeline.



الشكل ٢١٥ أنماط القيم المفقودة وكيفية تشخيصها

```python
كشف القيم المفقودة قبل المعالجة
```

```python
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

X = np.array([[1.0, np.nan], [2.0, 3.0], [4.0, 5.0], [5.0, 6.0]])
y = np.array([0, 0, 1, 1])
print("عدد المفقودات لكل عمود:", np.isnan(X).sum(axis=0).tolist())
model = make_pipeline(SimpleImputer(strategy="median"),
                      LogisticRegression())
model.fit(X, y)
print("عدد التنبؤات:", model.predict(X).shape[0])
```

يجب أن يطابق التعويض ما تسمح به الخوارزمية ومخطط البيانات. وضعه داخل الخط يمنع تعلم الوسيط من الاختبار، ويتيح إعادة استخدامه في الطيات.

١١.٣ حالة النموذج: قبل fit وبعده

استثناء `NotFittedError` لا يعني أن النموذج غير صالح؛ يعني أن استدعاء `predict` أو `transform` سبق `fit`. أما `AttributeError` على `coef_` فقد يشير إلى استخدام سمة متعلمة قبل التدريب أو إلى مقدّر لا يملكها أصلاً. استعمل `check_is_fitted` في الكود المخصص، ولا تفحص السمات الداخلية لتقرر هل التدريب نجح إذا توفر سلوك واجهة أوضح.

```python
تمييز NotFittedError عن نتيجة التدريب
```

```python
from sklearn.exceptions import NotFittedError
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
try:
    model.predict([[1.0, 2.0]])
except NotFittedError as exc:
    print("قبل fit:", type(exc).__name__)
model.fit([[0.0, 1.0], [1.0, 0.0], [2.0, 1.0], [3.0, 0.0]],
          [0, 0, 1, 1])
print("بعد fit:", model.predict([[1.5, 0.5]]).tolist())
```

لا تعالج الخطأ بإنشاء نموذج جديد عند كل تنبؤ؛ أصلح دورة الحياة حتى يكون الكائن الذي دربته هو نفسه الذي تستخدمه في التطبيق أو تحفظه.

١١.٤ عدم التقارب والنتائج غير الصالحة

قد لا يرفع scikit-learn استثناءً عندما لا يتقارب مقدّر تكراري؛ قد يصدر تحذيراً ويعيد نموذجاً يحتاج مراجعة. اقرأ التحذير، افحص عدد التكرارات ومقياس البيانات، ثم زد الحد أو غيّر المقياس بناءً على تشخيص. لا ترفع `max_iter` بلا نهاية ولا تخفِ التحذير.

```python التقاط التحذير مع إظهاره
import warnings
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression

X, y = make_classification(n_samples=300, n_features=20,
                           n_informative=10, random_state=42)
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    model = LogisticRegression(max_iter=1).fit(X, y)
print("التكرارات:", model.n_iter_.tolist())
print("التحذيرات:", [type(w.message).__name__ for w in caught])
```

التقاط التحذير هنا للتشخيص فقط، ثم نعرض نوعه صراحة. في الشيفرة النهائية، إما أصلح السبب أو وجّه التحذير إلى سجل مراقب؛ لا تستخدم `simplefilter("ignore")` كحل.

> **خطأ شائع**
>
> لا تجعل `error_score=np.nan` أو `warnings.filterwarnings("ignore")` وسيلة لإخفاء فشل بحث. أثناء التطوير استخدم `error_score="raise"`، وافحص كل تحذير عدم تقارب قبل اعتماد النتيجة.

١١.٥ من الإصلاح إلى الوقاية

بعد تحديد السبب، أضف اختباراً يثبت الإصلاح: أطوال متوافقة، شكل ثنائي الأبعاد، معالجة للقيم المفقودة، أو اختبار لعدم استدعاء `predict` قبل `fit`. إذا كان الخطأ بسبب تقسيم خاطئ أو تسرب، فاختبار الوحدة وحده لا يكفي؛ راجع تصميم التقييم كله، كما تعلمنا في الجزء التاسع.

تمارين الفصل الثامن والتسعين

- **ملاحظة**

التمرين ١١.١

أعد إنتاج خطأ عدد صفوف غير متطابق. اكتب فحصاً صريحاً للأشكال قبل الملاءمة يمنع الوصول إلى `fit`.

التمرين ١١.٢

مرر قيمة مفقودة إلى نموذج خطي، ثم ضع `SimpleImputer` في Pipeline وأعد التجربة.

التمرين ١١.٣

درّب `LogisticRegression` بحد تكرارات منخفض، والتقط التحذير مع تسجيل نوعه. اذكر سبب عدم اعتبار التحذير نجاحاً كاملاً.

- **ملاحظة**

فحص أطوال X و y قبل fit — python

```python
import numpy as np
from sklearn.linear_model import LogisticRegression

X = np.ones((5, 2))
y = np.array([0, 1, 0, 1])
if X.shape[0] != y.shape[0]:
    print(f"عدد الصفوف مختلف: X={X.shape[0]}, y={y.shape[0]}")
else:
    LogisticRegression().fit(X, y)
```

في الإنتاج يفضّل رفع استثناء واضح إذا لم يتطابق العقد.

> **ملاحظة**

إصلاح القيم المفقودة داخل Pipeline python

```python
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

X = np.array([[1.0, np.nan], [2.0, 3.0], [4.0, 5.0], [5.0, 6.0]])
y = [0, 0, 1, 1]
model = make_pipeline(SimpleImputer(strategy="median"), LogisticRegression())
model.fit(X, y)
print(model.predict(X).tolist())
```

> **ملاحظة**

١١.٧ الأخطاء الشائعة

الرسالة أو العرض	السبب المحتمل	الخطوة التشخيصية
inconsistent numbers of samples	طول `X` و `y` مختلف	اطبع `shape[0]` لكل مدخل
ΓD array Expected	مصفوفة أحادية البعد	أعد تشكيل العينة إلى `(1, -1)`
NaN أو infinity غير مقبول	بيانات مفقودة أو غير منتهية	افحص `isna` أو `isfinite` وعالج داخل Pipeline
NotFittedError	استدعاء قبل `fit`	راجع ترتيب دورة حياة الكائن
تحذير عدم التقارب	حد تكرار أو مقياس غير ملائم	افحص المقياس والحد والوسم

١١.٨ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
traceback	تسلسل الاستدعاءات المؤدي إلى الخطأ	traceback
استثناء	فشل يوقف مسار التنفيذ	exception
تحذير	تنبيه لا يوقف التنفيذ بالضرورة	warning
إعادة إنتاج	مثال أدنى يعيد ظهور المشكلة	reproduction

الجزء ١٢

مشاريع واقعية كاملة

من الفكرة إلى التقرير، بتفاصيل لا تُحذف

ماذا يغطّي هذا الجزء

كل ما سبق يلتقي هنا في مشاريع متكاملة تشبه ما يُطلب في العمل الحقيقي: تنظيف بيانات فوضوية، واختيار نموذج، وضبطه، ثم تقرير يقرؤه فريق غير تقني. كل مشروع يوضح القرارات، ويتضمّن الأخطاء التي وقعت فعلاً في الطريق، وكيف عولجت.

الجزء ١٢ · مشاريع واقعية كاملة

الفصل ٩٩ — مشروع ١: تصنيف البريد المزعج وتحليل المشاعر

TF-IDF، Bayes الساذج، LinearSVC ثم تقرير

الوحدة **scikit-learn ١.٩** الحجم **٤٩٩** كلمة تقريباً

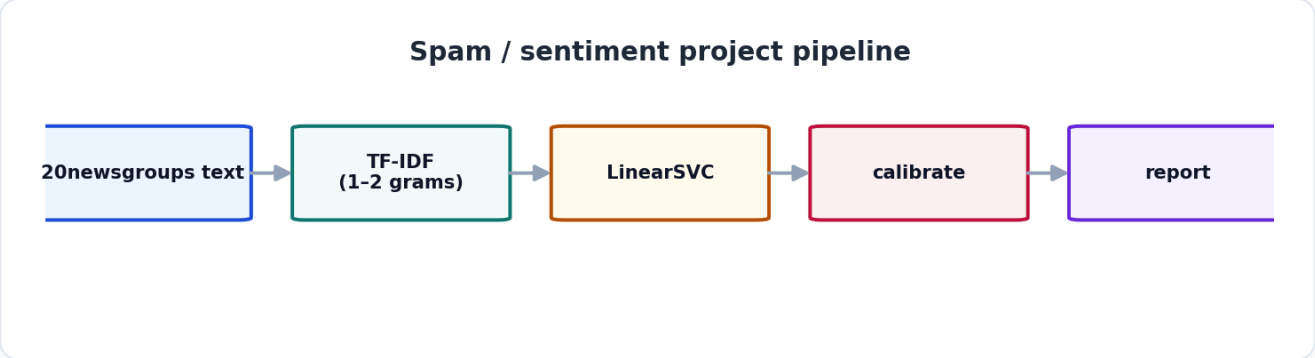
ما الذي ستتقنه في هذا الفصل

- أن يبني Pipeline نصياً يجمع TfidfVectorizer ومصنفاً
- أن يفصل التدريب والاختبار قبل تعلم قاموس الكلمات
- أن يقارن الاسترجاع والدقة ومقياس Average Precision بدلاً من الاكتفاء بالدقة
- أن يكتب تقريراً يميز بوضوح بين بيانات العرض الاصطناعية وبيانات الإنتاج

١٢.١ قبل الخوارزمية: ماذا نعني برسالة مزعجة؟

صندوق بريد الفريق يمتلئ برسائل تروّج لجوائز وهمية وروابط ضغط، لكن بعض الرسائل التسويقية مشروعة، وبعض الرسائل المهمة قصيرة جداً. السؤال ليس «هل النص يبدو غريباً؟» بل ما الفئة التي نريد اكتشافها، وما تكلفة كل خطأ؟ سنبني نموذجاً تعليمياً على رسائل محلية اصطناعية؛ لا ندّعي أن هذه البيانات تمثل سوقاً حقيقياً، ولا نحمّلها دقة إنتاجية.

ننشئ مجموعة متوازنة صغيرة كي لا يخفي اختلال الفئات أخطاء المصنف، ثم نقسمها **قبل** ملاءمة المفردات. في التطبيق الحقيقي احصل على موافقة مناسبة، أزل المعلومات الشخصية، وثبّت تعريف الرسالة المزعجة وتاريخ الجمع. لا تستخدم أرشيفاً عاماً أو رسائل عملاء بلا مراجعة الخصوصية.



الشكل ٢١٦خط تصنيف البريد من النص إلى القرار

```python
# توليد مجموعة رسائل محلية تعليمية
import numpy as np

spam_templates = [
    "claim your free prize today",
    "limited offer click to win cash",
    "urgent winner confirm your reward",
    "cheap deal buy now special discount",
]
ham_templates = [
    "meeting moved to tomorrow morning",
    "please review the project report",
    "your appointment is confirmed for Monday",
    "the deployment completed successfully",
]
docs, y = [], []
for label, templates in ((1, spam_templates), (0, ham_templates)):
    for i in range(40):
        docs.append(templates[i % len(templates)] + f" reference {i % 7}")
        y.append(label)
order = np.random.default_rng(42).permutation(len(y))
docs = np.array(docs, dtype=object)[order].tolist()
y = np.asarray(y)[order]
print("عدد الرسائل:", len(docs), "| التوزيع:", np.bincount(y).tolist())
print("مثال مجهول الهوية:", docs[0])
```

القوالب المتكررة تجعل المثال قابلاً لإعادة الإنتاج ولا تمنحه قيمة قياس خارج هذا الدرس. البيانات الحقيقية تحتاج تنوعاً لغوياً، ومراجعة بشرية للتسميات، ومجموعة اختبار زمنية تحاكي وصول الرسائل الجديدة.

١٢.٢ قاموس TF-IDF داخل الأنبوب

يحوّل `sklearn.feature_extraction.text.TfidfVectorizer` المستندات إلى مصفوفة متفرقة، ويزن الكلمات بحسب شيوعها في الوثيقة والمجموعة. يجب أن يتعلم القاموس من تدريب كل طية، لذا نضعه في `Pipeline` مع المصنّف. البدء بتحويل كل النصوص ثم تقسيم المصفوفة يسرّب معلومات قاموس الاختبار إلى التدريب.

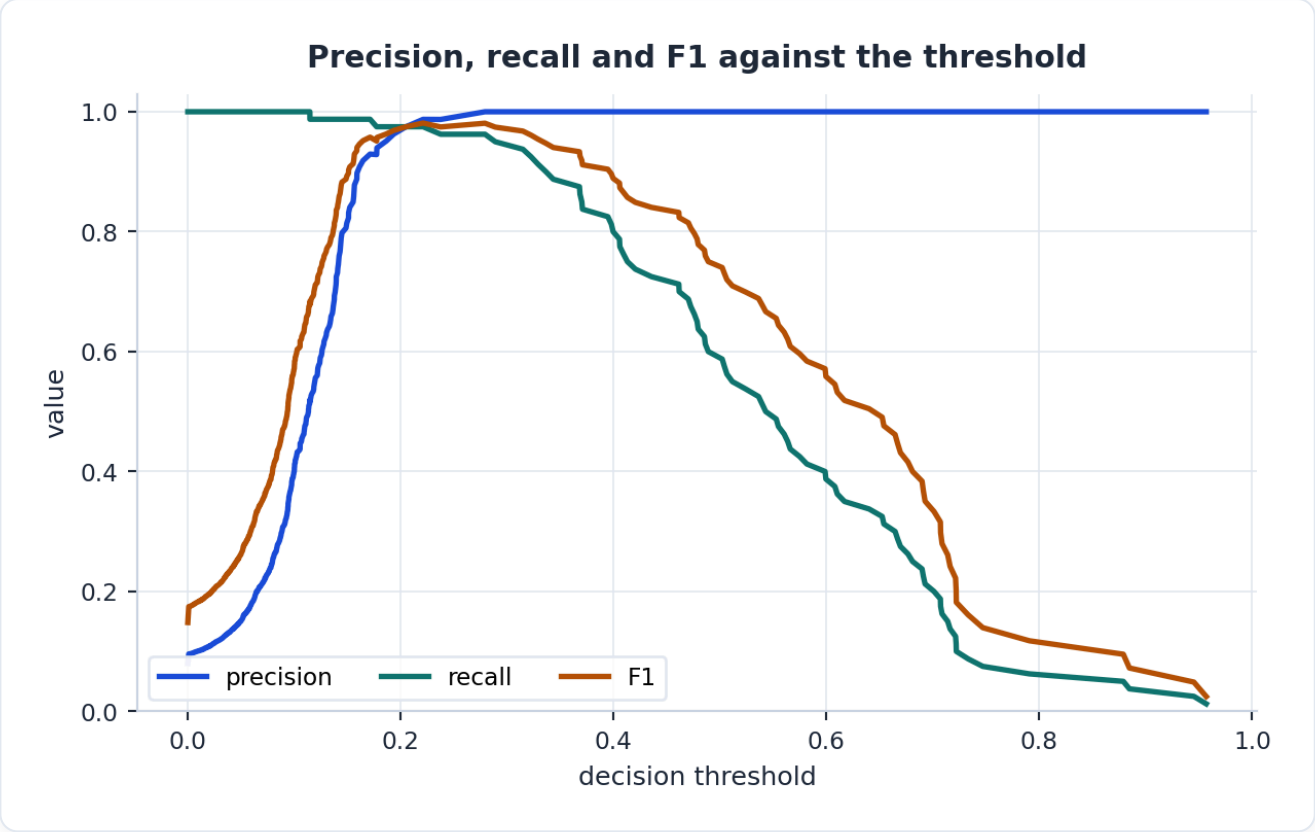
تـدريـب أول خط نـصي python

```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

spam = ["claim free prize click now", "limited cash reward today",
        "buy cheap deal now", "urgent winner claim reward"] * 30
ham = ["meeting tomorrow at nine", "please review the report",
       "deployment completed successfully", "appointment confirmed"] * 30
X = np.array(spam + ham, dtype=object)
y = np.r_[np.ones(len(spam), dtype=int), np.zeros(len(ham), dtype=int)]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced")),
])
model.fit(Xtr, ytr)
pred = model.predict(Xte)
print(classification_report(yte, pred, target_names=["مزعجة" ,"مشروعة"],
                            zero_division=0, digits=3))
```

المخرجات تقرير لهذا المثال الاصطناعي فقط. لا تنتقِ إعدادات `min_df` أو طول n-gram على الاختبار؛ استخدم تحققاً متقاطعاً على التدريب ثم احتفظ بالاختبار لتقدير نهائي واحد.

١٢.٣ المقارنة لا تختصر في الدقة

في مرشح البريد، قد يكون الخطأ الإيجابي الكاذب (إخفاء رسالة مشروعة) أشد من ترك رسالة دعائية في الوارد. نقارن `MultinomialNB` و`LinearSVC` داخل الأنبوب نفسه. الأول نموذج احتمالي بسيط يناسب تكرارات الكلمات غير السالبة، والثاني هامشه الخطي قوي في فضاءات النص المتفرقة لكنه لا يخرج `predict_proba` افتراضياً.



الشكل ٢١٧ مقايضة الدقة والاسترجاع في منحنى PR

مقارنة مصنفين على تقسيم ثابت python

```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import average_precision_score, precision_score,
recall_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC

spam = ["free prize click now", "claim cash reward", "cheap offer buy today"] *
40
ham = ["project meeting tomorrow", "review attached report", "schedule
confirmed"] * 40
X = np.array(spam + ham, dtype=object)
y = np.r_[np.ones(len(spam), dtype=int), np.zeros(len(ham), dtype=int)]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=7)
for name, clf in (("MultinomialNB", MultinomialNB()),
                  ("LinearSVC", LinearSVC(C=1.0))):
    pipe = Pipeline([("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
                     ("clf", clf)]).fit(Xtr, ytr)
    pred = pipe.predict(Xte)
    score = (pipe.predict_proba(Xte)[:, 1] if hasattr(clf, "predict_proba")
             else pipe.decision_function(Xte))
    print(name, "precision=", round(precision_score(yte, pred), 3),
          "recall=", round(recall_score(yte, pred), 3),
          "average_precision=", round(average_precision_score(yte, score), 3))
```

درجة القرار في `LinearSVC` تصلح لترتيب الرسائل، لكنها ليست احتمالاً معايراً. إذا كانت واجهة العمل تحتاج احتمالاً، استخدم معايرة داخل تحقق متقاطع، ثم افحص منحنى المعايرة قبل تفسير القيمة على أنها ثقة.

١٢.٤ تحليل المشاعر ليس مرادفاً لمكافحة البريد

يمكن إعادة استخدام مسار TF-IDF لتصنيف مراجعة إلى إيجابية أو سلبية، لكن مفردات المهمة تختلف: النفي والسياق والسخرية وتغير المجال تحديات حقيقية. لا تمزج فئتي «مزعج/مشروع» و«إيجابي/سلبي» في هدف واحد ما لم يكن ذلك تعريفاً صريحاً للمسألة. بنية البيانات والتسميات تحدد معنى النموذج قبل نوع الخوارزمية.

مثال مصغر لتحليل المشاعر python

```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.metrics import confusion_matrix

positive = ["excellent service and helpful staff",
            "the product works very well",
            "fast delivery and great quality"] * 25
negative = ["poor service and rude staff",
            "the product stopped working",
            "late delivery and bad quality"] * 25
X = np.array(positive + negative, dtype=object)
y = np.r_[np.ones(len(positive), dtype=int), np.zeros(len(negative), dtype=int)]
a, b, c, d = train_test_split(X, y, stratify=y, random_state=42)
model = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)),
                      LogisticRegression(max_iter=1000)).fit(a, c)
print("مصفوفة الخلط [سلبية، إيجابية]:\n", confusion_matrix(d, model.predict(b)))
print("تنبؤ جملة جديدة:", model.predict(["excellent quality and fast service"])[0])
```

الجملة الجديدة مشابهة للقوالب التي بُنيت عليها العينة؛ لذلك لا نستخدمها كدليل على فهم اللغة. في مشروع حقيقي اجمع مراجعات متنوعة، واحجب التكرارات بين التدريب والاختبار، وقس الأداء على فترة أو مصدر لم يره التدريب.

في الواقع

التقرير التشغيلي يذكر مصدر الرسائل وفترة جمعها، سياسة إخفاء البيانات، عدد كل فئة، طريقة تقسيم التدريب والاختبار، مقياس الاسترجاع للرسائل المزعجة، ومعدل حجب الرسائل المشروعة. اعرض أمثلة أخطاء بعد إخفاء الهوية كي يستطيع فريق البريد مراجعة أثر القرار.

تمارين الفصل التاسع والتسعين

- **ملاحظة**

التمرين ١٢.١

أضف n-grams ثنائية إلى `TfidfVectorizer`، وقارن عدد الخصائص مع unigram فقط.

التمرين ١٢.٢

اكتب تقريراً يعرض precision، recall للفئة المزعجة، ثم ناقش أيهما أهم في سياسة صندوق بريدك.

التمرين ١٢.٣

غيّر التقسيم إلى تقسيم زمني مصطنع، بحيث تكون الرسائل الأحدث في الاختبار. اشرح لماذا لا يجوز خلط تكرارات قالب الرسالة بين الجزأين.

- **ملاحظة**

حجم القاموس بحسب n-grams

```python
from sklearn.feature_extraction.text import TfidfVectorizer

texts = ["project status meeting tomorrow", "project report is ready",
         "meeting about deployment status"]
for ngram in ((1, 1), (1, 2)):
    v = TfidfVectorizer(ngram_range=ngram)
    matrix = v.fit_transform(texts)
    print(ngram, matrix.shape, v.get_feature_names_out().tolist())
```

ملاحظة

```python
# مقاييس الفئة المزعجة
from sklearn.metrics import classification_report

y_true = [0, 0, 0, 1, 1, 1]
y_pred = [0, 1, 0, 1, 0, 1]
print(classification_report(y_true, y_pred,
                            target_names=["مزعجة" ,"مشروعة"],
                            zero_division=0, digits=3))
```

الاختيار بين precision، recall قرار تشغيلي: هل كلفة حجب رسالة سليمة أعلى أم كلفة ترك رسالة مزعجة؟

> **ملاحظة**

١٢.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
ملاءمة المفردات قبل التقسيم	يدخل قاموس الاختبار في التمثيل	ضع المحوّل داخل Pipeline
عرض الدقة فقط	قد تخفي أخطاء الرسائل المهمة	اعرض precision، recall، ومصفوفة الخلط
تفسير هامش SVM كاحتمال	تصبح الثقة العددية مضللة	عاير الاحتمالات على تحقق مناسب
تقييم قوالب مكررة	يحفظ النموذج صياغة متطابقة	افصل بالمصدر أو الوقت واحجب التكرارات

١٢.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
TF-IDF	وزن للكلمة بحسب ندرتها	term frequency-inverse document frequency
n-gram	تسلسل من كلمات متجاورة	n-gram
متوسط الدقة	مساحة ملخصة لمنحنى precision-recall	average precision
معايرة	توافق الاحتمال المتوقع مع التواتر المرصود	calibration

الجزء ١٢ · مشاريع واقعية كاملة

الفصل ١٠٠ — مشروع ٢: توقّع مغادرة العملاء

تحليل RFM، والبيانات غير المتوازنة، ومعيار الربح

الوحدة **١.٩ scikit-learn** الحجم **٤٨٥** كلمة تقريباً

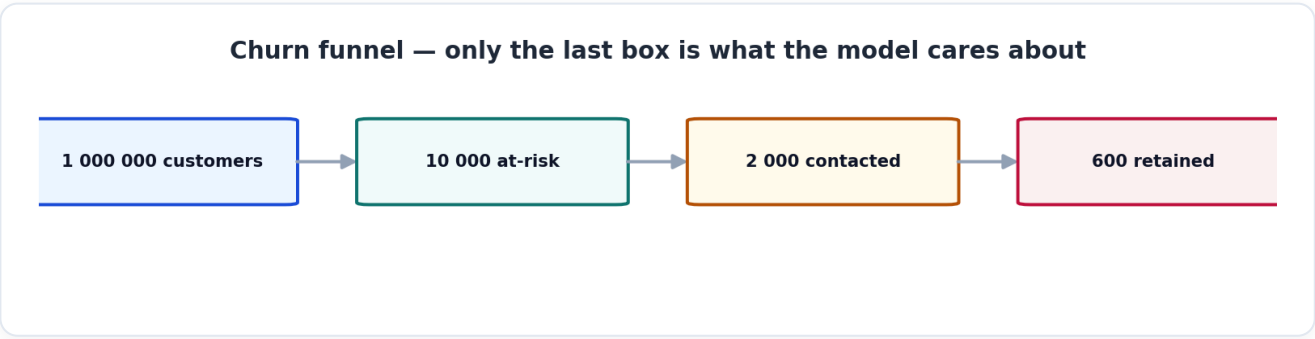
ما الذي ستتقنه في هذا الفصل

- أن يبني خط معالجة مختلطاً داخل Pipeline
- أن يعرّف مغادرة العميل والفترة الزمنية قبل إنشاء السمات
- أن يحسب عائد حملة تدخل من مصفوفة الخلط والافتراضات المالية
- أن يقترح مراقبة زمنية للبيانات والنتائج بعد الإطلاق

١٢.١ ما الذي نعدّه مغادرة؟

تصل لوحة المدير بقائمة «عملاء معرضين للمغادرة». قبل النموذج نسأل: هل المغادرة إلغاء اشتراك خلال الثلاثين يوماً القادمة، أم عدم شراء خلال ستة أشهر؟ أي تاريخ نستخدمه لتكوين السمات؟ إذا استُخدمت معاملات حدثت بعد تاريخ القرار، يصبح النموذج عرّافاً ببيانات المستقبل.

سنستخدم بيانات اصطناعية محلية لا تمثل سلوك شركة. الأعمدة تشبه مؤشرات تشغيلية: مدة العلاقة، الرسوم، تفاعلات الدعم، ونوع الخطة. في المشروع الحقيقي اكتب تعريف الهدف ونافذة السمات ونافذة النتيجة، ثم ابنِ تجزئة زمنية تمنع التدريب على المستقبل واختباره على الماضي.



الشكل ٢١٨ مسار تحليل العملاء من السلوك إلى التدخل

إنشاء جدول عملاء اصطناعي python

```python
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification

Xn, y = make_classification(n_samples=1200, n_features=3, n_informative=3,
                            n_redundant=0, weights=[0.78, 0.22],
                            class_sep=0.8, random_state=42)
rng = np.random.default_rng(42)
customers = pd.DataFrame({
    "tenure_months": np.clip((Xn[:, 0] * 12 + 30).round(), 1, 120),
    "monthly_charge": np.clip(Xn[:, 1] * 18 + 65, 10, 180).round(2),
    "support_calls": np.clip((Xn[:, 2] * 2 + 3).round(), 0, 15).astype(int),
    "contract": np.where(rng.random(len(y)) > 0.5, "annual", "monthly"),
    "churn": y,
})
print("الأبعاد:", customers.shape)
print("نسبة المغادرة:", round(customers["churn"].mean(), 3))
print(customers.head(3).to_string(index=False))
```

الأرقام مولدة للتوضيح؛ لا تحمل معنى تجارياً ولا ينبغي نقلها إلى عرض تنفيذي بوصفها توقعات حقيقية. المثال يساعدنا على بناء خط معالجة وتقييم، أما النموذج التشغيلي فيحتاج بيانات بتاريخ معلوم وتسميات مراجعة.

١٢.٢ المعالجة على مستوى العمود

الأعمدة العددية تحتاج تعويضاً وتحجيماً بحسب النموذج؛ `contract` فئة تحتاج ترميزاً. نستخدم `ColumnTransformer` داخل `Pipeline` كي يتعلم كل محوّل من جزء التدريب في كل طية. القيم المفقودة قد تحمل معنى، لكن لا نفترض ذلك بلا معرفة مجال؛ هنا نعرض سياسة وسيط للأرقام وأكثر فئة تكراراً للفئة.

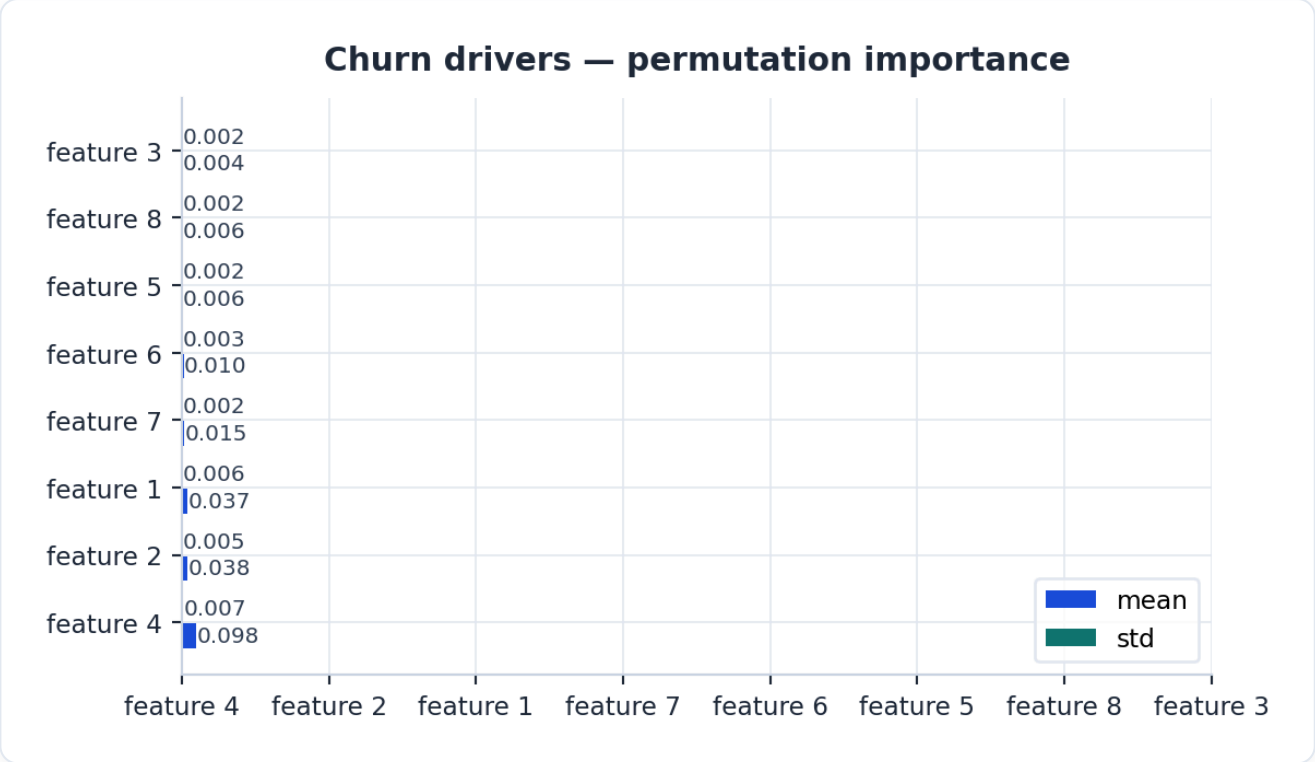
خط المعالجة والتصنيف python

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

Xn, y = __import__("sklearn.datasets", fromlist=
["make_classification"]).make_classification(
    n_samples=1000, n_features=3, n_informative=3, n_redundant=0,
    weights=[0.8, 0.2], random_state=42)
rng = np.random.default_rng(1)
X = pd.DataFrame({"tenure": Xn[:, 0] * 10 + 24,
                  "charges": Xn[:, 1] * 20 + 70,
                  "calls": np.maximum(0, (Xn[:, 2] * 2 + 3).round()),
                  "contract": np.where(rng.random(len(y)) > 0.5, "annual",
"monthly")})
a, b, c, d = train_test_split(X, y, stratify=y, test_size=0.25, random_state=42)
numeric = Pipeline([("impute", SimpleImputer(strategy="median")),
                    ("scale", StandardScaler())])
categorical = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                        ("encode", OneHotEncoder(handle_unknown="ignore",
                                                 sparse_output=False))])
prep = ColumnTransformer([("num", numeric, ["tenure", "charges", "calls"]),
                          ("cat", categorical, ["contract"])])
model = Pipeline([("prep", prep),
                  ("clf", LogisticRegression(max_iter=2000,
                                             class_weight="balanced"))])
model.fit(a, c)
print("Balanced accuracy:", round(balanced_accuracy_score(d, model.predict(b)),
3))
print("شكل التدريب:", a.shape, "نسبة المغادرة في الاختبار:", round(float(d.mean()), 3))
```

في هذا المثال استخدمنا `class_weight="balanced"` لإظهار معالجة اختلال الفئات، لكن الوزن ليس علاجاً عاماً ولا يحدد سياسة التواصل. ودرجة `score` هنا accuracy؛ لهذا سنفحص الاسترجاع والربح لاحقاً بدلاً من الاعتماد عليها.

١٢.٣ من الاحتمال إلى حملة قابلة للتبرير

ترتيب العملاء حسب الاحتمال لا يجيب وحده عن سؤال «من نتصل به؟». افترض، لأغراض الحساب فقط، أن الاحتفاظ بعميل مغادر يوفر قيمة ١٢٠ وحدة، وأن الاتصال يكلف ١٠ وحدات، وأن الحملة لا تنجح مع الجميع. نقارن قيمة true positive بكلفة false positive، ونختار عتبة على تحقق منفصل لا على الاختبار النهائي.



الشكل ٢١٩أهمية خصائص المغادرة وعتبة التدخل

حساب أثر سياسة عتبة على الاختبار python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=1400, n_features=8, n_informative=5,
                           weights=[0.8, 0.2], class_sep=0.8, random_state=42)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=2000,
class_weight="balanced"))
model.fit(Xtr, ytr)
p = model.predict_proba(Xte)[:, 1]
threshold = 0.35
pred = (p >= threshold).astype(int)
tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
value = tp * (120 - 10) - fp * 10
print(f"العتبة={threshold:.2f} | TP={tp} FP={fp} FN={fn} TN={tn}")
print("قيمة الحملة وفق الافتراضات:", value, "وحدة")
```

الحساب يتضمن افتراضاً مبسطاً: كل حالة موجبة اكتشفها النموذج تستجيب للتدخل، وكل اتصال كلفته ثابتة. في الواقع نحتاج تجربة عشوائية لقياس **الأثر الإضافي** للحملة؛ العميل الذي كان سيبقى على أي حال لا يُنسب بقاؤه إلى النموذج.

١٢.٤ الإطلاق والمراقبة

قبل تشغيل قائمة فعلية، افصل مرحلة التحقق عن التقييم النهائي، وحدد سقف عدد الاتصالات يومياً، وراجع الفروق بين مجموعات العملاء. بعد النشر راقب نسبة المغادرة، وتوزيع السمات، وعدد العملاء المختارين، ومعدل الاستجابة، والقيمة الصافية. إذا تغيرت سياسة الأسعار أو تعريف المنتج، أعد التحقق؛ تغير المجال يجعل النموذج القديم أقل موثوقية.

ملخص أداء قابل للتقرير python

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (average_precision_score, precision_score,
                             recall_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(n_samples=1000, n_features=8, n_informative=5,
                           weights=[0.82, 0.18], random_state=24)
a, b, c, d = train_test_split(X, y, stratify=y, random_state=24)
m = make_pipeline(StandardScaler(),
                  LogisticRegression(max_iter=2000, class_weight="balanced"))
m.fit(a, c)
p = m.predict_proba(b)[:, 1]
pred = (p >= 0.4).astype(int)
print("انتشار الهدف:", round(float(d.mean()), 3))
print("العملاء المستهدفون:", int(pred.sum()), "من", len(pred))
print("precision:", round(precision_score(d, pred, zero_division=0), 3))
print("recall:", round(recall_score(d, pred), 3))
print("average precision:", round(average_precision_score(d, p), 3))
```

هذه أرقام تقييم اصطناعي لا تقدير أثر تجاري. يجب أن يتضمن التقرير قيود البيانات وافتراضات الكلفة والتاريخ والمقاييس، وأن يفصل قيمة التنبؤ عن قيمة التدخل السببي.

> **في الواقع**
>
> أفضل قرار ليس دائماً الاتصال بكل من تجاوز عتبة واحدة. إذا كانت قدرة الفريق محدودة، قد تختار أعلى نسبة من العملاء بحسب كلفة الفرصة، ثم تقيس نتائج عينة تدخل عشوائية لتعرف من يستجيب فعلاً. النموذج يرتب الاحتمال؛ التجربة تقيس أثر السياسة.

تمارين الفصل المئة

- **ملاحظة**

التمرين ١٢.١

غيّر تعريف الفئة المستهدفة إلى مغادرة خلال نافذة زمنية مختلفة، واشرح كيف تتغير السمات المسموح بها.

التمرين ١٢.٢

احسب TP وFP عند عتبتين مختلفتين، ثم احسب قيمة حملة وفق قيمة احتفاظ وكلفة اتصال تختارهما وتذكر افتراضاتك.

التمرين ١٢.٣

اقترح مخطط مراقبة شهرياً يميز تغير توزيع البيانات عن تراجع جودة التسميات.

- **ملاحظة**

تحديد نوافذ السمات والهدف python

```python
decision_days = 90
feature_window = (decision_days - 180, decision_days)
label_window = (decision_days, decision_days + 30)
print("نافذة السمات:", feature_window)
print("نافذة الهدف:", label_window)
assert feature_window[1] <= label_window[0]
```

يجب ألا تتجاوز أي سمة تاريخ القرار. تغيير أفق الهدف يغير تعريف التسميات والملاءمة الزمنية.

> **ملاحظة**

```python قيمة سياستين للعتبة
import numpy as np
from sklearn.metrics import confusion_matrix

y = np.array([0, 0, 0, 1, 1, 1])
p = np.array([0.1, 0.35, 0.8, 0.2, 0.55, 0.9])
for threshold in (0.3, 0.6):
    pred = (p >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y, pred).ravel()
    print(threshold, "TP/FP/FN/TN =", tp, fp, fn, tn,
          "القيمة =", tp * (120 - 10) - fp * 10)
```

> **ملاحظة**

١٢.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تسرب أحداث بعد تاريخ القرار	يصبح النموذج مطلعاً على النتيجة	ابنِ سمات كل صف من نافذته السابقة فقط
accuracy كهدف للحملة	قد تتجاهل الفئة التي تهم العمل	اعرض precision، recall، والربح
تفسير churn على أنه استجابة للتدخل	تستهدف من سيغادر لا من يمكن إنقاذه	اختبر uplift بتجربة عشوائية
إعادة استخدام test لاختيار العتبة	يصبح الاختبار جزءاً من التطوير	استخدم تحققاً منفصلاً ثم قياساً نهائياً

١٢.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
مغادرة العميل	توقف العميل عن النشاط وفق تعريف زمني	churn
نافذة السمات	الفترة السابقة التي تجمع منها المدخلات	feature window
قيمة العميل	منفعة أو إيراد مرتبط بالعميل	customer value
الأثر الإضافي	النتيجة التي سببها التدخل فوق ما كان سيحدث	incremental uplift

الجزء ١٢ · مشاريع واقعية كاملة

الفصل ١٠١ — مشروع ٣: كشف عمليات الاحتيال

البيانات شديدة الاختلال، وضبط العتبة، وتفسير النتائج

الوحدة **scikit-learn ١.٩** الحجم **٤٣٠** كلمة تقريباً

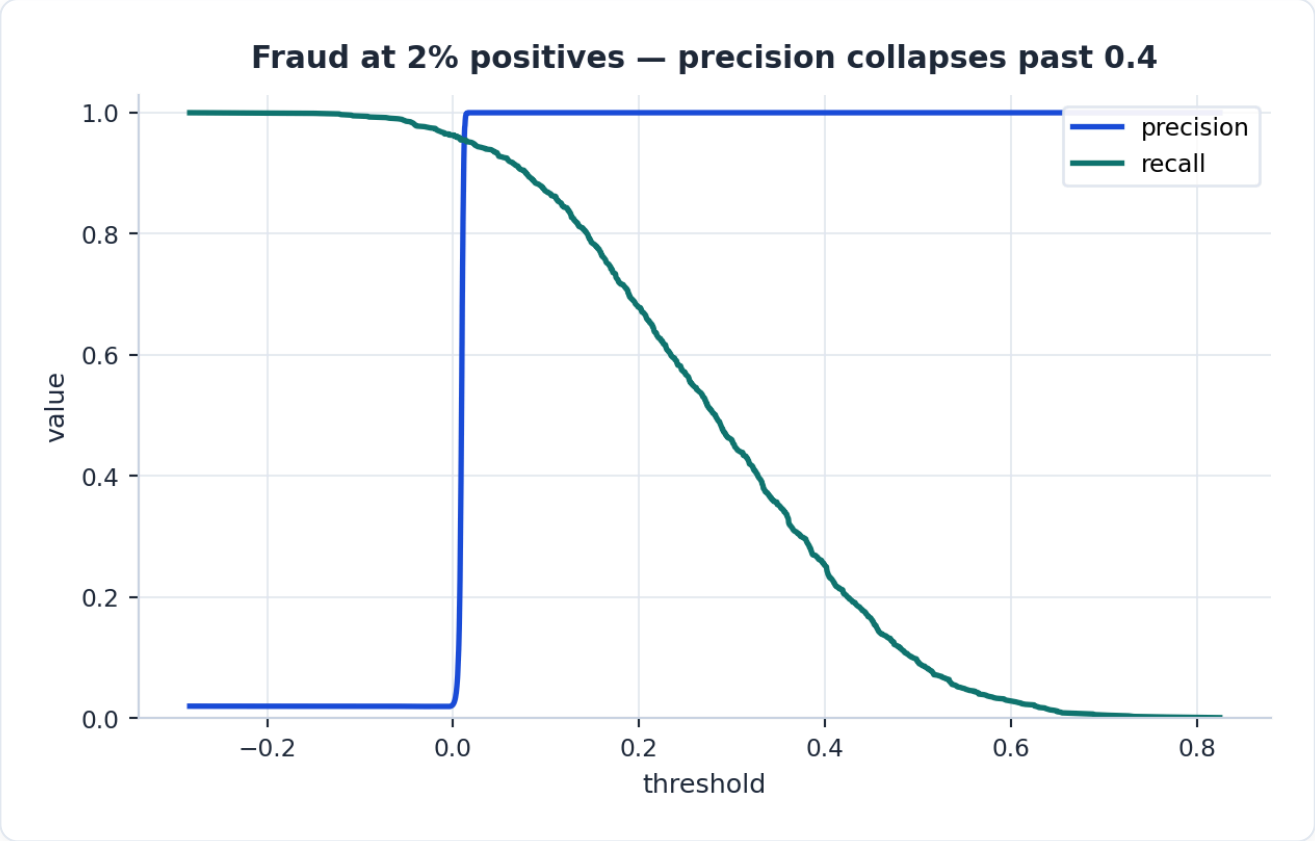
ما الذي ستتقنه في هذا الفصل

- أن يختار مقاييس ملائمة لفئة احتيال نادرة
- أن يضبط العتبة على تحقق مستقل مع الإبقاء على اختبار نهائي
- أن يحسب حجم قائمة المراجعة ومصفوفة الخلط
- أن يشرح حدود البيانات الاصطناعية والتسميات المتأخرة

١٢.١ لماذا accuracy مضللة هنا؟

لو كانت واحدة من كل ألف معاملة احتيالية، فقد يحقق نظام يقول «سليمة» دائماً دقة ٩٩٫٩٪ ولا يلتقط أي حالة. المطلوب ليس تعظيم رقم واحد؛ نريد ترتيب المخاطر واختيار عدد معاملات يمكن للمحققين مراجعتها، مع وزن كلفة تفويت الاحتيال والإنذارات الكاذبة.

سنولد سجلات اصطناعية موزونة لتوضيح المنهج. لا تمثل هذه السجلات أنماط احتيال واقعية ولا ينبغي استخدامها لاتخاذ قرارات عن أشخاص. البيانات الحقيقية تحتاج حوكمة وصول، وسجل قرار، وتسميات تحققها فرق مختصة.



الشكل ٢٢٠اختيار عتبة لكشف الحالات عالية المخاطر

إنشاء معاملات شديدة الاختلال python

```python
from sklearn.datasets import make_classification
import numpy as np

X, y = make_classification(
    n_samples=5000, n_features=24, n_informative=7, n_redundant=4,
    weights=[0.985, 0.015], class_sep=0.6, flip_y=0.01, random_state=42)
print("المعاملات:", X.shape[0])
print("حالات الفئة النادرة:", int(y.sum()))
print("انتشار الإيجابي:", round(float(y.mean()), 4))
print("خط أساس دائماً سليم:", round(float((y == 0).mean()), 4))
```

هذا الخط الأساس يوضح أن accuracy لا تكفي. نحتاج مقاييس تتعامل مع ترتيب الإيجابيات النادرة، مثل `average_precision_score`، ومقاييس القرار عند عتبة تشغيل.

١٢.٢ التقسيم ثم التعلم

في الاحتيال تتغير الأنماط بسرعة. إن كانت المعاملات مرتبة زمنياً، ينبغي أن يحاكي الاختبار فترة مستقبلية، لا تقسيم عشوائي قد يخلط نمط احتيال واحداً بين جانبي التقييم. في مثالنا لا يوجد وقت حقيقي، فنستخدم تقسيم طبقي ثابت لتوضيح الآلية فقط.

نضع التحجيم والمصنف في Pipeline. `class_weight="balanced"` يعيد وزن خسارة الفئات، لكنه لا يغير انتشار الاحتيال ولا يضمن عتبة مناسبة. نبدأ بـ `LogisticRegression` لسهولة الحصول على درجات احتمالية، ثم نراجع معايرة هذه الدرجات قبل تفسيرها.

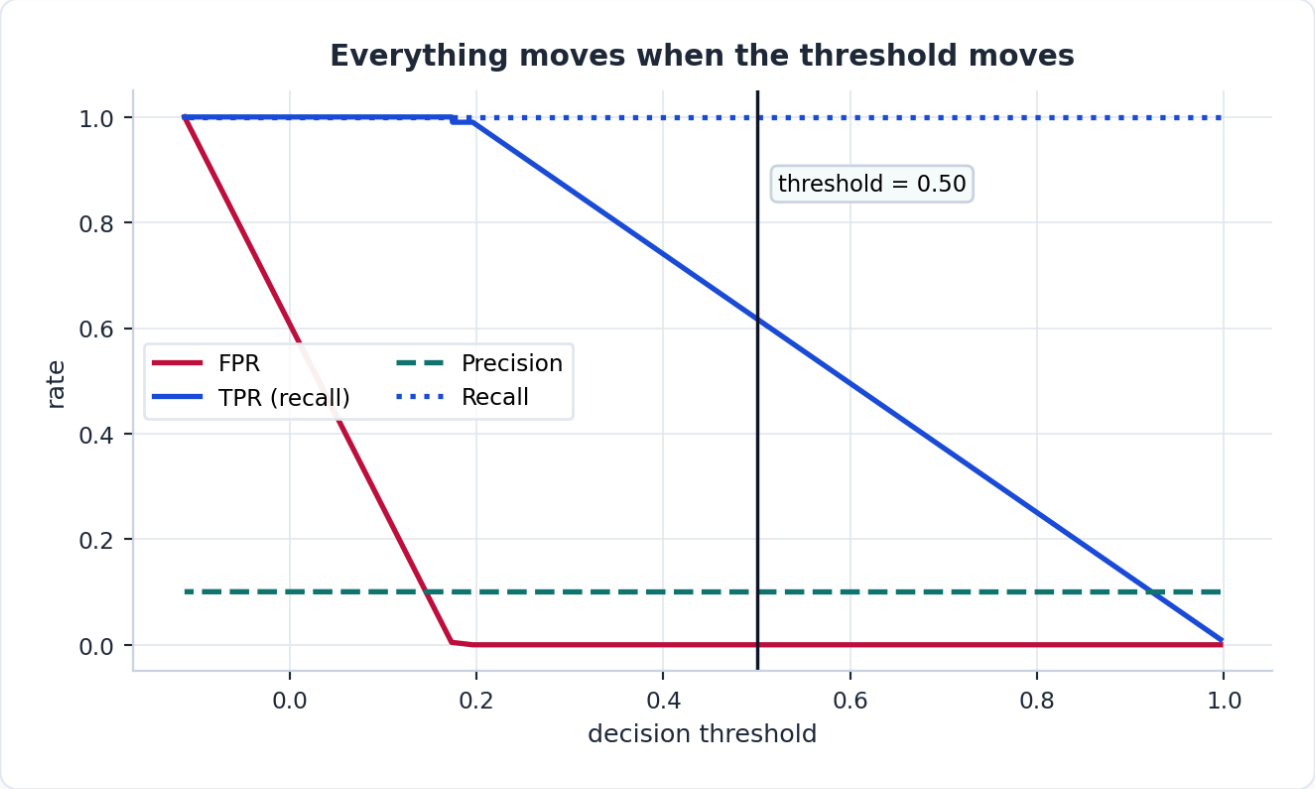
تدريب نموذج خط أساس مع أوزان الفئات — python

```python
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import average_precision_score, roc_auc_score

X, y = make_classification(
    n_samples=5000, n_features=24, n_informative=7, n_redundant=4,
    weights=[0.985, 0.015], class_sep=0.6, random_state=42)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=2000,
class_weight="balanced"))
model.fit(Xtr, ytr)
prob = model.predict_proba(Xte)[:, 1]
print("Average precision:", round(average_precision_score(yte, prob), 4))
print("ROC AUC:", round(roc_auc_score(yte, prob), 4))
print("انتشار الاختبار:", round(float(yte.mean()), 4))
```

إذا اقترب Average Precision من انتشار الفئة، فالنموذج لا يرتب الحالات أفضل كثيراً من خط أساس عشوائي في هذا المقياس. ROC AUC قد يبدو مرتفعاً مع فئة نادرة؛ اقرأ المقاييس مع كلفة التشغيل وحجم التنبيه.

١٢.٣ العتبة ميزانية مراجعة

الاحتمال ليس قراراً حتى تختار عتبة. قد تخفض العتبة لالتقاط مزيد من الحالات، لكن قائمة الإنذارات تكبر. نختار العتبة على مجموعة تحقق منفصلة، ونثبتها قبل كشف الاختبار. المثال التالي يقارن عتبتين على الاختبار لأغراض العرض؛ في مشروع فعلي لا نختار العتبة بالنظر إلى هذه المقارنة، بل نقيّم سياسة مثبتة سلفاً.



الشكل ٢٢١مقايضة عتبة القرار في كشف الاحتيال

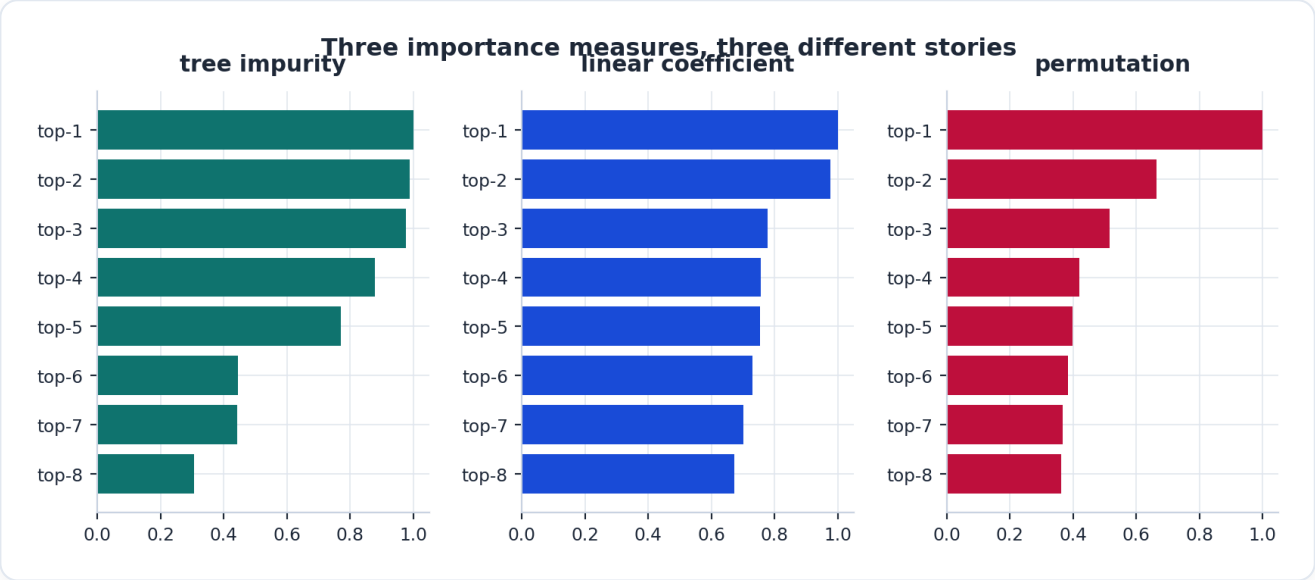
الاسترجاع وحجم التحقيق لكل عتبة python

```python
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(
    n_samples=4000, n_features=20, n_informative=7, weights=[0.985, 0.015],
    class_sep=0.65, random_state=17)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=17)
m = make_pipeline(StandardScaler(),
                  LogisticRegression(max_iter=2000, class_weight="balanced"))
m.fit(Xtr, ytr)
p = m.predict_proba(Xte)[:, 1]
for threshold in (0.35, 0.65):
    pred = (p >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    print(f"t={threshold:.2f}: review={int(pred.sum())}, TP={tp}, FP={fp}, FN=
{fn}, "
          f"precision={precision_score(yte, pred, zero_division=0):.3f}, "
          f"recall={recall_score(yte, pred):.3f}")
```

كلفة الإنذار لا تقاس بعدد FP وحده؛ راجع زمن المحققين ومعدل تأكيد الحالات وسرعة تجميد المعاملة. وقد تكون سياسة أفضل هي مراجعة أعلى `k` معاملات يومياً بدلاً من عتبة ثابتة إذا كانت القدرة اليومية هي القيد.

١٢.٤ تفسير، مراجعة، وتغذية راجعة

أهمية الخصائص ليست تفسيراً سببياً ولا تبريراً كافياً لرفض معاملة. استخدم أدوات تفسير مناسبة، وافحص أخطاء الفئات والمجموعات، واحتفظ بمسار مراجعة بشرية للحالات ذات الأثر. ولأن تأكيد الاحتيال قد يصل بعد أسابيع، يجب أن تتعامل لوحة الجودة مع تأخر التسميات: «غير معروف بعد» ليس «سليم».



الشكل ٢٢٢ مقارنة طرق أهمية الخصائص

مقارنة أثر العتبة وقياس AP على اختبار محجوز python

```python
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, precision_recall_curve
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np

X, y = make_classification(n_samples=3000, n_features=18, n_informative=6,
                           weights=[0.98, 0.02], random_state=5)
a, b, c, d = train_test_split(X, y, stratify=y, random_state=5)
m = make_pipeline(StandardScaler(),
                  LogisticRegression(max_iter=2000,
class_weight="balanced")).fit(a, c)
p = m.predict_proba(b)[:, 1]
precision, recall, thresholds = precision_recall_curve(d, p)
print("AP:", round(average_precision_score(d, p), 4))
print("منحنى PR:", len(precision), len(recall), len(thresholds))
print("عتبات مرشحة:", np.round(np.quantile(thresholds, [0.25, 0.5, 0.75]),
3).tolist())
```

استخدم منحنى PR على مجموعة التحقق لاختيار سياسة، ثم أعد قياس السياسة المثبتة على الاختبار النهائي. لا تنتقِ وسيط العتبات المعروض هنا؛ إنه وصف لتوزيع درجات النموذج فقط.

> **انتبه**
>
> لا تحوّل درجة النموذج إلى تجميد تلقائي للأموال من دون مراجعة واعتراض وتدقيق للانحياز. النموذج أداة ترتيب للمراجعة، والتسمية المتأخرة والإنذارات الكاذبة تؤثران على أشخاص ومعاملات حقيقية.

> **في الواقع**
>
> ينبغي أن يشرح التقرير عدد المعاملات اليومية التي تدخل المراجعة، ونسبة تأكيد المحققين، وزمن اكتشاف النمط الجديد، وسياسة التعامل مع مجهولية النتيجة. راقب اختلاف الأداء عبر قنوات الدفع أو المناطق المسموح استخدامها قانونياً، مع ضوابط الخصوصية.

تمارين الفصل المئة وواحد

- **ملاحظة**

التمرين ١٢.١

قارن accuracy، average precision على بيانات بالانتشار نفسه، ثم اشرح لماذا لا يكفي المقياس الأول.

التمرين ١٢.٢

احسب عدد المعاملات التي ستدخل المراجعة عند عتبتين، واربطه بسعة المحققين.

التمرين ١٢.٣

صمم تقسيماً زمنياً يترك آخر فترة للاختبار، واذكر ما الذي يجب أن يسبق أي ضبط للعتبة.

- **ملاحظة**

مقارنة accuracy وAverage Precision python

```python
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, average_precision_score
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=2000, n_features=10, weights=[0.99, 0.01],
                           random_state=42)
a, b, c, d = train_test_split(X, y, stratify=y, random_state=42)
m = DummyClassifier(strategy="most_frequent").fit(a, c)
p = m.predict(b)
print("accuracy:", round(accuracy_score(d, p), 4))
print("average precision:", round(average_precision_score(d, p), 4))
```

المصنف يحصل على accuracy عالية بإهمال الفئة الإيجابية؛ الاسترجاع يساوي صفراً.

> **ملاحظة**

```python حجم قائمة المراجعة
import numpy as np

scores = np.array([0.05, 0.12, 0.22, 0.31, 0.48, 0.61, 0.8])
for threshold in (0.25, 0.5):
    print(threshold, "عدد المراجعات:", int((scores >= threshold).sum()))
```

> **ملاحظة**

١٢.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
الاعتماد على accuracy	قد لا يكتشف النموذج أي احتيال	اعرض AP والاسترجاع ومصفوفة الخلط
اختيار العتبة على الاختبار	يتفائل التقرير بسبب تكرار الاختيار	عتبة على validation واختبار نهائي واحد
اعتبار كل مجهول سليماً	تتأخر نتائج التحقيق وتتحيز التسميات	افصل المجهول عن السليم
إنذار آلي بلا مراجعة	يسبب أثراً غير متناسب للإنذارات الكاذبة	احتفظ بمراجعة واعتراض وتدقيق

١٢.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
اختلال الفئات	تفاوت كبير في أحجام الفئات	class imbalance
معدل الإيجابيات الكاذبة	نسبة السليمين الذين أُطلق عليهم إنذار	false positive rate
مراجعة بشرية	تدقيق خبير في الحالات المعلّمة	human review
تأخر التسمية	وصول الحقيقة بعد قرار النموذج	label delay

الجزء ١٢ · مشاريع واقعية كاملة

الفصل ١٠٢ — مشروع ٤: التعرّف على الأرقام المكتوبة بخط اليد

دورة حياة كاملة من التحميل حتى تقييم التصنيف متعدد الفئات

الوحدة **scikit-learn ١.٩** الحجم **٣٨٦** كلمة تقريباً

ما الذي ستتقنه في هذا الفصل

- أن يحمل load_digits ويفهم مصفوفة الصور وخصائصها
- أن يبني Pipeline ويقيّمه على اختبار طبقي محجوز
- أن يقرأ تقرير التصنيف ومصفوفة الخلط متعددة الفئات
- أن يعرض أخطاء النموذج مع قيود حجم البيانات ومجالها

١٢.١ صورة صغيرة، وسؤال واضح

يحتوي `sklearn.datasets.load_digits` صوراً رمادية صغيرة لأرقام مكتوبة باليد، وهي مناسبة لتجربة مصنف متعدد الفئات. كل صورة شبكة من ٨×٨ بكسلات، وتصبح كل قيمة بكسل خاصية عند استخدام `data`. ليست هذه المجموعة بديلاً عن صور مستندات حقيقية؛ الخلفيات والقص والتباين والدقة في تطبيق OCR أوسع بكثير.

قبل التدريب افحص عدد الأمثلة، وأبعاد الصور، ومدى قيم البكسل، وتوزيع الأرقام. هذا الفحص يمنع أن ننسى أن الهدف عشر فئات وأن البيانات صغيرة نسبياً.

الشكل ٢٢٣بكسلات صورة رقمية وكيف تمثلها المصفوفة

فحص مجموعة الأرقام python

```python
from sklearn.datasets import load_digits
import numpy as np

digits = load_digits()
print("صور:", digits.images.shape)
print("خصائص مسطحة:", digits.data.shape)
print("الفئات:", np.unique(digits.target).tolist())
print("مدى البكسل:", int(digits.images.min()), int(digits.images.max()))
print("توزيع الفئات:", np.bincount(digits.target).tolist())
```

كل صف في `digits.data` صورة ممدودة إلى ٦٤ قيمة. عند استخدام نموذج يعتمد على المسافة أو تنظيم خطي، قد يفيد تحجيم الخصائص؛ وفي نموذج شجري قد لا يكون ضرورياً. سنبدأ بنموذج مسافة بسيط ونقيّم على بيانات لم تدخل التدريب.

١٢.٢ خط أساس لا يحفظ الاختبار

نقسم البيانات تقسيمًا طبقياً ونثبت البذرة، ثم نضع التحجيم و`KNeighborsClassifier` داخل `Pipeline`. لا نلائم المقياس على كامل البيانات. يقرر الجار الأقرب الرقم من أمثلة التدريب القريبة؛ لذلك تتأثر النتيجة باختيار k ومقياس المسافة وتوزيع الكتابات.

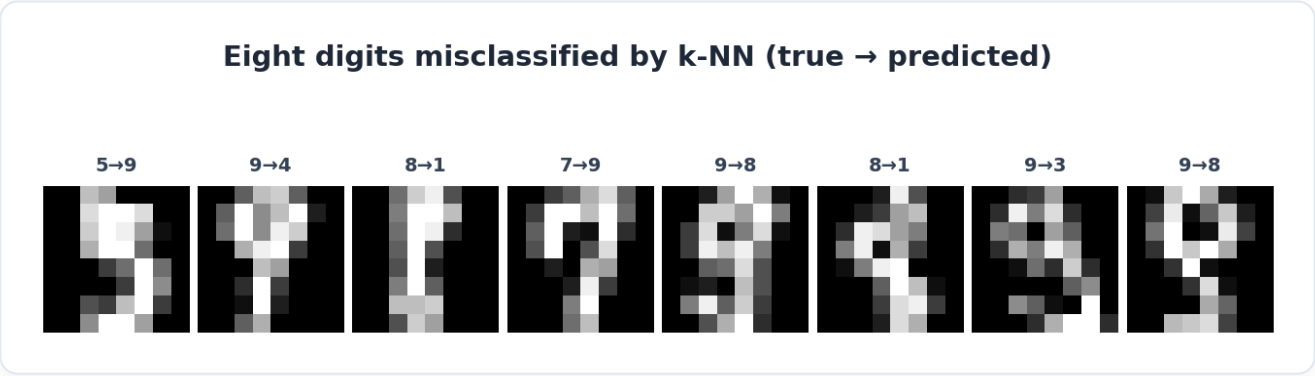
تدريب k-NN واختبار محجوز python

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

digits = load_digits()
Xtr, Xte, ytr, yte = train_test_split(
    digits.data, digits.target, test_size=0.25,
    stratify=digits.target, random_state=42)
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=3))
model.fit(Xtr, ytr)
pred = model.predict(Xte)
print("حجم التدريب والاختبار:", Xtr.shape[0], Xte.shape[0])
print("دقة الاختبار:", round(accuracy_score(yte, pred), 4))
```

النتيجة تقيس هذا التقسيم فقط. إذا قارنت قيماً مختلفة لـ k، فاستخدم تحققاً متقاطعاً على التدريب، ثم أعد التدريب على التدريب كله وقس مرة واحدة على الاختبار.

١٢.٣ أي الأرقام يخلط بينها؟

الدقة المجمعة لا تقول إن أخطاء الرقم ١ مثل أخطاء الرقم ٨. `classification_report` يعرض precision، recall لكل فئة، ومصفوفة الخلط توضح الاتجاهات: الصفوف هي الحقيقة والأعمدة التوقع. لا تفسر القطر وحده؛ انظر أيضاً إلى الخانات خارج القطر واعرض أمثلة الخطأ للتحقق من أن المشكلة ليست في القص أو التسمية.



الشكل ٢٢٤ الأمثلة التي يخطئ فيها مصنف الأرقام

تقرير ومصفوفة خلط لكل رقم

```python
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

digits = load_digits()
a, b, c, d = train_test_split(
    digits.data, digits.target, stratify=digits.target,
    test_size=0.25, random_state=42)
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=3000)).fit(a, c)
pred = model.predict(b)
print(classification_report(d, pred, digits=3, zero_division=0))
cm = confusion_matrix(d, pred)
print("شكل المصفوفة:", cm.shape)
print("أكثر أزواج الالتباس خارج القطر:")
for true_label, pred_label in sorted(
        ((i, j) for i in range(10) for j in range(10) if i != j),
        key=lambda pair: cm[pair[0], pair[1]], reverse=True)[:5]:
    print(f"{true_label} -> {pred_label}: {cm[true_label, pred_label]}")
```

اختلاف الأداء بين الفئات يقود إلى سؤال بيانات: هل بعض الأرقام أقل تمثيلاً أو أصعب كتابة؟ لا تعالج كل التفاوت بتغيير المصنف؛ افحص أمثلة الخطأ ومصدرها.

١٢.٤ من صورة إلى واجهة

في واجهة الاستخدام، يجب تثبيت خطوات التحويل نفسها التي استُخدمت في التدريب، وتحديد شكل الصورة وترتيب البكسلات ونطاق القيم. لا تدرّب على صورة ممدودة بترتيب، ثم تستقبل الصورة في الإنتاج بترتيب آخر. كما يجب عرض درجة الثقة أو تنبيه مراجعة عند حالات الغموض بدلاً من تقديم التنبؤ كحقيقة مؤكدة.

تنبؤ صورة واحدة من الشكل الأصلي python

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

digits = load_digits()
a, b, c, d = train_test_split(
    digits.data, digits.target, stratify=digits.target,
    test_size=0.25, random_state=42)
model = make_pipeline(StandardScaler(), SVC(C=3, gamma="scale"))
model.fit(a, c)
image = digits.images[0]
flat = image.reshape(1, -1)
print("شكل الصورة:", image.shape)
print("الرقم الحقيقي:", int(digits.target[0]))
print("التنبؤ:", int(model.predict(flat)[0]))
```

المثال يبين تحويل صورة المجموعة إلى صف خصائص، ولا يضمن أن الصورة الأولى لم تكن ضمن تدريب النموذج؛ إنها لتوضيح واجهة الإدخال فقط. لقياس عام استخدم الاختبار المحجوز أو cross-validation، لا هذا المثال الفردي.

في الواقع

نظام قراءة أرقام على نموذج ورقي يحتاج مراقبة انحراف جودة الصورة: الإضاءة، والقص، والدقة، والكتابة خارج النطاق. اختبر على عينات من الأجهزة والمستخدمين المستهدفين، وضع مساراً لمراجعة الحالات منخفضة الثقة.

تمارين الفصل المئة واثنين

- **ملاحظة**

التمرين ١٢.١

اعرض صورة واحدة وقيم البكسلات الخاصة بها، ثم تحقق أن تسطيحها ينتج ٦٤ خاصية.

التمرين ١٢.٢

قارن `KNeighborsClassifier` و `LogisticRegression` عبر نفس التقسيم، واطبع تقريراً لكل فئة.

التمرين ١٢.٣

اعثر على أمثلة أخطأ فيها النموذج واعرض الحقيقة والتنبؤ، ثم فسر لماذا لا يكفي إحصاء مصفوفة الخلط.

- **ملاحظة**

تسطيح الصورة والتحقق من البعد python

```python
from sklearn.datasets import load_digits

image = load_digits().images[0]
flat = image.reshape(-1)
print("الشكل:", image.shape, "عدد الخصائص:", flat.size)
assert flat.size == 64
```

ملاحظة

مقارنة خطين أساسيين python

```python
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

D = load_digits()
a, b, c, d = train_test_split(
    D.data, D.target, stratify=D.target, random_state=42)
models = [
    ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(3))),
    ("logistic", make_pipeline(StandardScaler(),
                               LogisticRegression(max_iter=3000))),
]
for name, model in models:
    pred = model.fit(a, c).predict(b)
    print(name)
    print(classification_report(d, pred, digits=2, zero_division=0))
```

> 📝 **ملاحظة**

١٢.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
قياس التدريب	يبدو النموذج أدق مما يعمم	استخدم اختباراً محجوزاً لم يره النموذج
تجاهل شكل الصورة	يختلف ترتيب البكسلات بين التدريب والواجهة	ثبّت التحجيم والتسطيح والترتيب
إخفاء تفاوت الفئات	تخفي الدقة أخطاء رقم بعينه	اعرض تقريراً متعدد الفئات ومصفوفة خلط
تعميم digits إلى OCR	تختلف الدقة والخلفية والمجال	اختبر على صور ممثلة للنشر

١٢.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
صورة رمادية	صورة بقناة سطوع واحدة	grayscale image
مصفوفة الخلط	عدد التنبؤات لكل زوج حقيقة وتوقع	confusion matrix
متعدد الفئات	هدف يضم أكثر من فئتين	multiclass
خط الأساس	نموذج مرجعي بسيط للمقارنة	baseline

الجزء ١٢ · مشاريع واقعية كاملة

الفصل ١٠٣ — مشروع ٥: محرّك توصيات

الترشيح المبني على المحتوى، والتعاون، ومعايير NDCG

الوحدة **scikit-learn ١.٩** الحجم **٣٧٧** كلمة تقريباً

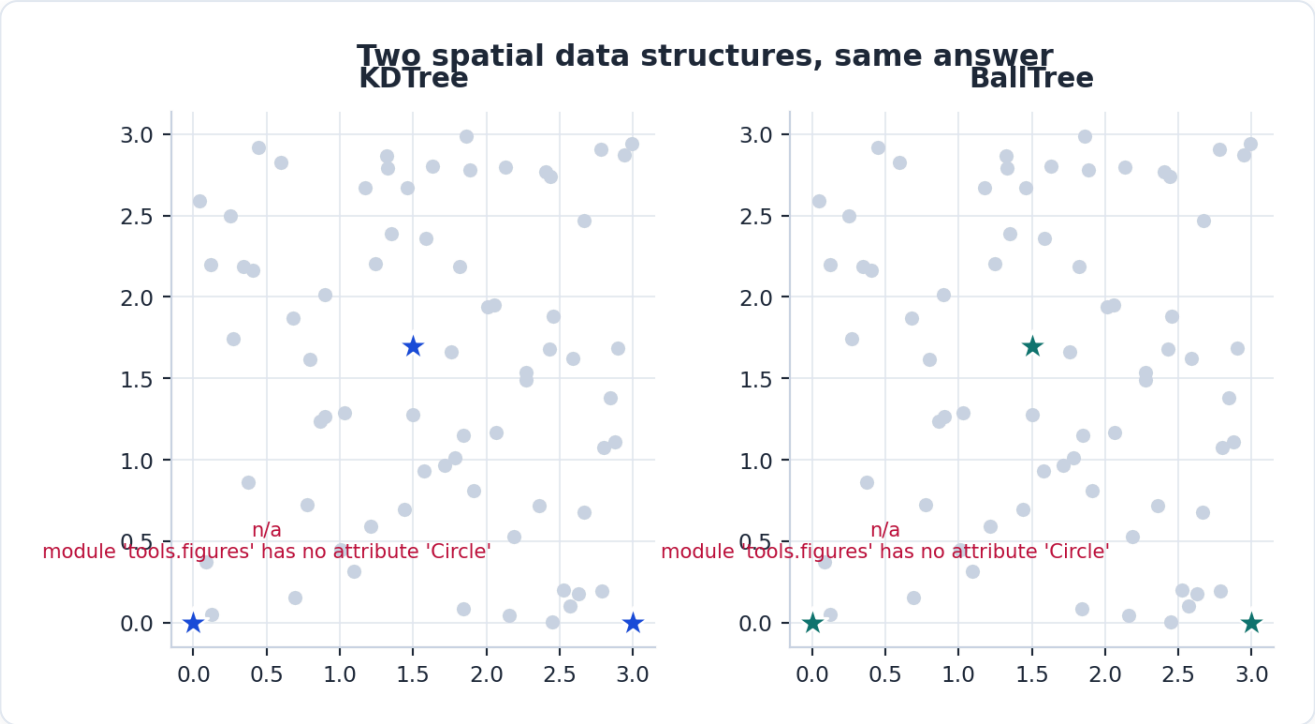
ما الذي ستتقنه في هذا الفصل

- أن يبني توصيات محتوى باستخدام تشابه العناصر
- أن يشرح الفرق بين التصفية القائمة على المحتوى والتعاونية
- أن يستخدم NDCG أو Precision@K لتقييم ترتيب التوصيات
- أن يعالج البداية الباردة وتسرب التفاعل إلى التقييم

١٢.١ التوصية ترتيب لا تصنيف بسيط

مستخدم شاهد بعض الأفلام، والنظام يريد اقتراح ثلاثة أخرى. لا نملك «فئة صحيحة» لكل عنصر؛ لدينا تفاعلات ناقصة، وربما لم يظهر الفيلم للمستخدم أصلاً. التوصية إذن مهمة ترتيب (ranking): نضع العناصر ذات المنفعة الأعلى في أعلى القائمة، ونقيس جودة الترتيب عند K.

سنستخدم مجموعة أفلام صغيرة اصطناعية بخصائص نوعية وتقييمات قليلة. في الإنتاج يجب فصل التفاعلات بحسب الزمن، وعدم اعتبار غياب التقييم رفضاً صريحاً. ابدأ بنموذج محتوى واضح، ثم قارن بتصفية تعاونية عندما يتوفر تاريخ كافٍ.



الشكل ٢٢٥جيران متقاربون في فضاء الخصائص

تمثيل العناصر بخصائص محتوى python

```python
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

items = ["طبيعة" ,"ألغاز" ,"طبخ" ,"مدينة" ,"مغامرة" ,"فضاء"]
features = np.array([
    [1, 0, 0, 0], [1, 1, 0, 0], [0, 1, 1, 0],
    [0, 0, 0, 1], [0, 1, 0, 0], [0, 0, 1, 0],
], dtype=float)
similarity = cosine_similarity(features)
target = items.index("فضاء")
neighbors = np.argsort(similarity[target])[::-1]
print("ترتيب التشابه لفضاء:")
for idx in neighbors[1:4]:
    print(items[idx], round(float(similarity[target, idx]), 3))
```

هذا التشابه يوصي بعناصر قريبة من خصائص عنصر أعجب به المستخدم. لا يستخدم تاريخ مستخدمين آخرين، ولذلك يستطيع اقتراح عنصر جديد فور توفر وصفه، لكنه لا يتعلم تفضيلات معقدة من التفاعلات وحدها.

١٢.٢ بناء ملف اهتمام مستخدم

في الترشيح القائم على المحتوى نكوّن ملفاً من تفضيلات المستخدم: متوسط أو مجموع موزون لخصائص العناصر التي تفاعل معها، ثم نرتب العناصر غير المشاهدة بتشابهها مع هذا الملف. يجب استبعاد العناصر التي شاهدها المستخدم بالفعل، واختيار وزن مناسب للتفاعل، مثل تقييم أو مشاهدة مكتملة.

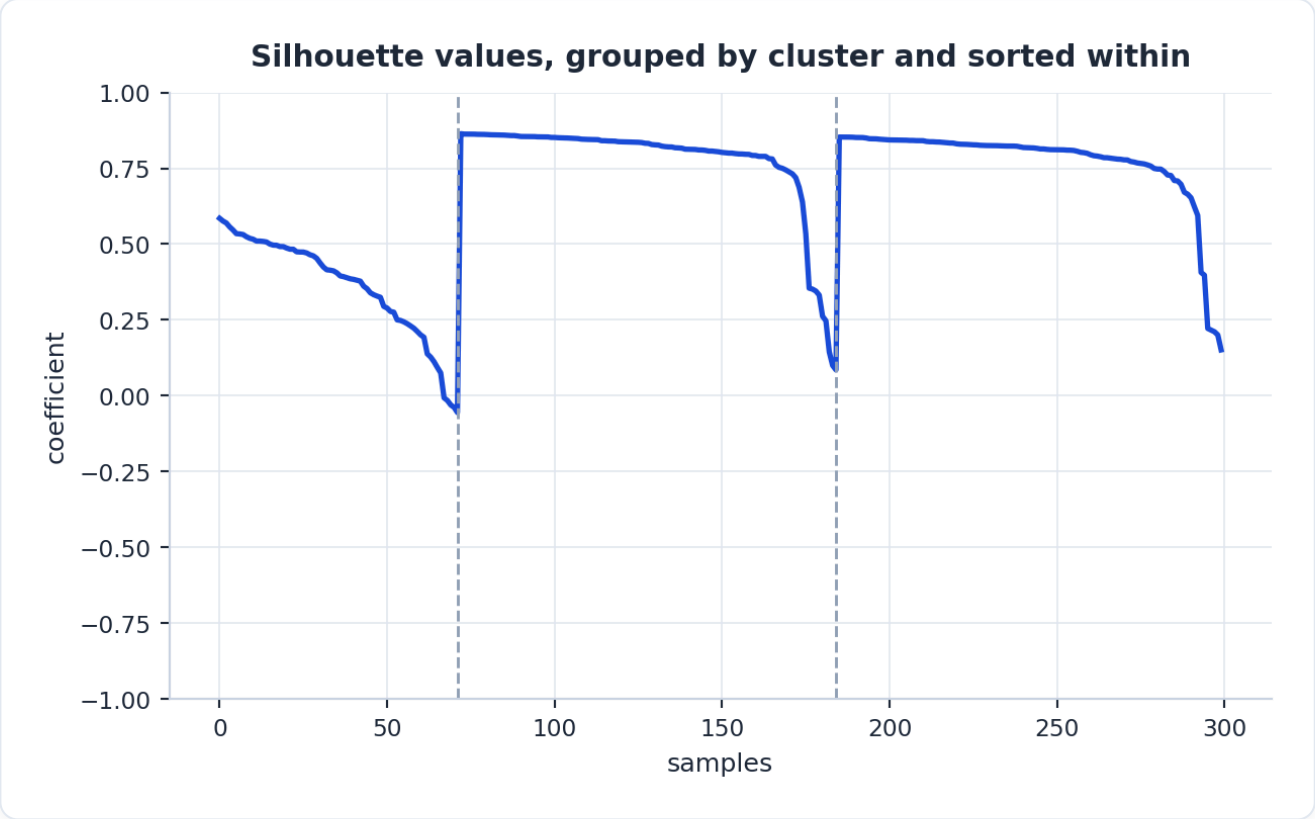
ترتيب عناصر جديدة وفق ملف مستخدم python

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

names = ["خيال", "مغامرة", "تاريخ", "طبخ", "ألغاز", "طبيعة"]
V = np.array([[1, 0, 0, 0], [1, 1, 0, 0], [0, 1, 1, 0],
              [0, 0, 0, 1], [0, 1, 0, 0], [0, 0, 1, 0]], dtype=float)
seen = np.array([0, 1])
profile = V[seen].mean(axis=0, keepdims=True)
scores = cosine_similarity(profile, V).ravel()
scores[seen] = -np.inf
ranking = np.argsort(scores)[::-1]
print("التوصيات:", [(names[i], round(float(scores[i]), 3))
                    for i in ranking[:3]])
```

ملف الاهتمام يبدأ من القليل، لكنه قد يكرر نوعاً واحداً من المحتوى. نظم التنويع أو القيود التجارية تأتي بعد حساب الصلة، ويجب أن تظل موثقة كي لا تتحول «الملاءمة» إلى غموض.

١٢.٣ التصفية التعاونية: من يشبه من؟

تستخدم التصفية التعاونية التفاعلات المشتركة بين المستخدمين أو العناصر. مصفوفة التقييم عادةً متفرقة جداً؛ لا تملأ كل خانة صفرًا لأن الصفر قد يعني «لم يُشاهد» لا «كرهه». يمكن استخدام تشابه جيب التمام على المستخدمين ذوي التفاعلات المشتركة، مع وزن التنبؤات بمقدار التشابه.



الشكل ٢٢٦ تشخيص جودة مجموعات أو جيران متشابهين

تشابه العناصر من تفاعلات المستخدمين python

```python
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

ratings = np.array([
    [5, 4, 0, 0, 1],
    [4, 5, 0, 0, 1],
    [0, 0, 5, 4, 0],
    [0, 1, 4, 5, 0],
], dtype=float)
item_similarity = cosine_similarity(ratings.T)
print("تشابه العنصرين 0 و1:", round(float(item_similarity[0, 1]), 3))
print("تشابه العنصرين 2 و3:", round(float(item_similarity[2, 3]), 3))
print("نسبة الخانات المرصودة:", round(float(np.count_nonzero(ratings) / ratings.size),
3))
```

المصفوفة الصغيرة توضح التشابه فقط. لا تكفي لبناء نظام إنتاجي، إذ تحتاج إلى إزالة أثر المستخدم النشط، وتحديد طريقة التقييم، ومنع استخدام التفاعلات المستقبلية في التدريب.

١٢.٤ تقييم الترتيب عند K

تقيس `sklearn.metrics.ndcg_score` جودة ترتيب درجات العناصر مقارنة بملاءمات حقيقية. المقياس يعاقب وضع العناصر ذات الصلة في مراتب متأخرة، ويقبل `k` للتركيز على أعلى القائمة. في نظام توصية حقيقي يجب بناء المرشحين من الماضي، ثم اختبار التفاعلات اللاحقة التي لم تدخل أي خطوة من خطوات التدريب.

حساب NDCG عند أعلى ثلاثة عناصر — python

```python
from sklearn.metrics import ndcg_score
import numpy as np

relevance = np.array([[3, 0, 2, 1, 0]])
ranking_a = np.array([[0.9, 0.8, 0.7, 0.2, 0.1]])
ranking_b = np.array([[0.1, 0.2, 0.3, 0.9, 0.8]])
print("NDCG@3 جيد:", round(float(ndcg_score(relevance, ranking_a, k=3)), 4))
print("NDCG@3 ضعيف:", round(float(ndcg_score(relevance, ranking_b, k=3)), 4))
```

المقياس الأعلى يعني أن ترتيب الدرجات أقرب لترتيب الصلة في هذا المثال. لا يكفي وحده: قد تكون القائمة متكررة أو غير متنوعة أو غير مفيدة تجارياً. أضف مقاييس تغطية وتنوع واختباراً بشرياً عندما يتطلب المنتج ذلك.

في الواقع

في الخدمة الحقيقية، قسم التفاعلات زمنياً: درّب حتى تاريخ معين، وابنِ التوصيات كما لو أن الوقت توقف، ثم قس التفاعلات اللاحقة. اختبر أيضاً مستخدماً جديداً وعنصراً جديداً؛ حالات البداية الباردة تحتاج قواعد شعبية أو محتوى أو سؤالاً للمستخدم، لا تخمين تشابه غير موجود.

تمارين الفصل المئة وثلاثة

- **ملاحظة**

التمرين ١٢.١

أضف سمة محتوى إلى العناصر، وأعد ترتيب تشابهها بعد تفاعل المستخدم مع عنصرين.

التمرين ١٢.٢

احسب NDCG@٣ لترتيبين مختلفين على قائمة صلة ثابتة، وحدد أي ترتيب يضع العنصر الأعلى صلة أولاً.

التمرين ١٢.٣

صمم اختباراً زمنياً لمستخدمين لديهم عدة تفاعلات، مع استبعاد التفاعلات المحجوبة من ملف الاهتمام.

- **ملاحظة**

تحديث تشابه المحتوى python

```python
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

features = np.array([[1, 0, 0], [1, 1, 0], [0, 1, 1], [0, 0, 1]], float)
profile = features[[0, 1]].mean(axis=0, keepdims=True)
scores = cosine_similarity(profile, features).ravel()
scores[[0, 1]] = -np.inf
print("العنصر التالي:", int(np.argmax(scores)))
```

ملاحظة

تقييم ترتيبين python

```python
from sklearn.metrics import ndcg_score

truth = [[3, 2, 0, 1]]
for scores in ([[0.9, 0.8, 0.2, 0.1]], [[0.1, 0.2, 0.9, 0.8]]):
    print(round(float(ndcg_score(truth, scores, k=3)), 4))
```

> **ملاحظة**

١٢.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
اعتبار التفاعل الغائب رفضاً	يتحول نقص الملاحظة إلى إشارة سلبية	تعامل معه كقيمة مجهولة
تسرب التفاعلات المستقبلية	يرفع جودة الترتيب ظاهرياً	افصل حسب الزمن قبل بناء الملف
اقتراح عنصر تمت مشاهدته	تهدر القائمة المحدودة	استبعد العناصر المعروفة
NDCG وحده معيار نجاح	قد ينتج تنوعاً وتغطية ضعيفين	أضف قياسات تجربة ومحتوى

١٢.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
ترشيح المحتوى	التوصية بحسب خصائص العنصر	content-based filtering
التصفية التعاونية	التوصية من أنماط التفاعلات المشتركة	collaborative filtering
البداية الباردة	غياب تاريخ لمستخدم أو عنصر جديد	cold start
NDCG	جودة ترتيب مطبّعة ومخصومة عند الرتبة	normalized discounted cumulative gain

الجزء ١٢ · مشاريع واقعية كاملة

الفصل ١٠٤ — مشروع ٦: الصيانة التنبّؤية للآلات

بيانات المستشعرات، وكشف الشذوذ، وتنبيهات الصيانة

الوحدة **مشاريع تطبيقية** الحجم **٤٥٩** كلمة تقريباً

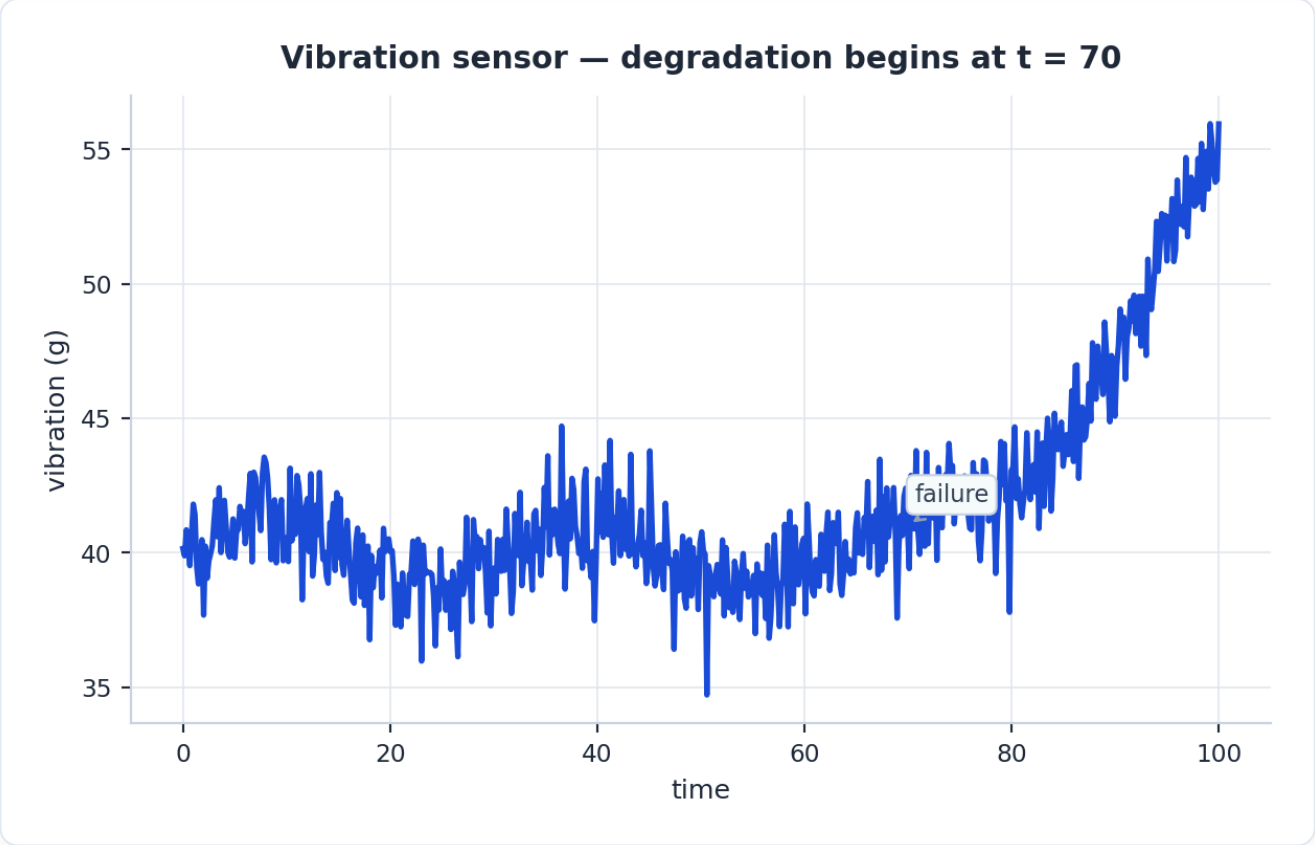
ما الذي ستتقنه في هذا الفصل

- أن يبني بيانات مستشعرات متتابعة وتسمية هدف زمنية واضحة
- أن يقسم التدريب والاختبار زمنياً بدلاً من خلط القراءات
- أن يقارن مصنف أعطال بدرجات IsolationForest
- أن يصمم سياسة تنبيه وتقييم تعكس زمن التوقف وكلفة الإنذار

١٢.١ المستشعر لا يرى المستقبل

تخيل آلة ترسل اهتزازاً وحرارة وضغطاً كل ساعة. نريد تقدير احتمال عطل خلال نافذة قادمة كي يصل الفني قبل التوقف. إذا استخدمنا قراءة بعد العطل كميزة، يصبح التقييم رائعاً لكنه عديم الفائدة. يجب أن تحدد لحظة القرار، نافذة السمات السابقة، ونافذة النتيجة التالية، ثم تحذف الصفوف التي لا تملك نتيجة مكتملة.

سنصنع بيانات اصطناعية مرتبة للتدريب التعليمي. ليست محاكاة فيزيائية ولا نموذج صيانة معتمداً. في المصنع الحقيقي تتغير الآلات والخامات، وتؤثر أعمال الصيانة السابقة في المستشعر، لذلك يجب تسجيل معرف الآلة وتاريخ الصيانة واختبار التعميم على آلات وفترات جديدة.



الشكل ٢٢٧ اتجاهات مستشعرات الصيانة ومؤشرات العطل

إنشاء قراءات زمنية اصطناعية python

```python
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 1500
t = np.arange(n)
degradation = t / n
vibration = 0.4 + 0.25 * degradation + rng.normal(0, 0.08, n)
temperature = 60 + 12 * degradation + rng.normal(0, 3, n)
pressure = 100 - 10 * degradation + rng.normal(0, 4, n)
risk = 1 / (1 + np.exp(-(-5 + 5 * degradation + 4 * (vibration - 0.5))))
failure_next_24h = (rng.random(n) < risk * 0.12).astype(int)
data = pd.DataFrame({"time": t, "vibration": vibration,
                     "temperature": temperature, "pressure": pressure,
                     "failure_next_24h": failure_next_24h})
print("شكل الجدول:", data.shape)
print("معدل الهدف:", round(float(data["failure_next_24h"].mean()), 3))
print(data.tail(3).round(2).to_string(index=False))
```

هذا الهدف احتمال عشوائي اصطناعي مرتبط بتدهور متزايد، وليس أعطالاً حقيقية. نستخدمه لإظهار التقييم الزمني؛ لا نعلن منه معدل فشل صالحاً للمصنع.

١٢.٢ اختبار المستقبل فقط

نحجز آخر جزء من التسلسل للاختبار. لا نخلط الصفوف، لأن القراءات المتجاورة متشابهة، ولأن النظام سينشر على المستقبل. نضع التحجيم والمصنف في Pipeline، ونستعمل ميزات المستشعر المسجلة قبل نقطة القرار. في تطبيق نافذة متحركة، يجب أيضاً أن تمنع التداخل بين صفوف التدريب والاختبار عبر `gap` يطابق أفق الهدف.

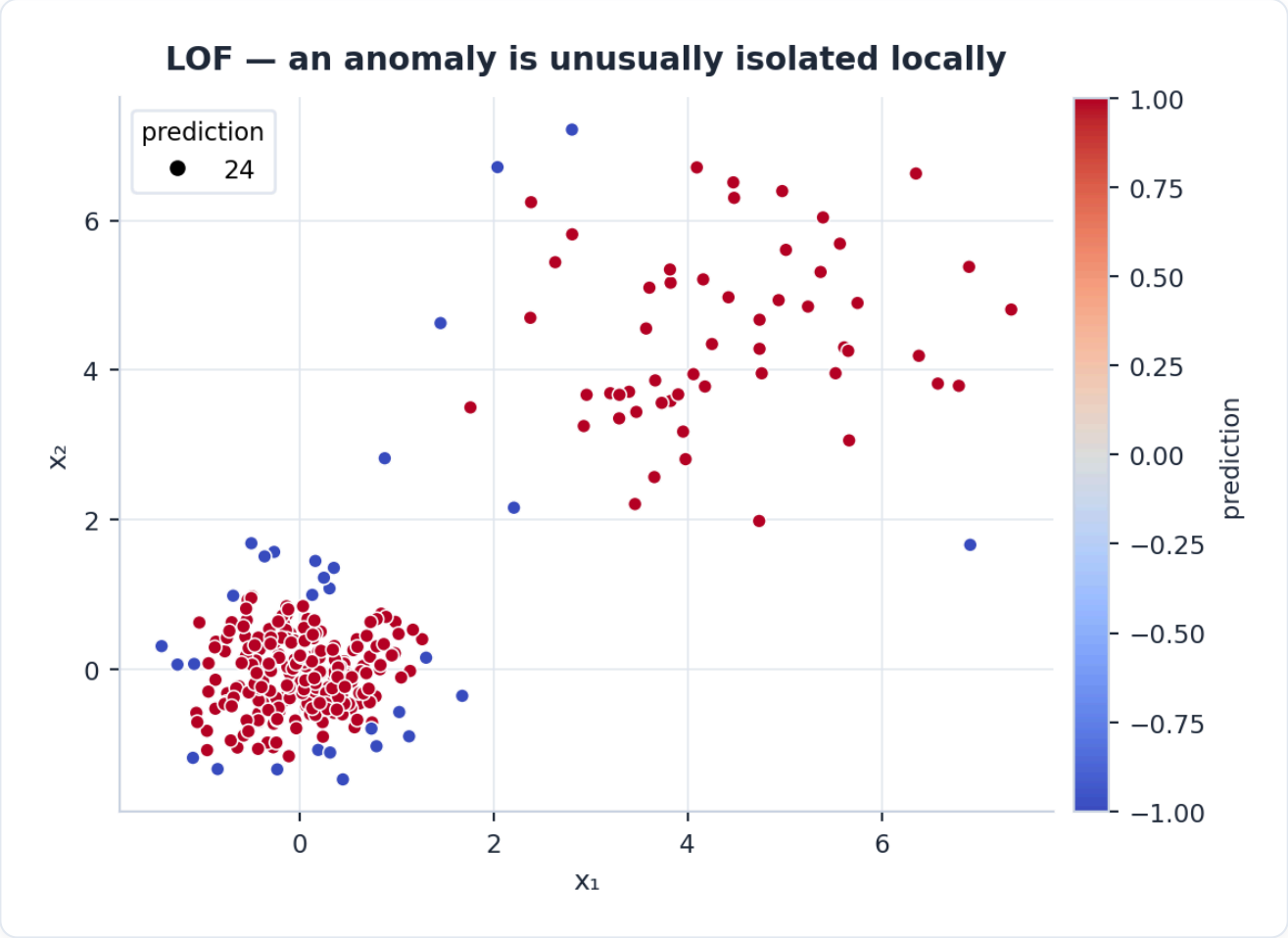
python تدريب على الماضي واختبار على الفترة اللاحقة

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, recall_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(17)
n = 1800
t = np.arange(n)
age = t / n
vibration = 0.35 + 0.3 * age + rng.normal(0, 0.1, n)
temperature = 55 + 18 * age + rng.normal(0, 4, n)
pressure = 105 - 13 * age + rng.normal(0, 5, n)
prob = 1 / (1 + np.exp(-(-5 + 5 * age + 3 * (vibration - 0.5))))
y = (rng.random(n) < prob * 0.16).astype(int)
X = np.c_[vibration, temperature, pressure]
cut = int(n * 0.75)
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=2000,
class_weight="balanced"))
model.fit(X[:cut], y[:cut])
p = model.predict_proba(X[cut:])[:, 1]
pred = (p >= 0.5).astype(int)
print("زمن التدريب والاختبار:", cut, n - cut)
print("AP آخر فترة:", round(average_precision_score(y[cut:], p), 4))
print("Recall 0.5 عند:", round(recall_score(y[cut:], pred, zero_division=0), 4))
```

إذا كان عدد الأعطال في الفترة المحجوزة قليلاً جداً، فالمقاييس متقلبة؛ اعرض عدد الحالات وعدد التنبيهات إلى جانب النسبة. وسّع الفترة أو اجمع بيانات إضافية بدلاً من اختراع دقة مستقرة.

١٢.٣ غير خاضع للإشراف: شذوذ ليس عطلًا

لا تتوفر دائماً تسميات أعطال موثوقة. `sklearn.ensemble.IsolationForest` ينتج درجة شذوذ من نمط القراءات، لكنه لا يعرف معنى «العطل» ولا يضمن أن كل نقطة غير مألوفة خطرة. ندرّبه على فترة مرجعية مستقرة، ثم نرتب القراءات اللاحقة للمراجعة. إذا كانت فترة التدريب تحوي أعطالاً أو تغير تشغيل طبيعي، سيتعلم مرجعاً خاطئاً.



الشكل ٢٢٨ كشف الشذوذ في قراءات مستشعر

ترتيب نقاط شاذة من بيانات مستشعر — python

```python
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(11)
normal = rng.normal(size=(500, 3))
later = rng.normal(size=(120, 3))
later[:8] += np.array([4.0, 4.0, -4.0])
detector = IsolationForest(contamination=0.03, random_state=42)
detector.fit(normal)
scores = -detector.score_samples(later)
order = np.argsort(scores)[::-1]
print("أعلى درجات الشذوذ:", np.round(scores[order[:8]], 3).tolist())
print("عدد النقاط المرصودة:", len(later))
```

القيم هنا مجرد درجات ترتيب؛ `contamination` يضبط عتبة تصنيف تقريبية وليس نسبة أعطال مؤكدة في العالم. راجع أعلى الحالات مع خبير الآلة، وقارنها بسجل الصيانة والحساسات الأخرى.

١٢.٤ من درجة إلى تنبيه قابل للتشغيل

تنبيه كل ساعة قد يرهق الفنيين ويؤدي إلى تجاهل الإنذارات. عرّف ميزانية المراجعة، وفترة تهدئة بعد التنبيه، وكلفة التوقف، وزمن توريد القطعة. قد يكون القرار «صيانة خلال وردية» لا «أوقف الآلة فوراً». قِس الاسترجاع قبل العطل بفترة نافعة، والإنذارات الكاذبة لكل آلة-أسبوع، لا نسبة تصنيف عامة وحدها.

قياس عبء التنبيهات على نافذة اختبار — python

```python
import numpy as np
from sklearn.metrics import confusion_matrix

y_true = np.array([0, 0, 1, 0, 1, 0, 0, 1, 0, 0])
risk_score = np.array([0.08, 0.2, 0.8, 0.35, 0.61,
                       0.1, 0.42, 0.73, 0.18, 0.55])
for threshold in (0.4, 0.65):
    alert = (risk_score >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, alert).ravel()
    print(f"threshold={threshold}: alerts={int(alert.sum())}, "
          f"TP={tp}, FP={fp}, FN={fn}, TN={tn}")
```

النقطة ليست أن عتبة واحدة صالحة لكل الآلات. يمكن ضبطها حسب حساسية الخط وضرر التوقف، ثم مراجعتها على فترة مستقلة مع ملاحظات الفنيين. لا تُنهِ الإنذار بعد صيانة ناجحة من غير توثيق ما تغير في البيانات.

في الواقع

خط صيانة تنبؤية ناضج يسجل تنبؤ كل آلة مع وقت القرار والبيانات المتاحة آنذاك، ثم يربطه بأمر العمل والنتيجة. بهذا يمكن معرفة إن كان الإنذار سبق العطل بالوقت الكافي، لا إن كان قد طابق تسمية ثنائية فقط.

تمارين الفصل المئة وأربعة

- **ملاحظة**

التمرين ١٢.١

غيّر نقطة الفصل الزمني وقارن عدد الأعطال في التدريب والاختبار. لماذا يؤثر ذلك على عدم اليقين؟

التمرين ١٢.٢

درّب IsolationForest على فترة مرجعية مختلفة، وقارن ترتيب أعلى الحالات. اشرح حساسية خط الأساس.

التمرين ١٢.٣

صمم مقياسين تشغيليين لفعالية التنبيه يحددان زمن الإنذار قبل العطل وعبء الإنذارات لكل آلة.

- **ملاحظة**

توزيع الهدف عند نقاط فصل زمنية مختلفة python

```python
import numpy as np

rng = np.random.default_rng(42)
n = 1000
t = np.arange(n)
p = 1 / (1 + np.exp(-(-5 + 6 * t / n)))
y = (rng.random(n) < p * 0.15).astype(int)
for fraction in (0.7, 0.8):
    cut = int(n * fraction)
    print(fraction, "أعطال تدريب/اختبار:",
          int(y[:cut].sum()), int(y[cut:].sum()))
```

الفصل المتأخر قد يواجه معدلاً مختلفاً للتدهور، لذلك يجب فحص تغير التوزيع إلى جانب الأداء.

> **ملاحظة**

```python تغير خط الأساس يغير ترتيب الشذوذ
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(42)
reference_a = rng.normal(0, 1, (400, 2))
reference_b = rng.normal(0.5, 1.2, (400, 2))
observations = np.array([[2.5, 2.5], [0.0, 0.0], [-3.0, 2.0]])
for reference in (reference_a, reference_b):
    model = IsolationForest(contamination=0.05, random_state=42).fit(reference)
    print(np.round(-model.score_samples(observations), 3).tolist())
```

> 📝 **ملاحظة**

١٢.٦ الأخطاء الشائعة

الخطأ	ما يحدث بالضبط	الحل البرمجي الصحيح
تقسيم القراءات عشوائياً	تتسرب القراءات المجاورة بين التدريب والاختبار	استخدم اختباراً زمنياً و `gap` عند الحاجة
إدخال بيانات ما بعد العطل	يقرأ النموذج المستقبل ضمن السمات	اجعل كل ميزة متاحة عند لحظة القرار
تفسير شذوذ كعطل مؤكد	تكثر التنبيهات الخاطئة	راجع الحالات مع بيانات الصيانة والخبرة
إغفال زمن التنبيه	يصاب النموذج بعد فوات فرصة التدخل	قس وقت السبق والإنذارات لكل فترة

١٢.٧ المصطلحات

المصطلح	المعنى	المقابل الإنجليزي
الصيانة التنبؤية	تخطيط صيانة اعتماداً على إشارات الخطر	predictive maintenance
نافذة زمنية	فترة تجمع منها سمات أو نتائج	time window
الشذوذ	عينة تختلف عن النمط المرجعي	anomaly
زمن السبق	المدة بين التنبيه والحدث	lead time

ملحق: إحصاء الكتاب

هذه وثيقة تقنية عملية كُتبت ومُتحقَّق منها آلياً: كل سطر برمجي في الصفحات التالية نُفِّذ فعلياً على تثبيت **scikit-learn ١.٩.١** داخل بيئة **Python ٣.١٤**، والمخرجات المعروضة مخرجات حقيقية لا مكتوبة بخط اليد.

البند	القيمة
عدد الأجزاء	١٢
عدد الفصول	١٠٤
عدد الأشكال والمخططات	٢٢٨
عدد المعادلات	٠
عدد مقتطفات الشيفرة	٧٧٠
عدد الجداول	٣١٧
بيئة التنفيذ	scikit-learn ١.٩.١ ▪ Python ٣.١٤ ▪ NumPy ٢.٤ ▪ pandas ٣.٠

الإصدار الأول ٢٠٢٦

Python
بلغة بايثون

إتقان scikit-learn 1.9

من الصفر إلى الاحتراف

دليل شامل لتعلّم الآلة التطبيقي

- **شرح مبسّط** من الأساسيات إلى المتقدم
- **أمثلة عملية** وتطبيقات واقعية
- **مشاريع متكاملة** من الحياة العملية
- **أفضل الممارسات** والنصائح الاحترافية

م/ أحمد صالح المغز

2026